hushuqi

算法竞赛模板

ICPC/CCPC Algorithm Templates

目录

0. 比赛注意 20
0.1 赛前与比赛周 . . . 20
0.2 赛时心态与分工 . . . 20
0.3 开题与选题 . . . 20
0.4 上机与打印 . . . 20
0.5 卡题 . . . 21
0.6 提交前 . . . 21
0.7 AC . . . 21
0.8 WA . . . 21
0.9 TLE . . . 22
0.10 RE . . . 22

1. 基础与工程 23
1.1 基础代码框架 . . . 23
1.1.001 C++ 通用头文件与类型别名 . . . 23
1.1.002 标准 `main` / 多测框架 . . . 23
1.1.003 常量、无穷大、浮点误差与符号函数 . . . 24
1.1.004 常用函数重载与容器输出 . . . 24
1.1.005 Lambda 递归写法 . . . 25
1.1.006 C 语言基础模板与常用 C 库 . . . 26
1.1.007 Python 基础模板 . . . 26
1.1.008 静态取模类 . . . 27
1.2 输入输出 . . . 30
1.2.001 快速输入输出 . . . 30
1.2.002 `__int128` 输入输出 . . . 32
1.2.003 泛型读写封装 . . . 34
1.2.004 浮点数与输出精度控制 . . . 34
1.2.005 未知长度的一行输入 . . . 34
1.2.006 文件输入输出 . . . 35
1.2.007 手动扩大栈空间 . . . 35
1.2.008 无原生 `__int128` 时的双字 `i128` . . . 35
1.3 编译、调试与测试 . . . 39
1.3.001 编译参数与调试构建 . . . 39
1.3.002 Makefile（Windows / Linux） . . . 42
1.3.003 bashrc / vimrc . . . 42
1.3.004 对拍器（Windows / Linux） . . . 43
1.3.005 随机数据生成器 . . . 47
1.3.006 GNU C++ 环境、本地计时与评测机测速 . . . 47
1.3.007 调试输出与断言 . . . 49
1.3.008 常见未定义行为和易错点清单 . . . 50
1.4 STL、扩展库与防卡 . . . 50
1.4.001 `vector`、`array`、`string`、`tuple`、`pair` . . . 50

1.4.002 stack、queue、deque、priority_queue . . . 50
1.4.003 set、multiset、map、multimap . . . 51
1.4.004 unordered_map / unordered_set 防 hack 哈希 . . . 51
1.4.005 bitset 与手写 bitset . . . 52
1.4.006 GNU PBDS . . . 54
1.4.007 lower_bound、upper_bound、unique、iota . . . 55
1.4.008 next_permutation、prev_permutation、shuffle . . . 56
1.4.009 accumulate、is_sorted、迭代器 . . . 56
1.4.010 __builtin_* 与 C++20 位运算函数 . . . 56
1.4.011 sort killer 与哈希 hack . . . 56
1.4.012 何时才需要卡常 . . . 57
1.4.013 字符串分割：getline / Split . . . 57
1.4.014 optional . . . 58
1.5 平台优化与极限卡常 . . . 58
1.5.001 卡常使用边界与专题索引 . . . 59

2. 基础算法与离线算法 61
2.1 搜索与枚举 . . . 61
2.1.001 整数二分、实数二分 . . . 61
2.1.002 整数三分、实数三分、黄金分割三分 . . . 63
2.1.003 全排列与不重复排列 . . . 65
2.1.004 子集枚举、固定大小子集枚举 . . . 66
2.1.005 Gray Code . . . 66
2.1.006 DFS / BFS 基础搜索 . . . 67
2.1.007 启发式搜索与 A*（第 K 短路） . . . 68
2.1.008 模拟退火 . . . 70
2.1.009 DLX 精确覆盖 . . . 71
2.1.010 IDDFS、IDA* 与双向 BFS . . . 75
2.2 排序、分治与序列处理 . . . 79
2.2.001 快速排序、归并排序、基数排序 . . . 79
2.2.002 离散化与坐标压缩 . . . 81
2.2.003 逆序对及其扩展 . . . 82
2.2.004 整体二分 . . . 84
2.2.005 CDQ 分治与三维偏序 . . . 87
2.2.006 分数级联（Fractional Cascading） . . . 92
2.2.007 扫描线 . . . 94
2.2.008 单调栈、单调队列 . . . 96
2.2.009 滑动窗口与区间最值 . . . 98
2.2.010 最长上升子序列及输出方案 . . . 98
2.2.011 二维 LIS / LCIS . . . 99
2.2.012 最大全 1 子矩形 . . . 100
2.2.013 约瑟夫问题 . . . 101
2.2.014 快速选择 / 线性期望第 k 元素 . . . 102
2.2.015 折半搜索 . . . 103
2.3 莫队与离线查询 . . . 106

2.3.001 普通莫队 . . . 106
2.3.002 带修改莫队 . . . 109
2.3.003 回滚莫队 . . . 111
2.3.004 二次离线莫队 . . . 112
2.3.005 区间不同数统计 . . . 116
2.3.006 区间第 K 小整体二分 . . . 117
2.3.007 树上莫队 . . . 117
2.4 差分与前缀技巧 . . . 122
2.4.001 一维/二维前缀和 . . . 122
2.4.002 一维/二维差分 . . . 123
2.4.003 K 阶差分 . . . 125
2.4.004 子段和、子矩阵和 . . . 126
2.4.005 所有区间 GCD . . . 127
2.4.006 连续区间按位异或 . . . 128

3. 数据结构 130
3.1 并查集 . . . 130
3.1.001 普通并查集 . . . 130
3.1.002 按大小/按秩合并 . . . 131
3.1.003 带权并查集 . . . 131
3.1.004 可撤销并查集 . . . 135
3.1.005 可持久化并查集 . . . 137
3.1.006 区间并行并查集 . . . 140
3.1.007 可删除并查集 . . . 142
3.1.008 非交换群势能并查集 . . . 144
3.2 树状数组 . . . 147
3.2.001 单点修改、前缀和 . . . 147
3.2.002 单点赋值、区间和 . . . 150
3.2.003 前缀和上的二分 . . . 151
3.2.004 区间修改、区间求和 . . . 152
3.2.005 前缀/后缀最大值 . . . 153
3.2.006 后缀和 . . . 154
3.2.007 取模树状数组 . . . 155
3.2.008 二维树状数组 . . . 156
3.2.009 矩形修改、矩形求和 . . . 158
3.2.010 静态点集矩形和 . . . 161
3.2.011 矩形加、单点求值 . . . 164
3.3 线段树家族 . . . 164
3.3.001 单点修改线段树与摘要二分 . . . 164
3.3.002 懒标记线段树 . . . 168
3.3.003 区间加、乘、赋值（仿射懒标记） . . . 174
3.3.004 区间加、区间最小值 . . . 180
3.3.005 ZKW 线段树 . . . 183
3.3.006 非递归线段树 . . . 184
3.3.007 动态开点线段树 . . . 185

3.3.008 可持久化线段树 / 主席树 . . . 189
3.3.009 线段树合并 . . . 192
3.3.010 线段树分裂 . . . 195
3.3.011 Segment Tree Beats . . . 199
3.3.012 扫描线线段树 . . . 205
3.3.013 矩形面积并、周长并 . . . 209
3.3.014 静态矩形加、矩形和 . . . 209
3.3.015 扫描线求矩形至少二重覆盖面积 . . . 209
3.3.016 线段树分治 . . . 212
3.3.017 动态区间严格众数 . . . 214
3.3.018 可排序线段树 . . . 218
3.3.019 区间一次函数加、区间最小值 . . . 224
3.3.020 可持久化仿射区间和 . . . 229
3.3.021 稀疏仿射区间和 . . . 235
3.3.022 区间赋同函数、区间复合 . . . 238
3.4 RMQ 与静态区间结构 . . . 242
3.4.001 ST 表 . . . 242
3.4.002 基于状压的 RMQ . . . 245
3.4.003 静态区间半群 . . . 249
3.4.004 笛卡尔树 . . . 250
3.4.005 小波矩阵 . . . 252
3.4.006 区间第 K 小、排名、前驱后继 . . . 257
3.4.007 离线静态区间频次 . . . 257
3.4.008 静态区间众数（秩优化） . . . 258
3.4.009 静态区间 LIS 查询 . . . 261
3.4.010 静态区间不同值个数 . . . 268
3.4.011 静态区间逆序对查询 . . . 268
3.4.012 静态区间和 . . . 271
3.4.013 Fischer-Heun 线性预处理 RMQ . . . 271
3.5 平衡树与堆 . . . 275
3.5.001 Treap . . . 275
3.5.002 FHQ Treap . . . 280
3.5.003 可持久化 Treap . . . 284
3.5.004 Splay . . . 288
3.5.005 Splay 重构 . . . 294
3.5.006 珂朵莉树（ODT） . . . 295
3.5.007 PBDS 平衡树 . . . 298
3.5.008 左偏树 / 可并堆 . . . 298
3.5.009 带删堆 . . . 300
3.5.010 Top-K 和维护 . . . 302
3.5.011 隐式 Treap（动态序列、翻转、仿射与区间和） . . . 304
3.5.012 可持久化队列 . . . 310
3.5.013 Radix Heap / 基数堆 . . . 313
3.6 Trie 与集合结构 . . . 315

3.6.001 字符 Trie . . . 315
3.6.002 01 Trie / 压位 Trie . . . 318
3.6.003 可删二进制 Trie . . . 323
3.6.004 Trie Graph . . . 324
3.6.005 线段树优化 Trie Graph . . . 326
3.6.006 哈希表 . . . 329
3.6.007 可持久化数组 . . . 333
3.6.008 手写 bitset . . . 335
3.6.009 动态区间频率索引 . . . 339
3.6.010 可持久化 01 Trie . . . 341
3.6.011 静态区间 MEX . . . 344
3.6.012 按标签维护的 01 Trie . . . 346
3.6.013 左儿子右兄弟 Trie . . . 348
3.6.014 分层位集前驱/后继集合 . . . 350
3.7 函数与空间数据结构 . . . 354
3.7.001 李超线段树 . . . 354
3.7.002 动态开点李超树 . . . 356
3.7.003 可持久化李超树 . . . 359
3.7.004 K-D Tree . . . 362
3.7.005 K-D Tree 二进制分组 . . . 365
3.7.006 分散层叠 . . . 368
3.7.007 双端队列全局查询 . . . 368
3.7.008 析合树（$O(n \log n)$ 构造） . . . 371
3.7.009 动态点集矩形仿射修改与矩形和 . . . 376
3.7.010 离散坐标线段李超树（线段插入、点查询） . . . 380
3.8 线性基 . . . 383
3.8.001 异或线性基 . . . 383
3.8.002 第 K 大线性基 . . . 386
3.8.003 区间线性基 . . . 388
3.8.004 线性基合并 . . . 390
3.9 GNU PBDS . . . 391
3.9.001 PBDS 头文件、命名空间、模板策略与迭代器体系 . . . 391
3.9.002 PBDS `priority_queue` 策略 . . . 392
3.9.003 PBDS 堆的 `modify`、`erase`、`join` 与点迭代器 . . . 393
3.9.004 PBDS 堆和 STL 堆、二叉堆、线段树的性能与适用范围 . . . 393
3.9.005 `tree`：`rb_tree_tag`、`splay_tree_tag`、`ov_tree_tag` . . . 394
3.9.006 PBDS `tree` 的 set/map 用法、`split` 与 `join` . . . 394
3.9.007 `tree_order_statistics_node_update` . . . 395
3.9.008 `find_by_order` 与 `order_of_key` . . . 395
3.9.009 自定义 `Node_Update`、`metadata_type` 与节点迭代器 . . . 395
3.9.010 自定义子树信息：区间和、前缀和及其他可合并信息 . . . 396
3.9.011 `cc_hash_table` 与 `gp_hash_table` . . . 398
3.9.012 哈希策略、探测策略、扩容策略与性能测试 . . . 398
3.9.013 编译器版本兼容性、常数、失效规则与不适用场景 . . . 398

3.10 根号分块 . . . 399
3.10.001 基础数列分块 . . . 399
3.10.002 有序数列分块 . . . 402
3.10.003 王室联邦树分块 . . . 406
3.10.004 图上按度数分块 . . . 407

4. 字符串 409
4.1 基础匹配与哈希 . . . 409
4.1.001 字符串哈希（Mersenne 单模、随机底数） . . . 409
4.1.002 KMP / 前缀函数 . . . 414
4.1.003 扩展 KMP / Z 函数 . . . 417
4.1.004 Manacher . . . 418
4.1.005 最小表示法 . . . 420
4.1.006 Lyndon 分解 . . . 421
4.1.007 Runs . . . 422
4.1.008 Karp-Rabin . . . 425
4.1.009 Sunday Algorithm . . . 425
4.1.010 `strstr` 类匹配 . . . 426
4.1.011 通配符模式匹配 . . . 427
4.1.012 随机单卷积通配符匹配 . . . 428
4.1.013 编辑距离与 Myers 位并行 . . . 430
4.2 Trie 与多模式匹配 . . . 432
4.2.001 Trie . . . 432
4.2.002 AC 自动机 . . . 434
4.2.003 AC 自动机 fail 树应用 . . . 436
4.2.004 子序列自动机 . . . 437
4.3 后缀结构 . . . 438
4.3.001 后缀数组：倍增法 . . . 438
4.3.002 后缀数组：SA-IS . . . 439
4.3.003 LCP / Height / RMQ 应用 . . . 443
4.3.004 后缀自动机 SAM . . . 446
4.3.005 广义 SAM / ExSAM . . . 450
4.3.006 在线广义 SAM . . . 454
4.3.007 SAMSA / CDAWG . . . 456
4.3.008 隐式后缀树 Ukkonen . . . 458
4.3.009 后缀平衡树（前插、回滚与动态排名） . . . 461
4.3.010 区间本质不同子串 . . . 467
4.3.011 Dictionary of Basic Factors . . . 469
4.3.012 BWT / FM-index . . . 474
4.4 回文结构 . . . 477
4.4.001 回文自动机 PAM / 回文树 . . . 477
4.4.002 双端可撤销回文树 . . . 479
4.4.003 Palindrome Series . . . 485
4.4.004 SqAM . . . 487
4.5 子序列与高级问题 . . . 488

4.5.001 最长公共子序列 LCS . . . 488
4.5.002 Bitset 优化 LCS . . . 489
4.5.003 区间 LCS / 区间 LIS . . . 490
4.5.004 多串最长公共子串/子序列 . . . 491
4.5.005 最短公共超序列 . . . 495
4.5.006 字符串周期、循环节与前后缀 . . . 496
4.5.007 前缀-子串 LCS 批量查询 . . . 496

5. 图论 499
5.1 图的表示与基础遍历 . . . 499
5.1.001 邻接表、链式前向星、邻接矩阵 . . . 499
5.1.002 DFS / BFS . . . 500
5.1.003 拓扑排序 . . . 501
5.1.004 二分图判定 . . . 502
5.1.005 DAG 最长路 . . . 503
5.1.006 DAG 删点最长路 . . . 505
5.1.007 差分约束 . . . 507
5.2 最短路 . . . 508
5.2.001 Dijkstra：稀疏图与稠密图 . . . 508
5.2.002 Bellman-Ford . . . 510
5.2.003 SPFA、SLF 优化与负环判定/输出 . . . 511
5.2.004 Floyd 与最小环 . . . 512
5.2.005 Johnson 全源最短路 . . . 514
5.2.006 区间图最短路 . . . 516
5.2.007 平面图对偶最短路 . . . 519
5.2.008 最短路径树 . . . 520
5.2.009 单源最短路/次短路计数 . . . 521
5.2.010 第 K 短路（Dijkstra / A*） . . . 524
5.2.011 0-1 BFS . . . 526
5.2.012 第 K 短游走（Eppstein） . . . 527
5.2.013 第 K 短简单路 Yen . . . 530
5.3 连通性 . . . 534
5.3.001 Tarjan 强连通分量 . . . 534
5.3.002 Kosaraju 强连通分量 . . . 536
5.3.003 Bitset 优化 Kosaraju . . . 537
5.3.004 割边与边双连通分量 . . . 538
5.3.005 割点与点双连通分量 . . . 541
5.3.006 缩点与块割树 . . . 543
5.3.007 圆方树、广义圆方树 . . . 544
5.3.008 无向图连通度、点连通度、边连通度 . . . 544
5.3.009 最佳/最小点割集与边割集 . . . 546
5.3.010 仙人掌森林分解 . . . 546
5.3.011 补图连通块 . . . 549
5.3.012 离线增量 SCC . . . 550
5.3.013 三边连通分量 . . . 552

5.4 2-SAT 554
5.4.001 Tarjan 2-SAT 554
5.4.002 Kosaraju 2-SAT 556
5.4.003 字典序或答案不唯一时的构造 558
5.5 最小生成结构 560
5.5.001 Kruskal 560
5.5.002 次小生成树 562
5.5.003 K 棵树的最小生成森林 566
5.5.004 带修改最小生成树 567
5.5.005 最小乘积生成树 568
5.5.006 切比雪夫距离最小生成树 572
5.5.007 平面欧氏距离最小生成树 574
5.5.008 最小树形图 / 朱刘算法 576
5.5.009 最小斯坦纳树 578
5.5.010 Kruskal 重构树 580
5.5.011 增量最小生成森林 584
5.5.012 最小直径生成树 589
5.5.013 Prim：稀疏图与稠密图 593
5.5.014 Borůvka 595
5.6 二分图匹配与覆盖 596
5.6.001 匈牙利算法 596
5.6.002 Bitset 优化匈牙利 598
5.6.003 Hopcroft-Karp 600
5.6.004 CSR Hopcroft-Karp 603
5.6.005 二分图最大权/最小权匹配（KM） 606
5.6.006 二分图边染色 610
5.6.007 最小点覆盖、最小边覆盖 615
5.6.008 最大独立集 617
5.6.009 DAG 最小不相交链覆盖 619
5.6.010 DAG 最大互不可达集 620
5.6.011 最大权闭合子图 622
5.7 一般图匹配 623
5.7.001 一般图最大匹配（Blossom） 623
5.7.002 一般图最大权匹配 627
5.7.003 Tutte Matrix 628
5.7.004 Edmonds Matrix 629
5.7.005 稳定婚姻 629
5.7.006 一般图最大权匹配（带权 Blossom） 631
5.7.007 稀疏稳定匹配（不完全偏好） 639
5.8 网络流与割 641
5.8.001 Dinic 最大流 641
5.8.002 FIFO 预流推进 644
5.8.003 SPFA 多路增广最小费用流 647
5.8.004 原始对偶最小费用流 650

5.8.005 有上下界可行流、最大流、最小流 . . . 654
5.8.006 最大流最小割与建图模型 . . . 656
5.8.007 Stoer-Wagner 全局最小割 . . . 658
5.8.008 Gomory-Hu 最小割树 . . . 660
5.8.009 ISAP 最大流 . . . 661
5.8.010 HLPP / 最高标号预流推进 . . . 664
5.8.011 费用缩放最小费用 b 流与带费用上下界 . . . 670
5.8.012 动态星增广图全局最小割 . . . 675
5.8.013 CSR 优化原始对偶最小费用流 . . . 680
5.8.014 有向图全局最小割 . . . 686
5.9 欧拉图与环 . . . 687
5.9.001 有向/无向欧拉路径与欧拉回路 . . . 687
5.9.002 字典序最小欧拉路径 . . . 692
5.9.003 BEST 定理与有向欧拉回路计数 . . . 692
5.9.004 无向图最小环 . . . 693
5.9.005 最小平均环 / 最小密度环 . . . 694
5.9.006 三元环、四元环计数 . . . 697
5.9.007 简单环存在性、输出与计数 . . . 703
5.9.008 基环树 . . . 707
5.9.009 函数图与后继跳跃 . . . 710
5.10 特殊图与图论结论 . . . 714
5.10.001 弦图、完美消除序列与判错 . . . 714
5.10.002 欧拉公式与圆上弦分侧 . . . 720
5.10.003 支配树（DAG / 一般图） . . . 721
5.10.004 竞赛图与 Landau 定理 . . . 725
5.10.005 度序列可图性 / Erdős-Gallai . . . 725
5.10.006 最大团、极大团计数 . . . 726
5.10.007 最大独立集与点染色 . . . 731
5.10.008 DSATUR 实践型精确点染色 . . . 732
5.10.009 有向图最小点基 . . . 735
5.10.010 有向无环定向与色多项式 . . . 736
5.10.011 拟阵交 . . . 742
5.10.012 小规模暴力双极定向 . . . 744
5.10.013 线性 st 编号 . . . 746
5.10.014 宽度 2 树分解 . . . 749
5.10.015 Ramsey 定理 . . . 751
5.10.016 外环加弦平面性判定（2-SAT） . . . 752
5.10.017 Havel-Hakimi / Gale-Ryser 构造 . . . 753
5.10.018 小图一般图同构 . . . 756

6. 树论与动态树 760
6.1 树的基础性质 . . . 760
6.1.001 树的直径、重心、中心 . . . 760
6.1.002 最近公共祖先：倍增 . . . 763
6.1.003 最近公共祖先：树链剖分 . . . 765

6.1.004 最近公共祖先：ST 表 / RMQ . . . 765
6.1.005 最近公共祖先：Tarjan 离线 . . . 768
6.1.006 树上距离、K 级祖先 . . . 769
6.1.007 树上路径交 . . . 771
6.1.008 树哈希与基环树哈希 . . . 772
6.1.009 Prüfer 序列与树的计数 . . . 777
6.1.010 树的邻接表转前序表 . . . 779
6.1.011 公共区间分解树 . . . 780
6.2 树上分治与启发式合并 . . . 784
6.2.001 点分治 . . . 784
6.2.002 点分树 . . . 787
6.2.003 边分治 . . . 790
6.2.004 DSU on Tree . . . 793
6.2.005 长链剖分 . . . 795
6.2.006 毛毛虫剖分 . . . 795
6.3 树链与虚树 . . . 795
6.3.001 轻重链剖分 / HLD . . . 795
6.3.002 换根树剖 . . . 798
6.3.003 虚树 . . . 801
6.3.004 树上路径/子树数据结构 . . . 803
6.3.005 树上差分 . . . 806
6.4 动态树 . . . 808
6.4.001 Link-Cut Tree . . . 808
6.4.002 带子树信息的 LCT . . . 813
6.4.003 动态割边、割点 LCT . . . 818
6.4.004 全局平衡二叉树 . . . 824
6.4.005 动态 DP . . . 827
6.4.006 离线动态图连通块点权和 . . . 831
6.4.007 Euler Tour Tree . . . 834
6.5 树形 DP . . . 839
6.5.001 树上背包 . . . 839
6.5.002 单调队列优化树上背包 . . . 841
6.5.003 长链剖分 DP 合并 . . . 842
6.5.004 树上依赖背包 . . . 844
6.5.005 有根树最小逆序拓扑序 . . . 846
6.5.006 换根 DP / Rerooting . . . 849

7. 数论 852
7.1 整数运算与模运算 . . . 852
7.1.001 快速幂 . . . 852
7.1.002 GCD、快速 GCD、LCM . . . 853
7.1.003 扩展欧几里得 . . . 854
7.1.004 线性同余方程与同余方程组 . . . 855
7.1.005 CRT / exCRT . . . 856
7.1.006 模逆元：快速幂、exgcd、线性与批量 . . . 857

7.1.007 静态取模类 . . . 858
7.1.008 动态取模类 . . . 858
7.1.009 Barrett 约减 . . . 861
7.1.010 Montgomery 模乘 . . . 861
7.1.011 Long Long 防溢出乘法 . . . 864
7.1.012 快速整除判定 . . . 864
7.1.013 模意义真分数还原 . . . 865
7.1.014 整数 k 次方根 . . . 866
7.1.015 线性同余不等式 . . . 867
7.1.016 基于值域优化的 GCD . . . 868
7.1.017 高斯整数与最大公约数 . . . 870
7.2 素数与因数分解 . . . 872
7.2.001 埃氏筛、线性筛 . . . 872
7.2.002 区间筛 . . . 874
7.2.003 Miller-Rabin . . . 875
7.2.004 Pollard-Rho . . . 877
7.2.005 质因数分解 . . . 879
7.2.006 约数枚举、约数个数和 . . . 880
7.2.007 因数最多的数 . . . 881
7.2.008 阶乘最后非零位 . . . 881
7.2.009 阶乘模合数 . . . 882
7.2.010 哥德巴赫猜想及竞赛用结论 . . . 884
7.2.011 麦乐鸡定理 / 两数 Frobenius 问题 . . . 885
7.3 算术函数与筛法 . . . 885
7.3.001 欧拉函数 . . . 885
7.3.002 莫比乌斯函数 . . . 886
7.3.003 积性函数 . . . 887
7.3.004 Dirichlet 卷积 . . . 888
7.3.005 莫比乌斯反演 . . . 889
7.3.006 杜教筛 . . . 890
7.3.007 Min_25 筛 . . . 893
7.3.008 Meissel-Lehmer 质数计数 . . . 896
7.3.009 整除分块 . . . 898
7.3.010 调和级数分块 . . . 899
7.3.011 素数、$\omega(n)$ 与 $d(n)$ . . . 899
7.3.012 商集上的 Dirichlet 卷积与除法 . . . 900
7.3.013 无平方因子数计数 . . . 905
7.4 离散对数与剩余 . . . 907
7.4.001 BSGS / exBSGS . . . 907
7.4.002 Pohlig-Hellman . . . 910
7.4.003 原根与全部原根筛 . . . 911
7.4.004 二次剩余 . . . 912
7.4.005 K 次剩余 . . . 914
7.4.006 扩展欧拉定理与欧拉降幂 . . . 915

7.4.007 Carmichael 函数 . . . 918
7.4.008 固定模批量离散对数 . . . 918
7.4.009 Hensel 简单根提升 . . . 921
7.5 类欧几里得 . . . 922
7.5.001 `floor_sum` . . . 922
7.5.002 类欧几里得格点统计 . . . 923
7.5.003 万能欧几里得 . . . 924
7.5.004 `Min of Mod of Linear` . . . 929
7.5.005 Stern-Brocot Tree / Farey 序列 . . . 929
7.5.006 Pell 方程 . . . 931
7.5.007 圆上整点 . . . 932

8. 组合数学 934
8.1 组合数 . . . 934
8.1.001 阶乘、逆阶乘与组合数 . . . 934
8.1.002 杨辉三角 . . . 935
8.1.003 Lucas 定理 . . . 936
8.1.004 扩展 Lucas . . . 937
8.1.005 任意模数组合数 . . . 940
8.1.006 Vandermonde 卷积 . . . 941
8.1.007 容斥原理 . . . 942
8.1.008 扩展 min-max 容斥与 KMAX-MIN 反演 . . . 943
8.1.009 单位根反演 . . . 945
8.1.010 球盒模型与鸽巢原理 . . . 946
8.1.011 q-二项式与 q-Lucas . . . 948
8.1.012 二项式恒等式与反演 . . . 950
8.2 经典数列与计数 . . . 951
8.2.001 Fibonacci / Lucas 数 . . . 951
8.2.002 Catalan 数 . . . 952
8.2.003 Motzkin 数 . . . 953
8.2.004 Bell 数 . . . 953
8.2.005 Stirling 数 . . . 954
8.2.006 Eulerian 数 . . . 956
8.2.007 Bernoulli 数与幂和 . . . 956
8.2.008 错排数 . . . 957
8.2.009 分拆数与五边形数 . . . 958
8.2.010 有根树、无根树计数 . . . 959
8.2.011 Matrix-Tree / Kirchhoff 生成树计数 . . . 960
8.2.012 Pólya / Burnside 置换计数 . . . 962
8.2.013 杨氏矩阵与钩子公式 . . . 964
8.2.014 固定 k 的第一类 Stirling 数 . . . 966
8.2.015 固定 k 的第二类 Stirling 数 . . . 967
8.2.016 小素数大下标第一类 Stirling 数 . . . 968
8.2.017 小素数大下标第二类 Stirling 数 . . . 970
8.3 生成与构造 . . . 972

8.3.001 排列组合生成 . . . 972
8.3.002 康托展开与逆展开 . . . 973
8.3.003 勾股数枚举 . . . 975
8.3.004 阿达马矩阵 . . . 976
8.3.005 幻方 . . . 977

9. 线性代数与数值算法 980
9.1 矩阵 . . . 980
9.1.001 普通矩阵与取模矩阵 . . . 980
9.1.002 矩阵快速幂 . . . 982
9.1.003 Frobenius 标准形矩阵快速幂（高级版） . . . 983
9.1.004 `+min` / `+max` 半环矩阵 . . . 991
9.1.005 矩阵求逆（质数模数 / 任意模数） . . . 993
9.1.006 行列式（质数模数 / 任意模数） . . . 997
9.1.007 矩阵特征多项式 . . . 1001
9.1.008 稀疏矩阵 . . . 1006
9.1.009 Strassen 矩阵乘法 . . . 1007
9.1.010 Woodbury 矩阵恒等式 . . . 1009
9.1.011 伴随矩阵 . . . 1013
9.1.012 Hafnian 与一般图完美匹配计数 . . . 1016
9.1.013 Permanent / Ryser . . . 1018
9.1.014 GF(2) 位集高斯消元 . . . 1019
9.2 线性方程与规划 . . . 1022
9.2.001 高斯消元：唯一解、无解、多解 . . . 1022
9.2.002 高斯消元：列主元 . . . 1027
9.2.003 高斯消元最小范数解 . . . 1028
9.2.004 线性基（参见数据结构） . . . 1031
9.2.005 Simplex 单纯形 . . . 1031
9.2.006 0-1 分数规划 . . . 1035
9.2.007 追赶法解三对角/周期方程 . . . 1037
9.2.008 Pfaffian . . . 1039
9.2.009 对称矩阵 LDL^T 分解 . . . 1040
9.3 数值计算 . . . 1043
9.3.001 自适应 Simpson . . . 1043
9.3.002 Romberg 积分 . . . 1044
9.3.003 Newton 多项式求根 . . . 1045
9.3.004 一元三次方程 . . . 1046
9.3.005 线性插值与幂和 . . . 1049
9.3.006 指数乘多项式有限和 . . . 1051
9.3.007 指数乘多项式无穷和 . . . 1053
9.3.008 LLL 浮点实用版 . . . 1054

10. 多项式、变换与线性递推 1057
10.1 卷积与变换 . . . 1057
10.1.001 DFT / IDFT . . . 1057

10.1.002 FFT . . . 1057
10.1.003 NTT . . . 1060
10.1.004 MTT 任意模数卷积 . . . 1072
10.1.005 二元 NTT / 二元多项式卷积 . . . 1078
10.1.006 FWT / FST . . . 1081
10.1.007 偶模 FWT 与光速乘 . . . 1087
10.1.008 K 进制 FWT . . . 1089
10.1.009 全在线卷积：基准版与 NTT 版 . . . 1090
10.1.010 整除关系卷积 . . . 1092
10.1.011 子集卷积 . . . 1093
10.1.012 Min-Plus 卷积 . . . 1095
10.1.013 集合形式幂级数 . . . 1099
10.1.014 多元卷积 . . . 1104
10.1.015 模 2^n 乘法卷积 . . . 1108
10.1.016 $GF(2^{64})$ 卷积 . . . 1110
10.1.017 模素数乘法幺半群卷积 . . . 1118
10.1.018 模 2^{64} 精确整数卷积 . . . 1119
10.2 多项式基础运算 . . . 1120
10.2.001 多项式封装 . . . 1120
10.2.002 乘法、除法、取模 . . . 1123
10.2.003 乘法逆元 . . . 1125
10.2.004 导数、积分 . . . 1128
10.2.005 ln、exp、开根、快速幂 . . . 1129
10.2.006 稀疏多项式除法 . . . 1136
10.2.007 多点求值 . . . 1137
10.2.008 拉格朗日单点求值、多点插值 . . . 1142
10.2.009 连续点值插值与点值平移 . . . 1143
10.2.010 快速阶乘 . . . 1146
10.2.011 Power Projection . . . 1149
10.2.012 多项式复合 . . . 1151
10.2.013 下降幂多项式乘法 . . . 1154
10.2.014 稀疏 FPS 运算 . . . 1155
10.2.015 有限域多项式求根与因式分解 . . . 1161
10.2.016 等差、等比节点求值与插值 . . . 1172
10.2.017 二元形式幂级数逆 . . . 1175
10.2.018 多项式序列乘积 . . . 1177
10.2.019 单项式基转 Newton 基 . . . 1178
10.2.020 多项式 half-GCD 与模逆 . . . 1180
10.2.021 拉格朗日非连续点单点求值 . . . 1187
10.2.022 计数版子集和生成函数 . . . 1188
10.3 生成函数 . . . 1189
10.3.001 普通生成函数 OGF . . . 1189
10.3.002 指数生成函数 EGF . . . 1190
10.3.003 q-int / q-Binomial . . . 1191

10.3.004 二项式型生成函数 . . . 1192
10.3.005 拉格朗日反演 . . . 1192
10.3.006 两边界间非降序列计数 . . . 1195
10.4 线性递推 . . . 1200
10.4.001 Berlekamp-Massey（BM / 最小线性递推） . . . 1200
10.4.002 最短递推式 . . . 1202
10.4.003 Linear Recurrence . . . 1202
10.4.004 递推式转分式 . . . 1204
10.4.005 Bostan-Mori / 分式第 N 项 . . . 1205
10.4.006 有理生成函数远处连续区间 . . . 1206

11. 计算几何 1210
11.1 二维基础对象 . . . 1210
11.1.001 浮点误差与精确整数几何 . . . 1210
11.1.002 Point / Vector . . . 1210
11.1.003 点积、叉积、长度、夹角、旋转 . . . 1213
11.1.004 极角排序与象限分类 . . . 1214
11.1.005 Line / Ray / Segment . . . 1215
11.1.006 平行、垂直、共线、同侧/异侧 . . . 1216
11.1.007 直线交点、投影、对称点 . . . 1217
11.1.008 点到直线/线段距离 . . . 1218
11.1.009 线段相交与距离 . . . 1219
11.1.010 线段中垂线 . . . 1221
11.1.011 四点组成矩形 / 正方形判定 . . . 1221
11.1.012 三角形内心、外心、垂心与旁心 . . . 1222
11.2 多边形 . . . 1223
11.2.001 多边形面积、周长、重心 . . . 1223
11.2.002 点在多边形内与有符号绕数 . . . 1224
11.2.003 线段是否在多边形内 . . . 1225
11.2.004 多边形切割 . . . 1227
11.2.005 多边形的核 . . . 1227
11.2.006 Ear Clipping 三角剖分 . . . 1228
11.2.007 多边形交、并 . . . 1230
11.2.008 多边形与圆面积交 . . . 1234
11.2.009 Pick 定理与格点多边形 . . . 1236
11.2.010 三角形点集批量计数 . . . 1237
11.2.011 多个简单多边形的覆盖层面积 . . . 1239
11.2.012 直线平面嵌入的面枚举与对偶图 . . . 1245
11.3 凸包 . . . 1250
11.3.001 Andrew / Graham 静态凸包 . . . 1250
11.3.002 单点插入动态凸包 . . . 1251
11.3.003 点在凸包内 . . . 1253
11.3.004 凸包极点、切点、最近点查询 . . . 1254
11.3.005 凸包切线 . . . 1257
11.3.006 旋转卡壳、凸包直径 . . . 1258

11.3.007 最大面积三角形、四边形 . . . 1259
11.3.008 闵可夫斯基和 . . . 1261
11.3.009 半平面交 . . . 1262
11.3.010 整数半平面交 . . . 1267
11.3.011 Delaunay 三角剖分 / Voronoi 图 . . . 1271
11.3.012 三角形四心（内心、外心、垂心、费马点） . . . 1280
11.3.013 凸包分层 . . . 1281
11.3.014 点集最小 / 最大面积三角形 . . . 1286
11.4 圆 . . . 1290
11.4.001 点、线、圆的位置关系 . . . 1290
11.4.002 直线与圆、两圆交点 . . . 1292
11.4.003 两圆相交面积 . . . 1293
11.4.004 圆的切线与两圆公切线 . . . 1293
11.4.005 三点外接圆、三角形内切圆 . . . 1295
11.4.006 最小圆覆盖 . . . 1296
11.4.007 固定半径圆覆盖最多点 . . . 1299
11.4.008 圆反演与阿波罗尼斯圆 . . . 1301
11.4.009 圆并与多圆覆盖层面积 . . . 1302
11.4.010 最近圆对 . . . 1306
11.4.011 圆弧表示与圆弧重心 . . . 1307
11.4.012 圆上整点 . . . 1308
11.5 距离、扫描线与特殊坐标 . . . 1309
11.5.001 平面最近点对 . . . 1309
11.5.002 最远曼哈顿距离 . . . 1312
11.5.003 曼哈顿距离与切比雪夫距离转换 . . . 1312
11.5.004 矩形面积并、周长并 . . . 1313
11.5.005 六边形坐标系 . . . 1316
11.5.006 球面经纬度与球面距离 . . . 1317
11.5.007 多线段任意相交对 . . . 1318
11.6 三维几何 . . . 1323
11.6.001 三维点、向量、直线、平面 . . . 1323
11.6.002 共线、共面、平行、垂直 . . . 1324
11.6.003 直线与平面、两平面求交 . . . 1327
11.6.004 点线面距离与夹角 . . . 1328
11.6.005 空间线段与三角形相交 . . . 1329
11.6.006 三维凸包 . . . 1331
11.6.007 最小覆盖球 . . . 1333
11.6.008 四面体体积与内切球 . . . 1336
11.6.009 高维球体积、超球坐标与立体角 . . . 1336
11.6.010 投影、最近点与空间距离 . . . 1337
11.6.011 射线与 AABB 查询 . . . 1341
11.6.012 空间三角形与平面裁剪 . . . 1345
11.6.013 球、空间圆与球冠 / 球带 . . . 1350
11.6.014 共面基底、凸包与网格截面 . . . 1355

11.6.015 反射、二面角与球面多边形 1360

12. 动态规划与优化 1364
12.1 背包 1364
12.1.001 01 背包 1364
12.1.002 完全背包 1364
12.1.003 多重背包 1365
12.1.004 混合背包 1366
12.1.005 二维费用背包 1367
12.1.006 分组背包 1368
12.1.007 有依赖背包 1369
12.1.008 背包方案数与具体方案 1370
12.1.009 树上背包及单调队列优化 1371
12.1.010 布尔母函数 / 有界子集和可达性 1373
12.1.011 基础区间 DP 1374
12.2 常见 DP 1377
12.2.001 数位 DP 1377
12.2.002 状压 DP 1380
12.2.003 棋盘分割 1381
12.2.004 两台机器工作调度 1383
12.2.005 找硬币问题 1384
12.2.006 汉诺塔 1385
12.2.007 最长上升/公共子序列 1385
12.2.008 子集和 Bitset 优化 1386
12.2.009 树形 DP 与动态 DP 1388
12.2.010 不同子序列计数 1389
12.2.011 插头 DP 1390
12.3 DP 优化 1394
12.3.001 单调队列优化 1394
12.3.002 四边形不等式优化 1395
12.3.003 分治优化 1396
12.3.004 斜率优化 / 李超树优化 1398
12.3.005 WQS 二分 1400
12.3.006 SMAWK 全单调矩阵优化 1402
12.3.007 Slope Trick 1403

13. 博弈论、概率与杂项数学 1407
13.1 博弈论 1407
13.1.001 Bash 与扩展 Bash 1407
13.1.002 Nim、Anti-Nim、Nim-K 1407
13.1.003 阶梯 Nim 1408
13.1.004 SG、Anti-SG、Multi-SG、Every-SG 1408
13.1.005 Wythoff 博弈 1410
13.1.006 Fibonacci 博弈 1411
13.1.007 树上删边、无向图删边游戏 1412

13.1.008 取石子游戏 . . . 1413
13.1.009 64 位 nim-product . . . 1414
13.2 概率与分布 . . . 1416
13.2.001 常见概率公式 . . . 1416
13.2.002 常见概率分布 . . . 1417
13.2.003 随机变量期望与方差 . . . 1419
13.2.004 随机算法与 Xorshift . . . 1420
13.3 常用公式表 . . . 1421
13.3.001 三角恒等式、导数与不定积分表 . . . 1421
13.3.002 调和数与幂和 . . . 1426
13.3.003 三角形、四边形与圆公式 . . . 1428
13.3.004 常用面积、体积公式 . . . 1429
13.3.005 日期公式：Zeller / 基姆拉尔森 . . . 1430
13.3.006 常用常数与 NTT / 哈希素数表 . . . 1430
13.3.007 数论数量级与极值表 . . . 1432
13.3.008 裂项相消、有限差分与几何级数 . . . 1434
13.3.009 泛型复数与三次单位根代数 . . . 1435

14. 其他实用模板 1440
14.1 其他实用模板 . . . 1440
14.1.001 高精度整数（BigInt） . . . 1440
14.1.002 分数类与有理数还原 . . . 1448
14.1.003 表达式解析：中缀转后缀、表达式树 . . . 1449
14.1.004 德州扑克牌型判断（五张牌与多手牌最佳五张） . . . 1453
14.1.005 日期与时间换算 . . . 1456
14.1.006 高精度进制转换与快速幂 . . . 1458
14.1.007 9×9 数独字典序最小解 . . . 1459
14.1.008 装箱问题 . . . 1461
14.1.009 选数与网格路径计数 . . . 1463
14.1.010 坐标类 . . . 1464
14.1.011 Python：基础语法、容器、字符串、数学库 . . . 1464
14.1.012 Python：并查集、线段树、最大流、费用流、FFT、NTT . . . 1466
14.1.013 Python：Splay . . . 1472
14.1.014 Python：`datetime`、`Fraction`、递归深度与文件 IO . . . 1476
14.1.015 完全图部分边染色的 1-因子扩展尝试 . . . 1477
14.1.016 高精度整数平方根 . . . 1480
14.1.017 一元二次方程实根 . . . 1483
14.1.018 N 皇后构造解 . . . 1484
14.1.019 十六进制百万位大整数 . . . 1486
14.1.020 任意精度定点与十进制数 . . . 1500
14.1.021 高精度圆周率：Machin 与 Chudnovsky . . . 1509

15. ACM 常用英文单词 1512
15.1 按字母索引 . . . 1512
15.1.001 A . . . 1512

15.1.002 B 1513
15.1.003 C 1513
15.1.004 D 1514
15.1.005 E 1515
15.1.006 F 1516
15.1.007 G 1516
15.1.008 H 1517
15.1.009 I 1517
15.1.010 J 1518
15.1.011 K 1518
15.1.012 L 1518
15.1.013 M 1519
15.1.014 N 1520
15.1.015 O 1520
15.1.016 P 1521
15.1.017 Q 1521
15.1.018 R 1522
15.1.019 S 1522
15.1.020 T 1524
15.1.021 U 1524
15.1.022 V 1525
15.1.023 W 1525

0. 比赛注意

0.1 赛前与比赛周

到赛地后记住自己是来比赛的，状态比临时多学一点更重要。

1. 赛前一天睡够 10 小时以上；前一天晚饭和当天早饭要吃好。
2. 到新环境注意防病。感冒、肠炎不算大病，却最影响成绩。
3. 比赛周不学新算法，也不连续几天高强度训练。可以看看 M67 等博客、翻小学奥数题，保持思维活跃。
4. 平时每场练习赛都按正式比赛打：所有题都要看过，赛后尽量补完没做出的题。
5. 有条件就看看往届比赛的总结、照片和录像，熟悉正式赛场，避免临场紧张到做不出题。
6. 队内最好三人都能做各类题，但各有侧重。要保证每出现一道题，都有人会做、敢做，至少知道做法。

0.2 赛时心态与分工

可以紧张，但不能慌张，慌张最影响出题。任何时候发现自己或队友慌了，就提醒对方深呼吸。

1. 第一道签到题交给队里最稳的人。遇到麻烦也别慌，很多队都过了就更不用急：它一定是简单题，晚几分钟也比罚时 20 分钟好。
2. 除签到题外尽量每人都看，并写下自己一眼能看出的观察。无论有人过没过，所有题都必须读完，最好每题都有两人以上读过，尽量避免靠讲题传题意。
3. 完全弄清题意，正确判断难易，不要想当然。觉得某题更适合队友做就转手；非万不得已不要中途换人写同一题。
4. 英语看不懂就查词典，偷懒一次可能就少一道题。
5. 比赛发的餐食留到赛后吃。
6. 最后一小时是出题高峰，谁松懈谁落后。最后一小时出一道是正常，出两道更好。
7. 没思路或头晕时，可以去洗手间洗把脸，或在座位旁站一会儿。

0.3 开题与选题

1. 时空效率危险且代码量大的题优先级放低。
2. 除非很多队都过了，复杂几何题留到最后考虑，精度带来的问题太多。
3. 非签到的构造题往往要想很久，过的队不多时不要开。
4. 尽量多想题目能用上的数学性质。
5. 常规方法不行时换思路：不常见的数据结构、加预处理、限时搜索等。效率第一，觉得 DP 难写就用记忆化搜索，想清楚后用最短时间出题。
6. 卡在一道全场通过率很高的题（按本队实力，例如超过 60% 的队伍通过）超过 1 小时，可以请求整队重做这题，不沿用任何旧代码。

0.4 上机与打印

1. 先在纸上把程序写得尽量完整再上机。每道题上机时间（输入、测试、调试合计）不超过 1 小时。

2. 有队友等着写题时，当前 coder 对着键盘发呆超过 3 分钟就立即下机，说明这题还没想好；下机时打印代码。
3. 程序出错一时排除不了，就打印出来读，把机器让给队友。
4. 每道题交完先打印，不等评测结果，直接写下一题。
5. 少复制粘贴，重复逻辑写成函数便于修改。必须块复制时，逐处检查对应部分是否已经改对。
6. 照纸敲代码和样例时不要敲错，尤其是字符串、提示文字等文本信息；不要自作主张改动模板。

0.5 卡题

不要急。通过队伍多的题不一定难，不要一开始就把题目想得过于复杂。

1. 一个人想 5 分钟没有思路，立即找队友讨论；再过 10 分钟还不会，再找第三个队友。队友在忙就先放下这题去看别的，并留下观察。
2. 把观察到的任何有用的点都记下来，不论看起来多显然。
3. 重新仔细阅读题面，思考每个可能有用的条件，尤其是数据范围。
4. 尝试倒着思考，从目标状态或答案反推必要条件。
5. 在题单上写下已有想法，整理已经确定的结论和仍待验证的分支。
6. 寻求队友帮助时直接说明卡住的具体位置，不要只转述题意。
7. 猜到结论但证明不了时，多造几组数据验证结论。
8. 要否定队友的结论，给出一组反例，而不是只说”显然不对”。
9. 觉得思路正确时多测试几个样例，优先补充边界情况。

0.6 提交前

恭喜你马上要通过这一题了，但不要因为样例正确就放松检查。比赛中要比平时更稳，尽量 1Y。

1. 手算至少 3 组样例，必须包含边界情况；这通常花费不到 5 分钟，不要省略。
2. 检查程序中的关键位置、需要初始化的变量和数组大小。
3. 重新估算时间复杂度和空间复杂度，确认还有没有可优化的部分。
4. 删除调试输出和临时断言。
5. 提交代码。
6. 打印代码前再确认提交的是最后修改并验证过的版本。

0.7 AC

恭喜，通过后去看下一题；若这份代码以后要写入模板，再补充边界测试和接口说明。

0.8 WA

能通过样例只说明代码距离正确不远，不能说明实现已经覆盖所有情况。压力下也要仔细：不要改一处就交，很可能还有别处错误。

1. 手算至少 5 组样例，必须包含边界情况。测试要完整，每个分支都测到，数据不必特别复杂，例如矩阵为 1×1、$1 \times n$、$m \times 1$。
2. 检查是否会溢出 `long long` 或 `__int128` 的取值范围。
3. 检查所有需要初始化的变量。
4. 删除调试信息，确认调试分支不会影响提交代码。
5. 检查数组的行列含义是否写反。
6. 检查排序方向是从小到大还是从大到小，严格与非严格关系是否一致。
7. 检查输出格式是否匹配题面，包括前导零、空格、换行和精度。
8. 使用随机数据进行多组对拍，缩小到最小反例。
9. 重新读一遍题意，逐句确认自己的理解没有遗漏。
10. 让队友根据题面独立构造几组样例。
11. 使用小黄鸭调试法，逐行解释代码并重点检查变量名是否写错。

0.9 TLE

程序已经接近正确时，先排除实现死循环，再确认复杂度和数据规模。

1. 优先检查是否存在死循环、递归不收敛或队列没有及时清空。
2. 重新核对复杂度，确认最坏情况而不是平均情况能够通过。
3. 检查读入数据的格式和题目约定是否一致，避免把错误输入当成算法瓶颈。
4. 评估能否用线性数据结构替代 `map`、`set` 等对数结构。
5. 删除调试输出。
6. 识别可以预处理的量，避免在每次查询或每个状态中重复计算。
7. 使用小黄鸭调试法，逐行检查循环边界、状态转移和变量名。

0.10 RE

运行时错误通常来自越界、非法数学运算或资源使用不当。先根据错误类型缩小范围。

1. 检查数组是否开小，所有下标是否满足题面和实现的边界。
2. 检查是否存在除以零、对负数开平方等非法运算。
3. 检查输入格式是否和题目匹配，读入失败后是否继续使用了无效数据。
4. 检查数组是否开得过大；内存超限有时会以 RE 的形式返回。
5. 检查循环边界和递归终止条件。
6. 使用小黄鸭调试法，逐行检查变量名、引用和指针是否写错。

1. 基础与工程

1.1 基础代码框架

1.1.001 C++ 通用头文件与类型别名

统一头文件、整数宽度、无穷大与浮点容差是全书其余代码的公共前导，后续章节不再重复。`#define int long long` 必须写在系统头之后；若先于 `<bits/stdc++.h>` 展开，标准库中的 `int` 也会被替换，代码无法编译。宏展开后 `INF` 的类型和值分别是 `long long` 与 `numeric_limits<long long>::max()/4`。它只适合在已经证明数值范围受控时作哨兵；可能接近完整整数范围的算法必须把“不可达”与数值分开表示。

```cpp
#include <bits/stdc++.h>

using namespace std;

#define int long long

constexpr long long INF = numeric_limits<long long>::max() / 4;
constexpr long double EPS = 1E-12L;
```

一般整数按题目范围使用 `int`；外部二进制格式或系统接口要求固定位宽时使用 `int`、`unsigned long long` 等明确类型。乘积可能超 9×10^{18} 时在运算前转成 `__int128`，见 1.2.002 `__int128` 输入输出。

1.1.002 标准 `main` / 多测框架

`signed main()` 是规避 `int` 宏的常用写法，因为单独的 `signed` 等价于未被宏替换的 `signed int`。它不是唯一写法；也可以不用该宏，或在入口前 `#undef int` 后使用标准的 `int main()`。单测逻辑放进 `solve()`，多测只在 `main` 中循环调用，便于对拍时复用同一份 `solve()`。

```cpp
void solve()
{
    // 处理一组数据，n 为本组规模
    int n;
    cin >> n;
}

signed main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t = 1;
    cin >> t;          // 单测题删除本行，保留 t = 1
    while (t--)
    {
        solve();
    }
}
```

交互题不要解绑 cin/cout，且每次询问后 cout << flush，否则输出滞留缓冲区会导致死等。框架额外开销 $O(t)$，总复杂度为各组 solve() 之和。

1.1.003 常量、无穷大、浮点误差与符号函数

不能用浮点字面量表达要求精确的整数，例如 100000000000000001LL 转成 double 后会舍入；纯整数常量应写整数后缀。sgn 只处理“按给定绝对容差与零比较”，返回 -1、0 或 1。

```cpp
// 依赖 1.1.001 C++ 通用头文件与类型别名
int sgn(long double x, long double eps = EPS)
{
    // 返回 x 相对 0 的符号：-1、0 或 1
    return (x > eps) - (x < -eps);
}
```

用法如 if (sgn(cross, eps) == 0) 判共线。eps 必须按数据尺度和量纲选择；比较两个一般浮点数时还要考虑相对误差或 ULP，不能把固定绝对 EPS 当成通用判据。整数几何优先使用精确叉积。

1.1.004 常用函数重载与容器输出

chmin/chmax 把“比较、赋值、返回是否更新”三步并成一次调用，返回值还能顺带记录转移来源（如最短路的前驱）。vector 的流重载只为本地对拍省样板，正式提交若与他人代码合并需留意命名冲突。

```cpp
template <class T> bool chmin(T &x, const T &y)
{
    // y < x 时把 x 降到 y 并返回 true
    if (y < x)
    {
        x = y;
        return true;
    }
    return false;
}

template <class T> bool chmax(T &x, const T &y)
{
    // x < y 时把 x 升到 y 并返回 true
    if (x < y)
    {
        x = y;
        return true;
    }
    return false;
}

template <class T> istream &operator>>(istream &in, vector<T> &a)
{
    // 按 a 的既有长度依次读入
    for (auto &x : a)
```

```
    {
        in >> x;
    }
    return in;
}

template <class T> ostream &operator<<(ostream &out, const vector<T> &a)
{
    // 空格分隔输出，不带行尾空格
    for (int i = 0; i < (int)a.size(); i++)
    {
        if (i)
        {
            out << ' ';
        }
        out << a[i];
    }
    return out;
}
```

`vector<int> a(n); cin >> a;` 依赖预设长度，重载不改变容器大小。`chmin/chmax` 要求两个实参类型相同，避免混合有符号比较或隐式窄化；需要转换时由调用处显式完成。更新为 $O(1)$，容器读写为 $O(n)$。

1.1.005 Lambda 递归写法

只服务单个算法的 DFS 留在函数内部，避免全局状态与冗长函数名。C++ 里 lambda 无法直接引用自身，用 `auto &&self` 显式传入（Y 组合子写法）；有返回值时必须写出 `→ T`，否则编译器推不出递归返回类型。

```
auto dfs = [&](auto &&self, int x, int fa) -> void
{
    // self 传自身用于递归，fa 为父结点，dep 记录深度
    for (auto y : g[x])
    {
        if (y == fa)
        {
            continue;
        }
        dep[y] = dep[x] + 1;
        self(self, y, x);
    }
};
dfs(dfs, 0, -1);
```

递归深度可能达 $O(n)$（链状图）时须确认系统栈足够，必要时参见 1.2.007 手动扩大栈空间；本模板仍直接使用递归函数或递归 lambda，不用容器模拟调用帧。复杂度与普通递归一致，每层仅多传一个引用。

1.1.006 C 语言基础模板与常用 C 库

仅用于强制 C、嵌入式接口或评测明确要求 `.c` 的场景，不与本模板的 C++ 宏混用。64 位整数的读写用 `<inttypes.h>` 的 `SCNd64`/`PRId64` 宏，跨平台格式串才正确。

```c
#include <assert.h>
#include <inttypes.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void)
{
    int64_t n;
    if (scanf("%" SCNd64, &n) != 1)
    {
        return 0;
    }
    printf("%" PRId64 "\n", n);
    return 0;
}
```

`malloc/calloc` 后必须 `free`；`memset` 等库函数复杂度取决于参数（$O(n)$），框架本身 $O(1)$。

1.1.007 Python 基础模板

适合大整数、字符串处理和对拍暴力。`input = sys.stdin.buffer.readline` 绕开 Python 慢速行读取；正式解题前先估常数与递归深度，Python 递归默认上限约 1000 层。

```python
import sys

input = sys.stdin.buffer.readline

def solve() -> None:
    n = int(input())
    a = list(map(int, input().split()))
    print(sum(a[:n]))

def main() -> None:
    t = int(input())
    for _ in range(t):
        solve()

if __name__ == "__main__":
    main()
```

未知组数时遍历 `sys.stdin.buffer`；深递归优先改迭代，确需时再 `sys.setrecursionlimit`（并注意实际系统栈限制）。

1.1.008 静态取模类

模数在编译期固定且不超过 10^9+7 时，把它做成模板参数 `MInt<P>`，使有限域上的加、减、乘、除、幂与逆元统一使用值语义接口。代表元固定用 32 位无符号整数保存，乘法显式提升到 64 位；这一布局不受公共 `#define int long long` 影响，适合组合数、矩阵、多项式与 NTT 的大数组。构造时把整数规范到 $[0,P)$。`inv()` 与除法使用费马小定理，只在 P 为素数且除数非零时有效；只需环上的加减乘时不要求 P 为素数。运行期模数使用 7.1.008 动态取模类，接近 64 位上界或合数模的专用算法使用 7.1.001 快速幂 的宽乘法原语，不应复制一份私有取模类。

```
template <uint32_t P> struct MInt
{
    static_assert(1 < P && P <= 1000000007);
    uint32_t x = 0; // 固定 32 位以保证对象为 4 字节，提高大数组的缓存密度。

    MInt(long long v = 0)
    {
        // v 是要转入模 P 剩余类的整数；构造其最小非负代表元。
        v %= (long long)P;
        if (v < 0)
        {
            v += P;
        }
        x = (uint32_t)v;
    }

    static constexpr uint32_t mod()
    {
        // 无参数；返回编译期模数 P。
        return P;
    }

    uint32_t val() const
    {
        // 无参数；返回当前剩余类的最小非负代表元。
        return x;
    }

    MInt operator-() const
    {
        // 无参数；返回当前剩余类的加法逆元。
        MInt ans;
        ans.x = x ? P - x : 0;
        return ans;
    }

    MInt &operator+=(const MInt &o)
    {
        // o 是要加到当前值上的同模剩余类；原地完成模加并返回当前对象引用。
        x += o.x;
        if (x >= P)
        {
```

```
            x -= P;
        }
        return *this;
    }

    MInt &operator-=(const MInt &o)
    {
        // o 是要从当前值减去的同模剩余类；原地完成模减并返回当前对象引用。
        x += P - o.x;
        if (x >= P)
        {
            x -= P;
        }
        return *this;
    }

    MInt &operator*=(const MInt &o)
    {
        // o 是要乘到当前值上的同模剩余类；原地完成模乘并返回当前对象引用。
        // 乘积可能超过 32 位，显式提升到无符号 64 位。
        x = (uint64_t)x * o.x % P;
        return *this;
    }

    MInt &operator/=(const MInt &o)
    {
        // o 是非零同模剩余类且 P 必须为素数；原地乘以 o 的逆元并返回当前对象引用。
        return *this *= o.inv();
    }

    friend MInt operator+(MInt a, const MInt &b)
    {
        // a、b 是同模剩余类；返回 a+b。
        return a += b;
    }

    friend MInt operator-(MInt a, const MInt &b)
    {
        // a、b 是同模剩余类；返回 a-b。
        return a -= b;
    }

    friend MInt operator*(MInt a, const MInt &b)
    {
        // a、b 是同模剩余类；返回 a*b。
        return a *= b;
    }

    friend MInt operator/(MInt a, const MInt &b)
    {
        // a、b 是同模剩余类且 b 非零；返回 a/b。
        return a /= b;
```

```cpp
    }

    friend bool operator==(const MInt &, const MInt &) = default;

    MInt pow(uint64_t b) const
    {
        // b 是非负指数；返回当前剩余类的 b 次幂。
        MInt a = *this;
        MInt ans = 1;
        while (b)
        {
            if (b & 1)
            {
                ans *= a;
            }
            a *= a;
            b >>= 1;
        }
        return ans;
    }

    MInt inv() const
    {
        // 当前值必须非零且 P 必须为素数；返回乘法逆元。
        assert(x != 0); // 调试检查，可删
        return pow(P - 2);
    }
};

template <uint32_t P> const vector<MInt<P>> &invTable(int n)
{
    // n 是需要的最大下标且 0<=n<P；返回至少覆盖 0..n 的共享模逆元表，其中位置 0 为 0。
    assert(0 <= n && (uint64_t)n < P); // 调试检查，可删
    static vector<MInt<P>> iv{0, 1};
    int old = iv.size();
    if (old <= n)
    {
        iv.resize(n + 1);
        for (int i = old; i <= n; i++)
        {
            iv[i] = MInt<P>(0) - MInt<P>(P / i) * iv[P % i];
        }
    }
    return iv;
}

constexpr uint32_t mod = 998244353;
using Z = MInt<mod>;
```

`MInt<P>` 的单次加减乘为 $O(1)$，模幂和单个逆元为 $O(\log P)$。`invTable<P>(n)` 要求 $0 \le n < P$，首次扩到 n 为 $O(n)$，之后读取为 $O(1)$。下面的固定验证同时覆盖常用模数 998244353 与 1000000007，检查负数构造、跨模加减、乘法、除法、逆元和共享逆元表；复制实现后可直接运行。

```cpp
template <uint32_t P> void checkMInt(uint32_t divAns)
{
    // divAns 是 (P-2)/5 的已知最小非负余数；核对模 P 的基础四则运算与逆元。
    using M = MInt<P>;
    assert(M(-2).val() == P - 2);
    assert((M(P - 2) + M(5)).val() == 3);
    assert((M(2) - M(5)).val() == P - 3);
    assert((M(P - 1) * M(P - 1)).val() == 1);
    assert((M(P - 2) / M(5)).val() == divAns);
    assert((M(5) * M(5).inv()).val() == 1);
    assert(invTable<P>(5)[5] == M(5).inv());
}

signed main()
{
    checkMInt<998244353>(798595482);
    checkMInt<1000000007>(200000001);
}
```

1.2 输入输出

1.2.001 快速输入输出

普通题关闭同步的 `cin/cout` 已足够；输入达数百万整数且 I/O 成瓶颈时才用 `FastInput`。核心是一次 `fread` 灌满 64 KiB 缓冲、逐字节手动解析，省掉 `cin` 的格式化开销。读整数时先跳空白、再处理可选负号、逐位累加；EOF 或格式错误返回 `false`。为缩短热路径，整数越界不再逐位检查，使用时必须保证题面输入能由目标类型表示；无符号目标不能读负数。构造时可传入 `FILE *` 作为读取流，比赛中保持默认的 `stdin`，本地测试可传临时流而不依赖外部标准输入。

```cpp
struct FastInput
{
    static constexpr int S = 1 << 16; // 缓冲区容量（字节）
    array<char, S> buf{};             // fread 批量读入的原始字节
    FILE *src = stdin;                // 当前读取的 C 输入流
    int p = 0;                        // 下一个未读位置
    int n = 0;                        // 当前有效字节数

    // 参数 src 为读取来源，默认从标准输入读取。
    FastInput(FILE *src = stdin) : src(src) {}

    char get()
    {
        // 返回下一个输入字节，EOF 返回 0
        if (p == n)
        {
            n = fread(buf.data(), 1, S, src);
            p = 0;
            if (!n)
            {
```

```
                return 0;
            }
        }
        return buf[p++];
    }

    template <class T> bool read(T &x)
    {
        // 题面保证整数能由 T 表示；EOF 或格式错误返回 false
        static_assert(is_integral_v<T> && !is_same_v<remove_cv_t<T>, bool>);
        using U = make_unsigned_t<T>;
        auto c = get();
        while (c && c <= ' ')
        {
            c = get();
        }
        if (!c)
        {
            return false;
        }
        bool neg = c == '-';
        if (neg)
        {
            c = get();
        }
        if (c < '0' || c > '9')
        {
            return false;
        }
        U v = 0;
        while ('0' <= c && c <= '9')
        {
            v = v * 10 + (U)(c - '0');
            c = get();
        }
        if constexpr (is_unsigned_v<T>)
        {
            if (neg)
            {
                return false;
            }
            x = (T)v;
        }
        else if (neg)
        {
            x = (T)(U(0) - v);
        }
        else
        {
            x = (T)v;
        }
        return true;
```

```
    }

    bool read(string &s)
    {
        // 读一个非空白字符串（以空白或 EOF 结束）
        auto c = get();
        while (c && c <= ' ')
        {
            c = get();
        }
        if (!c)
        {
            return false;
        }
        s.clear();
        while (c > ' ')
        {
            s.push_back(c);
            c = get();
        }
        return true;
    }
};
```

`FastInput in; in.read(n);` 直接取代 `cin >> n`。整数目标可以是有符号或无符号整型；负号不能写入无符号目标，调用者须保证数值在 `T` 范围内。同一程序内不要混用 `FastInput`、`cin` 与 `scanf`，它们各自维护缓冲。读长度 L 的输入为 $O(L)$，缓冲固定为 $O(2^{16})$。

1.2.002 `__int128` 输入输出

`__int128` 没有流重载，读写需要经字符串中转。`toString` 用无符号幅值逐位取模，避开 -2^{127} 取负溢出；`parseInt128` 严格检查符号、数字和范围，失败时返回空值。计算 `a*b%mod` 时写 `(__int128)a*b%mod`，让乘法在 128 位中完成。

```
string toString(__int128 x)
{
    // 128 位整数转十进制字符串
    if (!x)
    {
        return "0";
    }
    auto neg = x < 0;
    auto y = neg ? 0 - (unsigned __int128)x : (unsigned __int128)x;
    string s;
    while (y)
    {
        s.push_back(char('0' + y % 10));
        y /= 10;
    }
    if (neg)
```

```
    {
        s.push_back('-');
    }
    reverse(s.begin(), s.end());
    return s;
}

optional<__int128> parseInt128(const string &s)
{
    // 严格解析十进制串，非法或越界时返回空
    if (s.empty())
    {
        return nullopt;
    }
    bool neg = s[0] == '-';
    int p = neg || s[0] == '+';
    if (p == (int)s.size())
    {
        return nullopt;
    }
    unsigned __int128 lim = (unsigned __int128)1 << 127;
    if (!neg)
    {
        lim--;
    }
    unsigned __int128 x = 0;
    for (; p < (int)s.size(); p++)
    {
        if (s[p] < '0' || s[p] > '9')
        {
            return nullopt;
        }
        unsigned d = (unsigned)(s[p] - '0');
        if (x > (lim - d) / 10)
        {
            return nullopt;
        }
        x = x * 10 + d;
    }
    if (!neg)
    {
        return (__int128)x;
    }
    if (x == ((unsigned __int128)1 << 127))
    {
        return -(__int128)(x - 1) - 1;
    }
    return -(__int128)x;
}
```

解析与输出均为 $O(d)$，其中 d 是十进制位数。下面用最小值往返、带正号输入和一次乘法快速检查主路径，并保留一条非法字符检查：

```
signed main()
{
    string s = "-170141183460469231731687303715884105728";
    auto x = parseInt128(s);
    assert(x && toString(*x) == s);
    assert(parseInt128("+42") == optional<__int128>(42));
    assert(!parseInt128("12x"));
    auto a = parseInt128("123456789");
    auto b = parseInt128("987654321");
    assert(a && b && toString(*a * *b) == "121932631112635269");
}
```

1.2.003 泛型读写封装

容器长度已知时，1.1.004 常用函数重载与容器输出 已提供 `vector` 流重载。本条只展示调用，不重复定义同一组函数；复制模板时保留 1.1.004 的唯一实现即可。

```
vector<int> a(n);
cin >> a;
cout << a << '\n';
```

必须先 `vector<int> a(n)` 定好长度再 `cin >> a`——重载按既有 `size()` 读取，不会自动扩容。长度 n 时 $O(n)$ 时间、$O(1)$ 额外空间。

1.2.004 浮点数与输出精度控制

`fixed` 下 `setprecision(k)` 是小数位数，`defaultfloat` 下是有效数字，二者含义不同，误用会少输出几位导致精度不足。判题给绝对/相对误差时，多输出 2 到 4 位保险。

```
cout << fixed << setprecision(12) << x << '\n';
cout << scientific << setprecision(6) << x << '\n';
cout << defaultfloat;   // 恢复默认模式
```

单次格式化与位数线性相关，竞赛中视为 $O(1)$。

1.2.005 未知长度的一行输入

一行元素个数未知但行边界有语义时，不能用 `while (cin >> x)`——它跨行读取会吞掉下一行数据。正确做法是整行 `getline` 后用 `stringstream` 切分。

```
string line;
getline(cin >> ws, line);
stringstream ss(line);
vector<int> a;
for (int x; ss >> x;)
{
    a.push_back(x);
```

```
}
```

`cin >> ws` 跳过前导空白（含上一行残留换行）。若空行本身有效，则不能吃空白，应先单独消费前一个换行再 `getline(cin, line)`。行长 L 时 $O(L)$。

1.2.006 文件输入输出

本地固定样例、传统文件题与对拍时重定向标准流。用 `#ifndef ONLINE_JUDGE` 包裹，保证提交到评测机时不生效——否则普通标准输入题会因重定向读不到数据。

```
#ifndef ONLINE_JUDGE
freopen("input.txt", "r", stdin);
freopen("output.txt", "w", stdout);
#endif
```

若题面要求固定文件名（如 `data.in`），按题面改。只改变流来源，算法复杂度不变。

1.2.007 手动扩大栈空间

仅在递归写法已经清楚、且确认栈溢出时使用。C++ 的 DFS、分治和回溯仍直接写普通递归函数或递归 lambda；256 MiB 栈不等于可安全递归任意层——栈帧里的大对象、局部容器仍会吃掉配额。

Linux 运行前抬高软限制：

```
ulimit -s 262144
./main
```

Windows MinGW 在链接期设定栈大小：

```
g++ main.cpp -Wl,--stack,268435456 -o main.exe
```

不改变时间复杂度，只提高进程栈上限。

1.2.008 无原生 `__int128` 时的双字 `i128`

仅当编译器没有原生 `__int128` 时才使用本实现；它是兼容性兜底方案，加减乘除和十进制转换均明显慢于原生 `__int128`。用两个 `unsigned long long` 保存高低机器字比搬入任意精度整数更短。位型采用二进制补码：加减显式传递进位与借位，乘法把 64 位数拆成两个 32 位块求高半积，除法按位恢复商和余数。机器字必须是无符号型；若改成两个有符号整数，中间溢出会产生未定义行为。

```
struct i128
{
    unsigned long long hi = 0; // 补码的高 64 位
    unsigned long long lo = 0; // 补码的低 64 位

    i128(int x = 0) : hi(x < 0 ? ~0ULL : 0), lo((unsigned long long)x)
    {
```

```
        // 把 64 位有符号整数按补码符号扩展到 128 位
    }

    explicit operator bool() const
    {
        // 返回当前 128 位整数是否非零
        return hi || lo;
    }

private:
    i128(unsigned long long h, unsigned long long l) : hi(h), lo(l)
    {
        // h、l 是已经计算好的高、低 64 位补码
    }

    static bool lessU(i128 a, i128 b)
    {
        // 把 a、b 当作无符号 128 位整数，返回 a 是否小于 b
        return a.hi != b.hi ? a.hi < b.hi : a.lo < b.lo;
    }

    static unsigned long long mulHi(unsigned long long a, unsigned long long b)
    {
        // 返回 64 位无符号乘积的高 64 位
        constexpr unsigned long long MASK = 0xffffffffULL;
        unsigned long long a0 = a & MASK;
        unsigned long long a1 = a >> 32;
        unsigned long long b0 = b & MASK;
        unsigned long long b1 = b >> 32;
        unsigned long long p = a0 * b0;
        unsigned long long x = a0 * b1;
        unsigned long long y = a1 * b0;
        unsigned long long mid = (p >> 32) + (x & MASK) + (y & MASK);
        return a1 * b1 + (x >> 32) + (y >> 32) + (mid >> 32);
    }

public:
    friend i128 operator-(i128 x)
    {
        // 返回 x 的 128 位补码相反数
        auto lo = ~x.lo + 1;
        return i128(~x.hi + (lo == 0), lo);
    }

    friend i128 operator+(i128 a, i128 b)
    {
        // 返回 a+b 的低 128 位，低位溢出时向高位进位
        auto lo = a.lo + b.lo;
        return i128(a.hi + b.hi + (lo < a.lo), lo);
    }

    friend i128 operator-(i128 a, i128 b)
```

```
    {
        // 返回 a-b 的低 128 位，低位借位计入高位
        auto lo = a.lo - b.lo;
        return i128(a.hi - b.hi - (a.lo < b.lo), lo);
    }

    friend i128 operator*(i128 a, i128 b)
    {
        // 返回 a*b 的低 128 位
        return i128(mulHi(a.lo, b.lo) + a.hi * b.lo + a.lo * b.hi,
                    a.lo * b.lo);
    }

private:
    static pair<i128, i128> divU(i128 n, i128 d)
    {
        // n、d 按无符号解释且 d 非零；返回商与余数
        i128 q;
        i128 r;
        for (int i = 127; i >= 0; i--)
        {
            auto carry = r.hi >> 63;
            auto bit = i < 64 ? n.lo >> i & 1ULL : n.hi >> (i - 64) & 1ULL;
            r = i128(r.hi << 1 | r.lo >> 63, r.lo << 1 | bit);
            if (carry || !lessU(r, d))
            {
                r = r - d;
                if (i < 64)
                {
                    q.lo |= 1ULL << i;
                }
                else
                {
                    q.hi |= 1ULL << (i - 64);
                }
            }
        }
        return {q, r};
    }

    static pair<i128, i128> divS(i128 a, i128 b)
    {
        // b 非零且商可表示；返回向零取整的商和与 a 同号的余数
        auto na = a.hi >> 63;
        auto nb = b.hi >> 63;
        auto [q, r] = divU(na ? -a : a, nb ? -b : b);
        if (na != nb)
        {
            q = -q;
        }
        if (na)
        {
```

```
            r = -r;
        }
        return {q, r};
    }

public:
    friend i128 operator/(i128 a, i128 b)
    {
        // 返回 a/b 向零取整的商
        return divS(a, b).first;
    }

    friend i128 operator%(i128 a, i128 b)
    {
        // 返回 a%b，非零余数与 a 同号
        return divS(a, b).second;
    }

    string toString() const
    {
        // 返回当前值的十进制表示
        auto neg = hi >> 63;
        i128 x = neg ? -*this : *this;
        string s;
        do
        {
            auto [q, r] = divU(x, i128(10));
            s.push_back(char('0' + r.lo));
            x = q;
        } while (x);
        if (neg)
        {
            s.push_back('-');
        }
        reverse(s.begin(), s.end());
        return s;
    }

    friend ostream &operator<<(ostream &out, const i128 &x)
    {
        // 把 x 的十进制表示写入 out，并返回 out
        return out << x.toString();
    }
};
```

用 `i128 x = (long long)v` 从 64 位整数构造，`cout << x` 直接输出。加、减、乘保留补码乘积的低 128 位；若按有符号整数使用，应保证数学结果落在 $[-2^{127}, 2^{127}-1]$。除数必须非零，且不能计算结果无法表示的 $\frac{-2^{127}}{-1}$；商向零取整，余数与被除数同号。加减乘为 $O(1)$，除法固定做 128 轮，转十进制为 $O(128d)$（$d \le 39$），额外空间均为 $O(1)$。编译器支持原生类型时优先用 1.2.002 `__int128` 输入输出；范围超过 128 位时改用 14.1.001 高精度整数（BigInt）。

下面用一组负数运算和一个跨机器字的数检查五则运算、符号规则与除法：

```
signed main()
{
    i128 x(-123456789), y(12345);
    assert((x + y).toString() == "-123444444");
    assert((x - y).toString() == "-123469134");
    assert((x * y).toString() == "-1524074060205");
    assert((x / y).toString() == "-10000");
    assert((x % y).toString() == "-6789");

    i128 cross;
    cross.hi = 1;
    cross.lo = 1;
    assert((cross / i128((1LL << 32) + 1)).toString() == "4294967295");
    assert((cross % i128((1LL << 32) + 1)).toString() == "2");
}
```

1.3 编译、调试与测试

1.3.001 编译参数与调试构建

CLion 中为同一个 CMake 项目建立 `Release` 和 `Debug` 两个 Profile。Release 用于本地性能和提交前测试，Debug 用于定位越界、未定义行为和 STL 迭代器错误。下面的缓存变量应填入 CLion 的 CMake options；命令行构建时另加对应的 `-DCMAKE_BUILD_TYPE`。

最小 `CMakeLists.txt`：

```
cmake_minimum_required(VERSION 3.20)
project(acm)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
add_executable(main main.cpp)
```

Release Profile 的 CMake options：

```
-DCMAKE_CXX_FLAGS_RELEASE="-g -O2" -DCMAKE_EXE_LINKER_FLAGS_RELEASE="-static"
```

Debug Profile 的 CMake options：

```
-DCMAKE_CXX_FLAGS_DEBUG="-O0 -g3 -gdwarf-4 -Wall -Wextra -Wshadow -Wconversion
↪ -Wsign-conversion -Wformat=2 -Warray-bounds=2 -Wstringop-overflow=4 -Wnull-dereference
↪ -Wduplicated-cond -Wlogical-op -Wvla -Wstack-usage=1048576 -Werror=return-type
↪ -fsanitize=undefined,bounds-strict -fsanitize-undefined-trap-on-error
↪ -fno-omit-frame-pointer -fno-optimize-sibling-calls -fstack-protector-all -DLOCAL
↪ -D_GLIBCXX_DEBUG -D_GLIBCXX_DEBUG_PEDANTIC -D_GLIBCXX_ASSERTIONS"
↪ -DCMAKE_EXE_LINKER_FLAGS_DEBUG="-static"
```

参数作用如下：`-DCMAKE_CXX_FLAGS_RELEASE` 和 `-DCMAKE_CXX_FLAGS_DEBUG` 分别设置两种构建的 C++ 编译参数；`-DCMAKE_EXE_LINKER_FLAGS_RELEASE` 和 `-DCMAKE_EXE_LINKER_FLAGS_DEBUG` 分别设置可执行文

件的链接参数。`-g` 保留 Release 调试信息，`-O2` 开启常用安全优化；`-O0` 关闭优化便于单步，`-g3` 生成包含宏的完整调试信息，`-gdwarf-4` 使用 DWARF v4。

`-Wall` 开启常见警告，`-Wextra` 开启额外警告，`-Wshadow` 检查变量遮蔽，`-Wconversion` 检查可能改变数值的隐式转换，`-Wsign-conversion` 检查有符号和无符号转换，`-Wformat=2` 加强 `printf` / `scanf` 格式检查，`-Warray-bounds=2` 加强数组越界分析，`-Wstringop-overflow=4` 加强字符串和内存写入越界分析，`-Wnull-dereference` 检查可证明的空指针解引用，`-Wduplicated-cond` 检查重复条件，`-Wlogical-op` 检查可疑逻辑运算，`-Wvla` 警告运行时长度数组，`-Wstack-usage=1048576` 在函数栈使用量可能超过 1 MiB 时警告，`-Werror=return-type` 把缺失返回值等返回类型错误视为编译错误。

`-fsanitize=undefined` 检查常见未定义行为，`-fsanitize=bounds-strict` 加强边界检查，`-fsanitize-undefined-trap-on-error` 在发现 UBSan 错误后直接触发 trap；需要可读的诊断文本时可以暂时删掉最后一项。`-fno-omit-frame-pointer` 保留帧指针，`-fno-optimize-sibling-calls` 禁止尾调用优化，二者便于回溯；`-fstack-protector-all` 为函数增加栈破坏保护。`-DLOCAL` 开启本地代码，`-D_GLIBCXX_DEBUG` 开启 libstdc++ 调试迭代器，`-D_GLIBCXX_DEBUG_PEDANTIC` 开启更严格的 STL 检查，`-D_GLIBCXX_ASSERTIONS` 开启标准库断言。`-static` 要求静态链接可执行文件，缺少对应静态库时会链接失败；`_GLIBCXX_DEBUG` 改变 STL ABI，Debug 和 Release 对象不能混合链接。

临时不用 CLion 时，可在 PowerShell 用下面的精简命令；完整告警、STL 检查和静态链接仍以上面的两个 Profile 为准。

```
g++ -std=c++20 -O2 -Wall -Wextra -Wshadow main.cpp -o main.exe
g++ -std=c++20 -O0 -g3 -gdwarf-4 -Wall -Wextra -Wshadow '-fsanitize=undefined,bounds-strict'
↪  -fsanitize-undefined-trap-on-error -fno-omit-frame-pointer -fno-optimize-sibling-calls
↪  -DLOCAL main.cpp -o main_debug.exe
```

下面是 GCC 优化选项清单。它是卡常查询表，不是默认提交前导；必须先用普通 `-O2` 版本和固定输入确认瓶颈，再逐项打开并对拍。`#pragma GCC optimize` 只影响其后的函数或翻译单元，`#pragma GCC target` 还会改变生成的指令集。

```
#if defined(__GNUC__)
#pragma GCC optimize("O3,unroll-loops") // O3 增强优化；展开循环，减少循环控制开销。
#if defined(__x86_64__) || defined(__i386__)
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt,fma") // 开启下文逐项说明的六组 x86 指令。
#pragma GCC target("avx", "sse2") // 再开启 AVX 与 SSE2；x86-64 上 SSE2 通常已默认支持。
#endif
#pragma GCC optimize("Ofast") // 在 O3 上放宽浮点语义，可能破坏严格 IEEE 结果。
#pragma GCC optimize("inline") // 鼓励内联函数，减少调用开销。
#pragma GCC optimize("unroll-loops") // 展开循环，可能增大代码体积。
#pragma GCC optimize("-fgcse") // 消除全局公共子表达式。
#pragma GCC optimize("-fgcse-lm") // 加强循环中的加载移动和公共子表达式消除。
#pragma GCC optimize("-fipa-sra") // 跨函数拆分聚合参数，减少无用字段传递。
#pragma GCC optimize("-ftree-pre") // 对树形中间表示做部分冗余消除。
#pragma GCC optimize("-ftree-vrp") // 传播变量取值范围，删除不可能分支。
#pragma GCC optimize("-fpeephole2") // 执行目标相关的局部指令改写。
#pragma GCC optimize("-ffast-math") // 允许不严格 IEEE 规则的浮点优化。
#pragma GCC optimize("-fsched-spec") // 允许更激进地提前调度指令。
#pragma GCC optimize("-falign-jumps") // 对齐跳转目标，可能改善取指。
#pragma GCC optimize("-falign-loops") // 对齐循环入口。
#pragma GCC optimize("-falign-labels") // 对齐普通代码标签。
```

```
#pragma GCC optimize("-fdevirtualize") // 将可确定目标的虚调用改为直接调用。
#pragma GCC optimize("-fcaller-saves") // 在调用附近保存寄存器，减少溢出到内存。
#pragma GCC optimize("-fcrossjumping") // 合并不同分支末尾的相同代码。
#pragma GCC optimize("-fthread-jumps") // 重排条件跳转，减少连续跳转。
#pragma GCC optimize("-funroll-loops") // 再次显式指定循环展开。
#pragma GCC optimize("-freorder-blocks") // 重排基本块，让常走路径顺序执行。
#pragma GCC optimize("-fschedule-insns") // 在寄存器分配前调度指令。
#pragma GCC optimize("inline-functions") // 鼓励普通函数内联。
#pragma GCC optimize("-ftree-tail-merge") // 合并不同分支的相同尾部。
#pragma GCC optimize("-fschedule-insns2") // 在寄存器分配后再次调度指令。
#pragma GCC optimize("-fstrict-aliasing") // 假设不同类型指针不别名；违反规则会出错。
#pragma GCC optimize("-falign-functions") // 对齐函数入口。
#pragma GCC optimize("-fcse-follow-jumps") // 沿条件跳转路径继续消除公共子表达式。
#pragma GCC optimize("-fsched-interblock") // 允许跨基本块调度指令。
#pragma GCC optimize("-fpartial-inlining") // 只内联函数中实际需要的部分。
#pragma GCC optimize("no-stack-protector") // 关闭栈保护，只适合确认安全的性能实验。
#pragma GCC optimize("-freorder-functions") // 重排函数布局，改善代码局部性。
#pragma GCC optimize("-findirect-inlining") // 间接目标可确定时尝试内联。
#pragma GCC optimize("-fhoist-adjacent-loads") // 提前条件两侧相邻的加载操作。
#pragma GCC optimize("-frerun-cse-after-loop") // 循环优化后再次消除公共子表达式。
#pragma GCC optimize("inline-small-functions") // 鼓励内联小函数。
#pragma GCC optimize("-finline-small-functions") // 与上一项同类，鼓励内联小函数。
#pragma GCC optimize("-ftree-switch-conversion") // 将部分 switch 转为查表或位判断。
#pragma GCC optimize("-foptimize-sibling-calls") // 做尾调用优化，减少栈使用。
#pragma GCC optimize("-fexpensive-optimizations") // 开启耗时但可能有效的优化。
#pragma GCC optimize("inline-functions-called-once") // 内联只调用一次的函数。
#pragma GCC optimize("-fdelete-null-pointer-checks") // 按未定义行为假设删除无效空指针检查。
// 当前 GCC 不接受下面四个名称作为有效 optimize pragma，仅保留作历史索引：
// -fwhole-program：假设整个程序一次性编译。
// -fstrict-overflow：基于没有有符号溢出的假设优化。
// -fcse-skip-blocks：旧版 CSE 跳块选项。
// -funsafe-loop-optimizations：基于循环满足额外约束优化。
#endif
```

目标参数中，`avx2` 把多数整数 SIMD 扩展到 256 位，`bmi` 提供 `andn`、`blsi` 等基础位操作，`bmi2` 提供 `pext`、`pdep` 等扩展位操作，`lzcnt` 提供前导零计数，`popcnt` 提供置位计数，`fma` 提供单次舍入的融合乘加，`avx` 提供 256 位浮点 SIMD，`sse2` 提供 128 位整数与双精度浮点 SIMD。这些指令只能在评测机明确支持时开启，否则程序可能直接收到非法指令；本机可用 `-march=native` 让 GCC 按当前 CPU 选择，但不能据此假定评测机也支持。

选项边界必须记住：`-O3` 通常已经包含大多数普通 `-f*` 优化，重复列出不等于再次加速；`-Ofast` 与 `-ffast-math` 允许破坏严格 IEEE 浮点语义；`-fstrict-aliasing` 依赖程序遵守别名规则；`no-stack-protector` 会降低栈保护；展开、对齐、内联和调度可能增大代码并使指令缓存变差。`#pragma once` 只是头文件保护，与性能无关。

这些参数只影响常数，不改变渐进复杂度。`-fsanitize` 用于调试，不能与面向提交的极限优化混用；Linux 提栈用 1.2.007 的 `ulimit`，MinGW 才使用 `-Wl,--stack`，字节数。卡常前先用 1.5.001 卡常使用边界与专题索引 的基准步骤确认瓶颈。

1.3.002 Makefile（Windows / Linux）

固定编译参数，减少比赛中漏写标准版本或调试开关。配方行开头必须是真实 Tab（不是空格），否则 `make` 报错。

```
CXX := g++
CXXFLAGS := -std=c++20 -O2 -Wall -Wextra -Wshadow

main: main.cpp
	$(CXX) $(CXXFLAGS) $< -o $@

debug: main.cpp
	$(CXX) -std=c++20 -O1 -g -fsanitize=address,undefined $< -o main_debug

clean:
	rm -f main main.exe main_debug main_debug.exe
```

Linux/WSL 用 `make`，Windows 用 `mingw32-make`。

1.3.003 bashrc / vimrc

只保留不改变提交语义的快捷命令与编辑器设置。比赛环境不确定时不要依赖只存在于个人 shell 的别名。

```
alias c20='g++ -std=c++20 -O2 -Wall -Wextra -Wshadow'
alias asan='g++ -std=c++20 -O1 -g -fsanitize=address,undefined'
ulimit -c unlimited
```

```
set number
set expandtab
set shiftwidth=4
set tabstop=4
set autoindent
```

若比赛环境允许改自己的 shell，可把编译、解压样例、批量跑样例封成函数（`ulimit -s` 提栈上限，避免深递归段错误）：

```
ulimit -s 262144

build() {
    # $1: 源文件名，不带 .cpp
    if [ "$#" -ne 1 ]; then
        echo "usage: build <source>" >&2
        return 2
    fi
    if ! g++ -g -gdwarf-4 -O0 -Wall -Wextra -Wshadow -std=gnu++23 \
        -fsanitize=address,undefined -fno-omit-frame-pointer \
        -fno-optimize-sibling-calls "$1.cpp" -o "$1"; then
        echo "compile failed" >&2
        return 1
    fi
```

```
    echo "compile finished"
}

testcase() {
    # $1: 样例压缩包前缀
    if [ "$#" -ne 1 ]; then
        echo "usage: testcase <prefix>" >&2
        return 2
    fi
    unzip "samples-$1.zip" -d "./samples-$1/"
}

run() {
    # $1: 源文件名, $2: 样例前缀, $3: 起始编号, $4: 结束编号
    if [ "$#" -ne 4 ]; then
        echo "usage: run <source> <prefix> <first> <last>" >&2
        return 2
    fi
    build "$1" || return 1
    for ((i=$3;i<=$4;i++)); do
        echo "===== input: ./samples-$2/$i.in ====="
        if ! "./$1" < "./samples-$2/$i.in"; then
            echo "runtime error on sample $i" >&2
            return 1
        fi
    done
}
```

这些函数只包装本地命令，不改变被测程序复杂度。运行当前目录中的程序时保留 `./main`，不要把可写的当前目录加入 `PATH`。

1.3.004 对拍器（Windows / Linux）

对拍 = 生成器反复造随机数据，喂给待测程序和一个信得过的参考程序，比对结果，首个反例立即停并保留那组输入。生成器种子取自命令行参数（`gen 1`、`gen 2`……），同一种子永远产生同一组数据，反例可复现。按判定方式分两类：逐字比对（答案唯一）与 special judge（答案多解，用 checker 程序判）。

固定命名：`gen` 生成数据，`my` 是待测程序，`brute` 是可信的暴力或参考程序。生成器从 `argv[1]` 读取种子。每轮只输出 `AC i` 或 `WA i`，不打印程序输出、比较器说明或其他进度；首个 `WA i` 后停止，反例保存在 `in.txt`。

常规题用 `fc /b` 或 `diff -q` 严格比较；按空白分隔 token 或允许浮点误差时，改用题目对应的 checker。`dp.bat`：

```
@echo off
for %%x in (gen my brute) do (g++ -std=c++20 -O2 %%x.cpp -o %%x.exe || exit /b)
set i=0
:loop
set /a i+=1
gen.exe %i% > in.txt 2> nul
```

```
if errorlevel 1 goto wa
my.exe < in.txt > my.out 2> nul
if errorlevel 1 goto wa
brute.exe < in.txt > bf.out 2> nul
if errorlevel 1 goto wa
fc /b my.out bf.out > nul
if errorlevel 1 goto wa
echo AC %i%
goto loop
:wa
echo WA %i%
```

`dp.sh`:

```
#!/bin/bash
for x in gen my brute; do g++ -std=c++20 -O2 "$x.cpp" -o "$x" || exit; done
for ((i=1;;i++)); do
    if ! ./gen "$i" > in.txt 2>/dev/null ||
       ! ./my < in.txt > my.out 2>/dev/null ||
       ! ./brute < in.txt > bf.out 2>/dev/null; then
        echo "WA $i"
        break
    elif diff -q my.out bf.out > /dev/null 2>&1; then
        echo "AC $i"
    else
        echo "WA $i"
        break
    fi
done
```

只有浮点答案且答案唯一时，不需要编写题目专用 SPJ。把下面的 `f_cmp.py` 与 `dp.bat` 或 `dp.sh` 放在同一目录后直接调用。它接收待测输出和正确输出两个文件，要求两边浮点 token 数量相同，逐项使用题面给出的 $|x-y| \le 10^{-6}\max(1,y)$；它自己不输出内容，只用退出码表示是否全部通过：

```
import sys

x = [float(v) for v in open(sys.argv[1]).read().split()]
y = [float(v) for v in open(sys.argv[2]).read().split()]
ok = len(x) == len(y) and all(
    abs(a - b) <= 1e-6 * max(1.0, b)
    for a, b in zip(x, y)
)
raise SystemExit(not ok)
```

把逐字版脚本中的 `fc /b` 判定换成以下 Windows 片段：

```
python f_cmp.py my.out bf.out > nul 2> nul
if errorlevel 1 goto wa
echo AC %i%
goto loop
:wa
```

```
echo WA %i%
```

Linux / WSL 中把 `diff` 判定换成：

```
if python3 f_cmp.py my.out bf.out > /dev/null 2>&1; then
    echo "AC $i"
else
    echo "WA $i"
    break
fi
```

SPJ 题有多组合法输出，checker 直接验证待测输出，退出码 `0` 表示 AC。下面示例检查输出正整数 x, y 且 $x + y = n$；比赛时只替换中间的题目约束：

```
#include <bits/stdc++.h>
using namespace std;
signed main(signed argc, char *argv[])
{
    if (argc != 3) return 1;
    ifstream in(argv[1]), out(argv[2]);
    int n, x, y;
    if (!(in >> n) || !(out >> x >> y)) return 1;
    out >> ws;
    return !(x > 0 && y > 0 && x + y == n && out.eof());
}
```

最优化题另把标程答案作为第三个文件传给 checker。`spj.bat`：

```
@echo off
for %%x in (gen my spj) do (g++ -std=c++20 -O2 %%x.cpp -o %%x.exe || exit /b)
set i=0
:loop
set /a i+=1
gen.exe %i% > in.txt 2> nul
if errorlevel 1 goto wa
my.exe < in.txt > my.out 2> nul
if errorlevel 1 goto wa
spj.exe in.txt my.out > nul 2> nul
if errorlevel 1 goto wa
echo AC %i%
goto loop
:wa
echo WA %i%
```

`spj.sh`：

```
#!/bin/bash
for x in gen my spj; do g++ -std=c++20 -O2 "$x.cpp" -o "$x" || exit; done
for ((i=1;;i++)); do
    if ! ./gen "$i" > in.txt 2>/dev/null ||
       ! ./my < in.txt > my.out 2>/dev/null; then
```

```
        echo "WA $i"
        break
    elif ./spj in.txt my.out > /dev/null 2>&1; then
        echo "AC $i"
    else
        echo "WA $i"
        break
    fi
done
```

把脚本与源码放在同一目录，直接运行 `dp.bat`、`spj.bat`、`bash dp.sh` 或 `bash spj.sh`。每轮恰好输出一行 `AC i` 或 `WA i`；出现 `WA i` 时用留下的 `in.txt` 调试。对拍器是开发工具，允许用独立暴力程序校验待测程序；各算法条目的比赛内快速查验仍不得附第二套实现。

交互题由 Python 把解答与交互器的标准流相连。`interact.py` 的三个参数依次是解答程序、交互器和隐藏数据文件：

```
import subprocess, sys

sol = subprocess.Popen([sys.argv[1]], stdin=subprocess.PIPE,
                       stdout=subprocess.PIPE, stderr=subprocess.DEVNULL)
itr = subprocess.Popen([sys.argv[2], sys.argv[3]], stdin=sol.stdout,
                       stdout=sol.stdin, stderr=subprocess.DEVNULL)
sol.stdout.close()
sol.stdin.close()
a = itr.wait()
b = sol.wait()
raise SystemExit(a or b)
```

Windows 的 `itr.bat`：

```
@echo off
for %%x in (gen my interactor) do (g++ -std=c++20 -O2 %%x.cpp -o %%x.exe || exit /b)
set i=0
:loop
set /a i+=1
gen.exe %i% > in.txt 2> nul
if errorlevel 1 goto wa
python interact.py my.exe interactor.exe in.txt > nul 2> nul
if errorlevel 1 goto wa
echo AC %i%
goto loop
:wa
echo WA %i%
```

Linux / WSL 的 `itr.sh`：

```
#!/bin/bash
for x in gen my interactor; do g++ -std=c++20 -O2 "$x.cpp" -o "$x" || exit; done
for ((i=1;;i++)); do
    if ! ./gen "$i" > in.txt 2>/dev/null; then
```

```
        echo "WA $i"
        break
    elif python3 interact.py ./my ./interactor in.txt > /dev/null 2>&1; then
        echo "AC $i"
    else
        echo "WA $i"
        break
    fi
done
```

交互器从隐藏数据文件读取测试并在答错时返回非零；解答和交互器每次输出后都必须刷新。直接运行 `itr.bat` 或 `bash itr.sh`，每轮也只输出 `AC i` 或 `WA i`。单轮耗时为本轮所有进程之和，脚本不负责时限。

1.3.005 随机数据生成器

生成可复现随机数据，覆盖边界（最小、最大、重复、有序）与随机情形。种子取自 `argv`，同一种子永远产生同一组数据。绝不用 `rand()`——它的低位周期短、分布差。

```cpp
signed main(signed argc, char **argv)
{
    // 种子来自命令行第一个参数，缺省为 1
    auto seed = argc > 1 ? stoull(argv[1]) : 1ULL;
    mt19937_64 rng(seed);
    int n = uniform_int_distribution<int>(1, 50)(rng);
    cout << n << '\n';
    for (int i = 0; i < n; i++)
    {
        cout << uniform_int_distribution<int>(-100, 100)(rng)
             << " \n"[i + 1 == n];
    }
}
```

图数据生成后要额外检查连通、无重边等题目约束。生成 n 个元素为 $O(n)$。

1.3.006 GNU C++ 环境、本地计时与评测机测速

先输出标准版本、编译器版本、指针位数和优化状态；`__cplusplus >= 202002L` 才能依赖 C++20。实际程序耗时用 `steady_clock` 包住 `solve()`，只在 `LOCAL` 下输出到 `cerr`。

```cpp
cout << __cplusplus << '\n';
cout << __VERSION__ << '\n';
cout << sizeof(void *) * 8 << '\n';
#ifdef __OPTIMIZE__
cout << "optimized\n";
#endif

#ifdef LOCAL
auto beg = chrono::steady_clock::now();
```

```
#endif
solve();
#ifdef LOCAL
auto end = chrono::steady_clock::now();
cerr << chrono::duration<double, milli>(end - beg).count() << " ms\n";
#endif
```

热身赛比较本机与评测机速度时，使用固定的 `mt19937` 工作块。每块生成 10^7 个随机数并滚动异或；`speedCheck` 是该固定过程的真实结果。轮数和块数各做一次 `volatile` 读取，循环内部仍是普通局部变量；返回值必须进入最终输出或错误分支，因此 `-O2` 不能把整段计算当作无用代码删除。

```
volatile int speedRounds = 10000000; // 每块生成次数；修改后必须重算 speedCheck
volatile int speedBlocks = 1;        // 重复块数；测速时只调整这一项
constexpr long long speedCheck = 3894306128LL;

long long rot32(long long x, int k)
{
    // x 是低 32 位有效的无符号字；k 是 1..31 的左旋位数；返回低 32 位结果。
    return ((x << k) | (x >> (32 - k))) & 0xffffffffLL;
}

#if defined(__GNUC__)
__attribute__((noinline))
#endif
long long speedBlock()
{
    // 返回固定随机序列的滚动异或摘要
    int n = speedRounds;
    mt19937 rng(712367821);
    long long h = 0;
    for (int i = 0; i < n; i++)
    {
        h = rot32(h, 7) ^ (unsigned long long)rng();
        h &= 0xffffffffLL;
    }
    return h;
}

long long speedTest()
{
    // 重复 speedBlocks 块；全部摘要正确时返回 0
    int k = speedBlocks;
    long long bad = 0;
    for (int i = 0; i < k; i++)
    {
        bad |= speedBlock() ^ speedCheck;
    }
    return bad;
}
```

本地计时调用如下；`bad` 必须打印或判断，不能只测一个未使用的返回值。

```cpp
auto beg = chrono::steady_clock::now();
long long bad = speedTest();
auto end = chrono::steady_clock::now();
cerr << chrono::duration<double, milli>(end - beg).count()
     << " ms, check = " << bad << '\n';
```

评测机测速先令 `speedBlocks=0` 提交一次，记基线时间为 t_0；再令 `speedBlocks=k` 提交，记时间为 t_k。以下时间均换算为秒。一个块约耗时 $(t_k - t_0)/k$，该评测机每秒约执行

$$\frac{10^7 k}{t_k - t_0}$$

次本测试循环，一秒对应的块数约为 $k/(t_k - t_0)$。时间显示较粗时取 $k = 8$ 或 16，直到 $t_k > t_0$；只返回 AC/TLE 时，提交一个只运行 `speedTest()` 并输出检查值的程序，按 $1, 2, 4, 8, \ldots$ 增大块数，首次 TLE 后在最后两个值间二分。若时限是 L 秒、边界附近可通过 k 块，则一秒约能运行 k/L 块，即约 $10^7 k/L$ 次本测试循环；该值受时间取整和机器负载影响，只取近似。整数答案可写 `ans ^= speedTest()`，正确时异或零而不改变答案；字符串或多行输出在正式输出前写 `if (speedTest()) return 1;`。测速通常只调整 `speedBlocks`；修改 `speedRounds` 后必须本地输出一次 `speedBlock()`，用新摘要替换 `speedCheck`。测速只代表当前编译器下的 `mt19937` 和整数运算速度，不能替代图、内存、I/O 或具体算法的最坏数据测试。环境信息为 $O(1)$；测速为 $O(\mathit{speedRounds} \cdot \mathit{speedBlocks})$ 时间和 $O(1)$ 额外空间。

使用上述固定工作块实测时，各平台每秒完成的块数约为：

平台	每秒块数	每秒本测试循环次数
Codeforces	42	4.2×10^8
QOJ	51	5.1×10^8
洛谷	47	4.7×10^8
牛客	$30 \sim 31$	$3.0 \times 10^8 \sim 3.1 \times 10^8$
AtCoder	72	7.2×10^8
PTA	27	2.7×10^8

本次实测未记录具体评测机型号、编译器版本与重复轮数，以上数据只用于赛前粗略估算，不能视为平台的固定性能。

1.3.007 调试输出与断言

`debug(x)` 只在本地定义 `LOCAL` 时打印到 `cerr`，提交时展开为空语句，不影响 `cout` 结果。`assert` 内不要放有副作用的语句——`-DNDEBUG` 会整体删除表达式，副作用随之消失。

```cpp
#ifdef LOCAL
#define debug(x) cerr << #x << " = " << (x) << '\n'
#else
#define debug(x) ((void)0)
#endif

assert(0 <= l && l <= r && r <= n);
```

关闭时 $O(1)$，开启时取决于输出量。

1.3.008 常见未定义行为和易错点清单

提交前快速排查“本地能跑、评测漂移”的问题：

- 有符号整数溢出是未定义行为；只有乘积可能超出当前类型时，才对该表达式局部转成 `__int128`，普通加减优先用边界比较改写。
- 不对负数或超位宽做位移；`1ULL << k` 要求 $0 \le k < 64$。
- 不解引用 `end()`，不在容器扩容后沿用旧迭代器、指针或引用。
- `sort` 比较器必须严格弱序，不能写成 $a \le b$（自反会崩溃或死循环）。
- 不同时使用关闭同步的 iostream 与 stdio。
- 递归深度、数组下标、空容器 `back()`、除零、负数取模都要单独检查。

只在 `-O2` 下出错时，第一步跑 ASan/UBSan（1.3.001），不要先加 `volatile` 或关优化掩盖问题。清单本身无运行时复杂度。

1.4 STL、扩展库与防卡

1.4.001 `vector`、`array`、`string`、`tuple`、`pair`

`vector` 管运行时动态连续空间，`array` 管编译期定长，`string` 管字节串，`pair/tuple` 管小型聚合。已知最终长度用 `resize` 一次到位，只知上界用 `reserve` 预留避免反复搬迁。

```
vector<int> a(n);
array<int, 4> d{1, 0, -1, 0};     // 常用于四方向偏移
pair<int, int> e{x, y};
auto [u, v] = e;                  // 结构化绑定
```

`string` 下标按字节，不按 Unicode 字符。随机访问 $O(1)$，`push_back` 摊还 $O(1)$，中间插删 $O(n)$。

1.4.002 `stack`、`queue`、`deque`、`priority_queue`

分别对应后进先出、先进先出、双端、堆。`priority_queue` 默认大根堆，小根堆传 `greater<>`；它不支持删除任意元素，需要动态删除时用延迟删除双堆或带删堆（3.5.009）。

```
priority_queue<int> mx;                              // 大根堆
priority_queue<int, vector<int>, greater<>> mn;      // 小根堆
deque<int> q;
```

需要遍历全部元素时直接用底层 `vector/deque`。栈、队列、双端两端操作 $O(1)$；堆插入、弹出 $O(\log n)$，取顶 $O(1)$。

1.4.003 `set`、`multiset`、`map`、`multimap`

需要有序键、前驱后继或稳定最坏复杂度时用红黑树容器。`multiset::erase(x)` 按值删除全部等于 `x` 的元素；只删一个时先用 `find` 精确查找，再删除该迭代器。

```
multiset<int> s;
auto it = s.find(x);
if (it ≠ s.end())
{
    s.erase(it);
}
```

不要用 `distance(s.begin(), it)` 求排名——它对树迭代器是 $O(n)$；要排名用 PBDS 有序树（1.4.006）。查找、插删均 $O(\log n)$。

1.4.004 `unordered_map` / `unordered_set` 防 hack 哈希

标准哈希对整数近乎恒等，出题人能构造大量同桶键把它卡成 $O(n)$ 每次。用 SplitMix64 充分扰动、再混入进程启动时间做随机种子，构造者无法预测桶分布。

```
struct CustomHash
{
    static unsigned long long mix(unsigned long long x)
    {
        // SplitMix64 finalizer：雪崩扰动
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }

    size_t operator()(unsigned long long x) const
    {
        // 混入随机种子，抵御针对固定哈希的构造
        static const unsigned long long seed =
            (unsigned long long)chrono::steady_clock::now().time_since_epoch().count();
        return mix(x + seed);
    }

    size_t operator()(const pair<int, int> &x) const
    {
        // 只接受两个整数分量的 pair
        static const unsigned long long seed =
            (unsigned long long)chrono::steady_clock::now().time_since_epoch().count();
        unsigned long long a = mix((unsigned long long)x.first + seed);
       unsigned long long b = mix((unsigned long long)x.second + seed + 0x9e3779b97f4a7c15ULL);
        return mix(a ^ rotl(b, 32));
    }
};
```

使用时把它作为哈希容器的第三个模板参数，并预留容量：

```
unordered_map<int, int, CustomHash> cnt;
cnt.reserve(n * 2);          // 预留减少重哈希
cnt.max_load_factor(.7);
```

单个整数键和由两个整数组成的 `pair` 可以直接使用该哈希；字符串或复合键应另写与键类型对应的哈希器。操作期望为 $O(1)$，最坏仍为 $O(n)$；要求稳定最坏复杂度时改用 `map`。

1.4.005 `bitset` 与手写 bitset

编译期定长首选 `bitset<N>`；长度运行时才定用 `DynBitset`——按 64 位机器字打包，位运算天然 $\frac{1}{64}$ 加速。下面这份提供置位、清空、查询、计数、按位与/或，够覆盖“布尔可达性并行转移”一类用途；需要移位再按题补充，避免做成难核对的完整容器。

```
struct DynBitset
{
    int n = 0;                      // 有效位数
    vector<unsigned long long> a; // 按 64 位机器字打包

    DynBitset(int m = 0)
    {
        // 构造 m 位的全零集合，要求 m>=0
        init(m);
    }

    void init(int m)
    {
        // 把集合重设为 m 位并清零
        assert(m >= 0); // 调试检查，可删
        n = m;
        a.assign((n + 63) / 64, 0);
    }

    void set(int p, bool v = true)
    {
        // 第 p 位写成 v
        assert(0 <= p && p < n); // 调试检查，可删
        auto b = 1ULL << (p & 63);
        if (v)
        {
            a[p >> 6] |= b;
        }
        else
        {
            a[p >> 6] &= ~b;
        }
    }

    void reset()
```

```
    {
        // 全部有效位清零
        fill(a.begin(), a.end(), 0);
    }

    bool test(int p) const
    {
        // 返回第 p 位
        assert(0 <= p && p < n); // 调试检查，可删
        return a[p >> 6] >> (p & 63) & 1ULL;
    }

    int count() const
    {
        // 1 的总数
        int ans = 0;
        for (auto x : a)
        {
            ans += popcount(x);
        }
        return ans;
    }

    DynBitset &operator|=(const DynBitset &o)
    {
        // 逐位或入 o，返回当前集合
        assert(n == o.n); // 调试检查，可删
        for (int i = 0; i < (int)a.size(); i++)
        {
            a[i] |= o.a[i];
        }
        return *this;
    }

    DynBitset &operator&=(const DynBitset &o)
    {
        // 逐位与入 o，返回当前集合
        assert(n == o.n); // 调试检查，可删
        for (int i = 0; i < (int)a.size(); i++)
        {
            a[i] &= o.a[i];
        }
        return *this;
    }
};
```

`init(m)` 要求 $m \geq 0$，单点接口要求 $0 \leq p < n$，两个集合做按位运算时位数必须相同。单点操作为 $O(1)$，计数与按位运算为 $O\left(\frac{n}{64}\right)$。以下固定例覆盖机器字边界 $63/64$、末位、清位与按位或：

```
signed main()
{
```

```
    DynBitset a(70);
    a.set(0);
    a.set(63);
    a.set(69);
    DynBitset b(70);
    b.set(64);
    a |= b;
    assert(a.count() == 4 && a.test(63) && a.test(64));
    a.set(69, false);
    assert(a.count() == 3 && !a.test(69));
    a.reset();
    assert(a.count() == 0);
}
```

1.4.006 GNU PBDS

PBDS 是 libstdc++ 的非标准扩展，只能在提供相应头文件的 GNU 环境中使用。常用接口包括支持排名的 `tree`、可改键与合并的 `priority_queue`，以及 `cc_hash_table`、`gp_hash_table`。哈希表的默认哈希仍然可预测；需要抵御构造碰撞时，应显式使用 1.4.004 的随机化哈希器。三个扩展头必须写在 `#define int long long` 之前（放在文件最开头），否则宏会改写库内部的 `int` 声明而编译失败。下面给出四类接口的最小调用方式：

```
// 依赖 1.4.004 unordered_map / unordered_set 防 hack 哈希
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/priority_queue.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace __gnu_pbds;

template <class T> using OrderedSet = tree<
    T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

using MinHeap = __gnu_pbds::priority_queue<
    int, greater<int>, pairing_heap_tag>;

using ResizePolicy = hash_standard_resize_policy<
    hash_exponential_size_policy<>,
    hash_load_check_resize_trigger<>,
    true>;

using SizedHashMap = gp_hash_table<
    unsigned long long, // 64 位无符号键，保留完整机器字范围
    int,
    CustomHash,
    equal_to<unsigned long long>,
    direct_mask_range_hashing<>,
    linear_probe_fn<>,
    ResizePolicy>;
```

```cpp

void pbdsExample(int x, int id, int k)
{
    // x、id 是插入有序树的值与编号，k 是 0 起排名；演示四类 PBDS 接口，输出 x 的排名与第 k 小的值。
    OrderedSet<pair<int, int>> s;
    s.insert({x, id});
    auto rank = s.order_of_key({x, numeric_limits<int>::min()});
    assert(0 <= k && k < (int)s.size()); // 调试检查，可删
    auto value = *s.find_by_order(k);

    MinHeap a;
    MinHeap b;
    auto it = a.push(5);
    a.modify(it, 2);
    a.erase(it);
    a.join(b);

    cc_hash_table<int, int, CustomHash> chain;
    gp_hash_table<int, int, CustomHash> probe;
    chain[x]++;
    probe[x]++;

    SizedHashMap mp1;
    mp1.resize(1ULL << 20); // 批量插入前把桶数调到 2 的幂
    mp1[(unsigned long long)x]++;
    cout << rank << ' ' << value.first << '\n';
}
```

有序树的插入、删除、排名和第 k 小为 $O(\log n)$；解引用 `find_by_order(k)` 前要求 $0 \le k < |s|$，比较器必须是严格弱序，不能用 `less_equal` 模拟多重集合。`pairing_heap_tag` 的 `modify` 按摊还 $O(\log n)$ 使用，`join` 为 $O(1)$；哈希表操作只有期望 $O(1)$。

`gp_hash_table` 的 `resize` 只有当扩容策略 `hash_standard_resize_policy` 的第三个模板参数 `External_Size_Access` 为 `true` 时才能从外部调用；默认策略取 `false`，直接调用会编译失败。它接收扩容策略的表容量目标，不等同于按元素数预留的 `unordered_map::reserve`，也不保证能容纳同样数量的元素而不再扩容。这里的 `direct_mask_range_hashing` 用掩码定位桶，容量应取 2 的幂；`resize` 会重散列已有元素，故应在批量插入前调用。`CustomHash` 仍负责抵御构造碰撞。具体策略与兼容性见 3.9 PBDS 专题。

1.4.007 `lower_bound`、`upper_bound`、`unique`、`iota`

二分有序区间、去重、生成连续编号。`unique` 只把重复项移到尾部并返回新逻辑末尾，必须配 `erase` 才真正删除。二分前必须满足对应有序/单调条件。

```cpp
sort(a.begin(), a.end());
a.erase(unique(a.begin(), a.end()), a.end());               // 排序去重
auto l = lower_bound(a.begin(), a.end(), x) - a.begin();    // 首个 >= x
auto r = upper_bound(a.begin(), a.end(), x) - a.begin();    // 首个 > x
iota(id.begin(), id.end(), 0);                              // 0,1,2,...
```

随机访问迭代器二分 $O(\log n)$；`unique`、`iota` 为 $O(n)$。

1.4.008 next_permutation、prev_permutation、shuffle

按字典序枚举排列或随机打乱。从排序后的序列起循环 next_permutation，可无重复地枚举多重集合的全部排列。random_shuffle 已被移除，一律用 shuffle 配 mt19937。

```
sort(a.begin(), a.end());
do
{
    use(a);
} while (next_permutation(a.begin(), a.end()));

shuffle(a.begin(), a.end(), rng);
```

每次排列推进 $O(n)$，枚举 P 个排列共 $O(Pn)$；打乱 $O(n)$。

1.4.009 accumulate、is_sorted、迭代器

求和、验有序、写泛型区间代码。累加初值决定结果类型：#define int long long 改不了字面量 0 的 int 类型，大整数求和仍须显式写 0LL，否则中途溢出。

```
auto sum = accumulate(a.begin(), a.end(), 0LL);   // 初值 0LL 定死 long long
assert(is_sorted(a.begin(), a.end()));
for (auto it = a.begin(); it != a.end(); it++)
{
    use(*it);
}
```

均为 $O(n)$ 时间、$O(1)$ 额外空间。

1.4.010 __builtin_* 与 C++20 位运算函数

统计位数、找最高位、二进制上取整。C++20 的 popcount/bit_width/bit_ceil/countr_zero 要求无符号实参，比 __builtin_* 更安全：countr_zero(0) 有定义（返回位宽），而 __builtin_ctzll(0) 未定义。

```
auto u = (unsigned long long)x;
auto ones = popcount(u);                                  // 1 的个数
int hi = u ? (int)bit_width(u) - 1 : -1;                  // 0 的最高位约定为 -1
auto p2 = bit_ceil((unsigned long long)max<int>(1, n));   // >= n 的最小 2 的幂
auto z = countr_zero(u);                                  // 末尾 0 的个数
```

这些操作通常编译为单条或少量指令，视为 $O(1)$。调用 bit_ceil 时还要保证结果能由实参的无符号类型表示。

1.4.011 sort killer 与哈希 hack

理解旧式快排和固定哈希会被构造数据卡退化，据此做防御（非鼓励攻击评测）：

1. 排序直接用标准库 sort——它是 introsort，最坏退化时自动转堆排序，保证 $O(n \log n)$ 次比较。

2. 哈希表用 `CustomHash`、`reserve` 与较低装载因子（1.4.004）。
3. 需要稳定最坏复杂度时用 `map/set` 或排序离线处理。

哈希仍只有期望 $O(1)$。

1.4.012 何时才需要卡常

只在复杂度正确且剖析已经定位瓶颈后降低常数。卡常不能替代算法改进，具体入口见 1.5.001 卡常使用边界与专题索引。常规提交先检查以下事项：

- 连续数据优先 `vector` 并提前 `reserve`，避开链式容器与频繁分配。
- 热循环避免 `endl`（每次 flush）、虚函数、重复取模、`std::function` 类型擦除。
- 用 `bitset`/机器字并行处理布尔转移。
- 快读只在 I/O 确占主要时间时启用。
- 编译器、评测机指令集和内存限制必须满足所用优化的条件。

只改常数，不改渐进复杂度。

1.4.013 字符串分割：getline / Split

按单字符分隔符切分整行，并保留连续分隔符产生的空字段——这是它与 `stringstream` 按空白切分的关键区别：后者会吞掉空字段。

```
vector<string> split(const string &s, char sep = ' ')
{
    // 按 sep 切分，保留空字段
    vector<string> ans;
    string cur;
    for (auto c : s)
    {
        if (c == sep)
        {
            ans.push_back(cur);
            cur.clear();
        }
        else
        {
            cur.push_back(c);
        }
    }
    ans.push_back(cur);
    return ans;
}
```

`split("a,,bc,", ',')` 得到 `{"a", "", "bc", ""}`。按任意空白切数字用 `stringstream`，按固定字符且空字段有意义时用 `split`。长度 n 时 $O(n)$。

1.4.014 optional

`optional<T>` 表示“一个 `T` 或没有值”，适合表达查找失败、无解等状态，避免拿 `-1`、`INF` 等可能与合法答案冲突的哨兵。默认构造和 `nullopt` 都表示空；装入一个 `T` 后才可以解引用。

```
optional<int> firstPos(const vector<int> &a, int x)
{
    // 返回 x 在 a 中首次出现的下标，不存在时返回空
    for (int i = 0; i < (int)a.size(); i++)
    {
        if (a[i] == x)
        {
            return i;
        }
    }
    return nullopt;
}

void optExample(const vector<int> &a, int x)
{
    // 输出 x 的首次出现位置，并演示置值与置空
    auto pos = firstPos(a, x);
    if (pos)
    {
        cout << *pos << '\n';
    }
    else
    {
        cout << "not found\n";
    }

    optional<int> ans;
    ans = 42;                   // 现在含有值 42
    ans.reset();                // 重新置空
    int v = ans.value_or(-1);   // 空时取得默认值 -1
    cout << v << '\n';
}
```

用 `if (x)` 或 `x.has_value()` 判空，确认非空后用 `*x`、`x->member` 访问；`value()` 在空值上会抛出 `bad_optional_access`，竞赛代码通常先判空。`value_or(defaultValue)` 只提供一个替代值，不会改变原对象；`optional` 只区分成功与无值，需要区分多种失败原因时改用状态码或 `variant`。除所存对象自身的构造、析构成本外，判空、访问、`reset` 均为 $O(1)$；设 `a` 的长度为 n，示例 `firstPos` 的时间复杂度为 $O(n)$、额外空间为 $O(1)$。

1.5 平台优化与极限卡常

本节只保留比赛现场能直接使用的卡常入口。先确认复杂度正确，再用 1.3.006 GNU C++ 环境、本地计时与评测机测速 定位常数；依赖具体测试答案的硬编码、越界访问和有符号溢出不属于优化。

1.5.001 卡常使用边界与专题索引

普通提交先用 `-O2`、连续容器、预留容量和 1.2.001 快速输入输出。确实接近时限时再从 1.3.001 编译参数与调试构建 的 GCC 清单逐项试验，并对同一固定输入核对答案和时间。下表保留各类平台优化的有效结论；具体快慢依赖 CPU、编译器、数据分布和问题规模，不把一次本机结果写成普遍顺序。

技术	适用条件	复杂度与边界
AVX2 自动向量化或专用循环	x86 评测机明确支持 AVX2，数据连续且循环无依赖	扫描 n 项仍为 $O(n)$ 时间、$O(1)$ 额外空间；一次处理更多元素只改变常数，不同 CPU 上收益可能反转
PCLMUL / NEON PMULL	固定宽度二进制多项式乘法，且运行平台支持对应指令	固定 64 位硬件乘法视为 $O(1)$；标量回退需逐位处理，推广到 w 位时为 $O(w^2)$，均为 $O(1)$ 额外空间
延迟约减	已证明一个分块内全部乘加不会超过累计类型	长度 n 的点积为 $O(n)$ 时间、$O(1)$ 额外空间；块长必须由模数和累计类型上界推出，不能只靠经验值
radix-4 NTT、根表复用与频域驻留	固定友好模数、长度满足单位根条件，并有大量同规模卷积	10.1.003 NTT 已给 `FastNTT`、radix-2 回退、固定校验和与本机基准；两版均为 $O(N \log N)$ 时间、$O(N)$ 空间，基准差异只作当前环境参考
`mmap` / `fread` / 手写输出缓冲	输入确实占主要时间；`mmap` 还要求普通文件，管道输入必须回退	读写 B 字节均为 $O(B)$；固定缓冲额外空间为 $O(S)$，映射额外占用 $O(B)$ 虚拟地址空间，是否更快依赖系统与输入来源
arena / 空闲表	大量等生命周期小对象，析构可统一处理	预留 C 字节后单次顺序分配为 $O(1)$，清空为 $O(1)$，占用 $O(C)$；需要逐对象析构或未知峰值时不适合
CSR / SoA / 分块与预取	图或记录静态，遍历受缓存而非运算限制	CSR 建表 $O(n+m)$ 时间和空间，完整扫描 $O(m)$；预取和字段拆分不改变复杂度，收益必须按目标平台实测
位表、分支提示与经验阈值	状态域固定且表可由算法定义生成	固定 64 位 `popcount` 或查表视为 $O(1)$；阈值选择为 $O(1)$，被选择算法仍按自身复杂度计算，阈值须记录基准环境

固定模乘可选 7.1.009 Barrett 约减 或 7.1.010 Montgomery 模乘。AVX2、PCLMUL、`mmap`、通用内存池、手写预取和经验阈值没有一份能直接适用于所有题目的短接口，因此这里只保留选择条件与严格界；需要时在当前题目代码旁写专用版本和可移植回退。依赖具体测试答案的硬编码、越界访问和有符号溢出

不属于优化。

2. 基础算法与离线算法

2.1 搜索与枚举

2.1.001 整数二分、实数二分

在单调谓词上定位分界点。整数版维护「答案仍在闭区间 $[l, r]$ 中」的不变量，每轮用 `midpoint` 安全折半，`firstTrue` 返回第一个为真的位置、`lastTrue` 返回最后一个为真的位置；`lastTrue` 里 `m == l` 时上取到 `r`，是为了偶数长度不陷入死循环。两者都不设 $l-1$ / $r+1$ 哨兵，调用者须保证答案确在给定闭区间内。实数版不比较浮点端点是否相等，改用固定迭代轮数把误差压到初始长度的 2^{-it}。二分只对单调判定成立，不能对任意函数盲套。

```
template <class F> int firstTrue(int l, int r, F f)
{
    // l、r 是答案所在闭区间，f 是由假变真的谓词；返回第一个为真的位置。
    while (l < r)
    {
        auto m = midpoint(l, r);
        if (f(m))
        {
            r = m;
        }
        else
        {
            l = m + 1;
        }
    }
    return l;
}

template <class F> int lastTrue(int l, int r, F f)
{
    // l、r 是答案所在闭区间，f 是由真变假的谓词；返回最后一个为真的位置。
    while (l < r)
    {
        auto m = midpoint(l, r);
        if (m == l)
        {
            m = r;
        }
        if (f(m))
        {
            l = m;
        }
        else
        {
            r = m - 1;
        }
    }
```

```
    return l;
}

template <class F> long double firstReal(long double l, long double r, F f, int it = 100)
{
    // l、r 是初始实数区间，f 是单调谓词，it 是迭代轮数；返回分界点近似值。
    for (int t = 0; t < it; t++)
    {
        auto m = midpoint(l, r);
        if (f(m))
        {
            r = m;
        }
        else
        {
            l = m;
        }
    }
    return midpoint(l, r);
}
```

若要在已有随机访问有序区间上反复做下界/上界查询，分支结果通常难以预测，可以使用无分支步长收缩版：

```
template <class It, class T, class F = less<>> It fastLower(It first, It last, const T &x, F f
↪ = {})
{
    // [first,last) 必须支持随机访问；返回第一个不小于 x 的位置。
    assert(first <= last);
    int n = last - first;
    for (n++; n > 1;)
    {
        int h = n / 2;
        n -= h;
        first += f(first[h - 1], x) * h;
    }
    return first;
}

template <class It, class T, class F = less<>> It fastUpper(It first, It last, const T &x, F f
↪ = {})
{
    // [first,last) 必须支持随机访问；返回第一个大于 x 的位置。
    assert(first <= last);
    int n = last - first;
    for (n++; n > 1;)
    {
        int h = n / 2;
        n -= h;
        first += !f(x, first[h - 1]) * h;
    }
```

```
    return first;
}
```

谓词方向决定用哪支：`firstTrue` 的谓词须由假变真，`lastTrue` 的谓词须由真变假；无解时先在调用前验证边界谓词，或把返回区间外扩一格作显式哨兵。三个谓词接口的最小调用如下：

```
signed main()
{
    int x = firstTrue(0, 10, [](int p)
    {
        return p * p >= 20;
    });
    int y = lastTrue(0, 10, [](int p)
    {
        return p * p <= 20;
    });
    long double z = firstReal(0, 2, [](long double p)
    {
        return p * p >= 2;
    });
    assert(x == 5 && y == 4 && fabsl(z * z - 2) < 1E-12L);
}
```

整数版调用谓词 $\lceil \log_2(r-l+1) \rceil$ 次、额外空间 $O(1)$；实数版固定调用 it 次。`fastLower/fastUpper` 与标准库语义相同，允许空区间，但只适合随机访问迭代器；它们仍是 $O(\log n)$，优势只在不可预测比较下减少分支。本机一百万次随机下界查询的 7 轮中位数约为 `66ms`，标准 `lower_bound` 约为 `112ms`；小数组或比较器很贵时差距可能消失，默认仍可用标准库版本。

2.1.002 整数三分、实数三分、黄金分割三分

求单峰函数的极值，模板求最大值；求最小值时应反转比较，只有确认值域取负不溢出时才传 `-f(x)`。比较两个三等分点后能证明峰值不在较差点外侧那一段，于是每轮丢掉约三分之一。整数版收窄到常数宽度后逐点枚举剩余的至多 5 个位置，专门兜住平台峰与整除取整；`ternaryReal` 每轮独立取两点，`goldenMax` 用黄金比 $\varphi - 1 \approx 0.618$ 布点，使收缩后有一个内点可复用、每轮只新增一次求值，因此函数求值昂贵时更省。三分要求真单峰，函数有多个相隔局部峰时没有正确性保证。

```
template <class F> int ternaryInt(int l, int r, F f)
{
    // l、r 是整数闭区间，f 是单峰函数；返回一个最大值位置。
    // 求最小值要把收缩循环和末尾扫描中的 f 比较同时反向；只改一处会得到错误区间。
    while (r - l > 4)
    {
        auto d = r - l;
        auto x = l + d / 3;
        auto y = r - d / 3;
        if (f(x) < f(y))
        {
            l = x + 1;
        }
```

```
        else
        {
            r = y - 1;
        }
    }
    auto p = l;
    auto x = l;
    while (x < r)
    {
        x++;
        if (f(p) < f(x))
        {
            p = x;
        }
    }
    return p;
}

template <class F> long double ternaryReal(long double l, long double r, F f, int it = 120)
{
    // l、r 是实数区间，f 是单峰函数，it 是迭代轮数；返回最大值位置近似值。
    // 求最小值时反向循环内的 f(x)、f(y) 比较；若改传 -f，先确认取负不会溢出。
    for (int t = 0; t < it; t++)
    {
        auto x = lerp(l, r, 1.0L / 3);
        auto y = lerp(l, r, 2.0L / 3);
        if (f(x) < f(y))
        {
            l = x;
        }
        else
        {
            r = y;
        }
    }
    return midpoint(l, r);
}

template <class F> long double goldenMax(long double l, long double r, F f, int it = 120)
{
    // l、r 是实数区间，f 是单峰函数，it 是迭代轮数；返回黄金分割搜索结果。
    // 求最小值时反向 fx、fy 的比较；其余复用点和区间更新方向不变。
    constexpr long double ratio = 0.61803398874989484820L;
    auto x = lerp(l, r, 1 - ratio);
    auto y = lerp(l, r, ratio);
    auto fx = f(x);
    auto fy = f(y);
    for (int t = 0; t < it; t++)
    {
        if (fx < fy)
        {
            l = x;
```

```
            x = y;
            fx = fy;
            y = lerp(l, r, ratio);
            fy = f(y);
        }
        else
        {
            r = y;
            y = x;
            fy = fx;
            x = lerp(l, r, 1 - ratio);
            fx = f(x);
        }
    }
    return midpoint(l, r);
}
```

只在能证明单峰时使用。三个函数都可直接接收 Lambda：

```
signed main()
{
    auto fi = [](int x)
    {
        return -(x - 3) * (x - 3);
    };
    auto fr = [](long double x)
    {
        return -(x - 1.5L) * (x - 1.5L);
    };
    assert(ternaryInt(-10, 10, fi) == 3);
    assert(fabsl(ternaryReal(0, 3, fr) - 1.5L) < 1E-12L);
    assert(fabsl(goldenMax(0, 3, fr) - 1.5L) < 1E-12L);
}
```

整数版要求调用前保证 `r-l` 能由 64 位有符号整数表示，循环中的差值和三分点因此不会溢出；实数版要求两个端点有限，`midpoint` 与 `lerp` 避免先把同号大端点相加或先计算跨极值的 `r-l`。整数版进行 $O(\log(r-l+1))$ 轮、每轮两次求值，末尾至多检查 5 点；`ternaryReal` 固定 $2\cdot it$ 次求值，`goldenMax` 初始化 2 次、之后每轮新增 1 次共 $it+2$ 次；额外空间均为 $O(1)$。

2.1.003 全排列与不重复排列

按字典序枚举多重集合的全部不同排列。先排序得到最小字典序，再反复 `next_permutation`：标准库按「下一个严格更大的序列」推进，遇到相同元素时自动跳过，因此不会产生重复排列。

```
vector<int> a{1, 1, 2}; // a 是待枚举的多重集合。
sort(a.begin(), a.end());
do
{
    use(a); // use 处理当前排列。
} while (next_permutation(a.begin(), a.end()));
```

必须先排序再进入 `do while`，故 `{1,1,2}` 恰好产生 3 个排列；`use(a)` 在循环体内直接处理当前排列，需要留存时自行复制一份 `vector`。不同排列数为 P 时，排序 $O(n \log n)$、枚举 $O(Pn)$；`next_permutation` 本身为 $O(1)$ 额外空间，常用 libstdc++/libc++ 的 `sort` 栈空间为 $O(\log n)$，若换用带缓冲的排序实现则以该实现为准，回调自行复制才额外需要 $O(n)$。

2.1.004 子集枚举、固定大小子集枚举

用位掩码枚举子集：普通枚举把第 i 位是否为 1 解释为是否选第 i 个元素，直接从 0 递增到 2^n-1。Gosper's hack 由当前掩码直接构造下一个 popcount 相同的掩码——`low = s & -s` 取最低置位，`s + low` 进位后再把低位补齐——因此只走恰有 k 位的状态，不扫描其余掩码。两段都限定 $n < 63$，避免 `1ULL << n` 越过无符号 64 位边界。

```
// n 是元素数，s 是当前子集掩码，use 处理当前子集。
assert(0 <= n && n < 63); // 调试检查，可删
for (auto s = 0ULL; s < (1ULL << n); s++)
{
    use(s);
}
```

只枚举恰含 k 个元素的子集时，用 Gosper's hack 直接跳到下一个同位数掩码：

```
// n 是元素数，k 是子集大小，s 是当前掩码，use 处理当前子集。
assert(0 <= k && k <= n && n < 63); // 调试检查，可删
if (k == 0)
{
    use(0ULL);
}
else
{
    for (auto s = (1ULL << k) - 1; s < (1ULL << n);)
    {
        use(s);
        auto low = s & -s;
        auto nxt = s + low;
        s = nxt | (s ^ nxt) / low >> 2;
    }
}
```

两段都要求 $0 \le n < 63$，固定大小版本还要求 $0 \le k \le n$；$k = 0$ 时直接处理唯一的空集。若只要枚举某个掩码 `s` 的非空子集，用 `for (t = s; t; t = (t - 1) & s)`，不要遍历整个状态空间。全部子集共 2^n 个、固定大小子集恰 $\binom{n}{k}$ 个，每次推进 $O(1)$、额外空间 $O(1)$。

2.1.005 Gray Code

Gray Code 把 $0, 1, \ldots, 2^n-1$ 重编码为 n 位序列，使相邻编码的汉明距离恒为 1，适合枚举子集时每步只增删一个元素（增量维护子集和、行列式符号、状态代价等），它只改变访问顺序、不减少 2^n 个状态。二进制反射编码为 $g = x \oplus (x \gg 1)$：相邻整数加一会翻转从最低位起的一段连续位，与右移一位的自身异或后这段翻转两两抵消，只剩一位变化；当 $n > 0$ 时，反射构造还保证完整序列首尾只差一位，形成

哈密顿环。逆变换满足 $x_i = g_i \oplus x_{i+1}$，即从最高位向低位做前缀异或，代码靠不断右移并累积异或一次完成。

```
// x 是普通二进制编号，g 是对应的 Gray Code。
auto g = x ^ (x >> 1);
```

从 Gray Code 还原普通编号时，把 g 的各次右移逐一异或：

```
// g 是 Gray Code，x 是还原后的普通二进制编号。
auto x = g;
for (auto y = g >> 1; y; y >>= 1)
{
    x ^= y;
}
```

对普通编号 `x` 执行第一段得 `g`，对 `g` 执行第二段还原 `x`；想知道第 x 个状态相对前一个翻转了哪一位，算 `g ^ preG` 取其唯一置位即可。当 $n > 0$ 时，连续枚举完整的 2^n 个 n 位 Gray Code，其相邻项与首尾都恰差一位；截取任意前缀后不再保证首尾相邻。逆变换仅适用于此处的二进制反射编码。编码在 64 位机器字上 $O(1)$，解码执行 g 的二进制位数减一次那么多轮，即 $O(\log(g+1))$ 时间、$O(1)$ 空间。

2.1.006 DFS / BFS 基础搜索

BFS 求无权图最短边数：队列按距离层弹出，节点首次被访问时的距离即最短。DFS 沿一条分支深入到底再回溯，访问顺序由邻接表决定；本模板按约定直接写递归 lambda，深链场景需确认系统栈足够。两者都靠 `d` / `vis` 保证每个点只正式处理一次。

```
vector<int> bfs(const vector<vector<int>> &g, int s)
{
    // g 是邻接表，s 是起点；返回各顶点到 s 的最少边数，-1 表示不可达。
    vector<int> d(g.size(), -1);
    queue<int> q;
    d[s] = 0;
    q.push(s);
    while (!q.empty())
    {
        auto x = q.front();
        q.pop();
        for (auto y : g[x])
        {
            if (d[y] != -1)
            {
                continue;
            }
            d[y] = d[x] + 1;
            q.push(y);
        }
    }
    return d;
}

```

```cpp
vector<int> dfsOrder(const vector<vector<int>> &g, int s)
{
    // g 是邻接表，s 是起点；返回从 s 出发的递归 DFS 首次访问顺序。
    vector<int> ans, vis(g.size());
    auto dfs = [&](auto &&self, int x) -> void
    {
        vis[x] = 1;
        ans.push_back(x);
        for (auto y : g[x])
        {
            if (!vis[y])
            {
                self(self, y);
            }
        }
    };
    dfs(dfs, s);
    return ans;
}
```

图用 `vector<vector<int>>` 邻接表、顶点编号 $[0, n)$，`bfs(g, s)` 返回各点到 `s` 的最少边数（`-1` 表示不可达），`dfsOrder(g, s)` 返回首次访问序（具体顺序由邻接表顺序决定）。邻接表图上均为 $O(n+m)$ 时间、$O(n)$ 额外空间；递归 DFS 另占最坏 $O(n)$ 调用栈。

2.1.007 启发式搜索与 A*（第 K 短路）

求第 k 短游走（允许重复经过顶点和边，不是第 k 条简单路径）。先在反图上跑 Dijkstra，得到每点到终点的真实最短距离 $h(x)$，它既不高估又满足一致性，是理想的 A* 启发函数；随后正图 A* 每次弹出全局最小的 $g+h$ 状态，同一顶点第 c 次有效弹出对应到它的第 c 条候选游走，终点达到所需弹出次数时即得答案。每点只扩展至多 `need` 次；通常 `need == k`，仅在起终点相同且不计空路时为 `k + 1`。负边会同时破坏反图 Dijkstra 与这一弹出顺序，故要求边权非负；可达状态用 `optional` 与距离分开，调用者必须保证实际距离、估价与二者之和可由 64 位有符号整数表示。

```cpp
// WEdge[0] 表示终点，WEdge[1] 表示边权。
using WEdge = array<int, 2>;

optional<int> kthShortest(const vector<vector<WEdge>> &g, int s, int t, int k, bool allowEmpty
    = false)
{
    // g 是非负权图，s、t 是起终点，k 是排名，allowEmpty 表示是否计入空路；不存在第 k 条游走时返回空。
    auto n = (int)g.size();
    assert(0 <= s && s < n && 0 <= t && t < n); // 调试检查，可删
    assert(k >= 1);                             // 调试检查，可删
    vector<vector<WEdge>> rg(n);
    for (int x = 0; x < n; x++)
    {
        for (auto [y, w] : g[x])
        {
            assert(0 <= y && y < n && w >= 0); // 调试检查，可删
```

```
            rg[y].push_back({x, w});
        }
    }
    vector<optional<int>> h(n);
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> dq;
    h[t] = 0;
    dq.push({0, t});
    while (!dq.empty())
    {
        auto [dx, x] = dq.top();
        dq.pop();
        if (!h[x] || dx != *h[x])
        {
            continue;
        }
        for (auto [y, w] : rg[x])
        {
            auto d = dx + w;
            if (!h[y] || d < *h[y])
            {
                h[y] = d;
                dq.push({d, y});
            }
        }
    }
    if (!h[s])
    {
        return nullopt;
    }
    auto need = (unsigned long long)k + (s == t && !allowEmpty);
    // State 依次保存估价距离、已走距离和当前顶点。
    using State = tuple<int, int, int>;
    priority_queue<State, vector<State>, greater<>> q;
    vector<unsigned long long> cnt(n);
    q.push({*h[s], 0, s});
    while (!q.empty())
    {
        auto cur = q.top();
        q.pop();
        auto d = get<1>(cur);
        auto x = get<2>(cur);
        cnt[x]++;
        if (cnt[x] > need)
        {
            continue;
        }
        if (x == t && cnt[x] == need)
        {
            return d;
        }
        for (auto [y, w] : g[x])
        {
```

```
            if (!h[y])
            {
                continue;
            }
            auto nd = d + w;
            q.push({nd + *h[y], nd, y});
        }
    }
    return nullopt;
}
```

边表 `WEdge = {to, weight}`（`[0]` 终点、`[1]` 非负边权），建图后调用 `kthShortest(g,s,t,k)`，`nullopt` 只表示没有第 k 条游走；调用者须保证距离与估价和能由 `int` 表示。默认 `s == t` 时不把空路径计入，传 `allowEmpty = true` 才计入长度 0。以边集 $0\to1(1),0\to2(2),0\to3(10),1\to2(1),1\to3(4),2\to3(1)$ 为例，$0\to3$ 的前三短游走长度为 `3,3,5`。反图 Dijkstra 为 $O((n+m)\log n)$；A* 中每点至多有效弹出 $k+1$ 次、扩边至多 $O(km)$ 次，堆复杂度 $O(km\log(kn+km))$、最坏空间 $O(kn+km)$。改成简单路径则问题性质不同，不能套此模板。

2.1.008 模拟退火

无可靠精确算法、但状态可局部扰动且评分连续时求近似最优解，它没有正确性保证，不能替代可证明的凸优化、DP 或搜索。更优状态必然接受，更差状态以 $\exp(-\Delta/t)$ 的概率接受：高温放行较差跳转以逃出局部最优，降温后逐渐收敛稳定。结果质量高度依赖邻域设计、初温、降温率 `rate`、每层尝试次数 `each` 和多起点策略，因此接口把这些旋钮全部显式交给调用者。

```
template <class S, class Score, class Next, class Rng>
pair<S, long double> anneal(S cur,
                            Score score,
                            Next next,
                            Rng &rng,
                            long double t = 1,
                            long double stop = 1E-8L,
                            long double rate = .995L,
                            int each = 1)
{
    // cur 是初态，score/next 是评分与邻域函数，rng 是随机源；其余参数控制降温，返回最佳状态及分数。
    assert(t > stop && stop > 0 && 0 < rate && rate < 1 && each > 0); // 调试检查，可删
    long double val = (long double)score(cur);
    auto best = cur;
    long double ans = val;
    uniform_real_distribution<long double> dis(0, 1);
    while (t > stop)
    {
        for (int z = 0; z < each; z++)
        {
            auto p = next(cur, rng, t);
            long double v = (long double)score(p);
            long double d = v - val;
            if (d <= 0 || dis(rng) < expl(-d / t))
```

```
            {
                cur = move(p);
                val = v;
            }
            if (val < ans)
            {
                best = cur;
                ans = val;
            }
        }
        t *= rate;
    }
    return {best, ans};
}
```

`score` 越小越好且结果须可转为 `long double`，`next(cur,rng,t)` 生成邻居。下面从已知最优点出发，固定种子并核对最优状态不会丢失：

```
signed main()
{
    mt19937_64 rng(1);
    auto score = [](int x)
    {
        return (long double)(x - 3) * (x - 3);
    };
    auto next = [](int x, auto &rng, long double t)
    {
        (void)t;
        return x + uniform_int_distribution<int>(-1, 1)(rng);
    };
    auto [state, value] = anneal(3LL, score, next, rng, 1.L, 1E-4L, .9L, 4);
    assert(state == 3 && value == 0);
}
```

参数要求 $t_0 > stop > 0$、$0 < rate < 1$、`each > 0`，固定随机种子便于复现，正式提交宜多起点运行取最好值。温度层数为 $\left\lceil \frac{\log(stop/t_0)}{\log(rate)} \right\rceil$，候选总数为该值乘 `each`，总时间再乘单次 `score` 与 `next` 的代价，空间只两个状态；正确率无渐进保证。

2.1.009 DLX 精确覆盖

精确覆盖要求从 $0/1$ 矩阵中选若干行，使每一列恰被一个所选行覆盖；数独和棋盘精确铺放都能把候选决策写成行、把必须唯一满足的约束写成列。DLX 用循环双向十字链表实现 Algorithm X：递归时选择当前节点最少的列 c，尝试该列中的每一行；选行后删除它覆盖的全部列以及与这些列冲突的其他行，回溯时严格按相反顺序恢复，链表就回到进入递归前的状态。这里所有列都是必须覆盖一次的主列，列编号为 $1 \dots m$；同一行的列不得重复，只有“至多一次”的次要列需要另行扩展表头规则。

```
struct DLX
{
    // m 是必须恰好覆盖的列数。
```

```
    int m;
    // l、r、u、d 是十字链表节点向左、右、上、下的指针。
    vector<int> l, r, u, d;
    // row、col 分别是数据节点对应的行编号和列编号。
    vector<int> row, col;
    // sz[c] 是当前列 c 中仍未删除的数据节点数。
    vector<int> sz;
    // cur 是当前递归路径选中的行编号，ans 是找到的完整答案。
    vector<int> cur, ans;

    explicit DLX(int cols, int reserveNodes = 0)
        : m(cols), l(m + 1), r(m + 1), u(m + 1), d(m + 1), row(m + 1), col(m + 1), sz(m + 1)
    {
        // cols 是列数，reserveNodes 是预估的数据节点数；建立带根节点的列链表。
        assert(m >= 0 && reserveNodes >= 0); // 调试检查，可删。
        for (int i = 0; i <= m; i++)
        {
            l[i] = i - 1;
            r[i] = i + 1;
            u[i] = d[i] = col[i] = i;
        }
        l[0] = m;
        r[m] = 0;
        if (reserveNodes > 0)
        {
            auto cap = m + 1 + reserveNodes;
            l.reserve(cap);
            r.reserve(cap);
            u.reserve(cap);
            d.reserve(cap);
            row.reserve(cap);
            col.reserve(cap);
        }
    }

    void addRow(int id, const vector<int> &columns)
    {
        // id 是返回答案时使用的行编号，columns 是该行覆盖的列编号；把一行加入十字链表。
        int head = -1;
        for (int k = 0; k < (int)columns.size(); k++)
        {
            int c = columns[k];
            assert(1 <= c && c <= m); // 调试检查，可删。
            assert(find(columns.begin(), columns.begin() + k, c) == columns.begin() + k); // 调
            ↪ 试检查，可删。
            int x = (int)l.size();
            l.push_back(x);
            r.push_back(x);
            u.push_back(u[c]);
            d.push_back(c);
            row.push_back(id);
            col.push_back(c);
```

```
            d[u[c]] = x;
            u[c] = x;
            sz[c]++;
            if (head == -1)
            {
                head = x;
            }
            else
            {
                l[x] = l[head];
                r[x] = head;
                r[l[head]] = x;
                l[head] = x;
            }
        }
    }

    optional<vector<int>> solve()
    {
        // 无参数；返回一组覆盖全部列的行编号，无解时返回空。
        cur.clear();
        ans.clear();
        if (!dfs())
        {
            return nullopt;
        }
        return ans;
    }

  private:
    void cover(int c)
    {
        // c 是当前仍在列链表中的列编号；从结构中删除该列及与其冲突的行。
        r[l[c]] = r[c];
        l[r[c]] = l[c];
        for (int i = d[c]; i != c; i = d[i])
        {
            for (int j = r[i]; j != i; j = r[j])
            {
                d[u[j]] = d[j];
                u[d[j]] = u[j];
                sz[col[j]]--;
            }
        }
    }

    void uncover(int c)
    {
        // c 是最近一次删除的列编号；按相反顺序恢复列和冲突行。
        for (int i = u[c]; i != c; i = u[i])
        {
            for (int j = l[i]; j != i; j = l[j])
```

```
            {
                sz[col[j]]++;
                d[u[j]] = j;
                u[d[j]] = j;
            }
        }
        r[l[c]] = c;
        l[r[c]] = c;
    }

    bool dfs()
    {
        // 无参数；返回当前子问题是否有解，并在成功时把行编号写入 ans。
        if (r[0] == 0)
        {
            ans = cur;
            return true;
        }
        int c = r[0];
        for (int j = r[c]; j != 0; j = r[j])
        {
            if (sz[j] < sz[c])
            {
                c = j;
            }
        }
        if (sz[c] == 0)
        {
            return false;
        }
        cover(c);
        for (int i = d[c]; i != c; i = d[i])
        {
            cur.push_back(row[i]);
            for (int j = r[i]; j != i; j = r[j])
            {
                cover(col[j]);
            }
            bool found = dfs();
            for (int j = l[i]; j != i; j = l[j])
            {
                uncover(col[j]);
            }
            cur.pop_back();
            if (found)
            {
                uncover(c);
                return true;
            }
        }
        uncover(c);
        return false;
```

```
    }
};
```

先构造 `DLX dlx(m,z)`，其中 `z` 只是用于预留数据节点容量的可选估计，再以 `addRow(id,columns)` 加入候选行并调用 `solve()`；返回值中的行编号按搜索选择顺序排列，空值表示无解，$m = 0$ 时空行集就是合法答案。设矩阵中 1 的总数为 z，建表时间与空间均为 $O(m + z)$；搜索最坏为指数级，单次删除或恢复的代价等于实际遍历的链表节点数，最小列启发式只改善搜索树而不改变最坏上界。若只需一般集合覆盖的最少行数，问题不再是精确覆盖，应使用状态压缩 DP、搜索剪枝或近似算法。

下面用七列标准实例核对唯一解由第 $2, 4, 6$ 行组成，再检查无列实例和无解实例：

```cpp
signed main()
{
    DLX dlx(7, 18);
    dlx.addRow(1, {1, 4, 7});
    dlx.addRow(2, {1, 4});
    dlx.addRow(3, {4, 5, 7});
    dlx.addRow(4, {3, 5, 6});
    dlx.addRow(5, {2, 3, 6, 7});
    dlx.addRow(6, {2, 7});
    auto ans = dlx.solve();
    assert(ans);
    sort(ans->begin(), ans->end());
    assert(*ans == vector<int>({2, 4, 6}));
    auto empty = DLX(0).solve();
    assert(empty && empty->empty());
    DLX bad(2);
    bad.addRow(1, {1});
    assert(!bad.solve());
    cout << (*ans)[0] << ' ' << (*ans)[1] << ' ' << (*ans)[2] << '\n'; // 期望输出：2 4 6
    return 0;
}
```

2.1.010 IDDFS、IDA* 与双向 BFS

三种方法都用于求起点到终点的最少步数或最小代价，适合状态空间很大、无法先建出完整图的搜索题。代码按显式邻接表写成，套到状态搜索时只需把 `g[u]` 换成状态 u 的后继生成，把 `used` 换成路径上状态的判重。

- IDDFS：依次以边数上限 $k = 0, 1, 2, \ldots$ 做限深 DFS，第一次找到终点时的 k 就是最少边数。任何时刻只保存当前路径，内存 $O(d)$（d 为答案深度）；浅层会被重复搜索，但分支因子 $b \geq 2$ 时最后一层占主导，总量仍是 $O(b^d)$。
- IDA*：把限深换成限制 $f(u) = g(u) + h(u)$，其中 $g(u)$ 是已走代价，$h(u)$ 是 u 到终点代价的下界（可采纳：$h(u) \leq \mathrm{dist}(u, t)$，且 $h(t) = 0$）。一轮中超过阈值的结点被剪掉，下一轮阈值取本轮被剪掉的最小 f。由于阈值单调增加且每轮只放行 f 不超过阈值的路径，第一次到达终点时的代价就是最小代价。
- 双向 BFS：从 s 沿原图、从 t 沿反图同时按层扩展，每次扩展规模较小一侧的整层。一个点同时被两侧到达时得到长度 $d_s(v) + d_t(v)$ 的路径，记最好值为 $best$。设两侧待扩展层号为 L_s, L_t，尚未发

现的路径长度至少 $L_s + L_t + 1$，所以 $L_s + L_t \geq best$ 时停止。

```cpp
namespace iter_search
{
optional<vector<int>> iddfs(const vector<vector<int>> &g, int s, int t, int lim)
{
    // g 是有向图邻接表，s、t 是起点和终点，lim 是允许的最多边数；
    // 返回边数最少的简单路径（含两端点的顶点序列），lim 内不可达时返回空。
    vector<char> used(g.size());
    vector<int> path;
    auto dfs = [&](auto &&self, int u, int d) -> bool
    {
        // u 是当前点，d 是还能走的边数；找到 t 时保留 path 并返回 true。
        path.push_back(u);
        used[u] = true;
        if (u == t)
        {
            return true;
        }
        if (d > 0)
        {
            for (int v : g[u])
            {
                if (!used[v] && self(self, v, d - 1))
                {
                    return true;
                }
            }
        }
        used[u] = false;
        path.pop_back();
        return false;
    };
    for (int k = 0; k <= min(lim, (int)g.size() - 1); k++)
    {
        if (dfs(dfs, s, k))
        {
            return path;
        }
    }
    return nullopt;
}

struct Path
{
    __int128 cost;        // cost 是路径边权和。
    vector<int> vertices; // vertices 是从起点到终点的顶点序列。
};

optional<Path> ida(const vector<vector<pair<int, int>>> &g, int s, int t, const vector<int> &h)
{
    // g[u] 存 {v,w}，w 为非负边权；h[u] 是 u 到 t 的可采纳下界，且 h[t]=0。
```

```
    // 返回 s 到 t 的一条最短简单路径，不可达时返回空。
    assert(h.size() == g.size() && h[t] == 0); // 调试检查，可删。
    const __int128 INF = (__int128)1 << 120;
    __int128 lim = h[s], next = INF, found = 0;
    vector<int> path;
    vector<char> used(g.size());
    auto dfs = [&](auto &&self, int u, __int128 cost) -> bool
    {
        // u 是当前点，cost 是已走代价；f=cost+h[u] 超过阈值时记录下一轮阈值并剪枝。
        __int128 f = cost + h[u];
        if (f > lim)
        {
            next = min(next, f);
            return false;
        }
        path.push_back(u);
        used[u] = true;
        if (u == t)
        {
            found = cost;
            return true;
        }
        for (auto [v, w] : g[u])
        {
            if (!used[v] && self(self, v, cost + w))
            {
                return true;
            }
        }
        used[u] = false;
        path.pop_back();
        return false;
    };
    while (true)
    {
        next = INF;
        if (dfs(dfs, s, 0))
        {
            return Path{found, path};
        }
        if (next == INF)
        {
            return nullopt;
        }
        lim = next;
    }
}

optional<vector<int>> biBfs(const vector<vector<int>> &g, int s, int t)
{
    // g 是有向无权图邻接表；返回 s 到 t 的一条最短路径顶点序列，不可达时返回空。
    int n = g.size();
```

```
    vector<vector<int>> rev(n);
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u])
        {
            rev[v].push_back(u);
        }
    }
    // da、db 是正向、反向 BFS 距离，pa、pb 是对应的前驱，a、b 是两侧当前层。
    vector<int> da(n, -1), db(n, -1), pa(n, -1), pb(n, -1), a{s}, b{t};
    da[s] = db[t] = 0;
    int best = n + 1, meet = -1;
    auto relax = [&](int v)
    {
        // v 是刚被访问的点；若两侧都到过 v，就用 v 更新当前最短拼接路径。
        if (da[v] >= 0 && db[v] >= 0 && da[v] + db[v] < best)
        {
            best = da[v] + db[v];
            meet = v;
        }
    };
    relax(s);
    while (!a.empty() && !b.empty() && da[a[0]] + db[b[0]] < best)
    {
        // 每次扩展较小的一侧整层，保证两侧层数之和单调增加。
        bool fw = a.size() <= b.size();
        auto &cur = fw ? a : b;
        auto &d = fw ? da : db;
        auto &p = fw ? pa : pb;
        const auto &adj = fw ? g : rev;
        vector<int> nxt;
        for (int u : cur)
        {
            for (int v : adj[u])
            {
                if (d[v] < 0)
                {
                    d[v] = d[u] + 1;
                    p[v] = u;
                    nxt.push_back(v);
                }
                relax(v);
            }
        }
        cur.swap(nxt);
    }
    if (meet < 0)
    {
        return nullopt;
    }
    vector<int> path;
    for (int u = meet; u >= 0; u = pa[u])
```

```
    {
        path.push_back(u);
    }
    reverse(path.begin(), path.end());
    for (int u = pb[meet]; u >= 0; u = pb[u])
    {
        path.push_back(u);
    }
    return path;
}
}
```

`iddfs(g,s,t,lim)` 只找边数不超过 $\min(lim, n-1)$ 的简单路径，`ida(g,s,t,h)` 要求非负边权、可采纳的 `h` 且 `h[t]==0`，`biBfs(g,s,t)` 用于无权有向图；三者都返回含两端点的顶点序列（`ida` 另带代价），不可达时返回空。IDDFS 与 IDA* 是指数时间：IDDFS 至多枚举长度不超过答案深度的全部简单路径，IDA* 的轮数等于不同阈值个数，整数边权时不超过最优代价加一；二者都只用 $O(n)$ 额外空间（n 为点数，即路径与判重数组）。双向 BFS 时间与空间都是 $O(n+m)$，比单向 BFS 少搜索的部分只在状态图分支很多时才明显。

显式图上的普通最短路直接用 5.1.002 DFS / BFS 或 5.2.001 Dijkstra：稀疏图与稠密图；内存足够时带启发函数的单轮搜索用 2.1.007 启发式搜索与 A*（第 K 短路）。下面的图里 $0 \to 4$ 的直连边只有 1 条边但代价为 9，用来区分最少边数与最小代价：

```
signed main()
{
    vector<vector<int>> g{{1, 2, 4}, {3}, {3}, {4}, {}};
    vector<vector<pair<int, int>>> wg{{{1, 1}, {2, 2}, {4, 9}}, {{3, 5}}, {{3, 1}}, {{4, 1}},
↪   {}};
    vector<int> h{2, 1, 1, 1, 0}; // 每条边代价至少 1，取到 4 的最少边数，可采纳。
    auto p = iter_search::iddfs(g, 0, 3, 5);
    auto q = iter_search::biBfs(g, 0, 4);
    auto r = iter_search::ida(wg, 0, 4, h);
    assert(p && *p == vector<int>({0, 1, 3}));
    assert(q && *q == vector<int>({0, 4}));
    assert(r && r->cost == 4 && r->vertices == vector<int>({0, 2, 3, 4}));
    assert(!iter_search::iddfs(g, 4, 0, 5) && !iter_search::biBfs(g, 4, 0));
}
```

2.2 排序、分治与序列处理

2.2.001 快速排序、归并排序、基数排序

三种排序不是同一功能的快慢版本，接口和内存约束各异。通用比较排序 `sort` 只依赖严格弱序，能处理任意对象和自定义比较器，是默认首选；`mergeSort` 在合并两个有序段时保持相等元素原顺序，需要稳定性或想在归并过程中顺带统计（如逆序对）时用它；`radixSort` 把有符号 64 位整数翻转符号位（`^ (1ULL << 63)`）后按 16 位一段稳定分桶、四趟完成，数据是大量纯整数且内存充裕时最快。

```
sort(a.begin(), a.end());
```

```cpp
template <class T, class Cmp = less<T>> void mergeSort(vector<T> &a, Cmp cmp = {})
{
    // a 是待排序数组，cmp 是严格弱序比较器；排序结果直接写回 a。
    vector<T> b;
    b.reserve(a.size());
    auto dfs = [&](auto &&self, int l, int r) -> void
    {
        if (r - l <= 1)
        {
            return;
        }
        auto m = midpoint(l, r);
        self(self, l, m);
        self(self, m, r);
        auto i = l;
        auto j = m;
        b.clear();
        while (i < m || j < r)
        {
            if (j == r || (i < m && !cmp(a[j], a[i])))
            {
                b.push_back(move(a[i++]));
            }
            else
            {
                b.push_back(move(a[j++]));
            }
        }
        move(b.begin(), b.end(), a.begin() + l);
    };
    dfs(dfs, 0, a.size());
}

void radixSort(vector<int> &a)
{
    // a 是待排序的有符号 64 位整数数组；排序结果直接写回 a。
    static_assert(numeric_limits<int>::digits == 63);
    vector<unsigned long long> x(a.size()), b(a.size());
    for (int i = 0; i < (int)a.size(); i++)
    {
        x[i] = (unsigned long long)a[i] ^ (1ULL << 63);
    }
    constexpr int B = 1 << 16;
    vector<int> cnt(B);
    for (int sh = 0; sh < 64; sh += 16)
    {
        fill(cnt.begin(), cnt.end(), 0);
        for (auto v : x)
        {
            cnt[v >> sh & (B - 1)]++;
        }
        partial_sum(cnt.begin(), cnt.end(), cnt.begin());
```

```
        for (auto it = x.rbegin(); it != x.rend(); it++)
        {
            auto p = *it >> sh & (B - 1);
            b[--cnt[p]] = *it;
        }
        x.swap(b);
    }
    for (int i = 0; i < (int)a.size(); i++)
    {
        a[i] = bit_cast<int>(x[i] ^ (1ULL << 63));
    }
}
```

`sort(a.begin(), a.end())` 通用首选，`mergeSort(a)` 稳定备用，`radixSort(a)` 专供纯整数大数据。归并缓冲通过 `reserve` 后逐个构造，不要求 `T` 可默认构造；基数版要求 64 位二补码有符号整数，末步用 `bit_cast<int>` 恢复位模式，不依赖超范围无符号数到有符号数的实现定义转换。本机基准（MinGW g++ 14.2、`-O2`、一百万随机 64 位整数、7 轮中位数）为 `radixSort=16ms`、`sort=41ms`、`mergeSort=67ms`，基数排序最快，但比较器、自定义对象和低内存场景仍用 `sort`。`sort` 与 `mergeSort` 均 $O(n \log n)$、归并额外空间 $O(n)$；基数排序固定 4 趟、每趟 $O(n + 2^{16})$，总空间 $O(n + 2^{16})$。

传自定义比较器时，例如按降序稳定归并：

```
signed main()
{
    vector<int> a{2, 1, 3};
    auto cmp = [](int x, int y)
    {
        return x > y;
    };
    mergeSort(a, cmp);
    assert((a == vector<int>{3, 2, 1}));
}
```

2.2.002 离散化与坐标压缩

把任意可比较值映射到 $[0, V)$，保留大小关系和相等关系，是本章多处（整体二分、区间不同数等）共用的底层结构，故在此首次完整给出。做法是复制全部候选值，排序去重后用 `lower_bound` 求排名：`id(x)` 要求 `x` 已存在并返回其编号，`lower(x)` 返回第一个不小于 `x` 的位置，`value(p)` 反查原值。注意编号只能用于比较顺序、频率和树状数组下标；原值差、几何距离等运算必须回到 `value(id)`，否则会把不等距坐标误当成等距。

```
template <class T> struct Discrete
{
    vector<T> a; // a 保存升序且去重后的原值。

    Discrete() = default;

    explicit Discrete(vector<T> v) : a(move(v))
    {
```

```
        // v 收集所有可能出现的值。
        sort(a.begin(), a.end());
        a.erase(unique(a.begin(), a.end()), a.end());
    }

    int id(const T &x) const
    {
        // x 必须已存在；返回 x 从 0 开始的离散编号。
        auto it = lower_bound(a.begin(), a.end(), x);
        assert(it != a.end() && *it == x); // 调试检查，可删
        return it - a.begin();
    }

    int lower(const T &x) const
    {
        // x 是查询值；返回第一个不小于 x 的离散位置。
        return lower_bound(a.begin(), a.end(), x) - a.begin();
    }

    const T &value(int p) const
    {
        // p 是合法离散编号；返回对应原值的常量引用。
        return a[p];
    }
};
```

以 `Discrete<int> dis(a)` 构造后，`dis.id(x)`、`dis.lower(x)`、`dis.value(p)` 分别取编号、下界位置和原值。这是静态离散化：在线出现新值时会失效，须提前收集所有可能的更新值，或改用平衡树；也不要把离散编号当原值做距离运算。构造 $O(n \log n)$ 时间、$O(n)$ 空间，单次映射 $O(\log V)$。

2.2.003 逆序对及其扩展

统计 $i < j$ 且 $a_i > a_j$ 的二元组。两种同为 $O(n \log n)$ 但扩展方向不同的做法都完整保留，不合并成黑盒：归并版在合并时，每当右半元素先被取出，就一次性加上左半尚未取出的元素数 `m - i`，无需离散化且便于顺带累计其他归并贡献；BIT 版先离散化，再从右向左扫描、查询严格小于当前值的已加入数量，便于改成按值排名、动态频率或别的前缀统计。相等元素在两版中都不计入逆序对。

```
// 依赖 2.2.002 离散化与坐标压缩
template <class T> int invMerge(vector<T> a)
{
    // a 是按值复制的序列；返回严格逆序对数量。
    vector<T> b;
    b.reserve(a.size());
    int ans = 0;
    auto dfs = [&](auto &&self, int l, int r) -> void
    {
        if (r - l <= 1)
        {
            return;
```

```
        }
        auto m = midpoint(l, r);
        self(self, l, m);
        self(self, m, r);
        auto i = l;
        auto j = m;
        b.clear();
        while (i < m || j < r)
        {
            if (j == r || (i < m && !(a[j] < a[i])))
            {
                b.push_back(move(a[i++]));
            }
            else
            {
                ans += m - i;
                b.push_back(move(a[j++]));
            }
        }
        move(b.begin(), b.end(), a.begin() + l);
    };
    dfs(dfs, 0, a.size());
    return ans;
}

template <class T> int invBit(const vector<T> &a)
{
    // a 是原序列；返回用离散化和 Fenwick 统计的严格逆序对数量。
    Discrete<T> dis(a);
    vector<int> bit(dis.a.size() + 1);
    auto add = [&](int p)
    {
        for (p++; p < (int)bit.size(); p += p & -p)
        {
            bit[p]++;
        }
    };
    auto sum = [&](int p)
    {
        int ans = 0;
        for (; p > 0; p -= p & -p)
        {
            ans += bit[p];
        }
        return ans;
    };
    int ans = 0;
    for (auto it = a.rbegin(); it != a.rend(); it++)
    {
        auto p = dis.id(*it);
        ans += sum(p);
        add(p);
```

```
    }
    return ans;
}
```

`invMerge(a)` 与 `invBit(a)` 结果相同；返回值为 `int`，在本模板即 64 位。只做一次静态统计选归并版更直接，还要按值维护频率或排名选 BIT 版；扩展到其他偏序时必须重新确认严格/非严格比较和重复值处理。本机基准（同一组一百万随机 64 位整数、7 轮中位数）归并版 `69ms`、BIT 版 `177ms`，故静态统计默认归并版，BIT 版作值域统计扩展保留。两版均 $O(n \log n)$ 时间、$O(n)$ 额外空间。

2.2.004 整体二分

当一批询问都在同一有序答案域上二分、且一次判定能用可撤销的数据结构批量完成时，把「每个询问各自二分」改成递归地一起划分询问与元素：在值域中点 `vm` 处把离散排名 $< vm$ 的元素加入 Fenwick，每个询问看自己区间内已加入的元素数 `c` 决定落入左半还是右半答案域（落右半时把 `k` 扣掉 `c`），本层递归前务必回滚 Fenwick 修改。它要求判定关于答案单调，不支持在线强制查询。本条实例 `rangeKth` 求静态数组区间第 k 小；其中的离散化依赖 2.2.002 离散化与坐标压缩 的 `Discrete`，此处不再重复，而 `Fenwick` 树状数组在本章首次完整给出，后续 2.2.005 CDQ 分治与三维偏序、2.3.005 区间不同数统计 均按编号引用本条。

```
struct Fenwick
{
    int n = 0;      // n 是下标范围长度。
    vector<int> a; // a 保存从 1 开始的树状数组节点和。

    Fenwick(int m = 0)
    {
        // m 是初始下标范围长度。
        init(m);
    }

    void init(int m)
    {
        // m 是新的下标范围长度；调用后全部频率清零。
        assert(m >= 0); // 调试检查，可删
        n = m;
        a.assign(n + 1, 0);
    }

    void add(int p, int v)
    {
        // p 是从 0 开始的位置，v 是增量。
        assert(0 <= p && p < n); // 调试检查，可删
        for (p++; p <= n; p += p & -p)
        {
            a[p] += v;
        }
    }

    int sum(int r) const
```

```
    {
        // r 是右端点；返回半开前缀 [0,r) 的和。
        assert(0 <= r && r <= n); // 调试检查，可删
        int ans = 0;
        for (; r > 0; r -= r & -r)
        {
            ans += a[r];
        }
        return ans;
    }

    int sum(int l, int r) const
    {
        // l、r 是半开区间端点；返回 [l,r) 的和。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return sum(r) - sum(l);
    }
};

// KthQuery[0..2] 依次表示左端点、右端点、从 1 开始的排名。
using KthQuery = array<int, 3>;

vector<int> rangeKth(const vector<int> &a, vector<KthQuery> qs)
{
    // a 是静态数组，qs 中每项是 {l,r,k}；返回各半开区间第 k 小值。
    if (qs.empty())
    {
        return {};
    }
    assert(!a.empty()); // 调试检查，可删
    for (auto q : qs)
    {
        assert(0 <= q[0] && q[0] < q[1] && q[1] <= (int)a.size()); // 调试检查，可删
        assert(1 <= q[2] && q[2] <= q[1] - q[0]);                  // 调试检查，可删
    }
    Discrete<int> dis(a);
    vector<int> rk(a.size());
    for (int i = 0; i < (int)a.size(); i++)
    {
        rk[i] = dis.id(a[i]);
    }
    vector<int> es(a.size()), ids(qs.size()), ans(qs.size());
    iota(es.begin(), es.end(), 0);
    iota(ids.begin(), ids.end(), 0);
    Fenwick bit(a.size());
    auto solve = [&](auto &&self, int vl, int vr, vector<int> ps, vector<int> qi) -> void
    {
        if (qi.empty())
        {
            return;
        }
        if (vr - vl == 1)
```

```
        {
            for (auto id : qi)
            {
                ans[id] = dis.value(vl);
            }
            return;
        }
        auto vm = midpoint(vl, vr);
        vector<int> pl, pr, ql, qr;
        for (auto p : ps)
        {
            if (rk[p] < vm)
            {
                bit.add(p, 1);
                pl.push_back(p);
            }
            else
            {
                pr.push_back(p);
            }
        }
        for (auto id : qi)
        {
            auto &q = qs[id];
            auto c = bit.sum(q[0], q[1]);
            if (q[2] <= c)
            {
                ql.push_back(id);
            }
            else
            {
                q[2] -= c;
                qr.push_back(id);
            }
        }
        for (auto p : pl)
        {
            bit.add(p, -1);
        }
        self(self, vl, vm, move(pl), move(ql));
        self(self, vm, vr, move(pr), move(qr));
    };
    solve(solve, 0, dis.a.size(), move(es), move(ids));
    return ans;
}
```

`KthQuery = {l, r, k}`（半开区间加从 1 开始的排名，须 $1 \le k \le r-l$），`rangeKth(a, qs)` 返回各区间第 k 小值；空询问集直接返回空数组，非空询问要求原数组非空。函数内部只改询问副本里的 `k`、不动调用者数据；上面的代码需与本章前导及 2.2.002 的 `Discrete` 拼接编译。整体二分须满足答案单调、修改可按答案域分组、判定状态可回滚，若只是一次静态 `nth_element` 就别套它。答案域离散后大小为 V 时，每层处理全部元素与询问并做 Fenwick 操作，时间 $O((n+q)\log V \log n)$、空间 $O(n+q+V)$。

固定数组同时覆盖半开子区间、重复值和排名两端：

```cpp
signed main()
{
    vector<int> a{5, 1, 4, 1, 3};
    vector<KthQuery> qs{{0, 5, 1}, {0, 5, 3}, {0, 5, 5},
                        {1, 4, 1}, {1, 4, 2}, {1, 4, 3}};
    auto ans = rangeKth(a, qs);
    assert((ans == vector<int>{1, 3, 5, 1, 1, 4}));
    cout << ans[0] << ' ' << ans[1] << ' ' << ans[2] << ' '
         << ans[3] << ' ' << ans[4] << ' ' << ans[5] << '\n'; // 1 3 5 1 1 4
    return 0;
}
```

2.2.005 CDQ 分治与三维偏序

统计每个三维点被多少点在三维上同时不大于。先按 `x`，`y`，`z` 排序，第一维借排序天然处理，再对完全相同的点合组（否则递归顺序会让相同点得到不同答案）；CDQ 分治把第二维当作「时间维」，只统计左半对右半的贡献，合并时按 `y` 归并、用 Fenwick 查询 `z` 前缀。所用 `Fenwick` 与 2.2.004 整体二分 完全相同，此处依赖该条的 `Fenwick`，不再重复粘贴。

```cpp
// Point3[0..3] 依次是 x、y、z 坐标与原输入编号。
using Point3 = array<int, 4>;

// answer[i] 是三维坐标均不大于第 i 个点的点数。
inline vector<int> dom3(vector<Point3> points)
{
    // points 的每项为 {x,y,z,id}；返回每个原编号的三维非严格支配计数。
    int n = points.size();
    vector<int> coord;
    for (auto point : points)
    {
        coord.push_back(point[2]);
    }
    ranges::sort(coord);
    coord.erase(unique(coord.begin(), coord.end()), coord.end());
    ranges::sort(points,
                 [](const Point3 &a, const Point3 &b)
                 {
                     return tie(a[0], a[1], a[2]) < tie(b[0], b[1], b[2]);
                 });
    struct Node
    {
        // x、y 是原坐标，z 是离散编号，weight 是该坐标的重复点数。
        int x, y, z, weight;
        // ids 保存该组重复点的原编号。
        vector<int> ids;
        // answer 保存当前累计的非严格支配计数。
        long long answer;
    };
```

```
    vector<Node> nodes;
    for (auto point : points)
    {
        int z = lower_bound(coord.begin(), coord.end(), point[2]) - coord.begin();
        if (!nodes.empty() && tie(nodes.back().x, nodes.back().y, nodes.back().z) ==
        ↪ tie(point[0], point[1], z))
        {
            ++nodes.back().weight;
            ++nodes.back().answer;
            nodes.back().ids.push_back(point[3]);
        }
        else
        {
            nodes.push_back({point[0], point[1], z, 1, {point[3]}, 1});
        }
    }
    vector<int> bit(coord.size() + 1), order(nodes.size()), buffer(nodes.size()), answer(n);
    iota(order.begin(), order.end(), 0);
    auto add = [&](int x, int value)
    {
        for (++x; x < bit.size(); x += x & -x)
        {
            bit[x] += value;
        }
    };
    auto sum = [&](int x)
    {
        int result = 0;
        for (++x; x > 0; x -= x & -x)
        {
            result += bit[x];
        }
        return result;
    };
    function<void(int, int)> cdq = [&](int left, int right)
    {
        if (right - left <= 1)
        {
            return;
        }
        int middle = midpoint(left, right);
        cdq(left, middle);
        cdq(middle, right);
        int i = left, j = middle, output = left;
        while (j < right)
        {
            while (i < middle && nodes[order[i]].y <= nodes[order[j]].y)
            {
                add(nodes[order[i]].z, nodes[order[i]].weight);
                buffer[output++] = order[i++];
            }
            nodes[order[j]].answer += sum(nodes[order[j]].z);
```

```
            buffer[output++] = order[j++];
        }
        auto used = i;
        while (i < middle)
        {
            buffer[output++] = order[i++];
        }
        for (int k = left; k < used; ++k)
        {
            add(nodes[order[k]].z, -nodes[order[k]].weight);
        }
        copy(buffer.begin() + left, buffer.begin() + right, order.begin() + left);
    };
    cdq(0, nodes.size());
    for (auto &node : nodes)
    {
        for (int id : node.ids)
        {
            answer[id] = node.answer;
        }
    }
    return answer;
}

struct Point4
{
    // x、y、z、w 是四维坐标。
    int x, y, z, w;
};

struct Dom4Event
{
    // y、z、w 是后三维坐标；src 表示该点来自较小 x 的左半部。
    int y, z, w;
    bool src;
};

inline long long dom4Cross(vector<Dom4Event> a, int m)
{
    // a 是一次跨段统计的事件，m 是离散后第四维大小；返回后三维均严格递增的跨段点对数。
    sort(a.begin(), a.end(), [](const auto &u, const auto &v)
    {
        return tie(u.y, u.src) < tie(v.y, v.src);
    });
    vector<Dom4Event> buf(a.size());
    vector<int> bit(m + 1);
    auto add = [&](int p, int v)
    {
        for (++p; p <= m; p += p & -p)
        {
            bit[p] += v;
        }
```

```
    };
    auto sum = [&](int p)
    {
        int ans = 0;
        for (; p; p -= p & -p)
        {
            ans += bit[p];
        }
        return ans;
    };
    long long ans = 0;
    auto cdq = [&](auto &&self, int l, int r) -> void
    {
        if (r - l <= 1) return;
        int mid = (l + r) / 2;
        self(self, l, mid);
        self(self, mid, r);
        int i = l;
        for (int j = mid; j < r; j++)
        {
            while (i < mid && a[i].z < a[j].z)
            {
                if (a[i].src) add(a[i].w, 1);
                i++;
            }
            if (!a[j].src) ans += sum(a[j].w);
        }
        for (int k = l; k < i; k++)
        {
            if (a[k].src) add(a[k].w, -1);
        }
        merge(a.begin() + l, a.begin() + mid, a.begin() + mid, a.begin() + r,
              buf.begin() + l, [](const auto &u, const auto &v)
        {
            return u.z < v.z;
        });
        copy(buf.begin() + l, buf.begin() + r, a.begin() + l);
    };
    cdq(cdq, 0, a.size());
    return ans;
}

inline long long countFourDimensional(vector<Point4> a)
{
    // a 是四维点集；返回两点四维坐标均严格递增的有序点对数量。
    vector<int> ws;
    for (auto p : a)
    {
        ws.push_back(p.w);
    }
    sort(ws.begin(), ws.end());
    ws.erase(unique(ws.begin(), ws.end()), ws.end());
```

```
    for (auto &p : a)
    {
        p.w = lower_bound(ws.begin(), ws.end(), p.w) - ws.begin();
    }
    sort(a.begin(), a.end(), [](const auto &u, const auto &v)
    {
        return tie(u.x, u.y, u.z, u.w) < tie(v.x, v.y, v.z, v.w);
    });
    long long ans = 0;
    auto solve = [&](auto &&self, int l, int r) -> void
    {
        if (r - l <= 1) return;
        int mid = (l + r) / 2;
        while (mid < r && a[mid - 1].x == a[mid].x)
        {
            mid++;
        }
        if (mid == r)
        {
            mid = (l + r) / 2;
            while (mid > l && a[mid - 1].x == a[mid].x)
            {
                mid--;
            }
            if (mid == l) return;
        }
        self(self, l, mid);
        self(self, mid, r);
        vector<Dom4Event> e;
        e.reserve(r - l);
        for (int i = l; i < mid; i++)
        {
            e.push_back({a[i].y, a[i].z, a[i].w, true});
        }
        for (int i = mid; i < r; i++)
        {
            e.push_back({a[i].y, a[i].z, a[i].w, false});
        }
        ans += dom4Cross(move(e), ws.size());
    };
    solve(solve, 0, a.size());
    return ans;
}
```

`Point3 = {x, y, z, id}`，`id` 须覆盖 $[0,n)$ 且唯一，`dom3(p)` 返回每个原下标的支配计数；例如 `{{1,1,1,0},{1,1,1,1},{2,0,2,2},{2,2,2,3}}` 得 `{2,2,1,4}`。计数含点自身和完全相同的其他点，严格偏序需调整排序、分组和查询边界。排序与 CDQ 共 $O(n\log^2 n)$ 时间，Fenwick、归并缓冲和节点数组共 $O(n)$ 空间。同一条目的 `countFourDimensional` 统计四维坐标均严格递增的有序点对；外层按第一维分治，内层 CDQ 处理后三维，复杂度 $O(n\log^3 n)$，坐标任一维相等均不计入。

固定例覆盖重复点、不可比点、被全部点支配的点和全同点（`Fenwick` 取自 2.2.004）：

```cpp
signed main()
{
    vector<Point3> p{{1, 1, 1, 0}, {1, 1, 1, 1},
                     {2, 0, 2, 2}, {2, 2, 2, 3}};
    auto ans = dom3(p);
    assert((ans == vector<int>{2, 2, 1, 4}));
    assert((dom3({{7, 7, 7, 0}, {7, 7, 7, 1}, {7, 7, 7, 2}}) ==
            vector<int>{3, 3, 3}));
    cout << ans[0] << ' ' << ans[1] << ' ' << ans[2] << ' ' << ans[3] << '\n'; // 期望输出：2 2
    ↪ 1 4
    return 0;
}
```

2.2.006 分数级联（Fractional Cascading）

给定 k 个静态有序表，需要很多次把同一个值 x 分别送进每张表求 `lower_bound`。逐表二分单次是 $O(\sum_i \log n_i)$；分数级联预处理后只对第一张增强表二分，随后逐层沿「桥」转移，把单次查询降到 $O(\log N + k)$，其中 $N = \sum_i n_i$。构造从最后一层向前：增强表 B_i 把原表 A_i 与下一层 B_{i+1} 的约一半元素（隔一个采样一个）合并，每个增强元素记录它在 A_i 中的下界 `own` 与在 B_{i+1} 中的下界 `down`。因为下层每隔一个就被采样，沿 `down` 落到的位置与真实下界至多差一步（查询里的 `p--` 修正这一步），所以后续每层只需常数次比较；采样保证所有增强表总长仍为 $O(N)$。典型用于线段树或树链分解后同时查多份有序目录，以及计算几何中沿相邻层级定位同一坐标。

```cpp
template <class T> struct FracCascade
{
    struct Item
    {
        T x;           // x 是增强表中的键值。
        int own, down; // own 指向本层原表下界，down 指向下一层增强表下界。
    };

    vector<vector<T>> a;    // a 保存各层原始有序表。
    vector<vector<Item>> b; // b 保存采样合并后的增强表和桥指针。

    explicit FracCascade(vector<vector<T>> src) : a(move(src)), b(a.size())
    {
        // src 是若干个按同一规则升序排列的静态表。
        for (auto &x : a)
        {
            assert(is_sorted(x.begin(), x.end())); // 调试检查，可删
        }
        for (int i = (int)a.size() - 1; i >= 0; i--)
        {
            auto v = a[i];
            if (i + 1 < (int)b.size())
            {
                for (int j = 1; j < (int)b[i + 1].size(); j += 2)
                {
                    v.push_back(b[i + 1][j].x);
```

```
                }
            }
            sort(v.begin(), v.end());
            v.erase(unique(v.begin(), v.end()), v.end());
            for (auto x : v)
            {
                auto p = lower_bound(a[i].begin(), a[i].end(), x) - a[i].begin();
                int q = 0;
                if (i + 1 < (int)b.size())
                {
                    q = lower_bound(b[i + 1].begin(),
                                    b[i + 1].end(),
                                    x,
                                    [](const Item &u, const T &rhs)
                                    {
                                        return u.x < rhs;
                                    }) -
                        b[i + 1].begin();
                }
                b[i].push_back({x, p, q});
            }
        }
    }

    vector<int> lowerBound(const T &x) const
    {
        // x 是所有表共用的查询值；返回每个原表中的 lower_bound 位置。
        vector<int> ans(a.size());
        int p = 0;
        if (!b.empty())
        {
            p = lower_bound(b[0].begin(),
                            b[0].end(),
                            x,
                            [](const Item &u, const T &rhs)
                            {
                                return u.x < rhs;
                            }) -
                b[0].begin();
        }
        for (int i = 0; i < (int)a.size(); i++)
        {
            ans[i] = p == (int)b[i].size() ? (int)a[i].size() : b[i][p].own;
            if (i + 1 == (int)a.size())
            {
                break;
            }
            p = p == (int)b[i].size() ? (int)b[i + 1].size() : b[i][p].down;
            if (p && b[i + 1][p - 1].x >= x)
            {
                p--;
            }
```

```
        }
        return ans;
    }
};
```

每个输入表须按同一规则升序排列，构造后不支持插入、删除或改值。`fc.lowerBound(x)` 返回长度 k 的位置数组，第 i 项是 A_i 中第一个不小于 x 的下标、无则等于 A_i 长度；例如三张表 `{{-3,1,5,9},{0,2,8},{-5,4,4,10}}` 查 4 得 `{2,2,1}`。只有一次查询或表数很少时预处理不划算，有动态修改时改用平衡树、线段树套有序结构或重建分块。当前实现逐层排序建桥，构造 $O(N \log N)$ 时间、$O(N)$ 空间，单次查询 $O(\log N + k)$；改用线性归并建每层可把预处理降到 $O(N)$。

固定查询覆盖小于全部元素、重复值、空表、命中元素和大于全部元素：

```
signed main()
{
    vector<vector<int>> a{{-3, 1, 5, 9}, {}, {0, 2, 8}, {-5, 4, 4, 10}};
    FracCascade<int> fc(a);
    assert((fc.lowerBound(-10) == vector<int>{0, 0, 0, 0}));
    assert((fc.lowerBound(4) == vector<int>{2, 0, 2, 1}));
    assert((fc.lowerBound(5) == vector<int>{2, 0, 2, 3}));
    assert((fc.lowerBound(11) == vector<int>{4, 0, 3, 4}));
    auto ans = fc.lowerBound(4);
    cout << ans[0] << ' ' << ans[1] << ' ' << ans[2] << ' ' << ans[3] << '\n'; // 期望输出：2 0
    ↪ 2 1
    return 0;
}
```

2.2.007 扫描线

矩形面积并是扫描线最常用、也最便于完整测试的实例：把每个矩形拆成左右两条竖边，按 x 排序成进入（+1）/离开（−1）事件，线段树维护当前 y 轴被覆盖的总长度。相邻两事件横坐标之间活跃矩形集合不变，故面积增量为「当前 y 覆盖长度 × 横向宽度」。线段树每个节点记覆盖次数 `cov` 和实际覆盖长度 `len`：`cov` 非零时整段有效，为零时才向下合并子节点的 `len`——正因如此这棵覆盖线段树不需要 lazy 下传。相同 x 的事件必须成组处理，避免出现零宽度的中间状态。

```
// Rect[0..3] 依次表示左横坐标、下纵坐标、右横坐标、上纵坐标。
using Rect = array<int, 4>;

int rectArea(const vector<Rect> &rs)
{
    // rs 中每项是 {左、下、右、上}；返回所有半开矩形的面积并。
    // Event[0..3] 依次表示横坐标、下纵坐标、上纵坐标、覆盖次数变化量。
    using Event = array<int, 4>;
    vector<Event> es;
    vector<int> ys;
    for (auto [xl, yl, xr, yr] : rs)
    {
        if (xl >= xr || yl >= yr)
        {
```

```
            continue;
        }
        es.push_back({xl, yl, yr, 1});
        es.push_back({xr, yl, yr, -1});
        ys.push_back(yl);
        ys.push_back(yr);
    }
    if (es.empty())
    {
        return 0;
    }
    sort(ys.begin(), ys.end());
    ys.erase(unique(ys.begin(), ys.end()), ys.end());
    sort(es.begin(),
         es.end(),
         [](const auto &a, const auto &b)
         {
             return a[0] < b[0];
         });
    vector<int> cov(ys.size() * 4);
    vector<int> len(ys.size() * 4);
    auto add = [&](auto &&self, int p, int l, int r, int ql, int qr, int d) -> void
    {
        if (ql <= l && r <= qr)
        {
            cov[p] += d;
        }
        else
        {
            auto m = midpoint(l, r);
            if (ql < m)
            {
                self(self, p * 2, l, m, ql, qr, d);
            }
            if (m < qr)
            {
                self(self, p * 2 + 1, m, r, ql, qr, d);
            }
        }
        if (cov[p])
        {
            len[p] = ys[r] - ys[l];
        }
        else if (r - l == 1)
        {
            len[p] = 0;
        }
        else
        {
            len[p] = len[p * 2] + len[p * 2 + 1];
        }
    };
```

```
    int ans = 0;
    auto pre = es[0][0];
    for (int i = 0; i < (int)es.size();)
    {
        auto width = es[i][0] - pre;
        ans += len[1] * width;
        auto j = i;
        while (j < (int)es.size() && es[j][0] == es[i][0])
        {
            auto [x, yl, yr, d] = es[j];
            auto l = lower_bound(ys.begin(), ys.end(), yl) - ys.begin();
            auto r = lower_bound(ys.begin(), ys.end(), yr) - ys.begin();
            add(add, 1, 0, ys.size() - 1, l, r, d);
            j++;
        }
        pre = es[i][0];
        i = j;
    }
    return ans;
}
```

`Rect = {xl, yl, xr, yr}`，区域为 $[x_l, x_r) \times [y_l, y_r)$，退化矩形自动忽略；`rectArea(rs)` 返回面积并，例如 `{{0,0,2,2},{1,1,3,2}}` 得 5。调用者须保证坐标差、扫描线覆盖长度及最终面积都能由 `int` 表示；确实需要更宽面积时把这份短代码中的坐标和面积类型一起改宽。$2n$ 个事件、$2n$ 个纵坐标，排序与更新共 $O(n \log n)$ 时间、$O(n)$ 空间。

2.2.008 单调栈、单调队列

共同直觉是：新元素到来时，所有既不优于它、又比它更早的候选都可永久删除，因为新元素值更优且失效更晚。`prevLess` 用栈求每个位置左侧最近严格更小元素；`windowMin` / `windowMax` 用双端队列维护固定窗口的最值，比栈多一步从队首删除滑出窗口（`q.front() <= i - k`）的下标。比较符号决定「严格」还是「允许相等」——这里弹出条件用 `>=`（最小值）/ `<=`（最大值），留下的是严格意义的最近更小/更大，照抄前务必按题意核对。

```
template <class T> vector<int> prevLess(const vector<T> &a)
{
    // a 是原序列；返回每个位置左侧最近严格更小元素的下标，不存在时为 -1。
    // 改求最近严格更大时，把弹栈条件 >= 改为 <=；若允许相等，则分别用 > 或 <。
    vector<int> stk, ans(a.size(), -1);
    for (int i = 0; i < (int)a.size(); i++)
    {
        while (!stk.empty() && a[stk.back()] >= a[i])
        {
            stk.pop_back();
        }
        if (!stk.empty())
        {
            ans[i] = stk.back();
        }
```

```
        stk.push_back(i);
    }
    return ans;
}

template <class T> vector<T> windowMin(const vector<T> &a, int k)
{
    // a 是原序列，k 是固定窗口长度；返回每个连续窗口的最小值。
    // 改最大值时把维护队尾的 >= 改为 <=；过期下标的队首删除条件不变。
    assert(1 <= k && k <= (int)a.size()); // 调试检查，可删
    deque<int> q;
    vector<T> ans;
    for (int i = 0; i < (int)a.size(); i++)
    {
        while (!q.empty() && q.front() <= i - k)
        {
            q.pop_front();
        }
        while (!q.empty() && a[q.back()] >= a[i])
        {
            q.pop_back();
        }
        q.push_back(i);
        if (i + 1 >= k)
        {
            ans.push_back(a[q.front()]);
        }
    }
    return ans;
}

template <class T> vector<T> windowMax(const vector<T> &a, int k)
{
    // a 是原序列，k 是固定窗口长度；返回每个连续窗口的最大值。
    // 改最小值时把维护队尾的 <= 改为 >=；过期下标的队首删除条件不变。
    assert(1 <= k && k <= (int)a.size()); // 调试检查，可删
    deque<int> q;
    vector<T> ans;
    for (int i = 0; i < (int)a.size(); i++)
    {
        while (!q.empty() && q.front() <= i - k)
        {
            q.pop_front();
        }
        while (!q.empty() && a[q.back()] <= a[i])
        {
            q.pop_back();
        }
        q.push_back(i);
        if (i + 1 >= k)
        {
            ans.push_back(a[q.front()]);
```

```
        }
    }
    return ans;
}
```

`prevLess(a)` 返回各位置左侧最近严格更小元素的下标（无则 -1），`windowMin(a, k)` / `windowMax(a, k)` 返回每个连续窗口的最值。模板弹出满足 $a_{stk.back()} \geq a_i$ 的元素，故留下的是严格更小；要「更小或相等」就改成只在严格大于时弹出。窗口接口要求 $1 \leq k \leq n$。每个下标至多进出一次，时间 $O(n)$、空间 $O(n)$。

2.2.009 滑动窗口与区间最值

连续定长窗口的最值直接复用 2.2.008 单调栈、单调队列 的 `windowMin` / `windowMax`：队列保存值单调且下标仍在窗口内的候选，队首恒为当前答案，固定窗口下每个下标只进出一次，$O(n)$ 时间；除返回数组外额外空间为 $O(k)$，代码不再重复。真正的分界在于查询是否按右端点顺序滑动——一旦不是（窗口长度可变或任意区间查询），单调队列的不变量就失效，应换成 `multiset`（$O(n \log k)$）、对顶双堆、ST 表或线段树。切忌用 `priority_queue` 后再线性删除过期元素；若用堆，必须给元素带上下标并在堆顶循环清理过期项。

2.2.010 最长上升子序列及输出方案

求严格上升子序列并恢复一组原数组下标。核心不变量是 `val[len]` 保存长度为 $len+1$ 的上升子序列所能取得的最小结尾——结尾越小越有利于将来扩展，因此用 `lower_bound` 找到第一个不小于 `a[i]` 的位置替换即可，不损失最优性。`pos[len]` 记录该最小结尾来自哪个原下标，`pre[i]` 记录位置 i 在方案中的前驱，最后从最长长度的结尾沿 `pre` 反向恢复一条合法方案。

```
vector<int> lis(const vector<int> &a)
{
    // a 是原序列；返回一组严格上升子序列的原数组下标。
    vector<int> val, pos, pre(a.size(), -1);
    for (int i = 0; i < (int)a.size(); i++)
    {
        auto p = lower_bound(val.begin(), val.end(), a[i]) - val.begin();
        if (p == (int)val.size())
        {
            val.push_back(a[i]);
            pos.push_back(i);
        }
        else
        {
            val[p] = a[i];
            pos[p] = i;
        }
        if (p)
        {
            pre[i] = pos[p - 1];
        }
```

```
    }
    vector<int> ans;
    for (auto x = pos.empty() ? -1 : pos.back(); x >= 0; x = pre[x])
    {
        ans.push_back(x);
    }
    reverse(ans.begin(), ans.end());
    return ans;
}
```

`lis(a)` 返回一条最长严格上升子序列的原数组下标（不是值），需要打印子序列元素时用 `a[p]`。严格上升用 `lower_bound`，改求最长非降子序列则换成 `upper_bound`。时间 $O(n \log n)$，尾值、结尾下标和前驱数组共 $O(n)$ 空间。

2.2.011 二维 LIS / LCIS

本条是两个只有名字相近、不能互相替代的问题。二维 LIS 在点集里找两维都严格递增的最长链（信封嵌套、二维偏序、宽高同增等）：先按第一维升序、同第一维时按第二维降序排序，再对第二维做一维严格 LIS。同第一维降序是保证严格性的关键——它让同组元素在第二维上不构成上升关系，一维化后就不会误选两个第一维相等的点；若某维允许相等，排序并列规则与 `lower_bound` / `upper_bound` 须一并调整。LCIS 在两个序列中找既是公共子序列、值又严格递增的最长序列，无法用「先 LCS 再 LIS」拼出：令 f_j 为已扫描 a 的前缀中以 b_j 结尾的 LCIS 长度，处理 $x = a_i$ 时从左到右维护所有满足 $b_j < x$ 的 f_j 最大值 `best`，遇到 $b_j = x$ 就令 $f_j \leftarrow \max(f_j, best + 1)$；因为 `best` 只从严格小于 x 的状态更新，同一轮不会重复用到 x 自身。

```
int lis2D(vector<pair<int, int>> a)
{
    // a 保存二维点并按值复制；返回两维都严格增加的最长链长度。
    sort(a.begin(),
         a.end(),
         [](const auto &x, const auto &y)
         {
             if (x.first != y.first)
             {
                 return x.first < y.first;
             }
             return x.second > y.second;
         });
    vector<int> d;
    for (auto [x, y] : a)
    {
        auto it = lower_bound(d.begin(), d.end(), y);
        if (it == d.end())
        {
            d.push_back(y);
        }
        else
        {
            *it = y;
```

```
        }
    }
    return d.size();
}

int lcis(const vector<int> &a, const vector<int> &b)
{
    // a、b 是两个原序列；返回最长公共严格上升子序列长度。
    vector<int> f(b.size());
    for (auto x : a)
    {
        int best = 0;
        for (int j = 0; j < (int)b.size(); j++)
        {
            if (b[j] < x)
            {
                chmax(best, f[j]);
            }
            else if (b[j] == x)
            {
                chmax(f[j], best + 1);
            }
        }
    }
    return f.empty() ? 0 : *max_element(f.begin(), f.end());
}
```

`lis2D(points)` 接收二元组副本、只返回链长而不恢复点链，重复点最多选一个；`lcis(a, b)` 只返回长度，要恢复方案得为每个 f_j 额外记录前驱位置和来自 a 的层次。二维 LIS 为 $O(n \log n)$ 时间、$O(n)$ 空间；两序列长 n, m 时 LCIS 为 $O(nm)$ 时间、$O(m)$ 空间。

2.2.012 最大全 1 子矩形

把 01 矩阵逐行压成直方图：以当前行为底的全 1 矩形，其每列可用高度就是该列往上连续 1 的数量，于是问题化为「直方图中最大矩形」并对每行调用一次。直方图里每根柱子的最大贡献由它左右第一个更矮的柱子界定，单调栈在柱子被弹出时一次性确定其右边界，末尾补一个高度 0 的哨兵负责清空栈中残余柱子。

```
int maxRect(const vector<int> &h)
{
    // h 是非负直方图柱高；返回由 Acc 可表示的最大矩形面积。
    vector<int> stk;
    int ans = 0;
    for (int i = 0; i <= (int)h.size(); i++)
    {
        auto x = i == (int)h.size() ? 0 : h[i];
        while (!stk.empty() && h[stk.back()] >= x)
        {
            auto p = stk.back();
```

```
            stk.pop_back();
            auto l = stk.empty() ? 0 : stk.back() + 1;
            chmax(ans, h[p] * (i - l));
        }
        stk.push_back(i);
    }
    return ans;
}

int maxRect(const vector<vector<int>> &a)
{
    // a 是规则矩阵，非零元素视为 1；返回由 Acc 可表示的最大全 1 子矩形面积。
    if (a.empty() || a[0].empty())
    {
        return 0;
    }
    vector<int> h(a[0].size());
    int ans = 0;
    for (auto &row : a)
    {
        assert(row.size() == h.size()); // 调试检查，可删
        for (int j = 0; j < (int)h.size(); j++)
        {
            h[j] = row[j] ? h[j] + 1 : 0;
        }
        chmax(ans, maxRect(h));
    }
    return ans;
}
```

`maxRect(grid)` 接收规则矩阵、非零元素视为 1，返回最大全 1 子矩形面积；重载 `maxRect(h)` 直接对非负柱高的直方图求最大矩形。调用者须保证柱高、宽度乘积和最终面积能由 `int` 表示。若还要矩形的坐标，须在弹栈时一并记录左右边界和当前行。$n \times m$ 矩阵总时间 $O(nm)$、额外空间 $O(m)$；直方图长度为 m 时为 $O(m)$ 时间和 $O(m)$ 额外空间。

2.2.013 约瑟夫问题

n 人编号 $0 \dots n-1$、每次数到第 k 人出局，求最后幸存者。把规模 i 的答案 $f(i)$ 由 $f(i-1)$ 反推：删掉第一个出局者后余下 $i-1$ 人重新编号，相当于整体偏移 k，于是 $f(i) = (f(i-1)+k) \bmod i$，$f(1) = 0$。只有一个状态和一段短循环，直接内联进解题代码即可，无需封装。

```
// n 是人数，k 是报数步长，ans 是从 0 开始的幸存者编号。
assert(n > 0 && k > 0); // 调试检查，可删
int ans = 0;
if (n >= 2)
{
    int i = 2;
    while (true)
    {
```

```
        int x = k % i;
        ans = ans >= i - x ? ans - (i - x) : ans + x;
        if (i == n)
        {
            break;
        }
        i++;
    }
}
```

循环结束后的 `ans` 是从 0 开始的幸存者编号，题目若从 1 编号则答案加 1。每轮先取 `k % i`，再用比较和减法实现模加，避免 `ans + k % i` 溢出；循环在处理 `i == n` 后先退出再递增，因此 `n`、`k` 取完整正 64 位范围时不会在取模前或末次自增时溢出。该递推只求最后一人，不输出完整删除顺序。$O(n)$ 时间、$O(1)$ 空间。

2.2.014 快速选择 / 线性期望第 k 元素

只要第 k 小而不需完整排序时，划分后只递归含第 k 位的那一侧、丢弃另一侧，期望线性。`nthValue` 直接包了标准库 `nth_element`，经过充分优化，是默认实现；`quickSelect` 手写随机枢轴 + 三路划分，把等于枢轴的元素聚在中间，对高重复数据更稳，供不能用 STL 或需定制状态时备用。

```
template <class T> T nthValue(vector<T> a, int k)
{
    // a 是按值复制的数组，k 是从 0 开始的排名；返回第 k 小值。
    assert(0 <= k && k < (int)a.size()); // 调试检查，可删
    nth_element(a.begin(), a.begin() + k, a.end());
    return a[k];
}

template <class T, class Rng> T quickSelect(vector<T> a, int k, Rng &rng)
{
    // a 是按值复制的数组，k 是从 0 开始的排名，rng 是随机源；返回第 k 小值。
    assert(0 <= k && k < (int)a.size()); // 调试检查，可删
    int l = 0, r = a.size();
    while (r - l > 1)
    {
        auto p = a[uniform_int_distribution<int>(l, r - 1)(rng)];
        auto m1 = partition(a.begin() + l,
                            a.begin() + r,
                            [&](const T &x)
                            {
                                return x < p;
                            });
        auto m2 = partition(m1,
                            a.begin() + r,
                            [&](const T &x)
                            {
                                return !(p < x);
                            });
```

```
        auto x = m1 - a.begin();
        auto y = m2 - a.begin();
        if (k < x)
        {
            r = x;
        }
        else if (k >= y)
        {
            l = y;
        }
        else
        {
            return p;
        }
    }
    return a[l];
}
```

两者的 `k` 都从 0 开始：`nthValue(a, k)` 是主版本，`quickSelect(a, k, rng)` 是随机备用。本机基准（100 组、每组十万元素、7 轮中位数）`nth_element=50ms`、手写三路 `61ms`，故默认 `nth_element`。标准库对 `nth_element` 保证线性期望比较次数，手写版期望 $O(n)$、最坏 $O(n^2)$，两者都按值复制输入需 $O(n)$ 空间。

2.2.015 折半搜索

折半搜索把 n 个元素拆成前后两半，各自枚举 $2^{\lceil n/2 \rceil}$ 个子集和，再对其中一半排序，用另一半逐个二分配对，从而把 $O(2^n)$ 的子集枚举降到 $O(2^{n/2}n)$。适用于 $n \le 40$ 左右、值域太大不能做背包的子集和问题；元素可以为负。

`bound` 求不超过 `target` 的最大子集和（`ceil=true` 时求不小于 `target` 的最小子集和）：对前半的每个和 x，在排好序的后半中二分找最接近 $target - x$ 的一侧。`closest` 取两者中离 `target` 更近的一个。`fourSum` 是同一思想的四数组版本：把 $a_0 + a_1$ 的 N^2 个和排序，再对 $a_2 + a_3$ 的每个和二分查找差值。

```
namespace mitm
{
struct State
{
    __int128 sum;            // sum 是该子集的元素和。
    unsigned long long mask; // mask 的第 i 位表示半边内第 i 个元素是否被选。
};

vector<State> sums(const vector<int> &a, int l, int r)
{
    // a 是原数组，[l,r) 是一半的下标范围；返回这一半全部 2^(r-l) 个子集的和与选择掩码。
    vector<State> v{{0, 0}};
    for (int i = l; i < r; i++)
    {
        int n = v.size();
        for (int j = 0; j < n; j++)
```

```
        {
            v.push_back({v[j].sum + a[i], v[j].mask | (1ULL << (i - l))});
        }
    }
    return v;
}

struct Result
{
    __int128 sum;         // sum 是所选子集的和。
    vector<int> indices; // indices 是所选元素的原下标，升序。
};

optional<Result> bound(const vector<int> &a, __int128 target, bool ceil = false)
{
    // a 至多 46 个元素且可为负；ceil=false 求不超过 target 的最大子集和，
    // ceil=true 求不小于 target 的最小子集和；返回和与下标，不存在时返回空。
    assert(a.size() <= 46); // 调试检查，可删。
    int k = a.size() / 2;
    auto x = sums(a, 0, k), y = sums(a, k, a.size());
    sort(y.begin(), y.end(), [](const State &u, const State &v) { return u.sum < v.sum; });
    optional<Result> ans;
    unsigned long long mx = 0, my = 0;
    for (auto u : x)
    {
        // 在后半找最接近 target-u.sum 的和：it 是第一个不小于它的位置。
        auto it = lower_bound(y.begin(), y.end(), target - u.sum,
                              [](const State &v, __int128 s) { return v.sum < s; });
        if (!ceil)
        {
            if (it == y.end() || it->sum ≠ target - u.sum)
            {
                if (it == y.begin())
                {
                    continue;
                }
                --it;
            }
        }
        else if (it == y.end())
        {
            continue;
        }
        __int128 s = u.sum + it->sum;
        if (!ans || (ceil ? s < ans->sum : s > ans->sum))
        {
            ans = Result{s, {}};
            mx = u.mask;
            my = it->mask;
        }
    }
    if (ans)
```

```
    {
        for (int i = 0; i < (int)a.size(); i++)
        {
            if (i < k ? (mx >> i & 1) : (my >> (i - k) & 1))
            {
                ans->indices.push_back(i);
            }
        }
    }
    return ans;
}

Result closest(const vector<int> &a, __int128 target)
{
    // a 至多 46 个元素；返回与 target 绝对差最小的子集和及下标，两侧等距时取较小的和。
    auto lo = bound(a, target), hi = bound(a, target, true);
    if (!lo)
    {
        return *hi;
    }
    if (!hi || target - lo->sum <= hi->sum - target)
    {
        return *lo;
    }
    return *hi;
}

optional<array<int, 4>> fourSum(const array<vector<int>, 4> &a, int target)
{
    // a 是四个独立数组；返回 {i,j,k,l} 使 a[0][i]+a[1][j]+a[2][k]+a[3][l]=target，不存在时返回空。
    // v 的三个分量依次是 a[0][i]+a[1][j]、i、j。
    vector<tuple<__int128, int, int>> v;
    for (int i = 0; i < (int)a[0].size(); i++)
    {
        for (int j = 0; j < (int)a[1].size(); j++)
        {
            v.emplace_back((__int128)a[0][i] + a[1][j], i, j);
        }
    }
    sort(v.begin(), v.end());
    for (int i = 0; i < (int)a[2].size(); i++)
    {
        for (int j = 0; j < (int)a[3].size(); j++)
        {
            __int128 need = (__int128)target - a[2][i] - a[3][j];
            auto it = lower_bound(v.begin(), v.end(), tuple<__int128, int, int>{need, -1, -1});
            if (it != v.end() && get<0>(*it) == need)
            {
                return array<int, 4>{get<1>(*it), get<2>(*it), i, j};
            }
        }
    }
```

```
    return nullopt;
}
}
```

`bound`、`closest` 要求 $n \le 46$（掩码装入 64 位），返回的 `indices` 是所选元素的原下标、升序；空集也是合法子集，所以 `closest` 总有解，`bound` 只在所有子集和都在 `target` 另一侧时返回空。时间 $O(2^{n/2}n)$；每一半保存 $2^{\lceil n/2 \rceil}$ 个 32 字节的 `State`，$n = 40$ 时两半约 64 MiB，$n = 46$ 时超过 500 MiB，要先核对内存限制。`fourSum` 的时间 $O(N^2 \log N)$、空间 $O(N^2)$，N 是单个数组长度。

只需判定某个和能否取到且值域不大时用 12.1.001 01 背包 或 12.1.010 布尔母函数 / 有界子集和可达性；计数版见 10.2.022 计数版子集和生成函数。下面核对固定数组上的三种查询：

```
signed main()
{
    vector<int> a{8, -3, 5, 12, -7, 4};
    assert(mitm::bound(a, 10)->sum == 10);        // 取 8-3+5。
    assert(mitm::bound(a, 11, true)->sum == 11);
    auto c = mitm::closest(a, 100);               // 所有正数之和 29 最接近 100。
    assert(c.sum == 29 && c.indices == vector<int>({0, 2, 3, 5}));
    auto f = mitm::fourSum({vector<int>{1, 5}, {2, 7}, {3}, {4, 9}}, 20);
    assert(f && *f == (array<int, 4>{0, 1, 0, 1})); // 取 1+7+3+9。
}
```

2.3 莫队与离线查询

2.3.001 普通莫队

当单个端点的加入/删除都能 $O(1)$ 维护、且询问允许离线重排时，把大量区间查询降到约 $O(n\sqrt{q})$ 次状态更新。按左端点分块排序，使同块内右端点大体单调移动，奇偶块内右端点反向排序进一步减少折返。调度函数本身只负责移动当前 $[l, r)$，真正的答案状态交由四个端点回调（`addL` / `addR` / `delL` / `delR`）维护，因此保持为单函数而非封装成类。

```
// MoQuery[0..2] 依次表示左端点、右端点、答案编号。
using MoQuery = array<int, 3>;

template <class Ans, class AddL, class AddR, class DelL, class DelR, class Get>
vector<Ans> mo(int n, vector<MoQuery> qs, AddL addL, AddR addR, DelL delL, DelR delR, Get get)
{
    // n 是数组长度，qs 是询问；四个端点回调维护状态，get 返回当前答案。
    auto z = max<int>(1, (int)(n / sqrtl(max<int>(1, qs.size()))));
    sort(qs.begin(),
         qs.end(),
         [&](const auto &a, const auto &b)
         {
             auto x = a[0] / z, y = b[0] / z;
             if (x != y)
             {
                 return x < y;
```

```
            }
            return x & 1 ? a[1] > b[1] : a[1] < b[1];
        });
    vector<Ans> ans(qs.size());
    int l = 0, r = 0;
    for (auto q : qs)
    {
        while (l > q[0])
        {
            addL(--l);
        }
        while (r < q[1])
        {
            addR(r++);
        }
        while (l < q[0])
        {
            delL(l++);
        }
        while (r > q[1])
        {
            delR(--r);
        }
        ans[q[2]] = get();
    }
    return ans;
}
```

用 `auto ans = mo<int>(n, qs, addL, addR, delL, delR, get)` 调用，`MoQuery={l,r,id}`。询问使用 `[l, r)`，`id` 必须覆盖 $[0, q)$。左右端操作不对称时分别实现四个回调，对称时可以对 `add/del` 传同一个函数。排序为 $O(q \log q)$；块长为 B 时端点移动上界为 $O\left(\frac{n^2}{B} + qB\right)$，代码取 $B \approx \frac{n}{\sqrt{q}}$，得到 $O(n\sqrt{q} + q \log q)$ 时间；除维护状态外，调度额外空间为 $O(q)$。

当区间端点位于较大坐标范围、又不想为块长调参时，可以按二维端点的 Hilbert 曲线排序。递归把每个 $2^k \times 2^k$ 正方形分成四块，并在相邻块之间旋转方向，使相邻询问的端点通常同时较近；这只改变访问顺序，不改变莫队状态。`moHilbert` 仍使用半开区间和单个 `add/del` 回调，空区间也允许；实现要求数组长度不超过 2^{31}，以便把二维键压进无符号 64 位。

```
unsigned long long hilbertOrder(int x, int y, int power, int rotate = 0)
{
    // x、y 是 [0,2^power) 内的坐标，power 不超过 31；返回 Hilbert 序键。
    if (!power)
    {
        return 0;
    }
    int h = 1LL << (power - 1);
    int seg = x < h ? (y < h ? 0 : 3) : (y < h ? 1 : 2);
    seg = (seg + rotate) & 3;
    static constexpr int delta[4] = {3, 0, 0, 1};
    int nextRot = (rotate + delta[seg]) & 3;
    unsigned long long z = 1ULL << (2 * power - 2);
```

```
    unsigned long long sub = hilbertOrder(x & (h - 1), y & (h - 1), power - 1, nextRot);
    return (unsigned long long)seg * z + (seg == 1 || seg == 2 ? sub : z - sub - 1);
}

template <class Ans, class Add, class Del, class Get>
vector<Ans> moHilbert(int n, vector<MoQuery> qs, Add add, Del del, Get get)
{
    // n 是数组长度，qs 是半开区间询问；add/del 接收下标，get 返回当前答案。
    int power = 0;
    while ((1LL << power) < max<int>(1, n))
    {
        power++;
    }
    assert(power <= 31);
    sort(qs.begin(), qs.end(), [&](const MoQuery &a, const MoQuery &b)
    {
        int ax = min<int>(a[0], max<int>(0, n - 1));
        int ay = min<int>(max<int>(0, a[1] - 1), max<int>(0, n - 1));
        int bx = min<int>(b[0], max<int>(0, n - 1));
        int by = min<int>(max<int>(0, b[1] - 1), max<int>(0, n - 1));
        return hilbertOrder(ax, ay, power) < hilbertOrder(bx, by, power);
    });
    vector<Ans> ans(qs.size());
    int l = 0, r = 0;
    for (auto q : qs)
    {
        while (l > q[0])
        {
            add(--l);
        }
        while (r < q[1])
        {
            add(r++);
        }
        while (l < q[0])
        {
            del(l++);
        }
        while (r > q[1])
        {
            del(--r);
        }
        ans[q[2]] = get();
    }
    return ans;
}
```

`moHilbert(n,qs,add,del,get)` 与普通 `mo` 的答案接口相同，`MoQuery={l,r,id}` 且 `id` 从 0 开始。排序为 $O(q \log q)$，端点移动次数取决于 Hilbert 曲线的局部性，通常不需要显式块长；它不适合带修改时间轴，带修改仍使用 2.3.002 带修改莫队。

下面是普通莫队、Hilbert 莫队的最小可运行调用：

```cpp
signed main()
{
    vector<int> a{1, 2, 1, 3};
    vector<MoQuery> qs{{0, 3, 0}, {1, 4, 1}};
    int cur = 0;
    auto add = [&](int p) { cur += a[p]; };
    auto del = [&](int p) { cur -= a[p]; };
    auto get = [&]() { return cur; };
    assert((mo<int>(4, qs, add, add, del, del, get) == vector<int>{4, 6}));
    cur = 0;
    assert((moHilbert<int>(4, qs, add, del, get) == vector<int>{4, 6}));
}
```

2.3.002 带修改莫队

当询问之间夹有单点修改时，给莫队再加一维时间轴 `t`，让修改既能向前应用又能向后撤销。每个询问记录它之前已发生的修改数，块长取 $n^{2/3}$，排序时同时控制左端、右端和时间指针三者的移动。应用或撤销一次修改时，如果修改位置正落在当前区间内，必须先移除旧贡献、再改值、再加入新贡献，漏掉任一步都会让状态与时间轴不一致。

```cpp
// ModifyQuery[0..3] 依次表示左端点、右端点、答案编号、修改时间。
using ModifyQuery = array<int, 4>;

template <class Ans, class Add, class Del, class Change, class Get>
vector<Ans> moModify(int n, vector<ModifyQuery> qs, Add add, Del del, Change change, Get get)
{
    // n 是数组长度，qs 含修改时间；add/del 维护端点，change(p,forward,l,r) 切换第 p 次修改，get 返回答案。
    auto z = max<int>(1, (int)powl(max<int>(1, n), 2.0L / 3));
    sort(qs.begin(),
         qs.end(),
         [&](const auto &a, const auto &b)
         {
             auto al = a[0] / z, bl = b[0] / z;
             if (al != bl)
             {
                 return al < bl;
             }
             auto ar = a[1] / z, br = b[1] / z;
             if (ar != br)
             {
                 return al & 1 ? ar > br : ar < br;
             }
             return ar & 1 ? a[3] > b[3] : a[3] < b[3];
         });
    vector<Ans> ans(qs.size());
    int l = 0, r = 0, t = 0;
    for (auto q : qs)
    {
        while (t < q[3])
```

```
        {
            change(t++, true, l, r);
        }
        while (t > q[3])
        {
            change(--t, false, l, r);
        }
        while (l > q[0])
        {
            add(--l);
        }
        while (r < q[1])
        {
            add(r++);
        }
        while (l < q[0])
        {
            del(l++);
        }
        while (r > q[1])
        {
            del(--r);
        }
        ans[q[2]] = get();
    }
    return ans;
}
```

调用 `moModify<int>(n, qs, add, del, change, get)`，`ModifyQuery={l,r,id,t}` 中 `id` 须覆盖 $[0,q)$，`t` 是该询问前的修改次数且满足 $0 \le t \le c$。`change(p, forward, l, r)` 中 `p` 是从 0 开始的修改编号；修改位置当前位于 $[l,r)$ 时，回调必须同步删除旧贡献、写入新值、加入新贡献，反向时恢复旧值。块长取 $n^{2/3}$，在 n、q 和修改数同阶时状态移动为 $O\left(n^{5/3}\right)$，排序为 $O(q\log q)$，空间为 $O(n+q+c)$（c 是修改数）。

调用示例（区间和、一次修改）：

```
signed main()
{
    vector<int> b{1, 2, 3};
    vector<ModifyQuery> mq{{0, 3, 0, 1}};
    int sum = 0;
    auto addM = [&](int p) { sum += b[p]; };
    auto delM = [&](int p) { sum -= b[p]; };
    auto change = [&](int p, bool forward, int l, int r)
    {
        int pos = 1, old = 2, now = 5;
        if (p != 0) return;
        if (l <= pos && pos < r) sum += forward ? now - old : old - now;
        b[pos] = forward ? now : old;
    };
    auto getM = [&]() { return sum; };
```

```
    assert(moModify<int>(3, mq, addM, delM, change, getM)[0] == 9);
}
```

2.3.003 回滚莫队

当状态只能廉价加入、却难以删除时（如维护区间最大值、可撤销并查集），改用回滚莫队：右端在同一左端块内按升序单调扩展并跨询问保留，左端只对当前询问临时补入、回答后立刻撤销到快照；询问区间完全落在一个块内时直接临时加入再撤销。算法本身仍只是一次离线调度，历史栈等可回滚状态由调用者维护。

```
// 依赖 2.3.001 普通莫队
template <class Ans, class Add, class Snap, class Undo, class Get>
vector<Ans> rollbackMo(int n, vector<MoQuery> qs, Add add, Snap snap, Undo undo, Get get)
{
    // n 是数组长度，qs 是询问；add 加入位置，snap/undo 管理快照，get 返回当前答案。
    auto z = max<int>(1, (int)sqrtl(max<int>(1, n)));
    sort(qs.begin(),
         qs.end(),
         [&](const auto &a, const auto &b)
         {
             auto x = a[0] / z, y = b[0] / z;
             return x ≠ y ? x < y : a[1] < b[1];
         });
    vector<Ans> ans(qs.size());
    auto empty = snap();
    for (int i = 0; i < (int)qs.size();)
    {
        auto k = qs[i][0] / z;
        auto j = i;
        while (j < (int)qs.size() && qs[j][0] / z == k)
        {
            j++;
        }
        auto mid = min<int>(n, (k + 1) * z);
        auto r = mid;
        for (auto p = i; p < j; p++)
        {
            auto q = qs[p];
            if (q[1] <= mid)
            {
                auto s = snap();
                for (auto x = q[0]; x < q[1]; x++)
                {
                    add(x);
                }
                ans[q[2]] = get();
                undo(s);
                continue;
            }
            while (r < q[1])
```

```
            {
                add(r++);
            }
            auto s = snap();
            for (auto x = mid - 1; x >= q[0]; x--)
            {
                add(x);
            }
            ans[q[2]] = get();
            undo(s);
        }
        undo(empty);
        i = j;
    }
    return ans;
}
```

调用 `rollbackMo<int>(n, qs, add, snap, undo, get)`，`MoQuery={l,r,id}`；`snap()` 返回历史栈大小等快照，`undo(s)` 必须精确恢复到该状态，块内短询问在临时加入后回滚。块长为 B 时右端扩展为 $O\left(\frac{n^2}{B}\right)$，短询问与左端临时扩展为 $O(qB)$，取 $B \approx \sqrt{n}$ 得 $O((n+q)\sqrt{n})$，排序为 $O(q \log q)$；除调用者保存的回滚历史外，调度额外空间为 $O(q)$。

回调可以用一个历史栈实现可撤销的区间和：

```
signed main()
{
    vector<int> c{1, 2, 3};
    vector<MoQuery> rq{{0, 2, 0}};
    int val = 0;
    vector<int> hist;
    auto addR = [&](int p) { hist.push_back(val); val += c[p]; };
    auto snap = [&]() { return (int)hist.size(); };
    auto undo = [&](int s)
    {
        while ((int)hist.size() > s)
        {
            val = hist.back();
            hist.pop_back();
        }
    };
    auto getR = [&]() { return val; };
    assert(rollbackMo<int>(3, rq, addR, snap, undo, getR)[0] == 3);
}
```

2.3.004 二次离线莫队

当区间答案是点对贡献之和、普通莫队每移动一个端点仍要扫描整段才能算出新增贡献时，用二次离线把这份代价均摊掉。前提是答案能写成

$$F(l,r)=\sum_{l\le i<j<r} w(a_i,a_j),$$

并且能预处理或离线回答前缀贡献 $G(p,x)=\sum_{0\le i<x} w(a_i,a_p)$——此时加入右端点 p 的增量是 $G(p,p)-G(p,l)$，加入左端点同理。于是第一次莫队并不立即计算昂贵的端点贡献，而是把它拆成若干带正负号的「端点对某段前缀」的 G 事件；第二次沿前缀位置从左到右扫描，用值域计数一次性回答整批事件。下面给出的异或点对是一个完整问题实例（统计区间内 $\mathrm{popcount}(a_i\oplus a_j)=k$ 的无序点对数），其中的值域计数只对这一贡献函数成立，不能当成对任意 w 都通用的类。

```cpp
vector<int> xorPairs(const vector<int> &a, const vector<pair<int, int>> &qs, int k, int bits)
{
    // a 是 bits 位非负整数，qs 是半开区间，k 是目标异或位数；返回各区间点对数。
    assert(0 <= bits && bits <= 20); // 调试检查，可删
    auto n = (int)a.size();
    auto lim = 1LL << bits;
    for (auto x : a)
    {
        assert(0 <= x && x < lim); // 调试检查，可删
        (void)x;
    }
    vector<int> ans(qs.size());
    if (k < 0 || k > bits)
    {
        return ans;
    }

    // Query[0..2] 依次表示从 1 开始的左端点、右端点、原询问编号。
    using Query = array<int, 3>;
    vector<Query> ord;
    for (int id = 0; id < (int)qs.size(); id++)
    {
        auto [l, r] = qs[id];
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l < r)
        {
            ord.push_back({l + 1, r, id});
        }
    }
    if (ord.empty())
    {
        return ans;
    }

    auto z = max<int>(1, (int)(n / sqrtl((long double)ord.size())));
    sort(ord.begin(),
         ord.end(),
         [&](const auto &x, const auto &y)
         {
             auto bx = (x[0] - 1) / z, by = (y[0] - 1) / z;
             if (bx != by)
```

```
            {
                return bx < by;
            }
            return bx & 1 ? x[1] > y[1] : x[1] < y[1];
        });

    vector<int> masks;
    for (int s = 0; s < lim; s++)
    {
        if (popcount((unsigned long long)s) == k)
        {
            masks.push_back(s);
        }
    }
    vector<int> v(n + 1), cnt(lim), pre(n + 1);
    copy(a.begin(), a.end(), v.begin() + 1);
    for (int i = 1; i <= n; i++)
    {
        pre[i] = pre[i - 1];
        for (auto s : masks)
        {
            pre[i] += cnt[v[i] ^ s];
        }
        cnt[v[i]]++;
    }

    // Event[0..3] 依次表示左端点、右端点、贡献符号、排序后询问编号。
    using Event = array<int, 4>;
    vector<vector<Event>> ev(n + 1);
    vector<int> cur(ord.size());
    int l = 1, r = 0;
    for (int id = 0; id < (int)ord.size(); id++)
    {
        auto q = ord[id];
        cur[id] = pre[q[1]] - pre[r] + pre[q[0] - 1] - pre[l - 1];
        if (!k)
        {
            cur[id] += q[0] - l;
        }
        if (r < q[1])
        {
            ev[l - 1].push_back({r + 1, q[1], -1, id});
            r = q[1];
        }
        if (l > q[0])
        {
            ev[r].push_back({q[0], l - 1, 1, id});
            l = q[0];
        }
        if (r > q[1])
        {
            ev[l - 1].push_back({q[1] + 1, r, 1, id});
```

```
            r = q[1];
        }
        if (l < q[0])
        {
            ev[r].push_back({l, q[0] - 1, -1, id});
            l = q[0];
        }
    }

    fill(cnt.begin(), cnt.end(), 0);
    for (int i = 1; i <= n; i++)
    {
        for (auto s : masks)
        {
            cnt[v[i] ^ s]++;
        }
        for (auto e : ev[i])
        {
            int val = 0;
            for (int j = e[0]; j <= e[1]; j++)
            {
                val += cnt[v[j]];
            }
            cur[e[3]] += e[2] * val;
        }
    }
    for (int i = 1; i < (int)cur.size(); i++)
    {
        cur[i] += cur[i - 1];
    }
    for (int i = 0; i < (int)ord.size(); i++)
    {
        ans[ord[i][2]] = cur[i];
    }
    return ans;
}
```

用 `auto ans = xorPairs(a, qs, k, bits)` 调用，`qs` 是若干 $[l,r)$，返回每个区间内满足 $\mathrm{popcount}(a_i \oplus a_j) = k$ 的无序点对数，要求 $0 \le a_i < 2^{bits}$ 且 $bits \le 20$。令 $C = \binom{bits}{k}$、块长为 B，本实例时间为 $O\left(2^{bits} + nC + \frac{n^2}{B} + qB + q\log q\right)$，取 $B \approx \frac{n}{\sqrt{q}}$ 时莫队部分为 $O(n\sqrt{q})$，空间为 $O(2^{bits} + n + q + C)$。二次离线只在前缀贡献 G 能显著快于逐段扫描时才划算；区间不同数、区间和这类端点本就 $O(1)$ 的问题应继续用普通莫队，而换一个贡献函数 w 都必须重新推导并对拍。下面用四个两位数同时检查整段、子段与空区间：

```
signed main()
{
    vector<int> a{0, 1, 2, 3};
    vector<pair<int, int>> q{{0, 4}, {1, 4}, {2, 2}};
    assert((xorPairs(a, q, 1, 2) == vector<int>{4, 2, 0}));
}
```

2.3.005 区间不同数统计

静态数组上的区间不同数，比起普通莫队有更快且确定性更强的做法：把询问按右端点离线，扫描右端点 `r` 时用树状数组只保留每个值的最后一次出现——遇到新的 `a[r-1]` 就把它旧的最后出现位置减 1、当前位置加 1，于是 Fenwick 中每个不同值恰好留下一个标记，区间 $[l, r)$ 的标记和即不同值数量。这依赖查询静态，一旦夹杂修改，「最后出现位置」会整体漂移而失效。这里先离散化值域，最后位置、Fenwick 和查询桶都只存下标与频次，打印版按普通 `int` 处理；`Fenwick` 的查询接口不直接复用是因为它的节点类型必须保留一般的 64 位和，而此处计数上界为数组长度。此处只给新增的 `distinct`：

```
vector<int> distinct(const vector<int> &a, const vector<pair<int, int>> &qs)
{
    // a 是静态数组，qs 是半开区间；返回每个区间中的不同值数量。
    // 只有下标、排名和频次的热点数据使用普通 int，避免无意义地扩大工作集。
    vector<int> val = a;
    sort(val.begin(), val.end());
    val.erase(unique(val.begin(), val.end()), val.end());
    vector<int> rank(a.size());
    for (int i = 0; i < (int)a.size(); i++)
    {
        rank[i] = (int)(lower_bound(val.begin(), val.end(), a[i]) - val.begin());
    }
    // Ask[0..1] 依次表示左端点与答案编号，按右端点分桶。
    using Ask = array<int, 2>;
    vector<vector<Ask>> ask(a.size() + 1);
    for (int i = 0; i < (int)qs.size(); i++)
    {
        auto [l, r] = qs[i];
        assert(0 <= l && l <= r && r <= (int)a.size()); // 调试检查，可删
        assert(r <= numeric_limits<int>::max());    // 调试检查，可删
        ask[r].push_back({(int)l, (int)i});
    }
    vector<int> bit(a.size() + 1), last(val.size(), -1);
    auto add = [&](int p, int delta)
    {
        // p 是从 0 开始的位置，delta 是出现位置标记的变化。
        for (p++; p < (int)bit.size(); p += p & -p)
        {
            bit[p] += (int)delta;
        }
    };
    auto sum = [&](int p)
    {
        // p 是半开前缀右端点；返回 [0,p) 的最后出现位置标记数。
        int ans = 0;
        for (; p; p -= p & -p)
        {
            ans += bit[p];
        }
        return ans;
    };
    vector<int> ans(qs.size());
```

```
    for (int r = 0; r <= (int)a.size(); r++)
    {
        if (r)
        {
            auto x = rank[r - 1];
            if (last[x] >= 0)
            {
                add(last[x], -1);
            }
            add(r - 1, 1);
            last[x] = r - 1;
        }
        for (auto [l, id] : ask[r])
        {
            ans[id] = sum(r) - sum(l);
        }
    }
    return ans;
}
```

用 `auto ans = distinct(a, qs)` 调用，`qs` 里每个区间为 $[l,r)$，允许空区间。数组值经防卡哈希记录最后位置，需要稳定最坏复杂度时可先离散化再把 `last` 换成 `vector<int>`。每个位置至多执行两次 Fenwick 修改、每个询问执行一次区间和，时间为 $O((n+q)\log n)$，空间为 $O(n+q)$。

2.3.006 区间第 K 小整体二分

在静态数组上批量回答 $[l,r)$ 第 k 小，用整体二分套树状数组即可，无需建可持久化线段树：把元素按离散值域中点分到左右两组，Fenwick 只暂存左半值在各位置的出现次数，每个询问用区间计数 `c` 判断第 k 小落在哪组——落右组时把 k 扣掉 `c`，每层递归前务必撤销本层的 Fenwick 修改。这套代码与 2.2.004 整体二分 给出的 `rangeKth` 完全一致（那里连同离散化 `Discrete`、`Fenwick` 一并给出并附固定样例），此处不再重复。`KthQuery={l,r,k}` 三维为半开区间左右端点与从 1 开始的排名，须满足 $1 \le k \le r-l$，函数只改询问副本中的 k、不动调用者数据；时间 $O((n+q)\log V \log n)$、空间 $O(n+q+V)$。若有在线强制查询，改用主席树或小波矩阵。

2.3.007 树上莫队

欧拉括号序把树上路径查询降到序列区间查询：DFS 时进入结点 u 追加一次记录 st_u、离开时再追加一次记录 ed_u，得到长 $2n$ 的序列 seq，每个结点恰出现两次。对路径 (u,v)（先令 $st_u \le st_v$），设 $w=\mathrm{lca}(u,v)$：

- 若 $w=u$（u 是 v 的祖先），路径点集恰为 $seq[st_u..st_v]$ 中「出现奇数次」的那些结点；
- 否则取区间 $seq[ed_u..st_v]$，其奇数次结点为路径去掉 w 后的部分，需额外单独加入 w。

因此把每个询问转成一段区间（外加一个可选的 w），用普通莫队（2.3.001 普通莫队）离线跑，`work(u)` 按 u 当前出现次数的奇偶切换其贡献（偶 → 奇计入、奇 → 偶移除）。LCA 用倍增（6.1.002 最近公共祖先：倍增）。下例统计路径上不同权值个数，权值需先离散化到 $[0,n)$。

```cpp
struct TreeMo
{
    int n = 0;                      // n 是树的顶点数。
    int lg = 1;                     // lg 是倍增祖先表的层数。
    int blk = 1;                    // blk 是当前询问排序的块长。
    vector<vector<int>> g;          // g 是树的邻接表。
    vector<int> col;                // col[u] 是顶点 u 离散到 [0,n) 的权值。
    vector<int> st;                 // st[u] 是顶点 u 进入欧拉序的位置。
    vector<int> ed;                 // ed[u] 是顶点 u 离开欧拉序的位置。
    vector<int> seq;                // seq 是每个顶点进出各出现一次的欧拉序。
    vector<int> dep;                // dep[u] 是顶点 u 到当前根的边数。
    vector<vector<int>> up;         // up[k][u] 是顶点 u 的第 2^k 级祖先。
    vector<array<int, 4>> qs;       // 每项是 {欧拉左端、欧拉右端、额外 LCA、答案编号}。

    explicit TreeMo(int m) : n(m), g(m), col(m), st(m), ed(m), dep(m)
    {
        // m 是非空树的顶点数。
        assert(m > 0); // 调试检查，可删
    }

    void addTreeEdge(int u, int v)
    {
        // u、v 是无向树边的两个端点；调用后把边加入邻接表。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void build(int root = 0)
    {
        // root 是本次选定的根；重建欧拉序和祖先表，并清空依赖旧序的询问。
        assert(0 <= root && root < n); // 调试检查，可删
        lg = 1;
        while ((1ULL << lg) < (unsigned long long)n)
        {
            lg++;
        }
        up.assign(lg, vector<int>(n, root));
        fill(dep.begin(), dep.end(), 0);
        seq.clear();
        seq.reserve((size_t)n * 2);
        qs.clear();

        st[root] = 0;
        seq.push_back(root);
        auto dfs = [&](auto &&self, int u, int p) -> void
        {
            for (auto v : g[u])
            {
                if (v == p)
                {
                    continue;
```

```
                }
                up[0][v] = u;
                dep[v] = dep[u] + 1;
                st[v] = seq.size();
                seq.push_back(v);
                self(self, v, u);
            }
            ed[u] = seq.size();
            seq.push_back(u);
        };
        dfs(dfs, root, -1);
        for (int k = 1; k < lg; k++)
        {
            for (int u = 0; u < n; u++)
            {
                up[k][u] = up[k - 1][up[k - 1][u]];
            }
        }
    }

    int lca(int u, int v) const
    {
        // u、v 是树上顶点；返回它们在当前根下的最近公共祖先。
        assert(!up.empty() && 0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删
        if (dep[u] < dep[v])
        {
            swap(u, v);
        }
        auto d = dep[u] - dep[v];
        for (int k = 0; k < lg; k++)
        {
            if (d >> k & 1)
            {
                u = up[k][u];
            }
        }
        if (u == v)
        {
            return u;
        }
        for (int k = lg - 1; k >= 0; k--)
        {
            if (up[k][u] != up[k][v])
            {
                u = up[k][u];
                v = up[k][v];
            }
        }
        return up[0][u];
    }

    void addQuery(int u, int v)
```

```
    {
        // u、v 是路径两端；把该路径转成半开欧拉区间并按加入顺序编号。
        assert(!up.empty() && 0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删
        if (st[u] > st[v])
        {
            swap(u, v);
        }
        auto w = lca(u, v);
        auto id = (int)qs.size();
        if (w == u)
        {
            qs.push_back({st[u], st[v] + 1, -1, id});
        }
        else
        {
            qs.push_back({ed[u], st[v] + 1, w, id});
        }
    }

    vector<int> solve()
    {
        // 返回每条已加入路径上的不同权值数，顺序与 addQuery 一致。
        if (qs.empty())
        {
            return {};
        }
        for (auto x : col)
        {
            assert(0 <= x && x < n); // 调试检查，可删
        }
        auto m = (int)qs.size();
        blk = max<int>(1, (int)((long double)seq.size() / sqrtl(m)));
        sort(qs.begin(),
             qs.end(),
             [&](const auto &a, const auto &b)
             {
                 auto x = a[0] / blk;
                 auto y = b[0] / blk;
                 if (x != y)
                 {
                     return x < y;
                 }
                 return x & 1 ? a[1] > b[1] : a[1] < b[1];
             });
        vector<int> cnt(n);
        vector<int> used(n);
        vector<int> ans(m);
        int cur = 0;
        auto work = [&](int u)
        {
            // u 是本次切换的顶点；奇数次出现计入路径，偶数次出现移出路径。
            used[u] ^= 1;
```

```
157             if (used[u])
158             {
159                 if (cnt[col[u]]++ == 0)
160                 {
161                     cur++;
162                 }
163             }
164             else
165             {
166                 if (--cnt[col[u]] == 0)
167                 {
168                     cur--;
169                 }
170             }
171         };
172         int l = 0;
173         int r = 0;
174         for (auto q : qs)
175         {
176             auto ql = q[0];
177             auto qr = q[1];
178             auto w = q[2];
179             auto id = q[3];
180             while (l > ql)
181             {
182                 work(seq[--l]);
183             }
184             while (r < qr)
185             {
186                 work(seq[r++]);
187             }
188             while (l < ql)
189             {
190                 work(seq[l++]);
191             }
192             while (r > qr)
193             {
194                 work(seq[--r]);
195             }
196             if (w >= 0)
197             {
198                 work(w);
199             }
200             ans[id] = cur;
201             if (w >= 0)
202             {
203                 work(w);
204             }
205         }
206         return ans;
207     }
208 };
```

先用 addTreeEdge 连成一棵树，填好 col 后调用 build(root)，再用 addQuery(u,v) 挂路径询问；solve() 返回按加入顺序排列的不同权值数。再次 build 会重置深度、欧拉序、祖先表，并清空基于旧根编码的询问。带修改版再增加时间轴，思路同 2.3.002 带修改莫队。欧拉序长 $2n$，代码取块长约 $2n/\sqrt{q}$，排序和状态切换共 $O(n\sqrt{q}+q\log q)$ 时间，祖先表与其余状态共 $O(n\log n+q)$ 空间。

下面的四点树覆盖跨分支路径、祖孙路径、单点路径与根路径，并检查换根重建会清空旧询问：

```
signed main()
{
    TreeMo tr(4);
    tr.addTreeEdge(0, 1);
    tr.addTreeEdge(0, 2);
    tr.addTreeEdge(1, 3);
    tr.col = {1, 2, 1, 3};
    tr.build(0);
    for (auto [u, v] : vector<pair<int, int>>{{3, 2}, {3, 1}, {2, 2}, {3, 0}})
    {
        tr.addQuery(u, v);
    }
    assert((tr.solve() == vector<int>{3, 2, 1, 3}));
    tr.build(3);
    assert(tr.qs.empty());
}
```

2.4 差分与前缀技巧

2.4.001 一维/二维前缀和

静态数组区间和与静态矩阵子矩形和的基础工具。一维前缀定义 $s_i=\sum_{0\le j<i}a_j$，于是 $[l,r)$ 的区间和为 s_r-s_l；二维前缀沿两个轴累计，查询时用四个角做容斥。全程半开区间，让空区间、边界与长度计算保持一致。一维只需 partial_sum：

```
// a 是原数组，s 是前缀和，l、r 是半开区间端点，sum 是区间和。
vector<int> s(a.size() + 1);
partial_sum(a.begin(), a.end(), s.begin() + 1);
auto sum = s[r] - s[l];
```

二维把它封装成 Prefix2，构造时预处理带零行零列的前缀矩阵，sum 用四角容斥回答子矩形：

```
template <class T> struct Prefix2
{
    int n = 0, m = 0;     // n、m 是原矩阵行数与列数。
    vector<vector<T>> s; // s 是带额外零行零列的二维前缀和。

    explicit Prefix2(const vector<vector<T>> &a)
    {
        // a 是规则原矩阵。
        n = a.size();
        m = n ? a[0].size() : 0;
```

```
        s.assign(n + 1, vector<T>(m + 1));
        for (int i = 0; i < n; i++)
        {
            assert((int)a[i].size() == m); // 调试检查，可删
            for (int j = 0; j < m; j++)
            {
                s[i + 1][j + 1] = a[i][j] + s[i][j + 1] + s[i + 1][j] - s[i][j];
            }
        }
    }

    T sum(int x1, int y1, int x2, int y2) const
    {
        // x1、y1、x2、y2 是半开子矩形的上、左、下、右边界；返回子矩形和。
        return s[x2][y2] - s[x1][y2] - s[x2][y1] + s[x1][y1];
    }
};
```

调用形如 `Prefix2<int> ps(mat); auto rect = ps.sum(x1, y1, x2, y2)`，所有接口都用半开区间，二维矩阵必须规则矩形，空矩阵只能构造不能查询。模板参数 `T` 必须能表示每个前缀、四角容斥和查询结果；能由范围证明 64 位安全时使用 `int`，否则把累计类型实例化为更宽类型，不要只看单个元素范围。一维构造 $O(n)$、查询 $O(1)$、空间 $O(n)$；二维构造 $O(nm)$、查询 $O(1)$、空间 $O(nm)$。

2.4.002 一维/二维差分

前缀和的逆运算：离线执行大量区间加、矩形加，最后一次性恢复全部值。差分只在区间边界记录「从这里开始增加、到那里停止」，最终做一次前缀和即恢复所有位置；二维矩形修改在四个角写入容斥符号，再沿两个轴分别做前缀和。若修改后还要在线查询，则应改用 Fenwick 或线段树。一维封装为 `Diff`：

```
template <class T> struct Diff
{
    int n = 0;   // n 是目标数组长度。
    vector<T> d; // d 保存长度为 n+1 的边界差分。

    Diff(int m = 0) : n(m), d(m + 1)
    {
        // m 是目标数组长度。
    }

    void add(int l, int r, T x)
    {
        // l、r 是半开区间端点，x 是区间增量。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        d[l] += x;
        d[r] -= x;
    }

    vector<T> build() const
    {
```

```
        // 返回执行全部区间修改后的长度 n 数组。
        vector<T> a(n);
        T cur{};
        for (int i = 0; i < n; i++)
        {
            cur += d[i];
            a[i] = cur;
        }
        return a;
    }
};
```

二维版 `Diff2` 在矩形四角写差分、`build()` 时沿两轴前缀和还原：

```
template <class T> struct Diff2
{
    int n = 0, m = 0;     // n、m 是目标矩阵行数与列数。
    vector<vector<T>> d; // d 保存带额外边界的二维差分。

    Diff2(int x = 0, int y = 0) : n(x), m(y), d(x + 1, vector<T>(y + 1))
    {
        // x、y 是目标矩阵行数与列数。
    }

    void add(int x1, int y1, int x2, int y2, T v)
    {
        // 四个坐标描述半开子矩形，v 是矩形增量。
        assert(0 <= x1 && x1 <= x2 && x2 <= n); // 调试检查，可删
        assert(0 <= y1 && y1 <= y2 && y2 <= m); // 调试检查，可删
        d[x1][y1] += v;
        d[x1][y2] -= v;
        d[x2][y1] -= v;
        d[x2][y2] += v;
    }

    vector<vector<T>> build() const
    {
        // 返回执行全部矩形修改后的 n 行 m 列矩阵。
        auto a = d;
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < m; j++)
            {
                if (i)
                {
                    a[i][j] += a[i - 1][j];
                }
                if (j)
                {
                    a[i][j] += a[i][j - 1];
                }
                if (i && j)
```

```
                {
                    a[i][j] -= a[i - 1][j - 1];
                }
            }
        }
        a.resize(n);
        for (auto &row : a)
        {
            row.resize(m);
        }
        return a;
    }
};
```

调用形如 `Diff<int> d(n); d.add(l, r, x); auto a = d.build();` 与 `Diff2<int> d2(n, m); d2.add(x1, y1, x2, y2, v); auto mat = d2.build();`，修改同样是半开区间，`build()` 不清空差分故可重复恢复同一结果。模板参数 `T` 必须能表示边界差分、前缀恢复中的所有中间量和最终数组；若单次修改或多次叠加可能越过 64 位，应显式选择更宽累计类型。每次修改 $O(1)$；一维恢复 $O(n)$、空间 $O(n)$；二维恢复 $O(nm)$、空间 $O(nm)$。

2.4.003 K 阶差分

定义一阶差分 $\Delta a_i = a_{i+1} - a_i$，它把多项式次数降低 1，其逆过程即前缀和。于是普通差分重复 k 次后，次数不超过 k 的多项式序列会变成常量——这可用来识别多项式序列、从有限差分恢复值，以及设计以组合数为基的多项式区间更新。后者的依据是组合数基满足

$$\binom{x}{k} - \binom{x-1}{k} = \binom{x-1}{k-1},$$

对它做一次差分会自然降阶，区间加一个多项式可转成少量高阶差分端点修改。但各题的端点补偿取决于所选基底、下标起点和模数，不宜塞进一个没有明确语义的通用类，因此本节的短函数只负责对一个静态序列求 k 阶差分；

```
template <class T> vector<T> kthDiff(vector<T> a, int k)
{
    // a 是按值复制的原序列，k 是差分阶数；返回 k 阶差分序列。
    assert(k >= 0); // 调试检查，可删
    auto t = min<int>(k, a.size());
    for (int z = 0; z < t; z++)
    {
        for (int i = 1; i < (int)a.size(); i++)
        {
            a[i - 1] = a[i] - a[i - 1];
        }
        a.pop_back();
    }
    return a;
}
```

调用 `kthDiff(a, k)` 返回长度为 $\max(0, n-k)$ 的新序列、不修改传入数组，例如 `kthDiff(vector<int>{0,1,4,9,16}, 2)` 得 `{2,2,2}`（平方序列的二阶差分为常量）。序列在第 n 阶后已经为空，`k>n` 时不再执行无意义循环。若题目要多次区间加后统一输出，应先把所加多项式写成 $\binom{i}{j}$ 的线性组合，再据上式推导左右端点补偿；需在线查询则把各阶差分放入 Fenwick 或线段树。令 $h=\min(k,n)$，当前实现执行 $\sum_{j=0}^{h-1}(n-j-1)$ 次减法，时间 $O(hn)$、额外空间 $O(n)$；所有差分值须能由 `T` 表示。组合数区间更新的题目特化通常单次修改 $O(k)$、最终恢复 $O(kn)$。

2.4.004 子段和、子矩阵和

Kadane 求非空最大子段和：在每个位置只比较「接到前一段后面」与「从当前位置重新开始」，全负数时也能返回最大单元素而不误判为空：

```cpp
template <class T> T maxSub(const vector<T> &a)
{
    // a 是非空序列；返回非空最大子段和。
    // 若要最小子段和，保留首元素初始化，把 max 改成 min、chmax 改成 chmin；不能只改最终答案比较。
    assert(!a.empty()); // 调试检查，可删
    T ans = a[0];
    T cur = a[0];
    for (int i = 1; i < (int)a.size(); i++)
    {
        T x = a[i];
        cur = max(x, cur + x);
        chmax(ans, cur);
    }
    return ans;
}
```

最大子矩阵和枚举上下边界，把夹在两行间的每列和压成一维数组再复用同一个 Kadane；调用前让较小维度承担平方枚举，可明显降低常数：

```cpp
template <class T> T maxSubMat(const vector<vector<T>> &a)
{
    // a 是非空规则矩阵；返回非空最大子矩阵和。
    // 若要最小子矩阵和，同步使用最小子段版本并把 chmax 改成 chmin；边界仍以 a[0][0] 初始化。
    assert(!a.empty() && !a[0].empty()); // 调试检查，可删
    auto n = (int)a.size();
    auto m = (int)a[0].size();
    for (auto &row : a)
    {
        assert((int)row.size() == m); // 调试检查，可删
    }
    T ans = a[0][0];
    for (int u = 0; u < n; u++)
    {
        vector<T> s(m);
        for (int d = u; d < n; d++)
        {
            for (int j = 0; j < m; j++)
```

```
            {
                s[j] += a[d][j];
            }
            chmax(ans, maxSub(s));
        }
    }
    return ans;
}
```

直接 `maxSub(a)`、`maxSubMat(mat)` 调用，两者都要求非空输入并已正确处理全负数，返回类型与元素类型 `T` 相同。调用者须保证全部累计量能由 `T` 表示；确实需要 `__int128` 时先把输入元素转成 `__int128`，不为这两个短函数再加一层累计类型参数。当前实现平方枚举行数，故当行数大于列数时应先转置。最大子段 $O(n)$ 时间、$O(1)$ 空间；$n \times m$ 矩阵当前实现 $O(n^2m)$ 时间、$O(m)$ 空间，令较小维度作为行数后为 $O(\min(n,m)^2 \max(n,m))$。

2.4.005 所有区间 GCD

令 $V = \max_i |a_i|$。固定右端点 r、让左端点从大到小移动时，$\gcd(a_l, \dots, a_r)$ 除了从 0 变为首个非零值外，只会变成前一个正值的真因数，因此不同取值至多 $O(\log(V+2))$ 个。据此对每个右端点维护一张压缩列表：从 `r-1` 的列表转移到 `r` 时，把每个旧 gcd 与 `a[r]` 再取 gcd 并合并相邻相同结果，同时记录每个 gcd 生效的最小左端点，查询固定右端点后按左端点二分即可。因为要保存所有右端点的列表并支持多次查询，这里封装成结构体是必要的：

```
template <class T> struct GcdRanges
{
    using U = make_unsigned_t<T>;

    vector<vector<pair<int, U>>> g; // g[r] 保存以 r 结尾的不同 GCD 及最小左端点。

    static U mag(T x)
    {
        // x 是原数组元素；返回不对最小负数取正值的无符号幅值。
        auto u = (U)x;
        if constexpr (is_signed_v<T>)
        {
            if (x < 0)
            {
                return (U)0 - u;
            }
        }
        return u;
    }

    explicit GcdRanges(const vector<T> &a) : g(a.size())
    {
        // a 是用于预处理所有右端点 GCD 分段的原数组。
        static_assert(is_integral_v<T>);
        vector<pair<int, U>> pre;
        for (int r = 0; r < (int)a.size(); r++)
```

```
        {
            auto v = mag(a[r]);
            vector<pair<int, U>> cur{{r, v}};
            for (auto [l, x] : pre)
            {
                x = gcd(x, v);
                if (cur.back().second == x)
                {
                    cur.back().first = l;
                }
                else
                {
                    cur.push_back({l, x});
                }
            }
            reverse(cur.begin(), cur.end());
            g[r] = cur;
            pre = cur;
            reverse(pre.begin(), pre.end());
        }
    }

    U query(int l, int r) const
    {
        // l、r 是闭区间端点；返回 a[l..r] 的绝对值 GCD。
        assert(0 <= l && l <= r && r < (int)g.size()); // 调试检查，可删
        auto it = upper_bound(g[r].begin(),
                              g[r].end(),
                              l,
                              [](int x, const auto &p)
                              {
                                  return x < p.first;
                              });
        return prev(it)->second;
    }
};
```

调用形如 `GcdRanges<int> gr(a); auto g = gr.query(l, r);`，其中 `query` 是本章唯一使用闭区间 $[l, r]$ 的接口——因为结构按「以 r 结尾」存储，这里特意保留差异以防与半开区间混用。GCD 以 `make_unsigned_t<T>` 保存，负数先按位转换后在无符号环中取幅值，因此最小有符号整数也得到可表示的 2^{63}，返回值同样是无符号类型。每个右端点至多保留 $O(\log(V+2))$ 个不同 GCD；按 64 位 gcd 为常数操作的竞赛 RAM 模型，构造为 $O(n\log(V+2))$，若计入 Euclid 迭代则保守上界为 $O(n\log^2(V+2))$；空间 $O(n\log(V+2))$，单次查询 $O(\log(1+\log(V+2)))$。

2.4.006 连续区间按位异或

连续四个整数的低两位模式，使前缀异或 $0\oplus 1\oplus\cdots\oplus n$ 只由 $n \bmod 4$ 决定（依次为 n、1、$n+1$、0），于是闭区间 $[l, r]$ 的异或可用两个前缀异或相消得到，常数时间完成。这是纯公式，没有状态，也不需要结构体：

```cpp
// l、r 是闭区间端点，ans 是区间内所有整数的异或和。
auto ans = r;
if ((r & 3) == 1) ans = 1;
else if ((r & 3) == 2) ans = r + 1;
else if ((r & 3) == 3) ans = 0;

if (l)
{
    auto x = l - 1;
    auto pre = x;
    if ((x & 3) == 1) pre = 1;
    else if ((x & 3) == 2) pre = x + 1;
    else if ((x & 3) == 3) pre = 0;
    ans ^= pre;
}
```

代码结束后的 ans 即闭区间 $[l,r]$ 的异或和，要求 $0 \le l \le r$，且 l、r 使用无符号 64 位整数；若要对数组下标区间异或仍应改用普通前缀异或数组。$O(1)$ 时间、$O(1)$ 空间。

3. 数据结构

3.1 并查集

3.1.001 普通并查集

并查集维护若干互不相交的集合，支持合并、判连通、查询集合大小，是动态加边连通性、Kruskal 求最小生成树、等价类划分的基础工具。每个集合是一棵父指针树，`find` 用路径压缩把访问路径上的点直接挂到根，`merge` 按集合大小把小树接到大树下；两者配合后单次操作均摊到近似常数。要留意路径压缩会原地改写父链，因此这份实现无法撤销、也保存不了历史版本，需要那些能力时见下文的可撤销与可持久化变体。

```
struct DSU
{
    // f[x] 是 x 的父亲；s[x] 仅在 x 为根时表示集合大小。
    vector<int> f, s;

    DSU(int n = 0)
    {
        // n 是元素个数；初始时每个元素自成集合。
        init(n);
    }

    void init(int n)
    {
        // n 是新的元素个数；清空旧状态并重建单点集合。
        assert(n >= 0); // 调试检查，可删
        f.resize(n);
        iota(f.begin(), f.end(), 0);
        s.assign(n, 1);
    }

    int find(int x)
    {
        // x 是元素编号；压缩父链并返回集合根。
        return x == f[x] ? x : f[x] = find(f[x]);
    }

    bool merge(int x, int y)
    {
        // x、y 是元素编号；合并两集合并返回是否真正发生合并。
        x = find(x);
        y = find(y);
        if (x == y)
        {
            return false;
        }
        if (s[x] < s[y])
        {
```

```
            swap(x, y);
        }
        f[y] = x;
        s[x] += s[y];
        return true;
    }

    bool same(int x, int y)
    {
        // x、y 是元素编号；返回二者是否同属一个集合。
        return find(x) == find(y);
    }

    int size(int x)
    {
        // x 是元素编号；返回其集合大小。
        return s[find(x)];
    }
};
```

公开接口为 `DSU`：`DSU(n)` 或 `init(n)` 建立 n 个单点集合（下标 $[0,n)$），`merge(x,y)` 把两点所在块并起来并返回是否真正合并，`same(x,y)` 判同集，`size(x)` 取 x 所在块的元素数。构造 $O(n)$；路径压缩加按大小合并后 m 次操作总代价 $O(m\,\alpha(n))$，单次最坏 $O(\log n)$，空间 $O(n)$。只需加边连通性用本条；维护点间差值或异或约束改用 3.1.003 带权并查集，离线撤销最近操作用 3.1.004 可撤销并查集，在线访问任意历史版本用 3.1.005 可持久化并查集，一次合并两段等长区间用 3.1.006 区间并行并查集。

3.1.002 按大小/按秩合并

这不是一份新代码，而是对合并策略的说明：本章并查集统一采用按大小合并，直接依赖 3.1.001 普通并查集 的 `DSU`。`merge(x,y)` 里 `if (s[x] < s[y]) swap(x, y)` 保证较小的树挂到较大的树下，树高因此不超过 $O(\log n)$；再叠加路径压缩，m 次操作均摊 $O(m\,\alpha(n))$。按秩合并是等价的替代口径——把 `s` 换成树高上界，仅当两根秩相等时把新根的秩加一，其余只比较不更新；两种口径都能压树高，但「大小」和「秩」语义不同，不能混用同一个数组。单独用路径压缩而不按大小/秩合并时，可以先形成长度为 n 的父链，第一次查询链尾的单次代价为 $O(n)$；标准的反阿克曼均摊界要求路径压缩与按秩或按大小合并配合。带权、可撤销、可持久化等变体分别见 3.1.003 带权并查集、3.1.004 可撤销并查集、3.1.005 可持久化并查集。

3.1.003 带权并查集

带权并查集在连通性之外，还沿父边维护每个点到其父亲的势能，从而回答同一集合内两点的差值，并检验新约束是否与已有约束矛盾。加法势能版把约束 $val_y - val_x = d$ 视作一条带权边：`find` 沿路径压缩时把 `w` 累加成到根的势能，`merge` 先各自找根，再由 $v = d + w_x - w_y$ 算出两根间应有的差；若两点已同根，只需检查 v 是否为零，否则把小树挂到大树并令新根边权等于 $\pm v$。势能类型 `T` 必须支持零元、加减、相反数和相等比较。

```cpp
template <class T> struct WDSU
{
    // f[x] 是 x 的父亲；s[x] 仅在 x 为根时表示集合大小。
    vector<int> f, s;
    // w[x] 是 x 到父亲的势能差 val[x]-val[f[x]]。
    vector<T> w;

    WDSU(int n = 0)
    {
        // n 是元素个数；初始势能均为零。
        init(n);
    }

    void init(int n)
    {
        // n 是新的元素个数；清空旧状态并重建单点集合。
        assert(n >= 0); // 调试检查，可删
        f.resize(n);
        iota(f.begin(), f.end(), 0);
        s.assign(n, 1);
        w.assign(n, T{});
    }

    int find(int x)
    {
        // x 是元素编号；压缩父链、累加到根的势能并返回根。
        if (x == f[x])
        {
            return x;
        }
        int p = f[x];
        f[x] = find(p);
        w[x] += w[p];
        return f[x];
    }

    // 加入 val[y]-val[x]=d；已连通时返回该约束是否一致。
    bool merge(int x, int y, T d)
    {
        // 施加 val[y]-val[x]=d；已连通时返回该约束是否一致。
        int a = find(x), b = find(y);
        T v = d + w[x] - w[y];
        if (a == b)
        {
            return v == T{};
        }
        if (s[a] < s[b])
        {
            f[a] = b;
            w[a] = -v;
            s[b] += s[a];
        }
```

```
        else
        {
            f[b] = a;
            w[b] = v;
            s[a] += s[b];
        }
        return true;
    }

    bool same(int x, int y)
    {
        // x、y 是元素编号；返回二者是否同属一个集合。
        return find(x) == find(y);
    }

    // 要求 x,y 已连通，返回 val[y]-val[x]。
    T diff(int x, int y)
    {
        // x、y 必须连通；返回 val[y]-val[x]。
        assert(same(x, y)); // 调试检查，可删
        return w[y] - w[x];
    }
};
```

`WDSU<T>(n)` 建 n 个孤立点。`merge(x,y,d)` 施加约束 $val_y - val_x = d$：两点原本不连通时返回 `true` 并合并，已连通时返回该约束是否与现状一致（不修改结构）。`diff(x,y)` 要求 x, y 已同集（内部 `assert`），返回 $val_y - val_x$。

异或势能不能直接套用加法版的模板参数。下面令 `xr[x]` 表示 $val_x \oplus val_{f_x}$；路径压缩时沿父链累积异或值，加入约束 $val_x \oplus val_y = w$ 时，两根之间的异或值为 `xr[x]^xr[y]^w`。

```
struct XorDSU
{
    // f[x] 是 x 的父亲；s[x] 仅在 x 为根时表示集合大小。
    vector<int> f, s, xr;

    XorDSU(int n = 0)
    {
        // n 是元素个数；初始异或势能均为零。
        init(n);
    }

    void init(int n)
    {
        // n 是新的元素个数；清空旧状态并重建单点集合。
        assert(n >= 0); // 调试检查，可删
        f.resize(n);
        iota(f.begin(), f.end(), 0);
        s.assign(n, 1);
        xr.assign(n, 0);
    }
```

```

    int find(int x)
    {
        // x 是元素编号；压缩父链、累积到根的异或值并返回根。
        if (x == f[x])
        {
            return x;
        }
        int p = f[x];
        f[x] = find(p);
        xr[x] ^= xr[p];
        return f[x];
    }

    bool merge(int x, int y, int w)
    {
        // 施加 val[x]^val[y]=w；已连通时返回该约束是否一致。
        int a = find(x), b = find(y);
        int v = xr[x] ^ xr[y] ^ w;
        if (a == b)
        {
            return v == 0;
        }
        if (s[a] < s[b])
        {
            f[a] = b;
            xr[a] = v;
            s[b] += s[a];
        }
        else
        {
            f[b] = a;
            xr[b] = v;
            s[a] += s[b];
        }
        return true;
    }

    bool same(int x, int y)
    {
        // x、y 是元素编号；返回二者是否同属一个集合。
        return find(x) == find(y);
    }

    int diff(int x, int y)
    {
        // x、y 必须连通；返回 val[x]^val[y]。
        assert(same(x, y)); // 调试检查，可删
        return xr[x] ^ xr[y];
    }

    int value(int x)
```

```
    {
        // x 是元素编号；把当前根的值视为零，返回 val[x]。
        find(x);
        return xr[x];
    }
};
```

`XorDSU(n)` 建 n 个孤立点。`merge(x,y,w)` 施加 $val_x \oplus val_y = w$，返回该约束是否与现状一致；`diff(x,y)` 返回同一集合内两点的异或值。`value(x)` 把当前集合根的值视为零，只给出这一约定下的点值；未固定根的真实值时，只有两点异或值是确定的。两版构造均为 $O(n)$，单次操作最坏 $O(\log n)$、均摊 $O(\alpha(n))$，空间 $O(n)$。仅需连通性用 3.1.001 普通并查集；若势能属于矩阵、置换等非交换乘法群，不能改写 `WDSU` 的加减顺序，应使用 3.1.008 非交换群势能并查集；若约束需要撤销或回退到历史版本，则分别改用 3.1.004 可撤销并查集 与 3.1.005 可持久化并查集，并相应维护势能（为保历史不能使用路径压缩）。

3.1.004 可撤销并查集

可撤销并查集支持回退最近若干次合并，用于线段树分治、时间线回退和离线分治：处理完一段时间/一个分治区间后，把期间做过的合并原样撤销，恢复到进入前的状态。路径压缩会不可控地改写大量父指针、无法逐条回滚，因此这里彻底放弃压缩，只保留按大小合并（用负数 `p[root]` 直接编码集合大小）。每次 `merge` 都往栈里留痕：真正合并时压两条记录（两个被改根的旧父值），两点已同集时压一条哨兵 `{-1,0}`；`snap` 取当前栈高作为快照编号，`rollback(s)` 弹栈直到高度回到 `s` 并逐一还原父指针。因为没有压缩，`find` 单次是 $O(\log n)$ 而非 $\alpha(n)$。

```
struct RollbackDSU
{
    // p[x]<0 表示 x 是根且集合大小为 -p[x]；否则 p[x] 是父亲。
    vector<int> p;
    // st 依次保存每个被改位置及其修改前的 p 值；{-1,0} 表示一次空合并。
    vector<pair<int, int>> st;

    RollbackDSU(int n = 0) : p(n, -1)
    {
        // n 是元素个数；初始时每个元素自成集合。
        assert(n >= 0); // 调试检查，可删
    }

    int find(int x) const
    {
        // x 是元素编号；沿父链返回根，不做路径压缩。
        while (p[x] >= 0)
        {
            x = p[x];
        }
        return x;
    }

    int snap() const
    {
```

```
        // 返回当前回滚栈高度，作为之后回滚所用的快照编号。
        return (int)st.size();
    }

    bool merge(int a, int b)
    {
        // a、b 是元素编号；按大小合并并返回是否真正发生合并。
        a = find(a);
        b = find(b);
        if (a == b)
        {
            st.push_back({-1, 0});
            return false;
        }
        if (p[a] > p[b])
        {
            swap(a, b);
        }
        st.push_back({b, p[b]});
        st.push_back({a, p[a]});
        p[a] += p[b];
        p[b] = a;
        return true;
    }

    void rollback(int s)
    {
        // s 是此前由 snap 返回的栈高；撤销其后的所有合并。
        assert(0 <= s && s <= (int)st.size()); // 调试检查，可删
        while ((int)st.size() > s)
        {
            auto [u, x] = st.back();
            st.pop_back();
            if (u >= 0)
            {
                p[u] = x;
            }
        }
    }

    bool same(int a, int b) const
    {
        // a、b 是元素编号；返回二者是否同属一个集合。
        return find(a) == find(b);
    }

    int size(int x) const
    {
        // x 是元素编号；返回其集合大小。
        return -p[find(x)];
    }
};
```

典型用法：`snap()` 记下当前进度，做若干 `merge`，事后 `rollback(snap)` 撤回。`merge(a,b)` 返回是否真正合并，`same/size` 只读查询。`find`、`merge` 单次 $O(\log n)$，`rollback` 与被撤销的合并数成正比，栈空间 $O(n+q)$。它正是 3.3.016 线段树分治 与整体二分里维护连通性的标准件；只要求普通连通性、无需回退时用 3.1.001 普通并查集（能用路径压缩更快），需要在线跳到任意历史版本而非仅撤销栈顶时改用 3.1.005 可持久化并查集。

3.1.005 可持久化并查集

可持久化并查集为每次合并保留一个不可变的历史版本，允许从任意旧版本继续分叉、并查询当时的连通关系，适合强制在线的历史状态题。底层是一棵稀疏可持久化线段树（当作可持久化数组用）：未写叶子隐含为单点集合；写入的根以负数编码集合大小，非根写父亲编号。这样初始版本只用根 0，每次合并才按两条修改路径复制 $O(\log n)$ 个结点，旧版本原封不动。因为版本要共享结构，绝不能用路径压缩（那会原地改链、污染旧版本），只能按大小合并压树高——于是 `find` 里每跳一步父亲都要走一次 $O(\log n)$ 的线段树查询，单次 `find` 是 $O(\log^2 n)$。

```
struct PerDSU
{
    struct N
    {
        // l、r 是持久化数组的左右儿子；value 为叶子覆盖值，负数编码根的集合大小，非负数是父亲。
        int l = 0, r = 0, value = -1;
    };

    // n 是元素个数；t[0] 是全未写的初始版本，隐含每个元素自成集合。
    int n;
    vector<N> t{{}};

    PerDSU(int n) : n(n)
    {
        // n 是元素个数；允许 n=0，此时 build 返回空根 0。
        assert(n >= 0); // 调试检查，可删
    }

    int build(int l, int r) const
    {
        // [l,r) 是初始数组范围；初始值全是隐含单点集合，返回空根 0。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return 0;
    }

    int build() const
    {
        // 返回 n 个单点集合组成的初始版本根；空结构也返回 0。
        return 0;
    }

    int value(int x, int l, int r, int p) const
    {
        // x 是版本树节点，[l,r) 是其区间，p 是元素下标；返回覆盖值，未写位置返回 -1。
```

```
        if (!x)
        {
            return -1;
        }
        if (r - l == 1)
        {
            return t[x].value;
        }
        int m = (l + r) / 2;
        return p < m ? value(t[x].l, l, m, p) : value(t[x].r, m, r, p);
    }

    pair<int, int> get(int rt, int p) const
    {
        // rt 是版本根，p 是元素下标；返回 {父亲，根大小}，非根的第二项不使用。
        assert(0 <= p && p < n); // 调试检查，可删
        int v = value(rt, 0, n, p);
        return v < 0 ? pair<int, int>{p, -v} : pair<int, int>{v, 0};
    }

    int set(int x, int l, int r, int p, int v)
    {
        // x 是旧版本节点，[l,r) 是其区间，p 是元素下标；写入覆盖值 v 并返回新根。
        assert(t.size() < numeric_limits<int>::max()); // 调试检查，可删
        int y = (int)t.size();
        t.push_back(t[x]);
        if (r - l == 1)
        {
            t[y].value = v;
            return y;
        }
        int m = (l + r) / 2;
        if (p < m)
        {
            t[y].l = set(t[y].l, l, m, p, v);
        }
        else
        {
            t[y].r = set(t[y].r, m, r, p, v);
        }
        return y;
    }

    int find(int rt, int x) const
    {
        // rt 是版本根，x 是元素编号；沿父链返回根，不做路径压缩。
        while (true)
        {
            int p = get(rt, x).first;
            if (p == x)
            {
                return x;
```

```
            }
            x = p;
        }
    }

    int merge(int rt, int x, int y)
    {
        // rt 是旧版本根，x、y 是元素编号；返回按大小合并后的新版本根。
        x = find(rt, x);
        y = find(rt, y);
        if (x == y)
        {
            return rt;
        }
        int sx = get(rt, x).second;
        int sy = get(rt, y).second;
        if (sx < sy)
        {
            swap(x, y);
            swap(sx, sy);
        }
        rt = set(rt, 0, n, y, x);
        return set(rt, 0, n, x, -(sx + sy));
    }

    bool same(int rt, int x, int y) const
    {
        // rt 是版本根，x、y 是元素编号；返回二者是否同属一个集合。
        return find(rt, x) == find(rt, y);
    }

    int size(int rt, int x) const
    {
        // rt 是版本根，x 是元素编号；返回其集合大小。
        return get(rt, find(rt, x)).second;
    }
};
```

用法：`PerDSU pd(n)` 后 `int root = pd.build()` 取初始版本根，之后所有操作都显式带上版本根——`merge(rt,x,y)` 返回新版本根（无合并时返回原 `rt`），`find(rt,x)`、`same(rt,x,y)`、`size(rt,x)` 在指定版本上只读查询。`n=0` 时 `build()` 返回空根 0，此时没有合法元素可供查询或合并。可把每个版本根存进数组以随时回访。`find`/`same`/`size` 最坏 $O(\log^2 n)$，`merge` 最坏 $O(\log^2 n)$ 且新增 $O(\log n)$ 个结点；初建 $O(1)$，总空间 $O(q \log n)$。打印版的结点下标和叶子值使用普通 `int`；性能版为缩小持久化节点固定成 32 位，因此使用性能头时还要求 n、节点总数和集合大小不超过 $2^{31}-1$。它比 3.1.004 可撤销并查集 强在能访问任意历史版本而非只能撤销栈顶，代价是多一个 $\log$；若只需撤销最近操作，可撤销版更省。底层是按下标单点持久化的线段树，与 3.3.008 可持久化线段树 / 主席树 同源。

3.1.006 区间并行并查集

区间并行并查集一次把两个等长区间 $[x, x+len)$ 与 $[y, y+len)$ 的对应位置分别合并，用于「区间 A 与区间 B 逐位相等」这类约束（如强制某两段子串相等）。朴素做法逐位 `merge` 是 $O(len\,\alpha)$，这里借鉴 ST 表倍增：对每个层级 k 维护一层「长度 2^k 的块」的并查集 `d[k]`，先在合适层级把两个长块整体合并，仅当该层块是首次被合并时才递归下拆到两个半块，已合并过的块直接剪枝。真正的单点等价通过回调 `f(a,b)` 落到第 0 层，再由调用方交给 3.1.001 普通并查集 的 `DSU` 记录最终连通性；下面只打印新增的 `RangeDSU`。

```
// 依赖 3.1.001 普通并查集
struct RangeDSU
{
    struct D
    {
        // f[x]<0 表示 x 是块集合根且大小为 -f[x]；否则 f[x] 是父亲。
        vector<int> f;
        D(int n = 0) : f(n, -1)
        {
            // n 是本层可作为块起点的位置数。
        }
        int find(int x)
        {
            // x 是本层块起点；压缩父链并返回块集合根。
            return f[x] < 0 ? x : f[x] = find(f[x]);
        }
        pair<int, int> merge(int x, int y)
        {
            // x、y 是本层块起点；返回 {新根, 被吸收根}，已经同集时返回 {-1,-1}。
            x = find(x);
            y = find(y);
            if (x == y)
            {
                return {-1, -1};
            }
            if (f[x] > f[y])
            {
                swap(x, y);
            }
            f[x] += f[y];
            f[y] = x;
            return {x, y};
        }
    };

    // n 是原序列长度；d[k] 维护所有长度为 2^k 的块之间的等价关系。
    int n;
    vector<D> d;

    RangeDSU(int n) : n(n)
    {
        // n 是原序列长度；预建每个倍增层的块并查集。
        assert(n >= 0); // 调试检查，可删
```

```
        int m = max<int>(1, bit_width((unsigned long long)n));
        for (int k = 0; k < m; k++)
        {
            d.emplace_back(n - (1LL << k) + 1);
        }
    }

    template <class F> void mergeBlock(int k, int x, int y, F &f)
    {
        // k 是块层级，x、y 是两个块起点；首次连通单点块时调用 f(新根, 被吸收根)。
        auto [a, b] = d[k].merge(x, y);
        if (a < 0)
        {
            return;
        }
        if (!k)
        {
            return f(a, b);
        }
        int h = 1LL << (k - 1);
        mergeBlock(k - 1, x, y, f);
        mergeBlock(k - 1, x + h, y + h, f);
    }

    template <class F> void merge(int x, int y, int len, F f)
    {
        // 合并 [x,x+len) 与 [y,y+len)；每个新单点合并调用一次 f(新根, 被吸收根)。
        assert(0 <= x && x <= n && 0 <= y && y <= n && 0 <= len && len <= n - x && len <= n -
        ↪ y); // 调试检查，可删
        if (!len)
        {
            return;
        }
        int k = bit_width((unsigned long long)len) - 1;
        int z = 1LL << k;
        mergeBlock(k, x, y, f);
        mergeBlock(k, x + len - z, y + len - z, f);
    }
};
```

用法：`RangeDSU rd(n)` 建好各倍增层；对每条「两段相等」约束调用 `rd.merge(x, y, len, [&](int root, int absorbed){ … })`。lambda 只会在第 0 层真正合并时调用，两个参数分别是此结构第 0 层并查集的新根和被吸收根，因此既可只把它们交给外部 `DSU base(n)` 做 `base.merge(root, absorbed)`，也可把连通块和、计数等聚合状态直接维护在 `root` 上后再吸收 `absorbed`。要求回调返回 `void`，且 $[x, x+len)$、$[y, y+len)$ 都落在 $[0, n)$ 内；`len=0` 不产生操作。每层每对块至多真正合并一次，全部 q 次区间操作的总时间为 $O((n\log n + q)\alpha(n))$，总空间 $O(n\log n)$。它是逐位 `merge` 的批量加速版：区间约束很多、单点朴素合并成为瓶颈时才值得用，约束零散时直接 3.1.001 普通并查集 更简单。

外部 `DSU` 只记录第 0 层真实连通关系时，最小调用如下：

```
signed main()
{
    RangeDSU rd(6);
    DSU base(6);
    rd.merge(0, 3, 3, [&](int root, int absorbed)
    {
        base.merge(root, absorbed);
    });
    assert(base.same(0, 3) && base.same(1, 4) && base.same(2, 5));
}
```

3.1.007 可删除并查集

普通并查集的父指针一旦压缩，就无法从一棵集合树中直接摘掉任意元素；可删除并查集给每个原元素 x 保存一个当前替身 `pos[x]`，删除时不改旧父链，只把旧集合的有效元素数减一，再为 x 新建一个单点替身。`p` 记录只增不减的物理节点数并据此按大小合并，`cnt` 单独记录当前有效元素数；不能拿会随删除减小的 `cnt` 决定合并方向，否则一棵物理大树反复失去元素后可能被当成小树悬挂，破坏并查集的高度界。这里的删除语义是“把元素移到新的单点集合”，不是让元素从编号域中消失。

```
struct ErasableDSU
{
    // p[x]<0 表示 x 是内部树根，-p[x] 是只增不减的物理节点数；否则 p[x] 是父亲。
    vector<int> p;
    // cnt[x] 仅在 x 是根时表示集合中的当前元素数。
    vector<int> cnt;
    // pos[x] 是当前代表原元素 x 的内部节点。
    vector<int> pos;

    ErasableDSU(int n = 0)
    {
        // n 是原元素个数；初始时每个元素自成集合。
        init(n);
    }

    void init(int n)
    {
        // n 是新的原元素个数；清空全部合并与删除状态，无返回值。
        assert(n >= 0); // 调试检查，可删
        p.assign(n, -1);
        cnt.assign(n, 1);
        pos.resize(n);
        iota(pos.begin(), pos.end(), 0);
    }

    int findNode(int x)
    {
        // x 是内部节点编号；压缩父链并返回内部树根。
        return p[x] < 0 ? x : p[x] = findNode(p[x]);
    }
```

```
31
32      bool merge(int x, int y)
33      {
34          // x、y 是原元素编号；合并所在集合并返回是否真正发生合并。
35          assert(0 <= x && x < (int)pos.size() && 0 <= y && y < (int)pos.size()); // 调试检查，可删
36          int a = findNode(pos[x]);
37          int b = findNode(pos[y]);
38          if (a == b)
39          {
40              return false;
41          }
42          if (p[a] > p[b])
43          {
44              swap(a, b);
45          }
46          p[a] += p[b];
47          p[b] = a;
48          cnt[a] += cnt[b];
49          return true;
50      }
51
52      bool erase(int x)
53      {
54          // x 是原元素编号；把它移到新的单点集合，原本已是单点时返回 false。
55          assert(0 <= x && x < (int)pos.size()); // 调试检查，可删
56          int r = findNode(pos[x]);
57          if (cnt[r] == 1)
58          {
59              return false;
60          }
61          cnt[r]--;
62          pos[x] = p.size();
63          p.push_back(-1);
64          cnt.push_back(1);
65          return true;
66      }
67
68      bool same(int x, int y)
69      {
70          // x、y 是原元素编号；返回二者当前是否同属一个集合。
71          assert(0 <= x && x < (int)pos.size() && 0 <= y && y < (int)pos.size()); // 调试检查，可删
72          return findNode(pos[x]) == findNode(pos[y]);
73      }
74
75      int size(int x)
76      {
77          // x 是原元素编号；返回其当前集合中的原元素数。
78          assert(0 <= x && x < (int)pos.size()); // 调试检查，可删
79          return cnt[findNode(pos[x])];
80      }
81  };
```

`ErasableDSU(n)` 的原元素编号固定为 $[0,n)$；`merge`、`same`、`size` 与普通并查集同义，`erase(x)` 仅当 x 原来不是单点时新建替身并返回 `true`。若共有 e 次成功删除，构造为 $O(n)$，其余操作均摊 $O(\alpha(n+e))$、单次最坏 $O(\log(n+e))$，空间 $O(n+e)$。没有删除时直接用 3.1.001 普通并查集；按时间撤销合并用 3.1.004 可撤销并查集，访问任意历史版本用 3.1.005 可持久化并查集，这两种历史语义都不能由替身删除代替。

```
signed main()
{
    ErasableDSU d(4);
    d.merge(0, 1);
    d.merge(1, 2);
    assert(d.size(0) == 3 && d.erase(1));
    assert(!d.same(0, 1) && d.same(0, 2));
    assert(d.size(0) == 2 && d.size(1) == 1);
    assert(!d.erase(1));
    d.merge(1, 3);
    assert(d.same(1, 3) && d.size(3) == 2);
}
```

3.1.008 非交换群势能并查集

矩阵、置换、刚体变换等势能约束的乘法次序不能交换；把加法版 `WDSU` 里的 `+`、`-` 机械替换为乘法和逆元会在路径压缩或按大小换根时反转因子，常常只在链形数据上才暴露。这里把未知量记为群元素 $value_x$，群运算按书写顺序记为 $a\circ b$，并在父边保存右势能

$$value_x = value_{f_x}\circ pot_x.$$

模板参数 `Group` 必须提供 `using T`、静态函数 `id()`、`op(a,b)`、`inv(a)`，分别返回单位元、$a\circ b$ 与 a^{-1}；`T` 还须支持 `==`。`find` 把一条两边路径压成根边时严格按 $pot_x \leftarrow pot_{f_x}\circ pot_x$ 更新。`merge(x,y,d)` 的语义固定为 $value_x\circ d = value_y$，因此同根校验是 $pot_x^{-1}\circ pot_y = d$。若根 a 挂到根 b，应令

$$pot_a = pot_y\circ d^{-1}\circ pot_x^{-1};$$

反方向则令 $pot_b = pot_x\circ d\circ pot_y^{-1}$。两条式子都不能重排。对于 Library Checker 的 $a_u = a_v x$，直接调用 `merge(v,u,x)`；查询 $a_v^{-1}a_u$ 则调用 `diff(v,u)`。

```
template <class Group> struct PotentialDSU
{
    using T = typename Group::T;

    // f[x] 是 x 的父亲；s[x] 仅在 x 为根时表示集合大小。
    vector<int> f, s;
    // pot[x] 满足 value[x]=value[f[x]]*pot[x]；根的 pot 是单位元。
    vector<T> pot;

    PotentialDSU(int n = 0)
    {
```

```
        // n 是元素个数；初始时每个元素自成集合且势能为群单位元。
        init(n);
    }

    void init(int n)
    {
        // n 是新的元素个数；清空旧状态并重建单点集合。
        assert(n >= 0); // 调试检查，可删
        f.resize(n);
        iota(f.begin(), f.end(), 0);
        s.assign(n, 1);
        pot.assign(n, Group::id());
    }

    int find(int x)
    {
        // x 是元素编号；压缩父链、更新到根的右势能并返回集合根。
        if (x == f[x])
        {
            return x;
        }
        int p = f[x];
        f[x] = find(p);
        pot[x] = Group::op(pot[p], pot[x]);
        return f[x];
    }

    bool merge(int x, int y, const T &d)
    {
        // 施加 value[x]*d=value[y]；返回约束是否与已有信息相容。
        int a = find(x), b = find(y);
        if (a == b)
        {
            return Group::op(Group::inv(pot[x]), pot[y]) == d;
        }
        if (s[a] < s[b])
        {
            f[a] = b;
            pot[a] = Group::op(Group::op(pot[y], Group::inv(d)), Group::inv(pot[x]));
            s[b] += s[a];
        }
        else
        {
            f[b] = a;
            pot[b] = Group::op(Group::op(pot[x], d), Group::inv(pot[y]));
            s[a] += s[b];
        }
        return true;
    }

    bool same(int x, int y)
    {
```

```
        // x、y 是元素编号；返回二者是否同属一个集合。
        return find(x) == find(y);
    }

    T diff(int x, int y)
    {
        // x、y 必须连通；返回 value[x] 的逆乘 value[y]，即 value[x]^{-1}*value[y]。
        assert(same(x, y)); // 调试检查，可删
        return Group::op(Group::inv(pot[x]), pot[y]);
    }

    int size(int x)
    {
        // x 是元素编号；返回 x 所在集合的元素个数。
        return s[find(x)];
    }
};
```

`PotentialDSU<Group>(n)` 建立下标 $[0,n)$ 的单点集合；`merge(x,y,d)` 返回新约束是否相容、相容且原来不连通时合并；`same(x,y)` 判连通；`diff(x,y)` 要求二者连通，返回 $value_x^{-1}\circ value_y$；`size(x)` 返回集合大小。设一次 `Group` 运算为 $O(1)$，构造 $O(n)$，路径压缩配合按大小合并后 m 次操作均摊 $O(m\alpha(n))$、单次最坏 $O(\log n)$，额外空间 $O(n)$。加法差分用 3.1.003 带权并查集 更短，异或约束也用该条的 `XorDSU`；需要回滚或持久版本时现有 3.1.004 可撤销并查集、3.1.005 可持久化并查集 只维护连通性，不能直接替代这里的群势能。

下面以置换群固定验证非交换顺序、反向查询与把小集合挂到大集合的分支；`op(a,b)` 定义为先作用 `b` 再作用 `a`：

```
struct PermGroup
{
    using T = array<int, 3>; // T[i] 是置换把 i 映到的像。

    static T id()
    {
        // 返回三元置换群的恒等置换。
        return {0, 1, 2};
    }

    static T op(const T &a, const T &b)
    {
        // a、b 是置换；返回先作用 b 再作用 a 的复合 a*b。
        T c{};
        for (int i = 0; i < 3; i++)
        {
            c[i] = a[b[i]];
        }
        return c;
    }

    static T inv(const T &a)
    {
```

```
        // a 是置换；返回其逆置换。
        T b{};
        for (int i = 0; i < 3; i++)
        {
            b[a[i]] = i;
        }
        return b;
    }
};

signed main()
{
    array<int, 3> a{1, 2, 0}; // a[i] 是循环置换 a 对 i 的像。
    array<int, 3> b{1, 0, 2}; // b[i] 是交换 0、1 的置换 b 对 i 的像。
    assert(PermGroup::op(a, b) != PermGroup::op(b, a));
    PotentialDSU<PermGroup> d(4);
    assert(d.merge(0, 1, a));
    assert(d.merge(1, 2, b));
    assert(d.merge(3, 0, a));
    assert(d.diff(3, 2) == PermGroup::op(a, PermGroup::op(a, b)));
    assert(!d.merge(3, 2, PermGroup::op(PermGroup::op(a, b), a)));
    cout << "OK\n";
}
```

3.2 树状数组

3.2.001 单点修改、前缀和

树状数组（Fenwick / BIT）以 $O(\log n)$ 支持单点增量与前缀和，区间和由两个前缀相减得到，是本章最轻量的可修改前缀结构，也是主席树套 BIT、二维数点等的底座。核心是 `a[x]` 保存以 x 结尾、长度为 $\mathrm{lowbit}(x) = x \,\&\, (-x)$ 的块和：`add` 沿 `x += x & -x` 逐级更新覆盖该点的祖先块，`sum` 沿 `x -= x & -x` 把前缀拆成 $O(\log n)$ 个不相交块累加。内部下标从 1 开始（接口里先 `++x`），对外一律 0 下标、区间用左闭右开 $[l, r)$。它只支持有逆元的可加聚合（求和、异或），最大值这类不可逆运算要另用 3.2.005 前缀/后缀最大值 或线段树。附带的 `kth` 借二进制提升，在各点权非负、前缀和单调时按前缀和二分位置，避免「外层二分 + 每步 `sum`」的双 log。这份 `BIT<T>` 是全章树状数组的公共底座，后续条目按编号引用它、不再重复粘贴。

```
template <class T> struct BIT
{
    // n 是数组长度；a[i] 是以内部下标 i 结尾的 lowbit 块和。
    int n;
    vector<T> a; // a[pos(i)] 是 Fenwick 节点 i 管辖区间之和，i 从 1 开始。

    BIT(int n = 0)
    {
        // n 是数组长度；初值全为零。
        init(n);
    }
```

```
12
13      void init(int n_)
14      {
15          // n_ 是新的数组长度；清空旧状态并置零。
16          assert(n_ >= 0); // 调试检查，可删
17          n = n_;
18          a.assign(n + 1, T{});
19      }
20
21      void add(int x, T v)
22      {
23          // x 是 0 下标位置，v 是增量；把 a[x] 增加 v。
24          assert(0 <= x && x < n); // 调试检查，可删
25          for (++x; x <= n; x += x & -x)
26          {
27              a[x] += v;
28          }
29      }
30
31      T sum(int x) const
32      {
33          // x 是前缀右端点；返回 [0,x) 的和。
34          assert(0 <= x && x <= n); // 调试检查，可删
35          T ans{};
36          for (; x; x -= x & -x)
37          {
38              ans += a[x];
39          }
40          return ans;
41      }
42
43      T sum(int l, int r) const
44      {
45          // l、r 是半开区间端点；返回 [l,r) 的和。
46          assert(0 <= l && l <= r && r <= n); // 调试检查，可删
47          return sum(r) - sum(l);
48      }
49
50      int kth(T k) const
51      {
52          // 各点权必须非负且 k>=0；返回满足 sum(x)<=k 的最大前缀长度 x。
53          assert(k >= T{}); // 调试检查，可删
54          int x = 0;
55          T cur{};
56          for (int d = bit_floor((unsigned long long)n); d; d >>= 1)
57          {
58              if (x + d <= n && cur + a[x + d] <= k)
59              {
60                  x += d;
61                  cur += a[x];
62              }
63          }
```

```
        return x;
    }
};
```

当 n 很大且更新、查询密集时，Fenwick 的 2 的幂位置会集中映射到少数缓存组。下面的 `FastBIT` 每隔 1024 个逻辑槽插入一个空洞，只增加约 0.1% 空间，却能打散这类冲突；接口与 `BIT` 相同，并补充了 $O(n)$ 建树：

```
template <class T> struct FastBIT
{
    int n = 0; // n 是逻辑长度（元素个数）。
    vector<T> a;

    static constexpr int pos(int x)
    {
        // x 是 Fenwick 下标；返回每 1024 个槽插入一个空洞后的物理下标。
        return x + (x >> 10);
    }

    FastBIT(int n = 0) { init(n); }

    explicit FastBIT(const vector<T> &v)
    {
        // v 是初值数组；用前缀和 O(n) 建树，a[pos(i)] = pre[i] - pre[i & (i-1)]。
        init(v.size());
        vector<T> pre(n + 1);
        for (int i = 1; i <= n; i++)
        {
            pre[i] = pre[i - 1] + v[i - 1];
            a[pos(i)] = pre[i] - pre[i & (i - 1)];
        }
    }

    void init(int m)
    {
        // m 是元素个数；清空为 m 个零元素。
        n = m;
        a.assign(pos(n) + 1, T{});
    }

    void add(int x, T v)
    {
        // x 是 0 下标位置，v 是增量；把第 x 个元素增加 v。
        assert(0 <= x && x < n); // 调试检查，可删
        for (x++; x <= n; x += x & -x)
        {
            a[pos(x)] += v;
        }
    }

    T sum(int x) const
```

```
    {
        // x 是前缀长度；返回 [0,x) 的和。
        assert(0 <= x && x <= n); // 调试检查，可删
        T ans{};
        for (; x; x &= x - 1)
        {
            ans += a[pos(x)];
        }
        return ans;
    }

    T sum(int l, int r) const
    {
        // l、r 是半开区间端点；返回 [l,r) 的和。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return sum(r) - sum(l);
    }

    int kth(T k) const
    {
        // 各元素必须非负且 k>=0；返回满足 sum(x)<=k 的最大前缀长度 x。
        assert(k >= T{}); // 调试检查，可删
        int x = 0;
        T cur{};
        for (int d = bit_floor((unsigned long long)n); d; d >>= 1)
        {
            if (x + d <= n && cur + a[pos(x + d)] <= k)
            {
                x += d;
                cur += a[pos(x)];
            }
        }
        return x;
    }
};
```

`BIT<T> b(n)`：`add(x,v)` 给 0 下标位置 x 加 v，`sum(x)` 返回前缀 $[0,x)$ 之和，`sum(l,r)` 返回 $[l,r)$，`kth(k)` 返回满足 `sum(x)<=k` 的最大前缀长度 $x\in[0,n]$，要求各点权非负且 $k\geq 0$；若总和也不超过 k，结果就是 n。全部操作 $O(\log n)$，构造与空间 $O(n)$。`FastBIT<T>` 语义完全相同，由数组构造时是 $O(n)$；本机两百万次随机更新/前缀查询的 7 轮中位数约为 `29ms`，普通连续布局约为 `69ms`。小数据优先用更短的 `BIT`，大数据热路径再换 `FastBIT`。需要单点赋值而非增量见 3.2.002 单点赋值、区间和，区间加与区间和改用 3.2.004 区间修改、区间求和，二维点加矩形和用 3.2.008 二维树状数组、矩形加矩形和升级为 3.2.009 矩形修改、矩形求和；只有不可逆的单调最大值才选 3.2.005 前缀/后缀最大值。

3.2.002 单点赋值、区间和

树状数组天生做增量，要支持「把某点赋成新值」需要一层薄封装：额外用 `a[x]` 记住每个位置的当前值，赋值时把它翻译成增量 `v - a[x]` 喂给底层 BIT，再更新 `a[x]`。关键坑是绝不能直接覆盖 `bit[x]`——那样会破坏所有包含该点的 lowbit 块；必须走「差值 + `add`」这条路。底层直接复用 3.2.001 单点修改、前缀

和 的 BIT<T>。

```cpp
// 依赖 3.2.001 单点修改、前缀和
template <class T> struct AssignBIT
{
    // n 保存数组长度；a 保存每个位置的当前值；bit 维护增量的前缀和。
    int n;
    vector<T> a;
    BIT<T> bit;

    AssignBIT(int n = 0) : n(n), a(n), bit(n)
    {
        // n 表示数组长度；初始值全部为零。
        assert(n >= 0); // 调试检查，可删
    }

    void set(int x, T v)
    {
        // x 表示 0 下标位置，v 表示新值；把赋值转成增量 v-a[x]。
        assert(0 <= x && x < n); // 调试检查，可删
        bit.add(x, v - a[x]);
        a[x] = v;
    }

    T sum(int l, int r) const
    {
        // l、r 表示半开区间 [l,r)；返回区间和。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return bit.sum(l, r);
    }
};
```

`AssignBIT<T> b(n)`（初值全零）：`set(x,v)` 把 0 下标位置 x 赋成 v，`sum(l,r)` 查 $[l,r)$。两者都是 $O(\log n)$，额外 $O(n)$ 存当前值。若本就只做增量、不需要赋值语义，直接用 3.2.001 单点修改、前缀和 更省一份数组。

3.2.003 前缀和上的二分

给定目标前缀和，在树状数组上直接沿二进制层级定位位置，比「外层二分 + 每步 `sum`」少一个 log。这一功能已经内建为 3.2.001 单点修改、前缀和 的 `BIT<T>::kth`，不另立代码：从最高位 $d = \text{bit_floor}(n)$ 向下试探，每步判断把当前下标右移 d 后累计的前缀和是否仍不超过目标 k，是则接受该跳跃。原理依赖前缀和单调，因此各点权必须非负，出现负权时前缀和不单调、二进制提升失效。`kth(k)` 返回使前 x 个元素之和不超过 k 的最大下标 x，单次 $O(\log n)$。典型用途是权值树状数组上求第 k 小、区间排名，功能更全的静态区间第 k 小见 3.4.006 区间第 K 小、排名、前驱后继。

3.2.004 区间修改、区间求和

要让区间加与区间求和都只需对数时间，对差分序列 $d_i = a_i - a_{i-1}$ 下手：区间加 $[l, r)$ $+= v$ 在差分上只改两个端点 d_l $+= v$、d_r $-= v$。前缀和 $\sum_{i<x} a_i = \sum_{i<x}(x-i)d_i = x\sum_{i<x} d_i - \sum_{i<x} i\,d_i$，于是维护两个 BIT——`bit0` 存 d_i、`bit1` 存 $i\,d_i$——即可由 $x \cdot \mathrm{sum}(bit0, x) - \mathrm{sum}(bit1, x)$ 还原任意前缀。区间加只在两端各更新两棵树，区间和是两个前缀相减。它比单纯用一棵 BIT 多存一倍空间，换来了区间修改能力。

```
template <class T> struct RangeFenwick
{
    // n 是数组长度；bit0 维护 d[i]，bit1 维护 i*d[i]。
    int n;
    vector<T> bit0, bit1;

    void add(vector<T> &bit, int x, T value)
    {
        // bit 是一棵内部 BIT，x 是差分下标，value 是增量。
        for (++x; x <= n; x += x & -x)
        {
            bit[x] += value;
        }
    }

    T sum(const vector<T> &bit, int x) const
    {
        // bit 是一棵内部 BIT；返回其前 x 个差分量之和。
        T result{};
        for (; x > 0; x -= x & -x)
        {
            result += bit[x];
        }
        return result;
    }

    explicit RangeFenwick(int n = 0) : n(n), bit0(n + 1), bit1(n + 1)
    {
        // n 是数组长度；初值全为零。
        assert(n >= 0); // 调试检查，可删
    }

    explicit RangeFenwick(const vector<T> &a) : RangeFenwick((int)a.size())
    {
        // a 是初始数组；逐点加入其初值。
        for (int i = 0; i < n; ++i)
        {
            rangeAdd(i, i + 1, a[i]);
        }
    }

    void rangeAdd(int l, int r, T value)
    {
```

```
        // l、r 是半开区间端点，value 是增量；给 [l,r) 整段增加 value。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        add(bit0, l, value);
        add(bit0, r, -value);
        add(bit1, l, value * l);
        add(bit1, r, -value * r);
    }

    T prefixSum(int r) const
    {
        // r 是前缀右端点；返回 [0,r) 的元素和。
        assert(0 <= r && r <= n); // 调试检查，可删
        return sum(bit0, r) * r - sum(bit1, r);
    }

    T rangeSum(int l, int r) const
    {
        // l、r 是半开区间端点；返回 [l,r) 的元素和。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return prefixSum(r) - prefixSum(l);
    }
};
```

`RangeFenwick<T> b(n)` 或由 `vector<T>` 构造（逐点 `rangeAdd`）：`rangeAdd(l,r,v)` 给 $[l,r)$ 每个元素加 v，`rangeSum(l,r)` 查区间和，`prefixSum(r)` 查前缀 $[0,r)$；端点断言 $0 \le l \le r \le n$。`T` 必须能表示负差分以及所有 id_i、前缀和中间量和答案，需要宽和时直接选用更宽的 `T`；由于实现会保存负差分，不应使用无符号整数。从 `vector` 构造为 $O(n \log n)$，其余操作 $O(\log n)$，空间 $O(n)$。只需单点改、区间查时退回 3.2.001 单点修改、前缀和 省一半常数；要在区间加之上再做区间最值、区间历史最值等复杂标记时，树状数组无能为力，应换 3.3.002 懒标记线段树 或 3.3.004 区间最值与历史最值。

3.2.005 前缀/后缀最大值

最大值没有逆元，所以树状数组维护前缀最大值时只能「单点抬高」而不能「任意改小」：`chmax` 沿 `x += x & -x` 把途经块与新值取 max，`maxPrefix` 沿 `x -= x & -x` 合并覆盖前缀的块最大值。若某次要把某点值调小，旧的最大值无法从块里减掉，必须重建受影响块或直接换线段树——因此本结构仅适用于点值单调不减的场景（如按位置维护 LIS 的 dp 值）。构造时的幺元 `e` 取类型最小值，作为空查询返回。改成前缀最小值时，成对替换 `max/lowest/chmax/maxPrefix` 为 `min/numeric_limits<T>::max()/chmin/minPrefix`，并把适用条件改为点值单调不增；不能只替换查询函数。

```
template <class T> struct MaxBIT
{
    // 当前是“单点只增大、前缀取最大值”版。改成最小值版时，
    // 将 max/lowest/chmax/maxPrefix 分别换成 min/numeric_limits<T>::max()/chmin/minPrefix，且更
    ↪ 新方向改为单点只减小。
    // n 是数组长度，e 是空前缀的幺元，a[i] 是 lowbit 块最大值。
    int n;
    T e;
    vector<T> a;
```

```

    MaxBIT(int n = 0, T e = numeric_limits<T>::lowest()) : n(n), e(e), a(n + 1, e)
    {
        // n 是数组长度，e 是无元素贡献时的返回值。
        assert(n >= 0); // 调试检查，可删
    }

    // 只支持把单点值增大；若需要任意修改应改用线段树。
    void chmax(int x, T v)
    {
        // x 是 0 下标位置，v 是候选新值；令该点不低于 v。
        assert(0 <= x && x < n); // 调试检查，可删
        for (++x; x <= n; x += x & -x)
        {
            a[x] = max(a[x], v);
        }
    }

    T maxPrefix(int r) const
    {
        // r 是前缀右端点；返回 [0,r) 的最大值。
        assert(0 <= r && r <= n); // 调试检查，可删
        T ans = e;
        for (; r; r -= r & -r)
        {
            ans = max(ans, a[r]);
        }
        return ans;
    }
};
```

`MaxBIT<T> b(n, e)`（`e` 默认取 `numeric_limits<T>::lowest()`）：`chmax(x,v)` 把 0 下标位置 x 的值上抬到不低于 v，`maxPrefix(r)` 返回前缀 $[0,r)$ 的最大值（注意接口按前缀最大值实现，只能查前缀而非任意区间）。两操作均 $O(\log n)$，空间 $O(n)$。一旦需要任意方向的单点修改或任意区间最值，就必须换 3.3.001 单点修改线段树与摘要二分；求和类需求回到 3.2.001 单点修改、前缀和。

3.2.006 后缀和

有时更自然的查询是「从某位置到末尾」的后缀和。与其重写一套反向循环，不如把 lowbit 的推进方向对调：`add` 沿 `x -= x & -x` 更新，`sumSuffix` 沿 `x += x & -x` 累加，等价于在反转下标上跑标准 BIT，原数组后缀恰好对应 BIT 前缀。这样修改与查询共用同一套 lowbit 逻辑，不必同时维护两套相反循环。

```
template <class T> struct SuffixBIT
{
    // n 是数组长度；a[i] 是反向 lowbit 块和。
    int n;
    vector<T> a;

    SuffixBIT(int n = 0) : n(n), a(n + 1)
```

```
    {
        // n 是数组长度；初值全为零。
        assert(n >= 0); // 调试检查，可删
    }

    void add(int x, T v)
    {
        // x 是 0 下标位置，v 是增量；把该点增加 v。
        assert(0 <= x && x < n); // 调试检查，可删
        for (++x; x; x -= x & -x)
        {
            a[x] += v;
        }
    }

    T sumSuffix(int l) const
    {
        // l 是后缀左端点；返回 [l,n) 的和，允许 l=n。
        assert(0 <= l && l <= n); // 调试检查，可删
        T ans{};
        for (++l; l <= n; l += l & -l)
        {
            ans += a[l];
        }
        return ans;
    }

    T sum(int l, int r) const
    {
        // l、r 是半开区间端点；返回 [l,r) 的和。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return sumSuffix(l) - sumSuffix(r);
    }
};
```

`SuffixBIT<T> b(n)`：`add(x,v)` 给 0 下标位置 x 加 v，`sumSuffix(l)` 返回后缀 $[l,n)$ 之和，`sum(l,r)` 返回 $[l,r)$（由两个后缀相减）。均 $O(\log n)$，空间 $O(n)$。本质与 3.2.001 单点修改、前缀和 完全对称，仅方向相反；同时要前缀又要后缀时，直接用一棵普通 BIT 加「总和减前缀」即可，无需两棵。

3.2.007 取模树状数组

这是一个配置型条目，不引入新结构：树状数组只要求值类型带零元、加法、减法，于是普通求和与模意义求和共用同一份 3.2.001 单点修改、前缀和 的 `BIT<T>`——`BIT<int>` 存普通整数，`BIT<MInt>`（模整数类型自动把每次加减归一到 $[0,\mathrm{mod})$）存模意义前缀和，二者的结构、lowbit 推进与复杂度完全一致，不该复制第二份 BIT，只用一行 `using` 起个语义别名。

```
// 依赖 3.2.001 单点修改、前缀和
template <class MInt> using ModBIT = BIT<MInt>;
```

`ModBIT<MInt> b(n)` 即 `BIT<MInt>`，接口与 3.2.001 单点修改、前缀和 相同，各操作 $O(\log n)$、空间

$O(n)$；模板参数换成 `int` 便退回普通版。举例：模 7、依次在下标 $0,2,2$ 加 $5,6,4$，则 `sum(0,3)` 得 $(5+0+10) \bmod 7 = 1$、区间 $[1,3)$ 之和为 $10 \bmod 7 = 3$。模整数类型 `MInt` 的完整实现见数论 / 组合数学章。

3.2.008 二维树状数组

二维点集上的单点加权、矩形和：把一维 lowbit 套两层，节点 `tr[i][j]` 保存两段 lowbit 张成的小矩形和，修改在两维都沿 `+= lowbit` 上行、前缀查询在两维都沿 `-= lowbit` 下行，任意矩形和由四个二维前缀容斥。坐标范围大时不能开满二维数组，故这里采用「离线离散 + 树套树」：先由所有可能的更新点确定外层横坐标离散表，再对每个外层 BIT 节点 i 收集它会访问到的纵坐标，各自离散成一条内层 BIT；因此更新点必须在构造时全部给出，之后 `add` 的 (x, y) 只能落在预登记的坐标上。

```
class BIT2
{
    // xs 是可更新横坐标；ys[i] 是外层节点 i 收集的纵坐标。
    vector<int> xs;
    vector<vector<int>> ys;
    // tr[i] 是外层节点 i 对应的纵坐标 BIT。
    vector<vector<long long>> tr;

  public:
    explicit BIT2(const vector<pair<int, int>> &pts = {})
    {
        // pts 列出所有可能发生更新的二维坐标。
        prepare(pts);
    }

    void prepare(const vector<pair<int, int>> &pts)
    {
        // pts 列出所有可能更新点；清空旧权值并重建两层离散索引。
        xs.clear();
        for (auto [x, y] : pts)
        {
            xs.push_back(x);
        }
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()), xs.end());
        int n = xs.size();
        ys.assign(n + 1, {});
        for (auto [x, y] : pts)
        {
            int p = lower_bound(xs.begin(), xs.end(), x) - xs.begin() + 1;
            for (int i = p; i <= n; i += i & -i)
            {
                ys[i].push_back(y);
            }
        }
        tr.resize(n + 1);
        for (int i = 1; i <= n; ++i)
        {
```

```
            auto &v = ys[i];
            sort(v.begin(), v.end());
            v.erase(unique(v.begin(), v.end()), v.end());
            tr[i].assign(v.size() + 1, 0);
        }
    }

    void add(int x, int y, long long v)
    {
        // (x,y) 必须在构造时登记，v 是该点的权值增量。
        int n = xs.size();
        auto it = lower_bound(xs.begin(), xs.end(), x);
        assert(it != xs.end() && *it == x); // 调试检查，可删
        int p = it - xs.begin() + 1;
        for (int i = p; i <= n; i += i & -i)
        {
            auto jt = lower_bound(ys[i].begin(), ys[i].end(), y);
            assert(jt != ys[i].end() && *jt == y); // 调试检查，可删
            int q = jt - ys[i].begin() + 1;
            for (int j = q; j < (int)tr[i].size(); j += j & -j)
            {
                tr[i][j] += v;
            }
        }
    }

    long long prefixSum(int x, int y) const
    {
        // x、y 是开前缀上界；返回横坐标小于 x 且纵坐标小于 y 的点权和。
        int p = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
        long long ans = 0;
        for (int i = p; i > 0; i -= i & -i)
        {
            int q = lower_bound(ys[i].begin(), ys[i].end(), y) - ys[i].begin();
            for (int j = q; j > 0; j -= j & -j)
            {
                ans += tr[i][j];
            }
        }
        return ans;
    }

    void addRectangle(int l, int d, int r, int u, long long w)
    {
        // [l,r) x [d,u) 是半开加权矩形，w 是增量；四角必须已在构造时预登记。
        assert(l < r && d < u); // 调试检查，可删
        add(l, d, w);
        add(l, u, -w);
        add(r, d, -w);
        add(r, u, w);
    }

```

```
    long long pointGet(int x, int y) const
    {
        // (x,y) 是查询点；返回所有包含该点的半开加权矩形的权值和。
        return prefixSum(x + 1, y + 1);
    }
};
```

`BIT2 b(updatePoints)` 传入所有将被 `add` 的坐标（可含重复）；`add(x,y,v)` 在预登记点 (x,y) 加 v，`prefixSum(X,Y)` 返回 $x<X\wedge y<Y$ 的点权和，矩形 $[x_1,x_2)\times[y_1,y_2)$ 用四个 `prefixSum` 容斥。若反复需要“矩形加、单点值”这一固定组合，`addRectangle(l,d,r,u,w)` 会将严格非空半开矩形拆成四个差分角，`pointGet(x,y)` 返回其在整数点 (x,y) 的差分前缀；四个角同样必须预登记，查询点不必登记。它们不增加第二棵树或额外状态，只避免四角符号、端点开闭写错。设更新点数 q，离线建表 $O(q\log^2 q)$，`add`、`prefixSum`、`addRectangle` 与 `pointGet` 均为 $O(\log^2 q)$，空间 $O(q\log q)$。它原生支持单点加、矩形和；矩形加、单点值可用上述差分入口，矩形加、矩形和需要四棵二维 BIT 做二维差分，见 3.2.009 矩形修改、矩形求和。坐标可完全在线、还要带修矩形最值时改用 3.3.013 矩形面积并、周长并 一类扫描线或 KD-Tree 3.7.004 K-D Tree。

3.2.009 矩形修改、矩形求和

矩形加、矩形和：对二维差分 $d_{x,y}$ 下手，一次矩形加 $[x_1,x_2)\times[y_1,y_2)\mathrel{+}=w$ 拆成四个角上的差分事件。二维前缀和 $\sum_{x<X,\,y<Y}a=\sum(X-x)(Y-y)d_{x,y}$ 展开为 $XY\sum d-X\sum yd-Y\sum xd+\sum xyd$ 四项，于是需维护四棵二维 BIT 分别累计 d、yd、xd、xyd，查询时按上式组合再对矩形做四角容斥。本实现走完全离线：把所有矩形加与查询各拆成四个角点，按 x 排成扫描线、纵坐标离散后只用四棵一维 BIT 边扫边累加，每个查询结果按 $\pm$ 系数汇总。

```
template <class T> struct RectAddSum
{
    struct R
    {
        // l、d、r、u 依次是加权矩形的左、下、右、上边界。
        long long l, d, r, u;
        // w 是矩形内每个位置增加的权值。
        T w;
    };

    struct Q
    {
        // l、d、r、u 依次是询问矩形的左、下、右、上边界。
        long long l, d, r, u;
    };

    // a 保存全部矩形加操作，q 保存全部矩形和询问。
    vector<R> a;
    vector<Q> q;

    void add(long long l, long long d, long long r, long long u, T w)
    {
        // 给半开矩形 [l,r) x [d,u) 内每个位置增加 w；退化矩形无操作。
```

```
        if (l < r && d < u)
        {
            a.push_back({l, d, r, u, w});
        }
    }

    int query(long long l, long long d, long long r, long long u)
    {
        // 登记半开矩形 [l,r) x [d,u) 的权值和询问并返回询问编号。
        q.push_back({l, d, r, u});
        return q.size() - 1;
    }

    vector<T> work() const
    {
        // 统一处理全部操作；返回值按 query 的登记编号排列。
        struct E
        {
            // x、y 是二维差分事件坐标，w 是该点的带符号权值。
            long long x, y;
            T w;
        };
        struct P
        {
            // x、y 是前缀询问上界；id 是原询问编号，s 是容斥符号。
            long long x, y;
            int id, s;
        };

        vector<E> e;
        vector<P> p;
        vector<long long> ys;
        auto addE = [&](long long x, long long y, T w)
        {
            e.push_back({x, y, w});
            ys.push_back(y);
        };
        for (auto [l, d, r, u, w] : a)
        {
            addE(l, d, w);
            addE(l, u, -w);
            addE(r, d, -w);
            addE(r, u, w);
        }
        auto addP = [&](long long x, long long y, int id, int s)
        {
            p.push_back({x, y, id, s});
            ys.push_back(y);
        };
        for (int i = 0; i < (int)q.size(); i++)
        {
            auto [l, d, r, u] = q[i];
```

```
            addP(r, u, i, 1);
            addP(l, u, i, -1);
            addP(r, d, i, -1);
            addP(l, d, i, 1);
        }
        sort(e.begin(),
             e.end(),
             [](const E &x, const E &y)
             {
                 return x.x < y.x;
             });
        sort(p.begin(),
             p.end(),
             [](const P &x, const P &y)
             {
                 return x.x < y.x;
             });
        sort(ys.begin(), ys.end());
        ys.erase(unique(ys.begin(), ys.end()), ys.end());

        struct B
        {
            // a 保存一棵扫描线纵轴 BIT 的 lowbit 块和。
            vector<T> a;

            B(int n) : a(n + 1)
            {
                // n 是离散纵坐标数；初值全为零。
            }
            void add(int x, T v)
            {
                // x 是离散纵坐标，v 是增量；更新该点。
                for (++x; x < (int)a.size(); x += x & -x)
                {
                    a[x] += v;
                }
            }
            T sum(int x) const
            {
                // x 是前缀长度；返回前 x 个离散位置的和。
                T s{};
                for (; x; x -= x & -x)
                {
                    s += a[x];
                }
                return s;
            }
        };

        array<B, 4> b{B(ys.size()), B(ys.size()), B(ys.size()), B(ys.size())};
        vector<T> ans(q.size());
        int k = 0;
```

```
        for (auto z : p)
        {
            while (k < (int)e.size() && e[k].x < z.x)
            {
                auto [x, y, w] = e[k++];
                int v = lower_bound(ys.begin(), ys.end(), y) - ys.begin();
                b[0].add(v, w);
                b[1].add(v, w * T(y));
                b[2].add(v, w * T(x));
                b[3].add(v, w * T(x) * T(y));
            }
            int y = lower_bound(ys.begin(), ys.end(), z.y) - ys.begin();
            T v = T(z.x) * T(z.y) * b[0].sum(y) - T(z.x) * b[1].sum(y) - T(z.y) * b[2].sum(y) +
 ↪  b[3].sum(y);
            ans[z.id] += T(z.s) * v;
        }
        return ans;
    }
};
```

用法分两阶段：先用 `add(x1,y1,x2,y2,w)` 登记矩形加、`query(x1,y1,x2,y2)` 登记查询（返回该查询的编号），登记完调用 `work()` 一次性离线算出 `vector<T>`，其下标即 `query` 的返回值。区间左闭右开，退化的加法矩形被忽略，退化询问自然得到零；坐标与权值的乘积、中间和及答案都必须能由 `T` 表示。设矩形数 n、查询数 q，时间 $O((n+q)\log(n+q))$、空间 $O(n+q)$。举例：先对 $[0,2)\times[0,3)$ 加 4、再对 $[1,4)\times[2,5)$ 加 -2，查询 $[0,4)\times[0,5)$、$[1,2)\times[2,3)$、空矩形，`work()` 依次返回 $6,2,0$。只做单点加、矩形和时用离线树套树 3.2.008 二维树状数组 更省；强制在线的矩形加矩形和需要二维树套树或其他在线二维结构，本条不能处理依赖前一次答案的操作。

3.2.010 静态点集矩形和

静态带权点集的矩形和只需一次离线扫描，不该用查询最坏 $O(n)$ 的 KD-Tree。把每个半开矩形 $[l,r)\times[d,u)$ 拆为横向前缀差：横坐标小于 r 的纵向区间和减去横坐标小于 l 的纵向区间和。点和前缀事件都按横坐标排序，扫描到事件 X 时加入所有 $x<X$ 的点；点的纵坐标离散后，一棵 Fenwick 维护已加入点的权值，两个 `lower_bound` 得到 $[d,u)$。因此端点相同的点不会被错误计入左闭右开边界。

```
template <class T> struct StaticRect
{
    struct P
    {
        // x、y 是静态点坐标，w 是该点权值。
        long long x, y;
        T w;
    };

    struct Q
    {
        // l、d、r、u 依次是半开查询矩形的左、下、右、上边界。
        long long l, d, r, u;
    };
```

```
15
16      // p 保存全部静态带权点，q 保存全部待回答的矩形询问。
17      vector<P> p;
18      vector<Q> q;
19
20      void add(long long x, long long y, T w)
21      {
22          // 把坐标为 (x,y)、权值为 w 的静态点加入点集。
23          p.push_back({x, y, w});
24      }
25
26      int query(long long l, long long d, long long r, long long u)
27      {
28          // 登记半开矩形 [l,r) x [d,u) 的点权和询问，返回其编号。
29          q.push_back({l, d, r, u});
30          return (int)q.size() - 1;
31      }
32
33      vector<T> work() const
34      {
35          // 离线回答全部登记询问，返回值按 query 的返回编号排列。
36          struct E
37          {
38              // x 是扫描线横坐标；d、u 是纵向半开范围；id 是询问编号，sgn 是容斥符号。
39              long long x, d, u;
40              int id, sgn;
41          };
42
43          vector<P> a = p;
44          vector<E> e;
45          e.reserve(2 * q.size());
46          for (int i = 0; i < (int)q.size(); ++i)
47          {
48              auto [l, d, r, u] = q[i];
49              e.push_back({l, d, u, i, -1});
50              e.push_back({r, d, u, i, 1});
51          }
52          sort(a.begin(), a.end(), [](const P &x, const P &y)
53               {
54                   return x.x < y.x;
55               });
56          sort(e.begin(), e.end(), [](const E &x, const E &y)
57               {
58                   return x.x < y.x;
59               });
60
61          vector<long long> ys;
62          ys.reserve(a.size());
63          for (auto [x, y, w] : a)
64          {
65              ys.push_back(y);
66          }
```

```
        sort(ys.begin(), ys.end());
        ys.erase(unique(ys.begin(), ys.end()), ys.end());
        vector<T> bit(ys.size() + 1);
        auto addBit = [&](int x, T w)
        {
            // x 是 0 下标离散纵坐标，w 是加入该点的权值。
            for (++x; x < (int)bit.size(); x += x & -x)
            {
                bit[x] += w;
            }
        };
        auto sumBit = [&](int x)
        {
            // x 是离散前缀长度，返回所有离散下标小于 x 的权值和。
            T ans{};
            for (; x; x -= x & -x)
            {
                ans += bit[x];
            }
            return ans;
        };

        vector<T> ans(q.size());
        int at = 0;
        for (auto [x, d, u, id, sgn] : e)
        {
            while (at < (int)a.size() && a[at].x < x)
            {
                int y = lower_bound(ys.begin(), ys.end(), a[at].y) - ys.begin();
                addBit(y, a[at].w);
                ++at;
            }
            int l = lower_bound(ys.begin(), ys.end(), d) - ys.begin();
            int r = lower_bound(ys.begin(), ys.end(), u) - ys.begin();
            T cur = sumBit(r);
            cur -= sumBit(l);
            if (sgn == 1)
            {
                ans[id] += cur;
            }
            else
            {
                ans[id] -= cur;
            }
        }
        return ans;
    }
};
```

先用 `add(x,y,w)` 登记全部静态点，再用 `query(l,d,r,u)` 登记半开矩形并取得编号，最后调用一次 `work()`；答案向量按编号排列。点可重合、权值可为负，`T` 必须默认构造且支持 `+=`、`-=`；坐标、累计权值和答案均须能由相应类型表示。设点数为 n、询问数为 q，排序和扫描总时间为 $O((n+q)\log(n+q))$，额外空

间为 $O(n+q)$。需要在线单点加时用 3.2.008 二维树状数组，需要矩形加则用 3.2.009 矩形修改、矩形求和，静态点集但允许较松平均复杂度的多种几何查询才考虑 3.7.004 K-D Tree。

3.2.011 矩形加、单点求值

这是 3.2.008 二维树状数组 的四角差分入口，不另造第二棵结构。一次半开矩形 $[l,r)\times[d,u)$ 加 w，在二维差分上写 $(l,d):+w$、$(l,u):-w$、$(r,d):-w$、$(r,u):+w$；因此任意整数点 (x,y) 的当前值正是所有横坐标不超过 x、纵坐标不超过 y 的差分点权和。离线先把初始矩形及每条 0 型更新的四个角都收集进 `updatePoints`，再构造 `BIT2`；查询点无需登记。

```
// updatePoints 已收集每个初始和未来矩形的四个角。
BIT2 bit(updatePoints);
bit.addRectangle(l, d, r, u, w); // 给 [l,r) x [d,u) 加 w。
auto ans = bit.pointGet(x, y);   // 取整数点 (x,y) 的当前值。
```

`addRectangle(l,d,r,u,w)` 要求 $l<r, d<u$，四个角必须在构造 `BIT2` 时预登记；`pointGet(x,y)` 不要求查询点预登记，按题面半开边界返回恰好包含该点的所有矩形权值和。设 U 是去重前的全部矩形角数，预处理 $O(U\log^2 U)$，每次矩形加与点查询均为 $O(\log^2 U)$，空间 $O(U\log U)$；答案及中间差分和必须可由 `long long` 表示。要查询整个矩形的加权和而不只是单点，改用 3.2.009 矩形修改、矩形求和；若矩形或查询坐标不能预读，当前离线树套树不适用。

3.3 线段树家族

3.3.001 单点修改线段树与摘要二分

维护满足结合律的区间信息，支持单点修改与半开区间查询：叶子挂在不小于 n 的最小二次幂基址下，父节点存左右儿子合并的结果，`set` 只沿根到该叶的链自底向上重算，`query` 把 $[l,r)$ 拆成 $O(\log n)$ 段。查询里用 `x`、`y` 分别累计左边界与右边界扫上来的贡献，最后 `x + y`，这样即使合并不满足交换律（最大子段和、区间赋值后的仿射信息等）也能保持左右顺序正确。信息类型 `Info` 只需给出合并单位元（默认构造）与 `operator+`，示例 `Sum` 维护区间和，`MaxInfo` 可维护区间最大值。`findFirst` / `findLast` 是在同一棵树上的摘要搜索：谓词返回假时，必须保证该节点内没有答案；例如 `MaxInfo` 查第一个不小于 `x` 的位置时传入 `info.val >= x`。搜索遇到完全落入 $[l,r)$ 的节点时先用摘要剪枝，随后分别左优先或右优先递归，和 jiangly 模板的剪枝语义一致。这是自底向上的迭代维护配递归搜索，空间 $O(2\,\mathrm{pow2}(n))=O(n)$，代价是不带懒标记、仅支持单点改。

```
// 区间和实例：Sum{} 是合并单位元，Sum{x} 表示一个值为 x 的叶子。
struct Sum
{
    // val 是当前区间的元素和。
    long long val = 0;

    Sum() = default;
    Sum(long long x) : val(x)
    {
        // x 是单个叶子的值。
    }
```

```cpp

    friend Sum operator+(Sum a, Sum b)
    {
        // a、b 是相邻区间信息；返回合并后的区间和。
        return a.val + b.val;
    }
};

struct MaxInfo
{
    // 当前信息取区间最大值。改成最小值信息时，把 lowest 改为 numeric_limits<long long>::max()、max 改为 min，
    // 并同步把类型名、查询谓词中的 >= 改成 <=。
    // val 是区间最大值；空区间以 long long 最小值作为 max 的单位元。
    long long val = numeric_limits<long long>::lowest();

    MaxInfo() = default;
    MaxInfo(long long x) : val(x)
    {
        // x 是单个叶子的值。
    }

    friend MaxInfo operator+(MaxInfo a, MaxInfo b)
    {
        // a、b 是按顺序相邻的区间信息；返回两段的最大值。
        return max(a.val, b.val);
    }
};

template <class Info> struct Seg
{
    // n 是真实叶子数，size 是不小于 n 的二次幂；tr 保存区间信息。
    int n;
    int size;
    vector<Info> tr;

    Seg(int n = 0)
    {
        // n 是叶子数；初始信息均为合并单位元。
        init(n);
    }
    Seg(const vector<Info> &a)
    {
        // a 是各叶子的初始信息。
        init(a);
    }

    void init(int n_)
    {
        // n_ 是新的叶子数；幂次二基址让每个内部节点对应连续区间。
        assert(n_ >= 0); // 调试检查，可删
        n = n_;
```

```
        size = 1;
        while (size < n)
        {
            size <<= 1;
        }
        tr.assign(2 * size, Info{});
    }

    void init(const vector<Info> &a)
    {
        // a 是新的叶子信息；线性重建整棵树。
        init(a.size());
        copy(a.begin(), a.end(), tr.begin() + size);
        for (int i = size - 1; i > 0; --i)
        {
            pull(i);
        }
    }

    void pull(int p)
    {
        // p 是内部节点编号；按左右顺序合并两个儿子。
        tr[p] = tr[2 * p] + tr[2 * p + 1];
    }

    void set(int p, Info v)
    {
        // p 是 0 下标位置，v 是新叶子信息；修改后重算祖先。
        assert(0 <= p && p < n); // 调试检查，可删
        for (tr[p += size] = v; p >>= 1;)
        {
            pull(p);
        }
    }

    Info query(int l, int r) const
    {
        // l、r 是半开区间端点；按原顺序返回 [l,r) 的合并结果。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        Info x{}, y{};
        for (l += size, r += size; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1)
            {
                x = x + tr[l++];
            }
            if (r & 1)
            {
                y = tr[--r] + y;
            }
        }
        return x + y;
```

```
    }

    template <class F>
    int findFirst(int p, int l, int r, int ql, int qr, F &ok) const
    {
        // p 管辖 [l,r)，查询区间 [ql,qr)，ok 对区间信息单调（父真则至少一个儿子真）；
        // 返回 [ql,qr) 内使 ok(tr[叶]) 为真的最小下标，不存在返回 -1。
        if (r <= ql || qr <= l)
        {
            return -1;
        }
        if (ql <= l && r <= qr && !ok(tr[p]))
        {
            return -1;
        }
        if (r - l == 1)
        {
            return l < n ? l : -1;
        }
        int m = l + (r - l) / 2;
        int x = findFirst(2 * p, l, m, ql, qr, ok);
        return x == -1 ? findFirst(2 * p + 1, m, r, ql, qr, ok) : x;
    }

    template <class F>
    int findFirst(int l, int r, F ok) const
    {
        // l、r 是半开区间，ok(Info) 判断区间是否含目标；返回最左满足的叶子下标，不存在返回 -1。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return l == r ? -1 : findFirst(1, 0, size, l, r, ok);
    }

    template <class F>
    int findLast(int p, int l, int r, int ql, int qr, F &ok) const
    {
        // 参数同 findFirst 的递归版；返回 [ql,qr) 内使 ok 为真的最大下标，不存在返回 -1。
        if (r <= ql || qr <= l)
        {
            return -1;
        }
        if (ql <= l && r <= qr && !ok(tr[p]))
        {
            return -1;
        }
        if (r - l == 1)
        {
            return l < n ? l : -1;
        }
        int m = l + (r - l) / 2;
        int x = findLast(2 * p + 1, m, r, ql, qr, ok);
        return x == -1 ? findLast(2 * p, l, m, ql, qr, ok) : x;
    }
```

```
167
168     template <class F>
169     int findLast(int l, int r, F ok) const
170     {
171         // l、r 是半开区间，ok(Info) 判断区间是否含目标；返回最右满足的叶子下标，不存在返回 -1。
172         assert(0 <= l && l <= r && r <= n); // 调试检查，可删
173         return l == r ? -1 : findLast(1, 0, size, l, r, ok);
174     }
175 };
```

用 `Seg<Sum> seg(a)` 由 `vector<Sum>` 建树，构造 $O(n)$；`set(p, v)` 把下标 p 改成 `Info v`，`query(l, r)` 返回 $[l, r)$ 的合并结果，`set` 和 `query` 为 $O(\log n)$，空间 $O(n)$。`findFirst`/`findLast` 只要求“谓词为假就一定无解”时正确，但允许摘要假阳性会使最坏时间退化为 $O(n)$；若 `ok(info)` 准确等价于该节点内存在答案，例如区间最大值能准确判断是否存在不小于 x 的元素，则搜索只走一条成功路径及其被剪掉的兄弟，时间为 $O(\log n)$。懒标记版本需要在搜索递归前 `push`，不能直接套本条函数。它与 3.3.005 ZKW 线段树 是同一类固定规模、单点修改、结合律查询的结构，ZKW 常数更小；一旦需要区间整体修改就得上懒标记，见 3.3.002 懒标记线段树；值域巨大且访问稀疏改 3.3.007 动态开点线段树，要查历史版本改 3.3.008 可持久化线段树 / 主席树。区间 chmin/chmax 不是普通懒标记，必须用 3.3.011 Segment Tree Beats。

`ok` 是调用者传入的摘要谓词，例如查第一个不小于 4 的位置：

```
1 signed main()
2 {
3     Seg<MaxInfo> seg(vector<MaxInfo>{{2}, {5}, {1}, {4}});
4     int p = seg.findFirst(0, 4, [](MaxInfo x)
5     {
6         return x.val >= 4;
7     });
8     assert(p == 1);
9 }
```

3.3.002 懒标记线段树

要支持区间整体修改，需在递归线段树上挂懒标记：一次落到完整整段的修改先把效果 `apply` 到该节点的聚合信息、并把标记复合进 `tag`，直到后续要递归进儿子前才 `push` 下传。正确性系于两条：`Info::apply(Tag)` 必须能仅凭区间长度 `len` 把标记作用到聚合值（所以 `Info` 存了 `len`），且标记复合 `Tag::apply(Tag)` 的方向要与真实执行时间一致——「先有旧标记、再叠新标记」。这里以区间加、区间和为例，换题一般只改 `Tag` 与 `Info` 两个结构（例如加入乘法、赋值时复合规则更复杂，见下一条）。用 $4n$ 结点的堆式存储。

```
1 // 区间加、区间和实例。换题时通常只需要修改这两个结构。
2 struct Tag
3 {
4     // add 是尚未下传的区间增量。
5     long long add = 0;
6
7     // 先前标记后再执行 t：区间加直接累加。
8     void apply(const Tag &t)
```

```
    {
        // t 是后执行的区间加标记；与当前标记复合。
        add += t.add;
    }
};

struct Info
{
    // sum 是区间和，len 是区间叶子数。
    long long sum = 0;
    int len = 1;

    Info() = default; // 默认构造一个值为 0 的叶子信息
    Info(long long x) : sum(x), len(1)
    {
        // x 是叶子的初始值。
    }

    static Info identity()
    {
        // 无参数；返回长度为 0 的空区间信息，作为合并单位元。
        Info c;
        c.len = 0;
        return c;
    }

    // 把区间加标记作用到当前节点。
    void apply(const Tag &t)
    {
        // t 是区间加标记；更新当前区间和。
        sum += t.add * len;
    }

    friend Info operator+(Info a, Info b)
    {
        // a、b 是相邻区间信息；返回合并结果。
        Info c;
        c.sum = a.sum + b.sum;
        c.len = a.len + b.len;
        return c;
    }
};

template <class Info, class Tag> struct LazySeg
{
    // n 是叶子数；tr 保存已计入本节点修改的区间信息，tag 和 has 保存尚未下传给儿子的修改。
    int n;
    vector<Info> tr;
    vector<Tag> tag;
    vector<unsigned char> has;

    static Info identity()
```

```
    {
        // 无参数；Info 自带 identity() 时用它，否则用默认构造值作为合并单位元。
        if constexpr (requires { Info::identity(); })
            return Info::identity();
        else
            return Info{};
    }

    LazySeg(int n = 0)
    {
        // n 是叶子数；每个叶子由 Info{} 默认构造成初始信息。
        init(n);
    }
    LazySeg(const vector<Info> &a)
    {
        // a 是各叶子的初始信息。
        init(a);
    }

    void init(int n_)
    {
        // n_ 是新的叶子数；用 n_ 个 Info{} 重建默认数组。
        assert(n_ >= 0); // 调试检查，可删
        n = n_;
        tr.assign(4 * max<int>(n, 1), Info{});
        tag.assign(4 * max<int>(n, 1), Tag{});
        has.assign(4 * max<int>(n, 1), 0);
        if (n)
        {
            vector<Info> a(n);
            build(1, 0, n, a);
        }
    }

    void init(const vector<Info> &a)
    {
        // a 是新的叶子信息；清空旧状态并重建。
        n = a.size();
        tr.assign(4 * max<int>(n, 1), Info{});
        tag.assign(4 * max<int>(n, 1), Tag{});
        has.assign(4 * max<int>(n, 1), 0);
        if (n)
        {
            build(1, 0, n, a);
        }
    }

    void build(int p, int l, int r, const vector<Info> &a)
    {
        // p 是当前节点，[l,r) 是其区间，a 是叶子信息。
        if (r - l == 1)
        {
```

```
            tr[p] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(2 * p, l, m, a);
        build(2 * p + 1, m, r, a);
        pull(p);
    }

    void pull(int p)
    {
        // p 是内部节点编号；按左右顺序合并两个儿子。
        tr[p] = tr[2 * p] + tr[2 * p + 1];
    }

    void apply(int p, const Tag &v)
    {
        // p 是整段命中的节点，v 是本次区间修改标记。
        tr[p].apply(v);
        tag[p].apply(v);
        has[p] = 1;
    }

    void push(int p)
    {
        // p 是当前节点；仅在有待执行标记时把它下传给两个儿子。
        if (!has[p])
        {
            return;
        }
        apply(2 * p, tag[p]);
        apply(2 * p + 1, tag[p]);
        tag[p] = Tag{};
        has[p] = 0;
    }

    void apply(int p, int l, int r, int ql, int qr, const Tag &v)
    {
        // p、[l,r) 是当前节点，  [ql,qr) 是修改区间，v 是本次标记。
        if (qr <= l || r <= ql)
        {
            return;
        }
        if (ql <= l && r <= qr)
        {
            apply(p, v);
            return;
        }
        push(p);
        int m = (l + r) / 2;
        apply(2 * p, l, m, ql, qr, v);
        apply(2 * p + 1, m, r, ql, qr, v);
```

```
        pull(p);
    }

    Info query(int p, int l, int r, int ql, int qr)
    {
        // p、[l,r) 是当前节点，  [ql,qr) 是询问区间；返回相交部分的信息。
        if (qr <= l || r <= ql)
        {
            return identity();
        }
        if (ql <= l && r <= qr)
        {
            return tr[p];
        }
        push(p);
        int m = (l + r) / 2;
        return query(2 * p, l, m, ql, qr) + query(2 * p + 1, m, r, ql, qr);
    }

    Info get(int p, int l, int r, int i) const
    {
        // p、[l,r) 是当前节点，i 是目标叶子下标；只读返回该叶子的最新信息。
        if (r - l == 1)
        {
            return tr[p];
        }
        int m = (l + r) / 2;
        Info ans;
        if (i < m)
        {
            ans = get(2 * p, l, m, i);
        }
        else
        {
            ans = get(2 * p + 1, m, r, i);
        }
        if (has[p])
        {
            ans.apply(tag[p]);
        }
        return ans;
    }

    template <class F>
    int findFirst(int p, int l, int r, int ql, int qr, F &ok)
    {
        // p、[l,r) 是当前节点，  [ql,qr) 是搜索区间；返回其中第一个满足条件的下标。
        if (r <= ql || qr <= l)
        {
            return -1;
        }
        if (ql <= l && r <= qr && !ok(tr[p]))
```

```
        {
            return -1;
        }
        if (r - l == 1)
        {
            return l < n ? l : -1;
        }
        push(p);
        int m = l + (r - l) / 2;
        int x = findFirst(2 * p, l, m, ql, qr, ok);
        return x == -1 ? findFirst(2 * p + 1, m, r, ql, qr, ok) : x;
    }

    template <class F>
    int findLast(int p, int l, int r, int ql, int qr, F &ok)
    {
        // p、[l,r) 是当前节点，  [ql,qr) 是搜索区间；返回其中最后一个满足条件的下标。
        if (r <= ql || qr <= l)
        {
            return -1;
        }
        if (ql <= l && r <= qr && !ok(tr[p]))
        {
            return -1;
        }
        if (r - l == 1)
        {
            return l < n ? l : -1;
        }
        push(p);
        int m = l + (r - l) / 2;
        int x = findLast(2 * p + 1, m, r, ql, qr, ok);
        return x == -1 ? findLast(2 * p, l, m, ql, qr, ok) : x;
    }

    void apply(int l, int r, const Tag &v)
    {
        // l、r 是半开区间端点，v 是区间修改标记。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        apply(1, 0, n, l, r, v);
    }
    Info query(int l, int r)
    {
        // l、r 是半开区间端点；返回 [l,r) 的信息。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return query(1, 0, n, l, r);
    }
    Info get(int i) const
    {
        // i 是叶子下标；只读返回该位置在所有待下传标记生效后的信息。
        assert(0 <= i && i < n); // 调试检查，可删
        return get(1, 0, n, i);
```

```
    }

    template <class F>
    int findFirst(int l, int r, F ok)
    {
        // l、r 是半开搜索区间，ok 判断节点摘要是否可能包含答案；返回第一个满足条件的下标。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return l == r ? -1 : findFirst(1, 0, n, l, r, ok);
    }

    template <class F>
    int findLast(int l, int r, F ok)
    {
        // l、r 是半开搜索区间，ok 判断节点摘要是否可能包含答案；返回最后一个满足条件的下标。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return l == r ? -1 : findLast(1, 0, n, l, r, ok);
    }
};
```

`LazySeg<Info, Tag> seg(a)` 由 `vector<Info>` 建树（$O(n)$），`apply(l, r, Tag{ ... })` 对 $[l,r)$ 施加修改，`query(l, r)` 返回区间聚合。节点摘要 `tr[p]` 已计入本节点标记，`has[p]` 为真时 `tag[p]` 尚未作用到儿子；部分区间操作和上二分才调用 `push`。`get(i) const` 不修改树，而是在从叶子回溯时按由内到外的顺序施加尚未下传的祖先标记，返回单个叶子。`findFirst` / `findLast` 返回指定区间内第一个或最后一个可能满足条件的位置；谓词为假时必须保证该节点内没有答案，搜索下传懒标记后再递归，若谓词能由节点摘要准确剪枝，搜索为 $O(\log n)$；若只保证假值时整段无解，接口仍正确但最坏可能退化到 $O(n)$，空间为 $O(n)$。上二分接口会修改懒标记的存放位置，所以不能通过 `const LazySeg` 调用。只做单点修改时用不带标记的 3.3.001 单点修改线段树与摘要二分 更省；需要同时支持加、乘、赋值等仿射修改见 3.3.003 区间加、乘、赋值（仿射懒标记），区间取 $\min$ / $\max$ 这类无法直接复合的标记必须用 3.3.011 Segment Tree Beats，历史版本见 3.3.008 可持久化线段树 / 主席树。

下面对 $[1,2,3,4]$ 做区间加并检查区间和、单点值与上二分：

```
signed main()
{
    LazySeg<Info, Tag> seg(vector<Info>{1, 2, 3, 4});
    seg.apply(1, 4, Tag{2}); // 序列变为 [1,4,5,6]。
    assert(seg.query(0, 4).sum == 16 && seg.query(1, 3).sum == 9);
    assert(seg.get(2).sum == 5);
}
```

3.3.003 区间加、乘、赋值（仿射懒标记）

区间加、乘、赋值这三种修改可以统一成一个仿射变换 $x \mapsto mul \cdot x + add$：区间加是 $(mul, add) = (1, x)$，区间乘是 $(x, 0)$，区间赋值是 $(0, x)$。仿射变换在复合下封闭，「先做当前标记、再做 t」的复合为 $mul \leftarrow mul \cdot t.mul$、$add \leftarrow add \cdot t.mul + t.add$（`AffineTag<T>::apply` 正是这条，方向不能反），作用到区间和则是 $sum \leftarrow sum \cdot mul + len \cdot add$。`LazySeg<AffineInfo<T>, AffineTag<T>>` 因而同时支持区间仿射修改和区间和；若题目只有区间仿射修改与单点查询，则 `AffPointSeg<T>` 不维护区间摘要，省去每次部分修改的 `pull`。两者都使用同一 `AffineTag<T>`，`T` 必须支持 $0, 1, +, \times$；整数可

取 `long long`，模题直接取本书的 `Z`，不再把模数逻辑散落到适配代码。异或、取模不落在同一仿射族里：异或需按位维护，取模要靠「区间最大值不超过模数就剪枝」的势能分析，不能硬塞进仿射公式。

```cpp
// 依赖 3.3.002 懒标记线段树
template <class T> struct AffineTag
{
    // mul、add 表示待执行变换 x -> mul*x+add。
    T mul = T(1), add{};

    // 已有变换后再执行 t。
    void apply(const AffineTag<T> &t)
    {
        // t 是后执行的仿射变换；把它复合到当前标记之后。
        mul *= t.mul;
        add = add * t.mul + t.add;
    }
};

template <class T> struct AffineInfo
{
    // sum 是区间和，len 是区间叶子数。
    T sum{};
    int len = 0;

    AffineInfo() = default;
    AffineInfo(const T &x) : sum(x), len(1)
    {
        // x 是单个叶子的初值。
    }

    AffineInfo(const T &s, int n) : sum(s), len(n)
    {
        // s 是当前区间元素和，n 是该区间叶子数。
    }

    void apply(const AffineTag<T> &t)
    {
        // t 是本次仿射标记；更新当前区间和。
        sum = sum * t.mul + T(len) * t.add;
    }

    friend AffineInfo operator+(AffineInfo a, AffineInfo b)
    {
        // a、b 是相邻区间信息；返回合并结果。
        AffineInfo c;
        c.sum = a.sum + b.sum;
        c.len = a.len + b.len;
        return c;
    }
};

template <class T> class AffPointSeg
```

```
{
    // n 是叶子数；a 保存初始点值；tag 和 has 保存尚未下传的仿射修改。
    int n; // n 是叶子数。
    vector<T> a;
    vector<AffineTag<T>> tag;
    vector<unsigned char> has;

    void apply(int p, const AffineTag<T> &v)
    {
        // p 是完整命中节点，v 是后执行的仿射变换；只复合该段延迟标记。
        tag[p].apply(v);
        has[p] = 1;
    }

    void push(int p)
    {
        // p 是内部节点；把较早的整段修改下传后，才允许对子段施加较晚修改。
        if (!has[p])
        {
            return;
        }
        apply(2 * p, tag[p]);
        apply(2 * p + 1, tag[p]);
        tag[p] = AffineTag<T>{};
        has[p] = 0;
    }

    void affine(int p, int l, int r, int ql, int qr, const AffineTag<T> &v)
    {
        // p、[l,r) 是当前节点，[ql,qr) 是修改区间，v 是后执行的仿射变换。
        if (qr <= l || r <= ql)
        {
            return;
        }
        if (ql <= l && r <= qr)
        {
            apply(p, v);
            return;
        }
        push(p);
        int m = (l + r) / 2;
        affine(2 * p, l, m, ql, qr, v);
        affine(2 * p + 1, m, r, ql, qr, v);
    }

    T get(int p, int l, int r, int i) const
    {
        // p、[l,r) 是当前节点，i 是目标叶子下标；只读返回所有仿射修改后的点值。
        T ans;
        if (r - l == 1)
        {
            ans = a[i];
```

```
        }
        else
        {
            int m = (l + r) / 2;
            ans = i < m ? get(2 * p, l, m, i) : get(2 * p + 1, m, r, i);
        }
        if (has[p])
        {
            ans = ans * tag[p].mul + tag[p].add;
        }
        return ans;
    }

  public:
    AffPointSeg(vector<T> a) : n(a.size()), a(move(a))
    {
        // a 是初始点值；构造仅分配延迟标记，不维护区间摘要。
        tag.assign(4 * max<int>(n, 1), AffineTag<T>{});
        has.assign(4 * max<int>(n, 1), 0);
    }

    void affine(int l, int r, const T &mul, const T &add)
    {
        // l、r 是半开端点；把 [l,r) 内每个值变为 value*mul+add。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r)
        {
            return;
        }
        affine(1, 0, n, l, r, {mul, add});
    }

    T get(int i) const
    {
        // i 是叶子下标；只读返回该点当前值。
        assert(0 <= i && i < n); // 调试检查，可删
        return get(1, 0, n, i);
    }
};

class HistMaxSeg
{
  public:
    struct Info
    {
        // cur、hist 分别是区间当前最大值和历次修改后的历史最大值。
        long long cur, hist;
    };

  private:
    static constexpr long long neg = numeric_limits<long long>::lowest() / 2; // neg 表示
    ↪ max-plus 意义下的负无穷，取一半防止相加溢出。
```

```
153     struct Tag
154     {
155         // cc、ch、hh、vc、vh 描述 max-plus 变换 (cur,hist,1) 的五个有限系数。
156         long long cc = 0, ch = neg, hh = 0, vc = neg, vh = neg;
157     };
158     int n; // n 是叶子数。
159     vector<Info> tr; // tr[p] 是节点 p 区间的当前最大值与历史最大值。
160     vector<Tag> lz; // lz[p] 是节点 p 尚未下传给儿子的复合标记。
161 
162     static long long plus(long long a, long long b)
163     {
164         // a、b 是普通值或负无穷；返回 max-plus 乘法。
165         return a == neg || b == neg ? neg : a + b;
166     }
167 
168     static Tag compose(const Tag &a, const Tag &b)
169     {
170         // a、b 是先后执行的标记；返回先 a 后 b 的复合标记。
171         return {plus(a.cc, b.cc),
172                 max(plus(a.cc, b.ch), plus(a.ch, b.hh)),
173                 plus(a.hh, b.hh),
174                 max(plus(a.vc, b.cc), b.vc),
175                 max({plus(a.vh, b.hh), plus(a.vc, b.ch), b.vh})};
176     }
177 
178     static Info apply(Info a, const Tag &t)
179     {
180         // a 是节点摘要，t 是区间标记；返回变换后的摘要。
181         return {max(plus(a.cur, t.cc), t.vc),
182                 max({plus(a.cur, t.ch), plus(a.hist, t.hh), t.vh})};
183     }
184 
185     void pull(int p)
186     {
187         // p 是内部节点；由两个儿子重算摘要。
188         tr[p] = {max(tr[2 * p].cur, tr[2 * p + 1].cur),
189                  max(tr[2 * p].hist, tr[2 * p + 1].hist)};
190     }
191 
192     void build(int p, int l, int r, const vector<long long> &a)
193     {
194         // p、[l,r) 是当前节点，a 是初始数组。
195         if (r - l == 1) return void(tr[p] = {a[l], a[l]});
196         int m = (l + r) / 2;
197         build(2 * p, l, m, a), build(2 * p + 1, m, r, a), pull(p);
198     }
199 
200     void apply(int p, const Tag &t)
201     {
202         // p 是整段命中节点，t 是后执行的标记。
203         tr[p] = apply(tr[p], t);
204         lz[p] = compose(lz[p], t);
```

```
205     }
206
207     void push(int p)
208     {
209         // p 是内部节点；把累计标记下传。
210         apply(2 * p, lz[p]), apply(2 * p + 1, lz[p]), lz[p] = {};
211     }
212
213     void update(int p, int l, int r, int ql, int qr, const Tag &t)
214     {
215         // p、[l,r) 是当前节点，[ql,qr) 是修改区间，t 是修改标记。
216         if (qr <= l || r <= ql) return;
217         if (ql <= l && r <= qr) return apply(p, t);
218         push(p);
219         int m = (l + r) / 2;
220         update(2 * p, l, m, ql, qr, t), update(2 * p + 1, m, r, ql, qr, t), pull(p);
221     }
222
223     Info query(int p, int l, int r, int ql, int qr)
224     {
225         // p、[l,r) 是当前节点，[ql,qr) 是询问区间；返回相交部分摘要。
226         if (qr <= l || r <= ql) return {neg, neg};
227         if (ql <= l && r <= qr) return tr[p];
228         push(p);
229         int m = (l + r) / 2;
230         Info a = query(2 * p, l, m, ql, qr), b = query(2 * p + 1, m, r, ql, qr);
231         return {max(a.cur, b.cur), max(a.hist, b.hist)};
232     }
233
234   public:
235     explicit HistMaxSeg(const vector<long long> &a) : n(a.size()), tr(4 * max<int>(n, 1),
      ↪  {neg, neg}), lz(4 * max<int>(n, 1))
236     {
237         // a 是非空初始数组；建立当前最大值与历史最大值线段树。
238         assert(n > 0); // 调试检查，可删
239         build(1, 0, n, a);
240     }
241
242     void add(int l, int r, long long v)
243     {
244         // l、r 是半开端点，v 是增量；给区间加 v 并记录修改后的历史峰值。
245         assert(0 <= l && l <= r && r <= n); // 调试检查，可删
246         update(1, 0, n, l, r, {v, v, 0, neg, neg});
247     }
248
249     void assign(int l, int r, long long v)
250     {
251         // l、r 是半开端点，v 是新值；给区间赋值并记录修改后的历史峰值。
252         assert(0 <= l && l <= r && r <= n); // 调试检查，可删
253         update(1, 0, n, l, r, {neg, neg, 0, v, v});
254     }
255
```

```
    Info query(int l, int r)
    {
        // l、r 是非空半开端点；返回区间当前最大值和历史最大值。
        assert(0 <= l && l < r && r <= n); // 调试检查，可删
        return query(1, 0, n, l, r);
    }
};
```

用 `LazySeg<AffineInfo<T>, AffineTag<T>> seg(a)` 后，区间加 `seg.apply(l, r, {T(1), x})`、区间乘 `seg.apply(l, r, {x, T(0)})`、区间赋值 `seg.apply(l, r, {T(0), x})`，`seg.query(l, r).sum` 取区间和。仅有区间仿射和单点查询时，把初值存为 `vector<T>` 后构造 `AffPointSeg<T> seg(a)`，调用 `seg.affine(l,r,mul,add)` 和 `seg.get(i)`；它没有 `query`，不能替代通用 `LazySeg` 的区间和接口。完整命中的更新只复合节点标记，部分更新先 `push` 较早标记，保证后来子段更新的时间顺序；`get(i) const` 从叶到根施加尚未下传标记。显式 `has` 既不要求 `T` 支持相等比较，也防止 $(mul, add) = (0, 0)$ 的赋零操作被误判为没有待下传标记。两种结构构造 $O(n)$，每次修改与各自查询 $O(\log n)$，空间 $O(n)$。混用加、乘时务必让所有系数与 `sum` 在同一类型中；例如 Library Checker 的模题取 `T=Z`，有界整数题才取 `long long` 并自行保证不溢出。只有单一区间加时用 3.3.002 懒标记线段树 的最简 `Tag` 更清爽；值域很大、初值全零且访问稀疏时用 3.3.021 稀疏仿射区间和。区间 chmin/chmax 见 3.3.011 Segment Tree Beats。3.3.004 区间加、区间最小值 只维护当前最小值，不提供历史最值。

下面依次检查通用仿射区间和、仿射单点查询与历史最大值：

```
signed main()
{
    LazySeg<AffineInfo<int>, AffineTag<int>> seg(vector<AffineInfo<int>>{1, 2, 3, 4});
    seg.apply(0, 2, {1, 5}); // 加 5: [6,7,3,4]。
    seg.apply(1, 3, {2, 0}); // 乘 2: [6,14,6,4]。
    seg.apply(3, 4, {0, 7}); // 赋 7: [6,14,6,7]。
    assert(seg.query(0, 4).sum == 33 && seg.query(1, 3).sum == 20);

    AffPointSeg<int> ps(vector<int>{1, 2, 3, 4});
    ps.affine(0, 3, 2, 1);  // [3,5,7,4]。
    ps.affine(1, 4, 1, -2); // [3,3,5,2]。
    assert(ps.get(1) == 3 && ps.get(3) == 2);

    HistMaxSeg h(vector<int>{1, 5, 2});
    h.add(0, 3, -4); // [-3,1,-2]，历史最大值仍是 5。
    auto q = h.query(0, 3);
    assert(q.cur == 1 && q.hist == 5);
    h.assign(0, 1, 10);
    h.add(0, 1, -20); // a[0] 先变 10 再变 -10。
    q = h.query(0, 1);
    assert(q.cur == -10 && q.hist == 10);
}
```

3.3.004 区间加、区间最小值

`AddMinSeg` 维护当前数组的区间最小值：每个节点存最小值 `minimum` 与待下传增量 `lazy`，区间加时把二者同步增加，`push` 把父标记加到两个儿子，`pull` 取两儿子最小值。本条不维护历史时刻达到过的最小值；

历史最值还需要额外保存历史极值和懒标记执行过程中的最小前缀增量，不能把当前最值接口当作历史查询使用。若改成区间最大值，`mn/rangeMin/min/`正无穷单位元必须一起换成 `mx/rangeMax/max/`负无穷，区间加和懒标记不变。

```
class AddMinSeg
{
    // 当前是区间加、区间最小值版。改成最大值只需把 mn/rangeMin/min/正无穷单位元
    // 成对换成 mx/rangeMax/max/负无穷；区间加和懒标记逻辑不变。
    // n 是数组长度；mn、lz 分别保存区间最小值与待下传增量。
    int n;
    vector<long long> mn, lz;

    void build(int p, int l, int r, const vector<long long> &a)
    {
        // p 是当前节点，[l,r) 是其区间，a 是初始数组。
        if (r - l == 1)
        {
            mn[p] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(2 * p, l, m, a);
        build(2 * p + 1, m, r, a);
        pull(p);
    }

    void apply(int p, long long v)
    {
        // p 是整段命中的节点，v 是区间增量。
        mn[p] += v;
        lz[p] += v;
    }

    void push(int p)
    {
        // p 是当前节点；把待执行增量下传给两个儿子。
        if (!lz[p])
        {
            return;
        }
        apply(2 * p, lz[p]);
        apply(2 * p + 1, lz[p]);
        lz[p] = 0;
    }

    void pull(int p)
    {
        // p 是内部节点；由两个儿子的最小值重算 mn[p]。
        mn[p] = min(mn[2 * p], mn[2 * p + 1]);
    }

    void add(int p, int l, int r, int ql, int qr, long long v)
```

```
    {
        // p、[l,r) 是当前节点，  [ql,qr) 是修改区间，v 是增量。
        if (qr <= l || r <= ql)
        {
            return;
        }
        if (ql <= l && r <= qr)
        {
            apply(p, v);
            return;
        }
        push(p);
        int m = (l + r) / 2;
        add(2 * p, l, m, ql, qr, v);
        add(2 * p + 1, m, r, ql, qr, v);
        pull(p);
    }

    long long rangeMin(int p, int l, int r, int ql, int qr)
    {
        // p、[l,r) 是当前节点；返回 [ql,qr) 内的最小值。
        if (qr <= l || r <= ql)
        {
            return numeric_limits<long long>::max();
        }
        if (ql <= l && r <= qr)
        {
            return mn[p];
        }
        push(p);
        int m = (l + r) / 2;
        return min(rangeMin(2 * p, l, m, ql, qr), rangeMin(2 * p + 1, m, r, ql, qr));
    }

  public:
    explicit AddMinSeg(const vector<long long> &a)
        : n(a.size()), mn(4 * max<int>(n, 1)), lz(4 * max<int>(n, 1))
    {
        // a 是初始数组；空数组建立空结构。
        if (n)
        {
            build(1, 0, n, a);
        }
    }

    void add(int l, int r, long long v)
    {
        // 给半开区间 [l,r) 增加 v；空区间无操作。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r)
        {
            return;
```

```
        }
        add(1, 0, n, l, r, v);
    }

    long long rangeMin(int l, int r)
    {
        // 返回非空半开区间 [l,r) 的最小值。
        assert(0 <= l && l < r && r <= n); // 调试检查，可删
        return rangeMin(1, 0, n, l, r);
    }
};
```

`AddMinSeg seg(values)` 由 `vector<long long>` 建树，空数组也能构造；`add(l, r, v)` 对 $[l, r)$ 区间加并允许空区间，`rangeMin(l, r)` 查询时要求区间非空。两种操作均为 $O(\log n)$，空间 $O(n)$。求和类换 3.3.002 懒标记线段树；区间对某值取 $\min$ / $\max$（不是加）这类操作要靠势能分析，见 3.3.011 Segment Tree Beats。

3.3.005 ZKW 线段树

ZKW 线段树把叶子铺在连续数组 `tr[n..2n)` 上（`n` 取不小于原长的 2 的幂），全程自底向上迭代、无递归、无懒标记，因而缓存友好、常数小于递归版。`set` 改叶子后沿父链逐级重算；`prod(l,r)` 从左右端点同时上跳，用 `a`、`b` 分别累计左边界与右边界扫上来的贡献、最后 `op(a,b)`，从而对非交换的 `op` 也保持顺序正确。它要求 `op` 满足结合律并带幺元 `e`，只支持单点修改与区间查询，且规模在构造时固定、之后不能扩容。

```
template <class T, class F> struct ZkwSeg
{
    // n 保存补到二次幂后的叶子数；e 保存幺元；op 保存满足结合律的合并运算；tr 保存连续线段树节点。
    int n = 1;
    T e;
    F op;
    vector<T> tr;

    ZkwSeg(const vector<T> &a, T e, F op) : e(e), op(op)
    {
        // a 表示初始序列，e 表示幺元，op 表示区间合并运算；构造固定规模的迭代线段树。
        while (n < (int)a.size())
        {
            n *= 2;
        }
        tr.assign(2 * n, e);
        copy(a.begin(), a.end(), tr.begin() + n);
        for (int p = n - 1; p; p--)
        {
            tr[p] = op(tr[2 * p], tr[2 * p + 1]);
        }
    }

    void set(int x, T v)
    {
```

```
        // x 表示 0 下标位置，v 表示新值；修改叶子并沿父链重算。
        for (tr[x += n] = v; x >>= 1;)
        {
            tr[x] = op(tr[2 * x], tr[2 * x + 1]);
        }
    }

    T prod(int l, int r) const
    {
        // l、r 表示半开区间 [l,r)；返回保持原顺序合并后的区间信息。
        T a = e, b = e;
        for (l += n, r += n; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1)
            {
                a = op(a, tr[l++]);
            }
            if (r & 1)
            {
                b = op(tr[--r], b);
            }
        }
        return op(a, b);
    }
};
```

构造 `ZkwSeg<T, F> seg(a, e, op)`：`e` 是幺元，`op` 为满足结合律的二元合并；`set(x, v)` 单点赋值，`prod(l, r)` 查 $[l, r)$。调用时必须显式给出合并函数，例如：

```
signed main()
{
    vector<int> a{2, -1, 4, 3};
    auto op = [](int x, int y)
    {
        return x + y;
    };
    ZkwSeg<int, decltype(op)> seg(a, 0, op);
    assert(seg.prod(1, 4) == 6);
    seg.set(2, 7);
    assert(seg.prod(0, 3) == 8);
}
```

build $O(n)$，`set`/`prod` 为 $O(\log n)$，空间 $O(n)$。需要区间整体修改时它无能为力，改用 3.3.002 懒标记线段树；这份 ZKW 就是下一条「非递归线段树」的同一实现。

3.3.006 非递归线段树

「非递归线段树」在本书里与 3.3.005 ZKW 线段树 是同一个实现模型——固定规模、单点修改、结合律区间查询、自底向上迭代无递归——因此不再重复打印代码，直接使用那里的 `ZkwSeg`。唯一要反复强调的坑是非交换运算：`prod` 里左边界用 `op(a, tr[l])`、右边界用 `op(tr[--r], b)`，两侧顺序不能颠倒，否则

字符串拼接、矩阵乘法等会出错（例如取 op 为字符串拼接、a = {"a","b","c","d"}，prod(1, 4) 应得 "bcd"）。需要区间整体懒修改、动态开点或复杂下传时都要换回递归模板：分别见 3.3.002 懒标记线段树 与 3.3.007 动态开点线段树。

3.3.007 动态开点线段树

当值域跨度极大（甚至含负数、达 10^9）而真正被赋值的位置很少时，普通动态开点线段树会为每个新点开出整条根到叶路径，空间达到 $O(q \log U)$。这里改为每个出现的位置仅存一个节点：节点保存该位置的值、左右子树和按下标顺序合并的积；插入时按当前值域二分方向下探，必要时交换当前节点与待插点，使一个节点仍只代表一个实际位置。于是树高至多 $\lceil \log_2 U \rceil$，而节点数恰为出现过的位置数（即使后来赋回幺元也保留该节点）。未出现的位置统一取幺元 e，故它适合“初始全是幺元、单点赋值、区间结合积”，尤其是非交换的函数复合；查询累积时必须保持左、中、右的顺序。

泛型 SparsePtSeg<T,F> 的 op(left,right) 必须返回左段在前、右段在后的结合结果，e 是该运算幺元。旧的 DynamicSeg 只是其加法包装，保留单点加、区间和接口；新题优先直接使用泛型版本，不能把非交换函数复合伪装成可交换求和。

```
template <class T, class F> class SparsePtSeg
{
    struct Node
    {
        // l、r 是孩子编号，pos 是本节点实际保存的位置，x、prod 分别是该点值与子树按下标顺序的积。
        int l = 0, r = 0;
        long long pos = 0;
        T x{}, prod{};
    };

    // lo、hi 是总半开值域，rt 是根编号，e 是空位置值，op 是按左到右合并的结合运算，t 是节点池。
    long long lo, hi;
    int rt = 0;
    T e;
    F op;
    vector<Node> t{Node{}};

    int newNode(long long x, const T &v)
    {
        // x 是新节点位置，v 是该点值；新建只保存此点的节点并返回编号。
        Node q;
        q.pos = x;
        q.x = q.prod = v;
        t.push_back(move(q));
        assert(t.size() <= (size_t)numeric_limits<int>::max()); // 调试检查，可删
        return (int)t.size() - 1;
    }

    void pull(int p)
    {
        // p 是非空节点；按下标顺序合并左右子树与本点，重算子树积。
        T cur = t[p].x;
```

```
        if (t[p].l)
        {
            cur = op(t[t[p].l].prod, cur);
        }
        if (t[p].r)
        {
            cur = op(cur, t[t[p].r].prod);
        }
        t[p].prod = cur;
    }

    int set(int p, long long l, long long r, long long x, T v)
    {
        // p、[l,r) 是当前稀疏子树，x、v 是待写位置和值；返回更新后的子树根。
        if (!p)
        {
            return newNode(x, v);
        }
        if (t[p].pos == x)
        {
            t[p].x = v;
            pull(p);
            return p;
        }
        long long m = l + (r - l) / 2;
        if (x < m)
        {
            if (t[p].pos < x)
            {
                swap(t[p].pos, x);
                swap(t[p].x, v);
            }
            int a = set(t[p].l, l, m, x, move(v));
            t[p].l = a;
        }
        else
        {
            if (x < t[p].pos)
            {
                swap(t[p].pos, x);
                swap(t[p].x, v);
            }
            int b = set(t[p].r, m, r, x, move(v));
            t[p].r = b;
        }
        pull(p);
        return p;
    }

    void prod(int p, long long l, long long r, long long ql, long long qr, T &ans) const
    {
        // p、[l,r) 是当前子树，[ql,qr) 是查询区间，ans 按从左到右顺序累积答案。
```

```
        ql = max(ql, l);
        qr = min(qr, r);
        if (ql >= qr || !p)
        {
            return;
        }
        if (ql == l && qr == r)
        {
            ans = op(ans, t[p].prod);
            return;
        }
        long long m = l + (r - l) / 2;
        prod(t[p].l, l, m, ql, qr, ans);
        if (ql <= t[p].pos && t[p].pos < qr)
        {
            ans = op(ans, t[p].x);
        }
        prod(t[p].r, m, r, ql, qr, ans);
    }

  public:
    SparsePtSeg(long long lo, long long hi, T e, F op) : lo(lo), hi(hi), e(move(e)),
    ↪ op(move(op))
    {
        // lo、hi 是非空半开整数值域，e 是未写位置的值，op 是保持左右顺序的结合运算。
        assert(lo < hi); // 调试检查，可删
    }

    void reserveNodes(int cap)
    {
        // cap 是预计出现的不同位置数；预留节点池容量，不改变结构语义。
        assert(cap >= 0); // 调试检查，可删
        t.reserve(cap + 1);
    }

    void set(long long x, const T &v)
    {
        // x 是值域内位置，v 是新值；把该点赋为 v，未写过的位置默认取 e。
        assert(lo <= x && x < hi); // 调试检查，可删
        rt = set(rt, lo, hi, x, v);
    }

    T get(long long x) const
    {
        // x 是值域内位置；返回该点当前值，未出现过时返回 e。
        assert(lo <= x && x < hi); // 调试检查，可删
        int p = rt;
        while (p)
        {
            if (t[p].pos == x)
            {
                return t[p].x;
```

```
            }
            p = x < t[p].pos ? t[p].l : t[p].r;
        }
        return e;
    }

    T prod(long long l, long long r) const
    {
        // l、r 是值域内半开端点；返回 [l,r) 按下标从左到右合并的积，空区间返回 e。
        assert(lo <= l && l <= r && r <= hi); // 调试检查，可删
        T ans = e;
        prod(rt, lo, hi, l, r, ans);
        return ans;
    }
};

class DynamicSeg
{
    struct Add
    {
        long long operator()(long long a, long long b) const
        {
            // a、b 是相邻两段的元素和；返回两段合并后的和。
            return a + b;
        }
    };

    // tr 是泛型稀疏线段树的加法实例，保留旧的单点加、区间和接口。
    SparsePtSeg<long long, Add> tr;

  public:
    DynamicSeg(long long lo, long long hi) : tr(lo, hi, 0, Add{})
    {
        // lo、hi 是非空半开值域边界；初始所有点均为零。
    }

    void reserveNodes(int cap)
    {
        // cap 是预计被修改过的不同位置数；转交给内部节点池预留容量。
        tr.reserveNodes(cap);
    }

    void add(long long x, long long v)
    {
        // x 是值域内位置，v 是有符号增量；把该点增加 v。
        tr.set(x, tr.get(x) + v);
    }

    long long sum(long long l, long long r) const
    {
        // l、r 是查询半开区间；返回其中元素和。
        return tr.prod(l, r);
```

```
    }
};
```

构造 `SparsePtSeg<T,F> tr(lo, hi, e, op)`，值域为半开 $[lo, hi)$（可为负）；`set(x,v)` 单点赋值，`get(x)` 取点值，`prod(l,r)` 查半开区间的有序积。调用时例如用加法维护稀疏位置：

```
signed main()
{
    auto op = [](int x, int y)
    {
        return x + y;
    };
    SparsePtSeg<int, decltype(op)> tr(0, 1000000000, 0, op);
    tr.set(5, 7);
    tr.set(900000000, 3);
    assert(tr.get(5) == 7);
    assert(tr.prod(0, 1000000000) == 10);
}
```

设 $U = hi - lo$，出现过的不同位置数为 s，`set`、`get`、`prod` 最坏均为 $O(\log U)$，节点数 $O(s)$；`reserveNodes(s)` 只优化扩容，不能把位置数预留成 U。例如取 `T=array<Z,2>`，令 `e={1,0}`、`op(f,g)=g\circ f`，即可处理长度至多 10^9、初始全为恒等函数的单点赋值与区间函数复合。仍需要旧的加法接口时构造 `DynamicSeg tr(lo,hi)` 并调用 `add(x,v)`、`sum(l,r)`；其复杂度相同。若操作是大值域上的区间仿射、区间和，节点不能只表示单点，应改用 3.3.021 稀疏仿射区间和；要把整段函数统一赋成一个值并查询有序复合，改用 3.3.022 区间赋同函数、区间复合。它不提供两棵树合并——需要合并权值树时用 3.3.009 线段树合并，值域不大时直接离散化后用 3.3.002 懒标记线段树 常数更小，要保历史版本用 3.3.008 可持久化线段树 / 主席树。

3.3.008 可持久化线段树 / 主席树

可持久化线段树（主席树）每次单点修改只新建根到叶的一条链、其余结点与旧版本共享，于是每个版本根都是一份完整快照，仅用 $O(\log n)$ 额外结点保存历史。结点同时保存出现次数 `sum` 与可独立指定的附加权值和 `weight`，版本差便能在同一值域区间返回计数和总权值。静态区间第 k 小仍只依赖非负计数：第 r 版减第 l 版得到下标区间 $[l, r)$ 的值域计数，再按左子树计数二分；`kth` 的 k 从 0 开始，附加权值允许为负但不参与排名。

```
struct PerSeg
{
    struct Node
    {
        // l、r 是左右儿子编号，sum、weight 是当前值域的计数和与附加权值和。
        int l = 0, r = 0;
        long long sum = 0;
        long long weight = 0;
    };

    // n 是离散值域大小，t 是所有版本共享的节点池。
    int n;
```

```
    vector<Node> t{{}};

    PerSeg(int n) : n(n)
    {
        // n 是非空离散值域大小；0 号根表示空版本。
        assert(n > 0); // 调试检查，可删
    }

    int add(int root, int x, long long v)
    {
        // root 是旧版本根，x 是值域下标，v 是计数增量；返回新版本根。
        return add(root, x, v, 0);
    }

    int add(int root, int x, long long dc, long long dw)
    {
        // root 是旧版本根，x 是值域下标；dc、dw 是计数与附加权值增量，返回新版本根。
        assert(0 <= x && x < n); // 调试检查，可删
        return add(root, 0, n, x, dc, dw);
    }

    long long query(int root, int l, int r) const
    {
        // root 是版本根；返回值域 [l,r) 的计数和。
        return queryInfo(root, l, r)[0];
    }

    array<long long, 2> queryInfo(int root, int l, int r) const
    {
        // root 是版本根，l、r 是值域半开区间；返回 {计数和，附加权值和}。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return queryInfo(root, 0, n, l, r);
    }

    // 返回右版本减左版本后的第 k 小下标，k 从 0 开始；sum 必须表示非负计数。
    int kth(int rootL, int rootR, long long k) const
    {
        // rootL、rootR 是两个前缀版本根，k 是 0 基排名；返回版本差中的第 k 小值域下标。
        assert(0 <= k && k < t[rootR].sum - t[rootL].sum); // 调试检查，可删
        int l = 0, r = n;
        while (r - l > 1)
        {
            int m = (l + r) / 2;
            long long z = t[t[rootR].l].sum - t[t[rootL].l].sum;
            if (k < z)
            {
                rootL = t[rootL].l;
                rootR = t[rootR].l;
                r = m;
            }
            else
            {
```

```
                k -= z;
                rootL = t[rootL].r;
                rootR = t[rootR].r;
                l = m;
            }
        }
        return l;
    }

  private:
    int clone(int p)
    {
        // p 是旧节点编号；复制它并返回新节点编号。
        t.push_back(t[p]);
        return (int)t.size() - 1;
    }

    int add(int p, int l, int r, int x, long long dc, long long dw)
    {
        // p 是旧节点，[l,r) 是其值域，x 是更新位置；两个增量分别修改计数与附加权值，返回新节点。
        int q = clone(p);
        t[q].sum += dc;
        t[q].weight += dw;
        if (r - l == 1)
        {
            return q;
        }
        int m = (l + r) / 2;
        if (x < m)
        {
            t[q].l = add(t[q].l, l, m, x, dc, dw);
        }
        else
        {
            t[q].r = add(t[q].r, m, r, x, dc, dw);
        }
        return q;
    }

    array<long long, 2> queryInfo(int p, int l, int r, int ql, int qr) const
    {
        // p 是当前节点，[l,r) 是其值域；返回 [ql,qr) 的计数和与附加权值和。
        if (!p || qr <= l || r <= ql)
        {
            return {0, 0};
        }
        if (ql <= l && r <= qr)
        {
            return {t[p].sum, t[p].weight};
        }
        int m = (l + r) / 2;
        auto a = queryInfo(t[p].l, l, m, ql, qr);
```

```
        auto b = queryInfo(t[p].r, m, r, ql, qr);
        return {a[0] + b[0], a[1] + b[1]};
    }
};
```

用法：`PerSeg ps(n)`，其中 n 为非空离散值域大小。维护版本根数组，令 `root[0]=0` 为空树；只维护计数时调用三参数 `add(root,x,dc)`，同时维护权值时调用四参数 `add(root,x,dc,dw)`。`queryInfo(root,l,r)` 返回 `{计数和，附加权值和}`，原 `query` 仍只返回计数。逐点构造前缀版本可写成

```
root[i + 1] = ps.add(root[i], a[i], 1);
```

查询下标半开区间 $[l,r)$ 的第 k 小写 `ps.kth(root[l], root[r], k)`，返回离散值域下标。`add`、`queryInfo`、`kth` 均 $O(\log n)$，每次 `add` 新增 $O(\log n)$ 结点，总空间 $O(n\log n)$。同源的可持久化数组 / 并查集见 3.1.005 可持久化并查集。值域计数若还需在线带修（插删），单纯主席树不够，要树状数组套主席树或离线整体二分，见 3.4.006 区间第 K 小、排名、前驱后继。下面用含重复值的固定数组核对版本差和排名：

```
signed main()
{
    vector<int> a{5, 1, 4, 1, 3};
    PerSeg ps(6);
    vector<int> root(a.size() + 1);
    for (int i = 0; i < (int)a.size(); i++)
    {
        root[i + 1] = ps.add(root[i], a[i], 1);
    }
    vector<int> got{ps.kth(root[0], root[5], 0), ps.kth(root[0], root[5], 2),
                    ps.kth(root[0], root[5], 4), ps.kth(root[1], root[4], 0),
                    ps.kth(root[1], root[4], 2)};
    assert(got == vector<int>({1, 3, 5, 1, 4}));
}
```

3.3.009 线段树合并

线段树合并把两棵同值域的动态开点权值线段树递归合成一棵：某一侧为空就直接返回另一侧（`p | q`，其中一个必为 0），到叶子把计数相加，内部结点合并左右儿子后重算和。它是「每个树上结点先各建一棵权值树，再自底向上把儿子合并到父亲」这一套路的核心，可替代 DSU on tree 做子树众数 / 子树第 k 小等统计。这里是破坏式合并：`merge(p,q)` 复用 `p` 的结点，调用后 `q`（及被并入的结点）不应再独立使用。总复杂度靠势能分析——每次合并消掉一个重叠结点，全程重叠结点总数是 $O(\text{总插入次数}\cdot\log U)$，故所有合并累计线性对数。

```
struct MergeSeg
{
    struct Node
    {
        // l、r 是左右儿子编号，sum 是当前值域的权值和。
        int l = 0, r = 0;
        long long sum = 0;
    };
```

```
9
10      // lo、hi 是整棵权值树的半开值域，t 是所有树共享的节点池。
11      int lo, hi;
12      vector<Node> t{{}};
13
14      MergeSeg(int lo, int hi) : lo(lo), hi(hi)
15      {
16          // lo、hi 是非空半开值域边界。
17          assert(lo < hi); // 调试检查，可删
18      }
19      int newNode()
20      {
21          // 分配并返回一个全零节点编号。
22          t.push_back({});
23          return (int)t.size() - 1;
24      }
25      int makeRoot() const
26      {
27          // 返回表示空树的根编号 0。
28          return 0;
29      }
30
31      int add(int p, int l, int r, int x, long long v)
32      {
33          // p、[l,r) 是当前树，x 是值域位置，v 是权值增量；返回更新后的根。
34          if (!p)
35          {
36              p = newNode();
37          }
38          t[p].sum += v;
39          if (l + 1 == r)
40          {
41              return p;
42          }
43          int m = midpoint(l, r);
44          if (x < m)
45          {
46              t[p].l = add(t[p].l, l, m, x, v);
47          }
48          else
49          {
50              t[p].r = add(t[p].r, m, r, x, v);
51          }
52          return p;
53      }
54
55      void add(int &p, int x, long long v)
56      {
57          // p 是根，x 是值域位置，v 是权值增量；原地更新 p。
58          assert(lo <= x && x < hi); // 调试检查，可删
59          p = add(p, lo, hi, x, v);
60      }
```

```
61
62      int merge(int p, int q, int l, int r)
63      {
64          // p、q 是当前值域 [l,r) 上的两棵树根；破坏性合并并返回根。
65          if (!p || !q)
66          {
67              return p | q;
68          }
69          if (l + 1 == r)
70          {
71              t[p].sum += t[q].sum;
72              return p;
73          }
74          int m = midpoint(l, r);
75          t[p].l = merge(t[p].l, t[q].l, l, m);
76          t[p].r = merge(t[p].r, t[q].r, m, r);
77          t[p].sum = t[t[p].l].sum + t[t[p].r].sum;
78          return p;
79      }
80
81      int merge(int p, int q)
82      {
83          // p、q 是两棵共享同一值域的树根；破坏性合并并返回根。
84          return merge(p, q, lo, hi);
85      }
86
87      long long sum(int p, int l, int r, int x, int y) const
88      {
89          // p、[l,r) 是当前树；返回值域 [x,y) 的权值和。
90          if (!p || y <= l || r <= x)
91          {
92              return 0;
93          }
94          if (x <= l && r <= y)
95          {
96              return t[p].sum;
97          }
98          int m = midpoint(l, r);
99          return sum(t[p].l, l, m, x, y) + sum(t[p].r, m, r, x, y);
100     }
101
102     long long rangeSum(int p, int l, int r) const
103     {
104         // p 是树根；返回值域 [l,r) 的权值和。
105         assert(lo <= l && l <= r && r <= hi); // 调试检查，可删
106         return sum(p, lo, hi, l, r);
107     }
108 };
```

构造 `MergeSeg ms(lo, hi)` 共用一个结点池；每棵子树各持一个根编号，`ms.add(root, x, v)` 单点加、`ms.rangeSum(root, l, r)` 查区间和、`ms.merge(rootP, rootQ)` 返回合并根。所有树共享 `t` 数组，根用整型编号区分。单点 `add`、区间查询 $O(\log U)$，全部 `merge` 累计 $O(\text{有效更新数} \cdot \log U)$，空间同阶。它

与 3.3.010 线段树分裂 互为逆操作、常配合使用（分裂出一段再并回别处）；只在单棵权值树上做区间第 k 小、无需合并时用 3.3.008 可持久化线段树 / 主席树。

3.3.010 线段树分裂

线段树分裂是合并的逆操作：按值域区间把一棵权值线段树拆成两棵。递归时整段落在保留侧就原地不动、整段落在分出侧就把结点所有权整体转移给新根，只有跨边界的结点才 `newNode` 复制一份并递归拆分左右儿子，最后两侧都 `pull` 重算、和为零的结点回收成 0。这样一次分裂只新建 $O(\log U)$ 个结点。配合按排名分裂即可实现「取出前 k 小 / 某值域段」再并到别处，是可并集合、动态排名截取的主力；`kth` 在单棵树上按计数二分求第 k 小（k 从 1 开始，越界返回 -1）。

```
template <class T> struct SplitSeg
{
    struct Node
    {
        // l、r 是左右儿子编号，sum 是子树内的非负计数和。
        int l = 0, r = 0;
        T sum{};
    };

    // lo、hi 是整棵计数树的半开值域，t 是所有树共享的节点池。
    int lo, hi;
    vector<Node> t{{}};

    SplitSeg(int lo, int hi) : lo(lo), hi(hi)
    {
        // lo、hi 是非空半开值域边界。
        assert(lo < hi); // 调试检查，可删
    }
    int newNode()
    {
        // 分配并返回一个全零节点编号。
        t.push_back({});
        return (int)t.size() - 1;
    }
    int makeRoot() const
    {
        // 返回表示空树的根编号 0。
        return 0;
    }

    void pull(int p)
    {
        // p 是内部节点；由两个儿子的计数和重算当前节点。
        t[p].sum = t[t[p].l].sum + t[t[p].r].sum;
    }

    int add(int p, int l, int r, int x, T v)
    {
        // p、[l,r) 是当前计数树，x 是值域位置，v 是计数增量；返回新根。
```

```
        if (!p)
        {
            p = newNode();
        }
        if (l + 1 == r)
        {
            t[p].sum += v;
            assert(t[p].sum >= T{}); // 调试检查，可删
            return t[p].sum == T{} ? 0 : p;
        }
        int m = midpoint(l, r);
        if (x < m)
        {
            t[p].l = add(t[p].l, l, m, x, v);
        }
        else
        {
            t[p].r = add(t[p].r, m, r, x, v);
        }
        pull(p);
        return t[p].sum == T{} ? 0 : p;
    }

    void add(int &p, int x, T v)
    {
        // p 是根，x 是值域位置，v 是计数增量；更新后所有叶子计数必须非负。
        assert(lo <= x && x < hi); // 调试检查，可删
        p = add(p, lo, hi, x, v);
    }

    pair<int, int> split(int p, int l, int r, int x, int y)
    {
        // p、[l,r) 是当前树，[x,y) 是分出值域；返回 {保留根，分出根}。
        if (!p || y <= l || r <= x)
        {
            return {p, 0};
        }
        if (x <= l && r <= y)
        {
            return {0, p};
        }
        int q = newNode(), m = midpoint(l, r);
        auto [al, bl] = split(t[p].l, l, m, x, y);
        auto [ar, br] = split(t[p].r, m, r, x, y);
        t[p].l = al;
        t[p].r = ar;
        t[q].l = bl;
        t[q].r = br;
        pull(p);
        pull(q);
        if (t[p].sum == T{})
        {
```

```
            p = 0;
        }
        if (t[q].sum == T{})
        {
            q = 0;
        }
        return {p, q};
    }

    int split(int &p, int l, int r)
    {
        // p 是原根；把值域 [l,r) 分出并返回新根，p 保留其余部分。
        assert(lo <= l && l <= r && r <= hi); // 调试检查，可删
        auto [a, b] = split(p, lo, hi, l, r);
        p = a;
        return b;
    }

    int merge(int p, int q, int l, int r)
    {
        // p、q 是当前值域 [l,r) 上的两棵树根；破坏性合并并返回根。
        if (!p || !q)
        {
            return p | q;
        }
        if (l + 1 == r)
        {
            t[p].sum += t[q].sum;
            return p;
        }
        int m = midpoint(l, r);
        t[p].l = merge(t[p].l, t[q].l, l, m);
        t[p].r = merge(t[p].r, t[q].r, m, r);
        pull(p);
        return p;
    }

    int merge(int p, int q)
    {
        // p、q 是两棵所有权独立的计数树根；破坏性合并并返回根。
        return merge(p, q, lo, hi);
    }

    T sum(int p, int l, int r, int x, int y) const
    {
        // p、[l,r) 是当前树；返回值域 [x,y) 的计数和。
        if (!p || y <= l || r <= x)
        {
            return T{};
        }
        if (x <= l && r <= y)
        {
```

```
            return t[p].sum;
        }
        int m = midpoint(l, r);
        return sum(t[p].l, l, m, x, y) + sum(t[p].r, m, r, x, y);
    }

    T rangeSum(int p, int l, int r) const
    {
        // p 是根；返回值域 [l,r) 内的计数和。
        assert(lo <= l && l <= r && r <= hi); // 调试检查，可删
        return sum(p, lo, hi, l, r);
    }

    int kth(int p, T k) const
    {
        // p 是根，k 是从 1 开始的排名；返回第 k 小值，越界返回 -1。
        if (!p || k <= T{} || t[p].sum < k)
        {
            return -1;
        }
        int l = lo, r = hi;
        while (l + 1 < r)
        {
            int m = midpoint(l, r), q = t[p].l;
            if (t[q].sum >= k)
            {
                p = q;
                r = m;
            }
            else
            {
                k -= t[q].sum;
                p = t[p].r;
                l = m;
            }
        }
        return l;
    }
};
```

构造 `SplitSeg<long long> ss(lo, hi)`：`ss.add(root, x, v)` 在值 x 处加计数 v，要求更新后每个叶子计数非负；这一条件保证 `sum==0` 才等价于空树，也保证 `kth` 的方向判断成立。`ss.rangeSum(root, l, r)` 查值域 $[l,r)$ 计数和，`int b = ss.split(root, l, r)` 把 `root` 中值域 $[l,r)$ 的部分拆到新根 `b`（`root` 保留其余），`ss.merge(p, q)` 合回两棵所有权独立的树，`ss.kth(root, k)` 求第 k 小（k 从 1 起）。`add`、`kth`、`split` 均 $O(\log U)$，`merge` 与访问结点数成正比，空间 $O((\text{更新数}+\text{split 次数})\log U)$。它与 3.3.009 线段树合并 互逆、常成对使用；单纯静态区间第 k 小无需拆分时用 3.3.008 可持久化线段树 / 主席树。下面用一次固定拆分、合并闭环核对计数与排名：

```
signed main()
{
    SplitSeg<long long> ss(0, 10);
```

```
    int root = 0;
    for (int x : {1, 1, 4, 7, 9})
    {
        ss.add(root, x, 1);
    }
    int mid = ss.split(root, 4, 8);
    assert(ss.rangeSum(root, 0, 10) == 3 && ss.rangeSum(mid, 0, 10) == 2);
    root = ss.merge(root, mid);
    assert(ss.kth(root, 1) == 1 && ss.kth(root, 4) == 7 && ss.kth(root, 5) == 9);
}
```

3.3.011 Segment Tree Beats

吉司机线段树（Segment Tree Beats）是普通懒标记之外唯一能在线支持区间 chmin/chmax 的做法。关键在于每个节点除最大值外还维护严格次大值 `mx2` 与最大值出现次数 `mxc`（最小侧对称）。执行区间 chmin(x) 时分三种情形：若 $x \geq$ 最大值则整段无变化直接返回；若严格次大值 $< x <$ 最大值，则只有那 `mxc` 个最大值会被压到 x，可以 $O(1)$ 打标记整段完成（`sum` 减去 `(mx-x)*mxc`）；否则 x 会切断次大值层，必须递归下传。势能分析表明这种“标记回收”下连续 q 次操作总代价是 $O((n+q)\log^2 n)$。严格次值不能与普通整数哨兵共用表示，下面用 `hasMx2`、`hasMn2` 单独记录其是否存在，因此所有合法 `long long` 元素值都能参与维护；调用方仍须保证区间和及加法中间量可由 `long long` 表示。

```
struct SegBeats
{
    struct Node
    {
        // sum 是区间和，add 是待下传增量。
        long long sum = 0, add = 0;
        // mx、mn 是最值；mx2、mn2 仅在对应 has 标志为真时保存严格次值。
        long long mx = 0, mx2 = 0, mn = 0, mn2 = 0;
        bool hasMx2 = false, hasMn2 = false;
        // mxc、mnc 是最值出现次数，len 是区间长度。
        int mxc = 0, mnc = 0, len = 0;
    };

    // n 是数组长度，tr 保存各线段树节点的信息。
    int n;
    vector<Node> tr;

    SegBeats(const vector<long long> &a) : n(a.size()), tr(4 * n)
    {
        // a 是非空初始数组。
        assert(n); // 调试检查，可删
        build(1, 0, n, a);
    }

    void add(int l, int r, long long x)
    {
        // 给半开区间 [l,r) 增加 x。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
```

```
        add(1, 0, n, l, r, x);
    }
    void chmin(int l, int r, long long x)
    {
        // 把半开区间 [l,r) 内每个值改成 min(value,x)。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        chmin(1, 0, n, l, r, x);
    }
    void chmax(int l, int r, long long x)
    {
        // 把半开区间 [l,r) 内每个值改成 max(value,x)。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        chmax(1, 0, n, l, r, x);
    }
    long long sum(int l, int r)
    {
        // 返回半开区间 [l,r) 的和，空区间返回 0。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return sum(1, 0, n, l, r);
    }
    long long min(int l, int r)
    {
        // 返回非空半开区间 [l,r) 的最小值。
        assert(0 <= l && l < r && r <= n); // 调试检查，可删
        return min(1, 0, n, l, r);
    }
    long long max(int l, int r)
    {
        // 返回非空半开区间 [l,r) 的最大值。
        assert(0 <= l && l < r && r <= n); // 调试检查，可删
        return max(1, 0, n, l, r);
    }

  private:
    static Node merge(const Node &a, const Node &b)
    {
        // a、b 是相邻区间节点；返回合并后的 Beats 信息。
        Node c;
        c.sum = a.sum + b.sum;
        c.len = a.len + b.len;
        c.mx = a.mx < b.mx ? b.mx : a.mx;
        c.mxc = (a.mx == c.mx ? a.mxc : 0) + (b.mx == c.mx ? b.mxc : 0);
        auto takeMax = [&](long long x)
        {
            if (x != c.mx && (!c.hasMx2 || c.mx2 < x))
            {
                c.mx2 = x;
                c.hasMx2 = true;
            }
        };
        takeMax(a.mx);
        takeMax(b.mx);
```

```
        if (a.hasMx2)
        {
            takeMax(a.mx2);
        }
        if (b.hasMx2)
        {
            takeMax(b.mx2);
        }
        c.mn = a.mn < b.mn ? a.mn : b.mn;
        c.mnc = (a.mn == c.mn ? a.mnc : 0) + (b.mn == c.mn ? b.mnc : 0);
        auto takeMin = [&](long long x)
        {
            if (x != c.mn && (!c.hasMn2 || x < c.mn2))
            {
                c.mn2 = x;
                c.hasMn2 = true;
            }
        };
        takeMin(a.mn);
        takeMin(b.mn);
        if (a.hasMn2)
        {
            takeMin(a.mn2);
        }
        if (b.hasMn2)
        {
            takeMin(b.mn2);
        }
        return c;
    }

    void build(int p, int l, int r, const vector<long long> &a)
    {
        // p 是当前节点，[l,r) 是其区间，a 是初始数组。
        if (r - l == 1)
        {
            tr[p].sum = tr[p].mx = tr[p].mn = a[l];
            tr[p].mxc = tr[p].mnc = tr[p].len = 1;
            return;
        }
        int m = (l + r) / 2;
        build(2 * p, l, m, a);
        build(2 * p + 1, m, r, a);
        pull(p);
    }

    void pull(int p)
    {
        // p 是内部节点；合并两个儿子的最值、次值、次数与区间和。
        tr[p] = merge(tr[2 * p], tr[2 * p + 1]);
    }

```

```
    void applyAdd(int p, long long x)
    {
        // p 是整段命中的节点，x 是区间增量。
        auto &v = tr[p];
        v.sum += x * v.len;
        v.mx += x;
        v.mn += x;
        if (v.hasMx2)
        {
            v.mx2 += x;
        }
        if (v.hasMn2)
        {
            v.mn2 += x;
        }
        v.add += x;
    }

    void applyChmin(int p, long long x)
    {
        // p 是可整段处理的节点，x 是上界；仅把当前最大值压到 x。
        auto &v = tr[p];
        if (v.mx <= x)
        {
            return;
        }
        v.sum += (x - v.mx) * v.mxc;
        long long old = v.mx;
        if (v.mn == old)
        {
            v.mn = x;
        }
        else if (v.hasMn2 && v.mn2 == old)
        {
            v.mn2 = x;
        }
        v.mx = x;
    }

    void applyChmax(int p, long long x)
    {
        // p 是可整段处理的节点，x 是下界；仅把当前最小值抬到 x。
        auto &v = tr[p];
        if (v.mn >= x)
        {
            return;
        }
        v.sum += (x - v.mn) * v.mnc;
        long long old = v.mn;
        if (v.mx == old)
        {
            v.mx = x;
```

```
185         }
186         else if (v.hasMx2 && v.mx2 == old)
187         {
188             v.mx2 = x;
189         }
190         v.mn = x;
191     }
192 
193     void push(int p)
194     {
195         // p 是当前节点；依次下传区间加和父节点的最值界。
196         if (tr[p].add)
197         {
198             applyAdd(2 * p, tr[p].add);
199             applyAdd(2 * p + 1, tr[p].add);
200             tr[p].add = 0;
201         }
202         applyChmin(2 * p, tr[p].mx);
203         applyChmin(2 * p + 1, tr[p].mx);
204         applyChmax(2 * p, tr[p].mn);
205         applyChmax(2 * p + 1, tr[p].mn);
206     }
207 
208     void add(int p, int l, int r, int ql, int qr, long long x)
209     {
210         // p、[l,r) 是当前节点， [ql,qr) 是修改区间，x 是增量。
211         if (qr <= l || r <= ql)
212         {
213             return;
214         }
215         if (ql <= l && r <= qr)
216         {
217             applyAdd(p, x);
218             return;
219         }
220         push(p);
221         int m = (l + r) / 2;
222         add(2 * p, l, m, ql, qr, x);
223         add(2 * p + 1, m, r, ql, qr, x);
224         pull(p);
225     }
226 
227     void chmin(int p, int l, int r, int ql, int qr, long long x)
228     {
229         // p、[l,r) 是当前节点；把 [ql,qr) 内大于 x 的值压到 x。
230         if (qr <= l || r <= ql || tr[p].mx <= x)
231         {
232             return;
233         }
234         if (ql <= l && r <= qr && (!tr[p].hasMx2 || tr[p].mx2 < x))
235         {
236             applyChmin(p, x);
```

```
            return;
        }
        push(p);
        int m = (l + r) / 2;
        chmin(2 * p, l, m, ql, qr, x);
        chmin(2 * p + 1, m, r, ql, qr, x);
        pull(p);
    }

    void chmax(int p, int l, int r, int ql, int qr, long long x)
    {
        // p、[l,r) 是当前节点；把 [ql,qr) 内小于 x 的值抬到 x。
        if (qr <= l || r <= ql || tr[p].mn >= x)
        {
            return;
        }
        if (ql <= l && r <= qr && (!tr[p].hasMn2 || x < tr[p].mn2))
        {
            applyChmax(p, x);
            return;
        }
        push(p);
        int m = (l + r) / 2;
        chmax(2 * p, l, m, ql, qr, x);
        chmax(2 * p + 1, m, r, ql, qr, x);
        pull(p);
    }

    long long sum(int p, int l, int r, int ql, int qr)
    {
        // p、[l,r) 是当前节点；返回 [ql,qr) 的区间和。
        if (qr <= l || r <= ql)
        {
            return 0;
        }
        if (ql <= l && r <= qr)
        {
            return tr[p].sum;
        }
        push(p);
        int m = (l + r) / 2;
        return sum(2 * p, l, m, ql, qr) + sum(2 * p + 1, m, r, ql, qr);
    }

    long long min(int p, int l, int r, int ql, int qr)
    {
        // p、[l,r) 是当前节点；返回 [ql,qr) 的最小值。
        if (qr <= l || r <= ql)
        {
            return numeric_limits<long long>::max();
        }
        if (ql <= l && r <= qr)
```

```
        {
            return tr[p].mn;
        }
        push(p);
        int m = (l + r) / 2;
        long long a = min(2 * p, l, m, ql, qr), b = min(2 * p + 1, m, r, ql, qr);
        return a < b ? a : b;
    }

    long long max(int p, int l, int r, int ql, int qr)
    {
        // p、[l,r) 是当前节点；返回 [ql,qr) 的最大值。
        if (qr <= l || r <= ql)
        {
            return numeric_limits<long long>::lowest();
        }
        if (ql <= l && r <= qr)
        {
            return tr[p].mx;
        }
        push(p);
        int m = (l + r) / 2;
        long long a = max(2 * p, l, m, ql, qr), b = max(2 * p + 1, m, r, ql, qr);
        return a < b ? b : a;
    }
};
```

构造 `SegBeats st(a)`（a 为 `vector<long long>`，下标 $[0,n)$），此后 `st.add(l,r,x)`、`st.chmin(l,r,x)`、`st.chmax(l,r,x)` 均作用于半开区间 $[l,r)$，`st.sum/min/max(l,r)` 查询同一区间。单次 add、sum、min、max 为 $O(\log n)$，chmin/chmax 均摊 $O(\log^2 n)$、连续 q 次总计 $O((n+q)\log^2 n)$，空间 $O(n)$。若只有区间加和普通区间查询而无 chmin/chmax，用 3.3.002 懒标记线段树 常数更小；只需单点修改则用 3.3.005 ZKW 线段树。下面一组操作覆盖区间取小、取大、加法以及三种查询：

```
signed main()
{
    SegBeats st({5, -2, 7, 1});
    st.chmin(0, 3, 4);
    st.chmax(1, 4, 0);
    st.add(0, 2, 3);
    assert(st.sum(0, 4) == 15);
    assert(st.min(0, 4) == 1);
    assert(st.max(0, 4) == 7);
}
```

3.3.012 扫描线线段树

扫描线线段树把二维矩形问题降到一维：把每个矩形拆成左右两条竖边作为事件按横坐标排序，从左到右扫描，线段树在纵轴上维护「当前有多少矩形覆盖每一段」。这里线段树的下标区间对应离散化后的纵坐标段 $[ys[l], ys[r])$，每个节点存覆盖标记次数 `cov`、被至少一层覆盖的总长度 `len`、覆盖段数 `seg` 以及左右

端点是否被覆盖 `lc/rc`。`pull` 有个关键点：因为区间加减是成对出现且只在完整区间打标记，这里用的是「永久化标记 + 不下传」写法——`cov[p]>0` 时整段被覆盖直接取满长，否则由儿子合并，合并 `seg` 时若左儿子右端与右儿子左端相接要减一去重。相邻两条事件线之间，`len[1]` 乘横向距离累加进面积，`2*seg[1]` 乘横向距离累加进上下横边；同一横坐标先加入再删除，每次 `len[1]` 的变化量绝对值之和恰是该竖线贡献的周长：

```
template <class T> struct Rectangle
{
    // x1、y1、x2、y2 依次是矩形的左、下、右、上边界。
    T x1, y1, x2, y2;
};

template <class T, class Acc = T> class RectUnion
{
    struct Event
    {
        // x 是扫描线横坐标，[y1,y2) 是纵向覆盖段，d 为加入 1 或删除 -1。
        T x, y1, y2;
        int d;
    };

    // ys 保存离散后的纵坐标；下标区间 [l,r) 对应实际区间 [ys[l],ys[r])。
    vector<T> ys;
    // cov、seg、lc、rc 分别保存覆盖次数、覆盖段数和左右端覆盖状态。
    vector<int> cov, seg, lc, rc;
    // len 保存每个节点的实际覆盖长度，类型由 Acc 指定。
    vector<Acc> len;

    void pull(int p, int l, int r)
    {
        // p 是节点编号，[l,r) 是纵坐标下标区间；由覆盖标记和儿子信息更新节点。
        if (cov[p])
        {
            len[p] = (Acc)ys[r] - (Acc)ys[l];
            seg[p] = lc[p] = rc[p] = 1;
        }
        else if (r - l == 1)
        {
            len[p] = 0;
            seg[p] = lc[p] = rc[p] = 0;
        }
        else
        {
            int ls = p * 2, rs = ls + 1;
            len[p] = len[ls] + len[rs];
            seg[p] = seg[ls] + seg[rs] - (rc[ls] && lc[rs]);
            lc[p] = lc[ls];
            rc[p] = rc[rs];
        }
    }

```

```
    void add(int p, int l, int r, int ql, int qr, int d)
    {
        // p、[l,r) 是当前节点，[ql,qr) 是修改区间，d 是覆盖次数增量。
        if (ql <= l && r <= qr)
        {
            cov[p] += d;
            pull(p, l, r);
            return;
        }
        int m = midpoint(l, r);
        if (ql < m)
        {
            add(p * 2, l, m, ql, qr, d);
        }
        if (m < qr)
        {
            add(p * 2 + 1, m, r, ql, qr, d);
        }
        pull(p, l, r);
    }

    pair<Acc, Acc> calc(const vector<Rectangle<T>> &a)
    {
        // a 是输入矩形集合；返回值 first、second 依次为面积并和周长并。
        vector<Event> ev;
        ys.clear();
        for (auto [x1, y1, x2, y2] : a)
        {
            if (x1 >= x2 || y1 >= y2)
            {
                continue;
            }
            ev.push_back({x1, y1, y2, 1});
            ev.push_back({x2, y1, y2, -1});
            ys.push_back(y1);
            ys.push_back(y2);
        }
        if (ev.empty())
        {
            return {};
        }
        ranges::sort(ys);
        ys.erase(unique(ys.begin(), ys.end()), ys.end());
        ranges::sort(ev, {}, &Event::x);
        int n = (int)ys.size() * 4;
        cov.assign(n, 0);
        seg.assign(n, 0);
        lc.assign(n, 0);
        rc.assign(n, 0);
        len.assign(n, 0);

        Acc ar = 0;
```

```
        Acc peri = 0;
        T px = ev[0].x;
        for (int i = 0; i < (int)ev.size();)
        {
            int j = i;
            while (j < (int)ev.size() && ev[j].x == ev[i].x)
            {
                j++;
            }
            Acc dx = (Acc)ev[i].x - (Acc)px;
            ar += len[1] * dx;
            peri += (Acc)2 * seg[1] * dx;

            // 同一横坐标先加入再删除，逐次长度变化之和恰为新旧覆盖集合的对称差长度。
            for (int d : {1, -1})
            {
                for (int k = i; k < j; k++)
                {
                    if (ev[k].d != d)
                    {
                        continue;
                    }
                    Acc old = len[1];
                    int l = lower_bound(ys.begin(), ys.end(), ev[k].y1) - ys.begin();
                    int r = lower_bound(ys.begin(), ys.end(), ev[k].y2) - ys.begin();
                    add(1, 0, (int)ys.size() - 1, l, r, d);
                    Acc delta = len[1] - old;
                    peri += delta < 0 ? -delta : delta;
                }
            }
            px = ev[i].x;
            i = j;
        }
        return {ar, peri};
    }

public:
    Acc area(const vector<Rectangle<T>> &a)
    {
        // a 是输入矩形集合；返回全部轴平行矩形并集的面积。
        return calc(a).first;
    }

    Acc perimeter(const vector<Rectangle<T>> &a)
    {
        // a 是输入矩形集合；返回全部轴平行矩形并集的周长。
        return calc(a).second;
    }

    Acc solve(const vector<Rectangle<T>> &a)
    {
        // a 是输入矩形集合；兼容旧接口并返回矩形并集面积。
```

```
        return area(a);
    }
};
```

构造 `RectUnion<int> q`（或 `RectUnion<long long>`），`q.area(a)` 与 `q.perimeter(a)` 对同一组矩形分别返回面积并与周长并，`q.solve(a)` 是只取面积的兼容接口；默认 `Acc` 与坐标类型 `T` 相同，需要宽累计时显式构造 `RectUnion<T, __int128> q`。调用者须保证坐标差、覆盖长度、面积和周长均能由 `Acc` 表示，代码先转换端点再相减。浮点坐标仍使用 `T`。矩形 `a[i]` 的四个字段依次为左、下、右、上边界，统一左闭右开；退化矩形（`x1>=x2` 或 `y1>=y2`）被跳过，例如矩形 `{0,0,2,2}`、`{1,1,3,2}` 与退化的 `{5,0,5,3}` 面积并为 5、周长并为 10。n 个矩形产生 $2n$ 个事件，时间 $O(n\log n)$，空间 $O(n)$。同一份实现在 3.3.013 矩形面积并、周长并 复用，仅需覆盖长度时删去 `seg`/`lc`/`rc` 即退化为纯面积扫描线。

3.3.013 矩形面积并、周长并

这一条与 3.3.012 扫描线线段树 是同一知识，只是按“矩形面积并、周长并”的目录名再列一次，完整实现（`Rectangle<T>` 与 `RectUnion<T, Acc>`）见 3.3.012，不再重复粘贴。构造 `RectUnion<int> q` 后，`q.area(a)` 返回轴平行矩形并集面积、`q.perimeter(a)` 返回周长并；需要宽累计时改用 `RectUnion<int, __int128>`。例如矩形 `{0,0,2,2}`、`{1,1,3,2}` 与退化的 `{5,0,5,3}` 得 `area(a)==5`、`perimeter(a)==10`。n 个矩形共 $2n$ 个事件，时间 $O(n\log n)$、空间 $O(n)$。若还要支持“矩形加、矩形求和”这类带权二维差分而非布尔覆盖，改用 3.3.014 静态矩形加、矩形和。

3.3.014 静态矩形加、矩形和

本条就是 3.2.009 矩形修改、矩形求和 的静态离线形式，完整的 `RectAddSum<T>` 只在该处打印。一次矩形加与一次矩形和询问各拆成四个角点，按横坐标扫描；四棵树状数组分别维护 $\sum w$、$\sum wy$、$\sum wx$、$\sum wxy$，从而计算 $w(X-x)(Y-y)$ 的前缀贡献。

先用 `add(x1, y1, x2, y2, w)` 登记合法半开矩形加，再用 `query(x1, y1, x2, y2)` 登记询问并保存返回编号，最后调用一次 `work()`；`ans[id]` 是对应询问矩形内的权值和。所有询问都看到全部已登记修改，因而不能处理依赖先前答案的在线操作。设修改数为 n、询问数为 q，时间 $O((n+q)\log(n+q))$，空间 $O(n+q)$；布尔覆盖面积与周长改用 3.3.013 矩形面积并、周长并。

3.3.015 扫描线求矩形至少二重覆盖面积

求「至少被两个矩形共同覆盖」的面积，普通面积并那套只维护「至少覆盖一次的长度」就不够了，必须同时维护至少一次 `len1` 和至少两次 `len2` 两个长度。这里用永久化的整段覆盖计数 `cnt`（不下传），`pull` 时分三档合成：`cnt[p]>=2` 时整段已被覆盖两次以上，`len1=len2=full`；`cnt[p]==1` 时整段至少一次，故 `len1=full`，而「至少两次」恰是儿子里「至少一次」的部分（本层这一次叠上儿子那一次），即 `len2=` 儿子 `len1` 之和；`cnt[p]==0` 时直接取儿子的 `len1`、`len2` 之和。扫描时每推进一段横距离，累加 `seg.twice()`（即根的 `len2`）乘该横距离：

```
struct Cover2Seg
{
    vector<int> ys;   // ys 保存离散化后的纵坐标，叶子 i 表示 [ys[i], ys[i + 1])。
```

```
    vector<int> cnt;  // cnt[p] 是节点 p 对应区间被整段覆盖的次数。
    vector<int> len1; // len1[p] 是节点 p 内至少覆盖一次的实际长度。
    vector<int> len2; // len2[p] 是节点 p 内至少覆盖两次的实际长度。

    Cover2Seg(vector<int> y = {}) : ys(move(y))
    {
        // y 是严格递增的纵坐标；初始覆盖次数和长度均为零。
        int n = max<int>(1, (int)ys.size() - 1);
        cnt.assign(4 * n, 0);
        len1.assign(4 * n, 0);
        len2.assign(4 * n, 0);
    }

    void pull(int p, int l, int r)
    {
        // p、[l,r) 是当前节点；重算至少一重与至少二重覆盖长度。
        int full = ys[r] - ys[l];
        if (cnt[p] >= 2)
        {
            len1[p] = len2[p] = full;
            return;
        }
        int one = 0;
        int two = 0;
        if (r - l > 1)
        {
            one = len1[2 * p] + len1[2 * p + 1];
            two = len2[2 * p] + len2[2 * p + 1];
        }
        if (cnt[p] == 1)
        {
            len1[p] = full;
            len2[p] = one;
        }
        else
        {
            len1[p] = one;
            len2[p] = two;
        }
    }

    void add(int p, int l, int r, int ql, int qr, int v)
    {
        // p、[l,r) 是当前节点，  [ql,qr) 是修改区间，v 是覆盖次数增量。
        if (qr <= l || r <= ql)
        {
            return;
        }
        if (ql <= l && r <= qr)
        {
            cnt[p] += v;
            pull(p, l, r);
```

```
            return;
        }
        auto m = (l + r) / 2;
        add(2 * p, l, m, ql, qr, v);
        add(2 * p + 1, m, r, ql, qr, v);
        pull(p, l, r);
    }

    void add(int l, int r, int v)
    {
        // [l,r) 是纵坐标段下标，v 为加入 1 或删除 -1。
        if (l < r)
        {
            add(1, 0, (int)ys.size() - 1, l, r, v);
        }
    }

    int twice() const
    {
        // 返回当前纵轴上至少被覆盖两次的总长度。
        return len2[1];
    }
};

// Rect[0..3] 依次是左 x、下 y、右 x、上 y，表示半开矩形 [x1,x2) x [y1,y2)。
using Rect2 = array<int, 4>;

int rectCover2(const vector<Rect2> &a)
{
    // a 是半开矩形集合；返回至少被两个矩形覆盖的区域面积。
    // Event[0..3] 依次是横坐标、下纵坐标、上纵坐标、覆盖增量。
    using Event = array<int, 4>;
    vector<int> ys;
    vector<Event> ev;
    for (auto [x1, y1, x2, y2] : a)
    {
        if (x1 >= x2 || y1 >= y2)
        {
            continue;
        }
        ys.push_back(y1);
        ys.push_back(y2);
        ev.push_back({x1, y1, y2, 1});
        ev.push_back({x2, y1, y2, -1});
    }
    if (ev.empty())
    {
        return 0;
    }
    sort(ys.begin(), ys.end());
    ys.erase(unique(ys.begin(), ys.end()), ys.end());
    sort(ev.begin(), ev.end());
```

```
    Cover2Seg seg(ys);
    int ans = 0;
    int pre = ev[0][0];
    for (int i = 0; i < (int)ev.size();)
    {
        auto x = ev[i][0];
        ans += (x - pre) * seg.twice();
        while (i < (int)ev.size() && ev[i][0] == x)
        {
            int l = (int)(lower_bound(ys.begin(), ys.end(), ev[i][1]) - ys.begin());
            int r = (int)(lower_bound(ys.begin(), ys.end(), ev[i][2]) - ys.begin());
            seg.add(l, r, ev[i][3]);
            i++;
        }
        pre = x;
    }
    return ans;
}
```

直接调用 `rectCover2(a)`：a 是 `vector<array<int,4>>`，每项 `{x1,y1,x2,y2}` 为左、下、右、上，按左闭右开处理，退化矩形被忽略。调用者须保证坐标差、至少一重和至少二重覆盖长度及最终面积都能由 `int` 表示。例如矩形 `{0,0,4,3}`、`{2,1,5,4}`、`{1,2,3,5}` 的至少二重覆盖面积为 6：三组两两交面积之和为 8，三重交面积为 1，故至少二重部分为 $8-2\times1=6$。时间 $O(n\log n)$、空间 $O(n)$。把 `Cover2Seg` 的 `len2` 层数推广到 k 层即可求“至少 k 重覆盖”；只要普通面积并用 3.3.013 矩形面积并、周长并 即可，无需这里的双长度维护。

3.3.016 线段树分治

把「操作只在某段时间内有效」的离线问题挂到时间线段树上。将时间轴 $[0,q-1]$ 建成线段树，一个存活于时间区间 $[l,r]$ 的操作，被拆挂到覆盖 $[l,r]$ 的 $O(\log q)$ 个结点上。DFS 这棵线段树：进入某结点前把它挂的操作作用到一个可撤销数据结构上，递归到叶子即读出该时刻答案，回溯时按栈撤销本结点的全部操作。这样每个操作只在其有效时间被加入、离开时被撤销，避免了删除难以支持的问题。

下例判定「每个时刻的动态图是否二分图」：边 (u,v) 只在若干时间段存在。用扩展域并查集，把每个点 x 拆成「本色 x」与「异色 $x+n$」，加边即合并 u 与 $v+n$、$u+n$ 与 v（强制 u,v 异色）；若某刻发现 u,v 已在同一集合（同色），则出现奇环，该时间段起图非二分。数据结构必须是可撤销并查集（按秩合并、不路径压缩才能 $O(\log)$ 回滚），见 3.1.004 可撤销并查集，此处直接复用其 `RollbackDSU`（接口 `snap` / `merge` / `rollback` / `same`）。

```
// 依赖 3.1.004 可撤销并查集
// 线段树分治：离线判每个时刻的图是否二分图（扩展域 + 可撤销并查集 3.1.004）。
struct OfflineBip
{
    int n, q; // n 是点数，q 是时间轴长度（操作数）。
    vector<vector<array<int, 2>>> seg; // seg[p] 保存时间节点 p 全程存在的边 {u,v}。
    vector<char> ans;                  // ans[t] 表示时刻 t 的图是否为二分图。

    OfflineBip(int n, int q) : n(n), q(q), seg(4 * max<int>(q, 1)), ans(q)
```

```
    {
        // n 是点数，q 是时刻数；允许空时间轴 q=0。
        assert(n >= 0 && q >= 0); // 调试检查，可删
    }

    void addEdge(int u, int v, int l, int r, int node = 1, int lo = 0, int hi = -1)
    {
        // 边 (u,v) 在时刻区间 [l,r] 存在；挂到覆盖它的 O(log q) 个结点。
        assert(q > 0 && 0 <= u && u < n && 0 <= v && v < n && 0 <= l && l <= r && r < q); // 调
        ↪ 试检查，可删
        if (hi < 0)
        {
            hi = q - 1;
        }
        if (l > hi || r < lo)
        {
            return;
        }
        if (l <= lo && hi <= r)
        {
            seg[node].push_back({u, v});
            return;
        }
        int mid = (lo + hi) / 2;
        addEdge(u, v, l, r, node * 2, lo, mid);
        addEdge(u, v, l, r, node * 2 + 1, mid + 1, hi);
    }

    void solve(RollbackDSU &dsu, int node = 1, int lo = 0, int hi = -1, bool ok = true)
    {
        // dsu 大小为 2n（扩展域），ok 记当前时间段是否仍是二分图。
        if (!q)
        {
            return;
        }
        if (hi < 0)
        {
            hi = q - 1;
        }
        int s = dsu.snap();
        for (auto [u, v] : seg[node])
        {
            if (dsu.same(u, v))
            {
                ok = false;
                break;
            }
            dsu.merge(u, v + n);
            dsu.merge(u + n, v);
        }
        if (lo == hi)
        {
```

```
            ans[lo] = ok;
        }
        else
        {
            int mid = (lo + hi) / 2;
            solve(dsu, node * 2, lo, mid, ok);
            solve(dsu, node * 2 + 1, mid + 1, hi, ok);
        }
        dsu.rollback(s);
    }
};
```

`OfflineBip(n,q)` 后对每条边调用 `addEdge(u,v,l,r)`，把它挂到闭时间段 $[l,r]$；再传入大小为 $2n$ 的 `RollbackDSU` 调 `solve`，`ans[t]` 即时刻 t 的图是否二分。`q=0` 时 `solve` 直接返回空答案。验证：$n=3$、边 $(0,1),(1,2)$ 全程存在、$(0,2)$ 仅在 $t=1$ 存在，答案为 `1 0 1`。同一框架换掉可撤销状态即可处理其他时间区间操作。

设登记了 m 条存活区间，初始化扩展域为 $O(n)$，遍历时间树为 $O(q)$，每条边挂到 $O(\log q)$ 个节点并各做常数次 $O(\log n)$ 的并查集操作，总时间 $O(n+q+m\log q\log n)$；时间树挂边、答案和回滚状态合计空间 $O(n+q+m\log q)$。

3.3.017 动态区间严格众数

严格众数是出现次数大于区间长度一半的值，若存在则唯一。Boyer-Moore 抵消把一段序列压成 {候选值，未抵消票数}：相同候选合并票数，不同候选让较小票数被抵消；无论线段树怎样分段，真正的严格众数都不会被全部抵消。不过抵消结果在没有严格众数时仍可能留下任意候选，甚至票数恰好为零，因此查询后必须先检查剩余票数，再数一次候选在 $[l,r)$ 内的真实频次。本实现预先登记所有可能的单点赋值 {位置，新值}，为每个值单独建立按位置离散的 Fenwick；线段树只找候选，Fenwick 负责验证。

```
class MajRange
{
    // S[0] 是候选值编号，S[1] 是抵消后的剩余票数。
    using S = array<int, 2>;

    // n 是数组长度，z 是线段树叶子基址；a 保存当前值编号，val 保存原值。
    int n, z = 1;
    vector<int> a, val;
    vector<S> tr;
    // pos[k] 是值 k 预登记的位置，bit[k] 维护这些位置当前是否取值 k。
    vector<vector<int>> pos, bit;

    static S merge(S x, S y)
    {
        // x、y 是相邻两段的抵消摘要；返回合并摘要。
        if (!x[1])
        {
            return y;
        }
        if (!y[1])
```

```
        {
            return x;
        }
        if (x[0] == y[0])
        {
            return {x[0], x[1] + y[1]};
        }
        return x[1] >= y[1] ? S{x[0], x[1] - y[1]} : S{y[0], y[1] - x[1]};
    }

    int id(int x) const
    {
        // x 是已预登记的原值；返回其离散编号。
        auto it = lower_bound(val.begin(), val.end(), x);
        assert(it != val.end() && *it == x); // 调试检查，可删
        return it - val.begin();
    }

    void add(int k, int p, int d)
    {
        // k 是值编号，p 是预登记位置，d 是出现次数增量。
        auto it = lower_bound(pos[k].begin(), pos[k].end(), p);
        assert(it != pos[k].end() && *it == p); // 调试检查，可删
        for (int i = it - pos[k].begin() + 1; i < (int)bit[k].size(); i += i & -i)
        {
            bit[k][i] += d;
        }
    }

    int pre(int k, int p) const
    {
        // k 是值编号，p 是位置上界；返回该值在 [0,p) 的出现次数。
        int i = lower_bound(pos[k].begin(), pos[k].end(), p) - pos[k].begin();
        int ans = 0;
        for (; i; i -= i & -i)
        {
            ans += bit[k][i];
        }
        return ans;
    }

    void setLeaf(int p, int x)
    {
        // p 是数组位置，x 是新值编号；修改候选线段树的对应叶子。
        p += z;
        tr[p] = {x, 1};
        while (p >>= 1)
        {
            tr[p] = merge(tr[2 * p], tr[2 * p + 1]);
        }
    }

```

```
    S prod(int l, int r) const
    {
        // [l,r) 是查询区间；返回按原顺序抵消后的候选摘要。
        S x{-1, 0}, y{-1, 0};
        for (l += z, r += z; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1)
            {
                x = merge(x, tr[l++]);
            }
            if (r & 1)
            {
                y = merge(tr[--r], y);
            }
        }
        return merge(x, y);
    }

  public:
    MajRange(const vector<int> &v, const vector<pair<int, int>> &plan = {}) : n(v.size())
    {
        // v 是初始数组，plan 列出以后所有可能的 {赋值位置，新值}。
        val = v;
        for (auto [p, x] : plan)
        {
            assert(0 <= p && p < n); // 调试检查，可删
            val.push_back(x);
        }
        sort(val.begin(), val.end());
        val.erase(unique(val.begin(), val.end()), val.end());
        a.resize(n);
        pos.resize(val.size());
        for (int i = 0; i < n; i++)
        {
            a[i] = id(v[i]);
            pos[a[i]].push_back(i);
        }
        for (auto [p, x] : plan)
        {
            pos[id(x)].push_back(p);
        }
        bit.resize(pos.size());
        for (int k = 0; k < (int)pos.size(); k++)
        {
            auto &p = pos[k];
            sort(p.begin(), p.end());
            p.erase(unique(p.begin(), p.end()), p.end());
            bit[k].assign(p.size() + 1, 0);
        }
        while (z < max<int>(1, n))
        {
            z *= 2;
```

```
        }
        tr.assign(2 * z, S{-1, 0});
        for (int i = 0; i < n; i++)
        {
            tr[z + i] = {a[i], 1};
            add(a[i], i, 1);
        }
        for (int p = z - 1; p; p--)
        {
            tr[p] = merge(tr[2 * p], tr[2 * p + 1]);
        }
    }

    void set(int p, int x)
    {
        // p 是数组位置，x 是已在 plan 登记的新值；完成单点赋值。
        assert(0 <= p && p < n); // 调试检查，可删
        int k = id(x);
        add(a[p], p, -1);
        add(k, p, 1);
        a[p] = k;
        setLeaf(p, k);
    }

    optional<int> query(int l, int r) const
    {
        // [l,r) 是查询区间；返回严格众数，空区间或不存在时返回空。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r)
        {
            return nullopt;
        }
        S s = prod(l, r);
        if (!s[1])
        {
            return nullopt;
        }
        int k = s[0];
        int c = pre(k, r) - pre(k, l);
        return 2 * c > r - l ? optional<int>(val[k]) : nullopt;
    }
};
```

构造 `MajRange q(a,plan)` 时，`plan` 必须覆盖以后每次 `set(p,x)` 的位置和值组合；查询端点不必预登记。`set` 与 `query` 均为 $O(\log n)$，预处理 $O((n+q)\log(n+q))$，空间 $O(n+q)$。若数组静态且只需任意众数而非严格过半，用 3.4.008 静态区间众数；严格众数的优势正是候选唯一，不能把本结构改名成一般区间众数。

```
signed main()
{
    vector<int> a{1, 2, 1, 3, 1};
```

```
    vector<pair<int, int>> plan{{1, 1}};
    MajRange q(a, plan);
    assert(q.query(0, 5) == optional<int>(1));
    assert(!q.query(0, 2));
    q.set(1, 1);
    assert(q.query(0, 4) == optional<int>(1));
}
```

3.3.018 可排序线段树

可排序线段树维护一列 {键，幺半群值}，支持单点赋值、区间按键升序或降序排序，以及按当前次序查询区间积。它把序列划成若干已经单调的段：`cut` 记录段首，每段用一棵动态值域线段树保存键，节点同时维护升序积 `f` 和降序积 `b`；外层线段树只在段首保存整段积。查询或排序先用 `splitAt` 在端点按元素个数切段，排序再破坏性合并区间内各值域根并设置方向。所有键必须始终全局互异，否则两棵值域树会在同一叶相撞，排序次序也不再由键唯一确定。

```
template <class M> class SortSeg
{
  public:
    using T = typename M::T;

  private:
    struct Node
    {
        // l、r 是值域左右儿子；sz 是元素数；f、b 分别按键升序、降序合并。
        int l = 0, r = 0, sz = 0;
        T f, b;

        Node(const T &e) : f(e), b(e) {}
    };

    // n 是序列长度，K 是键值域右端点，z 是外层线段树叶子基址。
    int n, K, z = 1;
    // t 是动态值域线段树节点池；rt[p] 是以 p 开头的有序段根；rev[p] 表示该段降序。
    vector<Node> t;
    vector<int> rt;
    vector<char> rev;
    // cut 保存所有有序段左端点及哨兵 n；seg 仅在段首保存整段幺半群积。
    set<int> cut;
    vector<T> seg;

    int newNode()
    {
        // 新建空值域节点并返回编号。
        t.emplace_back(M::id());
        return (int)t.size() - 1;
    }

    int size(int p) const
```

```
    {
        // p 是值域节点编号；返回其元素数，空节点返回 0。
        return p ? t[p].sz : 0;
    }

    void pull(int p)
    {
        // p 是非叶值域节点；由非空儿子重算元素数和双向积。
        int x = t[p].l, y = t[p].r;
        t[p].sz = size(x) + size(y);
        if (!x)
        {
            t[p].f = t[y].f;
            t[p].b = t[y].b;
        }
        else if (!y)
        {
            t[p].f = t[x].f;
            t[p].b = t[x].b;
        }
        else
        {
            t[p].f = M::op(t[x].f, t[y].f);
            t[p].b = M::op(t[y].b, t[x].b);
        }
    }

    int put(int p, int l, int r, int k, const T &x)
    {
        // p 管理键区间 [l,r)，k 是新键，x 是对应值；返回插入后的根。
        if (!p)
        {
            p = newNode();
        }
        if (r - l == 1)
        {
            t[p].sz = 1;
            t[p].f = t[p].b = x;
            return p;
        }
        int m = l + (r - l) / 2;
        if (k < m)
        {
            t[p].l = put(t[p].l, l, m, k, x);
        }
        else
        {
            t[p].r = put(t[p].r, m, r, k, x);
        }
        pull(p);
        return p;
    }
```

```
86
87      pair<int, int> split(int p, int k, int l, int r)
88      {
89          // p 管理键区间 [l,r)，k 是左侧所取元素数；返回按键升序切开的两根。
90          if (!k)
91          {
92              return {0, p};
93          }
94          if (k == size(p))
95          {
96              return {p, 0};
97          }
98          int m = l + (r - l) / 2;
99          int q = newNode();
100         int s = size(t[p].l);
101         if (k <= s)
102         {
103             auto [x, y] = split(t[p].l, k, l, m);
104             int v = t[p].r;
105             t[p].l = x;
106             t[p].r = 0;
107             t[q].l = y;
108             t[q].r = v;
109         }
110         else
111         {
112             auto [x, y] = split(t[p].r, k - s, m, r);
113             int v = t[p].l;
114             t[p].l = v;
115             t[p].r = x;
116             t[q].l = 0;
117             t[q].r = y;
118         }
119         pull(p);
120         pull(q);
121         return {p, q};
122     }
123
124     int join(int p, int q, int l, int r)
125     {
126         // p、q 管理同一键区间 [l,r) 且键集不交；破坏性合并并返回根。
127         if (!p || !q)
128         {
129             return p | q;
130         }
131         assert(r - l > 1); // 调试检查，可删：合法键全局互异，不会在同一叶相遇
132         int m = l + (r - l) / 2;
133         t[p].l = join(t[p].l, t[q].l, l, m);
134         t[p].r = join(t[p].r, t[q].r, m, r);
135         pull(p);
136         return p;
137     }
```

```
138
139     T all(int p, bool d) const
140     {
141         // p 是非空值域根，d 表示是否降序；返回该顺序下的整段积。
142         return d ? t[p].b : t[p].f;
143     }
144
145     void outSet(int p, const T &x)
146     {
147         // p 是序列位置，x 是新的段首积或幺元；修改外层线段树。
148         p += z;
149         seg[p] = x;
150         while (p >>= 1)
151         {
152             seg[p] = M::op(seg[2 * p], seg[2 * p + 1]);
153         }
154     }
155
156     T outProd(int l, int r) const
157     {
158         // [l,r) 是序列区间；返回外层线段树中段首积的顺序合并。
159         T x = M::id(), y = M::id();
160         for (l += z, r += z; l < r; l >>= 1, r >>= 1)
161         {
162             if (l & 1)
163             {
164                 x = M::op(x, seg[l++]);
165             }
166             if (r & 1)
167             {
168                 y = M::op(seg[--r], y);
169             }
170         }
171         return M::op(x, y);
172     }
173
174     void splitAt(int p)
175     {
176         // p 是序列边界；若它落在有序段内部，就把该段切成两段。
177         if (p == n || cut.contains(p))
178         {
179             return;
180         }
181         auto it = cut.upper_bound(p);
182         int r = *it, l = *prev(it);
183         cut.insert(p);
184         if (!rev[l])
185         {
186             auto [x, y] = split(rt[l], p - l, 0, K);
187             rt[l] = x;
188             rt[p] = y;
189             rev[l] = rev[p] = false;
```

```
        }
        else
        {
            auto [x, y] = split(rt[l], r - p, 0, K);
            rt[l] = y;
            rt[p] = x;
            rev[l] = rev[p] = true;
        }
        outSet(l, all(rt[l], rev[l]));
        outSet(p, all(rt[p], rev[p]));
    }

    void sortRange(int l, int r, bool d)
    {
        // [l,r) 是排序区间，d 表示是否降序；合并其中各段并设置方向。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r)
        {
            return;
        }
        splitAt(l);
        splitAt(r);
        auto it = cut.upper_bound(l);
        while (*it < r)
        {
            int p = *it;
            rt[l] = join(rt[l], rt[p], 0, K);
            rt[p] = 0;
            rev[p] = false;
            outSet(p, M::id());
            it = cut.erase(it);
        }
        rev[l] = d;
        outSet(l, all(rt[l], d));
    }

  public:
    SortSeg(int K, const vector<int> &key, const vector<T> &a) : n(key.size()), K(K), rt(n),
↪   rev(n)
    {
        // K 是键值域 [0,K) 的右端点；key、a 分别是每个位置的互异键和幺半群值。
        assert(K > 0 && key.size() == a.size()); // 调试检查，可删
        t.emplace_back(M::id());
        while (z < max<int>(1, n))
        {
            z *= 2;
        }
        seg.assign(2 * z, M::id());
        cut.insert(n);
        for (int i = 0; i < n; i++)
        {
            assert(0 <= key[i] && key[i] < K); // 调试检查，可删
```

```
            cut.insert(i);
            rt[i] = put(0, 0, K, key[i], a[i]);
            seg[z + i] = a[i];
        }
        for (int p = z - 1; p; p--)
        {
            seg[p] = M::op(seg[2 * p], seg[2 * p + 1]);
        }
    }

    void assign(int p, int k, const T &x)
    {
        // p 是序列位置，k 是替换后的全局唯一键，x 是新值；完成单点赋值。
        assert(0 <= p && p < n && 0 <= k && k < K); // 调试检查，可删
        splitAt(p);
        splitAt(p + 1);
        rt[p] = put(0, 0, K, k, x);
        rev[p] = false;
        outSet(p, x);
    }

    void sortAsc(int l, int r)
    {
        // [l,r) 是排序区间；按键从小到大重排。
        sortRange(l, r, false);
    }

    void sortDesc(int l, int r)
    {
        // [l,r) 是排序区间；按键从大到小重排。
        sortRange(l, r, true);
    }

    T prod(int l, int r)
    {
        // [l,r) 是查询区间；返回当前顺序下的幺半群积。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        splitAt(l);
        splitAt(r);
        return outProd(l, r);
    }

    T prodAll() const
    {
        // 返回整个序列当前顺序下的幺半群积。
        return seg[1];
    }
};
```

`M::T` 是值类型，`M::id()` 返回幺元，`M::op(x,y)` 按先左后右的次序结合；键位于 $[0,K)$，构造和每次 `assign(p,k,x)` 后都必须两两不同。`sortAsc(l,r)`、`sortDesc(l,r)` 重排 $[l,r)$，`prod(l,r)` 返回当前顺序下的积。设值域树高 $H=O(\log K)$；单点赋值和区间查询为 $O(\log n+H)$。一次排序可能合并很多

现存段，但每个被删段界都来自初始化或一次端点切分，每个被重新访问的值域分支也可计入此前建立或切分出的节点，因此从建树起的 q 次操作总时间为 $O((n+q)(\log n+H))$，空间为 $O(n+(n+q)H)$。下面用非交换的仿射函数复合检查升序、降序、区间积与单点替换：

```
constexpr int mod = 998244353;

struct Aff
{
    int a, b; // 仿射变换 x -> a*x+b 的两个系数（模 mod）。
    bool operator==(const Aff &) const = default;
};

struct AffMonoid
{
    using T = Aff;

    static T id()
    {
        // 无参数；返回恒等变换 x -> x。
        return {1, 0};
    }

    static T op(T x, T y)
    {
        // x、y 是仿射变换；返回先做 x 再做 y 的复合变换。
        return {y.a * x.a % mod, (y.a * x.b + y.b) % mod};
    }
};

signed main()
{
    vector<int> key{3, 1, 2, 0};
    vector<Aff> a{{1, 1}, {2, 0}, {1, 3}, {3, 1}};
    SortSeg<AffMonoid> st(8, key, a);
    assert((st.prodAll() == Aff{6, 16}));
    st.sortAsc(0, 4);
    assert((st.prodAll() == Aff{6, 6}));
    st.sortDesc(1, 4);
    assert((st.prod(1, 3) == Aff{1, 4}));
    st.assign(2, 7, {2, 1});
    st.sortAsc(1, 4);
    assert((st.prodAll() == Aff{12, 7}));
}
```

3.3.019 区间一次函数加、区间最小值

把数组看成点集 (i,a_i)；修改 $[l,r)$ 时给每个纵坐标加 $bi+c$，就是对相应点集施加保持横坐标的仿射剪切，整段修改不会改变该段下凸包的组合结构。`LineMinSeg` 在每个内部节点保存左右儿子下凸包的公切桥，懒标记保存尚未下放的一次函数。部分修改后，`pull` 沿两棵儿子的桥递归寻找新公切点；查询一个完整节点时比较当前桥两端的纵坐标，就能向含最小点的儿子下降。这里的桥判定不是普通线段树的常数时

间合并，不能删成 `min` 信息后仍声称支持区间一次函数加。复用为最大值版时，仅当初值、增量与函数值都能安全取反，才保存 `-a[i]`、把 `add(l,r,b,c)` 改为 `add(l,r,-b,-c)` 并对查询结果取负；只把最后的 `min` 改成 `max` 会破坏下凸包桥判定，遇到最小负数则必须完整反向比较。

```
class LineMinSeg
{
    // 当前维护区间最小值。若 a、b、c 均能安全取反，可保存 -a[i]：每次 add(l,r,b,c) 改为
    // add(l,r,-b,-c)，查询结果再取负；若存在最小负数则完整反向桥判定，不要只改最后一个 min。
    struct P
    {
        // x 是数组下标，y 是该位置的当前值。
        long long x = 0, y = 0;

        bool operator==(const P &) const = default;
    };

    // L[0]、L[1] 分别是一次函数的斜率和截距；B[0]、B[1] 是桥的左右端点。
    using L = array<long long, 2>;
    using B = array<P, 2>;

    // n 是原数组长度，z 是补成二次幂后的叶子数，h 是树高。
    int n, z = 1, h = 0;
    // pt 是叶子点；lz 是整段待加的一次函数；br 是两棵儿子下凸包的公切桥。
    vector<P> pt;
    vector<L> lz;
    vector<B> br;

    static L merge(L a, L b)
    {
        // a、b 是先后累加的一次函数；返回逐项相加后的函数。
        a[0] += b[0];
        a[1] += b[1];
        return a;
    }

    static P apply(P p, L f)
    {
        // p 是平面点，f 是待加的一次函数；返回纵坐标增加 f(p.x) 后的点。
        p.y = (long long)((__int128)p.y + (__int128)p.x * f[0] + f[1]);
        return p;
    }

    static __int128 cross(P a, P b, P c)
    {
        // a、b、c 是三个点；返回向量 ab 与 ac 的叉积。
        return ((__int128)b.x - a.x) * ((__int128)c.y - a.y)
             - ((__int128)b.y - a.y) * ((__int128)c.x - a.x);
    }

    static __int128 cross(P a, P b, P c, P d)
    {
        // a、b、c、d 是四个点；返回向量 ab 与 cd 的叉积。
```

```
        return ((__int128)b.x - a.x) * ((__int128)d.y - c.y)
             - ((__int128)b.y - a.y) * ((__int128)d.x - c.x);
    }

    void pull(int p)
    {
        // p 是内部节点；重新寻找左右儿子下凸包之间的公切桥。
        if (p >= z)
        {
            return;
        }
        L off{0, 0};
        for (int x = p; x; x >>= 1)
        {
            off = merge(off, lz[x]);
        }
        int l = 2 * p, r = 2 * p + 1;
        int mid = r;
        while (mid < z)
        {
            mid *= 2;
        }
        mid -= z;
        L fl = lz[l], fr = lz[r];
        while (l < z || r < z)
        {
            P a = apply(apply(br[l][0], off), fl);
            P b = apply(apply(br[l][1], off), fl);
            P c = apply(apply(br[r][0], off), fr);
            P d = apply(apply(br[r][1], off), fr);
            if (a != b && cross(a, b, c) < 0)
            {
                l *= 2;
                fl = merge(fl, lz[l]);
            }
            else if (c != d && cross(b, c, d) < 0)
            {
                r = 2 * r + 1;
                fr = merge(fr, lz[r]);
            }
            else if (a == b)
            {
                r *= 2;
                fr = merge(fr, lz[r]);
            }
            else if (c == d)
            {
                l = 2 * l + 1;
                fl = merge(fl, lz[l]);
            }
            else
            {
```

```
                __int128 c1 = cross(a, b, c, d);
                __int128 c2 = cross(a, b, c, b);
                bool left;
                if (!c1 && !c2)
                {
                    left = c.x < mid;
                }
                else
                {
                    left = (__int128)c.x * c1 + ((__int128)d.x - c.x) * c2
                         < c1 * mid;
                }
                if (left)
                {
                    l = 2 * l + 1;
                    fl = merge(fl, lz[l]);
                }
                else
                {
                    r *= 2;
                    fr = merge(fr, lz[r]);
                }
            }
        }
        br[p] = {apply(pt[l], fl), apply(pt[r], fr)};
    }

    long long subMin(int p) const
    {
        // p 是完整覆盖的线段树节点；返回其区间最小值。
        L f{0, 0};
        for (int x = p; x; x >>= 1)
        {
            f = merge(f, lz[x]);
        }
        while (p < z)
        {
            P a = apply(br[p][0], f), b = apply(br[p][1], f);
            p = a.y < b.y ? 2 * p : 2 * p + 1;
            f = merge(f, lz[p]);
        }
        return apply(pt[p], f).y;
    }

  public:
    LineMinSeg(const vector<long long> &a) : n(a.size())
    {
        // a 是非空初始数组；建立每个节点的下凸包桥。
        assert(n > 0); // 调试检查，可删
        while (z < n)
        {
            z *= 2;
```

```
            h++;
        }
        pt.resize(2 * z);
        lz.assign(2 * z, L{0, 0});
        br.resize(2 * z);
        for (int i = 0; i < z; i++)
        {
            pt[z + i] = {i, i < n ? a[i] : 4'000'000'000'000'000'000LL};
            br[z + i] = {pt[z + i], pt[z + i]};
        }
        for (int p = z - 1; p; p--)
        {
            pull(p);
        }
    }

    void add(int l, int r, long long b, long long c)
    {
        // [l,r) 是修改区间，b、c 定义一次函数 b*i+c；把它加到每个 a[i]。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r)
        {
            return;
        }
        int x = l + z, y = r + z;
        while (x < y)
        {
            if (x & 1)
            {
                lz[x] = merge(lz[x], L{b, c});
                x++;
            }
            if (y & 1)
            {
                --y;
                lz[y] = merge(lz[y], L{b, c});
            }
            x >>= 1;
            y >>= 1;
        }
        x = l + z;
        y = r + z;
        for (int d = 1; d <= h; d++)
        {
            if ((x >> d << d) != x)
            {
                pull(x >> d);
            }
            if ((y >> d << d) != y)
            {
                pull((y - 1) >> d);
            }
```

```
        }
    }

    long long min(int l, int r) const
    {
        // [l,r) 是非空查询区间；返回其中最小值。
        assert(0 <= l && l < r && r <= n); // 调试检查，可删
        long long ans = 4'000'000'000'000'000'000LL;
        for (l += z, r += z; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1)
            {
                long long v = subMin(l++);
                ans = ans < v ? ans : v;
            }
            if (r & 1)
            {
                long long v = subMin(--r);
                ans = ans < v ? ans : v;
            }
        }
        return ans;
    }
};
```

构造 `LineMinSeg st(a)` 后，`st.add(l,r,b,c)` 执行 $a_i \mathrel{+}= bi + c$ $(l \le i < r)$，`st.min(l,r)` 查询非空区间最小值。建树 $O(n \log n)$，一次修改或查询均为 $O(\log^2 n)$，空间 $O(n)$。调用者必须保证初值、维护中的每个 a_i、累积斜率与截距及 `a_i+bi+c` 都可由 `long long` 表示，且 $|a_i| < 4 \times 10^{18}$；所有叉积和桥侧判定还须落在 `__int128` 内，例如 $n \le 2 \times 10^5$ 且所有维护值绝对值严格小于 4×10^{18} 时满足这一项。只有区间加常数时用 3.3.002 懒标记线段树 更短；只有一次函数修改而查询区间和时，按下标和长度维护两项和即可，不需要凸包桥。下面用四个数检查局部一次函数修改和跨段查询：

```
signed main()
{
    LineMinSeg st(vector<long long>{5, -2, 4, 1});
    assert(st.min(0, 4) == -2);
    st.add(1, 4, 2, -3);
    assert(st.min(0, 4) == -3);
    assert(st.min(2, 4) == 4);
    st.add(0, 2, -1, 4);
    assert(st.min(0, 4) == 0);
}
```

3.3.020 可持久化仿射区间和

给定固定长度的数组，若每次修改都要从任意历史版本分叉，对半开区间 $[l, r)$ 施加 $x \mapsto ax + b$，还要把另一个历史版本的同下标区间覆盖过来，就不能直接使用会原地改写节点的 3.3.003 区间加、乘、赋值（仿射懒标记）。`PerAffSeg<T>` 为每个版本保存一个根编号；`affine(root,l,r,mul,add)` 返回新根，`splice(target,source,l,r)` 返回把 `target` 的 $[l, r)$ 替换为 `source` 同一坐标 $[l, r)$ 后的新根，

`sum(root,l,r)` 则只读查询任意历史版本。它处理的是固定坐标上的版本拼接，不支持插入、删除或变长序列。

每个节点的 `sum` 已经计入自身懒标记 (m, a)，而两个标记按先旧后新的次序复合：

$$(m_2, a_2) \circ (m_1, a_1) = (m_1 m_2, a_1 m_2 + a_2), \qquad S \leftarrow mS + (R - L)a.$$

所有会写节点的路径先复制；`push` 只接收刚复制的父节点，并把标记施加到新复制的两个儿子，所以旧根永不被改写。查询不下传标记，而是携带祖先尚未落实的仿射变换。`splice` 部分相交时同样只复制目标路径；源节点若带标记，仅复制实际进入的源儿子并落实该标记。这样覆盖整段时可以直接复用源根，分叉和交叉复制都保持持久化。

```
// 依赖 1.1.008 静态取模类
template <class T> class PerAffSeg
{
    struct Node
    {
        // l、r 是左右儿子编号；sum 是当前区间已经计入本节点懒标记后的元素和。
        int l = 0, r = 0;
        T sum{};
        // mul、add 是尚未下传的变换 x -> mul*x+add。
        T mul{}, add{};
    };

    // n 是初始数组长度，one 是 T 的乘法单位元，base 是初始版本根。
    int n;
    T one;
    int base = 0;
    // t 是全部版本共用的节点池；0 号节点只作哨兵，不是合法版本根。
    vector<Node> t;

    int newNode()
    {
        // 新建空节点并返回编号；新节点的仿射标记初始化为恒等变换。
        t.push_back(Node{});
        int p = (int)t.size() - 1;
        assert(p <= numeric_limits<int>::max()); // 调试检查，可删
        t[p].mul = one;
        return p;
    }

    int clone(int p)
    {
        // p 是旧版本中的非空节点；复制它并返回新节点编号。
        assert(p > 0); // 调试检查，可删
        t.push_back(t[p]);
        int q = (int)t.size() - 1;
        assert(q <= numeric_limits<int>::max()); // 调试检查，可删
        return q;
    }

```

```
    bool idle(int p) const
    {
        // p 是非空节点；返回其待下传仿射标记是否为恒等变换。
        return t[p].mul == one && t[p].add == T{};
    }

    void pull(int p)
    {
        // p 是已经无本层懒标记的内部节点；由两个儿子重算当前区间和。
        t[p].sum = t[t[p].l].sum + t[t[p].r].sum;
    }

    void applyAt(int p, int l, int r, const T &mul, const T &add)
    {
        // p 是可独占修改的节点，[l,r) 是其区间；就地叠加后执行的仿射变换。
        t[p].sum = t[p].sum * mul + T(r - l) * add;
        t[p].mul = t[p].mul * mul;
        t[p].add = t[p].add * mul + add;
    }

    int copyApply(int p, int l, int r, const T &mul, const T &add)
    {
        // p 是旧节点，[l,r) 是其区间；复制后施加仿射变换并返回新根。
        int q = clone(p);
        applyAt(q, l, r, mul, add);
        return q;
    }

    void push(int p, int l, int r)
    {
        // p 是已复制且可修改的内部节点，[l,r) 是其区间；持久化地下传本层懒标记。
        if (r - l == 1 || idle(p))
        {
            return;
        }
        int m = (l + r) / 2;
        T mul = t[p].mul, add = t[p].add;
        t[p].l = copyApply(t[p].l, l, m, mul, add);
        t[p].r = copyApply(t[p].r, m, r, mul, add);
        t[p].mul = one;
        t[p].add = T{};
    }

    int srcL(int p, int l, int r)
    {
        // p 是源版本节点，[l,r) 是其内部区间；返回已落实 p 标记的左儿子版本根。
        if (idle(p))
        {
            return t[p].l;
        }
        int m = (l + r) / 2;
        T mul = t[p].mul, add = t[p].add;
```

```
92          return copyApply(t[p].l, l, m, mul, add);
93      }
94
95      int srcR(int p, int l, int r)
96      {
97          // p 是源版本节点，[l,r) 是其内部区间；返回已落实 p 标记的右儿子版本根。
98          if (idle(p))
99          {
100             return t[p].r;
101         }
102         int m = (l + r) / 2;
103         T mul = t[p].mul, add = t[p].add;
104         return copyApply(t[p].r, m, r, mul, add);
105     }
106
107     int build(const vector<T> &a, int l, int r)
108     {
109         // a 是初始数组，[l,r) 是当前建树区间；返回该区间的初始版本根。
110         int p = newNode();
111         if (r - l == 1)
112         {
113             t[p].sum = a[l];
114             return p;
115         }
116         int m = (l + r) / 2;
117         t[p].l = build(a, l, m);
118         t[p].r = build(a, m, r);
119         pull(p);
120         return p;
121     }
122
123     int affine(int p, int l, int r, int ql, int qr, const T &mul, const T &add)
124     {
125         // p 是旧版本节点，[l,r) 是其区间；对交集施加仿射变换并返回新节点。
126         if (qr <= l || r <= ql)
127         {
128             return p;
129         }
130         int q = clone(p);
131         if (ql <= l && r <= qr)
132         {
133             applyAt(q, l, r, mul, add);
134             return q;
135         }
136         push(q, l, r);
137         int m = (l + r) / 2;
138         t[q].l = affine(t[q].l, l, m, ql, qr, mul, add);
139         t[q].r = affine(t[q].r, m, r, ql, qr, mul, add);
140         pull(q);
141         return q;
142     }
143
```

```
144     int splice(int target, int source, int l, int r, int ql, int qr)
145     {
146         // target、source 是两个版本节点；把 source 在交集的位置覆盖到 target 并返回新节点。
147         if (qr <= l || r <= ql)
148         {
149             return target;
150         }
151         if (ql <= l && r <= qr)
152         {
153             return source;
154         }
155         int q = clone(target);
156         push(q, l, r);
157         int m = (l + r) / 2;
158         if (ql < m)
159         {
160             int x = srcL(source, l, r);
161             t[q].l = splice(t[q].l, x, l, m, ql, qr);
162         }
163         if (m < qr)
164         {
165             int x = srcR(source, l, r);
166             t[q].r = splice(t[q].r, x, m, r, ql, qr);
167         }
168         pull(q);
169         return q;
170     }
171 
172     T sum(int p, int l, int r, int ql, int qr, const T &outerMul, const T &outerAdd) const
173     {
174         // p 是版本节点，[l,r) 是其区间；outer 表示祖先尚未落实的仿射变换，返回交集和。
175         if (qr <= l || r <= ql)
176         {
177             return T{};
178         }
179         if (ql <= l && r <= qr)
180         {
181             return t[p].sum * outerMul + T(r - l) * outerAdd;
182         }
183         T mul = t[p].mul * outerMul;
184         T add = t[p].add * outerMul + outerAdd;
185         int m = (l + r) / 2;
186         return sum(t[p].l, l, m, ql, qr, mul, add)
187              + sum(t[p].r, m, r, ql, qr, mul, add);
188     }
189 
190   public:
191     explicit PerAffSeg(const vector<T> &a) : n((int)a.size()), one(T(1)), t(1)
192     {
193         // a 是非空初始数组；T 需支持 0、1、加法、乘法、相等比较和由长度构造。
194         assert(n > 0); // 调试检查，可删
195         t.reserve(2 * n + 1);
```

```
        t[0].mul = one;
        base = build(a, 0, n);
    }

    int root() const
    {
        // 返回初始数组对应的版本根；此根可与后续版本根一起长期保存。
        return base;
    }

    int affine(int root, int l, int r, const T &mul, const T &add)
    {
        // root 是旧版本根，[l,r) 是修改区间；返回执行 x->mul*x+add 后的新版本根。
        assert(0 < root && root < (int)t.size()); // 调试检查，可删
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r || (mul == one && add == T{}))
        {
            return root;
        }
        return affine(root, 0, n, l, r, mul, add);
    }

    int splice(int target, int source, int l, int r)
    {
        // target、source 是旧版本根；返回 target 的 [l,r) 被 source 同位置覆盖后的新版本根。
        assert(0 < target && target < (int)t.size()); // 调试检查，可删
        assert(0 < source && source < (int)t.size()); // 调试检查，可删
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (target == source || l == r)
        {
            return target;
        }
        return splice(target, source, 0, n, l, r);
    }

    T sum(int root, int l, int r) const
    {
        // root 是任意历史版本根，[l,r) 是查询区间；返回该区间当前元素和，空区间返回零。
        assert(0 < root && root < (int)t.size()); // 调试检查，可删
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return sum(root, 0, n, l, r, one, T{});
    }
};
```

初始数组必须非空，所有公开区间都采用 $0 \le l \le r \le n$ 的半开约定；根编号只能来自 `root`、`affine` 或 `splice`。`T` 至少要能构造 0、1 和长度，支持加法、乘法和相等比较；模 998244353 时直接取 1.1.008 静态取模类 的 `Z`。例如先给整个初始版本乘 3 加 1，再从另一条分支复制中段：

```
signed main()
{
    vector<Z> a{1, 2, 3, 4};
```

```
    PerAffSeg<Z> seg(a);
    int r0 = seg.root();
    int r1 = seg.affine(r0, 0, 4, Z(3), Z(1));
    int r2 = seg.affine(r0, 1, 3, Z(5), Z(0));
    int r3 = seg.splice(r1, r2, 1, 3);
    assert(seg.sum(r0, 0, 4) == Z(10));
    assert(seg.sum(r3, 0, 4) == Z(42));
}
```

建树时间和初始空间为 $O(n)$；一次 `affine`、`splice` 或 `sum` 的时间为 $O(\log n)$，前两者新增 $O(\log n)$ 个节点，因而保存 q 次修改/覆盖的总空间为 $O(n + q \log n)$。`splice` 的常数会受需要落实的源端懒标记影响，节点池按实际需求增长；$n, q \le 10^5$ 的场景应预留足够内存，不能把普通 3.3.008 可持久化线段树 / 主席树 的单点修改节点数上界照搬过来。只需当前版本的仿射区间和时用更短的 3.3.003 区间加、乘、赋值（仿射懒标记）；只做值域上的历史单点计数或第 k 小时用 3.3.008 可持久化线段树 / 主席树；坐标会插入、删除或翻转时改用 3.5.003 可持久化 Treap。

3.3.021 稀疏仿射区间和

若数组长度可达 10^9、初值全部为零、操作却只有 q 次区间仿射 $x \mapsto ax+b$ 与区间和，就不能建立 3.3.003 区间加、乘、赋值（仿射懒标记） 的 $O(n)$ 棵完整树。`SparseAffSeg<T>` 把未创建子树解释成长度已知、元素全为 0 的区间；只有修改边界经过的节点才开点。每个节点复用 `AffineInfo<T>` 与 `AffineTag<T>`：节点摘要已含本层标记，部分查询不下传、不创建节点，而是把祖先尚未落实且后执行的标记 `outer` 带入子树。因此空子树仍能正确贡献 $len \cdot outer.add$，不会遗漏先前整段修改。

这里的空间性质和 3.3.007 动态开点线段树 不同：一次区间修改最多访问并新建 $O(\log U)$ 个规范分段及其边界孩子，$U = hi - lo$，故总节点数最坏为 $O(q \log U)$，而不是按出现位置计数。`T` 除 $0, 1, +, \times$ 外还必须支持相等比较与由区间长度构造；模题取 `Z`，长度先转为 `Z` 再参与计算。它只表达全零初值的稀疏值域，若初始值不是零且没有可压缩表示，应离散化或用普通线段树。

```
// 依赖 3.3.003 区间加、乘、赋值（仿射懒标记）
template <class T> class SparseAffSeg
{
    struct Node
    {
        // l、r 是孩子编号，info 是该完整区间的和与长度，tag 是尚未下传的仿射变换。
        int l = 0, r = 0;
        AffineInfo<T> info;
        AffineTag<T> tag;
    };

    // lo、hi 是总半开值域，rt 是根编号，t 是动态节点池。
    long long lo, hi;
    int rt = 0;
    vector<Node> t{Node{}};

    int newNode(long long l, long long r)
    {
        // l、r 是新节点覆盖的非空半开区间；建立全零区间及恒等仿射标记。
```

```
        Node q;
        q.info = AffineInfo<T>(T{}, r - l);
        t.push_back(move(q));
        assert(t.size() <= (size_t)numeric_limits<int>::max()); // 调试检查，可删
        return (int)t.size() - 1;
    }

    bool idle(int p) const
    {
        // p 是非空节点；返回其延迟标记是否为恒等变换。
        return t[p].tag.mul == T(1) && t[p].tag.add == T{};
    }

    void apply(int p, long long, long long, const AffineTag<T> &tag)
    {
        // p、[l,r) 是完整命中节点，tag 是后执行的仿射变换；原地更新摘要和延迟标记。
        t[p].info.apply(tag);
        t[p].tag.apply(tag);
    }

    void push(int p, long long l, long long r)
    {
        // p、[l,r) 是内部节点；若有标记则按持久化无关的普通方式下传给两个孩子。
        if (r - l == 1 || idle(p))
        {
            return;
        }
        long long m = l + (r - l) / 2;
        AffineTag<T> tag = t[p].tag;
        int a = t[p].l;
        int b = t[p].r;
        if (!a)
        {
            a = newNode(l, m);
        }
        if (!b)
        {
            b = newNode(m, r);
        }
        apply(a, l, m, tag);
        apply(b, m, r, tag);
        t[p].l = a;
        t[p].r = b;
        t[p].tag = AffineTag<T>{};
    }

    void pull(int p, long long l, long long r)
    {
        // p 是已下传标记的内部节点，[l,r) 是其区间；由孩子的和重算摘要。
        T a = t[p].l ? t[t[p].l].info.sum : T{};
        T b = t[p].r ? t[t[p].r].info.sum : T{};
        t[p].info = AffineInfo<T>(a + b, r - l);
```

```
    }

    int affine(int p, long long l, long long r, long long ql, long long qr,
                   const AffineTag<T> &tag)
    {
        // p、[l,r) 是当前节点，[ql,qr) 是修改区间，tag 是后执行变换；返回更新后的根。
        if (qr <= l || r <= ql)
        {
            return p;
        }
        if (!p)
        {
            p = newNode(l, r);
        }
        if (ql <= l && r <= qr)
        {
            apply(p, l, r, tag);
            return p;
        }
        push(p, l, r);
        long long m = l + (r - l) / 2;
        int a = affine(t[p].l, l, m, ql, qr, tag);
        int b = affine(t[p].r, m, r, ql, qr, tag);
        t[p].l = a;
        t[p].r = b;
        pull(p, l, r);
        return p;
    }

    T sum(int p, long long l, long long r, long long ql, long long qr,
          const AffineTag<T> &outer) const
    {
        // p、[l,r) 是当前节点，[ql,qr) 是查询区间，outer 是祖先尚未下传且后执行的变换。
        if (qr <= l || r <= ql)
        {
            return T{};
        }
        long long a = max(l, ql), b = min(r, qr);
        if (!p)
        {
            // 空子树原值全为零，仍要接受祖先已经施加的常数项。
            return T(b - a) * outer.add;
        }
        if (ql <= l && r <= qr)
        {
            return t[p].info.sum * outer.mul + T(r - l) * outer.add;
        }
        AffineTag<T> next = t[p].tag;
        next.apply(outer);
        long long m = l + (r - l) / 2;
        return sum(t[p].l, l, m, ql, qr, next)
             + sum(t[p].r, m, r, ql, qr, next);
```

```
    }

  public:
    SparseAffSeg(long long lo, long long hi) : lo(lo), hi(hi)
    {
        // lo、hi 是非空半开整数值域；初始所有位置为零。
        assert(lo < hi); // 调试检查，可删
    }

    void reserveNodes(int cap)
    {
        // cap 是预计节点数；预留节点池容量，不改变结构语义。
        assert(cap >= 0); // 调试检查，可删
        t.reserve(cap + 1);
    }

    void affine(long long l, long long r, const T &mul, const T &add)
    {
        // l、r 是值域内半开端点，mul、add 表示 x->x*mul+add；原地修改 [l,r)。
        assert(lo <= l && l <= r && r <= hi); // 调试检查，可删
        if (l == r)
        {
            return;
        }
        rt = affine(rt, lo, hi, l, r, AffineTag<T>{mul, add});
    }

    T sum(long long l, long long r) const
    {
        // l、r 是值域内半开端点；返回 [l,r) 的元素和，空区间返回零。
        assert(lo <= l && l <= r && r <= hi); // 调试检查，可删
        return sum(rt, lo, hi, l, r, AffineTag<T>{});
    }
};
```

构造 `SparseAffSeg<T> tr(lo,hi)` 后，`tr.affine(l,r,mul,add)` 对 $[l,r)$ 执行 $x \mapsto x \cdot mul + add$，`tr.sum(l,r)` 返回区间和；半开端点必须位于总值域内，空修改和空查询均允许。单次修改、查询均为 $O(\log U)$，总空间 $O(q \log U)$，其中 q 是修改次数；`reserveNodes(cap)` 只可按可承受内存预留实际节点数。Library Checker `range_affine_range_sum_large_array` 取 `SparseAffSeg<Z> tr(0,N)`，初始零数组与模运算都正好符合接口。固定长度、已有完整初值时仍优先用 3.3.003 区间加、乘、赋值（仿射懒标记），因为它的空间为 $O(n)$、常数更小；历史版本要求改用 3.3.020 可持久化仿射区间和。

3.3.022 区间赋同函数、区间复合

`range_set_range_composite` 的元素本身是一元函数，操作是把一段每个位置改成同一个函数 f，查询却要求按位置顺序算 $f_{r-1} \circ \cdots \circ f_l$。这不是 3.3.003 区间加、乘、赋值（仿射懒标记） 的“把数值作用仿射变换”，所以不能混用 `AffineTag`；节点必须保存函数序列的有序积，整段赋值 f 后的摘要为 f^{r-l}。`RangeSetSeg<T,F>` 把“求幂”写成只依赖结合律和幺元的二进制重复，故同样可用于矩阵、置换等非交换幺半群。

op(left,right) 一律表示先执行左段、再执行右段。对仿射函数 $f(x) = ax + b$，用 array<Z,2> 存 {a,b}，并定义 $op(f, g) = g \circ f$；恒等元是 {1,0}。一次全覆盖赋值只覆盖旧标记而不是复合，因为题意是替换每个元素；push 才在需要访问儿子时把这个统一值下传。

```
template <class T, class F> class RangeSetSeg
{
    struct Node
    {
        // prod 是该区间按从左到右顺序合并的积，tag 是等待下传的统一赋值幂表编号，-1 表示无标记。
        T prod{};
        int tag = -1;
    };

    // n 是实际序列长度，base 是补齐后的二次幂叶子数，height 是树高。
    int n = 0, base = 1, height = 0;
    T e;
    F op;
    vector<Node> tr;
    // powers 的第 id 块保存同一次赋值 x 的 x^(2^0), ... ,x^(2^height)。
    vector<T> powers;

    const T &power(int id, int level) const
    {
        // id 是赋值幂表编号，level 对应长度 2^level；返回 x^(2^level)。
        return powers[(size_t)id * (height + 1) + level];
    }

    int makeTag(const T &x)
    {
        // x 是一次新的统一赋值；建立按二次幂长度索引的重复积并返回编号。
        assert(powers.size() / (height + 1) < (size_t)numeric_limits<int>::max()); // 调试检查，
        ↪ 可删
        int id = (int)(powers.size() / (height + 1));
        T cur = x;
        for (int level = 0; level <= height; level++)
        {
            powers.push_back(cur);
            if (level != height)
            {
                cur = op(cur, cur);
            }
        }
        return id;
    }

    void pull(int p)
    {
        // p 是内部节点；按原下标顺序合并两个孩子的积。
        tr[p].prod = op(tr[2 * p].prod, tr[2 * p + 1].prod);
    }

    void apply(int p, int level, int tag)
```

```
    {
        // p 是长度 2^level 的完整命中节点，tag 是统一赋值幂表编号；覆盖该段并记录标记。
        tr[p].prod = power(tag, level);
        tr[p].tag = tag;
    }

    void push(int p, int level)
    {
        // p 是长度 2^level 的当前节点；把待执行统一赋值按幂表编号下传给两个孩子。
        if (level == 0 || tr[p].tag < 0)
        {
            return;
        }
        apply(2 * p, level - 1, tr[p].tag);
        apply(2 * p + 1, level - 1, tr[p].tag);
        tr[p].tag = -1;
    }

    void build(const vector<T> &a)
    {
        // a 是完整初始序列；补齐幺元叶子后自底向上建立有序区间积。
        for (int i = 0; i < n; i++)
        {
            tr[base + i].prod = a[i];
        }
        for (int i = n; i < base; i++)
        {
            tr[base + i].prod = e;
        }
        for (int p = base - 1; p; p--)
        {
            pull(p);
        }
    }

    void assign(int p, int l, int r, int ql, int qr, int tag, int level)
    {
        // p、[l,r) 是长度 2^level 的当前节点，[ql,qr) 是赋值区间，tag 是统一新值幂表编号。
        if (qr <= l || r <= ql)
        {
            return;
        }
        if (ql <= l && r <= qr)
        {
            apply(p, level, tag);
            return;
        }
        push(p, level);
        int m = (l + r) / 2;
        assign(2 * p, l, m, ql, qr, tag, level - 1);
        assign(2 * p + 1, m, r, ql, qr, tag, level - 1);
        pull(p);
```

```
    }

    T prod(int p, int l, int r, int ql, int qr, int level)
    {
        // p、[l,r) 是当前节点；返回 [ql,qr) 与该节点交集按原顺序的积。
        if (qr <= l || r <= ql)
        {
            return e;
        }
        if (ql <= l && r <= qr)
        {
            return tr[p].prod;
        }
        push(p, level);
        int m = (l + r) / 2;
        return op(prod(2 * p, l, m, ql, qr, level - 1),
                  prod(2 * p + 1, m, r, ql, qr, level - 1));
    }

  public:
    RangeSetSeg(const vector<T> &a, T e, F op) : n(a.size()), e(move(e)), op(move(op))
    {
        // a 是初始序列，e 是运算幺元，op 是从左段到右段的结合运算；允许空序列。
        while (base < max<int>(n, 1))
        {
            base *= 2;
            height++;
        }
        tr.assign(2 * base, Node{});
        build(a);
    }

    void reserveTags(int cap)
    {
        // cap 是预计的非空赋值总次数；预留幂缓存容量，不改变结构语义。
        assert(cap >= 0); // 调试检查，可删
        powers.reserve((size_t)cap * (height + 1));
    }

    void assign(int l, int r, const T &x)
    {
        // l、r 是半开端点，x 是新值；把 [l,r) 每个元素统一赋为 x，空区间无操作。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r)
        {
            return;
        }
        assign(1, 0, base, l, r, makeTag(x), height);
    }

    T prod(int l, int r)
    {
```

```
        // l、r 是半开端点；返回 [l,r) 的有序积，空区间返回 e。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r)
        {
            return e;
        }
        return prod(1, 0, base, l, r, height);
    }
};
```

构造 `RangeSetSeg<T,F> st(a,e,op)`，其中 `a` 是初始序列；可先以预计的非空赋值数调用 `reserveTags(cap)` 预留幂缓存。调用时例如用加法合并：

```
signed main()
{
    auto op = [](int x, int y)
    {
        return x + y;
    };
    RangeSetSeg<int, decltype(op)> st(vector<int>{1, 2, 3, 4}, 0, op);
    st.assign(1, 3, 5);
    assert(st.prod(0, 4) == 15);
}
```

`assign(l,r,x)` 为此次赋值建立一次 $x, x^2, x^4, \ldots$ 的表，节点只保存表编号并按自身二次幂长度 $O(1)$ 取积；`prod(l,r)` 返回有序积并允许空区间。建树 $O(n)$，一次赋值和查询都是 $O(\log n)$；幂缓存额外占 $O(u \log n)$ 个 `T`，其中 u 是发生过的非空赋值次数，故总空间为 $O(n + u \log n)$。对模仿射函数这已覆盖 Library Checker 的 `range_set_range_composite`；其 $N, Q \le 5 \times 10^5$ 时应调用 `reserveTags(Q)`，避免幂表扩容。若只做单点赋函数、区间复合，普通 3.3.001 单点修改线段树与摘要二分 或大值域的 3.3.007 动态开点线段树 更短。若更新语义是把函数施加到数值数组而非替换函数元素，回到 3.3.003 区间加、乘、赋值（仿射懒标记）。

3.4 RMQ 与静态区间结构

3.4.001 ST 表

ST 表用倍增预处理静态数组，回答幂等运算（min、max、gcd、按位与或）的区间查询。`a[k][i]` 存以 i 开头、长度 2^k 的区间结果，递推 `a[k][i]=f(a[k-1][i], a[k-1][i+2^{k-1}])`。查询 $[l, r)$ 时取 $k = \lfloor \log_2(r-l) \rfloor$，用覆盖左右端的两个长度 2^k 的块 `f(a[k][l], a[k][r-2^k])`；因为运算幂等，两块重叠部分被重复计入也不影响结果，这正是它区别于下面 DST 的地方——只有幂等才能这样重叠取块：

```
template <class T, class F> struct ST
{
    // 最小值与最大值只由 f 决定：分别传 min 或 max；ST 的建表和查询代码无需修改。
    // n 是数组长度，f 是幂等合并运算，a[k][i] 保存长度 2^k 的区间信息。
    int n;
    F f;
    vector<vector<T>> a;
```

```

    ST(const vector<T> &v, F f = {}) : n((int)v.size()), f(f)
    {
        // v 是静态数组，f 是满足结合律和幂等性的合并运算。
        int m = n ? bit_width((unsigned)n) : 0;
        a.assign(m, vector<T>(n));
        if (!n)
        {
            return;
        }
        a[0] = v;
        for (int k = 1; k < m; k++)
        {
            for (int i = 0; i + (1 << k) <= n; i++)
            {
                a[k][i] = f(a[k - 1][i], a[k - 1][i + (1 << (k - 1))]);
            }
        }
    }

    // 仅适用于幂等运算（min/max/gcd/and/or），查询 [l,r)。
    T query(int l, int r) const
    {
        // [l,r) 是非空查询区间；返回其合并结果。
        assert(0 <= l && l < r && r <= n); // 调试检查，可删
        int k = bit_width((unsigned)(r - l)) - 1;
        return f(a[k][l], a[k][r - (1 << k)]);
    }
};
```

若普通 ST 的 $O(n \log n)$ 内存或构建常数过大，可按固定块长 B 保存块内前后缀，并只给整块建 ST：

```
template <class T, class F, int B = 16> struct BlockST
{
    // 与 ST 相同，最小/最大只切换 f；不要改重叠块的查询结构。
    int n = 0, m = 0;
    F f;
    vector<T> val, pre, suf;
    vector<vector<T>> st;

    BlockST(const vector<T> &v, F f = {}) : f(f) { build(v); }

    void build(const vector<T> &v)
    {
        // v 是静态数组；按块长 B 建块内前后缀与块间稀疏表，末块用最后一个元素补齐。
        n = v.size();
        m = (n + B - 1) / B;
        if (!n)
        {
            val.clear(); pre.clear(); suf.clear(); st.clear();
            return;
        }
```

```
        int z = m * B;
        val.assign(z, v.back());
        copy(v.begin(), v.end(), val.begin());
        pre.resize(z);
        suf.resize(z);
        vector<T> top(m);
        for (int i = 0; i < z; i++)
        {
            pre[i] = i % B ? f(pre[i - 1], val[i]) : val[i];
        }
        for (int l = 0; l < z; l += B)
        {
            suf[l + B - 1] = val[l + B - 1];
            for (int i = l + B - 1; i-- > l;)
            {
                suf[i] = f(val[i], suf[i + 1]);
            }
            top[l / B] = suf[l];
        }
        int h = bit_width((unsigned long long)m);
        st.assign(h, vector<T>(m));
        st[0] = move(top);
        for (int k = 1; k < h; k++)
        {
            for (int i = 0; i + (1 << k) <= m; i++)
            {
                st[k][i] = f(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
            }
        }
    }

    T query(int l, int r) const
    {
        // [l,r) 是非空半开区间；返回区间内全部元素按 f 合并的结果。
        assert(0 <= l && l < r && r <= n); // 调试检查，可删
        r--;
        int x = l / B, y = r / B;
        if (x == y)
        {
            T ans = val[l];
            for (int i = l + 1; i <= r; i++)
            {
                ans = f(ans, val[i]);
            }
            return ans;
        }
        T ans = f(suf[l], pre[r]);
        if (++x < y)
        {
            int k = bit_width((unsigned long long)(y - x)) - 1;
            ans = f(ans, f(st[k][x], st[k][y - (1 << k)]));
        }
```

```
        return ans;
    }
};
```

构造 `ST<T,F> st(v, f)`，`f` 是幂等的二元合并。最小值实例为：

```
signed main()
{
    vector<int> v{5, 2, 7, 3};
    auto f = [](int x, int y)
    {
        return min(x, y);
    };
    ST<int, decltype(f)> st(v, f);
    assert(st.query(1, 4) == 2);
    BlockST<int, decltype(f)> bst(v, f);
    assert(bst.query(1, 4) == 2);
}
```

`st.query(l, r)` 返回半开区间 $[l,r)$ 的合并结果，要求 $0 \le l < r \le n$。普通版预处理 $O(n \log n)$ 时间与空间，单次查询 $O(1)$。`BlockST<T,F,B>` 的预处理和空间为 $O(n + (n/B)\log(n/B))$，跨块查询 $O(1)$，同块查询至多合并 B 项；固定 $B = 16$ 时仍为最坏 $O(1)$。本机 $n = q = 10^6$ 的随机 RMQ 中，分块版构建约 `6ms`、查询约 `10ms`，普通版约为 `31ms+10ms`，同时大幅减少常驻内存。运算只有结合律但不幂等（矩阵乘、字符串拼接）时不能重叠取块，改用 3.4.003 静态区间半群；要第 k 小、排名、前驱后继用 3.4.005 小波矩阵；按小块笛卡尔树形状压缩查询表见 3.4.002 基于状压的 RMQ。

3.4.002 基于状压的 RMQ

把静态数组切成长度约 $b = \lfloor \frac{\log_2 n}{2} \rfloor$ 的小块；两个块若笛卡尔树形状相同，任意块内区间的最优局部下标也相同，因此可按形状共享一张 `l*len+r` 查询表，块间最值另建 ST 表。`shape` 用单调栈建块内笛卡尔树并输出括号序列编码，查询 $[l,r)$ 时合并左块后缀、右块前缀与中间整块。当前实现只为实际出现的形状现场建立 len^2 表，并用 `map` 管理形状；设形状数为 S，表规模是 $O(Sb^2)$，由 $S \le \min(n/b, C_b)$ 可得最坏 $O(n\sqrt{\log n})$，不是 Fischer-Heun 的严格线性构造。`pick` 在值相等时保留较小下标，保证并列最优取最靠前：

```
template <class T, class Cmp = less<T>> struct BlockRMQ
{
    // 默认 less<T> 求最小值；改用 greater<T> 即求最大值，其他建表、并列取较早下标逻辑不变。
    using Key = pair<unsigned long long, int>;

    // a 保存静态序列；cmp 判断前者是否更优；b 是小块长度；type 保存各块的笛卡尔树形状与实际长度。
    vector<T> a;
    Cmp cmp;
    int b = 1;
    vector<Key> type;
    // tab 为每种块形状保存所有块内询问的最优局部下标；mn 保存每块最优全局下标；st 是块间稀疏表。
    map<Key, vector<int>> tab;
    vector<int> mn;
```

```
    vector<vector<int>> st;

    BlockRMQ(const vector<T> &a = {}, Cmp cmp = {}) : a(a), cmp(cmp)
    {
        // a 是静态序列，cmp 是严格弱序比较器；建立块内表与块间 ST 表。
        build();
    }

    int pick(int x, int y) const
    {
        // x、y 表示两个候选下标；返回值更优者，空下标 -1 不参与比较。
        if (x < 0)
        {
            return y;
        }
        if (y < 0)
        {
            return x;
        }
        if (cmp(a[y], a[x]))
        {
            return y;
        }
        if (cmp(a[x], a[y]))
        {
            return x;
        }
        return min(x, y);
    }

    Key shape(int l, int r) const
    {
        // [l,r) 表示一个小块；返回其笛卡尔树括号编码与实际长度。
        int n = r - l;
        vector<int> lc(n, -1), rc(n, -1), stk;
        for (int i = 0; i < n; i++)
        {
            int last = -1;
            while (!stk.empty() && cmp(a[l + i], a[l + stk.back()]))
            {
                last = stk.back();
                stk.pop_back();
            }
            if (!stk.empty())
            {
                rc[stk.back()] = i;
            }
            if (last >= 0)
            {
                lc[i] = last;
            }
            stk.push_back(i);
```

```
        }
        int root = stk.front();
        unsigned long long code = 0;
        auto dfs = [&](auto &&self, int u) -> void
        {
            if (u < 0)
            {
                code <<= 1;
                return;
            }
            code = code << 1 | 1;
            self(self, lc[u]);
            self(self, rc[u]);
        };
        dfs(dfs, root);
        return {code, n};
    }

    void build()
    {
        // 重建块形状表、块最值和块间 ST 表。
        b = 1;
        type.clear();
        tab.clear();
        mn.clear();
        st.clear();
        int n = a.size();
        if (!n)
        {
            return;
        }
        b = max<int>(1, ((int)bit_width((unsigned long long)n) - 1) / 2);
        int m = (n + b - 1) / b;
        type.resize(m);
        mn.resize(m);
        for (int k = 0; k < m; k++)
        {
            int l = k * b, r = min(n, l + b), len = r - l;
            type[k] = shape(l, r);
            if (!tab.contains(type[k]))
            {
                vector<int> f(len * len);
                for (int i = 0; i < len; i++)
                {
                    int best = i;
                    for (int j = i; j < len; j++)
                    {
                        if (cmp(a[l + j], a[l + best]))
                        {
                            best = j;
                        }
                        f[i * len + j] = best;
```

```
                    }
                }
                tab[type[k]] = move(f);
            }
            mn[k] = inBlock(k, 0, len);
        }
        st.push_back(mn);
        for (int d = 1; (1LL << d) <= m; d++)
        {
            int len = m - (1LL << d) + 1;
            st.push_back(vector<int>(len));
            for (int i = 0; i < len; i++)
            {
                st[d][i] = pick(st[d - 1][i], st[d - 1][i + (1LL << (d - 1))]);
            }
        }
    }

    int inBlock(int k, int l, int r) const
    {
        // k 表示块编号，[l,r) 表示块内非空区间；返回对应全局最优下标。
        int len = type[k].second;
        return k * b + tab.at(type[k])[l * len + r - 1];
    }

    int index(int l, int r) const
    {
        // [l,r) 表示非空查询区间；返回其中最优元素的全局下标。
        assert(0 <= l && l < r && r <= (int)a.size()); // 调试检查，可删
        int x = l / b, y = (r - 1) / b;
        if (x == y)
        {
            return inBlock(x, l % b, (r - 1) % b + 1);
        }
        int ans = pick(inBlock(x, l % b, type[x].second), inBlock(y, 0, (r - 1) % b + 1));
        int ql = x + 1, qr = y;
        if (ql < qr)
        {
            int d = (int)bit_width((unsigned)(qr - ql)) - 1;
            ans = pick(ans, pick(st[d][ql], st[d][qr - (1LL << d)]));
        }
        return ans;
    }

    const T &query(int l, int r) const
    {
        // [l,r) 表示非空查询区间；返回其中最优元素的常引用。
        return a[index(l, r)];
    }
};
```

构造 `BlockRMQ<T,Cmp> q(a, cmp)`，`q.index(l, r)` 返回半开区间 $[l, r)$ 内最优元素的全局下标、

`q.query(l, r)` 返回该元素常引用，区间必须非空，`cmp` 须为严格弱序；修改公开成员 `a` 后可再次调用 `build()` 完整重建。例如 `a={5,2,2,7,-1,3,0}` 时 `index(0,7)==4`、`query(1,4)==2`、`index(1,3)==1`（并列最小取较早下标）。当前实现最坏预处理时间与空间均为 $O(n\sqrt{\log n})$，单次查询 $O(1)$；一般幂等 RMQ 用 3.4.001 ST 表 更简单且界更直接，确需严格线性预处理时应换完整 Fischer-Heun 构造。非幂等结合运算用 3.4.003 静态区间半群。下面用固定数组检查并列最小值和重复建表：

```
signed main()
{
    BlockRMQ<int> q({5, 2, 2, 7, -1, 3, 0});
    assert(q.index(0, 7) == 4);
    assert(q.query(1, 4) == 2);
    assert(q.index(1, 3) == 1);
    q.a = {9, 8, 7, 6, 5};
    q.build();
    assert(q.index(0, 5) == 4);
    cout << "OK\n";
}
```

3.4.003 静态区间半群

Disjoint Sparse Table（DST）回答静态数组上任意满足结合律的区间运算，不要求幂等或可逆，典型如矩阵乘、字符串拼接。它的第 k 层把数组按 2^{k+1} 分段，每段以中点为界向左记后缀积、向右记前缀积存进 `t[k]`。查询 $[l,r)$ 时（$r-1$ 记末位），取 l 与 $r-1$ 最高不同二进制位确定唯一的层 $k=\lfloor\log_2(l\oplus(r-1))\rfloor$，在这一层 l 与 $r-1$ 恰好分居某段中点两侧，于是答案就是”从 l 到中点的后缀积”合并”从中点到 $r-1$ 的前缀积”，即 `f(t[k][l], t[k][r-1])`——两段不重叠、恰好拼满，所以只需结合律：

```
// 静态半群：运算只需满足结合律，不要求可逆或幂等。
template <class T, class F> struct DST
{
    // n 是数组长度，f 是结合律运算，a 是原数组，t 保存各层左右半段积。
    int n;
    F f;
    vector<T> a;
    vector<vector<T>> t;

    DST(const vector<T> &a, F f = {}) : n((int)a.size()), f(f), a(a)
    {
        // a 是静态数组，f 是满足结合律的合并运算。
        int lg = n <= 1 ? 0 : bit_width((unsigned)(n - 1));
        t.assign(lg, vector<T>(n));
        for (int k = 0; k < lg; k++)
        {
            int len = 1 << k;
            for (int l = 0; l < n; l += 2 * len)
            {
                int m = min(l + len, n), r = min(l + 2 * len, n);
                if (m == l || m == r)
                {
                    continue;
```

```
                }
                t[k][m - 1] = a[m - 1];
                for (int i = m - 2; i >= l; i--)
                {
                    t[k][i] = f(a[i], t[k][i + 1]);
                }
                t[k][m] = a[m];
                for (int i = m + 1; i < r; i++)
                {
                    t[k][i] = f(t[k][i - 1], a[i]);
                }
            }
        }
    }

    T query(int l, int r) const
    {
        // [l,r) 是非空查询区间；返回按原顺序合并的结果。
        assert(0 <= l && l < r && r <= n); // 调试检查，可删
        if (r - l == 1)
        {
            return a[l];
        }
        int k = bit_width((unsigned)(l ^ (r - 1))) - 1;
        return f(t[k][l], t[k][r - 1]);
    }
};
```

构造 `DST<T,F> d(a, f)`，例如维护字符串拼接：

```
signed main()
{
    vector<string> a{"a", "b", "c", "d"};
    auto f = [](const string &x, const string &y)
    {
        return x + y;
    };
    DST<string, decltype(f)> d(a, f);
    assert(d.query(1, 4) == "bcd");
}
```

`d.query(l, r)` 返回半开区间 $[l, r)$ 上按 `f` 折叠的结果（长度 1 时直接返回 `a[l]`），要求 $0 \le l < r \le n$。预处理 $O(n \log n)$ 时间与空间，单次查询 $O(1)$。它比 ST 表适用面更广（不要求幂等），代价是常数略大、无法像 ST 表那样省一层；若运算幂等就用更简单的 3.4.001 ST 表，状压块内查询见 3.4.002 基于状压的 RMQ。

3.4.004 笛卡尔树

笛卡尔树对下标 $0..n-1$ 建一棵二叉树，使中序遍历仍是原下标顺序、而结点值满足堆序（这里建最小堆，即父值不大于子值）。它把”区间最小值位置”转化为”两下标的 LCA”，也是很多分治计数、单调栈

问题的结构骨架。构造用单调栈维护当前根到最右结点的这条”右链”：新元素 i 沿右链弹出所有比它大的结点（弹出的最后一个成为 i 的左儿子），再把 i 接到栈顶结点的右儿子，$O(n)$ 完成。相等元素靠前者在上，保证形状唯一：

```cpp
struct Cart
{
    // root 是树根；parent、left、right 分别保存父亲与左右儿子编号。
    int root = -1;
    vector<int> parent, left, right;
};

// 最小笛卡尔树；中序遍历为原下标，相等时靠前元素在上方。
template <class T> Cart cartesian(const vector<T> &a)
{
    // a 是静态数组；返回其最小笛卡尔树的根、父亲和左右儿子。
    int n = a.size();
    Cart c{-1, vector<int>(n, -1), vector<int>(n, -1), vector<int>(n, -1)};
    vector<int> s;
    for (int i = 0; i < n; i++)
    {
        int last = -1;
        while (!s.empty() && a[i] < a[s.back()])
        {
            last = s.back();
            s.pop_back();
        }
        if (!s.empty())
        {
            c.parent[i] = s.back();
            c.right[s.back()] = i;
        }
        if (last != -1)
        {
            c.parent[last] = i;
            c.left[i] = last;
        }
        s.push_back(i);
    }
    if (n)
    {
        c.root = s[0];
    }
    return c;
}
```

调用 `Cart c = cartesian(a)`,返回的 `Cart` 结构里 `c.root` 是根下标,`c.parent[i]`、`c.left[i]`、`c.right[i]` 分别是结点 i 的父、左儿子、右儿子（无则为 -1）。$O(n)$ 时间与空间。它给出的是“结构”而非直接的区间查询接口：配一棵 LCA 即得 $O(1)$ RMQ（下标形式），本身也常用于 3.4.002 基于状压的 RMQ 里对小块求形状；若只想直接查区间最值用 3.4.001 ST 表 更省事。

3.4.005 小波矩阵

小波矩阵在静态序列上按二进制位从高到低逐层「稳定划分」：每层把当前序列里该位为 0 的元素依原相对顺序放到前半、为 1 的放到后半，并记录每个前缀里 1 的个数 `prefixOne` 以及前缀里 0 元素的值和 `pre0`。有了这些，任一区间 $[l, r)$ 在下一层都能 $O(1)$ 映射到对应子区间：为 0 的元素落在 `[l-onesLeft, r-onesR)`，为 1 的落在 `[middle+onesLeft, middle+onesR)`。`kth` 从高位到低位，看当前区间里 0 的个数决定第 k 小走哪半；`countSumLT` 沿上界 `hi` 的各位走，遇到该位为 1 就把左半（更小）整段计入个数与值和。频次由两次 `countLess` 相减得到。注意 `rank` 是 0 基的，返回的是原值：

```cpp
template <int LOG = 30> struct Wavelet
{
    static_assert(1 <= LOG && LOG <= 62);

    // size 是序列长度，middle[bit] 是该层零段长度。
    int size;
    array<int, LOG> middle;
    // prefixOne[bit][i] 是该层前 i 位的一数；pre0 是零分支原值和。
    array<vector<int>, LOG> prefixOne;
    array<vector<int>, LOG> pre0;
    // prefixSum[i] 是原序列前 i 项的值和。
    vector<int> prefixSum;
    explicit Wavelet(vector<int> values = {})
    {
        // values 是位于 [0,2^LOG) 的初始序列。
        init(move(values));
    }

    void init(vector<int> values)
    {
        // values 是新的静态序列；清空旧层并重新稳定划分。
        size = values.size();
        prefixSum.assign(size + 1, 0);
        constexpr unsigned long long limit = 1ULL << LOG;
        for (int i = 0; i < size; i++)
        {
            assert(0 <= values[i] && (unsigned long long)values[i] < limit); // 调试检查，可删
            prefixSum[i + 1] = prefixSum[i] + values[i];
        }
        vector<int> next(size);
        for (int bit = LOG - 1; bit >= 0; --bit)
        {
            prefixOne[bit].assign(size + 1, 0);
            pre0[bit].assign(size + 1, 0);
            for (int i = 0; i < size; ++i)
            {
                bool one = values[i] >> bit & 1;
                prefixOne[bit][i + 1] = prefixOne[bit][i] + one;
                pre0[bit][i + 1] = pre0[bit][i] + (one ? 0 : values[i]);
            }
            middle[bit] = size - prefixOne[bit][size];
            int zero = 0, one = middle[bit];
```

```
            for (int value : values)
            {
                if (value >> bit & 1)
                {
                    next[one++] = value;
                }
                else
                {
                    next[zero++] = value;
                }
            }
            values.swap(next);
        }
    }

    int kth(int left, int right, int rank) const
    {
        // left、right 是半开区间端点，rank 从 0 开始；返回第 rank 小的原值。
        assert(0 <= left && left < right && right <= size); // 调试检查，可删
        assert(0 <= rank && rank < right - left); // 调试检查，可删
        int answer = 0;
        for (int bit = LOG - 1; bit >= 0; --bit)
        {
            int onesLeft = prefixOne[bit][left];
            int onesR = prefixOne[bit][right];
            int zeros = right - left - onesR + onesLeft;
            if (rank < zeros)
            {
                left -= onesLeft;
                right -= onesR;
            }
            else
            {
                answer |= 1LL << bit;
                rank -= zeros;
                left = middle[bit] + onesLeft;
                right = middle[bit] + onesR;
            }
        }
        return answer;
    }

    pair<int, int> countSumLT(int left, int right, int hi) const
    {
        // 返回 [left,right) 内严格小于 hi 的元素个数与值和。
        assert(0 <= left && left <= right && right <= size); // 调试检查，可删
        if (hi <= 0)
        {
            return {0, 0};
        }
        constexpr unsigned long long limit = 1ULL << LOG;
        if ((unsigned long long)hi >= limit)
```

```
        {
            return {right - left, prefixSum[right] - prefixSum[left]};
        }
        int count = 0;
        int sum = 0;
        for (int bit = LOG - 1; bit >= 0; --bit)
        {
            int onesLeft = prefixOne[bit][left];
            int onesR = prefixOne[bit][right];
            int zeros = right - left - onesR + onesLeft;
            if (hi >> bit & 1)
            {
                count += zeros;
                sum += pre0[bit][right] - pre0[bit][left];
                left = middle[bit] + onesLeft;
                right = middle[bit] + onesR;
            }
            else
            {
                left -= onesLeft;
                right -= onesR;
            }
        }
        return {count, sum};
    }

    int countLess(int left, int right, int hi) const
    {
        // 返回 [left,right) 内严格小于 hi 的元素个数。
        return countSumLT(left, right, hi).first;
    }

    int frequency(int left, int right, int value) const
    {
        // 返回 [left,right) 内 value 的出现次数，值域外返回 0。
        assert(0 <= left && left <= right && right <= size); // 调试检查，可删
        constexpr unsigned long long limit = 1ULL << LOG;
        if (value < 0 || (unsigned long long)value >= limit)
        {
            return 0;
        }
        return countLess(left, right, value + 1) - countLess(left, right, value);
    }

    pair<int, int> countSumLE(int left, int right, int hi) const
    {
        // 返回 [left,right) 内不大于 hi 的元素个数与值和。
        assert(0 <= left && left <= right && right <= size); // 调试检查，可删
        if (hi < 0)
        {
            return {0, 0};
        }
```

```
        constexpr unsigned long long limit = 1ULL << LOG;
        if ((unsigned long long)hi >= limit - 1)
        {
            return {right - left, prefixSum[right] - prefixSum[left]};
        }
        return countSumLT(left, right, hi + 1);
    }
};
```

构造 `Wavelet<LOG> w(values)`（$1 \le LOG \le 62$，`values` 位于 $[0, 2^{LOG})$），`w.kth(l,r,rank)` 返回 $[l, r)$ 内第 `rank` 小的值（`rank` 从 0 起）、`w.frequency(l,r,v)` 返回 v 的出现次数、`w.countLess(l,r,x)` 返回严格小于 x 的个数、`w.countSumLT/LE(l,r,x)` 返回小于/不大于 x 的个数与值和。调用者必须保证任意前缀值和可由 64 位有符号整数表示。计数接口允许空区间，`kth` 要求非空且排名有效。构造 $O(n \cdot LOG)$，各查询 $O(LOG)$。它是静态区间顺序统计的首选，把这些接口打包成的对外用法见 3.4.006 区间第 K 小、排名、前驱后继；带修改则要树套树或离线算法。

若题目只求第 k 小，不需要频次、排名与值和，可使用下面的 `KthWavelet`。它只保存每层的 `prefixOne` 和零段长度，省去 `Wavelet` 的零分支和值前缀和，常驻空间约减半；索引和计数遵循打印版的普通 `int` 规则。其取值范围、半开区间和排名约定与 `Wavelet::kth` 相同：

```
template <int LOG = 30> struct KthWavelet
{
    static_assert(1 <= LOG && LOG <= 62);

    // size 是序列长度，middle[bit] 是该层零段长度；只保存第 k 小查询所需的紧凑状态。
    int size = 0;
    array<int, LOG> middle;
    // prefixOne[bit][i] 是该层前 i 位的一数。
    array<vector<int>, LOG> prefixOne;

    explicit KthWavelet(vector<int> values = {})
    {
        // values 是位于 [0,2^LOG) 的初始序列。
        init(move(values));
    }

    void init(vector<int> values)
    {
        // values 是新的静态序列；清空旧层并重新稳定划分。
        assert(values.size() <= numeric_limits<int>::max()); // 调试检查，可删
        size = (int)values.size();
        constexpr unsigned long long limit = 1ULL << LOG;
        for (int value : values)
        {
            assert(0 <= value && (unsigned long long)value < limit); // 调试检查，可删
        }
        vector<int> next(size);
        for (int bit = LOG - 1; bit >= 0; --bit)
        {
            prefixOne[bit].assign(size + 1, 0);
            for (int i = 0; i < size; ++i)
```

```
            {
                prefixOne[bit][i + 1] = prefixOne[bit][i] + (values[i] >> bit & 1);
            }
            middle[bit] = size - prefixOne[bit][size];
            int zero = 0, one = middle[bit];
            for (int value : values)
            {
                if (value >> bit & 1)
                {
                    next[one++] = value;
                }
                else
                {
                    next[zero++] = value;
                }
            }
            values.swap(next);
        }
    }

    int kth(int left, int right, int rank) const
    {
        // left、right 是半开区间端点，rank 从 0 开始；返回第 rank 小的原值。
        assert(0 <= left && left < right && right <= size); // 调试检查，可删
        assert(0 <= rank && rank < right - left); // 调试检查，可删
        int answer = 0;
        for (int bit = LOG - 1; bit >= 0; --bit)
        {
            int onesLeft = prefixOne[bit][left];
            int onesR = prefixOne[bit][right];
            int zeros = right - left - onesR + onesLeft;
            if (rank < zeros)
            {
                left -= onesLeft;
                right -= onesR;
            }
            else
            {
                answer |= 1LL << bit;
                rank -= zeros;
                left = middle[bit] + onesLeft;
                right = middle[bit] + onesR;
            }
        }
        return answer;
    }
};
```

`KthWavelet<LOG> w(values)` 构造为 $O(n \cdot LOG)$，`w.kth(l,r,rank)` 为 $O(LOG)$，额外空间为 $O(n \cdot LOG)$，但没有查询频次、排名和值和的接口；需要它们时仍用完整的 `Wavelet`。下面用值域两端的固定数据核对全部边界：

```
signed main()
{
    vector<int> a{0, 1, 255, 511, 511};
    Wavelet<9> w(a);
    assert(w.kth(0, 5, 0) == 0);
    assert(w.kth(0, 5, 4) == 511);
    assert(w.countLess(0, 5, 0) == 0);
    assert(w.countLess(0, 5, 512) == 5);
    assert(w.frequency(0, 5, 511) == 2);
    auto [cnt, sum] = w.countSumLE(0, 5, 511);
    assert(cnt == 5 && sum == 1278);
    assert(w.countLess(2, 2, 512) == 0);
    cout << "0 511 5 2 1278\n";
    return 0;
}
```

3.4.006 区间第 K 小、排名、前驱后继

这是「区间第 K 小、排名、前驱后继」这一类需求的对外入口，底层就是 3.4.005 小波矩阵 的 `Wavelet`，不再重复粘贴代码。第 k 小直接 `kth(l, r, k)`（0 基）；某值 v 在 $[l,r)$ 内的排名即 `countLess(l, r, v)`；前驱（严格小于 v 的最大值）在个数 `c=countLess(l,r,v)` 大于 0 时为 `kth(l, r, c-1)`，后继（严格大于 v 的最小值）在 `c2=countSumLE(l,r,v).first < r-l` 时为 `kth(l, r, c2)`。全部为 $O(LOG)$。静态区间顺序统计就用它；若元素需要插入删除，用 3.5.002 FHQ Treap 维护，或对下标序列套线段树；单点带修区间第 k 小则要树状数组套主席树等结构。

3.4.007 离线静态区间频次

若全部询问 (l,r,x) 会预先给出，只求 x 在 $[l,r)$ 的出现次数，无需建立小波矩阵。把询问拆成“前 r 项计数”减“前 l 项计数”两个事件，按前缀长度从 0 扫到 n，哈希表维护各值已经出现的次数；每个数组元素与事件都只处理一次。下面依赖 1.4.004 防 hack 哈希 的 `CustomHash`：

```
vector<int> offlineFreq(const vector<int> &a, const vector<array<int, 3>> &qs)
{
    // a 是静态数组；qs 每项为 {l,r,x}；返回各询问中 x 在 [l,r) 的出现次数。
    int n = a.size();
    using E = array<int, 3>; // 三维依次是询问编号、查询值、容斥符号。
    vector<vector<E>> ev(n + 1);
    for (int i = 0; i < (int)qs.size(); i++)
    {
        auto [l, r, x] = qs[i];
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        ev[l].push_back({i, x, -1});
        ev[r].push_back({i, x, 1});
    }

    unordered_map<int, int, CustomHash> cnt;
    cnt.reserve(2 * (a.size() + qs.size()) + 1);
```

```
    vector<int> ans(qs.size());
    for (int p = 0; p <= n; p++)
    {
        if (p)
        {
            cnt[a[p - 1]]++;
        }
        for (auto [i, x, s] : ev[p])
        {
            auto it = cnt.find(x);
            int c = it == cnt.end() ? 0 : it->second;
            ans[i] += s * c;
        }
    }
    return ans;
}
```

`offlineFreq(a,qs)` 保持询问顺序返回答案，允许空区间和从未在数组出现的查询值。例如取 `a={1,2,1,3,1}`，查询分别为 `{0,5,1}`、`{1,4,1}`、`{2,2,1}`、`{0,5,4}`，答案是 $3,1,0,0$。在 `CustomHash` 的通常假设下，时间期望 $O(n+q)$、空间 $O(n+q)$；若要求确定性或还要第 k 小、值域计数与值和，改用 3.4.005 小波矩阵。

3.4.008 静态区间众数（秩优化）

把数组按块长 $B \approx \sqrt{n}$ 分块，预处理任意连续整块区间的众数及其次数。查询 $[l,r)$ 时先取中间整块答案，再只枚举两端至多 $2B$ 个位置。为去掉“每个边界候选在出现位置表上二分”的额外对数，预先保存 `rk[i]`，即下标 i 在同值出现序列 `pos[a[i]]` 中的秩；已知候选在询问内最左或最右的一次出现后，可直接检查相距当前最优次数的那一次出现是否仍在区间内。只有真正超过当前答案时才继续沿出现序列扩展，而答案总共至多增加 $O(B)$ 次。

```
class RangeMode
{
    int n, B, m; // n 是序列长度，B 是块长，m 是块数。
    // a 是离散值，val 是原值，rk[i] 是 i 在 pos[a[i]] 中的秩。
    vector<int> a, val, rk;
    vector<vector<int>> pos;
    // mode[l][r]、mc[l][r] 是第 l 到第 r 块合并后的众数及次数。
    vector<vector<int>> mode, mc;
    vector<unsigned long long> seen;
    unsigned long long stamp = 0;

    void nextStamp()
    {
        // 开始一次新查询；极端回绕时清空访问标记。
        stamp++;
        if (!stamp)
        {
            fill(seen.begin(), seen.end(), 0);
            stamp = 1;
```

```
        }
    }

public:
    explicit RangeMode(vector<int> x) : n(x.size())
    {
        // x 是静态数组；完成离散化、出现秩与整块众数预处理。
        val = x;
        sort(val.begin(), val.end());
        val.erase(unique(val.begin(), val.end()), val.end());
        a.resize(n);
        rk.resize(n);
        pos.resize(val.size());
        for (int i = 0; i < n; i++)
        {
            a[i] = lower_bound(val.begin(), val.end(), x[i]) - val.begin();
            rk[i] = pos[a[i]].size();
            pos[a[i]].push_back(i);
        }

        B = max<int>(1, sqrt((long double)max<int>(1, n)));
        m = (n + B - 1) / B;
        mode.assign(m, vector<int>(m, -1));
        mc.assign(m, vector<int>(m));
        for (int l = 0; l < m; l++)
        {
            vector<int> f(val.size());
            int best = -1, c = 0;
            for (int i = l * B; i < n; i++)
            {
                int v = a[i];
                f[v]++;
                if (f[v] > c)
                {
                    best = v;
                    c = f[v];
                }
                if ((i + 1) % B == 0 || i + 1 == n)
                {
                    mode[l][i / B] = best;
                    mc[l][i / B] = c;
                }
            }
        }
        seen.assign(val.size(), 0);
    }

    optional<pair<int, int>> query(int l, int r)
    {
        // l、r 是半开区间端点；返回 {一个众数，出现次数}，空区间返回空。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r)
```

```
        {
            return nullopt;
        }
        nextStamp();
        auto use = [&](int v)
        {
            if (seen[v] == stamp)
            {
                return false;
            }
            seen[v] = stamp;
            return true;
        };

        int L = (l + B - 1) / B;
        int R = r / B;
        int best = -1, c = 0;
        if (L < R)
        {
            best = mode[L][R - 1];
            c = mc[L][R - 1];
            for (int i = l; i < L * B; i++)
            {
                c += a[i] == best;
            }
            for (int i = R * B; i < r; i++)
            {
                c += a[i] == best;
            }
            use(best);
        }

        int le = min(r, L * B);
        for (int i = l; i < le; i++)
        {
            int v = a[i];
            if (!use(v))
            {
                continue;
            }
            int first = rk[i], e = first + c;
            if (e < (int)pos[v].size() && pos[v][e] < r)
            {
                e++;
                while (e < (int)pos[v].size() && pos[v][e] < r)
                {
                    e++;
                }
                best = v;
                c = e - first;
            }
        }
```

```
124
125         int rs = max(l, R * B);
126         for (int i = r - 1; i >= rs; i--)
127         {
128             int v = a[i];
129             if (!use(v))
130             {
131                 continue;
132             }
133             int last = rk[i], e = last - c;
134             if (e >= 0 && pos[v][e] >= l)
135             {
136                 e--;
137                 while (e >= 0 && pos[v][e] >= l)
138                 {
139                     e--;
140                 }
141                 best = v;
142                 c = last - e;
143             }
144         }
145         return pair{val[best], c};
146     }
147 };
```

`RangeMode rm(a)` 后，`query(l,r)` 返回任意一个众数及其出现次数；并列众数不保证取最小值，空区间返回 `nullopt`。例如 `a={1,2,1,3,2,1,2,2}` 时，`query(0,8)` 与 `query(1,8)` 均返回 $2,4$，`query(0,5)` 可返回 $1,2$。预处理 $O(n\sqrt{n})$ 时间、$O(n+(n/B)^2)$ 空间，单次查询 $O(B)$；只查某个指定值的频次且询问离线时用 3.4.007 离线静态区间频次，要顺序统计则用 3.4.005 小波矩阵。

下面用同一组数核对跨块区间、并列众数和空区间：

```
1  signed main()
2  {
3      vector<int> a{1, 2, 1, 3, 2, 1, 2, 2};
4      RangeMode rm(a);
5      assert(rm.query(0, 8) == optional(pair{2LL, 4LL}));
6      assert(rm.query(1, 8) == optional(pair{2LL, 4LL}));
7      auto q = rm.query(0, 5);
8      assert(q && q->second == 2 && (q->first == 1 || q->first == 2));
9      assert(!rm.query(3, 3));
10 }
```

3.4.009 静态区间 LIS 查询

当静态数组本身是值域 $[0,n)$ 的排列、而且要回答很多个区间的严格 LIS 时，逐次调用普通 patience sorting 的 $O((r-l)\log(r-l))$ 不能承受。把排列按值域递归二分，可以把每半边的半局部 LCS/LIS 临界路径压成一个允许空位的部分排列（seaweed 编码）。两个相邻值域半边的编码对应 subunit-Monge 矩阵乘法：补齐空位后得到两个单位 Monge 排列，分治地相乘，再丢回补齐路径。递归结束后得到 `edge`，把空位替换为 n。

其关键不变量是对任意 $0 \le l \le r \le n$:

$$\mathrm{LIS}(p[l,r)) = (r-l) - \#\{i \in [l,n) \mid edge_i < r\}.$$

右边仅是一段静态数组的“小于阈值计数”，故复用 3.4.005 小波矩阵 的 `Wavelet<LOG>::countLess`。这里 `LOG` 要使 $n < 2^{LOG}$；默认 `17` 正好覆盖 $n \le 10^5$，内部临界路径和工作区索引使用普通 `int`。它只接受排列，重复值的非严格/严格 LIS 都不能直接套用。

```cpp
// 依赖 3.4.005 小波矩阵
template <int LOG = 17> class StaticRngLIS
{
    static constexpr int none = -1; // none 表示部分排列中未定义的位置。

    // n 是原排列长度；wt 保存海草边界编码后的静态顺序统计结构。
    int n = 0;
    Wavelet<LOG> wt;

    static vector<int> inverse(const vector<int> &p)
    {
        // p 是允许 none 的部分排列；返回其反向部分排列，none 保持为 none。
        int n = p.size();
        vector<int> q(n, none);
        for (int i = 0; i < n; i++)
        {
            if (p[i] == none)
            {
                continue;
            }
            assert(0 <= p[i] && p[i] < n && q[p[i]] == none); // 调试检查，可删
            q[p[i]] = (int)i;
        }
        return q;
    }

    static int workSize(int n)
    {
        // n 是单位 Monge 排列阶数；返回一次递归乘法所需的 int 工作区长度。
        int ans = 1;
        while (n > 1)
        {
            ans += 2 * n;
            n = (n + 1) / 2;
            ans += 4 * n;
        }
        return ans;
    }

    static void unitMongeMul(int n, int *work, const int *a, const int *b)
    {
        // n 是排列阶数，work 是足够大的工作区，a、b 是两个单位 Monge 排列；把乘积行映射写入 work[0,n)。
        if (n == 1)
```

```
        {
            work[0] = 0;
            return;
        }
        int *rowToCol = work;
        work += n;
        int *colToRow = work;
        work += n;
        int mid = n / 2;

        auto solveHalf = [&](int len, auto inHalf, auto shift)
        {
            // len 是当前值域半边长度；inHalf 判定元素是否属于该半边；shift 把值压到半边内。
            int *aHead = work;
            int *aWhere = work + len;
            int *bHead = work + 2 * len;
            int *bWhere = work + 3 * len;
            auto split = [&](const int *src, int *head, int *where)
            {
                // src 是完整排列；head 写压缩后的子排列，where 写对应原下标。
                for (int i = 0; i < n; i++)
                {
                    if (inHalf(src[i]))
                    {
                        *head++ = (int)shift(src[i]);
                        *where++ = (int)i;
                    }
                }
            };
            split(a, aHead, aWhere);
            split(b, bHead, bWhere);
            unitMongeMul(len, work + 4 * len, aHead, bHead);
            int *part = work + 4 * len;
            for (int i = 0; i < len; i++)
            {
                int row = aWhere[i];
                int col = bWhere[part[i]];
                rowToCol[row] = (int)col;
                colToRow[col] = (int)row;
            }
        };
        solveHalf(mid, [=](int x)
        {
            return x < mid;
        }, [](int x)
        {
            return x;
        });
        solveHalf(n - mid, [=](int x)
        {
            return x >= mid;
        }, [=](int x)
```

```
        {
            return x - mid;
        });

        struct Cursor
        {
            // delta 是当前 Monge 边界相对平衡，col 是当前列位置。
            int delta = 0, col = 0;
        };
        int row = n;
        auto goRight = [&](Cursor &it)
        {
            // it 是当前边界游标；向右越过一列并维护其平衡量。
            if (b[it.col] < mid)
            {
                if (colToRow[it.col] >= row)
                {
                    it.delta++;
                }
            }
            else if (colToRow[it.col] < row)
            {
                it.delta++;
            }
            it.col++;
        };
        auto goUp = [&](Cursor &it)
        {
            // it 是当前边界游标；把行边界上移一格并维护其平衡量。
            if (a[row] < mid)
            {
                if (rowToCol[row] >= it.col)
                {
                    it.delta--;
                }
            }
            else if (rowToCol[row] < it.col)
            {
                it.delta--;
            }
        };
        Cursor neg, pos;
        while (row)
        {
            while (pos.col != n)
            {
                Cursor next = pos;
                goRight(next);
                if (next.delta != 0)
                {
                    break;
                }
```

```
                pos = next;
            }
            row--;
            goUp(neg);
            goUp(pos);
            while (neg.delta != 0)
            {
                goRight(neg);
            }
            if (neg.col > pos.col)
            {
                rowToCol[row] = (int)pos.col;
            }
        }
    }

    static vector<int> partMongeMul(vector<int> a, vector<int> b)
    {
        // a、b 是同阶部分 Monge 排列；返回它们复合后的部分排列。
        int n = a.size();
        vector<int> aInv = inverse(a), bInv = inverse(b);
        swap(b, bInv);
        vector<int> aMap, bMap;
        for (int i = n - 1; i >= 0; i--)
        {
            if (a[i] != none)
            {
                aMap.push_back((int)i);
                a[n - (int)aMap.size()] = a[i];
            }
        }
        reverse(aMap.begin(), aMap.end());
        int ptr = 0;
        for (int i = 0; i < n; i++)
        {
            if (aInv[i] == none)
            {
                a[ptr++] = (int)i;
            }
        }
        for (int i = 0; i < n; i++)
        {
            if (b[i] != none)
            {
                b[(int)bMap.size()] = b[i];
                bMap.push_back((int)i);
            }
        }
        ptr = bMap.size();
        for (int i = 0; i < n; i++)
        {
            if (bInv[i] == none)
```

```
            {
                b[ptr++] = (int)i;
            }
        }

        vector<int> work(workSize(n));
        unitMongeMul(n, work.data(), a.data(), b.data());
        vector<int> c(n, none);
        int start = n - (int)aMap.size();
        for (int i = 0; i < (int)aMap.size(); i++)
        {
            int to = work[start + i];
            if (to < (int)bMap.size())
            {
                c[aMap[i]] = bMap[to];
            }
        }
        return c;
    }

    static vector<int> seaweed(const vector<int> &p)
    {
        // p 是值域 [0,n) 的排列；返回其半局部 LIS 的 seaweed 部分排列编码。
        int n = p.size();
        if (n <= 1)
        {
            return vector<int>(n, none);
        }
        int mid = n / 2;
        vector<int> low, high, lowMap, highMap;
        low.reserve(mid);
        high.reserve(n - mid);
        lowMap.reserve(mid);
        highMap.reserve(n - mid);
        for (int i = 0; i < n; i++)
        {
            assert(0 <= p[i] && p[i] < n); // 调试检查，可删
            if (p[i] < mid)
            {
                low.push_back(p[i]);
                lowMap.push_back((int)i);
            }
            else
            {
                high.push_back((int)(p[i] - mid));
                highMap.push_back((int)i);
            }
        }
        low = seaweed(low);
        high = seaweed(high);
        vector<int> lowPad(n), highPad(n);
        iota(lowPad.begin(), lowPad.end(), (int)0);
```

```
        iota(highPad.begin(), highPad.end(), (int)0);
        for (int i = 0; i < mid; i++)
        {
            if (low[i] == none)
            {
                lowPad[lowMap[i]] = none;
            }
            else
            {
                lowPad[lowMap[i]] = lowMap[low[i]];
            }
        }
        for (int i = 0; mid + i < n; i++)
        {
            if (high[i] == none)
            {
                highPad[highMap[i]] = none;
            }
            else
            {
                highPad[highMap[i]] = highMap[high[i]];
            }
        }
        return partMongeMul(move(lowPad), move(highPad));
    }

  public:
    explicit StaticRngLIS(vector<int> p = {})
    {
        // p 是值域 [0,n) 的静态排列；构造其区间严格 LIS 查询结构。
        init(move(p));
    }

    void init(vector<int> p)
    {
        // p 是值域 [0,n) 的新静态排列；重建结构，不保留旧查询状态。
        n = p.size();
        assert(n <= numeric_limits<int>::max()); // 调试检查，可删
        assert((unsigned long long)n < (1ULL << LOG)); // 调试检查，可删
        vector<char> used(n);
        vector<int> q(n);
        for (int i = 0; i < n; i++)
        {
            assert(0 <= p[i] && p[i] < n && !used[p[i]]); // 调试检查，可删
            used[p[i]] = true;
            q[i] = (int)p[i];
        }
        vector<int> edge = seaweed(q);
        vector<int> code(n);
        for (int i = 0; i < n; i++)
        {
            code[i] = edge[i] == none ? n : edge[i];
```

```
        }
        wt.init(move(code));
    }

    int query(int l, int r) const
    {
        // l、r 是半开区间端点；返回排列 p[l,r) 的严格最长递增子序列长度。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return r - l - wt.countLess(l, n, r);
    }
};
```

构造 `StaticRngLIS<LOG> st(p)` 或重建 `st.init(p)`；`p` 必须是 $[0,n)$ 的排列且 $n < 2^{LOG}$。`st.query(l,r)` 允许空半开区间并返回其严格 LIS 长度。预处理为 $O(n\log^2 n)$ 时间、$O(n\log n)$ 额外空间，单次查询 $O(\log n)$；这是为大量查询准备的专用重型结构。只做一个区间或数组允许重复时用 4.5.003 区间 LCS / 区间 LIS，普通静态排名/频次则直接用 3.4.005 小波矩阵。下面固定样例同时核对空区间：

```
signed main()
{
    StaticRngLIS<5> st({0, 2, 1, 3});
    assert(st.query(0, 2) == 2);
    assert(st.query(0, 4) == 3);
    assert(st.query(1, 3) == 1);
    assert(st.query(2, 2) == 0);
}
```

3.4.010 静态区间不同值个数

若数组和全部区间都已给出，只要每个半开区间 $[l,r)$ 内不同值的数量，直接复用 2.3.005 区间不同数统计 的 `distinct(a,qs)`。这是一条面向静态区间结构的配置入口，不再复制其中的「按右端点扫、Fenwick 仅保留每个值最后位置」实现：`a` 是静态数组，`qs` 按原顺序保存半开区间，返回的 `vector<int>` 与 `qs` 一一对应，空数组、空询问集和空区间均可用。原接口已经逐项满足这里的语义，不能把只统计指定值频次的 3.4.007 离线静态区间频次 或普通莫队替代为同一个问题。

调用仍是 `auto ans = distinct(a, qs);`。`distinct` 内部用防卡哈希记录每个值的最后位置，故时间在 `CustomHash` 的通常假设下为 $O((n+q)\log n)$、空间为 $O(n+q)$，其中 $n=|a|$、$q=|qs|$；需确定性值域索引或还要顺序统计时，先离散化，或改用 3.4.005 小波矩阵。查询间若夹杂修改，最后出现位置会整体变化，本接口不适用，应另行选择带修改的离线算法。

3.4.011 静态区间逆序对查询

2.2.003 逆序对及其扩展 的 `invMerge` 和 `invBit` 只能统计整个数组；对很多预先给出的区间，要离线重排询问。下面复用 2.2.002 离散化与坐标压缩 的 `Discrete`，并在函数内用普通 `int` 频次数组实现 Fenwick：本题热点只存排名、端点和频次，公共 `Fenwick` 的一般 64 位和类型不适合这条固定容量路径。查询按普通莫队的块长和蛇形右端排序。`len` 是当前长度，`ans` 是当前严格逆序对数。向左加入排名 `x` 时，新元素在所有旧元素之前，新增的是旧元素中严格小于 `x` 的数量 `sum(x)`；向右加入时，新元素在所有旧元素之

后，新增的是严格大于 x 的数量 `len-sum(x+1)`。删除时先从 Fenwick 去掉本元素，再减去对应贡献，因此相等元素在四种移动中都不计入答案。

```cpp
// 依赖 2.2.002 离散化与坐标压缩
vector<int> rangeInv(const vector<int> &a, const vector<pair<int, int>> &qs)
{
    // a 是静态数组，qs 是半开区间；返回每个区间内满足 i<j 且 a[i]>a[j] 的数量。
    int n = (int)a.size();
    if (qs.empty())
    {
        return {};
    }

    // Query[0..2] 依次表示左端点、右端点、原询问编号；普通 int 足以覆盖题目下标范围。
    using Query = array<int, 3>;
    vector<Query> ord(qs.size());
    for (int id = 0; id < (int)qs.size(); id++)
    {
        auto [l, r] = qs[id];
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        ord[id] = {(int)l, (int)r, (int)id};
    }
    Discrete<int> dis(a);
    vector<int> rank(n);
    for (int i = 0; i < n; i++)
    {
        rank[i] = dis.id(a[i]);
    }
    vector<int> bit(dis.a.size() + 1);
    auto add = [&](int x, int delta)
    {
        // x 是离散排名，delta 是频次变化；修改紧凑的 Fenwick 计数树。
        for (x++; x < (int)bit.size(); x += x & -x)
        {
            bit[x] += delta;
        }
    };
    auto sum = [&](int x)
    {
        // x 是排名上界；返回所有排名小于 x 的当前元素个数。
        int ans = 0;
        for (; x; x -= x & -x)
        {
            ans += bit[x];
        }
        return ans;
    };

    int block = max<int>(1, (int)(n / sqrtl(max<int>(1, (int)ord.size()))));
    sort(ord.begin(), ord.end(), [&](const Query &x, const Query &y)
         {
             int bx = x[0] / block, by = y[0] / block;
```

```
            if (bx != by)
            {
                return bx < by;
            }
            return bx & 1 ? x[1] > y[1] : x[1] < y[1];
        });

    vector<int> out(qs.size());
    int l = 0, r = 0, len = 0, ans = 0;
    for (auto q : ord)
    {
        while (r < q[1])
        {
            int x = rank[r++];
            ans += len - sum(x + 1);
            add(x, 1);
            len++;
        }
        while (r > q[1])
        {
            int x = rank[--r];
            add(x, -1);
            len--;
            ans -= len - sum(x + 1);
        }
        while (l > q[0])
        {
            int x = rank[--l];
            ans += sum(x);
            add(x, 1);
            len++;
        }
        while (l < q[0])
        {
            int x = rank[l++];
            add(x, -1);
            len--;
            ans -= sum(x);
        }
        out[q[2]] = ans;
    }
    return out;
}
```

`rangeInv(a,qs)` 返回顺序与 `qs` 相同，区间均为 $[l,r)$，空区间答案为 0；本模板的 `int` 为 64 位，能保存题目规模内至多 $n(n-1)/2$ 个逆序对。令 V 为不同值数，离散化花 $O(n\log V)$，莫队端点移动次数为 $O(n\sqrt{q})$，每次移动做 $O(\log V)$ 的 Fenwick 操作，连同排序总时间为 $O(n\log V+(n\sqrt{q}+q\log q)\log V)$，额外空间 $O(n+q+V)$。紧凑节点要求 $n,q,V\le 2^{31}-1$，这覆盖常见题目限制。只问整个数组时 2.2.003 的归并版更短且为 $O(n\log n)$；区间中有修改时必须改用带时间轴的 2.3.002 带修改莫队，不能继续使用本静态接口。

```cpp
signed main()
{
    vector<int> a{3, 1, 2, 1};
    vector<pair<int, int>> qs{{0, 4}, {1, 4}, {2, 2}};
    assert((rangeInv(a, qs) == vector<int>{4, 1, 0}));
    cout << "4 1 0\n";
}
```

3.4.012 静态区间和

静态数组只做区间加法和时，线段树或 Fenwick 都多了一层不需要的修改能力，直接复用 2.4.001 一维/二维前缀和 的一维 `prefix` 即可。令 `pre=prefix(a)`，其中 `pre[0]=0`、`pre[i+1]=pre[i]+a[i]`，则半开区间的答案是 `pre[r]-pre[l]`；这与题目输入的左右端点完全一致。

```cpp
auto pre = prefix(a);
auto ans = pre[r] - pre[l]; // [l,r) 的元素和。
```

`a` 可以含负数，`0\le l\le r\le |a|`，空区间自然返回零。模板参数 `T` 必须容纳全部前缀和及其差；例如元素与总和都在 64 位范围内时使用 `vector<int>`，需更宽累计时显式用 `vector<__int128>`。预处理 $O(n)$、每次查询 $O(1)$、额外空间 $O(n)$。有单点修改时改用 3.2.001 单点修改、前缀和，有区间修改时改用 3.2.004 区间修改、区间求和。

3.4.013 Fischer-Heun 线性预处理 RMQ

静态区间最小值，$O(n)$ 预处理、$O(1)$ 查询。数组的笛卡尔树（小根堆序，中序是原下标）上，区间 $[l, r)$ 的最小值就是结点 l 与 $r-1$ 的 LCA；对笛卡尔树做欧拉序，LCA 又变成欧拉序上两次首次出现位置之间深度最小的位置。欧拉序长 $m = 2n-1$，相邻深度恰差 ± 1。

把欧拉序切成长 $b = \max(1, \lfloor \frac{\lfloor \log_2 m \rfloor + 1}{2} \rfloor)$ 的块：块内相对深度由 $b-1$ 个 ± 1 决定，至多 $2^{b-1} \le \sqrt{2m}$ 种模式，每种模式暴力建 b^2 的块内答案表，总计 $O(\sqrt{m}\log^2 m)$；块最小值另建稀疏表，只有 $O(\frac{m}{b}\log m) = O(m)$ 项。查询拆成左块后缀、右块前缀和中间整块三段。

```cpp
struct FischerHeun
{
    // a 是原序列；first[i] 是 i 在欧拉序中第一次出现的位置。
    vector<int> a, first;
    // euler、dep 是笛卡尔树欧拉序上的结点编号与深度，相邻深度恰差 1。
    vector<int> euler, dep;
    // b 是块长；pat[k] 是第 k 块深度差模式对应的块内表编号。
    int b = 1;
    vector<int> pat;
    // tab[id][l*b+r] 是该模式块内 [l,r] 的最小深度位置；st 是块最小值的稀疏表。
    vector<vector<int>> tab, st;

    FischerHeun(const vector<int> &v) : a(v), first(v.size(), -1)
    {
        // v 是静态序列；建立 O(n) 预处理、O(1) 查询的区间最小值结构。
```

```
        int n = a.size();
        if (!n)
        {
            return;
        }
        // 单调栈建笛卡尔树：lc、rc 是左右儿子，栈底是根。
        // 改区间最大值：只把下面的 a[i] < a[stk.back()] 改成 a[i] > a[stk.back()]，其余深度比较不变。
        vector<int> lc(n, -1), rc(n, -1), stk;
        for (int i = 0; i < n; i++)
        {
            int last = -1;
            while (!stk.empty() && a[i] < a[stk.back()])
            {
                last = stk.back();
                stk.pop_back();
            }
            if (!stk.empty())
            {
                rc[stk.back()] = i;
            }
            lc[i] = last;
            stk.push_back(i);
        }
        auto emit = [&](int v, int d)
        {
            // v 是结点，d 是深度；把 v 追加进欧拉序。
            if (first[v] < 0)
            {
                first[v] = euler.size();
            }
            euler.push_back(v);
            dep.push_back(d);
        };
        auto dfs = [&](auto &&self, int v, int d) -> void
        {
            // v 是当前结点，d 是其深度；进入以及每个儿子返回后各记录一次 v。
            emit(v, d);
            if (lc[v] >= 0)
            {
                self(self, lc[v], d + 1);
                emit(v, d);
            }
            if (rc[v] >= 0)
            {
                self(self, rc[v], d + 1);
                emit(v, d);
            }
        };
        dfs(dfs, stk[0], 0);
        int m = dep.size();
        b = max(1LL, (int)bit_width((unsigned long long)m) / 2);
        int cnt = (m + b - 1) / b;
```

```
        pat.resize(cnt);
        vector<int> mins(cnt);
        map<int, int> id; // id[code] 是该深度差模式在 tab 中的编号。
        for (int k = 0; k < cnt; k++)
        {
            int l = k * b, r = min(m, l + b), code = 0;
            mins[k] = l;
            for (int i = l + 1; i < r; i++)
            {
                if (dep[i] > dep[i - 1])
                {
                    code |= 1LL << (i - l - 1);
                }
                if (dep[i] < dep[mins[k]])
                {
                    mins[k] = i;
                }
            }
            if (!id.count(code))
            {
                // 按模式还原相对深度 d，再暴力填满块内所有 [i,j] 的最小位置。
                vector<int> d(b), t(b * b);
                for (int i = 1; i < b; i++)
                {
                    d[i] = d[i - 1] + ((code >> (i - 1) & 1) ? 1 : -1);
                }
                for (int i = 0; i < b; i++)
                {
                    int best = i;
                    for (int j = i; j < b; j++)
                    {
                        if (d[j] < d[best])
                        {
                            best = j;
                        }
                        t[i * b + j] = best;
                    }
                }
                id[code] = tab.size();
                tab.push_back(move(t));
            }
            pat[k] = id[code];
        }
        st.push_back(move(mins));
        for (int k = 1; (1LL << k) <= cnt; k++)
        {
            st.emplace_back(cnt - (1LL << k) + 1);
            for (int i = 0; i < (int)st[k].size(); i++)
            {
                st[k][i] = pick(st[k - 1][i], st[k - 1][i + (1LL << (k - 1))]);
            }
        }
```

```
    }

    int pick(int x, int y) const
    {
        // x、y 是欧拉序位置；返回深度较小者，相等时取 x。
        return dep[x] <= dep[y] ? x : y;
    }

    int inside(int k, int l, int r) const
    {
        // k 是块号，l、r 是块内闭区间；返回其中深度最小的欧拉序位置。
        return k * b + tab[pat[k]][l * b + r];
    }

    int argmin(int l, int r) const
    {
        // [l,r) 是非空半开区间；返回其中一个最小值的下标。
        assert(0 <= l && l < r && r <= (int)a.size()); // 调试检查，可删。
        int x = first[l], y = first[r - 1];
        if (x > y)
        {
            swap(x, y);
        }
        int u = x / b, v = y / b;
        if (u == v)
        {
            return euler[inside(u, x % b, y % b)];
        }
        int ans = pick(inside(u, x % b, b - 1), inside(v, 0, y % b));
        if (v > u + 1)
        {
            int k = bit_width((unsigned long long)(v - u - 1)) - 1;
            ans = pick(ans, pick(st[k][u + 1], st[k][v - (1LL << k)]));
        }
        return euler[ans];
    }

    int query(int l, int r) const
    {
        // [l,r) 是非空半开区间；返回区间最小值。
        return a[argmin(l, r)];
    }
};
```

构造 `FischerHeun rmq(a)`；`rmq.argmin(l,r)` 返回半开区间 $[l,r)$ 中一个最小值的下标，`rmq.query(l,r)` 返回最小值，区间必须非空。模式编号用 `map` 去重，查找次数是块数 $\frac{m}{b}$、每次 $O(\log m)$，整体仍是 $O(n)$ 预处理时间与空间，查询 $O(1)$。改求区间最大值只需改代码中注明的单调栈比较。

$O(n\log n)$ 内存可以接受时，3.4.001 ST 表 代码短得多；块内用位运算的近线性写法见 3.4.002 基于状压的 RMQ，笛卡尔树本身见 3.4.004 笛卡尔树。下面用固定种子生成 2×10^5 个数和同样多次查询，核对滚动哈希：

```cpp
signed main()
{
    mt19937 rng(20261015);
    vector<int> a(200000);
    for (auto &x : a)
    {
        x = rng() % 1000000000;
    }
    FischerHeun rmq(a);
    unsigned long long h = 0;
    for (int i = 0; i < 200000; i++)
    {
        int l = rng() % a.size(), r = rng() % a.size();
        if (l > r)
        {
            swap(l, r);
        }
        h = h * 1000003 + rmq.query(l, r + 1);
    }
    cout << rmq.query(3, 4) << ' ' << h << '\n';
    // 期望输出：98350465 3687840806893658951
}
```

3.5 平衡树与堆

3.5.001 Treap

这是旋转式 Treap：每个结点同时带一个键值（满足二叉搜索树的中序有序）和一个随机优先级（满足小根堆序）。插入后若儿子优先级更小就旋转上来修复堆序，删除则把待删结点转成叶子再摘掉。由于优先级随机，树高期望 $O(\log n)$。相同键用 `cnt` 计数而不新建结点，`sz` 维护含重复的子树大小，从而支持 `rank`（严格小于某键的个数）、`kth`（第 k 小，0 基）、`prev`/`next`（严格前驱后继）。这里用 xorshift 自带随机、结点池从下标 1 起（0 是空哨兵）：

```cpp
template <class T> struct Treap
{
    struct Node
    {
        T x{}; // x 是节点保存的键。
        // pri 是随机优先级；l、r 是儿子；cnt 是键重复数；sz 是含重复的子树大小。
        unsigned pri = 0;
        int l = 0, r = 0, cnt = 0, sz = 0;
    };

    vector<Node> t{{}};
    // root 是树根，seed 是优先级生成器状态。
    int root = 0;
    unsigned seed = 712367821;

    unsigned rnd()
```

```
    {
        // 推进内部伪随机状态并返回新值。
        seed ^= seed << 13;
        seed ^= seed >> 17;
        seed ^= seed << 5;
        return seed;
    }

    int size(int u) const
    {
        // u 是节点编号；返回含重复键的子树大小，空节点返回 0。
        return u ? t[u].sz : 0;
    }

    void pull(int u)
    {
        // u 是非空节点；由两个儿子和本键重数重算子树大小。
        t[u].sz = size(t[u].l) + size(t[u].r) + t[u].cnt;
    }

    int rot(int u, int d)
    {
        // u 是子树根；d=1 右旋、d=0 左旋，返回旋转后的根。
        int v = d ? t[u].l : t[u].r;
        if (d)
        {
            t[u].l = t[v].r;
            t[v].r = u;
        }
        else
        {
            t[u].r = t[v].l;
            t[v].l = u;
        }
        pull(u);
        pull(v);
        return v;
    }

    int add(int u, const T &x)
    {
        // u 是子树根，x 是待插入键；返回插入后的根。
        if (!u)
        {
            u = t.size();
            t.push_back({x, rnd(), 0, 0, 1, 1});
            return u;
        }
        if (t[u].x == x)
        {
            t[u].cnt++;
        }
```

```
        else if (x < t[u].x)
        {
            t[u].l = add(t[u].l, x);
            if (t[t[u].l].pri < t[u].pri)
            {
                u = rot(u, 1);
            }
        }
        else
        {
            t[u].r = add(t[u].r, x);
            if (t[t[u].r].pri < t[u].pri)
            {
                u = rot(u, 0);
            }
        }
        pull(u);
        return u;
    }

    void add(const T &x)
    {
        // x 是待插入键；插入一份并更新树根。
        root = add(root, x);
    }

    pair<int, bool> erase(int u, const T &x)
    {
        // u 是子树根，x 是待删键；返回删除后的根与删除前是否存在。
        if (!u)
        {
            return {0, false};
        }
        bool ok;
        if (x < t[u].x)
        {
            auto [v, z] = erase(t[u].l, x);
            t[u].l = v;
            ok = z;
        }
        else if (t[u].x < x)
        {
            auto [v, z] = erase(t[u].r, x);
            t[u].r = v;
            ok = z;
        }
        else
        {
            ok = true;
            if (t[u].cnt > 1)
            {
                t[u].cnt--;
```

```
            }
            else if (!t[u].l || !t[u].r)
            {
                u = t[u].l | t[u].r;
            }
            else
            {
                int d = t[t[u].l].pri < t[t[u].r].pri;
                u = rot(u, d);
                if (d)
                {
                    auto [v, z] = erase(t[u].r, x);
                    t[u].r = v;
                }
                else
                {
                    auto [v, z] = erase(t[u].l, x);
                    t[u].l = v;
                }
            }
        }
        if (u)
        {
            pull(u);
        }
        return {u, ok};
    }

    bool erase(const T &x)
    {
        // x 是待删键；删除一份并返回删除前是否存在。
        auto [u, ok] = erase(root, x);
        root = u;
        return ok;
    }

    int rank(const T &x) const
    {
        // x 是查询键；返回严格小于 x 的元素数。
        int u = root, ans = 0;
        while (u)
        {
            if (x <= t[u].x)
            {
                u = t[u].l;
            }
            else
            {
                ans += size(t[u].l) + t[u].cnt;
                u = t[u].r;
            }
        }
```

```
        return ans;
    }

    T kth(int k) const
    {
        // k 是 0 基排名；返回第 k 小键。
        assert(0 <= k && k < size(root)); // 调试检查，可删
        int u = root;
        while (true)
        {
            int z = size(t[u].l);
            if (k < z)
            {
                u = t[u].l;
            }
            else if (k < z + t[u].cnt)
            {
                return t[u].x;
            }
            else
            {
                k -= z + t[u].cnt;
                u = t[u].r;
            }
        }
    }

    optional<T> prev(const T &x) const
    {
        // x 是查询键；返回严格小于 x 的最大键，不存在时返回空。
        int k = rank(x);
        return k ? optional<T>(kth(k - 1)) : nullopt;
    }

    optional<T> next(const T &x) const
    {
        // x 是查询键；返回严格大于 x 的最小键，不存在时返回空。
        int u = root, k = 0;
        while (u)
        {
            if (x < t[u].x)
            {
                u = t[u].l;
            }
            else
            {
                k += size(t[u].l) + t[u].cnt;
                u = t[u].r;
            }
        }
        return k < size(root) ? optional<T>(kth(k)) : nullopt;
    }
```

```
};
```

用法：`Treap<int> tr;` 之后 `tr.add(x)` 插入、`tr.erase(x)` 删除一份、`tr.rank(x)` 返回严格小于 x 的个数、`tr.kth(k)` 返回第 k 小（0 基，需 $0 \le k <$ 元素数）、`tr.prev(x)`/`tr.next(x)` 返回严格前驱/后继的 `optional`（不存在为 `nullopt`）。单次操作期望 $O(\log n)$、确定性最坏 $O(n)$，空间与插入次数同阶。只需标准有序集合用 `set`；要序列翻转、区间懒标记用 3.5.002 FHQ Treap（分裂合并更自然）；只求第 k 小/排名且允许 GNU 扩展用 3.5.007 PBDS 平衡树。下面用固定序列覆盖重复值、删除、排名以及不存在的前驱后继：

```
signed main()
{
    Treap<int> tr;
    for (int x : {4, 1, 7, 1, 3})
    {
        tr.add(x);
    }
    assert(tr.rank(3) == 2);
    assert(tr.kth(0) == 1 && tr.kth(2) == 3 && tr.kth(4) == 7);
    assert(tr.erase(1) && !tr.erase(9));
    assert(tr.kth(0) == 1 && tr.kth(1) == 3);
    assert(!tr.prev(1) && tr.prev(4) == optional<int>(3));
    assert(tr.next(4) == optional<int>(7) && !tr.next(7));
    cout << "2 1 3 7\n";
    return 0;
}
```

3.5.002 FHQ Treap

FHQ（范浩强）Treap 完全不写旋转，只靠 `split` 与 `merge` 两个操作组合出一切。`split(p, x)` 把树按键拆成「小于 x」与「不小于 x」两棵，`splitLE(p, x)` 拆成「不大于 x」与「大于 x」；`merge(p, q)` 要求 `p` 的所有键不大于 `q` 的所有键，按随机优先级合并成一棵。有了这两把刀，插入就是「按键切两半、中间塞新结点、合三段」，删除一份是「切出等于 x 的那一个结点、丢掉它自身、把它的左右儿子并回去」，`rank` 是「切出小于 x 的子树看大小再合回」。这种无旋写法天然适合可持久化和序列区间操作（把键换成隐式下标即可维护可翻转序列）：

```
template <class T> struct FHQ
{
    struct Node
    {
        // x 保存键值；l、r 保存左右儿子；sz 保存子树大小；pri 保存随机优先级。
        T x{};
        int l = 0, r = 0, sz = 1;
        unsigned pri = 0;
    };

    // tr 保存节点池；root 保存根；seed 保存随机数状态。
    vector<Node> tr{{}};
    int root = 0; // root 是当前根，0 表示空树。
```

```
    unsigned seed = 712367821; // seed 是 xorshift 随机优先级状态。

    unsigned rnd()
    {
        // 推进内部伪随机状态并返回新的优先级。
        seed ^= seed << 13;
        seed ^= seed >> 17;
        seed ^= seed << 5;
        return seed;
    }

    int size(int p) const
    {
        // p 是节点编号；返回其子树大小，空节点返回 0。
        return p ? tr[p].sz : 0;
    }

    void pull(int p)
    {
        // p 是非空节点；根据两个儿子重算子树大小。
        tr[p].sz = size(tr[p].l) + size(tr[p].r) + 1;
    }

    pair<int, int> split(int p, const T &x)
    {
        // p 是树根，x 是分界键；返回键值小于 x 与不小于 x 的两棵树根。
        if (!p)
        {
            return {0, 0};
        }
        if (tr[p].x < x)
        {
            auto [a, b] = split(tr[p].r, x);
            tr[p].r = a;
            pull(p);
            return {p, b};
        }
        auto [a, b] = split(tr[p].l, x);
        tr[p].l = b;
        pull(p);
        return {a, p};
    }

    pair<int, int> splitLE(int p, const T &x)
    {
        // p 是树根，x 是分界键；返回键值不大于 x 与大于 x 的两棵树根。
        if (!p)
        {
            return {0, 0};
        }
        if (!(x < tr[p].x))
        {
```

```
            auto [a, b] = splitLE(tr[p].r, x);
            tr[p].r = a;
            pull(p);
            return {p, b};
        }
        auto [a, b] = splitLE(tr[p].l, x);
        tr[p].l = b;
        pull(p);
        return {a, p};
    }

    int merge(int p, int q)
    {
        // p、q 是两棵树根，且 p 中所有键不大于 q 中所有键；返回合并后的根。
        if (!p || !q)
        {
            return p | q;
        }
        if (tr[p].pri < tr[q].pri)
        {
            tr[p].r = merge(tr[p].r, q);
            pull(p);
            return p;
        }
        tr[q].l = merge(p, tr[q].l);
        pull(q);
        return q;
    }

    void insert(const T &x)
    {
        // x 是待插入键；允许重复值。
        auto [p, q] = split(root, x);
        int u = tr.size();
        tr.push_back({x, 0, 0, 1, rnd()});
        root = merge(merge(p, u), q);
    }

    bool erase(const T &x)
    {
        // x 是待删除键；只删除一个副本，成功返回 true。
        auto [p, q] = split(root, x);
        auto [u, v] = splitLE(q, x);
        bool ok = u;
        if (u)
        {
            u = merge(tr[u].l, tr[u].r);
        }
        root = merge(merge(p, u), v);
        return ok;
    }

```

```
    int rank(const T &x)
    {
        // x 是查询键；返回严格小于 x 的元素个数。
        auto [p, q] = split(root, x);
        int ans = size(p);
        root = merge(p, q);
        return ans;
    }

    T kth(int k) const
    {
        // k 是 0 基排名；返回第 k 小键。
        assert(0 <= k && k < size(root)); // 调试检查，可删
        int p = root;
        while (p)
        {
            int z = size(tr[p].l);
            if (k < z)
            {
                p = tr[p].l;
            }
            else if (k == z)
            {
                return tr[p].x;
            }
            else
            {
                k -= z + 1;
                p = tr[p].r;
            }
        }
        return T{};
    }
};
```

构造 `FHQ<int> tr` 后 `tr.insert(x)`（允许重复）、`tr.erase(x)`（删一份，成功返回 `true`）、`tr.rank(x)`（严格小于 x 的个数）、`tr.kth(k)`（第 k 小，0 基）。`split`/`splitLE`/`merge` 也公开可用，`merge(p,q)` 必须满足左树键全部不大于右树键的前提。单次操作期望 $O(\log n)$、最坏 $O(n)$，空间与插入次数同阶。相比 3.5.001 Treap 的旋转写法，它代码更短且是 3.5.003 可持久化 Treap、序列区间翻转的基础；静态有序数据无需修改时直接排序即可。下面用固定序列核对重复值、删除与排名：

```
signed main()
{
    FHQ<int> tr;
    for (int x : {4, 1, 7, 1, 3})
    {
        tr.insert(x);
    }
    assert(tr.rank(3) == 2);
    assert(tr.kth(0) == 1 && tr.kth(2) == 3 && tr.kth(4) == 7);
    assert(tr.erase(1) && !tr.erase(9));
```

```
    assert(tr.kth(0) == 1 && tr.kth(1) == 3 && tr.kth(3) == 7);
    cout << "2 1 3 7\n";
    return 0;
}
```

3.5.003 可持久化 Treap

把无旋 FHQ Treap 可持久化：`split` 与 `merge` 每经过一个结点就先 `copy` 出一份副本再改，未被触碰的子树在新旧版本间共享，于是每个历史根都对应一个不可变的完整版本。这里用隐式下标（按 `sz` 定位而非按键），因此支持序列的按位置插入、删除、区间翻转和区间求和。翻转懒标记 `rev` 是难点：`push` 下传前必须先 `flip`（复制并交换左右儿子）两个儿子，否则会破坏其他版本共享的子树——这也是可持久化带懒标记时最容易写错的地方。`values` 遍历时把一路累积的 `rev` 折进方向，故不落实标记也能按正确中序取值：

```
template <class T> struct PerTreap
{
    struct N
    {
        // l、r 是左右儿子编号，sz 是子树大小。
        int l = 0, r = 0, sz = 0;
        // p 是随机优先级；v 是本点值，s 是子树和。
        unsigned p = 0;
        T v{}, s{};
        // rev 表示当前子树的中序顺序需要翻转。
        bool rev = false;
    };

    // t 是持久化节点池，seed 是优先级生成器状态。
    vector<N> t{N{}};
    unsigned seed = 712367821; // seed 是 xorshift 随机优先级状态。

    unsigned rnd()
    {
        // 推进内部伪随机状态并返回新值。
        seed ^= seed << 13;
        seed ^= seed >> 17;
        seed ^= seed << 5;
        return seed;
    }

    int size(int x) const
    {
        // x 是节点编号；返回其子树大小，空节点返回 0。
        return t[x].sz;
    }
    T sum(int x) const
    {
        // x 是节点编号；返回其子树和，空节点返回零。
        return t[x].s;
    }
```

```
37
38      int node(T v)
39      {
40          // v 是新元素的值；新建单点并返回节点编号。
41          t.push_back({0, 0, 1, rnd(), v, v, false});
42          return t.size() - 1;
43      }
44      int copy(int x)
45      {
46          // x 是旧节点编号；复制非空节点并返回副本编号，空节点仍返回 0。
47          if (x)
48          {
49              t.push_back(t[x]);
50          }
51          return x ? (int)t.size() - 1 : 0;
52      }
53      void pull(int x)
54      {
55          // x 是非空节点编号；由两个儿子重算大小与和。
56          t[x].sz = size(t[x].l) + 1 + size(t[x].r);
57          t[x].s = sum(t[x].l) + t[x].v + sum(t[x].r);
58      }
59      int flip(int x)
60      {
61          // x 是旧子树根；复制根、交换儿子并翻转标记，返回新根。
62          x = copy(x);
63          if (x)
64          {
65              swap(t[x].l, t[x].r);
66              t[x].rev ^= 1;
67          }
68          return x;
69      }
70      void push(int x)
71      {
72          // x 是已复制的非空节点；持久化下传翻转标记。
73          if (!t[x].rev)
74          {
75              return;
76          }
77          t[x].l = flip(t[x].l);
78          t[x].r = flip(t[x].r);
79          t[x].rev = false;
80      }
81
82      pair<int, int> split(int x, int k)
83      {
84          // x 是旧版本根，k 是左段长度；返回前 k 项与其余项的新版本根。
85          if (!x)
86          {
87              return {0, 0};
88          }
```

```
        x = copy(x);
        push(x);
        if (size(t[x].l) >= k)
        {
            auto [a, b] = split(t[x].l, k);
            t[x].l = b;
            pull(x);
            return {a, x};
        }
        auto [a, b] = split(t[x].r, k - size(t[x].l) - 1);
        t[x].r = a;
        pull(x);
        return {x, b};
    }

    int merge(int x, int y)
    {
        // x、y 是相邻序列的根；返回保持中序顺序的合并版本根。
        if (!x || !y)
        {
            return x | y;
        }
        if (t[x].p < t[y].p)
        {
            x = copy(x);
            push(x);
            t[x].r = merge(t[x].r, y);
            pull(x);
            return x;
        }
        y = copy(y);
        push(y);
        t[y].l = merge(x, t[y].l);
        pull(y);
        return y;
    }

    int insert(int rt, int k, T v)
    {
        // rt 是旧版本根，k 是插入位置，v 是新值；返回插入后的版本根。
        assert(0 <= k && k <= size(rt)); // 调试检查，可删
        auto [a, b] = split(rt, k);
        return merge(merge(a, node(v)), b);
    }
    int erase(int rt, int k)
    {
        // rt 是旧版本根，k 是删除位置；返回删除后的版本根。
        assert(0 <= k && k < size(rt)); // 调试检查，可删
        auto [a, b] = split(rt, k);
        auto [c, d] = split(b, 1);
        return merge(a, d);
    }
```

```
    int reverse(int rt, int l, int r)
    {
        // rt 是旧版本根；翻转半开区间 [l,r) 并返回新版本根。
        assert(0 <= l && l <= r && r <= size(rt)); // 调试检查，可删
        auto [a, b] = split(rt, l);
        auto [c, d] = split(b, r - l);
        return merge(merge(a, flip(c)), d);
    }
    T query(int x, int l, int r, bool f) const
    {
        // x 是当前子树根，[l,r) 是相对子树的询问，f 是祖先累计的翻转状态。
        if (!x || l == r)
        {
            return T{};
        }
        if (l == 0 && r == size(x))
        {
            return sum(x);
        }
        int a = f ? t[x].r : t[x].l;
        int b = f ? t[x].l : t[x].r;
        bool g = f ^ t[x].rev;
        int z = size(a);
        T ans{};
        if (l < z)
        {
            ans += query(a, l, min(r, z), g);
        }
        if (l <= z && z < r)
        {
            ans += t[x].v;
        }
        if (z + 1 < r)
        {
            ans += query(b, max<int>(0, l - z - 1), r - z - 1, g);
        }
        return ans;
    }

    T query(int rt, int l, int r) const
    {
        // rt 是版本根；只读返回半开区间 [l,r) 的和，不新建节点。
        assert(0 <= l && l <= r && r <= size(rt)); // 调试检查，可删
        return query(rt, l, r, false);
    }

    vector<T> values(int rt) const
    {
        // rt 是版本根；返回该版本的序列值，顺序与隐式下标一致。
        vector<T> a;
        auto dfs = [&](auto &&self, int x, bool f) -> void
        {
```

```
            if (!x)
            {
                return;
            }
            int l = f ? t[x].r : t[x].l;
            int r = f ? t[x].l : t[x].r;
            bool g = f ^ t[x].rev;
            self(self, l, g);
            a.push_back(t[x].v);
            self(self, r, g);
        };
        dfs(dfs, rt, false);
        return a;
    }
};
```

用法：`PerTreap<long long> tr`，根用 `int` 版本号表示，0 是空树。`insert(rt, k, v)`、`erase(rt, k)`、`reverse(rt, l, r)` 分别在旧版本上插入、删除和翻转，并返回新版本根；三者不改旧根。`query(rt, l, r)` 只读返回 $[l, r)$ 的和，不分裂也不分配节点；`values(rt)` 按隐式下标展开该版本。更新与查询期望 $O(\log n)$、最坏 $O(n)$，每次更新新增期望 $O(\log n)$ 个结点，查询额外空间为递归栈高度。不需要历史版本时用更省内存的 3.5.002 FHQ Treap 或 3.5.004 Splay。下面检查插入、翻转、删除、区间和及旧版本持久性：

```
signed main()
{
    PerTreap<long long> tr;
    int a = tr.insert(0, 0, 1);
    int b = tr.insert(a, 1, 2);
    int c = tr.insert(b, 1, 3);       // 序列变为 [1,3,2]。
    int d = tr.reverse(c, 0, 3);      // 序列变为 [2,3,1]。
    int e = tr.erase(d, 1);           // 序列变为 [2,1]。
    assert(tr.query(c, 0, 3) == 6);
    assert(tr.query(d, 0, 2) == 5);
    assert(tr.values(e) == vector<long long>({2, 1}));
    assert(tr.values(c) == vector<long long>({1, 3, 2}));
}
```

3.5.004 Splay

Splay 靠「每次访问就把目标结点旋转到根」维持均摊 $O(\log n)$，不依赖随机也不存额外平衡信息。`splay(u, g)` 把 u 旋到 g 的儿子处（$g = 0$ 即旋到根），内部按 zig-zig / zig-zag 分类：祖父、父、自身三点共线时先转父再转自身，否则连转两次自身。查询、插入、删除、`rank`、`kth` 完成后都要 splay 一下相关结点保证均摊。删除时把待删点旋到根，若计数大于 1 只减计数，否则把它的前驱旋到根再挂上右子树。它相比 Treap 的价值在于「把区间 $[l, r)$ 夹成一棵子树」（左端前驱旋到根、右端后继旋到根的左儿子），从而支持区间翻转、区间懒标记：

```
template <class T> struct Splay
{
```

```
    struct Node
    {
        // x 是键值，ch[0]、ch[1] 是左右儿子。
        T x{};
        array<int, 2> ch{};
        // fa 是父亲，cnt 是本键重数，sz 是子树元素总数。
        int fa = 0, cnt = 0, sz = 0;
    };

    // t 是节点池，root 是当前根编号。
    vector<Node> t{{}};
    int root = 0; // root 是当前根，0 表示空树。

    int size(int u) const
    {
        // u 是节点编号；返回其子树元素数，空节点返回 0。
        return u ? t[u].sz : 0;
    }

    void pull(int u)
    {
        // u 是非空节点编号；由儿子与本键重数重算子树大小。
        t[u].sz = size(t[u].ch[0]) + size(t[u].ch[1]) + t[u].cnt;
    }

    void rotate(int u)
    {
        // u 是非根节点；把 u 向父亲方向旋转一层。
        int p = t[u].fa, g = t[p].fa;
        int d = t[p].ch[1] == u, v = t[u].ch[d ^ 1];
        if (g)
        {
            t[g].ch[t[g].ch[1] == p] = u;
        }
        t[u].fa = g;
        t[u].ch[d ^ 1] = p;
        t[p].fa = u;
        t[p].ch[d] = v;
        if (v)
        {
            t[v].fa = p;
        }
        pull(p);
        pull(u);
    }

    void splay(int u, int g = 0)
    {
        // u 是待旋节点，g 是停止位置；把 u 旋成 g 的儿子，g=0 时旋到根。
        while (t[u].fa != g)
        {
            int p = t[u].fa, q = t[p].fa;
```

```
            if (q != g)
            {
                if ((t[p].ch[1] == u) == (t[q].ch[1] == p))
                {
                    rotate(p);
                }
                else
                {
                    rotate(u);
                }
            }
            rotate(u);
        }
        if (!g)
        {
            root = u;
        }
    }

    int find(const T &x)
    {
        // x 是查询键；返回对应节点，不存在时返回 0，并伸展最后访问点。
        int u = root, p = 0;
        while (u && t[u].x != x)
        {
            p = u;
            u = t[u].ch[t[u].x < x];
        }
        if (u)
        {
            splay(u);
        }
        else if (p)
        {
            splay(p);
        }
        return u;
    }

    void add(const T &x)
    {
        // x 是待插入键；重复键只增加重数。
        if (!root)
        {
            root = t.size();
            t.push_back({x, {}, 0, 1, 1});
            return;
        }
        int u = root, p = 0, d = 0;
        while (u)
        {
            if (t[u].x == x)
```

```
            {
                t[u].cnt++;
                pull(u);
                splay(u);
                return;
            }
            p = u;
            d = t[u].x < x;
            u = t[u].ch[d];
        }
        u = t.size();
        t.push_back({x, {}, p, 1, 1});
        t[p].ch[d] = u;
        splay(u);
    }

    bool erase(const T &x)
    {
        // x 是待删除键；删除一份并返回删除前是否存在。
        int u = find(x);
        if (!u)
        {
            return false;
        }
        if (t[u].cnt > 1)
        {
            t[u].cnt--;
            pull(u);
            return true;
        }
        int l = t[u].ch[0], r = t[u].ch[1];
        if (!l)
        {
            root = r;
            if (r)
            {
                t[r].fa = 0;
            }
            return true;
        }
        t[l].fa = 0;
        root = l;
        int p = l;
        while (t[p].ch[1])
        {
            p = t[p].ch[1];
        }
        splay(p);
        t[p].ch[1] = r;
        if (r)
        {
            t[r].fa = p;
```

```
159         }
160         pull(p);
161         return true;
162     }
163 
164     int rank(const T &x)
165     {
166         // x 是查询键；返回严格小于 x 的元素数。
167         int u = root, p = 0, ans = 0;
168         while (u)
169         {
170             p = u;
171             if (x <= t[u].x)
172             {
173                 u = t[u].ch[0];
174             }
175             else
176             {
177                 ans += size(t[u].ch[0]) + t[u].cnt;
178                 u = t[u].ch[1];
179             }
180         }
181         if (p)
182         {
183             splay(p);
184         }
185         return ans;
186     }
187 
188     T kth(int k)
189     {
190         // k 是 0 基排名；返回第 k 小并把对应节点伸展到根。
191         assert(0 <= k && k < size(root)); // 调试检查，可删
192         int u = root;
193         while (true)
194         {
195             int z = size(t[u].ch[0]);
196             if (k < z)
197             {
198                 u = t[u].ch[0];
199             }
200             else if (k < z + t[u].cnt)
201             {
202                 splay(u);
203                 return t[u].x;
204             }
205             else
206             {
207                 k -= z + t[u].cnt;
208                 u = t[u].ch[1];
209             }
210         }
```

```
    }

    optional<T> prev(const T &x)
    {
        // x 是查询键；返回严格前驱，不存在时返回空。
        int k = rank(x);
        return k ? optional<T>(kth(k - 1)) : nullopt;
    }

    optional<T> next(const T &x)
    {
        // x 是查询键；返回严格后继，不存在时返回空。
        int u = root, p = 0, best = 0;
        while (u)
        {
            p = u;
            if (x < t[u].x)
            {
                best = u;
                u = t[u].ch[0];
            }
            else
            {
                u = t[u].ch[1];
            }
        }
        if (best)
        {
            splay(best);
            return t[best].x;
        }
        if (p)
        {
            splay(p);
        }
        return nullopt;
    }
};
```

用法与前两种平衡树一致：`Splay<int> tr;` 后 `tr.add(x)`、`tr.erase(x)`（成功返回 `true`）、`tr.rank(x)`（严格小于 x 的个数）、`tr.kth(k)`（第 k 小，0 基）、`tr.prev(x)`/`tr.next(x)`（严格前驱后继的 `optional`）。单次操作最坏 $O(n)$、连续 m 次摊还 $O(m \log n)$，空间与插入次数同阶。纯有序集合它并不比 3.5.002 FHQ Treap 有优势且常数更大；它的不可替代之处是 LCT 的底层和需要频繁“区间提取”的场景。长期只增不查退化时可用 3.5.005 Splay 重构 拍平。下面用一组固定操作覆盖重复键、删除、排名和前驱后继：

```
signed main()
{
    Splay<int> tr;
    for (int x : {4, 1, 7, 1, 3})
    {
        tr.add(x);
```

```
    }
    assert(tr.rank(3) == 2);
    assert(tr.kth(0) == 1 && tr.kth(2) == 3 && tr.kth(4) == 7);
    assert(tr.erase(1) && !tr.erase(9));
    assert(tr.kth(0) == 1 && tr.kth(1) == 3);
    assert(!tr.prev(1) && tr.prev(4) == optional<int>(3));
    assert(tr.next(4) == optional<int>(7) && !tr.next(7));
    cout << "2 1 3 7\n";
    return 0;
}
```

3.5.005 Splay 重构

当 Splay 长期只按同向插入而退化成一条链，或想一次性批量建树时，可按中序收集结点后重建。收集和按中点建树都直接使用递归 lambda；建树阶段深度只有 $O(\log n)$。重建后树高回到 $O(\log n)$，之后仍靠正常 Splay 操作维持均摊。它依赖 3.5.004 Splay 的结点结构，作为其外挂函数直接访问 `s.t`、`s.root`、`s.pull`：

```
template <class T> void rebuild(Splay<T> &s)
{
    // s 表示待重构的 Splay；保留键、重复计数和节点编号，重连成高度为 O(log n) 的平衡树。
    vector<int> id;
    auto collect = [&](auto &&self, int p) -> void
    {
        if (!p)
        {
            return;
        }
        self(self, s.t[p].ch[0]);
        id.push_back(p);
        self(self, s.t[p].ch[1]);
    };
    collect(collect, s.root);

    auto build = [&](auto &&self, int l, int r, int fa) -> int
    {
        if (l == r)
        {
            return 0;
        }
        int m = midpoint(l, r), p = id[m];
        s.t[p].fa = fa;
        s.t[p].ch[0] = self(self, l, m, p);
        s.t[p].ch[1] = self(self, m + 1, r, p);
        s.pull(p);
        return p;
    };
    s.root = build(build, 0, id.size(), 0);
}
```

对已有 `Splay<T> tr` 调用 `rebuild(tr)`：保留所有键与重复计数、结点编号不变，只重连成 $O(\log n)$ 高的平衡形态，重建本身 $O(n)$。例如依次插入 $4, 1, 7, 1, 3$ 再 `rebuild` 后 `size(root)==5`、`kth(0)==1`、`kth(2)==3`、`rank(7)==4`。它只是 3.5.004 Splay 的辅助手段；若本就用无旋 3.5.002 FHQ Treap 则无需重建，随机优先级已保证期望树高。

3.5.006 珂朵莉树（ODT）

珂朵莉树（ODT / Chtholly Tree）用一个 `set` 维护若干两两不交的常值区间，每个结点是 $[l, r)$ 加统一值 `v`。核心只有一个 `split(x)`：把包含 x 的区间在 x 处切成两段并返回后一段的迭代器，之后任何区间操作都先 `split(r)` 再 `split(l)`（顺序不能反，否则 `split(l)` 可能使 `split(r)` 得到的迭代器失效）拿到恰好覆盖 $[l, r)$ 的一批结点。真正让它在部分数据上有效的是 `assign`——区间赋值把触及的结点全删掉换成一个，实际段数可能长期较小；但没有明确随机模型时不存在可复用的期望对数保证，构造数据可令一系列操作达到二次时间。`mutable T v` 是为了能在 `set` 里原地改值：

```cpp
template <class T> struct ODT
{
    struct Node
    {
        // [l,r) 是常值段，v 是该段的值。
        int l, r;
        mutable T v;

        bool operator<(const Node &o) const
        {
            // o 是另一常值段；只按左端点排序。
            return l < o.l;
        }
    };

    // n 是数组长度，s 按左端点保存两两不交的常值段。
    int n;
    set<Node> s;

    ODT(const vector<T> &a) : n(a.size())
    {
        // a 是初始数组；空数组得到空段集。
        for (int l = 0, r; l < n; l = r)
        {
            for (r = l + 1; r < n && a[r] == a[l]; r++)
            {
                ;
            }
            s.insert({l, r, a[l]});
        }
    }

    auto split(int x)
    {
        // x 是分割位置；返回左端点为 x 的段，x=n 时返回 end()。
```

```
        assert(0 <= x && x <= n); // 调试检查，可删
        if (x == n)
        {
            return s.end();
        }
        auto it = prev(s.upper_bound({x, 0, {}}));
        if (it->l == x)
        {
            return it;
        }
        auto [l, r, v] = *it;
        s.erase(it);
        s.insert({l, x, v});
        return s.insert({x, r, v}).first;
    }

    void assign(int l, int r, const T &v)
    {
        // l、r 是半开区间端点，v 是新值；空区间无操作。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r)
        {
            return;
        }
        auto y = split(r), x = split(l);
        s.erase(x, y);
        s.insert({l, r, v});
    }

    template <class F> void apply(int l, int r, F f)
    {
        // l、r 是半开区间端点；对相交常值段的值逐段调用 f(T&)，空区间不调用。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r)
        {
            return;
        }
        auto y = split(r), x = split(l);
        for (auto it = x; it != y; it++)
        {
            f(it->v);
        }
    }

    T kth(int l, int r, int k)
    {
        // l、r 是非空半开区间端点，k 是 0 基排名；返回按位置计重后的第 k 小值。
        assert(0 <= l && l < r && r <= n && 0 <= k && k < r - l); // 调试检查，可删
        auto y = split(r), x = split(l);
        vector<pair<T, int>> a;
        for (auto it = x; it != y; it++)
        {
```

```
            a.push_back({it->v, it->r - it->l});
        }
        sort(a.begin(), a.end());
        for (auto [v, c] : a)
        {
            if (k < c)
            {
                return v;
            }
            k -= c;
        }
        assert(false);
        return {};
    }

    vector<T> values() const
    {
        // 返回当前数组，元素顺序按下标排列。
        vector<T> a(n);
        for (auto [l, r, v] : s)
        {
            fill(a.begin() + l, a.begin() + r, v);
        }
        return a;
    }
};
```

构造 `ODT<long long> odt(a)`，此后所有区间都用左闭右开 $[l, r)$：`assign(l, r, v)` 区间赋值，`apply(l, r, f)` 对区间内每个段的值调用 `f(T&)`，`kth(l, r, k)` 返回区间内按位置计重的第 k 小（0 基，要求 $l < r$），`values()` 展开成完整数组。空初始数组合法；`assign` 和 `apply` 接受空区间并直接返回。`split` 单次 $O(\log S)$（S 为段数），一次区间操作触及 k 段则 $O((k+1)\log S)$，最坏 $O(n \log n)$，空间最坏 $O(n)$。没有大量区间赋值时它会退化，此时该用 3.3.002 懒标记线段树；确定性复杂度需求下也应优先线段树。下面用固定数组覆盖空结构、空修改、赋值、逐段加和合法排名：

```
signed main()
{
    ODT<long long> empty(vector<long long>{});
    empty.assign(0, 0, 7);
    empty.apply(0, 0, [](long long &x)
    {
        x++;
    });
    assert(empty.values().empty());

    ODT<long long> odt({1, 1, 2, 2, 3});
    odt.assign(1, 4, 5);
    odt.assign(2, 2, 9);
    odt.apply(2, 5, [](long long &x)
    {
        x += 2;
    });
```

```
    assert((odt.values() == vector<long long>{1, 5, 7, 7, 5}));
    assert(odt.kth(0, 5, 0) == 1);
    assert(odt.kth(0, 5, 2) == 5);
    assert(odt.kth(0, 5, 4) == 7);
    cout << "1 5 7\n";
    return 0;
}
```

3.5.007 PBDS 平衡树

这是一个选型入口而非新实现：当你需要「有序集合 + 第 k 小 / 排名」且允许 GNU 扩展时，用 PBDS 的 `tree` 最省事，它在红黑树结点上多维护子树大小，`order_of_key(x)` 给出严格小于 x 的元素数、`find_by_order(k)` 给出第 k 小（0 基）。完整类型定义见 3.9.007 tree_order_statistics_node_update，调用细节见 3.9.008 find_by_order 与 order_of_key，此处不提前重复。只需按键插删、前驱后继而不要排名时，标准 `set`/`multiset` 就够：

```
using OrderedSet = set<int>;
using OrderedMSet = multiset<int>;

// 只需要按键插删、前驱和后继时直接使用上述标准容器；第 k 小与排名接口见 3.9.007、3.9.008。
```

PBDS `tree` 的 insert、erase、order_of_key、find_by_order 均 $O(\log n)$，空间 $O(n)$。不想引入 GNU 依赖、又要第 k 小和排名时，改用手写的 3.5.001 Treap 或 3.5.002 FHQ Treap。PBDS 接口省事，但节点布局、更新策略、编译器和数据分布都会影响常数；此前某个平台上的快慢只能保留为该环境的参考，接近时限时应与手写实现用同一数据实测，不能写成普遍优劣。

3.5.008 左偏树 / 可并堆

左偏树是可并堆：$O(\log n)$ 合并两个堆，同时支持取最值、删堆顶。每个结点维护「零路径长」`distance`（到最近空儿子的距离），并保持右儿子的 `distance` 不大于左儿子（这就是「左偏」），从而右链长度是 $O(\log n)$。合并 `merge(x,y)` 只沿右链递归：取根较优者作根，把它的右子树与另一棵合并回去，若合并后右儿子比左儿子深就交换，再更新 `distance`。插入是与单点堆合并，删堆顶是把根的左右子树合并。它比 STL 堆多出的能力就是合并，典型用于图上连通块各持一堆、频繁 union 的题：

```
template <class T, class Compare = less<T>> class LeftistHeap
{
    struct Node
    {
        // x 是键值；l、r 是儿子；d 是到空儿子的最短距离。
        T x{};
        int l = 0, r = 0, d = 1;
    };

    // cmp 决定堆序，t 是节点池，0 表示空节点且 d=0。
    Compare cmp;
    vector<Node> t{{T{}, 0, 0, 0}};

```

```
  public:
    int makeHeap(const T &x)
    {
        // x 是键值；新建零路径长为 1 的单点堆并返回根编号。
        t.push_back({x});
        return t.size() - 1;
    }

    int merge(int x, int y)
    {
        // x、y 是节点不共享的两个堆根；破坏性合并并返回新根，两个旧堆句柄随即失效。
        if (!x || !y)
        {
            return x | y;
        }
        if (cmp(t[y].x, t[x].x))
        {
            swap(x, y);
        }
        t[x].r = merge(t[x].r, y);
        if (t[t[x].l].d < t[t[x].r].d)
        {
            swap(t[x].l, t[x].r);
        }
        t[x].d = t[t[x].r].d + 1;
        return x;
    }

    const T &top(int root) const
    {
        // root 是非空堆根；返回堆顶键值。
        assert(root); // 调试检查，可删
        return t[root].x;
    }

    int pop(int root)
    {
        // root 是非空堆根；删除堆顶并返回新根。
        assert(root); // 调试检查，可删
        return merge(t[root].l, t[root].r);
    }
};
```

用法：`LeftistHeap<int> h;`，堆用根编号表示，0 是空堆。`int r = h.makeHeap(v)` 建单点堆，`r = h.merge(x, y)` 破坏性合并两棵节点不共享的堆并返回新根，`h.top(r)` 取最优（默认 `less` 即小根堆返回最小），`r = h.pop(r)` 删堆顶返回新根。`merge` 后 `x`、`y` 都不能再代表原来的独立堆；`pop` 后旧根及由它表示的旧堆句柄同样失效。`makeHeap`、`top` 为 $O(1)$，`merge`、`pop` 最坏 $O(\log n)$，空间 $O(n)$。只取最值不合并时 STL `priority_queue` 更简单更快；这里唯一价值是可并性。下面分别建立两堆，合并后依次弹出：

```
signed main()
{
```

```
    LeftistHeap<int> h;
    int a = h.merge(h.makeHeap(4), h.makeHeap(1));
    a = h.merge(a, h.makeHeap(7));
    int b = h.merge(h.makeHeap(3), h.makeHeap(2));
    assert(h.top(a) == 1 && h.top(b) == 2);
    int root = h.merge(a, b);
    for (int x : {1, 2, 3, 4, 7})
    {
        assert(h.top(root) == x);
        root = h.pop(root);
    }
    assert(root == 0);
    cout << "1 2 3 4 7\n";
}
```

3.5.009 带删堆

带删堆（懒删除堆）用两个普通 `priority_queue` 模拟支持任意值删除的堆：`a` 存所有插入、`b` 存所有待删除。删除某值只是把它压进 `b`，并不立即从 `a` 移除；每次要读堆顶时先 `clean()`——当 `a` 与 `b` 的堆顶相等时成对弹出，这样暴露出来的 `a.top()` 一定是真正有效的最值。前提是删除的值此前确实存在（否则会错误抵消掉未来出现的相等值），调用方需自行保证。用 `n` 单独记录有效元素个数，避免遍历统计：

```
template <class T, class Cmp = less<T>> struct ErasableHeap
{
    // a 保存全部插入，b 保存待抵消删除，n 是当前有效元素数。
    priority_queue<T, vector<T>, Cmp> a, b;
    int n = 0;

    void clean()
    {
        // 抵消已经同时暴露在两个堆顶的失效元素。
        while (!a.empty() && !b.empty() && a.top() == b.top())
        {
            a.pop();
            b.pop();
        }
    }
    void push(const T &x)
    {
        // x 是新键；插入一份。
        a.push(x);
        n++;
    }
    void erase(const T &x)
    {
        // x 是当前确实存在的键；登记删除一份。
        b.push(x);
        n--;
        clean();
    }
```

```
    const T &top()
    {
        // 返回当前非空堆的最优键。
        assert(n > 0); // 调试检查，可删
        clean();
        return a.top();
    }
    void pop()
    {
        // 删除当前非空堆的最优键。
        assert(n > 0); // 调试检查，可删
        clean();
        a.pop();
        n--;
        clean();
    }
    int size() const
    {
        // 返回有效元素数。
        return n;
    }
    bool empty() const
    {
        // 返回是否没有有效元素。
        return n == 0;
    }
};
```

用法：`ErasableHeap<int> h;`（默认 `less` 为大根堆，取最大值），`h.push(x)` 插入，`h.erase(x)` 删除一份已存在的 x，`h.top()` 取当前最值，`h.pop()` 删最值，`h.size()`/`h.empty()` 查询规模。各操作摊还 $O(\log n)$，某次 `clean` 若清掉 k 个失效元素则耗时 $O(k\log n)$，但每个元素至多被清一次，空间 $O(q)$。要按任意值删且还要第 k 小或前驱后继时用 3.5.002 FHQ Treap；只删堆顶用 STL `priority_queue` 即可。下面固定检查重复最大值、删除非堆顶与连续弹出：

```
signed main()
{
    ErasableHeap<int> h;
    for (int x : {4, 1, 7, 7, 3})
    {
        h.push(x);
    }
    assert(h.size() == 5 && h.top() == 7);
    h.erase(7);
    assert(h.size() == 4 && h.top() == 7);
    h.erase(1);
    h.pop();
    assert(h.size() == 2 && h.top() == 4);
    h.erase(3);
    h.pop();
    assert(h.empty());
    cout << "7 7 4 0\n";
```

```
    return 0;
}
```

3.5.010 Top-K 和维护

动态维护一个可增删的多重集合里「最大的 k 个元素之和」。用两个 `multiset`：`hi` 存当前入选的最大 k 个、`lo` 存其余，`sum` 是 `hi` 的总和。插入时按它和 `hi` 最小值的大小决定进哪边，删除时在两边查找并相应更新 `sum`，之后统一 `fix()`：若 `hi` 多了就把它的最小值移到 `lo`，若 `hi` 少了且 `lo` 非空就把 `lo` 的最大值移进来，始终保持 `hi` 恰含最大的 $\min(k, \text{总数})$ 个。这样 `query` 只需 $O(1)$ 读 `sum`，避免每次重新求和：

```
template <class T> struct TopK
{
    // k 是目标个数，sum 是 hi 中元素和。
    int k;
    T sum{};
    // hi 保存当前最大的 min(k,n) 个元素，lo 保存其余元素。
    multiset<T> hi, lo;

    TopK(int k) : k(k)
    {
        // k 是要取的最大元素个数，允许 k=0。
        assert(k >= 0); // 调试检查，可删
    }

    void fix()
    {
        // 调整两组大小，使 hi 恰含最大的 min(k,n) 个元素。
        while ((int)hi.size() > k)
        {
            auto it = hi.begin();
            sum -= *it;
            lo.insert(*it);
            hi.erase(it);
        }
        while ((int)hi.size() < k && !lo.empty())
        {
            auto it = prev(lo.end());
            sum += *it;
            hi.insert(*it);
            lo.erase(it);
        }
    }

    void insert(T x)
    {
        // x 是新元素；插入一份并恢复分组。
        if (!hi.empty() && x >= *hi.begin())
        {
            hi.insert(x);
            sum += x;
```

```
        }
        else
        {
            lo.insert(x);
        }
        fix();
    }

    bool erase(T x)
    {
        // x 是待删元素；删除一份并返回删除前是否存在。
        auto it = hi.find(x);
        if (it != hi.end())
        {
            sum -= x;
            hi.erase(it);
            fix();
            return true;
        }
        it = lo.find(x);
        if (it == lo.end())
        {
            return false;
        }
        lo.erase(it);
        return true;
    }

    optional<T> query() const
    {
        // 返回最大 k 个元素之和；总数不足 k 时返回空，k=0 时返回零。
        if ((int)hi.size() < k)
        {
            return nullopt;
        }
        return sum;
    }
};
```

构造 `TopK<long long> t(k)`，`t.insert(x)` 插入、`t.erase(x)` 删一份（成功返回 `true`）、`t.query()` 返回当前最大 k 个之和；元素不足 k 个时为 `nullopt`，$k = 0$ 时恒为零。`insert`、`erase` 均 $O(\log n)$，`query` 为 $O(1)$，空间 $O(n)$。若要的是第 k 大的值而非前 k 大之和，用 3.5.002 FHQ Treap 的 `kth`；只求静态一次 Top-K 用 `nth_element` 即可。下面固定检查元素不足、重复值、删除和 $k = 0$：

```
signed main()
{
    TopK<long long> t(3);
    assert(!t.query());
    t.insert(5);
    t.insert(1);
    assert(!t.query());
```

```
    t.insert(4);
    assert(t.query() == optional<long long>(10));
    t.insert(9);
    assert(t.query() == optional<long long>(18));
    t.insert(4);
    assert(t.query() == optional<long long>(18));
    assert(t.erase(5) && !t.erase(8));
    assert(t.query() == optional<long long>(17));
    assert(t.erase(9));
    assert(t.query() == optional<long long>(9));

    TopK<long long> zero(0);
    zero.insert(7);
    assert(zero.query() == optional<long long>(0));
    cout << "10 18 17 9 0\n";
    return 0;
}
```

3.5.011 隐式 Treap（动态序列、翻转、仿射与区间和）

当元素按当前位置插入、删除，且查询或修改区间会随之移动时，固定下标的线段树不能直接复用。隐式 Treap 把中序排名当作键：结点的随机优先级满足小根堆序，`sz` 记录左子树大小，于是 `split(root,k)` 可以切成前 k 项和其余项，`merge` 再把相邻两段接回。当前结构只维护一个可变版本；它与 3.5.003 可持久化 Treap 的历史根语义不同，也比 3.5.002 FHQ Treap 的有序键集合多出位置、区间和与懒标记。

每个结点还维护子树和 `sum`、翻转标记 `rev` 与仿射标记 (mul, add)。后者表示尚未下传的 $g(x) = x \cdot mul + add$；若随后对整段施加 $f(x) = x \cdot a + b$，组合是 $f(g(x)) = x \cdot (mul \cdot a) + (add \cdot a + b)$，故 `mul=mul*a`、`add=add*a+b`。区间和同步变成 $sum \cdot a + len \cdot b$；翻转只交换儿子，且与逐点仿射可交换。`T` 需要零元 `T{}`、单位元 `T(1)`、`+`、`*` 和相等比较，乘法须可交换；普通精确和可取 `long long`（调用者保证所有值和区间和不溢出），模 998244353 则直接复用 1.1.008 模整数 的 `Z`。节点池不回收删除结点，预知最多会插入 q 个元素时以 `reserveNodes(n+q)` 预留容量：

```
template <class T> struct ImpTreap
{
  private:
    struct Node
    {
        // l、r 是左右儿子，sz 是子树元素数，pri 是随机小根堆优先级。
        int l = 0, r = 0, sz = 0;
        unsigned pri = 0;
        // val 是本点值，sum 是子树和，mul、add 表示尚未下传的 x -> x*mul+add。
        T val{}, sum{}, mul{1}, add{};
        // rev 表示当前子树的中序顺序需要翻转。
        bool rev = false;
    };

    // tr 是节点池，root 是当前序列根，seed 是 xorshift 优先级状态。
    vector<Node> tr{Node{}};
    int root = 0; // root 是当前序列根，0 表示空序列。
```

```
    unsigned seed = 712367821; // seed 是 xorshift 随机优先级状态。

    unsigned rnd()
    {
        // 推进内部伪随机状态并返回新的优先级。
        seed ^= seed << 13;
        seed ^= seed >> 17;
        seed ^= seed << 5;
        return seed;
    }

    int sizeNode(int x) const
    {
        // x 是节点编号；返回其子树元素数，0 号空节点返回 0。
        return tr[x].sz;
    }

    T sumNode(int x) const
    {
        // x 是节点编号；返回其子树和，0 号空节点返回零元。
        return tr[x].sum;
    }

    int newNode(const T &x)
    {
        // x 是新元素值；新建一个无儿子的节点并返回编号。
        tr.push_back({0, 0, 1, rnd(), x, x, T(1), T{}, false});
        return (int)tr.size() - 1;
    }

    void pull(int x)
    {
        // x 是非空节点；由儿子与本点值重算大小和子树和。
        tr[x].sz = sizeNode(tr[x].l) + 1 + sizeNode(tr[x].r);
        tr[x].sum = sumNode(tr[x].l) + tr[x].val + sumNode(tr[x].r);
    }

    void applyAffine(int x, const T &mul, const T &add)
    {
        // x 是子树根，mul、add 表示后执行的 x -> x*mul+add；原地更新节点摘要和懒标记。
        if (!x)
        {
            return;
        }
        tr[x].val = tr[x].val * mul + add;
        tr[x].sum = tr[x].sum * mul + add * T(sizeNode(x));
        tr[x].mul = tr[x].mul * mul;
        tr[x].add = tr[x].add * mul + add;
    }

    void applyReverse(int x)
    {
```

```
        // x 是子树根；交换左右儿子并记录一次延迟翻转。
        if (!x)
        {
            return;
        }
        swap(tr[x].l, tr[x].r);
        tr[x].rev = !tr[x].rev;
    }

    void push(int x)
    {
        // x 是非空节点；把翻转与仿射懒标记按语义下传给两个儿子。
        if (tr[x].rev)
        {
            applyReverse(tr[x].l);
            applyReverse(tr[x].r);
            tr[x].rev = false;
        }
        if (tr[x].mul != T(1) || tr[x].add != T{})
        {
            applyAffine(tr[x].l, tr[x].mul, tr[x].add);
            applyAffine(tr[x].r, tr[x].mul, tr[x].add);
            tr[x].mul = T(1);
            tr[x].add = T{};
        }
    }

    void pullAll(int x)
    {
        // x 是新建 Cartesian 树的根；递归重算该子树全部摘要。
        if (!x)
        {
            return;
        }
        pullAll(tr[x].l);
        pullAll(tr[x].r);
        pull(x);
    }

    pair<int, int> split(int x, int k)
    {
        // x 是子树根，k 是左段长度；返回前 k 项与其余项的两棵树根。
        if (!x)
        {
            return {0, 0};
        }
        push(x);
        int z = sizeNode(tr[x].l);
        if (k <= z)
        {
            auto [a, b] = split(tr[x].l, k);
            tr[x].l = b;
```

```
122             pull(x);
123             return {a, x};
124         }
125         auto [a, b] = split(tr[x].r, k - z - 1);
126         tr[x].r = a;
127         pull(x);
128         return {x, b};
129     }
130 
131     int merge(int x, int y)
132     {
133         // x、y 是相邻两段的根；按中序保持 x 在前，返回合并后的根。
134         if (!x || !y)
135         {
136             return x ? x : y;
137         }
138         if (tr[x].pri < tr[y].pri)
139         {
140             push(x);
141             tr[x].r = merge(tr[x].r, y);
142             pull(x);
143             return x;
144         }
145         push(y);
146         tr[y].l = merge(x, tr[y].l);
147         pull(y);
148         return y;
149     }
150 
151   public:
152     explicit ImpTreap(int cap = 0)
153     {
154         // cap 是预计节点总数，只用于预留池容量；构造空序列。
155         assert(cap >= 0); // 调试检查，可删
156         reserveNodes(cap);
157     }
158 
159     explicit ImpTreap(const vector<T> &a, int cap = 0)
160     {
161         // a 是初始序列，cap 是预计节点总数；在线性时间建出当前序列。
162         assert(cap >= 0); // 调试检查，可删
163         reserveNodes(max<int>(cap, (int)a.size()));
164         build(a);
165     }
166 
167     void reserveNodes(int cap)
168     {
169         // cap 是预计节点总数；预留 0 号哨兵外的池容量，不改变现有序列。
170         assert(cap >= 0); // 调试检查，可删
171         tr.reserve(cap + 1);
172     }
173 
```

```
    void clear()
    {
        // 清空当前序列和节点池内容，保留已申请容量并重置随机状态。
        tr.clear();
        tr.push_back({});
        root = 0;
        seed = 712367821;
    }

    void build(const vector<T> &a)
    {
        // a 是新的完整序列；清空旧序列，用优先级 Cartesian 树在线性时间重建。
        clear();
        reserveNodes((int)a.size());
        vector<int> st;
        st.reserve(a.size());
        for (const T &x : a)
        {
            int u = newNode(x), last = 0;
            while (!st.empty() && tr[st.back()].pri > tr[u].pri)
            {
                last = st.back();
                st.pop_back();
            }
            tr[u].l = last;
            if (!st.empty())
            {
                tr[st.back()].r = u;
            }
            st.push_back(u);
        }
        root = st.empty() ? 0 : st[0];
        pullAll(root);
    }

    int size() const
    {
        // 返回当前序列元素数。
        return sizeNode(root);
    }

    bool empty() const
    {
        // 返回当前序列是否为空。
        return root == 0;
    }

    void insert(int k, const T &x)
    {
        // k 是插入位置，x 是新值；把 x 插到原第 k 项前，允许 k=size()。
        assert(0 <= k && k <= size()); // 调试检查，可删
        auto [a, b] = split(root, k);
```

```
226         root = merge(merge(a, newNode(x)), b);
227     }
228 
229     void erase(int k)
230     {
231         // k 是删除位置；删除原第 k 项，已分配节点不回收。
232         assert(0 <= k && k < size()); // 调试检查，可删
233         auto [a, b] = split(root, k);
234         root = merge(a, split(b, 1).second);
235     }
236 
237     void reverse(int l, int r)
238     {
239         // l、r 是半开区间端点；原地翻转 [l,r)，空区间允许。
240         assert(0 <= l && l <= r && r <= size()); // 调试检查，可删
241         auto [a, b] = split(root, l);
242         auto [c, d] = split(b, r - l);
243         applyReverse(c);
244         root = merge(merge(a, c), d);
245     }
246 
247     void affine(int l, int r, const T &mul, const T &add)
248     {
249         // l、r 是半开区间端点，mul、add 是变换参数；对 [l,r) 每项执行 x -> x*mul+add。
250         assert(0 <= l && l <= r && r <= size()); // 调试检查，可删
251         auto [a, b] = split(root, l);
252         auto [c, d] = split(b, r - l);
253         applyAffine(c, mul, add);
254         root = merge(merge(a, c), d);
255     }
256 
257     T sum(int l, int r)
258     {
259         // l、r 是半开区间端点；返回 [l,r) 的元素和，空区间返回零元。
260         assert(0 <= l && l <= r && r <= size()); // 调试检查，可删
261         auto [a, b] = split(root, l);
262         auto [c, d] = split(b, r - l);
263         T ans = sumNode(c);
264         root = merge(merge(a, c), d);
265         return ans;
266     }
267 
268     vector<T> values()
269     {
270         // 把当前序列按下标顺序复制到 vector；会下传懒标记但不改变序列语义。
271         vector<T> a;
272         a.reserve(size());
273         auto dfs = [&](auto &&self, int x) -> void
274         {
275             if (!x)
276             {
277                 return;
```

```
            }
            push(x);
            self(self, tr[x].l);
            a.push_back(tr[x].val);
            self(self, tr[x].r);
        };
        dfs(dfs, root);
        return a;
    }
};
```

构造 `ImpTreap<T> tr(cap)` 得空序列，或 `ImpTreap<T> tr(a,cap)` 以 $O(n)$ 建好 `a`；`cap` 仅预留节点数。`insert(k,x)` 在原第 k 项前插入，`erase(k)` 删除第 k 项，`reverse(l,r)` 翻转、`affine(l,r,mul,add)` 施加仿射、`sum(l,r)` 求和，所有位置都是 0 基半开区间且允许空区间（删除除外）；`values()` 仅用于调试，线性返回当前全序列。随机优先级模型下，建树 $O(n)$，每个区间操作期望 $O(\log n)$、最坏 $O(n)$，`values()` 为 $O(n)$；空间 $O(n+i)$，其中 i 是累计插入数。固定长度的区间仿射和优先用 3.3.003 区间加、乘、赋值（仿射懒标记），要保存历史版本用 3.5.003 可持久化 Treap，只维护按值排序的集合用 3.5.002 FHQ Treap。

下面固定核对位置插删、翻转、仿射、非空与空区间和：

```
signed main()
{
    ImpTreap<long long> tr(vector<long long>{1, 2, 3, 4}, 16);
    tr.reverse(1, 4);
    tr.affine(1, 3, 2, 1);
    tr.insert(2, 5);
    tr.erase(1);
    assert(tr.values() == vector<long long>({1, 5, 7, 2}));
    assert(tr.sum(1, 4) == 14 && tr.sum(2, 2) == 0);
    cout << "1 5 7 2 14 0\n";
    return 0;
}
```

3.5.012 可持久化队列

可持久化队列从任意历史版本分叉，只支持尾部追加和删除队首。不能把普通两栈队列直接持久化：一次把后栈倒到前栈会复制整段，`two_stacks_killer` 这类分叉输入可把均摊重建反复触发，达到二次时间。这里把每次 `push` 的元素看成一条从新队尾指向旧队尾的不可变父链；一个版本只保存 `tail` 和逻辑长度 `len`。当 `len>0` 时，从 `tail` 往前数 `len-1` 步恰是队首；`pop` 不修改节点，只把 `len` 减一并新建一个版本。

为了快速跳到队首，每个元素节点保存二进制祖先 `up[k]`：

$$up(v,0)=parent(v),\qquad up(v,k)=up(up(v,k-1),k-1).$$

于是把距离按二进制拆开即可 $O(\log Q)$ 取祖先。旧节点和旧版本从不改写，所有版本编号都可以长期保存。构造时应传入本题总操作数 Q 作为 `cap` 以预留节点与版本数组；默认 `LOG=20` 覆盖 $Q\le 5\times 10^5$。

若操作数上界更大，实例化 `PersistQueue<T, LOG>` 时要选满足 $2^{LOG} > Q$ 的 `LOG`。这不是支持任意区间修改的通用持久化序列；需要插入、删除、翻转和区间和时用 3.5.003 可持久化 Treap。

```
template <class T, int LOG = 20> class PersistQueue
{
    static_assert(1 <= LOG && LOG < 63);

    struct Node
    {
        // value 是本次入队的值；up[k] 是沿历史尾链向前 2^k 步的节点编号。
        T value{};
        array<int, LOG> up{};
    };

    struct Version
    {
        // tail 是逻辑队尾节点，len 是该版本未被弹出的逻辑元素数。
        int tail = 0, len = 0;
    };

    // nodes[0] 是空哨兵；versions[0] 是初始空队列。
    vector<Node> nodes{Node{}};
    vector<Version> versions{Version{}};

    int anc(int p, int distance) const
    {
        // p 是非空尾节点，distance 是向前跳的步数；返回对应祖先节点编号。
        assert(0 < p && p < (int)nodes.size()); // 调试检查，可删
        assert(0 <= distance && (unsigned long long)distance < (1ULL << LOG)); // 调试检查，可删
        for (int k = 0; distance; k++, distance >>= 1)
        {
            if (distance & 1)
            {
                p = nodes[p].up[k];
            }
        }
        return p;
    }

  public:
    explicit PersistQueue(int cap = 0)
    {
        // cap 是预计总操作数，仅用于预留节点和版本容量；默认 LOG=20 覆盖 Q<=5e5。
        assert(0 <= cap && (unsigned long long)cap < (1ULL << LOG)
               && cap <= numeric_limits<int>::max()); // 调试检查，可删
        nodes.reserve(cap + 1);
        versions.reserve(cap + 1);
    }

    int emptyVer() const
    {
        // 返回初始空队列的版本编号。
```

```
        return 0;
    }

    int verCnt() const
    {
        // 返回当前已经创建的版本数，包含初始空版本。
        return (int)versions.size();
    }

    int size(int version) const
    {
        // version 是版本编号；返回该版本的逻辑队列长度。
        assert(0 <= version && version < (int)versions.size()); // 调试检查，可删
        return versions[version].len;
    }

    bool empty(int version) const
    {
        // version 是版本编号；返回该版本是否为空。
        return size(version) == 0;
    }

    int push(int version, const T &value)
    {
        // version 是旧版本，value 是新队尾元素；返回追加后的新版本编号。
        assert(0 <= version && version < (int)versions.size()); // 调试检查，可删
        int p = (int)nodes.size();
        assert((unsigned long long)p < (1ULL << LOG)
               && p <= numeric_limits<int>::max()); // 调试检查，可删
        Node cur;
        cur.value = value;
        cur.up[0] = versions[version].len ? versions[version].tail : 0;
        for (int k = 1; k < LOG; k++)
        {
            cur.up[k] = nodes[cur.up[k - 1]].up[k - 1];
        }
        nodes.push_back(move(cur));
        int len = versions[version].len + 1;
        assert(len <= numeric_limits<int>::max()); // 调试检查，可删
        versions.push_back({(int)p, (int)len});
        return (int)versions.size() - 1;
    }

    T front(int version) const
    {
        // version 是非空版本编号；返回该版本的队首值，不生成新版本。
        assert(0 <= version && version < (int)versions.size()); // 调试检查，可删
        assert(versions[version].len > 0); // 调试检查，可删
        int p = anc(versions[version].tail, versions[version].len - 1);
        return nodes[p].value;
    }

```

```
    int pop(int version)
    {
        // version 是非空版本编号；删除其队首并返回新版本编号。
        assert(0 <= version && version < (int)versions.size()); // 调试检查，可删
        assert(versions[version].len > 0); // 调试检查，可删
        versions.push_back({versions[version].tail, (int)(versions[version].len - 1)});
        return (int)versions.size() - 1;
    }
};
```

`emptyVer()` 返回初始空版本；`push(version,value)` 和 `pop(version)` 都返回一个新的版本编号，`front(version)` 只读队首，`size` / `empty` 查询版本状态。所有版本号必须来自这些接口，`front` / `pop` 要求非空。本题可写成 `PersistQueue<int> q(Q)`，把第 i 次操作得到的版本编号保存到 `ver[i+1]`，并以 `ver[t+1]` 访问题面里的 S_t：

```
signed main()
{
    PersistQueue<int> q(10);
    int v0 = q.emptyVer();
    int v1 = q.push(v0, 4);
    int v2 = q.push(v1, 7);
    int v3 = q.pop(v2);
    int v4 = q.push(v1, 9);
    assert(q.front(v2) == 4 && q.front(v3) == 7);
    assert(q.front(v4) == 4 && q.size(v4) == 2);
}
```

在预留 `cap` 后，`push` 和 `front` 最坏 $O(LOG) = O(\log Q)$，`pop` 最坏 $O(1)$；每个 `push` 新增一个含 LOG 个祖先的节点，每次 `push` / `pop` 新增一个版本记录，所以 q 次操作总空间 $O(q \log Q)$。普通两栈队列只适用于不保留分叉历史、接受均摊界的场景；若版本只是单点修改的定长数组，使用 3.6.007 可持久化数组 更直接；若要一般持久化序列操作改用 3.5.003 可持久化 Treap。

3.5.013 Radix Heap / 基数堆

Radix Heap 是键单调的整数最小优先队列：每次新插入的键都必须不小于最近一次弹出的键。它把键按“与最近弹出键 `last` 的最高不同位”分到 65 个桶；当第 0 桶为空时，找到首个非空桶，以桶内最小键更新 `last`，再把整桶重新分配。更新后该最小键必进入第 0 桶，而其余元素进入编号更小的桶，所以每个元素至多被重新分配 64 次。这个前提正好适用于非负边权 Dijkstra 等松弛只会产生不小于当前距离的新键的算法；普通可减小键、负权或任意顺序优先队列不能使用本结构。

```
template <class T> class RadixHeap
{
    using Key = unsigned long long; // 键依赖 64 位无符号异或与位宽。
    // b[i] 保存与 last 的最高不同位落在第 i 桶的键值对。
    array<vector<pair<Key, T>>, 65> b;
    // last 是最近弹出的键，n 是当前元素数。
    Key last = 0;
    int n = 0;
```

```

    int id(Key x) const
    {
        // x 是不小于 last 的键；返回它当前所属的桶编号。
        return x == last ? 0 : bit_width(x ^ last);
    }

    void pull()
    {
        // 无参数；把首个非空高桶按其中最小键重新分桶，使 0 号桶非空。
        if (!b[0].empty())
        {
            return;
        }
        int p = 1;
        while (b[p].empty())
        {
            p++;
        }
        last = min_element(b[p].begin(), b[p].end(),
                           [](const auto &x, const auto &y)
                           {
                               return x.first < y.first;
                           })->first;
        for (auto x : b[p])
        {
            b[id(x.first)].push_back(move(x));
        }
        b[p].clear();
    }

  public:
    bool empty() const
    {
        // 无参数；返回堆是否为空。
        return n == 0;
    }

    int size() const
    {
        // 无参数；返回当前元素数。
        return n;
    }

    void push(Key k, const T &v)
    {
        // k 是单调键，v 是关联值；插入一项，要求 k 不小于最近弹出的键。
        assert(k >= last); // 调试检查，可删
        b[id(k)].push_back({k, v});
        n++;
    }

```

```
    const pair<Key, T> &top()
    {
        // 无参数；返回键最小的一项，要求堆非空。
        assert(n > 0); // 调试检查，可删
        pull();
        return b[0].back();
    }

    pair<Key, T> pop()
    {
        // 无参数；删除并返回键最小的一项，要求堆非空。
        auto ans = top();
        b[0].pop_back();
        n--;
        return ans;
    }
};
```

构造 `RadixHeap<T> q` 后，`push(k,v)` 插入键值对，`top()` 只读一个最小键元素，`pop()` 删除并返回一个最小键元素；同键元素之间没有稳定顺序。`top` 返回的引用会在下一次修改操作后失效。键必须是 $[0, 2^{64})$ 内的整数，并始终满足 `k>=last`；这里保留 `unsigned long long` 是因为分桶确实依赖精确的 64 位无符号异或和位宽。`push` 为 $O(1)$，每个元素一生最多移动 64 次，故 n 次插入、弹出共 $O(64n)$ 时间、$O(n+64)$ 空间；若把键宽记为 W，则为 $O(Wn)$。不满足单调键时使用标准 `priority_queue`；需要可并堆时使用 3.5.008 左偏树 / 可并堆。

下面检查重复键、跨高位键和单调弹出：

```
signed main()
{
    RadixHeap<int> q;
    q.push(7, 1);
    q.push(3, 2);
    q.push(3, 3);
    q.push(1ULL << 40, 4);
    assert(q.pop().first == 3);
    assert(q.pop().first == 3);
    q.push(8, 5);
    assert(q.pop().first == 7);
    assert(q.pop().first == 8);
    assert(q.pop().first == (1ULL << 40) && q.empty());
}
```

3.6 Trie 与集合结构

3.6.001 字符 Trie

字符 Trie 按前缀建树，公共前缀只存一次。根是空前缀，每条边代表一个字符，每个结点唯一对应一段前缀。两个计数是核心：`cnt[u]` 记经过 u 的插入次数（即以 u 对应前缀为前缀的串数），`end[u]` 记恰好在 u 结束的完整串数。于是 `prefixCnt` 读 `cnt`、`stringCnt` 读 `end`，`lcp` 沿边走到走不动为止。字符集大小

A 与最小字符 S 作模板参数，字符必须落在 $[S, S+A)$。它与 4.2.001 是同一知识分属数据结构与字符串两章的入口，两处都完整打印：

```
template <int A = 26, char S = 'a'> struct Trie
{
    // ch[u][c] 是节点 u 沿字符 S+c 的儿子。
    vector<array<int, A>> ch;
    // cnt[u] 是经过 u 的插入次数，end[u] 是恰在 u 结束的字符串数。
    vector<int> cnt, end;

    Trie()
    {
        // 建立表示空前缀的根节点 0。
        newNode();
    }

    int newNode()
    {
        // 新建全零节点并返回编号。
        ch.push_back({});
        cnt.push_back(0);
        end.push_back(0);
        return (int)ch.size() - 1;
    }

    int insert(const string &s)
    {
        // s 是字符均位于 [S,S+A) 的字符串；插入一份并返回终点节点。
        int p = 0;
        cnt[p]++;
        for (char c : s)
        {
            int x = c - S;
            assert(0 <= x && x < A); // 调试检查，可删
            if (!ch[p][x])
            {
                ch[p][x] = newNode();
            }
            p = ch[p][x];
            cnt[p]++;
        }
        end[p]++;
        return p;
    }

    int find(const string &s) const
    {
        // s 是查询串；返回其前缀节点，不存在或字符越界时返回 -1。
        int p = 0;
        for (char c : s)
        {
            int x = c - S;
```

```
            if (x < 0 || x >= A || !ch[p][x])
            {
                return -1;
            }
            p = ch[p][x];
        }
        return p;
    }

    int lcp(const string &s) const
    {
        // s 是查询串；返回它与已插入字符串的最长公共前缀长度。
        int p = 0, n = 0;
        for (char c : s)
        {
            int x = c - S;
            if (x < 0 || x >= A || !ch[p][x])
            {
                break;
            }
            p = ch[p][x];
            n++;
        }
        return n;
    }

    int prefixCnt(const string &s) const
    {
        // s 是查询前缀；返回以 s 为前缀的已插入字符串数。
        int p = find(s);
        return p < 0 ? 0 : cnt[p];
    }
    int stringCnt(const string &s) const
    {
        // s 是完整查询串；返回它被插入的次数。
        int p = find(s);
        return p < 0 ? 0 : end[p];
    }
};
```

构造 `Trie<26,'a'> tr` 后 `tr.insert(s)` 插入并返回终点结点、`tr.find(s)` 返回终点结点（不存在返回 -1）、`tr.prefixCnt(s)` 返回以 s 为前缀的串数、`tr.stringCnt(s)` 返回 s 被完整插入的次数、`tr.lcp(s)` 返回 s 与库中任意串的最长公共前缀长度。例如依次插入 `ab`、`abc`、`ab` 后 `stringCnt("ab")==2`、`prefixCnt("ab")==3`、`lcp("abd")==2`、`find("ac")==-1`。各操作 $O(|s|)$，空间 $O(\text{总串长} \times A)$。字符集为 01 且要求异或最值时用 3.6.002 01 Trie / 压位 Trie；需要多模式串匹配则在此基础上加 fail 指针成为 3.6.004 Trie Graph；整数异或查询用 3.6.002 01 Trie，需要删除时启用 3.6.003 可删二进制 Trie，查询历史区间异或时改用 3.6.010 可持久化 01 Trie。本条与 4.2.001 Trie 都完整打印字符 Trie（同一知识分属两章、各有独立阅读价值）；只做键值映射时哈希表比 Trie 更直接，但最坏复杂度不稳定。

3.6.002 01 Trie / 压位 Trie

把整数按二进制位从高到低插入 Trie，两个儿子对应下一位取 0 或 1，每个节点额外记一个经过计数以支持重复值、第 k 小与阈值计数。求最大异或时贪心走与当前位相反的分支（若非空），求最小异或时贪心走相同分支；第 k 小异或沿当前位为 0 的分支计数决定向左还是向右；`countXorLess` 则在 k 的当前位为 1 时把与之同侧的整棵子树计数累加、再转向另一侧。位宽 W 由模板参数固定，本节不做删除、故计数只增：

```cpp
template <int W = 30> struct BinaryTrie
{
    // Node[0..2] 依次是 0 儿子、1 儿子、经过节点的元素个数。
    using Node = array<int, 3>;

    // tr[0] 是空节点，tr[1] 是根；节点只增不删。
    vector<Node> tr;

    BinaryTrie(int n = 0)
    {
        // n 是预计插入次数，只用于预留空间；W 是参与比较的二进制位数。
        static_assert(1 <= W && W <= 64);
        if (n)
        {
            tr.reserve(n * (W + 1) + 2);
        }
        clear();
    }

    int newNode()
    {
        // 新建全零节点并返回编号。
        tr.push_back({});
        return (int)tr.size() - 1;
    }

    bool valid(unsigned long long x) const
    {
        // x 是待操作整数；返回 x 是否能由 W 位无符号整数表示。
        if constexpr (W == 64)
        {
            return true;
        }
        else
        {
            return x < ((unsigned long long)1 << W);
        }
    }

    int size() const
    {
        // 返回 Trie 中元素总数，重复值重复计数。
        return tr[1][2];
```

```
    }

    bool empty() const
    {
        // 返回 Trie 是否为空。
        return size() == 0;
    }

    void clear()
    {
        // 清空全部元素并重建空节点和根节点。
        tr.clear();
        newNode();
        newNode();
    }

    int count(unsigned long long x) const
    {
        // x 是待查询整数；返回 x 的出现次数。
        assert(valid(x)); // 调试检查，可删
        int p = 1;
        for (int i = W - 1; i >= 0; i--)
        {
            p = tr[p][x >> i & 1];
            if (!p)
            {
                return 0;
            }
        }
        return tr[p][2];
    }

    void insert(unsigned long long x, int v = 1)
    {
        // x 是待插入整数，v 是一次插入的份数，必须为正。
        assert(valid(x) && v > 0); // 调试检查，可删
        int p = 1;
        tr[p][2] += v;
        for (int i = W - 1; i >= 0; i--)
        {
            int b = x >> i & 1;
            if (!tr[p][b])
            {
                tr[p][b] = newNode();
            }
            p = tr[p][b];
            tr[p][2] += v;
        }
    }

    unsigned long long maxXor(unsigned long long x) const
    {
```

```
96          // x 是询问整数；返回集合中最大的 x xor y。
97          assert(valid(x) && !empty()); // 调试检查，可删
98          int p = 1;
99          unsigned long long ans = 0;
100         for (int i = W - 1; i >= 0; i--)
101         {
102             int b = x >> i & 1;
103             int q = tr[p][b ^ 1];
104             int c = q && tr[q][2] ? b ^ 1 : b;
105             if (c != b)
106             {
107                 ans |= (unsigned long long)1 << i;
108             }
109             p = tr[p][c];
110         }
111         return ans;
112     }
113
114     unsigned long long maxElement(unsigned long long x) const
115     {
116         // x 是询问整数；返回一个使 x xor y 最大的集合元素 y。
117         return x ^ maxXor(x);
118     }
119
120     unsigned long long minXor(unsigned long long x) const
121     {
122         // x 是询问整数；返回集合中最小的 x xor y。
123         assert(valid(x) && !empty()); // 调试检查，可删
124         int p = 1;
125         unsigned long long ans = 0;
126         for (int i = W - 1; i >= 0; i--)
127         {
128             int b = x >> i & 1;
129             int q = tr[p][b];
130             int c = q && tr[q][2] ? b : b ^ 1;
131             if (c != b)
132             {
133                 ans |= (unsigned long long)1 << i;
134             }
135             p = tr[p][c];
136         }
137         return ans;
138     }
139
140     unsigned long long minElement(unsigned long long x) const
141     {
142         // x 是询问整数；返回一个使 x xor y 最小的集合元素 y。
143         return x ^ minXor(x);
144     }
145
146     unsigned long long kthXor(unsigned long long x, int k) const
147     {
```

```
148         // x 是询问整数，k 从 0 开始；返回第 k 小的 x xor y。
149         assert(valid(x) && 0 <= k && k < size()); // 调试检查，可删
150         int p = 1;
151         unsigned long long ans = 0;
152         for (int i = W - 1; i >= 0; i--)
153         {
154             int b = x >> i & 1;
155             int q = tr[p][b];
156             int c = q ? tr[q][2] : 0;
157             if (k < c)
158             {
159                 p = q;
160             }
161             else
162             {
163                 k -= c;
164                 p = tr[p][b ^ 1];
165                 ans |= (unsigned long long)1 << i;
166             }
167         }
168         return ans;
169     }
170 
171     unsigned long long kthElement(unsigned long long x, int k) const
172     {
173         // x 是询问整数，k 从 0 开始；返回使 x xor y 第 k 小的元素 y。
174         return x ^ kthXor(x, k);
175     }
176 
177     int countXorLess(unsigned long long x, unsigned long long k) const
178     {
179         // x、k 是询问参数；返回满足 (x xor y) < k 的元素个数。
180         assert(valid(x)); // 调试检查，可删
181         if constexpr (W < 64)
182         {
183             if (k >= ((unsigned long long)1 << W))
184             {
185                 return size();
186             }
187         }
188         int p = 1, ans = 0;
189         for (int i = W - 1; i >= 0 && p; i--)
190         {
191             int a = x >> i & 1;
192             int b = k >> i & 1;
193             if (b)
194             {
195                 int q = tr[p][a];
196                 if (q)
197                 {
198                     ans += tr[q][2];
199                 }
```

```
                p = tr[p][a ^ 1];
            }
            else
            {
                p = tr[p][a];
            }
        }
        return ans;
    }
};
```

构造 `BinaryTrie<W> tr(n)`，W 是参与比较的二进制位数、n 仅用于 `reserve`；`insert(x,v)` 插入 v 份，`count`、`maxXor`、`maxElement`、`minXor`、`minElement`、`kthXor`（k 从 0 起）、`kthElement`、`countXorLess(x,k)` 查询。所有值须落在 $[0,2^W)$，$W=64$ 时允许全部 `unsigned long long`。例如 `BinaryTrie<4> tr` 依次插入 1、6、6 后 `count(6)==2`、`maxXor(8)==14`、`maxElement(8)==6`、`minXor(7)==1`、`kthXor(2,0)==3`、`kthXor(2,2)==4`、`countXorLess(2,4)==1`。每次操作 $O(W)$，空间 $O(nW)$（n 为插入总次数）。需要删除时启用 3.6.003 可删二进制 Trie（在此基础上加 `erase` 并令查询检查正计数）；查询历史版本或区间异或最值时改用 3.6.010 可持久化 01 Trie；仅做键值映射而非异或查询时 3.6.006 哈希表 更直接，但最坏复杂度不稳定。

下面用 64 位边界值固定检查重复计数、异或最值、第 k 小、对应原元素和阈值计数；另用 $W=4$ 检查阈值达到 2^W 时返回全部元素：

```
signed main()
{
    constexpr unsigned long long HI = 1ULL << 63;
    constexpr unsigned long long ALL = numeric_limits<unsigned long long>::max();
    BinaryTrie<64> tr;
    tr.insert(0);
    tr.insert(HI);
    tr.insert(ALL, 2);
    assert(tr.size() == 4 && tr.count(ALL) == 2);
    assert(tr.maxXor(0) == ALL && tr.maxElement(0) == ALL);
    assert(tr.minXor(0) == 0 && tr.minElement(0) == 0);
    assert(tr.kthXor(ALL, 0) == 0);
    assert(tr.kthXor(ALL, 2) == HI - 1);
    assert(tr.kthElement(ALL, 2) == HI);
    assert(tr.countXorLess(ALL, 1) == 2);
    assert(tr.countXorLess(ALL, HI) == 3);

    BinaryTrie<4> small;
    small.insert(15);
    assert(small.countXorLess(0, 16) == 1);
    cout << "4 2 0 9223372036854775807 3\n";
    return 0;
}
```

3.6.003 可删二进制 Trie

在 3.6.002 01 Trie 的每个节点上维护经过次数，`erase(x,v)` 沿根到叶把路径计数各减 v、但不回收节点，从而支持重复值与单值删除。关键坑点在于删除后计数归零的分支仍然物理存在，`maxXor`/`minXor` 因此都必须同时检查儿子编号非零和其经过计数为正（代码中 `q && tr[q][2]` 一并判断），否则会误走进已删空的历史分支。本节复用 3.6.002，只打印新增的派生层：

```cpp
// 依赖 3.6.002 01 Trie / 压位 Trie
template <int W = 30> struct EraseBinTrie : BinaryTrie<W>
{
    using BinaryTrie<W>::BinaryTrie;
    using BinaryTrie<W>::count;
    using BinaryTrie<W>::tr;
    using BinaryTrie<W>::valid;

    bool erase(unsigned long long x, int v = 1)
    {
        // x 是待删除整数，v 是删除份数；数量不足时不修改并返回 false。
        assert(valid(x) && v > 0); // 调试检查，可删
        if (count(x) < v)
        {
            return false;
        }
        int p = 1;
        tr[p][2] -= v;
        for (int i = W - 1; i >= 0; i--)
        {
            p = tr[p][x >> i & 1];
            tr[p][2] -= v;
        }
        return true;
    }
};
```

先放置 3.6.002 的 `BinaryTrie`，再构造 `EraseBinTrie<W> tr(n)`；`insert(x,v)` 插入 v 份，`erase(x,v)` 删除 v 份并在数量不足时不改动而返回 `false`。其余查询接口继承自普通版：`maxXor`、`minXor`、`kthXor` 返回异或值，`maxElement`、`minElement`、`kthElement` 返回取得该异或值的现存元素，`countXorLess` 做阈值计数。删除不回收节点，但计数归零的分支不会再参与查询。各操作 $O(W)$，空间 $O(\text{插入次数} \times W)$。不需要删除时用更精简的 3.6.002 01 Trie；需要查询历史版本则改用 3.6.010 可持久化 01 Trie。

下面沿用 64 位边界值，删除重复最大值后检查剩余分支、对应原元素和删除失败不改状态：

```cpp
signed main()
{
    constexpr unsigned long long HI = 1ULL << 63;
    constexpr unsigned long long ALL = numeric_limits<unsigned long long>::max();
    EraseBinTrie<64> tr;
    tr.insert(0);
    tr.insert(HI);
    tr.insert(ALL, 2);
    assert(tr.erase(ALL) && tr.count(ALL) == 1);
```

```
    assert(tr.maxXor(0) == ALL && tr.maxElement(0) == ALL);
    assert(tr.minXor(0) == 0 && tr.minElement(0) == 0);
    assert(tr.kthXor(ALL, 1) == HI - 1);
    assert(tr.kthElement(ALL, 1) == HI);
    assert(tr.erase(ALL) && !tr.erase(ALL));
    assert(tr.size() == 2 && tr.maxXor(0) == HI);
    assert(tr.countXorLess(ALL, HI) == 1);
    cout << "1 9223372036854775808 1\n";
    return 0;
}
```

3.6.004 Trie Graph

Trie 只负责把一组字符串的公共前缀压成状态节点，题目给出的跳转、代价或约束再作为普通有向边挂到这些状态上，随后按边权性质跑 BFS、Dijkstra 或 DP。要点是把「压前缀」与「图上转移」分开看待，而不是把 AC 自动机的 fail 转移当成通用 Trie Graph——这里的 `shortest` 用 Dijkstra，故要求边权非负：

```
template <int A = 26, char B = 'a', class Dist = long long> struct TrieGraph
{
    struct Node
    {
        // ch 保存 Trie 字符转移；e 保存从当前状态出发的 {目标状态, 非负边权}。
        array<int, A> ch{};
        vector<pair<int, Dist>> e;
    };

    // tr 保存所有 Trie 状态及额外图边。
    vector<Node> tr{{}};

    int insert(const string &s)
    {
        // s 表示待插入字符串；返回其终止状态编号。
        int p = 0;
        for (char c : s)
        {
            int x = c - B;
            assert(0 <= x && x < A); // 调试检查，可删
            if (!tr[p].ch[x])
            {
                tr[p].ch[x] = tr.size();
                tr.push_back({});
            }
            p = tr[p].ch[x];
        }
        return p;
    }

    void addEdge(int u, int v, Dist w, bool directed = true)
    {
        // u、v 表示状态编号，w 表示非负边权；directed 为 false 时同时加入反向边。
```

```
        assert(0 <= u && u < (int)tr.size() && 0 <= v && v < (int)tr.size() && w >= 0); // 调试检查，可删
        tr[u].e.push_back({v, w});
        if (!directed)
        {
            tr[v].e.push_back({u, w});
        }
    }

    vector<optional<Dist>> shortest(int s) const
    {
        // s 是源状态；返回到各状态的 Dist 距离，不可达位置为空。
        assert(0 <= s && s < (int)tr.size()); // 调试检查，可删
        vector<optional<Dist>> d(tr.size());
        priority_queue<pair<Dist, int>, vector<pair<Dist, int>>, greater<>> q;
        d[s] = 0;
        q.push({0, s});
        while (!q.empty())
        {
            auto [du, u] = q.top();
            q.pop();
            if (!d[u] || du != *d[u])
            {
                continue;
            }
            for (auto [v, w] : tr[u].e)
            {
                Dist nd = du + w;
                if (!d[v] || nd < *d[v])
                {
                    d[v] = nd;
                    q.push({nd, v});
                }
            }
        }
        return d;
    }
};
```

构造 `TrieGraph<> g`，用 `insert(s)` 得到字符串终止状态编号，再以 `addEdge(u,v,w,directed)` 加题目额外转移（`directed=false` 同时加反向边）。边权非负时，`shortest(s)` 默认返回 `vector<optional<long long>>`；需要宽距离时构造 `TrieGraph<A, B, __int128>`。有值表示可达并保存最短距离，`nullopt` 表示不可达；调用者须保证最短路长度能由 `Dist` 表示。Trie 的父子字符边是否加入、方向与权值均由题意决定，本结构不自动连它们。例如插入 `ab`、`ac` 得终点 u、v，加根到 u 权 5、u 到 v 权 2、根到 v 权 10，则 `*d[u]==5`、`*d[v]==7`。插入长度 L 的串为 $O(L)$，`shortest` 为 $O((S+E)\log S)$，空间 $O(AS+E)$，其中 S、E 是状态数与额外边数。需要多模式串在文本上匹配的 fail 自动机见 4.2.002 AC 自动机；纯字符字典查询见 3.6.001 字符 Trie。

3.6.005 线段树优化 Trie Graph

当一个状态需要向整段连续编号的目标批量连边时，直接连会退化成稠密图；线段树优化建图把编号区间建成两棵线段树 DAG（一棵「向下」用于点到区间、一棵「向上」用于区间到点），每条覆盖 $[l, r)$ 的批量转移只落在 $O(\log n)$ 个区间节点上，再统一跑最短路。`pointRange`/`rangePoint`/`rangeRange` 分别对应三种批量连边，附带的 `dualShortest` 是平面图对偶最短路的独立小工具。这份底层与图论章的线段树优化建图 5.2.006 同源：

```
template <class Dist = long long> struct RangeGraph
{
    // n 是原图点数，sz 是补成二次幂后的线段树叶子数。
    int n, sz;
    // g 保存两棵线段树节点、中转点及其非负权边。
    vector<vector<pair<int, Dist>>> g;

    RangeGraph(int n) : n(n), sz(bit_ceil((unsigned)(max<int>(1, n)))), g(4 * sz)
    {
        // n 是原图点数；建立向下树、向上树及对应叶子的零权连接。
        assert(n >= 0); // 调试检查，可删
        for (int i = 1; i < sz; i++)
        {
            g[i].push_back({i * 2, 0});
            g[i].push_back({i * 2 + 1, 0});
            g[2 * sz + i * 2].push_back({2 * sz + i, 0});
            g[2 * sz + i * 2 + 1].push_back({2 * sz + i, 0});
        }
        for (int i = 0; i < sz; i++)
        {
            g[sz + i].push_back({3 * sz + i, 0});
            g[3 * sz + i].push_back({sz + i, 0});
        }
    }

    vector<int> cover(int l, int r, int off) const
    {
        // l、r 是原点半开区间，off 是 0 或 2*sz；返回其线段树分解节点。
        assert(0 <= l && l <= r && r <= n && (off == 0 || off == 2 * sz)); // 调试检查，可删
        vector<int> q;
        for (l += sz, r += sz; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1)
            {
                q.push_back(off + l++);
            }
            if (r & 1)
            {
                q.push_back(off + --r);
            }
        }
        return q;
    }
```

```cpp

    void pointRange(int u, int l, int r, Dist w)
    {
        // u 是起点，[l,r) 是终点区间，w 是非负边权。
        assert(0 <= u && u < n && w >= 0); // 调试检查，可删
        for (int v : cover(l, r, 0))
        {
            g[sz + u].push_back({v, w});
        }
    }

    void rangePoint(int l, int r, int v, Dist w)
    {
        // [l,r) 是起点区间，v 是终点，w 是非负边权。
        assert(0 <= v && v < n && w >= 0); // 调试检查，可删
        for (int u : cover(l, r, 2 * sz))
        {
            g[u].push_back({sz + v, w});
        }
    }

    void rangeRange(int l, int r, int x, int y, Dist w)
    {
        // [l,r) 是起点区间，[x,y) 是终点区间，w 是非负边权。
        assert(0 <= l && l <= r && r <= n && 0 <= x && x <= y && y <= n && w >= 0); // 调试检查，
        ↪ 可删
        if (l == r || x == y)
        {
            return;
        }
        int p = g.size();
        g.push_back({});
        for (int u : cover(l, r, 2 * sz))
        {
            g[u].push_back({p, 0});
        }
        for (int v : cover(x, y, 0))
        {
            g[p].push_back({v, w});
        }
    }

    vector<optional<Dist>> dijkstra(int s) const
    {
        // s 是原图源点；返回到各原点的 Dist 距离，不可达位置为空。
        assert(0 <= s && s < n); // 调试检查，可删
        vector<optional<Dist>> d(g.size());
        priority_queue<pair<Dist, int>, vector<pair<Dist, int>>, greater<>> q;
        d[sz + s] = 0;
        q.push({0, sz + s});
        while (!q.empty())
        {
```

```
            auto [du, u] = q.top();
            q.pop();
            if (!d[u] || du != *d[u])
            {
                continue;
            }
            for (auto [v, w] : g[u])
            {
                Dist nd = du + w;
                if (!d[v] || nd < *d[v])
                {
                    d[v] = nd;
                    q.push({nd, v});
                }
            }
        }
        vector<optional<Dist>> ans(n);
        for (int i = 0; i < n; i++)
        {
            ans[i] = d[sz + i];
        }
        return ans;
    }
};

template <class Dist = long long>
vector<optional<Dist>> dualShortest(int faces, int s, const vector<array<long long, 3>> &edge)
{
    // faces 是面数，s 是源面；edge 每项为 {左面，右面，非负边权}，负面编号表示外部而忽略。
    assert(0 < faces && 0 <= s && s < faces); // 调试检查，可删
    vector<vector<pair<int, Dist>>> g(faces);
    for (auto [l, r, w] : edge)
    {
        if (l >= 0 && r >= 0)
        {
            assert(l < faces && r < faces && w >= 0); // 调试检查，可删
            g[l].push_back({(int)r, (Dist)w});
            g[r].push_back({(int)l, (Dist)w});
        }
    }
    vector<optional<Dist>> d(faces);
    priority_queue<pair<Dist, int>, vector<pair<Dist, int>>, greater<>> q;
    d[s] = 0;
    q.push({0, s});
    while (!q.empty())
    {
        auto [du, u] = q.top();
        q.pop();
        if (!d[u] || du != *d[u])
        {
            continue;
        }
```

```
        for (auto [v, w] : g[u])
        {
            Dist nd = du + w;
            if (!d[v] || nd < *d[v])
            {
                d[v] = nd;
                q.push({nd, v});
            }
        }
    }
    return d;
}
```

构造 `RangeGraph<> g(n)`（原点编号 $[0,n)$），`pointRange(u,l,r,w)` 从点 u 向区间 $[l,r)$ 连权 w 边，`rangePoint(l,r,v,w)` 反向，`rangeRange(l,r,x,y,w)` 从区间连向区间；空起点或终点区间不加边。所有边权须非负。默认 `Dist=long long`，需要宽距离时构造 `RangeGraph<__int128> g(n)` 或调用 `dualShortest<__int128>`；两者都用 `optional` 表示不可达，调用者须保证最短路长度能由 `Dist` 表示。n 个原点、q 次区间加边后图规模 $O(n+q\log n)$，每次加边 $O(\log n)$，最短路为 $O((n+q\log n)\log(n+q))$；对偶建图 $O(m)$、最短路 $O((f+m)\log f)$。批量连边模型与图论章的 5.2.006 线段树优化建图 同源，只是这里放在 Trie 状态图语境下使用。

3.6.006 哈希表

键到值的映射用 `unordered_map`，但整数键不能保留默认哈希：它近似恒等，存在针对桶号的构造性攻击。这里直接复用 1.4.004 `unordered_map` / `unordered_set` 防 hack 哈希 已定义的 `CustomHash`，不再复制 `splitmix64`；插入前先设装载因子并预留容量，避免 $Q=10^6$ 时反复重哈希。若题目能事先确定不同键数上界且不删除键，可用定长开放寻址版：

```
// 依赖 1.4.004 unordered_map / unordered_set 防 hack 哈希
template <class K, class V> using HashMap = unordered_map<K, V, CustomHash>;
template <class K> using HashSet = unordered_set<K, CustomHash>;

template <class K, class V, int N, class H = CustomHash> class FixedHashMap
{
    static_assert(1 <= N && N <= 30);
    static constexpr int CAP = 1LL << N; // CAP 是槽数 2^N，须严格大于不同键数。
    static constexpr int MASK = CAP - 1; // MASK 用于线性探测时回绕下标。

    int cnt = 0; // cnt 是当前不同键的个数。
    unique_ptr<K[]> key{new K[CAP]}; // key[p] 是槽 p 中的键。
    unique_ptr<V[]> val{new V[CAP]}; // val[p] 是槽 p 中键对应的值。
    vector<unsigned long long> used = vector<unsigned long long>((CAP + 63) / 64); // used 的第 p 位表示槽 p 已占用。
    H hs; // hs 是哈希函数对象。

    bool test(int p) const { return used[p >> 6] >> (p & 63) & 1ULL; }
    void set(int p) { used[p >> 6] |= 1ULL << (p & 63); }

    int slot(const K &x) const
```

```
    {
        // x 是键；返回 Fibonacci 散列后的初始槽位，取乘积的高 N 位。
        unsigned long long h = (unsigned long long)hs(x);
        h *= 11400714819323198485ULL;
        return h >> (64 - N);
    }

    int findPos(const K &x) const
    {
        // x 是键；线性探测返回 x 所在槽，不存在时返回第一个空槽。
        int p = slot(x);
        while (test(p) && key[p] != x)
        {
            p = (p + 1) & MASK;
        }
        return p;
    }

  public:
    V &operator[](const K &x)
    {
        // x 是键；不存在时插入值为 V{} 的新键，返回值的引用。
        assert(cnt < CAP); // 调试检查，可删；容量必须严格大于不同键数。
        int p = findPos(x);
        if (!test(p))
        {
            set(p);
            key[p] = x;
            val[p] = V{};
            cnt++;
        }
        return val[p];
    }

    V *find(const K &x)
    {
        // x 是键；返回值的指针，不存在时返回 nullptr。
        int p = findPos(x);
        return test(p) ? &val[p] : nullptr;
    }

    const V *find(const K &x) const
    {
        // x 是键；返回值的只读指针，不存在时返回 nullptr。
        int p = findPos(x);
        return test(p) ? &val[p] : nullptr;
    }

    bool contains(const K &x) const { return find(x) != nullptr; }

    V get(const K &x) const
    {
```

```
        // x 是键；返回对应值，不存在时返回 V{} 且不插入。
        auto p = find(x);
        return p ? *p : V{};
    }

    int size() const { return cnt; }

    void clear()
    {
        // 无参数；清空全部键，键值数组不重置，只清占用标记。
        fill(used.begin(), used.end(), 0);
        cnt = 0;
    }
};

template <class K, int N, class H = CustomHash> class FixedHashSet
{
    static_assert(1 <= N && N <= 30);
    static constexpr int CAP = 1LL << N; // CAP 是槽数 2^N，须严格大于元素数。
    static constexpr int MASK = CAP - 1; // MASK 用于线性探测时回绕下标。

    // cnt 是当前元素数，key 保存槽内键，used 标记槽是否占用。
    int cnt = 0;
    unique_ptr<K[]> key{new K[CAP]};
    vector<unsigned long long> used = vector<unsigned long long>((CAP + 63) / 64);
    H hs;

    bool test(int p) const
    {
        // p 是槽编号；返回该槽是否已被占用。
        return used[p >> 6] >> (p & 63) & 1ULL;
    }

    void set(int p)
    {
        // p 是空槽编号；把该槽标记为占用。
        used[p >> 6] |= 1ULL << (p & 63);
    }

    int slot(const K &x) const
    {
        // x 是待查键；返回经过随机哈希与乘法混合后的初始槽。
        unsigned long long h = (unsigned long long)hs(x);
        h *= 11400714819323198485ULL;
        return h >> (64 - N);
    }

    int findPos(const K &x) const
    {
        // x 是待查键；返回保存 x 的槽，或其探测链上的第一个空槽。
        int p = slot(x);
        while (test(p) && key[p] != x)
```

```
        {
            p = (p + 1) & MASK;
        }
        return p;
    }

  public:
    bool insert(const K &x)
    {
        // x 是待插入键；首次插入返回 true，已经存在返回 false。
        assert(cnt < CAP); // 调试检查，可删；容量必须严格大于不同键数。
        int p = findPos(x);
        if (test(p))
        {
            return false;
        }
        set(p);
        key[p] = x;
        cnt++;
        return true;
    }

    bool contains(const K &x) const
    {
        // x 是待查询键；返回集合是否包含 x。
        return test(findPos(x));
    }

    int size() const
    {
        // 返回当前不同键的数量。
        return cnt;
    }

    void clear()
    {
        // 清空全部键，保留已经分配的固定容量。
        fill(used.begin(), used.end(), 0);
        cnt = 0;
    }
};
```

以整数键为例，构造 `HashMap<int,int> mp;` 后先执行 `mp.max_load_factor(.7); mp.reserve(2 * q);`，再用 `mp[x]++` 计数或用标准迭代器 `find`；只保存键时使用 `HashSet<int>`。已知不同键数小于 2^N 时可构造 `FixedHashMap<int,int,N> mp`，用 `operator[]`、返回指针的 `find`、`contains`、缺失时返回默认值的 `get`、`size` 和 `clear`；只需判重或成员查询时改用 `FixedHashSet<int,N> st`，其 `insert(x)` 返回是否首次插入，并提供 `contains`、`size` 和 `clear`。Set 不分配值数组：例如 $N = 21$ 且键为 64 位整数时，键和占用位图约需 `16.25MiB`，同规格 64 位键值 Map 约需 `32.25MiB`。

两种定长结构都建议装载率不超过 0.7，实际不同键数必须严格小于容量，否则探测无法终止；均不支持删除、扩容和迭代。不能通过直接清除占用位实现删除，因为那会截断后续键的线性探测链；需要删除时

改用通用容器或另写墓碑/后移删除版本。各版本默认都用随机种子的 `CustomHash`，不接受确定性乘法哈希的构造性碰撞；字符串或其他复合键须提供对应哈希器。操作期望 $O(1)$、最坏仍为 $O(n)$，通用容器空间期望 $O(n)$、定长版空间 $O(2^N)$。

在 Ryzen 9 7945HX、MinGW GCC 14.2、`-O2 -DNDEBUG` 下，以 2^{20} 个槽执行一百万次固定随机混合操作（约三分之二插入，最终装载率约 0.64），预热两轮后的 7 轮中位数约为 `12.2ms`；同样使用 `CustomHash`、装载因子 0.7 且预留一百万项的 `unordered_set` 约为 `122.0ms`。计时不包含容器构造和 `reserve`，每轮都核对查询校验和与最终大小；这是特定机器与分布下的热点结果。Map 在此前同机一百万次混合读写中约为 `15ms`，同样防卡的 `unordered_map` 约为 `70ms`。要求稳定最坏复杂度时改用 `map` / `set`；需要删除或未知容量时仍用 `HashMap` / `HashSet` 或 3.9.011 cc_hash_table 与 gp_hash_table。若键是字符串前缀且需前缀/整串计数，用 3.6.001 字符 Trie 更稳定。

3.6.007 可持久化数组

把数组用一棵线段树表示、叶子存单个下标的值，则单点赋值只需复制从根到该叶的一条链（$O(\log n)$ 个新节点），其余节点与旧版本共享，于是既能查询任意历史版本的某位置、又能从任意旧版本继续修改。这是可持久化数据结构的最基本形态，也是可持久化并查集等结构的底座：

```
template <class T> class PerArray
{
    struct Node
    {
        // left、right 是儿子编号，value 仅在叶子保存数组值。
        int left = 0, right = 0;
        T value{};
    };

    // n 是数组长度，nodes 是节点池，roots[v] 是版本 v 的根。
    int n;
    vector<Node> nodes{{}};
    vector<int> roots;

    int build(const vector<T> &a, int l, int r)
    {
        // a 是初始数组，[l,r) 是当前区间；返回建成的节点编号。
        int p = nodes.size();
        nodes.push_back({});
        if (r - l == 1)
        {
            nodes[p].value = a[l];
            return p;
        }
        int m = midpoint(l, r);
        nodes[p].left = build(a, l, m);
        nodes[p].right = build(a, m, r);
        return p;
    }

    int set(int p, int l, int r, int x, const T &value)
```

```
    {
        // p 是旧节点，[l,r) 是其区间；把位置 x 改为 value 并返回新节点。
        int q = nodes.size();
        nodes.push_back(nodes[p]);
        if (r - l == 1)
        {
            nodes[q].value = value;
            return q;
        }
        int m = midpoint(l, r);
        if (x < m)
        {
            nodes[q].left = set(nodes[p].left, l, m, x, value);
        }
        else
        {
            nodes[q].right = set(nodes[p].right, m, r, x, value);
        }
        return q;
    }

  public:
    explicit PerArray(const vector<T> &a) : n(a.size())
    {
        // a 是初始数组；建立版本 0，空数组的根为 0。
        roots.push_back(n ? build(a, 0, n) : 0);
    }

    int set(int version, int position, const T &value)
    {
        // version 是旧版本，position 是位置，value 是新值；返回新增版本编号。
        assert(0 <= version && version < (int)roots.size()); // 调试检查，可删
        assert(0 <= position && position < n); // 调试检查，可删
        roots.push_back(set(roots[version], 0, n, position, value));
        return roots.size() - 1;
    }

    T get(int version, int position) const
    {
        // version 是历史版本，position 是位置；按值返回该位置的内容。
        assert(0 <= version && version < (int)roots.size()); // 调试检查，可删
        assert(0 <= position && position < n); // 调试检查，可删
        int p = roots[version], l = 0, r = n;
        while (r - l > 1)
        {
            int m = midpoint(l, r);
            if (position < m)
            {
                p = nodes[p].left;
                r = m;
            }
            else
```

```
            {
                p = nodes[p].right;
                l = m;
            }
        }
        return nodes[p].value;
    }
};
```

用初始序列构造 `PerArray<T> pa(a)` 并生成版本 0；空数组也有版本 0，但没有合法位置可改查。`set(version,position,value)` 基于旧版本生成新版本并返回编号，`get(version,position)` 按值返回某版本的位置内容，因此随后扩容节点池也不会使查询结果失效。初始构造 $O(n)$，每次 `set` 为 $O(\log n)$ 时间与新增空间，`get` 为 $O(\log n)$。需要按版本回答区间第 k 小用 3.3.008 可持久化线段树 / 主席树；需要可持久化的整数异或查询用 3.6.010 可持久化 01 Trie；把它套上按大小合并即得 3.1.005 可持久化并查集。

3.6.008 手写 bitset

标准 `bitset` 的长度必须编译期确定，运行期才知长度时改用这份把位装进 64 位字数组的手写版：按位与、或、异或逐字执行；左右移要同时处理整字偏移 `shift/64` 与字内偏移 `shift%64`，并把跨字进位用相邻字的补偿位拼上。唯一容易写错的地方是每次可能改动最高字的操作后都要 `trim()`，把最后一个字里超出逻辑长度 `n` 的高位清零，否则 `count`、移位链会把垃圾位算进去：

```
class BitVec
{
    // n 是逻辑位数，words 按低位到高位保存 64 位块。
    int n = 0;
    vector<unsigned long long> words;

    void trim()
    {
        // 清除最高字中超出逻辑长度的位。
        if (n % 64)
        {
            words.back() &= (1ULL << (n % 64)) - 1;
        }
    }

  public:
    explicit BitVec(int n = 0) : n(n)
    {
        // n 是逻辑位数；初值全为零。
        assert(n >= 0); // 调试检查，可删
        words.resize(n / 64 + (n % 64 != 0));
    }

    bool test(int x) const
    {
        // x 是位下标；返回该位是否为一。
```

```
        assert(0 <= x && x < n); // 调试检查，可删
        return words[x / 64] >> (x % 64) & 1;
    }

    void set(int x, bool value = true)
    {
        // x 是位下标，value 是新位值。
        assert(0 <= x && x < n); // 调试检查，可删
        if (value)
        {
            words[x / 64] |= 1ULL << (x % 64);
        }
        else
        {
            reset(x);
        }
    }

    void reset(int x)
    {
        // x 是位下标；把该位清零。
        assert(0 <= x && x < n); // 调试检查，可删
        words[x / 64] &= ~(1ULL << (x % 64));
    }

    void setAll()
    {
        // 把全部有效位设为一。
        ranges::fill(words, ~0ULL);
        if (!words.empty())
        {
            trim();
        }
    }

    void resetAll()
    {
        // 把全部位清零。
        ranges::fill(words, 0);
    }

    int count() const
    {
        // 返回一的个数。
        int result = 0;
        for (auto word : words)
        {
            result += popcount(word);
        }
        return result;
    }

```

```
    BitVec &operator|=(const BitVec &other)
    {
        // other 是等长位向量；逐位或后返回自身。
        assert(n == other.n); // 调试检查，可删
    for (int i = 0; i < (int)words.size(); ++i)
        {
            words[i] |= other.words[i];
        }
        return *this;
    }

    BitVec &operator&=(const BitVec &other)
    {
        // other 是等长位向量；逐位与后返回自身。
        assert(n == other.n); // 调试检查，可删
    for (int i = 0; i < (int)words.size(); ++i)
        {
            words[i] &= other.words[i];
        }
        return *this;
    }

    BitVec &operator^=(const BitVec &other)
    {
        // other 是等长位向量；逐位异或后返回自身。
        assert(n == other.n); // 调试检查，可删
    for (int i = 0; i < (int)words.size(); ++i)
        {
            words[i] ^= other.words[i];
        }
        return *this;
    }

    BitVec &operator<<=(int shift)
    {
        // shift 是非负左移量；越界位丢弃，返回自身。
        assert(shift >= 0); // 调试检查，可删
        if (shift >= n)
        {
            resetAll();
            return *this;
        }
        int block = shift / 64, offset = shift % 64;
    for (int i = (int)words.size() - 1; i >= 0; --i)
        {
            unsigned long long value = i >= block ? words[i - block] << offset : 0;
            if (offset && i > block)
            {
                value |= words[i - block - 1] >> (64 - offset);
            }
            words[i] = value;
        }
```

```
        trim();
        return *this;
    }

    BitVec &operator>>=(int shift)
    {
        // shift 是非负右移量；越界位丢弃，返回自身。
        assert(shift >= 0); // 调试检查，可删
        if (shift >= n)
        {
            resetAll();
            return *this;
        }
        int block = shift / 64, offset = shift % 64;
    for (int i = 0; i < (int)words.size(); ++i)
        {
        unsigned long long value = i + block < (int)words.size() ? words[i + block] >> offset :
        ↪ 0;
        if (offset && i + block + 1 < (int)words.size())
            {
                value |= words[i + block + 1] << (64 - offset);
            }
            words[i] = value;
        }
        return *this;
    }
};
```

构造 `BitVec v(n)` 得长度 n 的全零位向量，`test`/`set`/`reset` 单点读写，`setAll`/`resetAll` 整表置位清零，`count` 返回一的个数，`|= & ^=` 要求两侧等长，`<<= >>=` 按位移量整体移动。单点操作 $O(1)$，`count`、整表位运算与移位 $O(n/64)$，空间 $O(n/64)$。长度编译期已知时直接用标准 `bitset` 常数更小；只需按块判 01 异或最值用 3.6.002 01 Trie。

下面固定检查第 $63/64$ 位交界、整字移动、最高字裁剪、按位运算、全置位与空位向量：

```
signed main()
{
    BitVec b(70);
    for (int x : {0, 63, 64, 69})
    {
        b.set(x);
    }
    assert(b.count() == 4);

    BitVec l = b;
    l <<= 1;
    assert(l.count() == 3 && l.test(1) && l.test(64) && l.test(65));
    BitVec r = b;
    r >>= 1;
    assert(r.count() == 3 && r.test(62) && r.test(63) && r.test(68));
    BitVec x = b;
```

```
    x <<= 64;
    assert(x.count() == 1 && x.test(64));
    BitVec y = b;
    y >>= 64;
    assert(y.count() == 2 && y.test(0) && y.test(5));
    x = b;
    x <<= 70;
    assert(x.count() == 0);

    BitVec p(70), q(70);
    p.set(63);
    p.set(64);
    q.set(64);
    q.set(69);
    BitVec z = p;
    z |= q;
    assert(z.count() == 3);
    z = p;
    z &= q;
    assert(z.count() == 1 && z.test(64));
    z = p;
    z ^= q;
    assert(z.count() == 2 && z.test(63) && z.test(69));

    BitVec all(70);
    all.setAll();
    assert(all.count() == 70 && all.test(69));
    all <<= 70;
    assert(all.count() == 0);
    BitVec empty(0);
    empty >>= 0;
    assert(empty.count() == 0);
    cout << "OK\n";
}
```

3.6.009 动态区间频率索引

要在带单点修改的数组上查「值 v 在下标区间 $[l, r)$ 出现几次」，做法是离线先扫出每个位置将来可能取到的所有值，为每个值单独建一棵「以该值出现过的位置为下标」的树状数组；区间频次就是该值树状数组上两次前缀和之差。因为位置被离散化进各值自己的坐标系，`add` 里先 `lower_bound` 找到该位置在此值坐标下的秩再更新。修改一个位置就是在旧值树状数组减一、新值树状数组加一：

```
class Freq
{
    // a 是当前数组，id 把值映射到内部编号。
    vector<int> a;
    unordered_map<int, int, CustomHash> id;
    // pos[k] 是值 k 预登记过的位置，bit[k] 是对应树状数组。
    vector<vector<int>> pos, bit;

```

```
    void add(int v, int p, int d)
    {
        // v 是值，p 是预登记位置，d 是出现次数增量。
        auto it = id.find(v);
        assert(it != id.end()); // 调试检查，可删
        int k = it->second;
        auto jt = lower_bound(pos[k].begin(), pos[k].end(), p);
        assert(jt != pos[k].end() && *jt == p); // 调试检查，可删
        int x = jt - pos[k].begin() + 1;
        for (int i = x; i < (int)bit[k].size(); i += i & -i)
        {
            bit[k][i] += d;
        }
    }

    int pre(int k, int p) const
    {
        // k 是内部值编号，p 是位置上界；返回该值在 [0,p) 的出现次数。
        int x = lower_bound(pos[k].begin(), pos[k].end(), p) - pos[k].begin();
        int ans = 0;
        for (int i = x; i > 0; i -= i & -i)
        {
            ans += bit[k][i];
        }
        return ans;
    }

public:
    Freq(vector<int> init, const vector<pair<int, int>> &qs) : a(move(init))
    {
        // init 是初始数组，qs 是以后可能写入的全部 {位置, 值}。
        vector<pair<int, int>> all = qs;
        all.reserve(all.size() + a.size());
        for (int p = 0; p < (int)a.size(); p++)
        {
            all.push_back({p, a[p]});
        }
        id.max_load_factor(.7);
        id.reserve(2 * all.size() + 1);
        for (auto [p, v] : all)
        {
            assert(0 <= p && p < (int)a.size()); // 调试检查，可删
            auto [it, ok] = id.emplace(v, id.size());
            if (ok)
            {
                pos.emplace_back();
            }
            pos[it->second].push_back(p);
        }
        bit.resize(pos.size());
        for (int k = 0; k < (int)pos.size(); k++)
        {
```

```
            auto &p = pos[k];
            sort(p.begin(), p.end());
            p.erase(unique(p.begin(), p.end()), p.end());
            bit[k].assign(p.size() + 1, 0);
        }
        for (int p = 0; p < (int)a.size(); p++)
        {
            add(a[p], p, 1);
        }
    }

    void setValue(int p, int v)
    {
        // p 是位置，v 是新值；要求 {p,v} 已在构造时预登记。
        assert(0 <= p && p < (int)a.size()); // 调试检查，可删
        auto it = id.find(v);
        assert(it != id.end() && binary_search(pos[it->second].begin(), pos[it->second].end(),
        ↪  p)); // 调试检查，可删
        add(a[p], p, -1);
        a[p] = v;
        add(v, p, 1);
    }

    int frequency(int l, int r, int v) const
    {
        // l、r 是半开区间端点，v 是查询值；返回 v 在 [l,r) 的出现次数。
        assert(0 <= l && l <= r && r <= (int)a.size()); // 调试检查，可删
        auto it = id.find(v);
        if (it == id.end())
        {
            return 0;
        }
        int k = it->second;
        return pre(k, r) - pre(k, l);
    }
};
```

构造 `Freq f(initial, qs)`：`initial` 是初始数组、`qs` 列出所有将来会被 `setValue` 写入的 `{位置, 值}`（离线预登记，保证每个值都有坐标）；`setValue(position,value)` 单点改值，`frequency(left,right,value)` 查半开区间 $[left, right)$ 内 `value` 的出现次数（未登记的值返回 0）。这里显式依赖 1.4.004 `unordered_map` / `unordered_set` 防 hack 哈希 的 `CustomHash`，并在建表前设置装载因子与桶容量；在该哈希的通常假设下，预处理为 $O((n+q)\log(n+q))$，`setValue` 与 `frequency` 各期望 $O(\log(n+q))$，最坏哈希冲突时可退化，空间 $O(n+q)$。若查询的是区间第 k 小/排名而非指定值频次，用带修主席树或树状数组套线段树；离线且无修改时莫队更简单。

3.6.010 可持久化 01 Trie

给数组每个前缀存一棵 01 Trie 版本，`rt[i]` 是前 i 个数构成的 Trie 根；每次 `append` 只复制从最高位到最低位的一条链，其余节点与上一版共享。查询下标区间 $[l, r)$ 时用版本 r 与版本 l 两棵 Trie 对应位置的

计数差（count）判断某个方向的子树在该区间内是否有元素，从高位起贪心走能令答案该位为 1 的儿子，即得区间与 x 的最大异或：

```
struct PerBitTrie
{
    // Node[0..2] 依次是 0 儿子编号、1 儿子编号、经过该节点的版本区间元素数。
    using Node = array<int, 3>;
    int bit;          // bit 是处理的最高二进制位，默认 62 覆盖非负 long long。
    vector<Node> tr; // tr 保存所有版本共享的节点，0 号节点为空节点。
    vector<int> rt;  // rt[i] 是前 i 个数构成的 Trie 根，rt[0] 为空版本。

    PerBitTrie(int bit = 62) : bit(bit), tr(1, Node{}), rt(1, 0)
    {
        // bit 是最高处理位；构造只含空版本的持久化 Trie。
        assert(0 <= bit && bit <= 62); // 调试检查，可删
    }

    int clone(int p)
    {
        // p 是旧节点编号；复制该节点并返回新节点编号。
        tr.push_back(tr[p]);
        return tr.size() - 1;
    }

    void append(int x)
    {
        // x 是能由 bit+1 位表示的非负整数；追加后生成新前缀版本。
        assert(0 <= x && (bit == 62 || (unsigned long long)x < (1ULL << (bit + 1)))); // 调试检
        ↪ 查，可删
        int old = rt.back();
        int cur = clone(old);
        rt.push_back(cur);
        tr[cur][2]++;
        for (int b = bit; b >= 0; b--)
        {
            int c = x >> b & 1;
            int nxt = clone(tr[old][c]);
            tr[cur][c] = nxt;
            old = tr[old][c];
            cur = nxt;
            tr[cur][2]++;
        }
    }

    int count(int l, int r, int c) const
    {
        // l、r 是两个版本节点，c 是儿子方向；返回版本差中该儿子的元素数。
        return tr[tr[r][c]][2] - tr[tr[l][c]][2];
    }

    int maxXor(int l, int r, int x) const
    {
```

```
        // [l,r) 是下标区间，x 是询问值；返回最大异或值，空区间返回 -1。
        assert(0 <= l && l <= r && r < (int)rt.size()); // 调试检查，可删
        assert(0 <= x && (bit == 62 || (unsigned long long)x < (1ULL << (bit + 1)))); // 调试检
        ↪ 查，可删
        if (l == r)
        {
            return -1;
        }
        int u = rt[l];
        int v = rt[r];
        int ans = 0;
        for (int b = bit; b >= 0; b--)
        {
            int c = x >> b & 1;
            int go = c ^ 1;
            if (count(u, v, go) == 0)
            {
                go ^= 1;
            }
            if (go != c)
            {
                ans |= 1LL << b;
            }
            u = tr[u][go];
            v = tr[v][go];
        }
        return ans;
    }
};
```

构造 `PerBitTrie t(bit)`，`bit` 是最高处理位，默认 62，恰好覆盖全部非负 `long long`。逐个 `append(x)` 建立前缀版本，`maxXor(l,r,x)` 返回下标半开区间 $[l,r)$ 内元素与 x 的最大异或，空区间返回 -1。建树 $O(n\cdot bit)$，单次查询 $O(bit)$，空间 $O(n\cdot bit)$。不需要区间限制、只对全集查异或最值用 3.6.002 01 Trie；查区间第 k 小而非异或时用 3.3.008 可持久化线段树 / 主席树。

下面固定检查默认最高位、单元素区间、普通小值和空区间：

```
signed main()
{
    PerBitTrie t;
    t.append(0);
    t.append(numeric_limits<int>::max());
    t.append(1LL << 62);
    t.append(5);
    assert(t.maxXor(0, 4, 0) == numeric_limits<int>::max());
    assert(t.maxXor(2, 3, numeric_limits<int>::max()) == (1LL << 62) - 1);
    assert(t.maxXor(3, 4, 2) == 7);
    assert(t.maxXor(2, 2, 0) == -1);
    cout << "9223372036854775807 4611686018427387903 7 -1\n";
    return 0;
}
```

3.6.011 静态区间 MEX

区间 MEX 是「区间内没出现的最小非负整数」。做法是在值域上建可持久化线段树：扫描到位置 i 时把值 `a[i]` 的「最后出现位置」更新为 i 生成版本 `rt[i+1]`，每个值域节点维护子树内最小的最后出现位置。查询 $[l,r)$ 就在版本 r 的值域树上找第一个「最后出现位置 $< l$」的值——它在 $[l,r)$ 里没出现过且是最小的，即为 MEX。空节点视作最后出现位置 -1，天然小于任何 l：

```
struct StaticMex
{
    // Node[0..2] 依次是左儿子、右儿子、该值域区间内最小的最后出现位置。
    using Node = array<int, 3>;
    int n;           // n 是原序列长度，值域只需保留 [0,n]。
    vector<Node> tr; // tr 保存所有前缀版本的动态线段树节点，0 号节点表示全为 -1。
    vector<int> rt;  // rt[i] 表示扫描完前 i 个元素后的版本根。

    StaticMex(const vector<int> &a) : n(a.size()), tr(1, Node{0, 0, -1}), rt(n + 1)
    {
        // a 是原序列；构造所有前缀版本供区间 MEX 查询。
        for (int i = 0; i < n; i++)
        {
            rt[i + 1] = rt[i];
            if (0 <= a[i] && a[i] <= n)
            {
                rt[i + 1] = set(rt[i], 0, n + 1, a[i], i);
            }
        }
    }

    int copy(int p)
    {
        // p 是旧节点编号；复制节点并返回新编号。
        tr.push_back(tr[p]);
        return tr.size() - 1;
    }

    int val(int p) const
    {
        // p 是节点编号；返回子树内最小最后出现位置，空节点返回 -1。
        return p ? tr[p][2] : -1;
    }

    int set(int p, int l, int r, int x, int v)
    {
        // p、[l,r) 是旧节点和值域，x 是值，v 是最后出现下标；返回新根。
        auto q = copy(p);
        if (r - l == 1)
        {
            tr[q][2] = v;
            return q;
        }
        auto m = (l + r) / 2;
```

```
        if (x < m)
        {
            tr[q][0] = set(tr[q][0], l, m, x, v);
        }
        else
        {
            tr[q][1] = set(tr[q][1], m, r, x, v);
        }
        tr[q][2] = min(val(tr[q][0]), val(tr[q][1]));
        return q;
    }

    int find(int p, int l, int r, int lim) const
    {
        // p、[l,r) 是当前节点和值域，lim 是询问左端点；返回首个最后出现位置小于 lim 的值。
        if (r - l == 1)
        {
            return l;
        }
        auto m = (l + r) / 2;
        if (val(tr[p][0]) < lim)
        {
            return find(tr[p][0], l, m, lim);
        }
        return find(tr[p][1], m, r, lim);
    }

    int query(int l, int r) const
    {
        // [l,r) 是原序列下标区间；返回其中没有出现的最小非负整数。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return find(rt[r], 0, n + 1, l);
    }
};
```

构造 `StaticMex sm(a)` 传入完整数组，`query(l,r)` 查半开区间 $[l,r)$ 的 MEX；负数与大于 n 的元素不影响 MEX，直接跳过不更新。例如 `a={0,1,3,1,0}` 时 `query(0,5)==2`、`query(1,4)==0`、`query(2,5)==2`。预处理 $O(n\log n)$，单次查询 $O(\log n)$，空间 $O(n\log n)$。带修改的区间 MEX 需回滚莫队或树套树；求整个数组的 MEX 只需一次线性扫描。

下面固定检查整段、子段、空区间，以及会被忽略的负数和大于 n 的值：

```
signed main()
{
    StaticMex sm({0, 1, 3, 1, 0, -2, 100});
    assert(sm.query(0, 7) == 2);
    assert(sm.query(1, 4) == 0);
    assert(sm.query(2, 5) == 2);
    assert(sm.query(3, 3) == 0);
    assert(sm.query(0, 2) == 2);
    StaticMex empty(vector<int>{});
```

```
    assert(empty.query(0, 0) == 0);
    cout << "2 0 2 0 2 0\n";
    return 0;
}
```

3.6.012 按标签维护的 01 Trie

普通 01 Trie 的一个节点只记录总经过次数，无法限定查询属于哪个集合。本版让每个节点保存 `cnt[u][0]`、`cnt[u][1]` 两套计数，相同整数可以分别带两个标签插入；查询标签 t 时仍从高位到低位贪心，但只有 `cnt[son][t]>0` 的儿子可走。`add(x,t,v)` 允许用正 `v` 插入、负 `v` 删除，删除属于合法调用的前提是该标签下已有足够多份 x；标签集合可能为空，所以异或值与对应元素都用 `optional` 返回。

```
template <int W = 30> struct TaggedTrie
{
    using U = unsigned long long;
    static_assert(1 <= W && W <= 64);

    struct Node
    {
        array<int, 2> ch{}; // ch[0..1] 是比特 0、1 方向的儿子，0 表示空。
        array<int, 2> cnt{}; // cnt[t] 是标签 t 的元素经过本节点的次数。
    };

    // tr[0] 是空节点，tr[1] 是根；cnt[u][t] 是标签 t 经过 u 的次数。
    vector<Node> tr{Node{}, Node{}};

    bool valid(U x) const
    {
        // x 是待插入或查询的值；返回 x 是否落在 [0,2^W)。
        if constexpr (W == 64)
        {
            return true;
        }
        return x < (U(1) << W);
    }

    int newNode()
    {
        // 无参数；新建空节点并返回编号。
        tr.push_back(Node{});
        return (int)tr.size() - 1;
    }

    void add(U x, int t, int v = 1)
    {
        // t 是 0/1 标签，v 为正时插入、为负时删除。
        assert(valid(x) && 0 <= t && t < 2 && v != 0); // 调试检查，可删
        assert(v > 0 || tr[1].cnt[t] >= -v);           // 调试检查，可删
        int p = 1;
        tr[p].cnt[t] += v;
```

```
        for (int i = W - 1; i >= 0; i--)
        {
            int b = x >> i & 1;
            if (!tr[p].ch[b])
            {
                assert(v > 0); // 删除时路径必须已经存在
                tr[p].ch[b] = newNode();
            }
            int q = tr[p].ch[b];
            assert(v > 0 || tr[q].cnt[t] >= -v); // 调试检查，可删
            p = q;
            tr[p].cnt[t] += v;
        }
    }

    optional<U> maxXor(U x, int t) const
    {
        // 只在标签 t 的元素中查询最大的 x xor y。
        assert(valid(x) && 0 <= t && t < 2); // 调试检查，可删
        if (!tr[1].cnt[t])
        {
            return nullopt;
        }
        U ans = 0;
        int p = 1;
        for (int i = W - 1; i >= 0; i--)
        {
            int b = x >> i & 1;
            int c = b ^ 1;
            int q = tr[p].ch[c];
            if (!q || !tr[q].cnt[t])
            {
                c ^= 1;
            }
            if (c != b)
            {
                ans |= U(1) << i;
            }
            p = tr[p].ch[c];
        }
        return ans;
    }

    optional<U> maxElement(U x, int t) const
    {
        // 在标签 t 的元素中取使异或值最大的元素；集合为空时返回空。
        auto ans = maxXor(x, t);
        return ans.has_value() ? optional<U>(*ans ^ x) : nullopt;
    }
};
```

构造 `TaggedTrie<W> tr` 后用 `add(x,t)` 插入、`add(x,t,-1)` 删除一份，`maxXor(x,t)` 与 `maxElement(x,t)`

只查看标签 t 的集合；数值须落在 $[0,2^W)$，$W=64$ 时覆盖全部 `unsigned long long`。例如标签 0 插入 $1,6$、标签 1 插入 $6,10$ 后，`maxXor(8,0)==14`、`maxElement(1,1)==10`；删去标签 0 的 6 不影响标签 1。每次操作 $O(W)$，空间 $O(nW)$。不需要标签维度时使用接口更全的 3.6.002 01 Trie / 压位 Trie，需要历史区间时使用 3.6.010 可持久化 01 Trie。

```
signed main()
{
    TaggedTrie<4> tr;
    tr.add(1, 0);
    tr.add(6, 0);
    tr.add(6, 1);
    tr.add(10, 1);
    assert(tr.maxXor(8, 0) == optional<unsigned long long>(14));
    assert(tr.maxElement(1, 1) == optional<unsigned long long>(10));
    tr.add(6, 0, -1);
    assert(tr.maxXor(8, 0) == optional<unsigned long long>(9));
    tr.add(1, 0, -1);
    assert(!tr.maxXor(8, 0));

    TaggedTrie<4> one;
    one.add(5, 0);
    assert(one.maxXor(5, 0) == optional<unsigned long long>(0));
    assert(one.maxElement(5, 0) == optional<unsigned long long>(5));
    cout << "OK\n";
    return 0;
}
```

3.6.013 左儿子右兄弟 Trie

固定数组 Trie 给每个节点预留整个字符集的儿子表，字符集大而实际分支稀疏时浪费明显。左儿子右兄弟表示把 `child` 指向首个儿子、`next` 串起同一父节点的其余儿子，每个非根节点的 `c` 同时表示父边字符；插入新边时挂到兄弟链表头。它把每节点儿子表降成两个下标，但 `childOf` 必须在线性兄弟链上找字符，所以不能沿用固定数组版每字符 $O(1)$ 的说法。

```
template <class Val = int> struct SiblingTrie
{
    struct Node
    {
        char c = 0; // c 是从父亲走到本节点的字符。
        int child = 0; // child 是第一个儿子，0 表示没有。
        int next = 0; // next 是下一个兄弟，0 表示没有。
        int pass = 0; // pass 是经过本节点（以其为前缀）的串数。
        int end = 0; // end 是恰好在本节点结束的串数。
        Val val{}; // val 是最后一次插入在此结束的串时附带的值。
    };

    // 根为 0；非根节点同时充当字符边，child/next 构成儿子兄弟链。
    vector<Node> tr{Node{}};

```

```
    int newNode(char c)
    {
        // c 是边上的字符；新建节点并返回编号。
        tr.push_back(Node{});
        tr.back().c = c;
        return (int)tr.size() - 1;
    }

    int childOf(int p, char c) const
    {
        // p 是节点，c 是字符；顺兄弟链查找字符为 c 的儿子，不存在返回 0。
        for (int q = tr[p].child; q; q = tr[q].next)
        {
            if (tr[q].c == c)
            {
                return q;
            }
        }
        return 0;
    }

    int insert(string_view s, Val v = Val(1))
    {
        // s 是待插入串，v 是附带值；沿途 pass 加一、终点 end 加一并记录 v，返回终点编号。
        int p = 0;
        tr[p].pass++;
        for (char c : s)
        {
            int q = childOf(p, c);
            if (!q)
            {
                q = newNode(c);
                tr[q].next = tr[p].child;
                tr[p].child = q;
            }
            p = q;
            tr[p].pass++;
        }
        tr[p].end++;
        tr[p].val = v;
        return p;
    }

    int find(string_view s) const
    {
        // s 是查询串；返回 s 对应的节点编号，s 不是任何插入串的前缀时返回 -1。
        int p = 0;
        for (char c : s)
        {
            p = childOf(p, c);
            if (!p)
            {
```

```
                return -1;
            }
        }
        return p;
    }

    int lcp(string_view s) const
    {
        // s 是查询串；返回 s 与全部插入串的最长公共前缀长度。
        int p = 0, len = 0;
        for (char c : s)
        {
            p = childOf(p, c);
            if (!p)
            {
                break;
            }
            len++;
        }
        return len;
    }
};
```

`insert(s,v)` 插入一份字符串、把终点值更新为 `v` 并返回节点编号；`find(s)` 返回前缀节点，缺边时返回 -1；`lcp(s)` 返回与已建 Trie 的最长可走前缀长度。节点的 `pass` 是经过次数，`end` 是完整串次数，重复插入会累加两者而覆盖终点 `val`。若沿字符串各节点的度数为 $d_1, d_2, \ldots$，一次操作为 $O(\sum d_i)$，最坏 $O(|s|D)$，空间 $O(\text{总新边数})$；小固定字符集仍用 $O(|s|)$ 的 3.6.001 字符 Trie。

```
signed main()
{
    SiblingTrie<int> tr;
    int ab = tr.insert("ab", 7);
    int ac = tr.insert("ac", 8);
    int ab2 = tr.insert("ab", 9);
    assert(ab == ab2 && tr.tr[ab].end == 2 && tr.tr[ab].val == 9);
    assert(tr.tr[ac].end == 1 && tr.find("ad") == -1);
    assert(tr.lcp("abd") == 2 && tr.lcp("ax") == 1);
    cout << "OK\n";
    return 0;
}
```

3.6.014 分层位集前驱/后继集合

在宇宙 $[0, n)$ 上只做插入、删除、成员判定和前驱/后继时，`set<int>` 为每个已插入元素分配红黑树节点；当 $n \le 10^7$、查询达到 10^6 时，这个节点开销和 $O(\log |S|)$ 查询都不如按机器字组织的集合。3.6.008 手写 bitset 的 `BitVec` 虽也按字存位，但没有非空字摘要和前驱接口，逐字扫描最坏为 $O(n/64)$，不能直接替代。`FastSet` 的第 0 层按元素存位，之后每层把下一层中非零 64 位字再压成一位。插入/删除只在某一字从零变非零或从非零变零时继续修改摘要层；`next` 在当前层右移并找最低位，若该字无元素就上升到摘要层找下一个非空字，找到后沿各层最低位下降；`prev` 对称地用最高位。

这不是动态键域结构：宇宙大小必须先给定，空间随 n 而不是当前 $|S|$ 增长。第 0 层有 $\lceil n/64 \rceil$ 个 64 位字，后续层构成等比级数；$n = 10^7$ 时有效字载荷为 $156250 + 2442 + 39 + 1 = 158732$ 个字，即 1269856 字节，且没有每个元素一个平衡树节点的分配。

```
class FastSet
{
    // n 是元素宇宙 [0,n) 的大小。
    int n = 0;
    // a[0] 按元素位存集合；a[k+1] 按 a[k] 中非空 64 位字存摘要位。
    vector<vector<unsigned long long>> a;

  public:
    FastSet(int n = 0)
    {
        // n 是元素宇宙大小；构造一个初始为空的集合。
        init(n);
    }

    void init(int m)
    {
        // m 是新的元素宇宙大小；清空集合并建立所有 64 叉摘要层。
        assert(m >= 0); // 调试检查，可删
        n = m;
        a.clear();
        do
        {
            a.emplace_back((m + 63) / 64, 0);
            m = (m + 63) / 64;
        } while (m > 1);
    }

    void set(int x)
    {
        // x 是 [0,n) 内元素；把 x 插入集合，已经存在时不改变状态。
        assert(0 <= x && x < n); // 调试检查，可删
        for (auto &level : a)
        {
            int p = x / 64;
            unsigned long long b = 1ULL << (x % 64);
            if (level[p] & b)
            {
                break;
            }
            level[p] |= b;
            x /= 64;
        }
    }

    void reset(int x)
    {
        // x 是 [0,n) 内元素；把 x 从集合删除，原本不存在时不改变状态。
        assert(0 <= x && x < n); // 调试检查，可删
```

```
        for (auto &level : a)
        {
            int p = x / 64;
            unsigned long long b = 1ULL << (x % 64);
            if (!(level[p] & b))
            {
                return;
            }
            level[p] &= ~b;
            if (level[p])
            {
                break;
            }
            x /= 64;
        }
    }

    bool test(int x) const
    {
        // x 是 [0,n) 内元素；返回 x 当前是否属于集合。
        assert(0 <= x && x < n); // 调试检查，可删
        return a[0][x / 64] >> (x % 64) & 1ULL;
    }

    int next(int x) const
    {
        // x 是 [0,n] 内起点；返回不小于 x 的最小集合元素，不存在时返回 -1。
        assert(0 <= x && x <= n); // 调试检查，可删
        if (x == n)
        {
            return -1;
        }
        for (int h = 0; h < (int)a.size(); h++)
        {
            int p = x / 64;
            if (p >= (int)a[h].size())
            {
                return -1;
            }
            unsigned long long w = a[h][p] >> (x % 64);
            if (!w)
            {
                x = p + 1;
                continue;
            }
            x += countr_zero(w);
            while (h)
            {
                h--;
                x *= 64;
                x += countr_zero(a[h][x / 64]);
            }
```

```
            return x < n ? x : -1;
        }
        return -1;
    }

    int prev(int x) const
    {
        // x 是 [-1,n) 内起点；返回不大于 x 的最大集合元素，不存在时返回 -1。
        assert(-1 <= x && x < n); // 调试检查，可删
        if (x < 0)
        {
            return -1;
        }
        for (int h = 0; h < (int)a.size() && x >= 0; h++)
        {
            int p = x / 64;
            unsigned long long w = a[h][p] << (63 - x % 64);
            if (!w)
            {
                x = p - 1;
                continue;
            }
            x -= countl_zero(w);
            while (h)
            {
                h--;
                x = x * 64 + 63;
                x -= countl_zero(a[h][x / 64]);
            }
            return x;
        }
        return -1;
    }
};
```

`FastSet st(n)` 与 `init(n)` 建立宇宙 $[0,n)$ 的空集合，`set(x)`/`reset(x)` 是幂等插入/删除，`test(x)` 判成员；`next(x)` 返回最小的 $y\in S, y\ge x$，允许 $x=n$ 并在无解时返回 `-1`；`prev(x)` 返回最大的 $y\in S, y\le x$，允许 $x=-1$ 并在无解时返回 `-1`。其余下标均须落在 $[0,n)$。若 $H=1+\lfloor \log_{64}\max(1,n-1)\rfloor$ 是层数，`set`、`reset`、`next`、`prev` 均为 $O(H)$ 最坏时间，`test` 为 $O(1)$，初始化和空间均为 $O(\lceil n/64\rceil)$ 个机器字。需要秩、第 k 个或区间计数时用 3.2.003 前缀和上的二分 的树状数组；宇宙很小且编译期已知时标准 `bitset` 更直接；只需少量元素迭代或键域无法预先确定时再用 3.5.007 PBDS 平衡树 或 `set`。

下面固定检查空集、$0/63/64$ 字边界、四层摘要、最大宇宙端点和幂等删除：

```
signed main()
{
    FastSet s(130);
    for (int x : {0, 63, 64, 129})
    {
        s.set(x);
    }
```

```
    assert(s.next(1) == 63 && s.next(65) == 129 && s.next(130) == -1);
    assert(s.prev(-1) == -1 && s.prev(62) == 0 && s.prev(128) == 64);
    s.reset(64);
    s.reset(64);
    assert(s.next(64) == 129 && s.prev(64) == 63);

    FastSet deep(64 * 64 + 1);
    deep.set(0);
    deep.set(4096);
    assert(deep.next(1) == 4096 && deep.prev(4095) == 0);
    cout << "OK\n";
}
```

3.7 函数与空间数据结构

3.7.001 李超线段树

李超线段树维护一族一次函数 $y = kx + b$，在预先给定的横坐标集合上查某点的最小（或最大）值。每个线段树节点保存「在本区间中点处最优」的那条直线：插入新直线时先在中点比较，把中点更优者留在节点上，另一条至多只能在左半或右半区间的一侧翻盘，于是递归进入那一侧。这样每条直线只沿 $O(\log)$ 条边下沉。本版把可能查询的横坐标离散化后建静态树，比较时用真实横坐标 `xs[·]` 取值以避免斜率与整型除法的精度问题：

```
template <class T, class C = T> class LiChao
{
    // 当前实现维护最小值。改成最大值有两种等价方式：反向所有优劣比较并把空值改为负无穷，
    // 或在 k、b 与函数值都能安全取反时插入 {-k,-b}，再把非空结果取反。
  public:
    static_assert(is_floating_point_v<T> || (is_integral_v<T> && is_signed_v<T>));

    struct Line
    {
        // k、b 是一次函数 y=kx+b 的斜率与截距。
        T k{}, b{};

        C get(T x) const
        {
            // x 是查询横坐标；用比较类型 C 计算函数值。
            return (C)k * x + b;
        }
    };

  private:
    // xs 是全部允许查询的横坐标，tr[p] 是线段树节点 p 保存的候选直线。
    vector<T> xs;
    vector<optional<Line>> tr;

    void add(int p, int l, int r, Line f)
    {
```

```
        // p 是节点编号，[l,r) 是横坐标下标段，f 是待加入直线。
        if (!tr[p])
        {
            tr[p] = f;
            return;
        }
        int m = midpoint(l, r);
        bool lef = f.get(xs[l]) < tr[p]->get(xs[l]);
        bool mid = f.get(xs[m]) < tr[p]->get(xs[m]);
        if (mid)
        {
            swap(f, *tr[p]);
        }
        if (r - l == 1)
        {
            return;
        }
        if (lef != mid)
        {
            add(p << 1, l, m, f);
        }
        else
        {
            add(p << 1 | 1, m, r, f);
        }
    }

public:
    LiChao(vector<T> xs) : xs(move(xs))
    {
        // xs 是所有可能查询的横坐标；排序去重并建立最小值李超树。
        // 改最大值时可在系数可安全取反时优先对直线取反再对答案取反，可保留李超树的比较不变量。
        sort(this->xs.begin(), this->xs.end());
        this->xs.erase(unique(this->xs.begin(), this->xs.end()), this->xs.end());
        assert(!this->xs.empty()); // 调试检查，可删
        tr.resize(4 * this->xs.size());
    }

    void add(Line f)
    {
        // f 是待加入直线；使其参与全部预存横坐标的最小值查询。
        // 最大值版可在 -f.k、-f.b 不溢出时调用 add({-f.k, -f.b})，并在 query 的返回值上取负。
        add(1, 0, xs.size(), f);
    }

    optional<C> query(T x) const
    {
        // x 必须在构造坐标中；返回最小函数值，尚未加入直线时返回空。
        // 最大值版若按直线取反，调用者取负即可；空结果仍保持 nullopt。
        int q = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
        assert(q < (int)xs.size() && xs[q] == x); // 调试检查，可删
        optional<C> ans;
```

```
        int p = 1, l = 0, r = xs.size();
        while (true)
        {
            if (tr[p])
            {
                C y = tr[p]->get(x);
                if (!ans || y < *ans)
                {
                    ans = y;
                }
            }
            if (r - l == 1)
            {
                break;
            }
            int m = midpoint(l, r);
            if (q < m)
            {
                p <<= 1;
                r = m;
            }
            else
            {
                p = p << 1 | 1;
                l = m;
            }
        }
        return ans;
    }
};
```

先用全部可能查询的横坐标构造 `LiChao<T> lc(xs)`（自动排序去重），`add({k,b})` 插入直线 $y = kx + b$，`query(x)` 返回 `optional<C>`：有直线时保存最小值，空函数集返回 `nullopt`；x 必须在构造坐标中。`T` 是系数与横坐标类型，`C` 是 $kx + b$ 的计算、比较和返回类型，默认与 `T` 相同；若 `T=long long` 且乘积可能溢出，使用 `LiChao<long long, __int128>`。求最大值可把 k, b 取反并对答案取反。插入与查询均 $O(\log |xs|)$，空间 $O(|xs|)$。横坐标未知时用 3.7.002 动态开点李超树，需要历史版本用 3.7.003 可持久化李超树；斜率与查询都单调时可换 12.3.004 斜率优化 / 李超树优化 的单调凸壳。

3.7.002 动态开点李超树

当横坐标定义域 $[lo, hi)$ 很大而无法离散化时，把 3.7.001 李超线段树 改成动态开点：节点在插线或查询实际经过时才创建，比较规则不变（比较的是整数横坐标本身），未分配的子树表示该段没有候选直线。这样空间只随实际访问路径增长，适合横坐标范围大、操作数相对少的场景：

```
template <class T, class C = T> class DynLiChao
{
    // 当前实现维护最小值。改求最大值时，插入 {-k,-b} 并对非空 query 结果取负；
    // 若直接改比较，插线中 lef/mid 与查询累计的比较必须全部同步反向。
  public:
```

```
    struct Line
    {
        // k、b 是一次函数 y=kx+b 的斜率与截距。
        T k{}, b{};

        C get(T x) const
        {
            // x 是查询横坐标；用比较类型 C 计算函数值。
            return (C)k * x + b;
        }
    };

  private:
    struct Node
    {
        // f 是当前区间中点占优的直线，ch[0..1] 是左右儿子编号，-1 表示尚未分配。
        optional<Line> f;
        array<int, 2> ch{-1, -1};
    };

    // lo、hi 定义整数查询域 [lo,hi)，tr 保存实际访问到的节点。
    T lo, hi;
    vector<Node> tr{{}};

    int newNode()
    {
        // 分配一个空节点并返回编号。
        tr.push_back({});
        return tr.size() - 1;
    }

    void add(int p, T l, T r, Line f)
    {
        // p 是节点编号，[l,r) 是整数域，f 是待加入直线。
        if (!tr[p].f)
        {
            tr[p].f = f;
            return;
        }
        T m = midpoint(l, r);
        bool lef = f.get(l) < tr[p].f->get(l);
        bool mid = f.get(m) < tr[p].f->get(m);
        if (mid)
        {
            swap(f, *tr[p].f);
        }
        if (l + 1 == r)
        {
            return;
        }
        int d = lef != mid ? 0 : 1;
        if (tr[p].ch[d] == -1)
```

```
        {
            tr[p].ch[d] = newNode();
        }
        if (d == 0)
        {
            add(tr[p].ch[d], l, m, f);
        }
        else
        {
            add(tr[p].ch[d], m, r, f);
        }
    }

public:
    DynLiChao(T lo, T hi) : lo(lo), hi(hi)
    {
        // lo、hi 定义非空整数半开区间；构造最小值动态李超树。
        // 最大值版优先采用直线和答案整体取反，动态开点方向无需变化。
        static_assert(is_integral_v<T> && is_signed_v<T>);
        assert(lo < hi); // 调试检查，可删
    }

    void add(Line f)
    {
        // f 是待加入直线；使其参与整个整数域的最小值查询。
        // 最大值版可插入 {-f.k,-f.b}，查询非空结果再取负。
        add(0, lo, hi, f);
    }

    optional<C> query(T x) const
    {
        // x 是 [lo,hi) 内整数；返回最小函数值，空函数集返回空。
        // 若直接维护最大值，插线与这里累计答案的全部优劣比较必须同步反向。
        assert(lo <= x && x < hi); // 调试检查，可删
        optional<C> ans;
        int p = 0;
        T l = lo, r = hi;
        while (p != -1)
        {
            if (tr[p].f)
            {
                C y = tr[p].f->get(x);
                if (!ans || y < *ans)
                {
                    ans = y;
                }
            }
            if (l + 1 == r)
            {
                break;
            }
            T m = midpoint(l, r);
```

```
            if (x < m)
            {
                p = tr[p].ch[0];
                r = m;
            }
            else
            {
                p = tr[p].ch[1];
                l = m;
            }
        }
        return ans;
    }
};
```

用 `DynLiChao<T> lc(lo, hi)` 维护有符号整数域 $[lo, hi)$；`add({k,b})` 插入直线，`query(x)` 返回 `optional<C>`，空函数集为 `nullopt`。求最大值时插入 `{-k,-b}` 并对非空结果取反，空集仍为 `nullopt`。`T` 是系数与横坐标类型，`C` 是 $kx + b$ 的计算、比较和返回类型，默认与 `T` 相同；乘积有溢出风险时使用 `DynLiChao<long long, __int128>`。`midpoint` 使定义域横跨有符号数两端时也不会因求中点溢出。插入与查询均 $O(\log U)$，U 是定义域内整数个数；空间 $O(q \log U)$。查询坐标预先已知时用 3.7.001 李超线段树 常数更小，需要历史版本时用 3.7.003 可持久化李超树。

3.7.003 可持久化李超树

给动态开点李超树加上可持久化：每次插线只复制被访问路径上的节点、返回新根，未访问的子树与旧版本共享，于是可以在任意历史版本上查询、也可从任意旧版本分叉出新版本。常用于按时间轴维护函数集合、或树上把父版本传给子节点：

```
template <class T, class C = T> struct PerLiChao
{
    // 当前实现维护最小值。改最大值可反向所有优劣比较并把空值改为负无穷，
    // 或在 k、b 与函数值都能安全取反时对插入直线同时取反、再对 query 结果取反；后者不改变持久化不变量。
    struct Line
    {
        // k、b 是一次函数 y=kx+b 的斜率与截距。
        T k{}, b{};
        C get(T x) const
        {
            // x 是查询横坐标；用比较类型 C 计算函数值。
            return (C)k * x + b;
        }
    };
    struct Node
    {
        // f 是当前区间中点占优的直线，l、r 是左右儿子编号。
        optional<Line> f;
        int l = -1, r = -1;
    };

```

```
    // lo、hi 是整数定义域，t 是所有版本共享的节点池。
    T lo, hi;
    vector<Node> t;

    PerLiChao(T lo, T hi) : lo(lo), hi(hi)
    {
        // lo、hi 定义非空整数半开区间；初始空版本根为 -1。
        static_assert(is_integral_v<T> && is_signed_v<T>);
        assert(lo < hi); // 调试检查，可删
    }

    int clone(int p)
    {
        // p 是旧节点编号；复制它，-1 则新建空节点，返回新编号。
        t.push_back(p < 0 ? Node{} : t[p]);
        return t.size() - 1;
    }

    int add(int root, Line f)
    {
        // root 是旧版本根，f 是新直线；返回插入后的版本根。
        return add(root, lo, hi, f);
    }

    int add(int p, T l, T r, Line f)
    {
        // p 是旧节点，[l,r) 是当前整数域，f 是新直线；返回新节点。
        p = clone(p);
        if (!t[p].f)
        {
            t[p].f = f;
            return p;
        }
        T m = midpoint(l, r);
        bool x = f.get(l) < t[p].f->get(l);
        bool y = f.get(m) < t[p].f->get(m);
        if (y)
        {
            swap(f, *t[p].f);
        }
        if (l + 1 == r)
        {
            return p;
        }
        if (x != y)
        {
            t[p].l = add(t[p].l, l, m, f);
        }
        else
        {
            t[p].r = add(t[p].r, m, r, f);
        }
```

```
        return p;
    }

    optional<C> query(int p, T x) const
    {
        // p 是版本根，x 是查询横坐标；返回最小值，空版本返回空。
        // 最大值版按直线取反时，对非空返回值取负，空版本仍返回 nullopt。
        assert(lo <= x && x < hi); // 调试检查，可删
        return qval(p, lo, hi, x);
    }

    optional<C> qval(int p, T l, T r, T x) const
    {
        // p 是当前节点，[l,r) 是其整数域，x 是横坐标；返回路径最小值。
        // 若改为直接维护最大值，必须同步反向这里的比较与空值语义。
        if (p < 0)
        {
            return nullopt;
        }
        optional<C> ans;
        if (t[p].f)
        {
            ans = t[p].f->get(x);
        }
        if (l + 1 == r)
        {
            return ans;
        }
        T m = midpoint(l, r);
        auto z = x < m ? qval(t[p].l, l, m, x) : qval(t[p].r, m, r, x);
        if (!ans || (z && *z < *ans))
        {
            ans = z;
        }
        return ans;
    }
};
```

用 `PerLiChao<T> lc(lo, hi)` 建对象，`T` 是有符号整数，版本根 `-1` 表示空函数集。`add(root,{k,b})` 基于旧版本插线并返回新根，`query(root,x)` 返回 `optional<C>`，不把空版本与某条极大值直线混用。最大值版仍可对每个版本插入的 `k,b` 取反并对非空查询结果取反，不改变根和节点共享关系。`C` 是 $kx+b$ 的计算、比较和返回类型，默认与 `T` 相同；乘积有溢出风险时使用 `PerLiChao<long long, __int128>`。中点使用 `midpoint`。例如值域 $[-5,6)$，令 `r1=add(-1,{2,3})`、`r2=add(r1,{-1,1})`，再从 `r1` 分叉 `r3=add(r1,{0,-3})`，则 `*query(r1,0)==3`、`*query(r2,0)==1`、`*query(r3,0)==-3`，而 `query(-1,0)==nullopt`。每次插线与查询均为 $O(\log U)$，每次插线新增 $O(\log U)$ 个节点；不需要历史版本时用 3.7.002 动态开点李超树。

3.7.004 K-D Tree

K-D Tree 递归地按当前跨度较大的坐标轴用 `nth_element` 把点集对半劈开，每个节点存自己子树的包围盒（x, y 的上下界）与权值和。矩形求和时用包围盒做整块命中/整块剪除、只在跨界时下递归；最近点查询先用「查询点到子树包围盒的最小距离平方」`boxDist` 估下界，先递归更近的一侧、当另一侧下界已不小于当前最优就剪掉。距离平方类型由 `Dist` 指定：

```
template <class Dist = long long> class KDTree
{
    // a[i][0..2] 依次是第 i 个点的横坐标、纵坐标和可加权值。
    vector<array<int, 3>> a;
    // t[u][0..9] 依次是横坐标、纵坐标、点权、左儿子、右儿子、横坐标下界、横坐标上界、纵坐标下界、纵坐标
    ↪ 上界、子树权值和。
    vector<array<int, 10>> t;
    // rt 是 K-D Tree 根编号，-1 表示空树。
    int rt = -1;

    void pull(int u, int v)
    {
        // u 是父节点，v 是儿子编号；把非空儿子的包围盒与权值和合入 u。
        if (v < 0)
        {
            return;
        }
        t[u][5] = min(t[u][5], t[v][5]);
        t[u][6] = max(t[u][6], t[v][6]);
        t[u][7] = min(t[u][7], t[v][7]);
        t[u][8] = max(t[u][8], t[v][8]);
        t[u][9] += t[v][9];
    }

    int build(int l, int r)
    {
        // l、r 是 a 中待建树的半开区间；返回该批点的 K-D Tree 根编号。
        if (l == r)
        {
            return -1;
        }
        int xl = a[l][0], xr = xl, yl = a[l][1], yr = yl;
        for (int i = l + 1; i < r; i++)
        {
            xl = min(xl, a[i][0]);
            xr = max(xr, a[i][0]);
            yl = min(yl, a[i][1]);
            yr = max(yr, a[i][1]);
        }
        int d = (Dist)xr - (Dist)xl < (Dist)yr - (Dist)yl;
        int m = midpoint(l, r);
        nth_element(a.begin() + l,
                    a.begin() + m,
                    a.begin() + r,
```

```
                    [&](const auto &x, const auto &y)
                    {
                        // x、y 是两个三维点记录；按当前跨度更大的坐标排序。
                        return x[d] < y[d];
                    });
        int u = (int)t.size();
        t.push_back({a[m][0], a[m][1], a[m][2], -1, -1, a[m][0], a[m][0], a[m][1], a[m][1],
↪   a[m][2]});
        t[u][3] = build(l, m);
        t[u][4] = build(m + 1, r);
        pull(u, t[u][3]);
        pull(u, t[u][4]);
        return u;
    }

    int rectSum(int u, int xl, int xr, int yl, int yr) const
    {
        // u 是当前节点，四个边界给出闭矩形；返回矩形内点权和。
        if (u < 0 || t[u][6] < xl || xr < t[u][5] || t[u][8] < yl || yr < t[u][7])
        {
            return 0;
        }
        if (xl <= t[u][5] && t[u][6] <= xr && yl <= t[u][7] && t[u][8] <= yr)
        {
            return t[u][9];
        }
        int ans = 0;
        if (xl <= t[u][0] && t[u][0] <= xr && yl <= t[u][1] && t[u][1] <= yr)
        {
            ans += t[u][2];
        }
        return ans + rectSum(t[u][3], xl, xr, yl, yr) + rectSum(t[u][4], xl, xr, yl, yr);
    }

    Dist boxDist(int u, int x, int y) const
    {
        // u 是非空子树根，x、y 是查询点；返回查询点到该子树包围盒的最小距离平方。
        Dist dx = 0, dy = 0;
        if (x < t[u][5])
        {
            dx = (Dist)t[u][5] - (Dist)x;
        }
        else if (t[u][6] < x)
        {
            dx = (Dist)x - (Dist)t[u][6];
        }
        if (y < t[u][7])
        {
            dy = (Dist)t[u][7] - (Dist)y;
        }
        else if (t[u][8] < y)
        {
```

```
            dy = (Dist)y - (Dist)t[u][8];
        }
        return dx * dx + dy * dy;
    }

    void nearest(int u, int x, int y, Dist &ans) const
    {
        // u 是当前子树根，x、y 是查询点，ans 是当前最优距离平方。
        if (u < 0 || boxDist(u, x, y) >= ans)
        {
            return;
        }
        Dist dx = (Dist)t[u][0] - (Dist)x;
        Dist dy = (Dist)t[u][1] - (Dist)y;
        ans = min(ans, dx * dx + dy * dy);
        int p = t[u][3], q = t[u][4];
        if (p >= 0 && q >= 0 && boxDist(p, x, y) > boxDist(q, x, y))
        {
            swap(p, q);
        }
        if (p >= 0)
        {
            nearest(p, x, y, ans);
        }
        if (q >= 0)
        {
            nearest(q, x, y, ans);
        }
    }

  public:
    KDTree(vector<array<int, 3>> a) : a(move(a))
    {
        // a 的三维依次是横坐标、纵坐标和点权；一次性建立静态 K-D Tree。
        t.reserve(this->a.size());
        rt = build(0, (int)this->a.size());
    }

    int rectSum(int xl, int xr, int yl, int yr) const
    {
        // xl、xr、yl、yr 给出闭矩形且左右、上下边界有序；返回矩形内全部点权和。
        assert(xl <= xr && yl <= yr); // 调试检查，可删
        return rectSum(rt, xl, xr, yl, yr);
    }

    optional<Dist> nearest(int x, int y) const
    {
        // x、y 是查询点坐标；返回到输入点集的最小欧氏距离平方，空点集返回空。
        if (rt < 0)
        {
            return nullopt;
        }
```

```
        Dist ans = numeric_limits<Dist>::max();
        nearest(rt, x, y, ans);
        return ans;
    }
};
```

把每个点写成 `{x,y,w}` 装进 `vector<array<int,3>>` 后构造 `KDTree tr(a)`；`rectSum(xl,xr,yl,yr)` 返回闭矩形内点权和，`nearest(x,y)` 返回到点集的最小欧氏距离平方，空点集返回 `nullopt`。默认 `Dist=long long`，调用者须保证坐标差、坐标平方和能由 `Dist` 表示；需要宽距离时构造 `KDTree<__int128> tr(a)`。`Dist` 只控制距离计算，点权与子树权值和仍由 `int` 表示，调用者必须另行保证它们不溢出；扩大 `Dist` 不会扩大点权累计范围，反之亦然。点集构造后固定。例如点 `{(0,0,2),(3,0,5),(1,2,7)}` 在 $[0,1]\times[0,2]$ 内权和为 9，查询点 $(2,0)$ 的最近距离平方为 1。建树 $O(n\log n)$，两类查询确定性最坏 $O(n)$；随机均匀点集上矩形聚合通常接近 $O(\sqrt{n})$，但不是确定性保证。在线加点用 3.7.005 K-D Tree 二进制分组。

下面检查空集、闭矩形边界、最近点和宽距离类型：

```
signed main()
{
    KDTree<> empty(vector<array<int, 3>>{});
    assert(!empty.nearest(0, 0));
    assert(empty.rectSum(-1, 1, -1, 1) == 0);

    KDTree<> tr(vector<array<int, 3>>{{0, 0, 2}, {3, 0, 5}, {1, 2, 7}});
    assert(tr.rectSum(0, 1, 0, 2) == 9);
    assert(tr.rectSum(3, 3, 0, 0) == 5);
    assert(tr.nearest(2, 0) == optional<long long>(1));

    KDTree<__int128> wide(vector<array<int, 3>>{{3000000000LL, 3000000000LL, 1}});
    __int128 d = 6000000000LL;
    assert(wide.nearest(-3000000000LL, -3000000000LL) == optional<__int128>(2 * d * d));
}
```

3.7.005 K-D Tree 二进制分组

K-D Tree 难以在线增量维护平衡,二进制分组用「重建代替旋转」绕开:把已插入的点按块大小 $2^0, 2^1, 2^2, \dots$ 组织成若干静态 K-D Tree，插入一个点时若第 k 层已占用就把它与新点合并、进位到第 $k+1$ 层重建（类似二进制加一）。查询时遍历所有非空块累加。因为每个点被重建的次数是它经过的层数（对数级），整体插入摊还可控：

```
template <class T = long long> struct KDBlocks
{
    struct KDPoint
    {
        // x、y 是点坐标，w 是可加权值。
        int x, y;
        T w;
    };

    struct Block
```

```
    {
        struct Node
        {
            // p 是本点，l、r 是儿子，四个边界与 sum 描述整棵子树。
            KDPoint p;
            int l = -1, r = -1;
            int xl, xr, yl, yr;
            T sum{};
        };

        // a 是本块的全部点，t 是静态 K-D Tree 节点池，root 是根。
        vector<KDPoint> a;
        vector<Node> t;
        int root = -1;

        Block(vector<KDPoint> a = {}) : a(move(a))
        {
            // a 是本块点集；一次性建立静态 K-D Tree。
            t.reserve(this->a.size());
            root = build(0, this->a.size(), 0);
        }

        int build(int l, int r, int d)
        {
            // [l,r) 是待建点段，d=0/1 表示按横/纵坐标切分；返回根编号。
            if (l == r)
            {
                return -1;
            }
            int m = midpoint(l, r);
            nth_element(a.begin() + l,
                        a.begin() + m,
                        a.begin() + r,
                        [&](const KDPoint &p, const KDPoint &q)
                        {
                            return d ? p.y < q.y : p.x < q.x;
                        });
            int u = t.size();
            t.push_back({a[m], -1, -1, a[m].x, a[m].x, a[m].y, a[m].y, a[m].w});
            t[u].l = build(l, m, d ^ 1);
            t[u].r = build(m + 1, r, d ^ 1);
            pull(u, t[u].l);
            pull(u, t[u].r);
            return u;
        }

        void pull(int u, int v)
        {
            // u 是父节点，v 是儿子；把非空儿子的包围盒与权值和合入 u。
            if (v < 0)
            {
                return;
```

```
            }
            t[u].xl = min(t[u].xl, t[v].xl);
            t[u].xr = max(t[u].xr, t[v].xr);
            t[u].yl = min(t[u].yl, t[v].yl);
            t[u].yr = max(t[u].yr, t[v].yr);
            t[u].sum += t[v].sum;
        }

        T query(int u, int xl, int xr, int yl, int yr) const
        {
            // u 是当前节点；返回闭矩形 [xl,xr] x [yl,yr] 内的点权和。
            if (u < 0 || t[u].xr < xl || xr < t[u].xl || t[u].yr < yl || yr < t[u].yl)
            {
                return {};
            }
            if (xl <= t[u].xl && t[u].xr <= xr && yl <= t[u].yl && t[u].yr <= yr)
            {
                return t[u].sum;
            }
            T ans{};
            if (xl <= t[u].p.x && t[u].p.x <= xr && yl <= t[u].p.y && t[u].p.y <= yr)
            {
                ans += t[u].p.w;
            }
            return ans + query(t[u].l, xl, xr, yl, yr) + query(t[u].r, xl, xr, yl, yr);
        }

        T query(int xl, int xr, int yl, int yr) const
        {
            // 四个参数是有序的闭矩形边界；返回本块内点权和。
            assert(xl <= xr && yl <= yr); // 调试检查，可删
            return query(root, xl, xr, yl, yr);
        }
    };

    // b[k] 保存大小为 2^k 的静态块，n 是当前点数。
    vector<optional<Block>> b;
    int n = 0;

    void add(int x, int y, T w)
    {
        // x、y 是新点坐标，w 是权值；插入后按二进制进位重建块。
        vector<KDPoint> a{{x, y, w}};
        int k = 0;
        while (k < (int)b.size() && b[k])
        {
            a.insert(a.end(), b[k]->a.begin(), b[k]->a.end());
            b[k].reset();
            k++;
        }
        if (k == (int)b.size())
        {
```

```
            b.emplace_back();
        }
        b[k].emplace(move(a));
        n++;
    }

    T query(int xl, int xr, int yl, int yr) const
    {
        // 四个参数是有序的闭矩形边界；返回所有块内的点权和。
        assert(xl <= xr && yl <= yr); // 调试检查，可删
        T ans{};
        for (auto &x : b)
        {
            if (x)
            {
                ans += x->query(xl, xr, yl, yr);
            }
        }
        return ans;
    }
};
```

默认 `KDBlocks<> kd`（权值类型 `long long`），`add(x,y,w)` 在线插入带权点、`query(xl,xr,yl,yr)` 求闭矩形 $[xl, xr] \times [yl, yr]$ 内权值和。只支持插入与矩形和，不支持删除或最近点（分块后最近点无法跨块统一剪枝）。插入 n 点总建树 $O(n \log^2 n)$、空间 $O(n)$，单次矩形查询遍历 $O(\log n)$ 个块、确定性最坏 $O(n)$；随机均匀点集通常快得多，但没有统一的次线性最坏界。点集静态时直接用常数更小的 3.7.004 K-D Tree；只需一维带修区间和用 3.2.001 树状数组；带删除的二维数点常改用 CDQ 分治或树套树离线处理。

3.7.006 分散层叠

分散层叠（分数级联）用于在 k 张静态有序表上反复把同一值 x 送进每张表求 `lower_bound`：预处理后只在第一张增强表二分，随后沿预存的相邻层「桥」指针每层常数转移，把单次多表查询从 $O(\sum_i \log n_i)$ 降到 $O(\log N + k)$（$N = \sum_i n_i$）。这份底层与其接口 `lowerBound` 已在基础算法章完整给出且带自测，本节不重复粘贴，直接见 2.2.006 分数级联（Fractional Cascading）——构造 `FracCascade<T> fc(tabs)`（各表升序）、`fc.lowerBound(x)` 返回各表下界位置数组，构造 $O(N \log N)$、单次查询 $O(\log N + k)$。它服务于线段树/树链分解后同时查多份有序目录，或几何中沿相邻层定位同一坐标；只有一两次查询时预处理不划算，有修改时改用平衡树或树套有序结构。

3.7.007 双端队列全局查询

用两个栈模拟双端队列，即可在两端插入删除的同时维护整段的结合律聚合（甚至非交换的，如仿射函数复合）。左栈 `l` 从队首往里、右栈 `r` 从队尾往里，每个栈元素额外存「从该栈栈底到本元素」的前缀聚合，因此两栈栈顶的聚合按队列方向 `op(l 顶, r 顶)` 合并就是全局值。当某一侧栈空又要从该侧弹出或取端点时，把当前所有元素按队列顺序取出、对半 `rebuild` 回两栈，保证均摊 $O(1)$：

```cpp
template <class T, class F> struct AggDeque
{
    // op 是结合律运算，e 是幺元。
    F op;
    T e;
    // l、r 是队首侧与队尾侧栈；每项为 {原值，按队列方向的本栈聚合}。
    vector<pair<T, T>> l, r;

    AggDeque(F combine, T identity) : op(combine), e(identity)
    {
        // combine 是结合律二元运算，identity 是其幺元；初始队列为空。
    }

    int size() const
    {
        // 返回队列元素数。
        return l.size() + r.size();
    }
    bool empty() const
    {
        // 返回队列是否为空。
        return l.empty() && r.empty();
    }
    void pushFront(T x)
    {
        // x 是新元素；插入队首并更新左栈聚合。
        l.push_back({x, l.empty() ? x : op(x, l.back().second)});
    }
    void pushBack(T x)
    {
        // x 是新元素；插入队尾并更新右栈聚合。
        r.push_back({x, r.empty() ? x : op(r.back().second, x)});
    }
    void rebuild(int k)
    {
        // k 是重建后放入队首侧的元素数；保持整个队列顺序不变。
        assert(0 <= k && k <= size()); // 调试检查，可删
        vector<T> a;
        for (auto i = l.rbegin(); i != l.rend(); i++)
        {
            a.push_back(i->first);
        }
        for (auto x : r)
        {
            a.push_back(x.first);
        }
        l.clear();
        r.clear();
        for (int i = k - 1; i >= 0; i--)
        {
            pushFront(a[i]);
        }
```

```
        for (int i = k; i < (int)a.size(); i++)
        {
            pushBack(a[i]);
        }
    }
    void popFront()
    {
        // 删除队首元素。
        assert(!empty()); // 调试检查，可删
        if (l.empty())
        {
            rebuild((size() + 1) / 2);
        }
        l.pop_back();
    }
    void popBack()
    {
        // 删除队尾元素。
        assert(!empty()); // 调试检查，可删
        if (r.empty())
        {
            rebuild(size() / 2);
        }
        r.pop_back();
    }
    T front() const
    {
        // 返回队首元素；只读端点，不为读取重建两侧栈。
        assert(!empty()); // 调试检查，可删
        return l.empty() ? r.front().first : l.back().first;
    }
    T back() const
    {
        // 返回队尾元素；只读端点，不为读取重建两侧栈。
        assert(!empty()); // 调试检查，可删
        return r.empty() ? l.front().first : r.back().first;
    }
    T query() const
    {
        // 按队首到队尾顺序聚合全部元素；空队列返回 e。
        if (l.empty())
        {
            return r.empty() ? e : r.back().second;
        }
        if (r.empty())
        {
            return l.back().second;
        }
        return op(l.back().second, r.back().second);
    }
};
```

构造 `AggDeque<T,F> dq(op, e)`，例如维护队列和：

```
signed main()
{
    auto op = [](int x, int y)
    {
        return x + y;
    };
    AggDeque<int, decltype(op)> dq(op, 0);
    dq.pushBack(2);
    dq.pushFront(3);
    assert(dq.query() == 5);
    dq.popFront();
    assert(dq.front() == 2);
}
```

`op` 是结合律二元运算、`e` 是其幺元（可非交换，注意 `op(l,r)` 的左右顺序对应队列方向）；`pushFront/pushBack/popFront/popBack` 两端增删，`front/back` 只读对应一侧栈顶或另一侧栈底，`query()` 返回按队首到队尾顺序 `op` 折叠的全局聚合（空队列返回 `e`）。只有删除时所在侧为空才会 `rebuild`；读取端点绝不重建，因而所有公开操作均摊 $O(1)$，单次 `rebuild` 最坏 $O(n)$，空间 $O(n)$。若只在一端进出用普通栈即可；若聚合可逆（如加法、异或）用双指针前缀更简单；需要区间（而非全局）查询用线段树。它区别于滑动窗口最值 12.3.001 单调队列优化 之处在于支持任意结合律与非交换运算。

3.7.008 析合树（$O(n \log n)$ 构造）

排列区间 $[l, r)$ 是连续段当且仅当其中的值构成连续整数，即 $\max - \min = r - l - 1$；不与其他连续段交叉的连续段称为强区间，全部强区间按包含关系构成析合树。增量扫描右端点 i 时，线段树在位置 j 维护 $q_j = \max_{j \le k \le i} a_k - \min_{j \le k \le i} a_k - (i - j)$，两个单调栈把新元素造成的最大值、最小值变化拆成区间加；最左的 $q_j = 0$ 给出本轮可能合并到的最远左端点，再在线性维护的析合森林中建立叶、析点或合点。线段树的差分与懒标记使用 `__int128`，避免输入从 `LLONG_MIN` 开始时对端点取负溢出。

```
struct PermTree
{
    struct Node
    {
        int l = 0;        // l 保存强区间左端点。
        int r = 0;        // r 保存强区间右端点，区间为 [l,r)。
        int type = 0;     // type 为 0、1、2 时分别表示叶、析点、合点。
        int par = -1;     // par 保存父结点，根的父亲为 -1。
        int mid = -1;     // 合点中 mid 保存最右孩子的左端点。
        vector<int> ch;   // ch 按原排列顺序保存直接孩子。
    };

    struct RMQ
    {
        vector<int> lg;         // lg[len] 保存 floor(log2(len))。
        vector<vector<int>> mn; // mn[k][i] 保存长 2^k 区间最小值。
        vector<vector<int>> mx; // mx[k][i] 保存长 2^k 区间最大值。

```

```
        RMQ(const vector<int> &a)
        {
            // a 是排列；建立区间最大值和最小值稀疏表。
            int n = a.size();
            lg.assign(n + 1, 0);
            for (int i = 2; i <= n; i++)
            {
                lg[i] = lg[i / 2] + 1;
            }
            int z = n ? lg[n] + 1 : 0;
            mn.assign(z, vector<int>(n));
            mx.assign(z, vector<int>(n));
            if (n)
            {
                mn[0] = mx[0] = a;
            }
            for (int k = 1; k < z; k++)
            {
                for (int i = 0; i + (1LL << k) <= n; i++)
                {
                    mn[k][i] = min(mn[k - 1][i], mn[k - 1][i + (1LL << (k - 1))]);
                    mx[k][i] = max(mx[k - 1][i], mx[k - 1][i + (1LL << (k - 1))]);
                }
            }
        }

        pair<int, int> query(int l, int r) const
        {
            // l、r 是非空半开区间端点；返回该区间的 {最小值, 最大值}。
            int k = lg[r - l];
            int d = r - (1LL << k);
            return {min(mn[k][l], mn[k][d]), max(mx[k][l], mx[k][d])};
        }
    };

    struct Seg
    {
        int n;                // n 保存叶子个数。
        vector<__int128> mn, tag; // mn、tag 保存区间最小值与区间加懒标记。

        Seg(int n) : n(n), mn(4 * n + 4), tag(4 * n + 4)
        {
            // n 是排列长度；初始 q 全为 0。
        }

        void apply(int u, __int128 v)
        {
            // u 是线段树结点，v 是增量；给整段加 v。
            mn[u] += v;
            tag[u] += v;
        }

```

```
71          void push(int u)
72          {
73              // u 是线段树结点；把区间加标记下传给两个儿子。
74              if (tag[u])
75              {
76                  apply(u * 2, tag[u]);
77                  apply(u * 2 + 1, tag[u]);
78                  tag[u] = 0;
79              }
80          }
81
82          void add(int u, int l, int r, int ql, int qr, __int128 v)
83          {
84              // u 管理 [l,r)，给 [ql,qr) 加 v。
85              if (ql <= l && r <= qr)
86              {
87                  apply(u, v);
88                  return;
89              }
90              push(u);
91              int m = (l + r) / 2;
92              if (ql < m)
93              {
94                  add(u * 2, l, m, ql, qr, v);
95              }
96              if (m < qr)
97              {
98                  add(u * 2 + 1, m, r, ql, qr, v);
99              }
100             mn[u] = min(mn[u * 2], mn[u * 2 + 1]);
101         }
102
103         void add(int l, int r, __int128 v)
104         {
105             // l、r 是半开区间端点；给 q[l,r) 加 v。
106             if (l < r)
107             {
108                 add(1, 0, n, l, r, v);
109             }
110         }
111
112         int firstZero(int u, int l, int r)
113         {
114             // u 管理 [l,r)，且段内最小值为 0；返回最左的零位置。
115             if (r - l == 1)
116             {
117                 return l;
118             }
119             push(u);
120             int m = (l + r) / 2;
121             if (mn[u * 2] == 0)
122             {
```

```
                return firstZero(u * 2, l, m);
            }
            return firstZero(u * 2 + 1, m, r);
        }

        int firstZero()
        {
            // 无参数；返回 q 中最左的零位置。
            assert(mn[1] == 0); // 调试检查，可删
            return firstZero(1, 0, n);
        }
    };

    vector<Node> t; // t 保存全部析合树结点。
    int root = -1;  // root 保存整段排列对应的根结点。

    PermTree(const vector<int> &a)
    {
        // a 是一段连续整数的排列；用增量析合森林构造析合树。
        int n = a.size();
        if (!n)
        {
            return;
        }
        vector<int> chk = a;
        sort(chk.begin(), chk.end());
        assert(unique(chk.begin(), chk.end()) == chk.end()); // 调试检查，可删
        assert((__int128)chk.back() - (__int128)chk[0] == n - 1); // 调试检查，可删
        RMQ rmq(a);
        Seg seg(n);
        vector<int> low, high, forest;

        auto common = [&](int l, int r)
        {
            auto [mn, mx] = rmq.query(l, r);
            return (__int128)mx - (__int128)mn == r - l - 1;
        };
        auto node = [&](int l, int r, int type)
        {
            t.push_back({l, r, type, -1, -1, {}});
            return (int)t.size() - 1;
        };
        auto link = [&](int u, int v)
        {
            t[u].ch.push_back(v);
            t[v].par = u;
        };

        for (int i = 0; i < n; i++)
        {
            int lp = low.empty() ? 0 : low.back() + 1;
            while (!low.empty() && a[i] <= a[low.back()])
```

```
            {
                int p = low.back();
                low.pop_back();
                int l = low.empty() ? 0 : low.back() + 1;
                seg.add(l, p + 1, (__int128)a[p]);
                lp = l;
            }
            seg.add(lp, i + 1, -(__int128)a[i]);
            low.push_back(i);

            lp = high.empty() ? 0 : high.back() + 1;
            while (!high.empty() && a[i] >= a[high.back()])
            {
                int p = high.back();
                high.pop_back();
                int l = high.empty() ? 0 : high.back() + 1;
                seg.add(l, p + 1, -(__int128)a[p]);
                lp = l;
            }
            seg.add(lp, i + 1, (__int128)a[i]);
            high.push_back(i);

            int now = node(i, i + 1, 0);
            int le = seg.firstZero();
            while (!forest.empty() && t[forest.back()].l >= le)
            {
                int x = forest.back();
                if (t[x].type == 2 && common(t[x].mid, i + 1))
                {
                    forest.pop_back();
                    t[x].r = i + 1;
                    t[x].mid = t[now].l;
                    link(x, now);
                    now = x;
                }
                else if (common(t[x].l, i + 1))
                {
                    forest.pop_back();
                    int u = node(t[x].l, i + 1, 2);
                    t[u].mid = t[now].l;
                    link(u, x);
                    link(u, now);
                    now = u;
                }
                else
                {
                    int u = node(0, i + 1, 1);
                    link(u, now);
                    while (true)
                    {
                        x = forest.back();
                        forest.pop_back();
```

```
                        link(u, x);
                        if (common(t[x].l, i + 1))
                        {
                            break;
                        }
                    }
                    t[u].l = t[x].l;
                    now = u;
                }
            }
            forest.push_back(now);
            seg.add(0, i + 1, -1);
        }
        assert(forest.size() == 1); // 调试检查，可删
        root = forest[0];
        for (auto &x : t)
        {
            sort(x.ch.begin(), x.ch.end(), [&](int u, int v)
            {
                return t[u].l < t[v].l;
            });
        }
    }
};
```

构造 `PermTree tr(a)` 后，`tr.t[u]` 给出强区间 $[l,r)$、类型、父亲和按位置排列的孩子，`tr.root` 是整段排列；空排列得到 `root=-1` 和空结点表，非空输入的值不必从 0 开始，但必须互异并恰好构成连续整数（包括从 `LLONG_MIN` 开始的连续排列）。稀疏表预处理为 $O(n\log n)$ 时间和空间，每次查询 $O(1)$；线段树操作合计 $O(n\log n)$ 时间、占 $O(n)$ 空间，析合森林总共只压入、弹出线性次。因此整个构造为 $O(n\log n)$ 时间、$O(n\log n)$ 空间。合点的连续若干孩子仍可组成连续段，析点则只能取整个结点；需要统计全部连续段时按这个区别做树上 DP。

3.7.009 动态点集矩形仿射修改与矩形和

把初始点和以后可能插入的点坐标全部离线收集，用 `optional` 空值表示尚未激活的点。平衡 K-D Tree 每个子树维护包围盒、激活点数、权值和与仿射懒标记；矩形整体覆盖子树时，$w\leftarrow aw+b$ 使子树和变成 $a\cdot sum+b\cdot cnt$。未来点激活前会先下传祖先标记再覆盖叶子，因此不会错误继承它插入以前的修改。

```
template <class Coord = long long, class T = long long>
struct KDRectAffine
{
    struct Node
    {
        Coord xl{}, xr{}, yl{}, yr{}; // 子树内全部点的坐标包围盒（闭区间）。
        int cnt = 0; // cnt 是子树内已激活点数。
        T sum = 0, mul = 1, add = 0; // sum 是激活点权值和，mul、add 是未下传的 w->mul*w+add 标记。
    };

    int n; // n 是预收集的点数。
```

```
    vector<Coord> x, y; // x[id]、y[id] 是第 id 个点的坐标。
    vector<int> ord, pos; // ord 是建树时的点序，pos[id] 是点 id 所在叶子的节点编号。
    vector<Node> tr; // tr 是按堆式编号存放的 KD 树节点。

    KDRectAffine(vector<Coord> x, vector<Coord> y, const vector<optional<T>> &w)
        : n(x.size()), x(move(x)), y(move(y)), ord(n), pos(n), tr(max<int>(1, 4 * n))
    {
        // x、y 是全部已知点坐标，w[id] 为空表示该点尚未激活。
        assert((int)this->y.size() == n && (int)w.size() == n); // 调试检查，可删。
        iota(ord.begin(), ord.end(), 0);
        if (n)
        {
            build(1, 0, n, 0, w);
        }
    }

    void pull(int k)
    {
        // k 是内部节点；由两个儿子重算包围盒、激活点数与权值和。
        auto &a = tr[2 * k];
        auto &b = tr[2 * k + 1];
        tr[k].xl = min(a.xl, b.xl);
        tr[k].xr = max(a.xr, b.xr);
        tr[k].yl = min(a.yl, b.yl);
        tr[k].yr = max(a.yr, b.yr);
        tr[k].cnt = a.cnt + b.cnt;
        tr[k].sum = a.sum + b.sum;
    }

    void build(int k, int l, int r, int d, const vector<optional<T>> &w)
    {
        // k 管辖 ord[l..r)，d 是切分维度（0 按 x、1 按 y），w 是初始权值；递归建 KD 树。
        if (r - l == 1)
        {
            int id = ord[l];
            pos[id] = k;
            tr[k].xl = tr[k].xr = x[id];
            tr[k].yl = tr[k].yr = y[id];
            if (w[id])
            {
                tr[k].cnt = 1;
                tr[k].sum = *w[id];
            }
            return;
        }
        int m = (l + r) / 2;
        nth_element(ord.begin() + l, ord.begin() + m, ord.begin() + r, [&](int a, int b)
        {
            return d == 0 ? x[a] < x[b] : y[a] < y[b];
        });
        build(2 * k, l, m, d ^ 1, w);
        build(2 * k + 1, m, r, d ^ 1, w);
```

```
64          pull(k);
65      }
66
67      void applyAt(int k, T a, T b)
68      {
69          // k 是节点；对其子树内激活点执行 w=a*w+b，并复合到延迟标记。
70          tr[k].sum = a * tr[k].sum + b * T(tr[k].cnt);
71          tr[k].mul = a * tr[k].mul;
72          tr[k].add = a * tr[k].add + b;
73      }
74
75      void push(int k)
76      {
77          // k 是内部节点；把延迟标记下传给两个儿子。
78          if (tr[k].mul == T(1) && tr[k].add == T(0))
79          {
80              return;
81          }
82          applyAt(2 * k, tr[k].mul, tr[k].add);
83          applyAt(2 * k + 1, tr[k].mul, tr[k].add);
84          tr[k].mul = T(1);
85          tr[k].add = T(0);
86      }
87
88      bool disjoint(int k, Coord xl, Coord xr, Coord yl, Coord yr) const
89      {
90          // 返回节点 k 的包围盒是否与半开矩形 [xl,xr)x[yl,yr) 不相交。
91          return xr <= tr[k].xl || tr[k].xr < xl || yr <= tr[k].yl || tr[k].yr < yl;
92      }
93
94      bool covered(int k, Coord xl, Coord xr, Coord yl, Coord yr) const
95      {
96          // 返回节点 k 的包围盒是否完全落在半开矩形 [xl,xr)x[yl,yr) 内。
97          return xl <= tr[k].xl && tr[k].xr < xr && yl <= tr[k].yl && tr[k].yr < yr;
98      }
99
100     T query(int k, Coord xl, Coord xr, Coord yl, Coord yr)
101     {
102         // k 是当前节点；返回其子树内落在半开矩形中的激活点权值和。
103         if (disjoint(k, xl, xr, yl, yr))
104         {
105             return T(0);
106         }
107         if (covered(k, xl, xr, yl, yr))
108         {
109             return tr[k].sum;
110         }
111         push(k);
112         return query(2 * k, xl, xr, yl, yr) + query(2 * k + 1, xl, xr, yl, yr);
113     }
114
115     void apply(int k, Coord xl, Coord xr, Coord yl, Coord yr, T a, T b)
```

```
    {
        // k 是当前节点；对其子树内落在半开矩形中的激活点执行 w=a*w+b。
        if (disjoint(k, xl, xr, yl, yr))
        {
            return;
        }
        if (covered(k, xl, xr, yl, yr))
        {
            applyAt(k, a, b);
            return;
        }
        push(k);
        apply(2 * k, xl, xr, yl, yr, a, b);
        apply(2 * k + 1, xl, xr, yl, yr, a, b);
        pull(k);
    }

    void set(int id, T w)
    {
        // id 是预收集点编号，w 是当前新权值；激活或覆盖该点。
        assert(0 <= id && id < n); // 调试检查，可删。
        int k = pos[id];
        vector<int> path;
        for (int z = k / 2; z; z /= 2)
        {
            path.push_back(z);
        }
        reverse(path.begin(), path.end());
        for (int z : path)
        {
            push(z);
        }
        tr[k].cnt = 1;
        tr[k].sum = w;
        tr[k].mul = T(1);
        tr[k].add = T(0);
        for (k /= 2; k; k /= 2)
        {
            pull(k);
        }
    }

    T query(Coord xl, Coord xr, Coord yl, Coord yr)
    {
        // 四个边界给出半开矩形；返回全部激活点权值和。
        assert(xl <= xr && yl <= yr); // 调试检查，可删。
        return n ? query(1, xl, xr, yl, yr) : T(0);
    }

    void apply(Coord xl, Coord xr, Coord yl, Coord yr, T a, T b)
    {
        // 四个边界给出半开矩形；对其中激活点执行 w=a*w+b。
```

```
        assert(xl <= xr && yl <= yr); // 调试检查，可删。
        if (n)
        {
            apply(1, xl, xr, yl, yr, a, b);
        }
    }
};
```

构造时必须预收集全部点编号及坐标，初始未出现的点令 `w[id]=nullopt`；随后 `set(id,w)` 激活或重设单点，`apply(xl,xr,yl,yr,a,b)` 修改半开矩形，`query` 求和。重复坐标按不同编号独立保存并一起受矩形操作影响。`T` 必须容纳仿射乘加与子树和，模意义题直接使用模整数。建树期望 $O(n\log n)$、单点设置 $O(\log n)$、空间 $O(n)$；矩形操作依赖包围盒剪枝，确定性最坏 $O(n)$，普通分布下通常接近 $O(\sqrt{n})$，具体常数会随数据和平台变化。

下面用四个点覆盖重复坐标、延迟激活与半开边界：

```
signed main()
{
    vector<long long> x{0, 1, 1, 3}, y{0, 1, 1, 0};
    vector<optional<long long>> w{2, nullopt, 5, 4};
    KDRectAffine tr(x, y, w);
    assert(tr.query(0, 2, 0, 2) == 7);
    tr.apply(0, 2, 0, 2, 2, 1);
    assert(tr.query(0, 2, 0, 2) == 16);
    tr.set(1, 3);
    assert(tr.query(0, 2, 0, 2) == 19);
    tr.apply(1, 2, 1, 2, -1, 4);
    assert(tr.query(0, 2, 0, 2) == -1);
    assert(tr.query(3, 4, 0, 1) == 4);
}
```

3.7.010 离散坐标线段李超树（线段插入、点查询）

若直线只在自己的横坐标半开区间 $[l,r)$ 内有效，先离线收集所有点查询的横坐标并排序去重。把每条线段映射为这些坐标中的连续下标段 $[\text{lower_bound}(l),\text{lower_bound}(r))$；在线段树上递归时，完整覆盖的节点按普通李超规则放一条“在该节点中点最优”的直线。被换下的直线只会在一个儿子区间翻盘，故继续向那个儿子下传即可。所有节点共用同一棵离散线段树，每个节点至多存一条线，不为每个区间额外分配一棵李超树：

```
template <class T, class C = T> class SegLiChao
{
    // 当前实现维护最小值。改最大值可直接反向所有优劣比较并把空值改为负无穷，
    // 或在系数与函数值能安全取反时对每条线取反后复用本实现，查询非空结果再取反。
  public:
    static_assert(is_floating_point_v<T> || (is_integral_v<T> && is_signed_v<T>));

    struct Line
    {
        // k、b 是一次函数 y=kx+b 的斜率与截距。
```

```
        T k{}, b{};

        C get(T x) const
        {
            // x 是查询横坐标；用比较类型 C 计算函数值。
            return (C)k * x + b;
        }
    };

  private:
    // xs 是全部允许查询的横坐标，tr[p] 是节点 p 上在中点占优且整段有效的直线。
    vector<T> xs;
    vector<optional<Line>> tr;

    void addLine(int p, int l, int r, Line f)
    {
        // p 是节点编号，[l,r) 是横坐标下标段，f 已在整段内有效；把 f 加入该段。
        if (!tr[p])
        {
            tr[p] = f;
            return;
        }
        int m = midpoint(l, r);
        bool lef = f.get(xs[l]) < tr[p]->get(xs[l]);
        bool mid = f.get(xs[m]) < tr[p]->get(xs[m]);
        if (mid)
        {
            swap(f, *tr[p]);
        }
        if (r - l == 1)
        {
            return;
        }
        if (lef != mid)
        {
            addLine(p << 1, l, m, f);
        }
        else
        {
            addLine(p << 1 | 1, m, r, f);
        }
    }

    void addSegment(int p, int l, int r, int ql, int qr, Line f)
    {
        // p、[l,r) 是当前节点，[ql,qr) 是 f 有效的离散下标段；插入二者交集。
        if (qr <= l || r <= ql)
        {
            return;
        }
        if (ql <= l && r <= qr)
        {
```

```
            addLine(p, l, r, f);
            return;
        }
        int m = midpoint(l, r);
        addSegment(p << 1, l, m, ql, qr, f);
        addSegment(p << 1 | 1, m, r, ql, qr, f);
    }

public:
    SegLiChao(vector<T> xs) : xs(move(xs))
    {
        // xs 是全部可能查询的横坐标；排序去重并建立最小值线段李超树。
        // 最大值版可对插入线的 k、b 取反，并对 query 返回值取反。
        sort(this->xs.begin(), this->xs.end());
        this->xs.erase(unique(this->xs.begin(), this->xs.end()), this->xs.end());
        assert(!this->xs.empty()); // 调试检查，可删
        tr.resize(4 * this->xs.size());
    }

    void add(Line f, T l, T r)
    {
        // f 是待插入直线，[l,r) 是其按真实横坐标定义的有效半开区间。
        assert(l < r); // 调试检查，可删
        int ql = lower_bound(xs.begin(), xs.end(), l) - xs.begin();
        int qr = lower_bound(xs.begin(), xs.end(), r) - xs.begin();
        if (ql < qr)
        {
            addSegment(1, 0, xs.size(), ql, qr, f);
        }
    }

    optional<C> query(T x) const
    {
        // x 必须在构造坐标中；返回所有在 x 有效直线的最小值，无线时返回空。
        // 采用取反线的最大值版仍保留无线时的 nullopt 语义。
        int q = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
        assert(q < (int)xs.size() && xs[q] == x); // 调试检查，可删
        optional<C> ans;
        int p = 1, l = 0, r = xs.size();
        while (true)
        {
            if (tr[p])
            {
                C y = tr[p]->get(x);
                if (!ans || y < *ans)
                {
                    ans = y;
                }
            }
            if (r - l == 1)
            {
                break;
```

```
            }
            int m = midpoint(l, r);
            if (q < m)
            {
                p <<= 1;
                r = m;
            }
            else
            {
                p = p << 1 | 1;
                l = m;
            }
        }
        return ans;
    }
};
```

先收集至少一个可能被 query 的横坐标，以 `SegLiChao<T,C> lc(xs)` 构造；`lc.add({k,b},l,r)` 插入仅在真实区间 $[l,r)$ 有效的 $y=kx+b$，`lc.query(x)` 返回 `optional<C>`，没有有效线段时为 `nullopt`。最大值版对每条线段的 `k,b` 取反后按相同有效区间插入，并对非空查询结果取反；有效域和空值语义不变。端点不需要出现在 `xs` 中；若 $[l,r)$ 内没有任何查询坐标，本次插入自动忽略。T 是横坐标与系数类型，C 是函数值类型；积可能超过 T 时用 `SegLiChao<long long,__int128>`。对 $m=|xs|$，一次插入为 $O(\log^2 m)$，点查询为 $O(\log m)$，空间为 $O(m)$；前者来自区间分解的 $O(\log m)$ 个完整节点及每次李超下沉。若所有直线都在完整定义域有效，使用更短的 3.7.001 李超线段树；横坐标不能离线收集时改用动态开点结构并为线段有效域补充动态区间插入；需要历史版本时再扩展 3.7.003 可持久化李超树。

3.8 线性基

3.8.001 异或线性基

线性基把一组整数在异或（$\mathbb{F}_2$ 上的向量加法）意义下的张成空间压成至多 W 个主元向量：插入 x 时从最高位往低扫，遇到某位已有主元就异或消去，否则让 x 占据这个主元位并使秩加一；最终非零主元张成的空间与原集合相同。于是能判定某值可否表示（`has`）、求与初值异或的最大值（`maxXor`，逐位贪心取更大者）、求秩，以及在约化后按排名取第 k 小可表示值（`kth`）：

```
template <int W = 64> struct XorBasis
{
    static_assert(1 <= W && W <= 64);

    static bool valid(unsigned long long x)
    {
        // x 是待处理的整数；返回它是否只使用低 W 位。
        if constexpr (W == 64)
        {
            return true;
        }
        else
        {
```

```
            return x < (1ULL << W);
        }
    }

    // a[i] 是最高位为 i 的主元，rk 是当前秩。
    array<unsigned long long, W> a{};
    int rk = 0;

    bool add(unsigned long long x)
    {
        // x 是待插入向量；返回它是否使秩增加。
        assert(valid(x)); // 调试检查，可删。
        for (int i = W - 1; i >= 0; i--)
        {
            if (x >> i & 1)
            {
                if (!a[i])
                {
                    a[i] = x;
                    rk++;
                    return true;
                }
                x ^= a[i];
            }
        }
        return false;
    }

    bool has(unsigned long long x) const
    {
        // x 是查询向量；返回它能否由当前基表示。
        assert(valid(x)); // 调试检查，可删。
        for (int i = W - 1; i >= 0; i--)
        {
            if ((x >> i & 1) && a[i])
            {
                x ^= a[i];
            }
        }
        return x == 0;
    }

    unsigned long long maxXor(unsigned long long x = 0) const
    {
        // x 是初值；返回 x 与张成空间中某值异或后的最大结果。
        assert(valid(x)); // 调试检查，可删。
        for (int i = W - 1; i >= 0; i--)
        {
            chmax(x, x ^ a[i]);
        }
        return x;
    }
```

```cpp

    void merge(const XorBasis &o)
    {
        // o 是另一组基；把它的张成空间并入当前基。
        for (auto x : o.a)
        {
            if (x)
            {
                add(x);
            }
        }
    }

    optional<unsigned long long> kth(unsigned long long k) const
    {
        // k 是 0 基排名；返回第 k 小不同可表示值，越界返回空。
        if (rk < 64 && k >= (1ULL << rk))
        {
            return nullopt;
        }
        auto b = a;
        for (int i = 0; i < W; i++)
        {
            if (!b[i])
            {
                continue;
            }
            for (int j = i + 1; j < W; j++)
            {
                if (b[j] >> i & 1)
                {
                    b[j] ^= b[i];
                }
            }
        }
        vector<unsigned long long> v;
        for (auto x : b)
        {
            if (x)
            {
                v.push_back(x);
            }
        }
        unsigned long long ans = 0;
        for (int i = 0; i < (int)v.size(); i++)
        {
            if (k >> i & 1)
            {
                ans ^= v[i];
            }
        }
        return ans;
```

```
    }
};
```

默认 `XorBasis<> xb`（$W = 64$ 位）：`add(x)` 插入并返回是否使秩增加，`has(x)` 判可表示性，`maxXor(x)` 求与初值 x 异或的最大值，`rk` 是当前秩，`merge(o)` 并入另一组基，`kth(k)`（k 从 0 起，含 0）返回第 k 小不同可表示值、越界返回 `nullopt`。当 $W < 64$ 时，`add`、`has`、`maxXor` 以及并入的主元都必须在 $[0, 2^W)$ 内，代码以 `assert` 检查；$W = 64$ 时允许任意 `unsigned long long`。`add`、`has`、`maxXor` 均 $O(W)$，`kth`/`merge` 为 $O(W^2)$，空间 $O(W)$。只需第 k 大且从裸序列一次算出用 3.8.002 第 K 大线性基；带下标区间限制用 3.8.003 区间线性基；树上/分治合并两组基用 3.8.004 线性基合并。

下面固定检查线性相关向量、可表示性、排名、合并，以及秩为 64 时的边界：

```
signed main()
{
    XorBasis<4> a;
    assert(a.add(3));
    assert(a.add(5));
    assert(!a.add(6));
    assert(a.rk == 2 && a.has(6) && !a.has(1));
    assert(a.maxXor() == 6);
    array<unsigned long long, 4> want{0, 3, 5, 6};
    for (int k = 0; k < 4; k++)
    {
        assert(a.kth(k) == optional<unsigned long long>(want[k]));
    }
    assert(!a.kth(4));

    XorBasis<4> b;
    b.add(8);
    a.merge(b);
    assert(a.rk == 3 && a.maxXor() == 14);
    assert(a.kth(5) == optional<unsigned long long>(11));

    XorBasis<> full;
    for (int i = 0; i < 64; i++)
    {
        full.add(1ULL << i);
    }
    assert(full.rk == 64);
    assert(full.kth(ULLONG_MAX) == optional<unsigned long long>(ULLONG_MAX));
    cout << "2 6 0 3 5 6 3 14 11 18446744073709551615\n";
    return 0;
}
```

3.8.002 第 K 大线性基

要按大小有序枚举可表示的异或值，先把线性基化成约化阶梯形——让每个主元位只在一个主元中出现为 1。这样把非零主元从低位到高位排好后，它们两两「互不影响高位」，于是排名 k 的二进制各位恰好对应「选或不选」每个主元，异或起来就是第 k 小可表示值；求第 k 大只需把 k 换成 $2^{rk} - 1 - k$。本条把「插

入建基 + 约化 + 按排名取值」打包成一个独立函数：

```cpp
template <int W = 64>
optional<unsigned long long> kthXor(const vector<unsigned long long> &a, unsigned long long k,
    ↪ bool largest = false)
{
    // a 是生成空间的整数集合，k 从 0 开始，largest 表示按从大到小排名；返回第 k 个不同异或值，越界返回
    ↪ 空。
    static_assert(1 <= W && W <= 64);
    array<unsigned long long, W> b{};
    int rk = 0;
    for (auto x : a)
    {
        if constexpr (W < 64)
        {
            assert(x < (1ULL << W)); // 调试检查，可删。
        }
        for (int i = W - 1; i >= 0; i--)
        {
            if (!(x >> i & 1))
            {
                continue;
            }
            if (b[i])
            {
                x ^= b[i];
            }
            else
            {
                b[i] = x;
                rk++;
                break;
            }
        }
    }
    if (rk < 64 && k >= (1ULL << rk))
    {
        return nullopt;
    }
    for (int i = 0; i < W; i++)
    {
        if (!b[i])
        {
            continue;
        }
        for (int j = i + 1; j < W; j++)
        {
            if (b[j] >> i & 1)
            {
                b[j] ^= b[i];
            }
        }
```

```
    }
    if (largest)
    {
        k = (rk == 64 ? ULLONG_MAX : (1ULL << rk) - 1) - k;
    }
    unsigned long long ans = 0;
    int p = 0;
    for (int i = 0; i < W; i++)
    {
        if (!b[i])
        {
            continue;
        }
        if (k >> p & 1)
        {
            ans ^= b[i];
        }
        p++;
    }
    return ans;
}
```

`kthXor(a,k)` 从整数集合 `a` 求第 k 小不同异或值，`kthXor(a,k,true)` 求第 k 大；k 从 0 起，越界返回 `nullopt`，线性相关的输入不增加可表示值个数。注意 $k=0$ 对应最小值 0。当 $W<64$ 时，`a` 的每项必须在 $[0,2^W)$ 内，代码以 `assert` 检查；$W=64$ 时允许任意 `unsigned long long`。例如 `a={1,2,3}` 可表示 $\{0,1,2,3\}$：第 2 小（$k=2$）为 2，第 1 大（`largest`, $k=1$）为 2，$k=4$ 返回空。总时间 $O(nW+W^2)$、额外空间 $O(W)$。若还要反复插入/查询同一组基，改用维护状态的 3.8.001 异或线性基 的 `kth` 成员，避免每次重建。

3.8.003 区间线性基

要回答「下标区间 $[l,r]$ 内元素能异或出的最大值」，给每个主元额外记它当前来自哪个下标位置：对每个右端点保存一份前缀线性基，插入新元素时贪心让「出现位置更靠右」的向量占据主元（把旧主元换下继续消元）。查询时在版本 r 的基上只采纳「位置 $\geq l$」的主元做最大异或贪心，就等价于只用区间内元素建的基：

```
template <class T, int Bits = numeric_limits<T>::digits> class RangeBasis
{
    static_assert(is_integral_v<T> && is_unsigned_v<T>);
    static_assert(1 <= Bits && Bits <= numeric_limits<T>::digits);

    struct Entry
    {
        // v 是主元，p 是它在前缀基中保留的最靠右来源位置。
        T v{};
        int p = -1;
    };

    // pre[r] 是扫描到下标 r 后的带位置线性基。
```

```
    vector<array<Entry, Bits>> pre;

    static bool valid(T x)
    {
        // x 是待处理的无符号整数；返回它是否只使用低 Bits 位。
        if constexpr (Bits == numeric_limits<T>::digits)
        {
            return true;
        }
        else
        {
            return !(x >> Bits);
        }
    }

  public:
    explicit RangeBasis(const vector<T> &a) : pre(a.size())
    {
        // a 是静态无符号整数序列；建立每个右端点的前缀基。
        for (int i = 0; i < (int)a.size(); i++)
        {
            assert(valid(a[i])); // 调试检查，可删。
            if (i)
            {
                pre[i] = pre[i - 1];
            }
            T v = a[i];
            int p = i;
            for (int b = Bits - 1; b >= 0 && v; b--)
            {
                if (!(v >> b & 1))
                {
                    continue;
                }
                auto &e = pre[i][b];
                if (e.p < p)
                {
                    swap(e.v, v);
                    swap(e.p, p);
                }
                v ^= e.v;
            }
        }
    }

    T maxXor(int l, int r, T x = 0) const
    {
        // l、r 是闭区间端点，x 是初值；返回可取得的最大异或值。
        assert(0 <= l && l <= r && r < (int)pre.size()); // 调试检查，可删
        assert(valid(x)); // 调试检查，可删。
        for (int b = Bits - 1; b >= 0; b--)
        {
```

```
            auto [v, p] = pre[r][b];
            if (p >= l && (x ^ v) > x)
            {
                x ^= v;
            }
        }
        return x;
    }
};
```

用无符号整数序列构造 `RangeBasis<T> rb(a)`，`rb.maxXor(l,r,x)` 返回闭区间 $[l,r]$ 内元素与初值 x（默认 0）异或的最大值，要求 $0 \le l \le r < n$。有符号类型不满足按位右移与无符号大小比较的语义，已由静态断言禁止。当 `Bits` 小于 `T` 的有效位数时，`a` 的每项与 `x` 都必须在 $[0, 2^{Bits})$ 内，构造和查询会以 `assert` 检查；取满位宽时允许任意 `T` 值。构造 $O(nW)$ 时间与空间，单次查询 $O(W)$。无下标限制时用 3.8.001 异或线性基，离线带修改时用线段树分治套线性基。

3.8.004 线性基合并

合并两个子空间的线性基，只需把第二组的每个主元依次插入第一组：插入时的消元会自动丢弃线性相关的向量，最终张成空间恰为两者子空间之和。常用于树上启发式合并、线段树/分治节点自底向上归并各区间的线性基：

```
template <int W = 64>
array<unsigned long long, W> mergeBasis(array<unsigned long long, W> a, const array<unsigned
↪ long long, W> &b)
{
    // a、b 都按最高位保存一组异或线性基主元；返回两者张成空间之和的线性基。
    static_assert(1 <= W && W <= 64);
    if constexpr (W < 64)
    {
        for (auto x : a)
        {
            assert(x < (1ULL << W)); // 调试检查，可删。
        }
    }
    for (auto x : b)
    {
        if constexpr (W < 64)
        {
            assert(x < (1ULL << W)); // 调试检查，可删。
        }
        for (int i = W - 1; i >= 0; i--)
        {
            if (!(x >> i & 1))
            {
                continue;
            }
            if (a[i])
            {
```

```
                x ^= a[i];
            }
            else
            {
                a[i] = x;
                break;
            }
        }
    }
    return a;
}
```

两组基都用 `array<unsigned long long,W>`（下标 i 保存最高位为 i 的主元，与 3.8.001 异或线性基 的 `a` 布局一致），调用 `mergeBasis(a,b)` 返回并空间的基，两个输入数组本身不被修改（`a` 按值传入）。当 $W < 64$ 时，两个数组的每个值都必须在 $[0, 2^W)$ 内，代码以 `assert` 检查；$W = 64$ 时允许任意 `unsigned long long`。$O(W^2)$ 时间、$O(W)$ 返回空间。这与 3.8.001 异或线性基 的成员 `merge` 等价，区别只是这里操作裸数组、便于在没有对象包装的分治节点上直接合并。

3.9 GNU PBDS

3.9.001 PBDS 头文件、命名空间、模板策略与迭代器体系

GNU PBDS 的容器类型由底层标签（`rb_tree_tag`、各种堆 tag 等）、比较器和节点更新策略三者共同拼出，使用前必须显式包含 `ext/pb_ds` 下对应的头文件，且要区分两类迭代器：`tree`/哈希表的普通迭代器与堆的“点迭代器”能力不同，后者代表一个具体节点、可用于 `modify`/`erase`，不能与普通容器迭代器混用。下面的示例把有序树封在语义命名空间里，用它同时求排名与后继。

```
// 知识点 3.9.001：PBDS 头文件、命名空间、模板策略与迭代器体系。
namespace pbds_ordered
{
using namespace __gnu_pbds;

template <class T> using OrdSet = tree<T, null_type, less<T>, rb_tree_tag,
 ↪  tree_order_statistics_node_update>;

array<int, 2> rankInfo(const vector<int> &a, int x)
{
    // a 是待插入值，x 是查询值；返回严格小于 x 的数量与第一个不小于 x 的值，无后继时第二维为 -1。
    OrdSet<pair<int, int>> tr;
    for (int i = 0; i < (int)a.size(); i++)
    {
        tr.insert({a[i], i});
    }
    int rk = tr.order_of_key({x, -1});
    auto it = tr.lower_bound({x, -1});
    return {rk, it == tr.end() ? -1 : it->first};
}
}
```

PBDS 是 GNU libstdc++ 扩展。第 3 章公共前导已在 `#define int long long` 之前包含以下三个头文件：

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/priority_queue.hpp>
#include <ext/pb_ds/tree_policy.hpp>
```

它们分别提供 `tree`/哈希表、PBDS 堆、`tree_order_statistics_node_update`。切忌全局 `using namespace __gnu_pbds`，否则其 `priority_queue` 会与 STL 同名冲突，故示例都封在命名空间里、按 `pbds_ordered::rankInfo(a,x)` 这样限定调用；Clang 配 libc++ 或 MSVC 不能直接用这些头。插入与排名查询最坏 $O(\log n)$，空间 $O(n)$。后续 3.9.002–3.9.006 分述各种 tag，3.9.007 tree_order_statistics_node_update 与 3.9.008 find_by_order 与 order_of_key 给出排名/第 k 小的最小接口，3.9.011 gp_hash_table 讨论哈希容器。

3.9.002 PBDS `priority_queue` 策略

PBDS 的 `priority_queue` 可换五种底层堆标签，它们在合并、改键和常数上各有取舍：`binary_heap_tag` 常数最小但不支持高效合并；`binomial_heap_tag`、`rc_binomial_heap_tag` 支持对数级合并；`pairing_heap_tag` 的 `push`、`join` 均摊 $O(1)$，`modify` 均摊 $O(\log n)$，是需要句柄修改与合并时的默认选择；`thin_heap_tag` 在只减键的场景（如 Dijkstra）表现好。纯取堆顶而无需合并或改键时，STL 二叉堆通常更快。

```
// 知识点 3.9.002：PBDS priority_queue 的五种策略标签。
namespace pbdsHeap
{
using namespace __gnu_pbds;

template <class T> using BinaryHeap = __gnu_pbds::priority_queue<T, less<T>, binary_heap_tag>;
template <class T> using BinomialHeap = __gnu_pbds::priority_queue<T, less<T>,
↪ binomial_heap_tag>;
template <class T> using RcBinHeap = __gnu_pbds::priority_queue<T, less<T>,
↪ rc_binomial_heap_tag>;
template <class T> using PairingHeap = __gnu_pbds::priority_queue<T, less<T>,
↪ pairing_heap_tag>;
template <class T> using ThinHeap = __gnu_pbds::priority_queue<T, less<T>, thin_heap_tag>;

int heapTop(const vector<int> &a)
{
    // a 是初始键值；用默认推荐的 pairing heap 返回最大值，空表返回 -1。
    PairingHeap<int> hp;
    for (int x : a)
    {
        hp.push(x);
    }
    return hp.empty() ? -1 : hp.top();
}
}
```

`pbdsHeap::heapTop(a)` 用推荐的 pairing heap 建堆并取最大值，空表返回 `-1`；换 tag 只需改 `using` 别名。示例建堆 $O(n \log n)$、取堆顶 $O(1)$，各 tag 的 `modify`/`join` 界以 GNU PBDS 实现为准。要真正用

上点迭代器改键、删点、合并见 3.9.003 PBDS 堆点迭代器；只取堆顶时直接用 STL `priority_queue`。

3.9.003 PBDS 堆的 `modify`、`erase`、`join` 与点迭代器

PBDS 堆相对 STL 堆的核心增值在于点迭代器：`push` 返回的句柄代表一个具体节点，据此可 `modify(it, 新值)` 改键或 `erase(it)` 删除任意元素，而 `join(b)` 把另一堆的全部节点并入当前堆、并使 `b` 变空。句柄一旦对应节点被删就失效，改键/删点务必用仍有效的句柄。

```cpp
// 知识点 3.9.003：PBDS 堆的点迭代器、modify、erase 与 join。
namespace pbdsMutHeap
{
using namespace __gnu_pbds;

using Heap = __gnu_pbds::priority_queue<int, less<int>, pairing_heap_tag>;

int editHeap()
{
    // 演示点迭代器改键、删除与合并，并返回最终堆顶。
    Heap a, b;
    auto p = a.push(10);
    auto q = a.push(30);
    a.modify(p, 40);
    a.erase(q);
    b.push(25);
    a.join(b);
    return a.top();
}
}
```

`pbdsMutHeap::editHeap()` 走一遍改键、删点、合并后返回堆顶。pairing heap 的 `push`、`join` 均摊 $O(1)$，`modify`、`pop`、`erase` 均摊 $O(\log n)$，空间 $O(n)$。这套接口正是 Dijkstra 减键、堆式贪心合并等场景相对 STL 堆的优势所在；tag 的选型见 3.9.002 五种堆策略标签，与其他容器的取舍见 3.9.004 PBDS 堆与 STL 堆、线段树的选型。

3.9.004 PBDS 堆和 STL 堆、二叉堆、线段树的性能与适用范围

同是“取极值”的需求，STL `priority_queue` 接口稳定、常数最小，适合只做 push/pop/top；PBDS 堆的价值在可合并与句柄改键；手写二叉堆便于内联定制；线段树则适合按下标随机聚合与区间极值。选型看真正需要哪些操作，而不是只比理论复杂度——为用不到的 `modify`/`join` 付常数是常见浪费。

只做 push/pop/top 时直接使用 STL `priority_queue`；需要 `modify`、`erase` 或 `join` 时使用 3.9.003 PBDS 堆点迭代器；键具有固定下标并需要区间查询或修改时使用本章线段树。这里是接口选型，不另造比较函数；具体性能取决于操作比例、数据分布、编译器和平台，应在目标环境用实际数据单独基准。

3.9.005 `tree`：`rb_tree_tag`、`splay_tree_tag`、`ov_tree_tag`

PBDS `tree` 的底层平衡策略同样可换 tag：`rb_tree_tag` 红黑树给稳定的最坏对数界，是默认选择；`splay_tree_tag` 偏向访问局部性，适合热点集中的序列；`ov_tree_tag` 用连续向量存储换取查询紧凑，但插删最坏 $O(n)$，只宜读多写少。三种策略与节点更新策略（下面几条）互相独立，可任意组合。

```cpp
// 知识点 3.9.005：PBDS tree 的三种底层平衡策略。
namespace pbdsTree
{
using namespace __gnu_pbds;

template <class T> using RBTree = tree<T, null_type, less<T>, rb_tree_tag>;
template <class T> using SplayTree = tree<T, null_type, less<T>, splay_tree_tag>;
template <class T> using OVTree = tree<T, null_type, less<T>, ov_tree_tag>;
}
```

按操作特征选一个别名实例化即可，例如 `pbdsTree::RBTree<int> s`。红黑树操作最坏 $O(\log n)$，Splay 均摊 $O(\log n)$，ov_tree 查询 $O(\log n)$ 而插删最坏 $O(n)$。挂上 order-statistics 或自定义更新策略见 3.9.006 set/map/split/join、3.9.007 tree_order_statistics_node_update 与 3.9.009 自定义 Node_Update。

3.9.006 PBDS `tree` 的 set/map 用法、`split` 与 `join`

PBDS `tree` 把值类型设为 `null_type` 即是 set，设为具体类型即是 map，键须满足严格弱序。它比 STL `set`/`map` 多出 $O(\log n)$ 的整体 `split`/`join`：`split(x, b)` 把所有严格大于 `x` 的键搬到（必须为空的）`b`，`join(b)` 要求 `a` 的全部键严格小于 `b` 的全部键（键域完全分离）才合法，否则不满足接口前提。

```cpp
// 知识点 3.9.006：PBDS tree 的 set/map、split 与 join。
namespace pbdsSplit
{
using namespace __gnu_pbds;

using Map = tree<int, int, less<int>, rb_tree_tag>;

void splitAt(Map &a, Map &b, int x)
{
    // a 是原映射，b 必须为空，x 是分界键；把所有严格大于 x 的键移入 b。
    a.split(x, b);
}

void joinOrdered(Map &a, Map &b)
{
    // a、b 的键域必须严格分离且 a 的键全小于 b；把 b 合入 a 并清空 b。
    a.join(b);
}
}
```

`splitAt(a, b, x)` 要求 `b` 为空，`joinOrdered(a, b)` 要求键域分离且 `a` 全小于 `b`，两者都原地修改传入的树。红黑树插入、查找最坏 $O(\log n)$，`split`/`join` 的界依赖策略与键域前提，空间 $O(n)$。要顺带

维护子树大小以支持排名见 3.9.007 tree_order_statistics_node_update，要维护子树和等可合并信息见 3.9.009 自定义 Node_Update。

3.9.007 `tree_order_statistics_node_update`

把节点更新策略设为 `tree_order_statistics_node_update`，红黑树的每个节点就自动维护子树大小并在旋转后同步，从而开放“按排名取值”和“统计严格小于某键的个数”两个接口——这正是竞赛里最常用的“权值平衡树”配置，本书 3.5.007 等条目就直接借它当第 k 小结构。

```
template <class T>
using OSet = __gnu_pbds::
    tree<T, __gnu_pbds::null_type, less<T>, __gnu_pbds::rb_tree_tag,
    __gnu_pbds::tree_order_statistics_node_update>;
```

`OSet<T>` 的模板参数 `T` 是键类型，默认 `less<T>` 升序；`tree` 的键唯一，要存重复值须把键换成 `pair<` 值，唯一编号 `>` 以人为区分。该类型依赖 GNU PBDS 与 `tree_policy.hpp`，insert、erase、order_of_key、find_by_order 均最坏 $O(\log n)$，空间 $O(n)$。定义好后按 3.9.008 find_by_order 与 order_of_key 做第 k 小与排名查询；要维护除大小以外的可合并信息见 3.9.009 自定义 Node_Update。

3.9.008 `find_by_order` 与 `order_of_key`

有了 3.9.007 的 `OSet` 后，两个成员即可完成排名类查询：`find_by_order(k)` 返回 0 下标第 k 小元素的迭代器、越界时等于 `end()`；`order_of_key(x)` 返回严格小于 `x` 的键数（即 `x` 的 0 基排名）。两者互为逆向，组合即“查第 k 小”与“查某值排名”。

```
// 依赖 3.9.007 tree_order_statistics_node_update
OSet<int> tr;

// 直接在题目逻辑中执行 tr.insert(x)。
// tr.find_by_order(k) 返回 0 下标第 k 小的迭代器，越界时等于 tr.end()。
// tr.order_of_key(x) 返回严格小于 x 的键数。
```

类型定义取自 3.9.007 tree_order_statistics_node_update，本条只演示这两个查询接口，均最坏 $O(\log n)$，空间 $O(n)$。若查询需求超出“大小/排名”（如子树和、区间和），改用 3.9.009 自定义 Node_Update 或 3.9.010 自定义子树信息 定制 metadata。

3.9.009 自定义 `Node_Update`、`metadata_type` 与节点迭代器

要维护子树大小以外的信息，就自定义一个 `Node_Update` 仿函数：声明它的 `metadata_type`，并实现 `operator()(it, end)`——PBDS 会在插入、删除、旋转后对受影响节点回调它，函数内通过节点迭代器 `get_l_child`/`get_r_child`/`get_metadata` 读左右儿子的 metadata，按“当前键 + 左子树 + 右子树”的公式算出本节点 metadata 写回。合并公式只能依赖这三者，才能随旋转局部正确重算。下例维护子树节点数（等价于内置的 order-statistics）。

```cpp
// 知识点 3.9.009：自定义 Node_Update、metadata_type 与节点迭代器。
namespace pbdsMeta
{
using namespace __gnu_pbds;

template <class NodeIt, class NodeCIt, class Cmp, class Alloc> struct SizeUpdate
{
    using metadata_type = int;

    void operator()(NodeIt it, NodeCIt end) const
    {
        // it 是待更新节点，end 是空节点；把元信息更新为子树节点数。
        int s = 1;
        auto l = it.get_l_child();
        auto r = it.get_r_child();
        if (l != end)
        {
            s += l.get_metadata();
        }
        if (r != end)
        {
            s += r.get_metadata();
        }
        const_cast<int &>(it.get_metadata()) = s;
    }
};

using Tree = tree<int, null_type, less<int>, rb_tree_tag, SizeUpdate>;

int rootSize(const vector<int> &a)
{
    // a 是待插入值；返回去重后的树根元信息，空树返回 0。
    Tree tr;
    for (int x : a)
    {
        tr.insert(x);
    }
    return tr.empty() ? 0 : tr.node_begin().get_metadata();
}
}
```

`pbdsMeta::rootSize(a)` 插入去重后返回树根 metadata（即元素总数），空树返回 0。每次 Node_Update 合并 $O(1)$，红黑树单次插删最坏 $O(\log n)$，空间 $O(n)$。只求大小/排名时直接用现成的 3.9.007 tree_order_statistics_node_update 即可；把 metadata 换成子树和就得到 3.9.010 自定义子树信息 的前缀和查询。

3.9.010 自定义子树信息：区间和、前缀和及其他可合并信息

把 3.9.009 的 metadata 换成“子树内所有键之和”，就能在动态插删的平衡树上回答键值前缀和：查“严格小于 x 的键之和”时沿搜索路径下行，每当当前键小于 x 就把它连同其左子树的 metadata 整块累加、

再转向右子树，否则只转向左子树。整块跳过左子树是 $O(\log n)$ 的关键，前提是键值更新后 metadata 已由树维护到位。

```cpp
// 知识点 3.9.010：用自定义子树和回答键值前缀和。
namespace pbdsSubSum
{
using namespace __gnu_pbds;

template <class NodeIt, class NodeCIt, class Cmp, class Alloc> struct SumUpdate
{
    using metadata_type = int;

    void operator()(NodeIt it, NodeCIt end) const
    {
        // it 是待更新节点，end 是空节点；元信息保存本节点键与左右子树键之和。
        int s = **it;
        auto l = it.get_l_child();
        auto r = it.get_r_child();
        if (l != end)
        {
            s += l.get_metadata();
        }
        if (r != end)
        {
            s += r.get_metadata();
        }
        const_cast<int &>(it.get_metadata()) = s;
    }
};

using Tree = tree<int, null_type, less<int>, rb_tree_tag, SumUpdate>;

int prefixSum(const Tree &tr, int x)
{
    // tr 是键集合，x 是右边界；返回所有严格小于 x 的不同键之和。
    int ans = 0;
    auto it = tr.node_begin();
    auto end = tr.node_end();
    while (it != end)
    {
        auto l = it.get_l_child();
        int v = **it;
        if (v < x)
        {
            if (l != end)
            {
                ans += l.get_metadata();
            }
            ans += v;
            it = it.get_r_child();
        }
        else
```

```
        {
            it = l;
        }
    }
    return ans;
}
}
```

`pbdsSubSum::prefixSum(tr, x)` 返回所有严格小于 `x` 的不同键之和，更新与查询均最坏 $O(\log n)$，空间 $O(n)$。这是“可修改的权值前缀和”，键固定时用普通 3.2.001 树状数组 更省常数；只要计数不要和则回到 3.9.007 tree_order_statistics_node_update。metadata 的重算框架同 3.9.009 自定义 Node_Update。

3.9.011 `cc_hash_table` 与 `gp_hash_table`

PBDS 提供两种哈希表替代 `unordered_map`：`cc_hash_table` 用拉链法处理冲突，`gp_hash_table` 用开放寻址，后者局部性好、通常更快，但对负载因子和哈希质量更敏感，二者接口也与 `unordered_map` 不完全一致。防 hack 的混合函数直接复用 1.4.004 `unordered_map` / `unordered_set` 防 hack 哈希 的 `CustomHash`，这里仅定义整数键 PBDS 别名，不重复打印实现。

```
// 依赖 1.4.004 unordered_map / unordered_set 防 hack 哈希
namespace pbds_hash
{
using namespace __gnu_pbds;

template <class K, class V> using HashMap = gp_hash_table<K, V, CustomHash>;
}
```

`pbds_hash::HashMap<K,V>` 是挂有 `CustomHash` 的 `gp_hash_table`，按 `[]`、`find` 使用；`K` 使用 64 位整数。若要使用 `pair<int,int>` 或其它键，按实际键型直接定义 `gp_hash_table`，不要再给这一行别名堆约束。操作期望 $O(1)$、确定性最坏 $O(n)$，空间 $O(n)$。追求稳定最坏界或有序排名时改用 3.9.007 tree_order_statistics_node_update，策略与常数见 3.9.012 哈希策略与性能测试。

3.9.012 哈希策略、探测策略、扩容策略与性能测试

这一条是选型总结而非新代码，底层实现即 3.9.011 cc_hash_table 与 gp_hash_table 的 `HashMap` 与 1.4.004 防 hack 哈希 的 `CustomHash`。三类策略各管一段：哈希函数决定键映射是否均匀，探测或拉链策略决定冲突后的查找路径长度，扩容策略决定重散列频率与内存峰值。`gp_hash_table` 的开放寻址在高负载下探测链会急剧变长，故它比 `cc_hash_table` 更依赖负载因子和哈希质量。性能比较必须注明数据规模、键分布、读写比例与编译参数，并覆盖接近题面的连续、等差和聚簇键；不能把单组随机计时当作普遍结论。操作期望 $O(1)$、确定性最坏 $O(n)$，空间 $O(n)$。需要有序或排名时用 3.9.007 tree_order_statistics_node_update。

3.9.013 编译器版本兼容性、常数、失效规则与不适用场景

PBDS 是 GCC/libstdc++ 的私有扩展，不属于 C++ 标准，Clang 配 libc++、MSVC 都不提供，因此它只在确认评测机用 GNU 工具链时才可依赖；受限环境要提前准备 STL 或自写替代。此外堆的点迭代器在对

应节点被删后即失效，跨 GCC 大版本升级时接口细节、句柄失效规则与常数都可能变，移植前应重新核对。下例用宏 `__GLIBCXX__` 做一次编译期可用性判断。

```cpp
// 知识点 3.9.013：PBDS 的编译器边界与最小可用性检查。
namespace pbds_compat
{
using namespace __gnu_pbds;

bool available()
{
    // 当前源文件已在 GNU libstdc++ 环境中通过预处理时返回 true。
#ifdef __GLIBCXX__
    return true;
#else
    return false;
#endif
}
}
```

`pbds_compat::available()` 只能说明这份源文件已经在 libstdc++ 环境中成功通过预处理；若环境根本没有 PBDS 头，文件会在到达该函数之前编译失败，因此它不能在同一源文件中运行时探测或补救“缺少 PBDS”。真要维护跨工具链版本，应在构建配置中选择不同源文件，或在包含扩展头之前用 `__has_include` 条件编译整段 PBDS 代码。有序容器可回退 STL `set`/`map`，排名用手写平衡树或 3.2.001 树状数组，哈希表用 1.4.004 防 hack 哈希 的 `CustomHash` 配标准 `unordered_map`。

3.10 根号分块

3.10.001 基础数列分块

把长度 n 的数列按块长 $B \approx \sqrt{n}$ 划分，整块保存加法标记、区间和与最值，零散部分直接枚举。部分修改前把整块标记下放，修改后重算该块；完整块只改摘要，因此代码短且适合同时维护多种容易合并的块信息。

```cpp
struct SqrtBlock
{
    int n, B, m; // n 是长度，B 是块长，m 是块数。
    vector<int> a, tag, sum, mn, mx; // a 是元素（不含所在块标记），tag 是块加标记，sum、mn、mx 是含标记的块和、块最小、块最大。

    int L(int k) const
    {
        // k 是块编号；返回第 k 块的起始下标（含）。
        return k * B;
    }

    int R(int k) const
    {
        // k 是块编号；返回第 k 块的结束下标（不含），末块截到 n。
        return min(n, (k + 1) * B);
```

```
    }

    void pull(int k)
    {
        // k 是块编号；按块内 a 值重算 sum、mn、mx，要求该块标记已下放（tag[k]=0）。
        sum[k] = 0;
        mn[k] = numeric_limits<int>::max();
        mx[k] = numeric_limits<int>::lowest();
        for (int i = L(k); i < R(k); i++)
        {
            sum[k] += a[i];
            mn[k] = min(mn[k], a[i]);
            mx[k] = max(mx[k], a[i]);
        }
    }

    void push(int k)
    {
        // k 是块编号；把 tag[k] 加到块内每个 a[i] 后清零，块摘要已含标记故不变。
        if (!tag[k])
        {
            return;
        }
        for (int i = L(k); i < R(k); i++)
        {
            a[i] += tag[k];
        }
        tag[k] = 0;
    }

    SqrtBlock(vector<int> v) : n(v.size()), a(move(v))
    {
        // v 是初始数列（0 起下标）；取块长 B≈√n 分块，清零标记并算出每块的和、最小值、最大值。
        B = max<int>(1, (int)sqrtl(max<int>(1, n)));
        m = (n + B - 1) / B;
        tag.assign(m, 0);
        sum.resize(m);
        mn.resize(m);
        mx.resize(m);
        for (int k = 0; k < m; k++)
        {
            pull(k);
        }
    }

    void add(int l, int r, int v)
    {
        // [l,r) 是 0 起半开区间，v 是增量；给区间内每个元素加 v，整块只改标记与摘要，零散块下放后重算。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删。
        if (l == r)
        {
            return;
```

```
        }
        for (int k = l / B; L(k) < r; k++)
        {
            int x = max(l, L(k)), y = min(r, R(k));
            if (x == L(k) && y == R(k))
            {
                tag[k] += v;
                sum[k] += v * (y - x);
                mn[k] += v;
                mx[k] += v;
            }
            else
            {
                push(k);
                for (int i = x; i < y; i++)
                {
                    a[i] += v;
                }
                pull(k);
            }
        }
    }

    array<int, 3> query(int l, int r) const
    {
        // 返回区间的 {和, 最小值, 最大值}。
        assert(0 <= l && l < r && r <= n); // 调试检查，可删。
        array<int, 3> ans{0, numeric_limits<int>::max(), numeric_limits<int>::lowest()};
        for (int k = l / B; L(k) < r; k++)
        {
            int x = max(l, L(k)), y = min(r, R(k));
            if (x == L(k) && y == R(k))
            {
                ans[0] += sum[k];
                ans[1] = min(ans[1], mn[k]);
                ans[2] = max(ans[2], mx[k]);
            }
            else
            {
                for (int i = x; i < y; i++)
                {
                    int v = a[i] + tag[k];
                    ans[0] += v;
                    ans[1] = min(ans[1], v);
                    ans[2] = max(ans[2], v);
                }
            }
        }
        return ans;
    }
};
```

构造 `SqrtBlock q(a)` 后，`add(l,r,v)` 修改 $[l,r)$，`query(l,r)` 对非空区间返回 `{和，最小值，最大值}`。所有单点值、块标记、块和与 `v*(r-l)` 必须能由当前 64 位 `int` 表示。构造 $O(n)$，单次修改或查询 $O(\sqrt{n})$，空间 $O(n)$；只维护区间和时用 3.2.004 区间修改、区间求和 更快。

3.10.002 有序数列分块

每块再保存一份排序后的元素，完整块可二分统计小于或小于等于给定值的个数；区间前驱、下界和第 K 小都建立在同一计数上。整块加只改标记，零散修改下放标记后重排该块。

```cpp
struct OrderBlock
{
    int n, B, m; // n 是长度，B 是块长，m 是块数。
    vector<int> a, tag; // a 是元素（不含所在块标记），tag 是块加标记。
    vector<vector<int>> s; // s[k] 是第 k 块元素（不含标记）的升序副本。

    int L(int k) const
    {
        // k 是块编号；返回第 k 块的起始下标（含）。
        return k * B;
    }

    int R(int k) const
    {
        // k 是块编号；返回第 k 块的结束下标（不含），末块截到 n。
        return min(n, (k + 1) * B);
    }

    void pull(int k)
    {
        // k 是块编号；用块内 a 值重建升序副本 s[k]，要求该块标记已下放（tag[k]=0）。
        s[k].assign(a.begin() + L(k), a.begin() + R(k));
        sort(s[k].begin(), s[k].end());
    }

    void push(int k)
    {
        // k 是块编号；把 tag[k] 加到块内每个 a[i] 后清零。
        if (!tag[k])
        {
            return;
        }
        for (int i = L(k); i < R(k); i++)
        {
            a[i] += tag[k];
        }
        tag[k] = 0;
    }

    OrderBlock(vector<int> v) : n(v.size()), a(move(v))
    {
```

```
        // v 是初始数列（0 起下标）；取块长 B≈√n 分块，清零标记并为每块建立升序副本。
        B = max<int>(1, (int)sqrtl(max<int>(1, n)));
        m = (n + B - 1) / B;
        tag.assign(m, 0);
        s.resize(m);
        for (int k = 0; k < m; k++)
        {
            pull(k);
        }
    }

    void add(int l, int r, int v)
    {
        // [l,r) 是 0 起半开区间，v 是增量；给区间内每个元素加 v，整块只改标记，零散块下放后重排。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删。
        if (l == r)
        {
            return;
        }
        for (int k = l / B; L(k) < r; k++)
        {
            int x = max(l, L(k)), y = min(r, R(k));
            if (x == L(k) && y == R(k))
            {
                tag[k] += v;
            }
            else
            {
                push(k);
                for (int i = x; i < y; i++)
                {
                    a[i] += v;
                }
                pull(k);
            }
        }
    }

    int count(int l, int r, int x, bool le = false) const
    {
        // [l,r) 是查询区间，x 是阈值；le 为假时返回区间内 <x 的元素个数，为真时返回 ≤x 的个数。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删。
        int ans = 0;
        for (int k = l / B; L(k) < r; k++)
        {
            int p = max(l, L(k)), q = min(r, R(k));
            if (p == L(k) && q == R(k))
            {
                auto it = partition_point(s[k].begin(), s[k].end(),
                                          [&](int v)
                                          {
                                              return le ? v + tag[k] <= x : v + tag[k] < x;
```

```
                                        });
                ans += it - s[k].begin();
            }
            else
            {
                for (int i = p; i < q; i++)
                {
                    int v = a[i] + tag[k];
                    ans += le ? v <= x : v < x;
                }
            }
        }
        return ans;
    }

    optional<int> prev(int l, int r, int x) const
    {
        // [l,r) 是查询区间，x 是阈值；返回区间内严格小于 x 的最大元素，不存在时为空。
        optional<int> ans;
        for (int k = l / B; L(k) < r; k++)
        {
            int p = max(l, L(k)), q = min(r, R(k));
            if (p == L(k) && q == R(k))
            {
                auto it = partition_point(s[k].begin(), s[k].end(),
                                          [&](int v)
                                          {
                                              return v + tag[k] < x;
                                          });
                if (it != s[k].begin())
                {
                    int v = *(it - 1) + tag[k];
                    if (!ans || *ans < v)
                    {
                        ans = v;
                    }
                }
            }
            else
            {
                for (int i = p; i < q; i++)
                {
                    int v = a[i] + tag[k];
                    if (v < x && (!ans || *ans < v))
                    {
                        ans = v;
                    }
                }
            }
        }
        return ans;
    }
```

```
146
147     optional<int> next(int l, int r, int x) const
148     {
149         // [l,r) 是查询区间，x 是阈值；返回区间内大于等于 x 的最小元素，不存在时为空。
150         optional<int> ans;
151         for (int k = l / B; L(k) < r; k++)
152         {
153             int p = max(l, L(k)), q = min(r, R(k));
154             if (p == L(k) && q == R(k))
155             {
156                 auto it = partition_point(s[k].begin(), s[k].end(),
157                                           [&](int v)
158                                           {
159                                               return v + tag[k] < x;
160                                           });
161                 if (it != s[k].end())
162                 {
163                     int v = *it + tag[k];
164                     if (!ans || v < *ans)
165                     {
166                         ans = v;
167                     }
168                 }
169             }
170             else
171             {
172                 for (int i = p; i < q; i++)
173                 {
174                     int v = a[i] + tag[k];
175                     if (x <= v && (!ans || v < *ans))
176                     {
177                         ans = v;
178                     }
179                 }
180             }
181         }
182         return ans;
183     }
184
185     int kth(int l, int r, int k, int lo, int hi) const
186     {
187         // [l,r) 是查询区间，k 从 0 计数，[lo,hi] 是须覆盖答案的整数值域；二分返回区间第 k 小的元素值。
188         assert(0 <= k && k < r - l && lo <= hi); // 调试检查，可删。
189         while (lo < hi)
190         {
191             int mid = midpoint(lo, hi);
192             if (count(l, r, mid, true) > k)
193             {
194                 hi = mid;
195             }
196             else
197             {
```

```
                lo = mid + 1;
            }
        }
        return lo;
    }
};
```

`count(l,r,x)` 默认统计 $< x$，末参传 `true` 时统计 $\le x$；`prev` 返回严格前驱，`next` 返回大于等于 `x` 的下界，均以空 `optional` 表示不存在。`kth(l,r,k,lo,hi)` 的 `k` 从零计数，且调用者给出的整数值域必须覆盖真实答案。所有元素加块标记后须能由 64 位 `int` 表示。修改、计数、前驱和下界均为 $O(\sqrt{n}\log n)$，第 K 小再乘值域二分次数，空间 $O(n)$；静态区间查询优先用 3.4.005 小波矩阵。

3.10.003 王室联邦树分块

树分块把 DFS 完成的子树点暂存在栈中；某个儿子返回后，若当前节点以下尚未分块的点达到 B 个，就弹出为一块并把当前节点记作代表点。最后不足 B 个点并入最后一块。代表点只负责连接这一批点，不保证自身属于该块。

```
struct TreeBlock
{
    vector<int> bel, cap; // bel[u] 是点 u 所属块编号，cap[k] 是第 k 块的块根（省会）。

    TreeBlock(const vector<vector<int>> &g, int root, int B) : bel(g.size(), -1)
    {
        // g 是树的邻接表，root 是根，B 是块大小下界；王室联邦分块，每块至少 B 个点（剩余点并入最后一
        ↪ 块），块根可能不在块内。
        assert(!g.empty() && 0 <= root && root < (int)g.size() && B > 0); // 调试检查，可删。
        vector<int> st;
        auto dfs = [&](auto &&self, int u, int f) -> void
        {
            int base = st.size();
            for (int v : g[u])
            {
                if (v == f)
                {
                    continue;
                }
                self(self, v, u);
                if ((int)st.size() - base >= B)
                {
                    cap.push_back(u);
                    while ((int)st.size() > base)
                    {
                        bel[st.back()] = (int)cap.size() - 1;
                        st.pop_back();
                    }
                }
            }
            st.push_back(u);
```

```
        };
        dfs(dfs, root, -1);
        if (cap.empty())
        {
            cap.push_back(root);
        }
        while (!st.empty())
        {
            bel[st.back()] = (int)cap.size() - 1;
            st.pop_back();
        }
    }
};
```

输入必须是以 `root` 为根的连通无向树，`B>0`；`bel[u]` 是点 `u` 所属块，`cap[k]` 是第 `k` 块代表点。除最后一块外每块至少有 B 个点，块数 $O(n/B)$；构造 $O(n)$ 时间、$O(n)$ 空间，递归深度最坏 $O(n)$。它只给出分块，不自动维护块间路径；需要完整路径查询时使用第 6 章的 LCA、树链剖分或点分治。

3.10.004 图上按度数分块

把度数至少为 $B \approx \sqrt{2m}$ 的点设为重点。重点预存邻点权值和；修改点权时只更新与该点相邻的重点摘要，查询轻点则直接枚举邻边。重点数至多 $2m/B$，于是修改和查询都被根号界控制。

```
struct DegreeBlock
{
    int B; // B 是重点度数阈值，约为 sqrt(2m)。
    vector<vector<int>> g, big; // g 是邻接表，big[u] 是 u 的重点邻居编号（id 值）。
    vector<int> id, w, sum; // id[u] 是重点编号（轻点为 -1），w 是点权，sum[k] 是重点 k 的邻点权值和。

    DegreeBlock(int n, const vector<array<int, 2>> &e, vector<int> init = {})
        : g(n), big(n), id(n, -1), w(move(init))
    {
        // n 是点数，e 是无向边 {u,v}（0 起、无自环），init 是初始点权（空表示全 0）。
        // 建邻接表，把度数不小于 B≈⌊√(2m)⌋ 的点设为重点并预存其邻点权值和。
        assert(n >= 0 && (w.empty() || (int)w.size() == n)); // 调试检查，可删。
        w.resize(n);
        for (auto [u, v] : e)
        {
            assert(0 <= u && u < n && 0 <= v && v < n && u != v); // 调试检查，可删。
            g[u].push_back(v);
            g[v].push_back(u);
        }
        B = max<int>(1, (int)sqrtl(2 * (int)e.size() + 1));
        for (int u = 0; u < n; u++)
        {
            if ((int)g[u].size() >= B)
            {
                id[u] = sum.size();
                sum.push_back(0);
            }
```

```
        }
        for (int u = 0; u < n; u++)
        {
            for (int v : g[u])
            {
                if (id[v] >= 0)
                {
                    big[u].push_back(id[v]);
                }
                if (id[u] >= 0)
                {
                    sum[id[u]] += w[v];
                }
            }
        }
    }

    void add(int u, int v)
    {
        // u 是点编号，v 是增量；令 w[u]+=v，并同步更新与 u 相邻的所有重点的邻点和。
        w[u] += v;
        for (int k : big[u])
        {
            sum[k] += v;
        }
    }

    int query(int u) const
    {
        // u 是点编号；返回 u 所有邻点的权值和（重边按重数计），重点直接取预存值。
        if (id[u] >= 0)
        {
            return sum[id[u]];
        }
        int ans = 0;
        for (int v : g[u])
        {
            ans += w[v];
        }
        return ans;
    }
};
```

构造 `DegreeBlock q(n,e,w)` 后，`add(u,v)` 给点 `u` 的权值增加 `v`，`query(u)` 返回所有邻点权值和。输入必须是简单无向图；重边会被当成多条边重复累计。点权和任意邻点权值和须能由 64 位 `int` 表示。构造 $O(n+m)$，单次修改、查询 $O(\sqrt{m})$，空间 $O(n+m)$；若操作是边增删而不是点权修改，需要另外维护动态重点结构。

4. 字符串

4.1 基础匹配与哈希

4.1.001 字符串哈希（Mersenne 单模、随机底数）

把长度 n 的串预处理成前缀哈希 $h[i]=\sum_{j<i}(s_j+1)base^{i-1-j} \bmod P$ 后，子串 $s[l,r)$ 的哈希就是 $h[r]-h[l]base^{r-l}$。这里取 Mersenne 素数 $P=2^{61}-1$：两个代表元相乘先放入 `__uint128_t`，再利用 $2^{61}\equiv 1 \pmod P$ 把高 61 位与低 61 位相加约简，避免普通 `%`。`HASH_BASE` 在进程启动时从 $[P/2,P-2]$ 随机一次，所有对象共享它；因此固定测试数据不能预先针对一个公开底数构造碰撞。对长度不超过 n 的两个不同等长串，其差是次数至多 $n-1$ 的非零多项式，在当前候选集合上碰撞概率至多约 $2(n-1)/P$，但哈希判等仍不是确定性证明。

```
struct HashValue
{
    // x 是模 2^61-1 的代表元；无符号 64 位与 128 位乘积是 Mersenne 快速约简所必需的。
    static constexpr unsigned long long mod = (1ULL << 61) - 1;
    unsigned long long x = 0;

    HashValue(unsigned long long x = 0) : x((x & mod) + (x >> 61))
    {
        // x 是待规范化整数；构造模 mod 的哈希值。
        if (this->x >= mod)
        {
            this->x -= mod;
        }
    }

    friend HashValue operator+(HashValue a, HashValue b)
    {
        // a、b 是同模哈希值；返回模 mod 的和。
        unsigned long long x = a.x + b.x;
        if (x >= mod)
        {
            x -= mod;
        }
        return x;
    }

    friend HashValue operator-(HashValue a, HashValue b)
    {
        // a、b 是同模哈希值；返回模 mod 的差。
        unsigned long long x = a.x + mod - b.x;
        if (x >= mod)
        {
            x -= mod;
        }
        return x;
    }
```

```cpp

    friend HashValue operator*(HashValue a, HashValue b)
    {
        // a、b 是同模哈希值；用 128 位乘积返回模 mod 的积。
        __uint128_t x = (__uint128_t)a.x * b.x;
        return (unsigned long long)(x >> 61) + (unsigned long long)(x & mod);
    }

    friend bool operator==(HashValue, HashValue) = default;

    friend ostream &operator<<(ostream &out, HashValue a)
    {
        // out 是输出流，a 是哈希值；输出 a 的规范代表元并返回 out。
        return out << a.x;
    }
};

inline HashValue hashBase()
{
    // 无参数；返回本进程随机选定的大底数。
    unsigned long long seed = (unsigned long
    ↪  long)chrono::steady_clock::now().time_since_epoch().count();
    seed ^= (unsigned long long)random_device{}() << 32;
    seed ^= random_device{}();
    mt19937_64 mt(seed);
    uniform_int_distribution<unsigned long long> dist(HashValue::mod / 2, HashValue::mod - 2);
    return dist(mt);
}

inline const HashValue HASH_BASE = hashBase();

struct HashBase
{
    // pw[i] 是 HASH_BASE 的 i 次幂，按实际需要延长。
    vector<HashValue> pw{HashValue(1)};

    void ensure(int n)
    {
        // n 是需要支持的最大字符串长度；保证 pw[0..n] 已计算。
        while ((int)pw.size() <= n)
        {
            pw.push_back(pw.back() * HASH_BASE);
        }
    }
};

inline HashBase HASH_POWER;

struct DynStrHash
{
    // pre[i] 是当前串前 i 个字符的哈希。
    vector<HashValue> pre{HashValue(0)};
```

```
88
89      int size() const
90      {
91          // 无参数；返回当前字符串长度。
92          return pre.size() - 1;
93      }
94
95      void push_back(char c)
96      {
97          // c 是追加字符；在串尾加入 c。
98          HASH_POWER.ensure(size() + 1);
99          pre.push_back(pre.back() * HASH_BASE + HashValue((unsigned char)c + 1));
100     }
101
102     HashValue get(int l, int r) const
103     {
104         // l、r 是当前串下标；返回半开子串 [l,r) 的哈希值。
105         assert(0 <= l && l <= r && r <= size()); // 调试检查，可删。
106         return pre[r] - pre[l] * HASH_POWER.pw[r - l];
107     }
108
109     HashValue concat(HashValue value, int l, int r) const
110     {
111         // value 是已有串的哈希，l、r 指定当前串的半开子串；返回两段拼接后的哈希。
112         return value * HASH_POWER.pw[r - l] + get(l, r);
113     }
114 };
115
116 struct StrHash
117 {
118     // pre[i] 是原串前 i 个字符的哈希，rev[i] 是反转串前 i 个字符的哈希。
119     vector<HashValue> pre, rev;
120
121     StrHash(const string &s) : pre(s.size() + 1), rev(s.size() + 1)
122     {
123         // s 是原字符串；预处理正向、反向前缀哈希与所需底数幂。
124         int n = s.size();
125         HASH_POWER.ensure(n);
126         for (int i = 0; i < n; i++)
127         {
128             pre[i + 1] = pre[i] * HASH_BASE + HashValue((unsigned char)s[i] + 1);
129             rev[i + 1] = rev[i] * HASH_BASE + HashValue((unsigned char)s[n - 1 - i] + 1);
130         }
131     }
132
133     int size() const
134     {
135         // 无参数；返回原字符串长度。
136         return pre.size() - 1;
137     }
138
139     HashValue get(int l, int r) const
```

```
    {
        // l、r 是原串下标；返回半开子串 [l,r) 的正向哈希。
        assert(0 <= l && l <= r && r <= size()); // 调试检查，可删。
        return pre[r] - pre[l] * HASH_POWER.pw[r - l];
    }

    HashValue getrev(int l, int r) const
    {
        // l、r 是原串下标；返回反转 s[l,r) 后的哈希。
        assert(0 <= l && l <= r && r <= size()); // 调试检查，可删。
        int n = size();
        return rev[n - l] - rev[n - r] * HASH_POWER.pw[r - l];
    }

    HashValue concat(HashValue value, int l, int r) const
    {
        // value 是已有串的哈希，l、r 指定原串半开子串；返回两段拼接后的哈希。
        return value * HASH_POWER.pw[r - l] + get(l, r);
    }

    HashValue concatrev(HashValue value, int l, int r) const
    {
        // value 是已有串的哈希，l、r 指定原串半开子串；返回拼接反转子串后的哈希。
        return value * HASH_POWER.pw[r - l] + getrev(l, r);
    }

    static vector<int> match(const string &s, const string &t)
    {
        // s 是文本串，t 是模式串；返回 t 的全部匹配起点，按下标递增排列。
        if (t.size() > s.size())
        {
            return {};
        }
        StrHash a(s), b(t);
        HashValue target = b.get(0, t.size());
        vector<int> ans;
        for (int i = 0; i + (int)t.size() <= (int)s.size(); i++)
        {
            if (a.get(i, i + t.size()) == target)
            {
                ans.push_back(i);
            }
        }
        return ans;
    }
};
```

构造 `StrHash a(s)` 后，`get(l,r)`、`getrev(l,r)` 分别取半开区间 $[l,r)$ 的正向哈希与反转哈希，`concat`、`concatrev` 把对应子串接到已有哈希后；`DynStrHash` 支持逐字符 `push_back` 和同样的正向查询。幂表由全部对象共享并按最大实际长度增长，不要求预设 `N`。构造与追加总计 $O(n)$，子串哈希和拼接 $O(1)$，`match(s,t)` 为 $O(n+m)$，空间 $O(n+m)$；单模静态版只保存正反两组前缀与一份共享幂表，百万字

符约占 24 MB，旧双模正向实现的四个整数数组约占 32 MB。Windows x86-64、GCC 14.2、`-O2` 下，对随机百万字符冷构造并执行 200 万组最长 1000 的等长子串比较，7 轮中位数从旧版 `8.029 ms + 100.425 ms` 降至 `7.437 ms + 55.973 ms`；新版构造还同时得到旧版没有的反向前缀，该结果只代表当前机器。要确定性线性匹配改用 4.1.002 KMP / 前缀函数 或 4.1.003 扩展 KMP / Z 函数；滑窗用法单列为 4.1.008 Karp-Rabin，只找首次出现见 4.1.010 `strstr` 类匹配。

比赛现场只需要静态子串判等时，可改抄下面的 `ShortHash`。它保留固定底数、$2^{61}-1$ 单模、`__uint128_t` 乘积和 Mersenne 快速约简，只去掉值类型运算符、反向前缀、动态追加、拼接和模式匹配；原版接口仍用于需要这些能力的题目。固定底数不具备运行时随机化，不能抵御针对该常数构造的碰撞数据。

```cpp
struct ShortHash
{
    static constexpr unsigned long long M = (1ULL << 61) - 1; // M 是梅森素数模数 2^61-1。
    // B 从 [M / 2, M - 2] 中选取并固定，避免现场引入随机设备。
    static constexpr unsigned long long B = 123456789012345678 9ULL;
    inline static vector<unsigned long long> p{1}; // p[i] 是 B 的 i 次幂。
    vector<unsigned long long> h;                  // h[i] 是原串前 i 个字符的哈希。

    static unsigned long long red(unsigned long long x)
    {
        // x 小于 2M；返回 x 模 M 的规范代表元。
        x = (x & M) + (x >> 61);
        return x >= M ? x - M : x;
    }

    static unsigned long long mul(unsigned long long a, unsigned long long b)
    {
        // a、b 是模 M 代表元；返回二者乘积模 M。
        __uint128_t x = (__uint128_t)a * b;
        return red((unsigned long long)(x >> 61) + (unsigned long long)(x & M));
    }

    ShortHash(string_view s) : h(s.size() + 1)
    {
        // s 是原字符串；预处理前缀哈希和所需底数幂。
        while (p.size() <= s.size())
        {
            p.push_back(mul(p.back(), B));
        }
        for (int i = 0; i < (int)s.size(); i++)
        {
            h[i + 1] = red(mul(h[i], B) + (unsigned char)s[i] + 1);
        }
    }

    unsigned long long get(int l, int r) const
    {
        // l、r 是原串下标；返回半开子串 [l,r) 的哈希。
        return red(h[r] + M - mul(h[l], p[r - l]));
    }
};
```

构造 `ShortHash h(s)` 后用 `h.get(l,r)` 查询 $s[l,r)$。需要反向查询时，把反串 `t` 另建为 `ShortHash rev(t)`，则反转 $s[l,r)$ 的哈希为 `rev.get(n-r,n-l)`。构造为 $O(n)$，查询为 $O(1)$，每次查询仍只做一次 128 位乘法与 M61 约简；短版每个字符串只保存一组前缀，额外空间为 $O(n)$。底数固定在 $[M/2, M-2]$ 的选定常数，哈希仍是概率判定；需要抵御构造碰撞时使用上面的运行时随机底数版本。性能数据只代表对应机器与数据分布。

4.1.002 KMP / 前缀函数

前缀函数 `pi[i]` 记录 $s[0..i]$ 的最长真 border（既是前缀又是后缀的最长长度），求它时若 `s[i]` 与当前候选位 `s[j]` 失配就沿 `pi` 链跳到次长 border 继续试，均摊线性。有了模式串的前缀函数，KMP 扫描文本时文本指针从不回退：失配就让模式指针沿 `pi` 回跳，匹配到串尾记录一次出现再回跳继续找重叠出现。`borders` 顺着 `pi.back()` 链枚举全部 border，`minPeriod` 用「最短周期 $= n - \pi_{n-1}$、当它整除 n 才是真整周期」判断循环节。

```cpp
vector<int> prefix(const string &s)
{
    // s 是原字符串；返回前缀函数，p[i] 是 s[0..i] 的最长真 border 长度。
    vector<int> p(s.size());
    for (int i = 1; i < (int)s.size(); i++)
    {
        int j = p[i - 1];
        while (j && s[i] != s[j])
        {
            j = p[j - 1];
        }
        if (s[i] == s[j])
        {
            j++;
        }
        p[i] = j;
    }
    return p;
}

vector<int> kmp(const string &s, const string &t)
{
    // s 是文本串，t 是模式串；返回全部匹配起点，按下标递增排列。
    if (t.empty())
    {
        vector<int> a(s.size() + 1);
        iota(a.begin(), a.end(), 0);
        return a;
    }
    auto p = prefix(t);
    vector<int> ans;
    int j = 0;
    for (int i = 0; i < (int)s.size(); i++)
    {
        while (j && s[i] != t[j])
```

```
        {
            j = p[j - 1];
        }
        if (s[i] == t[j])
        {
            j++;
        }
        if (j == (int)t.size())
        {
            ans.push_back(i - j + 1);
            j = p[j - 1];
        }
    }
    return ans;
}

vector<int> borders(const string &s)
{
    // s 是原字符串；返回全部非空真 border 的长度，按递增排列。
    auto p = prefix(s);
    vector<int> a;
    int x = s.empty() ? 0 : p.back();
    while (x)
    {
        a.push_back(x);
        x = p[x - 1];
    }
    reverse(a.begin(), a.end());
    return a;
}

int minPeriod(const string &s)
{
    // s 是原字符串；返回最短整周期长度，空串返回 0。
    if (s.empty())
    {
        return 0;
    }
    auto p = prefix(s);
    int x = s.size() - p.back();
    return (int)s.size() % x == 0 ? x : (int)s.size();
}
```

`prefix(s)` 返回前缀函数表；`kmp(s,t)` 返回 `t` 在 `s` 的全部出现起点（空模式返回 $0, 1, \dots, n$）；`borders(s)` 按长度递增给出全部非空真 border，不包含整串；`minPeriod(s)` 返回最短整周期，非周期串返回 n。四者均为线性时间和线性空间。相比 4.1.001 字符串哈希 的概率匹配，KMP 确定性正确；只关心某后缀与前缀的匹配长度时 4.1.003 扩展 KMP / Z 函数 更直接。周期、border、循环节的汇总应用另见 4.5.006 字符串周期、循环节与前后缀，它直接复用这里的前缀函数。

需要在自动机或数位 DP 中反复读取字符时，可把同一回退过程预计算成 DFA。`KMPAutomaton<A,F>` 的状态 $0 \dots |p|$ 表示当前后缀与模式 `pattern` 的最长匹配长度；完整匹配状态读入新字符时先沿 `pi` 回退，因

此重叠出现不会丢失。字符集必须是连续区间 $[F, F + A)$，空模式也有合法的唯一状态。

```
template <int A = 26, char F = 'a'> struct KMPAutomaton
{
    // pattern 保存模式串，pi 是其前缀函数，go[state][字符] 是 DFA 转移后的匹配长度。
    string pattern;
    vector<int> pi;
    vector<array<int, A>> go;

    KMPAutomaton() = default;

    explicit KMPAutomaton(const string &s)
    {
        // s 是模式串；构造其所有前缀匹配状态的自动机。
        build(s);
    }

    int id(char c) const
    {
        // c 是字符集 [F,F+A) 中的字符；返回从 0 开始的字符编号。
        int x = c - F;
        assert(0 <= x && x < A);
        return x;
    }

    void build(const string &s)
    {
        // s 是模式串；重建前缀函数和状态转移表，无返回值。
        pattern = s;
        int n = pattern.size();
        for (char c : pattern)
        {
            id(c);
        }
        pi.assign(n, 0);
        for (int i = 1, j = 0; i < n; i++)
        {
            while (j && pattern[i] != pattern[j])
            {
                j = pi[j - 1];
            }
            if (pattern[i] == pattern[j])
            {
                j++;
            }
            pi[i] = j;
        }
        go.assign(n + 1, array<int, A>());
        for (int state = 0; state <= n; state++)
        {
            for (int x = 0; x < A; x++)
            {
```

```
                int j = state;
                char c = (char)(F + x);
                while (j && (j == n || pattern[j] != c))
                {
                    j = pi[j - 1];
                }
                if (j < n && pattern[j] == c)
                {
                    j++;
                }
                go[state][x] = j;
            }
        }
    }

    int next(int state, int x) const
    {
        // state 是当前已匹配前缀长度，x 是字符编号；返回读入该字符后的匹配长度。
        assert(0 <= state && state <= (int)pattern.size() && 0 <= x && x < A);
        return go[state][x];
    }

    int next(int state, char c) const
    {
        // state 是当前已匹配前缀长度，c 是字符集内字符；返回读入该字符后的匹配长度。
        return next(state, id(c));
    }

    bool matched(int state) const
    {
        // state 是当前状态；返回是否已经匹配完整模式串。
        return state == (int)pattern.size();
    }
};
```

构造 `KMPAutomaton<26,'a'> autom("aba")` 后，状态从 `0` 开始，逐字符令 `state=autom.next(state,c)`，`autom.matched(state)` 为真时记录一次出现；完整状态仍可继续转移以保留重叠匹配。预处理时间和空间为 $O(A|p|)$，每次转移 $O(1)$；若只扫描一次文本，直接使用本条前面的 `kmp` 可省掉这张表。

4.1.003 扩展 KMP / Z 函数

Z 函数 `z[i]` 记录后缀 $s[i..n)$ 与整串前缀的最长公共前缀长度（约定 `z[0]=n`）。维护一个当前已知能匹配到前缀的最靠右区间 $[l,r)$（Z-box）：若 `i` 落在盒内，`z[i]` 至少是镜像位置 `z[i-l]` 与盒剩余长度 `r-i` 的较小值，再从这个下界向右暴力扩展；每次扩展都推进 `r`，因此总匹配字符数线性。它和 KMP 互补——KMP 看前缀函数，Z 看与整串前缀的对齐，用于模式匹配（把模式接文本前）、周期判断、前缀出现统计。

```
vector<int> zfunc(const string &s)
{
    // s 是原字符串；返回 z 数组，z[i]=lcp(s,s[i..n))。
```

```
    int n = s.size();
    vector<int> z(n);
    if (!n)
    {
        return z;
    }
    z[0] = n;
    for (int i = 1, l = 0, r = 0; i < n; ++i)
    {
        if (i < r)
        {
            z[i] = min(r - i, z[i - l]);
        }
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
        {
            ++z[i];
        }
        if (i + z[i] > r)
        {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

`zfunc(s)` 返回长度 n 的 Z 数组，空串返回空数组，时间与空间均为 $O(n)$。要全部出现位置且确定性时它与 4.1.002 KMP / 前缀函数 等价；大量任意子串比较仍用 4.1.001 字符串哈希。下面四个固定串分别覆盖空串、单字符、整段留在 Z-box 内以及越过右端继续扩展：

```
signed main()
{
    assert(zfunc("").empty());
    assert(zfunc("a") == vector<int>{1});
    assert(zfunc("aaaaa") == vector<int>({5, 4, 3, 2, 1}));
    assert(zfunc("abacaba") == vector<int>({7, 0, 1, 0, 3, 0, 1}));
    cout << "7 0 1 0 3 0 1\n";
    return 0;
}
```

真实输出为 `7 0 1 0 3 0 1`。

4.1.004 Manacher

Manacher 在线性时间求出以每个中心的最长回文半径。奇数长度回文以字符为中心，偶数长度以字符缝为中心，本实现分两趟分别求 `odd[i]`、`even[i]`，各自维护当前最靠右回文区间 $[l, r]$：中心落在区间内时先用关于区间中心的镜像半径当下界，再向外扩展，扩展总量线性。`odd[i]` 对应长度 $2 \cdot odd[i] - 1$ 的回文，`even[i]` 对应中心缝 $(i-1, i)$ 的半径，因此 `isPal` 查询半开区间 $[l, r)$ 时要按长度奇偶分别取 `odd` 或 `even` 中点比较，混用会错。

```cpp
struct Manacher
{
    // odd、even 分别保存奇回文与偶回文半径。
    vector<int> odd, even;

    Manacher(const string &s)
    {
        // s 是要预处理回文半径的原字符串。
        int n = s.size();
        odd.resize(n);
        for (int i = 0, l = 0, r = -1; i < n; i++)
        {
            int k = i > r ? 1 : min(odd[l + r - i], r - i + 1);
            while (i - k >= 0 && i + k < n && s[i - k] == s[i + k])
            {
                k++;
            }
            odd[i] = k--;
            if (i + k > r)
            {
                l = i - k;
                r = i + k;
            }
        }
        even.resize(n);
        for (int i = 0, l = 0, r = -1; i < n; i++)
        {
            int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
            while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k])
            {
                k++;
            }
            even[i] = k--;
            if (i + k > r)
            {
                l = i - k - 1;
                r = i + k;
            }
        }
    }

    // odd[i] 对应长度 2*odd[i]-1；even[i] 对应中心缝 (i-1,i) 的半径。
    bool isPal(int l, int r) const
    {
        // l、r 是原串下标；返回半开区间 [l,r) 是否为回文串。
        assert(0 <= l && l <= r && r <= (int)odd.size()); // 调试检查，可删
        if (l == r)
        {
            return true;
        }
        int n = r - l;
        if (n & 1)
```

```
        {
            return odd[(l + r) / 2] > n / 2;
        }
        return even[(l + r) / 2] >= n / 2;
    }
};
```

构造 `Manacher m(s)` 后 `m.isPal(l,r)` $O(1)$ 判断 $[l,r)$ 是否回文；构造 $O(n)$ 时间与空间。它只回答连续回文，需要枚举本质不同回文串或其出现次数应改用 4.4.001 回文自动机 PAM / 回文树；只做一次「某子串是否回文」的概率判定也可用 4.1.001 字符串哈希 正反哈希比对。

4.1.005 最小表示法

求循环串所有旋转里字典序最小的那个起点，用于环形串的规范化与同构判定。用双指针 `i`、`j` 各代表一个候选起点，`k` 是已确认相等的公共长度：比较 $a[(i+k) \bmod n]$ 与 $a[(j+k) \bmod n]$，相等则 `k++`，否则字典序较大的那个起点直接跳过 `k+1` 个位置（这段区间内任何起点都不可能更优），两指针相遇时把落后的一个再推进一格；`k` 每次失配归零但两指针单调前进，总比较线性。

```
template <class T>
int minRepr(const vector<T> &a)
{
    // a 是循环序列；返回其字典序最小旋转的起点。
    int n = a.size();
    if (!n)
    {
        return 0;
    }
    int i = 0, j = 1, k = 0;
    while (i < n && j < n && k < n)
    {
        T x = a[(i + k) % n], y = a[(j + k) % n];
        if (x == y)
        {
            k++;
            continue;
        }
        if (x > y)
        {
            i += k + 1;
        }
        else
        {
            j += k + 1;
        }
        if (i == j)
        {
            j++;
        }
        k = 0;
```

```
    }
    return min(i, j);
}

template <class T>
vector<T> minRotation(const vector<T> &a)
{
    // a 是循环序列；返回其字典序最小的完整旋转。
    int p = minRepr(a);
    vector<T> b;
    for (int i = 0; i < (int)a.size(); i++)
    {
        b.push_back(a[(p + i) % a.size()]);
    }
    return b;
}
```

`minRepr(a)` 返回最小字典序旋转的起点下标，`minRotation(a)` 返回旋转后的完整序列；求起点 $O(n)$ 时间、$O(1)$ 额外空间，`minRotation` 另需 $O(n)$ 输出。模板参数 `T` 只需支持比较与拷贝，故字符串、整数序列通用。求唯一 Lyndon 分解或用它做后缀比较时见 4.1.006 Lyndon 分解，最小旋转恰是整串旋转的一个 Lyndon 相关副产品。

4.1.006 Lyndon 分解

Lyndon 串是严格小于自己所有真后缀的串；任何串都能唯一分解成字典序非增的若干 Lyndon 串（CFL 分解）。Duval 算法在线扫描：维护当前正在生长的近似 Lyndon 块，用 `k` 指向块内与新字符 `s[j]` 对应比较的位置——`s[k]<s[j]` 说明还能扩展，把 `k` 拉回块首；`s[k]==s[j]` 则 `k` 随 `j` 前进保持周期；一旦 `s[k]>s[j]` 就把已定型的整块（长度 `j-k`）逐段切出，`i` 前移。返回各段左端点，末尾补哨兵 `n`。

```
vector<int> lyndon(const string &s)
{
    // s 是待分解字符串；返回各段左端点，并在末尾附加 s.size()。
    int n = s.size();
    vector<int> a;
    for (int i = 0; i < n;)
    {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j])
        {
            if (s[k] < s[j])
            {
                k = i;
            }
            else
            {
                k++;
            }
            j++;
        }
```

```
        while (i <= k)
        {
            a.push_back(i);
            i += j - k;
        }
    }
    a.push_back(n);
    return a;
}
```

`lyndon(s)` 返回各 Lyndon 段的左端点并以 n 收尾，故第 i 段是 $[a_i, a_{i+1})$；$O(n)$ 时间、$O(n)$ 输出空间。它与 4.1.005 最小表示法 都利用字典序下的周期结构；4.1.007 Runs 使用两次 Lyndon 方向枚举，但维护独立的单调栈。

4.1.007 Runs

一个 run（极大重复）是指数至少为 2 的极大周期区间：其最小周期为 p，区间长度至少为 2p，且向左右都无法继续保持周期 p。当前实现采用两次 Lyndon 方向枚举：单调栈确定候选 Lyndon 根，再用前后缀哈希二分求 LCP/LCS，把根扩展成极大周期区间；第二次把字符序翻转后重复枚举，最后排序去重。

```
struct Run
{
    // p 是最小周期，[l,r) 是极大周期区间；32 位字段用于降低百万规模候选的内存占用。
    int p, l, r;
    auto operator<=>(const Run &) const = default;
};

struct RunHash
{
    // a 是 1 下标字符串值；h、pw 用于单模哈希 LCE。
    int n;
    vector<int> a, h, pw;

    explicit RunHash(const string &s) : n(s.size()), a(n + 1), h(n + 1), pw(n + 1, 1)
    {
        // s 是小写字母串；字符映射为 1..26 存入 a[1..n] 后建立前缀哈希。
        for (int i = 1; i <= n; i++)
        {
            a[i] = s[i - 1] - 'a' + 1;
        }
        build();
    }

    void build()
    {
        // 根据当前 a 建立前缀哈希。
        constexpr int mod = 998244353, bs = 31;
        h[0] = 0;
        pw[0] = 1;
        for (int i = 1; i <= n; i++)
```

```
        {
            pw[i] = 1LL * pw[i - 1] * bs % mod;
            h[i] = (1LL * h[i - 1] * bs + a[i]) % mod;
        }
    }

    int get(int l, int len) const
    {
        // 返回 1 下标区间 a[l..l+len) 的哈希值。
        constexpr int mod = 998244353;
        return (h[l + len - 1] - 1LL * h[l - 1] * pw[len] % mod + mod) % mod;
    }

    int lcp(int x, int y, int up) const
    {
        // 返回从 x、y 开始的最长公共前缀，长度不超过 up。
        if (x > n || y > n || a[x] != a[y])
        {
            return 0;
        }
        int l = 0, r = min({up, n - x + 1, n - y + 1});
        while (l < r)
        {
            int m = (l + r + 1) / 2;
            if (get(x, m) == get(y, m))
            {
                l = m;
            }
            else
            {
                r = m - 1;
            }
        }
        return l;
    }

    int lcs(int x, int y, int up) const
    {
        // 返回以 x、y 结尾的最长公共后缀，长度不超过 up。
        if (x <= 0 || y <= 0 || a[x] != a[y])
        {
            return 0;
        }
        int l = 0, r = min({up, x, y});
        while (l < r)
        {
            int m = (l + r + 1) / 2;
            if (get(x - m + 1, m) == get(y - m + 1, m))
            {
                l = m;
            }
            else
```

```
            {
                r = m - 1;
            }
        }
        return l;
    }

    void flip()
    {
        // 互补字符值，枚举另一种 Lyndon 方向。
        for (int i = 1; i <= n; i++)
        {
            a[i] = 27 - a[i];
        }
        build();
    }

    void getRuns(vector<Run> &ans)
    {
        // 用 Lyndon 单调栈枚举当前方向的全部 Runs。
        vector<int> st(n + 1);
        st[0] = n + 1;
        for (int i = n, top = 0, lt = 0; i >= 1; i--)
        {
            while (top)
            {
                int x = min(st[top] - i, st[top - 1] - st[top]);
                lt = lcp(i, st[top], x);
                if ((lt == x && st[top] - i < st[top - 1] - st[top]) ||
                    (lt < x && a[i + lt] < a[st[top] + lt]))
                {
                    top--;
                    lt = 0;
                }
                else
                {
                    break;
                }
            }
            int j = st[top];
            st[++top] = i;
            int x = lcs(i - 1, j - 1, j - i), y;
            if (x < j - i)
            {
                y = lt + lcp(i + lt, j + lt, n);
                if (x + y >= j - i)
                {
                    ans.push_back({j - i, i - x - 1, j + y - 1});
                }
            }
        }
    }
```

```
};

vector<Run> runs(const string &s)
{
    // s 是原字符串；返回全部极大周期区间及其最小周期。哈希碰撞概率极低。
    RunHash e(s);
    vector<Run> ans;
    e.getRuns(ans);
    e.flip();
    e.getRuns(ans);
    sort(ans.begin(), ans.end());
    ans.erase(unique(ans.begin(), ans.end()), ans.end());
    return ans;
}
```

`runs(s)` 返回全部 run，每个 `Run{p,l,r}` 给出最小周期 p 与半开区间 $[l,r)$，结果按 `(p,l,r)` 排序。单调栈枚举为线性次候选，每次 LCE 由哈希二分完成，时间 $O(n\log n)$、空间 $O(n)$；QOJ 21648 的百万长度提交 2830439 已通过。实现使用模 998244353、底数 31 的单模滚动哈希，因此概率正确，不能抵抗专门构造的碰撞；要求绝对正确时保留旧 Main-Lorentz/Z-LCE 实现或改用确定性 LCE。full 版使用 32 位字段保存字符串长度、周期和端点，打印版按模板规则展开为 `int`。

4.1.008 Karp-Rabin

Karp-Rabin 用滚动哈希筛选与模式等长的窗口，命中后按需逐字符复核。它与 4.1.001 字符串哈希 是同一份代码，这里不重复：直接依赖其中的 `StrHash::match(s,t)` 即得全部匹配起点，构造 $O(n)$、匹配 $O(n+m)$、空间 $O(n+m)$。Mersenne 单模与进程级随机底数仍是概率正确；要求绝对无误时对每个哈希命中的窗口再逐字符比对一次，或改用确定性的 4.1.002 KMP / 前缀函数、4.1.003 扩展 KMP / Z 函数。

4.1.009 Sunday Algorithm

Sunday 是启发式匹配：失配时不看当前窗口，而是看紧贴窗口右侧的下一个文本字符 `s[i+m]`，把模式中该字符最右出现位置对齐过来，一次可能跳过多达 $m+1$ 个位置。预处理 `sh[c]` 存「模式中字符 c 到末尾再加一的距离」，不含则为 $m+1$。它实现简单、随机数据上很快，但最坏仍是 $O(nm)$，不保证线性。

```
vector<int> sunday(string_view s, string_view p)
{
    // s 是文本串，p 是模式串；返回全部匹配起点，按下标递增排列。
    vector<int> ans;
    if (p.empty())
    {
        for (int i = 0; i <= (int)s.size(); i++)
        {
            ans.push_back(i);
        }
        return ans;
    }
```

```
    array<int, 256> sh;
    sh.fill(p.size() + 1);
    for (int i = 0; i < (int)p.size(); i++)
    {
        sh[(unsigned char)p[i]] = p.size() - i;
    }
    for (int i = 0; i + (int)p.size() <= (int)s.size();)
    {
        int j = 0;
        while (j < (int)p.size() && s[i + j] == p[j])
        {
            j++;
        }
        if (j == (int)p.size())
        {
            ans.push_back(i);
        }
        if (i + (int)p.size() == (int)s.size())
        {
            break;
        }
        i += sh[(unsigned char)(s[i + p.size()])];
    }
    return ans;
}
```

`sunday(s,p)` 返回 `p` 在 `s` 的全部匹配起点；空模式用 $O(n+1)$ 时间和输出空间返回所有位置，非空模式最坏 $O(nm)$ 时间、$O(|\Sigma|+occ)$ 空间，其中 occ 是匹配数。需要最坏线性保证时改用 4.1.002 KMP / 前缀函数 或 4.1.003 扩展 KMP / Z 函数；只找首次出现见 4.1.010 `strstr` 类匹配。

4.1.010 `strstr` 类匹配

只需要模式首次出现位置时，短模式直接用 `string::find` 即可。模式较长、字符分布较散，或要拿同一模式搜索多个文本时，可以用 C++17 的 `boyer_moore_horspool_searcher`：它先按模式建立坏字符位移表，再让 `search` 从模式末端比较，失配后一次跳过多个位置。

```
// s 是文本串，t 是模式串；pos 是首次匹配位置，找不到为 -1，空模式为 0。
boyer_moore_horspool_searcher bm(t.begin(), t.end());
auto it = search(s.begin(), s.end(), bm);
int pos = it == s.end() ? -1 : (int)(it - s.begin());
```

这里不能写 `boyer_moore_horspool_searcher<>`：它的首个模板参数没有默认值。C++17 起像代码中那样省略整个 `<>`，由 `t.begin()` 推导迭代器类型；需要显式写出时，普通 `string` 用 `boyer_moore_horspool_searcher<string::iterator>`，`const string` 用 `string::const_iterator`。模式串 `t` 及其迭代器必须在 `bm` 使用期间保持有效，期间不要修改 `t`；同一个 `bm` 可以传给多次 `search` 以复用预处理。该接口要求模式和文本都是随机访问序列，`string`、`vector`、`array` 均满足；`string` 中含 `\0` 也能正常匹配。

设文本长 n、模式长 m，建立搜索器通常需 $O(m)$ 时间和空间，搜索最坏为 $O((n-m+1)m)$；只有失

配能产生较大跳跃时常数才占优，重复字符等数据可能退化。因此单次短模式优先写 `s.find(t)`，需要确定性最坏 $O(n+m)$ 或全部匹配位置时改用 4.1.002 KMP / 前缀函数 或 4.1.003 扩展 KMP / Z 函数，批量子串比较用 4.1.001 字符串哈希。

4.1.011 通配符模式匹配

带单字符通配符（`s` 与 `t` 两侧的 `any` 都匹配任意一个字符）的匹配可化成卷积。把普通小写字母映成 $1 \dots 26$，另用 $0/1$ 掩码标记非通配符位置，则文本窗口与反转模式在位置 i 完全匹配当且仅当 $\sum_j (x_{i+j} - y_j)^2 p_{i+j} q_j = 0$。每项均非负：任一侧通配符都使对应项为 0，普通位置只有字符相等才为 0。展开后用三次卷积计算 $\sum x^2 pq - 2\sum xypq + \sum y^2 pq$。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.1.003 NTT
vector<bool> wildcard(const string &s, string t, char any = '*')
{
    // s、t 只含 a-z 与 any，any 在两侧都表示通配符；返回每个合法起点是否匹配。
    int n = s.size(), m = t.size();
    if (!m)
    {
        return vector<bool>(n + 1, true);
    }
    if (m > n)
    {
        return {};
    }
    reverse(t.begin(), t.end());
    vector<Z> sx(n), sx2(n), sm(n);
    vector<Z> ty(m), ty2(m), tm(m);
    for (int i = 0; i < n; i++)
    {
        if (s[i] != any)
        {
            int x = s[i] - 'a' + 1;
            assert(1 <= x && x <= 26); // 调试检查，可删。
            sx[i] = x;
            sx2[i] = x * x;
            sm[i] = 1;
        }
    }
    for (int i = 0; i < m; i++)
    {
        if (t[i] != any)
        {
            int y = t[i] - 'a' + 1;
            assert(1 <= y && y <= 26); // 调试检查，可删。
            ty[i] = y;
            ty2[i] = y * y;
            tm[i] = 1;
        }
    }
```

```
    auto c0 = conv(move(sx2), move(tm));
    auto c1 = conv(move(sx), move(ty));
    auto c2 = conv(move(sm), move(ty2));
    vector<bool> ans(n - m + 1);
    for (int i = 0; i + m <= n; i++)
    {
        int k = i + m - 1;
        ans[i] = c0[k] - c1[k] * 2 + c2[k] == Z(0);
    }
    return ans;
}
```

`wildcard(s,t,any)` 把两侧的 `any` 都视为通配符，返回长度 $n-m+1$ 的布尔向量，`ans[i]` 表示窗口在起点 i 是否匹配；接口按 Library Checker 原题约束要求普通字符属于 `a-z`。代码直接复用 10.1.003 NTT 的 `Z` 与 `conv`，这里不重复写 NTT。原题 $m \le 524288$，每项至多 25^2，故真实总和至多 $625m \le 327680000 < 998244353$；单模结果不会绕回 0，判定是确定性的。设补齐后的变换长度为 N，三次卷积的时间为 $O(N \log N)$、空间为 $O(N)$；相较原浮点 FFT 版消除了舍入风险，公共 NTT 的四进制蝶形也更适合这里的整数卷积。无通配符时首选 4.1.002 KMP / 前缀函数。

4.1.012 随机单卷积通配符匹配

确定性版展开平方恒等式要做三次卷积；随机版给每个字节独立取正实数 r_c，文本写 r_c，反转模式的普通字符写 $1/r_c$、通配符写 0。若窗口完全匹配，卷积值必为模式普通字符数 k；错配窗口只有在随机比值恰好抵消到 k 时才会误判。`wildcardMC` 从 2^{48} 个离散实数中取权，理想精确运算模型下可把单轮代数碰撞粗略估为 $O(m/2^{48})$；浮点 FFT 的舍入误差不服从该界，阈值附近仍可能误判，独立换 `seed` 重跑可降低随机碰撞但不能消除数值风险。浮点卷积直接复用 10.1.002 FFT 的 `convolution`；需要确定性结果时使用 4.1.011 通配符模式匹配。

```
// 依赖 10.1.002 FFT
vector<bool> wildcardMC(const string &s, string t, unsigned long long seed, char any = '*')
{
    // s 是文本串，t 是模式串，seed 是本轮随机种子；返回每个合法起点是否匹配。
    int n = s.size();
    int m = t.size();
    if (!m)
    {
        return vector<bool>(n + 1, true);
    }
    if (m > n)
    {
        return {};
    }
    mt19937_64 rng(seed);
    constexpr unsigned long long mask = (1ULL << 48) - 1;
    constexpr long double scale = 1ULL << 48;
    array<long double, 256> w;
    for (long double &x : w)
    {
```

```
        x = 1 + ((rng() & mask) + 0.5L) / scale;
    }
    vector<long double> a(n), b(m);
    for (int i = 0; i < n; i++)
    {
        a[i] = w[(unsigned char)s[i]];
    }
    int fixed = 0;
    for (int i = 0; i < m; i++)
    {
        char c = t[m - 1 - i];
        if (c != any)
        {
            b[i] = 1 / w[(unsigned char)c];
            fixed++;
        }
    }
    auto c = convolution(a, b);
    long double tol = 1E-9L * max<int>(1, fixed);
    vector<bool> ans(n - m + 1);
    for (int i = 0; i + m <= n; i++)
    {
        ans[i] = fabsl(c[i + m - 1] - fixed) <= tol;
    }
    return ans;
}
```

`wildcardMC(s,t,seed,any)` 在普通分支只做一次浮点卷积，单项在 $(1/2, 2)$ 内，卷积绝对值小于 $2m$；时间、峰值空间为 $O(N \log N)$、$O(N)$。它比 4.1.011 通配符模式匹配 少两次卷积，代价是同时承担 Monte Carlo 碰撞和平台相关的浮点阈值误判。`seed` 应由调用方控制；调试时固定种子可复现，多轮结果取交能压低代数假阳性，却会放大浮点假阴性的风险。下面的固定例检查空模式、模式过长、任意字节，以及文本中的 `any` 会按普通字节参与：

```
signed main()
{
    assert(wildcardMC("abc", "", 1).size() == 4);
    assert(wildcardMC("a", "aa", 1).empty());
    assert(!wildcardMC("*b", "ab", 1, '*')[0]);
    assert(wildcardMC("*b", "*b", 1, '*')[0]);
    auto a = wildcardMC("ababa", "a*a", 0);
    assert(a.size() == 3 && a[0] && !a[1] && a[2]);
    string x;
    x.push_back((char)0x00);
    x.push_back((char)0xFF);
    assert(wildcardMC(x, x, 7, '*')[0]);
}
```

4.1.013 编辑距离与 Myers 位并行

编辑距离（Levenshtein 距离）是把 a 改成 b 所需的最少单字符插入、删除、替换次数。设 $D_{i,j}$ 为 a 的前 i 个字符与 b 的前 j 个字符的距离，则 $D_{i,0}=i$、$D_{0,j}=j$，

$$D_{i,j}=\min\bigl(D_{i-1,j}+1,\ D_{i,j-1}+1,\ D_{i-1,j-1}+[a_i\neq b_j]\bigr).$$

`editDist` 按行滚动，只保存一行。

Myers 位并行算法按 t 的字符逐列推进 DP，利用同列相邻两行之差 $D_{i,j}-D_{i-1,j}\in\{-1,0,1\}$：用两个位集 `vp`、`vn` 记录差为 $+1$、-1 的行，`eq[c]` 记录模式中等于 c 的位置。每读入一个字符，用常数次字运算（含一次加法进位）算出新一列的差分，并根据最高位的横向差更新最后一行的值 `ans`。模式长不超过 64 时一整列放进一个机器字。

```cpp
int editDist(const string &a, const string &b)
{
    // a、b 是字节串；返回把 a 改成 b 的最少插入、删除、替换次数。
    // row[j] 滚动保存 a 前 i 个字节与 b 前 j 个字节的编辑距离。
    vector<int> row(b.size() + 1);
    iota(row.begin(), row.end(), 0);
    for (int i = 0; i < (int)a.size(); i++)
    {
        int pre = row[0]; // pre 是上一行的 row[j]，即左上角的值。
        row[0] = i + 1;
        for (int j = 0; j < (int)b.size(); j++)
        {
            int old = row[j + 1];
            row[j + 1] = min({row[j + 1] + 1, row[j] + 1, pre + (a[i] != b[j])});
            pre = old;
        }
    }
    return row.back();
}

int myersDist(const string &p, const string &t)
{
    // p 是至多 64 字节的串，t 是任意字节串；返回二者的编辑距离，与 editDist(p,t) 相同。
    int n = p.size();
    if (!n)
    {
        return t.size();
    }
    assert(n <= 64); // 调试检查，可删。
    // eq[c] 的第 i 位表示 p[i]==c；vp、vn 是 DP 当前列竖向差为 +1、-1 的位集。
    array<unsigned long long, 256> eq{};
    for (int i = 0; i < n; i++)
    {
        eq[(unsigned char)p[i]] |= 1ULL << i;
    }
    unsigned long long vp = ~0ULL, vn = 0, top = 1ULL << (n - 1);
    int ans = n; // ans 是当前列最后一行的值，即 p 与 t 已读前缀的编辑距离。
    for (unsigned char c : t)
```

```
    {
        unsigned long long x = eq[c] | vn;
        unsigned long long d = (((eq[c] & vp) + vp) ^ vp) | eq[c] | vn;
        unsigned long long hp = vn | ~(d | vp), hn = vp & d;
        if (hp & top)
        {
            ans++;
        }
        if (hn & top)
        {
            ans--;
        }
        hp = (hp << 1) | 1;
        hn <<= 1;
        vp = hn | ~(x | hp);
        vn = d & hp;
    }
    return ans;
}
```

`editDist(a,b)` 的时间 $O(|a||b|)$、额外空间 $O(|b|)$。`myersDist(p,t)` 要求 $|p| \le 64$，结果与 `editDist(p,t)` 相同，时间 $O(|t|)$、空间 $O(\sigma)$（$\sigma = 256$ 为字节表）。这里的第 0 行是 $D_{0,j} = j$，对应代码中 `hp = (hp << 1) | 1`；改成 `hp <<= 1` 即第 0 行全为 0，读入 t 的第 j 个字符后 `ans` 变成 p 与以 t_j 结尾的某个子串的最小编辑距离，可用于近似匹配。模式更长时要把位集拆成多个字并逐字传递进位，此处未实现。

只允许插入删除时编辑距离是 $|a| + |b| - 2\,\mathrm{LCS}(a, b)$；匹配含通配符时见 4.1.011 通配符模式匹配。下面先核对经典例子，再用固定种子的 64 字节模式和 10^5 字节文本核对结果：

```
signed main()
{
    assert(editDist("kitten", "sitting") == 3 && myersDist("kitten", "sitting") == 3);
    assert(editDist("", "abc") == 3 && myersDist("", "abc") == 3);
    mt19937 rng(20261015);
    string p(64, 'a'), t(100000, 'a');
    for (auto &c : p)
    {
        c = 'a' + rng() % 4;
    }
    for (auto &c : t)
    {
        c = 'a' + rng() % 4;
    }
    cout << myersDist(p, t) << ' ' << editDist(p, t.substr(0, 200)) << '\n';
    // 期望输出：99936 140
}
```

4.2 Trie 与多模式匹配

4.2.001 Trie

Trie 把一组串按字符建成前缀树：根是空前缀，每条边一个字符，一个节点唯一对应从根到它的前缀，公共前缀只存一次。`cnt[u]` 记录有多少次插入经过 `u`（即以该前缀开头的串数），`end[u]` 记录有多少完整串在此结束，二者语义不同不能混用——前缀计数查 `cnt`、整串计数查 `end`。模板参数 `A` 是字符集大小、`S` 是最小字符，插入的每个字符须落在 $[S, S+A)$。本条与第 3 章 3.6.001 都完整讲字符 Trie，二者都有独立阅读价值，故此处按规范保留完整实现。

```
template <int A = 26, char S = 'a'> struct Trie
{
    // ch[u][c] 是节点 u 沿字符 c 的儿子编号，0 表示不存在。
    vector<array<int, A>> ch;
    // cnt[u] 是经过节点 u 的插入次数，end[u] 是在 u 结束的字符串次数。
    vector<int> cnt, end;

    Trie()
    {
        // 建立只含根节点的空 Trie。
        newNode();
    }

    int newNode()
    {
        // 分配并返回一个清空后的节点编号。
        ch.push_back({});
        cnt.push_back(0);
        end.push_back(0);
        return (int)ch.size() - 1;
    }

    int insert(const string &s)
    {
        // s 是待插入字符串；返回它在 Trie 中的终止节点编号。
        int p = 0;
        cnt[p]++;
        for (char c : s)
        {
            int x = c - S;
            assert(0 <= x && x < A); // 调试检查，可删
            if (!ch[p][x])
            {
                ch[p][x] = newNode();
            }
            p = ch[p][x];
            cnt[p]++;
        }
        end[p]++;
        return p;
    }
```

```
42
43      int find(const string &s) const
44      {
45          // s 是查询串；返回它的终止节点编号，不存在时返回 -1。
46          int p = 0;
47          for (char c : s)
48          {
49              int x = c - S;
50              if (x < 0 || x >= A || !ch[p][x])
51              {
52                  return -1;
53              }
54              p = ch[p][x];
55          }
56          return p;
57      }
58
59      int lcp(const string &s) const
60      {
61          // s 是查询串；返回它与任意已插入字符串的最长公共前缀长度。
62          int p = 0, n = 0;
63          for (char c : s)
64          {
65              int x = c - S;
66              if (x < 0 || x >= A || !ch[p][x])
67              {
68                  break;
69              }
70              p = ch[p][x];
71              n++;
72          }
73          return n;
74      }
75
76      int prefixCnt(const string &s) const
77      {
78          // s 是查询前缀；返回以 s 为前缀的已插入字符串数量。
79          int p = find(s);
80          return p < 0 ? 0 : cnt[p];
81      }
82      int stringCnt(const string &s) const
83      {
84          // s 是查询串；返回 s 被完整插入的次数。
85          int p = find(s);
86          return p < 0 ? 0 : end[p];
87      }
88  };
```

构造 `Trie<A,S> tr`；`insert(s)` 返回终点节点，`find(s)` 返回节点或 `-1`，`prefixCnt(s)`、`stringCnt(s)` 分别查前缀出现次数与整串出现次数，`lcp(s)` 查与字典内任意串的最长公共前缀长度。各操作最坏 $O(|s|)$，空间 $O((1+\sum_i |s_i|)A)$。依次插入 `ab`、`abc`、`ab` 后 `stringCnt("ab")==2`、`prefixCnt("ab")==3`、`lcp("abd")==2`、`find("ac")==-1`。要在长文本里同时匹配多个模式需在其上加 fail 指针，见 4.2.002 AC 自动机；判断子

序列而非前缀用 4.2.004 子序列自动机。

4.2.002 AC 自动机

AC 自动机是在 Trie 上补 fail 指针的多模式匹配机：`fail` 指向当前状态所代表串的最长真后缀状态，`nxt[c]` 是补全后的自动机转移（无真实儿子时顺着 fail 走）。BFS 建图时先令根的儿子以根为 fail，再逐层用父亲的 `nxt` 填儿子的 fail 与自身 `nxt`，使一次文本扫描每步 $O(1)$ 转移即可同时命中所有以当前状态结尾、且在其 fail 链上的模式。累计每个模式出现次数时把文本经过的状态在 fail 树上向上求和。

```cpp
template <int A = 26> class AC
{
  public:
    struct Node
    {
        // ch 是 Trie 实边，nxt 是补全失配后的自动机转移。
        array<int, A> ch, nxt;
        // fail 指向当前状态的最长真后缀状态。
        int fail = 0;
        Node()
        {
            // 无参数；Trie 实边置为 -1（不存在），补全转移默认指向根 0。
            ch.fill(-1);
            nxt.fill(0);
        }
    };

  private:
    // t 是状态表，pos 按插入顺序保存各模式终态。
    vector<Node> t{1};
    vector<int> pos;

  public:
    int add(const string &s, char fst = 'a')
    {
        // s 是非空模式串，fst 是字符集起始字节；返回模式终态。
        assert(!s.empty()); // 调试检查，可删
        int u = 0;
        for (char c : s)
        {
            int x = (unsigned char)c - (unsigned char)fst;
            assert(0 <= x && x < A); // 调试检查，可删
            if (t[u].ch[x] == -1)
            {
                t[u].ch[x] = t.size();
                t.push_back({});
            }
            u = t[u].ch[x];
        }
        pos.push_back(u);
        return u;
```

```
    }

    void build()
    {
        // 补全全部失配链接和自动机转移。
        queue<int> q;
        for (int c = 0; c < A; c++)
        {
            int v = t[0].ch[c];
            if (v == -1)
            {
                continue;
            }
            t[0].nxt[c] = v;
            q.push(v);
        }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            t[u].nxt = t[t[u].fail].nxt;
            for (int c = 0; c < A; c++)
            {
                int v = t[u].ch[c];
                if (v == -1)
                {
                    continue;
                }
                t[v].fail = t[u].nxt[c];
                t[u].nxt[c] = v;
                q.push(v);
            }
        }
    }

    int step(int u, int c) const
    {
        // u 是当前状态，c 是 [0,A) 内字符编号；返回下一状态。
        assert(0 <= u && u < (int)t.size()); // 调试检查，可删
        assert(0 <= c && c < A); // 调试检查，可删
        return t[u].nxt[c];
    }
    const vector<Node> &nodes() const
    {
        // 返回按编号排列的状态表。
        return t;
    }
    const vector<int> &ends() const
    {
        // 返回各模式按插入顺序对应的终态。
        return pos;
    }
```

```
};
```

`add(s,fst)` 插入非空模式并返回终态；所有模式须使用同一个 `fst`，字符均落在 $[fst, fst + A)$，并在全部插入后只调用一次 `build()`。`step(u,c)` 用已经离散到 $[0, A)$ 的字符走一步；扫描原始文本遇到域外字符时把状态重置为根。插入总长 L 为 $O(L)$，`build` 的时间与空间均为 $O(SA)$（S 为状态数）。空模式不属于本接口；若题目允许空模式，它在长度 n 的文本中出现 $n + 1$ 次，应在自动机外单独统计。下面用经典 fail 链样例核对四个模式的出现次数：

```
signed main()
{
    AC<26> ac;
    vector<string> pat{"he", "she", "his", "hers"};
    for (const string &s : pat)
    {
        ac.add(s);
    }
    ac.build();
    const auto &nd = ac.nodes();
    vector<int> hit(nd.size());
    int u = 0;
    for (char c : string("ahishers"))
    {
        u = ac.step(u, c - 'a');
        hit[u]++;
    }
    vector<int> cnt(nd.size());
    for (int s = 1; s < (int)nd.size(); s++)
    {
        for (int p = s; p > 0; p = nd[p].fail)
        {
            cnt[p] += hit[s];
        }
    }
    vector<int> ans;
    for (int p : ac.ends())
    {
        ans.push_back(cnt[p]);
    }
    assert(ans == vector<int>({1, 1, 1, 1}));
    cout << "1 1 1 1\n";
    return 0;
}
```

真实输出为 `1 1 1 1`。

4.2.003 AC 自动机 fail 树应用

把每个状态向它的 `fail` 连边（反向）就得到 fail 树：一个状态的所有失配后缀都成为它在树上的祖先。于是「文本扫描经过某状态」这一贡献，对该状态到根路径上的每个模式终态都有效——转成子树求和或 DFS 序上的区间操作，即可离线统计每个模式的出现次数、或做子树加 / 祖先查。

```cpp
// 依赖 4.2.002 AC 自动机
template <int A> vector<vector<int>> failTree(const AC<A> &ac)
{
    // ac 是已经 build 的 AC 自动机；返回 fail 边反向后的树形邻接表。
    const auto &t = ac.nodes();
    vector<vector<int>> g(t.size());
    for (int u = 1; u < (int)t.size(); u++)
    {
        g[t[u].fail].push_back(u);
    }
    return g;
}
```

先对 4.2.002 AC 自动机 调用 `build()`，再 `failTree(ac)` 得反向邻接表，`g[u]` 是 fail 树上 `u` 的孩子；建树 $O(S)$，其后可求 DFS 序做子树加与祖先统计。它只重排 4.2.002 AC 自动机 已算好的 fail 指针，不新建自动机；在线逐字符统计而非离线子树和时直接沿 `fail` 上跳即可。

4.2.004 子序列自动机

子序列自动机为「判断某串是否为文本的子序列」而建：`nxt[i][c]` 存位置 i（含）之后字符 c 第一次出现的下标，从右往左递推一遍即可。匹配时从位置 0 起，对模式每个字符跳到它的下一出现位置再加一，任一步越界即非子序列，查询只与模式长度有关。它按文本下标离线建整张表，和回文小节的 4.4.004 SqAM 是不同结构——后者在线维护、这里离线预处理。

```cpp
template <int A = 26> class SubseqAM
{
    // nxt[i][c] 是位置 i（含）之后字符 c 的首次出现位置。
    vector<array<int, A>> nxt;

  public:
    explicit SubseqAM(const vector<int> &s) : nxt(s.size() + 1)
    {
        // s 是 [0,A) 内的文本序列；从右向左建立跳转表。
        int n = s.size();
        nxt.back().fill(n);
        for (int i = n - 1; i >= 0; --i)
        {
            assert(0 <= s[i] && s[i] < A); // 调试检查，可删
            nxt[i] = nxt[i + 1];
            nxt[i][s[i]] = i;
        }
    }

    bool contains(const vector<int> &t) const
    {
        // t 是 [0,A) 内的模式序列；返回它是否为文本的子序列。
        int p = 0, n = nxt.size() - 1;
        for (int c : t)
        {
```

```
            assert(0 <= c && c < A); // 调试检查，可删
            if (p == n || nxt[p][c] == n)
            {
                return false;
            }
            p = nxt[p][c] + 1;
        }
        return true;
    }
};
```

构造 `SubseqAM<A> am(s)`（`s` 为 $[0, A)$ 的整数序列），`am.contains(t)` 判断 `t` 是否为子序列；构造的时间、空间均为 $O((n+1)A)$，长度 m 查询 $O(m)$。它只判子序列，连续模式匹配用 4.2.002 AC 自动机或后缀结构；在线加字符维护同样功能见 4.4.004 SqAM。

4.3 后缀结构

4.3.001 后缀数组：倍增法

后缀数组 `sa` 按字典序给出全部后缀的起点。倍增法先按首字符给每个后缀一个排名，再反复以「长度 2^k 前缀的排名对 (rk_i, rk_{i+2^k})」为关键字排序并重新编号，k 每轮翻倍，$O(\log n)$ 轮后排名互异即得 `sa`。`saNaive` 是直接对后缀 `sort` 的 $O(n^2 \log n)$ 朴素版，只用于很短输入的内部回退；`saDouble` 是上述倍增，每轮用 `sort` 故 $O(n \log^2 n)$。

```
vector<int> saNaive(const vector<int> &s)
{
    // s 是整数序列；返回按字典序排列的全部后缀起点。
    int n = s.size();
    vector<int> sa(n);
    iota(sa.begin(), sa.end(), 0);
    sort(sa.begin(),
         sa.end(),
         [&](int x, int y)
         {
             while (x < n && y < n && s[x] == s[y])
             {
                 x++;
                 y++;
             }
             return y < n && (x == n || s[x] < s[y]);
         });
    return sa;
}

vector<int> saDouble(const vector<int> &s)
{
    // s 是整数序列；返回倍增法求得的后缀数组。
    int n = s.size();
    vector<int> sa(n), rk(n), nrk(n), v = s;
```

```
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end()), v.end());
    for (int i = 0; i < n; i++)
    {
        rk[i] = lower_bound(v.begin(), v.end(), s[i]) - v.begin();
    }
    iota(sa.begin(), sa.end(), 0);
    for (int k = 1; k < n; k *= 2)
    {
        auto cmp = [&](int x, int y)
        {
            if (rk[x] != rk[y])
            {
                return rk[x] < rk[y];
            }
            int a = x + k < n ? rk[x + k] : -1;
            int b = y + k < n ? rk[y + k] : -1;
            return a < b;
        };
        sort(sa.begin(), sa.end(), cmp);
        nrk[sa[0]] = 0;
        for (int i = 1; i < n; i++)
        {
            nrk[sa[i]] = nrk[sa[i - 1]] + cmp(sa[i - 1], sa[i]);
        }
        swap(rk, nrk);
    }
    return sa;
}
```

`saNaive`、`saDouble` 都接受整数序列返回后缀数组下标。朴素版 $O(n^2 \log n)$、倍增版 $O(n \log^2 n)$、空间 $O(n)$。数据大时换线性的 4.3.002 后缀数组：SA-IS（`suffixArray` 综合入口内部对小串就回退到这两者），求两后缀 LCP 再接 4.3.003 LCP / Height / RMQ 应用。统计本质不同子串与出现次数往往 4.3.004 后缀自动机 SAM 更省心。

4.3.002 后缀数组：SA-IS

SA-IS 用诱导排序在线性时间构造后缀数组。把每个位置按其后缀比右邻后缀小 / 大分成 S 型与 L 型，两个相邻 LMS 位置（L 转 S 的边界）之间的 LMS 子串先被诱导排序、命名，再对命名后的缩短串递归求 SA，最后由已排好的 LMS 顺序两趟诱导（先 L 型从左入桶、再 S 型从右入桶）得到全部后缀顺序。实现必须正确处理哨兵、字符离散化与 LMS 命名，故 `saIs` 对小规模直接退回朴素版。`suffixArray` 是对外入口：字符串版按字节离散，整数版先坐标压缩再调 `saIs`。

```
// 依赖 4.3.001 后缀数组：倍增法
vector<int> saIs(const vector<int> &s, int m)
{
    // s 是整数序列，元素须在 [0,m] 内，m 是最大符号值；返回后缀数组。
    int n = s.size();
    if (n == 0)
```

```
    {
        return {};
    }
    if (n == 1)
    {
        return {0};
    }
    if (n == 2)
    {
        return s[0] < s[1] ? vector<int>{0, 1} : vector<int>{1, 0};
    }
    if (n < 10)
    {
        return saNaive(s);
    }
    if (n < 40)
    {
        return saDouble(s);
    }

    vector<int> sa(n, -1), sl(m + 1), ss(m + 1);
    vector<bool> ls(n);
    for (int i = n - 2; i >= 0; i--)
    {
        ls[i] = s[i] == s[i + 1] ? ls[i + 1] : s[i] < s[i + 1];
    }
    for (int i = 0; i < n; i++)
    {
        if (ls[i])
        {
            sl[s[i] + 1]++;
        }
        else
        {
            ss[s[i]]++;
        }
    }
    for (int i = 0; i <= m; i++)
    {
        ss[i] += sl[i];
        if (i < m)
        {
            sl[i + 1] += ss[i];
        }
    }

    auto induce = [&](const vector<int> &lms)
    {
        fill(sa.begin(), sa.end(), -1);
        vector<int> b = ss;
        for (int x : lms)
        {
```

```
            if (x < n)
            {
                sa[b[s[x]]++] = x;
            }
        }
        b = sl;
        sa[b[s[n - 1]]++] = n - 1;
        for (int i = 0; i < n; i++)
        {
            int x = sa[i];
            if (x > 0 && !ls[x - 1])
            {
                sa[b[s[x - 1]]++] = x - 1;
            }
        }
        b = sl;
        for (int i = n - 1; i >= 0; i--)
        {
            int x = sa[i];
            if (x > 0 && ls[x - 1])
            {
                sa[--b[s[x - 1] + 1]] = x - 1;
            }
        }
    };

    vector<int> id(n + 1, -1), lms;
    for (int i = 1; i < n; i++)
    {
        if (!ls[i - 1] && ls[i])
        {
            id[i] = lms.size();
            lms.push_back(i);
        }
    }
    induce(lms);
    int z = lms.size();
    if (z)
    {
        vector<int> ord;
        for (int x : sa)
        {
            if (x >= 0 && id[x] != -1)
            {
                ord.push_back(x);
            }
        }
        vector<int> ns(z);
        int nm = 0;
        ns[id[ord[0]]] = 0;
        for (int i = 1; i < z; i++)
        {
```

```
            int x = ord[i - 1], y = ord[i];
            int ex = id[x] + 1 < z ? lms[id[x] + 1] : n;
            int ey = id[y] + 1 < z ? lms[id[y] + 1] : n;
            bool same = ex - x == ey - y;
            while (same && x < ex)
            {
                if (s[x] != s[y])
                {
                    same = false;
                }
                x++;
                y++;
            }
            if (same && (x == n || s[x] != s[y]))
            {
                same = false;
            }
            if (!same)
            {
                nm++;
            }
            ns[id[ord[i]]] = nm;
        }
        vector<int> rsa = saIs(ns, nm);
        for (int i = 0; i < z; i++)
        {
            ord[i] = lms[rsa[i]];
        }
        induce(ord);
    }
    return sa;
}

template <class T> vector<int> suffixArray(const vector<T> &s)
{
    // s 是可比较序列；先离散化，再返回按字典序排列的全部后缀起点。
    int n = s.size();
    vector<int> p(n), a(n);
    iota(p.begin(), p.end(), 0);
    sort(p.begin(),
         p.end(),
         [&](int x, int y)
         {
             return s[x] < s[y];
         });
    int m = 0;
    for (int i = 0; i < n; i++)
    {
        if (i && s[p[i - 1]] != s[p[i]])
        {
            m++;
        }
```

```
        a[p[i]] = m;
    }
    return saIs(a, m);
}

vector<int> suffixArray(const string &s)
{
    // s 按无符号字节比较；返回按后缀字典序排列的起点下标。
    vector<int> a;
    a.reserve(s.size());
    for (unsigned char c : s)
    {
        a.push_back(c);
    }
    return saIs(a, 255);
}
```

字符串入口按无符号字节比较，SA-IS 为 $O(n+256)$ 时间、$O(n+256)$ 空间；泛型 `vector` 入口先排序离散化，故为 $O(n\log n)$ 时间、$O(n)$ 空间。得到 `sa` 后用 4.3.003 LCP / Height / RMQ 应用 求任意两后缀 LCP；需要压缩后缀树边才用 4.3.008 隐式后缀树 Ukkonen。下面检查空串、重复字符、一般字符串与无符号字节顺序：

```
signed main()
{
    assert(suffixArray("").empty());
    assert(suffixArray("aaaa") == vector<int>({3, 2, 1, 0}));
    assert(suffixArray("banana") == vector<int>({5, 3, 1, 0, 4, 2}));
    string bytes;
    bytes.push_back((char)0xFF);
    bytes.push_back((char)0x00);
    bytes.push_back((char)0x80);
    assert(suffixArray(bytes) == vector<int>({1, 2, 0}));
}
```

4.3.003 LCP / Height / RMQ 应用

给定后缀数组，Kasai 算法线性求 Height：`ht[i]` 是排名相邻的 `sa[i]`、`sa[i+1]` 两后缀的 LCP。关键性质是删去首字符后 LCP 至多减一，故按原串下标顺序处理、`h` 只减不重置能均摊 $O(n)$。任意两后缀 $s[i..)$、$s[j..)$ 的 LCP 等于它们排名区间内 Height 的最小值，于是对 `ht` 建 ST 表即可 $O(1)$ 回答，`query` 先把下标换成排名再取区间最小。它不构造 SA，`sa` 必须是合法排列、空串不可查。

```
namespace suffix_lcp
{
vector<int> lcpArray(const string &s, const vector<int> &sa)
{
    // s 是原字符串，sa 是全部后缀按字典序排列的起点；返回相邻后缀 LCP，ans[i]=lcp(sa[i],sa[i+1])。
    auto n = (int)s.size();
    assert((int)sa.size() == n); // 调试检查，可删
    vector<int> rk(n), ht(max<int>(0, n - 1));
```

```
    vector<char> seen(n);
    for (int i = 0; i < n; i++)
    {
        assert(0 <= sa[i] && sa[i] < n && !seen[sa[i]]); // 调试检查，可删
        seen[sa[i]] = true;
        rk[sa[i]] = i;
    }
    int h = 0;
    for (int i = 0; i < n; i++)
    {
        if (rk[i] == n - 1)
        {
            h = 0;
            continue;
        }
        auto j = sa[rk[i] + 1];
        while (i + h < n && j + h < n && s[i + h] == s[j + h])
        {
            h++;
        }
        ht[rk[i]] = h;
        if (h)
        {
            h--;
        }
    }
    return ht;
}

struct LcpRMQ
{
    int n = 0;              // n 是字符串长度。
    vector<int> sa;         // sa[r] 是排名 r 的后缀起点。
    vector<int> rk;         // rk[i] 是后缀 s[i..n) 的排名。
    vector<int> ht;         // ht[i] 是 sa[i] 与 sa[i+1] 的最长公共前缀长度。
    vector<int> lg;         // lg[len] 是不超过 len 的最大二次幂指数。
    vector<vector<int>> st; // st[k][i] 是 ht[i..i+2^k) 的最小值。

    LcpRMQ(const string &s, vector<int> order)
        : n((int)s.size()), sa(move(order)), rk(n), ht(lcpArray(s, sa)), lg(n + 1)
    {
        // s 是原字符串，order 是后缀数组；建立 Height 的 ST 表。
        for (int i = 0; i < n; i++)
        {
            rk[sa[i]] = i;
        }
        for (int i = 2; i <= n; i++)
        {
            lg[i] = lg[i / 2] + 1;
        }
        if (n <= 1)
        {
```

```
            return;
        }
        st.assign(lg[n - 1] + 1, {});
        st[0] = ht;
        for (int k = 1; k < (int)st.size(); k++)
        {
            auto len = 1LL << k;
            st[k].resize(n - len);
            for (int i = 0; i + len <= n - 1; i++)
            {
                st[k][i] = min(st[k - 1][i], st[k - 1][i + len / 2]);
            }
        }
    }

    int queryRank(int l, int r) const
    {
        // l、r 是后缀排名且 l<r；返回 sa[l] 与 sa[r] 的最长公共前缀长度。
        assert(0 <= l && l < r && r < n); // 调试检查，可删
        auto k = lg[r - l];
        return min(st[k][l], st[k][r - (1LL << k)]);
    }

    int query(int i, int j) const
    {
        // i、j 是两个后缀起点；返回 s[i..n) 与 s[j..n) 的最长公共前缀长度。
        assert(0 <= i && i < n && 0 <= j && j < n); // 调试检查，可删
        if (i == j)
        {
            return n - i;
        }
        auto x = rk[i];
        auto y = rk[j];
        if (x > y)
        {
            swap(x, y);
        }
        return queryRank(x, y);
    }
};
}
```

实现放在命名空间 `suffix_lcp`。只要 Height 用 `suffix_lcp::lcpArray(s,sa)`；要多次询问就建 `suffix_lcp::LcpRMQ q(s,sa)` 再 `q.query(i,j)` 查后缀 $s[i..n)$ 与 $s[j..n)$ 的 LCP；构造 Height 与 ST 表 $O(n\log n)$ 时间与空间,单次查询 $O(1)$。以 `s="banana"`、`sa={5,3,1,0,4,2}` 为例,Height 为 `{1,3,0,0,2}`，`query(1,3)=3`、`query(0,2)=0`。SA 由 4.3.001 后缀数组：倍增法 或 4.3.002 后缀数组：SA-IS 提供；区间受限的本质不同子串不能套整串公式，见 4.3.010 区间本质不同子串。

4.3.004 后缀自动机 SAM

后缀自动机把一个串的全部子串压进至多 $2n-1$ 个状态：每个状态代表一组 endpos（结束位置集合）相同的子串，这组子串长度连续，`link` 指向其最长真后缀所在的（endpos 更大的）状态。在线加字符 `add`：新建末状态 `q`，沿 `last` 的 `link` 链把没有 `x` 转移的状态都指向 `q`；遇到已有转移 `v` 时，若 `len[v]==len[p]+1` 直接令 `link[q]=v`，否则克隆 `v` 为 `z`（复制转移、`len` 设为 `len[p]+1`）分裂出正确长度区间。不同子串数为 $\sum len[u]-len[link[u]]$；`count` 按 `len` 基数排序后沿 `link` 累加初始 `cnt=1`（只在非克隆末状态）得到每个状态代表子串的出现次数。

```cpp
template <int A = 26, char S = 'a'> struct SAM
{
    struct Node
    {
        array<int, A> next{}; // next[c] 是字符 c 的转移，0 表示不存在（根没有入边）。
        // link 是后缀链接，len 是最长长度，cnt 是当前出现次数，first 是一个最长串的末位置。
        int link = -1, len = 0, cnt = 0, first = -1;
    };

    vector<Node> t{{}};
    // last 是当前整串对应的末状态。
    int last = 0;
    // counted 表示出现次数是否已经沿后缀链接上传。
    bool counted = false;

    SAM() = default;
    SAM(const string &s)
    {
        // s 是原字符串；按顺序插入其全部字符。
        for (char c : s)
        {
            add(c);
        }
    }

    void add(char c)
    {
        // c 是要追加到原串末尾的字符。
        assert(!counted); // 调试检查，可删
        int x = c - S;
        assert(0 <= x && x < A); // 调试检查，可删
        int p = last, q = t.size();
        t.push_back({});
        t[q].len = t[p].len + 1;
        t[q].cnt = 1;
        t[q].first = t[q].len - 1;
        while (p != -1 && !t[p].next[x])
        {
            t[p].next[x] = q;
            p = t[p].link;
        }
        if (p == -1)
```

```
        {
            t[q].link = 0;
        }
        else
        {
            int v = t[p].next[x];
            if (t[v].len == t[p].len + 1)
            {
                t[q].link = v;
            }
            else
            {
                int z = t.size();
                t.push_back(t[v]);
                t[z].len = t[p].len + 1;
                t[z].cnt = 0;
                while (p ≠ -1 && t[p].next[x] == v)
                {
                    t[p].next[x] = z;
                    p = t[p].link;
                }
                t[v].link = t[q].link = z;
            }
        }
        last = q;
    }

    int find(const string &s) const
    {
        // s 是查询串；返回对应状态，不存在或含域外字符时返回 -1。
        int p = 0;
        for (char c : s)
        {
            int x = c - S;
            if (x < 0 || x >= A || !t[p].next[x])
            {
                return -1;
            }
            p = t[p].next[x];
        }
        return p;
    }

    long long distinct() const
    {
        // 返回当前自动机表示的本质不同非空子串数。
        long long ans = 0;
        for (int i = 1; i < (int)t.size(); i++)
        {
            ans += t[i].len - t[t[i].link].len;
        }
        return ans;
```

```
    }

    void count()
    {
        // 把各末状态的计数沿后缀链接上传；重复调用不再二次累计。
        if (counted)
        {
            return;
        }
        int n = 0;
        for (auto &v : t)
        {
            chmax(n, v.len);
        }
        vector<int> c(n + 1), p(t.size());
        for (auto &v : t)
        {
            c[v.len]++;
        }
        partial_sum(c.begin(), c.end(), c.begin());
        for (int i = (int)t.size() - 1; i >= 0; i--)
        {
            p[--c[t[i].len]] = i;
        }
        for (int i = (int)p.size() - 1; i; i--)
        {
            int x = p[i];
            t[t[x].link].cnt += t[x].cnt;
        }
        counted = true;
    }

    int occ(const string &s)
    {
        // s 是查询串；返回它在原串中的出现次数，空串返回原串长度加一。
        if (s.empty())
        {
            return t[last].len + 1;
        }
        int p = find(s);
        if (p < 0)
        {
            return 0;
        }
        count();
        return t[p].cnt;
    }
};

array<int, 4> commonSubstr(const string &a, const string &b)
{
    // a、b 是小写字符串；返回 {a_l,a_r,b_l,b_r} 的一组最长公共子串半开区间。
```

```
    SAM<26, 'a'> sam(a);
    int p = 0, len = 0, best = 0;
    array<int, 4> ans{};
    for (int i = 0; i < (int)b.size(); i++)
    {
        int x = b[i] - 'a';
        while (p && !sam.t[p].next[x])
        {
            p = sam.t[p].link;
            len = min(len, sam.t[p].len);
        }
        if (sam.t[p].next[x])
        {
            p = sam.t[p].next[x];
            len++;
        }
        else
        {
            len = 0;
        }
        if (len > best)
        {
            best = len;
            int end = sam.t[p].first;
            ans = {end - len + 1, end + 1, i - len + 1, i + 1};
        }
    }
    return ans;
}
```

构造 `SAM<A,S> sam(s)`；`find(s)` 定位状态，`distinct()` 返回本质不同子串数，`occ(t)` 返回 `t` 在原串中的出现次数，空串按出现位置定义返回 $n+1$。`commonSubstr(a,b)` 返回一组 `{a_l,a_r,b_l,b_r}`，两段半开区间长度相等且为最长公共子串长度；无公共字符时返回全零区间。首次非空查询触发一次 $O(n)$ 的 `count()`；调用 `count()` 或首次查询非空串后不得再 `add`。构造时间和空间均为 $O(nA)$；非空串的状态数不超过 $2n-1$，空串只有根状态；查长度 m 的串需 $O(m)$。最长公共子串在扫描第二串时也只沿后缀链接回退，故为 $O(|a|+|b|)$ 时间与 $O(|a|)$ 空间。下面用会产生状态分裂的 `ababa` 直接核对本质不同子串数和各节点次数：

```
signed main()
{
    SAM<3, 'a'> sam("ababa");
    assert(sam.distinct() == 9);
    assert(sam.occ("a") == 3);
    assert(sam.occ("b") == 2);
    assert(sam.occ("ab") == 2);
    assert(sam.occ("aba") == 2);
    assert(sam.occ("bab") == 1);
    assert(sam.occ("c") == 0);
    assert(sam.occ("") == 6);
    cout << "9 3 2 2 2 1 0 6\n";
    return 0;
```

```
}
```

真实输出为 `9 3 2 2 2 1 0 6`。

4.3.005 广义 SAM / ExSAM

广义 SAM 把多条串的全部子串压进一个自动机。离线版先把所有串插进公共 Trie（`pass` 记录每个 Trie 节点被多少串经过），再按 BFS 顺序对 Trie 的每条边调用 `extend(p,c)` 扩展 SAM：相同前缀天然共享状态，且不依赖插入顺序。`extend` 处理已有转移时同样按 `len[q]==len[p]+1` 决定直接复用还是克隆分裂。建完后把 Trie 节点的 `pass` 加到对应 SAM 状态，再按 `len` 降序沿 `link` 上推，得到每个子串跨全部串的出现次数。

```
template <int A = 26, char B = 'a'> struct ExSAM
{
    struct Node
    {
        array<int, A> ch{}; // ch[c] 是 SAM 中字符 c 的转移，0 表示不存在。
        // link 是后缀链接，len 是该状态的最长长度。
        int link = -1, len = 0;
        // cnt 是该状态所代表子串在全部输入串中的出现次数。
        long long cnt = 0;
    };

    struct TrieNode
    {
        array<int, A> ch{}; // ch[c] 是 Trie 中字符 c 的儿子，0 表示不存在。
        // pass 是经过该 Trie 节点的输入串数，重复插入重复计算。
        long long pass = 0;
    };

    vector<Node> t{{}};
    vector<TrieNode> tr{{}};
    // done 记录是否已经 build，emptyCnt 是空串在全部输入串中的出现位置数。
    bool done = false;
    long long emptyCnt = 0;

    void insert(const string &s)
    {
        // s 是待加入集合的整条字符串；相邻字符串之间不产生跨边界子串。
        assert(!done); // 调试检查，可删
        emptyCnt += (int)s.size() + 1;
        int u = 0;
        for (char c : s)
        {
            int x = c - B;
            assert(0 <= x && x < A); // 调试检查，可删
            if (!tr[u].ch[x])
            {
                tr[u].ch[x] = tr.size();
                tr.push_back({});
```

```
            }
            u = tr[u].ch[x];
            tr[u].pass++;
        }
    }

    int extend(int p, int c)
    {
        // p 是当前 SAM 状态，c 是离散字符；返回扩展后的状态。
        int q = t[p].ch[c];
        if (q)
        {
            if (t[q].len == t[p].len + 1)
            {
                return q;
            }
            int nq = t.size();
            t.push_back(t[q]);
            t[nq].len = t[p].len + 1;
            t[nq].cnt = 0;
            while (p != -1 && t[p].ch[c] == q)
            {
                t[p].ch[c] = nq;
                p = t[p].link;
            }
            t[q].link = nq;
            return nq;
        }

        int u = t.size();
        t.push_back({});
        t[u].len = t[p].len + 1;
        while (p != -1 && !t[p].ch[c])
        {
            t[p].ch[c] = u;
            p = t[p].link;
        }
        if (p == -1)
        {
            t[u].link = 0;
            return u;
        }
        q = t[p].ch[c];
        if (t[q].len == t[p].len + 1)
        {
            t[u].link = q;
            return u;
        }
        int nq = t.size();
        t.push_back(t[q]);
        t[nq].len = t[p].len + 1;
        t[nq].cnt = 0;
```

```
        while (p != -1 && t[p].ch[c] == q)
        {
            t[p].ch[c] = nq;
            p = t[p].link;
        }
        t[q].link = t[u].link = nq;
        return u;
    }

    void build()
    {
        // 从公共 Trie 构造广义 SAM，并上传各状态的跨串出现次数。
        assert(!done); // 调试检查，可删
        done = true;
        vector<int> pos(tr.size());
        queue<int> q;
        q.push(0);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int c = 0; c < A; c++)
            {
                int v = tr[u].ch[c];
                if (!v)
                {
                    continue;
                }
                pos[v] = extend(pos[u], c);
                t[pos[v]].cnt += tr[v].pass;
                q.push(v);
            }
        }
        int mx = 0;
        for (const Node &v : t)
        {
            mx = max(mx, v.len);
        }
        vector<int> c(mx + 1), p(t.size());
        for (const Node &v : t)
        {
            c[v.len]++;
        }
        partial_sum(c.begin(), c.end(), c.begin());
        for (int u = (int)t.size() - 1; u >= 0; u--)
        {
            p[--c[t[u].len]] = u;
        }
        for (int i = (int)p.size() - 1; i > 0; i--)
        {
            int u = p[i];
            if (t[u].link >= 0)
```

```
            {
                t[t[u].link].cnt += t[u].cnt;
            }
        }
    }

    int find(const string &s) const
    {
        // s 是查询串；返回对应状态，不存在或含域外字符时返回 -1。
        int u = 0;
        for (char c : s)
        {
            int x = c - B;
            if (x < 0 || x >= A)
            {
                return -1;
            }
            u = t[u].ch[x];
            if (!u)
            {
                return -1;
            }
        }
        return u;
    }

    long long count(const string &s) const
    {
        // s 是查询串；返回它在全部输入串中的出现次数，空串按各串分别计位置。
        assert(done); // 调试检查，可删
        if (s.empty())
        {
            return emptyCnt;
        }
        int u = find(s);
        return u < 0 ? 0 : t[u].cnt;
    }

    long long distinct() const
    {
        // 返回全部输入串并集中的本质不同非空子串数。
        assert(done); // 调试检查，可删
        long long ans = 0;
        for (int i = 1; i < t.size(); i++)
        {
            ans += t[i].len - t[t[i].link].len;
        }
        return ans;
    }
};
```

逐串 `insert(s)` 建 Trie，只能调用一次 `build()`；之后 `find`、`count(s)` 和 `distinct()` 可查。`count("")`

返回 $\sum_i(|s_i|+1)$，域外查询字符直接视为不存在。按长度计数排序上传状态后，总长 L 的构造时间和空间均为 $O((L+1)A)$，状态数不超过 $2L+1$，单次查询为 $O(|s|)$。要边到达边在线加串而非先收齐用 4.3.006 在线广义 SAM；单串场景直接 4.3.004 后缀自动机 SAM。

4.3.006 在线广义 SAM

在线广义 SAM 不预先收集全部文本：每来一条新串就从根开始逐字符 `extend`。与离线版的区别只在于没有 Trie 前置——`extend(p,c)` 若发现转移已存在但 `len[q]≠len[p]+1` 就克隆分裂到正确长度并沿 `link` 改边，从而在任意插入顺序下都维持广义 SAM 的正确结构，适合字典串陆续到达、需要随时查询的场景。

```cpp
template <int A = 26, char B = 'a'> struct OnlineExSAM
{
    struct Node
    {
        // ch 是字符转移，link 是后缀链接，len 是该状态的最长长度。
        array<int, A> ch{};
        int link = -1, len = 0;
    };

    // tr 是状态池。
    vector<Node> tr{{}};

    int extend(int p, int c)
    {
        // p 是当前状态，c 是离散字符；返回扩展后的状态。
        int q = tr[p].ch[c];
        if (q)
        {
            if (tr[q].len == tr[p].len + 1)
            {
                return q;
            }
            int nq = tr.size();
            tr.push_back(tr[q]);
            tr[nq].len = tr[p].len + 1;
            while (p != -1 && tr[p].ch[c] == q)
            {
                tr[p].ch[c] = nq;
                p = tr[p].link;
            }
            tr[q].link = nq;
            return nq;
        }

        int u = tr.size();
        tr.push_back({});
        tr[u].len = tr[p].len + 1;
        while (p != -1 && !tr[p].ch[c])
        {
            tr[p].ch[c] = u;
```

```
            p = tr[p].link;
        }
        if (p == -1)
        {
            tr[u].link = 0;
            return u;
        }
        q = tr[p].ch[c];
        if (tr[q].len == tr[p].len + 1)
        {
            tr[u].link = q;
            return u;
        }
        int nq = tr.size();
        tr.push_back(tr[q]);
        tr[nq].len = tr[p].len + 1;
        while (p != -1 && tr[p].ch[c] == q)
        {
            tr[p].ch[c] = nq;
            p = tr[p].link;
        }
        tr[q].link = tr[u].link = nq;
        return u;
    }

    void insert(const string &s)
    {
        // s 是新字符串；每条串从根开始，不产生跨边界子串。
        int p = 0;
        for (char c : s)
        {
            int x = c - B;
            assert(0 <= x && x < A); // 调试检查，可删
            p = extend(p, x);
        }
    }

    int find(const string &s) const
    {
        // s 是查询串；返回对应状态，不存在或含域外字符时返回 -1。
        int p = 0;
        for (char c : s)
        {
            int x = c - B;
            if (x < 0 || x >= A)
            {
                return -1;
            }
            p = tr[p].ch[x];
            if (!p)
            {
                return -1;
```

```
            }
        }
        return p;
    }

    long long distinct() const
    {
        // 返回当前所有输入串的本质不同非空子串数。
        long long ans = 0;
        for (int p = 1; p < (int)tr.size(); p++)
        {
            ans += tr[p].len - tr[tr[p].link].len;
        }
        return ans;
    }
};
```

构造 `OnlineExSAM<> sam`，每条新串 `insert(s)`，`distinct()` 随时返回当前串集合的本质不同子串总数；当前接口不分别记录各串出现次数。设总长 L，构造和空间均为 $O((L+1)A)$，状态数 $\le 2L+1$，`find` 为 $O(|s|)$。依次插入 `ababa`、`bcb` 后子串并集为 13 个，`distinct()==13`、`find("aba")>=0`、`find("ac")==-1`。需要跨串出现次数用离线的 4.3.005 广义 SAM / ExSAM。

4.3.007 SAMSA / CDAWG

把 SAM 转移 DAG 中入度、出度都为 1 的非关键状态压进边标签，得到紧凑的 C-SAM 视图：保留根以及入度或出度不为 1 的状态，每条保留边沿唯一出链走到下一个关键状态、把途经字符拼成标签。它减少显式节点而仍能判子串——SAM 路径的任意前缀都是合法子串，故查询允许停在标签中间。注意若题目要求接受全部后缀的严格 CDAWG，还须另标后缀终止态，本条只做子串判定视图。

```
// 依赖 4.3.004 后缀自动机 SAM
template <int A = 26, char B = 'a'> struct CSAM
{
    struct Edge
    {
        // to 是目标关键状态，s 是压缩路径的非空标签。
        int to = 0;
        string s;
    };

    // g 是压缩后的 DAG；id 把 SAM 状态映射到关键状态，非关键状态为 -1。
    vector<vector<Edge>> g;
    vector<int> id;

    CSAM(const SAM<A, B> &sam)
    {
        // sam 是已经建好的后缀自动机；压缩入度、出度均为 1 的路径。
        int n = sam.t.size();
        vector<int> in(n), out(n);
        for (int u = 0; u < n; u++)
```

```
        {
            for (int c = 0; c < A; c++)
            {
                int v = sam.t[u].next[c];
                if (v)
                {
                    in[v]++, out[u]++;
                }
            }
        }
        id.assign(n, -1);
        for (int u = 0; u < n; u++)
        {
            if (u == 0 || in[u] != 1 || out[u] != 1)
            {
                id[u] = g.size();
                g.push_back({});
            }
        }
        for (int u = 0; u < n; u++)
        {
            if (id[u] < 0)
            {
                continue;
            }
            for (int c = 0; c < A; c++)
            {
                int v = sam.t[u].next[c];
                if (!v)
                {
                    continue;
                }
                string s(1, (char)(B + c));
                while (id[v] < 0)
                {
                    for (int d = 0; d < A; d++)
                    {
                        int w = sam.t[v].next[d];
                        if (w)
                        {
                            s += (char)(B + d);
                            v = w;
                            break;
                        }
                    }
                }
                g[id[u]].push_back({id[v], move(s)});
            }
        }
    }

    bool contains(const string &t) const
```

```
    {
        // t 是查询串；返回它是否对应根出发的一条路径前缀，空串返回 true。
        int u = 0, p = 0;
        while (p < (int)t.size())
        {
            const Edge *e = nullptr;
            for (const auto &cur : g[u])
            {
                if (cur.s[0] == t[p])
                {
                    e = &cur;
                    break;
                }
            }
            if (!e)
            {
                return false;
            }
            for (char c : e->s)
            {
                if (p == (int)t.size())
                {
                    return true;
                }
                if (t[p++] != c)
                {
                    return false;
                }
            }
            u = e->to;
        }
        return true;
    }
};
```

先 `SAM<> sam(s)` 再 `CSAM<> cs(sam)`，`cs.contains(t)` 判断 `t` 是否为 `s` 的子串；边标签可能多字符，匹配时必须逐字符核对而非只看首字符。构造与 4.3.004 后缀自动机 SAM 同量级，时间、空间均为 $O((n+1)A)$。原串 `ababa` 上 `contains("")`、`contains("aba")`、`contains("bab")` 为真，`contains("abb")`、`contains("abac")` 为假。只判子串不需压缩时直接用 4.3.004 后缀自动机 SAM 的 `find`。

4.3.008 隐式后缀树 Ukkonen

Ukkonen 在线构造压缩后缀 Trie：边用原串区间 $[left, right)$ 表示，逐字符加入时维护活动点，借后缀链接从一个后缀跳到下一个，内部节点只在边分裂时创建。直接传入原串得到隐式后缀树，部分后缀会结束在边内而没有独立叶子；若每个后缀都要对应显式叶子，先在原串末尾追加一个未出现且小于原字母表所有字符的哨兵。

```
class SuffixTree
{
```

```
public:
  struct Node
  {
      // [left,right) 是父边在原串中的标签区间，parent 是父节点，suffixLink 是后缀链接。
      int left = 0, right = 0, parent = -1, suffixLink = -1;
      // children 把出边首字符映射到儿子节点。
      map<char, int> children;
      int length() const
      {
          // 返回当前压缩边长度。
          return right - left;
      }
  };

private:
  struct State
  {
      // node 是当前边的下端节点，position 是从该边起点走过的字符数。
      int node, position;
  };

  // text 是建树所用原串，nodes 是后缀树节点池。
  string text;
  vector<Node> nodes;
  State active{0, 0};

  State go(State state, int left, int right)
  {
      // state 是活动点，[left,right) 是待匹配文本；返回匹配后的活动点，失败返回 {-1,-1}。
      while (left < right)
      {
          if (state.position == nodes[state.node].length())
          {
              auto it = nodes[state.node].children.find(text[left]);
              if (it == nodes[state.node].children.end())
              {
                  return {-1, -1};
              }
              state = {it->second, 0};
          }
          else
          {
              if (text[nodes[state.node].left + state.position] != text[left])
              {
                  return {-1, -1};
              }
              int length = min(right - left, nodes[state.node].length() - state.position);
              state.position += length;
              left += length;
          }
      }
      return state;
```

```
    }

    int split(State state)
    {
        // state 是活动点；若它在边内则分裂该边，返回活动点对应的显式节点。
        if (state.position == nodes[state.node].length())
        {
            return state.node;
        }
        if (!state.position)
        {
            return nodes[state.node].parent;
        }
        Node original = nodes[state.node];
        int middle = nodes.size();
        nodes.push_back({original.left, original.left + state.position, original.parent, -1,
↪   {}});
        nodes[original.parent].children[text[original.left]] = middle;
        nodes[middle].children[text[original.left + state.position]] = state.node;
        nodes[state.node].parent = middle;
        nodes[state.node].left += state.position;
        return middle;
    }

    int suffixLink(int node)
    {
        // node 是显式后缀树节点；惰性计算并返回它的后缀链接。
        if (nodes[node].suffixLink != -1)
        {
            return nodes[node].suffixLink;
        }
        if (nodes[node].parent == -1)
        {
            return 0;
        }
        int parLink = suffixLink(nodes[node].parent);
        int skip = nodes[node].parent == 0;
        return nodes[node].suffixLink =
                    split(go({parLink, nodes[parLink].length()}, nodes[node].left + skip,
↪   nodes[node].right));
    }

    void extend(int position)
    {
        // position 是本次追加字符在 text 中的下标；补齐所有新后缀。
        while (true)
        {
            State next = go(active, position, position + 1);
            if (next.node != -1)
            {
                active = next;
                return;
```

```
            }
            int middle = split(active), leaf = nodes.size();
            nodes.push_back({position, (int)text.size(), middle, -1, {}});
            nodes[middle].children[text[position]] = leaf;
            active.node = suffixLink(middle);
            active.position = nodes[active.node].length();
            if (!middle)
            {
                break;
            }
        }
    }

  public:
    // 需要显式叶子时，在末尾加入一个小于字母表中所有字符的哨兵。
    explicit SuffixTree(string text) : text(move(text))
    {
        // text 是原字符串；按顺序完成 Ukkonen 在线构造。
        nodes.reserve(this->text.size() * 2 + 1);
        nodes.push_back({0, 0, -1, 0, {}});
        for (int i = 0; i < this->text.size(); ++i)
        {
            extend(i);
        }
    }

    const vector<Node> &getNodes() const
    {
        // 返回后缀树节点池的只读引用，顺序与节点编号一致。
        return nodes;
    }
    const string &getText() const
    {
        // 返回建树所用的完整文本。
        return text;
    }
};
```

构造 `SuffixTree st(s)` 得到隐式树；若 `s` 不含 `\0`，可用 `SuffixTree st(s + '\0')` 得到每个后缀均有独立叶子的显式树。`getNodes()`、`getText()` 分别返回节点池与实际建树文本，边 $[left, right)$ 与 `children` 描述压缩边；字符集大小为 A 时构造摊还 $O(n\log(A+1)+1)$，空间 $O(n+1)$。只需统计子串时，4.3.004 后缀自动机 SAM 更易写。

4.3.009 后缀平衡树（前插、回滚与动态排名）

后缀平衡树维护只在串首加入字符的全部后缀。把当前串反向保存为 `r` 后，每个后缀对应 `r` 的一个前缀终点；继续前插只会在 `r` 末尾追加字符，旧前缀代表的字符串不变，因而旧结点之间的字典序不会改变。Treap 用反向前缀的 LCP 比较键，子树大小支持排名和第 k 后缀；新后缀只与排序后的前驱、后继比较 LCP，即可累计本质不同子串数。哈希采用自然溢出的 64 位多项式哈希，是速度优先的 Monte Carlo 实现。

```cpp
struct SuffixBST
{
    struct Node
    {
        int l = 0;                  // l 保存 Treap 左儿子编号。
        int r = 0;                  // r 保存 Treap 右儿子编号。
        int sz = 1;                 // sz 保存子树结点数。
        int key = 0;                // key 是反向串前缀终点，也是后缀长度。
        unsigned long long pri = 0; // pri 保存固定随机优先级。
    };

    static constexpr unsigned long long base = 11995408973635179863ULL; // base 是多项式哈希的基数，哈希在 2^64 下自然溢出。
    string r;                       // r 保存当前字符串的反串。
    vector<unsigned long long> h{0};  // h 保存 r 的 64 位前缀哈希。
    vector<unsigned long long> pw{1}; // pw[i] 保存 base 的 i 次幂。
    vector<Node> t{{}};             // t[0] 是空结点，其余结点编号等于 key。
    vector<int> ways{0};            // ways[n] 保存长度为 n 时的不同子串数。
    int root = 0;                   // root 保存 Treap 根编号。

    static unsigned long long priority(unsigned long long x)
    {
        // x 是后缀长度；返回 SplitMix64 生成的固定优先级。
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }

    int size(int u) const
    {
        // u 是 Treap 结点编号；返回其子树大小，空结点返回 0。
        return u ? t[u].sz : 0;
    }

    void pull(int u)
    {
        // u 是非空结点；由两个儿子重算子树大小。
        t[u].sz = size(t[u].l) + size(t[u].r) + 1;
    }

    unsigned long long hash(int l, int rr) const
    {
        // l、rr 是 r 的半开区间端点；返回 r[l,rr) 的 64 位哈希。
        return h[rr] - h[l] * pw[rr - l];
    }

    int lcpKey(int a, int b) const
    {
        // a、b 是两个反向前缀终点；返回对应原串后缀的 LCP 长度。
        int l = 0, rr = min(a, b) + 1;
        while (l + 1 < rr)
```

```
        {
            int m = (l + rr) / 2;
            if (hash(a - m, a) == hash(b - m, b))
            {
                l = m;
            }
            else
            {
                rr = m;
            }
        }
        return l;
    }

    bool lessKey(int a, int b) const
    {
        // a、b 是后缀长度；返回长度 a 对应后缀是否字典序更小。
        if (a == b)
        {
            return false;
        }
        int p = lcpKey(a, b);
        if (p == min(a, b))
        {
            return a < b;
        }
        return (unsigned char)r[a - p - 1] < (unsigned char)r[b - p - 1];
    }

    void split(int u, int key, int &x, int &y)
    {
        // u 是原树，key 是分界后缀；x、y 分别接收小于 key 和不小于 key 的树。
        if (!u)
        {
            x = y = 0;
        }
        else if (lessKey(t[u].key, key))
        {
            x = u;
            split(t[u].r, key, t[u].r, y);
            pull(u);
        }
        else
        {
            y = u;
            split(t[u].l, key, x, t[u].l);
            pull(u);
        }
    }

    int merge(int x, int y)
    {
```

```
        // x 的全部键小于 y；返回合并后的 Treap 根。
        if (!x || !y)
        {
            return x | y;
        }
        if (t[x].pri > t[y].pri)
        {
            t[x].r = merge(t[x].r, y);
            pull(x);
            return x;
        }
        t[y].l = merge(x, t[y].l);
        pull(y);
        return y;
    }

    void insert(int u)
    {
        // u 是尚未入树的新结点；按其 key 插入 Treap。
        int x, y;
        split(root, t[u].key, x, y);
        root = merge(merge(x, u), y);
    }

    int erase(int u, int key)
    {
        // u 是当前根，key 是已存在后缀；删除后返回新根。
        if (t[u].key == key)
        {
            return merge(t[u].l, t[u].r);
        }
        if (lessKey(key, t[u].key))
        {
            t[u].l = erase(t[u].l, key);
        }
        else
        {
            t[u].r = erase(t[u].r, key);
        }
        pull(u);
        return u;
    }

    pair<int, int> neighbors(int key) const
    {
        // key 是尚未入树的后缀；返回其前驱、后继的后缀长度，不存在时为 0。
        int u = root, pre = 0, suc = 0;
        while (u)
        {
            if (lessKey(t[u].key, key))
            {
                pre = t[u].key;
```

```
156                 u = t[u].r;
157             }
158             else
159             {
160                 suc = t[u].key;
161                 u = t[u].l;
162             }
163         }
164         return {pre, suc};
165     }
166 
167     void pushFront(char c)
168     {
169         // c 是新字符；把 c 加到当前串首，并插入由整个新串形成的后缀。
170         r.push_back(c);
171         h.push_back(h.back() * base + (unsigned char)c + 1);
172         pw.push_back(pw.back() * base);
173         int key = r.size();
174         auto [pre, suc] = neighbors(key);
175         int same = 0;
176         if (pre)
177         {
178             same = max(same, lcpKey(key, pre));
179         }
180         if (suc)
181         {
182             same = max(same, lcpKey(key, suc));
183         }
184         ways.push_back(ways.back() + key - same);
185         t.push_back({0, 0, 1, key, priority(key)});
186         insert(key);
187     }
188 
189     void popFront()
190     {
191         // 无参数；删除最近一次加入串首的字符及其完整后缀。
192         assert(!r.empty()); // 调试检查，可删
193         int key = r.size();
194         root = erase(root, key);
195         r.pop_back();
196         h.pop_back();
197         pw.pop_back();
198         ways.pop_back();
199         t.pop_back();
200     }
201 
202     int rank(int i) const
203     {
204         // i 是当前串的后缀起点；返回该后缀从 0 开始的字典序排名。
205         int key = (int)r.size() - i;
206         assert(1 <= key && key <= (int)r.size()); // 调试检查，可删
207         int u = root, ans = 0;
```

```
        while (t[u].key ≠ key)
        {
            if (lessKey(key, t[u].key))
            {
                u = t[u].l;
            }
            else
            {
                ans += size(t[u].l) + 1;
                u = t[u].r;
            }
        }
        return ans + size(t[u].l);
    }

    int kth(int k) const
    {
        // k 是从 0 开始的字典序排名；返回当前串中对应后缀的起点。
        assert(0 <= k && k < size(root)); // 调试检查，可删
        int u = root;
        while (true)
        {
            int z = size(t[u].l);
            if (k < z)
            {
                u = t[u].l;
            }
            else if (k == z)
            {
                return (int)r.size() - t[u].key;
            }
            else
            {
                k -= z + 1;
                u = t[u].r;
            }
        }
    }

    int distinct() const
    {
        // 无参数；返回当前串本质不同子串个数。
        return ways.back();
    }
};
```

`pushFront(c)`、`popFront()`、`rank(i)` 和 `kth(k)` 的期望时间均为 $O(\log^2 n)$，其中一次 Treap 路径含 $O(\log n)$ 次比较，每次比较二分 LCP；`distinct()` 为 $O(1)$，总空间 $O(n)$。只允许撤销最近一次前插，不能删除中间后缀；64 位哈希存在极低碰撞概率，需要确定性结果时改用动态 LCP 结构。静态字符串仍优先使用 4.3.002 后缀数组：SA-IS。

```cpp
signed main()
{
    SuffixBST s;
    for (char c : string("ananab"))
    {
        s.pushFront(c);
    }
    vector<int> sa{5, 3, 1, 0, 4, 2};
    for (int k = 0; k < 6; k++)
    {
        assert(s.kth(k) == sa[k]);
        assert(s.rank(sa[k]) == k);
    }
    assert(s.distinct() == 15);
}
```

4.3.010 区间本质不同子串

限定下标区间 $[l, r)$ 内的本质不同子串数不能直接套整串 SAM 的 $\sum len - len_{link}$。本条给出最直接的做法：把子串 `s[l,r)` 取出，对它单独建后缀数组与 Height，答案就是所有长度之和 $\frac{L(L+1)}{2}$ 减去相邻 Height 之和。它对单次询问简单可靠；大量区间询问才需要持久化或离线的可持久化 SAM / SA 结构。

```cpp
// 依赖 3.3.001 单点修改线段树与摘要二分
// 依赖 3.3.002 懒标记线段树
// 依赖 4.3.003 LCP / Height / RMQ 应用
// 依赖 4.3.004 后缀自动机 SAM
inline long long subDistinct(const string &s, int l, int r)
{
    // s 是原字符串，l、r 是半开端点；返回 s[l,r) 的本质不同子串数量。
    assert(0 <= l && l <= r && r <= (int)s.size()); // 调试检查，可删
    string t = s.substr(l, r - l);
    auto sa = suffixArray(t);
    auto h = suffix_lcp::lcpArray(t, sa);
    long long n = t.size();
    return n * (n + 1) / 2 - accumulate(h.begin(), h.end(), 0LL);
}

struct RangeSubMax
{
    // n、z 是叶子数和补齐后的二次幂；tr 保存区间最大值。
    int n, z = 1;
    vector<int> tr;

    RangeSubMax(int n) : n(n)
    {
        // n 是叶子数；初始各位置均为 0。
        while (z < n)
        {
            z <<= 1;
        }
```

```
        tr.assign(2 * z, 0);
    }

    void setMax(int p, int v)
    {
        // p 是位置，v 是候选值；执行单点取最大。
        for (p += z, tr[p] = max(tr[p], v); p >>= 1;)
        {
            tr[p] = max(tr[2 * p], tr[2 * p + 1]);
        }
    }

    int query(int l, int r) const
    {
        // l、r 是半开端点；返回 [l,r) 的最大值。
        int ans = 0;
        for (l += z, r += z; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1) ans = max(ans, tr[l++]);
            if (r & 1) ans = max(ans, tr[--r]);
        }
        return ans;
    }
};

inline vector<int> rangeSubs(const string &s,
                             const vector<pair<int, int>> &qs)
{
    // s 是小写字母串；qs 是 0 下标半开区间；返回各区间的本质不同子串数。
    int n = s.size(), q = qs.size();
    assert(n > 0); // 调试检查，可删
    vector<vector<pair<int, int>>> at(n + 1);
    for (int i = 0; i < q; i++)
    {
        auto [l, r] = qs[i];
        assert(0 <= l && l < r && r <= n); // 调试检查，可删
        at[r].push_back({l + 1, i});
    }
    SAM<26, 'a'> sam(s);
    int states = sam.t.size();
    vector<vector<int>> son(states);
    for (int i = 1; i < states; i++)
    {
        son[sam.t[i].link].push_back(i);
    }
    vector<int> in(states), out(states), it(states), st{0};
    int tim = 0;
    while (!st.empty())
    {
        int u = st.back();
        if (!in[u]) in[u] = ++tim;
        if (it[u] < (int)son[u].size()) st.push_back(son[u][it[u]++]);
```

```
        else out[u] = tim, st.pop_back();
    }
    LazySeg<Info, Tag> seg(n);
    RangeSubMax mx(states + 1);
    vector<int> top(n + 1), ans(q);
    vector<unsigned char> seen(states);
    int u = 0;
    for (int r = 1; r <= n; r++)
    {
        u = sam.t[u].next[s[r - 1] - 'a'];
        int v = u;
        while (v && !seen[v])
        {
            seen[v] = 1, v = sam.t[v].link;
        }
        while (v)
        {
            int p = mx.query(in[v] - 1, out[v]);
            int old = top[p];
            seg.apply(p - sam.t[v].len, p - sam.t[old].len, Tag{-1});
            swap(top[p], v);
        }
        mx.setMax(in[u] - 1, r);
        top[r] = 0;
        for (auto [l, id] : at[r])
        {
            int len = r - l + 1;
            ans[id] = len * (len + 1) / 2 + seg.query(l - 1, r).sum;
        }
    }
    return ans;
}
```

`subDistinct(s,l,r)` 返回子串 $[l,r)$ 的本质不同子串数，单次 $O(L+1)$（$L=r-l$）时间与空间，依赖 4.3.002 后缀数组：SA-IS 与 4.3.003 LCP / Height / RMQ 应用。批量区间询问可调用 `rangeSubs(s,qs)`：离线扫描右端点，在 SAM 后缀链接树上维护贡献，复杂度 $O(n\log^2 n+q\log n)$，实现见 `include/full/4_3_010.hpp`。

4.3.011 Dictionary of Basic Factors

Dictionary of Basic Factors 把所有长度为 2 的幂的子串（基本因子）分层编号：第 k 层长度 2^k 的子串由第 $k-1$ 层的前、后两个 2^{k-1} 块的编号对决定，逐层排序压缩得离散排名 `rk[k][i]`。有了它，任意等长子串比较可拆成常数个基本因子——`compare(l,r,len)` 取覆盖 `len` 的两个 2^k 块（$k=\lfloor\log_2 len\rfloor$，右块从 `len` 减 2^k 起）比较排名对。

```
struct BasicFactors
{
    // s 是原字符串，rk[k][i] 是长 2^k 子串 s[i..i+2^k) 的排名。
    string s;
```

```
    vector<vector<int>> rk;

    BasicFactors(string s) : s(move(s))
    {
        // s 按无符号字节字典序比较；建立所有二次幂长度子串的分层排名。
        int n = this->s.size();
        vector<int> first;
        for (unsigned char c : this->s)
        {
            first.push_back(c);
        }
        rk.push_back(move(first));
        for (int k = 1; (1LL << k) <= n; k++)
        {
            int len = 1LL << (k - 1);
            vector<int> id(n - 2 * len + 1), next(id.size());
            iota(id.begin(), id.end(), 0);
            sort(id.begin(), id.end(), [&](int i, int j)
            {
                return pair{rk[k - 1][i], rk[k - 1][i + len]} <
                       pair{rk[k - 1][j], rk[k - 1][j + len]};
            });
            int cls = 0;
            for (int i = 0; i < (int)id.size(); i++)
            {
                if (i && pair{rk[k - 1][id[i - 1]], rk[k - 1][id[i - 1] + len]} !=
                         pair{rk[k - 1][id[i]], rk[k - 1][id[i] + len]}) cls++;
                next[id[i]] = cls;
            }
            rk.push_back(move(next));
        }
    }

    int compare(int l, int r, int len) const
    {
        // l、r 是两个子串起点，len 是共同长度；按字典序返回 -1、0 或 1。
        assert(0 <= l && 0 <= r && 0 <= len && l + len <= (int)s.size() && r + len <=
        ↪  (int)s.size()); // 调试检查，可删
        if (!len) return 0;
        int k = bit_width((unsigned long long)len) - 1, d = len - (1LL << k);
        pair<int, int> a{rk[k][l], rk[k][l + d]}, b{rk[k][r], rk[k][r + d]};
        return (a > b) - (a < b);
    }
};

struct BorderInfo
{
    // mn、mx 是最短和最长真 border 长度，cnt 是真 border 数量。
    int mn = 0, mx = 0, cnt = 0;
};

struct BasicDictQuery
```

```
{
    int l, r; // l、r 是询问子串的 1 起闭区间端点。
};

inline vector<BorderInfo> basicDict(const string &s,
                                    const vector<pair<int, int>> &qs)
{
    // s 是字符串；qs 是 0 下标闭区间；返回每个区间的真 border 统计。
    int n = s.size(), q = qs.size();
    vector<BorderInfo> ans(q);
    if (!n || !q) return ans;
    vector<BasicDictQuery> que(q);
    vector<vector<int>> atL(n + 2), atR(n + 2);
    for (int i = 0; i < q; i++)
    {
        int l = qs[i].first + 1, r = qs[i].second + 1;
        assert(1 <= l && l <= r && r <= n);
        que[i] = {l, r};
        atL[l].push_back(i);
        atR[r].push_back(i);
    }
    vector<int> sa(n + 1), rk(n + 1), id(2 * n + 3), cnt(256);
    for (int i = 1; i <= n; i++)
    {
        cnt[(unsigned char)s[i - 1]]++;
    }
    for (int i = 1; i < 256; i++)
    {
        cnt[i] += cnt[i - 1];
    }
    for (int i = n; i; i--)
    {
        sa[cnt[(unsigned char)s[i - 1]]--] = i;
    }
    int cls = 0;
    for (int i = 1; i <= n; i++)
    {
        if (i == 1 || s[sa[i] - 1] != s[sa[i - 1] - 1]) cls++;
        rk[sa[i]] = cls;
    }
    vector<int> nxt(n + 2), leftMost(q, n + 1), rightMost(q),
        leftRev(q, n + 1), rightRev(q), low(q, n + 1), high(q), number(q),
        buf(n + 3);
    auto addOne = [&](int idq, int x, int l, int r, int step)
    {
        if (l <= x && x <= r && (x - l) % step == 0)
        {
            low[idq] = min(low[idq], x);
            high[idq] = max(high[idq], x);
            number[idq]++;
        }
    };
```

```
    auto addTwo = [&](int idq, int l1, int r1, int d1,
                      int l2, int r2, int d2)
    {
        if (r1 - l1 <= d1)
        {
            for (int x = l1; x <= r1; x += d1)
            {
                addOne(idq, x, l2, r2, d2);
            }
            return;
        }
        if (r2 - l2 <= d2)
        {
            for (int x = l2; x <= r2; x += d2)
            {
                addOne(idq, x, l1, r1, d1);
            }
            return;
        }
        if ((l1 - l2) % d1) return;
        l1 = max(l1, l2), r1 = min(r1, r2);
        if (l1 <= r1)
        {
            low[idq] = min(low[idq], l1);
            high[idq] = max(high[idq], r1);
            number[idq] += (r1 - l1) / d1 + 1;
        }
    };
    auto work = [&](int len)
    {
        fill(buf.begin(), buf.begin() + cls + 1, n + 1);
        for (int i = n - len + 1; i >= 1; i--)
        {
            nxt[i] = buf[rk[i]], buf[rk[i]] = i;
        }
        fill(buf.begin(), buf.begin() + cls + 1, -1);
        for (int i = 2 - len; i <= n; i++)
        {
            if (1 <= i && i <= n) buf[rk[i]] = i;
            int j = i + len - 1;
            if (1 <= j && j <= n)
            {
                for (int idq : atR[j])
                {
                    if (que[idq].r - que[idq].l + 1 > len)
                    {
                        rightMost[idq] = buf[rk[que[idq].l]];
                    }
                }
            }
            j = i - len + 1;
            if (1 <= j && j <= n)
```

```
            {
                for (int idq : atL[j])
                {
                    if (que[idq].r - que[idq].l + 1 > len)
                    {
                      rightRev[idq] = min(buf[rk[que[idq].r - len + 1]], que[idq].r - len + 1);
                    }
                }
            }
        }
        fill(buf.begin(), buf.begin() + cls + 1, n + 1);
        for (int i = n; i + len - 1; i--)
        {
            if (1 <= i && i <= n) buf[rk[i]] = i;
            int j = i + 2 * len - 2;
            if (1 <= j && j <= n)
            {
                for (int idq : atR[j])
                {
                    if (que[idq].r - que[idq].l + 1 > len)
                    {
                        leftMost[idq] = max(buf[rk[que[idq].l]], nxt[que[idq].l]);
                    }
                }
            }
            if (1 <= i && i <= n)
            {
                for (int idq : atL[i])
                {
                    if (que[idq].r - que[idq].l + 1 > len)
                    {
                        leftRev[idq] = buf[rk[que[idq].r - len + 1]];
                    }
                }
            }
        }
        for (int i = 0; i < q; i++)
        {
            if (que[i].r - que[i].l + 1 <= len || leftMost[i] > rightMost[i] || leftRev[i] >
            ↪  rightRev[i]) continue;
            int d1 = nxt[leftMost[i]] - leftMost[i], d2 = nxt[leftRev[i]] - leftRev[i];
            swap(leftMost[i], rightMost[i]);
            leftRev[i] += len - que[i].l;
            rightRev[i] += len - que[i].l;
            leftMost[i] = que[i].r - leftMost[i] + 1;
            rightMost[i] = que[i].r - rightMost[i] + 1;
            addTwo(i, leftMost[i], rightMost[i], d1, leftRev[i], rightRev[i], d2);
        }
        if (cls == n) return;
        fill(buf.begin(), buf.begin() + cls + 1, 0);
        int z = 0;
        for (int i = n; i > n - len; i--)
```

```
        {
            id[++z] = i;
        }
        for (int i = 1; i <= n; i++)
        {
            if (sa[i] > len) id[++z] = sa[i] - len;
        }
        for (int i = 1; i <= n; i++)
        {
            buf[rk[i]]++;
        }
        for (int i = 2; i <= cls; i++)
        {
            buf[i] += buf[i - 1];
        }
        for (int i = n; i; i--)
        {
            sa[buf[rk[id[i]]]--] = id[i], id[i] = rk[i];
        }
        cls = 0;
        for (int i = 1; i <= n; i++)
        {
            int x = sa[i], y = sa[i - 1];
            if (i == 1 || id[x] != id[y] || id[x + len] != id[y + len]) cls++;
            rk[x] = cls;
        }
    };
    for (int len = 1; len <= n; len <<= 1)
    {
        work(len);
    }
    for (int i = 0; i < q; i++)
    {
        if (number[i]) ans[i] = {low[i], high[i], number[i]};
    }
    return ans;
}
```

构造 `BasicFactors bf(s)` 后 `bf.compare(l,r,len)` 按字典序比较两个等长子串返回 $-1/0/1$；非空串构造 $O(n\log^2 n)$ 时间、$O(n\log n)$ 空间，空串构造 $O(1)$，定长比较 $O(1)$。批量 border 统计调用 `basicDict(s,qs)`，其中 `qs` 为 0 下标闭区间，返回真 border 的最短长度、最长长度和数量，复杂度 $O((n+q)\log n)$，实现见 `include/full/4_3_011.hpp`。

4.3.012 BWT / FM-index

在文本末尾加入比所有字符都小的终止符 \$，把它的全部循环移位排序，排序结果的最后一列就是 BWT：第 i 行对应后缀数组中第 i 小的后缀，$L_i = s_{sa_i-1}$（$sa_i = 0$ 时为 \$）。记 $C[x]$ 为 L 中小于 x 的符号个数，$\mathrm{rank}(x, r)$ 为 $L_0 \dots L_{r-1}$ 中 x 的个数，则：

- LF 映射：第 i 行循环右移一位后位于第 $\mathrm{LF}(i) = C[L_i] + \mathrm{rank}(L_i, i)$ 行，因为同一字符在首列和

末列的相对顺序相同。

- 反向搜索：以模式 p 为前缀的行构成一段区间 $[l, r)$。从 p 的最后一个字符向前，每加一个字符 x 更新为 $l \leftarrow C[x] + \mathrm{rank}(x, l)$、$r \leftarrow C[x] + \mathrm{rank}(x, r)$。
- 还原：从 $L_i = \$$ 的那一行（即整个原文所在行）出发反复做 LF，依次得到原文从后往前的字符。

实现用 4.3.002 后缀数组：SA-IS 建后缀数组；字节 x 编码为 $x+1$，终止符编码为 0；`rank` 用每个符号的出现位置表二分，不做压缩。

```cpp
// 依赖 4.3.002 后缀数组：SA-IS
struct FMIndex
{
    // sa 是带终止符文本的后缀数组；bwt[i] 是第 i 小后缀前面的那个符号。
    // 字节 x 编码为 x+1，唯一的终止符编码为 0。
    vector<int> sa, bwt;
    // c[x] 是 bwt 中小于 x 的符号个数；at[x] 是符号 x 在 bwt 中出现的行号，升序。
    array<int, 258> c{};
    array<vector<int>, 257> at;

    FMIndex(const string &text)
    {
        // text 是任意字节串；建立后缀数组、BWT 与按符号分组的出现位置。
        int n = text.size() + 1;
        vector<int> s(n);
        for (int i = 0; i + 1 < n; i++)
        {
            s[i] = (unsigned char)text[i] + 1;
        }
        sa = suffixArray(s);
        bwt.resize(n);
        for (int i = 0; i < n; i++)
        {
            bwt[i] = s[(sa[i] + n - 1) % n];
            at[bwt[i]].push_back(i);
            c[bwt[i] + 1]++;
        }
        for (int x = 1; x < 258; x++)
        {
            c[x] += c[x - 1];
        }
    }

    int rank(int x, int r) const
    {
        // x 是符号编码，r 是行号上界；返回 bwt[0..r) 中 x 的出现次数。
        return lower_bound(at[x].begin(), at[x].end(), r) - at[x].begin();
    }

    int lf(int i) const
    {
        // i 是行号；返回把该行循环右移一位后所在的行（LF 映射）。
        return c[bwt[i]] + rank(bwt[i], i);
```

```
    }

    pair<int, int> search(const string &p) const
    {
        // p 是模式串；返回后缀数组中以 p 为前缀的行区间 [l,r)，不出现时 l==r。
        int l = 0, r = sa.size();
        for (int i = (int)p.size() - 1; i >= 0 && l < r; i--)
        {
            int x = (unsigned char)p[i] + 1;
            l = c[x] + rank(x, l);
            r = c[x] + rank(x, r);
        }
        return {l, r};
    }

    vector<int> locate(const string &p) const
    {
        // p 是模式串；返回 p 在原文中全部出现的起点，升序；空模式返回 0..|text|。
        auto [l, r] = search(p);
        vector<int> ans(sa.begin() + l, sa.begin() + r);
        sort(ans.begin(), ans.end());
        return ans;
    }

    string restore() const
    {
        // 只用 bwt 沿 LF 映射倒推，返回原文。
        int n = bwt.size(), i = at[0][0];
        string s(n - 1, '\0');
        for (int k = n - 2; k >= 0; k--)
        {
            i = lf(i);
            s[k] = (char)(bwt[i] - 1);
        }
        return s;
    }
};
```

`FMIndex fm(text)` 接受任意字节串。`search(p)` 返回以 p 为前缀的后缀数组行区间，$r-l$ 是出现次数；`locate(p)` 返回全部出现起点并升序排序，空模式返回 $0 \dots |text|$ 全部位置；`restore()` 只用 BWT 还原原文。构造 $O(n)$，`search` 为 $O(|p| \log n)$，`locate` 另加 $O(occ \log occ)$ 排序；保存完整的 `sa`、`bwt` 与出现位置表，空间 $O(n)$ 个整数，不是压缩索引。

只需要子串查找时，直接在后缀数组上二分（4.3.002 后缀数组：SA-IS）或用 4.3.004 后缀自动机 SAM 更直接；FM-index 的价值在于逐字符反向扩展区间以及由 BWT 还原原串。下面核对 `abracadabra`：

```
signed main()
{
    FMIndex fm("abracadabra");
    auto [l, r] = fm.search("abra");
    assert(r - l == 2);
```

```
    assert(fm.locate("a") == vector<int>({0, 3, 5, 7, 10}));
    assert(fm.locate("cad") == vector<int>({4}) && fm.locate("zzz").empty());
    assert(fm.restore() == "abracadabra");
}
```

4.4 回文结构

4.4.001 回文自动机 PAM / 回文树

回文自动机（回文树）每个节点恰对应一种本质不同回文子串，`link` 指向该回文的最长真回文后缀。有两个根：长度 -1 的奇根与长度 0 的偶根。在线 `add(c)` 时先沿 `getLink` 从 `last` 找到能左右扩展当前字符的回文后缀 `p`（即 $s[n-len_p-1]=s[n]$），若 `p` 没有 `x` 边就新建长度 len_p+2 的回文节点，其 `link` 在长度为 1 时接偶根、否则接 `getLink(link_p)` 的 `x` 边；至多新建一个节点。节点数减 2 即不同回文串数，`count` 沿 `link` 逆序上推 `cnt` 得每种回文的出现次数。

```
template <int A = 26, char S = 'a'> struct PAM
{
    struct Node
    {
        array<int, A> ch{}; // ch[c] 是两侧各加字符 c 得到的回文节点，0 表示不存在。
        // len 是回文长度，link 是回文后缀链接，cnt 是出现次数。
        int len = 0, link = 0, cnt = 0;
    };

    vector<Node> t{{{}, -1, 0, 0}, {{}, 0, 0, 0}};
    vector<int> s{-1}; // s 是已加入的字符，s[0]=-1 作为哨兵。
    // last 是当前最长回文后缀节点，counted 记录出现次数是否已经上传。
    int last = 1;
    bool counted = false;

    int getLink(int p) const
    {
        // p 是回文节点；返回可被当前末字符继续扩展的后缀回文节点。
        int n = s.size() - 1;
        while (n - t[p].len - 1 < 0 || s[n - t[p].len - 1] != s[n])
        {
            p = t[p].link;
        }
        return p;
    }

    int add(char c)
    {
        // c 是追加到原串末尾的字符；返回新的最长回文后缀节点。
        assert(!counted); // 调试检查，可删
        int x = c - S;
        assert(0 <= x && x < A); // 调试检查，可删
        s.push_back(x);
        int p = getLink(last);
```

```
        if (!t[p].ch[x])
        {
            int q = t.size();
            t.push_back({{}, t[p].len + 2, 0, 0});
            if (t[q].len == 1)
            {
                t[q].link = 1;
            }
            else
            {
                t[q].link = t[getLink(t[p].link)].ch[x];
            }
            t[p].ch[x] = q;
        }
        last = t[p].ch[x];
        t[last].cnt++;
        return last;
    }

    void build(const string &x)
    {
        // x 是要继续追加的字符串。
        for (char c : x)
        {
            add(c);
        }
    }

    void count()
    {
        // 沿后缀链接上传出现次数；重复调用不再二次累计。
        if (counted)
        {
            return;
        }
        for (int p = (int)t.size() - 1; p >= 2; p--)
        {
            t[t[p].link].cnt += t[p].cnt;
        }
        counted = true;
    }
};
```

`build(s)` 逐字符加入，`add(c)` 返回当前最长回文后缀节点，`count()` 幂等地上传一次出现次数；上传后不能再追加字符。节点数减二是本质不同回文数。构造 $O(n+1)$、计数 $O(p+1)$、空间 $O((p+2)A+n+1)$，其中 $p \le n$ 是本质不同回文数。需要在两端插入和删除时改用 4.4.002 双端可撤销回文树，回文划分 DP 的等差链优化见 4.4.003 Palindrome Series。下面加入 `ababa`，记录每次新增长后缀对应的节点，再逐节点核对五种回文的出现次数：

```
signed main()
{
```

```
    PAM<3, 'a'> pam;
    vector<int> id;
    for (char c : string("ababa"))
    {
        id.push_back(pam.add(c));
    }
    pam.count();
    pam.count();
    assert((int)pam.t.size() - 2 == 5);
    assert(pam.t[id[0]].cnt == 3);
    assert(pam.t[id[1]].cnt == 2);
    assert(pam.t[id[2]].cnt == 2);
    assert(pam.t[id[3]].cnt == 1);
    assert(pam.t[id[4]].cnt == 1);
    cout << "5 3 2 2 1 1\n";
    return 0;
}
```

真实输出为 `5 3 2 2 1 1`。

4.4.002 双端可撤销回文树

普通 PAM 只支持在右端追加；`DequePAM` 额外维护各位置对应的最长回文前缀 `pre` 和最长回文后缀 `suf`，从而能在 deque 两端插入和撤销。节点的 `cnt` 是当前 `pre`、`suf` 记录对它的引用数，`sons` 是后缀链接孩子数；二者都归零时，该回文已不在当前串中，可删除父转移并回收编号。`quick` 跳过一批不可能被新字符扩展的后缀回文，前后端使用对称的 `frontCan`、`backCan` 查找可扩展节点。字符必须落在 $[S, S+A)$，删除只允许作用于非空串。

```
template <int A = 26, char S = 'a'>
struct DequePAM
{
  private:
    struct Node
    {
        // ch 是回文树转移；par、link、quick 分别是转移父亲、后缀链接和快速跳转。
        array<int, A> ch{};
        int par = 0;
        int link = 0;
        int quick = 0;
        // len 是回文长度，cnt 是边界记录引用数，sons 是后缀链接孩子数。
        int len = 0;
        int cnt = 0;
        int sons = 0;
    };

    struct Pos
    {
        // c 是当前位置字符编号；pre、suf 记录当前最长回文前缀、后缀节点。
        int c = 0;
        int pre = 1;
```

```
        int suf = 1;
    };

    // t 是节点池，spare 是可复用编号，d 是当前字符串，kinds 是本质不同回文数。
    vector<Node> t;
    vector<int> spare;
    deque<Pos> d;
    int kinds = 0;

    int newNode()
    {
        // 返回一个已经清空的节点编号，优先复用回收节点。
        if (spare.empty())
        {
            t.push_back({});
            return (int)t.size() - 1;
        }
        int p = spare.back();
        spare.pop_back();
        t[p] = Node{};
        return p;
    }

    void drop(int p)
    {
        // p 是已经脱离所有转移和边界记录的节点；清空后回收其编号。
        t[p] = Node{};
        spare.push_back(p);
    }

    int backCan(int c, int p) const
    {
        // c 是待加字符，p 是当前最长回文后缀；返回最先能从右端扩展的后缀回文。
        int n = d.size();
        while (true)
        {
            if (t[p].len == -1 || (t[p].len < n && d[n - t[p].len - 1].c == c))
            {
                return p;
            }
            int q = t[p].link;
            if (t[q].len == -1 || (t[q].len < n && d[n - t[q].len - 1].c == c))
            {
                return q;
            }
            p = t[p].quick;
        }
    }

    int frontCan(int c, int p) const
    {
        // c 是待加字符，p 是当前最长回文前缀；返回最先能从左端扩展的前缀回文。
```

```
        int n = d.size();
        while (true)
        {
            if (t[p].len == -1 || (t[p].len < n && d[t[p].len].c == c))
            {
                return p;
            }
            int q = t[p].link;
            if (t[q].len == -1 || (t[q].len < n && d[t[q].len].c == c))
            {
                return q;
            }
            p = t[p].quick;
        }
    }

public:
    DequePAM() : t(2)
    {
        // 建立长度 -1 的奇根 0 和长度 0 的偶根 1。
        t[0].len = -1;
        t[0].par = 0;
        t[0].link = 0;
        t[0].quick = 0;
        t[1].len = 0;
        t[1].par = 0;
        t[1].link = 0;
        t[1].quick = 0;
    }

    void pushBack(char c)
    {
        // c 是新字符；加入右端并维护最长回文前后缀。
        int x = (unsigned char)c - (unsigned char)S;
        assert(0 <= x && x < A); // 调试检查，可删
        int par = d.empty() ? 0 : backCan(x, d.back().suf);
        int n = d.size();
        int v = t[par].ch[x];
        int w = 1;
        if (!v)
        {
            v = newNode();
            t[v].par = par;
            t[v].len = t[par].len + 2;
            if (par != 0)
            {
                w = t[backCan(x, t[par].link)].ch[x];
                assert(w); // 调试检查，可删
            }
            t[v].link = w;
            t[w].sons++;
            d.push_back({x, 1, 1});
```

```
            n++;
            if (t[w].link != 0 && d[n - t[w].len - 1].c == d[n - t[t[w].link].len - 1].c)
            {
                t[v].quick = t[w].quick;
            }
            else
            {
                t[v].quick = t[w].link;
            }
            t[par].ch[x] = v;
            kinds++;
        }
        else
        {
            d.push_back({x, 1, 1});
            n++;
            w = t[v].link;
        }
        d[n - 1].suf = v;
        d[n - t[v].len].pre = v;
        if (t[w].len >= 1 && d[n - t[v].len + t[w].len - 1].suf == w)
        {
            d[n - t[v].len + t[w].len - 1].suf = 1;
        }
        t[v].cnt++;
    }

    void pushFront(char c)
    {
        // c 是新字符；加入左端并维护最长回文前后缀。
        int x = (unsigned char)c - (unsigned char)S;
        assert(0 <= x && x < A); // 调试检查，可删
        int par = d.empty() ? 0 : frontCan(x, d.front().pre);
        int v = t[par].ch[x];
        int w = 1;
        if (!v)
        {
            v = newNode();
            t[v].par = par;
            t[v].len = t[par].len + 2;
            if (par != 0)
            {
                w = t[frontCan(x, t[par].link)].ch[x];
                assert(w); // 调试检查，可删
            }
            t[v].link = w;
            t[w].sons++;
            d.push_front({x, 1, 1});
            if (t[w].link != 0 && d[t[w].len].c == d[t[t[w].link].len].c)
            {
                t[v].quick = t[w].quick;
            }
```

```
            else
            {
                t[v].quick = t[w].link;
            }
            t[par].ch[x] = v;
            kinds++;
        }
        else
        {
            d.push_front({x, 1, 1});
            w = t[v].link;
        }
        d[0].pre = v;
        d[t[v].len - 1].suf = v;
        if (t[w].len >= 1 && d[t[v].len - t[w].len].pre == w)
        {
            d[t[v].len - t[w].len].pre = 1;
        }
        t[v].cnt++;
    }

    void popBack()
    {
        // 删除右端字符；当前串必须非空。
        assert(!d.empty()); // 调试检查，可删
        int n = d.size();
        int v = d.back().suf;
        int x = d.back().c;
        int w = t[v].link;
        if (t[v].len >= 2 && t[d[n - t[v].len + t[w].len - 1].suf].len < t[w].len)
        {
            d[n - t[v].len + t[w].len - 1].suf = w;
            d[n - t[v].len].pre = w;
        }
        else
        {
            d[n - t[v].len].pre = 1;
        }
        t[v].cnt--;
        if (!t[v].sons && !t[v].cnt)
        {
            t[t[v].par].ch[x] = 0;
            t[w].sons--;
            drop(v);
            kinds--;
        }
        d.pop_back();
    }

    void popFront()
    {
        // 删除左端字符；当前串必须非空。
```

```
        assert(!d.empty()); // 调试检查，可删
        int v = d.front().pre;
        int x = d.front().c;
        int w = t[v].link;
        if (t[v].len >= 2 && t[d[t[v].len - t[w].len].pre].len < t[w].len)
        {
            d[t[v].len - t[w].len].pre = w;
            d[t[v].len - 1].suf = w;
        }
        else
        {
            d[t[v].len - 1].suf = 1;
        }
        t[v].cnt--;
        if (!t[v].sons && !t[v].cnt)
        {
            t[t[v].par].ch[x] = 0;
            t[w].sons--;
            drop(v);
            kinds--;
        }
        d.pop_front();
    }

    int size() const
    {
        // 返回当前字符串长度。
        return d.size();
    }

    int distinct() const
    {
        // 返回当前字符串的本质不同非空回文子串数。
        return kinds;
    }

    int longPrefix() const
    {
        // 返回当前最长回文前缀长度，空串返回 0。
        return d.empty() ? 0 : t[d.front().pre].len;
    }

    int longSuffix() const
    {
        // 返回当前最长回文后缀长度，空串返回 0。
        return d.empty() ? 0 : t[d.back().suf].len;
    }
};
```

`pushFront`、`pushBack`、`popFront`、`popBack` 分别修改两端；`distinct()`、`longPrefix()`、`longSuffix()` 均为 $O(1)$ 查询。数组转移为 $O(1)$，一次修改沿 `quick` 链查找可扩展回文，保守最坏为相关回文链接链长。设操作过程中最大串长为 N、最多同时存在 P 种非空回文，节点编号由 `spare` 复用，峰值空间为

$O((P+2)A+N)$。下面从 abac 出发依次在两端增删，检查四种修改和节点回收：

```
signed main()
{
    DequePAM<3, 'a'> pam;
    for (char c : string("abac"))
    {
        pam.pushBack(c);
    }
    assert((array{pam.distinct(), pam.longPrefix(), pam.longSuffix()} == array{4LL, 3LL,
    ↪ 1LL}));
    pam.pushFront('c');
    assert((array{pam.distinct(), pam.longPrefix(), pam.longSuffix()} == array{5LL, 5LL,
    ↪ 5LL}));
    pam.pushBack('a');
    assert((array{pam.distinct(), pam.longPrefix(), pam.longSuffix()} == array{6LL, 5LL,
    ↪ 3LL}));
    pam.popFront();
    assert((array{pam.distinct(), pam.longPrefix(), pam.longSuffix()} == array{5LL, 3LL,
    ↪ 3LL}));
    pam.popBack();
    assert((array{pam.distinct(), pam.longPrefix(), pam.longSuffix()} == array{4LL, 3LL,
    ↪ 1LL}));
}
```

4.4.003 Palindrome Series

把回文树的后缀链接链按 diff=len[u]-len[link[u]] 分成等差系列，并维护 series 指针跳过同 diff 段。这样每个前缀的全部回文后缀被压成 $O(\log n)$ 个等差系列，回文划分类 DP（如把串划分成最少 / 方案数个回文）可按系列整体转移而非逐个回文，均摊 $O(n \log n)$。add 在建回文节点时顺带算出 diff 与 series，eachSeries 沿 series 链枚举关键状态。

```
template <int A = 26, char S = 'a'> struct PamSeries
{
    struct Node
    {
        array<int, A> ch{}; // ch[c] 是两侧各加字符 c 得到的回文节点，0 表示不存在。
        // diff=len-len[link]，series 指向上一条不同公差的系列。
        int len = 0, link = 0, diff = 0, series = 0;
    };

    vector<Node> t{{}, -1, 0, 0, 0}, {{}, 0, 0, 0, 0}};
    vector<int> s{-1}; // s 是已加入的字符，s[0]=-1 作为哨兵。
    // last 是当前最长回文后缀节点。
    int last = 1;

    int get(int p) const
    {
        // p 是一个回文节点；返回能被当前末字符继续扩展的后缀回文节点。
        int n = s.size() - 1;
```

```
        while (s[n - t[p].len - 1] != s[n])
        {
            p = t[p].link;
        }
        return p;
    }

    int add(char c)
    {
        // c 是追加到原串末尾的字符；返回新的最长回文后缀节点。
        int x = c - S;
        assert(0 <= x && x < A); // 调试检查，可删
        s.push_back(x);
        int p = get(last);
        if (!t[p].ch[x])
        {
            int q = t.size();
            t.push_back({{}, t[p].len + 2});
            t[q].link = t[q].len == 1 ? 1 : t[get(t[p].link)].ch[x];
            t[q].diff = t[q].len - t[t[q].link].len;
            t[q].series = t[q].diff == t[t[q].link].diff ? t[t[q].link].series : t[q].link;
            t[p].ch[x] = q;
        }
        return last = t[p].ch[x];
    }

    template <class F> void eachSeries(int p, F f) const
    {
        // p 是当前最长回文后缀，f 接收每条等差系列的首节点。
        for (; t[p].len > 0; p = t[p].series)
        {
            f(p);
        }
    }
};
```

`add(c)` 在线扩展并维护 `diff`、`series`；`eachSeries(p,f)` 沿等差链对每个关键状态调用回调 `f`。每次追加摊还 $O(1)$，状态数 $\le n+2$、空间 $O((n+2)A+n+1)$；一个前缀的系列链长为 $O(\log(n+1))$，故配合系列 DP 时每次枚举 $O(\log(n+1))$。只统计本质不同回文或其出现次数用 4.4.001 回文自动机 PAM / 回文树；本条专为回文划分 DP 的等差优化。

例如把当前前缀的每条系列首节点收集起来：

```
signed main()
{
    PamSeries<> pam;
    int p = 0;
    for (char c : string("abacaba"))
    {
        p = pam.add(c);
    }
```

```
    vector<int> head;
    pam.eachSeries(p, [&](int u)
    {
        head.push_back(u);
    });
    assert(!head.empty());
}
```

4.4.004 SqAM

SqAM 是在线子序列自动机：每加入一个文本字符建一个状态，`go[u][c]` 指向位置 `u` 之后字符 `c` 首次出现的状态。插入新字符 `x` 时沿每种字符 `c` 的「上一同字符位置」链 `pre` 补齐尚不存在的 `go[..][x]` 转移，每个状态与字符对最多被写一次，故总代价 $O(nA)$。沿 `go` 读完模式当且仅当它是文本子序列。

```
template <int A = 26> class SqAM
{
    // go[u][c] 是状态 u 读字符 c 后到达的最早后继，-1 表示不存在。
    vector<array<int, A>> go;
    // pre[u] 是与状态 u 字符相同的上一次出现位置，-1 表示此前未出现。
    vector<int> pre;
    // last[c] 是字符 c 最近一次出现的状态，-1 表示尚未出现。
    array<int, A> last;

  public:
    SqAM()
    {
        // 建立只含空文本状态的子序列自动机。
        last.fill(-1);
        go.push_back({});
        go[0].fill(-1);
        pre.push_back(-1);
    }

    void add(int x)
    {
        // x 是 [0,A) 内的新字符编号；把它追加到文本末尾。
        assert(0 <= x && x < A); // 调试检查，可删
        int v = go.size();
        go.push_back({});
        go.back().fill(-1);
        pre.push_back(last[x]);
        for (int c = 0; c < A; c++)
        {
            for (int u = last[c]; u != -1 && go[u][x] == -1; u = pre[u])
            {
                go[u][x] = v;
            }
        }
        if (last[x] == -1)
        {
```

```
            go[0][x] = v;
        }
        last[x] = v;
    }

    bool contains(const vector<int> &s) const
    {
        // s 是 [0,A) 内的模式序列；返回 s 是否为当前文本的子序列。
        int u = 0;
        for (int x : s)
        {
            if (x < 0 || x >= A || go[u][x] == -1)
            {
                return false;
            }
            u = go[u][x];
        }
        return true;
    }

    int size() const
    {
        // 返回当前文本长度。
        return go.size() - 1;
    }
};
```

构造 `SqAM<A> q`，`add(x)` 追加 $[0,A)$ 内字符编号，`contains(s)` 判断编号序列是否为当前文本子序列，`size()` 返回文本长度；只支持末尾追加、不支持删除，也不判连续子串。加入总长 n 的时间、空间均为 $O((n+1)A)$，长度 m 查询 $O(m)$。依次加入 `{0,1,0,2}` 后 `{0,0,2}` 与空序列是子序列，`{1,1}`、`{2,0}` 不是。离线一次建表的等价结构见 4.2.004 子序列自动机。

4.5 子序列与高级问题

4.5.001 最长公共子序列 LCS

最长公共子序列求两序列中不要求连续、但保持相对顺序的最长公共子列（勿与最长公共子串混淆）。令 $f_{i,j}$ 为 $a[0,i)$、$b[0,j)$ 的 LCS 长度：末字符相等时 $f_{i,j}=f_{i-1,j-1}+1$，否则 $f_{i,j}=\max(f_{i-1,j},f_{i,j-1})$。本实现存整张 `f` 表以便回溯：从 (n,m) 出发，相等就取该字符并双向回退，否则往 `f` 更大的一侧走，最后反转得到一条 LCS。

```
template <class T> vector<T> lcs(const vector<T> &a, const vector<T> &b)
{
    // a、b 是两个序列；返回二者的一条最长公共子序列。
    int n = a.size(), m = b.size();
    vector<vector<int>> f(n + 1, vector<int>(m + 1));
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
```

```
        {
            if (a[i] == b[j])
            {
                f[i + 1][j + 1] = f[i][j] + 1;
            }
            else
            {
                f[i + 1][j + 1] = max(f[i][j + 1], f[i + 1][j]);
            }
        }
    }
    vector<T> c;
    for (int i = n, j = m; i && j;)
    {
        if (a[i - 1] == b[j - 1])
        {
            i--;
            j--;
            c.push_back(a[i]);
        }
        else if (f[i - 1][j] >= f[i][j - 1])
        {
            i--;
        }
        else
        {
            j--;
        }
    }
    reverse(c.begin(), c.end());
    return c;
}
```

`lcs(a,b)` 返回一条具体的最长公共子序列（元素只需支持判等）；构造与回溯的时间、空间均为 $O((n+1)(m+1))$。只要长度且串长时用位并行的 4.5.002 Bitset 优化 LCS；连续公共片段是另一问题，见 4.5.004 多串最长公共子串/子序列。最短公共超序列 4.5.005 最短公共超序列 直接建在这里恢复出的 LCS 上。

4.5.002 Bitset 优化 LCS

只要 LCS 长度、且串较长时可用位并行。把第二串每个字符的出现位置编码成位集 `mask[c]`，用经典公式 `x=mask[c]|s; s=x&~(x-((s<<1)|1))` 逐个 `a` 中字符批量推进整行（跨多机器字用手写借位减法处理进位），最后各字长的 `popcount` 之和即 LCS 长度。它把一维转移加速约一个字长倍。

```
int lcsBitset(const string &a, const string &b)
{
    // a、b 是两个字节串；返回二者的最长公共子序列长度。
    int w = (b.size() + 63) / 64;
    vector<vector<unsigned long long>> mask(256, vector<unsigned long long>(w));
    vector<unsigned long long> s(w), x(w), z(w);
    for (int i = 0; i < (int)b.size(); i++)
```

```
    {
        mask[(unsigned char)b[i]][i / 64] |= 1ULL << (i % 64);
    }
    for (unsigned char c : a)
    {
        unsigned long long shift = 1, borrow = 0;
        for (int i = 0; i < w; i++)
        {
            x[i] = mask[c][i] | s[i];
            unsigned long long y = s[i] << 1 | shift;
            shift = s[i] >> 63;
            unsigned long long q = y + borrow;
            unsigned long long borrow = q < y || x[i] < q;
            z[i] = x[i] - q;
            borrow = borrow;
        }
        for (int i = 0; i < w; i++)
        {
            s[i] = x[i] & ~z[i];
        }
    }
    int ans = 0;
    for (unsigned long long x : s)
    {
        ans += popcount(x);
    }
    return ans;
}
```

`lcsBitset(a,b)` 只返回 LCS 长度，字符按 `unsigned char` 映射到 256 个掩码。答案不超过 $\min(n,m)$；两个串长和 $W=\lceil m/64\rceil$ 都须能由模板中的 64 位 `int` 表示。时间为 $O(nW+m+256W)$，空间为 $O(256(W+1))$；需要恢复具体序列时改用 4.5.001 最长公共子序列 LCS 的普通 DP。

4.5.003 区间 LCS / 区间 LIS

两个不同问题放在一起对比。`rangeLCS` 是限定两串各自下标区间 $[l,r)$、$[x,y)$ 的 LCS，用滚动一维 `f` 数组做标准转移，`pre` 暂存左上角值。`rangeLIS` 是区间内严格递增子序列长度，用经典的 `lower_bound` 维护「长度 k 的 LIS 的最小末元素」贪心数组。二者状态含义、答案都不同，不能相互套用；LCS 只有在一串元素互异时才能映射成另一串的位置序列转成 LIS。

```
int rangeLCS(const string &a, int l, int r, const string &b, int x, int y)
{
    // a、b 是两个字符串；[l,r)、[x,y) 是参与计算的半开区间；返回区间 LCS 长度。
    vector<int> f(y - x + 1);
    for (int i = l; i < r; i++)
    {
        int pre = 0;
        for (int j = x; j < y; j++)
        {
```

```
            int old = f[j - x + 1];
            if (a[i] == b[j])
            {
                f[j - x + 1] = pre + 1;
            }
            else
            {
                f[j - x + 1] = max(f[j - x + 1], f[j - x]);
            }
            pre = old;
        }
    }
    return f.back();
}

template <class T> int rangeLIS(const vector<T> &a, int l, int r)
{
    // a 是输入序列，[l,r) 是查询区间；返回严格递增子序列长度。
    vector<T> d;
    for (int i = l; i < r; i++)
    {
        auto it = lower_bound(d.begin(), d.end(), a[i]);
        if (it == d.end())
        {
            d.push_back(a[i]);
        }
        else
        {
            *it = a[i];
        }
    }
    return d.size();
}
```

`rangeLCS(a,l,r,b,x,y)` 查两个半开子串的 LCS 长度；令 $n = r - l$、$m = y - x$，时间为 $O((n+1)(m+1))$，空间为 $O(m+1)$。`rangeLIS(a,l,r)` 查半开区间内严格 LIS 长度，令 $L = r - l$，时间为 $O(1 + L\log(L+1))$，空间为 $O(L+1)$。求可恢复的整串 LCS 用 4.5.001 最长公共子序列 LCS；连续公共片段用 4.5.004 多串最长公共子串/子序列。

4.5.004 多串最长公共子串/子序列

公共子串（连续）与公共子序列（可跳字符）是两个问题。`multiSubstr` 对首串建 SAM，再让其余每串在自动机上跑匹配、用 `best` 记录每状态能匹配到的最长长度并沿 link 上推，`mn` 取各串在每状态的最小匹配长度，最终最大值即多串最长公共子串。`multiSubseq` 是 k 维 DP：把各串下标打包成混合进制状态 `s`，同层字符全相等则从「全体减一」转移 $+1$，否则从某一维减一取最大。二者状态与答案不能混用。

```
int multiSubstr(const vector<string> &a)
{
    // a 是全部输入串；返回它们的最长公共子串长度。
```

```
    if (a.empty())
    {
        return 0;
    }
    struct N
    {
        // len 是该状态的最长长度，link 是后缀链接。
        int len = 0, link = -1;
        // to 按字节保存转移目标状态。
        unordered_map<unsigned char, int> to;
    };
    vector<N> t(1);
    int last = 0;
    for (unsigned char c : a[0])
    {
        int u = t.size();
        t.push_back({t[last].len + 1});
        int p = last;
        while (p != -1 && !t[p].to.contains(c))
        {
            t[p].to[c] = u;
            p = t[p].link;
        }
        if (p == -1)
        {
            t[u].link = 0;
        }
        else
        {
            int q = t[p].to[c];
            if (t[p].len + 1 == t[q].len)
            {
                t[u].link = q;
            }
            else
            {
                int v = t.size();
                t.push_back(t[q]);
                t[v].len = t[p].len + 1;
                while (p != -1)
                {
                    auto it = t[p].to.find(c);
                    if (it == t[p].to.end() || it->second != q)
                    {
                        break;
                    }
                    it->second = v;
                    p = t[p].link;
                }
                t[q].link = t[u].link = v;
            }
        }
```

```
        last = u;
    }
    vector<int> ord(t.size()), cnt(a[0].size() + 1), mn(t.size());
    for (const N &v : t)
    {
        cnt[v.len]++;
    }
    partial_sum(cnt.begin(), cnt.end(), cnt.begin());
    for (int u = (int)t.size() - 1; u >= 0; u--)
    {
        ord[--cnt[t[u].len]] = u;
    }
    reverse(ord.begin(), ord.end());
    for (int i = 0; i < (int)t.size(); i++)
    {
        mn[i] = t[i].len;
    }
    for (int k = 1; k < (int)a.size(); k++)
    {
        vector<int> best(t.size());
        int v = 0, l = 0;
        for (unsigned char c : a[k])
        {
            while (v && !t[v].to.contains(c))
            {
                v = t[v].link;
                l = min(l, t[v].len);
            }
            if (t[v].to.contains(c))
            {
                v = t[v].to[c];
                l++;
            }
            else
            {
                l = 0;
            }
            best[v] = max(best[v], l);
        }
        for (int u : ord)
        {
            if (t[u].link >= 0)
            {
                best[t[u].link] = max(best[t[u].link], min(best[u], t[t[u].link].len));
            }
        }
        for (int i = 0; i < (int)t.size(); i++)
        {
            mn[i] = min(mn[i], best[i]);
        }
    }
    return *max_element(mn.begin(), mn.end());
```

```cpp
}

optional<int> multiSubseq(const vector<string> &a)
{
    // a 是全部输入串；返回多串 LCS 长度，状态数无法用 size_t 表示时返回空。
    if (a.empty())
    {
        return 0;
    }
    int k = a.size();
    size_t states = 1;
    vector<size_t> st(k), base(k);
    for (int i = 0; i < k; i++)
    {
        size_t len = a[i].size() + 1;
        if (len > numeric_limits<size_t>::max() / states)
        {
            return nullopt;
        }
        base[i] = states;
        states *= len;
    }
    if (states > vector<int>().max_size())
    {
        return nullopt;
    }
    vector<int> f(states);
    for (size_t s = 1; s < states; s++)
    {
        size_t q = s;
        bool zero = false, same = true;
        int c = -1;
        size_t back = 0;
        for (int i = k - 1; i >= 0; i--)
        {
            st[i] = q / base[i];
            q %= base[i];
            if (!st[i])
            {
                zero = true;
            }
            else
            {
                int cur = (unsigned char)a[i][st[i] - 1];
                if (c == -1)
                {
                    c = cur;
                }
                else if (c ≠ cur)
                {
                    same = false;
                }
```

```
            }
            back += base[i];
        }
        if (!zero && same)
        {
            f[s] = f[s - back] + 1;
        }
        else
        {
            for (int i = 0; i < k; i++)
            {
                if (st[i])
                {
                    f[s] = max(f[s], f[s - base[i]]);
                }
            }
        }
    }
    return f.back();
}
```

`multiSubstr(a)` 返回多串最长公共连续子串长度，答案不超过最短串长；首串产生至多 $2|a_0|-1$ 个状态，串长和状态编号须能由模板中的 64 位 `int` 表示。转移使用哈希表，总长 L 下为期望 $O(L+1)$ 时间、$O(|a_0|+1)$ 额外空间。`multiSubseq(a)` 的答案同样不超过最短串长；它在混合进制状态数不能由 `size_t` 或容器表示时返回空，非空输入需要 $O(k\prod_i(|a_i|+1))$ 时间和 $O(k+\prod_i(|a_i|+1))$ 空间，只适合串少且短。

4.5.005 最短公共超序列

最短公共超序列是同时以 `a`、`b` 为子序列的最短串，长度为 $|a|+|b|-\mathrm{LCS}(a,b)$。构造时先取一条 LCS，再沿 LCS 逐个对齐：把两串在下一个公共字符前的未匹配字符依次补入，公共字符只加一次，末尾补上各自剩余。

```
// 依赖 4.5.001 最长公共子序列 LCS
string scs(const string &a, const string &b)
{
    // a、b 是两个字符串；返回二者的一条最短公共超序列。
    vector<char> x(a.begin(), a.end()), y(b.begin(), b.end());
    auto c = lcs(x, y);
    string ans;
    int i = 0, j = 0;
    for (char z : c)
    {
        while (a[i] != z)
        {
            ans += a[i++];
        }
        while (b[j] != z)
        {
```

```
            ans += b[j++];
        }
        ans += z;
        i++;
        j++;
    }
    return ans + a.substr(i) + b.substr(j);
}
```

`scs(a,b)` 复用 4.5.001 最长公共子序列 LCS 恢复的 LCS 交错拼出一条最短公共超序列，时间、空间均为 $O((n+1)(m+1))$。它必须用能恢复序列的普通 LCS，位并行的 4.5.002 Bitset 优化 LCS 只给长度、无法回溯故不适用。

4.5.006 字符串周期、循环节与前后缀

字符串的周期、最小循环节、border、前后缀关系都能由前缀函数一次性得到，代码与 4.1.002 KMP / 前缀函数 完全相同，这里不重复：直接依赖其中的 `prefix`、`kmp`、`borders`、`minPeriod`。对非空串，前缀函数末端 π_{n-1} 的失配链枚举全部真 border，而 $n-\pi_{n-1}$ 是最短周期；它整除 n 时才是最小循环节，否则整串自身才是循环节。空串没有非空 border，`minPeriod("")` 返回 `0`。`prefix`、`borders`、`minPeriod` 为 $O(n+1)$ 时间与空间；`kmp` 对长度 n 的文本和长度 m 的模式为 $O(n+m+1)$ 时间、$O(m+occ+1)$ 空间，其中 occ 是匹配数。需要在文本中定位循环出现用 `kmp`，判前缀对齐用 4.1.003 扩展 KMP / Z 函数。

4.5.007 前缀-子串 LCS 批量查询

给定固定的首串 s、次串 t，每个查询要求 $s[0,a)$ 与 $t[l,r)$ 的 LCS 长度。逐个调用 4.5.003 区间 LCS / 区间 LIS 的动态规划要 $O(Q|s||t|)$，无法承受 $Q=5\times10^5$。对每个首串前缀维护 seaweed 边界的置换表示 `h`：扫描新字符时，交换恰能继续匹配或已经跨过当前边界的端点；边界值落在 $[0,l)$ 的数量正好给出查询矩形中没有被 LCS 使用的位置数。按右端点离线并用 Fenwick 计数，即可一次扫描回答该前缀的全部查询。

```
struct PrefixSubLCS
{
    struct Query
    {
        int a, l, r, id; // a 是首串前缀长度，l、r 是次串半开区间，id 是原查询编号。
    };

    static vector<int> solve(const string &s, const string &t, const vector<array<int, 3>>
    ↪ &ask)
    {
        // s、t 是小写字符串；ask 每项为 {前缀长度，左端，右端}，返回对应 LCS 长度。
        int m = s.size(), n = t.size(), q = ask.size();
        vector<int> ans(q);
        vector<vector<Query>> byA(m);
        for (int i = 0; i < q; i++)
        {
            auto [a, l, r] = ask[i];
```

```
            assert(0 <= a && a <= m && 0 <= l && l <= r && r <= n);
            if (a && l < r)
            {
                byA[a - 1].push_back({a, l, r, i});
            }
        }
        vector<int> h(n);
        iota(h.begin(), h.end(), 0);
        for (int a = 0; a < m; a++)
        {
            int cur = -1;
            for (int j = 0; j < n; j++)
            {
                if (s[a] == t[j] || h[j] < cur)
                {
                    swap(h[j], cur);
                }
            }
            vector<vector<Query>> byR(n);
            for (auto x : byA[a])
            {
                byR[x.r - 1].push_back(x);
            }
            vector<int> bit(n + 1);
            auto add = [&](int p)
            {
                for (p++; p <= n; p += p & -p)
                {
                    bit[p]++;
                }
            };
            auto sum = [&](int p)
            {
                int ret = 0;
                for (; p; p &= p - 1)
                {
                    ret += bit[p];
                }
                return ret;
            };
            int cnt = 0;
            for (int r = 0; r < n; r++)
            {
                if (h[r] != -1)
                {
                    add(h[r]);
                    cnt++;
                }
                for (auto x : byR[r])
                {
                    ans[x.id] = (x.r - x.l) - (cnt - sum(x.l));
                }
```

```
            }
        }
        return ans;
    }
};
```

`PrefixSubLCS::solve(s,t,ask)` 中每个 `ask[i]={a,l,r}` 使用半开下标，返回同顺序的 LCS 长度；允许空前缀和空子串。设 $m=|s|$、$n=|t|$，总时间为 $O(mn+Q\log n)$，额外空间为 $O(n+Q)$；题目范围 $m,n\le 1000$、$Q\le 5\times 10^5$。单个查询或需要恢复子序列时仍选 4.5.001 最长公共子序列 LCS 或 4.5.003 区间 LCS / 区间 LIS，本条只输出批量长度。

5. 图论

5.1 图的表示与基础遍历

5.1.001 邻接表、链式前向星、邻接矩阵

图的存储方式决定后续所有算法的常数与可行性，本条给出三种基本形态。邻接表只存实际出现的边，稀疏图上遍历与空间都最省；邻接矩阵按点对直接下标索引，$O(1)$ 判边和改权，代价是 $O(V^2)$ 空间，只在稠密图或需要常数时间查边权时用；链式前向星用一维边数组 `e` 加头指针 `head` 把同起点的边串成链，加边时新边接到链首，天然得到稳定的边编号。普通 `add` 只加入一条边；需要用 `id^1` 取得反向边时，必须改用 `addResEdge` 连续插入正反边。三者不解决任何最短路或连通性问题，只是承载它们的数据结构。

```cpp
// AdjEdge[0..1] 依次是边的终点与边权。
using AdjEdge = array<int, 2>;
using AdjList = vector<vector<AdjEdge>>;
using AdjMatrix = vector<vector<int>>;

class ForwardStar
{
    // head[u] 保存点 u 的第一条出边编号，-1 表示没有出边。
    vector<int> head;
    // e[i][0..3] 依次是第 i 条边的终点、边权、下一条同起点边和起点。
    vector<array<int, 4>> e;

  public:
    ForwardStar(int n) : head(n, -1)
    {
        // n 是点数；构造一张没有边的图，无返回值。
    }

    int add(int u, int v, int w = 1)
    {
        // u、v 是边的起点和终点，w 是边权；加入一条有向边并返回边编号。
        e.push_back({v, w, head[u], u});
        head[u] = e.size() - 1;
        return head[u];
    }

    int addResEdge(int u, int v, int w, int rw = 0)
    {
        // u、v 是正向边端点，w、rw 是正反边初值；返回正向边编号。
        int id = e.size();
        add(u, v, w);
        add(v, u, rw);
        return id;
    }

    const vector<int> &heads() const
    {
```

```
        // 无参数；返回各点第一条出边编号。
        return head;
    }

    const vector<array<int, 4>> &edges() const
    {
        // 无参数；返回链式前向星的边表。
        return e;
    }
};
```

邻接表用 `AdjList g(n)`，无边权时退化为 `vector<vector<int>>`；矩阵用 `AdjMatrix g(n, vector<int>(n, INF))` 并把对角置零；链式前向星构造 `ForwardStar g(n)` 后用 `add(u,v,w)` 加一条有向边，无向边加两次。`addResEdge(u,v,w,rw)` 返回正向边编号 `id`，且只对这一对边保证反向边编号为 `id^1`。邻接表与前向星空间 $O(V+E)$、加边均摊 $O(1)$，矩阵空间 $O(V^2)$、加边 $O(1)$。无权图上求可达性或最少边数直接在邻接表上跑 5.1.002 DFS / BFS；非负边权最短路交给 5.2.001 Dijkstra（稠密图正需要邻接矩阵版）；有负权则改 5.2.002 Bellman-Ford。需要成对反向边、按编号回退的网络流一律成对插边，见 5.8.001 Dinic。

5.1.002 DFS / BFS

两种最基本的遍历。BFS 用队列按边数分层扩展，首次到达某点即是无权图上的最短边数，`d[v]==-1` 兼作未访问标记；DFS 沿一条分支深入到底再回溯，天然给出访问顺序、时间戳与 DFS 搜索树，是后面 Tarjan 系连通性算法的骨架。两者都只处理「能否到达、几条边」，一旦边带非负权就得换 Dijkstra。DFS 的递归深度可达点数量级，链状图需确认系统栈足够，必要时使用 1.2.007 手动扩大栈空间。

```
vector<int> bfs(const vector<vector<int>> &g, int s)
{
    // g 是无权图邻接表，s 是起点；返回到各点的最少边数，不可达点为 -1。
    vector<int> d(g.size(), -1);
    queue<int> q;
    d[s] = 0;
    q.push(s);
    while (!q.empty())
    {
        int u = q.front();
        q.pop();
        for (auto v : g[u])
        {
            if (d[v] == -1)
            {
                d[v] = d[u] + 1;
                q.push(v);
            }
        }
    }
    return d;
}

```

```cpp
vector<int> dfs(const vector<vector<int>> &g, int s)
{
    // g 是图邻接表，s 是起点；返回从 s 出发的递归 DFS 首次访问顺序。
    vector<int> ord, vis(g.size());
    auto work = [&](auto &&self, int u) -> void
    {
        vis[u] = 1;
        ord.push_back(u);
        for (auto v : g[u])
        {
            if (!vis[v])
            {
                self(self, v);
            }
        }
    };
    work(work, s);
    return ord;
}
```

`bfs(g,s)` 返回各点最少边数（不可达为 -1），`dfs(g,s)` 返回首次访问顺序，点编号须在 $[0,n)$。二者均 $O(V+E)$ 时间、$O(V)$ 额外空间，递归 DFS 另有最坏 $O(V)$ 调用栈。边带非负权时最少边数不再等于最短路，改用 5.2.001 Dijkstra；有向无环图上要求「每条边从前指向后」的顺序而非任意访问序时用 5.1.003 拓扑排序。

5.1.003 拓扑排序

拓扑排序把 DAG 的点排成每条边都从前指向后的线性序，是 DAG 上一切递推（最长路、计数、DP）的前置条件，同时兼任判环。这里用 Kahn 算法：先把入度为零的点入队，每取出一个点就删去其全部出边、令新的零入度点入队；若最终排出的点数不足 n，剩下的点必然卡在某个有向环里，据此返回空。队列换成优先队列即可求字典序最小拓扑序，当前普通队列版不保证这一点。

```cpp
optional<vector<int>> topo(const vector<vector<int>> &g)
{
    // g 是有向图邻接表；返回一种拓扑序，图含有向环时返回空 optional。
    vector<int> deg(g.size()), ord;
    for (auto &e : g)
    {
        for (auto v : e)
        {
            deg[v]++;
        }
    }
    queue<int> q;
    for (int i = 0; i < (int)g.size(); i++)
    {
        if (deg[i] == 0)
        {
            q.push(i);
```

```
        }
    }
    while (!q.empty())
    {
        int u = q.front();
        q.pop();
        ord.push_back(u);
        for (auto v : g[u])
        {
            if (--deg[v] == 0)
            {
                q.push(v);
            }
        }
    }
    if ((int)ord.size() != (int)g.size())
    {
        return nullopt;
    }
    return ord;
}
```

`topo(g)` 返回空表示图含有向环，否则返回含全部点的一种拓扑序；$O(V+E)$ 时间、$O(V)$ 额外空间。有向边 `0→1,0→2,1→3,2→3` 的结果必以 0 开头、3 结尾，再加 `3→0` 成环后返回空。排出序后即可按序做 5.1.005 DAG 最长路 的松弛；要求字典序最小方案时把 `queue` 换 `priority_queue`；判环也可用 5.3.001 Tarjan 强连通分量：存在大小大于 1 的强连通分量，或某个单点分量中的点有自环，才说明图中有环。

5.1.004 二分图判定

二分图判定即检查无向图能否两染色使每条边两端异色，等价于「不含奇环」。做法是对每个未染色连通块从颜色 0 起 BFS，沿边把颜色异或 1 传播；一旦碰到已染成同色的邻点，说明存在奇环（树上到两端的路径加这条边构成奇数长度环），返回空。每个连通块独立处理，因此非连通图也能整体给出一组染色。

```
optional<vector<int>> bipartite(const vector<vector<int>> &g)
{
    // g 是无向图邻接表；返回每个点的 0/1 颜色，存在奇环时返回空 optional。
    vector<int> col(g.size(), -1);
    for (int s = 0; s < (int)g.size(); s++)
    {
        if (col[s] != -1)
        {
            continue;
        }
        queue<int> q;
        col[s] = 0;
        q.push(s);
        while (!q.empty())
        {
            int u = q.front();
```

```
            q.pop();
            for (auto v : g[u])
            {
                if (col[v] == -1)
                {
                    col[v] = col[u] ^ 1;
                    q.push(v);
                }
                else if (col[v] == col[u])
                {
                    return nullopt;
                }
            }
        }
    }
    return col;
}
```

无向边须在邻接表里加两个方向再调 `bipartite(g)`：空返回值表示存在奇环，否则返回 $0/1$ 染色；$O(V+E)$ 时间、$O(V)$ 额外空间。四边形 `0-1-2-3-0` 返回合法二染色，三角形 `0-1-2-0` 返回空。判定为二分图后，最大匹配走 5.6.001 匈牙利算法、最小点覆盖走 5.6.007 最小点覆盖、最小边覆盖；若染色本身要动态维护（加边判奇环）则改用带权并查集或 5.5.001 Kruskal 里的并查集扩展。

5.1.005 DAG 最长路

DAG 最长路把拓扑序当成 DP 顺序：按序处理点 u 时，所有指向 u 的路径都已算完，于是沿出边更新终点即可。函数复用 Kahn 拓扑框架先排序兼验 DAG 前提，拓扑序未覆盖全部点时返回空；这表示输入不满足本算法的定义域，并不声称任意有环图的路径权都会无限增大。参数 `s=-1` 时所有点初值为 0，求任意起点的全局最长路；指定 `s` 时用内层 `optional` 区分不可达点，路径和类型由 `Dist` 指定。

```
template <class Dist = long long>
optional<vector<optional<Dist>>> dagLongest(
    const vector<vector<array<int, 2>>> &g, int s = -1)
{
    // g[u] 的每项依次是终点和边权，s 是指定起点；返回各点最长路，图不是 DAG 时返回空。
    vector<int> deg(g.size()), ord;
    for (auto &e : g)
    {
        for (auto [v, w] : e)
        {
            deg[v]++;
        }
    }
    queue<int> q;
    for (int i = 0; i < (int)g.size(); i++)
    {
        if (deg[i] == 0)
        {
            q.push(i);
```

```
        }
    }
    while (!q.empty())
    {
        int u = q.front();
        q.pop();
        ord.push_back(u);
        for (auto [v, w] : g[u])
        {
            if (--deg[v] == 0)
            {
                q.push(v);
            }
        }
    }
    if ((int)ord.size() != (int)g.size())
    {
        return nullopt;
    }
    vector<optional<Dist>> d(g.size());
    if (s == -1)
    {
        fill(d.begin(), d.end(), 0);
    }
    else
    {
        d[s] = 0;
    }
    for (auto u : ord)
    {
        if (!d[u])
        {
            continue;
        }
        for (auto [v, w] : g[u])
        {
            Dist nd = *d[u] + (Dist)w;
            if (!d[v] || nd > *d[v])
            {
                d[v] = nd;
            }
        }
    }
    return d;
}
```

`g[u]` 每项 `array<int,2>` 为终点和边权。`dagLongest(g,s)` 的外层空值表示图不是 DAG；成功时返回各点的最长路，指定 `s` 后内层空值表示该点不可达，`s=-1` 时每个点都可作为起点。调用者必须保证所有参与比较的路径和都可由 `Dist` 表示；需要更宽距离时调用 `dagLongest<__int128>(g,s)`。算法用 $O(V+E)$ 时间、$O(V)$ 额外空间。图不保证无环时可先用 5.3.001 Tarjan 强连通分量 缩点；要求删去某点后的最长路见 5.1.006 DAG 删点最长路；求最短路而非最长路时对非负权用 5.2.001 Dijkstra。

5.1.006 DAG 删点最长路

要对每个点求「删掉它之后剩余 DAG 的最长路径边数」，逐点重跑 DP 是 $O(nm)$，本条用拓扑前后缀把它降到近线性。先拓扑排序，`left[v]` 记以 v 结尾的最长路径长、`right[u]` 记以 u 开头的最长路径长；删去拓扑位置为 i 的点后，一条不经过它的最长路径要么整段落在 i 左侧（前缀 `left` 最大值）、要么整段落在右侧（后缀 `right` 最大值）、要么是一条「跨过位置 i」的边 $u \to v$（`position[u]<i<position[v]`）拼接 `left[u]+1+right[v]`。跨越边按左端点挂到 `starts`，扫描时用大根堆维护当前仍跨过 i 的边的最大贡献，堆顶右端点越界即弹出，于是每个点的答案取前缀、后缀、活跃跨越边三者最大值。

```cpp
class DelDAG
{
    // n 是图的点数。
    int n;
    // graph 是输入 DAG 的邻接表。
    vector<vector<int>> graph;
    // edges 保存全部有向边的起点和终点。
    vector<pair<int, int>> edges;

  public:
    explicit DelDAG(int n) : n(n), graph(n)
    {
        // n 是图的点数；构造一张没有边的 DAG。
    }
    void addEdge(int u, int v)
    {
        // u、v 是边的起点和终点；加入一条有向边。
        graph[u].push_back(v);
        edges.push_back({u, v});
    }

    // 依次删除每个点，返回剩余图中的最长路径边数。
    vector<int> solve() const
    {
        // 无参数；返回依次删除每个点后，剩余 DAG 的最长路径边数。
        vector<int> indegree(n);
        for (auto [u, v] : edges)
        {
            ++indegree[v];
        }
        queue<int> q;
        for (int u = 0; u < n; ++u)
        {
            if (!indegree[u])
            {
                q.push(u);
            }
        }
        vector<int> order;
        while (!q.empty())
        {
            int u = q.front();
```

```
            q.pop();
            order.push_back(u);
            for (int v : graph[u])
            {
                if (!--indegree[v])
                {
                    q.push(v);
                }
            }
        }
        assert((int)order.size() == n); // 调试检查，可删：输入必须是 DAG。
        vector<int> position(n), left(n), right(n);
        for (int i = 0; i < n; ++i)
        {
            position[order[i]] = i;
        }
        for (int u : order)
        {
            for (int v : graph[u])
            {
                left[v] = max(left[v], left[u] + 1);
            }
        }
        for (int u : order | views::reverse)
        {
            for (int v : graph[u])
            {
                right[u] = max(right[u], right[v] + 1);
            }
        }
        vector<vector<pair<int, int>>> starts(n + 1);
        for (auto [u, v] : edges)
        {
            int begin = position[u] + 1, end = position[v];
            if (begin < end)
            {
                starts[begin].push_back({left[u] + 1 + right[v], end});
            }
        }
        vector<int> prefix(n + 1), suffix(n + 1), answer(n);
        for (int i = 0; i < n; ++i)
        {
            prefix[i + 1] = max(prefix[i], left[order[i]]);
        }
        for (int i = n - 1; i >= 0; --i)
        {
            suffix[i] = max(suffix[i + 1], right[order[i]]);
        }
        priority_queue<pair<int, int>> active;
        for (int i = 0; i < n; ++i)
        {
            for (auto item : starts[i])
```

```
            {
                active.push(item);
            }
            while (!active.empty() && active.top().second <= i)
            {
                active.pop();
            }
            answer[order[i]] = max(prefix[i], suffix[i + 1]);
            if (!active.empty())
            {
                answer[order[i]] = max(answer[order[i]], active.top().first);
            }
        }
        return answer;
    }
};
```

`DelDAG g(n)` 建图，`addEdge(u,v)` 加有向边，调用 `solve()` 前必须保证输入是 DAG；代码中的断言只用于调试时发现误用。返回数组第 i 项是删去点 i 后剩余图的最长路径边数。拓扑与前后缀 DP 用 $O(n+m)$ 时间，跨越边用堆维护，合计 $O(n+m\log m)$ 时间、$O(n+m)$ 空间。只需一次性最长路时用 5.1.005 DAG 最长路；若删的是边而非点，跨越集合需按被删边重新定义。

5.1.007 差分约束

差分约束求一组变量满足所有形如 $x_v - x_u \le w$ 的不等式，本质是把每条约束看成边 $u \to v$ 权 w 的松弛 $x_v \le x_u + w$，一组可行解就是最短路。所有初值置零等价于加入一个向每点连零权边的超级源，于是 Bellman-Ford 跑到收敛即得解；若第 n 轮仍能松弛，说明存在负环，约束不相容，返回空。代码就是朴素 n 轮松弛，某轮无更新即提前收敛返回。注意本条只判可行性，若要「最大化」或「最小化」某变量还需按题意选择连边方向和取最长/最短路。

```
template <class Dist = long long>
optional<vector<Dist>> diffCons(int n, const vector<array<int, 3>> &a)
{
    // n 是变量个数；a[i][0..2] 依次是 u、v、w，表示 x[v]-x[u]<=w；无解返回空，否则返回一组可行 x。
    vector<Dist> x(n);
    if (n == 0)
    {
        return x;
    }
    for (int k = 0; k < n; k++)
    {
        bool upd = false;
        for (auto [u, v, w] : a)
        {
            assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
            Dist nx = x[u] + (Dist)w;
            if (x[v] > nx)
            {
                x[v] = nx;
```

```
                upd = true;
            }
        }
        if (!upd)
        {
            return x;
        }
    }
    return nullopt;
}
```

`diffCons(n,a)` 中每个 `array<int,3>` 依次为 u, v, w，返回空表示约束含负环无解，否则返回一组满足全部约束的 `Dist` 势值；默认 `Dist=long long`，调用者须保证可行势值和每次松弛均可表示，需要宽势值时调用 `diffCons<__int128>(n,a)`。因初值全零，它给出一组逐次向下松弛得到的可行解，整体平移仍合法。算法用 $O(VE)$ 时间、$O(V)$ 额外空间。约束 `{0,1,3},{1,2,4},{2,0,-8}` 无解，把 -8 改成 -7 后可行。约束稠密、需要队列优化时见 5.2.003 SPFA；全部边非负、只是普通最短路则用 5.2.001 Dijkstra。

5.2 最短路

5.2.001 Dijkstra：稀疏图与稠密图

Dijkstra 求非负边权图的单源最短路。稀疏版使用不支持 decrease-key 的二叉堆，更新后直接压入新记录，再用 `du≠d[u]` 跳过旧记录；堆内最多有 $O(E)$ 个记录，所以按当前实现严格写作 $O((V+E)\log E)$，简单图下因 $E = O(V^2)$ 可化成常见的 $O((V+E)\log V)$。稠密版直接用 $O(V^2)$ 扫描选点，省去堆常数。

```
// WEdge[0..1] 依次是有向边终点和非负边权。
using WEdge = array<int, 2>;

template <class Dist = long long>
vector<optional<Dist>> dijkstraSp(const vector<vector<WEdge>> &g, int s)
{
    // g 是稀疏图邻接表，s 是源点；返回各点最短距离，不可达点为空。
    int n = g.size();
    vector<optional<Dist>> d(n);
    priority_queue<pair<Dist, int>, vector<pair<Dist, int>>, greater<>> q;
    d[s] = 0;
    q.push({0, s});
    while (!q.empty())
    {
        auto [du, u] = q.top();
        q.pop();
        if (!d[u] || du ≠ *d[u])
        {
            continue;
        }
        for (auto [v, w] : g[u])
        {
```

```
            assert(w >= 0); // 调试检查，可删：Dijkstra 要求边权非负。
            Dist ew = (Dist)w;
            Dist nd = du + ew;
            if (!d[v] || nd < *d[v])
            {
                d[v] = nd;
                q.push({nd, v});
            }
        }
    }
    return d;
}

template <class Dist = long long>
vector<optional<Dist>> dijkstraDn(
    const vector<vector<optional<int>>> &w, int s)
{
    // w 是邻接矩阵，空值表示缺边，s 是源点；返回各点最短距离，不可达点为空。
    int n = w.size();
    vector<optional<Dist>> d(n);
    vector<bool> vis(n);
    d[s] = 0;
    for (int it = 0; it < n; it++)
    {
        int u = -1;
        for (int i = 0; i < n; i++)
        {
            if (!vis[i] && d[i] && (u < 0 || *d[i] < *d[u]))
            {
                u = i;
            }
        }
        if (u < 0)
        {
            break;
        }
        vis[u] = true;
        for (int v = 0; v < n; v++)
        {
            if (w[u][v])
            {
                assert(*w[u][v] >= 0); // 调试检查，可删：Dijkstra 要求边权非负。
                Dist z = (Dist)*w[u][v];
                Dist nd = *d[u] + z;
                if (!d[v] || nd < *d[v])
                {
                    d[v] = nd;
                }
            }
        }
    }
    return d;
```

```
}
```

稀疏图调 `dijkstraSp(g,s)`,`g[u]` 每项 `WEdge` 为终点与非负边权;稠密图调 `dijkstraDn(w,s)`,空 `optional` 表示缺边，对角填 0。两版都用空距离表示不可达；调用者必须保证松弛中间量与有限最短路可由 `Dist` 表示，需要更宽距离时显式调用 `dijkstraSp<__int128>` 或 `dijkstraDn<__int128>`。上面的五边样例从 0 出发距离为 `0,2,3,4`。稀疏版严格界为 $O((V+E)\log E)$，简单图下常写 $O((V+E)\log V)$；稠密版为 $O(V^2)$，额外空间均为 $O(V)$。含负边时改用 5.2.002 Bellman-Ford。

5.2.002 Bellman-Ford

Bellman-Ford 允许负边权求单源最短路：反复扫描全部边，最多 $V-1$ 轮即得到无负环时的最短路；若第 V 轮仍能松弛，被松弛点位于或可达一个负环。距离用 `optional<Dist>` 分开表示不可达和有限值，避免有限哨兵截断合法答案；`p` 记前驱，检测到负环后先沿前驱回退 n 步进入环，再收集一组环上顶点。默认 `Dist=long long`，需要宽距离时显式实例化。`DEdge` 是后续 SPFA 和 Johnson 共用的有向边类型。

```
struct DEdge
{
    // x、y 是有向边的起点和终点。
    int x, y;
    // w 是边权。
    int w;
};

template <class Dist = long long> struct Bellman
{
    // d[v] 是源点到 v 的 Dist 最短距离，空值表示不可达。
    vector<optional<Dist>> d;
    // cycle 是一组源点可达的负环顶点，空数组表示没有这种负环。
    vector<int> cycle;
};

template <class Dist = long long>
Bellman<Dist> bellmanFord(int n, const vector<DEdge> &e, int s)
{
    // n 是点数，e 是边表，s 是源点；返回最短距离及一组可达负环。
    vector<optional<Dist>> d(n);
    vector<int> p(n, -1);
    d[s] = 0;
    int x = -1;
    for (int z = 0; z < n; z++)
    {
        x = -1;
        for (auto [u, v, w] : e)
        {
            if (!d[u])
            {
                continue;
            }
            Dist nd = *d[u] + (Dist)w;
```

```
            if (!d[v] || nd < *d[v])
            {
                d[v] = nd;
                p[v] = u;
                x = v;
            }
        }
    }
    vector<int> cycle;
    if (x != -1)
    {
        for (int i = 0; i < n; i++)
        {
            x = p[x];
        }
        int y = x;
        do
        {
            cycle.push_back(y);
            y = p[y];
        } while (y != x);
        reverse(cycle.begin(), cycle.end());
    }
    return {d, cycle};
}
```

`bellmanFord(n,e,s)` 返回的 `Bellman<Dist>::d` 用空值表示不可达，`cycle` 非空时给出一组从 s 可达的负环顶点。调用者必须保证松弛中间量与有限距离可由 `Dist` 表示，需要更宽距离时调用 `bellmanFord<__int128>`。算法最坏用 $O(nm)$ 时间、$O(n+m)$ 空间。需要同时区分有限距离、不可达与可由负环到达的负无穷点时，调用本条附带的 `negShortest(n,edges,s)`；它维护最短路树并传播负环影响，随机数据上较快，但最坏复杂度不作多项式保证，要求稳定上界时仍用 Bellman-Ford。边全非负时用 5.2.001 Dijkstra；只需队列式判负环时见 5.2.003 SPFA、SLF 优化与负环判定/输出；全源稀疏图见 5.2.005 Johnson 全源最短路。

5.2.003 SPFA、SLF 优化与负环判定/输出

SPFA 是 Bellman-Ford 的队列优化：只把成功松弛的点重新入队，避免每轮盲扫全部边。负环判定维护当前最短路的边数 `len`，一旦某点的边数达到 n，这条更短路径必然含环，返回空。它最坏仍是 $O(nm)$，容易被构造数据卡满；稳定性要求高时用 Bellman-Ford，边全非负时用 Dijkstra。本条复用 5.2.002 Bellman-Ford 的 `DEdge`，距离默认是 `Dist=long long`，并在负数加法前检查上下界。

```
// 依赖 5.2.002 Bellman-Ford
template <class Dist = long long>
optional<vector<optional<Dist>>> spfa(
    int n, const vector<DEdge> &edges, int s)
{
    // n 是点数，edges 是边表，s 是源点；有可达负环时返回空，否则返回最短距离。
    vector<vector<pair<int, Dist>>> e(n);
    for (auto [x, y, w] : edges)
```

```
    {
        e[x].push_back({y, w});
    }
    vector<optional<Dist>> d(n);
    vector<int> in(n), len(n);
    queue<int> q;
    d[s] = 0;
    in[s] = 1;
    q.push(s);
    while (!q.empty())
    {
        int x = q.front();
        q.pop();
        in[x] = 0;
        for (auto [y, w] : e[x])
        {
            Dist nd = *d[x] + w;
            if (d[y] && *d[y] <= nd)
            {
                continue;
            }
            d[y] = nd;
            len[y] = len[x] + 1;
            if (len[y] >= n)
            {
                return nullopt;
            }
            if (!in[y])
            {
                in[y] = 1;
                q.push(y);
            }
        }
    }
    return d;
}
```

`spfa(n,edges,s)` 的外层空值表示源点可达负环；成功时，内层空距离表示该点不可达，其余距离默认是 `long long`。调用者必须保证松弛中间量与有限最短路可由 `Dist` 表示；需要更宽距离时调用 `spfa<__int128>`。最坏用 $O(nm)$ 时间、$O(n+m)$ 空间。要输出具体负环顶点，用 5.2.002 Bellman-Ford 的前驱回溯版；边全非负时 5.2.001 Dijkstra 更稳。

5.2.004 Floyd 与最小环

Floyd 按「允许经过的中间点集合逐个扩大」做 DP：处理到第 k 个中间点时，用经过 k 的路径更新所有点对，全程 $O(n^3)$，适合稠密图全点对最短路，也容忍负边，但最短路语义要求相关路径上没有负环。无向图最小环利用同一层次结构：在把 k 并入中间点集合之前，$d[i][j]$ 只经过编号小于 k 的点，此时组合 $d[i][j]+a[i][k]+a[k][j]$ 得到一个以 k 为最大编号点的环。必须先统计环，再用 k 更新距离。矩阵空值表示缺边或不可达，有限值类型由 `Dist` 指定。

```cpp
template <class Dist = long long>
vector<vector<optional<Dist>>> floyd(vector<vector<optional<Dist>>> d)
{
    // d 是初始距离矩阵，空值表示没有路径；返回全点对最短距离。
    int n = d.size();
    for (int k = 0; k < n; k++)
    {
        for (int i = 0; i < n; i++)
        {
            if (!d[i][k])
            {
                continue;
            }
            for (int j = 0; j < n; j++)
            {
                if (d[k][j])
                {
                    Dist nd = *d[i][k] + *d[k][j];
                    if (!d[i][j] || nd < *d[i][j])
                    {
                        d[i][j] = nd;
                    }
                }
            }
        }
    }
    return d;
}

template <class Dist = long long>
optional<Dist> minCycle(const vector<vector<optional<Dist>>> &a)
{
    // a 是非负权无向简单图的邻接矩阵；返回最小环权值，无环时返回空。
    int n = a.size();
    auto d = a;
    optional<Dist> ans;
    for (int k = 0; k < n; k++)
    {
        for (int i = 0; i < k; i++)
        {
            for (int j = i + 1; j < k; j++)
            {
                if (d[i][j] && a[i][k] && a[k][j])
                {
                    Dist cur = *d[i][j] + *a[i][k] + *a[k][j];
                    if (!ans || cur < *ans)
                    {
                        ans = cur;
                    }
                }
            }
        }
```

```
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (d[i][k] && d[k][j])
                {
                    Dist nd = *d[i][k] + *d[k][j];
                    if (!d[i][j] || nd < *d[i][j])
                    {
                        d[i][j] = nd;
                    }
                }
            }
        }
    }
    return ans;
}
```

`floyd(d)` 要求对角为 0、缺边为空，返回全点对最短距离；调用后若 `d[i][i]<0`，说明从 i 可达负环。`minCycle(a)` 接受非负权无向简单图邻接矩阵，无环时返回空值。调用者必须保证有限路径与环权可由 `Dist` 表示；需要更宽距离时传入 `optional<__int128>` 矩阵。两者均用 $O(n^3)$ 时间、$O(n^2)$ 空间。稀疏图全源最短路见 5.2.005 Johnson 全源最短路；单源见 5.2.001 Dijkstra；负环输出见 5.2.002 Bellman-Ford。

5.2.005 Johnson 全源最短路

Johnson 让含负边的稀疏图也能用 Dijkstra 求全源最短路。关键是势函数重标权：先跑一次 Bellman-Ford 求出每点势 $h(u)$（等价于加超级源到各点的最短路），把边权重定义为 $w'(u,v) = w(u,v) + h(u) - h(v)$。由三角不等式 $h(v) \le h(u) + w$ 知 $w' \ge 0$，且任意路径 $u \rightsquigarrow v$ 的新权只比原权多常数 $h(u) - h(v)$，最短路结构不变；于是从每点跑 Dijkstra，最后减去 $h(v) - h(source)$ 还原真实距离。Bellman-Ford 第 n 轮仍能松弛则存在负环，返回空。

```
// 依赖 5.2.002 Bellman-Ford
template <class Dist = long long> struct Johnson
{
    // n 是图的点数。
    int n;
    // e 保存全部有向边。
    vector<DEdge> e;

    explicit Johnson(int n) : n(n)
    {
        // n 是图的点数；构造一张没有边的图。
    }
    void addEdge(int u, int v, int w)
    {
        // u、v 是边的起点和终点，w 是边权；加入一条有向边。
        e.push_back({u, v, w});
    }

```

```
    optional<vector<vector<optional<Dist>>>> solve() const
    {
        // 无参数；返回全点对最短距离，图含负环时返回空。
        vector<Dist> h(n);
        for (int k = 0; k < n; k++)
        {
            bool upd = false;
            for (auto [u, v, w] : e)
            {
                Dist nh = h[u] + (Dist)w;
                if (h[v] > nh)
                {
                    h[v] = nh;
                    upd = true;
                }
            }
            if (!upd)
            {
                break;
            }
            if (k == n - 1)
            {
                return nullopt;
            }
        }
        vector<vector<pair<int, Dist>>> g(n);
        for (auto [u, v, w] : e)
        {
            Dist nw = (Dist)w + h[u] - h[v];
            assert(nw >= 0); // 调试检查，可删：势函数应使重标权非负。
            g[u].push_back({v, nw});
        }
        vector ans(n, vector<optional<Dist>>(n));
        for (int s = 0; s < n; s++)
        {
            priority_queue<pair<Dist, int>, vector<pair<Dist, int>>, greater<>> q;
            ans[s][s] = 0;
            q.push({0, s});
            while (!q.empty())
            {
                auto [du, u] = q.top();
                q.pop();
                if (!ans[s][u] || du != *ans[s][u])
                {
                    continue;
                }
                for (auto [v, w] : g[u])
                {
                    Dist nd = du + w;
                    if (!ans[s][v] || nd < *ans[s][v])
                    {
                        ans[s][v] = nd;
```

```
                        q.push({nd, v});
                    }
                }
            }
            for (int v = 0; v < n; ++v)
            {
                if (ans[s][v])
                {
                    ans[s][v] = *ans[s][v] + h[v] - h[s];
                }
            }
        }
        return ans;
    }
};
```

`Johnson<> g(n)` 建图，`addEdge(u,v,w)` 允许负边；`solve()` 的外层空值表示图含负环，成功时矩阵中的空值表示两点不可达，有限距离默认是 `long long`。势、重标边权、堆键和答案都使用 `Dist`；调用者必须保证这些中间量均可表示，需要更宽距离时构造 `Johnson<__int128>`。势函数用 $O(nm)$ 时间，随后 n 次 Dijkstra 用 $O(n(m+n)\log n)$ 时间，总空间为 $O(n^2+m)$。全部边非负时直接逐点调用 5.2.001 Dijkstra；稠密图改用 5.2.004 Floyd 与最小环。

5.2.006 区间图最短路

当题目要求「一个点向一段连续下标批量连同权边」或反向时，逐点连边是 $O(n)$，用线段树优化建图能压到 $O(\log n)$ 条。建两棵零权线段树：向外树的父节点向两个儿子连边、叶子连到原点，代表「从某区间节点能走到区间内每个点」；向内树方向相反，代表「区间内每点都能汇到该区间节点」。一个区间在线段树上被拆成 $O(\log n)$ 个节点，于是「点 u 到区间 $[l,r)$」只需从 u 向向外树的 $O(\log n)$ 个覆盖节点各连一条边，「区间到点」对称地连向内树。最后在这张含辅助点的图上跑 Dijkstra，权值必须非负。

```
template<class Dist = long long>
class IntGraph
{
    // n 是原始点数，g 是包含线段树辅助点的带权有向图。
    int n;
    vector<vector<array<int, 2>>> g;
    // out[p]、in[p] 是原区间节点 p 对应的向外树、向内树图节点编号。
    vector<int> out, in;

    int node()
    {
        // 无参数；新增一个空图节点并返回编号。
        g.push_back({});
        return g.size() - 1;
    }

    void edge(int u, int v, int w)
    {
        // u、v 是图节点，w 是非负边权；加入 u 到 v 的有向边，无返回值。
```

```
        g[u].push_back({v, w});
    }

    void build(int p, int l, int r)
    {
        // p 是线段树节点，[l,r) 是原点区间；建立两棵方向相反的零权树，无返回值。
        out[p] = node();
        in[p] = node();
        if (r - l == 1)
        {
            edge(out[p], l, 0);
            edge(l, in[p], 0);
            return;
        }
        int m = (l + r) / 2;
        build(p << 1, l, m);
        build(p << 1 | 1, m, r);
        edge(out[p], out[p << 1], 0);
        edge(out[p], out[p << 1 | 1], 0);
        edge(in[p << 1], in[p], 0);
        edge(in[p << 1 | 1], in[p], 0);
    }

    void pointRange(int p, int l, int r, int u, int ql, int qr, int w)
    {
        // p、[l,r) 是线段树状态，u 是起点，[ql,qr) 是终点范围，w 是边权；加入压缩边。
        if (qr <= l || r <= ql)
        {
            return;
        }
        if (ql <= l && r <= qr)
        {
            edge(u, out[p], w);
            return;
        }
        int m = (l + r) / 2;
        pointRange(p << 1, l, m, u, ql, qr, w);
        pointRange(p << 1 | 1, m, r, u, ql, qr, w);
    }

    void rangePoint(int p, int l, int r, int ql, int qr, int v, int w)
    {
        // p、[l,r) 是线段树状态，[ql,qr) 是起点范围，v 是终点，w 是边权；加入压缩边。
        if (qr <= l || r <= ql)
        {
            return;
        }
        if (ql <= l && r <= qr)
        {
            edge(in[p], v, w);
            return;
        }
```

```
        int m = (l + r) / 2;
        rangePoint(p << 1, l, m, ql, qr, v, w);
        rangePoint(p << 1 | 1, m, r, ql, qr, v, w);
    }

public:
    IntGraph(int n) : n(n), g(n), out(4 * n), in(4 * n)
    {
        // n 是正的原始点数；建立区间图辅助节点。
        assert(n > 0); // 调试检查，可删。
        build(1, 0, n);
    }

    void addEdge(int u, int v, int w)
    {
        // u、v 是原始点，w 是非负边权；加入单点到单点边。
        assert(0 <= u && u < n && 0 <= v && v < n && w >= 0); // 调试检查，可删。
        edge(u, v, w);
    }

    void addPointRng(int u, int l, int r, int w)
    {
        // u 是起点，[l,r) 是终点范围，w 是非负边权；表示 r-l 条逻辑边。
        assert(0 <= u && u < n && 0 <= l && l <= r && r <= n && w >= 0); // 调试检查，可删。
        pointRange(1, 0, n, u, l, r, w);
    }

    void addRngPoint(int l, int r, int v, int w)
    {
        // [l,r) 是起点范围，v 是终点，w 是非负边权；表示 r-l 条逻辑边。
        assert(0 <= l && l <= r && r <= n && 0 <= v && v < n && w >= 0); // 调试检查，可删。
        rangePoint(1, 0, n, l, r, v, w);
    }

    vector<optional<Dist>> dijkstra(int s) const
    {
        // s 是原始源点；返回到全部原始点的最短路，不可达点为空。
        assert(0 <= s && s < n); // 调试检查，可删。
        vector<optional<Dist>> d(g.size());
        priority_queue<pair<Dist, int>, vector<pair<Dist, int>>, greater<>> q;
        d[s] = 0;
        q.push({0, s});
        while (!q.empty())
        {
            auto [du, u] = q.top();
            q.pop();
            if (!d[u] || du ≠ *d[u])
            {
                continue;
            }
            for (auto [v, w] : g[u])
            {
```

```
                Dist nd = du + (Dist)w;
                if (!d[v] || nd < *d[v])
                {
                    d[v] = nd;
                    q.push({nd, v});
                }
            }
        }
        d.resize(n);
        return d;
    }
};
```

构造 `IntGraph q(n)` 时要求 $n > 0$，三个加边接口都要求端点有效、区间左闭右开且边权非负；断言仅用于调试。`IntGraph<Dist>::dijkstra(s)` 返回原始点距离，空值表示不可达；调用者必须保证有限距离可由 `Dist` 表示，需要更宽距离时可显式使用 `IntGraph<__int128>`。$n = 5$、$0 \to [1,4)$ 权 7、$[1,3) \to 4$ 权 2、$3 \to 4$ 权 1 时，从 0 的距离为 `0,7,7,7,8`。辅助点与边为 $O(n)$，每次区间连边新增 $O(\log n)$ 条实际边，Dijkstra 用 $O((V+E)\log V)$ 时间。没有区间需求时直接用 5.2.001 Dijkstra。

5.2.007 平面图对偶最短路

在已知平面嵌入（面及相邻关系）时，「穿越一系列边从面 s 走到面 t 的最小总代价」可转成对偶图最短路：原图每条边恰分隔两个面，就在这两个面的对偶点间连一条权等于穿越代价的边，面路径的最短路即最小穿越代价，这正是平面图 s-t 最小割转最短路的经典技巧。函数本体就是一次记录前驱的 Dijkstra，最后回溯出一条面序列。前提是面信息已经给出——只有原图点边邻接而无嵌入时无法直接用。

```
template<class Dist = long long>
struct DualPath
{
    // dist 是最小穿越代价。
    Dist dist;
    // path 是从起始面到终止面的面编号序列。
    vector<int> path;
};

template<class Dist = long long>
optional<DualPath<Dist>> dualShortest(
    int faces, const vector<array<int, 3>> &dualEdges, int s, int t)
{
    // faces 是面数，边记录相邻面与代价，s、t 是起终面；不可达时返回空。
    assert(faces > 0 && 0 <= s && s < faces && 0 <= t && t < faces); // 调试检查，可删。
    vector<vector<array<int, 2>>> g(faces);
    for (auto [u, v, w] : dualEdges)
    {
        assert(0 <= u && u < faces && 0 <= v && v < faces && w >= 0); // 调试检查，可删。
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
    vector<optional<Dist>> d(faces);
```

```
    vector<int> pre(faces, -1);
    priority_queue<pair<Dist, int>, vector<pair<Dist, int>>, greater<>> q;
    d[s] = 0;
    q.push({0, s});
    while (!q.empty())
    {
        auto [du, u] = q.top();
        q.pop();
        if (!d[u] || du != *d[u])
        {
            continue;
        }
        for (auto [v, w] : g[u])
        {
            Dist nd = du + (Dist)w;
            if (!d[v] || nd < *d[v])
            {
                d[v] = nd;
                pre[v] = u;
                q.push({nd, v});
            }
        }
    }
    if (!d[t])
    {
        return nullopt;
    }
    vector<int> path;
    for (int u = t; u != -1; u = pre[u])
    {
        path.push_back(u);
    }
    reverse(path.begin(), path.end());
    return DualPath<Dist>{*d[t], path};
}
```

先给所有面编号，为每条可穿越原边提供 {左面，右面，代价}，并保证面编号有效、代价非负。`dualShortest(faces,dualEdges,s,t)` 默认以 `long long` 保存距离，返回 `DualPath<Dist>`，其中 `dist` 是最小代价，`path` 是一条最短面路径；不连通时返回空。调用者必须保证路径代价可由 `Dist` 表示，需要更宽距离时可显式调用 `dualShortest<__int128>`。算法用 $O((F+D)\log F)$ 时间、$O(F+D)$ 空间，建立平面嵌入的成本另计。它与原图 s-t 最小割对应，一般图最小割见 5.8.006 最大流最小割与建图模型。

5.2.008 最短路径树

最短路径树从每个可达非源点选一条满足 $d[v]=d[u]+w$ 的父边，便于回溯路径或继续做树上处理。不能在 Dijkstra 的等长松弛中各自挑最小父点：零权等距分量内可能互相选为父亲而成环。这里先求完整距离，再从源点沿最短路子图做一次 BFS；只有首次到达点 v 时才记录父边，因此父指针一定构成一棵有根树，零权环也不会破坏结构。

```cpp
template<class Dist = long long>
pair<vector<optional<Dist>>, vector<int>> shortestTree(
    const vector<vector<WEdge>> &g, int s)
{
    // g 是非负权有向图，s 是源点；返回最短距离与一棵最短路径树的父节点。
    int n = g.size();
    assert(0 <= s && s < n); // 调试检查，可删。
    auto d = dijkstraSp<Dist>(g, s);
    vector<int> par(n, -1), vis(n);
    queue<int> q;
    vis[s] = 1;
    q.push(s);
    while (!q.empty())
    {
        int u = q.front();
        q.pop();
        for (auto [v, w] : g[u])
        {
            Dist z = (Dist)w;
            if (!vis[v] && d[v] && *d[u] + z == *d[v])
            {
                vis[v] = 1;
                par[v] = u;
                q.push(v);
            }
        }
    }
    return {d, par};
}
```

本条依赖 5.2.001 Dijkstra：稀疏图与稠密图 的 `WEdge` 与 `dijkstraSp`。`Dist` 默认是 `long long`；若最短距离可能超出 64 位，可显式调用 `shortestTree<__int128>`。返回值为 `{d,par}`，空距离表示不可达，源点和不可达点的父亲均为 -1；从任一可达点沿 `par` 必能回到源点。算法用 $O((V+E)\log V)$ 时间、$O(V)$ 额外空间。含负边时先用 5.2.002 Bellman-Ford 求距离，再按相同办法遍历最短路子图。

5.2.009 单源最短路/次短路计数

这条统计从源点到每点的最短、严格次短两种游走长度及各自条数。先用 Dijkstra 式扩展只求每点最小的两个不同距离，再建立两层状态图：状态 $(v,0/1)$ 表示到 v 的最短/次短距离，原边恰好把一个已知距离转成另一个已知距离时才加入状态边，最后按拓扑序计数。这样相同距离的全部前驱都会在后继之前汇总，不依赖优先队列对并列元素的弹出顺序。边权要求非负，且这张可达状态图必须无环；若有零权环，按游走语义会产生无限条等长方案，函数返回空。

```cpp
template<class Dist = long long>
struct ShortestCnt
{
    // d[v][0..1] 是到 v 的最短、严格次短距离，空值表示不存在。
    vector<array<optional<Dist>, 2>> d;
    // cnt[v][0..1] 是对应距离的游走条数模 mod。
```

```
    vector<array<int, 2>> cnt;
};

template<class Dist = long long>
optional<ShortestCnt<Dist>> shortestCnt(
    const vector<vector<pair<int, int>>> &g, int s, int mod)
{
    // g 是非负权邻接表，s 是源点，mod 是正模数；零权环导致计数无穷时返回空。
    int n = g.size();
    assert(0 <= s && s < n); // 调试检查，可删。
    assert(mod > 0); // 调试检查，可删。
    ShortestCnt<Dist> z{vector<array<optional<Dist>, 2>>(n), vector<array<int, 2>>(n)};
    using Q = pair<Dist, int>;
    priority_queue<Q, vector<Q>, greater<>> q;
    z.d[s][0] = 0;
    q.push({0, s});
    while (!q.empty())
    {
        auto [du, u] = q.top();
        q.pop();
        if ((!z.d[u][0] || du != *z.d[u][0]) &&
            (!z.d[u][1] || du != *z.d[u][1]))
        {
            continue;
        }
        for (auto [v, w] : g[u])
        {
            assert(w >= 0); // 调试检查，可删。
            Dist ew = (Dist)w;
            Dist nd = du + ew;
            if (!z.d[v][0] || nd < *z.d[v][0])
            {
                z.d[v][1] = z.d[v][0];
                z.d[v][0] = nd;
                q.push({nd, v});
            }
            else if (nd > *z.d[v][0] && (!z.d[v][1] || nd < *z.d[v][1]))
            {
                z.d[v][1] = nd;
                q.push({nd, v});
            }
        }
    }
    vector<vector<int>> e(2 * n);
    vector<int> deg(2 * n), use(2 * n);
    for (int u = 0; u < n; u++)
    {
        for (int k = 0; k < 2; k++)
        {
            if (!z.d[u][k])
            {
                continue;
```

```
            }
            int x = 2 * u + k;
            use[x] = 1;
            for (auto [v, w] : g[u])
            {
                Dist ew = (Dist)w;
                Dist nd = *z.d[u][k] + ew;
                for (int p = 0; p < 2; p++)
                {
                    if (z.d[v][p] && nd == *z.d[v][p])
                    {
                        e[x].push_back(2 * v + p);
                        deg[2 * v + p]++;
                    }
                }
            }
        }
    }
    queue<int> que;
    int states = 0, done = 0;
    for (int x = 0; x < 2 * n; x++)
    {
        if (use[x])
        {
            states++;
            if (deg[x] == 0)
            {
                que.push(x);
            }
        }
    }
    z.cnt[s][0] = 1 % mod;
    while (!que.empty())
    {
        int x = que.front();
        que.pop();
        done++;
        for (auto y : e[x])
        {
            int v = y / 2, k = y % 2;
            int add = z.cnt[x / 2][x % 2];
            if (z.cnt[v][k] >= mod - add)
            {
                z.cnt[v][k] -= mod - add;
            }
            else
            {
                z.cnt[v][k] += add;
            }
            if (--deg[y] == 0)
            {
                que.push(y);
```

```
            }
        }
    }
    if (done != states)
    {
        return nullopt;
    }
    return z;
}
```

`shortestCnt(g,s,mod)` 要求 `mod>0`，返回空表示可达的最短/次短状态图含环，按游走语义计数无穷；否则 `d[v][0/1]` 与 `cnt[v][0/1]` 分别是最短/严格次短距离及其条数模 `mod`。调用者必须保证距离中间量可由 `Dist` 表示，需要更宽距离时可显式调用 `shortestCnt<__int128>`。不存在的名次为空。算法用 $O((n+m)\log n)$ 时间、$O(n+m)$ 空间。只要一棵最短路径树时用 5.2.008 最短路径树，只要距离时用 5.2.001 Dijkstra。

5.2.010 第 K 短路（Dijkstra / A*）

第 K 短路按总长从小到大枚举源到汇的游走（允许重复点边），本条用 A* 求。先在反图上跑一次 Dijkstra，得到每点到汇 t 的精确最短距离 $h(u)$，它是可采纳且一致的估价；主搜索的优先队列按 $f=$「已走距离 $+h(u)$」取最小，当汇点第 k 次出队时的实际距离就是第 K 短。用 `cnt[u]` 限制每点最多扩展 k 次即可保证复杂度。`keep0` 控制 $s=t$ 时那条长度为零的空游走是否计入排名。零权环下路径条数无限，需靠这种「游走」定义配合出队计数才有良定义。

```
struct KEdge
{
    // u、v 是有向边的起点和终点。
    int u, v;
    // w 是非负边权。
    int w;
};

template<class Dist = long long>
optional<Dist> kthWalk(
    int n, const vector<KEdge> &e, int s, int t,
    unsigned long long k, bool keep0 = true)
{
    // n 是点数，e 是边表，s、t 是起终点，k 从 1 开始；返回第 k 小游走长度。
    assert(k >= 1 && 0 <= s && s < n && 0 <= t && t < n); // 调试检查，可删。
    vector<vector<pair<int, int>>> g(n), rg(n);
    for (auto [u, v, w] : e)
    {
        assert(w >= 0); // 调试检查，可删。
        g[u].push_back({v, w});
        rg[v].push_back({u, w});
    }
    vector<optional<Dist>> h(n);
    priority_queue<pair<Dist, int>, vector<pair<Dist, int>>, greater<>> dq;
    h[t] = 0;
```

```
    dq.push({0, t});
    while (!dq.empty())
    {
        auto [d, u] = dq.top();
        dq.pop();
        if (!h[u] || d ≠ *h[u])
        {
            continue;
        }
        for (auto [v, w] : rg[u])
        {
            Dist ew = (Dist)w;
            Dist nd = d + ew;
            if (!h[v] || nd < *h[v])
            {
                h[v] = nd;
                dq.push({nd, v});
            }
        }
    }
    if (!h[s])
    {
        return nullopt;
    }
    if (s == t && keep0)
    {
        if (k == 1)
        {
            return (Dist)0;
        }
        k--;
    }
    using State = tuple<Dist, Dist, int>;
    priority_queue<State, vector<State>, greater<>> q;
    vector<unsigned long long> cnt(n);
    for (auto [v, w] : g[s])
    {
        if (!h[v])
        {
            continue;
        }
        Dist ew = (Dist)w;
        Dist nd = ew;
        q.push({nd + *h[v], nd, v});
    }
    while (!q.empty())
    {
        auto [f, d, u] = q.top();
        q.pop();
        if (cnt[u] == k)
        {
            continue;
```

```
        }
        cnt[u]++;
        if (u == t && cnt[u] == k)
        {
            return d;
        }
        for (auto [v, w] : g[u])
        {
            if (h[v] && cnt[v] < k)
            {
                Dist ew = (Dist)w;
                Dist nd = d + ew;
                q.push({nd + *h[v], nd, v});
            }
        }
    }
    return nullopt;
}
```

`kthWalk` 要求边权非负且 $k \geq 1$，返回第 k 小有向游走长度，不足 k 条或不可达时返回空。调用者必须保证实际距离、估价与二者之和可由 `Dist` 表示，需要更宽距离时可显式调用 `kthWalk<__int128>`。`keep0=false` 会在 $s = t$ 时跳过唯一的空游走；函数先把源点的每条出边放入队列，因此不会用 `k+1` 表示这个特例。反图 Dijkstra 用 $O((n+m)\log n)$ 时间，主搜索至多弹出每点 k 次、扫描每条出边 k 次，最坏用 $O(km\log(km))$ 时间和 $O(km+n)$ 空间。它枚举允许重复点边的游走；要求简单路径时应改用专门的 K 短简单路算法，不能直接套本条。

5.2.011 0-1 BFS

边权只取 0 或 1 时，用双端队列替代 Dijkstra 的优先队列。队首弹出的点具有当前最小距离：沿 0 边松弛得到的点仍属于当前距离层，压到队首；沿 1 边松弛得到下一层，压到队尾。因此每个点第一次正式弹出时距离已经确定，`done` 让它只扫描一次出边；距离用 `optional` 区分不可达，不借有限哨兵承担加法。若出现负边或大于 1 的边权，这个队列次序便不再成立。

```
// BinaryEdge[0] 是终点，BinaryEdge[1] 是只能取 0 或 1 的边权。
using BinaryEdge = array<int, 2>;

template <class Dist = long long> vector<optional<Dist>> zeroOneBfs(const
↪ vector<vector<BinaryEdge>> &g, int s)
{
    // g 是 0-1 权有向图，s 是源点；返回最短距离，不可达点为空。
    int n = g.size();
    assert(0 <= s && s < n); // 调试检查，可删
    vector<optional<Dist>> d(n);
    vector<bool> done(n);
    deque<int> q;
    d[s] = (Dist)0;
    q.push_front(s);
    while (!q.empty())
    {
```

```
        int u = q.front();
        q.pop_front();
        if (done[u])
        {
            continue;
        }
        done[u] = true;
        for (auto [v, w] : g[u])
        {
            assert(0 <= v && v < n && (w == 0 || w == 1)); // 调试检查，可删
            Dist nd = *d[u] + (Dist)w;
            if (!d[v].has_value() || nd < *d[v])
            {
                d[v] = nd;
                if (w == 0)
                {
                    q.push_front(v);
                }
                else
                {
                    q.push_back(v);
                }
            }
        }
    }
    return d;
}
```

`zeroOneBfs(g,s)` 使用 0 下标有向邻接表，建无向边时需向两端各加入一项；调用者须保证最短距离能由 `Dist` 表示，需要更宽结果时显式调用 `zeroOneBfs<__int128>`。设点数为 n、边数为 m，时间 $O(n+m)$、额外空间 $O(n)$。全为 1 时普通 5.1.002 BFS 更短，一般非负权改用 5.2.001 Dijkstra，存在负权则用 5.2.002 Bellman-Ford 或 5.2.003 SPFA。

```
signed main()
{
    vector<vector<BinaryEdge>> g(6);
    g[0] = {{1, 1}, {2, 0}};
    g[2] = {{1, 0}, {3, 1}};
    g[1] = {{3, 1}};
    g[3] = {{4, 0}};
    auto d = zeroOneBfs(g, 0);
    assert(d[0] == 0 && d[1] == 0 && d[2] == 0);
    assert(d[3] == 1 && d[4] == 1 && !d[5]);
}
```

5.2.012 第 K 短游走（Eppstein）

当 n, m, k 都达到 3×10^5 时，5.2.010 的逐次 A* 扩展会变成 $O(km \log(km))$，只适合小规模或很小的名次。这里保留它的接口和教学价值，另用 Eppstein 的偏离边枚举处理大规模有向非负权图。先在反图上以 t 为源运行 Dijkstra，选出一棵到汇最短路树；非树边 $u \to v$ 的偏离代价为 $w + d(v) - d(u) \geq 0$。

沿最短路树把各点可用偏离边合并为持久左偏堆，堆节点的左右儿子替换当前偏离边，堆根再接入终点处的偏离堆，便恰好枚举全部游走。

```cpp
struct EppsteinEdge
{
    // u、v 是有向边端点，w 是非负边权。
    int u, v, w;
};

template<class Dist = long long>
vector<Dist> eppWalks(int n, const vector<EppsteinEdge> &e, int s, int t,
                              unsigned long long k, bool keep0 = true)
{
    // n 是点数，e 是边表，s、t 是起终点，k 是要枚举的项数；返回前至多 k 小游走长度。
    assert(k >= 1 && 0 <= s && s < n && 0 <= t && t < n);
    vector<vector<array<int, 3>>> g(n), rg(n);
    for (int i = 0; i < (int)e.size(); i++)
    {
        auto [u, v, w] = e[i];
        assert(0 <= u && u < n && 0 <= v && v < n && w >= 0);
        g[u].push_back({v, w, i});
        rg[v].push_back({u, w, i});
    }
    vector<optional<Dist>> d(n);
    vector<int> par(n, -1), pe(n, -1);
    priority_queue<pair<Dist, int>, vector<pair<Dist, int>>, greater<>> q;
    d[t] = 0;
    q.push({0, t});
    while (!q.empty())
    {
        auto [du, u] = q.top();
        q.pop();
        if (!d[u] || du != *d[u])
        {
            continue;
        }
        for (auto [v, w, id] : rg[u])
        {
            Dist nd = du + (Dist)w;
            if (!d[v] || nd < *d[v])
            {
                d[v] = nd;
                par[v] = u;
                pe[v] = id;
                q.push({nd, v});
            }
        }
    }
    if (!d[s]) return {};
    struct Node
    {
        // l、r 是左偏堆儿子编号，rk 是右链长度，to 是偏离边终点，val 是偏离代价。
```

```
        int l = 0, r = 0, rk = 1, to = -1;
        Dist val{};
    };
    vector<Node> hp(1);
    hp[0].rk = 0;
    auto meld = [&](auto &&self, int a, int b) -> int
    {
        // a、b 是持久左偏堆根编号；返回合并后新堆根，不修改旧版本。
        if (!a) return b;
        if (!b) return a;
        if (hp[a].val > hp[b].val) swap(a, b);
        int x = hp.size();
        hp.push_back(hp[a]);
        hp[x].r = self(self, hp[x].r, b);
        if (hp[hp[x].l].rk < hp[hp[x].r].rk) swap(hp[x].l, hp[x].r);
        hp[x].rk = hp[hp[x].r].rk + 1;
        return x;
    };
    vector<vector<int>> child(n);
    for (int u = 0; u < n; u++)
    {
        if (par[u] != -1) child[par[u]].push_back(u);
    }
    vector<int> root(n);
    queue<int> bfs;
    bfs.push(t);
    while (!bfs.empty())
    {
        int u = bfs.front();
        bfs.pop();
        if (par[u] != -1) root[u] = meld(meld, root[u], root[par[u]]);
        for (auto [v, w, id] : g[u])
        {
            if (id == pe[u] || !d[v]) continue;
            // extra 是偏离最短路树后增加的非负长度。
            Dist extra = (Dist)w - *d[u] + *d[v];
            hp.push_back({0, 0, 1, v, extra});
            root[u] = meld(meld, root[u], (int)hp.size() - 1);
        }
        for (int v : child[u])
        {
            bfs.push(v);
        }
    }
    hp.push_back({0, 0, 1, s, *d[s]});
    priority_queue<pair<Dist, int>, vector<pair<Dist, int>>, greater<>> pq;
    pq.push({*d[s], (int)hp.size() - 1});
    bool skip0 = s == t && !keep0;
    vector<Dist> ans;
    ans.reserve(k);
    while (!pq.empty())
    {
```

```
        auto [len, x] = pq.top();
        pq.pop();
        if (skip0)
        {
            skip0 = false;
        }
        else
        {
            ans.push_back(len);
            if (ans.size() == k) return ans;
        }
        for (int y : {hp[x].l, hp[x].r})
        {
            if (y) pq.push({len + hp[y].val - hp[x].val, y});
        }
        int r = root[hp[x].to];
        if (r) pq.push({len + hp[r].val, r});
    }
    return ans;
}

template<class Dist = long long>
optional<Dist> eppWalk(int n, const vector<EppsteinEdge> &e, int s, int t,
                                unsigned long long k, bool keep0 = true)
{
    // 参数含义同 eppWalks；返回第 k 小游走长度，不足 k 条时返回空。
    auto ans = eppWalks<Dist>(n, e, s, t, k, keep0);
    if (ans.size() < k) return nullopt;
    return ans.back();
}
```

`eppWalks` 一次返回前至多 k 项，`eppWalk` 是其第 k 项包装；二者与朴素 `kthWalk` 一样枚举允许重边、重点和同权平行边的游走，`keep0` 只影响 $s = t$ 的零长空游走。前者建反图最短路和持久堆用 $O((n+m)\log n)$ 时间，随后连续输出 k 项用 $O(k\log(n+m))$ 时间，持久堆空间为 $O(m\log n)$；后者实现较短，适合 k 很小或仅作原型。官方大规模第 K 短游走应调用批量接口。两者都要求非负边权；要枚举简单路径时必须使用专门算法。

5.2.013 第 K 短简单路 Yen

求有向图（边权非负）中 s 到 t 代价最小的前 k 条简单路径，路径按边编号区分，所以平行边给出不同路径。Yen 算法逐条产生答案：第一条是普通最短路；已得到第 i 条路径 $A_i = (v_0, \dots, v_\ell)$ 后，对每个偏离点 v_j：

1. 根路径取 A_i 的前 j 条边；
2. 对所有已输出且前 j 条边与根路径相同的路径，禁用它们的第 $j+1$ 条边，保证新路径与已有路径不同；
3. 禁用根路径上 v_j 之前的所有点，保证拼接后仍是简单路径；
4. 求 v_j 到 t 的最短路，与根路径拼成候选。

候选放入按（代价，边序列）排序的集合，每轮取出最小者作为下一条答案。任何第 $i+1$ 短简单路都与某条已输出路径有一段最长公共前缀，并在该前缀末端偏离，因此一定作为候选出现过。

```
struct YenPaths
{
    struct Path
    {
        __int128 cost = 0;    // cost 是路径边权和。
        vector<int> vertices; // vertices 是从起点到终点的顶点序列。
        vector<int> edges;    // edges 是依次经过的边编号，平行边据此区分。

        bool operator<(const Path &o) const
        {
            // 先比代价，再比边编号序列，使候选集合有确定的顺序。
            return tie(cost, edges) < tie(o.cost, o.edges);
        }
    };

    int n; // n 是点数。
    vector<array<int, 3>> edges; // edges[id] 的三维依次是有向边的起点、终点和非负边权。
    vector<vector<int>> adj; // adj[u] 是从 u 出发的边编号。

    YenPaths(int n) : n(n), adj(n)
    {
        // n 是点数，点编号为 [0,n)；构造空图。
    }

    int add(int u, int v, int w)
    {
        // 加入有向边 u->v，w 为非负边权；返回边编号。
        assert(0 <= u && u < n && 0 <= v && v < n && w >= 0); // 调试检查，可删。
        edges.push_back({u, v, w});
        adj[u].push_back(edges.size() - 1);
        return edges.size() - 1;
    }

    optional<Path> shortest(int s, int t, const vector<char> &banV, const vector<char> &banE)
    ↪ const
    {
        // 在删去 banV 中的点和 banE 中的边后求 s 到 t 的最短路；不可达时返回空。
        const __int128 INF = (__int128)1 << 120;
        if (banV[s] || banV[t])
        {
            return nullopt;
        }
        vector<__int128> d(n, INF);
        vector<int> pre(n, -1); // pre[v] 是最短路树上进入 v 的边编号。
        vector<char> done(n);
        priority_queue<pair<__int128, int>, vector<pair<__int128, int>>, greater<>> q;
        d[s] = 0;
        q.push({0, s});
        while (!q.empty())
```

```
        {
            auto [du, u] = q.top();
            q.pop();
            if (done[u])
            {
                continue;
            }
            done[u] = true;
            if (u == t)
            {
                break;
            }
            for (int id : adj[u])
            {
                auto [x, v, w] = edges[id];
                if (banE[id] || banV[v] || done[v])
                {
                    continue;
                }
                if (du + w < d[v])
                {
                    d[v] = du + w;
                    pre[v] = id;
                    q.push({d[v], v});
                }
            }
        }
        if (d[t] == INF)
        {
            return nullopt;
        }
        Path p;
        p.cost = d[t];
        for (int u = t;; u = edges[pre[u]][0])
        {
            p.vertices.push_back(u);
            if (u == s)
            {
                break;
            }
            p.edges.push_back(pre[u]);
        }
        reverse(p.vertices.begin(), p.vertices.end());
        reverse(p.edges.begin(), p.edges.end());
        return p;
    }

    vector<Path> solve(int s, int t, int k) const
    {
        // 返回 s 到 t 代价最小的前 k 条互不相同的简单路径，按代价升序；不足 k 条时全部返回。
        assert(0 <= s && s < n && 0 <= t && t < n); // 调试检查，可删。
        vector<Path> ans;
```

```
        vector<char> banV(n), banE(edges.size());
        auto first = shortest(s, t, banV, banE);
        if (k <= 0 || !first)
        {
            return ans;
        }
        ans.push_back(*first);
        set<Path> cand;                // cand 是尚未输出的候选路径。
        set<vector<int>> seen{first->edges}; // seen 是已生成过的边序列，避免重复入候选。
        while ((int)ans.size() < k)
        {
            const Path &last = ans.back();
            __int128 root = 0; // root 是 last 前 i 条边的代价，即根路径代价。
            fill(banV.begin(), banV.end(), 0);
            for (int i = 0; i < (int)last.edges.size(); i++)
            {
                // 偏离点是 last.vertices[i]：禁掉所有共享这段根路径的已选路径的下一条边，
                // 根路径上更早的点已在 banV 中，保证拼接后仍是简单路径。
                fill(banE.begin(), banE.end(), 0);
                for (const Path &p : ans)
                {
                    if ((int)p.edges.size() > i && equal(last.edges.begin(), last.edges.begin()
                     ↪  + i, p.edges.begin()))
                    {
                        banE[p.edges[i]] = 1;
                    }
                }
                auto spur = shortest(last.vertices[i], t, banV, banE);
                if (spur)
                {
                    Path p;
                    p.cost = root + spur->cost;
                    p.edges.assign(last.edges.begin(), last.edges.begin() + i);
                    p.edges.insert(p.edges.end(), spur->edges.begin(), spur->edges.end());
                    p.vertices.assign(last.vertices.begin(), last.vertices.begin() + i);
                    p.vertices.insert(p.vertices.end(), spur->vertices.begin(),
↪  spur->vertices.end());
                    if (seen.insert(p.edges).second)
                    {
                        cand.insert(move(p));
                    }
                }
                banV[last.vertices[i]] = 1;
                root += edges[last.edges[i]][2];
            }
            if (cand.empty())
            {
                break;
            }
            ans.push_back(*cand.begin());
            cand.erase(cand.begin());
        }
```

```
        return ans;
    }
};
```

`YenPaths g(n)` 后用 `g.add(u,v,w)` 加入有向边并得到边编号，`g.solve(s,t,k)` 按代价升序返回至多 k 条路径，每条含 `cost`、`vertices` 与 `edges`；不足 k 条时全部返回，$s=t$ 时只有一条零长路径。无向图把每条边按两个方向各加一次即可。设路径边数不超过 n，每条答案做至多 n 次 Dijkstra，时间 $O(kn\,m\log m)$；候选集合最多 $O(kn)$ 条路径，空间 $O(kn^2+m)$。

允许重复经过点的第 k 短路（游走）见 5.2.010 第 K 短路（Dijkstra / A*）与 5.2.012 第 K 短游走（Eppstein），它们比 Yen 快但不能保证简单。下面是 Yen 论文常用的 6 点示例：

```
signed main()
{
    // C=0,D=1,E=2,F=3,G=4,H=5，求 C 到 H 的前 3 条简单路。
    YenPaths g(6);
    for (auto [u, v, w] : vector<array<int, 3>>{{0, 1, 3}, {0, 2, 2}, {1, 3, 4}, {2, 1, 1}, {2, 3, 2},
                                                {2, 4, 3}, {3, 4, 2}, {3, 5, 1}, {4, 5, 2}})
    {
        g.add(u, v, w);
    }
    auto p = g.solve(0, 5, 3);
    assert(p.size() == 3);
    assert(p[0].cost == 5 && p[0].vertices == vector<int>({0, 2, 3, 5}));
    assert(p[1].cost == 7 && p[1].vertices == vector<int>({0, 2, 4, 5}));
    assert(p[2].cost == 8 && p[2].vertices == vector<int>({0, 1, 3, 5}));
}
```

5.3 连通性

5.3.001 Tarjan 强连通分量

Tarjan 一次 DFS 就求出有向图的强连通分量。给每点记进入时间戳 dfn_u 与「不经过已封闭分量、能回溯到的最早时间戳」low_u，并把访问中的点压入栈。回溯时若 $low_u=dfn_u$，说明 u 是当前栈中一个封闭强连通分量的根——从栈顶连续弹出直到弹出 u，这批点两两可达且不可能再与栈外点互达，即一个 SCC。分量编号按弹栈顺序递增，天然是缩点 DAG 的逆拓扑序，缩点后可直接做 DAG DP。

```
vector<int> tarjanSCC(const vector<vector<int>> &g)
{
    // g 是有向图邻接表；返回每点的强连通分量编号，编号按分量弹栈顺序递增。
    int n = g.size(), ti = 0, cc = 0;
    vector<int> dfn(n), low(n), st, bel(n, -1);
    vector<bool> on(n);
    auto dfs = [&](auto &&self, int u) -> void
    {
        // u 是当前点；完成以 u 为根的 Tarjan 搜索并弹出已封闭分量，无返回值。
        dfn[u] = low[u] = ++ti;
```

```
        st.push_back(u);
        on[u] = true;
        for (auto v : g[u])
        {
            if (!dfn[v])
            {
                self(self, v);
                chmin(low[u], low[v]);
            }
            else if (on[v])
            {
                chmin(low[u], dfn[v]);
            }
        }
        if (low[u] == dfn[u])
        {
            return;
        }
        while (true)
        {
            int v = st.back();
            st.pop_back();
            on[v] = false;
            bel[v] = cc;
            if (v == u)
            {
                break;
            }
        }
        cc++;
    };
    for (int u = 0; u < n; u++)
    {
        if (!dfn[u])
        {
            dfs(dfs, u);
        }
    }
    return bel;
}
```

`tarjanSCC(g)` 返回每点的分量编号，允许图不连通、自环和重边；编号只保证「同分量相等」，缩点建 DAG 时须按分量编号重新连边去重。$O(V+E)$ 时间与空间。有向边 `0→1,1→0,1→2,2→3,3→2` 得两个 SCC `{0,1}` 与 `{2,3}`；5.3.002 Kosaraju 强连通分量、5.3.003 Bitset 优化 Kosaraju 解决同一问题，各版都应单独用这个已知分组检查。需要按分量拓扑序正序处理时，Kosaraju 直接给出正序；无向图找桥/割点是另一套 low 值判据，见 5.3.004 割边与边双连通分量、5.3.005 割点与点双连通分量。

5.3.002 Kosaraju 强连通分量

Kosaraju 用两遍 DFS 求 SCC，是 Tarjan 的替代。第一遍在原图 DFS，按结束时间把点压入 `ord`；缩点 DAG 的源分量必然最后结束。第二遍按结束时间逆序（`ord` 反转）在反图 DFS，每从一个未定点出发所能到达的点集恰是一个 SCC——因为反图上从源分量出发无法越过 DAG 的边界。它比 Tarjan 直观、但需要显式建反图，分量编号按缩点 DAG 的拓扑正序递增，正好与 Tarjan 相反。

```
vector<int> kosarajuSCC(const vector<vector<int>> &g)
{
    // g 是有向图邻接表；返回每点的强连通分量编号，编号按缩点 DAG 拓扑序递增。
    int n = g.size(), cc = 0;
    vector<vector<int>> rg(n);
    for (int u = 0; u < n; u++)
    {
        for (auto v : g[u])
        {
            rg[v].push_back(u);
        }
    }
    vector<int> ord, bel(n, -1);
    vector<bool> vis(n);
    auto dfs1 = [&](auto &&self, int u) -> void
    {
        // u 是原图当前点；按 DFS 结束时间把点加入 ord，无返回值。
        vis[u] = true;
        for (auto v : g[u])
        {
            if (!vis[v])
            {
                self(self, v);
            }
        }
        ord.push_back(u);
    };
    auto dfs2 = [&](auto &&self, int u) -> void
    {
        // u 是反图当前点；把本次可达点标为同一强连通分量，无返回值。
        bel[u] = cc;
        for (auto v : rg[u])
        {
            if (bel[v] == -1)
            {
                self(self, v);
            }
        }
    };
    for (int u = 0; u < n; u++)
    {
        if (!vis[u])
        {
            dfs1(dfs1, u);
```

```
        }
    }
    reverse(ord.begin(), ord.end());
    for (auto u : ord)
    {
        if (bel[u] == -1)
        {
            dfs2(dfs2, u);
            cc++;
        }
    }
    return bel;
}
```

`kosarajuSCC(g)` 返回每点分量编号，编号按缩点 DAG 拓扑正序递增；$O(V+E)$ 时间与空间，需额外存反图，递归深度可达点数（深链要防爆栈）。只关心分组且想省掉反图时用 5.3.001 Tarjan 强连通分量；需要拓扑正序编号（如按序 DP）则 Kosaraju 更省事。点数很小的稠密图可用位运算并行的 5.3.003 Bitset 优化 Kosaraju。

5.3.003 Bitset 优化 Kosaraju

当点数小到能压位时，可以用位集传递闭包求 SCC。`reach[u]` 是 u 能到达的点集，用位集版 Floyd（枚举中间点 k，若 u 能到 k 就把 $reach[k]$ 或进 $reach[u]$）求出全部可达关系；随后 u, v 同属一个 SCC 当且仅当互相可达，互达是等价关系可直接分组。它把 SCC 转成 $O(V^3/64)$ 的闭包运算，只在稠密小图（V 几百内）划算，不是一般大图的默认做法——那时线性的 Tarjan/Kosaraju 远快。

```
vector<int> bitsetSCC(const vector<vector<int>> &g)
{
    // g 是点数较小的稠密有向图；返回每点强连通分量编号。
    int n = g.size(), w = (n + 63) / 64;
    vector reach(n, vector<unsigned long long>(w));
    for (int u = 0; u < n; u++)
    {
        reach[u][u / 64] |= 1ULL << (u % 64);
        for (auto v : g[u])
        {
            reach[u][v / 64] |= 1ULL << (v % 64);
        }
    }
    for (int k = 0; k < n; k++)
    {
        for (int u = 0; u < n; u++)
        {
            if (!(reach[u][k / 64] >> (k % 64) & 1))
            {
                continue;
            }
            for (int b = 0; b < w; b++)
            {
```

```
                reach[u][b] |= reach[k][b];
            }
        }
    }
    vector<int> bel(n, -1);
    int cc = 0;
    for (int u = 0; u < n; u++)
    {
        if (bel[u] != -1)
        {
            continue;
        }
        for (int v = u; v < n; v++)
        {
            bool uv = reach[u][v / 64] >> (v % 64) & 1;
            bool vu = reach[v][u / 64] >> (u % 64) & 1;
            if (uv && vu)
            {
                bel[v] = cc;
            }
        }
        cc++;
    }
    return bel;
}
```

`bitsetSCC(g)` 返回每点分量编号（编号按首个未定点顺序给出，不承诺拓扑序），仅在 $O(V^2/64)$ 位内存与 $O(V^3/64)$ 时间可接受时使用。点数一大就该回到线性的 5.3.001 Tarjan 强连通分量 或 5.3.002 Kosaraju 强连通分量；此处顺带求出的传递闭包本身可复用（如判两点可达性），这是位集法相对前两者的额外收益。

5.3.004 割边与边双连通分量

桥是删掉后连通块数增加的边，边双连通分量是删去所有桥后剩下的极大连通块。判据用 DFS 树的 low 值：树边 $u \to v$ 满足 $low_v > dfn_u$ 时是桥，因为 v 的子树没有任何返祖边能绕过这条树边回到 u 或更早。重边必须靠边编号区分（`id==pe` 跳的是父边编号而非父点，才不会把两条平行边误判成桥）。求出桥后再做一次「不走桥」的遍历，每个连通块即一个边双。

```
class EdgeBCC
{
    // n 是原图点数，ti 是 DFS 时间戳。
    int n, ti = 0;
    // ed[i][0..1] 依次是第 i 条无向边的两个端点。
    vector<array<int, 2>> ed;
    // g[u] 的每项依次是邻点与无向边编号。
    vector<vector<array<int, 2>>> g;
    // dfn[u] 是进入时间，low[u] 是不经过父边能到达的最早时间。
    vector<int> dfn, low;
    // bridge[i] 标记第 i 条边是否为桥。
```

```
    vector<bool> bridge;

    void dfs(int u, int pe)
    {
        // u 是当前点，pe 是进入 u 的边编号；计算 low 并标记桥，无返回值。
        dfn[u] = low[u] = ++ti;
        for (auto [v, id] : g[u])
        {
            if (id == pe)
            {
                continue;
            }
            if (!dfn[v])
            {
                dfs(v, id);
                chmin(low[u], low[v]);
                if (low[v] > dfn[u])
                {
                    bridge[id] = true;
                }
            }
            else
            {
                chmin(low[u], dfn[v]);
            }
        }
    }

  public:
    EdgeBCC(int n) : n(n), g(n), dfn(n), low(n)
    {
        // n 是点数；构造一张无边图，无返回值。
    }

    int addEdge(int u, int v)
    {
        // u、v 是 0 下标端点；加入无向边并返回边编号，允许重边。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        int id = ed.size();
        ed.push_back({u, v});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        return id;
    }

    void build()
    {
        // 无参数；重新计算全部桥，无返回值。
        ti = 0;
        fill(dfn.begin(), dfn.end(), 0);
        fill(low.begin(), low.end(), 0);
        bridge.assign(ed.size(), false);
```

```
        for (int u = 0; u < n; u++)
        {
            if (!dfn[u])
            {
                dfs(u, -1);
            }
        }
    }

    const vector<bool> &bridges() const
    {
        // 无参数；返回按边编号排列的桥标记。
        return bridge;
    }

    vector<int> comps() const
    {
        // 无参数；返回删除全部桥后每个点的边双连通分量编号。
        vector<int> bel(n, -1);
        int cc = 0;
        for (int s = 0; s < n; s++)
        {
            if (bel[s] != -1)
            {
                continue;
            }
            queue<int> q;
            q.push(s);
            bel[s] = cc++;
            while (!q.empty())
            {
                int u = q.front();
                q.pop();
                for (auto [v, id] : g[u])
                {
                    if (!bridge[id] && bel[v] == -1)
                    {
                        bel[v] = bel[u];
                        q.push(v);
                    }
                }
            }
        }
        return bel;
    }
};
```

构造 `EdgeBCC q(n)`，逐边 `addEdge(u,v)` 后 `build()` 重算，之后 `bridges()[id]` 判第 `id` 条边是否为桥、`comps()` 返回各点的边双编号；加边后必须重新 `build()`。$O(V+E)$ 时间与空间。点数 5、边 `0-1,1-2,2-0,1-3,3-4`：桥是后两条，边双里 $0,1,2$ 同号而 $3,4$ 各自成块。求的是割点/点双而非割边/边双时用 5.3.005 割点与点双连通分量（判据是 $low_v \geq dfn_u$，多一个等号）；把边双缩点连桥可得块割树，见

5.3.006 缩点与块割树。

5.3.005 割点与点双连通分量

割点是删去后连通块数增加的点，点双连通分量是极大的「任意两点间有两条内部不相交路径」的块；一个割点可同属多个点双。判据是树边 $u \to v$ 满足 $low_v \geq dfn_u$（比桥的严格大于多一个等号）：此时把边栈弹到这条树边为止，弹出的边所涉点集就是一个点双，且 u 是割点。根节点特殊——它只有在 DFS 儿子数 > 1 时才是割点。孤立点单独作为一个点双输出。栈存的是边编号而非点，才能正确处理割点被多个点双共享的情形。

```
class VertexBCC
{
    // n 是原图点数，ti 是 DFS 时间戳。
    int n, ti = 0;
    // ed[i][0..1] 依次是第 i 条无向边的两个端点。
    vector<array<int, 2>> ed;
    // g[u] 的每项依次是邻点与边编号。
    vector<vector<array<int, 2>>> g;
    // dfn、low 分别保存进入时间和 Lowlink 值，cut 标记割点。
    vector<int> dfn, low;
    vector<bool> cut;
    // st 保存尚未归入点双的边编号，bcc 保存各点双的点集。
    vector<int> st;
    vector<vector<int>> bcc;

    void dfs(int u, int pe)
    {
        // u 是当前点，pe 是父边编号；划分点双并标记割点，无返回值。
        dfn[u] = low[u] = ++ti;
        int son = 0;
        for (auto [v, id] : g[u])
        {
            if (id == pe)
            {
                continue;
            }
            if (!dfn[v])
            {
                son++;
                st.push_back(id);
                dfs(v, id);
                chmin(low[u], low[v]);
                if (low[v] >= dfn[u])
                {
                    if (pe != -1 || son > 1)
                    {
                        cut[u] = true;
                    }
                    vector<int> cur;
                    while (true)
```

```
                    {
                        int x = st.back();
                        st.pop_back();
                        cur.push_back(ed[x][0]);
                        cur.push_back(ed[x][1]);
                        if (x == id)
                        {
                            break;
                        }
                    }
                    sort(cur.begin(), cur.end());
                    cur.erase(unique(cur.begin(), cur.end()), cur.end());
                    bcc.push_back(move(cur));
                }
            }
            else if (dfn[v] < dfn[u])
            {
                st.push_back(id);
                chmin(low[u], dfn[v]);
            }
        }
        if (pe == -1 && son == 0)
        {
            bcc.push_back({u});
        }
    }

public:
    VertexBCC(int n) : n(n), g(n), dfn(n), low(n), cut(n)
    {
        // n 是点数；构造一张无边图，无返回值。
    }

    int addEdge(int u, int v)
    {
        // u、v 是 0 下标端点；加入无向边并返回边编号，允许重边。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        int id = ed.size();
        ed.push_back({u, v});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        return id;
    }

    void build()
    {
        // 无参数；清空旧结果并重新求割点和点双，无返回值。
        ti = 0;
        fill(dfn.begin(), dfn.end(), 0);
        fill(low.begin(), low.end(), 0);
        fill(cut.begin(), cut.end(), false);
        st.clear();
```

```
        bcc.clear();
        for (int u = 0; u < n; u++)
        {
            if (!dfn[u])
            {
                dfs(u, -1);
            }
        }
    }

    const vector<bool> &cutVertices() const
    {
        // 无参数；返回按点编号排列的割点标记。
        return cut;
    }

    const vector<vector<int>> &comps() const
    {
        // 无参数；返回全部点双连通分量，每项是去重后的原图点编号。
        return bcc;
    }
};
```

构造 `VertexBCC q(n)`，加完边后 `build()`，从 `cutVertices()[u]` 读割点标记、`comps()` 读各点双点集（每项去重）；自环需按题意预处理。$O(V+E)$ 时间与空间，每条边只入栈出栈一次。对边 `0-1,1-2,2-0,1-3,3-4` 的图，割点为 $1,3$，点双点集为 `{0,1,2}`、`{1,3}`、`{3,4}`。把点双与割点建成树即 5.3.006 缩点与块割树 或 5.3.007 圆方树、广义圆方树；求割边/边双而非割点/点双时用 5.3.004 割边与边双连通分量。

5.3.006 缩点与块割树

块割树把每个点双缩成一个块节点，与它所含的原图点相连，从而把「原图经割点串联的块结构」变成一棵树。因为不同点双至多共享一个割点，加入块节点后整体无环——原图每个连通块对应一棵块割树，圆点（原图点）与方点（块节点）交替，故也称二分结构。构造只是把每个点双 i 建成新点 $n+i$，向其点集里每个原图点连无向边。

```
vector<vector<int>> blockCutTree(int n, const vector<vector<int>> &bcc)
{
    // n 是原图点数，bcc[i] 是第 i 个点双的点集；返回 n+|bcc| 个点的块割树邻接表。
    vector<vector<int>> tr(n + bcc.size());
    for (int i = 0; i < (int)bcc.size(); i++)
    {
        int b = n + i;
        for (auto u : bcc[i])
        {
            tr[u].push_back(b);
            tr[b].push_back(u);
        }
    }
    return tr;
```

```
}
```

先由 5.3.005 割点与点双连通分量 得到 bcc，再 `blockCutTree(n, bcc)` 返回 $n + |bcc|$ 个点的邻接表：编号 $< n$ 是原图点，其余依次对应点双。$O(V + S)$ 时间与空间，S 为全部点双大小之和，无向图中 $S = O(V + E)$。同一份拓扑若保留「圆点/方点」语义用于树上路径限制，就是 5.3.007 圆方树、广义圆方树——两者代码一致，区别在方点是否被赋予点双相关的权信息。

5.3.007 圆方树、广义圆方树

圆方树把原图每个点作为圆点、每个点双作为方点，圆点与包含它的方点相连。它与 5.3.006 缩点与块割树 的拓扑完全相同，价值在语义：路径一旦进入某方点就代表在对应点双内自由移动，于是「经过割点的连通/路径限制」被转化为树上路径问题，可套 LCA、树上差分等工具。广义圆方树在此基础上按题意给方点赋权（如点双大小、内部最短环长），因此方点权不能简单复制某个圆点的权。

实现直接复用 5.3.006 缩点与块割树 的 `blockCutTree(n,bcc)`：返回图中编号 $[0, n)$ 的点作为圆点，第 i 个点双对应编号 $n + i$ 的方点。圆方树与块割树在这里是同一份拓扑，差别只在使用语义；广义圆方树还要按题意给每个方点汇总点双权值，不能复制某个圆点权。构造用 $O(V + S)$ 时间与空间，S 为全部点双大小之和；仙人掌的环结构见 5.3.010 仙人掌森林分解。

5.3.008 无向图连通度、点连通度、边连通度

全局连通度问至少删多少条边或多少个点才能使无向图不连通。边连通度 λ 就是全局最小割，调用 Stoer-Wagner。点连通度 κ 用拆点最大流：每点拆成入点、出点并连容量 1，指定源汇的内部边和原图边容量取 n，再枚举所有非相邻点对；完全图单独返回 $n - 1$。全局值不能用一次任意源汇最小割代替。本条只保留建模入口，底层统一引用 5.8 的主实现。

```
// 依赖 5.8.007 Stoer-Wagner 全局最小割
template <class T>
struct Flow;

struct GlobalCut;

GlobalCut stoerWagner(vector<vector<int>> a);

int edgeConn(int n, const vector<tuple<int, int, long long>> &e)
{
    // e 是非负权无向边；返回全局边连通度。
    vector a(n, vector<int>(n));
    for (auto [u, v, w] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && w >= 0); // 调试检查，可删。
        if (u != v)
        {
            a[u][v] += w;
            a[v][u] = a[u][v];
        }
    }
```

```
    return stoerWagner(move(a)).w;
}

template <class F = Flow<long long>>
int vertexConn(const vector<vector<int>> &g)
{
    // g 是简单无向图邻接表；返回全局点连通度。
    int n = g.size();
    if (n <= 1)
    {
        return 0;
    }
    vector a(n, vector<bool>(n));
    int m = 0;
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u])
        {
            assert(0 <= v && v < n); // 调试检查，可删。
            if (u == v)
            {
                continue;
            }
            if (!a[u][v])
            {
                a[u][v] = true;
                a[v][u] = true;
                m++;
            }
        }
    }
    if (m == n * (n - 1) / 2)
    {
        return n - 1;
    }
    int ans = n - 1;
    for (int s = 0; s < n; s++)
    {
        for (int t = s + 1; t < n; t++)
        {
            if (a[s][t])
            {
                continue;
            }
            F f(2 * n);
            for (int u = 0; u < n; u++)
            {
                f.add(2 * u, 2 * u + 1, u == s || u == t ? n : 1);
            }
            for (int u = 0; u < n; u++)
            {
                for (int v = u + 1; v < n; v++)
```

```
                {
                    if (a[u][v])
                    {
                        f.add(2 * u + 1, 2 * v, n);
                        f.add(2 * v + 1, 2 * u, n);
                    }
                }
            }
            ans = min(ans, f.flow(2 * s + 1, 2 * t));
        }
    }
    return ans;
}
```

本条提前声明接口，完整实现仍统一放在 5.8.001 Dinic 最大流 的 `Flow` 与 5.8.007 Stoer-Wagner 全局最小割；调用点须写在这两条定义之后。`edgeConn(n,e)` 接受非负 64 位边权，自环不影响割；调用者必须保证平行边与收缩过程中的容量和可由 64 位有符号整数表示。`vertexConn(g)` 处理简单无向图，$n \le 1$ 返回 0，完全图返回 $n-1$。边连通度为 $O(n^3)$；点连通度按通用界为 $O(n^4(n+m))$，仅适合小图。

5.3.009 最佳/最小点割集与边割集

要真正取出一组割而不只是割的大小，靠的是最大流最小割的对偶：把「删点」和「删边」都翻译成容量约束，最小割中被用满的那批容量就是要删的对象。求边割时直接给每条边赋容量、原点保持不动，跑源汇最大流后残量图里从源可达一侧与不可达一侧之间的满流边即为边割集。求点割时先把每个可删点 u 拆成 $u_{in} \to u_{out}$ 一条容量为其代价的内部边、原图的边设成无穷容量，这样最小割只能切在拆点边上，被切的拆点边对应的原点就是点割集；指定点对要断开时把该点对的拆点边设成无穷以禁止割掉端点自身。

这套建模直接复用 5.8.001 Dinic 最大流 的 `Flow` 与 5.8.007 Stoer-Wagner 全局最小割，不再重复代码。求指定点对边割时跑 `Flow::flow`，再用 `cut(源)` 分出残量源侧，跨越两侧的正容量原边就是一组最小边割；不指定点对时，`GlobalCut.side` 已给出全局最小割的一侧。求点割时在拆点图上取被割断的内部边，枚举非相邻点对取最小者。

复杂度与 5.3.008 相同：边割由 Stoer-Wagner 求得，$O(n^3)$ 时间、$O(n^2)$ 空间；点割枚举 $O(n^2)$ 个点对，拆点图有 $2n$ 个点和 $O(n+m)$ 条边，代入本章 Dinic 的通用界后为 $O(n^4(n+m))$ 时间、$O(n+m)$ 空间；输出割集只需在此基础上再做一次 $O(n+m)$ 的残量图遍历。不同评测机上的可用规模会受常数影响，但不改变这里的渐进上界。若只要连通度数值而不要具体割集，直接调用 5.3.008 即可；需要把断开代价推广到「删点或删边任选」的混合模型时，仍是本条的拆点加边建图，只是给拆点边与原边各自赋上对应代价。

5.3.010 仙人掌森林分解

仙人掌是每条边至多属于一个简单环的简单无向图；本实现也接受多个连通块，得到仙人掌森林。DFS 维护 `dfn`、`low` 与边栈，当 `low[v]>=dfn[u]` 时弹出一个点双分量。仙人掌中每个含多条边的点双必须恰为一个简单环，所以其边数等于不同端点数。端点去重使用时间戳数组，整段保持线性复杂度。标题只写实际提供的仙人掌分解，不再保留没有对应代码的“AFL”。

```cpp
struct Cactus
{
    // n 是图的点数，ti 是 DFS 时间戳。
    int n, ti = 0;
    // e[id] 是第 id 条无向边的两个端点。
    vector<pair<int, int>> e;
    // g[u] 的每项是邻点及无向边编号。
    vector<vector<pair<int, int>>> g;
    // Tarjan 时间戳、边栈及端点去重时间戳。
    vector<int> dfn, low, st, mark;
    int tag = 0;
    // cycles 的每项是一条简单环的边编号集合。
    vector<vector<int>> cycles;
    // ok 表示每个非桥点双是否都是单环。
    bool ok = true;

    Cactus(int n) : n(n), g(n), dfn(n), low(n), mark(n)
    {
        // n 是图的点数；构造一张没有边的图。
    }

    void add(int u, int v)
    {
        // u、v 是简单无向边端点；加入一条边。
        assert(0 <= u && u < n && 0 <= v && v < n && u != v); // 调试检查，可删。
        int id = e.size();
        e.push_back({u, v});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
    }

    void dfs(int u, int pe)
    {
        // u 是当前点，pe 是父边编号；划分以 u 为根的点双。
        dfn[u] = low[u] = ++ti;
        for (auto [v, id] : g[u])
        {
            if (id == pe)
            {
                continue;
            }
            if (!dfn[v])
            {
                st.push_back(id);
                dfs(v, id);
                low[u] = min(low[u], low[v]);
                if (low[v] >= dfn[u])
                {
                    vector<int> q;
                    while (true)
                    {
                        int x = st.back();
```

```
                        st.pop_back();
                        q.push_back(x);
                        if (x == id)
                        {
                            break;
                        }
                    }
                    if (q.size() > 1)
                    {
                        tag++;
                        int cnt = 0;
                        for (int x : q)
                        {
                            for (int v : {e[x].first, e[x].second})
                            {
                                if (mark[v] != tag)
                                {
                                    mark[v] = tag;
                                    cnt++;
                                }
                            }
                        }
                        if (cnt != (int)q.size())
                        {
                            ok = false;
                        }
                        else
                        {
                            cycles.push_back(q);
                        }
                    }
                }
            }
            else if (dfn[v] < dfn[u])
            {
                st.push_back(id);
                low[u] = min(low[u], dfn[v]);
            }
        }
    }

    bool build()
    {
        // 无参数；重新分解全部点双，返回图是否为仙人掌。
        ti = 0;
        fill(dfn.begin(), dfn.end(), 0);
        fill(low.begin(), low.end(), 0);
        fill(mark.begin(), mark.end(), 0);
        st.clear();
        cycles.clear();
        ok = true;
        tag = 0;
```

```
        for (int i = 0; i < n; i++)
        {
            if (!dfn[i])
            {
                dfs(i, -1);
            }
        }
        return ok;
    }
};
```

用 `add` 建简单无向图，`build()` 可重复调用；每次都会清空全部运行态。返回 `true` 时，`cycles` 的每项是一条环的原边编号，桥边不出现；返回 `false` 表示某个点双不是单环。允许图不连通，但不允许自环或重边。时间与空间均为 $O(n+m)$。一般点双见 5.3.005 割点与点双连通分量，只有一个环的连通图可用 5.9.008 基环树。

5.3.011 补图连通块

当原图稀疏而补图稠密（补图有近 n^2 条边）时，显式建补图再跑连通块不可行。诀窍是维护一个「尚未归类点」的有序集合 `left`，每次从中取一点做 BFS：先用一个时间戳 `tim` 把该点在原图中的邻居打上标记，然后遍历 `left`，凡是没被打上标记的点就是补图邻居，从 `left` 里删掉并入队。由于每个点一旦入队就永久离开 `left`，`left` 的总删除次数是 $O(n)$，而每次 BFS 扩展时对某点的原图邻居打标记的总量是 $O(m)$，于是不必真正枚举补图的 $O(n^2)$ 条边就能把补图连通块全部找出。

```
vector<vector<int>> complementCC(const vector<vector<int>> &g)
{
    // g 是原图邻接表；返回补图的全部连通块，每个点恰出现一次。
    int n = g.size();
    set<int> left;
    for (int u = 0; u < n; u++)
    {
        left.insert(u);
    }
    vector<int> mark(n);
    int tim = 0;
    vector<vector<int>> ans;
    while (!left.empty())
    {
        int s = *left.begin();
        left.erase(s);
        ans.push_back({});
        queue<int> q;
        q.push(s);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            ans.back().push_back(u);
            tim++;
```

```
            for (int v : g[u])
            {
                mark[v] = tim;
            }
            for (auto it = left.begin(); it != left.end();)
            {
                int v = *it;
                if (mark[v] != tim)
                {
                    it = left.erase(it);
                    q.push(v);
                }
                else
                {
                    it++;
                }
            }
        }
    }
    return ans;
}
```

`complementCC(g)` 传入原图邻接表，返回补图的全部连通块，每个点恰出现一次；`g` 里的重复边或自环不影响结果，因为判定只看「是否被标记」。集合遍历配合 `set` 的删除，最坏 $O((n+m)\log n)$ 时间、$O(n+m)$ 空间；把 `set` 换成带链表的并查集式结构可去掉对数因子。这里求的是无向图补图连通性，与原图上的连通分量（直接 BFS / 5.3.004 割边与边双连通分量 的连通框架）互补；若要的是有向图强连通而非补图连通，应改用 5.3.001 Tarjan 强连通分量。

5.3.012 离线增量 SCC

已知有向边按编号 $0, 1, \dots, m-1$ 依次加入，要求每条边最早在哪个前缀中处于有向环。对时间区间分治：在中点加入当前候选中编号小于中点的边并求 SCC；端点已经同分量的边进入左半继续找更早时间，其余边把端点缩成当前分量后进入右半。每层所有候选边只出现一次，缩点保证右半保留后续成环所需的全部可达关系。

```
// 依赖 5.3.001 Tarjan 强连通分量
vector<int> incrSCC(int n, const vector<pair<int, int>> &e)
{
    // e[i] 是第 i 次加入的有向边；返回它最早成环时的前缀长度，m+1 表示始终不成环。
    int m = e.size();
    vector<int> ans(m, m + 1), id(n, -1);
    vector<tuple<int, int, int>> all;
    for (int i = 0; i < m; i++)
    {
        auto [u, v] = e[i];
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        all.push_back({i, u, v});
    }
    auto solve = [&](auto &&self, const vector<tuple<int, int, int>> &a, int l, int r) -> void
```

```
    {
        if (a.empty() || r == l + 1)
        {
            return;
        }
        int mid = (l + r) / 2;
        int cnt = 0;
        for (auto [i, u, v] : a)
        {
            if (id[u] == -1)
            {
                id[u] = cnt++;
            }
            if (id[v] == -1)
            {
                id[v] = cnt++;
            }
        }
        vector<vector<int>> g(cnt);
        for (auto [i, u, v] : a)
        {
            if (i < mid)
            {
                g[id[u]].push_back(id[v]);
            }
        }
        vector<int> bel = tarjanSCC(g);
        vector<tuple<int, int, int>> left, right;
        for (auto [i, u, v] : a)
        {
            int x = id[u], y = id[v];
            if (i < mid && bel[x] == bel[y])
            {
                chmin(ans[i], mid);
                left.push_back({i, x, y});
            }
            else
            {
                right.push_back({i, bel[x], bel[y]});
            }
        }
        for (auto [i, u, v] : a)
        {
            id[u] = id[v] = -1;
        }
        self(self, left, l, mid);
        self(self, right, mid, r);
    };
    solve(solve, all, 0, m + 1);
    return ans;
}
```

`incrSCC(n,e)` 的答案 `t` 表示加入 `e[0..t)` 后该边的两端首次同属一个 SCC；自环答案是 `i+1`，始终不成环时为 `m+1`。这是离线算法，不支持插边后立即索取当前全部分量编号；那种小规模需求可每次重跑 5.3.001 Tarjan 强连通分量。时间 $O((n+m)\log(m+1))$、空间 $O(n+m)$，递归层数 $O(\log m)$。

下面用五条边检查三点环、之后并入第四点的环，以及始终不能成环的边：

```
signed main()
{
    vector<pair<int, int>> e{{0, 1}, {1, 2}, {2, 0}, {2, 3}, {3, 2}, {0, 0}};
    assert(incrSCC(4, e) == vector<int>({3, 3, 3, 5, 5, 6}));
}
```

5.3.013 三边连通分量

三边连通要求任意两点间至少有三条边不相交路径，强于 5.3.004 割边与边双连通分量 的两边连通。DFS 维护返祖边数 `deg`、最低返祖时间 `low` 与尚未并入的祖先链 `path`；一条子树只有至多一条返祖边时可把它的约束上提，否则按最低返祖边把整段祖先链与当前点合并。并查集的代表即最终三边连通分量。

```
vector<vector<int>> threeEdgeCC(const vector<vector<int>> &g)
{
    // g 是允许重边和自环的无向邻接表；返回任意顺序的三边连通点集划分。
    int n = g.size(), timer = 0;
    vector<int> in(n), out(n), low(n, n), deg(n), path(n, n), nxt(n);
    vector<char> vis(n);
    iota(nxt.begin(), nxt.end(), 0);
    auto find = [&](auto &&self, int x) -> int
    {
        return nxt[x] == x ? x : nxt[x] = self(self, nxt[x]);
    };
    auto join = [&](int x, int y)
    {
        x = find(find, x);
        y = find(find, y);
        if (x != y)
        {
            nxt[y] = x;
            deg[x] += deg[y];
        }
    };
    auto dfs = [&](auto &&self, int v, int p) -> void
    {
        vis[v] = true;
        in[v] = timer++;
        for (int w : g[v])
        {
            if (w == v)
            {
                continue;
            }
            if (w == p)
```

```
            {
                p = n;
                continue;
            }
            if (vis[w])
            {
                if (in[w] < in[v])
                {
                    deg[v]++;
                    low[v] = min(low[v], in[w]);
                }
                else
                {
                    deg[v]--;
                    int u = path[v];
                    while (u != n && in[u] <= in[w] && in[w] < out[u])
                    {
                        join(v, u);
                        u = path[u];
                    }
                    path[v] = u;
                }
                continue;
            }
            self(self, w, v);
            if (path[w] == n && deg[w] <= 1)
            {
                deg[v] += deg[w];
                low[v] = min(low[v], low[w]);
            }
            else
            {
                if (deg[w] == 0)
                {
                    w = path[w];
                }
                if (low[w] < low[v])
                {
                    low[v] = low[w];
                    swap(w, path[v]);
                }
                while (w != n)
                {
                    join(v, w);
                    w = path[w];
                }
            }
        }
        out[v] = timer;
    };
    for (int i = 0; i < n; i++)
    {
```

```
        if (!vis[i])
        {
            dfs(dfs, i, n);
        }
    }
    vector<vector<int>> ans(n);
    for (int i = 0; i < n; i++)
    {
        ans[find(find, i)].push_back(i);
    }
    ans.erase(remove_if(ans.begin(), ans.end(), [](const vector<int> &a)
                        { return a.empty(); }), ans.end());
    return ans;
}
```

`threeEdgeCC(g)` 的 `g` 是无向邻接表，每条无向边必须在两个端点各出现一次；允许重边和自环，返回分量顺序与分量内点顺序均未规定。时间 $O(n+m\alpha(n))$、额外空间 $O(n+m)$，适用于 $n, m \le 2\times 10^5$。只需要按桥切开的边双连通分量时用 5.3.004 割边与边双连通分量，不能把它替代本条。

5.4 2-SAT

5.4.001 Tarjan 2-SAT

2-SAT 要给每个布尔变量定真值以同时满足一批二元子句。把变量 x 拆成两个文字点 $2x$（假）与 $2x+1$（真），子句 $a \vee b$ 等价于 $\neg a \to b$ 及其逆否 $\neg b \to a$。可满足当且仅当没有变量的两个文字落在同一强连通分量里。Tarjan 按分量弹栈顺序编号：缩点 DAG 的汇分量先弹出、编号更小，因此每对互否文字中选编号更小者为真；代码里的比较式正按这个方向赋值。

```
class TarjanTwoSAT
{
    // n 是布尔变量数，g 是 2n 个文字构成的蕴含图。
    int n;
    vector<vector<int>> g;

  public:
    TarjanTwoSAT(int n) : n(n), g(2 * n)
    {
        // n 是变量数；构造没有子句的 2-SAT，无返回值。
    }

    void imply(int x, bool xv, int y, bool yv)
    {
        // x=xv 是前件，y=yv 是后件；加入蕴含及其逆否边。
        int a = 2 * x + xv, b = 2 * y + yv;
        g[a].push_back(b);
        g[b ^ 1].push_back(a ^ 1);
    }

    void addOr(int x, bool xv, int y, bool yv)
```

```
    {
        // x=xv、y=yv 是两个文字；加入二元子句 `(x=xv) or (y=yv)`。
        imply(x, !xv, y, yv);
    }

    optional<vector<bool>> solve() const
    {
        // 无参数；返回一组可行变量值，不可满足时返回空。
        int ti = 0, cc = 0;
        vector<int> dfn(2 * n), low(2 * n), st, bel(2 * n, -1);
        vector<bool> on(2 * n);
        auto dfs = [&](auto &&self, int u) -> void
        {
            // u 是当前文字；执行 Tarjan SCC 并给封闭分量编号，无返回值。
            dfn[u] = low[u] = ++ti;
            st.push_back(u);
            on[u] = true;
            for (auto v : g[u])
            {
                if (!dfn[v])
                {
                    self(self, v);
                    low[u] = min(low[u], low[v]);
                }
                else if (on[v])
                {
                    low[u] = min(low[u], dfn[v]);
                }
            }
            if (low[u] != dfn[u])
            {
                return;
            }
            while (true)
            {
                int v = st.back();
                st.pop_back();
                on[v] = false;
                bel[v] = cc;
                if (v == u)
                {
                    break;
                }
            }
            cc++;
        };
        for (int u = 0; u < 2 * n; u++)
        {
            if (!dfn[u])
            {
                dfs(dfs, u);
            }
```

```
        }
        vector<bool> ans(n);
        for (int x = 0; x < n; x++)
        {
            if (bel[2 * x] == bel[2 * x + 1])
            {
                return nullopt;
            }
            ans[x] = bel[2 * x] > bel[2 * x + 1];
        }
        return ans;
    }
};
```

构造 `TarjanTwoSAT q(n)`，用 `addOr(x,xv,y,yv)` 加二元子句，或用 `imply(x,xv,y,yv)` 加蕴含；后者会同时加入逆否边，不能把它当成只插一条普通有向边的接口。`solve()` 无解时返回空，否则返回变量取值。算法用 $O(V+E)$ 时间与空间，$V=2n$ 为文字数。Kosaraju 版本见 5.4.002 Kosaraju 2-SAT，字典序最小构造见 5.4.003 字典序或答案不唯一时的构造。

5.4.002 Kosaraju 2-SAT

同样的 2-SAT，只是把求 SCC 换成 Kosaraju：第一遍在原图上记录结束顺序，第二遍按逆结束序在反图上染色。本实现的分量编号按缩点 DAG 的拓扑正序递增，因此每对互否文字中选编号更大、拓扑更靠后的文字为真，正好与 Tarjan 版的编号比较方向相反。它同时保存正反两张图，逻辑直观，代价是边存储翻倍。

```
class Kosaraju2SAT
{
    // n 是变量数，g、rg 是文字蕴含图及其反图。
    int n;
    vector<vector<int>> g, rg;

    void edge(int u, int v)
    {
        // u、v 是文字节点；加入 u 到 v 的蕴含边及反边索引，无返回值。
        g[u].push_back(v);
        rg[v].push_back(u);
    }

  public:
    Kosaraju2SAT(int n) : n(n), g(2 * n), rg(2 * n)
    {
        // n 是变量数；构造没有子句的 2-SAT，无返回值。
    }

    void imply(int x, bool xv, int y, bool yv)
    {
        // x=xv 是前件，y=yv 是后件；加入蕴含及其逆否边。
        int a = 2 * x + xv, b = 2 * y + yv;
```

```
        edge(a, b);
        edge(b ^ 1, a ^ 1);
    }

    void addOr(int x, bool xv, int y, bool yv)
    {
        // x=xv、y=yv 是两个文字；加入二元子句。
        imply(x, !xv, y, yv);
    }

    optional<vector<bool>> solve() const
    {
        // 无参数；返回一组可行变量值，不可满足时返回空。
        vector<int> ord, bel(2 * n, -1);
        vector<bool> vis(2 * n);
        auto dfs1 = [&](auto &&self, int u) -> void
        {
            // u 是原图文字；记录 DFS 结束顺序，无返回值。
            vis[u] = true;
            for (auto v : g[u])
            {
                if (!vis[v])
                {
                    self(self, v);
                }
            }
            ord.push_back(u);
        };
        int cc = 0;
        auto dfs2 = [&](auto &&self, int u) -> void
        {
            // u 是反图文字；标记当前强连通分量，无返回值。
            bel[u] = cc;
            for (auto v : rg[u])
            {
                if (bel[v] == -1)
                {
                    self(self, v);
                }
            }
        };
        for (int u = 0; u < 2 * n; u++)
        {
            if (!vis[u])
            {
                dfs1(dfs1, u);
            }
        }
        reverse(ord.begin(), ord.end());
        for (auto u : ord)
        {
            if (bel[u] == -1)
```

```
            {
                dfs2(dfs2, u);
                cc++;
            }
        }
        vector<bool> ans(n);
        for (int x = 0; x < n; x++)
        {
            if (bel[2 * x] == bel[2 * x + 1])
            {
                return nullopt;
            }
            ans[x] = bel[2 * x] < bel[2 * x + 1];
        }
        return ans;
    }
};
```

接口与 5.4.001 Tarjan 2-SAT 完全一致，只把类型名换成 `Kosaraju2SAT`：`addOr` / `imply` 加约束、`solve()` 返回 `optional<vector<bool>>`。$O(V+E)$ 时间与空间，因存反图边数常数约为 Tarjan 版两倍。追求单遍 DFS、更省内存时用 5.4.001 Tarjan 2-SAT；需要字典序最小解或增量固定变量时见 5.4.003 字典序或答案不唯一时的构造。底层 SCC 原理见 5.3.002 Kosaraju 强连通分量。

5.4.003 字典序或答案不唯一时的构造

当 2-SAT 有多组解而要求字典序最小时，不能直接用一次 SCC 的赋值（它给出的是任意可行解）。做法是把「可满足性判定」和「构造」分开：先验证空约束下整体可满足，再按变量顺序逐个试着把 x_i 钉成 0——通过在蕴含图里加一条 $\neg(x_i{=}0) \to (x_i{=}0)$ 的强制边使该值成为必然，若加钉后整体仍可满足就固定为 0，否则回退为 1。因为每一步都保留了「已固定前缀 + 当前位取更小」仍可满足的不变量，最终得到的就是字典序最小解。判定函数每次都从头重建蕴含图并跑一遍 Kosaraju，故必须与构造循环彻底解耦。

```
// 知识点 5.4.003：2-SAT 的字典序最小可行赋值。
namespace lex_two_sat
{
optional<vector<int>> lexTwoSat(int n, const vector<array<int, 4>> &clauses)
{
    // n 是变量数；clauses[i] 四维依次是 u、u 的目标布尔值、v、v 的目标布尔值，表示二者至少一个成立。
    auto feasible = [&](const vector<int> &fix) -> bool
    {
        // fix[x] 为 -1 或强制布尔值；返回加入强制条件后公式是否可满足。
        vector<vector<int>> g(2 * n), rg(2 * n);
        auto add = [&](int u, int v)
        {
            // u、v 是文字编号；加入蕴含 u⇒v，无返回值。
            g[u].push_back(v);
            rg[v].push_back(u);
        };
        for (auto [u, a, v, b] : clauses)
        {
```

```
            assert(0 <= u && u < n && 0 <= v && v < n && (a == 0 || a == 1) && (b == 0 || b ==
            ↪ 1)); // 调试检查，可删。
            int x = 2 * u + a, y = 2 * v + b;
            add(x ^ 1, y);
            add(y ^ 1, x);
        }
        for (int i = 0; i < n; i++)
        {
            if (fix[i] >= 0)
            {
                add(2 * i + (fix[i] ^ 1), 2 * i + fix[i]);
            }
        }
        vector<int> vis(2 * n), ord, id(2 * n, -1);
        auto dfs = [&](auto &&self, int u) -> void
        {
            // self 是递归自身，u 是文字；完成正图后序遍历，无返回值。
            vis[u] = 1;
            for (int v : g[u])
            {
                if (!vis[v])
                {
                    self(self, v);
                }
            }
            ord.push_back(u);
        };
        auto rdfs = [&](auto &&self, int u, int c) -> void
        {
            // self 是递归自身，u 是文字，c 是分量编号；标记反图分量，无返回值。
            id[u] = c;
            for (int v : rg[u])
            {
                if (id[v] < 0)
                {
                    self(self, v, c);
                }
            }
        };
        for (int i = 0; i < 2 * n; i++)
        {
            if (!vis[i])
            {
                dfs(dfs, i);
            }
        }
        reverse(ord.begin(), ord.end());
        int c = 0;
        for (int u : ord)
        {
            if (id[u] < 0)
            {
```

```
                rdfs(rdfs, u, c++);
            }
        }
        for (int i = 0; i < n; i++)
        {
            if (id[2 * i] == id[2 * i + 1])
            {
                return false;
            }
        }
        return true;
    };
    vector<int> ans(n, -1);
    if (!feasible(ans))
    {
        return nullopt;
    }
    for (int i = 0; i < n; i++)
    {
        ans[i] = 0;
        if (!feasible(ans))
        {
            ans[i] = 1;
        }
    }
    return ans;
}
}
```

`lexTwoSat(n, clauses)` 中 `clauses[i]` 是四元组 (u, a, v, b)，表示子句「$x_u{=}a$ 或 $x_v{=}b$」$(a, b \in \{0, 1\})$；返回 `optional<vector<int>>`，无解为空，否则是字典序最小赋值（每位取 0 或 1）。逐变量各做一次判定、每次判定一遍 SCC，最坏 $O(n(V+E))$ 时间、$O(V+E)$ 空间，其中 $V = 2n$。只需任意一组解而不在意字典序时，用一次 5.4.001 Tarjan 2-SAT 或 5.4.002 Kosaraju 2-SAT 即可，快 n 倍；本条只在明确要字典序最小或要按优先级增量固定变量时才用。

5.5 最小生成结构

5.5.001 Kruskal

最小生成树的正确性来自割性质：任意割上的最轻边都属于某棵 MST。Kruskal 把边按权升序扫描，只接受连接不同分量的边；接受 $n-1$ 条后成树，不足则原图不连通。这里同时给出本节唯一的并查集底座 `DSU`，后续最小生成结构均复用它。`kruskal` 用可配置的累计类型保存总权，避免把通常不需要的宽整数带入默认实例。稠密图可另写邻接矩阵 Prim，把每轮选边降为扫描点集；本条只提供 Kruskal，不在标题中承诺未给出的实现。

```
struct DSU
{
    // f[x] 是并查集父亲，s[x] 仅在根处保存连通块大小。
```

```
    vector<int> f, s;

    DSU(int n = 0)
    {
        // n 是元素个数；建立 n 个单点集合。
        init(n);
    }

    void init(int n)
    {
        // n 是元素个数；清空旧状态并建立 n 个单点集合。
        f.resize(n);
        iota(f.begin(), f.end(), 0);
        s.assign(n, 1);
    }

    int find(int x)
    {
        // x 是元素编号；返回所在集合的根并压缩路径。
        return x == f[x] ? x : f[x] = find(f[x]);
    }

    bool merge(int x, int y)
    {
        // x、y 是元素编号；合并所在集合，原本不连通时返回 true。
        x = find(x);
        y = find(y);
        if (x == y)
        {
            return false;
        }
        if (s[x] < s[y])
        {
            swap(x, y);
        }
        f[y] = x;
        s[x] += s[y];
        return true;
    }

    bool same(int x, int y)
    {
        // x、y 是元素编号；返回二者是否属于同一集合。
        return find(x) == find(y);
    }
    int size(int x)
    {
        // x 是元素编号；返回所在集合的元素个数。
        return s[find(x)];
    }
};

```

```cpp
struct MEdge
{
    // x、y 是无向边的两个端点。
    int x, y;
    // w 是边权。
    int w;
};

optional<pair<int, vector<MEdge>>> kruskal(int n, vector<MEdge> e)
{
    // n 是点数，e 是无向边集；返回 MST 总权与所选边，图不连通时返回空。
    sort(e.begin(),
         e.end(),
         [](const MEdge &a, const MEdge &b)
         {
             return a.w < b.w;
         });
    DSU d(n);
    int ans = 0;
    vector<MEdge> use;
    for (auto v : e)
    {
        if (!d.merge(v.x, v.y))
        {
            continue;
        }
        ans += v.w;
        use.push_back(v);
    }
    if ((int)use.size() + 1 != n && n)
    {
        return nullopt;
    }
    return pair{ans, use};
}
```

`kruskal(n,e)` 中 `MEdge` 保存端点 `x,y` 和边权 `w`；图不连通时返回空，否则第一项是总权，第二项是所选边。调用者必须保证生成树总权可由 64 位有符号整数表示；确实需要更宽类型时，直接把 `ans` 改为 `__int128`。排序主导 $O(m \log m)$ 时间，空间为 $O(n+m)$。`DSU` 与 `MEdge` 供 5.5.003 K 棵树的最小生成森林、5.5.004 带修改最小生成树 和 5.5.010 Kruskal 重构树 复用；严格次小值见 5.5.002 次小生成树。

5.5.002 次小生成树

先求一棵 MST，任何非树边 (u, v, w) 加进来都会与树上 $u \to v$ 的唯一路径形成一个环，换掉环上一条树边就得到另一棵生成树，权值增量为 w 减去被删边权。要让增量最小又非零，就删这条路径上权值最大的树边；据此枚举所有非树边取最小增量即得次小生成树。「不同边集次小」允许增量为零（存在等权替换），「严格次小」要求权值真正变大：当 w 恰等于路径最大边权时，最大边换不出正增量，必须改删路径上与最大值不同的次大边权，因此倍增维护每段路径的最大与严格次大两个边权。实现用倍增 LCA 同时上跳求这两个量。

```cpp
// 依赖 5.5.001 Kruskal
struct MstTop2
{
    // a[0..1] 是一段路径上最大、严格次大的边权，不存在时为空。
    array<optional<int>, 2> a;
};

MstTop2 mstMergeTop(const MstTop2 &a, const MstTop2 &b)
{
    // a、b 是两段路径的最大值记录；返回合并后的两个互异最大值。
    vector<int> v;
    for (auto x : a.a)
    {
        if (x)
        {
            v.push_back(*x);
        }
    }
    for (auto x : b.a)
    {
        if (x)
        {
            v.push_back(*x);
        }
    }
    sort(v.begin(), v.end(), greater<>());
    v.erase(unique(v.begin(), v.end()), v.end());
    MstTop2 c;
    for (int i = 0; i < min<int>(2, v.size()); i++)
    {
        c.a[i] = v[i];
    }
    return c;
}

struct MstVariants
{
    // mst 是最小生成树权值。
    int mst;
    // other 是不同边集的最小权值，不存在时为空。
    optional<int> other;
    // strict 是严格大于 mst 的最小生成树权值，不存在时为空。
    optional<int> strict;
};

optional<MstVariants> secondMST(int n, const vector<array<int, 3>> &edges)
{
    // edges[i][0..2] 是端点 u、v 与边权；图不连通时返回空。
    if (n == 0)
    {
        return MstVariants{0, nullopt, nullopt};
    }
```

```
    int m = edges.size();
    vector<int> ord(m);
    iota(ord.begin(), ord.end(), 0);
    sort(ord.begin(),
         ord.end(),
         [&](int x, int y)
         {
             return edges[x][2] < edges[y][2];
         });
    DSU dsu(n);
    vector<bool> used(m);
    vector<vector<array<int, 2>>> g(n);
    int mst = 0;
    int cnt = 0;
    for (auto id : ord)
    {
        auto [u, v, w] = edges[id];
        if (!dsu.merge(u, v))
        {
            continue;
        }
        used[id] = true;
        mst += w;
        cnt++;
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
    if (cnt != n - 1)
    {
        return nullopt;
    }
    int lg = bit_width((unsigned long long)max<int>(1, n));
    vector up(lg, vector<int>(n));
    vector mx(lg, vector<MstTop2>(n));
    vector<int> dep(n);
    vector<int> par(n, -1), seq{0};
    par[0] = 0;
    for (int i = 0; i < (int)seq.size(); i++)
    {
        int u = seq[i];
        up[0][u] = par[u];
        for (auto [v, w] : g[u])
        {
            if (v == par[u])
            {
                continue;
            }
            par[v] = u;
            dep[v] = dep[u] + 1;
            mx[0][v].a[0] = w;
            seq.push_back(v);
        }
```

```
    }
    for (int j = 1; j < lg; j++)
    {
        for (int u = 0; u < n; u++)
        {
            mx[j][u] = mstMergeTop(mx[j - 1][u], mx[j - 1][up[j - 1][u]]);
            up[j][u] = up[j - 1][up[j - 1][u]];
        }
    }
    auto pathMax = [&](int u, int v) -> MstTop2
    {
        // u、v 是树上点；返回路径边权中最大的两个互异值。
        MstTop2 ans;
        if (dep[u] < dep[v])
        {
            swap(u, v);
        }
        for (int j = lg - 1; j >= 0; j--)
        {
            if (dep[u] - (1LL << j) >= dep[v])
            {
                ans = mstMergeTop(ans, mx[j][u]);
                u = up[j][u];
            }
        }
        if (u == v)
        {
            return ans;
        }
        for (int j = lg - 1; j >= 0; j--)
        {
            if (up[j][u] != up[j][v])
            {
                ans = mstMergeTop(ans, mx[j][u]);
                ans = mstMergeTop(ans, mx[j][v]);
                u = up[j][u];
                v = up[j][v];
            }
        }
        ans = mstMergeTop(ans, mx[0][u]);
        return mstMergeTop(ans, mx[0][v]);
    };
    optional<int> other, strict;
    auto upd = [] (optional<int> &x, int v)
    {
        // x 是当前最优候选，v 是新候选；保留较小值。
        if (!x || v < *x)
        {
            x = v;
        }
    };
    for (int id = 0; id < m; id++)
```

```
    {
        if (used[id])
        {
            continue;
        }
        auto [u, v, w] = edges[id];
        auto top = pathMax(u, v);
        if (top.a[0])
        {
            int cur = mst + w - *top.a[0];
            upd(other, cur);
            if (w > *top.a[0])
            {
                upd(strict, cur);
            }
            else if (w == *top.a[0] && top.a[1])
            {
                upd(strict, mst + w - *top.a[1]);
            }
        }
    }
    return MstVariants{mst, other, strict};
}
```

`secondMST(n,edges)` 的边记录依次是 u, v, w；图不连通时返回空，否则 `mst` 是 MST 权，`other` 是不同边集的最小权，`strict` 是严格更大的最小权，后两者不存在时各自为空。调用者必须保证生成树总权和替换后权值可由 64 位有符号整数表示。倍增建表和非树边查询合计 $O((V+E)\log V)$ 时间与空间。四点环加一条权 2 的对角边时三项依次为 $3, 3, 4$。本条复用 5.5.001 Kruskal 的 `DSU`。

5.5.003 K 棵树的最小生成森林

要把 n 个点组织成恰好 k 棵树的森林且总权最小，只需跑 Kruskal 到连通块数从 n 降到 k 时停手。按边权升序选不成环的边，在任意连通块数下都是该块数的最小权森林。实现复用 5.5.001 Kruskal 的 `DSU` 与 `MEdge`；`k=n` 必须在扫描边前直接返回零，否则第一次成功合并就再也回不到 n 个连通块。

```
// 依赖 5.5.001 Kruskal
optional<int> minKForest(int n, int k, vector<MEdge> e)
{
    // n 是点数，k 是目标连通块数，e 是无向边集；返回最小森林权值。
    assert(n >= 1 && 1 <= k && k <= n); // 调试检查，可删。
    if (k == n)
    {
        return 0;
    }
    sort(e.begin(),
         e.end(),
         [](auto a, auto b)
         {
             return a.w < b.w;
```

```
         });
    DSU d(n);
    int cc = n;
    int ans = 0;
    for (auto [u, v, w] : e)
    {
        if (!d.merge(u, v))
        {
            continue;
        }
        ans += w;
        if (--cc == k)
        {
            return ans;
        }
    }
    return cc == k ? optional<int>(ans) : nullopt;
}
```

`minKForest(n,k,e)` 要求 $n \geq 1$ 且 $1 \leq k \leq n$；图自身连通块数超过 k 时返回空，否则返回最小森林总权。调用者必须保证答案可由 64 位有符号整数表示。四点环上 $k = 2, 1, 4$ 的答案依次为 $2, 3, 0$。排序主导 $O(E \log E)$ 时间，额外空间为 $O(V)$；$k = 1$ 就是 5.5.001 Kruskal。

5.5.004 带修改最小生成树

这里给出最朴素的带修改版本：`DynamicMST` 保存全部边，`set(id,w)` 直接改权，`query()` 调用 5.5.003 K 棵树的最小生成森林 的 `minKForest(n,1,e)` 重跑 Kruskal。修改是 $O(1)$，查询承担全部排序成本，适合查询少的场景；修改和查询都密集时应换 LCT 或离线分治。本条复用 5.5.001 Kruskal 的 `MEdge`，不再定义同名边类型或复制森林算法。

```
// 依赖 5.5.001 Kruskal
// 依赖 5.5.003 K 棵树的最小生成森林
struct DynamicMST
{
    // n 是图的点数。
    int n;
    // e 是当前无向边集。
    vector<MEdge> e;

    DynamicMST(int n, vector<MEdge> e) : n(n), e(move(e))
    {
        // n 是点数，e 是初始边集；保存当前图。
    }

    void set(int id, int w)
    {
        // id 是边编号，w 是新边权；修改该边。
        e[id].w = w;
    }
```

```
    optional<int> query() const
    {
        // 无参数；返回当前 MST 总权，图不连通时返回空。
        return minKForest(n, 1, e);
    }
};
```

构造 `DynamicMST dm(n,e)`，边使用 `MEdge{x,y,w}`；`set(id,w)` 改边权，`query()` 返回 MST 总权，不连通时为空。调用者必须保证答案可由 64 位有符号整数表示。修改用 $O(1)$ 时间，查询用 $O(m \log m)$ 时间、$O(n)$ 额外空间。静态一次查询见 5.5.001 Kruskal，次小值见 5.5.002 次小生成树。

5.5.005 最小乘积生成树

每条边带两种非负代价，要最小化生成树两种总代价之积 $X \cdot Y$。把每棵生成树映射成平面点 (X, Y)，最优点落在这些点的下凸壳上；用 $aX + bY$ 作边权跑一次 MST，就得到沿方向 (a, b) 的支撑点。先以 $(1, 0)$、$(0, 1)$ 求凸壳两端，再用垂直于端点连线的方向继续找中间支撑点。两种权和与差先在 `__int128` 中形成；线性权、叉积和最终乘积只需比较两个 128 位乘积，辅助结构用四个 64 位肢精确完成比较，避免再次溢出且不依赖额外大整数库。底层复用 5.5.001 Kruskal 的 `DSU`。

```
// 依赖 5.5.001 Kruskal
struct MstU256
{
    // a 从低到高保存四个 64 位肢。
    array<unsigned long long, 4> a{};

    static MstU256 mul(unsigned __int128 x, unsigned __int128 y)
    {
        // x、y 是无符号 128 位整数；返回精确的 256 位乘积。
        unsigned long long p[2] = {(unsigned long long)x, (unsigned long long)(x >> 64)};
        unsigned long long q[2] = {(unsigned long long)y, (unsigned long long)(y >> 64)};
        MstU256 z;
        for (int i = 0; i < 2; i++)
        {
            unsigned __int128 c = 0;
            for (int j = 0; j < 2; j++)
            {
                unsigned __int128 t = (unsigned __int128)p[i] * q[j] + z.a[i + j] + c;
                z.a[i + j] = (unsigned long long)t;
                c = t >> 64;
            }
            z.a[i + 2] = (unsigned long long)c;
        }
        return z;
    }
};

int mstProdCmp(__int128 a, __int128 b, __int128 c, __int128 d)
{
```

```
    // 返回 a*b 与 c*d 的大小关系。
    auto mag = [](__int128 x) -> unsigned __int128
    {
        return x < 0 ? 0 - (unsigned __int128)x : (unsigned __int128)x;
    };
    auto x = MstU256::mul(mag(a), mag(b));
    auto y = MstU256::mul(mag(c), mag(d));
    bool nx = (a < 0) != (b < 0), ny = (c < 0) != (d < 0);
    if (x.a == array<unsigned long long, 4>{})
    {
        nx = false;
    }
    if (y.a == array<unsigned long long, 4>{})
    {
        ny = false;
    }
    if (nx != ny)
    {
        return nx ? -1 : 1;
    }
    int s = 0;
    for (int i = 3; i >= 0 && !s; i--)
    {
        s = (x.a[i] > y.a[i]) - (x.a[i] < y.a[i]);
    }
    return nx ? -s : s;
}

struct ProductMST
{
    using Cost = pair<__int128, __int128>;

    struct E
    {
        // u、v 是无向边端点。
        int u, v;
        // x、y 是两种非负边权。
        int x, y;
    };

    // n 是图的点数。
    int n;
    // e 是全部无向边。
    vector<E> e;

    static bool better(const Cost &a, const Cost &b)
    {
        // a、b 是两棵生成树的两种总权；返回 a 的乘积是否更优。
        int c = mstProdCmp(a.first, a.second, b.first, b.second);
        return c != 0 ? c < 0 : a < b;
    }

```

```
    ProductMST(int n = 0) : n(n)
    {
        // n 是图的点数；构造一张没有边的图。
    }

    void add(int u, int v, int x, int y)
    {
        // u、v 是端点，x、y 是两种非负边权；加入一条无向边。
        assert(x >= 0 && y >= 0); // 调试检查，可删。
        e.push_back({u, v, x, y});
    }

    optional<Cost> mst(__int128 a, __int128 b) const
    {
        // a、b 是线性目标系数；返回该方向下 MST 的两种原始总权。
        vector<int> p(e.size());
        iota(p.begin(), p.end(), 0);
        sort(p.begin(),
             p.end(),
             [&](int i, int j)
             {
                 int c = mstProdCmp(a,
                                    (__int128)e[i].x - e[j].x,
                                    b,
                                    (__int128)e[j].y - e[i].y);
                 if (c != 0)
                 {
                     return c < 0;
                 }
                 if (e[i].x != e[j].x)
                 {
                     return e[i].x < e[j].x;
                 }
                 return e[i].y < e[j].y;
             });
        DSU d(n);
        Cost s{};
        int z = 0;
        for (int i : p)
        {
            if (d.merge(e[i].u, e[i].v))
            {
                s.first += (__int128)e[i].x;
                s.second += (__int128)e[i].y;
                z++;
            }
        }
        if (z != n - 1)
        {
            return nullopt;
        }
        return s;
```

```
    }

    optional<Cost> solve() const
    {
        // 无参数；返回使两种边权总和乘积最小的生成树两项权值和。
        if (n <= 1)
        {
            return Cost{0, 0};
        }
        auto a = mst(1, 0);
        auto b = mst(0, 1);
        if (!a || !b)
        {
            return nullopt;
        }
        Cost ans = better(*a, *b) ? *a : *b;
        auto divide = [&](auto &&self, Cost l, Cost r) -> void
        {
            __int128 x = l.second - r.second;
            __int128 y = r.first - l.first;
            auto c = mst(x, y);
            if (!c || *c == l || *c == r)
            {
                return;
            }
            int cr = mstProdCmp(r.first - l.first,
                                c->second - l.second,
                                r.second - l.second,
                                c->first - l.first);
            if (cr >= 0)
            {
                return;
            }
            if (better(*c, ans))
            {
                ans = *c;
            }
            self(self, l, *c);
            self(self, *c, r);
        };
        divide(divide, *a, *b);
        return ans;
    }
};
```

构造 `ProductMST pm(n)`，用 `add(u,v,x,y)` 加非负双权边，`solve()` 返回 `pair<__int128,__int128>` 形式的最优 (X, Y)，不连通时为空。每次 Kruskal 用 $O(m \log m)$ 时间，若下凸壳访问了 h 个支撑点，总时间为 $O(hm \log m)$、空间为 $O(n + m + h)$。两种代价成固定线性关系时直接用 5.5.001 Kruskal。

5.5.006 切比雪夫距离最小生成树

平面上 n 个点两两以切比雪夫距离 $\max(|\Delta x|, |\Delta y|)$ 连边求 MST，完全图有 $O(n^2)$ 条边不可直接跑。关键结论是曼哈顿 / 切比雪夫距离的 MST 只需保留每个点在八分之一平面内的最近邻这类候选边——切比雪夫经 $(x+y,\ x-y)$ 变换即化为曼哈顿距离，而曼哈顿 MST 只要在四个（对称后八个）方向的每个锥区里为每点连一条最近点边就够了。用按坐标排序 + 树状数组维护「某区间内 $x+y$ 最小的点」即可在 $O(n \log n)$ 内生成 $O(n)$ 条候选边，再对这批稀疏边跑 Kruskal。

```cpp
// 依赖 5.5.001 Kruskal
// CP[0..1] 依次是原点的 x、y 坐标。
using CP = array<int, 2>;

struct GeometricMST
{
    __int128 weight = 0; // 最小生成树总权。
    vector<pair<int, int>> edge; // 所选边的原点编号。
};

GeometricMST chebyMST(const vector<CP> &input)
{
    // input[i][0..1] 是原点坐标；返回切比雪夫 MST 总权与所选边。
    struct Point
    {
        // x、y 是变换后的坐标，id 是原点编号。
        __int128 x, y;
        int id;
    };
    struct Edge
    {
        // w 是候选边权，u、v 是原点编号。
        __int128 w;
        int u, v;
    };
    int n = input.size();
    vector<Point> p;
    for (int i = 0; i < n; ++i)
    {
        p.push_back({(__int128)input[i][0] + input[i][1],
                     (__int128)input[i][0] - input[i][1], i});
    }
    vector<Edge> e;
    for (int dir = 0; dir < 4; ++dir)
    {
        ranges::sort(p,
                     [](const Point &a, const Point &b)
                     {
                         return a.x + a.y != b.x + b.y ? a.x + a.y < b.x + b.y : a.x < b.x;
                     });
        map<__int128, int> sweep;
        for (int i = 0; i < n; ++i)
        {
```

```
            for (auto it = sweep.lower_bound(-p[i].y); it != sweep.end();)
            {
                int j = it->second;
                if (p[i].x - p[j].x < p[i].y - p[j].y)
                {
                    break;
                }
                __int128 dx = p[i].x - p[j].x;
                __int128 dy = p[i].y - p[j].y;
                if (dx < 0)
                {
                    dx = -dx;
                }
                if (dy < 0)
                {
                    dy = -dy;
                }
                e.push_back({(dx + dy) / 2, p[i].id, p[j].id});
                it = sweep.erase(it);
            }
            sweep[-p[i].y] = i;
        }
        for (auto &v : p)
        {
            if (dir & 1)
            {
                v.x = -v.x;
            }
            else
            {
                swap(v.x, v.y);
            }
        }
    }
    ranges::sort(e,
                 {},
                 [](const Edge &a)
                 {
                     return a.w;
                 });
    DSU d(n);
    GeometricMST ans;
    for (auto [w, u, v] : e)
    {
        if (!d.merge(u, v))
        {
            continue;
        }
        ans.weight += w;
        ans.edge.push_back({u, v});
    }
    return ans;
```

```
}

__int128 chebyshevMst(const vector<CP> &input)
{
    // input[i][0..1] 是原点坐标；返回切比雪夫 MST 总权。
    return chebyMST(input).weight;
}
```

`chebyMST(input)` 返回总权和一组原下标 MST 边，`chebyshevMst(input)` 仅取总权；`CP` 存 64 位整点坐标，坐标和差、候选距离与返回总权均为 `__int128`。候选边生成和 Kruskal 各用 $O(n \log n)$ 时间，空间为 $O(n)$。原坐标 Manhattan 距离等于变换后坐标的切比雪夫距离，故需要 Manhattan MST 边时先传入 `(x+y,x-y)`。欧氏距离版本见 5.5.007 平面欧氏距离最小生成树。

5.5.007 平面欧氏距离最小生成树

二维点两两以欧氏距离连边求 MST，完全图 $O(n^2)$ 条边。这里用稠密图 Prim 避免显式建边：维护每个未选点到当前生成树的最短距离 `d[i]`，每轮 $O(n)$ 扫出最近的未选点加入，再用它 $O(n)$ 更新其余 `d`，边权按坐标即时算平方欧氏距离（比较用平方避免开根、累加时才开根）。总共 $O(n^2)$，无需存边表，适合中等点数；点数很大时应改用 Delaunay 三角剖分把候选边降到 $O(n)$。

```
// 依赖 5.5.001 Kruskal
long double euclideanMST(const vector<array<long double, 2>> &p)
{
    // p[i][0..1] 依次是第 i 个点的 x、y 坐标；返回完全图欧氏距离最小生成树总长。
    int n = p.size();
    if (n == 0)
    {
        return 0;
    }
    vector<long double> dis(n, numeric_limits<long double>::infinity());
    vector<bool> vis(n);
    dis[0] = 0;
    long double ans = 0;
    for (int t = 0; t < n; t++)
    {
        int u = -1;
        for (int i = 0; i < n; i++)
        {
            if (!vis[i] && (u == -1 || dis[i] < dis[u]))
            {
                u = i;
            }
        }
        vis[u] = true;
        ans += sqrtl(dis[u]);
        for (int v = 0; v < n; v++)
        {
            if (vis[v])
            {
```

```
                continue;
            }
            long double dx = p[u][0] - p[v][0], dy = p[u][1] - p[v][1];
            dis[v] = min(dis[v], dx * dx + dy * dy);
        }
    }
    return ans;
}

vector<pair<int, int>> euclidMST(const vector<array<long long, 2>> &p)
{
    // p[i][0..1] 是整数点坐标；返回完全图欧氏距离 MST 的原点编号边集。
    int n = p.size();
    if (n <= 1)
    {
        return {};
    }
    map<pair<long long, long long>, int> first;
    vector<IP> unique;
    vector<int> rep;
    vector<MEdge> edge;
    for (int i = 0; i < n; i++)
    {
        pair key{p[i][0], p[i][1]};
        auto [it, fresh] = first.emplace(key, i);
        if (!fresh)
        {
            edge.push_back({it->second, i, 0});
        }
        else
        {
            unique.push_back({p[i][0], p[i][1]});
            rep.push_back(i);
        }
    }
    auto tri = delaunay(unique);
    for (auto [x, y] : tri.edge)
    {
        long long dx = unique[x].x - unique[y].x;
        long long dy = unique[x].y - unique[y].y;
        edge.push_back({rep[x], rep[y], dx * dx + dy * dy});
    }
    auto mst = kruskal(n, move(edge));
    assert(mst.has_value()); // 调试检查，可删；Delaunay 边与重合点零边必然连通。
    vector<pair<int, int>> ans;
    for (auto [x, y, w] : mst->second)
    {
        ans.push_back({x, y});
    }
    return ans;
}
```

`euclideanMST(p)` 保留稠密 Prim 小规模版，返回浮点总长，时间 $O(n^2)$、空间 $O(n)$。大规模整数点调用 `euclidMST(p)`：先用零边连接重合点，再依赖 11.3.011 Delaunay 三角剖分 生成 $O(n)$ 条候选边并按距离平方 Kruskal，返回原下标边集；时间 $O(n \log n)$、空间 $O(n)$，要求坐标差四次方满足 Delaunay 条目的 `__int128` 范围。单位正方形的两版 MST 总权均为 3。

5.5.008 最小树形图 / 朱刘算法

最小树形图要在有向图里求一棵以 `root` 为根、能到达所有点的生成树，使总权最小；每个非根点恰有一条入边。朱刘算法的思想是：先给每个非根点选权值最小的入边，累加这些入边权得到一个下界。如果这些入边不成环，它们就直接构成答案；否则出现有向环，说明环上的点互相「抢」了对方的最小入边，需要把整个环收缩成一个超级点，并把所有指向环内点 v 的边权减去 v 当前选中的最小入边权（这样重新在收缩图上选最小入边时，等价于「换掉环上某条边、改从环外进入」的代价差），然后在收缩后的更小图上递归重复。每轮至少收缩掉一个环、点数严格减少，故最多 $n-1$ 轮。代码用 `pre` 记每点当前最小入边来源，用一次遍历给环染色 `id` 完成收缩。

```
struct DEdgeMST
{
    // u、v 是有向边的起点和终点。
    int u, v;
    // w 是当前收缩图中的边权。
    __int128 w;
};

optional<__int128> directedMST(int n, int root, vector<DEdgeMST> e)
{
    // n 是点数，root 是树根，e 是有向边集；返回最小树形图权值，不存在时返回空。
    assert(n >= 1 && 0 <= root && root < n); // 调试检查，可删。
    __int128 ans = 0;
    while (true)
    {
        vector<optional<__int128>> in(n);
        vector<int> pre(n, -1);
        for (auto [u, v, w] : e)
        {
            if (u != v && (!in[v] || w < *in[v]))
            {
                in[v] = w;
                pre[v] = u;
            }
        }
        in[root] = 0;
        for (int u = 0; u < n; u++)
        {
            if (!in[u])
            {
                return nullopt;
            }
            ans += *in[u];
        }
```

```
        int cnt = 0;
        vector<int> id(n, -1), vis(n, -1);
        for (int s = 0; s < n; s++)
        {
            int u = s;
            while (vis[u] != s && id[u] < 0 && u != root)
            {
                vis[u] = s;
                u = pre[u];
            }
            if (u != root && id[u] < 0)
            {
                for (int v = pre[u]; v != u; v = pre[v])
                {
                    id[v] = cnt;
                }
                id[u] = cnt++;
            }
        }
        if (!cnt)
        {
            return ans;
        }
        for (int u = 0; u < n; u++)
        {
            if (id[u] < 0)
            {
                id[u] = cnt++;
            }
        }
        for (auto &[u, v, w] : e)
        {
            int old = v;
            u = id[u];
            v = id[v];
            if (u != v)
            {
                w -= *in[old];
            }
        }
        root = id[root];
        n = cnt;
    }
}
```

`directedMST(n,root,e)` 要求 $n \geq 1$ 且 `root` 是合法点编号，返回以 `root` 为根的最小树形图总权，不存在时为空；`DEdgeMST::w` 和返回值均为 `__int128`。参数 `e` 按值传入，收缩只修改函数内副本，不会改调用方边集。算法至多收缩 $n-1$ 轮，用 $O(nm)$ 时间、$O(n+m)$ 空间，适合小规模、需要宽权值或只要总权的场景。大规模且需要父节点时使用同条目的 `dmstTree(n,root,e)`：它以配对堆维护每个收缩点的入边，`DMSTFastEdge` 用 `long long` 存边权，返回 `DMSTResult{weight,parent}`，其中 `parent[root]=root`。该版本在 $O((n+m)\log n)$ 期望时间和 $O(n+m)$ 空间内恢复一棵最优树，适用于 $n, m \leq 2 \times 10^5$；

两版并存，不能以快速版替代需要 `__int128` 总权的旧接口。有向根树与 5.5.001 Kruskal 的无向 MST 不同，不能互换。下面的四点图含一个非根有向环，最优值为 6；第二张图没有到点 2 的入边，故无解。

```
signed main()
{
    vector<DEdgeMST> e{{0, 1, 4}, {0, 2, 5}, {1, 2, 1},
                       {2, 3, 1}, {3, 1, 1}, {1, 3, 3}};
    assert(directedMST(4, 0, e) == 6);
    assert(!directedMST(3, 0, {{0, 1, 1}}));
}
```

5.5.009 最小斯坦纳树

斯坦纳树问题要连通指定的 k 个关键点，允许借道其余「斯坦纳点」，求最小总边权——当 k 等于全部点时就退化成 MST。关键点少时用状压 DP：`f[S][u]` 表示「已连通关键点集合 S、且当前树含点 u」的最小代价。转移分两种：一是子集合并，把 S 拆成两个不相交子集 a、b，在同一个汇点 u 处拼接，`f[S][u] = min(f[a][u]+f[b][u])`；二是在固定 S 后沿图边做一次 Dijkstra 松弛 `f[S][v] = min(f[S][u]+w(u,v))`，让这棵树「长」到相邻点上。初值把每个关键点单独成集的代价设 0，最终答案是 `f[full]` 在所有汇点上的最小值。

```
struct Steiner
{
    struct E
    {
        // to 是邻点编号。
        int to;
        // w 是非负边权。
        int w;
    };

    // n 是图的点数。
    int n;
    // e 是无向图邻接表。
    vector<vector<E>> e;

    Steiner(int n = 0) : n(n), e(n)
    {
        // n 是图的点数；构造一张没有边的图。
    }

    void add(int u, int v, int w)
    {
        // u、v 是端点，w 是非负边权；加入一条无向边。
        assert(w >= 0); // 调试检查，可删。
        e[u].push_back({v, w});
        e[v].push_back({u, w});
    }

    optional<int> solve(const vector<int> &key) const
```

```
    {
        // key 是关键点编号列表；返回连接全部关键点的最小权值，不可连通时返回空。
        size_t k = key.size();
        if (k == 0)
        {
            return 0;
        }
        assert(k < 63); // 调试检查，可删。
        size_t z = (size_t)1 << k;
        vector f(z, vector<optional<int>>(n));
        for (size_t i = 0; i < k; i++)
        {
            assert(0 <= key[i] && key[i] < n); // 调试检查，可删。
            f[(size_t)1 << i][key[i]] = 0;
        }
        for (size_t s = 1; s < z; s++)
        {
            for (size_t a = (s - 1) & s; a; a = (a - 1) & s)
            {
                size_t b = s ^ a;
                if (!b || a > b)
                {
                    continue;
                }
                for (int u = 0; u < n; u++)
                {
                    if (f[a][u] && f[b][u])
                    {
                        int nd = *f[a][u] + *f[b][u];
                        if (!f[s][u] || nd < *f[s][u])
                        {
                            f[s][u] = nd;
                        }
                    }
                }
            }
            priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> q;
            for (int u = 0; u < n; u++)
            {
                if (f[s][u])
                {
                    q.push({*f[s][u], u});
                }
            }
            while (!q.empty())
            {
                auto [d, u] = q.top();
                q.pop();
                if (!f[s][u] || d != *f[s][u])
                {
                    continue;
                }
```

```
                for (auto [v, w] : e[u])
                {
                    int nd = d + w;
                    if (!f[s][v] || nd < *f[s][v])
                    {
                        f[s][v] = nd;
                        q.push({nd, v});
                    }
                }
            }
        }
        optional<int> ans;
        for (auto x : f.back())
        {
            if (x && (!ans || *x < *ans))
            {
                ans = *x;
            }
        }
        return ans;
    }
};
```

构造 `Steiner g(n)` 后用 `add(u,v,w)` 加非负权无向边，`solve(key)` 返回最小权；关键点为空时为 0，关键点不能互相连通时为空。调用者必须保证边权和可由 64 位有符号整数表示，并且 $2^k n$ 的内存可承受。子集合并用 $O(3^k n)$ 时间，全部 Dijkstra 用 $O(2^k m \log n)$ 时间，空间为 $O(2^k n + m)$。关键点覆盖全部点时直接用 5.5.001 Kruskal。

5.5.010 Kruskal 重构树

把 5.5.001 Kruskal 的加边过程显式建成一棵二叉树。边按权升序处理，每当一条边成功合并两个连通块，就新建一个「代表这条边」的内部结点，其点权取该边权，并令它做两个块根的父亲。n 个原始点最终成为叶子，共得到 $2n-1$ 个结点（图连通时为一棵树，否则为森林）。两条关键性质：

- $\mathrm{lca}(u,v)$ 的点权 $=$ u,v 间所有路径上「最大边权」的最小值，即瓶颈路权、也等于最小生成树上 $u \to v$ 路径的最大边；
- 「从 u 出发只走权 $\le W$ 的边所能到达的点集」，恰为 u 在重构树上向上跳到「点权 $\le W$ 的最高祖先」，该祖先子树内的全部叶子。

若改按降序加边，则对应最大瓶颈（路径最小边的最大化）。第二条性质配合倍增找祖先 + DFS 序，能把「限制边权的可达性/第 K 大」类询问转成子树问题。

```
struct KruskalTree
{
    // n 是原始点数，tot 是当前重构树结点数。
    int n, tot;
    // val[x] 是内部结点代表的边权，叶子处为空。
    vector<optional<int>> val;
    // ch[x] 是内部结点的两个孩子，叶子为 {-1,-1}。
```

```
    vector<array<int, 2>> ch;
    // fa 是建树时使用的并查集父亲。
    vector<int> fa;
    // dep、up 是 build 后的深度和倍增祖先表。
    vector<int> dep;
    vector<vector<int>> up;

    KruskalTree(int n) : n(n), tot(n), val(2 * n), ch(2 * n, {-1, -1}), fa(2 * n)
    {
        // n 是原始点数；建立 n 个叶子。
        iota(fa.begin(), fa.end(), 0);
    }

    int find(int x)
    {
        // x 是当前结点；迭代查找并压缩所在块的根。
        int r = x;
        while (fa[r] != r)
        {
            r = fa[r];
        }
        while (fa[x] != x)
        {
            int p = fa[x];
            fa[x] = r;
            x = p;
        }
        return r;
    }

    void addEdge(int u, int v, int w)
    {
        // u、v 是原图端点，w 是当前边权；按非降权序合并两个块。
        int ru = find(u);
        int rv = find(v);
        if (ru == rv)
        {
            return;
        }
        int c = tot++;
        val[c] = w;
        ch[c] = {ru, rv};
        fa[ru] = c;
        fa[rv] = c;
        fa[c] = c;
    }

    void build()
    {
        // 无参数；加完所有边后重建深度和 LCA 倍增表。
        int lg = bit_width((unsigned long long)max<int>(1, tot));
        up.assign(lg, vector<int>(tot, -1));
```

```
        dep.assign(tot, 0);
        vector<char> vis(tot);
        for (int i = 0; i < tot; i++)
        {
            int r = find(i);
            if (vis[r])
            {
                continue;
            }
            vis[r] = 1;
            auto dfs = [&](auto &&self, int x) -> void
            {
                for (int c : {ch[x][0], ch[x][1]})
                {
                    if (c < 0)
                    {
                        continue;
                    }
                    dep[c] = dep[x] + 1;
                    up[0][c] = x;
                    self(self, c);
                }
            };
            dfs(dfs, r);
        }
        for (int k = 1; k < lg; k++)
        {
            for (int x = 0; x < tot; x++)
            {
                if (up[k - 1][x] >= 0)
                {
                    up[k][x] = up[k - 1][up[k - 1][x]];
                }
            }
        }
    }

    int lca(int u, int v)
    {
        // u、v 是原图点；返回重构森林 LCA，不连通时返回 -1。
        if (find(u) != find(v))
        {
            return -1;
        }
        if (dep[u] < dep[v])
        {
            swap(u, v);
        }
        int d = dep[u] - dep[v];
        for (int k = 0; d; k++)
        {
            if (d & 1)
```

```
            {
                u = up[k][u];
            }
            d >>= 1;
        }
        if (u == v)
        {
            return u;
        }
        for (int k = (int)up.size() - 1; k >= 0; k--)
        {
            if (up[k][u] != up[k][v])
            {
                u = up[k][u];
                v = up[k][v];
            }
        }
        return up[0][u];
    }

    optional<int> bottleneck(int u, int v)
    {
        // u、v 是原图点；返回瓶颈路权，不连通时返回空，同一点返回 0。
        if (u == v)
        {
            return 0;
        }
        int l = lca(u, v);
        if (l < 0)
        {
            return nullopt;
        }
        return val[l];
    }
};
```

把边按权非降序逐条传给 `addEdge(u,v,w)`，再调用 `build()`；之后 `bottleneck(u,v)` 返回瓶颈路权，不连通时为空。排序与建树用 $O(m \log m + n)$ 时间，查询用 $O(\log n)$ 时间，空间为 $O(n \log n)$。下面的图中点 4 孤立，路径 $0 \to 2$ 与 $0 \to 3$ 的瓶颈分别为 4 与 5。

```
signed main()
{
    vector<array<int, 3>> e{{2, 1, 2}, {4, 0, 1},
                            {5, 2, 3}, {7, 0, 2}};
    KruskalTree t(5);
    for (auto [w, u, v] : e)
    {
        t.addEdge(u, v, w);
    }
    t.build();
    assert(t.bottleneck(0, 0) == 0);
```

```
    assert(t.bottleneck(0, 2) == 4);
    assert(t.bottleneck(0, 3) == 5);
    assert(!t.bottleneck(0, 4));
}
```

5.5.011 增量最小生成森林

边只增加时，当前两端不连通的新边必进入最小生成森林；已经连通时，若新边比森林路径上的最重边更轻，就用它替换该最重边。把每条入选边建成一个 LCT 边点，原图端点连到边点，路径最大值便能同时给出权值和待删边编号。下面是 6.4.001 Link-Cut Tree 改成“路径最大边”的短专用版，不处理删去任意历史边或修改旧边权。

```
struct IncrMSF
{
    struct Node
    {
        array<int, 2> ch{}; // ch[0..1] 是 splay 左右儿子，0 表示空。
        int fa = 0; // fa 是 splay 父亲或所在链的路径父亲。
        bool rev = false, edge = false; // rev 是子树翻转标记，edge 表示本节点代表一条边。
        long long val = 0, mx = 0; // val 是边权（点节点为 0），mx 是 splay 子树内边节点的最大边权。
        int who = 0, u = 0, v = 0; // who 是取到 mx 的边节点（无边为 0），u、v 是边节点的两个端点。
    };

    int n, cc; // n 是点数，cc 是当前连通块数。
    int sum = 0; // sum 是当前最小生成森林总权。
    vector<Node> t; // t[1..n] 是点节点，之后依次是新建的边节点，t[0] 为空。

    IncrMSF(int n) : n(n), cc(n), t(n + 1)
    {
        // n 是点数；建立没有边的森林。
        assert(n >= 0); // 调试检查，可删。
    }

    bool isRoot(int x)
    {
        // x 是节点；返回 x 是否为所在 splay 的根。
        int f = t[x].fa;
        return !f || (t[f].ch[0] != x && t[f].ch[1] != x);
    }

    void pull(int x)
    {
        // x 是节点；由自身与儿子重算 mx 与取到它的边节点 who。
        t[x].who = t[x].edge ? x : 0;
        t[x].mx = t[x].val;
        for (int c : t[x].ch)
        {
            if (t[c].who && (!t[x].who || t[c].mx > t[x].mx))
            {
                t[x].who = t[c].who;
```

```
                t[x].mx = t[c].mx;
            }
        }
    }

    void reverseNode(int x)
    {
        // x 是节点（0 时忽略）；交换左右儿子并翻转标记。
        if (x)
        {
            swap(t[x].ch[0], t[x].ch[1]);
            t[x].rev ^= 1;
        }
    }

    void push(int x)
    {
        // x 是节点；把翻转标记下传给儿子。
        if (t[x].rev)
        {
            reverseNode(t[x].ch[0]);
            reverseNode(t[x].ch[1]);
            t[x].rev = false;
        }
    }

    void rotate(int x)
    {
        // x 是非根节点；把 x 旋转到其父亲的位置。
        int y = t[x].fa, z = t[y].fa, k = t[y].ch[1] == x, w = t[x].ch[k ^ 1];
        if (!isRoot(y))
        {
            t[z].ch[t[z].ch[1] == y] = x;
        }
        t[x].fa = z;
        t[x].ch[k ^ 1] = y;
        t[y].fa = x;
        t[y].ch[k] = w;
        if (w)
        {
            t[w].fa = y;
        }
        pull(y);
        pull(x);
    }

    void splay(int x)
    {
        // x 是节点；先自顶向下下传标记，再把 x 旋到所在 splay 的根。
        auto pushAll = [&](auto &&self, int y) -> void
        {
            if (!isRoot(y))
```

```
            {
                self(self, t[y].fa);
            }
            push(y);
        };
        pushAll(pushAll, x);
        while (!isRoot(x))
        {
            int y = t[x].fa, z = t[y].fa;
            if (!isRoot(y))
            {
                rotate((t[y].ch[1] == x) == (t[z].ch[1] == y) ? y : x);
            }
            rotate(x);
        }
    }

    void access(int x)
    {
        // x 是节点；打通根到 x 的偏好路径。
        int y = 0;
        while (x)
        {
            splay(x);
            int p = t[x].fa;
            t[x].ch[1] = y;
            pull(x);
            y = x;
            x = p;
        }
    }

    void makeRoot(int x)
    {
        // x 是节点；令 x 成为所在树的根。
        access(x);
        splay(x);
        reverseNode(x);
    }

    int findRoot(int x)
    {
        // x 是节点；返回 x 所在树的根。
        access(x);
        splay(x);
        while (true)
        {
            push(x);
            if (!t[x].ch[0])
            {
                break;
            }
```

```
            x = t[x].ch[0];
        }
        splay(x);
        return x;
    }

    void linkNode(int x, int y)
    {
        // x、y 在不同树中；连接 x 与 y。
        makeRoot(x);
        t[x].fa = y;
    }

    void cutNode(int x, int y)
    {
        // x、y 之间有树边；删除这条边。
        makeRoot(x);
        access(y);
        splay(y);
        assert(t[y].ch[0] == x && !t[x].ch[1]); // 调试检查，可删。
        t[y].ch[0] = t[x].fa = 0;
        pull(y);
    }

    int pathMax(int x, int y)
    {
        // x、y 连通；返回 x 到 y 路径上权值最大的边节点。
        makeRoot(x);
        access(y);
        splay(y);
        return t[y].who;
    }

    int newEdge(int u, int v, long long w)
    {
        // u、v 是 1 起端点，w 是边权；新建边节点并返回其编号。
        Node x;
        x.edge = true;
        x.val = x.mx = w;
        x.u = u;
        x.v = v;
        t.push_back(x);
        int id = t.size() - 1;
        t[id].who = id;
        return id;
    }

    bool add(int u, int v, long long w)
    {
        // u、v 是端点，w 是新边权；森林边集改变时返回 true。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        if (u == v)
```

```
        {
            return false;
        }
        u++;
        v++;
        if (findRoot(u) != findRoot(v))
        {
            int e = newEdge(u, v, w);
            linkNode(u, e);
            linkNode(e, v);
            sum += w;
            cc--;
            return true;
        }
        int e = pathMax(u, v);
        if (t[e].val <= w)
        {
            return false;
        }
        long long old = t[e].val;
        cutNode(e, t[e].u);
        cutNode(e, t[e].v);
        int ne = newEdge(u, v, w);
        linkNode(u, ne);
        linkNode(ne, v);
        sum += w - old;
        return true;
    }

    int weight() const
    {
        // 无参数；返回当前最小生成森林总权。
        return sum;
    }

    int comps() const
    {
        // 无参数；返回当前连通块数。
        return cc;
    }
};
```

构造 `IncrMSF q(n)`，按到达顺序调用 `q.add(u,v,w)`；`weight()` 始终是当前所有连通块的最小生成森林总权，`comps()` 给连通块数。调用者必须保证每次总权都可由 64 位有符号整数表示。每条边至多新建一个边点，单次加入均摊 $O(\log(n+m))$，总空间 $O(n+m)$；任意旧边修改仍用 5.5.004 带修改最小生成树 的重算版或另写完整动态算法。

下面按顺序加入边，检查成环时拒绝较重边、替换较重树边、负边以及自环：

```
signed main()
{
```

```
    IncrMSF q(4);
    q.add(0, 1, 5);
    q.add(1, 2, 4);
    q.add(0, 2, 10);
    assert(q.weight() == 9 && q.comps() == 2);
    q.add(0, 2, 1);
    q.add(2, 3, -2);
    q.add(0, 3, 6);
    q.add(3, 3, -7);
    assert(q.weight() == 3 && q.comps() == 1);
}
```

5.5.012 最小直径生成树

一般连通非负权无向图中，最小直径生成树不一定是 MST。最优树可以视为从某个绝对中心向各点选最短路：中心要么是顶点，要么位于一条原图边内部。先从每个顶点跑 Dijkstra 得全源最短路并按距离降序排列点；顶点中心的直径是两倍最大距离，边 uv 上的中心则扫描两端最远点归属变化的位置，候选直径由 $d(u,x)+w(u,v)+d(v,y)$ 给出。所有边权先乘二，使边内部的半整数中心仍能精确表示，最终恢复中心最短路树并把答案除二。

```
struct MinDiamTree
{
    struct Edge
    {
        int to; // to 是边的另一端。
        long long w; // w 是加倍后的边权。
    };

    struct State
    {
        long long d; // d 是当前距离。
        int u, fa; // u 是出堆点，fa 是它在最短路树上的前驱。

        bool operator>(const State &o) const
        {
            // o 是另一个状态；按 (d,u,fa) 字典序比较，供小根堆使用。
            return tie(d, u, fa) > tie(o.d, o.u, o.fa);
        }
    };

    int n; // n 是点数。
    vector<vector<Edge>> g; // g 是邻接表，边权已加倍以避免中点出现半整数。
    bool valid = true; // valid 为假表示加入过非法边或点数非正。
    __int128 totalWeight2 = 0; // totalWeight2 是全部加倍边权之和，用于检查溢出。

    MinDiamTree(int n_) : n(max(0LL, n_)), g(max(0LL, n_))
    {
        // n_ 是点数；构造空图，点数非正时标记为无效。
        valid = n_ > 0;
```

```
    }

    bool addEdge(int u, int v, long long w)
    {
        // u、v 是端点，w 是非负边权；合法时加入无向边并返回 true，否则标记无效并返回 false。
        if (u < 0 || u >= n || v < 0 || v >= n || w < 0 ||
            (__int128)w * 2 + totalWeight2 >= numeric_limits<long long>::max() / 4)
        {
            valid = false;
            return false;
        }
        totalWeight2 += (__int128)w * 2;
        g[u].push_back({v, w * 2});
        g[v].push_back({u, w * 2});
        return true;
    }

    pair<vector<long long>, vector<int>> dijkstra(int s, const vector<vector<Edge>> &a) const
    {
        // s 是源点，a 是邻接表；返回各点距离与最短路树前驱（源点前驱为自身，不可达为 -1）。
        int m = a.size();
        const long long INF = numeric_limits<long long>::max() / 4;
        vector<long long> dis(m, INF);
        vector<int> fa(m, -1);
        vector<char> vis(m);
        priority_queue<State, vector<State>, greater<State>> q;
        q.push({0, s, s});
        while (!q.empty())
        {
            auto [d, u, p] = q.top();
            q.pop();
            if (vis[u])
            {
                continue;
            }
            vis[u] = true;
            dis[u] = d;
            fa[u] = p;
            for (auto [v, w] : a[u])
            {
                if (!vis[v])
                {
                    if (d <= INF - w) q.push({d + w, v, u});
                }
            }
        }
        return {dis, fa};
    }

    optional<pair<long long, vector<pair<int, int>>>> solve() const
    {
        // 无参数；返回最小直径与一组生成树边，图非法或不连通时返回空。
```

```
        if (!valid || n == 0) return nullopt;
        const long long INF = numeric_limits<long long>::max() / 4;
        vector<vector<long long>> dis(n);
        vector<vector<int>> ord(n, vector<int>(n));
        long long bestP = INF;
        int center = 0;
        for (int s = 0; s < n; s++)
        {
            dis[s] = dijkstra(s, g).first;
            iota(ord[s].begin(), ord[s].end(), 0);
            stable_sort(ord[s].begin(), ord[s].end(), [&](int x, int y)
            {
                return dis[s][x] > dis[s][y];
            });
            if (dis[s][ord[s][0]] >= INF) return nullopt;
            long long cur = dis[s][ord[s][0]] * 2;
            if (cur < bestP)
            {
                bestP = cur;
                center = s;
            }
        }

        long long bestEdge = INF, left = 0, right = 0;
        int cu = 0, cv = 0;
        for (int u = 0; u < n; u++)
        {
            for (auto [v, w] : g[u])
            {
                if (v <= u)
                {
                    continue;
                }
                int y = ord[u][0];
                for (int x : ord[u])
                {
                    if (dis[v][x] > dis[v][y])
                    {
                        long long cur = dis[v][y] + dis[u][x] + w;
                        if (cur < bestEdge)
                        {
                            bestEdge = cur;
                            cu = u;
                            cv = v;
                            left = cur / 2 - dis[u][x];
                            right = w - left;
                        }
                        y = x;
                    }
                }
            }
        }
```

```

        vector<vector<Edge>> a = g;
        int root = center;
        bool inside = bestEdge < bestP;
        if (inside)
        {
            root = n;
            a.push_back({});
            a[root].push_back({cu, left});
            a[root].push_back({cv, right});
        }
        auto fa = dijkstra(root, a).second;
        vector<pair<int, int>> tree;
        if (inside)
        {
            for (int u = 0; u < n; u++)
            {
                if (u != cu && u != cv)
                {
                    tree.push_back({u, fa[u]});
                }
            }
            tree.push_back({cu, cv});
        }
        else
        {
            for (int u = 0; u < n; u++)
            {
                if (u != root)
                {
                    tree.push_back({u, fa[u]});
                }
            }
        }
        return pair<long long, vector<pair<int, int>>>{min(bestP, bestEdge) / 2, move(tree)};
    }
};
```

构造 `MinDiamTree q(n)`、加入全部无向边后调用 `q.solve()`；成功返回 `{最小直径, 树边端点对}`，非法边或非连通图返回 `nullopt`。边权非负，所有加倍边权之和须小于 `LLONG_MAX/4`。平行边只按端点恢复，输出原边编号时应在该端点对中取最轻边；自环不会改善答案。n 次 Dijkstra 加边中心扫描共 $O(nm\log n+nm)$ 时间、$O(n^2+m)$ 空间，适合 $n,m\le 2000$。只求最小权和仍用 5.5.001 Kruskal。

下面检查单点与三角形；三角形两条权为 1 的边构成直径为 2 的最优生成树：

```
signed main()
{
    MinDiamTree one(1);
    auto [d0, t0] = *one.solve();
    assert(d0 == 0 && t0.empty());

```

```
    MinDiamTree q(3);
    q.addEdge(0, 1, 1);
    q.addEdge(1, 2, 1);
    q.addEdge(0, 2, 3);
    auto [d, tree] = *q.solve();
    assert(d == 2 && tree.size() == 2);
}
```

5.5.013 Prim：稀疏图与稠密图

Prim 从一个点开始维护每个未选点到当前生成树的最轻连接边，每轮取连接代价最小的点并更新邻点。邻接表配优先队列适合稀疏图，邻接矩阵直接扫描适合边数接近 n^2 的稠密图；二者都保留父亲数组，避免只算权值却无法恢复树。负权无向边可以直接使用，图不连通时返回空。

```
struct PrimResult
{
    int sum;        // sum 保存最小生成树边权和。
    vector<int> fa; // fa[v] 保存 v 的父亲，根为 -1。
};

optional<PrimResult> primSparse(const vector<vector<pair<int, int>>> &g,
                                int s = 0)
{
    // g[u] 的每项是 {邻点, 边权}，s 是根；连通时返回 MST，否则返回空。
    int n = g.size();
    assert(n > 0 && 0 <= s && s < n); // 调试检查，可删
    vector<int> d(n, INF), fa(n, -1);
    vector<char> vis(n);
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> q;
    d[s] = 0;
    q.push({0, s});
    int sum = 0, cnt = 0;
    while (!q.empty())
    {
        auto [w, u] = q.top();
        q.pop();
        if (vis[u])
        {
            continue;
        }
        vis[u] = true;
        sum += w;
        cnt++;
        for (auto [v, c] : g[u])
        {
            if (!vis[v] && c < d[v])
            {
                d[v] = c;
                fa[v] = u;
                q.push({c, v});
```

```
            }
        }
    }
    if (cnt != n)
    {
        return nullopt;
    }
    return PrimResult{sum, move(fa)};
}

optional<PrimResult> primDense(const vector<vector<int>> &w, int s = 0)
{
    // w 是对称邻接矩阵，缺边填 INF，s 是根；连通时返回 MST，否则返回空。
    int n = w.size();
    assert(n > 0 && 0 <= s && s < n); // 调试检查，可删
    for (const auto &row : w)
    {
        assert((int)row.size() == n); // 调试检查，可删
    }
    vector<int> d(n, INF), fa(n, -1);
    vector<char> vis(n);
    d[s] = 0;
    int sum = 0;
    for (int t = 0; t < n; t++)
    {
        int u = -1;
        for (int v = 0; v < n; v++)
        {
            if (!vis[v] && (u < 0 || d[v] < d[u]))
            {
                u = v;
            }
        }
        if (u < 0 || d[u] == INF)
        {
            return nullopt;
        }
        vis[u] = true;
        sum += d[u];
        for (int v = 0; v < n; v++)
        {
            if (!vis[v] && w[u][v] < d[v])
            {
                d[v] = w[u][v];
                fa[v] = u;
            }
        }
    }
    return PrimResult{sum, move(fa)};
}
```

`primSparse(g,s)` 的时间为 $O((n+m)\log n)$、空间为 $O(n+m)$；`primDense(w,s)` 的时间为 $O(n^2)$、

除矩阵外空间为 $O(n)$。两版都要求无向边在邻接表或矩阵中对称出现，权值和须装入 64 位；只已有边表并需要排序时 5.5.001 Kruskal 通常更短，需要在线维护森林时用 5.5.011 增量最小生成森林。

```
signed main()
{
    vector<vector<pair<int, int>>> g(4);
    auto add = [&](int u, int v, int w)
    {
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    };
    add(0, 1, 1), add(1, 2, 2), add(2, 3, 3), add(0, 3, 9);
    assert(primSparse(g)->sum == 6);
}
```

5.5.014 Borůvka

Borůvka 算法维护当前最小生成森林，并在每一轮为每个连通块选择一条权值最小的出边。割性质保证这些边都可以安全加入某棵最小生成树；合并所有仍连接不同块的候选边后，连通块数至少减半，因此最多进行 $O(\log n)$ 轮。它不需要预排序，适合边按批次扫描、并行寻找各块最轻边，或作为其他最小生成树算法的收缩阶段；普通一次性边表通常仍优先使用更短的 Kruskal。

以下实现显式依赖 5.5.001 Kruskal 已定义的 `DSU` 与 `MEdge`，不重复实现并查集和边记录：

```
optional<pair<int, vector<MEdge>>> boruvka(int n, const vector<MEdge> &e)
{
    // n 是点数，e 是无向边集；返回 MST 总权与所选边，不连通时返回空。
    assert(n >= 0); // 调试检查，可删
    DSU d(n);
    int cnt = n, ans = 0;
    vector<MEdge> use;
    while (cnt > 1)
    {
        vector<int> best(n, -1);
        for (int i = 0; i < (int)e.size(); i++)
        {
            int x = d.find(e[i].x), y = d.find(e[i].y);
            if (x == y)
            {
                continue;
            }
            // 求最大生成树时，下面两处 < 要同时改成 >，其余逻辑不变。
            if (best[x] == -1 || e[i].w < e[best[x]].w)
            {
                best[x] = i;
            }
            if (best[y] == -1 || e[i].w < e[best[y]].w)
            {
                best[y] = i;
            }
```

```
        }
        bool upd = false;
        for (int x = 0; x < n; x++)
        {
            if (d.find(x) != x || best[x] == -1)
            {
                continue;
            }
            auto v = e[best[x]];
            if (d.merge(v.x, v.y))
            {
                ans += v.w;
                use.push_back(v);
                cnt--;
                upd = true;
            }
        }
        if (!upd)
        {
            return nullopt;
        }
    }
    return pair{ans, use};
}
```

调用 `boruvka(n,e)`，成功时返回最小生成树总权和按加入顺序排列的 $n-1$ 条边，图不连通时返回空；$n=0$ 或 $n=1$ 时返回权和 0 的空树。输入是无向边表，每条边只需出现一次，允许负权、重边和自环，端点必须落在 $[0,n)$，总权须能由 64 位 `int` 表示。每轮扫描全部 m 条边并做并查集操作，共 $O(m\log n\,\alpha(n))$ 时间、$O(n)$ 额外空间。求最大生成树时只需把代码中两处候选比较的 `<` 同时改为 `>`，无穷单位元和其余分支都不变；只改一处会破坏各连通块选择方向的一致性。已有边表且排序方便时使用 5.5.001 Kruskal，已有邻接表或矩阵时使用 5.5.013 Prim。

下面检查负权、重边和生成树恢复：

```
signed main()
{
    vector<MEdge> e{{0, 1, 4}, {1, 2, -2}, {2, 3, 5},
                    {0, 3, 10}, {0, 2, 1}, {0, 2, 8}};
    auto ans = boruvka(4, e);
    assert(ans.has_value());
    assert(ans->first == 4 && ans->second.size() == 3);
}
```

5.6 二分图匹配与覆盖

5.6.001 匈牙利算法

匈牙利算法求二分图最大基数匹配。核心是增广路：一条从未匹配左点出发、交替走「非匹配边、匹配边」并终止于未匹配右点的路径，把它上面的匹配 / 非匹配状态整体翻转，匹配数就增加 1。Berge 定理保证

当且仅当不存在增广路时匹配最大。实现逐个未匹配左点做一次 DFS 找增广路，用时间戳 `vis[v] == ti` 代替每轮清空访问标记（省去 $O(V)$ 重置）。代码短、常数小，适合中小稀疏图；稠密或大图应换下面的位运算版或 Hopcroft-Karp。

```
class Hungarian
{
    // nl、nr 是左右部点数，g[u] 保存左点 u 的全部邻点。
    int nl, nr;
    vector<vector<int>> g;
    // ml[u]、mr[v] 分别保存左右点当前匹配点，-1 表示未匹配。
    vector<int> ml, mr;
    // vis[v] 是本轮增广是否已经访问右点 v 的时间戳。
    vector<int> vis;
    int ti = 0;

    bool dfs(int u)
    {
        // u 是尝试增广的左点；找到增广路并改写匹配时返回 true。
        for (auto v : g[u])
        {
            if (vis[v] == ti)
            {
                continue;
            }
            vis[v] = ti;
            if (mr[v] == -1 || dfs(mr[v]))
            {
                ml[u] = v;
                mr[v] = u;
                return true;
            }
        }
        return false;
    }

  public:
    Hungarian(int nl, int nr) : nl(nl), nr(nr), g(nl), ml(nl, -1), mr(nr, -1), vis(nr)
    {
        // nl、nr 是左右部点数；构造空二分图，无返回值。
    }

    void addEdge(int u, int v)
    {
        // u 是左点，v 是右点；加入一条二分图边，无返回值。
        assert(0 <= u && u < nl && 0 <= v && v < nr); // 调试检查，可删。
        g[u].push_back(v);
    }

    int matching()
    {
        // 无参数；从空匹配重算并返回最大匹配大小。
        fill(ml.begin(), ml.end(), -1);
```

```
        fill(mr.begin(), mr.end(), -1);
        fill(vis.begin(), vis.end(), 0);
        ti = 0;
        for (int u = 0; u < nl; u++)
        {
            if (ml[u] == -1)
            {
                ti++;
                dfs(u);
            }
        }
        return count_if(ml.begin(),
                        ml.end(),
                        [](int v)
                        {
                            return v != -1;
                        });
    }

    const vector<int> &leftMatch() const
    {
        // 无参数；返回每个左点匹配的右点，未匹配为 -1。
        return ml;
    }

    const vector<int> &matchR() const
    {
        // 无参数；返回每个右点匹配的左点，未匹配为 -1。
        return mr;
    }
};
```

构造 `Hungarian q(nl,nr)` 后加边，`matching()` 每次从空匹配重算，之后用 `leftMatch()`、`matchR()` 读取互逆匹配数组。最坏用 $O(VE)$ 时间、$O(V+E)$ 空间。稠密图见 5.6.002 Bitset 优化匈牙利，大规模稀疏图见 5.6.003 Hopcroft-Karp。

5.6.002 Bitset 优化匈牙利

仍是 5.6.001 匈牙利算法 的增广路思想，只是把「遍历左点 u 的邻居中尚未访问者」用位运算加速：邻接集合按右点编号压进若干个 64 位块，本轮访问集合 `vis` 同样是位集，`g[u][b] & ~vis[b]` 一次取出该块里所有「相邻且未访问」的右点，`countr_zero` 逐位弹出。这样在稠密二分图上把内层邻居扫描的常数压到 $1/64$。匹配正确性完全来自增广路翻转，与朴素版一致。

```
class BitHungarian
{
    // nl、nr 是左右部点数，w 是存放 nr 位所需的 64 位块数。
    int nl, nr, w;
    // g[u][b] 的第 k 位表示左点 u 是否连接右点 64b+k。
    vector<vector<unsigned long long>> g;
```

```
    // ml、mr 是左右匹配点，vis 是一轮增广中访问过的右点位集。
    vector<int> ml, mr;
    vector<unsigned long long> vis;

    bool dfs(int u)
    {
        // u 是尝试增广的左点；找到增广路并改写匹配时返回 true。
        for (int b = 0; b < w; b++)
        {
            auto x = g[u][b] & ~vis[b];
            while (x)
            {
                int k = countr_zero(x);
                x &= x - 1;
                int v = b * 64 + k;
                if (v >= nr)
                {
                    continue;
                }
                vis[b] |= 1ULL << k;
                if (mr[v] == -1 || dfs(mr[v]))
                {
                    ml[u] = v;
                    mr[v] = u;
                    return true;
                }
                x &= ~vis[b];
            }
        }
        return false;
    }

public:
    BitHungarian(int nl, int nr)
        : nl(nl), nr(nr), w((nr + 63) / 64), g(nl), ml(nl, -1), mr(nr, -1), vis(w)
    {
        // nl、nr 是左右部点数；构造位集邻接矩阵，无返回值。
        for (auto &row : g)
        {
            row.assign(w, 0);
        }
    }

    void addEdge(int u, int v)
    {
        // u 是左点，v 是右点；把对应邻接位设为 1，无返回值。
        assert(0 <= u && u < nl && 0 <= v && v < nr); // 调试检查，可删。
        g[u][v / 64] |= 1ULL << (v % 64);
    }

    int matching()
    {
```

```
        // 无参数；从空匹配重算并返回最大匹配边数。
        fill(ml.begin(), ml.end(), -1);
        fill(mr.begin(), mr.end(), -1);
        int ans = 0;
        for (int u = 0; u < nl; u++)
        {
            fill(vis.begin(), vis.end(), 0);
            ans += dfs(u);
        }
        return ans;
    }

    const vector<int> &leftMatch() const
    {
        // 无参数；返回每个左点匹配的右点，未匹配为 -1。
        return ml;
    }

    const vector<int> &matchR() const
    {
        // 无参数；返回每个右点匹配的左点，未匹配为 -1。
        return mr;
    }
};
```

接口与 5.6.001 匈牙利算法 相同（`addEdge` 写位、`matching()` 返回最大匹配数），适合右部不太大且边稠密的图；右部极大而边稀疏时位集反而浪费内存，用普通邻接表版更省。保守最坏 $O(n_l^2\lceil n_r/64\rceil + n_l E)$ 时间、$O(n_l\lceil n_r/64\rceil + n_r)$ 空间。要更好的最坏渐进界用 5.6.003 Hopcroft-Karp。

5.6.003 Hopcroft-Karp

Hopcroft-Karp 把匈牙利的「逐条增广」升级成「逐层批量增广」。每轮先做一次 BFS，从所有未匹配左点出发按交替图给左点分层 `dep`；再用受层次约束的 DFS 一次找出一批点不相交的增广路同时翻转。这里完整展开可达层，并允许 DFS 在任意层接到未匹配右点；仍只沿层数递增的匹配边前进，故每轮得到极大增广路集。`dfs` 失败时把 `dep[u]` 置 -1 剪枝，`it` 记录本轮当前弧，避免重复扫描已经失败的边。邻接点与热点状态使用固定 32 位整数，不受公共前导的 `#define int long long` 放大；BFS 队列也预分配并复用。下面先给适合随时加边的邻接表版。

```
class HopcroftKarp
{
    // nl、nr 是左右部点数，g 是从左部出发的邻接表。
    int nl, nr;
    vector<vector<int>> g;
    // ml、mr 是左右匹配点；dep、it、que 分别是层数、当前弧和复用队列。
    vector<int> ml, mr, dep, it, que;

    bool bfs()
    {
        // 无参数；给交替图分层，存在增广路时返回 true。
```

```
        fill(dep.begin(), dep.end(), -1);
        int ql = 0, qr = 0;
        bool found = false;
        for (int u = 0; u < nl; u++)
        {
            if (ml[u] == -1)
            {
                dep[u] = 0;
                que[qr++] = u;
            }
        }
        while (ql < qr)
        {
            int u = que[ql++];
            for (int v : g[u])
            {
                int x = mr[v];
                if (x == -1)
                {
                    found = true;
                }
                else if (dep[x] == -1)
                {
                    dep[x] = dep[u] + 1;
                    que[qr++] = x;
                }
            }
        }
        return found;
    }

    bool dfs(int u)
    {
        // u 是当前左点；沿 BFS 层寻找增广路，成功时返回 true。
        for (int &i = it[u]; i < (int)g[u].size(); i++)
        {
            int v = g[u][i];
            int x = mr[v];
            if (x == -1)
            {
                ml[u] = v;
                mr[v] = u;
                return true;
            }
            if (x != -1 && dep[x] == dep[u] + 1 && dfs(x))
            {
                ml[u] = v;
                mr[v] = u;
                return true;
            }
        }
        dep[u] = -1;
```

```
        return false;
    }

public:
    HopcroftKarp(int nl, int nr)
        : nl(nl), nr(nr), g(nl), ml(nl, -1), mr(nr, -1), dep(nl), it(nl), que(nl)
    {
        // nl、nr 是左右部点数；构造空二分图，无返回值。
        assert(nl <= numeric_limits<int>::max() && nr <= numeric_limits<int>::max()); // 调试检
        ↪ 查，可删。
    }

    void reserve(int u, int m)
    {
        // u 是左点，m 是预计邻边数；预留空间但不改变图。
        assert(0 <= u && u < nl && m >= 0); // 调试检查，可删。
        g[u].reserve(m);
    }

    void addEdge(int u, int v)
    {
        // u 是左点，v 是右点；加入一条边，无返回值。
        assert(0 <= u && u < nl && 0 <= v && v < nr); // 调试检查，可删。
        g[u].push_back((int)v);
    }

    int matching()
    {
        // 无参数；从空匹配重算并返回最大匹配边数。
        fill(ml.begin(), ml.end(), -1);
        fill(mr.begin(), mr.end(), -1);
        int ans = 0;
        while (bfs())
        {
            fill(it.begin(), it.end(), 0);
            for (int u = 0; u < nl; u++)
            {
                if (ml[u] == -1)
                {
                    ans += dfs(u);
                }
            }
        }
        return ans;
    }

    vector<int> leftMatch() const
    {
        // 无参数；返回左点到右点的匹配数组，未匹配为 -1。
        return {ml.begin(), ml.end()};
    }

```

```
    vector<int> matchR() const
    {
        // 无参数；返回右点到左点的匹配数组，未匹配为 -1。
        return {mr.begin(), mr.end()};
    }
};
```

构造 `HopcroftKarp q(nl, nr)`，加边后 `matching()` 返回最大匹配数，`leftMatch()` / `matchR()` 取匹配（-1 未匹配）；点从 0 编号。已知左点 u 的邻边数时可先调用 `reserve(u,m)` 避免扩容；因内部使用 32 位编号，左右部点数都不能超过 `numeric_limits<int>::max()`。匹配数组接口按公共 `int` 类型返回一份副本。$O(E\sqrt{V})$ 时间、$O(V+E)$ 空间。它允许随时向各左点的邻接表追加边，接口最直接；若边在求解前一次给齐且更看重连续存储，改用 5.6.004 CSR Hopcroft-Karp。

5.6.004 CSR Hopcroft-Karp

CSR 版仍做完全相同的 BFS 分层和 DFS 批量增广，只把每个左点各自持有的邻接表压成连续的 `to` 数组。`start[u]` 与 `start[u+1]` 圈出左点 u 的邻边区间，免去 n_l 次独立分配并改善顺序扫描的局部性；代价是求解前必须按当前边集重建 CSR，不适合频繁加边、求解、再加边的交替流程。它是确定性实现，不增加失败概率，也不依赖平台指令。

```
// 依赖 5.6.001 匈牙利算法
// 依赖 5.6.002 Bitset 优化匈牙利
// 依赖 5.6.003 Hopcroft-Karp
class HopKarpCSR
{
    int nl, nr; // nl、nr 是左右部点数。
    // edge[i][0]、edge[i][1] 是第 i 条边的左右端点。
    vector<array<int, 2>> edge;
    // 左点 u 的邻边存放在 to[start[u]..start[u+1])。
    vector<int> start, to;
    // ml、mr 是左右匹配点，dep 是左点层数，lim 是本轮最短增广路长度。
    vector<int> ml, mr, dep;
    int lim = -1;

    void build()
    {
        // 无参数；按当前边集重建 CSR 邻接表，无返回值。
        start.assign(nl + 1, 0);
        for (auto e : edge)
        {
            start[e[0] + 1]++;
        }
        partial_sum(start.begin(), start.end(), start.begin());
        vector<int> cur = start;
        to.assign(edge.size(), 0);
        for (auto e : edge)
        {
            to[cur[e[0]]++] = e[1];
        }
```

```
    }

    bool bfs()
    {
        // 无参数；给全部最短增广路分层，存在增广路时返回 true。
        queue<int> q;
        fill(dep.begin(), dep.end(), -1);
        lim = -1;
        for (int u = 0; u < nl; u++)
        {
            if (ml[u] == -1)
            {
                dep[u] = 0;
                q.push(u);
            }
        }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            if (lim != -1 && dep[u] >= lim)
            {
                continue;
            }
            for (int i = start[u]; i < start[u + 1]; i++)
            {
                int x = mr[to[i]];
                if (x == -1)
                {
                    lim = dep[u] + 1;
                }
                else if (dep[x] == -1 && (lim == -1 || dep[u] + 1 < lim))
                {
                    dep[x] = dep[u] + 1;
                    q.push(x);
                }
            }
        }
        return lim != -1;
    }

    bool dfs(int u)
    {
        // u 是当前左点；沿 BFS 层寻找最短增广路，成功时返回 true。
        for (int i = start[u]; i < start[u + 1]; i++)
        {
            int v = to[i];
            int x = mr[v];
            if (x == -1 && dep[u] + 1 == lim)
            {
                ml[u] = v;
                mr[v] = u;
```

```
                return true;
            }
            if (x != -1 && dep[x] == dep[u] + 1 && dfs(x))
            {
                ml[u] = v;
                mr[v] = u;
                return true;
            }
        }
        dep[u] = -1;
        return false;
    }

public:
    HopKarpCSR(int nl, int nr) : nl(nl), nr(nr), ml(nl, -1), mr(nr, -1), dep(nl)
    {
        // nl、nr 是左右部点数；构造空边集，无返回值。
    }

    void addEdge(int u, int v)
    {
        // u 是左点，v 是右点；暂存一条边，无返回值。
        assert(0 <= u && u < nl && 0 <= v && v < nr); // 调试检查，可删。
        edge.push_back({u, v});
    }

    int matching()
    {
        // 无参数；重建 CSR，并从空匹配重算最大匹配边数。
        build();
        fill(ml.begin(), ml.end(), -1);
        fill(mr.begin(), mr.end(), -1);
        int ans = 0;
        while (bfs())
        {
            for (int u = 0; u < nl; u++)
            {
                if (ml[u] == -1)
                {
                    ans += dfs(u);
                }
            }
        }
        return ans;
    }

    const vector<int> &leftMatch() const
    {
        // 无参数；返回左点到右点的匹配数组，未匹配为 -1。
        return ml;
    }

```

```
    const vector<int> &matchR() const
    {
        // 无参数；返回右点到左点的匹配数组，未匹配为 -1。
        return mr;
    }
};
```

CSR 版与 5.6.003 Hopcroft-Karp 的正确性、$O(E\sqrt{V})$ 时间和 $O(V+E)$ 峰值空间界完全相同；它少做小 `vector` 分配，但每次 `matching()` 多做一次 $O(V+E)$ 建表。大规模静态稀疏图可优先试 CSR，动态图式调用仍用邻接表版；小规模或代码从简用 5.6.001 匈牙利算法，稠密图用 5.6.002 Bitset 优化匈牙利，带权最优匹配见 5.6.005 二分图最大权 / 最小权匹配（KM）。下面每个实现分别在一张已知答案为 3 的图上检查，不互相比较内部状态。

```
template <class F>
void testMatching()
{
    // F 是待检查的匹配结构；构造存在大小为 3 的完美匹配的图，无返回值。
    F g(3, 3);
    for (auto [u, v] : vector<array<int, 2>>{{0, 0}, {0, 1}, {1, 1}, {1, 2}, {2, 0}})
    {
        g.addEdge(u, v);
    }
    assert(g.matching() == 3);
}

signed main()
{
    testMatching<Hungarian>();
    testMatching<BitHungarian>();
    testMatching<HopcroftKarp>();
    testMatching<HopKarpCSR>();
    return 0;
}
```

5.6.005 二分图最大权/最小权匹配（KM）

带权二分图完备匹配（指派问题）要在 n 行 m 列（$n \le m$）的权值矩阵里给每行选一个不同的列，使总权最优。这里用最短增广路形式的匈牙利指派实现：为左点 `u` 和右点 `v` 维护可行顶标，保持任意边的归约代价 $c(i,j)-u_i-v_j \ge 0$；每次从一个新左点出发在「相等子图」（归约代价为 0 的边）里用最短增广路 + `way` 记录前驱地扩展，找不到增广时按未访问列的最小松弛量 `delta` 整体调整顶标，使至少一条新边进入相等子图，直到增广成功。求最大权时对权取负即可（`mx` 参数），故一份代码兼顾最大 / 最小。

```
template<class Acc = long long>
struct Assignment
{
    // cost 是最优匹配总权。
    Acc cost;
    // col[i] 是第 i 行匹配的列编号。
    vector<int> col;
```

```cpp
};

template<class Acc = long long>
optional<Assignment<Acc>> assignment(const vector<vector<long long>> &a, bool mx = false)
{
    // a 是权值矩阵，mx 表示求最大权；返回最优完备匹配，不存在时返回空。
    static_assert(numeric_limits<Acc>::is_integer && numeric_limits<Acc>::is_signed &&
                  sizeof(Acc) >= sizeof(long long));
    int n = a.size(), m = n ? a[0].size() : 0;
    for (const auto &row : a)
    {
        if ((int)row.size() != m)
        {
            return nullopt;
        }
    }
    if (n > m)
    {
        return nullopt;
    }
    auto add = []<class T>(T x, T y) -> optional<T>
    {
        // x、y 是同类型整数；和超出该类型范围时返回空。
        if ((y > 0 && x > numeric_limits<T>::max() - y) ||
            (y < 0 && x < numeric_limits<T>::min() - y))
        {
            return nullopt;
        }
        return x + y;
    };
    auto sub = []<class T>(T x, T y) -> optional<T>
    {
        // x、y 是同类型整数；差超出该类型范围时返回空。
        if ((y > 0 && x < numeric_limits<T>::min() + y) ||
            (y < 0 && x > numeric_limits<T>::max() + y))
        {
            return nullopt;
        }
        return x - y;
    };
    vector<Acc> u(n + 1), v(m + 1);
    vector<int> p(m + 1), way(m + 1);
    for (int i = 1; i <= n; i++)
    {
        p[0] = i;
        int j0 = 0;
        vector<optional<Acc>> d(m + 1);
        vector<bool> used(m + 1);
        do
        {
            used[j0] = true;
            int i0 = p[j0], j1 = 0;
```

```
            optional<Acc> delta;
            for (int j = 1; j <= m; j++)
            {
                if (used[j])
                {
                    continue;
                }
                Acc cur = (Acc)a[i0 - 1][j - 1];
                if (mx)
                {
                    auto z = sub((Acc)0, cur);
                    if (!z)
                    {
                        return nullopt;
                    }
                    cur = *z;
                }
                auto z = sub(cur, u[i0]);
                if (!z)
                {
                    return nullopt;
                }
                z = sub(*z, v[j]);
                if (!z)
                {
                    return nullopt;
                }
                cur = *z;
                if (!d[j] || cur < *d[j])
                {
                    d[j] = cur;
                    way[j] = j0;
                }
                if (!delta || *d[j] < *delta)
                {
                    delta = d[j];
                    j1 = j;
                }
            }
            assert(delta); // 调试检查，可删：n<=m 时总有未访问列。
            for (int j = 0; j <= m; j++)
            {
                if (used[j])
                {
                    auto x = add(u[p[j]], *delta);
                    if (!x)
                    {
                        return nullopt;
                    }
                    u[p[j]] = *x;
                    if (j)
                    {
```

```
                        x = sub(v[j], *delta);
                        if (!x)
                        {
                            return nullopt;
                        }
                        v[j] = *x;
                    }
                }
            }
            for (int j = 1; j <= m; j++)
            {
                if (!used[j] && d[j])
                {
                    auto x = sub(*d[j], *delta);
                    if (!x)
                    {
                        return nullopt;
                    }
                    d[j] = *x;
                }
            }
            j0 = j1;
        } while (p[j0]);
        do
        {
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;
        } while (j0);
    }
    vector<int> col(n);
    for (int j = 1; j <= m; j++)
    {
        if (p[j])
        {
            col[p[j] - 1] = j - 1;
        }
    }
    Acc cost = 0;
    for (int i = 0; i < n; i++)
    {
        auto x = add(cost, (Acc)a[i][col[i]]);
        if (!x)
        {
            return nullopt;
        }
        cost = *x;
    }
    return Assignment<Acc>{cost, col};
}
```

`assignment` 的 `Acc` 默认是 `long long`，必须是至少与输入同宽的有符号整数；`mx=true` 求最大权，否则

求最小权。矩阵的每个位置都是有效边，不能用极值表示缺边。返回的 `cost` 为 `Acc`，锯齿矩阵、行数大于列数或中间值越界时返回空；顶标或总权可能超过 64 位时可用 `assignment<__int128>(a,mx)`。算法用 $O(n^2m)$ 时间、$O(n+m)$ 额外空间。

```
signed main()
{
    vector<vector<long long>> a{{4, 1, 3, 8},
                                {2, 0, 5, 7},
                                {3, 2, 2, 6}};
    assert(assignment(a, false)->cost == 5);
    assert(assignment(a, true)->cost == 16);
}
```

5.6.006 二分图边染色

König 边染色定理：二分图的边色数恰等于最大度 Δ。构造性证明也给出算法：把两侧补齐成同样多的点、必要时添加虚拟边使每个点度数都恰为 Δ，得到一个 Δ-正则二分多重图；正则二分图必有完美匹配，取出一个完美匹配当作一种颜色、从图中删去，剩下的是 $(\Delta-1)$-正则图，递归 Δ 次即得恰 Δ 种颜色的合法边染色。实现里每轮用 Hopcroft-Karp 式的分层增广求一个覆盖所有点的完美匹配，真实边记颜色、虚拟边（`id == -1`）丢弃。

```
struct EdgeColor
{
    // 颜色数。
    int k;
    // c[i] 是第 i 条原边的颜色。
    vector<int> c;
};

struct BipEdgeColor
{
    struct E
    {
        // 左端点、右端点、原边编号；虚拟边编号为 -1。
        int u, v, id;
    };

    // 左、右部点数。
    int n, m;
    // 原图边集。
    vector<E> e;

    BipEdgeColor(int n, int m) : n(n), m(m)
    {
        // n、m 是左右部点数；构造空二分图。
    }

    void add(int u, int v)
    {
```

```cpp
        // u、v 是左右端点；按加入顺序给边编号。
        assert(0 <= u && u < n && 0 <= v && v < m); // 调试检查，可删。
        e.push_back({u, v, (int)e.size()});
    }

    EdgeColor work() const
    {
        // 返回最少颜色数及每条原边的颜色；先合并低度点，避免孤立点使补图膨胀。
        vector<int> dl(n), dr(m);
        int k = 0;
        for (auto x : e)
        {
            k = max(k, ++dl[x.u]);
            k = max(k, ++dr[x.v]);
        }
        if (!k) return {0, {}};
        auto compress = [&](const vector<int> &deg) -> pair<vector<int>, int>
        {
            // deg 是一侧原点度数；返回原点到合并点的映射及合并后点数。
            int sz = deg.size();
            vector<int> fa(sz, -1);
            auto find = [&](auto &&self, int u) -> int
            {
                // u 是并查集节点；返回所在集合代表。
                return fa[u] < 0 ? u : fa[u] = self(self, fa[u]);
            };
            priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> q;
            for (int i = 0; i < sz; i++)
            {
                q.push({deg[i], i});
            }
            while (q.size() > 1)
            {
                auto [x, u] = q.top(); q.pop();
                auto [y, v] = q.top(); q.pop();
                if (x + y > k)
                {
                    q.push({x, u});
                    q.push({y, v});
                    break;
                }
                if (fa[u] > fa[v]) swap(u, v);
                fa[u] += fa[v];
                fa[v] = u;
                q.push({x + y, u});
            }
            vector<int> id(sz), pos(sz, -1);
            int cnt = 0;
            for (int i = 0; i < sz; i++)
            {
                int u = find(find, i);
                if (pos[u] == -1) pos[u] = cnt++;
```

```
                id[i] = pos[u];
            }
            return {id, cnt};
        };
        auto [pl, nl] = compress(dl);
        auto [pr, nr] = compress(dr);
        int z = max(nl, nr);
        vector<E> es;
        es.reserve(2 * e.size() + 2 * z);
        vector<int> cl(z), cr(z);
        for (auto x : e)
        {
            x.u = pl[x.u];
            x.v = pr[x.v];
            es.push_back(x);
            cl[x.u]++;
            cr[x.v]++;
        }
        int u = 0, v = 0;
        while (u < z && v < z)
        {
            while (u < z && cl[u] == k)
            {
                u++;
            }
            while (v < z && cr[v] == k)
            {
                v++;
            }
            if (u == z || v == z) break;
            int cnt = min(k - cl[u], k - cr[v]);
            for (int i = 0; i < cnt; i++)
            {
                es.push_back({u, v, -1});
            }
            cl[u] += cnt;
            cr[v] += cnt;
        }
        vector<int> ans(e.size());
        auto perfect = [&](const vector<int> &ids) -> vector<int>
        {
            // ids 是正则二分图边编号；返回一组覆盖全部合并点的完美匹配边。
            vector<vector<int>> g(z);
            for (int id : ids)
            {
                g[es[id].u].push_back(id);
            }
            vector<int> ml(z, -1), mr(z, -1), dep(z), it(z);
            auto bfs = [&]() -> bool
            {
                // 无参数；给最短增广路分层，存在时返回 true。
                queue<int> q;
```

```
                fill(dep.begin(), dep.end(), -1);
                int lim = -1;
                for (int x = 0; x < z; x++)
                {
                    if (ml[x] == -1) dep[x] = 0, q.push(x);
                }
                while (!q.empty())
                {
                    int x = q.front(); q.pop();
                    if (lim != -1 && dep[x] >= lim) continue;
                    for (int id : g[x])
                    {
                        int y = mr[es[id].v];
                        if (y == -1) lim = dep[x] + 1;
                        else if (dep[y] == -1 && (lim == -1 || dep[x] + 1 < lim)) dep[y] =
                            ↪ dep[x] + 1, q.push(y);
                    }
                }
                return lim != -1;
            };
            auto dfs = [&](auto &&self, int x) -> bool
            {
                // x 是当前左点；沿分层交替路增广，成功时返回 true。
                for (int &i = it[x]; i < (int)g[x].size(); i++)
                {
                    int id = g[x][i], y = mr[es[id].v];
                    if (y == -1 || (dep[y] == dep[x] + 1 && self(self, y)))
                    {
                        ml[x] = id;
                        mr[es[id].v] = x;
                        return true;
                    }
                }
                dep[x] = -1;
                return false;
            };
            int got = 0;
            while (bfs())
            {
                fill(it.begin(), it.end(), 0);
                for (int x = 0; x < z; x++)
                {
                    if (ml[x] == -1 && dfs(dfs, x)) got++;
                }
            }
            assert(got == z); // 调试检查，可删：正则二分图总有完美匹配。
            return ml;
        };
        auto color = [&](auto &&self, vector<int> ids, int d, int first) -> void
        {
            // ids 是 d 正则图边集，d 是度数，first 是该集合第一种可用颜色。
            if (!d) return;
```

```
            if (d == 1)
            {
                for (int id : ids)
                {
                    if (es[id].id != -1) ans[es[id].id] = first;
                }
                return;
            }
            if (d & 1)
            {
                auto take = perfect(ids);
                vector<char> used(es.size());
                for (int id : take)
                {
                    used[id] = true;
                    if (es[id].id != -1) ans[es[id].id] = first;
                }
                vector<int> rest;
                rest.reserve(ids.size() - take.size());
                for (int id : ids)
                {
                    if (!used[id]) rest.push_back(id);
                }
                self(self, move(rest), d - 1, first + 1);
                return;
            }
            vector<vector<pair<int, int>>> g(2 * z);
            for (int id : ids)
            {
                int a = es[id].u, b = z + es[id].v;
                g[a].push_back({b, id});
                g[b].push_back({a, id});
            }
            vector<char> used(es.size());
            vector<int> ptr(2 * z), a, b;
            for (int s = 0; s < 2 * z; s++)
            {
                while (ptr[s] < (int)g[s].size() && used[g[s][ptr[s]].second])
                {
                    ptr[s]++;
                }
                if (ptr[s] == (int)g[s].size()) continue;
                vector<pair<int, int>> stk{{s, -1}}, tour;
                while (!stk.empty())
                {
                    int x = stk.back().first;
                    while (ptr[x] < (int)g[x].size() && used[g[x][ptr[x]].second])
                    {
                        ptr[x]++;
                    }
                    if (ptr[x] == (int)g[x].size())
                    {
```

```
                        int id = stk.back().second;
                        stk.pop_back();
                        if (id != -1) tour.push_back({0, id});
                    }
                    else
                    {
                        auto [y, id] = g[x][ptr[x]++];
                        if (!used[id]) used[id] = true, stk.push_back({y, id});
                    }
                }
                for (int i = 0; i < (int)tour.size(); i++)
                {
                    (i & 1 ? b : a).push_back(tour[i].second);
                }
            }
            self(self, move(a), d / 2, first);
            self(self, move(b), d / 2, first + d / 2);
        };
        vector<int> ids(es.size());
        iota(ids.begin(), ids.end(), 0);
        color(color, move(ids), k, 0);
        return {k, ans};
    }
};
```

构造 `BipEdgeColor g(n, m)`（左、右部点数），`add(u, v)` 加边（按加入顺序隐式编号），`work()` 返回 `EdgeColor`：`k` 是颜色数（即 Δ），`c[i]` 是第 i 条原边的颜色（0 到 $k-1$）。先把同侧度数和不超过 Δ 的点合并，故参与补齐的合并点和边数都是 $O(E)$；再把图补成 Δ 正则图，奇数度时取一组完美匹配、偶数度时沿欧拉回路二分。设 E 为原边数，Hopcroft-Karp 版本的完美匹配分解总时间为 $O(E\sqrt{E}\log\Delta)$、额外空间为 $O(E)$，不会因大量孤立点膨胀。它反复调用二分图完美匹配，底层增广与 5.6.003 Hopcroft-Karp 同构；一般（非二分）图的边染色可能需要 $\Delta+1$ 色（Vizing 定理），不能套用本条的 Δ 色结论。

5.6.007 最小点覆盖、最小边覆盖

König 定理：二分图最小点覆盖大小等于最大匹配数。要还原一组具体的最小点覆盖，先求一份最大匹配，再从所有未匹配左点出发沿「未匹配边向右、匹配边向左」交替搜索，记可达左点集 Z_L、可达右点集 Z_R；则最小点覆盖恰为 $(L\setminus Z_L)\cup Z_R$。最小边覆盖则由 Gallai 定理给出：无孤立点时其大小为 $|V|-$ 最大匹配，构造上先取匹配边、再给每个未被覆盖的点补一条任意关联边。任一侧存在孤立点时最小边覆盖不存在。

```
pair<vector<int>, vector<int>>
minVtxCover(const vector<vector<int>> &g, const vector<int> &ml, const vector<int> &mr)
{
    // g 是左部邻接表，ml、mr 是一份最大匹配；返回最小点覆盖中的左点表与右点表。
    vector<bool> vl(g.size()), vr(mr.size());
    queue<int> q;
    for (int u = 0; u < (int)g.size(); u++)
    {
        if (ml[u] == -1)
```

```
        {
            vl[u] = true;
            q.push(u);
        }
    }
    while (!q.empty())
    {
        int u = q.front();
        q.pop();
        for (auto v : g[u])
        {
            if (ml[u] == v || vr[v])
            {
                continue;
            }
            vr[v] = true;
            if (mr[v] != -1 && !vl[mr[v]])
            {
                vl[mr[v]] = true;
                q.push(mr[v]);
            }
        }
    }
    vector<int> l, r;
    for (int u = 0; u < (int)g.size(); u++)
    {
        if (!vl[u])
        {
            l.push_back(u);
        }
    }
    for (int v = 0; v < (int)mr.size(); v++)
    {
        if (vr[v])
        {
            r.push_back(v);
        }
    }
    return {l, r};
}

optional<vector<array<int, 2>>> minEdgeCover(const vector<vector<int>> &g, const vector<int>
↪ &ml, const vector<int> &mr)
{
    // g 是左部邻接表，ml、mr 是最大匹配；返回最小边覆盖，存在孤立点时返回空。
    vector<array<int, 2>> ans;
    vector<bool> usedR(mr.size());
    for (int u = 0; u < (int)g.size(); u++)
    {
        if (g[u].empty())
        {
            return nullopt;
```

```
        }
        if (ml[u] != -1)
        {
            ans.push_back({u, ml[u]});
            usedR[ml[u]] = true;
        }
    }
    for (int u = 0; u < (int)g.size(); u++)
    {
        if (ml[u] == -1)
        {
            ans.push_back({u, g[u][0]});
            usedR[g[u][0]] = true;
        }
    }
    vector<int> pre(mr.size(), -1);
    for (int u = 0; u < (int)g.size(); u++)
    {
        for (auto v : g[u])
        {
            pre[v] = u;
        }
    }
    for (int v = 0; v < (int)mr.size(); v++)
    {
        if (pre[v] == -1)
        {
            return nullopt;
        }
        if (!usedR[v])
        {
            ans.push_back({pre[v], v});
        }
    }
    return ans;
}
```

把原左部邻接表 `g` 和一份最大匹配 `ml` / `mr`（由 5.6.003 Hopcroft-Karp 或 5.6.001 匈牙利算法 求得）传入：`minVtxCover` 返回最小点覆盖的左点表与右点表；`minEdgeCover` 返回最小边覆盖的边列表（`array<int,2>` 为左右端点），任一侧有孤立点时返回空。两函数都只做一遍交替搜索与扫描，$O(V+E)$ 时间与输出空间，前置匹配的开销另计。最小点覆盖的补集就是最大独立集，构造见 5.6.008 最大独立集；这些都是二分图专属对偶，一般图的最小点覆盖是 NP 难、不能套用。

5.6.008 最大独立集

独立集是两两不相邻的点集。二分图里它有简洁对偶：最大独立集 = 全部点去掉一个最小点覆盖，大小 $|V|-$ 最大匹配，直接用 5.6.007 最小点覆盖、最小边覆盖 的点覆盖取补集即可。一般图没有这种多项式公式（最大独立集与最大团互补，都是 NP 难），只能搜索。这里在补图上求最大团：贪心着色把候选点分成若干独立色类，色数就是当前团还能扩大的上界；按色数倒序分支，界不可能超过已知最优时立即剪枝。

这比逐点选或不选的朴素分支适合 $n \leq 40$ 的在线约束。

```cpp
// 这是补图最大团的精确分支限界，只适合 n<=63 的一般图。
inline vector<int> maxIS(int n, const vector<pair<int, int>> &edges)
{
    // n 是点数，edges 是无向边；返回一组最大独立集顶点，允许重边和自环。
    assert(0 <= n && n <= 63); // 调试检查，可删：位集使用 unsigned long long。
    vector<unsigned long long> adj(n);
    vector<bool> loop(n);
    unsigned long long all = n ? (1ULL << n) - 1 : 0;
    for (int u = 0; u < n; u++)
    {
        adj[u] = all ^ (1ULL << u);
    }
    for (auto [u, v] : edges)
    {
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        if (u == v)
        {
            loop[u] = true;
        }
        else
        {
            adj[u] &= ~(1ULL << v);
            adj[v] &= ~(1ULL << u);
        }
    }
    vector<int> best, cur;
    auto dfs = [&](auto &&self, unsigned long long can) -> void
    {
        // can 是仍与当前选点全相邻的候选点集合；更新 best，无返回值。
        vector<int> ord, col;
        unsigned long long rem = can;
        int cnt = 0;
        while (rem)
        {
            cnt++;
            unsigned long long able = rem;
            while (able)
            {
                int u = countr_zero(able);
                ord.push_back(u);
                col.push_back(cnt);
                rem ^= 1ULL << u;
                able &= ~(1ULL << u);
                able &= ~adj[u];
                able &= rem;
            }
        }
        for (int i = (int)ord.size() - 1; i >= 0; i--)
        {
            if ((int)cur.size() + col[i] <= (int)best.size()) return;
```

```
            int u = ord[i];
            cur.push_back(u);
            unsigned long long nxt = 0;
            for (int j = 0; j < i; j++)
            {
                if (adj[u] >> ord[j] & 1ULL) nxt |= 1ULL << ord[j];
            }
            if (!nxt)
            {
                if (cur.size() > best.size()) best = cur;
            }
            else self(self, nxt);
            cur.pop_back();
        }
    };
    unsigned long long can = all;
    for (int u = 0; u < n; u++)
    {
        if (loop[u]) can &= ~(1ULL << u);
    }
    dfs(dfs, can);
    return best;
}
```

`maxIS(n, edges)` 返回一组最大独立集；平行边不影响结果，自环点会被排除，且要求 $n \le 63$。补图着色上界分支的最坏时间仍为指数级、额外空间 $O(n^2)$（递归候选序列），但在 $n \le 40$ 时比朴素逐点分支稳定得多。二分图应直接对 5.6.007 最小点覆盖、最小边覆盖 取补；更大的一般图要按稀疏性和约束另选专用算法。

5.6.009 DAG 最小不相交链覆盖

要用最少条点不相交有向路径覆盖 DAG 的全部点。经典转化：把每个点 x 拆成左副本 x_L 与右副本 x_R，原有向边 $x \to y$ 变成二分图边 x_L-y_R。二分图里的一条匹配边 x_L-y_R 表示「在覆盖中让 x 的后继紧接 y」，即把两个点串进同一条路径；于是每多一条匹配边就少一条路径，最小路径数 $= V -$ 最大匹配数。求出匹配后，从每个「右副本未被匹配」的点（即某条路径的起点）出发沿 `ml` 链一路串下去即可恢复每条路径。

```
// 依赖 5.6.003 Hopcroft-Karp
// DAG 的最小点不相交路径覆盖，返回路径中的原图顶点。
vector<vector<int>> dagPathCover(int n, const vector<pair<int, int>> &edges)
{
    // n 是点数，edges 是 DAG 边；返回一组最小点不相交路径覆盖。
    HopcroftKarp hk(n, n);
    for (auto [x, y] : edges)
    {
        hk.addEdge(x, y);
    }
    hk.matching();
    const auto &ml = hk.leftMatch();
```

```
    const auto &mr = hk.matchR();
    vector<vector<int>> ans;
    for (int s = 0; s < n; s++)
    {
        if (mr[s] != -1)
        {
            continue;
        }
        vector<int> a;
        for (int x = s; x != -1; x = ml[x])
        {
            a.push_back(x);
        }
        ans.push_back(move(a));
    }
    return ans;
}
```

`dagPathCover(n, edges)` 传入 DAG 的点数与有向边集，返回一组点不相交路径，每条路径按顶点顺序给出；路径条数即为最小值。拆点后二分图两侧各 n 个点，x、y 都沿用原图点号；本条直接复用 5.6.003 Hopcroft-Karp，不再内嵌第二份匹配器。输入必须是 DAG；若只给一般偏序的覆盖关系而边集未传递闭包，这里求的是原图路径覆盖，不是偏序链覆盖。二分图增广最坏 $O(m\sqrt{n})$，重建所有路径 $O(n)$，空间 $O(n+m)$。最大反链及传递闭包的区别见 5.6.010 DAG 最大互不可达集；带权版本或最小路径可重叠覆盖则要改用最小流建模，见 5.8.001 Dinic 最大流。

5.6.010 DAG 最大互不可达集

求 DAG 中两两不存在可达关系的最大点集，等价于其可达偏序上的最大反链。Dilworth 定理给出关键等式：最大反链大小等于最小链覆盖数。必须先求传递闭包，再把每个可达关系 $u \rightsquigarrow v$ 变成二分图边 $u_L - v_R$；只使用原图边求到的是最小路径覆盖，答案可能不同。

最大反链不能把二分图最小点覆盖的两个点集简单合并后取补，因为同一个原图点在左右部各有一个副本。设最小点覆盖为 $C_L \cup C_R$，正确恢复式是

$$A = \{x \mid x_L \notin C_L \text{ 且 } x_R \notin C_R\}.$$

任意可比的 $x < y$ 对应边 $x_L - y_R$，其至少一个端点在点覆盖中，所以 A 内两点必不可比；由 König 与 Dilworth 定理可知其大小恰为 $n - |M|$。

```
// 依赖 5.6.003 Hopcroft-Karp
// 依赖 5.6.007 最小点覆盖、最小边覆盖
vector<int> dagAntichain(int n, const vector<pair<int, int>> &edges)
{
    // n 是点数，edges 是 DAG 边；返回一组最大互不可达点集。
    vector reach(n, vector<bool>(n));
    for (auto [u, v] : edges)
    {
        reach[u][v] = true;
    }
    for (int k = 0; k < n; k++)
```

```
    {
        for (int i = 0; i < n; i++)
        {
            if (!reach[i][k])
            {
                continue;
            }
            for (int j = 0; j < n; j++)
            {
                reach[i][j] = reach[i][j] || reach[k][j];
            }
        }
    }

    vector<vector<int>> g(n);
    HopcroftKarp hk(n, n);
    for (int u = 0; u < n; u++)
    {
        for (int v = 0; v < n; v++)
        {
            if (reach[u][v])
            {
                g[u].push_back(v);
                hk.addEdge(u, v);
            }
        }
    }
    hk.matching();
    auto [cl, cr] = minVtxCover(g, hk.leftMatch(), hk.matchR());
    vector<bool> inL(n), inR(n);
    for (int u : cl)
    {
        inL[u] = true;
    }
    for (int v : cr)
    {
        inR[v] = true;
    }
    vector<int> ans;
    for (int u = 0; u < n; u++)
    {
        if (!inL[u] && !inR[u])
        {
            ans.push_back(u);
        }
    }
    return ans;
}
```

`dagAntichain(n, edges)` 返回一组最大反链。代码用 Floyd 求传递闭包，时间 $O(n^3 + n^2\sqrt{n})$、空间 $O(n^2)$；稀疏大图可把闭包部分换成从每点出发的 BFS，匹配与恢复不变。只需最小链覆盖方案时，把闭包边交给 5.6.009 DAG 最小不相交链覆盖；`dagPathCover` 返回的链数也等于本函数答案大小。二分图最

小点覆盖的构造见 5.6.007 最小点覆盖、最小边覆盖。带权反链需另建最小割模型，不能直接给这里的基数匹配加权。

5.6.011 最大权闭合子图

在带点权的有向图里选一个点集，要求「选了点 u 就必须选它的所有后继依赖点」，并最大化所选点权和——这是闭合性约束（选一个就得连带选下去），不是匹配。经典的最小割建模：新建源 s、汇 t，正权点 u 连 $s \to u$ 容量 w_u，负权点 u 连 $u \to t$ 容量 $-w_u$，每条依赖 $u \Rightarrow v$ 连 $u \to v$ 容量取一个大于全部正权之和的「无穷」值（代码里取 $1 + \sum_{w_u>0} w_u$）。因为依赖边容量足够大，任何有限割都不会切它，于是割对应的源侧点集一定是闭合的；最小割 $=$ 未选正权点的损失 $+$ 已选负权点的代价，答案就是正权总和减最小割。求出最小割后，残量网络里从 s 出发 BFS 可达的点即被选点集。

```cpp
template <class Cap = long long>
struct ClosureRes
{
    Cap val;          // 最大闭合子图点权和。
    vector<int> pick; // 一组被选择的原图点编号。
};

template <class Cap = long long>
ClosureRes<Cap> maxClosure(
    const vector<long long> &w, const vector<array<int, 2>> &need)
{
    // w[u] 是点权，need 中的 {u, v} 表示选 u 必须选 v；返回最优闭集。
    int n = w.size(), s = n, t = n + 1;
    Cap sum = 0;
    for (long long x : w)
    {
        if (x > 0)
        {
            sum += (Cap)x;
        }
    }

    Cap inf = sum + 1;
    Flow<Cap> fl(n + 2);
    for (int u = 0; u < n; u++)
    {
        if (w[u] > 0)
        {
            fl.add(s, u, (Cap)w[u]);
        }
        else if (w[u] < 0)
        {
            fl.add(u, t, -(Cap)w[u]);
        }
    }
    for (auto [u, v] : need)
    {
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
```

```
        fl.add(u, v, inf);
    }

    Cap flow = fl.flow(s, t);
    vector<bool> vis = fl.cut(s);
    vector<int> pick;
    for (int u = 0; u < n; u++)
    {
        if (vis[u])
        {
            pick.push_back(u);
        }
    }
    return ClosureRes<Cap>{sum - flow, pick};
}
```

`maxClosure<Cap>(w,need)` 中 `Cap` 默认是 `long long`，`val` 是最大权和，`pick` 是一组最优点。调用者必须保证正权总和、负权取反和人工边容量可由 `Cap` 表示；确实需要时可显式使用 `maxClosure<__int128>`。本条依赖 5.8.001 Dinic 最大流，建图 $O(V+E)$，Dinic 一般网络最坏 $O(V^2E)$，空间 $O(V+E)$。

5.7 一般图匹配

5.7.001 一般图最大匹配（Blossom）

在允许奇环的一般无向图里求最大基数匹配。5.6.001 匈牙利算法 与 5.6.003 Hopcroft-Karp 的二分图增广之所以失效，是因为奇环会让 BFS 交替森林里出现「同层两点相连」的边，此时沿两条路径回溯到最近公共祖先 b 之间的那个奇环就是一朵「花」（blossom）。Edmonds 的关键洞见：把整朵花缩成一个点后再找增广路，缩点不改变是否存在增广路——花里任意点都能既作偶点又作奇点参与增广。实现上不真的缩点，而是用 `base[]` 记录每个点当前所属花的基点，`lca` 求两条交替路径的汇合点，`mark` 沿环把途经点的 `base` 统一指向 b 并把奇点也拉进队列继续 BFS。找到未匹配端点后，沿 `p[]` 父链回溯并翻转匹配边即完成一次增广。逐点尝试增广，匹配数每次至多加一。

```
struct Blossom
{
    // 点数与无向邻接表。
    int n;
    vector<vector<int>> g;
    // 当前配偶、交替森林父点、花基点和 BFS 队列。
    vector<int> match, p, base, q;
    // used 标记偶层点是否入队，flower 标记本轮被收缩的基点。
    vector<bool> used, flower;

    Blossom(int n = 0) : n(n), g(n), match(n, -1), p(n), base(n), q(n), used(n), flower(n)
    {
        // n 是点数；构造空图。
    }

    void add(int u, int v)
```

```
    {
        // u、v 是端点；自环不参与匹配，直接忽略。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        if (u == v)
        {
            return;
        }
        g[u].push_back(v);
        g[v].push_back(u);
    }

    int lca(int a, int b)
    {
        // a、b 是同棵交替树中的偶层点；返回新花的基点。
        vector<bool> seen(n);
        while (true)
        {
            a = base[a];
            seen[a] = true;
            if (match[a] < 0)
            {
                break;
            }
            a = p[match[a]];
        }
        while (true)
        {
            b = base[b];
            if (seen[b])
            {
                return b;
            }
            b = p[match[b]];
        }
    }

    void mark(int v, int b, int x)
    {
        // 从 v 沿交替树走到基点 b，并令第一步父点为 x。
        while (base[v] != b)
        {
            flower[base[v]] = true;
            flower[base[match[v]]] = true;
            p[v] = x;
            x = match[v];
            v = p[match[v]];
        }
    }

    int path(int s)
    {
        // 从未匹配点 s 建交替树；返回另一个未匹配端点，不存在则返回 -1。
```

```
        fill(used.begin(), used.end(), false);
        fill(p.begin(), p.end(), -1);
        iota(base.begin(), base.end(), 0);
        int ql = 0, qr = 0;
        q[qr++] = s;
        used[s] = true;
        while (ql < qr)
        {
            int v = q[ql++];
            for (int u : g[v])
            {
                if (base[v] == base[u] || match[v] == u)
                {
                    continue;
                }
                if (u == s || (match[u] >= 0 && p[match[u]] >= 0))
                {
                    int b = lca(v, u);
                    fill(flower.begin(), flower.end(), false);
                    mark(v, b, u);
                    mark(u, b, v);
                    for (int i = 0; i < n; i++)
                    {
                        if (flower[base[i]])
                        {
                            base[i] = b;
                            if (!used[i])
                            {
                                used[i] = true;
                                q[qr++] = i;
                            }
                        }
                    }
                }
                else if (p[u] < 0)
                {
                    p[u] = v;
                    if (match[u] < 0)
                    {
                        return u;
                    }
                    u = match[u];
                    used[u] = true;
                    q[qr++] = u;
                }
            }
        }
        return -1;
    }

    int work()
    {
```

```
        // 从空匹配重算，返回最大匹配边数。
        fill(match.begin(), match.end(), -1);
        int ans = 0;
        for (int s = 0; s < n; s++)
        {
            if (match[s] >= 0)
            {
                continue;
            }
            int v = path(s);
            if (v < 0)
            {
                continue;
            }
            ans++;
            while (v >= 0)
            {
                int u = p[v];
                int w = u < 0 ? -1 : match[u];
                if (u >= 0)
                {
                    match[v] = u;
                    match[u] = v;
                }
                v = w;
            }
        }
        return ans;
    }
};
```

构造 `Blossom bl(n)`，`add(u, v)` 加无向边（点 0 下标），`work()` 返回最大匹配数并把方案写进 `match[]`（`match[u]` 为 u 的配偶，未匹配为 -1）。图可以不连通，也可以含平行边或自环；自环不可能进入普通图匹配，`add` 会直接忽略。时间 $O(n^3)$、空间 $O(n+m)$。图是二分图时这份代码仍正确，但用 5.6.003 Hopcroft-Karp 的 $O(m\sqrt{n})$ 更快；需要带权最大匹配见 5.7.002 一般图最大权匹配（本模板仅给 $n \le 24$ 的指数版），只求匹配大小可用 5.7.003 Tutte Matrix 的随机代数法。

下面用五环加一片叶子检查奇环收缩；最优方案可取 $(0,5),(1,2),(3,4)$，匹配数为 3。

```
signed main()
{
    Blossom b(6);
    for (int i = 0; i < 5; i++)
    {
        b.add(i, (i + 1) % 5);
    }
    b.add(0, 5);
    assert(b.work() == 3);
}
```

5.7.002 一般图最大权匹配

在一般无向图中求总权最大的匹配（不要求匹配数最大，允许空匹配）。理论上正解是带权 Blossom + 原始对偶，$O(n^3)$；但那份代码极长且容易写错，竞赛里点数往往很小，所以这里给一个更短、更可靠的状压 DP：以 $dp[S]$ 表示只用点集 S 时的最大权匹配。取 S 中最低位点 u，要么 u 不匹配（继承 $dp[S \setminus \{u\}]$），要么 u 与 S 中某个更高位点 v 匹配（$dp[S \setminus \{u,v\}] + w_{uv}$），两者取大。`take[S]` 记录 u 在最优解里配给谁，便于回溯匹配边。因为每步固定处理最低位点，转移无后效，$O(n2^n)$，`assert(n <= 24)` 卡住规模上限。

```cpp
pair<int, vector<pair<int, int>>>
weightMatch(const vector<vector<optional<long long>>> &w)
{
    // w[u][v] 是边权，空值表示无边；返回最大总权和一组匹配边。
    int n = w.size();
    assert(n <= 24); // 调试检查，可删：状态数必须能承受。
    for (const auto &row : w)
    {
        assert((int)row.size() == n); // 调试检查，可删。
    }
    size_t tot = 1ULL << n;
    vector<int> dp(tot);
    vector<int> take(tot, -1);
    dp[0] = 0;
    for (size_t s = 1; s < tot; s++)
    {
        int u = countr_zero((unsigned long long)s);
        dp[s] = dp[s ^ (1ULL << u)];
        for (int v = u + 1; v < n; v++)
        {
            if ((s >> v & 1) && w[u][v].has_value())
            {
                int z = dp[s ^ (1ULL << u) ^ (1ULL << v)] + *w[u][v];
                if (z > dp[s])
                {
                    dp[s] = z;
                    take[s] = v;
                }
            }
        }
    }
    vector<pair<int, int>> e;
    for (size_t s = tot - 1; s;)
    {
        int u = countr_zero((unsigned long long)s), v = take[s];
        s ^= 1ULL << u;
        if (v >= 0)
        {
            s ^= 1ULL << v;
            e.push_back({u, v});
        }
    }
```

```
    return pair{dp.back(), e};
}
```

`weightMatch(w)` 接受 $n \times n$ 对称矩阵，空值表示无边；返回最大总权和一组匹配边。算法允许空匹配，故全负权答案为 0；调用者必须保证匹配总权可由 64 位有符号整数表示。要求 $n \le 24$，但 $n = 24$ 时内存约 $272\,\mathrm{MiB}$，常见限制下宜取 $n \le 22$；时间 $O(n2^n)$、空间 $O(2^n)$。

5.7.003 Tutte Matrix

图 G 的 Tutte 矩阵是一个反对称矩阵：对每条边 (u, v) 引入一个不定元 x_{uv}，令 $A_{uv} = x_{uv}$、$A_{vu} = -x_{uv}$，非边处为 0。Tutte 定理说 $\det A$（作为多项式）恒为零当且仅当 G 没有完美匹配；Lovász 进一步证明 $\operatorname{rank} A = 2\nu(G)$，$\nu$ 是最大匹配数。多项式行列式无法直接算，但可用 Schwartz–Zippel 引理：把每个 x_{uv} 代入大素数域上的随机值，则以极高概率 rank 不掉——单次误判概率约 $\frac{n}{P}$。于是把随机化后的数值矩阵做一次高斯消元求秩，除以 2 就得最大匹配大小。代码里 `rankField` 是有限域高斯消元求秩，`tutteMatch` 填反对称随机矩阵后取 rank $/2$；固定 `seed` 保证结果可复现。

```
int rankField(vector<vector<Z>> a)
{
    // a 是有限域上的矩阵；返回矩阵秩。
    int n = a.size(), m = n ? a[0].size() : 0, r = 0;
    for (int c = 0; c < m && r < n; c++)
    {
        int p = r;
        while (p < n && a[p][c] == Z(0))
        {
            p++;
        }
        if (p == n)
        {
            continue;
        }
        swap(a[p], a[r]);
        Z iv = a[r][c].inv();
        for (int i = r + 1; i < n; i++)
        {
            Z q = a[i][c] * iv;
            for (int j = c; j < m; j++)
            {
                a[i][j] -= q * a[r][j];
            }
        }
        r++;
    }
    return r;
}

int tutteMatch(int n, const vector<pair<int, int>> &e, unsigned seed = 712367)
{
    // n 是点数，e 是无向边端点列表，seed 是固定随机种子；返回最大匹配大小，存在有限域随机误判概率。
```

```
    mt19937 rng(seed);
    vector a(n, vector<Z>(n));
    for (auto [u, v] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        if (u == v)
        {
            continue;
        }
        Z x = rng() % (Z::mod() - 1) + 1;
        a[u][v] += x;
        a[v][u] -= x;
    }
    return rankField(move(a)) / 2;
}
```

依赖 1.1.008 静态取模整数类 的大素数域类型 `Z`。调用 `tutteMatch(n, e, seed)`，`e` 是无向边端点列表（点 0 下标）；平行边可重复出现，自环与普通图匹配无关，代码会忽略。返回最大匹配大小，不恢复边集；例如四点路径返回 2，三点路径返回 1。时间 $O(n^3+m)$、空间 $O(n^2)$，单次误判概率 $O(\frac{n}{P})$，P 是模数；可换几个 `seed` 取最大值进一步降低误判概率。需要确定性结果或恢复方案时改用 5.7.001 一般图最大匹配（Blossom）。

5.7.004 Edmonds Matrix

竞赛资料里「Edmonds Matrix」常指一般图匹配所用的 Tutte–Edmonds 反对称矩阵，与 5.7.003 Tutte Matrix 是同一个代数对象、同一份代码，这里不再重复打印，直接沿用那里的 `tutteMatch(n, e, seed)`：随机代入后求秩除以二即最大匹配大小，$O(n^3+m)$ 时间、$O(n^2)$ 空间，单次误判概率 $O(\frac{n}{P})$。需要注意的是「Edmonds 矩阵」在另一些资料里特指二分图的那个 x_{ij} 矩阵（左行右列，非反对称），其行列式非零当且仅当二分图有完美匹配——那是不同的对象，若题目按二分图定义使用，应改在二分图上建矩阵，而不能套用本条的一般图反对称版本。要确定性求解或恢复匹配边，用 5.7.001 一般图最大匹配（Blossom）。

5.7.005 稳定婚姻

给左右两侧各 n 人以及每人对另一侧从优到劣的完整偏好序列，求一个完美匹配，使不存在「阻塞对」——即不存在一对未配的 (x,y) 彼此都比各自现配偶更中意。稳定匹配一定存在，Gale–Shapley 算法给出其一：让尚未匹配的求婚方 x 按自己偏好从高到低依次向 y 求婚，y 若空闲则暂定，若已有对象则留下自己更偏好的那个、把另一个踢回求婚队列。每个 x 对每个 y 至多求婚一次，故必然终止；结论对求婚方最优、对被求婚方最劣，但都稳定。代码用 `rk[y][x]` 预存 y 眼中 x 的排名（越小越偏好）以 $O(1)$ 比较，`at[x]` 记 x 下一个要求婚的对象下标，队列 `q` 存当前空闲的求婚方。

```
struct Marriage
{
    // a[x]、b[y] 分别是左侧 x 与右侧 y 的配偶编号。
    vector<int> a, b;
};
```

```cpp

// pa[x] 是左侧 x 对右侧的偏好；pb[y] 是右侧 y 对左侧的偏好，均从优到劣。
Marriage stableMatch(const vector<vector<int>> &pa, const vector<vector<int>> &pb)
{
    // pa、pb 是左右两侧从优到劣的完整偏好表；返回两侧互逆的稳定配偶数组。
    int n = pa.size();
    assert((int)pb.size() == n); // 调试检查，可删。
    vector<vector<int>> rk(n, vector<int>(n));
    for (int y = 0; y < n; y++)
    {
        for (int i = 0; i < n; i++)
        {
            rk[y][pb[y][i]] = i;
        }
    }
    vector<int> a(n, -1), b(n, -1), at(n);
    queue<int> q;
    for (int x = 0; x < n; x++)
    {
        q.push(x);
    }
    while (!q.empty())
    {
        int x = q.front();
        q.pop();
        int y = pa[x][at[x]++];
        if (b[y] < 0)
        {
            a[x] = y;
            b[y] = x;
        }
        else if (rk[y][x] < rk[y][b[y]])
        {
            q.push(b[y]);
            a[b[y]] = -1;
            a[x] = y;
            b[y] = x;
        }
        else
        {
            q.push(x);
        }
    }
    return {a, b};
}
```

`stableMatch(pa, pb)` 中 `pa[x]`、`pb[y]` 都是长度 n 的排列，分别是左侧 x、右侧 y 从优到劣的对方编号序列；返回的 `Marriage` 里 `a[x]` 是左侧 x 的配偶、`b[y]` 是右侧 y 的配偶。要求两侧人数相等且偏好表是完整排列（每人都能接受所有对象），得到的是求婚方（左侧）最优稳定匹配；把 `pa`、`pb` 互换即得另一侧最优解。时间与空间 $O(n^2)$，瓶颈是 `rk` 表与偏好表本身。这属于稳定匹配问题，与追求总权最优的 5.6.005 KM 算法 或最大基数的一般图匹配 5.7.001 一般图最大匹配（Blossom） 目标不同，不能相互替代——稳

定解通常不是权最大解。

5.7.006 一般图最大权匹配（带权 Blossom）

上一条状压 DP 只适合 $n \le 24$。需要在一般图上直接处理更大点数时，使用带权 Blossom 的原始对偶算法：顶标 `lab` 保持边的对偶可行性，交替树上的零紧边扩展增广路，遇到奇环就收缩成 Blossom；顶标调整后再展开达到零顶标的花朵。实现按外部竞赛模板改成动态数组，保留矩阵松弛，时间 $O(n^3)$、空间 $O(n^2)$。边权非正时允许空匹配，因此不会为了凑匹配数强行加入负边。

```cpp
template <class W = long long>
struct WBlossom
{
    struct Edge
    {
        int u = 0, v = 0; // u、v 是边在当前收缩图中的两个端点。
        W w = 0;          // w 是边权。
    };

    int n, m, stamp = 0, head = 0, tail = 0; // n 是点数，m 是已用的点与花朵编号上界，stamp 是找 LCA 的时间戳，head、tail 是 BFS 队列首尾。
    W inf = numeric_limits<W>::max() / 4; // inf 是不可达权值。
    vector<vector<Edge>> e;          // e 是原点与花朵之间的最优代表边矩阵。
    vector<W> lab;                   // lab 是原点和花朵的对偶顶标。
    vector<int> link, slack, state;  // link 是匹配点，slack 是最松边，state 是交替树状态。
    vector<int> rep, fa, seen, q;    // rep 是当前花朵代表，fa 是交替树父边端点。
    vector<vector<int>> blossom;     // blossom[x] 按环序保存花朵 x 的子块。
    vector<vector<int>> base;        // base[x][v] 是花朵 x 内含原点 v 的直接子块。

    explicit WBlossom(int n)
        : n(n), m(n), e(2 * n + 5, vector<Edge>(2 * n + 5)),
          lab(2 * n + 5), link(2 * n + 5), slack(2 * n + 5),
          state(2 * n + 5), rep(2 * n + 5), fa(2 * n + 5), seen(2 * n + 5),
          q(2 * n + 5), blossom(2 * n + 5), base(2 * n + 5, vector<int>(2 * n + 5))
    {
        // n 是原图点数；构造最多含 n 个收缩花朵的工作区。
        assert(n >= 0);
        for (int i = 1; i < (int)e.size(); i++)
        {
            for (int j = 1; j < (int)e.size(); j++)
            {
                e[i][j] = {i, j, 0};
            }
        }
    }

    W delta(const Edge &x) const
    {
        // x 是边；返回它相对当前顶标的二倍松弛量。
        return lab[x.u] + lab[x.v] - x.w * 2;
    }
```

```

    void updateSlack(int u, int v)
    {
        // u 是外点，v 是待更新块；保留连接 v 的最松边起点。
        if (!slack[v] || delta(e[u][v]) < delta(e[slack[v]][v]))
        {
            slack[v] = u;
        }
    }

    void setSlack(int v)
    {
        // v 是当前块；重新寻找从外层交替树连接到 v 的最松边。
        slack[v] = 0;
        for (int u = 1; u <= n; u++)
        {
            if (e[u][v].w > 0 && rep[u] != v && !state[rep[u]])
            {
                updateSlack(u, v);
            }
        }
    }

    void enqueue(int u)
    {
        // u 是外层块；把其中全部原点加入交替树扫描队列。
        if (u <= n)
        {
            q[++tail] = u;
        }
        else
        {
            for (int v : blossom[u])
            {
                enqueue(v);
            }
        }
    }

    void modify(int u, int w)
    {
        // u 是块，w 是新代表；递归修改 u 内全部原点的代表。
        rep[u] = w;
        if (u > n)
        {
            for (int v : blossom[u])
            {
                modify(v, w);
            }
        }
    }

```

```
    int position(int u, int v)
    {
        // u 是花朵，v 是其中一个直接子块；调整环向后返回 v 的偶数下标。
        auto it = find(blossom[u].begin(), blossom[u].end(), v);
        int x = it - blossom[u].begin();
        if (x & 1)
        {
            reverse(blossom[u].begin() + 1, blossom[u].end());
            return (int)blossom[u].size() - x;
        }
        return x;
    }

    void setMatch(int u, int v)
    {
        // u、v 是两个待配对块；展开花朵 u 内部并建立对应匹配。
        link[u] = e[u][v].v;
        if (u <= n)
        {
            return;
        }
        Edge x = e[u][v];
        int z = base[u][x.u], k = position(u, z);
        for (int i = 0; i < k; i++)
        {
            setMatch(blossom[u][i], blossom[u][i ^ 1]);
        }
        setMatch(z, v);
        rotate(blossom[u].begin(), blossom[u].begin() + k, blossom[u].end());
    }

    void augment(int u, int v)
    {
        // u、v 是增广路首条边两端；沿交替树翻转匹配直到自由根。
        int x = rep[link[u]];
        setMatch(u, v);
        if (!x)
        {
            return;
        }
        setMatch(x, rep[fa[x]]);
        augment(rep[fa[x]], x);
    }

    int lca(int u, int v)
    {
        // u、v 是两个外层块；返回其交替树最低公共祖先，不相交时返回 0。
        ++stamp;
        while (u || v)
        {
            if (u)
            {
```

```
                if (seen[u] == stamp)
                {
                    return u;
                }
                seen[u] = stamp;
                u = rep[link[u]];
                if (u)
                {
                    u = rep[fa[u]];
                }
            }
            swap(u, v);
        }
        return 0;
    }

    void contract(int u, int a, int v)
    {
        // u、v 是零紧边两端，a 是交替树公共祖先；收缩对应奇环。
        int x = n + 1, i, j;
        while (x <= m && rep[x])
        {
            x++;
        }
        if (x > m)
        {
            m++;
        }
        lab[x] = state[x] = rep[x] = 0;
        link[x] = link[a];
        blossom[x].clear();
        blossom[x].push_back(a);
        for (i = u; i != a; i = rep[fa[j]])
        {
            blossom[x].push_back(i);
            blossom[x].push_back(j = rep[link[i]]);
            enqueue(j);
        }
        reverse(blossom[x].begin() + 1, blossom[x].end());
        for (i = v; i != a; i = rep[fa[j]])
        {
            blossom[x].push_back(i);
            blossom[x].push_back(j = rep[link[i]]);
            enqueue(j);
        }
        modify(x, x);
        for (i = 1; i <= m; i++)
        {
            e[x][i].w = e[i][x].w = 0;
        }
        fill(base[x].begin() + 1, base[x].begin() + n + 1, 0);
        for (int u0 : blossom[x])
```

```
        {
            for (int v0 = 1; v0 <= m; v0++)
            {
                if (!e[x][v0].w || delta(e[u0][v0]) < delta(e[x][v0]))
                {
                    e[x][v0] = e[u0][v0];
                    e[v0][x] = e[v0][u0];
                }
            }
            for (int v0 = 1; v0 <= n; v0++)
            {
                if (base[u0][v0])
                {
                    base[x][v0] = u0;
                }
            }
        }
        setSlack(x);
    }

    void expand(int u)
    {
        // u 是顶标降到零的内层花朵；按交替树方向展开。
        for (int x : blossom[u])
        {
            modify(x, x);
        }
        int a = base[u][e[u][fa[u]].u], r = position(u, a);
        for (int i = 0; i < r; i += 2)
        {
            int x = blossom[u][i], y = blossom[u][i + 1];
            fa[x] = e[y][x].u;
            state[x] = 1;
            state[y] = slack[x] = 0;
            setSlack(y);
            enqueue(y);
        }
        state[a] = 1;
        fa[a] = fa[u];
        for (int i = r + 1; i < (int)blossom[u].size(); i++)
        {
            state[blossom[u][i]] = -1;
            setSlack(blossom[u][i]);
        }
        rep[u] = 0;
    }

    bool onEdge(const Edge &x)
    {
        // x 是新出现的零紧边；扩展交替树、收花或增广，增广成功返回 true。
        int u = rep[x.u], v = rep[x.v];
        if (state[v] == -1)
```

```
        {
            fa[v] = x.u;
            state[v] = 1;
            int a = rep[link[v]];
            slack[v] = slack[a] = state[a] = 0;
            enqueue(a);
        }
        else if (!state[v])
        {
            int a = lca(u, v);
            if (!a)
            {
                augment(u, v);
                augment(v, u);
                return true;
            }
            contract(u, a, v);
        }
        return false;
    }

    bool bfs()
    {
        // 在当前匹配上寻找并执行一条增广路；不存在时返回 false。
        fill(state.begin() + 1, state.begin() + m + 1, -1);
        fill(slack.begin() + 1, slack.begin() + m + 1, 0);
        head = 1;
        tail = 0;
        for (int i = 1; i <= m; i++)
        {
            if (rep[i] == i && !link[i])
            {
                fa[i] = state[i] = 0;
                enqueue(i);
            }
        }
        if (head > tail)
        {
            return false;
        }
        while (true)
        {
            while (head <= tail)
            {
                int u = q[head++];
                if (state[rep[u]] == 1)
                {
                    continue;
                }
                for (int v = 1; v <= n; v++)
                {
                    if (e[u][v].w > 0 && rep[u] != rep[v])
```

```
                    {
                        if (delta(e[u][v]))
                        {
                            updateSlack(u, rep[v]);
                        }
                        else if (onEdge(e[u][v]))
                        {
                            return true;
                        }
                    }
                }
            }
            W x = inf;
            for (int i = n + 1; i <= m; i++)
            {
                if (rep[i] == i && state[i] == 1)
                {
                    x = min(x, lab[i] / 2);
                }
            }
            for (int i = 1; i <= m; i++)
            {
                if (rep[i] == i && slack[i] && rep[slack[i]] != i && state[i] != 1)
                {
                    x = min(x, delta(e[slack[i]][i]) >> (state[i] + 1));
                }
            }
            for (int i = 1; i <= n; i++)
            {
                if (state[rep[i]] != -1)
                {
                    lab[i] += (state[rep[i]] * 2 - 1) * x;
                    if (lab[i] <= 0)
                    {
                        return false;
                    }
                }
            }
            for (int i = n + 1; i <= m; i++)
            {
                if (rep[i] == i && state[i] != -1)
                {
                    lab[i] += (2 - state[i] * 4) * x;
                }
            }
            head = 1;
            tail = 0;
            for (int i = 1; i <= m; i++)
            {
                if (rep[i] == i && slack[i] && rep[slack[i]] != i &&
                    !delta(e[slack[i]][i]) && onEdge(e[slack[i]][i]))
                {
```

```
                    return true;
                }
            }
            for (int i = n + 1; i <= m; i++)
            {
                if (rep[i] == i && state[i] == 1 && !lab[i])
                {
                    expand(i);
                }
            }
        }
    }

    pair<W, vector<pair<int, int>>> solve(const vector<tuple<int, int, W>> &edges)
    {
        // edges 是 0 下标无向边 {u,v,w}；返回最大总权和及所选边端点。
        m = n;
        stamp = 0;
        fill(link.begin(), link.end(), 0);
        fill(seen.begin(), seen.end(), 0);
        for (int i = 1; i <= n; i++)
        {
            for (int j = 1; j <= n; j++)
            {
                e[i][j].w = 0;
            }
        }
        for (int i = 1; i <= n; i++)
        {
            rep[i] = i;
            blossom[i].clear();
        }
        W wm = 0;
        for (auto [u, v, w] : edges)
        {
            assert(0 <= u && u < n && 0 <= v && v < n && u != v);
            u++, v++;
            e[u][v].w = e[v][u].w = max(e[u][v].w, w);
            wm = max(wm, w);
        }
        for (int i = 1; i <= n; i++)
        {
            for (int j = 1; j <= n; j++)
            {
                base[i][j] = i == j ? i : 0;
                if (i != j)
                {
                    e[i][j].u = i;
                    e[i][j].v = j;
                }
            }
        }
```

```
        fill(lab.begin() + 1, lab.begin() + n + 1, wm);
        while (bfs())
        {
        }
        W ans = 0;
        vector<pair<int, int>> match;
        for (int i = 1; i <= n; i++)
        {
            if (link[i])
            {
                ans += e[i][link[i]].w;
                if (i < link[i])
                {
                    match.push_back({i - 1, link[i] - 1});
                }
            }
        }
        return {ans / 2, match};
    }
};
```

`edges` 是点编号从 0 开始的无向边三元组，重复边只保留较大权值；`solve` 返回最大总权值和匹配边集。权值必须能容纳顶标调整与答案累加，默认使用 `long long`；需要更宽中间量时显式实例化 `WBlossom<__int128>`。这是最大权匹配，不保证匹配边数最大；若只求最大基数，使用 5.7.001 一般图最大匹配（Blossom），小图还可用 5.7.002 一般图最大权匹配。固定四点完全图可检查返回值为 11：

```
signed main()
{
    vector<tuple<int, int, long long>> e{{0, 1, 4}, {0, 2, 5}, {0, 3, 1},
                                         {1, 2, 2}, {1, 3, 6}, {2, 3, 3}};
    WBlossom<long long> b(4);
    auto [v, m] = b.solve(e);
    assert(v == 11 && m.size() == 2);
}
```

5.7.007 稀疏稳定匹配（不完全偏好）

完整偏好表即使只运行 Gale–Shapley，也必须读入和保存 $\Theta(n^2)$ 个排名；当两侧人数不同、每个人只接受少量对象时，应只存可接受关系。设左右侧人数为 n_L, n_R，可接受边数为 m。每条边同时给出它在左端点和右端点偏好表中的严格排名，排名从 0 开始且越小越优；同一个人的排名必须恰为 $[0, d)$ 的排列，其中 d 是其可接受对象数。未列出的两人彼此不可接受，不会组成阻塞对。

算法仍让左侧依次求婚。代码先按 `leftRank` 将边线性放入连续偏好表；右侧只需保存当前对象的 `rightRank`，即可在 $O(1)$ 时间决定保留谁。每条边最多被求婚一次，因此返回的匹配对左侧最优且稳定。与需要按偏好分数排序的接口相比，显式排名省去 $O(m \log m)$ 的预处理；代价是调用方必须先给出合法的严格排名。

```cpp
struct StableEdge
{
    int left, right;           // left、right 是可接受关系的左右端点。
    int leftRank, rightRank;   // 两个排名分别表示双方对该关系的偏好，0 最优。
};

struct SparseMatch
{
    vector<int> left, right; // 两个数组分别记录左右侧配偶；未匹配位置为 -1。
};

SparseMatch sparseMatch(int nLeft, int nRight,
                                    const vector<StableEdge> &edges)
{
    // nLeft、nRight 是两侧人数，edges 给出全部可接受关系及双方严格排名；返回左侧最优稳定匹配。
    assert(nLeft >= 0 && nRight >= 0);
    vector<int> beginLeft(nLeft + 1), beginR(nRight + 1);
    for (auto [x, y, rx, ry] : edges)
    {
        assert(0 <= x && x < nLeft && 0 <= y && y < nRight);
        ++beginLeft[x + 1];
        ++beginR[y + 1];
    }
    partial_sum(beginLeft.begin(), beginLeft.end(), beginLeft.begin());
    partial_sum(beginR.begin(), beginR.end(), beginR.begin());

    // pref 按左侧排名连续存边；usedRight 检查每个右侧排名恰好出现一次。
    vector<pair<int, int>> pref(edges.size(), {-1, -1});
    vector<char> usedRight(edges.size());
    for (auto [x, y, rx, ry] : edges)
    {
        assert(0 <= rx && rx < beginLeft[x + 1] - beginLeft[x]);
        assert(0 <= ry && ry < beginR[y + 1] - beginR[y]);
        int px = beginLeft[x] + rx, py = beginR[y] + ry;
        assert(pref[px].first == -1 && !usedRight[py]);
        pref[px] = {y, ry};
        usedRight[py] = true;
    }

    vector<int> left(nLeft, -1), right(nRight, -1), heldRank(nRight, -1);
    vector<int> next = beginLeft, q;
    q.reserve(nLeft + edges.size());
    for (int x = 0; x < nLeft; ++x)
    {
        if (next[x] < beginLeft[x + 1])
        {
            q.push_back(x);
        }
    }
    for (int head = 0; head < (int)q.size(); ++head)
    {
        int x = q[head];
```

```
        auto [y, rank] = pref[next[x]++];
        if (right[y] == -1 || rank < heldRank[y])
        {
            int old = right[y];
            if (old != -1)
            {
                left[old] = -1;
                if (next[old] < beginLeft[old + 1])
                {
                    q.push_back(old);
                }
            }
            left[x] = y;
            right[y] = x;
            heldRank[y] = rank;
        }
        else if (next[x] < beginLeft[x + 1])
        {
            q.push_back(x);
        }
    }
    return {left, right};
}
```

`sparseMatch(nLeft,nRight,edges)` 使用 0 下标；每对可接受对象在 `edges` 中恰好出现一次，左右两端的排名都必须严格且连续。返回的 `left[x]` 与 `right[y]` 互逆，`-1` 表示没有可接受的稳定配偶。若允许并列偏好，需要另行规定弱稳定、强稳定或超稳定，不能直接把相同排名交给本实现。时间与额外空间均为 $O(n_L+n_R+m)$。若每个人接受另一侧全部对象且两侧同样大，输入已有 $\Theta(n^2)$ 项，使用 5.7.005 稳定婚姻 的双完整偏好表接口更自然；若目标是最大匹配边数或总权而不考虑偏好稳定性，分别使用 5.6.001 Hopcroft–Karp 或 5.6.005 KM 算法。固定的 3 对 4 示例中，左侧配偶为 $[1,0,2]$，右侧第 3 人未匹配：

```
signed main()
{
    vector<StableEdge> e{{0, 0, 0, 1}, {0, 1, 1, 0},
                         {1, 0, 0, 0}, {2, 2, 0, 0}};
    auto ans = sparseMatch(3, 4, e);
    assert((ans.left == vector<int>{1, 0, 2}));
    assert((ans.right == vector<int>{1, 0, 2, -1}));
}
```

5.8 网络流与割

5.8.001 Dinic 最大流

在有向容量网络里求源 s 到汇 t 的最大流。Dinic 每轮先 BFS 建残量层次图，再用当前弧优化的 DFS 推阻塞流。`flow` 的 `lim` 参数允许只继续增广指定流量，`cut(s)` 在最大流跑满后返回残量图的源侧。本条没有实现按容量位分阶段的 Scaling，因此标题只保留普通 Dinic；`Flow` 是本章其余流模型的统一底座。

```cpp
template <class T>
struct Flow
{
    struct E
    {
        // to 是终点，rev 是反向边在 e[to] 中的下标。
        int to, rev;
        // cap 是当前残量容量。
        T cap;
    };

    // n 是残量网络点数。
    int n;
    // e[u] 是点 u 发出的全部残量边。
    vector<vector<E>> e;
    // h 是层次图距离，cur[u] 是点 u 尚未尝试的第一条残量边下标。
    vector<int> h, cur;

    Flow(int n = 0) : n(n), e(n), h(n), cur(n)
    {
        // n 是残量网络点数；构造空网络。
    }

    int add(int x, int y, T c)
    {
        // x、y 是端点，c 是非负容量；返回正向边在 e[x] 中的下标。
        assert(0 <= x && x < n && 0 <= y && y < n && c >= 0); // 调试检查，可删。
        int id = e[x].size();
        int rev = e[y].size() + (x == y);
        e[x].push_back({y, rev, c});
        e[y].push_back({x, id, 0});
        return id;
    }

    bool bfs(int s, int t)
    {
        // s、t 是源汇点；重建层次图，存在增广路时返回 true。
        fill(h.begin(), h.end(), -1);
        queue<int> q;
        h[s] = 0;
        q.push(s);
        while (!q.empty())
        {
            int x = q.front();
            q.pop();
            for (auto [y, r, c] : e[x])
            {
                if (c > 0 && h[y] == -1)
                {
                    h[y] = h[x] + 1;
                    q.push(y);
                }
```

```
            }
        }
        return h[t] != -1;
    }

    T dfs(int x, int t, T f)
    {
        // x 是当前点，t 是汇点，f 是流量上限；返回沿层次图实际送出的流量。
        if (x == t)
        {
            return f;
        }
        T ans = 0;
        for (int &i = cur[x]; i < (int)e[x].size(); ++i)
        {
            auto &a = e[x][i];
            if (a.cap == 0 || h[a.to] != h[x] + 1)
            {
                continue;
            }
            T d = dfs(a.to, t, min(f - ans, a.cap));
            a.cap -= d;
            e[a.to][a.rev].cap += d;
            ans += d;
            if (ans == f)
            {
                break;
            }
        }
        return ans;
    }

    T flow(int s, int t, T lim = numeric_limits<T>::max())
    {
        // s、t 是源汇点，lim 是本次流量上限；返回现有残量网络上的新增流量。
        if (s == t)
        {
            return 0;
        }
        T ans = 0;
        while (ans < lim && bfs(s, t))
        {
            fill(cur.begin(), cur.end(), 0);
            ans += dfs(s, t, lim - ans);
        }
        return ans;
    }

    vector<bool> cut(int s) const
    {
        // s 是残量搜索起点；返回残量图中从 s 可达的点集。
        vector<bool> vis(n);
```

```
        queue<int> q;
        vis[s] = true;
        q.push(s);
        while (!q.empty())
        {
            int x = q.front();
            q.pop();
            for (auto [y, r, c] : e[x])
            {
                if (c > 0 && !vis[y])
                {
                    vis[y] = true;
                    q.push(y);
                }
            }
        }
        return vis;
    }
};
```

用 `Flow<T> fl(n)` 建 n 点网络（0 下标），`add(x, y, c)` 加容量非负的有向边并返回它在 `e[x]` 中的下标；自环也能正确维护反向边，但不会改变 s-t 最大流。`flow(s, t)` 返回本次在现有残量网络上新增的流，`s==t` 约定返回 0；`cut(s)` 返回当前残量图中从 s 可达的点集，只有最大流已经跑满时才对应最小割源侧。`T` 必须容纳总流量及残量加法；若每条容量是 `long long` 而容量总和可能越界，应直接用 `__int128`。同一对象再次调用 `flow` 会继续增广，独立求解需重建。Dinic 一般网络最坏 $O(V^2E)$，单位容量网络 $O(E\sqrt{E})$，二分图匹配网络 $O(E\sqrt{V})$，空间 $O(V+E)$。稠密图上可比较 5.8.002 FIFO 预流推进、5.8.009 ISAP 最大流 与 5.8.010 HLPP / 最高标号预流推进；边带费用改用 5.8.003 SPFA 多路增广最小费用流 或 5.8.004 原始对偶最小费用流；有上下界先用 5.8.005 有上下界可行流、最大流、最小流 转成普通网络；无指定源汇的全局最小割见 5.8.007 Stoer-Wagner 全局最小割。

Dinic、FIFO 预流推进、ISAP 与 HLPP 共用固定小图检查，见 5.8.010 HLPP / 最高标号预流推进 末尾。

5.8.002 FIFO 预流推进

预流推进（push-relabel）不寻找完整增广路，而允许中间点暂时有超额流：`ex[u]>0` 表示流入多于流出。高度 `h[u]` 限制流只能沿 `h[u]=h[v]+1` 的残量边推向邻点；没有允许边时就把 `u` 重标号到所有残量邻点的最低高度加一。源点高度初始化为 n 并饱和其出边，最终超额流要么退回源点，要么进入汇点。本实现按 FIFO 队列处理活跃点，配合当前弧 `cur`，最坏 $O(V^2E)$。它没有最高标号选择、全局重标号和 gap 启发式，因此不是 HLPP；标题与复杂度都按实际的 FIFO 预流推进书写。`PushRelabel` 与 5.8.001 Dinic 最大流 接口平行。

```
template <class T>
struct PushRelabel
{
    struct E
    {
        // 残量边终点与反向边下标。
        int v, r;
```

```
        // 当前残量容量。
        T c;
    };

    // 点数与残量邻接表。
    int n;
    vector<vector<E>> e;
    // 每个点的超额流。
    vector<T> ex;
    // 高度与当前弧。
    vector<int> h, cur;
    // 是否已在活跃队列中。
    vector<bool> in;
    // FIFO 活跃点队列。
    queue<int> q;

    PushRelabel(int n = 0) : n(n), e(n), ex(n), h(n), cur(n), in(n)
    {
        // n 是残量网络点数；构造空网络。
    }

    void add(int u, int v, T c)
    {
        // u、v 是端点，c 是非负容量；加入一对残量边。
        assert(0 <= u && u < n && 0 <= v && v < n && c >= 0); // 调试检查，可删。
        int id = e[u].size();
        int rev = e[v].size() + (u == v);
        e[u].push_back({v, rev, c});
        e[v].push_back({u, id, 0});
    }

    void active(int u, int s, int t)
    {
        // 超额为正且不是源汇时，把 u 加入活跃队列。
        if (u != s && u != t && ex[u] > 0 && !in[u])
        {
            in[u] = true;
            q.push(u);
        }
    }

    void push(int u, E &a, int s, int t)
    {
        // 若 a 是允许边，就从 u 沿 a 推出尽量多的超额流。
        if (!a.c || h[u] != h[a.v] + 1)
        {
            return;
        }
        T f = min(ex[u], a.c);
        a.c -= f;
        e[a.v][a.r].c += f;
        ex[u] -= f;
```

```
        ex[a.v] += f;
        active(a.v, s, t);
    }

    T flow(int s, int t)
    {
        // s、t 是源汇点；从初始网络求最大流并返回流量。
        if (s == t)
        {
            return 0;
        }
        h[s] = n;
        for (auto &a : e[s])
        {
            T f = a.c;
            a.c = 0;
            e[a.v][a.r].c += f;
            ex[a.v] += f;
            ex[s] -= f;
            active(a.v, s, t);
        }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            in[u] = false;
            while (ex[u] > 0)
            {
                if (cur[u] == (int)e[u].size())
                {
                    h[u] = 2 * n;
                    for (auto a : e[u])
                    {
                        if (a.c)
                        {
                            h[u] = min(h[u], h[a.v] + 1);
                        }
                    }
                    cur[u] = 0;
                    continue;
                }
                push(u, e[u][cur[u]], s, t);
                if (!e[u][cur[u]].c || h[u] != h[e[u][cur[u]].v] + 1)
                {
                    cur[u]++;
                }
            }
        }
        return ex[t];
    }
};
```

用 `PushRelabel<T> pr(n)` 建网络，`add(u, v, c)` 加有向边，`flow(s, t)` 返回最大流；`s==t` 约定返回 0，同一对象重复求解需重建。`T` 必须容纳总流量，容量总和可能超过 `long long` 时改用 `__int128`。FIFO 预流推进最坏 $O(V^2E)$，空间 $O(V+E)$。它代码短，稠密图上可与 Dinic、5.8.009 ISAP 最大流 和 5.8.010 HLPP / 最高标号预流推进 分别实测选型；需要最高标号、GAP 和全局重标号时应直接使用 HLPP。稀疏图和二分图匹配网络通常优先用 5.8.001 Dinic 最大流。带费用需求见 5.8.003 SPFA 多路增广最小费用流。

本实现与 Dinic、ISAP、HLPP 共用 5.8.010 HLPP / 最高标号预流推进 末尾的固定小图检查，不再重复驱动。

5.8.003 SPFA 多路增广最小费用流

最小费用流要求在发送指定流量（默认尽量发送最大流）的前提下，使 $\sum$(单位费用 $\times$ 流量) 最小。若当前流已是该流量下的最优解，沿残量图中的最短 $s \to t$ 路增广后，新流仍最优。残量反向边带负费用，因此本版每轮先用 SPFA 求最短距离 d，再把满足 $d_u + c_{uv} = d_v$ 的残量边称为紧边；从源点沿紧边用当前弧 DFS，一轮可同时推送多条等长最短路。每单位新增流的费用均为 d_t，所以批量增广仍保持最优性。零费用紧边可能成环，`vis` 只禁止当前递归路径重复点，下一轮 SPFA 会重建全部距离与当前弧。本实现常被笼统称作 ZKW 费用流，但它每轮重新运行 SPFA，没有使用经典 ZKW 的标号调整，故标题按实际过程命名。算法要求残量网络中不存在从 s 可达的负费用环。

```cpp
template <class T = long long, class Cost = long long>
struct SpfaCostFlow
{
    struct E
    {
        int to, rev;  // 终点与反向边下标。
        T cap;        // 当前残量容量。
        Cost cost;    // 单位流量费用。
    };

    int n;                // 残量网络点数。
    vector<vector<E>> e; // 每个点的残量边。
    // d 是本轮从源点出发的最短距离，空值表示不可达。
    vector<optional<Cost>> d;
    // in 标记 SPFA 队列中的点，cur 是紧边 DFS 的当前弧，vis 标记递归栈中的点。
    vector<int> in, cur, vis;

    SpfaCostFlow(int n) : n(n), e(n), d(n), in(n), cur(n), vis(n)
    {
        // n 是点数；构造空残量网络，无返回值。
    }

    void add(int u, int v, T cap, Cost cost)
    {
        // u、v 是端点，cap 是容量，cost 是单位费用；加入正向边和零容量反向边，无返回值。
        assert(0 <= u && u < n && 0 <= v && v < n && cap >= 0); // 调试检查，可删。
        assert(cost != numeric_limits<Cost>::min()); // 调试检查，可删。
        int id = e[u].size();
        int rev = e[v].size() + (u == v);
```

```
        e[u].push_back({v, rev, cap, cost});
        e[v].push_back({u, id, 0, -cost});
    }

    bool shortest(int s, int t)
    {
        // s、t 是源汇点；用 SPFA 建立残量最短距离，返回 t 是否可达。
        fill(d.begin(), d.end(), nullopt);
        fill(in.begin(), in.end(), 0);
        queue<int> q;
        d[s] = 0;
        q.push(s);
        in[s] = 1;
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            in[u] = 0;
            for (const auto &a : e[u])
            {
                if (a.cap == 0)
                {
                    continue;
                }
                Cost nd = *d[u] + a.cost;
                if (!d[a.to].has_value() || nd < *d[a.to])
                {
                    d[a.to] = nd;
                    if (!in[a.to])
                    {
                        q.push(a.to);
                        in[a.to] = 1;
                    }
                }
            }
        }
        return d[t].has_value();
    }

    T dfs(int u, int t, T lim)
    {
        // u、t 是当前点与汇点，lim 是流量上限；沿本轮紧边尽量增广并返回实际流量。
        if (u == t)
        {
            return lim;
        }
        vis[u] = 1;
        T f = 0;
        for (int &i = cur[u]; i < (int)e[u].size() && f < lim; i++)
        {
            auto &a = e[u][i];
            if (a.cap == 0 || vis[a.to] || !d[a.to].has_value() ||
```

```
                *d[u] + a.cost != *d[a.to])
            {
                continue;
            }
            T x = dfs(a.to, t, min(lim - f, a.cap));
            if (x == 0)
            {
                continue;
            }
            a.cap -= x;
            e[a.to][a.rev].cap += x;
            f += x;
        }
        vis[u] = 0;
        return f;
    }

    pair<T, Cost> flow(int s, int t, T lim = numeric_limits<T>::max())
    {
        // s、t 是源汇点，lim 是最多发送的流量；返回实际流量与最小费用。
        if (s == t)
        {
            return {0, 0};
        }
        assert(lim >= 0); // 调试检查，可删。
        T f = 0;
        Cost cost = 0;
        while (f < lim && shortest(s, t))
        {
            fill(cur.begin(), cur.end(), 0);
            fill(vis.begin(), vis.end(), 0);
            T x = dfs(s, t, lim - f);
            assert(x > 0); // 调试检查，可删：最短路存在时紧边 DFS 必须能增广。
            if (x == 0) break;
            f += x;
            cost += (Cost)x * *d[t];
        }
        return {f, cost};
    }
};
```

构造 `SpfaCostFlow<T,Cost> g(n)`，先用 `add(u,v,cap,cost)` 加完全部边，再调用 `flow`；容量与费用默认都是 `long long`。`flow(s,t,lim)` 最多发送 `lim`，返回 `{实际流量，最小费用}`。调用者必须保证反向费用、路径费用、单次乘积和总费用都可由 `Cost` 表示，且单位费用不等于 `numeric_limits<Cost>::min()`。`s==t` 约定返回 `{0,0}`，其余情况要求无源点可达的负费用环。同一边集上可以继续调用 `flow` 增广，但第一次求流后不要再加边。递归深度最坏为 $O(V)$；设 SPFA 轮数为 $K \le F$，其中 F 是整数容量下的最大流值，每轮 SPFA 最坏 $O(VE)$、紧边 DFS 按当前弧扫描 $O(E)$，故总时间最坏仍为 $O(FVE)$，空间为 $O(V+E)$。大量流能走费用相同的最短路时，K 可远小于 F；若最短费用大多互异或数据会卡 SPFA，则优先比较 5.8.004 原始对偶最小费用流。两版共用下一条末尾的检查。

5.8.004 原始对偶最小费用流

原始对偶法用 Johnson 势能把每轮最短路改成 Dijkstra。对残量边 (u, v) 定义约化费用 $c_{uv} + h_u - h_v$；若 h 是从源点出发的最短距离，则所有源点可达残量边的约化费用非负。首次调用先用一次 SPFA 初始化势能，所以允许负费用边，但仍要求不存在源点可达的负费用环。此后每轮 Dijkstra 求约化距离，并把可达点的距离累加到 `h`。势能保存在对象里，不会在下一次 `flow` 调用时清零，因此相同源汇可以分段继续增广。

```cpp
// 依赖 5.8.003 SPFA 多路增广最小费用流
template <class T = long long, class Cost = long long>
struct CostFlow
{
    struct E
    {
        // 终点与反向边下标。
        int to, rev;
        // 当前残量容量。
        T cap;
        // 单位流量费用。
        Cost cost;
    };

    // 点数、残量邻接表与跨调用保留的势能。
    int n;
    vector<vector<E>> e;
    vector<Cost> h;
    bool ready = false;
    int source = -1, sink = -1;

    CostFlow(int n = 0) : n(n), e(n), h(n)
    {
        // n 是残量网络点数；构造空网络。
    }

    void add(int x, int y, T cap, Cost cost)
    {
        // x、y 是端点，cap 是非负容量，cost 是单位费用；加入一对残量边。
        assert(0 <= x && x < n && 0 <= y && y < n && cap >= 0); // 调试检查，可删。
        assert(source == -1); // 调试检查，可删：所有边须在首次 flow 前加入。
        assert(cost != numeric_limits<Cost>::min()); // 调试检查，可删。
        int id = e[x].size();
        int rev = e[y].size() + (x == y);
        e[x].push_back({y, rev, cap, cost});
        e[y].push_back({x, id, 0, -cost});
        ready = false;
    }

    void init(int s)
    {
        // s 是源点；用 SPFA 求初始势能，要求不存在 s 可达的负费用环。
        vector<optional<Cost>> d(n);
```

```
        vector<bool> in(n);
        queue<int> q;
        d[s] = 0;
        q.push(s);
        in[s] = true;
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            in[u] = false;
            for (const auto &a : e[u])
            {
                Cost nd = *d[u] + a.cost;
                if (a.cap > 0 &&
                    (!d[a.to].has_value() || nd < *d[a.to]))
                {
                    d[a.to] = nd;
                    if (!in[a.to])
                    {
                        in[a.to] = true;
                        q.push(a.to);
                    }
                }
            }
        }
        fill(h.begin(), h.end(), 0);
        for (int u = 0; u < n; u++)
        {
            if (d[u].has_value())
            {
                h[u] = *d[u];
            }
        }
        ready = true;
    }

    pair<T, Cost> flow(int s, int t, T lim = numeric_limits<T>::max())
    {
        // s、t 是固定源汇，lim 是流量上限；返回本次新增流量与费用。
        if (s == t)
        {
            return {0, 0};
        }
        assert(lim >= 0); // 调试检查，可删。
        if (source == -1)
        {
            source = s;
            sink = t;
        }
        assert(source == s && sink == t); // 调试检查，可删：分段调用不能更换源汇。
        if (!ready)
        {
```

```
            init(s);
        }

        vector<int> pv(n), pe(n);
        T f = 0;
        Cost cost = 0;
        while (f < lim)
        {
            vector<optional<Cost>> dis(n);
            priority_queue<pair<Cost, int>, vector<pair<Cost, int>>, greater<>> q;
            dis[s] = 0;
            q.emplace(0, s);
            while (!q.empty())
            {
                auto [d, x] = q.top();
                q.pop();
                if (!dis[x].has_value() || d != *dis[x])
                {
                    continue;
                }
                for (int i = 0; i < (int)e[x].size(); ++i)
                {
                    auto &a = e[x][i];
                    if (a.cap == 0)
                    {
                        continue;
                    }
                    Cost nd = d + a.cost + h[x] - h[a.to];
                    assert(nd >= 0); // 调试检查，可删：势能应保证约化费用非负。
                    if (!dis[a.to].has_value() || nd < *dis[a.to])
                    {
                        dis[a.to] = nd;
                        pv[a.to] = x;
                        pe[a.to] = i;
                        q.emplace(nd, a.to);
                    }
                }
            }
            if (!dis[t].has_value())
            {
                break;
            }
            for (int i = 0; i < n; ++i)
            {
                if (dis[i].has_value())
                {
                    h[i] += *dis[i];
                }
            }
            T d = lim - f;
            for (int x = t; x != s; x = pv[x])
            {
```

```
                d = min(d, e[pv[x]][pe[x]].cap);
            }
            for (int x = t; x != s; x = pv[x])
            {
                auto &a = e[pv[x]][pe[x]];
                a.cap -= d;
                e[x][a.rev].cap += d;
            }
            f += d;
            cost += (Cost)d * (h[t] - h[s]);
        }
        return {f, cost};
    }
};
```

构造 `CostFlow<T,Cost> g(n)`，先用 `add(x,y,cap,cost)` 加完全部边，容量与费用默认都是 `long long`；`flow(s,t,lim)` 返回本次新增的 `{流量，最小费用}`。调用者必须保证反向费用、距离、势能、单次乘积和总费用都可由 `Cost` 表示，且单位费用不等于 `numeric_limits<Cost>::min()`。允许负费用边但不允许源点可达的负费用环；同一对象可以对相同 (s,t) 分段调用，不能中途更换源汇或继续加边。首次 SPFA 为 $O(VE)$，之后每次增广的 Dijkstra 为 $O(E\log V)$；整数容量下增广次数至多为最大流值 F，总复杂度 $O(VE+FE\log V)$，空间 $O(V+E)$。本版避免每轮 SPFA，最短费用大多互异或构造数据上更稳；它每轮只恢复一条路径，在大量等费用最短路并存时可能慢于 5.8.003 SPFA 多路增广最小费用流。两版各自核对下一段固定网络的已知答案；只求最大流用 5.8.001 Dinic 最大流。

本地性能检查使用 Windows 10、Ryzen 9 7945HX、GCC 14.2.0 与 `-O2 -pipe`，每个场景先预热一次，再重复 5 或 7 次取中位数，并把整个进程重复三轮。120×120 全零费用指派图上，多路 SPFA 为 0.486–0.541 ms，原始对偶为 4.540–4.714 ms；宽度 500、每点 8 条等费用边的分层图上，两者分别为 0.333–0.343 ms 与 14.016–14.803 ms。随机费用指派图上两者分别为 7.316–7.657 ms 与 8.122–8.297 ms，差距很小。这些结果只说明同费用最短路的批量增广收益，不替代 SPFA 与 Dijkstra 的最坏复杂度比较；

下面直接检查一张含负费用边的小图。最大流为 3，最小费用为 9；原始对偶版分两次调用时，第一单位费用为 0，余下两单位费用为 9。

```
template <class F>
void addFlowCase(F &g)
{
    // g 是四点费用流网络；加入含负费用边的固定样例。
    g.add(0, 1, 2, 1);
    g.add(0, 2, 1, 4);
    g.add(1, 2, 1, -2);
    g.add(1, 3, 1, 3);
    g.add(2, 3, 2, 1);
}

signed main()
{
    SpfaCostFlow<> a(4);
    addFlowCase(a);
    assert((a.flow(0, 3) == pair<long long, long long>{3, 9}));

```

```
    CostFlow<> b(4);
    addFlowCase(b);
    assert((b.flow(0, 3, 1) == pair<long long, long long>{1, 0}));
    assert((b.flow(0, 3) == pair<long long, long long>{2, 9}));
    return 0;
}
```

5.8.005 有上下界可行流、最大流、最小流

每条边带流量下界 l 与上界 r，先问是否存在满足所有 $l \le f_e \le r$ 与流量守恒的可行流，再在此基础上求指定源汇的最大流或最小流。标准做法是「拆下界」：让每条边先强制流 l，剩余可调容量为 $r-l$；强制的这部分会破坏守恒，于是给每个点算净盈亏 $d_v = \sum_{\text{入}} l - \sum_{\text{出}} l$，再建超级源 ss、超级汇 tt，对 $d_v > 0$ 的点连 $ss \to v$ 容量 d_v、$d_v < 0$ 的点连 $v \to tt$ 容量 $-d_v$。当 $ss \to tt$ 的最大流恰好灌满所有超级边（等于 $\sum_{d_v>0} d_v$）时可行流存在，每条原边的实际流量为 $l+$（其残量边反向容量）。若还要求原网络 $s \to t$ 的最大/最小流，先加一条 $t \to s$ 容量无穷的边把有源汇问题转成无源汇可行流问题，判可行后拆掉这条边，在残量网络上再对 $s \to t$（最大流）或 $t \to s$（最小流，从可行值里减去）增广。内部复用 5.8.001 Dinic 最大流 的 `Flow` 模板，此处不重复打印。

```
template <class Cap = long long>
struct BoundResult
{
    // 指定源汇时的流值；只求循环可行流时为 0。
    Cap value;
    // 每条原边的实际流量。
    vector<Cap> edge;
};

template <class Cap = long long>
struct BoundFlow
{
    struct E
    {
        // 端点与流量上下界。
        int u, v;
        Cap l, r;
    };

    // 点数与原图边集。
    int n;
    vector<E> e;

    BoundFlow(int n = 0) : n(n)
    {
        // n 是原图点数；构造空网络。
    }

    void add(int u, int v, Cap l, Cap r)
    {
        // u、v 是端点，l、r 是非负流量下界和上界。
```

```
        assert(0 <= u && u < n && 0 <= v && v < n && 0 <= l && l <= r); // 调试检查，可删。
        e.push_back({u, v, l, r});
    }

    optional<BoundResult<Cap>> solve(int s = -1, int t = -1, bool minimum = false) const
    {
        // s=t=-1 求循环可行流；否则 minimum 决定求最小流还是最大流。
        assert((s == -1 && t == -1) ||
               (0 <= s && s < n && 0 <= t && t < n && s != t)); // 调试检查，可删。
        int ss = n, tt = n + 1;
        Flow<Cap> f(n + 2);
        vector<Cap> d(n);
        vector<pair<int, int>> id;
        Cap sumR = 0;
        for (const auto &x : e)
        {
            id.push_back({x.u, f.add(x.u, x.v, x.r - x.l)});
            d[x.u] -= x.l;
            d[x.v] += x.l;
            sumR += x.r;
        }
        int art = -1;
        if (s != -1)
        {
            art = f.add(t, s, sumR + 1);
        }
        Cap need = 0;
        for (int i = 0; i < n; i++)
        {
            if (d[i] > 0)
            {
                f.add(ss, i, d[i]);
                need += d[i];
            }
            else if (d[i] < 0)
            {
                f.add(i, tt, -d[i]);
            }
        }
        if (f.flow(ss, tt) != need)
        {
            return nullopt;
        }
        Cap val = 0;
        if (s != -1)
        {
            auto &a = f.e[t][art];
            val = f.e[s][a.rev].cap;
            a.cap = 0;
            f.e[s][a.rev].cap = 0;
        }
        for (int u : {ss, tt})
```

```
        {
            for (auto &a : f.e[u])
            {
                f.e[a.to][a.rev].cap = 0;
                a.cap = 0;
            }
        }
        if (s != -1)
        {
            if (minimum)
            {
                val -= f.flow(t, s, val);
            }
            else
            {
                val += f.flow(s, t);
            }
        }
        vector<Cap> ans(e.size());
        for (int i = 0; i < (int)e.size(); i++)
        {
            auto [u, j] = id[i];
            auto &a = f.e[u][j];
            ans[i] = e[i].l + f.e[a.to][a.rev].cap;
        }
        return BoundResult<Cap>{val, ans};
    }
};
```

代码依赖 5.8.001 Dinic 最大流 的 `Flow`。构造 `BoundFlow<Cap> g(n)`，`Cap` 默认是 `long long`，`add(u,v,l,r)` 加一条同类型上下界边。`solve()` 求循环可行流；`solve(s,t,false)` 求非负 $s \to t$ 最大流，`solve(s,t,true)` 求非负最小流。只在无可行流时返回空，否则 `.value` 是流值，`.edge[i]` 是第 i 条原边的流量。调用者必须保证平衡量、所有上界之和、流值和边流可由 `Cap` 表示；人工边容量取所有上界之和加一，不使用可能偏小的固定“无穷”。复杂度为两次 Dinic 的量级，最坏 $O(V^2E)$，空间 $O(V+E)$。带费用的上下界流还需累计下界的固定费用，并把增广器换成 5.8.003 SPFA 多路增广最小费用流 或 5.8.004 原始对偶最小费用流。

5.8.006 最大流最小割与建图模型

最大流最小割定理（s-t 最大流等于最小割容量）让一大类「每个对象二选一、带一元代价与二元关系代价」的最小化问题变成求最小割。约定割后与 s 同侧记为取 1、与 t 同侧记为取 0。三种常用建模砖块：一元代价——变量 u 取 0 花 c_0、取 1 花 c_1，连 $s \to u$ 容量 c_0、$u \to t$ 容量 c_1；蕴含约束 $u{=}1 \Rightarrow v{=}1$——连 $u \to v$ 硬约束边；异值代价——u, v 取值不同时付 c，连两条方向相反、容量均为 c 的边。`BinaryCut` 只保存这些建模项，求解时统一调用 5.8.001 Dinic 最大流，硬约束容量自动取全部有限容量之和加一。

```
// 依赖 5.8.001 Dinic 最大流
template <class Cap = long long>
struct BinaryCut
```

```
{
    // 变量数与内部源汇编号。
    int n, s, t;
    // 有限容量边、硬约束边及全部有限容量之和。
    vector<tuple<int, int, Cap>> edge;
    vector<pair<int, int>> hard;
    Cap sum = 0;

    BinaryCut(int n) : n(n), s(n), t(n + 1)
    {
        // n 是二元变量数；构造空模型。
    }

    void addCost(int u, Cap c0, Cap c1)
    {
        // c0、c1 是 u 取 0、1 的非负代价。
        assert(0 <= u && u < n && c0 >= 0 && c1 >= 0); // 调试检查，可删。
        edge.push_back({s, u, c0});
        edge.push_back({u, t, c1});
        sum += c0 + c1;
    }

    void imply(int u, int v)
    {
        // 加入 u=1 推出 v=1 的硬约束。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        hard.push_back({u, v});
    }

    void addDiffCost(int u, int v, Cap c)
    {
        // 二者取值不同时付非负代价 c。
        assert(0 <= u && u < n && 0 <= v && v < n && c >= 0); // 调试检查，可删。
        edge.push_back({u, v, c});
        edge.push_back({v, u, c});
        sum += c + c;
    }

    pair<Cap, vector<int>> solve() const
    {
        // 返回最小代价与一组取值；源点侧变量取 1。
        Cap inf = sum + 1;
        Flow<Cap> fl(n + 2);
        for (auto [u, v, c] : edge)
        {
            fl.add(u, v, c);
        }
        for (auto [u, v] : hard)
        {
            fl.add(u, v, inf);
        }
        Cap ans = fl.flow(s, t);
```

```
        vector<bool> side = fl.cut(s);
        vector<int> val(n);
        for (int u = 0; u < n; u++)
        {
            val[u] = side[u];
        }
        return pair<Cap, vector<int>>{ans, val};
    }
};
```

构造 `BinaryCut<Cap> q(n)`，`Cap` 默认是 `long long`；`addCost(u,c0,c1)` 加一元代价，`imply(u,v)` 加 $u{=}1 \Rightarrow v{=}1$，`addDiffCost(u,v,c)` 加异值代价，有限代价都使用 `Cap`。`solve()` 返回 `{最小代价，取值向量}`，可重复调用。调用者必须保证全部有限代价之和加一可由 `Cap` 表示；这个和加一就是硬约束容量，不依赖全局 `INF`。例如两变量代价为 $(3,0)$、$(0,2)$，再加 `imply(0,1)`，最优值为 2，取值为 $\{1,1\}$。Dinic 最坏 $O(V^2E)$，空间 $O(V+E)$。最大权闭合子图见 5.6.011 最大权闭合子图，全局最小割见 5.8.007 Stoer-Wagner 全局最小割，所有点对最小割见 5.8.008 Gomory-Hu 最小割树。

5.8.007 Stoer-Wagner 全局最小割

求无向带权图的全局最小割——把点集分成非空两部分、被切断的边权和最小，且不指定哪两个点必须分开。若枚举源汇跑 $\binom{n}{2}$ 次最大流太慢。Stoer–Wagner 用「最大邻接序」把它压到 $O(n^3)$：每一阶段从空集出发，反复把「与当前已选集合边权和最大」的点加入，直到所有点加完；记最后加入的点为 t、倒数第二个为 s，则 t 到已选集合的边权和恰是一个「s-t 最小割」的值（最小割阶段引理）。用它更新全局答案后，把 s、t 合并成一个点（边权相加）继续下一阶段，$n-1$ 阶段后剩一个点，期间记录的最小割即全局最小割。`GlobalCut.side` 保存取到最小值那一刻 t 所代表的原始点集合，即最小割的一侧。

```
struct GlobalCut
{
    // 最小割值与割的一侧。
    int w;
    vector<int> side;
};

GlobalCut stoerWagner(vector<vector<int>> g)
{
    // g 是无向图的对称非负权矩阵；返回全局最小割值和一侧点集。
    int n = g.size();
    if (n <= 1)
    {
        return {0, n ? vector<int>{0} : vector<int>{}};
    }
    for (int i = 0; i < n; i++)
    {
        assert((int)g[i].size() == n); // 调试检查，可删。
        for (int j = 0; j < n; j++)
        {
            assert(g[i][j] >= 0 && g[i][j] == g[j][i]); // 调试检查，可删。
        }
```

```
        g[i][i] = 0;
    }
    vector<int> v(n);
    vector<vector<int>> part(n);
    iota(v.begin(), v.end(), 0);
    for (int i = 0; i < n; i++)
    {
        part[i] = {i};
    }
    GlobalCut ans{numeric_limits<int>::max(), {}};
    while (v.size() > 1)
    {
        vector<int> w(n);
        vector<bool> used(n);
        int s = -1, t = -1;
        for (int z = 0; z < (int)v.size(); z++)
        {
            s = t;
            t = -1;
            for (int u : v)
            {
                if (!used[u] && (t < 0 || w[u] > w[t]))
                {
                    t = u;
                }
            }
            if (z + 1 == (int)v.size())
            {
                if (w[t] < ans.w)
                {
                    ans = {w[t], part[t]};
                }
                for (int u : v)
                {
                    if (u != s)
                    {
                        g[s][u] += g[t][u];
                        g[u][s] = g[s][u];
                    }
                }
                part[s].insert(part[s].end(), part[t].begin(), part[t].end());
                v.erase(find(v.begin(), v.end(), t));
                break;
            }
            used[t] = true;
            for (int u : v)
            {
                w[u] += g[t][u];
            }
        }
    }
    return ans;
```

```
}
```

`stoerWagner(g)` 传入 $n \times n$ 对称非负权矩阵，无边填 0，对角线会被忽略；返回的 `w` 是全局最小割值，`side` 是割的一侧。参数按值传递，算法会直接在副本上收缩；调用者必须保证任意收缩边权和可由 64 位有符号整数表示。$n \le 1$ 时不存在非平凡割，约定返回 0。时间 $O(n^3)$、空间 $O(n^2)$。第 5.3 节只引用本条，不再保留第二份实现；指定源汇最小割用 5.8.001 Dinic 最大流，所有点对最小割用 5.8.008 Gomory-Hu 最小割树，二元建模见 5.8.006 最大流最小割与建图模型。

下面检查四点带权图与不连通图。第一张图的全局最小割为孤立点 3，割值是 4；第二张图已有两个连通块，割值是 0。

```
signed main()
{
    vector<vector<int>> g{{0, 3, 2, 0},
                          {3, 0, 4, 1},
                          {2, 4, 0, 3},
                          {0, 1, 3, 0}};
    auto a = stoerWagner(g);
    assert(a.w == 4 && !a.side.empty() && a.side.size() < 4);

    vector<vector<int>> h{{0, 2, 0}, {2, 0, 0}, {0, 0, 0}};
    assert(stoerWagner(h).w == 0);
}
```

5.8.008 Gomory-Hu 最小割树

无向图有 $\binom{n}{2}$ 个点对最小割值，Gomory-Hu 树用 $n-1$ 条带权树边编码它们：树上 u, v 路径的最小边权等于原图 u-v 最小割。因此不同割值至多有 $n-1$ 个，并不保证树边权两两不同。构造只需 $n-1$ 次最大流。这里用 Gusfield 简化版：初始父点均为 0，依次求 i 到 `p[i]` 的最小割，再按残量源侧修正父指针。每轮都重建 5.8.001 Dinic 最大流 的网络。

```
// 依赖 5.8.001 Dinic 最大流
template <class Cap = long long>
vector<tuple<int, int, Cap>>
gomoryHu(int n, const vector<tuple<int, int, long long>> &edges)
{
    // edges 是非负权无向边；返回 Gomory-Hu 树边及割值。
    vector<int> p(n);
    vector<Cap> w(n);
    for (int s = 1; s < n; s++)
    {
        int t = p[s];
        Flow<Cap> f(n);
        for (auto [u, v, c] : edges)
        {
            assert(0 <= u && u < n && 0 <= v && v < n && c >= 0); // 调试检查，可删。
            f.add(u, v, (Cap)c);
            f.add(v, u, (Cap)c);
        }
```

```
        Cap z = f.flow(s, t);
        auto cut = f.cut(s);
        for (int i = s + 1; i < n; i++)
        {
            if (p[i] == t && cut[i])
            {
                p[i] = s;
            }
        }
        if (t && cut[p[t]])
        {
            p[s] = p[t];
            p[t] = s;
            w[s] = w[t];
            w[t] = z;
        }
        else
        {
            w[s] = z;
        }
    }
    vector<tuple<int, int, Cap>> ans;
    for (int i = 1; i < n; i++)
    {
        ans.push_back({i, p[i], w[i]});
    }
    return ans;
}
```

`gomoryHu<Cap>(n,edges)` 中 `Cap` 默认是 `long long`，平行边可直接重复加入；返回 $n-1$ 条树边 `(点，父点，割值)`。调用者必须保证任意点对最小割可由 `Cap` 表示。任意两点最小割等于树上路径最小边权，可再配合 6.1.001 LCA / 倍增与 ST 表 查询。算法调用 $n-1$ 次最大流；Dinic 最坏 $O(n^3m)$，空间 $O(n+m)$。只查一个点对时直接用 5.8.001 Dinic 最大流，只要所有点对中的最小值用 5.8.007 Stoer-Wagner 全局最小割。

5.8.009 ISAP 最大流

ISAP（Improved Shortest Augmenting Path）是 Dinic 之外的另一支高效最大流实现，很多稠密图上常数更小。它给每个点维护到汇点 t 的距离标号 `h[]`（先反向 BFS 从 t 初始化），DFS 只沿「$h_x = h_y + 1$」的允许边推流；当前点走不通时不重跑全局 BFS，而是局部重标号——把 h_x 抬到「所有仍有残量的出边终点 h 的最小值加一」，这样标号单调递增、总重标号次数 $O(V^2)$。关键加速是 GAP 优化：`gap[d]` 记距离标号恰为 d 的点数，一旦某层被抬空（`--gap[h[x]]==0`），说明 s 与 t 之间出现了断层、不可能再有增广路，立即把源点标号设为 n 提前终止整个算法。

```
template <class T>
struct ISAP
{
    struct E
    {
```

```
        int to, rev; // 终点与反向边下标。
        T cap;       // 当前残量容量。
    };

    int n, src = 0;       // n 是点数，src 是本次增广的源点。
    vector<vector<E>> e; // e[u] 是点 u 发出的全部残量边。
    vector<int> h, gap;  // h 是到汇点的距离标号，gap[d] 是标号为 d 的点数。
    vector<int> cur;     // cur[u] 是点 u 尚未尝试的第一条残量边下标。

    ISAP(int n = 0) : n(n), e(n), h(n), gap(n + 1), cur(n)
    {
        // n 是点数；构造空残量网络。
    }

    void add(int x, int y, T c)
    {
        // x、y 是端点，c 是非负容量；加入一对残量边。
        assert(0 <= x && x < n && 0 <= y && y < n && c >= 0); // 调试检查，可删。
        int id = e[x].size();
        int rev = e[y].size() + (x == y);
        e[x].push_back({y, rev, c});
        e[y].push_back({x, id, 0});
    }

    void bfs(int t)
    {
        // t 是汇点；在反向残量意义下初始化距离标号和 gap 计数，无返回值。
        fill(h.begin(), h.end(), n);
        fill(gap.begin(), gap.end(), 0);
        queue<int> q;
        h[t] = 0;
        q.push(t);
        while (!q.empty())
        {
            auto x = q.front();
            q.pop();
            for (auto [y, rev, cap] : e[x])
            {
                if (h[y] != n || e[y][rev].cap == 0)
                {
                    continue;
                }
                h[y] = h[x] + 1;
                q.push(y);
            }
        }
        for (auto d : h)
        {
            gap[d]++;
        }
    }

```

```
    T dfs(int x, int t, T lim)
    {
        // x、t、lim 是当前点、汇点和流量上限；返回本次实际增广量。
        if (x == t)
        {
            return lim;
        }
        T used = 0;
        for (auto &i = cur[x]; i < (int)e[x].size(); i++)
        {
            auto &a = e[x][i];
            if (a.cap == 0 || h[x] != h[a.to] + 1)
            {
                continue;
            }
            T d = dfs(a.to, t, min(lim - used, a.cap));
            a.cap -= d;
            e[a.to][a.rev].cap += d;
            used += d;
            if (used == lim || h[x] == n)
            {
                return used;
            }
        }
        if (--gap[h[x]] == 0)
        {
            h[src] = n;
            return used;
        }
        int nh = n - 1;
        for (const auto &a : e[x])
        {
            if (a.cap > 0)
            {
                nh = min(nh, h[a.to]);
            }
        }
        h[x] = nh + 1;
        gap[h[x]]++;
        cur[x] = 0;
        return used;
    }

    T flow(int s, int t)
    {
        // s、t 是源汇点；返回现有残量上新增的最大流。
        if (s == t)
        {
            return 0;
        }
        src = s;
        bfs(t);
```

```
        fill(cur.begin(), cur.end(), 0);
        T ans = 0;
        while (h[s] < n && ans < numeric_limits<T>::max())
        {
            ans += dfs(s, t, numeric_limits<T>::max() - ans);
        }
        return ans;
    }
};
```

构造 `ISAP<T> is(n)`，`add(u, v, c)` 加非负容量边，`flow(s, t)` 返回现有残量网络上新增的最大流；`s==t` 约定返回 0。`T` 必须容纳总流量，必要时用 `__int128`。最坏 $O(V^2E)$、空间 $O(V+E)$。它与 5.8.001 Dinic 最大流 同解，稠密图上也可比较 5.8.002 FIFO 预流推进 与 5.8.010 HLPP / 最高标号预流推进。带费用需求见 5.8.003 SPFA 多路增广最小费用流。本类没有 `cut(s)`，需要割侧点集时使用 `Flow`。

5.8.010 HLPP / 最高标号预流推进

HLPP（Highest Label Push-Relabel）也是预流推进，但活跃点总按当前最高标号处理。实现用当前弧 `cur`、按标号组织的活跃点桶、GAP 链表和全局重标号：最高标号桶避免 `priority_queue` 的对数开销，GAP 在某个标号层变空时把更高层一次标成不可达，全局重标号则按反向残量边 BFS 重建到汇点的精确距离。`flow(s,t)` 从当前加入的容量网络求一次最大流，`s==t` 约定返回 0；结束时可能仍保留无法到达汇点的预流超额，因此同一对象不得再次调用 `flow`。容量类型 `T` 必须为能表示非负容量和总流量的有符号整数。

```
// 依赖 5.8.001 Dinic 最大流
// 依赖 5.8.002 FIFO 预流推进
// 依赖 5.8.009 ISAP 最大流
template <class T = long long>
struct HLPP
{
    struct E
    {
        // to 是终点，rev 是反向边在 e[to] 中的下标。
        int to, rev;
        // cap 是当前残量容量。
        T cap;
    };

    // n 是点数，src、sink 是最近一次求流使用的源汇点。
    int n, src = -1, sink = -1;
    // e[u] 是点 u 发出的全部残量边。
    vector<vector<E>> e;
    // ex 是超额流，h 是标号，cur 是当前弧。
    vector<T> ex;
    vector<int> h, cur;
    // actHead/actNext 按标号维护活跃点链表，in 标记是否已经入链。
    vector<int> actHead, actNext;
    vector<char> in;
    // gapHead/gapPrev/gapNext 按标号维护全部顶点链表。
```

```
    vector<int> gapHead, gapPrev, gapNext;
    int maxH = -1, work = 0;
    bool used = false; // used 标记是否已经调用过 flow。

    HLPP(int n = 0)
        : n(n), e(n), ex(n), h(n), cur(n), actHead(n, -1), actNext(n, -1), in(n),
          gapHead(n, -1), gapPrev(n, -1), gapNext(n, -1)
    {
        // n 是残量网络点数；构造空网络。
    }

    void add(int u, int v, T c)
    {
        // u、v 是端点，c 是非负容量；加入一对残量边。
        assert(0 <= u && u < n && 0 <= v && v < n && c >= 0); // 调试检查，可删。
        int id = e[u].size();
        int rev = e[v].size() + (u == v);
        e[u].push_back({v, rev, c});
        e[v].push_back({u, id, 0});
    }

    void insertGap(int u)
    {
        // 把标号小于 n 的点 u 插入对应 GAP 链表。
        int d = h[u];
        gapPrev[u] = -1;
        gapNext[u] = gapHead[d];
        if (gapHead[d] != -1)
        {
            gapPrev[gapHead[d]] = u;
        }
        gapHead[d] = u;
    }

    void removeGap(int u)
    {
        // 从当前标号链表移除点 u。
        int d = h[u];
        if (gapPrev[u] == -1)
        {
            gapHead[d] = gapNext[u];
        }
        else
        {
            gapNext[gapPrev[u]] = gapNext[u];
        }
        if (gapNext[u] != -1)
        {
            gapPrev[gapNext[u]] = gapPrev[u];
        }
        gapPrev[u] = gapNext[u] = -1;
    }
```

```
78
79      void enqueue(int u)
80      {
81          // 把有超额且可达汇点的非源汇点加入最高标号活跃链表。
82          if (u == src || u == sink || ex[u] <= 0 || h[u] >= n || in[u])
83          {
84              return;
85          }
86          in[u] = true;
87          actNext[u] = actHead[h[u]];
88          actHead[h[u]] = u;
89          maxH = max(maxH, h[u]);
90      }
91
92      void relabelAll()
93      {
94          // 从汇点按反向残量边 BFS，重建标号、GAP 链表与活跃点链表。
95          fill(h.begin(), h.end(), n);
96          fill(cur.begin(), cur.end(), 0);
97          fill(actHead.begin(), actHead.end(), -1);
98          fill(actNext.begin(), actNext.end(), -1);
99          fill(in.begin(), in.end(), false);
100         fill(gapHead.begin(), gapHead.end(), -1);
101         fill(gapPrev.begin(), gapPrev.end(), -1);
102         fill(gapNext.begin(), gapNext.end(), -1);
103         maxH = -1;
104         queue<int> q;
105         h[sink] = 0;
106         q.push(sink);
107         while (!q.empty())
108         {
109             int u = q.front();
110             q.pop();
111             for (const auto &a : e[u])
112             {
113                 int v = a.to;
114                 if (v ≠ src && h[v] == n && e[v][a.rev].cap > 0)
115                 {
116                     h[v] = h[u] + 1;
117                     q.push(v);
118                 }
119             }
120         }
121         for (int u = 0; u < n; u++)
122         {
123             if (h[u] < n)
124             {
125                 insertGap(u);
126             }
127         }
128         for (int u = 0; u < n; u++)
129         {
```

```
            enqueue(u);
        }
        work = 0;
    }

    void gapRelabel(int old)
    {
        // old 层为空时，所有更高标号点都不可达汇点，统一标为 n。
        for (int d = old + 1; d < n; d++)
        {
            int u = gapHead[d];
            gapHead[d] = -1;
            while (u != -1)
            {
                int v = gapNext[u];
                h[u] = n;
                gapPrev[u] = gapNext[u] = -1;
                if (in[u])
                {
                    in[u] = false;
                    actNext[u] = -1;
                }
                u = v;
            }
            actHead[d] = -1;
        }
        maxH = min(maxH, old - 1);
    }

    void relabel(int u)
    {
        // 当前弧耗尽时把 u 改到残量邻点的最低标号加一。
        int old = h[u], nh = n;
        for (const auto &a : e[u])
        {
            if (a.cap > 0)
            {
                nh = min(nh, h[a.to] + 1);
            }
        }
        removeGap(u);
        bool gap = gapHead[old] == -1;
        if (gap)
        {
            gapRelabel(old);
        }
        h[u] = gap ? n : nh;
        cur[u] = 0;
        if (h[u] < n)
        {
            insertGap(u);
        }
```

```
        work++;
    }

    void push(int u, E &a)
    {
        // 沿允许边从 u 推出尽量多的超额流。
        T f = min(ex[u], a.cap);
        a.cap -= f;
        e[a.to][a.rev].cap += f;
        ex[u] -= f;
        ex[a.to] += f;
        enqueue(a.to);
    }

    void discharge(int u)
    {
        // 反复推流或重标号，直到 u 没有超额或已不可达汇点。
        while (ex[u] > 0 && h[u] < n)
        {
            if (cur[u] == (int)e[u].size())
            {
                relabel(u);
                continue;
            }
            E &a = e[u][cur[u]];
            if (a.cap == 0 || h[u] != h[a.to] + 1)
            {
                cur[u]++;
                continue;
            }
            push(u, a);
        }
    }

    T flow(int s, int t)
    {
        // s、t 是源汇点；首次调用时返回最大流。
        assert(0 <= s && s < n && 0 <= t && t < n); // 调试检查，可删。
        assert(!used); // 调试检查，可删。
        used = true;
        if (s == t)
        {
            return 0;
        }
        src = s;
        sink = t;
        ex.assign(n, 0);
        relabelAll();
        for (auto &a : e[s])
        {
            if (a.to == s || a.cap == 0)
            {
```

```
                continue;
            }
            T f = a.cap;
            a.cap = 0;
            e[a.to][a.rev].cap += f;
            ex[s] -= f;
            ex[a.to] += f;
            enqueue(a.to);
        }
        while (maxH >= 0)
        {
            if (actHead[maxH] == -1)
            {
                maxH--;
                continue;
            }
            int u = actHead[maxH];
            actHead[maxH] = actNext[u];
            actNext[u] = -1;
            in[u] = false;
            discharge(u);
            if (ex[u] > 0 && h[u] < n)
            {
                enqueue(u);
            }
            if (work >= 4 * max(1LL, n))
            {
                relabelAll();
            }
        }
        return ex[t];
    }
};
```

构造 `HLPP<T> hp(n)`，`add(u, v, c)` 加非负容量边，`flow(s, t)` 一次性返回最大流，同一对象不得再次调用；`s==t` 约定返回 0。最高标号预流推进最坏 $O(V^2E)$、空间 $O(V+E)$；数组桶、字节标志、当前弧、GAP 和约每 $4V$ 次重标号进行一次全局重标号，是本版保留的常数优化。它通常优先用于稠密网络或预流推进确有优势的场景；稀疏图、需要流量上限或割侧查询时优先使用 5.8.001 Dinic 最大流，需要 FIFO 调度的短实现使用 5.8.002 FIFO 预流推进，想保留距离标号与 GAP 但不需要活跃桶时使用 5.8.009 ISAP 最大流。HLPP 的全局重标号和 GAP 都维护同一份残量状态，不能把它们删掉后仍称为 HLPP。

下面让 Dinic、FIFO 预流推进、ISAP 与 HLPP 分别检查同一张已知答案为 5 的四点图；它们各自核对固定答案，不相互比较。

```
template <class F>
void testMaxFlow()
{
    // F 是待检查的最大流结构；直接核对一张固定图的答案。
    F f(4);
    f.add(0, 1, 3);
    f.add(0, 2, 2);
```

```
    f.add(1, 2, 1);
    f.add(1, 3, 2);
    f.add(2, 3, 4);
    assert(f.flow(0, 3) == 5);
}

signed main()
{
    testMaxFlow<Flow<long long>>();
    testMaxFlow<PushRelabel<long long>>();
    testMaxFlow<ISAP<long long>>();
    testMaxFlow<HLPP<long long>>();
    return 0;
}
```

5.8.011 费用缩放最小费用 b 流与带费用上下界

把每条边流量限制写成 $l_e \le f_e \le r_e$，并把固定的下界先放进节点超额 `ex`：边 $u \to v$ 的下界使 `ex[u]-=l`、`ex[v]+=l`。费用缩放维护约化费用 $c_{uv}+p_u-p_v$；每一轮先把所有负约化费用残量边饱和，再沿负约化费用边消除正超额，没有可用边时下调该点势能。把费用乘以 $n+1$ 后缩放到 $\varepsilon=1$，任意长度不超过 n 的负环都不可能残留，因此得到最小费用可行 b 流；负费用边和负环不需要额外假设。

```
template <class Cap = long long, class Cost = long long, class Ans = long long>
struct MinCostBFlow
{
    struct Edge
    {
        int to, rev; // to、rev 是终点与反向边下标。
        Cap cap;     // cap 是当前残量容量。
        Cost w;      // w 是乘以 n+1 后的费用。
    };

    struct Ref
    {
        int u, i;    // u、i 定位加入的正向边。
        Cap low;     // low 是原边下界。
        Cost cost;   // cost 是原单位费用。
    };
    struct Certificate
    {
        // value 是最小费用，pot 是满足互补松弛的点势，flow 按 add 顺序给出每条边流量。
        Ans value;
        vector<Cost> pot;
        vector<Cap> flow;
    };

    int n;                      // n 是点数。
    vector<vector<Edge>> g;     // g 是残量网络。
    vector<Cap> ex;             // ex 是当前流入减流出的超额。
    vector<Cost> pot;           // pot 是费用缩放势能。
```

```
    vector<Ref> ref;            // ref 保存原边，便于查询最终流量。

    MinCostBFlow(int n) : n(n), g(n), ex(n), pot(n)
    {
        // n 是点数；初始没有边和节点供给。
        assert(n >= 0); // 调试检查，可删
    }

    void add(int u, int v, Cap low, Cap high, Cost cost)
    {
        // u、v 是端点，low/high 是下界和上界，cost 是单位费用；加入一条带上下界边。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删
        assert(low <= high); // 调试检查，可删
        Cost scale = (Cost)n + 1;
        assert((__int128)cost * scale < numeric_limits<Cost>::max()); // 调试检查，可删
        assert((__int128)cost * scale > numeric_limits<Cost>::min()); // 调试检查，可删
        Cost w = cost * scale;
        int i = g[u].size(), j = g[v].size() + (u == v);
        g[u].push_back({v, j, high - low, w});
        g[v].push_back({u, i, 0, -w});
        ex[u] -= low;
        ex[v] += low;
        ref.push_back({u, i, low, cost});
    }

    void supply(int u, Cap x)
    {
        // u 是点，x 是要求的净流出量；正值表示必须流出，负值表示必须流入。
        assert(0 <= u && u < n); // 调试检查，可删
        ex[u] += x;
    }

    Cost reduced(int u, const Edge &e) const
    {
        // u、e 是残量边起点和边；返回缩放后的约化费用。
        return e.w + pot[u] - pot[e.to];
    }

    void push(int u, int i, Cap f)
    {
        // u、i 定位残量边，f 是沿边发送的流量。
        Edge &e = g[u][i];
        e.cap -= f;
        g[e.to][e.rev].cap += f;
        ex[u] -= f;
        ex[e.to] += f;
    }

    bool refine(Cost eps)
    {
        // eps 是当前费用精度；饱和负边并消除所有正超额。
        vector<int> cur(n);
```

```
        vector<char> in(n);
        deque<int> q;
        auto canDeficit = [&](int s)
        {
            // s 是当前超额点；判断残量图中是否仍有路径到任一缺口点。
            vector<char> vis(n);
            deque<int> z;
            z.push_back(s);
            vis[s] = true;
            while (!z.empty())
            {
                int u = z.front();
                z.pop_front();
                if (ex[u] < 0)
                {
                    return true;
                }
                for (const auto &e : g[u])
                {
                    if (e.cap > 0 && !vis[e.to])
                    {
                        vis[e.to] = true;
                        z.push_back(e.to);
                    }
                }
            }
            return false;
        };
        for (int u = 0; u < n; u++)
        {
            for (int i = 0; i < (int)g[u].size(); i++)
            {
                if (g[u][i].cap > 0 && reduced(u, g[u][i]) < 0)
                {
                    push(u, i, g[u][i].cap);
                }
            }
        }
        for (int u = 0; u < n; u++)
        {
            if (ex[u] > 0)
            {
                q.push_back(u);
                in[u] = true;
            }
        }
        while (!q.empty())
        {
            int u = q.front();
            q.pop_front();
            in[u] = false;
            while (ex[u] > 0)
```

```
            {
                if (cur[u] == (int)g[u].size())
                {
                    Cost best = numeric_limits<Cost>::max();
                    for (const auto &e : g[u])
                    {
                        if (e.cap > 0)
                        {
                            best = min(best, reduced(u, e));
                        }
                    }
                    if (best == numeric_limits<Cost>::max())
                    {
                        return false;
                    }
                    if (!canDeficit(u))
                    {
                        return false;
                    }
                    pot[u] -= best + eps;
                    cur[u] = 0;
                    continue;
                }
                Edge &e = g[u][cur[u]];
                if (e.cap == 0 || reduced(u, e) >= 0)
                {
                    cur[u]++;
                    continue;
                }
                int v = e.to;
                Cap f = min(ex[u], e.cap);
                push(u, cur[u], f);
                if (ex[v] > 0 && !in[v])
                {
                    q.push_back(v);
                    in[v] = true;
                }
            }
        }
        return true;
    }

    optional<Ans> solve()
    {
        // 无参数；返回最小费用可行 b 流，不存在可行流时返回空。
        Cap total = 0;
        for (Cap x : ex)
        {
            total += x;
        }
        if (total != 0)
        {
```

```
            return nullopt;
        }
        Cost mx = 1;
        for (int u = 0; u < n; u++)
        {
            for (const auto &e : g[u])
            {
                mx = max(mx, e.w >= 0 ? e.w : -e.w);
            }
        }
        Cost eps = 1;
        while (eps < mx)
        {
            assert(eps <= numeric_limits<Cost>::max() / 2); // 调试检查，可删
            eps *= 2;
        }
        while (eps)
        {
            if (!refine(eps))
            {
                return nullopt;
            }
            eps /= 2;
        }
        for (Cap x : ex)
        {
            if (x != 0)
            {
                return nullopt;
            }
        }
        Ans ans = 0;
        for (const auto &x : ref)
        {
            Cap flow = x.low + g[g[x.u][x.i].to][g[x.u][x.i].rev].cap;
            ans += (Ans)flow * x.cost;
        }
        return ans;
    }

    optional<Certificate> solveCert()
    {
        // 无参数；求最小费用并返回可直接验证的点势和每条原边流量，不可行时返回空。
        auto ans = solve();
        if (!ans) return nullopt;
        vector<Cost> d(n);
        // 最优残量网络无负环；从超级源以零边到各点做 Bellman-Ford，得到可行势。
        for (int it = 0; it + 1 < n; it++)
        {
            bool upd = false;
            for (int u = 0; u < n; u++)
            {
```

```
                for (const auto &x : g[u])
                {
                    if (x.cap > 0 && d[x.to] > d[u] + x.w / ((Cost)n + 1))
                    {
                        d[x.to] = d[u] + x.w / ((Cost)n + 1);
                        upd = true;
                    }
                }
            }
            if (!upd) break;
        }
        vector<Cap> flow;
        flow.reserve(ref.size());
        for (const auto &x : ref)
        {
            const Edge &a = g[x.u][x.i];
            flow.push_back(x.low + g[a.to][a.rev].cap);
        }
        return Certificate{*ans, d, flow};
    }
};
```

`add(u,v,l,r,c)` 加边，`supply(u,x)` 指定净流出量，`solve()` 返回最小费用；`supply(s,F)` 与 `supply(t,-F)` 就是固定流量为 F 的源汇费用流，原有 5.8.005 的无费用最大/最小可行流可先确定 F，再调用本条求对应最小费用。需要提交最优性证书时改用 `solveCert()`，其 `flow` 按 `add` 顺序给出边流量，`pot` 满足残量边的约化费用非负，故可直接验证容量、守恒与互补松弛。容量和总费用须装入模板参数，缩放后的 $(n+1)c$ 也须装入 `Cost`；默认实现用 `long long`，总费用可能超界时把 `Ans` 改为 `__int128`。费用缩放的经典复杂度为 $O(VE\log(VC))$ 的理论上界，证书恢复额外 $O(VE)$ 时间和 $O(V+E)$ 空间；它直接处理负费用环，不要求源点可达子图无负环。

```
signed main()
{
    MinCostBFlow<> g(2);
    g.add(0, 1, 1, 3, 2);
    g.add(1, 0, 0, 3, 1);
    assert(g.solve() == optional<long long>(3));

    MinCostBFlow<> h(2);
    h.add(0, 1, 0, 3, 2);
    h.supply(0, 2);
    h.supply(1, -2);
    assert(h.solve() == optional<long long>(4));
}
```

5.8.012 动态星增广图全局最小割

原图固定，再加入一个新星点并维护它到每个原点的非负边权；每次单点改权后输出全局最小割。先对原图做 Stoer-Wagner 收缩，得到叶子为原点的二叉收缩树。原边和星边在每个收缩树节点上分别表示对应割的贡献；一条星边恰给叶子到根路径全部加同一增量。对收缩树重链剖分后，路径加与全局最小值由懒

线段树维护，因此每次更新只改 $O(\log^2 n)$ 个区间。

```
struct DynStarCut
{
    struct Seg
    {
        int n; // n 是补齐到 2 的幂的叶子数。
        vector<int> mn, tag; // mn 是区间最小值，tag 是整段增量。
        Seg(int n = 0) { init(n); }
        void init(int m)
        {
            // m 是叶子数；把全部叶子初始化为无穷大。
            n = 1;
            while (n < m)
            {
                n <<= 1;
            }
            mn.assign(n * 2, numeric_limits<int>::max() / 4);
            tag.assign(n * 2, 0);
        }
        void add(int l, int r, int x, int u, int a, int b)
        {
            // [l,r) 加 x，u 覆盖 [a,b)。
            if (r <= a || b <= l)
            {
                return;
            }
            if (l <= a && b <= r)
            {
                mn[u] += x;
                tag[u] += x;
                return;
            }
            int m = (a + b) / 2;
            add(l, r, x, u * 2, a, m);
            add(l, r, x, u * 2 + 1, m, b);
            mn[u] = tag[u] + min(mn[u * 2], mn[u * 2 + 1]);
        }
        int point(int p, int u, int a, int b) const
        {
            // 返回位置 p 的当前值，u 覆盖 [a,b)。
            if (b - a == 1)
            {
                return mn[u];
            }
            int m = (a + b) / 2;
            return tag[u] + (p < m ? point(p, u * 2, a, m) : point(p, u * 2 + 1, m, b));
        }
        void add(int l, int r, int x) { add(l, r, x, 1, 0, n); }
        void set(int p, int x) { add(p, p + 1, x - point(p, 1, 0, n)); }
        int allMin() const { return mn[1]; }
    };
```

```cpp

    int n; // n 是点数。
    vector<vector<int>> tree; // Stoer-Wagner 收缩树的子节点。
    vector<int> par, dep, heavy, head, pos, val; // HLD 父、深度、重儿子、链头、位置与星边权。
    Seg seg;

    void buildCutTree(vector<vector<pair<int, int>>> g)
    {
        // g 是原图的非零权邻接表；构造对应的收缩二叉树。
        tree.assign(2 * n - 1, {});
        vector<char> alive(2 * n - 1, true);
        for (int step = 0; step < n - 1; step++)
        {
            g.push_back({});
            vector<int> ord(n - step), cost(n + step);
            vector<char> used(n + step);
            priority_queue<pair<int, int>> q;
            for (int i = 0; i < n + step; i++)
            {
                if (alive[i])
                {
                    for (auto [j, w] : g[i])
                    {
                        cost[i] += w;
                    }
                    q.push({-cost[i], i});
                }
            }
            for (int k = 0; k < n - step; k++)
            {
                while (true)
                {
                    auto [neg, v] = q.top();
                    q.pop();
                    if (used[v])
                    {
                        continue;
                    }
                    used[v] = true;
                    ord[k] = v;
                    for (auto [to, w] : g[v])
                    {
                        if (!used[to])
                        {
                            cost[to] -= w;
                            q.push({-cost[to], to});
                        }
                    }
                    break;
                }
            }
            int x = ord[n - step - 1], y = ord[n - step - 2], z = n + step;
```

```
            g[x].clear();
            g[y].clear();
            alive[x] = alive[y] = false;
            for (int i = 0; i < n + step; i++)
            {
                if (!alive[i])
                {
                    continue;
                }
                for (auto &e : g[i])
                {
                    if (e.first == x || e.first == y)
                    {
                        e.first = z;
                        g[z].push_back({i, e.second});
                    }
                }
            }
            tree[z] = {x, y};
        }
    }

    int dfsSize(int u)
    {
        // u 是收缩树节点；设置父、深度和重儿子并返回子树大小。
        int size = 1, best = 0;
        for (int v : tree[u])
        {
            par[v] = u;
            dep[v] = dep[u] + 1;
            int s = dfsSize(v);
            size += s;
            if (s > best)
            {
                best = s;
                heavy[u] = v;
            }
        }
        return size;
    }

    void buildHld()
    {
        // 无参数；为收缩树建立重链编号并初始化线段树。
        int all = tree.size(), root = all - 1;
        par.assign(all, -1);
        dep.assign(all, 0);
        heavy.assign(all, -1);
        head.assign(all, -1);
        pos.assign(all, -1);
        dfsSize(root);
        queue<int> q;
```

```
        q.push(root);
        int timer = 0;
        while (!q.empty())
        {
            int h = q.front();
            q.pop();
            for (int u = h; u != -1; u = heavy[u])
            {
                head[u] = h;
                pos[u] = timer++;
                for (int v : tree[u])
                {
                    if (v != heavy[u])
                    {
                        q.push(v);
                    }
                }
            }
        }
        seg.init(all);
    }

    void addRootPath(int u, int x)
    {
        // u 是收缩树节点，x 是增量；把根到 u 的节点权整体加 x。
        while (u != -1)
        {
            seg.add(pos[head[u]], pos[u] + 1, x);
            u = par[head[u]];
        }
    }

    DynStarCut(int n, const vector<array<int, 3>> &ed, vector<int> a) : n(n), val(move(a))
    {
        // n 是原图点数，ed 每项为 {u,v,w}，val 是新星点到各原点的初始边权。
        assert(n >= 1 && (int)val.size() == n);
        vector<vector<pair<int, int>>> g(n);
        for (auto [u, v, w] : ed)
        {
            assert(0 <= u && u < n && 0 <= v && v < n && w >= 0);
            if (w)
            {
                g[u].push_back({v, w});
                g[v].push_back({u, w});
            }
        }
        buildCutTree(move(g));
        buildHld();
        vector<int> base(tree.size());
        for (auto [u, v, w] : ed)
        {
            if (w == 0)
```

```
            {
                continue;
            }
            while (u ≠ v)
            {
                if (dep[u] < dep[v])
                {
                    swap(u, v);
                }
                base[u] += w;
                u = par[u];
            }
        }
        for (int u = 0; u < (int)tree.size(); u++)
        {
            seg.set(pos[u], base[u]);
        }
        for (int u = 0; u < n; u++)
        {
            addRootPath(u, val[u]);
        }
    }

    int update(int u, int x)
    {
        // u 是被修改的原图点，x 是新星边权；更新后返回增广图的全局最小割值。
        assert(0 <= u && u < n && x >= 0);
        addRootPath(u, x - val[u]);
        val[u] = x;
        return seg.allMin();
    }
};
```

`DynStarCut(n,ed,val)` 中 `ed` 的每项为 `{u,v,w}`、`w\ge0`，`val[u]` 是星点到 `u` 的初始边权；`update(u,x)` 更新后返回最小割。预处理时间 $O(n^3+mn)$，额外空间 $O(n+m)$；单次更新 $O(\log^2 n)$，适用 $n,m \le 4000$、$Q \le 2\times10^5$。静态图只求一次割值时 5.8.007 Stoer-Wagner 全局最小割 更短；本条的收缩树专门服务固定原图上的星边动态更新。

5.8.013 CSR 优化原始对偶最小费用流

本版仍是逐路增广的原始对偶法，但把运行期残量图一次性整理成 CSR：点 u 的边连续存放在 `[start[u],start[u+1])`。对残量边 (u,v) 使用约化费用 $c_{uv}+h_u-h_v$；正容量原边费用全非负时，零势能已经可行，直接跳过初始化 SPFA；否则只从源点运行一次 SPFA。每轮 Dijkstra 把约化费用为零的松弛放入数组栈 `zero`，只有正约化费用候选进入最小堆，并在汇点出堆时提前停止。更新已确定点的势能后，所有可达残量边仍保持非负约化费用。

性能版只把点号和边下标固定为 32 位有符号整数，以缩小 CSR 边和堆元素；容量、费用与返回值仍由 `Cap`、`Cost` 控制。打印版按全书约定使用普通 `int`，算法、分支与接口不变。

```cpp
template <class Cap = long long, class Cost = long long>
struct FastCostFlow
{
    struct Raw
    {
        // from、to 是端点；性能版固定为 32 位，打印版使用普通 int。
        int from, to;
        Cap cap;   // 原边容量。
        Cost cost; // 原边单位费用。
    };
    struct E
    {
        // to 是终点，rev 是反向边全局下标；性能版固定为 32 位，打印版使用普通 int。
        int to, rev;
        Cap cap;   // 当前残量容量。
        Cost cost; // 单位费用。
    };
    struct Q
    {
        Cost d;    // 当前约化距离。
        int u; // u 是终点；性能版固定为 32 位以缩小堆元素。
        bool operator<(const Q &o) const { return d > o.d; }
    };

    // n 与端点在性能版固定为 32 位；容量、费用及答案仍由模板参数控制。
    int n, source = -1, sink = -1;
    vector<Raw> raw;        // 首次求流前暂存的原边。
    vector<int> start; // CSR 中点 u 的边区间为 [start[u],start[u+1])。
    vector<E> e;            // 按起点连续排列的残量边。
    vector<Cost> h, d;      // h 是势能，d 是本轮约化距离。
    vector<int> pre;   // pre[v] 是最短路进入 v 的残量边下标。
    vector<int> zero;  // 尚未确定的零约化距离点。
    vector<Q> heap;         // 正约化距离候选的最小堆。
    vector<char> vis, in;  // vis 是已确定/已到达标记，in 是 SPFA 入队标记。
    bool built = false;     // 是否已经构造 CSR 残量图。
    bool neg = false;       // 是否存在正容量负费用原边。

    FastCostFlow(int n = 0) : n(n)
    {
        // n 是点数；构造一张空网络。
        assert(n >= 0); // 调试检查，可删。
        static_assert(numeric_limits<Cap>::is_signed);
        static_assert(numeric_limits<Cost>::is_signed);
    }

    void add(int u, int v, Cap cap, Cost cost)
    {
        // u、v 是端点，cap 是非负容量，cost 是单位费用；加入一条原边。
        assert(!built); // 调试检查，可删：首次 flow 后不能继续加边。
        assert(0 <= u && u < n && 0 <= v && v < n && cap >= 0); // 调试检查，可删。
        assert(cost != numeric_limits<Cost>::min()); // 调试检查，可删。
        raw.push_back({u, v, cap, cost});
```

```
        neg |= cap > 0 && cost < 0;
    }

    void build()
    {
        // 无参数；把原边一次性展开成按起点连续存储的 CSR 残量图。
        assert(raw.size() <= (size_t)numeric_limits<int>::max() / 2); // 调试检查，可删。
        start.assign(n + 1, 0);
        for (const auto &a : raw)
        {
            start[a.from + 1]++;
            start[a.to + 1]++;
        }
        for (int u = 0; u < n; u++)
        {
            start[u + 1] += start[u];
        }
        vector<int> ptr = start;
        e.resize(raw.size() * 2);
        for (const auto &a : raw)
        {
            int x = ptr[a.from]++;
            int y = ptr[a.to]++;
            e[x] = {a.to, y, a.cap, a.cost};
            e[y] = {a.from, x, 0, -a.cost};
        }
        vector<Raw>().swap(raw);
        h.assign(n, 0);
        d.resize(n);
        pre.resize(n);
        vis.resize(n);
        in.resize(n);
        zero.reserve(n);
        heap.reserve(e.size());
        built = true;
    }

    void init(int s)
    {
        // s 是源点；有负费用边时用一次 SPFA 建立可行势能，否则保留零势能。
        fill(h.begin(), h.end(), 0);
        if (!neg)
        {
            return;
        }
        fill(vis.begin(), vis.end(), 0);
        fill(in.begin(), in.end(), 0);
        queue<int> q;
        d[s] = 0;
        vis[s] = in[s] = 1;
        q.push(s);
        while (!q.empty())
```

```
        {
            int u = q.front();
            q.pop();
            in[u] = 0;
            for (int i = start[u]; i < start[u + 1]; i++)
            {
                const E &a = e[i];
                if (a.cap == 0)
                {
                    continue;
                }
                Cost nd = d[u] + a.cost;
                if (!vis[a.to] || nd < d[a.to])
                {
                    d[a.to] = nd;
                    vis[a.to] = 1;
                    if (!in[a.to])
                    {
                        in[a.to] = 1;
                        q.push(a.to);
                    }
                }
            }
        }
        for (int u = 0; u < n; u++)
        {
            if (vis[u])
            {
                h[u] = d[u];
            }
        }
    }

    bool dualRef(int s, int t)
    {
        // s、t 是固定源汇；求一条最短增广路并更新势能，返回 t 是否可达。
        const Cost inf = numeric_limits<Cost>::max();
        fill(d.begin(), d.end(), inf);
        fill(vis.begin(), vis.end(), 0);
        zero.clear();
        heap.clear();
        d[s] = 0;
        zero.push_back(s);
        size_t heapSize = 0;
        while (!zero.empty() || !heap.empty())
        {
            int u;
            if (!zero.empty())
            {
                u = zero.back();
                zero.pop_back();
            }
```

```
            else
            {
                while (heapSize < heap.size())
                {
                    heapSize++;
                    push_heap(heap.begin(), heap.begin() + heapSize);
                }
                u = heap.front().u;
                pop_heap(heap.begin(), heap.end());
                heap.pop_back();
                heapSize--;
            }
            if (vis[u])
            {
                continue;
            }
            vis[u] = 1;
            if (u == t)
            {
                break;
            }
            Cost du = d[u];
            for (int i = start[u]; i < start[u + 1]; i++)
            {
                const E &a = e[i];
                if (a.cap == 0)
                {
                    continue;
                }
                Cost w = a.cost + h[u] - h[a.to];
                assert(w >= 0); // 调试检查，可删：势能必须使可达残量边约化费用非负。
                Cost nd = du + w;
                if (nd < d[a.to])
                {
                    d[a.to] = nd;
                    pre[a.to] = i;
                    if (nd == du)
                    {
                        zero.push_back(a.to);
                    }
                    else
                    {
                        heap.push_back({nd, a.to});
                    }
                }
            }
        }
        if (!vis[t])
        {
            return false;
        }
        for (int u = 0; u < n; u++)
```

```
        {
            if (vis[u])
            {
                h[u] -= d[t] - d[u];
            }
        }
        return true;
    }

    pair<Cap, Cost> flow(int s, int t,
                         Cap lim = numeric_limits<Cap>::max())
    {
        // s、t 是固定源汇，lim 是本次最多发送的流量；返回本次新增流量与最小费用。
        assert(0 <= s && s < n && 0 <= t && t < n && lim >= 0); // 调试检查，可删。
        if (s == t)
        {
            return {0, 0};
        }
        if (source == -1)
        {
            source = s;
            sink = t;
            build();
            init(s);
        }
        assert(source == s && sink == t); // 调试检查，可删：分段调用不能更换源汇。
        Cap f = 0;
        Cost cost = 0;
        while (f < lim && dualRef(s, t))
        {
            Cap x = lim - f;
            for (int v = t; v != s; v = e[e[pre[v]].rev].to)
            {
                x = min(x, e[pre[v]].cap);
            }
            for (int v = t; v != s; v = e[e[pre[v]].rev].to)
            {
                E &a = e[pre[v]];
                a.cap -= x;
                e[a.rev].cap += x;
            }
            f += x;
            cost += (Cost)x * (h[t] - h[s]);
        }
        return {f, cost};
    }
};
```

构造 `FastCostFlow<Cap,Cost> g(n)` 后，先用 `add(u,v,cap,cost)` 加完全部边，再调用 `flow(s,t,lim)`；点编号为 $0 \dots n-1$，默认容量和费用均为 `long long`。返回值是本次新增的 `{流量，费用}`；相同源汇可以分段继续增广，`s==t` 返回 `{0,0}` 且不锁定源汇。首次有效 `flow` 会构造 CSR，此后不能继续加边或更换源汇。调用者必须保证边数可由性能版 32 位下标容纳，且反向费用、路径费用、势能、`Cap` 到 `Cost` 的转

换、单次乘积和总费用均不溢出；单位费用不能等于 `numeric_limits<Cost>::min()`。

算法允许负费用边，但要求残量网络中不存在从源点可达的负费用环；需要处理负环及上下界时改用 5.8.011 费用缩放最小费用 b 流。设增广次数为 A，整数容量下 $A \le F$，其中 F 是发送的总流量。若存在正容量负费用原边，初始化 SPFA 最坏 $O(VE)$；否则没有这一步。每轮最短路最坏 $O(E \log E + V)$，建图 $O(V + E)$，额外空间 $O(V + E)$。

这版适合费用较分散、增广轮数难以批量合并的图；大量同费用最短路并存时，5.8.003 SPFA 多路增广 可在一轮 DFS 中推送多条路径，通常更快。5.8.004 普通原始对偶 使用嵌套邻接表和标准优先队列，代码更直观，适合不需要极致常数的场合。完备二分图指派仍优先使用 5.6.005 二分图最大权 / 最小权匹配。

本地性能检查使用 Windows 10、Ryzen 9 7945HX、GCC 14.2.0 与 `-O2 -pipe`，每个场景预热一次后取 5 或 7 次中位数，并重复整个进程三轮。120×120 随机费用指派图上，本版为 2.273–2.330 ms，5.8.003 为 7.377–7.679 ms，5.8.004 为 8.185–8.521 ms；全零费用指派图上三者分别为 1.682–1.723 ms、0.495–0.523 ms、4.658–4.855 ms。宽度 500、每点 8 条等费用边的分层图上，本版为 0.717–0.798 ms，5.8.003 为 0.321–0.336 ms。结果包含构造和求流时间，只说明这些固定分布下的常数取舍；

下面检查一张含负费用边的小图，并验证同源汇分段调用。

```
signed main()
{
    FastCostFlow<> g(4);
    g.add(0, 1, 2, 1);
    g.add(0, 2, 1, 4);
    g.add(1, 2, 1, -2);
    g.add(1, 3, 1, 3);
    g.add(2, 3, 2, 1);
    assert((g.flow(0, 3, 1) == pair<long long, long long>{1, 0}));
    assert((g.flow(0, 3) == pair<long long, long long>{2, 9}));
    return 0;
}
```

5.8.014 有向图全局最小割

有向图的全局最小割是所有非空真子集 $S \subset V$ 上 $c(S, V \setminus S)$ 的最小值，其中 $c(S, T)$ 是从 S 指向 T 的边容量之和。任取一个非平凡割，点 0 要么在 S 中而某个 v 不在，要么不在 S 中而某个 v 在；前者至少是 $0 \to v$ 的最小割，后者至少是 $v \to 0$ 的最小割。所以只要对每个 $v \neq 0$ 求两个方向的最大流，取最小值。

实现直接复用 5.8.001 Dinic 最大流 的 `Flow<T>`，容量类型取 `__int128` 以免容量和溢出；残量网络上从源点可达的点就是最小割的源侧。

```
// 依赖 5.8.001 Dinic 最大流
pair<__int128, vector<bool>> dirMinCut(int n, const vector<array<int, 3>> &e)
{
    // n>=2 是点数，e[i][0..2] 依次是有向边起点、终点和非负容量；
    // 返回有向图全局最小割的容量，以及一组割的源侧点集 side（side[v] 为真表示 v 在源侧）。
    assert(n >= 2); // 调试检查，可删。
    __int128 best = (__int128)1 << 120;
```

```
    vector<bool> side;
    // 任一非平凡割都把点 0 与某点 v 分开：枚举 v 和两个方向各求一次最大流即可覆盖。
    for (int v = 1; v < n; v++)
    {
        for (int dir = 0; dir < 2; dir++)
        {
            Flow<__int128> f(n);
            for (auto [x, y, c] : e)
            {
                f.add(x, y, c);
            }
            int s = dir ? v : 0, t = dir ? 0 : v;
            __int128 cur = f.flow(s, t);
            if (cur < best)
            {
                best = cur;
                side = f.cut(s);
            }
        }
    }
    return {best, side};
}
```

`dirMinCut(n,e)` 要求 $n \geq 2$，`e[i]` 依次是起点、终点和非负容量；返回最小割容量及源侧点集 `side`（`side[v]` 为真表示 v 在源侧），割边是从源侧指向另一侧的边。总共 $2(n-1)$ 次最大流，时间为 Dinic 单次 $O(n^2m)$ 上界的 $2(n-1)$ 倍，空间 $O(n+m)$。

无向图的全局最小割用 5.8.007 Stoer-Wagner 全局最小割，所有点对的最小割用 5.8.008 Gomory-Hu 最小割树。Hao-Orlin 算法能在一次最大流的时间内求出有向全局最小割，但代码长得多，这里没有实现。下面的三点图中把 $\{0,1\}$ 与 $\{2\}$ 分开只需切断 $1 \to 2$：

```
signed main()
{
    auto [c, side] = dirMinCut(3, {{0, 1, 5}, {1, 2, 3}, {2, 0, 4}, {1, 0, 1}});
    assert(c == 3 && side == vector<bool>({1, 1, 0}));
}
```

5.9 欧拉图与环

5.9.001 有向/无向欧拉路径与欧拉回路

欧拉迹是恰好经过每条边一次的路径，首尾相同则是欧拉回路。存在性判定很干净：无向图要求所有非零度点连通，且奇度点数为 0（有回路）或 2（有迹，起点必为某个奇度点）；有向图要求非零度点在忽略方向后连通，且每点入度等于出度（回路），或恰有一点出度比入度多 1（作起点）、一点入度比出度多 1（作终点）。构造用 Hierholzer：维护当前游走的语义栈，沿尚未用过的边走到底，再按弹栈顺序记录边和点，最后反转得到欧拉迹；无向图额外用 `used[]` 标记边是否已走。最后检查回溯出的点数等于 $m+1$，否则说明图不连通、无欧拉迹，返回空。`lex=true` 时先给每个邻接表按目标点排序，得到字典序最小的迹。

```cpp
struct EulerTrail
{
    // 依次经过的点与原边编号。
    vector<int> v, id;
};

optional<EulerTrail> dirEuler(int n, const vector<pair<int, int>> &edges, bool lex = false)
{
    // n 是点数，edges 是有向边，lex 控制是否取最小点序列；无欧拉迹时返回 nullopt。
    if (!n)
    {
        return edges.empty() ? optional<EulerTrail>(EulerTrail{}) : nullopt;
    }
    int m = edges.size(), s = m ? edges[0].first : 0, plus = 0, minus = 0;
    vector<int> in(n), out(n);
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < m; i++)
    {
        auto [u, v] = edges[i];
        g[u].push_back({v, i});
        out[u]++;
        in[v]++;
    }
    for (int u = 0; u < n; u++)
    {
        if (out[u] - in[u] == 1)
        {
            s = u;
            plus++;
        }
        else if (in[u] - out[u] == 1)
        {
            minus++;
        }
        else if (in[u] != out[u])
        {
            return nullopt;
        }
    }
    if (!((plus == 1 && minus == 1) || (!plus && !minus)))
    {
        return nullopt;
    }
    if (!plus)
    {
        for (int u = 0; u < n; u++)
        {
            if (out[u])
            {
                s = u;
                break;
            }
```

```
        }
    }
    if (lex)
    {
        for (auto &a : g)
        {
            sort(a.begin(), a.end());
        }
    }
    vector<int> it(n), rv, re, st{s}, enter{-1};
    while (!st.empty())
    {
        int u = st.back();
        if (it[u] < (int)g[u].size())
        {
            auto [v, id] = g[u][it[u]++];
            st.push_back(v);
            enter.push_back(id);
        }
        else
        {
            rv.push_back(u);
            if (enter.back() != -1) re.push_back(enter.back());
            st.pop_back();
            enter.pop_back();
        }
    }
    if ((int)rv.size() != m + 1)
    {
        return nullopt;
    }
    reverse(rv.begin(), rv.end());
    reverse(re.begin(), re.end());
    return EulerTrail{rv, re};
}

optional<EulerTrail> undirEuler(int n, const vector<pair<int, int>> &edges, bool lex = false)
{
    // n 是点数，edges 是无向边，lex 控制是否取最小点序列；无欧拉迹时返回 nullopt。
    if (!n)
    {
        return edges.empty() ? optional<EulerTrail>(EulerTrail{}) : nullopt;
    }
    int m = edges.size(), s = 0;
    vector<vector<pair<int, int>>> g(n);
    vector<int> deg(n);
    for (int i = 0; i < m; i++)
    {
        auto [u, v] = edges[i];
        g[u].push_back({v, i});
        g[v].push_back({u, i});
        deg[u]++;
```

```
        deg[v]++;
    }
    vector<int> odd;
    for (int u = 0; u < n; u++)
    {
        if (deg[u] & 1)
        {
            odd.push_back(u);
        }
    }
    if (odd.size() != 0 && odd.size() != 2)
    {
        return nullopt;
    }
    if (odd.size() == 2)
    {
        s = odd[0];
    }
    else
    {
        for (int u = 0; u < n; u++)
        {
            if (deg[u])
            {
                s = u;
                break;
            }
        }
    }
    if (lex)
    {
        for (auto &a : g)
        {
            sort(a.begin(), a.end());
        }
    }
    vector<int> it(n), rv, re, st{s}, enter{-1};
    vector<bool> used(m);
    while (!st.empty())
    {
        int u = st.back();
        while (it[u] < (int)g[u].size() && used[g[u][it[u]].second])
        {
            it[u]++;
        }
        if (it[u] < (int)g[u].size())
        {
            auto [v, id] = g[u][it[u]++];
            used[id] = true;
            st.push_back(v);
            enter.push_back(id);
        }
```

```
        else
        {
            rv.push_back(u);
            if (enter.back() != -1) re.push_back(enter.back());
            st.pop_back();
            enter.pop_back();
        }
    }
    if ((int)rv.size() != m + 1)
    {
        return nullopt;
    }
    reverse(rv.begin(), rv.end());
    reverse(re.begin(), re.end());
    return EulerTrail{rv, re};
}
```

`dirEuler(n, edges, lex)` 与 `undirEuler(n, edges, lex)` 返回 `optional<EulerTrail>`；边的数组下标就是原编号。无边时，$n > 0$ 返回只含点 0 的空迹，$n = 0$ 返回空点列。`lex=true` 时，开路径从度数强制的起点出发，回路从最小的非零度点出发，再按目标点升序取边，因而比较的是所有合法起点与走法中的最小点序列。排序前 $O(n+m)$，排序后 $O(n+m\log m)$，空间 $O(n+m)$。连通性由最终是否恰好回溯 $m+1$ 个点统一检查。5.9.002 直接复用本接口；计数见 5.9.003 BEST 定理与有向欧拉回路计数。

下面依次检查有向回路、有向开路径、无向奇度路径、自环与平行边、度数合法但不连通的图，以及字典序最小点序列。

```
signed main()
{
    vector<pair<int, int>> circuit = {{0, 1}, {1, 2}, {2, 0}};
    auto a = dirEuler(3, circuit);
    assert(a && a->v == vector<int>({0, 1, 2, 0}) && a->id == vector<int>({0, 1, 2}));

    vector<pair<int, int>> open = {{0, 1}, {1, 2}, {2, 1}};
    auto b = dirEuler(3, open);
    assert(b && b->v == vector<int>({0, 1, 2, 1}) && b->id == vector<int>({0, 1, 2}));

    vector<pair<int, int>> undirected = {{0, 0}, {0, 1}, {0, 1}, {1, 2}};
    auto c = undirEuler(3, undirected);
    assert(c && c->v == vector<int>({1, 0, 0, 1, 2}) && c->id == vector<int>({1, 0, 2, 3}));

    assert(!dirEuler(4, {{0, 1}, {1, 0}, {2, 3}, {3, 2}}).has_value());
    assert(!undirEuler(6, {{0, 1}, {1, 2}, {2, 0}, {3, 4}, {4, 5}, {5, 3}}).has_value());

    auto lex = dirEuler(3, {{0, 2}, {0, 1}, {1, 0}, {2, 0}}, true);
    assert(lex.has_value() && lex->v == vector<int>({0, 1, 0, 2, 0}));
    puts("OK"); // OK
    return 0;
}
```

5.9.002 字典序最小欧拉路径

求字典序最小的欧拉迹（点序列尽量小）用的正是 5.9.001 有向/无向欧拉路径与欧拉回路 里的 Hierholzer-lex：把每个邻接表按目标点升序排序，再跑迭代 Hierholzer，回溯后反转即得字典序最小的迹。因此本条不另立实现，直接调用 `dirEuler(n, edges, true)`（无向图用 `undirEuler(n, edges, true)`），返回的 `.v` 就是字典序最小的点序列、`.id` 是对应边序列。举例：有向边 $0\to2, 0\to1, 1\to0, 2\to0$ 的字典序最小欧拉回路点序列为 $0,1,0,2,0$。排序 $O(m\log m)$、Hierholzer $O(n+m)$，空间 $O(n+m)$。平行边终点相同时按原边编号先后消歧。只求任意一条迹（不要求字典序）省掉排序即可；统计回路数量而非构造用 5.9.003 BEST 定理与有向欧拉回路计数。

5.9.003 BEST 定理与有向欧拉回路计数

数有向欧拉图的欧拉回路条数是 BEST 定理的经典应用。前提是图欧拉：每点入度等于出度且所有非零度点连通。此时以任意根 r 固定一条首出边后，从 r 出发的欧拉回路数为 $t_r\prod_v(\deg^+(v)-1)!$，其中 t_r 是「所有边都朝向根 r」的内向生成树（arborescence）数目——BEST 定理保证这个乘积与根的选择无关。内向树形图计数正是把每条边反向后跑 8.2.011 Matrix-Tree / Kirchhoff 生成树计数 的 `arboCount`（对反向图求以 r 为根的外向树形图，即原图的内向树形图）。若只固定起点而不固定首边，还要乘 $\deg^+(r)$，因为每条首出边各对应一族回路。度数不平衡、非零度点不连通，或有边而 `root` 为孤立点时返回 0；与边无关的额外孤立点不影响计数。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 8.2.011 Matrix-Tree / Kirchhoff 生成树计数
Z bestTheorem(int n, int root, const vector<pair<int, int>> &e, bool fixFirst = false)
{
    // n 是点数，root 是起点，e 是有编号有向边；fixFirst 表示是否固定首边。
    if (n == 0)
    {
        return e.empty() ? Z(1) : Z(0);
    }
    assert(0 <= root && root < n); // 调试检查，可删。
    vector<int> out(n), in(n);
    for (auto [u, v] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        out[u]++;
        in[v]++;
    }
    for (int u = 0; u < n; u++)
    {
        if (out[u] != in[u])
        {
            return 0;
        }
    }
    if (e.empty())
    {
        return 1;
    }
```

```
    if (out[root] == 0)
    {
        return 0;
    }

    vector<int> id(n, -1), active;
    for (int u = 0; u < n; u++)
    {
        if (out[u] > 0)
        {
            id[u] = active.size();
            active.push_back(u);
        }
    }
    vector<pair<int, int>> rev;
    for (auto [u, v] : e)
    {
        rev.push_back({id[v], id[u]});
    }
    Z ans = arboCount(active.size(), id[root], rev);
    for (int u : active)
    {
        for (int k = 1; k < out[u]; k++)
        {
            ans *= k;
        }
    }
    if (!fixFirst)
    {
        ans *= out[root];
    }
    return ans;
}
```

调用前先备好 8.2.011 Matrix-Tree / Kirchhoff 生成树计数 的 `arboCount`。`bestTheorem(n, root, e, false)` 统计从 `root` 出发、首边任取的有编号边序列；若已指定 `root` 的某条首边，末参传 `true`。只有非零度点参与 Matrix-Tree，额外孤立点不会把答案误判成 0；有边而 `root` 孤立时才返回 0。全图无边约定空回路计数为 1。两点间各有两条双向平行边时，固定一条首边答案为 2，只固定起点答案为 4。设非零度点数为 k，时间 $O(k^3+m)$、空间 $O(k^2+m)$。

5.9.004 无向图最小环

非负权简单无向图的最小环可用 Floyd 变形求出。依次把顶点 k 加入中转点集合；在用 k 松弛前，$d_{i,j}$ 只允许经过编号小于 k 的内部点，因此对每个 $i<j<k$，路径 $i\rightsquigarrow j$ 加直连边 $i-k$、$k-j$ 就枚举了一个以 k 为最大编号点的简单环。

实现直接复用 5.2.004 Floyd 与最小环 的 `minCycle(a)`，不再维护第二份类型接口。输入是对称非负权矩阵，空值表示无边，无环返回空；调用者必须保证路径和环权可由矩阵的元素类型表示。算法要求简单图，平行边形成的二边环不属于该接口。稠密图 $O(n^3)$ 时间、$O(n^2)$ 空间；固定长度短环见 5.9.006 三元环、

四元环计数，全部简单环见 5.9.007 简单环存在性、输出与计数。

5.9.005 最小平均环 / 最小密度环

最小平均环求有向图中「环权和 / 环边数」的最小值，Karp 定理给出优雅的 $O(nm)$ 解法。设 $F_k(v)$ 是从某个虚拟源出发、恰好走 k 条边到达 v 的最小权（初始 $F_0(\cdot)=0$，逐层用所有边松弛），则最小平均环 $\mu^* = \min_v \max_{0\le k\le n-1} \dfrac{F_n(v)-F_k(v)}{n-k}$。直觉是：对最优环上任一点，$F_n$ 相对某个较短层 F_k 的「差商」恰好被环的平均权卡住，外层取 $\min$、内层取 $\max$ 正好抓出这个界。最小密度环（比率环）则求环上 $\sum a / \sum b$ 的最小值（要求每条边代价 $b>0$）：二分候选比率 x，把边权改成 $a-xb$ 判是否存在负环，可行则收紧上界。这是参数二分，不是按当前比率迭代更新的 Dinkelbach 算法。

```cpp
struct MeanEdge
{
    // 端点与边权。
    int u, v;
    long double w;
};

optional<long double> minMeanCycle(int n, const vector<MeanEdge> &e)
{
    // n 是点数，e 是有向边；返回最小平均环权，无环返回空。
    vector f(n + 1, vector<optional<long double>>(n));
    for (int v = 0; v < n; v++)
    {
        f[0][v] = 0;
    }
    for (int k = 1; k <= n; k++)
    {
        for (auto [u, v, w] : e)
        {
            assert(0 <= u && u < n && 0 <= v && v < n && isfinite(w)); // 调试检查，可删。
            if (f[k - 1][u].has_value())
            {
                long double z = *f[k - 1][u] + w;
                if (!f[k][v].has_value() || z < *f[k][v])
                {
                    f[k][v] = z;
                }
            }
        }
    }
    optional<long double> ans;
    for (int v = 0; v < n; v++)
    {
        if (!f[n][v].has_value())
        {
            continue;
        }
        optional<long double> cur;
```

```
        for (int k = 0; k < n; k++)
        {
            if (f[k][v].has_value())
            {
                long double z = (*f[n][v] - *f[k][v]) / (n - k);
                if (!cur.has_value() || z > *cur)
                {
                    cur = z;
                }
            }
        }
        if (cur.has_value() && (!ans.has_value() || *cur < *ans))
        {
            ans = *cur;
        }
    }
    return ans;
}

struct RatioEdge
{
    // 端点、收益与严格为正的代价。
    int u, v;
    long double a, b;
};

// b 必须严格为正；求环上 sum(a)/sum(b) 的最小值。
optional<long double> minRatCycle(int n, const vector<RatioEdge> &e, int it = 100)
{
    // n 是点数，e 是有向边，it 是二分轮数；返回最小环比率，无环返回空。
    if (e.empty())
    {
        return nullopt;
    }
    assert(it >= 0); // 调试检查，可删。
    vector<vector<int>> g(n);
    vector<int> deg(n);
    long double lo = 0, hi = 0;
    bool first = true;
    for (auto [u, v, a, b] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n &&
               isfinite(a) && isfinite(b) && b > 0); // 调试检查，可删。
        g[u].push_back(v);
        deg[v]++;
        long double z = a / b;
        if (first)
        {
            lo = hi = z;
            first = false;
        }
        else
```

```
        {
            lo = min(lo, z);
            hi = max(hi, z);
        }
    }
    queue<int> q;
    for (int u = 0; u < n; u++)
    {
        if (deg[u] == 0)
        {
            q.push(u);
        }
    }
    int seen = 0;
    while (!q.empty())
    {
        int u = q.front();
        q.pop();
        seen++;
        for (int v : g[u])
        {
            if (--deg[v] == 0)
            {
                q.push(v);
            }
        }
    }
    if (seen == n)
    {
        return nullopt;
    }
    auto has = [&](long double x)
    {
        vector<long double> d(n);
        for (int k = 0; k < n; k++)
        {
            bool any = false;
            for (auto [u, v, a, b] : e)
            {
                if (d[v] > d[u] + a - x * b)
                {
                    d[v] = d[u] + a - x * b;
                    any = true;
                }
            }
            if (!any)
            {
                return false;
            }
            if (k == n - 1)
            {
                return true;
```

```
            }
        }
        return false;
    };
    for (int z = 0; z < it; z++)
    {
        long double m = midpoint(lo, hi);
        if (has(m))
        {
            hi = m;
        }
        else
        {
            lo = m;
        }
    }
    return hi;
}
```

点从 0 编号。`minMeanCycle` 用 Karp 层数 DP 返回最小平均权；不可达状态与合法浮点值分开保存。`minRatCycle` 要求每条边 `b>0`，先用拓扑删除准确判定是否存在有向环，再在全部单边比率的最小值与最大值之间二分；中点用 `midpoint`，不会因同号大数相加溢出，也不依赖可能满足 `hi+1==hi` 的探测值。两函数都返回 `nullopt` 表示无环。Karp 算法 $O(nm)$ 时间、$O(n^2)$ 空间；比率环为 $O(it\,nm)$ 时间、$O(n+m)$ 空间。浮点输入与中间运算需保持有限，答案按题目误差要求比较。

5.9.006 三元环、四元环计数

三元环计数的经典技巧是「按度数定向」：把每条无向边定向为从度数小的端点指向度数大的（度数相同时按编号），得到 DAG。这样每个点的出度不超过 $O(\sqrt{m})$，枚举 $u \to v \to w$ 再检查 u, w 是否有边（`vis` 标记 u 的出邻居）即可，每个三角形在其定向后唯一的「源点」处被恰好数一次，总量 $O(m\sqrt{m})$。四元环则换一种思路：环 a—b—c—d—a 的两条对角点对 (a,c)、(b,d) 各是「一对拥有 ≥ 2 个公共邻居」的点对；对每对顶点 $u < v$ 用 bitset 求公共邻居数 c，累加 $\binom{c}{2}$，每个四元环被两条对角点对各数一次，故最后除以 2。

```
// 依赖 1.1.008 静态取模类
// 依赖 7.1.001 快速幂
unsigned long long triangles(int n, const vector<pair<int, int>> &e)
{
    // e 是简单无向图边集；返回三元环数量。
    vector<int> d(n);
    for (auto [u, v] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v); // 调试检查，可删。
        d[u]++;
        d[v]++;
    }
    vector<vector<int>> g(n);
    for (auto [u, v] : e)
    {
```

```cpp
        if (pair{d[u], u} > pair{d[v], v})
        {
            swap(u, v);
        }
        g[u].push_back(v);
    }
    vector<int> vis(n);
    unsigned long long ans = 0;
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u])
        {
            vis[v] = 1;
        }
        for (int v : g[u])
        {
            for (int w : g[v])
            {
                ans += vis[w];
            }
        }
        for (int v : g[u])
        {
            vis[v] = 0;
        }
    }
    return ans;
}

long long weightedTri(int n, const vector<long long> &val, const vector<pair<int, int>> &e,
    long long mod)
{
    // n 是点数，val 是每点权值，e 是简单无向边集，mod 是正模数；返回所有三元环三点权乘积之和模 mod。
    assert((int)val.size() == n && mod > 0); // 调试检查，可删。
    vector<int> d(n);
    for (auto [u, v] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v); // 调试检查，可删。
        d[u]++;
        d[v]++;
    }
    vector<vector<int>> g(n);
    for (auto [u, v] : e)
    {
        if (pair{d[u], u} > pair{d[v], v})
        {
            swap(u, v);
        }
        g[u].push_back(v);
    }
    vector<int> mark(n, -1);
    long long ans = 0;
```

```
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u])
        {
            mark[v] = u;
        }
        for (int v : g[u])
        {
            for (int w : g[v])
            {
                if (mark[w] == u)
                {
                    long long cur = mulMod(val[u], val[v], mod);
                    ans += mulMod(cur, val[w], mod);
                    if (ans >= mod)
                    {
                        ans -= mod;
                    }
                }
            }
        }
    }
    return (ans + mod) % mod;
}

inline int triWeight998(int n, const vector<int> &value,
                        const vector<pair<int, int>> &edges)
{
    // value 是点权，edges 是简单无向边；返回三元环三点权乘积和模 998244353。
    assert((int)value.size() == n); // 调试检查，可删。
    vector<int> deg(n), ord(n), rk(n);
    for (auto [u, v] : edges)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v); // 调试检查，可删。
        deg[u]++, deg[v]++;
    }
    iota(ord.begin(), ord.end(), 0);
    sort(ord.begin(), ord.end(), [&](int u, int v)
    {
        return pair{deg[u], u} < pair{deg[v], v};
    });
    for (int i = 0; i < n; i++)
    {
        rk[ord[i]] = i;
    }
    vector<Z> w(n);
    for (int i = 0; i < n; i++)
    {
        w[i] = value[ord[i]];
    }
    vector<vector<int>> g(n);
    for (auto [u, v] : edges)
```

```
    {
        u = rk[u], v = rk[v];
        if (u > v) swap(u, v);
        g[v].push_back(u);
    }
    vector<int> mark(n, -1);
    Z ans = 0;
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u])
        {
            mark[v] = u;
        }
        Z sum = 0;
        for (int v : g[u])
        {
            Z cur = 0;
            for (int x : g[v])
            {
                if (mark[x] == u) cur += w[x];
            }
            sum += cur * w[v];
        }
        ans += sum * w[u];
    }
    return ans.val();
}

unsigned long long fourCycles(int n, const vector<pair<int, int>> &e)
{
    // e 是简单无向图边集；返回四元环数量。
    int blocks = (n + 63) / 64;
    vector a(n, vector<unsigned long long>(blocks));
    for (auto [u, v] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v); // 调试检查，可删。
        a[u][v >> 6] |= 1ULL << (v & 63);
        a[v][u >> 6] |= 1ULL << (u & 63);
    }
    unsigned long long ans = 0;
    for (int u = 0; u < n; u++)
    {
        for (int v = u + 1; v < n; v++)
        {
            long long c = 0;
            for (int k = 0; k < blocks; k++)
            {
                c += popcount(a[u][k] & a[v][k]);
            }
            ans += (unsigned long long)c * (c - 1) / 2;
        }
    }
```

```cpp
    return ans / 2;
}

vector<long long> fourCycleCnt(int n, const vector<pair<int, int>> &edges)
{
    // n 是点数，edges 是允许重边的无向边；返回每条原边参与的四元环数量。
    int m = edges.size();
    vector<int> deg(n), ord(n), rk(n);
    for (auto [u, v] : edges)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v); // 调试检查，可删。
        deg[u]++;
        deg[v]++;
    }
    iota(ord.begin(), ord.end(), 0);
    stable_sort(ord.begin(), ord.end(), [&](int u, int v)
    {
        return deg[u] < deg[v];
    });
    for (int i = 0; i < n; i++)
    {
        rk[ord[i]] = i;
    }
    map<pair<int, int>, int> pos;
    vector<pair<int, int>> es;
    vector<vector<int>> group;
    for (int id = 0; id < m; id++)
    {
        auto [u, v] = edges[id];
        u = rk[u];
        v = rk[v];
        if (u < v) swap(u, v);
        auto [it, fresh] = pos.emplace(pair{u, v}, (int)es.size());
        if (fresh)
        {
            es.push_back({u, v});
            group.push_back({});
        }
        group[it->second].push_back(id);
    }
    vector<long long> mul(es.size()), ans(es.size());
    vector<vector<pair<int, int>>> g(n);
    for (int id = 0; id < (int)es.size(); id++)
    {
        auto [u, v] = es[id];
        mul[id] = group[id].size();
        g[u].push_back({v, id});
        g[v].push_back({u, id});
    }
    for (auto &a : g)
    {
        sort(a.begin(), a.end());
```

```
    }
    vector<long long> path(n);
    for (int top = n - 1; top >= 0; top--)
    {
        for (auto [mid, a] : g[top])
        {
            if (mid >= top) break;
           int lim = lower_bound(g[mid].begin(), g[mid].end(), pair{top, -1}) - g[mid].begin();
            for (int i = 0; i < lim; i++)
            {
                auto [bot, b] = g[mid][i];
                path[bot] += mul[a] * mul[b];
            }
        }
        for (auto [mid, a] : g[top])
        {
            if (mid >= top) break;
           int lim = lower_bound(g[mid].begin(), g[mid].end(), pair{top, -1}) - g[mid].begin();
            for (int i = 0; i < lim; i++)
            {
                auto [bot, b] = g[mid][i];
                long long other = path[bot] - mul[a] * mul[b];
                ans[a] += other * mul[b];
                ans[b] += other * mul[a];
            }
        }
        for (auto [mid, a] : g[top])
        {
            if (mid >= top) break;
           int lim = lower_bound(g[mid].begin(), g[mid].end(), pair{top, -1}) - g[mid].begin();
            for (int i = 0; i < lim; i++)
            {
                path[g[mid][i].first] = 0;
            }
        }
    }
    vector<long long> res(m);
    for (int i = 0; i < (int)es.size(); i++)
    {
        for (int id : group[i])
        {
            res[id] = ans[i];
        }
    }
    return res;
}
```

`triangles` 与 `fourCycles` 都要求简单无向图，不允许自环和重边，返回总数 `unsigned long long`。`weightedTri` 使用完全相同的定向枚举，但对每个环唯一地累加 `val[u]val[v]val[w]`；固定模数 998244353 时可用 `triWeight998`，它按度序重排后汇总公共邻点权，时间仍为 $O(m\sqrt{m})$，但减少循环内取模。`fourCycles` 适合顶点数不大但图较稠密的情形，时间为 $O(n^3/64)$。

`fourCycleCnt` 面向允许重边、要求每条输入边分别参与多少个四元环的场景；不允许自环。它先按度数给点重编号并把同端点的边压成一条带重数的边，再以最高编号点为顶点，汇总两条严格向下路径的重数积。每个长度三的折线只处理一次，第二轮减去当前折线后便能把其余同端点折线贡献回写给两条边，最后将压缩边答案复制回每条原边。令 m' 为压缩后简单图的边数，此实现排序和压缩为 $O(m \log m)$，路径枚举为 $O(m'\sqrt{m'})$，额外空间为 $O(n+m)$；在题目保证的 $n, m \le 3 \times 10^5$ 下应使用该接口而不是 bitset 总数接口。三种接口的结果都必须能放入 64 位有符号或无符号整数；若只做符号计算或图规模已使答案越界，调用者须另选数值类型。任意长度的简单环见 5.9.007 简单环存在性、输出与计数，最小权环见 5.9.004 无向图最小环。

5.9.007 简单环存在性、输出与计数

找环和数环是两类难度悬殊的任务，这里分开实现。找一条有向环用三色 DFS：白点未访问、灰点在当前递归栈上、黑点已回溯完毕；遇到指向灰点的边（返祖边）就找到了环，沿 `parent` 从当前点回溯到那个灰点即可复原环上顶点序列。而数出全部简单环是 #P 难的指数问题，只能在小图上用状态压缩 DP：为避免同一环因起点旋转或方向翻转被重复计数，固定「环上编号最小的顶点」为起点 s，DP 状态是已访问点集 `mask`（恒含 s）与当前落脚点 u，转移只走编号在 $[s, n)$ 的点，回到 s 且环长 ≥ 3 时累加；每个环被顺、逆两个方向各数一次，故 $/2$。

```cpp
vector<int> dirCycle(const vector<vector<int>> &g)
{
    // g 是有向邻接表；返回一个简单环，无环返回空。
    int n = g.size();
    vector<int> c(n), p(n, -1), ans;
    auto dfs = [&](auto &&self, int u) -> bool
    {
        c[u] = 1;
        for (int v : g[u])
        {
            assert(0 <= v && v < n); // 调试检查，可删。
            if (c[v] == 0)
            {
                p[v] = u;
                if (self(self, v))
                {
                    return true;
                }
            }
            else if (c[v] == 1)
            {
                ans = {v};
                for (int x = u; x != v; x = p[x])
                {
                    ans.push_back(x);
                }
                reverse(ans.begin() + 1, ans.end());
                return true;
            }
        }
```

```
        c[u] = 2;
        return false;
    };
    for (int s = 0; s < n; s++)
    {
        if (!c[s] && dfs(dfs, s))
        {
            return ans;
        }
    }
    return {};
}

vector<int> findDirCycle(int n, const vector<pair<int, int>> &edges)
{
    // n 是点数，edges 是有向边；返回一条环的原边编号顺序，无环返回空。
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < (int)edges.size(); i++)
    {
        auto [u, v] = edges[i];
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        g[u].push_back({v, i});
    }
    vector<int> state(n), it(n), par(n, -1), pe(n, -1), st;
    for (int s = 0; s < n; s++)
    {
        if (state[s]) continue;
        st.push_back(s);
        state[s] = 1;
        while (!st.empty())
        {
            int u = st.back();
            if (it[u] == (int)g[u].size())
            {
                state[u] = 2;
                st.pop_back();
                continue;
            }
            auto [v, id] = g[u][it[u]++];
            if (!state[v])
            {
                par[v] = u;
                pe[v] = id;
                state[v] = 1;
                st.push_back(v);
            }
            else if (state[v] == 1)
            {
                vector<int> ans{id};
                for (int x = u; x != v; x = par[x])
                {
                    ans.push_back(pe[x]);
```

```
                }
                reverse(ans.begin(), ans.end());
                return ans;
            }
        }
    }
    return {};
}

vector<int> findUndirCyc(int n, const vector<pair<int, int>> &edges)
{
    // n 是点数，edges 是无向边；返回一条环的原边编号顺序，无环返回空。
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < (int)edges.size(); i++)
    {
        auto [u, v] = edges[i];
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        g[u].push_back({v, i});
        if (u != v) g[v].push_back({u, i});
    }
    vector<int> state(n), it(n), par(n, -1), pe(n, -1), st;
    for (int s = 0; s < n; s++)
    {
        if (state[s]) continue;
        st.push_back(s);
        state[s] = 1;
        while (!st.empty())
        {
            int u = st.back();
            if (it[u] == (int)g[u].size())
            {
                state[u] = 2;
                st.pop_back();
                continue;
            }
            auto [v, id] = g[u][it[u]++];
            if (id == pe[u]) continue;
            if (!state[v])
            {
                par[v] = u;
                pe[v] = id;
                state[v] = 1;
                st.push_back(v);
            }
            else if (state[v] == 1)
            {
                vector<int> ans{id};
                for (int x = u; x != v; x = par[x])
                {
                    ans.push_back(pe[x]);
                }
                reverse(ans.begin(), ans.end());
```

```
                return ans;
            }
        }
    }
    return {};
}

unsigned __int128 simpleCycles(const vector<unsigned long long> &g)
{
    // g 是简单无向图的对称邻接位掩码；返回全部简单环数量。
    int n = g.size();
    assert(n <= 22); // 调试检查，可删：状态数必须能承受。
    for (int u = 0; u < n; u++)
    {
        assert(!(g[u] >> u & 1)); // 调试检查，可删。
        for (int v = 0; v < n; v++)
        {
            assert((g[u] >> v & 1) == (g[v] >> u & 1)); // 调试检查，可删。
        }
    }
    unsigned __int128 ans = 0;
    for (int s = 0; s < n; s++)
    {
        int m = n - s;
        size_t states = 1ULL << m;
        vector<unsigned long long> dp(states * m);
        dp[m] = 1;
        for (size_t mask = 1; mask < states; mask += 2)
        {
            for (int u = 0; u < m; u++)
            {
                unsigned long long x = dp[mask * m + u];
                if (!x)
                {
                    continue;
                }
                for (int v = 1; v < m; v++)
                {
                    if (!(mask >> v & 1) && (g[s + u] >> (s + v) & 1))
                    {
                        dp[((mask | 1ULL << v) * m) + v] += x;
                    }
                }
                if (popcount((unsigned)mask) >= 3 && (g[s + u] >> s & 1))
                {
                    ans += x;
                }
            }
        }
    }
    return ans / 2;
}
```

`dirCycle(g)` 用递归 DFS 返回一个简单有向环，首点不在末尾重复；无环返回空。需要在线性时间输出原边编号且避开递归深度限制时，用 `findDirCycle(n, edges)` 或 `findUndirCyc(n, edges)`：两者维护显式 DFS 语义栈，返回能按顺序首尾相接的一条环边编号，无环返回空，支持重边和自环，均为 $O(n+m)$ 时间与空间。`simpleCycles(g)` 接受 $n \le 22$ 的对称位掩码，返回 `unsigned __int128` 环数；DP 的单状态路径数在该范围内不超过 $20!$，可安全存于 `unsigned long long`，最终总数再用 128 位累加。时间 $O(n^2 2^n)$、峰值空间 $O(n2^n)$，实际内存限制往往比断言更早成为瓶颈。定长短环见 5.9.006 三元环、四元环计数，带权最小环见 5.9.004 无向图最小环。

5.9.008 基环树

基环树是恰有 n 条边的连通简单无向图，等价于一棵树多一条边。反复删除度数不超过 1 的点，未被删除的点恰为唯一环；随后以全部环点为根做多源 BFS，即可给每个树枝点确定所属环点、父点和深度。这里的接口按简单图定义，不处理自环或平行边形成的一、二元环。

```cpp
struct Unicycle
{
    // 点数与无向邻接表。
    int n;
    vector<vector<int>> e;
    // cycle 按顺序保存唯一环上的顶点；root[v] 记 v 所挂的环点；parent[v]、dep[v] 是 v 在挂树内的父亲与深度。
    vector<int> cycle, root, parent, dep;
    // on[u] 表示 u 是否位于环上（剥叶子后仍未被删）。
    vector<bool> on;

    Unicycle(int n = 0) : n(n), e(n)
    {
        // n 是点数；构造空图。
    }

    void add(int u, int v)
    {
        // 加入简单无向边 u-v。
        assert(0 <= u && u < n && 0 <= v && v < n && u != v); // 调试检查，可删。
        e[u].push_back(v);
        e[v].push_back(u);
    }

    bool build()
    {
        // 重建环与挂树信息；输入不是基环树时返回 false。
        cycle.clear();
        root.clear();
        parent.clear();
        dep.clear();
        on.clear();
        int m = 0;
        vector<int> d(n);
        queue<int> q;
```

```
        for (int u = 0; u < n; u++)
        {
            d[u] = e[u].size();
            m += d[u];
            if (d[u] <= 1)
            {
                q.push(u);
            }
        }
        if (m != 2 * n)
        {
            return false;
        }
        on.assign(n, true);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            on[u] = false;
            for (int v : e[u])
            {
                if (--d[v] == 1)
                {
                    q.push(v);
                }
            }
        }
        int s = find(on.begin(), on.end(), true) - on.begin();
        if (s == n)
        {
            return false;
        }
        for (int u = 0; u < n; u++)
        {
            if (!on[u])
            {
                continue;
            }
            int cnt = 0;
            for (int v : e[u])
            {
                cnt += on[v];
            }
            if (cnt != 2)
            {
                return false;
            }
        }
        cycle = {s};
        int p = -1, u = s;
        while (true)
        {
```

```
            int v = -1;
            for (int x : e[u])
            {
                if (on[x] && x != p)
                {
                    v = x;
                    break;
                }
            }
            if (v == s)
            {
                break;
            }
            if (v < 0 || (int)cycle.size() > n)
            {
                return false;
            }
            cycle.push_back(v);
            p = u;
            u = v;
        }
        root.assign(n, -1);
        parent.assign(n, -1);
        dep.assign(n, 0);
        for (int x : cycle)
        {
            root[x] = x;
            q.push(x);
        }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int v : e[u])
            {
                if (on[v] || root[v] != -1)
                {
                    continue;
                }
                parent[v] = u;
                root[v] = root[u];
                dep[v] = dep[u] + 1;
                q.push(v);
            }
        }
        return count(on.begin(), on.end(), true) == (int)cycle.size() &&
               count(root.begin(), root.end(), -1) == 0;
    }
};
```

先 `Unicycle uc(n)`、逐边 `add`，再调用 `build()`。只有边数等于点数、唯一剩余核是一个简单环且所有点都从该环可达时才返回 `true`；成功后 `cycle`、`root`、`parent`、`dep`、`on` 有效。`build` 每次都会清空旧结果，

可在继续加边或手工修改邻接表后重新调用。全过程 $O(n)$ 时间与空间。多个连通块各带一环时应分块处理；树上算法见第 6 章，只需找任意环见 5.9.007 简单环存在性、输出与计数。

5.9.009 函数图与后继跳跃

函数图是每个点恰有一条出边的有向图：每个弱连通块恰含一个环，其余点组成指向环的内向树。先反复删除入度为 0 的点（拓扑剥离），剩下的点恰好都在环上；再按删除顺序的逆序处理树上点，它们的后继总是先被处理，于是入环点 `entry`、到环深度 `depth`、所在块 `comp` 都可直接继承。

后继跳跃用倍增：`up[j][v]` 是走 2^j 步后的点，同时维护这 2^j 步离开的点权和与最小值，层数 L 满足 $2^L > n$。步数 $k \le n$ 时直接倍增；$k > n$ 时必然先走 $\text{depth}(v)$ 步到达入环点 e，再绕环至少一整圈，剩余 $k - \text{depth}(v) = q \cdot len + r$ 步的贡献是 q 倍整圈加上 r 步前缀，所以 k 可以取到 $2^{64}-1$ 而表只有 $O(n \log n)$。

```
struct FuncGraph
{
    int n; // n 是点数，点编号为 [0,n)。
    // to[v] 是 v 的唯一后继；comp[v] 是所在连通块（即所在环）的编号。
    vector<int> to, comp;
    // pos[v] 是环上点在 cycles[comp[v]] 中的下标，树上点为 -1；entry[v] 是 v 第一次到达的环点。
    vector<int> pos, entry;
    // depth[v] 是 v 走到环上所需的步数（环上点为 0）；len[v] 是所在环的长度。
    vector<int> depth, len;
    vector<vector<int>> cycles; // cycles[i] 按后继方向列出第 i 个环。
    int L; // L 是倍增层数，满足 2^L > n。
    // up[j][v] 是 v 走 2^j 步后的点；sum、mn 是这 2^j 步离开的各点权值之和与最小值。
    vector<vector<int>> up, sum, mn;

    FuncGraph(vector<int> f, vector<int> w = {}) : n(f.size()), to(move(f)), comp(n, -1),
↪   pos(n, -1),
        entry(n), depth(n), len(n), L(bit_width((unsigned long long)n)), up(L, vector<int>(n)),
        sum(L, vector<int>(n)), mn(L, vector<int>(n))
    {
        // f[v] 是 v 的后继，w[v] 是点权（为空时全为 0），须满足 n*max|w| 不超过 LLONG_MAX；
        // 建立环分解与 L 层倍增表。
        if (w.empty())
        {
            w.assign(n, 0);
        }
        assert(w.size() == to.size()); // 调试检查，可删。
        // 反复删入度为 0 的点，剩下的恰是环；order 是被删点的顺序。
        vector<int> deg(n), order;
        for (int v : to)
        {
            deg[v]++;
        }
        queue<int> q;
        for (int i = 0; i < n; i++)
        {
            if (!deg[i])
```

```
            {
                q.push(i);
            }
        }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            order.push_back(u);
            if (!--deg[to[u]])
            {
                q.push(to[u]);
            }
        }
        for (int s = 0; s < n; s++)
        {
            if (deg[s] && comp[s] < 0)
            {
                int id = cycles.size(), u = s;
                vector<int> c;
                do
                {
                    comp[u] = id;
                    pos[u] = c.size();
                    entry[u] = u;
                    c.push_back(u);
                    u = to[u];
                } while (u != s);
                for (int v : c)
                {
                    len[v] = c.size();
                }
                cycles.push_back(move(c));
            }
        }
        // 逆删点顺序处理树上点：后继总是先被处理，可直接继承其信息。
        reverse(order.begin(), order.end());
        for (int u : order)
        {
            int v = to[u];
            comp[u] = comp[v];
            entry[u] = entry[v];
            depth[u] = depth[v] + 1;
            len[u] = len[v];
        }
        for (int i = 0; i < n; i++)
        {
            up[0][i] = to[i];
            sum[0][i] = w[i];
            mn[0][i] = w[i];
        }
        for (int j = 1; j < L; j++)
```

```
        {
            for (int i = 0; i < n; i++)
            {
                int v = up[j - 1][i];
                up[j][i] = up[j - 1][v];
                sum[j][i] = sum[j - 1][i] + sum[j - 1][v];
                mn[j][i] = min(mn[j - 1][i], mn[j - 1][v]);
            }
        }
    }

    struct Walk
    {
        int end;          // end 是走完后到达的点。
        __int128 total;   // total 是途中离开的各点权值之和。
        optional<int> minimum; // minimum 是途中离开的各点权值最小值，不走时为空。
    };

    Walk lift(int v, int k) const
    {
        // 0<=k<2^L；用倍增表从 v 出发走 k 步。
        Walk r{v, 0, nullopt};
        for (int j = 0; j < L; j++)
        {
            if (k >> j & 1)
            {
                r.total += sum[j][r.end];
                r.minimum = r.minimum ? min(*r.minimum, mn[j][r.end]) : mn[j][r.end];
                r.end = up[j][r.end];
            }
        }
        return r;
    }

    Walk walk(int v, unsigned long long k) const
    {
        // 从 v 出发走 k 步；返回终点以及离开的 k 个点（含 v，不含终点）的权值和与最小值。
        if (k <= (unsigned long long)n)
        {
            return lift(v, k);
        }
        // k>n 时必然绕环至少一整圈：先走到入环点 e，再把整圈的和乘上圈数。
        Walk a = lift(v, depth[v]);
        int e = a.end;
        unsigned long long rest = k - depth[v], q = rest / len[e];
        Walk c = lift(e, len[e]), b = lift(e, rest % len[e]);
        a.total += (__int128)q * c.total + b.total;
        a.minimum = a.minimum ? min(*a.minimum, *c.minimum) : *c.minimum;
        a.end = b.end;
        return a;
    }

```

```
    int jump(int v, unsigned long long k) const
    {
        // 返回 v 走 k 步后的点。
        return walk(v, k).end;
    }

    optional<unsigned long long> meet(int u, int v) const
    {
        // u、v 同时每步各走一条边；返回第一次落在同一点时的步数，永不相遇时返回空。
        if (comp[u] != comp[v])
        {
            return nullopt;
        }
        // 都走到环上之后位置差不再改变，所以相遇只可能发生在前 max(depth) 步内。
        unsigned long long hi = max(depth[u], depth[v]);
        if (jump(u, hi) != jump(v, hi))
        {
            return nullopt;
        }
        unsigned long long lo = 0;
        while (lo < hi)
        {
            unsigned long long m = lo + (hi - lo) / 2;
            if (jump(u, m) == jump(v, m))
            {
                hi = m;
            }
            else
            {
                lo = m + 1;
            }
        }
        return lo;
    }
};
```

`FuncGraph f(to, w)` 中 `to[v]` 是后继，`w` 是可选点权，须满足 $n \cdot \max |w|$ 不超过 `LLONG_MAX`。`walk(v,k)` 返回走 k 步后的终点 `end`、离开的 k 个点（含起点、不含终点）的权值和 `total` 与最小值 `minimum`（$k = 0$ 时为空）；`jump(v,k)` 只返回终点。`meet(u,v)` 让两点每步各走一条边，返回第一次重合的步数：不在同一块时永不相遇；两点都进入环后位置差不再变化，所以只需检查前 $\max(\mathrm{depth}(u), \mathrm{depth}(v))$ 步，而一旦重合之后一直重合，可以二分。

预处理 $O(n \log n)$ 时间与空间，`walk` 为 $O(\log n)$，`meet` 为 $O(\log^2 n)$。只需找环、不需要跳跃时用 5.9.008 基环树 的拆环方式或直接拓扑剥离；树上的 k 级祖先见 6.1.006 树上距离、K 级祖先。下面的图中 $0 \to 1 \to 2 \to 3 \to 1$ 构成尾巴 0 和环 $\{1, 2, 3\}$，另有 $4 \to 2$：

```
signed main()
{
    FuncGraph f({1, 2, 3, 1, 2}, {5, -1, 2, 7, 3});
    auto w = f.walk(0, 1000000000000000000ULL); // 离开 0 一次，再绕环 333333333333333333 圈。
    assert(w.end == 1 && w.total == 5 + (__int128)8 * 333333333333333333 && *w.minimum == -1);
```

```
    assert(f.jump(0, 4) == 1 && f.depth[0] == 1 && f.len[0] == 3 && f.entry[4] == 2);
    assert(f.meet(0, 3) == 1 && !f.meet(0, 4)); // 0、4 进入环后始终相差一步。
}
```

5.10 特殊图与图论结论

5.10.001 弦图、完美消除序列与判错

弦图是每个长度 ≥ 4 的环都有弦（连接环上不相邻两点的边）的图，等价于存在完美消除序列（PEO）：按此序逆向删点时，每个点在删除瞬间其尚未删除的邻居构成一个团。最大势搜索（MCS）能高效产生候选 PEO——维护每点「已被选入序列的邻居数」w，每步贪心选 w 最大的未选点、并给它未选邻居的 w 加一；把选出顺序反转即得候选 PEO。这个序不一定真是 PEO（图可能非弦），所以还要验证：按 PEO 正序，对每个点 u 找它在序中最靠前的后继邻居 p，检查 u 的其余后继邻居是否都与 p 相邻（用 `mark` 标记 p 的邻居后 $O(1)$ 查），一旦某个 v 不与 p 相邻就找到反例三元组 (u, p, v)，判定非弦图。

```
struct Chordal
{
    // ok 表示图是否为弦图。
    bool ok;
    // peo 保存最大势搜索得到的完美消除序列，弦图时有效。
    vector<int> peo;
    // 非弦图时，bad 是破坏候选 PEO 的三元组 (u,p,v)。
    array<int, 3> bad{-1, -1, -1};
};

Chordal chordal(const vector<vector<int>> &g)
{
    // g 是简单无向图邻接表；返回弦图判定、候选 PEO 和失败三元组。
    int n = g.size();
    vector<int> w(n), pos(n), peo;
    vector<bool> used(n);
    for (int k = 0; k < n; k++)
    {
        int u = -1;
        for (int i = 0; i < n; i++)
        {
            if (!used[i] && (u == -1 || w[i] > w[u]))
            {
                u = i;
            }
        }
        used[u] = true;
        peo.push_back(u);
        for (int v : g[u])
        {
            if (!used[v])
            {
                w[v]++;
            }
```

```
        }
    }
    reverse(peo.begin(), peo.end());
    for (int i = 0; i < n; i++)
    {
        pos[peo[i]] = i;
    }
    vector<int> mark(n, -1);
    for (int i = 0; i < n; i++)
    {
        int u = peo[i], p = -1;
        for (int v : g[u])
        {
            if (pos[v] > i && (p == -1 || pos[v] < pos[p]))
            {
                p = v;
            }
        }
        if (p == -1)
        {
            continue;
        }
        for (int v : g[p])
        {
            mark[v] = u;
        }
        for (int v : g[u])
        {
            if (v != p && pos[v] > i && mark[v] != u)
            {
                return {false, peo, {u, p, v}};
            }
        }
    }
    return {true, peo};
}

struct ChordalCert
{
    // ok 表示图是否为弦图。
    bool ok;
    // peo 是弦图的一组完美消除序列；非弦图时为空。
    vector<int> peo;
    // cycle 是非弦图的一条诱导环；弦图时为空。
    vector<int> cycle;
};

ChordalCert chordalCert(int n, const vector<pair<int, int>> &e)
{
    // n 是点数，e 是简单无向边；返回 PEO 或长度至少四的诱导环证书。
    assert(n >= 0); // 调试检查，可删。
    vector<set<int>> g(n);
```

```
    for (auto [u, v] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v); // 调试检查，可删。
        g[u].insert(v);
        g[v].insert(u);
    }
    vector<int> w(n), pos(n), ord;
    set<pair<int, int>> cand;
    for (int u = 0; u < n; u++)
    {
        cand.insert({0, u});
    }
    while (!cand.empty())
    {
        int u = prev(cand.end())->second;
        cand.erase(prev(cand.end()));
        pos[u] = ord.size();
        ord.push_back(u);
        w[u] = -1;
        for (int v : g[u])
        {
            if (w[v] != -1)
            {
                cand.erase({w[v], v});
                cand.insert({++w[v], v});
            }
        }
    }
    for (int u : ord)
    {
        int pre = -1;
        for (int v : g[u])
        {
            if (pos[v] < pos[u]) pre = max(pre, pos[v]);
        }
        if (pre == -1) continue;
        int p = ord[pre];
        for (int s : g[u])
        {
            if (pos[s] >= pos[p] || g[p].contains(s)) continue;
            vector<int> from(n, -1);
            queue<int> q;
            from[s] = s;
            q.push(s);
            while (!q.empty() && from[p] == -1)
            {
                int x = q.front();
                q.pop();
                for (int v : g[x])
                {
                    if (v == u || (v != p && g[u].contains(v)) || from[v] != -1) continue;
                    from[v] = x;
```

```
                    q.push(v);
                }
            }
            assert(from[p] != -1); // MCS 反例保证该路径存在。
            vector<int> cyc;
            for (int x = p; x != s; x = from[x])
            {
                cyc.push_back(x);
            }
            cyc.push_back(s);
            cyc.push_back(u);
            return {false, {}, move(cyc)};
        }
    }
    reverse(ord.begin(), ord.end());
    return {true, move(ord), {}};
}

ChordalCert chordalCertFast(int n, const vector<pair<int, int>> &edges)
{
    // edges 是简单无向边；使用桶式 MCS 与 CSR 邻接，返回 PEO 或诱导环证书。
    vector<int> deg(n), off(n + 1);
    for (auto [u, v] : edges)
    {
        deg[u]++, deg[v]++;
    }
    for (int i = 0; i < n; i++)
    {
        off[i + 1] = off[i] + deg[i];
    }
    vector<int> to(2 * edges.size()), cur = off;
    for (auto [u, v] : edges)
    {
        to[cur[u]++] = v, to[cur[v]++] = u;
    }
    vector<int> head(n + 1, -1), nx(n, -1), prv(n, -1), wt(n);
    auto erase = [&](int u)
    {
        int x = wt[u];
        if (prv[u] == -1) head[x] = nx[u]; else nx[prv[u]] = nx[u];
        if (nx[u] != -1) prv[nx[u]] = prv[u];
    };
    auto insert = [&](int u)
    {
        int x = wt[u];
        prv[u] = -1, nx[u] = head[x];
        if (nx[u] != -1) prv[nx[u]] = u;
        head[x] = u;
    };
    for (int i = 0; i < n; i++)
    {
        insert(i);
```

```
    }
    vector<int> ord;
    vector<char> used(n);
    int best = 0;
    while ((int)ord.size() < n)
    {
        while (best >= 0 && head[best] == -1)
        {
            best--;
        }
        int u = head[best];
        erase(u), used[u] = 1, ord.push_back(u);
        for (int i = off[u]; i < off[u + 1]; i++)
        {
            if (!used[to[i]])
{
int v = to[i];
erase(v), ++wt[v], insert(v), best = max(best, wt[v]);
}
        }
    }
    vector<int> pos(n), par(n, -1), child(n, -1), sibling(n, -1);
    for (int i = 0; i < n; i++)
    {
        pos[ord[i]] = i;
    }
    for (int u = 0; u < n; u++)
    {
        for (int i = off[u]; i < off[u + 1]; i++)
{
int v = to[i];
if (pos[v] < pos[u] && (par[u] == -1 || pos[par[u]] < pos[v])) par[u] = v;
}
    }
    for (int u = 0; u < n; u++)
    {
        if (par[u] != -1) sibling[u] = child[par[u]], child[par[u]] = u;
    }
    vector<int> mark(n, -1);
    int bad = -1, parent = -1, other = -1;
    for (int u = 0; u < n; u++)
    {
        for (int i = off[u]; i < off[u + 1]; i++)
        {
            mark[to[i]] = u;
        }
        for (int v = child[u]; v != -1; v = sibling[v])
        {
            for (int i = off[v]; i < off[v + 1]; i++)
            {
if (pos[to[i]] < pos[u] && mark[to[i]] != u) bad = v, parent = u, other = to[i];
            }
```

```
        }
    }
    if (bad == -1)
    {
        reverse(ord.begin(), ord.end());
        return {true, move(ord), {}};
    }
    vector<char> ban(n);
    for (int i = off[bad]; i < off[bad + 1]; i++)
    {
        if (pos[to[i]] < pos[bad] && to[i] != parent && to[i] != other) ban[to[i]] = 1;
    }
    vector<int> from(n, -1), q(n);
    int l = 0, r = 0;
    from[other] = other, q[r++] = other;
    while (l < r && from[parent] == -1)
    {
        int u = q[l++];
        for (int i = off[u]; i < off[u + 1]; i++)
        {
            int v = to[i];
            if (pos[v] >= pos[bad] || ban[v] || from[v] != -1) continue;
            from[v] = u, q[r++] = v;
        }
    }
    if (from[parent] == -1) return {false, {}, {bad, parent, other}};
    vector<int> path;
    for (int u = parent;; u = from[u])
    {
        path.push_back(u);
        if (u == other) break;
    }
    reverse(path.begin(), path.end());
    vector<int> cycle{bad};
    cycle.insert(cycle.end(), path.begin(), path.end());
    return {false, {}, move(cycle)};
}
```

`chordal(g)` 是短小的邻接表教学入口：`.ok` 为真时 `.peo` 是 PEO，为假时 `.bad` 给出破坏候选 PEO 的三元组；其朴素 MCS 为 $O(n^2+m)$。顶点数约数千时它更短且便于调试。

`chordalCert(n,e)` 是大图和在线判题接口，输入简单无向边，除了 PEO 还能在非弦图时给出一条长度至少四的诱导环。它把 MCS 的未选点按 (势，编号) 放进有序集合，每次增势和邻接查询分别花 $O(\log n)$；找到 MCS 违反点对后，从其中一个不相邻后继出发，避开当前点及其其余邻居 BFS 到另一个后继，所得路径加上当前点正是一条无弦环。总时间 $O((n+m)\log n)$，额外空间 $O(n+m)$。大图可改用 `chordalCertFast`：桶式 MCS 与 CSR 邻接把排序和集合操作降为线性，仍返回 PEO/诱导环证书。判弦图后求团/独立集直接用 PEO 即可。

5.10.002 欧拉公式与圆上弦分侧

平面图的欧拉公式为 $V-E+F=1+C$，其中 C 是连通块数。它能推出简单平面图 $E\le 3V-6$ 等必要条件，但不能判定一般图是否平面。本条只给两个与平面嵌入有关的短工具：`faceCount` 从欧拉公式反推面数；`chordPlanar` 处理顶点顺次固定在圆周上、每条弦可选画在圆内或圆外的特殊模型。两条弦端点交错时必须异侧，所以问题等价于给弦的交叉图二染色。

```cpp
namespace planar_graph
{
optional<vector<int>> chordPlanar(int n, vector<array<int, 2>> es)
{
    // n 是已按环排列的点数，es[i] 是额外弦端点；返回每条弦放环内或环外的 0/1 方案，无解返回空。
    int m = es.size();
    vector<vector<int>> g(m);
    for (auto &[u, v] : es)
    {
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        if (u > v)
        {
            swap(u, v);
        }
    }
    for (int i = 0; i < m; i++)
    {
        auto [a, b] = es[i];
        for (int j = i + 1; j < m; j++)
        {
            auto [c, d] = es[j];
            bool cross = (a < c && c < b && b < d) || (c < a && a < d && d < b);
            if (cross)
            {
                g[i].push_back(j);
                g[j].push_back(i);
            }
        }
    }
    vector<int> col(m, -1);
    for (int s = 0; s < m; s++)
    {
        if (col[s] >= 0)
        {
            continue;
        }
        col[s] = 0;
        queue<int> q;
        q.push(s);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int v : g[u])
```

```
            {
                if (col[v] < 0)
                {
                    col[v] = col[u] ^ 1;
                    q.push(v);
                }
                else if (col[v] == col[u])
                {
                    return nullopt;
                }
            }
        }
    }
    return col;
}

long long faceCount(long long v, long long e, long long comp = 1)
{
    // v、e、comp 是平面图的点数、边数和连通块数；返回任一平面嵌入的面数。
    if (e >= v)
    {
        return e - v + comp + 1;
    }
    return comp - (v - e) + 1;
}
}
```

`chordPlanar(n, es)` 返回每条弦位于圆内或圆外的 $0/1$ 方案，无解返回空，$O(m^2)$ 时间与空间。共享端点的弦不算交叉。`faceCount(v,e,comp)` 返回 `long long` 面数 $e - v + comp + 1$；调用方须保证该计数在 64 位内，空图应传 `comp=0`，得到唯一的外部面。本条标题刻意不写“一般平面图判定”，任意图仍需真正的平面性算法；弦分侧也可编码为 5.4.001 Tarjan 2-SAT。

5.10.003 支配树（DAG / 一般图）

从入口 rt 出发，点 w 支配点 v 指的是「从 rt 到 v 的每条路径都经过 w」。所有支配 v 的点排成一条链，链上离 v 最近的那个（$\neq v$）就是 v 的直接支配点 $\mathrm{idom}(v)$；以 idom 为父指针得到的树即支配树，回答「删掉某点后哪些点从入口不可达」这类问题。这里用 Lengauer-Tarjan：先对图 DFS 建树、按 DFS 序 dfn 重标号；对每个点求半支配点 $\mathrm{sdom}(v)$——即所有满足「存在一条到 v 的路径、其中间点 dfn 都大于 v」的起点里 dfn 最小者，它是 idom 的良好近似。按 dfn 逆序处理，用带路径压缩的并查集 `dsu`（`find` 顺带把路径上 sdom 最小的顶点记进标签 `lab`）在已处理部分查询最小 sdom；再借「若 u 的 sdom 等于 v 的 sdom 则 $\mathrm{idom}(v) = \mathrm{sdom}(v)$，否则等于 $\mathrm{idom}(u)$」的定理两趟推出全部 idom。此实现对一般有向图（含环、DAG）都适用，DAG 也可退化成按拓扑序求前驱在支配树上的 LCA，但这份通用版无需区分。

```
struct Dominator
{
    // 点数与本次 DFS 到达的点数。
    int n, ti = 0;
    // 原图、按 DFS 序存的反图与半支配点桶。
```

```
    vector<vector<int>> g, rg, buc;
    // DFS 序及反查、DFS 父点、半支配点、直接支配点、并查集与最优标签。
    vector<int> dfn, rev, par, sdom, dom, dsu, lab;

    Dominator(const vector<vector<int>> &g)
        : n(g.size()), g(g), rg(n + 1), buc(n + 1), dfn(n), rev(n + 1), par(n + 1), sdom(n +
        ↪ 1), dom(n + 1), dsu(n + 1),
          lab(n + 1)
    {
        // g 是有向邻接表；构造求解器。
    }

    void buildDfs(int rt)
    {
        // 从 rt 递归 DFS，并按 DFS 序建立树父点与反图。
        dfn[rt] = ++ti;
        rev[ti] = rt;
        auto dfs = [&](auto &&self, int u) -> void
        {
            for (int v : g[u])
            {
                assert(0 <= v && v < n); // 调试检查，可删。
                if (!dfn[v])
                {
                    dfn[v] = ++ti;
                    rev[ti] = v;
                    par[ti] = dfn[u];
                    self(self, v);
                }
            }
        };
        dfs(dfs, rt);
        for (int u = 0; u < n; u++)
        {
            if (!dfn[u])
            {
                continue;
            }
            for (int v : g[u])
            {
                if (dfn[v])
                {
                    rg[dfn[v]].push_back(dfn[u]);
                }
            }
        }
        for (int i = 1; i <= ti; i++)
        {
            sdom[i] = dsu[i] = lab[i] = i;
        }
    }

```

```
    int find(int u)
    {
        // 返回并查集路径上半支配序最小的标签，并递归压缩路径。
        if (u == dsu[u])
        {
            return u;
        }
        if (dsu[u] == dsu[dsu[u]])
        {
            return lab[u];
        }
        auto compress = [&](auto &&self, int x) -> int
        {
            int p = dsu[x];
            if (p == dsu[p])
            {
                return x;
            }
            int base = self(self, p);
            if (sdom[lab[p]] < sdom[lab[x]])
            {
                lab[x] = lab[p];
            }
            dsu[x] = base;
            return base;
        };
        compress(compress, u);
        return lab[u];
    }

    vector<int> work(int rt)
    {
        // rt 是入口；返回直接支配点，不可达点为 -1。
        assert(0 <= rt && rt < n); // 调试检查，可删。
        ti = 0;
        rg.assign(n + 1, {});
        buc.assign(n + 1, {});
        dfn.assign(n, 0);
        rev.assign(n + 1, 0);
        par.assign(n + 1, 0);
        sdom.assign(n + 1, 0);
        dom.assign(n + 1, 0);
        dsu.assign(n + 1, 0);
        lab.assign(n + 1, 0);
        buildDfs(rt);
        for (int i = ti; i; i--)
        {
            for (int v : rg[i])
            {
                sdom[i] = min(sdom[i], sdom[find(v)]);
            }
            if (i > 1)
```

```
            {
                buc[sdom[i]].push_back(i);
            }
            for (int v : buc[i])
            {
                int u = find(v);
                dom[v] = sdom[u] == sdom[v] ? sdom[v] : u;
            }
            if (i > 1)
            {
                dsu[i] = par[i];
            }
        }
        for (int i = 2; i <= ti; i++)
        {
            if (dom[i] != sdom[i])
            {
                dom[i] = dom[dom[i]];
            }
        }
        vector<int> ans(n, -1);
        ans[rt] = rt;
        for (int i = 2; i <= ti; i++)
        {
            ans[rev[i]] = rev[dom[i]];
        }
        return ans;
    }
};
```

用 `Dominator dm(g)` 建器，`work(rt)` 返回直接支配点数组：`ans[rt]=rt`，不可达点为 -1。每次 `work` 都会重建 DFS 序、反图、桶和并查集状态，因此同一对象可更换入口重复求解；DFS 与并查集路径压缩均直接递归，深链需确认系统栈足够。多个真实入口可先加虚拟源。Lengauer-Tarjan 为近线性时间，常写作 $O((n+m)\alpha(n))$，空间 $O(n+m)$。DAG 也可按拓扑序配合支配树 LCA 增量构造，但通用图直接使用本实现。

下面用菱形 DAG 检查汇合点的直接支配者，再用带环图检查环内支配关系和不可达点。

```
signed main()
{
    vector<vector<int>> a{{1, 2}, {3}, {3}, {}};
    assert(Dominator(a).work(0) == vector<int>({0, 0, 0, 0}));

    vector<vector<int>> b{{1}, {2}, {1, 3}, {}, {}};
    assert(Dominator(b).work(0) == vector<int>({0, 0, 1, 2, -1}));
    return 0;
}
```

5.10.004 竞赛图与 Landau 定理

竞赛图是每对顶点之间恰有一条有向边的图（n 个人两两比赛的胜负关系）。Landau 定理刻画了哪些出度（胜场）多重集是合法的：把出度升序排成 $d_1 \le \dots \le d_n$，则存在对应竞赛图当且仅当每个前缀和 $\sum_{i=1}^{k} d_i \ge \binom{k}{2}$，且总和 $\sum d_i = \binom{n}{2}$。直觉是前 k 小的选手内部至少产生 $\binom{k}{2}$ 场胜负、这些胜场只能落在他们自己头上，故前缀和有下界；总胜场数恒等于总场数。这两个条件不仅必要也充分。

```cpp
bool tourScore(vector<long long> a)
{
    // a 是竞赛图各点出度；返回是否存在一个竞赛图恰好具有该出度多重集。
    sort(a.begin(), a.end());
    int n = a.size();
    __int128 sum = 0;
    for (int k = 1; k <= n; k++)
    {
        if (a[k - 1] < 0 || a[k - 1] >= n)
        {
            return false;
        }
        sum += (__int128)a[k - 1];
        if (sum < (__int128)k * (k - 1) / 2)
        {
            return false;
        }
    }
    return sum == (__int128)n * (n - 1) / 2;
}
```

`tourScore(a)` 接受任意顺序的出度多重集（函数内部会排序），返回是否可图；只判存在性、不构造具体竞赛图（要构造可用逐步满足前缀下界的贪心）。$O(n \log n)$ 时间、$O(n)$ 空间。例如 `{0,1,2}`、`{1,1,1}` 都返回 `true`，`{0,0,3}` 返回 `false`。无向图度序列的可图性是另一套判据，见 5.10.005 度序列可图性 / Erdős-Gallai；竞赛图本身的强连通性、哈密顿路径等结论可结合 5.3.001 Tarjan 强连通分量。

5.10.005 度序列可图性 / Erdős-Gallai

Erdős-Gallai 定理判定一个非负整数多重集能否作为无向简单图（无自环、无重边）的度序列。把度数降序排成 $d_1 \ge \dots \ge d_n$，可图当且仅当总和 $\sum d_i$ 为偶，且对每个 k 有 $\sum_{i=1}^{k} d_i \le k(k-1) + \sum_{i>k} \min(d_i, k)$。右端的含义是：前 k 个高度点的度数，最多有 $k(k-1)$ 连在彼此之间（每对贡献 2），其余每个点最多向它们连 $\min(d_i, k)$ 条。$\sum_{i>k} \min(d_i, k)$ 可用「找到第一个 $d_i < k$ 的位置 p」把它拆成前段整块贡献 k、后段贡献原度数（后缀和），从而每个 k 都 $O(\log n)$ 二分算出，总 $O(n \log n)$。

```cpp
bool graphical(vector<long long> a)
{
    // a 是无向简单图各点度数；返回是否存在一个简单图恰好具有该度数多重集。
    sort(a.begin(), a.end(), greater<>());
    int n = a.size();
    vector<long long> s(n + 1);
    for (int i = 0; i < n; i++)
```

```
    {
        if (a[i] < 0 || a[i] >= n)
        {
            return false;
        }
        if (s[i] > LLONG_MAX - a[i])
        {
            return false;
        }
        s[i + 1] = s[i] + a[i];
    }
    if (s[n] & 1)
    {
        return false;
    }
    for (int k = 1; k <= n; k++)
    {
        int p = lower_bound(a.begin() + k, a.end(), k, greater<>()) - a.begin();
        long long rhs = 1LL * k * (k - 1) + 1LL * (p - k) * k + s[n] - s[p];
        if (s[k] > rhs)
        {
            return false;
        }
    }
    return true;
}
```

`graphical(a)` 接受任意顺序的度数多重集，度数须落在 $[0, n)$ 内且前缀和与判定式在 64 位内，返回是否可图。只判存在性；要真正构造出边集应改用 Havel-Hakimi 贪心（每次取最大度点连向次大的若干点）。$O(n \log n)$ 时间、$O(n)$ 空间。例如 `{2,2,2}`、`{3,3,2,2,2,2}` 返回 `true`，`{3,3,1,1}` 返回 `false`。竞赛图（有向、任意两点一条边）的出度可图性判据不同，见 5.10.004 竞赛图与 Landau 定理；二部图度序列用 Gale-Ryser 定理（本书未单列，思路类似）。

5.10.006 最大团、极大团计数

最大团（任意两点相邻的最大点集）是 NP 难的，但用位运算 + 分支限界在 $n \le 64$ 时很快。`maxClique` 递归维护当前团 `r` 与还能加入的候选集 `p`（都是 64 位掩码），剪枝是「当前团大小 + 候选数 $\le$ 已知最优则回退」；每步选一个枢轴 u（候选中与 `p` 交集最大的点）只对 `p & ~e[u]`（非 u 邻居的候选）分支，因为最大团必含 u 或 u 的某个非邻居，这样能砍掉大量重复分支。补图上的最大团就是原图最大独立集，`maxIndSet` 据此把邻接取反后复用 `maxClique`。枚举全部极大团（无法再加点的团，未必最大）用带枢轴的 Bron-Kerbosch：维护当前团 `r`、候选 `p`、已排除 `x`，`p|x` 为空即输出一个极大团，同样按枢轴分支，最坏 $O(3^{n/3})$ 个极大团。

```
struct Clique64
{
    // n 是图的点数；best 是搜索过程中已知的最大团大小。
    int n, best = 0;
    // ans 是当前最大团的点位集（第 i 位为 1 表示点 i 在团中）。
```

```
    unsigned long long ans = 0;
    // e[u] 的第 v 位为 1 表示存在无向边 u-v。
    vector<unsigned long long> e;

    Clique64(int n = 0) : n(n), e(n)
    {
        // n 是点数；构造空图。
        assert(0 <= n && n <= 64); // 调试检查，可删。
    }

    void add(int u, int v)
    {
        // 加入无向边 u-v；自环与团定义无关，直接忽略。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        if (u == v)
        {
            return;
        }
        e[u] |= 1ULL << v;
        e[v] |= 1ULL << u;
    }

    void dfs(unsigned long long r, unsigned long long p)
    {
        // r 是已选团，p 是仍可加入的点集；用分支限界更新最大团，无返回值。
        int z = popcount(r);
        if (z + popcount(p) <= best)
        {
            return;
        }
        if (!p)
        {
            best = z;
            ans = r;
            return;
        }
        int u = countr_zero(p);
        for (auto q = p; q; q &= q - 1)
        {
            int v = countr_zero(q);
            if (popcount(p & e[v]) > popcount(p & e[u]))
            {
                u = v;
            }
        }
        unsigned long long q = p & ~e[u];
        while (q)
        {
            int v = countr_zero(q);
            dfs(r | 1ULL << v, p & e[v]);
            p &= ~(1ULL << v);
            q &= q - 1;
```

```
            if (z + popcount(p) <= best)
            {
                return;
            }
        }
    }

    vector<int> maxClique()
    {
        // 无参数；返回当前图的一组最大团顶点。
        best = 0;
        ans = 0;
        dfs(0, n == 64 ? ~0ULL : (1ULL << n) - 1);
        vector<int> a;
        for (int i = 0; i < n; i++)
        {
            if (ans >> i & 1)
            {
                a.push_back(i);
            }
        }
        return a;
    }

    vector<int> maxIndSet()
    {
        // 无参数；返回当前图的一组最大独立集顶点。
        unsigned long long all = n == 64 ? ~0ULL : (1ULL << n) - 1;
        auto old = e;
        for (int i = 0; i < n; i++)
        {
            e[i] = all & ~(old[i] | 1ULL << i);
        }
        auto res = maxClique();
        e = move(old);
        return res;
    }

    template <class F>
    void maxCliques(F f)
    {
        // f 接收一个极大团的点位集；枚举全部极大团并逐个调用 f，无返回值。
        auto go = [&](auto &&self, unsigned long long r, unsigned long long p, unsigned long
        ↪  long x) -> void
        {
            if (!(p | x))
            {
                f(r);
                return;
            }
            int u = countr_zero(p | x);
            unsigned long long q = p & ~e[u];
```

```
            while (q)
            {
                int v = countr_zero(q);
                self(self, r | 1ULL << v, p & e[v], x & e[v]);
                p &= ~(1ULL << v);
                x |= 1ULL << v;
                q &= q - 1;
            }
        };
        go(go, 0, n == 64 ? ~0ULL : (1ULL << n) - 1, 0);
    }
};

long long weightClique(int n, const vector<long long> &val, const vector<pair<int, int>> &e,
 ↪ long long mod)
{
    // n 是点数，val 是点权，e 是简单无向边，mod 是正模数；返回所有非空团点权乘积之和模 mod。
    assert((int)val.size() == n && mod > 0); // 调试检查，可删。
    vector<vector<int>> g(n);
    vector<vector<char>> adj(n, vector<char>(n));
    for (auto [u, v] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v && !adj[u][v]); // 调试检查，可删。
        g[u].push_back(v);
        g[v].push_back(u);
        adj[u][v] = adj[v][u] = 1;
    }
    int lim = sqrt(2.0 * e.size()) + 1;
    vector<char> alive(n, 1);
    auto go = [&](auto &&self, int rem) -> long long
    {
        if (rem == 0) return 0;
        int x = -1;
        for (int u = 0; u < n; u++)
        {
            if (alive[u] && (int)g[u].size() < lim)
            {
                x = u;
                break;
            }
        }
        if (x == -1)
        {
            vector<int> vs;
            for (int u = 0; u < n; u++)
            {
                if (alive[u]) vs.push_back(u);
            }
            long long ans = 0;
            for (int mask = 1; mask < (1 << vs.size()); mask++)
            {
                long long cur = 1;
```

```
                bool ok = true;
                for (int i = 0; i < (int)vs.size() && ok; i++)
                {
                    if (!(mask >> i & 1)) continue;
                    cur = (__int128)cur * (val[vs[i]] % mod) % mod;
                    for (int j = 0; j < i; j++)
                    {
                        if ((mask >> j & 1) && !adj[vs[i]][vs[j]])
                        {
                            ok = false;
                            break;
                        }
                    }
                }
                if (ok) ans = (ans + cur) % mod;
            }
            return ans;
        }
        long long ans = 0;
        int d = g[x].size();
        for (int mask = 0; mask < (1 << d); mask++)
        {
            long long cur = val[x] % mod;
            bool ok = true;
            for (int i = 0; i < d && ok; i++)
            {
                if (!(mask >> i & 1)) continue;
                int u = g[x][i];
                if (!alive[u])
                {
                    ok = false;
                    break;
                }
                cur = (__int128)cur * (val[u] % mod) % mod;
                for (int j = 0; j < i; j++)
                {
                    if ((mask >> j & 1) && !adj[u][g[x][j]])
                    {
                        ok = false;
                        break;
                    }
                }
            }
            if (ok) ans = (ans + cur) % mod;
        }
        alive[x] = 0;
        return (ans + self(self, rem - 1)) % mod;
    };
    return (go(go, n) + mod) % mod;
}
```

构造 `Clique64 g(n)`（$n \le 64$）、逐边 `g.add(u, v)`，然后 `maxClique()` 返回一组最大团的点编号、

`maxIndSet()` 返回一组最大独立集、`maxCliques(f)` 对每个极大团把其 64 位点掩码 `r` 交给回调 `f`。求最大团/独立集与极大团枚举最坏都是 $O(3^{n/3})$、$O(n)$ 递归栈，随机稀疏图上剪枝极快。

枚举全部极大团时，回调参数是 64 位点集：

```
signed main()
{
    Clique64 g(3);
    g.add(0, 1);
    g.add(1, 2);
    int cnt = 0;
    g.maxCliques([&](unsigned long long mask)
    {
        cnt++;
        assert(mask);
    });
    assert(cnt == 2);
}
```

`weightClique(n,val,e,mod)` 用于 $n, m \le 100$、需要累加每个非空团的点权乘积的情形。它不断选原图度数小于 $\lfloor\sqrt{2m}\rfloor+1$ 的未删点，枚举其仍存邻居的全部子集，因每个团按最早删点只算一次而不重不漏；若再没有低度点，剩余点数小于该阈值，直接枚举其所有子集。邻接矩阵使子集合法性检查为常数时间，时间为 $O(n2^{\sqrt{2m}+1})$，空间为 $O(n^2+m)$；`mod` 为正，乘积中间量使用 `__int128`。图非常大或有特殊结构时不要硬套指数算法：弦图上最大团用 5.10.001 弦图、完美消除序列与判错 的 PEO 多项式求；二分图最大独立集 = 总点数 − 最大匹配，用 5.6.008 最大独立集。最小点染色数（色数）与最大独立集划分见 5.10.007 最大独立集与点染色，那里给出至多 24 点的精确染色 DP。

5.10.007 最大独立集与点染色

最大独立集直接调用 5.10.006 最大团、极大团计数 的 `Clique64::maxIndSet()`，不再维护第二份补图搜索。点染色用独立集子集 DP：预处理每个点集是否为独立集，再把全集划分成尽量少的独立集。固定让当前集合的最低位属于新取的独立集，可消除颜色类排列造成的重复枚举而不丢最优解。它有明确的 $O(3^n)$ 最坏界，适合 $n \le 24$ 且内存充足。

```
int chromaticNum(const vector<unsigned long long> &e)
{
    // e 是简单无向图邻接位集；返回最小点染色数。
    int n = e.size();
    assert(n <= 24); // 调试检查，可删：状态数必须能承受。
    size_t z = 1ULL << n;
    vector<bool> ind(z);
    ind[0] = true;
    for (size_t s = 1; s < z; s++)
    {
        int v = countr_zero((unsigned long long)s);
        size_t t = s ^ (1ULL << v);
        ind[s] = ind[t] && !(e[v] & t);
    }
    vector<int> f(z, n);
```

```
    f[0] = 0;
    for (size_t s = 1; s < z; s++)
    {
        size_t b = s & -s;
        for (size_t t = s; t; t = (t - 1) & s)
        {
            if ((t & b) && ind[t])
            {
                chmin(f[s], f[s ^ t] + 1);
            }
        }
    }
    return f.back();
}
```

`chromaticNum(e)` 返回最小色数，$n \le 24$，时间 $O(3^n)$、空间 $O(2^n)$；它不恢复方案，但运行时间和峰值内存只由 n 决定，表现较稳定。稀疏或结构明显、并且需要一组具体染色时，可改用 5.10.008 DSATUR 实践型精确点染色。

5.10.008 DSATUR 实践型精确点染色

DSATUR 仍做精确搜索，但分支顺序由当前部分染色决定：优先选择「已染邻居使用的颜色种类最多」的点，再以未染邻居数打破平局。先贪心得到一组合法染色作为上界，随后枚举已有颜色或至多一个新颜色，并用当前颜色数剪枝。它是确定性算法且能恢复方案，通常比子集 DP 更快，但最坏时间仍为指数级，不能把 64 位邻接掩码的容量误当成保证可通过的规模。

```
struct Coloring
{
    int k;              // 使用的颜色数。
    vector<int> color; // 每个点的颜色，编号从 0 开始。
};

class Dsatur
{
    int n, best; // n 是点数，best 是当前最优颜色数。
    vector<unsigned long long> e; // e[u] 是 u 的邻接位集。
    vector<int> col, ans; // col 是搜索中的染色（-1 未染），ans 是最优染色。

    unsigned long long forbidden(int u) const
    {
        // u 是点；返回 u 的已染色邻居所用颜色的位集。
        unsigned long long mask = 0, q = e[u];
        while (q)
        {
            int v = countr_zero(q);
            q &= q - 1;
            if (col[v] >= 0)
            {
                mask |= 1ULL << col[v];
```

```
            }
        }
        return mask;
    }

    int pick() const
    {
        // 无参数；返回饱和度最大、其次未染色邻居最多的未染色点。
        int at = -1, sat = -1, deg = -1;
        unsigned long long left = 0;
        for (int u = 0; u < n; u++)
        {
            if (col[u] < 0)
            {
                left |= 1ULL << u;
            }
        }
        for (int u = 0; u < n; u++)
        {
            if (col[u] >= 0)
            {
                continue;
            }
            int s = popcount(forbidden(u));
            int d = popcount(e[u] & left);
            if (s > sat || (s == sat && d > deg))
            {
                at = u;
                sat = s;
                deg = d;
            }
        }
        return at;
    }

    void dfs(int done, int used)
    {
        // done 是已染色点数，used 是已用颜色数；分支定界搜索，更新 best 与 ans。
        if (used >= best)
        {
            return;
        }
        if (done == n)
        {
            best = used;
            ans = col;
            return;
        }
        int u = pick();
        unsigned long long ban = forbidden(u);
        for (int c = 0; c < used; c++)
        {
```

```
            if (ban >> c & 1)
            {
                continue;
            }
            col[u] = c;
            dfs(done + 1, used);
            col[u] = -1;
        }
        if (used + 1 < best)
        {
            col[u] = used;
            dfs(done + 1, used + 1);
            col[u] = -1;
        }
    }

public:
    Dsatur(vector<unsigned long long> e) : n(e.size()), best(n + 1), e(move(e)), col(n, -1)
    {
        // e 是简单无向图邻接位集；构造精确染色器。
        assert(n <= 64); // 调试检查，可删。
        for (int u = 0; u < n; u++)
        {
            assert(!(this->e[u] >> u & 1)); // 调试检查，可删。
            for (int v = 0; v < n; v++)
            {
                assert((this->e[u] >> v & 1) == (this->e[v] >> u & 1)); // 调试检查，可删。
            }
        }
    }

    Coloring solve()
    {
        // 返回最少颜色数和一组染色方案。
        fill(col.begin(), col.end(), -1);
        int used = 0;
        for (int step = 0; step < n; step++)
        {
            int u = pick();
            unsigned long long ban = forbidden(u);
            int c = 0;
            while (ban >> c & 1)
            {
                c++;
            }
            col[u] = c;
            used = max(used, c + 1);
        }
        best = used;
        ans = col;
        fill(col.begin(), col.end(), -1);
        dfs(0, 0);
```

```
        return {best, ans};
    }
};
```

`Dsatur(e).solve()` 返回精确色数与方案；同一层只新增一个颜色编号，消除颜色置换造成的重复。最坏时间仍为指数级，额外空间为 $O(n)$ 递归栈和状态；实现不随机、没有重试语义，只依赖 C++20 位运算。稀疏或结构明显的图通常优先试本版；需要 $n \le 24$ 下由规模决定的稳定时间和内存界时用 5.10.007 最大独立集与点染色。弦图色数可由 5.10.001 弦图、完美消除序列与判错 多项式求，二分图只需 BFS 二染色。

```
signed main()
{
    vector<unsigned long long> e(5);
    for (int u = 0; u < 5; u++)
    {
        int v = (u + 1) % 5;
        e[u] |= 1ULL << v;
        e[v] |= 1ULL << u;
    }
    Coloring res = Dsatur(e).solve();
    assert(res.k == 3 && res.color == vector<int>({0, 1, 0, 1, 2}));
}
```

5.10.009 有向图最小点基

有向图的最小点基是「选最少的起点，使得从它们出发能到达图中所有点」的一个最小点集。做法是先用 5.3.001 Tarjan 强连通分量 的 `tarjanSCC` 把图缩成 SCC-DAG：同一强连通分量内部互相可达，只需选其中一点即可覆盖整块。缩点后统计每个分量的入度，凡入度为 0 的分量都没有任何外部点能到达它，必须自己选一个起点；而所有入度非零的分量都能被某个入度为 0 的分量沿 DAG 路径到达，无需另选。因此最小点基的大小等于入度为 0 的分量数，从每个这样的分量各取一个代表点（这里取分量内最小编号点）即得一组方案。

```
// 依赖 5.3.001 Tarjan 强连通分量
vector<int> minPtBasis(const vector<vector<int>> &g)
{
    // g 是有向图邻接表；返回升序最小点基，每个源 SCC 取最小编号点。
    auto id = tarjanSCC(g);
    if (g.empty())
    {
        return {};
    }
    int n = g.size(), m = *max_element(id.begin(), id.end()) + 1;
    vector<int> in(m), mn(m, n);
    for (int u = 0; u < n; u++)
    {
        mn[id[u]] = min(mn[id[u]], u);
        for (int v : g[u])
        {
```

```
            if (id[u] != id[v])
            {
                in[id[v]]++;
            }
        }
    }
    vector<int> ans;
    for (int i = 0; i < m; i++)
    {
        if (!in[i])
        {
            ans.push_back(mn[i]);
        }
    }
    sort(ans.begin(), ans.end());
    return ans;
}
```

调用前需备好 5.3.001 Tarjan 强连通分量 的 `tarjanSCC`。`minPtBasis(g)` 吃邻接表（点从 0 编号），返回升序的代表点列表；空图返回空。缩点与统计都是线性的，整体 $O(n+m)$ 时间与空间。注意「最小点基」只保证覆盖可达性、且大小唯一，但代表点的选择不唯一（这里固定取分量最小编号）。若要的是「最少边使全图强连通」（强连通化）则是另一问题，答案为 $\max(\text{源分量数}, \text{汇分量数})$，同样基于 SCC-DAG；缩点后的其它 DAG 分析（拓扑、最长路）也从 5.3.001 Tarjan 强连通分量 出发。

5.10.010 有向无环定向与色多项式

图 G 的色多项式 $P_G(x)$ 是「用 x 种颜色作合法点染色的方案数」，它是关于 x 的 n 次多项式。关键观察：每个颜色类必是一个独立集，于是「用恰好 k 种有标号颜色染全图」的方案数等于把全集划分/覆盖成 k 个（可空）独立集的有序方式数。用子集 DP 求：`ind[s]` 标记点集 s 是否独立（增量判定），`dp` 每轮与「加入一个新颜色类 = 任选一个独立子集」做子集卷积，k 轮后 `dp[full]` 就是 $P_G(k)$。这样得到 $n+1$ 个样本 $P_G(0),\dots,P_G(n)$，因 $\deg P_G = n$ 足以唯一确定它。再用 Newton 前向差分把样本转成下降幂基系数，边算边乘 $x(x-1)\cdots$ 的展开多项式 `cur`，累加成普通幂次系数。有了色多项式，Stanley 定理立刻给出无环定向数 $= (-1)^n P_G(-1)$。

```
// 依赖 1.1.008 静态取模类
vector<array<Z, 21>> rankedZeta(vector<Z> f, int n)
{
    // f 是大小为 2^n 的子集函数；返回按子集大小分层后的子集 Zeta 变换。
    vector<array<Z, 21>> a(f.size());
    for (int s = 0; s < (int)f.size(); s++)
    {
        a[s][popcount((unsigned)s)] = f[s];
    }
    for (int i = 0; i < n; i++)
    {
        for (int s = 0; s < (1 << n); s++)
        {
            if (s >> i & 1)
```

```
            {
                for (int d = 0; d <= n; d++)
                {
                    a[s][d] += a[s ^ (1 << i)][d];
                }
            }
        }
    }
    return a;
}

vector<Z> rankedMobius(vector<array<Z, 21>> a, int n)
{
    // a 是按子集大小分层后的 Zeta 值；返回对应的子集函数。
    for (int i = 0; i < n; i++)
    {
        for (int s = 0; s < (1 << n); s++)
        {
            if (s >> i & 1)
            {
                for (int d = 0; d <= n; d++)
                {
                    a[s][d] -= a[s ^ (1 << i)][d];
                }
            }
        }
    }
    vector<Z> f(1 << n);
    for (int s = 0; s < (1 << n); s++)
    {
        f[s] = a[s][popcount((unsigned)s)];
    }
    return f;
}

vector<Z> subsetConv(const vector<Z> &a, const vector<Z> &b)
{
    // a、b 是相同 n 位全集上的子集函数；返回 (a*b)[S]=sum_{T subset S}a[T]b[S-T]。
    int n = countr_zero((unsigned)a.size());
    assert(a.size() == b.size() && n <= 20); // 调试检查，可删。
    auto x = rankedZeta(a, n), y = rankedZeta(b, n);
    for (int s = 0; s < (1 << n); s++)
    {
        for (int d = n; d >= 0; d--)
        {
            Z z = 0;
            for (int i = 0; i <= d; i++)
            {
                z += x[s][i] * y[s][d - i];
            }
            x[s][d] = z;
        }
```

```
    }
    return rankedMobius(move(x), n);
}

vector<Z> transSubConv(vector<Z> s, vector<Z> x)
{
    // s、x 是同一位数的子集函数；返回映射 a->a*s 对 x 的转置作用结果。
    reverse(x.begin(), x.end());
    x = subsetConv(x, s);
    reverse(x.begin(), x.end());
    return x;
}

vector<Z> powerProjEgf(vector<Z> wt, const vector<Z> &s, int n)
{
    // wt 是线性泛函权，s[0] 必须为 0；返回 sum_T wt[T](s^k/k!)[T]，k=0..n。
    vector<Z> ans(n + 1), dp = move(wt);
    ans[0] = dp[0];
    for (int i = 0; i < n; i++)
    {
        vector<Z> nxt(1 << (n - 1 - i));
        for (int j = 0; j < n - i; j++)
        {
            int l = 1 << j, r = 2 << j;
            vector<Z> a(s.begin() + l, s.begin() + r);
            vector<Z> b(dp.begin() + l, dp.begin() + r);
            b = transSubConv(move(a), move(b));
            for (int k = 0; k < (int)b.size(); k++)
            {
                nxt[k] += b[k];
            }
        }
        dp = move(nxt);
        ans[i + 1] = dp[0];
    }
    return ans;
}

vector<Z> chromPoly(const vector<unsigned long long> &g)
{
    // g 是允许自环的无向图邻接位集；返回按次数升序的色多项式系数，自环图返回全零，支持至多 20 点。
    int n = g.size(), N = 1 << n;
    assert(n <= 20 && n < Z::mod()); // 调试检查，可删：模数须为大于 n 的素数。
    for (int u = 0; u < n; u++)
    {
        if (g[u] >> u & 1) return vector<Z>(n + 1);
    }
    vector<char> ind(N, true);
    for (int s = 1; s < N; s++)
    {
        int u = countr_zero((unsigned)s);
        ind[s] = ind[s ^ (1 << u)] && !(g[u] & (s ^ (1 << u)));
```

```
    }
    vector<Z> f(N), wt(N);
    for (int s = 0; s < N; s++)
    {
        if (ind[s]) f[s] = 1;
    }
    wt[N - 1] = 1;
    Z c = f[0];
    f[0] -= c;
    vector<Z> egf = powerProjEgf(wt, f, n);
    vector<Z> val(n + 1), pw(n + 1), fact(n + 1, 1);
    for (int i = 1; i <= n; i++)
    {
        fact[i] = fact[i - 1] * i;
    }
    for (int k = 0; k <= n; k++)
    {
        if (k == 0) pw[k] = 1;
        else pw[k] = pw[k - 1] * c;
        Z sum = 0;
        for (int i = 0; i <= k; i++)
        {
            sum += egf[i] * pw[k - i] / fact[k - i];
        }
        val[k] = sum * fact[k];
    }
    vector<Z> ans(n + 1), cur{1};
    Z fac = 1;
    for (int k = 0; k <= n; k++)
    {
        Z d = val[0] / fac;
        for (int i = 0; i < (int)cur.size(); i++)
        {
            ans[i] += d * cur[i];
        }
        for (int i = 0; i + 1 < (int)val.size(); i++)
        {
            val[i] = val[i + 1] - val[i];
        }
        val.pop_back();
        if (k == n)
        {
            break;
        }
        fac *= k + 1;
        vector<Z> nxt(cur.size() + 1);
        for (int i = 0; i < (int)cur.size(); i++)
        {
            nxt[i] -= cur[i] * k;
            nxt[i + 1] += cur[i];
        }
        cur.swap(nxt);
```

```
    }
    return ans;
}

inline void tutteRanked(vector<Z> &a, int n, bool inverse)
{
    // a 按子集和次数压平存储，n 是位数；执行分层 Zeta 或 Mobius 变换。
    int N = 1 << n, w = n + 1;
    for (int bit = 0; bit < n; bit++)
    {
        for (int s = 0; s < N; s++)
        {
            if (s >> bit & 1)
{
Z *to = a.data() + (size_t)s * w;
Z *from = a.data() + (size_t)(s ^ (1 << bit)) * w;
for (int d = 0; d <= n; d++)
{
if (inverse) to[d] -= from[d]; else to[d] += from[d];
}
}
        }
    }
}

inline vector<Z> tutteSetExp(const vector<Z> &f, int n)
{
    // f 是常数项为 0 的 n 位集合幂级数；返回集合卷积意义下的 exp(f)。
    int N = 1 << n, w = n + 1;
    vector<Z> a((size_t)N * w);
    for (int s = 0; s < N; s++)
    {
        a[(size_t)s * w + popcount((unsigned)s)] = f[s];
    }
    tutteRanked(a, n, false);
    vector<Z> log(w);
    for (int s = 0; s < N; s++)
    {
        Z *v = a.data() + (size_t)s * w;
        copy(v, v + w, log.begin());
        v[0] = 1;
        for (int d = 1; d <= n; d++)
        {
            Z sum = 0;
            for (int k = 1; k <= d; k++)
            {
                sum += Z(k) * log[k] * v[d - k];
            }
            v[d] = sum / Z(d);
        }
    }
    tutteRanked(a, n, true);
```

```
    vector<Z> ans(N);
    for (int s = 0; s < N; s++)
    {
        ans[s] = a[(size_t)s * w + popcount((unsigned)s)];
    }
    return ans;
}

inline Z tutteEval(const vector<unsigned long long> &g, Z x, Z y)
{
    // g 是无自环无向图邻接位集；x、y 是 Tutte 多项式点值，点数不超过 20。
    int n = g.size(), N = 1 << n;
    assert(0 < n && n <= 20); // 调试检查，可删。
    vector<Z> connected(N), temp(N);
    vector<Z> geo(n + 1);
    for (int i = 1; i <= n; i++)
    {
        geo[i] = geo[i - 1] * y + Z(1);
    }
    for (int u = 0; u < n; u++)
    {
        int M = 1 << u;
        for (int s = 0; s < M; s++)
        {
            temp[s] = connected[s] * geo[popcount(g[u] & (unsigned long long)s)];
        }
        vector<Z> add(temp.begin(), temp.begin() + M);
        add = tutteSetExp(add, u);
        for (int s = 0; s < M; s++)
        {
            connected[M | s] = add[s];
        }
    }
    vector<char> vis(n), hasRoot(N);
    for (int start = 0; start < n; start++)
    {
        if (!vis[start])
{
vector<int> q{start};
vis[start] = 1;
for (int h = 0; h < (int)q.size(); h++)
{
int u = q[h];
for (int v = 0; v < n; v++)
{
    if ((g[u] >> v & 1) && !vis[v])
vis[v] = 1, q.push_back(v);
}
}
for (int s = 0; s < N; s++)
{
    if (s >> start & 1) hasRoot[s] = 1;
```

```
}
}
    }
    for (int s = 1; s < N; s++)
    {
        if (!hasRoot[s]) connected[s] *= x - Z(1);
    }
    return tutteSetExp(connected, n).back();
}
```

先准备 1.1.008 静态取模类 的素数域类型 `Z`，并要求模数严格大于 n。`chromPoly(g)` 返回色多项式系数；只需 Tutte 多项式在一点 (x,y) 的值时调用 `tutteEval(g,x,y)`，它复用分层 Zeta/Mobius 与集合幂级数指数，允许图不连通。两者均要求无自环简单无向图、$n \le 20$，时间 $O(n^2 2^n)$、空间 $O(n2^n)$。

5.10.011 拟阵交

拟阵交解决「在同一底集上选一个最大的、同时在两个拟阵 M_1、M_2 中都独立的子集」。很多经典问题是它的特例：二分图最大匹配（两侧各一个划分拟阵）、有向图最大公共森林、带颜色约束的生成树等。算法沿「交换图」增广：当前公共独立集为 I，对每个 $y \notin I$ 若 $I+y$ 在 M_1 中独立就作为源、在 M_2 中独立就作为汇；图中的边表示交换关系——删掉 I 中某个 x 后能否换进 y（保持某个拟阵独立）。找一条从源到汇的最短增广路，沿路翻转元素归属，$|I|$ 加一；找不到路时 I 即为最大。正确性由拟阵交的 min-max（König 型）定理保证。此实现用独立性回调 `first`、`second`（各接收一个 `vector<bool>` 选择标记、返回是否独立），对任意拟阵通用。

```
template <class M1, class M2>
vector<int> matroidMeet(int n, M1 first, M2 second)
{
    // n 是底集大小，first、second 判定选择集在两个拟阵中是否独立；返回升序最大公共独立集。
    vector<bool> in(n);
    while (true)
    {
        vector<int> pre(n, -2), q;
        for (int y = 0; y < n; y++)
        {
            if (!in[y])
            {
                auto a = in;
                a[y] = true;
                if (first(a))
                {
                    pre[y] = -1;
                    q.push_back(y);
                }
            }
        }
        int last = -1;
        for (int h = 0; h < (int)q.size() && last < 0; h++)
        {
            int x = q[h];
```

```
            if (!in[x])
            {
                auto a = in;
                a[x] = true;
                if (second(a))
                {
                    last = x;
                    break;
                }
                for (int y = 0; y < n; y++)
                {
                    if (in[y] && pre[y] == -2)
                    {
                        a[y] = false;
                        if (second(a))
                        {
                            pre[y] = x;
                            q.push_back(y);
                        }
                        a[y] = true;
                    }
                }
            }
            else
            {
                for (int y = 0; y < n; y++)
                {
                    if (!in[y] && pre[y] == -2)
                    {
                        auto a = in;
                        a[x] = false;
                        a[y] = true;
                        if (first(a))
                        {
                            pre[y] = x;
                            q.push_back(y);
                        }
                    }
                }
            }
        }
        if (last < 0)
        {
            break;
        }
        for (int x = last; x >= 0; x = pre[x])
        {
            in[x] = !in[x];
        }
    }
    vector<int> ans;
    for (int i = 0; i < n; i++)
```

```
    {
        if (in[i])
        {
            ans.push_back(i);
        }
    }
    return ans;
}
```

`matroidMeet(n, first, second)` 中 n 是底集大小、两个回调是独立性谕示（oracle），返回最终公共独立集的元素编号。增广轮数为 $O(r)$（r 是答案大小），每轮 $O(n^2)$ 次建边、每次建边调用一次独立性谕示，故独立性调用次数约 $O(r\,n^2)$，实际耗时取决于谕示自身复杂度（多为 $O(n)$ 或更高）。这是无权版；带权拟阵交需要给交换边赋代价并维护势函数，反复在交换图上求最短增广路，不能只按当前 BFS 层数或元素数贪心，本实现未含权。若两个拟阵恰是二分图两侧的划分拟阵，直接用更快的 5.6.003 Hopcroft-Karp；三个拟阵的交是 NP 难，无多项式算法。

最小调用可用两个“集合大小不超过 2”的拟阵谕示：

```
signed main()
{
    auto first = [](const vector<bool> &a)
    {
        return count(a.begin(), a.end(), true) <= 2;
    };
    auto second = [](const vector<bool> &a)
    {
        return !(a[0] && a[1]);
    };
    auto ans = matroidMeet(3, first, second);
    assert(ans.size() == 2);
}
```

5.10.012 小规模暴力双极定向

双极定向（bipolar orientation / st-orientation）把一个含边 s-t 的双连通无向图定向成无环图，使得 s 是唯一源（无入边）、t 是唯一汇（无出边），其余每个点都同时有入边和出边。它等价于求一个 st-编号：把点排成序列 $s = p_0, p_1, \dots, p_{n-1} = t$，使每个中间点 p_i 既有编号更小的邻居又有编号更大的邻居，然后每条边由编号小的一端指向大的一端。理论上 st-编号可由耳分解或 Lempel-Even-Cederbaum 在 $O(n+m)$ 内构造；这里给的是小规模（$n \le 12$）的直接实现——DFS 枚举从 s 到 t 的排列，末尾校验每个中间点两侧都有邻居，成立即按位置定向。它逻辑短，适合小数据直接求解，规模大时应换线性算法。

```
vector<pair<int, int>> bipolarOrd(const vector<vector<int>> &g, int s, int t)
{
    // g 是简单无向图；返回以 s 为唯一源、t 为唯一汇的边方向，无解返回空。
    int n = g.size();
    assert(n <= 12 && 0 <= s && s < n && 0 <= t && t < n && s != t); // 调试检查，可删。
    vector<unsigned long long> adj(n);
    for (int u = 0; u < n; u++)
```

```
    {
        for (int v : g[u])
        {
            assert(0 <= v && v < n && u != v); // 调试检查，可删。
            adj[u] |= 1ULL << v;
        }
    }
    for (int u = 0; u < n; u++)
    {
        for (int v = 0; v < n; v++)
        {
            assert((adj[u] >> v & 1) == (adj[v] >> u & 1)); // 调试检查，可删。
        }
    }
    if (!(adj[s] >> t & 1))
    {
        return {};
    }
    vector<int> p{s};
    vector<bool> use(n);
    use[s] = true;
    use[t] = true;
    auto dfs = [&](auto &&self) -> bool
    {
        if ((int)p.size() == n - 1)
        {
            p.push_back(t);
            unsigned long long left = 1ULL << s;
            for (int i = 1; i + 1 < n; i++)
            {
                int u = p[i];
                unsigned long long right = 0;
                for (int j = i + 1; j < n; j++)
                {
                    right |= 1ULL << p[j];
                }
                if (!(adj[u] & left) || !(adj[u] & right))
                {
                    p.pop_back();
                    return false;
                }
                left |= 1ULL << u;
            }
            return true;
        }
        for (int v = 0; v < n; v++)
        {
            if (!use[v])
            {
                use[v] = true;
                p.push_back(v);
                if (self(self))
```

```
                {
                    return true;
                }
                p.pop_back();
                use[v] = false;
            }
        }
        return false;
    };
    if (!dfs(dfs))
    {
        return {};
    }
    vector<int> at(n);
    for (int i = 0; i < n; i++)
    {
        at[p[i]] = i;
    }
    vector<pair<int, int>> ans;
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u])
        {
            if (u < v)
            {
                ans.push_back(at[u] < at[v] ? pair{u, v} : pair{v, u});
            }
        }
    }
    return ans;
}
```

`bipolarOrd(g, s, t)` 要求简单无向图含边 $s-t$ 且 $n \le 12$，返回 `{u,v}` 表示定向边 $u \to v$；无解返回空。有效 st-编号中每个中间点都有更小和更大的邻点，按编号从小到大定向后自然无环，并以 s, t 为唯一源汇。暴力最坏 $O(n!(n+m))$，只适合小图；它直接返回有向边，逻辑短，适合小数据或复核。规模较大时使用下一条线性版本；双连通分解见 5.3.005 割点与点双连通分量。

5.10.013 线性 st 编号

一般的 st 编号不必先假定整张图双连通：只要存在一个排列使 `s` 为首点、`t` 为末点，其余每点都同时有更小和更大的邻点即可。算法先把 `s` 标为已访问，再从 `t` 开始 DFS，使 DFS 序固定以 `s,t` 开头；`low[v]` 保存从 `v` 的 DFS 子树经至多一条返祖边能到达的最早顶点。初始双向链表只有 `s,t`，随后按 DFS 序处理其余点：由 `low[v]` 的符号决定把 `v` 插在父点前还是父点后，并把相反符号传给父点。lowpoint 的嵌套关系保证存在 st 编号时插入位置始终有效。最后把链表转成排名，并重新检查每个非 `s` 点有更小邻点、每个非 `t` 点有更大邻点；检查失败返回空。

```
optional<vector<int>> stNumbering(const vector<vector<int>> &g, int s, int t)
{
    // g 是简单无向图；返回 st 编号的排名，无法构造时返回空。
```

```
    int n = g.size();
    assert(n >= 2 && 0 <= s && s < n && 0 <= t && t < n && s != t); // 调试检查，可删。
    vector<int> pre(n, -1), low(n), par(n, -1), order;
    pre[s] = 0;
    low[s] = s;
    order.push_back(s);
    pre[t] = 1;
    low[t] = t;
    order.push_back(t);
    auto dfs = [&](auto &&self, int u) -> void
    {
        for (int v : g[u])
        {
            assert(0 <= v && v < n && v != u); // 调试检查，可删。
            if (pre[v] == -1)
            {
                pre[v] = order.size();
                low[v] = v;
                par[v] = u;
                order.push_back(v);
                self(self, v);
                if (pre[low[v]] < pre[low[u]])
                {
                    low[u] = low[v];
                }
            }
            else if (pre[v] < pre[low[u]])
            {
                low[u] = v;
            }
        }
    };
    dfs(dfs, t);
    if ((int)order.size() != n)
    {
        return nullopt;
    }

    vector<int> prev(n, -1), next(n, -1), sign(n);
    next[s] = t;
    prev[t] = s;
    sign[s] = -1;
    for (int i = 2; i < n; i++)
    {
        int v = order[i], p = par[v];
        if (p == -1)
        {
            return nullopt;
        }
        if (sign[low[v]] == -1)
        {
            int q = prev[p];
```

```
            if (q == -1)
            {
                return nullopt;
            }
            next[q] = v;
            next[v] = p;
            prev[v] = q;
            prev[p] = v;
            sign[p] = 1;
        }
        else
        {
            int q = next[p];
            if (q == -1)
            {
                return nullopt;
            }
            next[p] = v;
            next[v] = q;
            prev[v] = p;
            prev[q] = v;
            sign[p] = -1;
        }
    }

    vector<int> rank(n, -1);
    int u = s;
    for (int i = 0; i < n && u != -1; i++)
    {
        rank[u] = i;
        u = next[u];
    }
    if (u != -1 || rank[t] != n - 1 || count(rank.begin(), rank.end(), -1))
    {
        return nullopt;
    }
    for (int v = 0; v < n; v++)
    {
        bool left = false, right = false;
        for (int w : g[v])
        {
            assert(0 <= w && w < n && w != v); // 调试检查，可删。
            left |= rank[w] < rank[v];
            right |= rank[v] < rank[w];
        }
        if ((v != s && !left) || (v != t && !right))
        {
            return nullopt;
        }
    }
    return rank;
}
```

`stNumbering(g, s, t)` 返回长度为 n 的排名数组，`rank[s]=0`、`rank[t]=n-1`；按排名从小到大给每条边定向即可得到以 `s` 为唯一源、`t` 为唯一汇的无环定向。图不连通或不存在这种编号时返回空。实现确定、无随机失败，DFS 使用递归 lambda，时间 $O(n+m)$、空间 $O(n+m)$，深链需确认系统栈足够。它比上一条暴力版适用范围更广、复杂度更低，但只返回排名，不直接整理有向边；需要边表时遍历原边比较两端排名即可。

5.10.014 宽度 2 树分解

树分解把原图每个点放进若干个 bag，要求每条原边的两端同处某个 bag，且包含同一原图点的 bag 在分解树上连通；最大 bag 大小减一就是宽度。树宽至多 2 的图可用低度消元识别并构造：反复取当前度数不超过 2 的点 `v`，记录由 `v` 和剩余邻点组成的 bag；若恰有两个邻点，先给它们补边，使这两个点成为后续 bag 中必须共同出现的分隔集。若仍有存活点却再找不到低度点，则不存在宽度 2 分解。逆序恢复时，单邻点 bag 挂到一个包含该点的既有 bag，双邻点 bag 挂到一个同时包含这条补边两端的既有 bag；不同连通块的根 bag 可任意相连。

```cpp
struct TreeDecomp2
{
    // bag 是各树节点包含的原图点集，edge 是 bag 树的边。
    vector<vector<int>> bag;
    vector<pair<int, int>> edge;
};

optional<TreeDecomp2> treeDecomp2(int n, const vector<pair<int, int>> &edge)
{
    // n 是点数，edge 是简单无向边；返回宽度至多 2 的树分解，不存在时返回空。
    vector<set<int>> g(n);
    for (auto [u, v] : edge)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v); // 调试检查，可删。
        g[u].insert(v);
        g[v].insert(u);
    }
    struct Event
    {
        int d, v, a, b; // d 是删除时度数，v 是被删点，a、b 是它的邻居（不存在为 -1）。
    };
    vector<Event> ev;
    vector<bool> alive(n, true);
    queue<int> q;
    for (int u = 0; u < n; u++)
    {
        q.push(u);
    }
    while (!q.empty())
    {
        int v = q.front();
        q.pop();
        if (!alive[v] || g[v].size() >= 3)
        {
```

```
            continue;
        }
        int d = g[v].size(), a = -1, b = -1;
        if (d >= 1)
        {
            a = *g[v].begin();
        }
        if (d == 2)
        {
            b = *g[v].rbegin();
            g[a].insert(b);
            g[b].insert(a);
        }
        ev.push_back({d, v, a, b});
        alive[v] = false;
        for (int u : g[v])
        {
            g[u].erase(v);
            q.push(u);
        }
    }
    if (count(alive.begin(), alive.end(), true))
    {
        return nullopt;
    }

    reverse(ev.begin(), ev.end());
    TreeDecomp2 ans;
    vector<int> one(n, -1);
    map<pair<int, int>, int> two;
    for (auto [d, v, a, b] : ev)
    {
        int id = ans.bag.size(), p = -1;
        ans.bag.push_back({v});
        if (d >= 1)
        {
            ans.bag.back().push_back(a);
        }
        if (d == 2)
        {
            ans.bag.back().push_back(b);
            auto it = two.find(minmax(a, b));
            if (it == two.end())
            {
                return nullopt;
            }
            p = it->second;
        }
        else if (d == 1)
        {
            p = one[a];
            if (p < 0)
```

```
            {
                return nullopt;
            }
        }
        else if (id > 0)
        {
            p = id - 1;
        }
        if (p >= 0)
        {
            ans.edge.push_back({p, id});
        }
        for (int x : ans.bag.back())
        {
            one[x] = id;
        }
        for (int x : ans.bag.back())
        {
            for (int y : ans.bag.back())
            {
                if (x < y)
                {
                    two[{x, y}] = id;
                }
            }
        }
    }
    return ans;
}
```

`treeDecomp2(n, edge)` 要求简单无向图；成功时每个 bag 至多含 3 点，`.edge` 连接 bag 编号并构成一棵树，空图返回一个有效的空分解。失败返回空，说明图的树宽大于 2。`set` 使实现确定且不依赖哈希，时间 $O((n+m)\log n)$、空间 $O(n+m)$；若题目严格卡线性复杂度，可把动态邻接集合换成专门的小度邻接结构。返回结果可直接用于在树宽 2 图上做 bag DP。

5.10.015 Ramsey 定理

Ramsey 数 $R(s,t)$ 是「使得任意对完全图 K_n 的边二染色（红/蓝）都必然出现红色 K_s 或蓝色 K_t」的最小 n。它把「足够大的结构必含规则子结构」量化：$R(3,3)=6$ 是最著名的小值（6 个人中必有 3 人两两相识或两两不识），另有 $R(4,4)=18$，而 $R(5,5)$ 至今未知（只知在 $[43,46]$ 区间）。一般上界 $R(s,t)\le\binom{s+t-2}{s-1}$。竞赛里通常不需要算 Ramsey 数本身，而是用它的存在性保证：在规模达标的完全图上一定能找到单色团。下面给出对应 $R(3,3)=6$ 的构造性验证——在任意 $0/1$ 边染色的完全图里枚举三点找同色三角形，$n\ge 6$ 时必然找到。切忌把未确定的 Ramsey 数当精确常数用。

```
namespace ramsey
{
optional<array<int, 3>> monoTriangle(const vector<vector<int>> &col)
{
    // col 是完全图的对称 0/1 边染色；返回一个单色三角形，不存在时返回空。
```

```
    int n = col.size();
    for (int i = 0; i < n; i++)
    {
        assert((int)col[i].size() == n); // 调试检查，可删。
    }
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            assert((col[i][j] == 0 || col[i][j] == 1) &&
                   col[i][j] == col[j][i]); // 调试检查，可删。
            for (int k = j + 1; k < n; k++)
            {
                if (col[i][j] == col[i][k] && col[i][j] == col[j][k])
                {
                    return array<int, 3>{i, j, k};
                }
            }
        }
    }
    return nullopt;
}
}
```

`monoTriangle(col)` 传对称的 $0/1$ 边染色矩阵（`col[i][j]` 为该边颜色，对角线任意），返回一个同色三角形的三个顶点，不存在则返回 `nullopt`；当 $n \geq 6$ 时由 $R(3,3)=6$ 保证一定非空。枚举三点 $O(n^3)$、$O(1)$ 额外空间。要找更大的单色团（对应 $R(s,t)$）时枚举不再可行，得转成一般 5.10.006 最大团、极大团计数 分别在红图和蓝图上求团；构造证明 Ramsey 下界的极端染色（如 $R(4,4)$ 的 Paley 图）则是另一类专门构造。

5.10.016 外环加弦平面性判定（2-SAT）

这里的“平面性”有明确限制：原图是一条固定圆环，输入的每条额外边都是一条弦，只需要决定每条弦画在环内还是环外。两条端点交错的弦不能在同一侧，否则必相交；端点包含或互不交错的弦在同一侧可以嵌套或分离。于是对每一对交错弦加入两个子句，强制它们选相反侧，再调用 5.4.001 Tarjan 2-SAT。这不是任意图的平面图识别，不能把它推广成普适结论。

```
// 依赖 5.4.001 Tarjan 2-SAT
optional<vector<bool>> outerChordOK(int n, const vector<pair<int, int>> &chord)
{
    // n 是环上顶点数；chord[i] 是第 i 条弦的两个不同端点，编号按环顺序。
    assert(n >= 3);
    TarjanTwoSAT sat(chord.size());
    vector<pair<int, int>> a = chord;
    for (auto &[u, v] : a)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        if (u > v)
        {
```

```
            swap(u, v);
        }
    }
    for (int i = 0; i < (int)a.size(); i++)
    {
        for (int j = i + 1; j < (int)a.size(); j++)
        {
            auto [u, v] = a[i];
            auto [x, y] = a[j];
            bool cross = (u < x && x < v && v < y) || (x < u && u < y && y < v);
            if (cross)
            {
                sat.addOr(i, false, j, false);
                sat.addOr(i, true, j, true);
            }
        }
    }
    return sat.solve();
}
```

`outerChordOK(n,chord)` 返回每条弦的一组内外侧选择;无解表示无论怎样分侧都会有交叉。建模 $O(m^2)$，2-SAT 求解为 $O(m^2)$，空间 $O(m^2)$。例如环 $0,1,2,3$ 的弦 $(0,2)$ 与 $(1,3)$ 必须分到两侧：

```
signed main()
{
    auto side = outerChordOK(4, {{0, 2}, {1, 3}});
    assert(side && (*side)[0] != (*side)[1]);
}
```

5.10.017 Havel-Hakimi / Gale-Ryser 构造

Havel-Hakimi：非负整数序列 d 可简单图化，当且仅当去掉最大项 d_u 并把剩下最大的 d_u 项各减 1 后的序列仍可简单图化。交换论证：存在一种实现使度数最大的点恰好连向度数最大的那些点。用大根堆每次取剩余度数最大的点，按该规则连边即可得到一张简单图。

Gale-Ryser：二分图左侧度数 a、右侧度数 b 可实现为简单二分图，当且仅当 $\sum a = \sum b$，且把 a 降序排列后对每个 k 有

$$\sum_{i=1}^{k} a_i \le \sum_{j} \min(b_j, k).$$

`galeRyser` 把 b 升序后用双指针累计右端。条件成立时，左侧按度数从大到小、每个点连向右侧剩余度数最大的若干点，这个贪心总能完成构造（`bipDegree`）。

```
optional<vector<pair<int, int>>> havelHakimi(const vector<int> &d)
{
    // d[i] 是点 i 要求的度数；能构造简单无向图时返回其边表，否则返回空。
    int n = d.size();
    priority_queue<pair<int, int>> q; // q 存 (剩余度数, 点)，每次取剩余度数最大的点。
    vector<pair<int, int>> e;
```

```
    for (int i = 0; i < n; i++)
    {
        if (d[i] < 0 || d[i] >= n)
        {
            return nullopt;
        }
        if (d[i])
        {
            q.push({d[i], i});
        }
    }
    while (!q.empty())
    {
        auto [k, u] = q.top();
        q.pop();
        if (k > (int)q.size())
        {
            return nullopt;
        }
        // u 连向剩余度数最大的 k 个点，各自度数减一后放回。
        vector<pair<int, int>> tmp;
        while (k--)
        {
            auto [s, v] = q.top();
            q.pop();
            e.push_back({u, v});
            if (s > 1)
            {
                tmp.push_back({s - 1, v});
            }
        }
        for (auto x : tmp)
        {
            q.push(x);
        }
    }
    return e;
}

bool galeRyser(vector<int> a, vector<int> b)
{
    // a、b 是二分图左右两侧各点要求的度数；返回是否存在对应的简单二分图。
    __int128 sa = 0, sb = 0;
    for (int x : a)
    {
        if (x < 0 || x > (int)b.size())
        {
            return false;
        }
        sa += x;
    }
    for (int x : b)
```

```
    {
        if (x < 0 || x > (int)a.size())
        {
            return false;
        }
        sb += x;
    }
    if (sa != sb)
    {
        return false;
    }
    // a 降序后检查：前 k 大之和 <= sum min(b_j, k)。b 升序以便双指针累计 min。
    sort(a.rbegin(), a.rend());
    sort(b.begin(), b.end());
    __int128 lhs = 0, small = 0; // small 是 b 中小于 k 的那些度数之和。
    int j = 0;
    for (int k = 1; k <= (int)a.size(); k++)
    {
        lhs += a[k - 1];
        while (j < (int)b.size() && b[j] < k)
        {
            small += b[j++];
        }
        if (lhs > small + (__int128)k * (b.size() - j))
        {
            return false;
        }
    }
    return true;
}

optional<vector<pair<int, int>>> bipDegree(const vector<int> &a, const vector<int> &b)
{
    // a、b 同 galeRyser；可构造时返回边表（左点，右点），否则返回空。
    if (!galeRyser(a, b))
    {
        return nullopt;
    }
    vector<pair<int, int>> left, e;
    priority_queue<pair<int, int>> q; // q 存右侧（剩余度数，点）。
    for (int i = 0; i < (int)a.size(); i++)
    {
        left.push_back({a[i], i});
    }
    for (int j = 0; j < (int)b.size(); j++)
    {
        if (b[j])
        {
            q.push({b[j], j});
        }
    }
    // 左侧按度数从大到小，每个点连向右侧剩余度数最大的若干点。
```

```
    sort(left.rbegin(), left.rend());
    for (auto [k, u] : left)
    {
        vector<pair<int, int>> tmp;
        while (k--)
        {
            auto [s, v] = q.top();
            q.pop();
            e.push_back({u, v});
            if (s > 1)
            {
                tmp.push_back({s - 1, v});
            }
        }
        for (auto x : tmp)
        {
            q.push(x);
        }
    }
    return e;
}
```

`havelHakimi(d)` 可图化时返回一张简单无向图的边表 (u, v)，否则返回空；`galeRyser(a,b)` 只判定，`bipDegree(a,b)` 可实现时返回边表（左点, 右点）。度数为负或超过另一侧点数时直接不可实现。设边数为 m，`havelHakimi` 与 `bipDegree` 的时间 $O((n+m)\log n)$，`galeRyser` 为 $O(n \log n)$。

只判定一般图度序列时，Erdős-Gallai 条件见 5.10.005 度序列可图性 / Erdős-Gallai；竞赛图比分序列用 5.10.004 竞赛图与 Landau 定理。允许重边或要求连通等附加条件时贪心不再适用，通常改用网络流构造（5.8.005 有上下界可行流、最大流、最小流）。

```
signed main()
{
    auto e = havelHakimi({3, 3, 2, 2, 2});
    assert(e && e->size() == 6 && !havelHakimi({3, 3, 1, 1}));
    auto b = bipDegree({2, 1, 1}, {2, 2});
    assert(b && b->size() == 4 && !galeRyser({2, 2}, {2, 1, 0}));
    vector<int> x(3), y(2);
    for (auto [u, v] : *b)
    {
        x[u]++;
        y[v]++;
    }
    assert(x == vector<int>({2, 1, 1}) && y == vector<int>({2, 2}));
}
```

5.10.018 小图一般图同构

`graphIso` 判定两张小图是否同构并给出映射 p，使 $a_{u,v} = b_{p(u),p(v)}$ 对所有 u, v 成立。邻接矩阵可以有向、带自环，元素是边数或边权。回溯时先按（出度和，入度和）降序固定点的顺序，只在度数与自环值相同的点之间尝试，每确定一个点就检查它与所有已确定点之间的两个方向的边。

graphCanon 枚举全部 $n!$ 个顶点排列，取重排后按行展开的字典序最小矩阵作为规范编码；两图同构当且仅当编码相同，所以可以把很多小图按编码去重或分类。

```
optional<vector<int>> graphIso(const vector<vector<int>> &a, const vector<vector<int>> &b)
{
    // a、b 是方阵邻接矩阵（可有向、带自环与重边计数）；同构时返回映射 p，
    // 满足 a[u][v]==b[p[u]][p[v]]，否则返回空。
    int n = a.size();
    if ((int)b.size() != n)
    {
        return nullopt;
    }
    // da[i]、db[i] 是 (出度和，入度和)，只在度数相同的点之间尝试匹配。
    vector<pair<__int128, __int128>> da(n), db(n);
    for (int i = 0; i < n; i++)
    {
        assert((int)a[i].size() == n && (int)b[i].size() == n); // 调试检查，可删。
        for (int j = 0; j < n; j++)
        {
            da[i].first += a[i][j];
            da[i].second += a[j][i];
            db[i].first += b[i][j];
            db[i].second += b[j][i];
        }
    }
    vector<int> p(n, -1), order(n);
    vector<char> used(n);
    // 度数大的点先定，剪枝更早生效。
    iota(order.begin(), order.end(), 0);
    sort(order.begin(), order.end(), [&](int x, int y)
    {
        return da[x] > da[y];
    });
    auto dfs = [&](auto &&self, int k) -> bool
    {
        // 已为 order[0..k) 确定像，尝试为 order[k] 选像。
        if (k == n)
        {
            return true;
        }
        int u = order[k];
        for (int v = 0; v < n; v++)
        {
            if (used[v] || da[u] != db[v] || a[u][u] != b[v][v])
            {
                continue;
            }
            bool ok = true;
            for (int j = 0; j < k && ok; j++)
            {
                int x = order[j];
                ok = a[u][x] == b[v][p[x]] && a[x][u] == b[p[x]][v];
```

```
            }
            if (!ok)
            {
                continue;
            }
            p[u] = v;
            used[v] = true;
            if (self(self, k + 1))
            {
                return true;
            }
            used[v] = false;
        }
        return false;
    };
    if (dfs(dfs, 0))
    {
        return p;
    }
    return nullopt;
}

pair<vector<int>, vector<int>> graphCanon(const vector<vector<int>> &a)
{
    // a 是 n<=10 的方阵邻接矩阵；枚举全部顶点排列，返回字典序最小的重排矩阵（按行展开）
    // 以及取到它的顶点顺序。两图同构当且仅当规范编码相同。
    int n = a.size();
    assert(n <= 10); // 调试检查，可删。
    vector<int> p(n), best, order;
    iota(p.begin(), p.end(), 0);
    do
    {
        vector<int> code;
        for (int u : p)
        {
            for (int v : p)
            {
                code.push_back(a[u][v]);
            }
        }
        if (order.empty() || code < best)
        {
            best = move(code);
            order = p;
        }
    } while (next_permutation(p.begin(), p.end()));
    return {best, order};
}
```

`graphIso(a,b)` 要求两个矩阵都是方阵，阶数不同直接返回空。最坏时间 $O(n!\,n^2)$，度数剪枝对随机图通常有效，但正则图等对称结构仍可能接近最坏。`graphCanon(a)` 要求 $n \le 10$，时间 $O(n!\,n^2)$，返回编码与取到它的顶点顺序。

树同构见 6.1.008 树哈希与基环树哈希；大规模一般图同构需要 nauty/Traces 一类的划分细化算法，这里只适合小图和打表。下面把有向路 $0\to1\to2$ 与 $2\to0\to1$ 比较：

```cpp
signed main()
{
    vector<vector<int>> a{{0, 1, 0}, {0, 0, 1}, {0, 0, 0}}, b{{0, 1, 0}, {0, 0, 0}, {1, 0, 0}};
    auto p = graphIso(a, b);
    assert(p && *p == vector<int>({2, 0, 1}));
    assert(graphCanon(a).first == graphCanon(b).first);
    assert(!graphIso(a, {{0, 1, 1}, {0, 0, 0}, {0, 0, 0}}));
}
```

6. 树论与动态树

6.1 树的基础性质

6.1.001 树的直径、重心、中心

树上三个“中心性”对象定义互不相同，混用会直接出错：直径是最远点对及其路径，重心是删除后最大连通块最小的点，中心是到所有点最大距离最小的点。`treeDiameter` 用两遍搜索——先从任意点找到最远点 s，再从 s 出发找最远点 t，则 s-t 就是一条直径，这套两遍法只在非负边权下成立。`centroids` 按子树大小判断重心；`treeCenters` 反复剥去叶子，最后剩下中心。重心可能有一或两个，与直径奇偶没有直接对应关系；按边数计的无权树直径长度为偶数时有一个中心，为奇数时有两个中心。

```cpp
template <class Dist = long long>
struct Dia
{
    // dist 是直径长度，path 是一条直径上的完整点列。
    Dist dist;
    vector<int> path;
};

template <class Dist = long long>
Dia<Dist> treeDiameter(const vector<vector<pair<int, long long>>> &g)
{
    // g 是非负边权树；返回直径长度与一条直径路径。
    static_assert(numeric_limits<Dist>::is_integer && numeric_limits<Dist>::is_signed &&
                  sizeof(Dist) >= sizeof(long long));
    int n = g.size();
    if (!n)
    {
        return Dia<Dist>{0, {}};
    }
    auto far = [&](int s)
    {
        vector<int> p(n, -1);
        vector<Dist> d(n, -1);
        d[s] = 0;
        auto dfs = [&](auto &&self, int u) -> void
        {
            for (auto [v, w] : g[u])
            {
                if (v == p[u])
                {
                    continue;
                }
                assert(w >= 0); // 调试检查，可删。
                p[v] = u;
                d[v] = d[u] + (Dist)w;
                self(self, v);
            }
```

```
        };
        dfs(dfs, s);
        int u = max_element(d.begin(), d.end()) - d.begin();
        return tuple{u, move(p), move(d)};
    };
    auto first = far(0);
    int s = get<0>(first);
    auto second = far(s);
    auto [t, p, d] = move(second);
    vector<int> path;
    for (int u = t; u != -1; u = p[u])
    {
        path.push_back(u);
    }
    reverse(path.begin(), path.end());
    return Dia<Dist>{d[t], path};
}

vector<int> centroids(const vector<vector<int>> &g)
{
    // g 是无权树；返回一个或两个重心。
    int n = g.size();
    if (!n)
    {
        return {};
    }
    vector<int> sz(n), ans;
    auto dfs = [&](auto &&self, int u, int p) -> void
    {
        sz[u] = 1;
        int mx = 0;
        for (int v : g[u])
        {
            if (v == p)
            {
                continue;
            }
            self(self, v, u);
            sz[u] += sz[v];
            chmax(mx, sz[v]);
        }
        chmax(mx, n - sz[u]);
        if (2 * mx <= n)
        {
            ans.push_back(u);
        }
    };
    dfs(dfs, 0, -1);
    return ans;
}

vector<int> treeCenters(const vector<vector<int>> &g)
```

```
{
    // g 是无权树；返回一个或两个中心。
    int n = g.size();
    if (n <= 2)
    {
        vector<int> a(n);
        iota(a.begin(), a.end(), 0);
        return a;
    }
    vector<int> d(n);
    queue<int> q;
    for (int u = 0; u < n; u++)
    {
        d[u] = g[u].size();
        if (d[u] == 1)
        {
            q.push(u);
        }
    }
    int left = n;
    while (left > 2)
    {
        int z = q.size();
        left -= z;
        while (z--)
        {
            int u = q.front();
            q.pop();
            for (int v : g[u])
            {
                if (--d[v] == 1)
                {
                    q.push(v);
                }
            }
        }
    }
    vector<int> ans;
    while (!q.empty())
    {
        ans.push_back(q.front());
        q.pop();
    }
    return ans;
}
```

`treeDiameter` 的 `Dist` 默认是 `long long`，且不得窄于输入边权类型；调用者须保证直径长度能由 `Dist` 表示，若可能超过 64 位可用 `treeDiameter<__int128>`。`centroids`、`treeCenters` 分别返回无权树的重心和中心。三者均为 $O(n)$ 时间与空间；仅对无权树，直径中间的一个或两个顶点才是中心。

6.1.002 最近公共祖先：倍增

倍增法预处理 `up[k][u]` 为 u 的 2^k 级祖先，满足 $up[k][u] = up[k-1][up[k-1][u]]$。查询时先用进出时间戳判祖先关系（$in[u] \le in[v] \wedge out[v] \le out[u]$），若互不为祖先，就把 u 从高位往低位跳到“仍不是 v 祖先”的最高点，其父亲即 LCA。它面向静态有根树的在线询问；一旦连边断边就必须整表重建。根的 2^k 级祖先取自身（自环），使 `jump` 越界安全；`jump` 越过根返回 -1，`pathKth` 把路径第 k 点按 LCA 拆成两段各做一次 `jump`。

```
struct LCA
{
    // n 是点数，lg 是倍增层数，tim 是前序计时器。
    int n, lg, tim = 0;
    // dep 是深度，in/out 是子树前序区间的两端。
    vector<int> dep, in, out;
    // up[k][u] 是 u 的 2^k 级祖先。
    vector<vector<int>> up;

    LCA(const vector<vector<int>> &g, int root = 0)
       : n(g.size()), lg(bit_width((unsigned long long)max<int>(1, n))), dep(n), in(n), out(n),
         up(lg, vector<int>(n))
    {
        // g 是树，root 是根；空树只建立空表。
        assert(n == 0 || (0 <= root && root < n)); // 调试检查，可删
        if (!n)
        {
            return;
        }
        auto dfs = [&](auto &&self, int u, int p) -> void
        {
            in[u] = tim++;
            up[0][u] = p;
            for (int k = 1; k < lg; k++)
            {
                up[k][u] = up[k - 1][up[k - 1][u]];
            }
            for (int v : g[u])
            {
                if (v == p)
                {
                    continue;
                }
                dep[v] = dep[u] + 1;
                self(self, v, u);
            }
            out[u] = tim;
        };
        dfs(dfs, root, root);
    }

    bool anc(int u, int v) const
    {
```

```
        // 返回 u 是否为 v 的祖先，节点编号须合法。
        return in[u] <= in[v] && out[v] <= out[u];
    }

    int jump(int u, int d) const
    {
        // 返回 u 的第 d 级祖先；越过根返回 -1。
        assert(d >= 0); // 调试检查，可删
        if (d > dep[u])
        {
            return -1;
        }
        for (int k = 0; d; k++, d >>= 1)
        {
            if (d & 1)
            {
                u = up[k][u];
            }
        }
        return u;
    }

    int lca(int u, int v) const
    {
        // 返回 u、v 的最近公共祖先。
        if (anc(u, v))
        {
            return u;
        }
        if (anc(v, u))
        {
            return v;
        }
        for (int k = lg - 1; k >= 0; k--)
        {
            if (!anc(up[k][u], v))
            {
                u = up[k][u];
            }
        }
        return up[0][u];
    }

    int dist(int u, int v) const
    {
        // 返回 u、v 间的边数。
        int p = lca(u, v);
        return dep[u] + dep[v] - 2 * dep[p];
    }

    int pathKth(int u, int v, int k) const
    {
```

```
        // 返回 u 到 v 路径上从 0 开始的第 k 个点；越界返回 -1。
        int p = lca(u, v), a = dep[u] - dep[p], b = dep[v] - dep[p];
        if (k < 0 || k > a + b)
        {
            return -1;
        }
        return k <= a ? jump(u, k) : jump(v, a + b - k);
    }
};
```

构造 `LCA(g, root)`；`lca`、`dist`、`jump`、`pathKth` 均 $O(\log n)$，其中 `dist` 返回边数（带权距离需另维护根距），`jump`/`pathKth` 越界返回 -1。预处理 $O(n \log n)$ 时间与空间。这里的 `in`/`out` 时间戳判祖先技巧被 6.3.003 虚树 直接复用；只读且询问量极大时改用 6.1.004 最近公共祖先：ST 表 / RMQ 的 $O(1)$ 查询，若能离线则用 6.1.005 最近公共祖先：Tarjan 离线 省去每次 log。

倍增、RMQ 与离线 Tarjan 三版共用的固定检查放在 6.1.005 最近公共祖先：Tarjan 离线 后，期望值均可直接手算。

6.1.003 最近公共祖先：树链剖分

已经为换根或路径查询建好重链剖分时，求 LCA 不必再开倍增表：沿重链把链顶更深的一点跳到其链顶的父亲，两点进入同一条链后深度较小者即 LCA，每次询问 $O(\log n)$、零额外预处理。本条直接复用 6.3.002 换根树剖 的 `RerootHLD`，调用其 `lca(u, v)` 即可；若只要 LCA 而不需换根，6.3.001 轻重链剖分 / HLD 的 `HLD::lca` 更轻。相比 6.1.002 最近公共祖先：倍增，树剖版常数更小、省去 $O(n \log n)$ 空间，但同样只适用于静态树。

6.1.004 最近公共祖先：ST 表 / RMQ

把 LCA 归约成欧拉序上的区间最小值：DFS 游走记录 (深度，节点) 序列与每点首次出现位置 `first`，两点 LCA 就是它们首现位置之间深度最小的那一项。区间最小用幂等的 ST 表（比较器 `MinDepth` 取 `pair` 的 `min`，深度在前保证按深度比较），因此查询 $O(1)$。`ST` 只适用于 `min`/`max`/`gcd`/`and`/`or` 这类幂等运算，查询区间约定为 $[l, r)$。它适合询问量巨大、可接受 $O(n \log n)$ 预处理空间的只读场景。

```
template <class T, class F> struct ST
{
    // n 是序列长度，f 是幂等合并函数，a[k][i] 维护长 2^k 的区间。
    int n;
    F f;
    vector<vector<T>> a;

    ST(const vector<T> &v, F f = {}) : n(v.size()), f(f)
    {
        // v 是原序列，f 合并两个区间结果。
        int m = n ? bit_width((unsigned long long)n) : 0;
        a.assign(m, vector<T>(n));
        if (!n)
        {
```

```
            return;
        }
        a[0] = v;
        for (int k = 1; k < m; k++)
        {
            for (int i = 0; i + (1LL << k) <= n; i++)
            {
                a[k][i] = f(a[k - 1][i], a[k - 1][i + (1LL << (k - 1))]);
            }
        }
    }

    // 仅适用于幂等运算（min/max/gcd/and/or），查询 [l,r)。
    T query(int l, int r) const
    {
        // 返回半开区间 [l,r) 的合并结果。
        assert(0 <= l && l < r && r <= n); // 调试检查，可删
        int k = bit_width((unsigned long long)(r - l)) - 1;
        return f(a[k][l], a[k][r - (1LL << k)]);
    }
};

struct MinDepth
{
    pair<int, int> operator()(pair<int, int> a, pair<int, int> b) const
    {
        // 返回深度较小的一项；深度相同则取节点编号较小者。
        return min(a, b);
    }
};

struct LCARMQ
{
    // first[u] 是 u 在欧拉游走序列中的首次出现位置。
    vector<int> first;
    // st 维护欧拉序上的最小深度项。
    ST<pair<int, int>, MinDepth> st;

    static pair<vector<int>, vector<pair<int, int>>> tour(const vector<vector<int>> &g, int
    ↪ root)
    {
        // 返回 {首次出现位置，(深度，节点) 欧拉游走序列}。
        assert(!g.empty() && 0 <= root && root < (int)g.size()); // 调试检查，可删
        vector<int> first(g.size(), -1);
        vector<pair<int, int>> a;
        auto dfs = [&](auto &&self, int u, int p, int d) -> void
        {
            first[u] = a.size();
            a.push_back({d, u});
            for (int v : g[u])
            {
                if (v == p)
```

```
                {
                    continue;
                }
                self(self, v, u, d + 1);
                a.push_back({d, u});
            }
        };
        dfs(dfs, root, -1, 0);
        return {first, a};
    }

    LCARMQ(const vector<vector<int>> &g, int root = 0) : LCARMQ(tour(g, root))
    {
        // g 是非空树，root 是根。
    }

    int lca(int x, int y) const
    {
        // 返回 x、y 的最近公共祖先。
        int l = first[x], r = first[y];
        if (l > r)
        {
            swap(l, r);
        }
        return st.query(l, r + 1).second;
    }

  private:
    LCARMQ(pair<vector<int>, vector<pair<int, int>>> p) : first(move(p.first)), st(p.second,
    ↪ MinDepth{})
    {
        // p 保存首次出现位置与欧拉游走序列。
    }
};
```

构造 `LCARMQ(g, root)` 后 `lca(x, y)` 为 $O(1)$；欧拉序长 $2n-1$，ST 构造 $O(n\log n)$ 时间与空间。相比 6.1.002 最近公共祖先：倍增 查询更快，但空间常数更大，且不直接给 `dist`/`jump`；需要 K 级祖先或路径第 k 点仍回到 6.1.002 或 6.1.006 树上距离、K 级祖先。

单独使用本条 `ST` 时也要显式传入幂等合并对象：

```
signed main()
{
    vector<pair<int, int>> a{{2, 0}, {1, 3}, {4, 2}};
    ST<pair<int, int>, MinDepth> st(a, MinDepth{});
    assert((st.query(0, 3) == pair<int, int>{1, 3}));
}
```

固定检查与倍增版合并放在 6.1.005 最近公共祖先：Tarjan 离线 后。

6.1.005 最近公共祖先：Tarjan 离线

若全部询问预先给定，Tarjan 离线用一遍 DFS 加并查集一次性回答：DFS 回溯完一个儿子就把它并入父亲、并把该集合的祖先指向父亲；处理到某点时，凡另一端已访问完毕的询问，其所在集合的 `anc[find(v)]` 就是最近公共祖先。它必须离线——遍历结束后不能再追加询问。`col` 用 0/1/2 三态区分未访问、在栈中、已完成。

```cpp
// 依赖 6.1.002 最近公共祖先：倍增
// 依赖 6.1.004 最近公共祖先：ST 表 / RMQ
vector<int> offlineLCA(const vector<vector<int>> &g, int rt, const vector<pair<int, int>> &q)
{
    // g 是非空树，rt 是根，q 是询问端点对；按原顺序返回 LCA。
    int n = g.size();
    assert(n > 0 && 0 <= rt && rt < n); // 调试检查，可删
    vector<vector<pair<int, int>>> ask(n);
    for (int i = 0; i < (int)q.size(); i++)
    {
        auto [u, v] = q[i];
        ask[u].push_back({v, i});
        ask[v].push_back({u, i});
    }
    vector<int> p(n, -1), anc(n), ans(q.size(), -1), col(n);
    auto find = [&](auto &&self, int x) -> int
    {
        return p[x] < 0 ? x : p[x] = self(self, p[x]);
    };
    auto merge = [&](int x, int y)
    {
        x = find(find, x);
        y = find(find, y);
        if (x == y)
        {
            return x;
        }
        if (p[x] > p[y])
        {
            swap(x, y);
        }
        p[x] += p[y];
        p[y] = x;
        return x;
    };
    auto dfs = [&](auto &&self, int u, int f) -> void
    {
        anc[u] = u;
        col[u] = 1;
        for (int v : g[u])
        {
            if (v == f)
            {
                continue;
```

```
            }
            self(self, v, u);
            anc[merge(u, v)] = u;
        }
        col[u] = 2;
        for (auto [v, id] : ask[u])
        {
            if (col[v] == 2)
            {
                ans[id] = anc[find(find, v)];
            }
        }
    };
    dfs(dfs, rt, -1);
    return ans;
}
```

`offlineLCA(g, rt, q)` 按输入顺序返回每对询问的 LCA，q 是 `{u, v}` 列表。总计 $O((n+q)\alpha(n))$ 时间、$O(n+q)$ 空间，常数极小。需要在线追加询问时改用 6.1.002 最近公共祖先：倍增 或 6.1.004 最近公共祖先：ST 表 / RMQ。

下面让三种 LCA 分别核对同一组固定答案；各实现之间不互相比较：

```
signed main()
{
    vector<vector<int>> g(5);
    for (auto [u, v] : vector<pair<int, int>>{{0, 1}, {0, 2}, {1, 3}, {1, 4}})
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    LCA a(g);
    LCARMQ b(g);
    vector<pair<int, int>> q{{3, 4}, {2, 4}};
    vector<int> want{1, 0};
    assert(offlineLCA(g, 0, q) == want);
    for (int i = 0; i < (int)q.size(); i++)
    {
        auto [u, v] = q[i];
        assert(a.lca(u, v) == want[i]);
        assert(b.lca(u, v) == want[i]);
    }
    assert(a.jump(4, 2) == 0 && a.pathKth(3, 2, 2) == 0);
}
```

6.1.006 树上距离、K 级祖先

两点距离靠根距与 LCA：$dis(u,v) = dis(u) + dis(v) - 2\,dis(\mathrm{lca})$。K 级祖先则把 k 按二进制拆开，在倍增表 `parent[level][u]` 上逐位上跳，越过根用 -1 明确失败。本条 `KthAncestor` 只维护父倍增表与深度，并用 BFS 序（`order`）自顶向下建表，避免深树递归爆栈。

```cpp
class KthAncestor
{
    // parent[k][u] 是 u 的 2^k 级祖先，越过根记 -1。
    vector<vector<int>> parent;
    // depth[u] 是 u 的深度。
    vector<int> depth;

  public:
    KthAncestor(const vector<vector<int>> &tree, int root = 0) : depth(tree.size())
    {
        // tree 是非空树，root 是根。
        assert(!tree.empty() && 0 <= root && root < (int)tree.size()); // 调试检查，可删
        int levels = bit_width((unsigned long long)tree.size());
        parent.assign(levels, vector<int>(tree.size(), -1));
        vector<int> order{root};
        for (int i = 0; i < (int)order.size(); ++i)
        {
            int u = order[i];
            for (int v : tree[u])
            {
                if (v == parent[0][u])
                {
                    continue;
                }
                parent[0][v] = u;
                depth[v] = depth[u] + 1;
                order.push_back(v);
            }
        }
        for (int level = 1; level < levels; ++level)
        {
            for (int u = 0; u < (int)tree.size(); ++u)
            {
                int p = parent[level - 1][u];
                parent[level][u] = p == -1 ? -1 : parent[level - 1][p];
            }
        }
    }

    int query(int u, int distance) const
    {
        // 返回 u 的第 distance 级祖先；越过根返回 -1。
        assert(distance >= 0); // 调试检查，可删
        if (distance > depth[u])
        {
            return -1;
        }
        for (int bit = 0; distance; ++bit, distance >>= 1)
        {
            if (distance & 1)
            {
                u = parent[bit][u];
```

```
            }
        }
        return u;
    }
};
```

构造 `KthAncestor(tree, root)` 后 `query(u, k)` 返回 u 的第 k 级祖先，$k > dep_u$ 时返回 -1，单次 $O(\log k)$；构造 $O(n \log n)$ 时间与空间。带权距离需另配根距数组。6.2.005 长链剖分 的 $O(1)$ K 级祖先建立在同一倍增预处理之上；只求 LCA 则无需建此表，见 6.1.002 最近公共祖先：倍增。

6.1.007 树上路径交

树上简单路径是“凸”的：两条路径的交若非空，仍是一条路径。把两路径四个端点与它们两两的 LCA 作为候选点，凡同时落在两条路径上的点（用 $dis(a,x)+dis(x,b)=dis(a,b)$ 判定 x 在 a-b 上），取其中距离最远的一对即交段端点；候选全不满足则两路径不相交。本条只依赖 6.1.002 最近公共祖先：倍增 的 `lca` 与 `dist`。

```
optional<pair<int, int>> pathIsect(int a, int b, int c, int d, const LCA &l)
{
    // 返回路径 a-b 与 c-d 的公共段端点；不相交时返回 nullopt。
    vector<int> p = {a, b, c, d};
    for (int x : vector<int>{a, b})
    {
        for (int y : vector<int>{c, d})
        {
            p.push_back(l.lca(x, y));
        }
    }
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    vector<int> q;
    for (int x : p)
    {
        bool u = l.dist(a, x) + l.dist(x, b) == l.dist(a, b);
        bool v = l.dist(c, x) + l.dist(x, d) == l.dist(c, d);
        if (u && v)
        {
            q.push_back(x);
        }
    }
    if (q.empty())
    {
        return nullopt;
    }
    pair<int, int> ans{q[0], q[0]};
    for (int x : q)
    {
        for (int y : q)
        {
```

```
            if (l.dist(x, y) > l.dist(ans.first, ans.second))
            {
                ans = {x, y};
            }
        }
    }
    return ans;
}
```

`pathIsect(a, b, c, d, l)` 传入两条路径端点与已建好的 `LCA`，返回 `optional<pair>` 表示交段端点，无交返回 `nullopt`。候选点常数个、每次 LCA $O(\log n)$，总 $O(\log n)$ 时间、$O(1)$ 额外空间。用 dis 等式判点在路径上，省去了显式构造路径本身。

6.1.008 树哈希与基环树哈希

给无根树或基环树算一个与编号无关的结构指纹，用于同构分组。有根树哈希自底向上：每点把儿子哈希排序后经 `map` 映射成一个整数（AHU 编码），保证同构子树得同一值。无根树先取重心（一或两个）为根分别哈希，再取规范集合；基环树先剥叶找出唯一环，对环上每个点以“临时禁掉环其余点”的方式哈希其挂树，再把环序列用最小表示 `minRot` 消旋转、`reverse` 消翻转取规范。这里用 `map` 编码而非随机哈希，是确定性无碰撞的（代价是每次 log）。

```
struct TreeIso
{
    // mp 在当前对象内把儿子编码多重集映射为唯一整数。
    map<vector<int>, int> mp;

    int get(vector<int> a)
    {
        // a 是儿子子树编码；返回当前共享编码表中的 AHU 编号。
        sort(a.begin(), a.end());
        return mp.emplace(move(a), mp.size() + 1).first->second;
    }

    vector<int> ids(const vector<vector<int>> &g, int rt, const vector<bool> *ban = nullptr)
    {
        // 返回从 rt 可达的各点有根子树编码；ban 中的点不参与遍历。
        int n = g.size();
        vector<int> p(n, -2), q{rt};
        p[rt] = -1;
        for (int k = 0; k < (int)q.size(); k++)
        {
            int u = q[k];
            for (int v : g[u])
            {
                if (v == p[u] || p[v] != -2 || (ban && (*ban)[v]))
                {
                    continue;
                }
                p[v] = u;
```

```
                q.push_back(v);
            }
        }
        vector<int> h(n);
        for (int k = (int)q.size() - 1; k >= 0; k--)
        {
            int u = q[k];
            vector<int> a;
            for (int v : g[u])
            {
                if (p[v] == u)
                {
                    a.push_back(h[v]);
                }
            }
            h[u] = get(move(a));
        }
        return h;
    }

    int rooted(const vector<vector<int>> &g, int rt = 0)
    {
        // 返回以 rt 为根的整棵树编码，g 必须非空。
        assert(!g.empty() && 0 <= rt && rt < (int)g.size()); // 调试检查，可删
        return ids(g, rt)[rt];
    }

    vector<int> centroids(const vector<vector<int>> &g)
    {
        // 返回非空树的一个或两个重心。
        assert(!g.empty()); // 调试检查，可删
        int n = g.size();
        vector<int> p(n, -1), sz(n, 1), q{0};
        for (int k = 0; k < (int)q.size(); k++)
        {
            int u = q[k];
            for (int v : g[u])
            {
                if (v == p[u])
                {
                    continue;
                }
                p[v] = u;
                q.push_back(v);
            }
        }
        vector<int> ans;
        for (int k = n - 1; k >= 0; k--)
        {
            int u = q[k];
            for (int v : g[u])
            {
```

```
                if (p[v] != u)
                {
                    continue;
                }
                sz[u] += sz[v];
            }
            int mx = n - sz[u];
            for (int v : g[u])
            {
                if (p[v] != u)
                {
                    continue;
                }
                mx = max(mx, sz[v]);
            }
            if (mx * 2 <= n)
            {
                ans.push_back(u);
            }
        }
        return ans;
    }

    vector<int> unrooted(const vector<vector<int>> &g)
    {
        // 返回无根树在当前共享编码表中的规范编码。
        vector<int> ans;
        for (int r : centroids(g))
        {
            ans.push_back(rooted(g, r));
        }
        sort(ans.begin(), ans.end());
        return ans;
    }

    vector<int> minRot(const vector<int> &a)
    {
        // 返回循环序列 a 的字典序最小循环表示。
        int n = a.size(), i = 0, j = 1, k = 0;
        while (i < n && j < n && k < n)
        {
            int x = a[(i + k) % n], y = a[(j + k) % n];
            if (x == y)
            {
                k++;
                continue;
            }
            if (x > y)
            {
                i += k + 1;
            }
            else
```

```
            {
                j += k + 1;
            }
            if (i == j)
            {
                j++;
            }
            k = 0;
        }
        int s = min(i, j);
        vector<int> b;
        for (int x = 0; x < n; x++)
        {
            b.push_back(a[(s + x) % n]);
        }
        return b;
    }

    vector<int> unicycle(const vector<vector<int>> &g)
    {
        // 返回非空简单连通基环图在当前共享编码表中的规范编码。
        assert(!g.empty()); // 调试检查，可删
        int n = g.size();
        vector<int> d(n);
        queue<int> q;
        for (int i = 0; i < n; i++)
        {
            d[i] = g[i].size();
            if (d[i] == 1)
            {
                q.push(i);
            }
        }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            d[u] = 0;
            for (int v : g[u])
            {
                if (d[v] && --d[v] == 1)
                {
                    q.push(v);
                }
            }
        }
        vector<int> cyc;
        for (int i = 0; i < n; i++)
        {
            if (d[i])
            {
                cyc.push_back(i);
```

```
            }
        }
        if (cyc.empty())
        {
            return {};
        }
        vector<bool> ban(n);
        for (int x : cyc)
        {
            ban[x] = true;
        }
        vector<int> a;
        int s = cyc[0], p = -1, u = s;
        do
        {
            ban[u] = false;
            a.push_back(ids(g, u, &ban)[u]);
            ban[u] = true;
            int v = -1;
            for (int x : g[u])
            {
                if (d[x] && x != p)
                {
                    v = x;
                }
            }
            p = u;
            u = v;
        } while (u != s);
        auto x = minRot(a);
        reverse(a.begin(), a.end());
        return min(x, minRot(a));
    }
};
```

`TreeIso` 用精确 AHU 编码，不存在随机碰撞；`unrooted(g)`、`unicycle(g)` 返回规范整数序列。整数编号由子树形态在 `mp` 中首次出现的顺序决定，因此只有同一个 `TreeIso` 对象编码出的结果才能直接比较；两个各自从空 `mp` 开始的对象也不能比较编号。单棵树 $O(n \log n)$ 时间、$O(n)$ 空间，基环图须简单、连通且恰有一个环。

下面固定检查路径的重标号以及路径与星。所有待比较图都交给同一个 `TreeIso` 对象编码：

```
signed main()
{
    auto tree = [](int n, initializer_list<pair<int, int>> e)
    {
        vector<vector<int>> g(n);
        for (auto [u, v] : e)
        {
            g[u].push_back(v);
            g[v].push_back(u);
```

```
        }
        return g;
    };
    auto path = tree(5, {{0, 1}, {1, 2}, {2, 3}, {3, 4}});
    auto relabeled = tree(5, {{3, 1}, {1, 4}, {4, 0}, {0, 2}});
    auto star = tree(5, {{0, 1}, {0, 2}, {0, 3}, {0, 4}});
    TreeIso iso;
    assert(iso.unrooted(path) == iso.unrooted(relabeled));
    assert(iso.unrooted(path) != iso.unrooted(star));
}
```

6.1.009 Prüfer 序列与树的计数

Prüfer 序列在 n 个带标号点的树与长度 $n-2$ 的序列间建立双射：编码时反复删去编号最小的叶并记录其唯一邻点，解码时用度数数组与当前最小可用叶还原。由此点 i 在序列中出现次数恰为 $\deg(i)-1$，带标号树共 n^{n-2} 棵（Cayley 公式），给定度数 d_i 的树数是多项式系数 $\binom{n-2}{d_1-1,\dots,d_n-1}$。`pruferCode` 用小根堆取最小叶，`pruferTree` 逆流程重建边集。

```
vector<int> pruferCode(const vector<vector<int>> &g)
{
    // 返回 n>=2 的带标号树对应的 Prüfer 序列。
    int n = g.size();
    assert(n >= 2); // 调试检查，可删
    if (n == 2)
    {
        return {};
    }
    vector<int> d(n);
    priority_queue<int, vector<int>, greater<>> q;
    for (int i = 0; i < n; i++)
    {
        d[i] = g[i].size();
        if (d[i] == 1)
        {
            q.push(i);
        }
    }
    vector<int> a;
    for (int k = 0; k < n - 2; k++)
    {
        int u = q.top();
        q.pop();
        int v = -1;
        for (int x : g[u])
        {
            if (d[x])
            {
                v = x;
                break;
            }
```

```
        }
        a.push_back(v);
        d[u]--;
        if (--d[v] == 1)
        {
            q.push(v);
        }
    }
    return a;
}

vector<pair<int, int>> pruferTree(const vector<int> &a)
{
    // 返回 Prüfer 序列还原出的 n-1 条无向边；空序列表示两点一边。
    int n = a.size() + 2;
    vector<int> d(n, 1);
    for (int x : a)
    {
        assert(0 <= x && x < n); // 调试检查，可删
        d[x]++;
    }
    priority_queue<int, vector<int>, greater<>> q;
    for (int i = 0; i < n; i++)
    {
        if (d[i] == 1)
        {
            q.push(i);
        }
    }
    vector<pair<int, int>> e;
    for (int v : a)
    {
        int u = q.top();
        q.pop();
        e.push_back({u, v});
        if (--d[v] == 1)
        {
            q.push(v);
        }
    }
    int u = q.top();
    q.pop();
    e.push_back({u, q.top()});
    return e;
}
```

`pruferCode(g)` 的定义域是 $n \geq 2$ 的无向带标号树，`pruferTree(a)` 输入长 $n-2$ 的序列并返回 $n-1$ 条边；在此定义域内两者互逆。空序列固定表示两点一边，单点树不属于这组双射。编解码各 $O(n \log n)$ 时间、$O(n)$ 空间。它是随机生成树、按度数计数的基础工具，与 8.1.001 阶乘、逆阶乘与组合数 的多项式系数配合即可直接数给定度数序列的树数。

下面用一条含重复标签的序列检查编解码往返，并保留二点树特例：

```cpp
signed main()
{
    vector<int> a{3, 0, 3, 1};
    int n = a.size() + 2;
    auto e = pruferTree(a);
    vector<vector<int>> g(n);
    for (auto [u, v] : e)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    assert(pruferCode(g) == a);
    assert((pruferTree({}) == vector<pair<int, int>>{{0, 1}}));
}
```

6.1.010 树的邻接表转前序表

把邻接表重排成“父先于所有子孙”的前序表，同时给出父亲与每棵子树对应的连续区间，这是所有“子树问题转区间问题”的地基。递归 DFS 首次访问某点时分配前序编号，回溯时累加子树大小，于是每棵子树恰对应区间 $[pos[x], pos[x] + siz[x])$。

```cpp
struct TreeOrder
{
    vector<int> par; // par[x] 是 x 的父亲，根的父亲为 -1。
    vector<int> ord; // ord 按 DFS 前序保存全部点。
    vector<int> pos; // pos[x] 是 x 在 ord 中的下标。
    vector<int> siz; // siz[x] 是以 x 为根的子树大小，因此子树对应 ord 的连续区间。
};

TreeOrder preorderTab(const vector<vector<int>> &g, int root = 0)
{
    // g 是无向树，root 是根；空树返回四张空表。
    auto n = (int)g.size();
    if (!n)
    {
        return {};
    }
    assert(0 <= root && root < n); // 调试检查，可删
    TreeOrder z{vector<int>(n, -1), {}, vector<int>(n), vector<int>(n, 1)};
    z.ord.reserve(n);
    auto dfs = [&](auto &&self, int x, int p) -> void
    {
        z.par[x] = p;
        z.pos[x] = z.ord.size();
        z.ord.push_back(x);
        for (auto y : g[x])
        {
            if (y != p)
            {
                self(self, y, x);
```

```
                z.siz[x] += z.siz[y];
            }
        }
    };
    dfs(dfs, root, -1);
    return z;
}
```

`preorderTab(g, root)` 返回 `TreeOrder{par, ord, pos, siz}`：`par` 根为 -1，`pos[x]` 是 x 的前序下标，子树对应 `ord` 上区间 $[pos[x], pos[x]+siz[x])$。时间与额外空间均为 $O(n)$，递归深度最坏为 $O(n)$。这个连续子树区间正是 6.3.004 树上路径/子树数据结构 把子树查询交给一维数据结构的前提。

6.1.011 公共区间分解树

排列的公共区间是位置连续且值集合也连续的区间。所有强公共区间形成唯一的包含树：子区间的值域相邻时为线性节点，否则为素节点。算法从左向右插入排列元素，用两个单调栈维护区间最小值、最大值变化，并让区间加、区间最大值线段树判断栈顶段是否恰构成连续值域；一旦成立就合并为素节点，值域端点相邻的节点则直接合并为线性节点。

```
struct CommonNode
{
    int l, r, parent; // l、r 是闭区间端点，parent 是父节点编号，根为 -1。
    bool linear; // linear 为真表示线性节点，为假表示素节点。
};

vector<CommonNode> commonIntTr(const vector<int> &p)
{
    // p 是 0..n-1 的排列；返回先序编号的公共区间分解树，父节点编号严格更小。
    struct MaxAddSeg
    {
        int n; // n 是叶子数。
        vector<int> mx, tag; // mx 是区间最大值，tag 是整段加法懒标记。
        MaxAddSeg(int n) : n(n), mx(n * 4), tag(n * 4)
        {
            // n 是叶子数；初始每个叶子为 0。
        }
        void add(int ql, int qr, int x, int u, int l, int r)
        {
            // [ql,qr) 加 x，u 覆盖 [l,r)。
            if (qr <= l || r <= ql)
            {
                return;
            }
            if (ql <= l && r <= qr)
            {
                mx[u] += x;
                tag[u] += x;
                return;
            }
```

```
            int m = (l + r) / 2;
            add(ql, qr, x, u * 2, l, m);
            add(ql, qr, x, u * 2 + 1, m, r);
            mx[u] = tag[u] + max(mx[u * 2], mx[u * 2 + 1]);
        }
        void set(int p, int x, int u, int l, int r)
        {
            // 把位置 p 赋为 x，u 覆盖 [l,r)。
            if (r - l == 1)
            {
                mx[u] = x;
                tag[u] = 0;
                return;
            }
            int m = (l + r) / 2;
            if (p < m)
            {
                set(p, x - tag[u], u * 2, l, m);
            }
            else
            {
                set(p, x - tag[u], u * 2 + 1, m, r);
            }
            mx[u] = tag[u] + max(mx[u * 2], mx[u * 2 + 1]);
        }
        int prod(int ql, int qr, int u, int l, int r) const
        {
            // 返回 [ql,qr) 的最大值，u 覆盖 [l,r)。
            if (qr <= l || r <= ql)
            {
                return numeric_limits<int>::min() / 4;
            }
            if (ql <= l && r <= qr)
            {
                return mx[u];
            }
            int m = (l + r) / 2;
            return tag[u] + max(prod(ql, qr, u * 2, l, m), prod(ql, qr, u * 2 + 1, m, r));
        }
        void add(int l, int r, int x) { add(l, r, x, 1, 0, n); }
        void set(int p, int x) { set(p, x, 1, 0, n); }
        int prod(int l, int r) const { return prod(l, r, 1, 0, n); }
    };
    struct Node
    {
        int l, r, mn, mx; // l、r 是半开位置区间，mn、mx 是对应值半开区间。
        bool linear; // 是否为线性节点。
        vector<int> ch; // 从左到右的子节点编号。
    };
    int n = p.size();
    if (n == 0)
    {
```

```
        return {};
    }
    MaxAddSeg seg(n);
    vector<int> mn, mx, stk;
    vector<Node> tr;
    auto makeLeaf = [&](int i)
    {
        tr.push_back({i, i + 1, p[i], p[i] + 1, true, {}});
        return (int)tr.size() - 1;
    };
    auto merge = [&](vector<int> ch, bool linear)
    {
        int l = tr[ch[0]].l, r = tr[ch.back()].r;
        int lo = tr[ch[0]].mn, hi = tr[ch[0]].mx;
        for (int x : ch)
        {
            lo = min(lo, tr[x].mn);
            hi = max(hi, tr[x].mx);
        }
        tr.push_back({l, r, lo, hi, linear, move(ch)});
        return (int)tr.size() - 1;
    };
    for (int i = 0; i < n; i++)
    {
        int r = i;
        while (!mn.empty() && p[mn.back()] > p[i])
        {
            mn.pop_back();
            int l = mn.empty() ? 0 : mn.back() + 1;
            seg.add(l, r, p[i] - p[r - 1]);
            r = l;
        }
        mn.push_back(i);
        r = i;
        while (!mx.empty() && p[mx.back()] < p[i])
        {
            mx.pop_back();
            int l = mx.empty() ? 0 : mx.back() + 1;
            seg.add(l, r, -p[i] + p[r - 1]);
            r = l;
        }
        mx.push_back(i);
        seg.add(0, i, 1);
        int cur = makeLeaf(i);
        while (true)
        {
            if (!stk.empty() && tr[stk.back()].linear && !tr[stk.back()].ch.empty() &&
                (tr[cur].mn == tr[tr[stk.back()].ch.back()].mx || tr[cur].mx ==
                ↪ tr[tr[stk.back()].ch.back()].mn))
            {
                tr[stk.back()].ch.push_back(cur);
                tr[stk.back()].r = tr[cur].r;
```

```
                tr[stk.back()].mn = min(tr[stk.back()].mn, tr[cur].mn);
                tr[stk.back()].mx = max(tr[stk.back()].mx, tr[cur].mx);
                cur = stk.back();
                stk.pop_back();
                continue;
            }
            if (!stk.empty() && (tr[stk.back()].mx == tr[cur].mn || tr[stk.back()].mn ==
            ↪ tr[cur].mx))
            {
                cur = merge({stk.back(), cur}, true);
                stk.pop_back();
                continue;
            }
            stk.push_back(cur);
            if (stk.size() == 1)
            {
                break;
            }
            if (seg.prod(0, tr[stk.back()].l) != 0)
            {
                break;
            }
            vector<int> ch{stk.back()};
            stk.pop_back();
            int l = tr[ch[0]].l, r2 = tr[ch[0]].r, lo = tr[ch[0]].mn, hi = tr[ch[0]].mx;
            while (true)
            {
                int x = stk.back();
                stk.pop_back();
                ch.push_back(x);
                l = min(l, tr[x].l);
                r2 = max(r2, tr[x].r);
                lo = min(lo, tr[x].mn);
                hi = max(hi, tr[x].mx);
                if (r2 - l == hi - lo)
                {
                    break;
                }
            }
            reverse(ch.begin(), ch.end());
            tr.push_back({l, r2, lo, hi, false, move(ch)});
            cur = tr.size() - 1;
        }
        seg.set(i, 0);
    }
    assert(stk.size() == 1);
    vector<CommonNode> ans;
    auto dfs = [&](auto &&self, int u, int fa) -> void
    {
        int id = ans.size();
        ans.push_back({tr[u].l, tr[u].r - 1, fa, tr[u].linear});
        for (int v : tr[u].ch)
```

```
        {
            self(self, v, id);
        }
    };
    dfs(dfs, stk.back(), -1);
    return ans;
}
```

`commonIntTr(p)` 要求 `p` 是 `0..n-1` 的排列，返回先序编号的节点表；节点区间是闭区间，根的 `parent=-1`，`linear=false` 表示素节点。构造时间 $O(n \log n)$、空间 $O(n)$，可覆盖 $n \le 5 \times 10^5$。它分解的是排列公共区间，不是输入图上的树；普通树的 DFS 序仍使用 6.1.010 树的邻接表转前序表。

6.2 树上分治与启发式合并

6.2.001 点分治

点分治按重心分层统计跨子树的路径：任意点对在分治过程中恰有一层首次跨过当前重心，于是“经过重心的合法点对”只需在这一层数一次。做法是收集重心到各点的距离，对全体排序后双指针数出距离和 $\le lim$ 的对，再减去每个儿子子树内部（并未真正跨重心）被多算的对以去重，删去重心后对每个子块递归。每层都从当前块重新求重心，保证子块规模减半。

```
template <class Dist = long long>
class CentPairs
{
    static_assert(numeric_limits<Dist>::is_integer && numeric_limits<Dist>::is_signed &&
                  sizeof(Dist) >= sizeof(long long));
    // n 是树点数，g[u] 的每项依次是邻点和非负边权。
    int n;
    vector<vector<array<int, 2>>> g;
    // siz、par 是当前连通块的临时子树大小与父亲，ban 标记已选重心。
    vector<int> siz, par;
    vector<bool> ban;

    int cen(int s)
    {
        // 返回 s 所在未删除连通块的重心。
        vector<int> ord;
        auto dfs = [&](auto &&self, int u, int p) -> void
        {
            par[u] = p;
            siz[u] = 1;
            ord.push_back(u);
            for (auto x : g[u])
            {
                int v = x[0];
                if (v == p || ban[v])
                {
                    continue;
                }
```

```
                self(self, v, u);
                siz[u] += siz[v];
            }
        };
        dfs(dfs, s, -1);
        int total = ord.size(), c = s, best = total;
        for (int u : ord)
        {
            int mx = total - siz[u];
            for (auto x : g[u])
            {
                int v = x[0];
                if (!ban[v] && par[v] == u)
                {
                    chmax(mx, siz[v]);
                }
            }
            if (mx < best)
            {
                best = mx;
                c = u;
            }
        }
        return c;
    }

    vector<Dist> distances(int s, int p, long long w) const
    {
        // 返回 s 所在分支内各点到当前重心的距离。
        vector<Dist> a;
        auto dfs = [&](auto &&self, int u, int f, Dist d) -> void
        {
            a.push_back(d);
            for (auto [v, ew] : g[u])
            {
                if (v != f && !ban[v])
                {
                    self(self, v, u, d + (Dist)ew);
                }
            }
        };
        dfs(dfs, s, p, (Dist)w);
        return a;
    }

    int pairs(vector<Dist> a, Dist lim) const
    {
        // a 是一组到同一根的距离，lim 是距离上限；返回无序点对距离和不超过 lim 的数量。
        sort(a.begin(), a.end());
        int ans = 0;
        for (int l = 0, r = (int)a.size() - 1; l < r;)
        {
```

```
            if (a[r] <= lim && a[l] <= lim - a[r])
            {
                ans += r - l;
                l++;
            }
            else
            {
                r--;
            }
        }
        return ans;
    }

    int solve(int s, Dist lim)
    {
        // s 是当前未删除连通块内一点，lim 是距离上限；返回该块经当前层首次统计的点对数。
        int c = cen(s);
        ban[c] = true;
        vector<Dist> all{0};
        int ans = 0;
        for (auto [v, w] : g[c])
        {
            if (ban[v])
            {
                continue;
            }
            auto cur = distances(v, c, w);
            ans -= pairs(cur, lim);
            all.insert(all.end(), cur.begin(), cur.end());
        }
        ans += pairs(all, lim);
        for (auto x : g[c])
        {
            int v = x[0];
            if (!ban[v])
            {
                ans += solve(v, lim);
            }
        }
        return ans;
    }

  public:
    CentPairs(const vector<vector<array<int, 2>>> &g) : n(g.size()), g(g), siz(n), par(n),
    ↪ ban(n)
    {
        // g 是 0 下标加权树；保存输入并初始化点分治状态，无返回值。
    }

    int countAtMost(Dist lim)
    {
        // lim 是非负距离上限；返回距离不超过 lim 的无序不同点对数量。
```

```
        fill(ban.begin(), ban.end(), false);
        return n ? solve(0, lim) : 0;
    }
};
```

构造 `CentPairs<Dist> q(g)` 后调用 `countAtMost(lim)`；`Dist` 必须是至少与输入同宽的有符号整数，边权和 `lim` 均须非负，且调用者须保证树上距离能由 `Dist` 表示；若可能超过 64 位，应改用 `CentPairs<__int128>`。时间 $O(n \log^2 n)$，空间 $O(n)$。

下面用一棵六点带权树检查零距离、局部点对和全部点对边界：

```
signed main()
{
    vector<vector<array<int, 2>>> g(6);
    vector<array<int, 3>> e{{0, 1, 2}, {0, 2, 3}, {1, 3, 1}, {1, 4, 4}, {2, 5, 2}};
    for (auto [u, v, w] : e)
    {
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
    CentPairs q(g);
    assert(q.countAtMost(0) == 0);
    assert(q.countAtMost(2) == 3);
    assert(q.countAtMost(4) == 6);
    assert(q.countAtMost(5) == 9);
    assert(q.countAtMost(11) == 15);
}
```

6.2.002 点分树

反复删重心并记录父子关系，得到点分树。因重心删除后每个子块不超过原块一半，每个原点至多出现在 $O(\log n)$ 层，故点分树树高 $O(\log n)$。注意点分树的父子边表示“包含块”关系，不是原树的边；它常配合“每点到各级点分树祖先的距离”支持带修改的路径统计。

```
class CentroidTree
{
    // n 是树点数，g 是原树邻接表。
    int n;
    vector<vector<int>> g;
    // siz、tmp 是当前块的临时子树大小与父亲，ban 标记已选重心。
    vector<int> siz, tmp;
    vector<bool> ban;
    // par[u]、dep[u] 是点 u 在点分树中的父节点与深度。
    vector<int> par, dep;

    int cen(int s)
    {
        // 返回 s 所在未删除连通块的重心。
        vector<int> ord;
        auto dfs = [&](auto &&self, int u, int p) -> void
```

```
        {
            tmp[u] = p;
            siz[u] = 1;
            ord.push_back(u);
            for (int v : g[u])
            {
                if (v == p || ban[v])
                {
                    continue;
                }
                self(self, v, u);
                siz[u] += siz[v];
            }
        };
        dfs(dfs, s, -1);
        int total = ord.size(), c = s, best = total;
        for (int u : ord)
        {
            int mx = total - siz[u];
            for (int v : g[u])
            {
                if (!ban[v] && tmp[v] == u)
                {
                    chmax(mx, siz[v]);
                }
            }
            if (mx < best)
            {
                best = mx;
                c = u;
            }
        }
        return c;
    }

    int build(int s, int p, int d)
    {
        // s 是当前块内一点，p、d 是点分树父点和深度；返回当前块重心。
        int c = cen(s);
        par[c] = p;
        dep[c] = d;
        ban[c] = true;
        for (auto v : g[c])
        {
            if (!ban[v])
            {
                build(v, c, d + 1);
            }
        }
        return c;
    }

```

```
  public:
    CentroidTree(const vector<vector<int>> &g)
        : n(g.size()), g(g), siz(n), tmp(n), ban(n), par(n, -1), dep(n)
    {
        // g 是 0 下标树；建立点分树，无返回值。
        if (n)
        {
            build(0, -1, 0);
        }
    }

    const vector<int> &parents() const
    {
        // 无参数；返回点分树父节点数组，根的父节点为 -1。
        return par;
    }

    const vector<int> &depths() const
    {
        // 无参数；返回每个原树点在点分树中的深度。
        return dep;
    }
};
```

构造 `CentroidTree tr(g)`，`parents()`（根为 -1）与 `depths()` 给出点分树拓扑。当前实现每层重新求子树大小，$O(n \log n)$ 时间、$O(n)$ 空间。要在点分树上做距离查询还需另行预处理原树两点距离（如 6.1.002 最近公共祖先：倍增）。静态一次性路径统计直接用 6.2.001 点分治 更省。

下面在 7 点链、6 点星和 7 点完全二叉树上检查点分树父亲与深度：

```
signed main()
{
    auto makeTree = [](int n, initializer_list<pair<int, int>> e)
    {
        vector<vector<int>> g(n);
        for (auto [u, v] : e)
        {
            g[u].push_back(v);
            g[v].push_back(u);
        }
        return g;
    };
    CentroidTree chain(makeTree(7, {{0, 1}, {1, 2}, {2, 3}, {3, 4}, {4, 5}, {5, 6}}));
    assert((chain.parents() == vector<int>{1, 3, 1, -1, 5, 3, 5}));
    assert((chain.depths() == vector<int>{2, 1, 2, 0, 2, 1, 2}));
    CentroidTree star(makeTree(6, {{0, 1}, {0, 2}, {0, 3}, {0, 4}, {0, 5}}));
    assert((star.parents() == vector<int>{-1, 0, 0, 0, 0, 0}));
    CentroidTree binary(makeTree(7, {{0, 1}, {0, 2}, {1, 3}, {1, 4}, {2, 5}, {2, 6}}));
    assert((binary.parents() == vector<int>{-1, 0, 0, 1, 1, 2, 2}));
    cout << "OK\n";
    return 0;
}
```

6.2.003 边分治

边分治先用零权虚点把树“二叉化”（把每个原点的多个儿子串成一条零权链，度数降到 ≤ 3 且保持任意原点间距离不变），再反复选一条使两侧原点数尽量均衡的边切开。二叉化后必存在一条边令两侧原点数都在常数比例内，故递归深度 $O(\log n)$。它擅长那些天然按边分割、而点分治难以合并的贡献（如某些路径计数）。编号 $\ge$ `origNodes()` 的是虚点，统计时不计入。

```
class EdgeDecomp
{
    // original 是原树点数；扩展图中编号 >= original 的点都是零权虚点。
    int original;
    // g[u] 的每项依次是邻点、边权、扩展树边编号。
    vector<vector<array<int, 3>>> g;
    // real[u] 表示该扩展点代表几个原点，ban[id] 标记已经切断的扩展边。
    vector<int> real, sub, par, pe;
    vector<bool> ban;
    // cut[i][0..2] 依次是父切边编号、当前扩展边两个端点。
    vector<array<int, 3>> cut;

    int node(bool isReal)
    {
        // isReal 表示是否对应原点；新增扩展点并返回编号。
        g.push_back({});
        real.push_back(isReal);
        sub.push_back(0);
        par.push_back(-1);
        pe.push_back(-1);
        return g.size() - 1;
    }

    void edge(int u, int v, int w)
    {
        // u、v 是扩展点，w 是边权；加入无向边，无返回值。
        int id = ban.size();
        ban.push_back(false);
        g[u].push_back({v, w, id});
        g[v].push_back({u, w, id});
    }

    void build(int s, int cutPar)
    {
        // s 是当前块内一点，cutPar 是分治树父切边记录；递归建立边分树，无返回值。
        vector<int> ord;
        auto dfs = [&](auto &&self, int u, int p, int pid) -> void
        {
            par[u] = p;
            pe[u] = pid;
            sub[u] = real[u];
            ord.push_back(u);
            for (auto x : g[u])
            {
```

```
                int v = x[0], id = x[2];
                if (v == p || ban[id])
                {
                    continue;
                }
                self(self, v, u, id);
                sub[u] += sub[v];
            }
        };
        dfs(dfs, s, -1, -1);
        int total = sub[s];
        if (total <= 1)
        {
            return;
        }
        array<int, 4> best{INF, -1, -1, -1};
        for (int v : ord)
        {
            if (v == s)
            {
                continue;
            }
            array<int, 4> cur{max(sub[v], total - sub[v]), pe[v], par[v], v};
            if (cur[0] < best[0])
            {
                best = cur;
            }
        }
        assert(best[1] != -1); // 调试检查，可删
        int id = cut.size();
        cut.push_back({cutPar, best[2], best[3]});
        ban[best[1]] = true;
        build(best[2], id);
        build(best[3], id);
    }

  public:
    EdgeDecomp(const vector<vector<array<int, 2>>> &tree) : original(tree.size())
    {
        // tree[u] 的每项依次是邻点和边权；二叉化后建立边分治树，无返回值。
        for (int i = 0; i < original; i++)
        {
            node(true);
        }
        vector<int> p(original, -1);
        vector<vector<array<int, 2>>> child(original);
        if (original)
        {
            auto dfs = [&](auto &&self, int u) -> void
            {
                for (auto [v, w] : tree[u])
                {
```

```
                    if (v == p[u])
                    {
                        continue;
                    }
                    p[v] = u;
                    child[u].push_back({v, w});
                    self(self, v);
                }
            };
            dfs(dfs, 0);
        }
        for (int u = 0; u < original; u++)
        {
            int cur = u;
            for (int i = 0; i < (int)child[u].size(); i++)
            {
                auto [v, w] = child[u][i];
                if (i + 1 == (int)child[u].size())
                {
                    edge(cur, v, w);
                }
                else
                {
                    int z = node(false);
                    edge(cur, z, 0);
                    edge(z, v, w);
                    cur = z;
                }
            }
        }
        if (original)
        {
            build(0, -1);
        }
    }

    const vector<array<int, 3>> &cuts() const
    {
        // 无参数；每项依次是父切分记录编号、当前切边的两个扩展点，根记录的父编号为 -1。
        return cut;
    }

    int origNodes() const
    {
        // 无参数；返回原树点数，用于区分扩展图虚点。
        return original;
    }
};
```

构造 `EdgeDecomp q(tree)`（`tree[u]` 每项 `{v, w}`），`cuts()` 返回边分树，每条记录父切边与当前切边两端点。二叉化后点边仍 $O(n)$，建树 $O(n \log n)$ 时间、$O(n)$ 空间。跨子树路径能合并时优先 6.2.001 点分治（常数更小、无虚点开销）；只有必须按边划分时才用边分治。

6.2.004 DSU on Tree

DSU on Tree 离线回答每个子树的“频率型”询问：先处理并清空所有轻儿子，再处理重儿子并保留其统计，最后把所有轻子树暴力加入当前点后回答。每个点只在跨越轻边时被重复加入，而任一点到根的轻边数 $O(\log n)$，故总加删次数 $O(n \log n)$。`add`/`remove`/`answer` 三个回调把“如何计入一个点、如何撤销、如何取答案”留给调用者，本体只管调度。

```cpp
class DsuOnTree
{
    // n 是点数，tree 是树邻接表。
    int n;
    vector<vector<int>> tree;
    // parent、size、heavy 是父亲、子树大小与重儿子；ord 是前序点列，pos[u] 是 u 在 ord 中的下标。
    vector<int> parent, size, heavy, ord, pos;

    void prepare(int root)
    {
        // 按 root 重建父亲、前序、子树大小与重儿子。
        fill(parent.begin(), parent.end(), -1);
        fill(size.begin(), size.end(), 1);
        fill(heavy.begin(), heavy.end(), -1);
        ord.clear();
        auto dfs = [&](auto &&self, int u, int p) -> void
        {
            parent[u] = p;
            pos[u] = ord.size();
            ord.push_back(u);
            for (int v : tree[u])
            {
                if (v != p)
                {
                    self(self, v, u);
                    size[u] += size[v];
                    if (heavy[u] == -1 || size[v] > size[heavy[u]])
                    {
                        heavy[u] = v;
                    }
                }
            }
        };
        dfs(dfs, root, -1);
    }

    template <class F> void visitSubtree(int u, F &&f) const
    {
        // 按前序遍历 u 的整棵子树，并对每个点调用 f。
        for (int i = pos[u]; i < pos[u] + size[u]; i++)
        {
            f(ord[i]);
        }
    }
```

```cpp

public:
  explicit DsuOnTree(vector<vector<int>> tree)
      : n(tree.size()), tree(move(tree)), parent(n), size(n), heavy(n, -1), pos(n)
  {
      // 保存树；根相关信息在 run 时重建。
  }

  template <class Add, class Remove, class Answer> void run(Add &&add, Remove &&remove,
  ↪ Answer &&answer, int root = 0)
  {
      // add/remove 增删单点贡献，answer(u) 在 u 的子树状态齐备时读取答案。
      assert(n > 0 && 0 <= root && root < n); // 调试检查，可删
      prepare(root);
      auto dfs = [&](auto &&self, int u, bool keep) -> void
      {
          for (int v : tree[u])
          {
              if (v != parent[u] && v != heavy[u])
              {
                  self(self, v, false);
              }
          }
          if (heavy[u] != -1)
          {
              self(self, heavy[u], true);
          }
          for (int v : tree[u])
          {
              if (v != parent[u] && v != heavy[u])
              {
                  visitSubtree(v, add);
              }
          }
          add(u);
          answer(u);
          if (!keep)
          {
              visitSubtree(u, remove);
          }
      };
      dfs(dfs, root, false);
  }
};
```

构造 `DsuOnTree q(tree)` 后 `run(add, remove, answer, root)`：`add(u)`/`remove(u)` 增删单点贡献、`answer(u)` 在 u 子树贡献齐备时读答案。全程 $O(n \log n)$ 次回调、$O(n)$ 结构自身空间（不含回调内部数据结构）。它要求询问可离线且贡献可增量增删；可减性不成立或需在线时须另择方案。相比 6.2.001 点分治 面向路径统计，DSU on Tree 面向子树内的可加删信息。

例如统计每个子树的点权和：

```cpp
signed main()
{
    vector<vector<int>> tree{{1, 2}, {0}, {0}};
    vector<int> w{1, 2, 3}, ans(3);
    int sum = 0;
    auto add = [&](int u) { sum += w[u]; };
    auto remove = [&](int u) { sum -= w[u]; };
    auto answer = [&](int u) { ans[u] = sum; };
    DsuOnTree q(tree);
    q.run(add, remove, answer);
    assert((ans == vector<int>{6, 2, 3}));
}
```

6.2.005 长链剖分

长链剖分按“最深儿子”划分长链，与轻重链按子树大小划分互补，专治状态只沿深度平移一位的树形递推：重儿子的深度型 DP 数组可让父节点直接偏移一位复用，轻儿子逐项暴力合并，总合并量由各长链长度之和 $O(n)$ 控制。它有两个经典用途分别落在别处——以深度为下标的 DP 合并见 6.5.003 长链剖分 DP 合并；$O(1)$ 查询 K 级祖先则在长链基础上再配一张倍增表，本条直接复用 6.1.006 树上距离、K 级祖先 的 `KthAncestor` 倍增预处理即可。相较 6.2.004 DSU on Tree 的子树频率合并，长链剖分只对“深度平移”型状态有效，两者不可互换。

6.2.006 毛毛虫剖分

毛毛虫剖分把树拆成少量“主干路径 + 挂在其上一层附属点”的毛毛虫块，用于满足毛毛虫结构约束的路径覆盖或 DP；它的成立高度依赖题目对路径、距离或度数的特殊约束，没有统一模板，通常在轻重链框架上按题目附加规则切链。本条复用 6.3.001 轻重链剖分 / HLD 的 `HLD` 作为主干与 DFS 序底座，再在其重链结构上按题意归并附属点。一般的路径修改查询直接用 6.3.001 即可，无需毛毛虫特化。

6.3 树链与虚树

6.3.001 轻重链剖分 / HLD

轻重链剖分把任意树路径拆成 $O(\log n)$ 段 DFS 序连续区间：按子树大小选重儿子，重链在 DFS 序中连续，任一路径跨越的轻边至多 $O(\log n)$ 条。配一维数据结构即可支持路径修改/查询、子树查询与 LCA。`path(u, v, op, edge)` 把路径拆段回调 `op(l, r, rev)`，`rev` 标记该段应逆序读取——这对非交换信息（矩阵、仿射变换）至关重要；`edge = true` 时处理边权（跳过 LCA 自身那一点）。

```cpp
struct HLD
{
    // n 是点数，tim 是 DFS 序计数器。
    int n, tim = 0;
    // fa、dep、sz、son、top、in 依次是父亲、深度、子树大小、重儿子、链顶和 DFS 序位置。
    // rev[i] 是 DFS 序位置 i 对应的原节点。
    vector<int> fa, dep, sz, son, top, in, rev;
```

```cpp

    HLD(const vector<vector<int>> &g, int root = 0)
        : n(g.size()), fa(n, -1), dep(n), sz(n, 1), son(n, -1), top(n), in(n), rev(n)
    {
        // g 是非空树，root 是根。
        assert(n > 0 && 0 <= root && root < n); // 调试检查，可删
        auto dfs1 = [&](auto &&self, int u, int p) -> void
        {
            fa[u] = p;
            for (int v : g[u])
            {
                if (v == p)
                {
                    continue;
                }
                dep[v] = dep[u] + 1;
                self(self, v, u);
                sz[u] += sz[v];
                if (son[u] < 0 || sz[v] > sz[son[u]])
                {
                    son[u] = v;
                }
            }
        };
        auto dfs2 = [&](auto &&self, int u, int h) -> void
        {
            top[u] = h;
            rev[tim] = u;
            in[u] = tim++;
            if (son[u] >= 0)
            {
                self(self, son[u], h);
            }
            for (int v : g[u])
            {
                if (v != fa[u] && v != son[u])
                {
                    self(self, v, v);
                }
            }
        };
        dfs1(dfs1, root, -1);
        dfs2(dfs2, root, root);
    }

    int lca(int u, int v) const
    {
        // 返回 u、v 的最近公共祖先。
        while (top[u] != top[v])
        {
            if (dep[top[u]] < dep[top[v]])
            {
```

```
                swap(u, v);
            }
            u = fa[top[u]];
        }
        return dep[u] < dep[v] ? u : v;
    }

    // op(l,r,rev): 区间为 [l,r)，rev 表示沿路径应逆序读取。
    template <class F> void path(int u, int v, F op, bool edge = false) const
    {
        // 按 u 到 v 的顺序回调各段；edge=true 时每条边存到较深端点。
        vector<pair<int, int>> right;
        while (top[u] != top[v])
        {
            if (dep[top[u]] >= dep[top[v]])
            {
                op(in[top[u]], in[u] + 1, true);
                u = fa[top[u]];
            }
            else
            {
                right.push_back({in[top[v]], in[v] + 1});
                v = fa[top[v]];
            }
        }
        if (dep[u] >= dep[v])
        {
            int l = in[v] + edge, r = in[u] + 1;
            if (l < r)
            {
                op(l, r, true);
            }
        }
        else
        {
            int l = in[u] + edge, r = in[v] + 1;
            if (l < r)
            {
                right.push_back({l, r});
            }
        }
        reverse(right.begin(), right.end());
        for (auto [l, r] : right)
        {
            op(l, r, false);
        }
    }

    pair<int, int> subtree(int u) const
    {
        // 返回 u 的子树对应的 DFS 序半开区间。
        return {in[u], in[u] + sz[u]};
```

```
    }
};
```

构造 `HLD(g, root)`；`lca` $O(\log n)$、`subtree(u)` 返回子树区间 $O(1)$、`path` 拆成 $O(\log n)$ 段回调，构造 $O(n)$ 时间、$O(n)$ 空间。把 `op` 接到线段树/树状数组即得完整路径数据结构，示例见 6.3.004 树上路径/子树数据结构；只保留询问关键点时压成 6.3.003 虚树。非交换信息务必用 `rev` 标记分别维护左右两半再合并，不能直接把两段区间并起来。

最小调用只记录所有拆分段的长度：

```
signed main()
{
    vector<vector<int>> tree{{1}, {0, 2}, {1}};
    HLD hld(tree);
    int len = 0;
    hld.path(0, 2, [&](int l, int r, bool rev)
    {
        (void)rev;
        len += r - l;
    });
    assert(len == 3);
}
```

6.3.002 换根树剖

在不重建剖分的前提下回答“动态根”下的子树范围。固定一次剖分后，查询 u 在当前根 `root` 下的子树分三类：$u = root$ 时是全树；`root` 不在 u 的原子树内时仍是原区间；否则答案是全树挖去“通向 `root` 的那个儿子”的子树，即至多两段区间。`childOnPath` 沿重链找出 u 到 `root` 路径上紧邻 u 的那个儿子。

```
class RerootHLD
{
    // n 是点数，timer 是 DFS 序计数器。
    int n, timer = 0;
    // tree 是树邻接表。
    vector<vector<int>> tree;
    // parent、depth、size、heavy、head、position 依次是父亲、深度、子树大小、重儿子、链顶和 DFS 序位置。
    // vertexAt[i] 是 DFS 序位置 i 对应的原节点。
    vector<int> parent, depth, size, heavy, head, position, vertexAt;

    void firstDfs(int u)
    {
        // 从 u 出发递归计算父亲、深度、子树大小和重儿子。
        auto dfs = [&](auto &&self, int x, int p) -> void
        {
            parent[x] = p;
            for (int v : tree[x])
            {
                if (v == p)
                {
```

```
                    continue;
                }
                depth[v] = depth[x] + 1;
                self(self, v, x);
                size[x] += size[v];
                if (heavy[x] == -1 || size[v] > size[heavy[x]])
                {
                    heavy[x] = v;
                }
            }
        };
        fill(size.begin(), size.end(), 1);
        dfs(dfs, u, -1);
    }

    void secondDfs(int u, int top)
    {
        // 从 u 开始递归分配链顶和 DFS 序。
        auto dfs = [&](auto &&self, int x, int top) -> void
        {
            head[x] = top;
            position[x] = timer++;
            vertexAt[position[x]] = x;
            if (heavy[x] != -1)
            {
                self(self, heavy[x], top);
            }
            for (int v : tree[x])
            {
                if (v != parent[x] && v != heavy[x])
                {
                    self(self, v, v);
                }
            }
        };
        dfs(dfs, u, top);
    }

    int childOnPath(int anc, int desc) const
    {
        // anc 是 desc 的真祖先；返回路径上紧邻 anc 的儿子。
        while (head[anc] != head[desc])
        {
            if (parent[head[desc]] == anc)
            {
                return head[desc];
            }
            desc = parent[head[desc]];
        }
        return vertexAt[position[anc] + 1];
    }

```

```
73    public:
74      RerootHLD(vector<vector<int>> tree, int root = 0)
75          : n(tree.size()), tree(move(tree)), parent(n, -1), depth(n), size(n), heavy(n, -1),
            ↪  head(n), position(n),
76            vertexAt(n)
77      {
78          // tree 是非空树，root 是预处理根。
79          assert(n > 0 && 0 <= root && root < n); // 调试检查，可删
80          firstDfs(root);
81          secondDfs(root, root);
82      }
83
84      vector<pair<int, int>> pathSegments(int u, int v) const
85      {
86          // 返回覆盖 u-v 路径的 DFS 序区间；区间顺序不表示路径方向。
87          vector<pair<int, int>> result;
88          while (head[u] ≠ head[v])
89          {
90              if (depth[head[u]] < depth[head[v]])
91              {
92                  swap(u, v);
93              }
94              result.push_back({position[head[u]], position[u] + 1});
95              u = parent[head[u]];
96          }
97          if (depth[u] > depth[v])
98          {
99              swap(u, v);
100         }
101         result.push_back({position[u], position[v] + 1});
102         return result;
103     }
104
105     int lca(int u, int v) const
106     {
107         // 返回 u、v 的最近公共祖先。
108         while (head[u] ≠ head[v])
109         {
110             if (depth[head[u]] < depth[head[v]])
111             {
112                 swap(u, v);
113             }
114             u = parent[head[u]];
115         }
116         return depth[u] < depth[v] ? u : v;
117     }
118
119     int distance(int u, int v) const
120     {
121         // 返回 u、v 间的边数。
122         int p = lca(u, v);
123         return depth[u] + depth[v] - 2 * depth[p];
```

```
    }

    vector<pair<int, int>> subtree(int u, int root) const
    {
        // 返回以 root 为当前根时 u 子树对应的至多两个 DFS 序区间。
        if (u == root)
        {
            return {{0, n}};
        }
        if (!(position[u] <= position[root] && position[root] < position[u] + size[u]))
        {
            return {{position[u], position[u] + size[u]}};
        }
        int child = childOnPath(u, root);
        vector<pair<int, int>> result;
        if (position[child])
        {
            result.push_back({0, position[child]});
        }
        if (position[child] + size[child] < n)
        {
            result.push_back({position[child] + size[child], n});
        }
        return result;
    }
};
```

构造 `RerootHLD(tree, root)`；`pathSegments`、`lca`、`distance` 及带动态根的 `subtree(u, root)`（返回至多 2 段 DFS 序区间）。构造 $O(n)$，`pathSegments`/`subtree` 均 $O(\log n)$，空间 $O(n)$。本类的 `lca` 也是 6.1.003 最近公共祖先：树链剖分 所引用的实现。仅静态根时用 6.3.001 轻重链剖分 / HLD 更直接。

6.3.003 虚树

虚树只保留询问关键点及其两两必要的 LCA，把规模压到 $O(k)$，供单次关键点 DP 在压缩树上运行。做法：关键点按 DFS 序 `in` 排序，相邻两点补入它们的 LCA 后再排序去重，用单调栈按祖先关系连边——栈中始终是当前点到根路径上的祖先链，弹到栈顶是当前点祖先为止再连边。返回点数不超过 $2k-1$。本条只依赖 6.1.002 最近公共祖先：倍增 的 `in` 与 `lca`/`anc`。

```
struct VTree
{
    // root 是虚树根；空虚树时为 -1。
    int root = -1;
    // nodes 按原树 DFS 序保存虚树点，edges 按儿子 DFS 序保存从父亲指向儿子的边。
    vector<int> nodes;
    vector<pair<int, int>> edges;
};

VTree virtualTree(vector<int> a, const LCA &lca)
{
```

```
    // a 是关键点列表，lca 是原树 LCA；返回关键点及必要 LCA 构成的虚树。
    if (a.empty())
    {
        return {};
    }
    sort(a.begin(),
         a.end(),
         [&](int x, int y)
         {
             return lca.in[x] < lca.in[y];
         });
    a.erase(unique(a.begin(), a.end()), a.end());
    int n = a.size();
    for (int i = 1; i < n; i++)
    {
        a.push_back(lca.lca(a[i - 1], a[i]));
    }
    sort(a.begin(),
         a.end(),
         [&](int x, int y)
         {
             return lca.in[x] < lca.in[y];
         });
    a.erase(unique(a.begin(), a.end()), a.end());
    vector<int> st;
    VTree ans;
    ans.root = a[0];
    ans.nodes = a;
    for (int u : a)
    {
        while (!st.empty() && !lca.anc(st.back(), u))
        {
            st.pop_back();
        }
        if (!st.empty())
        {
            ans.edges.push_back({st.back(), u});
        }
        st.push_back(u);
    }
    return ans;
}
```

`virtualTree(a, lca)` 传入关键点集与已建好的 `LCA`，返回 `VTree{root, nodes, edges}`（`a` 为空返回空树）。排序 $O(k \log k)$，补 LCA 与建树 $O(k \log n)$，点数 $\le 2k-1$。虚树与原树同构地保留了关键点间的祖先/路径关系，但边“长度”需按原树 `dist` 另算。多次不同关键点集询问时每次重建。

下面用一棵八点树检查关键点去重、补 LCA、连边和空集合（需同时粘贴 6.1.002 的 `LCA`）：

```
signed main()
{
    vector<vector<int>> g(8);
```

```
    vector<pair<int, int>> e{{0, 1}, {0, 2}, {1, 3}, {1, 4}, {2, 5}, {5, 6}, {5, 7}};
    for (auto [u, v] : e)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    LCA lca(g);
    VTree vt = virtualTree({3, 4, 6, 7, 3}, lca);
    assert(vt.root == 0);
    assert((vt.nodes == vector<int>{0, 1, 3, 4, 5, 6, 7}));
    vector<pair<int, int>> edges{{0, 1}, {1, 3}, {1, 4}, {0, 5}, {5, 6}, {5, 7}};
    assert(vt.edges == edges);
    vt = virtualTree({}, lca);
    assert(vt.root == -1 && vt.nodes.empty() && vt.edges.empty());
}
```

6.3.004 树上路径/子树数据结构

静态树上“单点赋值 + 路径点权和 + 固定根子树点权和”的组合板子：路径交给重链剖分（每条根路径 $\leq O(\log n)$ 条重链、链内 DFS 序连续），子树交给 DFS 序连续区间，点权放在 DFS 序上的树状数组里。它不支持动态连边断边——那需要 6.4.001 Link-Cut Tree。

```
template <class Acc = long long>
class TreePathSum
{
    // n 保存点数，ti 保存 DFS 序计数器。
    int n, ti = 0;
    // g 保存静态树邻接表。
    vector<vector<int>> g;
    // fa、dep、siz、son、top、in 依次保存父节点、深度、子树大小、重儿子、链顶和 DFS 序。
    vector<int> fa, dep, siz, son, top, in;
    // val 保存各点当前权值，bit 保存 DFS 序上的前缀和。
    vector<Acc> val, bit;

    void build(int root)
    {
        // 递归建立父亲、重儿子、链顶与 DFS 序。
        auto dfs1 = [&](auto &&self, int u, int p) -> void
        {
            fa[u] = p;
            for (int v : g[u])
            {
                if (v == p)
                {
                    continue;
                }
                dep[v] = dep[u] + 1;
                self(self, v, u);
                siz[u] += siz[v];
                if (son[u] == -1 || siz[v] > siz[son[u]])
```

```
                {
                    son[u] = v;
                }
            }
        };
        auto dfs2 = [&](auto &&self, int u, int h) -> void
        {
            top[u] = h;
            in[u] = ti++;
            if (son[u] != -1)
            {
                self(self, son[u], h);
            }
            for (int v : g[u])
            {
                if (v != fa[u] && v != son[u])
                {
                    self(self, v, v);
                }
            }
        };
        dfs1(dfs1, root, -1);
        dfs2(dfs2, root, root);
    }

    void addBit(int p, Acc v)
    {
        // p 是 0 下标 DFS 序位置，v 是增量；更新树状数组，无返回值。
        for (p++; p <= n; p += p & -p)
        {
            bit[p] += v;
        }
    }

    Acc sumBit(int p) const
    {
        // p 是右端点；返回半开前缀 [0,p) 的点权和。
        Acc ans = 0;
        for (; p; p -= p & -p)
        {
            ans += bit[p];
        }
        return ans;
    }

  public:
    TreePathSum(const vector<vector<int>> &g, const vector<Acc> &a, int root = 0)
        : n(g.size()), g(g), fa(n, -1), dep(n), siz(n, 1), son(n, -1), top(n), in(n), val(a),
        ↪ bit(n + 1)
    {
        // g 是静态树，a 是各点初值，root 是根；完成重链剖分与树状数组初始化，无返回值。
        assert(n > 0 && (int)a.size() == n && 0 <= root && root < n); // 调试检查，可删
```

```
        build(root);
        for (int u = 0; u < n; u++)
        {
            addBit(in[u], val[u]);
        }
    }

    void set(int u, Acc v)
    {
        // u 是节点编号，v 是新点权；执行单点赋值，无返回值。
        assert(0 <= u && u < n); // 调试检查，可删
        addBit(in[u], v - val[u]);
        val[u] = v;
    }

    Acc pathSum(int u, int v) const
    {
        // u、v 是路径两端；返回简单路径上全部点权之和。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删
        Acc ans = 0;
        while (top[u] != top[v])
        {
            if (dep[top[u]] < dep[top[v]])
            {
                swap(u, v);
            }
            ans += sumBit(in[u] + 1) - sumBit(in[top[u]]);
            u = fa[top[u]];
        }
        if (dep[u] > dep[v])
        {
            swap(u, v);
        }
        return ans + sumBit(in[v] + 1) - sumBit(in[u]);
    }

    Acc subtreeSum(int u) const
    {
        // u 是子树根；返回固定构造根下 u 的整棵子树点权和。
        assert(0 <= u && u < n); // 调试检查，可删
        return sumBit(in[u] + siz[u]) - sumBit(in[u]);
    }
};
```

构造 `TreePathSum q(g, a, root)`；默认累加类型为 `long long`，点权、`set` 参数及两个求和函数均使用 `Acc`。必须保证路径和、子树和以及 `set` 产生的点权差不超出 `Acc`；可能超过 64 位时使用 `TreePathSum<__int128>`，并传入 `vector<__int128>` 点权。预处理 $O(n)$，`set`/`subtreeSum` $O(\log n)$、`pathSum` $O(\log^2 n)$，空间 $O(n)$。求和满足可减性才能用树状数组；最值或非交换信息改用线段树接到 6.3.001 轻重链剖分 / HLD 的 `path` 回调。

下面在固定 7 点分叉树上检查同点路径、跨重链路径、根与内部子树，并在修改根、内部点和叶子后复查：

```cpp
signed main()
{
    vector<vector<int>> g(7);
    for (auto [u, v] : vector<pair<int, int>>{{0, 1}, {0, 2}, {1, 3}, {1, 4}, {2, 5}, {5, 6}})
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<long long> a{5, -2, 4, 7, 1, -3, 6};
    TreePathSum q(g, a);
    assert(q.pathSum(4, 4) == 1);
    assert(q.pathSum(3, 6) == 17);
    assert(q.subtreeSum(0) == 18);
    assert(q.subtreeSum(1) == 6);
    q.set(0, 10);
    assert(q.pathSum(3, 6) == 22);
    q.set(1, 8);
    assert(q.pathSum(3, 6) == 32);
    assert(q.subtreeSum(1) == 16);
    q.set(6, -4);
    assert(q.pathSum(3, 6) == 22);
    assert(q.subtreeSum(5) == -7);
    cout << "OK\n";
    return 0;
}
```

6.3.005 树上差分

静态树上有许多次路径加、最后才一次性询问全部点或边时，无需树剖逐次修改。固定根并令 $p=\operatorname{lca}(u,v)$：路径点加 x 在差分数组中执行 $d_u+=x, d_v+=x, d_p-=x$，若 p 不是根再令 $d_{\text{parent}(p)}-=x$；路径边加则执行 $d_u+=x, d_v+=x, d_p-=2x$。全部修改结束后按 DFS 序逆序把儿子累加到父亲，点差分直接落到节点，边差分在节点 u 处表示边 $(\text{parent}(u),u)$ 的答案。下面复用 6.1.002 最近公共祖先：倍增 的 `LCA`，不重复祖先表实现。

```cpp
// 依赖 6.1.002 最近公共祖先：倍增
template <class T = long long> class TreeDiff
{
    // n 是点数，root 是固定树根。
    int n, root;
    // lca 保存倍增祖先、深度与 DFS 序。
    LCA lca;
    // order 按父亲先于儿子的 DFS 序保存节点。
    vector<int> order;
    // dv 与 edgeDelta 分别保存路径点加、路径边加的树上差分。
    vector<T> dv, edgeDelta;

    vector<T> collect(vector<T> value) const
    {
        // value 是一份树上差分；返回自底向上累加后的每点结果。
```

```
        for (int i = n - 1; i > 0; i--)
        {
            int u = order[i];
            value[lca.up[0][u]] += value[u];
        }
        return value;
    }

  public:
    TreeDiff(const vector<vector<int>> &g, int root = 0)
        : n(g.size()), root(root), lca(g, root), order(n), dv(n), edgeDelta(n)
    {
        // g 是非空无向树，root 是固定根；预处理祖先与汇总顺序。
        assert(n > 0 && 0 <= root && root < n); // 调试检查，可删
        for (int u = 0; u < n; u++)
        {
            order[lca.in[u]] = u;
        }
    }

    void clear()
    {
        // 无参数；清空已经记录的点路径与边路径增量，无返回值。
        fill(dv.begin(), dv.end(), T{});
        fill(edgeDelta.begin(), edgeDelta.end(), T{});
    }

    void addVtxPath(int u, int v, const T &delta)
    {
        // u、v 是路径两端，delta 是增量；给简单路径上的每个点增加 delta。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删
        int p = lca.lca(u, v);
        dv[u] += delta;
        dv[v] += delta;
        dv[p] -= delta;
        if (p != root)
        {
            dv[lca.up[0][p]] -= delta;
        }
    }

    void addEdgePath(int u, int v, const T &delta)
    {
        // u、v 是路径两端，delta 是增量；给简单路径上的每条边增加 delta。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删
        int p = lca.lca(u, v);
        edgeDelta[u] += delta;
        edgeDelta[v] += delta;
        edgeDelta[p] -= delta;
        edgeDelta[p] -= delta;
    }

```

```
    vector<T> val() const
    {
        // 无参数；返回全部点累计的路径点增量，按节点编号排列。
        return collect(dv);
    }

    vector<T> edgeValues() const
    {
        // 无参数；返回每点到父亲边的累计增量，根位置为零。
        auto ans = collect(edgeDelta);
        ans[root] = T{};
        return ans;
    }
};
```

构造 `TreeDiff<T>(g,root)` 后用 `addVtxPath` 与 `addEdgePath` 分别记录两套互不干扰的增量；`val()`、`edgeValues()` 都在副本上汇总，可重复调用且不会清空记录，`clear()` 才重置。预处理沿用 LCA 的 $O(n \log n)$ 时间与空间，每次路径修改 $O(\log n)$，一次汇总 $O(n)$；`T` 须支持默认零值、`+=` 与 `-=`，并能容纳累计结果。在线穿插查询用 6.3.004 树上路径/子树数据结构，动态连边断边则用 6.4.001 Link-Cut Tree。

```
signed main()
{
    vector<vector<int>> g{{1, 2}, {0, 3}, {0}, {1}};
    TreeDiff<int> d(g);
    d.addVtxPath(2, 3, 5);
    d.addVtxPath(1, 3, 2);
    d.addEdgePath(2, 3, 5);
    d.addEdgePath(1, 3, 2);
    assert(d.val() == vector<int>({5, 7, 5, 7}));
    // 根为 0，边答案依次对应空边、0-1、0-2、1-3。
    assert(d.edgeValues() == vector<int>({0, 5, 5, 7}));
}
```

6.4 动态树

6.4.001 Link-Cut Tree

LCT 维护由 `link`/`cut` 动态变化的森林，支持连通性、点权修改与路径点权和。核心是用 Splay 维护每条“首选路径”：`access(x)` 把根到 x 的路径改成首选路径并暴露在一棵 Splay 里，`makeRoot` 用翻转标记换根，`findRoot` 找路径最左端。Splay 旋转只重排辅助树、不改变它表示的原森林；路径一旦暴露，辅助树根的 `sum` 就是整条路径的答案。`isRoot` 判某点是否为其所在 Splay 的根（看父亲是否认它作儿子）。

```
template <class Acc = long long>
class LinkCutTree
{
    struct Node
    {
```

```
        // ch[0..1] 依次保存辅助树左右儿子编号。
        array<int, 2> ch{};
        // fa 保存辅助树父节点，rev 标记辅助子树翻转。
        int fa = 0;
        bool rev = false;
        // val 保存当前点权，sum 保存辅助树中路径节点的聚合和。
        Acc val = 0, sum = 0;
    };

    // t[0] 是空节点，t[u+1] 对应外部 0 下标节点 u。
    vector<Node> t;

    bool isRoot(int x) const
    {
        // x 是内部节点编号；返回 x 是否为当前辅助树根。
        int f = t[x].fa;
        return !f || (t[f].ch[0] != x && t[f].ch[1] != x);
    }

    void pull(int x)
    {
        // x 是内部节点编号；由两个儿子重新计算路径聚合和，无返回值。
        t[x].sum = t[t[x].ch[0]].sum + t[x].val + t[t[x].ch[1]].sum;
    }

    void reverseNode(int x)
    {
        // x 是内部节点编号；翻转辅助子树并记录懒标记，无返回值。
        if (!x)
        {
            return;
        }
        swap(t[x].ch[0], t[x].ch[1]);
        t[x].rev ^= 1;
    }

    void push(int x)
    {
        // x 是内部节点编号；下传翻转标记，无返回值。
        if (!t[x].rev)
        {
            return;
        }
        reverseNode(t[x].ch[0]);
        reverseNode(t[x].ch[1]);
        t[x].rev = false;
    }

    void rotate(int x)
    {
        // x 是要上旋的内部节点；执行一次 Splay 旋转并维护聚合，无返回值。
        int y = t[x].fa, z = t[y].fa, k = t[y].ch[1] == x, w = t[x].ch[k ^ 1];
```

```
        if (!isRoot(y))
        {
            t[z].ch[t[z].ch[1] == y] = x;
        }
        t[x].fa = z;
        t[x].ch[k ^ 1] = y;
        t[y].fa = x;
        t[y].ch[k] = w;
        if (w)
        {
            t[w].fa = y;
        }
        pull(y);
        pull(x);
    }

    void splay(int x)
    {
        // x 是内部节点；把它旋到当前辅助树根，无返回值。
        vector<int> stk{x};
        for (int y = x; !isRoot(y); y = t[y].fa)
        {
            stk.push_back(t[y].fa);
        }
        while (!stk.empty())
        {
            push(stk.back());
            stk.pop_back();
        }
        while (!isRoot(x))
        {
            int y = t[x].fa, z = t[y].fa;
            if (!isRoot(y))
            {
                rotate((t[y].ch[1] == x) == (t[z].ch[1] == y) ? y : x);
            }
            rotate(x);
        }
    }

    void access(int x)
    {
        // x 是内部节点；把动态树根到 x 的路径改为首选路径，无返回值。
        int y = 0;
        while (x)
        {
            splay(x);
            int p = t[x].fa;
            t[x].ch[1] = y;
            pull(x);
            y = x;
            x = p;
```

```
        }
    }

    void makeRootNode(int x)
    {
        // x 是内部节点；把它改为所在动态树的根，无返回值。
        access(x);
        splay(x);
        reverseNode(x);
    }

    int findRootNode(int x)
    {
        // x 是内部节点；返回所在动态树根的内部编号。
        access(x);
        splay(x);
        while (true)
        {
            push(x);
            if (!t[x].ch[0])
            {
                break;
            }
            x = t[x].ch[0];
        }
        splay(x);
        return x;
    }

  public:
    LinkCutTree(const vector<Acc> &a = {}) : t(a.size() + 1)
    {
        // a 是各节点初始点权；建立 n 个互不连通的节点，无返回值。
        for (int i = 0; i < (int)a.size(); i++)
        {
            t[i + 1].val = a[i];
            t[i + 1].sum = a[i];
        }
    }

    bool same(int u, int v)
    {
        // u、v 是外部节点编号；返回两点是否连通。
        return connected(u, v);
    }

    bool link(int u, int v)
    {
        // u、v 是外部节点编号；不连通时连边并返回 true，否则不修改并返回 false。
        u++;
        v++;
        makeRootNode(u);
```

```
        if (findRootNode(v) == u)
        {
            return false;
        }
        t[u].fa = v;
        return true;
    }

    bool cut(int u, int v)
    {
        // u、v 是外部节点编号；边存在时删除并返回 true，否则返回 false。
        u++;
        v++;
        makeRootNode(u);
        access(v);
        splay(v);
        if (t[v].ch[0] != u || t[u].ch[1])
        {
            return false;
        }
        t[v].ch[0] = 0;
        t[u].fa = 0;
        pull(v);
        return true;
    }

    bool connected(int u, int v)
    {
        // u、v 是外部节点编号；返回两点当前是否属于同一棵动态树。
        return findRootNode(u + 1) == findRootNode(v + 1);
    }

    void set(int u, Acc v)
    {
        // u 是外部节点编号，v 是新点权；执行单点赋值，无返回值。
        u++;
        access(u);
        splay(u);
        t[u].val = v;
        pull(u);
    }

    Acc pathSum(int u, int v)
    {
        // u、v 必须连通；返回两点简单路径上的点权和。
        assert(connected(u, v)); // 调试检查，可删
        u++;
        v++;
        makeRootNode(u);
        access(v);
        splay(v);
        return t[v].sum;
```

```
    }
};
```

构造 `LinkCutTree q(a)`；默认点权、`set` 参数和 `pathSum` 返回值均为 `long long`。必须保证任意时刻维护的路径和能装入 `Acc`；可能超过 64 位时使用 `LinkCutTree<__int128>`，并传入 `vector<__int128>` 点权。`link`/`cut` 失败时返回 `false`，`same`/`connected` 判连通；路径和要求两点连通。各操作均摊 $O(\log n)$、空间 $O(n)$，单次最坏 $O(n)$。它只维护首选路径、不含虚儿子子树信息；要子树聚合用 6.4.002 带子树信息的 LCT。

下面用六个点检查连边、断边、连通性、点权修改、路径和及失败操作：

```
signed main()
{
    LinkCutTree q({1, 2, 3, 4, 5, 6});
    assert(q.link(0, 1));
    assert(q.link(1, 2));
    assert(q.link(1, 3));
    assert(q.link(3, 4));
    assert(!q.link(2, 4));
    assert(q.pathSum(2, 4) == 14);
    q.set(1, -2);
    assert(q.pathSum(2, 4) == 10);
    assert(q.cut(1, 3));
    assert(!q.same(2, 4));
    assert(!q.cut(1, 3));
    assert(q.link(2, 5));
    assert(q.link(5, 4));
    assert(q.pathSum(0, 4) == 13);
    assert(!q.cut(0, 2));
}
```

6.4.002 带子树信息的 LCT

在 LCT 上额外维护虚儿子贡献，从而同时回答路径和与“指定根方向下的子树和”。关键在 `access` 更换首选右儿子时：旧右儿子转为虚儿子、新右儿子移出虚集合，用 `vir += 旧.sum - 新.sum` 维持所有非首选子树的贡献；`link` 时把接入子树的 `sum` 计入父亲的 `vir`。`path` 是首选路径和、`sum` 是含虚儿子的整个辅助子树和。

```
template <class Acc = long long>
class SubtreeLCT
{
    struct Node
    {
        // ch[0..1] 依次保存辅助树左右儿子编号。
        array<int, 2> ch{};
        // fa 保存辅助树父节点，rev 标记辅助子树翻转。
        int fa = 0;
        bool rev = false;
        // val、path、vir、sum 依次保存点权、首选路径和、虚儿子贡献、整棵辅助子树贡献。
```

```
        Acc val = 0, path = 0, vir = 0, sum = 0;
    };

    // t[0] 是空节点，t[u+1] 对应外部节点 u。
    vector<Node> t;

    bool isRoot(int x) const
    {
        // x 是内部节点编号；返回 x 是否为当前辅助树根。
        int f = t[x].fa;
        return !f || (t[f].ch[0] != x && t[f].ch[1] != x);
    }

    void pull(int x)
    {
        // x 是内部节点；重算首选路径和与包含虚儿子的总和，无返回值。
        auto [l, r] = t[x].ch;
        t[x].path = t[l].path + t[x].val + t[r].path;
        t[x].sum = t[l].sum + t[x].val + t[r].sum + t[x].vir;
    }

    void reverseNode(int x)
    {
        // x 是内部节点；翻转辅助子树并记录标记，无返回值。
        if (!x)
        {
            return;
        }
        swap(t[x].ch[0], t[x].ch[1]);
        t[x].rev ^= 1;
    }

    void push(int x)
    {
        // x 是内部节点；下传翻转标记，无返回值。
        if (!t[x].rev)
        {
            return;
        }
        reverseNode(t[x].ch[0]);
        reverseNode(t[x].ch[1]);
        t[x].rev = false;
    }

    void rotate(int x)
    {
        // x 是要上旋的内部节点；执行一次 Splay 旋转并维护信息，无返回值。
        int y = t[x].fa, z = t[y].fa, k = t[y].ch[1] == x, w = t[x].ch[k ^ 1];
        if (!isRoot(y))
        {
            t[z].ch[t[z].ch[1] == y] = x;
        }
```

```
        t[x].fa = z;
        t[x].ch[k ^ 1] = y;
        t[y].fa = x;
        t[y].ch[k] = w;
        if (w)
        {
            t[w].fa = y;
        }
        pull(y);
        pull(x);
    }

    void splay(int x)
    {
        // x 是内部节点；把它旋到辅助树根，无返回值。
        vector<int> stk{x};
        for (int y = x; !isRoot(y); y = t[y].fa)
        {
            stk.push_back(t[y].fa);
        }
        while (!stk.empty())
        {
            push(stk.back());
            stk.pop_back();
        }
        while (!isRoot(x))
        {
            int y = t[x].fa, z = t[y].fa;
            if (!isRoot(y))
            {
                rotate((t[y].ch[1] == x) == (t[z].ch[1] == y) ? y : x);
            }
            rotate(x);
        }
    }

    void access(int x)
    {
        // x 是内部节点；切换首选儿子并同步虚儿子贡献，无返回值。
        int y = 0, z = x;
        while (z)
        {
            splay(z);
            int p = t[z].fa;
            t[z].vir += t[t[z].ch[1]].sum - t[y].sum;
            t[z].ch[1] = y;
            pull(z);
            y = z;
            z = p;
        }
        splay(x);
    }
```

```
116
117     void makeRootNode(int x)
118     {
119         // x 是内部节点；把它改为所在动态树的根，无返回值。
120         access(x);
121         reverseNode(x);
122     }
123
124     int findRootNode(int x)
125     {
126         // x 是内部节点；返回所在动态树根的内部编号。
127         access(x);
128         while (true)
129         {
130             push(x);
131             if (!t[x].ch[0])
132             {
133                 break;
134             }
135             x = t[x].ch[0];
136         }
137         splay(x);
138         return x;
139     }
140
141   public:
142     SubtreeLCT(const vector<Acc> &a = {}) : t(a.size() + 1)
143     {
144         // a 是各点初始权值；建立 n 个互不连通节点，无返回值。
145         for (int i = 0; i < (int)a.size(); i++)
146         {
147             t[i + 1].val = a[i];
148             t[i + 1].path = a[i];
149             t[i + 1].sum = a[i];
150         }
151     }
152
153     bool link(int u, int v)
154     {
155         // u、v 是外部节点编号；不连通时连边并返回 true，否则返回 false。
156         u++;
157         v++;
158         makeRootNode(u);
159         if (findRootNode(v) == u)
160         {
161             return false;
162         }
163         access(v);
164         t[u].fa = v;
165         t[v].vir += t[u].sum;
166         pull(v);
167         return true;
```

```
    }

    bool cut(int u, int v)
    {
        // u、v 是外部节点编号；边存在时删除并返回 true，否则返回 false。
        u++;
        v++;
        makeRootNode(u);
        access(v);
        if (t[v].ch[0] != u || t[u].ch[1])
        {
            return false;
        }
        t[v].ch[0] = 0;
        t[u].fa = 0;
        pull(v);
        return true;
    }

    bool connected(int u, int v)
    {
        // u、v 是外部节点编号；返回两点当前是否属于同一棵动态树。
        return findRootNode(u + 1) == findRootNode(v + 1);
    }

    void set(int u, Acc v)
    {
        // u 是外部节点编号，v 是新点权；执行单点赋值，无返回值。
        u++;
        access(u);
        t[u].val = v;
        pull(u);
    }

    Acc pathSum(int u, int v)
    {
        // u、v 必须连通；返回两点简单路径上的点权和。
        assert(connected(u, v)); // 调试检查，可删
        u++;
        v++;
        makeRootNode(u);
        access(v);
        return t[v].path;
    }

    Acc subtreeSum(int root, int u)
    {
        // root 是有根树根，u 是子树根；返回该根方向下 u 的子树点权和。
        assert(connected(root, u)); // 调试检查，可删
        root++;
        u++;
        makeRootNode(root);
```

```
        access(u);
        return t[u].val + t[u].vir;
    }
};
```

构造 `SubtreeLCT q(a)`；默认点权、`set` 参数、路径和与子树和均为 `long long`。必须保证所有路径聚合、虚儿子贡献和子树和能装入 `Acc`；可能超过 64 位时使用 `SubtreeLCT<__int128>`，并传入 `vector<__int128>` 点权。`subtreeSum(root, u)` 要求两点连通，先把 `root` 置根再返回 u 的有根子树和。各操作均摊 $O(\log n)$、空间 $O(n)$。子树和依赖 `vir` 的差量维护，故聚合须可减。

下面用六个点检查路径和、换根子树、点权修改与动态连边断边：

```
signed main()
{
    SubtreeLCT q({1, 2, 3, 4, 5, 6});
    assert(q.link(0, 1));
    assert(q.link(1, 2));
    assert(q.link(1, 3));
    assert(q.link(3, 4));
    assert(q.link(3, 5));
    assert(q.pathSum(2, 5) == 15);
    assert(q.subtreeSum(0, 1) == 20);
    assert(q.subtreeSum(2, 1) == 18);
    q.set(3, -4);
    assert(q.subtreeSum(0, 1) == 12);
    assert(q.cut(1, 3));
    assert(!q.connected(0, 5));
    assert(q.subtreeSum(4, 3) == 2);
    assert(q.link(2, 5));
    assert(q.pathSum(0, 4) == 13);
    assert(q.subtreeSum(0, 1) == 12);
    assert(!q.link(0, 4));
}
```

6.4.003 动态割边、割点 LCT

这里给两套桥/割点维护。`OnlineBridge` 只支持加边并实时维护桥数：用“连通块 DSU + 边双 DSU + 生成森林”三件套，新边跨连通块则桥数 $+1$，成环则把环上（两端到 LCA）的桥全部收缩进一个边双、相应减少桥数；`makeRoot` 靠翻父边实现路径反转。`DynamicCut` 是通用基准版，按边编号加删，每次 `analyze` 重跑一遍 Tarjan 求 `low` 值判桥与割点，简单但每次线性。

```
class OnlineBridge
{
    // d2、dc 分别保存边双代表元和连通块代表元。
    vector<int> d2, dc;
    // fa 保存当前生成森林父节点，siz 保存连通块大小。
    vector<int> fa, siz;
    // vis 保存最近一次 mergePath 的访问轮次。
    vector<int> vis;
```

```
    // ti 是访问轮次，cnt 是当前桥数。
    int ti = 0, cnt = 0;

    int find2(int x)
    {
        // x 是节点编号；返回其当前边双连通分量代表元。
        if (x == -1)
        {
            return -1;
        }
        return d2[x] == x ? x : d2[x] = find2(d2[x]);
    }

    int findC(int x)
    {
        // x 是节点编号；返回其当前连通块代表元。
        if (x == -1)
        {
            return -1;
        }
        x = find2(x);
        return dc[x] == x ? x : dc[x] = findC(dc[x]);
    }

    void makeRoot(int x)
    {
        // x 是节点编号；翻转生成森林中 x 到原根的父边，无返回值。
        x = find2(x);
        int root = x, pre = -1;
        while (x != -1)
        {
            int p = find2(fa[x]);
            fa[x] = pre;
            dc[x] = root;
            pre = x;
            x = p;
        }
        siz[root] = siz[pre];
    }

    void mergePath(int a, int b)
    {
        // a、b 已在同一连通块；压缩新环上的全部桥，无返回值。
        ti++;
        vector<int> pa, pb;
        int lca = -1;
        while (lca == -1)
        {
            if (a != -1)
            {
                a = find2(a);
                pa.push_back(a);
```

```
                if (vis[a] == ti)
                {
                    lca = a;
                    break;
                }
                vis[a] = ti;
                a = fa[a];
            }
            swap(a, b);
            swap(pa, pb);
        }
        for (auto x : pa)
        {
            d2[x] = lca;
            if (x == lca)
            {
                break;
            }
            cnt--;
        }
        for (auto x : pb)
        {
            d2[x] = lca;
            if (x == lca)
            {
                break;
            }
            cnt--;
        }
    }

public:
    OnlineBridge(int n) : d2(n), dc(n), fa(n, -1), siz(n, 1), vis(n)
    {
        // n 是固定点数；建立没有边的无向图，无返回值。
        iota(d2.begin(), d2.end(), 0);
        iota(dc.begin(), dc.end(), 0);
    }

    void addEdge(int a, int b)
    {
        // a、b 是无向边两端；加入边并更新当前桥数，无返回值，允许重边和自环。
        a = find2(a);
        b = find2(b);
        if (a == b)
        {
            return;
        }
        int ca = findC(a), cb = findC(b);
        if (ca != cb)
        {
            cnt++;
```

```
            if (siz[ca] > siz[cb])
            {
                swap(a, b);
                swap(ca, cb);
            }
            makeRoot(a);
            fa[a] = b;
            dc[a] = b;
            siz[cb] += siz[a];
        }
        else
        {
            mergePath(a, b);
        }
    }

    int bridges() const
    {
        // 无参数；返回当前无向图中的桥数。
        return cnt;
    }
};

class DynamicCut
{
    // n 保存固定点数。
    int n;
    // e[id][0..1] 依次保存第 id 条边的两个端点，on 标记边当前是否存在。
    vector<array<int, 2>> e;
    vector<char> on;

  public:
    DynamicCut(int n) : n(n)
    {
        // n 是固定点数；建立空无向图，无返回值。
    }

    int addEdge(int u, int v)
    {
        // u、v 是边的两个端点；加入边并返回稳定边编号。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删
        e.push_back({u, v});
        on.push_back(1);
        return e.size() - 1;
    }

    bool eraseEdge(int id)
    {
        // id 是 addEdge 返回的边编号；边存在时删除并返回 true，否则返回 false。
        if (id < 0 || id >= (int)e.size() || !on[id])
        {
            return false;
```

```
        }
        on[id] = 0;
        return true;
    }

    pair<vector<int>, vector<int>> analyze() const
    {
        // 无参数；返回 {全部桥的边编号, 全部割点编号}，两部分都按编号升序排列。
        vector<vector<array<int, 2>>> g(n);
        for (int id = 0; id < (int)e.size(); id++)
        {
            if (!on[id])
            {
                continue;
            }
            auto [u, v] = e[id];
            g[u].push_back({v, id});
            g[v].push_back({u, id});
        }
        vector<int> dfn(n), low(n), cut(n), bridge;
        int ti = 0;
        auto dfs = [&](auto &&self, int u, int pe) -> void
        {
            dfn[u] = low[u] = ++ti;
            int child = 0;
            for (auto [v, id] : g[u])
            {
                if (id == pe)
                {
                    continue;
                }
                if (!dfn[v])
                {
                    child++;
                    self(self, v, id);
                    chmin(low[u], low[v]);
                    if (low[v] > dfn[u])
                    {
                        bridge.push_back(id);
                    }
                    if (pe != -1 && low[v] >= dfn[u])
                    {
                        cut[u] = 1;
                    }
                }
                else
                {
                    chmin(low[u], dfn[v]);
                }
            }
            if (pe == -1)
            {
```

```
                cut[u] = child > 1;
            }
        };
        for (int s = 0; s < n; s++)
        {
            if (dfn[s])
            {
                continue;
            }
            dfs(dfs, s, -1);
        }
        vector<int> point;
        for (int u = 0; u < n; u++)
        {
            if (cut[u])
            {
                point.push_back(u);
            }
        }
        sort(bridge.begin(), bridge.end());
        return {bridge, point};
    }
};
```

只加边且只问桥数用 `OnlineBridge::addEdge/bridges`（m 次加边总 $O((n+m)\log n)$、空间 $O(n)$，允许重边自环）；要删边或要割点列表用 `DynamicCut::addEdge/eraseEdge/analyze`，`analyze` 返回按编号升序的（桥边编号，割点编号），更新 $O(1)$、每次分析 $O(n+m)$。其中 Tarjan 的递归深度最坏为 $O(n)$，深链能否直接运行取决于当前评测机的线程栈容量。频繁删边而只要连通性时另有线段树分治 + 可撤销并查集（见图论章），本条 `DynamicCut` 只作正确性基准。

`DynamicCut` 用固定图覆盖链、环、平行边、自环和删边；`OnlineBridge` 用固定加边顺序覆盖跨连通块、成环、重边和自环：

```
signed main()
{
    {
        DynamicCut q(4);
        int e0 = q.addEdge(0, 1);
        int e1 = q.addEdge(1, 2);
        int e2 = q.addEdge(2, 0);
        int e3 = q.addEdge(2, 3);
        q.addEdge(3, 3);
        assert((q.analyze() == pair{vector<int>{e3}, vector<int>{2}}));
        assert(q.eraseEdge(e2));
        assert((q.analyze() == pair{vector<int>{e0, e1, e3}, vector<int>{1, 2}}));
    }
    {
        DynamicCut q(3);
        int e0 = q.addEdge(0, 1);
        int e1 = q.addEdge(0, 1);
        int e2 = q.addEdge(1, 2);
```

```
        assert((q.analyze() == pair{vector<int>{e2}, vector<int>{1}}));
        assert(q.eraseEdge(e1));
        assert((q.analyze() == pair{vector<int>{e0, e2}, vector<int>{1}}));
    }
    {
        OnlineBridge q(5);
        q.addEdge(0, 1);
        q.addEdge(1, 2);
        q.addEdge(2, 3);
        assert(q.bridges() == 3);
        q.addEdge(0, 2);
        assert(q.bridges() == 1);
        q.addEdge(1, 3);
        assert(q.bridges() == 0);
        q.addEdge(3, 4);
        assert(q.bridges() == 1);
        q.addEdge(3, 4);
        assert(q.bridges() == 0);
        q.addEdge(4, 4);
        assert(q.bridges() == 0);
    }
}
```

6.4.004 全局平衡二叉树

全局平衡二叉树把静态树的每条重链按“轻子树质量”选加权中点建成二叉树，使全局深度 $O(\log n)$，为树 DP 的单点修改提供统一的向上更新链。重链上点 u 的权重取 $siz[u]-siz[son[u]]$（挂在该位置的轻子树总质量），每次取累计质量的中点作根，保证向任一儿子下降时剩余质量至少减半；轻链递归建成子树后用虚父边挂到对应重链点上。

```
class GlobalTree
{
    // n 保存点数，root 保存全局平衡树根。
    int n, root;
    // g 保存原树邻接表。
    vector<vector<int>> g;
    // pa、siz、son 依次保存原树父节点、子树大小和重儿子。
    vector<int> pa, siz, son;
    // ch[u][0..1] 保存全局平衡树左右儿子，fa 保存平衡树父边或轻边父亲。
    vector<array<int, 2>> ch;
    vector<int> fa;

    void dfs(int root)
    {
        // 递归计算原树父亲、子树大小和重儿子。
        auto work = [&](auto &&self, int u) -> void
        {
            siz[u] = 1;
            for (int v : g[u])
            {
```

```
                if (v == pa[u])
                {
                    continue;
                }
                pa[v] = u;
                self(self, v);
                siz[u] += siz[v];
                if (son[u] == -1 || siz[v] > siz[son[u]])
                {
                    son[u] = v;
                }
            }
        };
        work(work, root);
    }

    int buildSeq(const vector<int> &v, const vector<int> &pre, int l, int r)
    {
        // v 是一条重链，pre 是轻子树质量前缀和，l、r 是当前闭区间；返回加权中点根。
        if (l > r)
        {
            return -1;
        }
        int need = (pre[l] + pre[r + 1]) / 2;
        int m = lower_bound(pre.begin() + l + 1, pre.begin() + r + 2, need) - pre.begin() - 1;
        m = clamp(m, l, r);
        int u = v[m];
        ch[u][0] = buildSeq(v, pre, l, m - 1);
        ch[u][1] = buildSeq(v, pre, m + 1, r);
        if (ch[u][0] != -1)
        {
            fa[ch[u][0]] = u;
        }
        if (ch[u][1] != -1)
        {
            fa[ch[u][1]] = u;
        }
        return u;
    }

    int buildChain(int x)
    {
        // x 是一条重链链顶；递归建立轻链并返回该链的平衡树根。
        vector<int> v;
        for (int u = x; u != -1; u = son[u])
        {
            v.push_back(u);
        }
        for (auto u : v)
        {
            for (auto w : g[u])
            {
```

```
                if (w == pa[u] || w == son[u])
                {
                    continue;
                }
                int z = buildChain(w);
                fa[z] = u;
            }
        }
        vector<int> pre(v.size() + 1);
        for (int i = 0; i < (int)v.size(); i++)
        {
            pre[i + 1] = pre[i] + siz[v[i]] - (son[v[i]] == -1 ? 0 : siz[son[v[i]]]);
        }
        return buildSeq(v, pre, 0, v.size() - 1);
    }

  public:
    GlobalTree(const vector<vector<int>> &g, int rt = 0)
        : n(g.size()), root(-1), g(g), pa(n, -1), siz(n), son(n, -1), ch(n, array<int, 2>{-1,
        ↪ -1}), fa(n, -1)
    {
        // g 是静态树，rt 是原树根；建立全局平衡二叉树，无返回值。
        assert(n > 0 && 0 <= rt && rt < n); // 调试检查，可删
        dfs(rt);
        root = buildChain(rt);
    }

    int getRoot() const
    {
        // 无参数；返回全局平衡二叉树根节点编号。
        return root;
    }

    const vector<array<int, 2>> &children() const
    {
        // 无参数；返回每个节点在平衡二叉树中的左右儿子。
        return ch;
    }

    const vector<int> &parents() const
    {
        // 无参数；返回平衡树父边与轻边共同组成的父指针。
        return fa;
    }

    const vector<int> &heavy() const
    {
        // 无参数；返回原树中每个节点的重儿子。
        return son;
    }

    const vector<int> &parTree() const
```

```
    {
        // 无参数；返回原树父节点数组。
        return pa;
    }
};
```

构造 `GlobalTree tr(g, root)`；`children()` 给二叉树实儿子、`parents()` 给“实儿子父边 + 连接轻链的虚父边”的混合父指针，`heavy()`/`parTree()` 给原树重儿子与父亲。构造 $O(n \log n)$ 时间、$O(n)$ 空间，混合父链长 $O(\log n)$。它只是拓扑底座、不含题目信息，动态 DP 用法见 6.4.005 动态 DP。相比用 LCT 做动态 DP，全局平衡树是静态结构、常数更小，但不支持连边断边。

6.4.005 动态 DP

维护固定树、点权可变时的最大权独立集（相邻不同选）。状态“不选/选当前点”，重儿子转移写成 2×2 的 max-plus 矩阵 $\begin{pmatrix} g_0 & g_0 \\ g_1 & -\infty \end{pmatrix}$（$g$ 为并入轻儿子后的节点常量），沿重链是矩阵连乘；轻儿子贡献并进 g。挂在 6.4.004 全局平衡二叉树 上后，一次单点修改只需沿混合父链重算 $O(\log n)$ 个矩阵，跨轻边时先扣除旧贡献、重算后再加回。允许选空集，故全负时答案为 0。

```
// 依赖 6.4.004 全局平衡二叉树
template <class Acc = long long>
class DynamicMWIS
{
    struct Mat
    {
        // nullopt 表示不可达，其余项保存 Acc 类型的 max-plus 值。
        array<array<optional<Acc>, 2>, 2> a{};
    };

    // tr 保存全局平衡二叉树拓扑。
    GlobalTree tr;
    // ch、fa、son、pa 分别引用平衡树儿子、混合父指针、重儿子和原树父节点。
    vector<array<int, 2>> ch;
    vector<int> fa, son, pa;
    // w 是点权，g[u] 是不含重儿子的“不选/选 u”贡献。
    vector<Acc> w;
    vector<array<Acc, 2>> g;
    // s 保存每个平衡树子树合并后的 max-plus 矩阵。
    vector<Mat> s;
    // tree 保存原树邻接表。
    vector<vector<int>> tree;

    static Mat mul(const Mat &x, const Mat &y)
    {
        // x、y 是两个 max-plus 矩阵；返回按顺序合并后的矩阵。
        Mat z;
        for (int i = 0; i < 2; i++)
        {
            for (int j = 0; j < 2; j++)
            {
```

```
                for (int k = 0; k < 2; k++)
                {
                    if (!x.a[i][k] || !y.a[k][j])
                    {
                        continue;
                    }
                    Acc v = *x.a[i][k] + *y.a[k][j];
                    if (!z.a[i][j] || *z.a[i][j] < v)
                    {
                        z.a[i][j] = v;
                    }
                }
            }
        }
        return z;
    }

    bool isChild(int x) const
    {
        // x 是节点编号；返回 x 到 fa[x] 是否为平衡二叉树实儿子边。
        return fa[x] != -1 && (ch[fa[x]][0] == x || ch[fa[x]][1] == x);
    }

    void pull(int x)
    {
        // x 是节点编号；由自身转移和左右儿子重算矩阵，无返回值。
        Mat cur;
        cur.a[0][0] = g[x][0];
        cur.a[0][1] = g[x][0];
        cur.a[1][0] = g[x][1];
        if (ch[x][0] != -1)
        {
            cur = mul(s[ch[x][0]], cur);
        }
        if (ch[x][1] != -1)
        {
            cur = mul(cur, s[ch[x][1]]);
        }
        s[x] = cur;
    }

    void init(int root)
    {
        // 递归求静态 DP，并初始化各点的轻儿子贡献。
        vector<array<Acc, 2>> f(w.size());
        auto dfs = [&](auto &&self, int u) -> void
        {
            f[u] = {0, w[u]};
            for (int v : tree[u])
            {
                if (pa[v] != u)
                {
```

```
                    continue;
                }
                self(self, v);
                f[u][0] += max(f[v][0], f[v][1]);
                f[u][1] += f[v][0];
                if (v != son[u])
                {
                    g[u][0] += max(f[v][0], f[v][1]);
                    g[u][1] += f[v][0];
                }
            }
            g[u][1] += w[u];
        };
        dfs(dfs, root);
    }

    void buildMat()
    {
        // 对平衡二叉树森林作递归后序，建立全部矩阵。
        auto dfs = [&](auto &&self, int u) -> void
        {
            for (int v : ch[u])
            {
                if (v != -1)
                {
                    self(self, v);
                }
            }
            pull(u);
        };
        for (int u = 0; u < (int)w.size(); u++)
        {
            if (!isChild(u))
            {
                dfs(dfs, u);
            }
        }
    }

public:
    DynamicMWIS(const vector<vector<int>> &tree, const vector<Acc> &w, int root = 0)
        : tr(tree, root), ch(tr.children()), fa(tr.parents()), son(tr.heavy()),
        ↪  pa(tr.parTree()), w(w), g(w.size()),
          s(w.size()), tree(tree)
    {
        // tree 是静态树，w 是可修改点权，root 是固定根；初始化动态最大权独立集，无返回值。
        assert(tree.size() == w.size()); // 调试检查，可删
        init(root);
        buildMat();
    }

    void set(int x, Acc v)
```

```
    {
        // x 是节点编号，v 是新点权；更新点权并维护全树最大权独立集，无返回值。
        assert(0 <= x && x < (int)w.size()); // 调试检查，可删
        g[x][1] += v - w[x];
        w[x] = v;
        while (fa[x] != -1)
        {
            if (isChild(x))
            {
                pull(x);
            }
            else
            {
                int p = fa[x];
                Acc old0 = max(*s[x].a[0][0], *s[x].a[1][0]);
                Acc old1 = *s[x].a[0][0];
                g[p][0] -= old0;
                g[p][1] -= old1;
                pull(x);
                g[p][0] += max(*s[x].a[0][0], *s[x].a[1][0]);
                g[p][1] += *s[x].a[0][0];
            }
            x = fa[x];
        }
        pull(x);
    }

    Acc answer() const
    {
        // 无参数；返回当前整棵树最大权独立集的权值，允许选择空集。
        int r = tr.getRoot();
        return max(*s[r].a[0][0], *s[r].a[1][0]);
    }
};
```

构造 `DynamicMWIS q(tree, w, root)`；默认点权、`set` 参数及 `answer()` 返回值均为 `long long`。必须保证所有有限 DP 值、轻儿子贡献和矩阵乘加结果能装入 `Acc`；可能超过 64 位时使用 `DynamicMWIS<__int128>`，并传入 `vector<__int128>` 点权。预处理 $O(n \log n)$、单点修改 $O(\log n)$、查询 $O(1)$、空间 $O(n)$。它要求树结构固定、只改点权。改成其他矩阵化树 DP 时，除了重新定义矩阵乘法，还必须保证轻子树旧贡献可以撤销；一般半环没有减法，不能直接照搬本实现。

下面用一棵六点树检查修改根、内部点、叶子与全负权：

```
signed main()
{
    {
        vector<vector<int>> g(6);
        vector<pair<int, int>> e{{0, 1}, {0, 2}, {1, 3}, {1, 4}, {2, 5}};
        for (auto [u, v] : e)
        {
            g[u].push_back(v);
```

```
            g[v].push_back(u);
        }
        DynamicMWIS q(g, {5, 4, 3, 2, 6, 7});
        assert(q.answer() == 20);
        q.set(0, -10);
        assert(q.answer() == 15);
        q.set(4, -8);
        assert(q.answer() == 11);
        q.set(5, -20);
        assert(q.answer() == 7);
        q.set(1, 100);
        assert(q.answer() == 103);
        q.set(1, -4);
        q.set(2, -3);
        q.set(3, -2);
        assert(q.answer() == 0);
    }
}
```

6.4.006 离线动态图连通块点权和

一般图上边可以加删、点权可以修改、询问某点所在连通块点权和；删边会拆开环，在线的 LCT 无法直接处理，操作全部已知时用线段树分治。把时刻 $0 \dots q-1$ 建成线段树：每条边在它存在的时间区间 $[$加入，删除$)$ 内有效，每次“点 u 加 x”在 $[i, q)$ 内有效，把它们挂到覆盖对应区间的 $O(\log q)$ 个结点上。DFS 线段树时，进入结点就用并查集合并挂在上面的边、把加值加到 u 当前所在块的根上，到叶子 i 回答该时刻的查询，退出结点时撤销本结点的全部修改。

点加值挂在结点上时直接加到当时的块根：更深的结点只会继续合并，块只会变大，所以这个加值始终被包含在 u 所在块的和里。撤销要求并查集不做路径压缩，只按大小合并，每次修改前把被改点的旧状态压栈。

```
vector<int> offlineCSum(const vector<int> &a, const vector<array<int, 4>> &op)
{
    // a 是初始点权；op[i] 的四维依次是类型、u、v、x：类型 0 加边 u-v，1 删边 u-v，
    // 2 给点 u 加 x，3 查询 u 所在连通块的点权和。加边时该边不存在，删边时该边存在。
    // 返回全部类型 3 查询的结果，顺序与操作出现顺序相同。
    int n = a.size(), q = op.size();
    // seg[p] 是时间线段树结点 p 上的事件，每个事件的四维依次是类型（0 合并、1 加值）、u、v、x。
    vector<vector<array<int, 4>>> seg(max(1LL, 4 * q));
    auto add = [&](auto &&self, int p, int l, int r, int ql, int qr, array<int, 4> e) -> void
    {
        // 把事件 e 挂到覆盖时间区间 [ql,qr) 的结点上；p 管辖 [l,r)。
        if (qr <= l || r <= ql)
        {
            return;
        }
        if (ql <= l && r <= qr)
        {
            seg[p].push_back(e);
```

```
            return;
        }
        int m = (l + r) / 2;
        self(self, p * 2, l, m, ql, qr, e);
        self(self, p * 2 + 1, m, r, ql, qr, e);
    };
    map<pair<int, int>, int> born; // born[{u,v}] 是当前存在的边的加入时刻。
    for (int i = 0; i < q; i++)
    {
        auto [type, u, v, x] = op[i];
        if (type == 0)
        {
            born[minmax(u, v)] = i;
        }
        else if (type == 1)
        {
            auto e = minmax(u, v);
            add(add, 1, 0, q, born[e], i, {0, e.first, e.second, 0});
            born.erase(e);
        }
        else if (type == 2)
        {
            // 点加值从此刻起一直有效，相当于在 [i,q) 内给所在连通块加 x。
            add(add, 1, 0, q, i, q, {1, u, 0, x});
        }
    }
    for (auto [e, l] : born)
    {
        add(add, 1, 0, q, l, q, {0, e.first, e.second, 0});
    }
    // fa 是不压缩路径的按大小合并并查集，根的 fa 为 -大小；sum[r] 是以 r 为根的连通块点权和。
    vector<int> fa(n, -1), sum = a, ans;
    // his 是修改记录，三维依次是被改的点、旧 fa、旧 sum。
    vector<array<int, 3>> his;
    auto root = [&](int u)
    {
        // 返回 u 所在连通块的根。
        while (fa[u] >= 0)
        {
            u = fa[u];
        }
        return u;
    };
    auto save = [&](int u)
    {
        // 记录点 u 修改前的状态。
        his.push_back({u, fa[u], sum[u]});
    };
    auto rollback = [&](int at)
    {
        // 撤销到 his 只有 at 条记录时的状态。
        while ((int)his.size() > at)
```

```
        {
            auto [u, f, s] = his.back();
            his.pop_back();
            fa[u] = f;
            sum[u] = s;
        }
    };
    auto solve = [&](auto &&self, int p, int l, int r) -> void
    {
        // 进入结点 p（管辖时刻 [l,r)）时应用其事件，到叶子回答查询，退出时回滚。
        int at = his.size();
        for (auto [type, u, v, x] : seg[p])
        {
            u = root(u);
            if (type == 1)
            {
                save(u);
                sum[u] += x;
                continue;
            }
            v = root(v);
            if (u == v)
            {
                continue;
            }
            if (fa[u] > fa[v])
            {
                swap(u, v);
            }
            save(u);
            save(v);
            fa[u] += fa[v];
            fa[v] = u;
            sum[u] += sum[v];
        }
        if (r - l == 1)
        {
            if (op[l][0] == 3)
            {
                ans.push_back(sum[root(op[l][1])]);
            }
        }
        else
        {
            int m = (l + r) / 2;
            self(self, p * 2, l, m);
            self(self, p * 2 + 1, m, r);
        }
        rollback(at);
    };
    if (q)
    {
```

```
        solve(solve, 1, 0, q);
    }
    return ans;
}
```

`offlineCSum(a,op)` 中 `op[i]` 的四维依次是类型、u、v、x：类型 0 加边 u-v（此时该边不存在），1 删边 u-v（此时该边存在），2 给点 u 加 x，3 查询 u 所在块的点权和；未用到的维写 0。返回全部查询结果，顺序与操作出现顺序相同。允许自环（合并时自动忽略），不允许同一时刻存在两条相同的边。设操作数为 q，每个事件挂到 $O(\log q)$ 个结点，按大小合并使 `root` 为 $O(\log n)$，总时间 $O(q \log q \log n)$，空间 $O(q \log q + n)$。

只有加边没有删边时直接用普通并查集；在森林上在线维护路径或子树信息时分别用 6.4.001 Link-Cut Tree 与 6.4.002 带子树信息的 LCT，在线维护森林的连通块整体信息用 6.4.007 Euler Tour Tree。下面的操作序列含加边、点加、删边与查询：

```
signed main()
{
    vector<int> a{1, 2, 3};
    vector<array<int, 4>> op{{3, 0, 0, 0}, {0, 0, 1, 0}, {3, 1, 0, 0}, {2, 0, 0, 5}, {3, 1, 0,
    ↪ 0},
                             {0, 1, 2, 0}, {3, 2, 0, 0}, {1, 0, 1, 0}, {3, 0, 0, 0}};
    assert(offlineCSum(a, op) == vector<int>({1, 3, 8, 11, 6}));
}
```

6.4.007 Euler Tour Tree

Euler Tour Tree 维护一片动态森林：每棵树存成一条欧拉序，换根就是把序列循环移位，连边与删边都只是序列的拼接与切分。序列用带父指针的隐式 treap 维护：

- `reroot(u)`：把 u 所在序列从 u 的点结点处切开、前后对调，u 成为序列开头；
- `link(u,v)`：分别换根后按 $E_u, (u \to v), E_v, (v \to u)$ 拼接；
- `cut(u,v)`：序列形如 $A\,(u \to v)\,B\,(v \to u)\,C$（两个边结点也可能顺序相反），$B$ 是被切下的子树，A 与 C 拼回剩下的树。

点结点带权、边结点权恒为 0，结点维护子树点数、点权和与整棵子树加的懒标记，因此可以 $O(\log n)$ 查询或修改整个连通块。父指针让任一结点能找到所在序列的根和自己的下标。

```
struct EulerTour
{
    // 每棵树存成一条欧拉序，序列用带父指针的隐式 treap 维护；节点 0 为空，1..n 是点节点，
    // 之后每条树边对应两个边节点 (u->v 与 v->u)。点节点带权，边节点权恒为 0。
    // l、r、p 是左右儿子与父亲；sz 是子树节点数，cnt 是子树内点节点数。
    vector<int> l, r, p, sz, cnt;
    vector<char> own;                  // own[x] 表示 x 是点节点。
    vector<unsigned long long> pri;    // pri 是 treap 随机优先级。
    vector<__int128> val, sum, lz;     // val 是点权，sum 是子树权和，lz 是整棵子树待加的值。
    map<pair<int, int>, pair<int, int>> edge; // edge[{u,v}] (u<v) 是该边的两个边节点。
    mt19937_64 rng{712367};

```

```
    EulerTour(int n)
    {
        // n 是点数，点编号为 [0,n)；初始为 n 个孤立点，点权为 0。
        make(false);
        sz[0] = 0; // 节点 0 是空节点，大小为 0。
        for (int i = 0; i < n; i++)
        {
            make(true);
        }
    }

    int make(bool v)
    {
        // 新建一个节点，v 表示是否为点节点；返回节点编号。
        for (auto *a : {&l, &r, &p})
        {
            a->push_back(0);
        }
        sz.push_back(1);
        cnt.push_back(v);
        own.push_back(v);
        pri.push_back(rng());
        val.push_back(0);
        sum.push_back(0);
        lz.push_back(0);
        return l.size() - 1;
    }

    void apply(int x, __int128 d)
    {
        // 给 x 的子树内所有点节点加 d。
        if (x)
        {
            val[x] += own[x] * d;
            sum[x] += d * cnt[x];
            lz[x] += d;
        }
    }

    void push(int x)
    {
        // 把 x 的懒标记下传给儿子。
        if (lz[x])
        {
            apply(l[x], lz[x]);
            apply(r[x], lz[x]);
            lz[x] = 0;
        }
    }

    void pull(int x)
    {
```

```
        // 由儿子重算 x 的子树信息，并修正儿子的父指针。
        sz[x] = 1 + sz[l[x]] + sz[r[x]];
        cnt[x] = own[x] + cnt[l[x]] + cnt[r[x]];
        sum[x] = val[x] + sum[l[x]] + sum[r[x]];
        p[l[x]] = p[r[x]] = x;
        p[0] = 0;
    }

    int merge(int a, int b)
    {
        // 拼接序列 a、b，返回新根（其父指针为 0）。
        if (!a || !b)
        {
            p[a | b] = 0;
            return a | b;
        }
        int x;
        if (pri[a] > pri[b])
        {
            push(a);
            r[a] = merge(r[a], b);
            x = a;
        }
        else
        {
            push(b);
            l[b] = merge(a, l[b]);
            x = b;
        }
        pull(x);
        p[x] = 0;
        return x;
    }

    pair<int, int> split(int x, int k)
    {
        // 把序列 x 拆成前 k 个节点和其余部分，两个根的父指针都为 0。
        if (!x)
        {
            return {0, 0};
        }
        push(x);
        pair<int, int> res;
        if (sz[l[x]] >= k)
        {
            auto [a, b] = split(l[x], k);
            l[x] = b;
            res = {a, x};
        }
        else
        {
            auto [a, b] = split(r[x], k - sz[l[x]] - 1);
```

```
            r[x] = a;
            res = {x, b};
        }
        pull(x);
        p[res.first] = p[res.second] = 0;
        return res;
    }

    int root(int x) const
    {
        // 返回 x 所在序列的根。
        while (p[x])
        {
            x = p[x];
        }
        return x;
    }

    int index(int x) const
    {
        // 返回 x 在所在序列中的下标（从 0 开始）。
        int i = sz[l[x]];
        for (; p[x]; x = p[x])
        {
            if (x == r[p[x]])
            {
                i += sz[l[p[x]]] + 1;
            }
        }
        return i;
    }

    int reroot(int u)
    {
        // 把点 u 所在树的欧拉序旋转为以 u 开头，返回序列根。
        int x = u + 1;
        auto [a, b] = split(root(x), index(x));
        return merge(b, a);
    }

    bool connected(int u, int v) const
    {
        // 返回 u、v 是否在同一棵树中。
        return root(u + 1) == root(v + 1);
    }

    bool link(int u, int v)
    {
        // 连边 u-v；已连通时不操作并返回 false。
        if (connected(u, v))
        {
            return false;
```

```
        }
        int a = reroot(u), b = reroot(v), uv = make(false), vu = make(false);
        edge[minmax(u, v)] = {uv, vu};
        merge(merge(merge(a, uv), b), vu);
        return true;
    }

    bool cut(int u, int v)
    {
        // 删除树边 u-v；边不存在时返回 false。
        auto it = edge.find(minmax(u, v));
        if (it == edge.end())
        {
            return false;
        }
        auto [x, y] = it->second;
        int i = index(x), j = index(y);
        if (i > j)
        {
            swap(i, j);
        }
        // 序列形如 A [e1] B [e2] C：B 是被切下的子树，A+C 是剩下的树。
        auto [a, rest] = split(root(x), i);
        auto [e1, tail] = split(rest, 1);
        auto [b, last] = split(tail, j - i - 1);
        auto [e2, c] = split(last, 1);
        merge(a, c);
        edge.erase(it);
        return true;
    }

    __int128 compSum(int u) const
    {
        // 返回 u 所在树的点权和。
        return sum[root(u + 1)];
    }

    int compSize(int u) const
    {
        // 返回 u 所在树的点数。
        return cnt[root(u + 1)];
    }

    void compAdd(int u, int d)
    {
        // 给 u 所在树的所有点加 d。
        apply(root(u + 1), d);
    }

    void add(int u, int d)
    {
        // 给点 u 加 d：先自顶向下下传标记，再自底向上重算。
```

```
        int x = u + 1;
        vector<int> path;
        for (int y = x; y; y = p[y])
        {
            path.push_back(y);
        }
        for (int i = path.size() - 1; i >= 0; i--)
        {
            push(path[i]);
        }
        val[x] += d;
        for (int y : path)
        {
            pull(y);
        }
    }
};
```

`EulerTour t(n)` 初始为 n 个点权为 0 的孤立点。`link(u,v)` 在已连通时返回 `false`，`cut(u,v)` 在边不存在时返回 `false`；`connected`、`compSize`、`compSum` 查询连通性、块大小与块点权和，`compAdd(u,d)` 给 u 所在块全体加 d，`add(u,d)` 只给点 u 加。每次操作期望 $O(\log n)$，空间 $O(n+m)$，m 为历史连过的边数（删边不回收结点）；`edge` 用 `map` 按无序点对查边，所以不支持重边。

它只维护连通块整体信息，不支持路径聚合；需要路径信息时用 6.4.001 Link-Cut Tree，同时需要子树信息时用 6.4.002 带子树信息的 LCT。操作离线已知时，6.4.006 离线动态图连通块点权和 更简单。下面的操作序列核对整块加、删边后重新连边：

```
signed main()
{
    EulerTour t(5);
    for (int i = 0; i < 5; i++)
    {
        t.add(i, i + 1); // 点 i 的初始权值为 i+1。
    }
    t.link(0, 1), t.link(1, 2), t.link(3, 4);
    t.compAdd(1, 10); // 0、1、2 各加 10。
    assert(t.compSum(2) == 36 && t.compSize(0) == 3);
    t.cut(0, 1);
    t.link(2, 3);
    assert(t.compSum(4) == 34 && !t.connected(0, 4) && !t.link(1, 4));
}
```

6.5 树形 DP

6.5.001 树上背包

树上背包基础版：$f_u[j]$ 为 u 子树中恰选 j 个点的最大权值，合并一棵儿子子树时对左右所选个数做 max-plus 卷积，每点可不选也可选，所选点之间无连通或依赖约束。数组长度只开到当前有效上界（各边两侧有效状态数之积求和才是真实复杂度，这正是经典“子树背包上界为 $O(nk)$”的来源）。

```cpp
template <class Acc = long long>
vector<optional<Acc>> treePack(const vector<vector<int>> &g, const vector<Acc> &val, int root,
    int k)
{
    // 返回 f[j]= 全树恰选 j 点的最大权值；nullopt 表示不可达。
    int n = g.size();
    assert(n > 0 && (int)val.size() == n && 0 <= root && root < n && k >= 0); // 调试检查，可删
    auto dfs = [&](auto &&self, int u, int p) -> vector<optional<Acc>>
    {
        vector<optional<Acc>> f(min<int>(1, k) + 1);
        f[0] = 0;
        if (k)
        {
            f[1] = val[u];
        }
        for (int v : g[u])
        {
            if (v == p)
            {
                continue;
            }
            auto h = self(self, v, u);
            vector<optional<Acc>> nf(min<int>(k, (int)f.size() + (int)h.size() - 2) + 1);
            for (int i = 0; i < (int)f.size(); i++)
            {
                for (int j = 0; j < (int)h.size() && i + j <= k; j++)
                {
                    if (f[i] && h[j])
                    {
                        Acc v = *f[i] + *h[j];
                        if (!nf[i + j] || *nf[i + j] < v)
                        {
                            nf[i + j] = v;
                        }
                    }
                }
            }
            f.swap(nf);
        }
        return f;
    };
    return dfs(dfs, root, -1);
}
```

`treePack(g, val, root, k)` 要求非空树且 $k \geq 0$，默认返回 `optional<long long>` 数组，`nullopt` 表示该选择数不可达；必须保证可达状态的价值累加能装入 `Acc`。可能超过 64 位时使用 `treePack<__int128>`，并传入 `vector<__int128>` 点权。$O(nk^2)$ 保守时间、$O(nk)$ 空间；链形树的递归深度为 $O(n)$，实际可承受深度取决于评测环境的线程栈。例如链 `0-1-2`、点权 `{5, -2, 4}`、$k = 3$ 时四个状态依次为 `0 5 9 7`。有父子依赖时改用 6.5.004 树上依赖背包。

6.5.002 单调队列优化树上背包

每点挂一类“有体积、价值、件数上限”的物品，用按体积余数分组的单调队列把本点多重背包的枚举件数一维消掉，再与儿子卷积。固定余数 r 后，容量 $r, r+w, r+2w, \dots$ 上的转移是长度 $cnt+1$ 的滑动窗口最大值：把 $f[r+jw]-jv$ 入队即可线性处理一类物品。该优化只作用于本点物品，儿子之间仍是树上背包卷积。

```cpp
template <class Acc = long long>
vector<optional<Acc>> treePack(const vector<vector<int>> &g,
                               const vector<array<long long, 3>> &a,
                               int root,
                               int cap)
{
    // a[u] 是 {体积, 价值, 件数}；返回恰用各容量的最大价值，nullopt 表示不可达。
    int n = g.size();
    assert(n > 0 && (int)a.size() == n && 0 <= root && root < n && cap >= 0); // 调试检查，可删
    auto dfs = [&](auto &&self, int u, int p) -> vector<optional<Acc>>
    {
        int w = a[u][0];
        Acc v = (Acc)a[u][1];
        int cnt = a[u][2];
        assert(w > 0 && cnt >= 0); // 调试检查，可删
        vector<optional<Acc>> pre(cap + 1), f(cap + 1);
        pre[0] = (Acc)0;
        for (int r = 0; r < w && r <= cap; r++)
        {
            deque<int> q;
            for (int j = 0, x = r; x <= cap; j++, x += w)
            {
                while (!q.empty() && q.front() < j - cnt)
                {
                    q.pop_front();
                }
                if (pre[x])
                {
                    Acc cur = *pre[x] - (Acc)j * v;
                    while (!q.empty())
                    {
                        int y = r + q.back() * w;
                        Acc old = *pre[y] - (Acc)q.back() * v;
                        if (old > cur)
                        {
                            break;
                        }
                        q.pop_back();
                    }
                    q.push_back(j);
                }
                if (!q.empty())
                {
                    int y = r + q.front() * w;
```

```
                    f[x] = *pre[y] + (Acc)(j - q.front()) * v;
                }
            }
        }
        for (int x : g[u])
        {
            if (x == p)
            {
                continue;
            }
            auto h = self(self, x, u);
            auto nf = f;
            for (int i = 0; i <= cap; i++)
            {
                for (int j = 1; i + j <= cap; j++)
                {
                    if (f[i] && h[j])
                    {
                        Acc cur = *f[i] + *h[j];
                        if (!nf[i + j] || *nf[i + j] < cur)
                        {
                            nf[i + j] = cur;
                        }
                    }
                }
            }
            f.swap(nf);
        }
        return f;
    };
    return dfs(dfs, root, -1);
}
```

`treePack(g, a, root, cap)` 要求非空树、$cap \geq 0$、体积为正且件数非负，各点物品独立。默认价值乘法、队列比较、卷积和状态表均使用 `long long`，必须保证这些中间值可表示；风险题显式调用 `treePack<__int128>`。不可达状态为 `nullopt`。$O(nC^2)$ 保守时间、$O(nC)$ 空间；链形树的递归深度为 $O(n)$，实际可承受深度取决于评测环境的线程栈。例如单点、物品 `{2,3,3}`、容量 7 时容量 `0,2,4,6` 的值为 `0,3,6,9`，其余不可达。

6.5.003 长链剖分 DP 合并

这是长链剖分 DP 合并的样例——求每个子树中出现次数最多的相对深度（并列取较小深度）。以深度为下标的 DP 让长儿子的数组与父节点错位一位直接继承（`at[h] = at[u]+1` 共享同一段内存 `f`），其余轻儿子只在其有效深度范围内逐项合并；分别核算每个轻子树的扫描长度，其总和是 $O(n)$ 而非平方。指针 `ptr` 线性分配各长链的内存段。

```
vector<int> depthMode(const vector<vector<int>> &g, int root = 0)
{
    // 返回每个子树中出现次数最多的相对深度，并列取较小者。
```

```
    int n = g.size();
    if (!n)
    {
        return {};
    }
    assert(0 <= root && root < n); // 调试检查，可删
    vector<int> len(n, 1), son(n, -1), ans(n), at(n), f(n + 1);
    auto dfs1 = [&](auto &&self, int u, int p) -> void
    {
        for (int v : g[u])
        {
            if (v == p)
            {
                continue;
            }
            self(self, v, u);
            if (len[v] + 1 > len[u])
            {
                len[u] = len[v] + 1;
                son[u] = v;
            }
        }
    };
    dfs1(dfs1, root, -1);
    int ptr = len[root];
    auto dfs2 = [&](auto &&self, int u, int p) -> void
    {
        int h = son[u];
        if (h != -1)
        {
            at[h] = at[u] + 1;
            self(self, h, u);
            ans[u] = ans[h] + 1;
        }
        f[at[u]] = 1;
        if (f[at[u] + ans[u]] <= 1)
        {
            ans[u] = 0;
        }
        for (int v : g[u])
        {
            if (v == p || v == h)
            {
                continue;
            }
            at[v] = ptr;
            ptr += len[v];
            self(self, v, u);
            for (int d = 0; d < len[v]; d++)
            {
                f[at[u] + d + 1] += f[at[v] + d];
                int x = f[at[u] + d + 1], y = f[at[u] + ans[u]];
```

```
                if (x > y || (x == y && d + 1 < ans[u]))
                {
                    ans[u] = d + 1;
                }
            }
        }
    };
    dfs2(dfs2, root, -1);
    assert(ptr == n); // 调试检查，可删
    return ans;
}
```

`depthMode(g, root)` 返回 `ans[u]` 为 u 子树中众数相对深度（并列取较小）。两遍 DFS 加长链内存复用，总 $O(n)$ 时间与空间——这是长链剖分把“深度平移”型 DP 从 $O(n^2)$ 降到线性的典型。前置的长链划分思想见 6.2.005 长链剖分；非深度平移型的子树合并回到 6.5.001 树上背包。

下面用分叉树、链、非零根与空树检查固定结果：

```
signed main()
{
    auto makeTree = [](int n, initializer_list<pair<int, int>> e)
    {
        vector<vector<int>> g(n);
        for (auto [u, v] : e)
        {
            g[u].push_back(v);
            g[v].push_back(u);
        }
        return g;
    };
    auto g = makeTree(8, {{0, 1}, {0, 2}, {1, 3}, {1, 4}, {2, 5}, {5, 6}, {5, 7}});
    assert((depthMode(g) == vector<int>{2, 1, 2, 0, 0, 1, 0, 0}));
    g = makeTree(4, {{0, 1}, {1, 2}, {2, 3}});
    assert((depthMode(g) == vector<int>{0, 0, 0, 0}));
    g = makeTree(4, {{0, 1}, {0, 2}, {0, 3}});
    assert((depthMode(g, 1) == vector<int>{1, 2, 0, 0}));
    assert(depthMode({}).empty());
}
```

6.5.004 树上依赖背包

树上依赖背包：每件物品有父子依赖，选任一非根物品前必须先选其父物品；求容量上限内的最大价值。$f_u[c]$ 为满足 u 子树依赖且恰用容量 c 的最大价值；关键约束是“非虚根节点在 $c=0$ 时禁止合并非空儿子状态”——没选父亲就不能选后代，正由此表达。用一个虚根 `rt` 把依赖森林统一成一棵树。

```
template <class Acc = long long>
vector<optional<Acc>> treeDepPack(const vector<int> &w,
                                  const vector<Acc> &val,
                                  const vector<int> &par,
                                  int cap)
```

```
{
    // 返回恰用容量 c 且满足父子依赖的最大价值；nullopt 表示不可达。
    int n = w.size(), rt = n;
    assert((int)val.size() == n && (int)par.size() == n && cap >= 0); // 调试检查，可删
    vector<vector<int>> son(n + 1);
    for (int u = 0; u < n; u++)
    {
        assert(w[u] > 0 && (-1 <= par[u] && par[u] < n)); // 调试检查，可删
        son[par[u] < 0 ? rt : par[u]].push_back(u);
    }
    int seen = 0;
    auto dfs = [&](auto &&self, int u) -> vector<optional<Acc>>
    {
        seen++;
        vector<optional<Acc>> f(cap + 1);
        if (u == rt)
        {
            f[0] = (Acc)0;
        }
        else if (w[u] <= cap)
        {
            f[w[u]] = val[u];
        }
        for (int v : son[u])
        {
            auto h = self(self, v);
            auto nf = f;
            for (int i = 0; i <= cap; i++)
            {
                if (!f[i] || (u != rt && i == 0))
                {
                    continue;
                }
                for (int j = 1; i + j <= cap; j++)
                {
                    if (h[j])
                    {
                        Acc cur = *f[i] + *h[j];
                        if (!nf[i + j] || *nf[i + j] < cur)
                        {
                            nf[i + j] = cur;
                        }
                    }
                }
            }
            f.swap(nf);
        }
        return f;
    };
    auto ans = dfs(dfs, rt);
    assert(seen == n + 1); // 调试检查，可删：par 必须构成森林
    return ans;
```

```
}
```

`treeDepPack(w, val, par, cap)` 要求 $cap \ge 0$ 且 `par` 构成森林，根记 -1；默认返回 `optional<long long>` 数组，必须保证所有价值和能装入 `Acc`。需要宽状态时调用 `treeDepPack<__int128>` 并传入 `vector<__int128>` 价值。$O(nC^2)$ 保守时间、$O(nC)$ 空间；链形依赖的递归深度为 $O(n)$，实际可承受深度取决于评测环境的线程栈。例如体积 `{2,2,3}`、价值 `{3,5,8}`、父亲 `{-1,0,0}`、容量 5 时最大值为 `11`（选点 0 与 2）。

6.5.005 有根树最小逆序拓扑序

根必须先于所有后代；对合法拓扑序 p，最小化 $\sum_i \sum_{j<i} c_{p_i} d_{p_j}$。把一段连续块记作 $(D, C) = (\sum d, \sum c)$，块 X 接在 Y 前产生的跨块代价是 $D_X C_Y$，交换论证给出最优顺序按 C/D 非增排列。树约束下，每次取当前比率最大的非根块并紧接到其父块后，再把两块收缩；DSU 找当前父块，循环链表用一次 `swap` 完成拼接。块和使用 `__int128` 累加，避免多个 `long long` 点值相加时溢出；交叉乘积和题目要求的答案也必须能由 `__int128` 表示。

```
vector<int> minInvOrder(const vector<int> &par, vector<long long> c,
                                vector<long long> d, int root = 0)
{
    // par 是父亲数组（根为 -1），c、d 是非负点值，root 是根；
    // 返回祖先先于后代、使 sum_i sum_{j<i} c[p_i] d[p_j] 最小的一个排列。
    int n = par.size();
    assert((int)c.size() == n && (int)d.size() == n && par[root] == -1); // 调试检查，可删。

    struct DSU
    {
        vector<int> p, id; // p 是并查集父亲（根为负的集合大小），id[r] 是集合 r 当前对应的块代表。

        DSU(int n) : p(n, -1), id(n)
        {
            // n 是点数；每个点自成一个集合，代表为自身。
            iota(id.begin(), id.end(), 0);
        }

        int find(int x)
        {
            // x 是点；返回所在集合的根，带路径压缩。
            return p[x] < 0 ? x : p[x] = find(p[x]);
        }

        int operator[](int x)
        {
            // x 是点；返回所在集合当前对应的块代表。
            return id[find(x)];
        }

        void merge(int x, int y, int z)
        {
            // x、y 是两点；合并其集合并把代表设为 z。
```

```
            x = find(x);
            y = find(y);
            if (x == y)
            {
                id[x] = z;
                return;
            }
            if (-p[x] < -p[y])
            {
                swap(x, y);
            }
            p[x] += p[y];
            p[y] = x;
            id[x] = z;
        }
    };

    struct Block
    {
        __int128 d, c; // d、c 是块内 d 值之和与 c 值之和。
        int root, ver; // root 是块代表点，ver 是入堆时的版本号，过期版本出堆时跳过。

        static bool ratioLess(const Block &x, const Block &y)
        {
            // x、y 是块；返回 c/d 比值是否 x<y，全零块视为最小。
            if (y.c == 0 && y.d == 0)
            {
                return false;
            }
            if (x.c == 0 && x.d == 0)
            {
                return true;
            }
            return (__int128)x.c * y.d < (__int128)x.d * y.c;
        }

        bool operator<(const Block &o) const
        {
            // o 是另一块；先比 c/d 比值，再比 (root,ver)，使大根堆先取比值最大的块。
            if (ratioLess(*this, o))
            {
                return true;
            }
            if (ratioLess(o, *this))
            {
                return false;
            }
            return pair(root, ver) < pair(o.root, o.ver);
        }
    };

    DSU dsu(n);
```

```
    priority_queue<Block> q;
    vector<int> nxt(n), ver(n);
    vector<__int128> sumC(n), sumD(n);
    iota(nxt.begin(), nxt.end(), 0);
    for (int u = 0; u < n; u++)
    {
        sumC[u] = c[u];
        sumD[u] = d[u];
        if (u != root)
        {
            assert(0 <= par[u] && par[u] < n); // 调试检查，可删。
            q.push({sumD[u], sumC[u], u, 0});
        }
    }
    while (!q.empty())
    {
        auto x = q.top();
        q.pop();
        int u = x.root;
        if (ver[u] != x.ver)
        {
            continue;
        }
        int v = dsu[par[u]];
        sumD[v] += sumD[u];
        sumC[v] += sumC[u];
        dsu.merge(u, v, v);
        if (v != root)
        {
            q.push({sumD[v], sumC[v], v, ++ver[v]});
        }
        swap(nxt[u], nxt[v]);
    }

    vector<int> ans{nxt[root]};
    while (ans.back() != root)
    {
        ans.push_back(nxt[ans.back()]);
    }
    reverse(ans.begin(), ans.end());
    return ans;
}
```

`minInvOrder(par,c,d,root)` 要求 `par[root]=-1`、其余父亲构成一棵以 `root` 为根的树，且 $c_i, d_i \geq 0$；返回一个最优排列，最小值按题意扫描排列即可。块和、任意两个块的交叉乘积以及题目要求的代价都必须能由 `__int128` 表示；输入点值仍可使用 `long long`，函数会在合并前提升为宽类型。堆中每个有效块只合并一次，时间 $O(n \log n)$、空间 $O(n)$。

下面用一棵三点树检查祖先必须在前、以及两个兄弟的最优交换顺序：

```cpp
__int128 orderValue(const vector<int> &p, const vector<long long> &c,
                    const vector<long long> &d)
{
    // p 是排列，c、d 是点值；返回 sum_i sum_{j<i} c[p_i] d[p_j]。
    __int128 ans = 0, sum = 0;
    for (int u : p)
    {
        ans += sum * c[u];
        sum += d[u];
    }
    return ans;
}

signed main()
{
    vector<int> par{-1, 0, 0};
    vector<long long> c{0, 4, 1}, d{1, 1, 5};
    auto got = minInvOrder(par, c, d);
    assert(got == vector<int>({0, 1, 2}));
    assert(orderValue(got, c, d) == 6);
}
```

6.5.006 换根 DP / Rerooting

固定根树形 DP 只得到一个根的答案；若每个点都要作为根，逐根重算会退化到 $O(n^2)$。设 `merge` 是带单位元 `identity` 的结合运算，`base[u]` 是点 u 自身贡献，`lift(value,from,to,edgeId)` 把 `from` 一侧的聚合跨边转成对 `to` 的贡献。第一遍 DFS 自底向上求 `down[u]`；第二遍对每个点按邻接表顺序建立贡献的前缀积与后缀积，给儿子时合并除该儿子外的全部方向，再跨父子边传下去。这样每条有向边只转移常数次，不要求 `merge` 可逆；若 `merge` 不可交换，结果按给定邻接表顺序定义。

```cpp
// RerootEdge[0] 是邻点，RerootEdge[1] 是无向边编号。
using RerootEdge = array<int, 2>;

template <class T, class Merge, class Lift>
vector<T> rerooting(
    const vector<vector<RerootEdge>> &g, const vector<T> &base, const T &identity, Merge
    ↪ merge, Lift lift, int root = 0)
{
    // g 是带边号无向树，base 是各点自身贡献，identity 是 merge 单位元；merge 合并贡献，lift 令贡献跨
    ↪ 边。
    int n = g.size();
    assert(n > 0 && (int)base.size() == n && 0 <= root && root < n); // 调试检查，可删
    vector<T> down(n, identity), fromPar(n, identity), ans(n, identity);

    auto dfsDown = [&](auto &&self, int u, int p) -> void
    {
        down[u] = base[u];
        for (auto [v, edgeId] : g[u])
        {
```

```
            if (v == p)
            {
                continue;
            }
            self(self, v, u);
            down[u] = merge(down[u], lift(down[v], v, u, edgeId));
        }
    };
    dfsDown(dfsDown, root, -1);

    auto dfsUp = [&](auto &&self, int u, int p) -> void
    {
        int degree = g[u].size();
        vector<T> part(degree, identity), prefix(degree + 1, identity), suffix(degree + 1,
↪   identity);
        for (int i = 0; i < degree; i++)
        {
            auto [v, edgeId] = g[u][i];
            part[i] = v == p ? fromPar[u] : lift(down[v], v, u, edgeId);
            prefix[i + 1] = merge(prefix[i], part[i]);
        }
        for (int i = degree - 1; i >= 0; i--)
        {
            suffix[i] = merge(part[i], suffix[i + 1]);
        }
        ans[u] = merge(base[u], prefix[degree]);
        for (int i = 0; i < degree; i++)
        {
            auto [v, edgeId] = g[u][i];
            if (v == p)
            {
                continue;
            }
            T withoutV = merge(base[u], merge(prefix[i], suffix[i + 1]));
            fromPar[v] = lift(withoutV, u, v, edgeId);
            self(self, v, u);
        }
    };
    dfsUp(dfsUp, root, -1);
    return ans;
}
```

`g[u]` 的每项为 `{v,edgeId}`，每条无向边须在两端各出现一次且使用同一边号；`base` 与 `g` 等长。`merge` 必须满足结合律并以 `identity` 为双侧单位元，`lift` 的返回类型为 `T`，可按方向与边号读取外部边权。设一次回调为 $O(1)$，总时间与额外空间均为 $O(n)$，递归调用栈最坏 $O(n)$。只需要固定根子树状态时普通树形 DP 更短；动态修改点权后反复询问全树最优值属于 6.4.005 动态 DP，不是本条的静态全根问题。

```
signed main()
{
    vector<vector<RerootEdge>> g(4);
    vector<int> w{2, 3, 1};
```

```
    auto add = [&](int u, int v, int id)
    {
        g[u].push_back({v, id});
        g[v].push_back({u, id});
    };
    add(0, 1, 0);
    add(1, 2, 1);
    add(1, 3, 2);
    // State[0] 是点数，State[1] 是到当前根的距离和。
    using State = array<int, 2>;
    auto merge = [](State a, State b)
    {
        return State{a[0] + b[0], a[1] + b[1]};
    };
    auto lift = [&](State a, int from, int to, int id)
    {
        (void)from;
        (void)to;
        return State{a[0], a[1] + a[0] * w[id]};
    };
    auto ans = rerooting(g, vector<State>(4, {1, 0}), State{0, 0}, merge, lift);
    assert((vector<int>{ans[0][1], ans[1][1], ans[2][1], ans[3][1]}
            == vector<int>{10, 6, 12, 8}));
}
```

7. 数论

7.1 整数运算与模运算

7.1.001 快速幂

指数二进制分解：底数每轮平方，遇到二进制置位就把它乘进答案，$O(\log(b+1))$ 次乘法。只用到乘法结合律和单位元，所以矩阵快速幂、幺半群快速幂都是同一套框架，换掉乘法即可。

`mulMod` 在模数不超过 `3037000499` 时直接用 64 位乘法，接近 2^{63} 的模数才对这一条乘法局部使用 `__int128`，因此任意 $m < 2^{63}$ 都不溢出；`powerMod` 的 `r` 初值取 $1 \bmod m$ 是为兼容 $m = 1$（此时答案恒为 0）。`mulMod` 是本章各处模乘的公共底座，防溢出细节见 7.1.011 Long Long 防溢出乘法。

```
long long mulMod(long long a, long long b, long long m)
{
    // a、b 是乘数，m 是正模数；返回 a*b mod m 的最小非负余数。
    assert(m > 0); // 调试检查，可删
    a %= m;
    b %= m;
    if (a < 0)
    {
        a += m;
    }
    if (b < 0)
    {
        b += m;
    }
    constexpr long long limit = 3037000499LL;
    if (m <= limit)
    {
        return a * b % m;
    }
    return (__int128)a * b % m;
}

long long powerMod(long long a, unsigned long long b, long long m)
{
    // a 是可为负的底数，b 是非负指数，m 是正模数；返回 a^b mod m。
    assert(m > 0); // 调试检查，可删
    long long r = 1 % m;
    a %= m;
    if (a < 0)
    {
        a += m;
    }
    while (b)
    {
        if (b & 1)
        {
            r = mulMod(r, a, m);
```

```
        }
        a = mulMod(a, a, m);
        b >>= 1;
    }
    return r;
}
```

复杂度 $O(\log(b+1))$ 次模乘。矩阵幂见线性代数章，模数为素数时求逆即 a^{m-2}，见 7.1.006 模逆元：快速幂、exgcd、线性与批量。

7.1.002 GCD、快速 GCD、LCM

标准 `gcd(a, b)`（$O(\log \max(|a|,|b|))$）一般够用，但参数含最小负数时其幅值无法用 `long long` 表示，应改用 `binaryGcd`。二进制 GCD 只用减法和移位、避开取模，取模昂贵时略快。求最小公倍数一律先除后乘 $\operatorname{lcm}(a,b) = a/\gcd(a,b)\cdot b$ 以压小中间值；`safeLcm` 再用 `unsigned __int128` 验算结果是否超出无符号 64 位，溢出返回空。约定 $\gcd(0,0)=0$。

```
unsigned long long magnitude(long long x)
{
    // 返回 x 的无符号幅值，最小负数也可表示
    auto u = (unsigned long long)x;
    return x < 0 ? 0ULL - u : u;
}

unsigned long long binaryGcd(long long a, long long b)
{
    // 返回 gcd(|a|,|b|)，约定 gcd(0,0)=0
    auto x = magnitude(a);
    auto y = magnitude(b);
    if (x == 0)
    {
        return y;
    }
    if (y == 0)
    {
        return x;
    }
    int z = countr_zero(x | y);
    x >>= countr_zero(x);
    do
    {
        y >>= countr_zero(y);
        if (x > y)
        {
            swap(x, y);
        }
        y -= x;
    } while (y ≠ 0);
    return x << z;
}
```

```cpp

optional<unsigned long long> safeLcm(long long a, long long b)
{
    // 返回非负 lcm(|a|,|b|)，超过 64 位无符号范围时返回空
    if (a == 0 || b == 0)
    {
        return 0;
    }
    auto x = magnitude(a);
    auto y = magnitude(b);
    auto g = gcd(x, y);
    unsigned __int128 v = (unsigned __int128)(x / g) * y;
    if (v > numeric_limits<unsigned long long>::max())
    {
        return nullopt;
    }
    return (unsigned long long)v;
}
```

`binaryGcd` 与 `safeLcm` 返回无符号幅值，因而能表示 $gcd(-2^{63},0)=2^{63}$；`safeLcm` 返回空表示结果超过 $2^{64}-1$。二者均为 $O(\log\max(|a|,|b|))$ 时间。

7.1.003 扩展欧几里得

欧几里得迭代 $\gcd(a,b)=\gcd(b,a \bmod b)$,同时维护余数对原数的线性表示,结束时即得到一组 Bézout 系数 x,y，满足 $ax+by=\gcd(a,b)$。两个直接应用：任意互素模数下的逆元 `invMod`（$\gcd(a,m)\neq 1$ 时无逆元，返回空），以及线性丢番图方程 $ax+by=c$（有解当且仅当 $\gcd(a,b)\mid c$）。`exgcd` 返回 `[gcd, x, y]`，gcd 取非负。`invMod`、`exgcd` 是后续同余方程、CRT 的公共底座。

```cpp
using Exgcd = array<__int128, 3>; // [0]=gcd(a,b)，[1]=x，[2]=y，后二者是一组 Bézout 系数

Exgcd exgcd(long long a, long long b)
{
    // a、b 是原方程系数；返回 {gcd(a,b),x,y}，满足 a*x+b*y=gcd(a,b)。
    __int128 r0 = a;
    __int128 r1 = b;
    __int128 x0 = 1;
    __int128 x1 = 0;
    __int128 y0 = 0;
    __int128 y1 = 1;
    while (r1)
    {
        __int128 q = r0 / r1;
        __int128 nr = r0 - q * r1;
        __int128 nx = x0 - q * x1;
        __int128 ny = y0 - q * y1;
        r0 = r1;
        r1 = nr;
        x0 = x1;
        x1 = nx;
```

```
        y0 = y1;
        y1 = ny;
    }
    if (r0 < 0)
    {
        r0 = -r0;
        x0 = -x0;
        y0 = -y0;
    }
    return {r0, x0, y0};
}

optional<long long> invMod(long long a, long long m)
{
    // a 是待求逆整数，m 是大于 1 的模数；返回最小非负逆元，不可逆时返回空。
    assert(m > 1); // 调试检查，可删
    auto [g, x, y] = exgcd(a, m);
    if (g != 1)
    {
        return nullopt;
    }
    x %= m;
    if (x < 0)
    {
        x += m;
    }
    return (long long)x;
}
```

复杂度 $O(\log \max(|a|,|b|))$，迭代实现只用 $O(1)$ 额外空间。`exgcd` 用 `__int128` 保存非负 gcd 与系数，能覆盖两个完整 `long long` 输入；`invMod` 要求 $m>1$。合数模、非互素情形的方程求解见 7.1.004 线性同余方程与同余方程组。

7.1.004 线性同余方程与同余方程组

单个方程 $ax \equiv b \pmod m$ 有解当且仅当 $\gcd(a,m) \mid b$。此时约去公因子 $g=\gcd(a,m)$，得到唯一剩余类 $x \equiv (b/g)\,(a/g)^{-1} \pmod{m/g}$，周期为 m/g。`congruence` 返回这一对 `[余数, 周期]`，无解返回空；多个约束的合并交给 7.1.005 CRT / exCRT。依赖 7.1.002 GCD、快速 GCD、LCM 的 `binaryGcd`、7.1.003 扩展欧几里得 的 `invMod` 与 7.1.001 快速幂 的 `mulMod`。

```
// 依赖 7.1.001 快速幂
// 依赖 7.1.003 扩展欧几里得
using Cong = array<int, 2>;   // [0]= 最小非负余数，[1]= 正周期（模）

optional<Cong> congruence(long long a, long long b, long long m)
{
    // 解 ax≡b (mod m)，返回 {最小非负解，周期}；无解返回空。
    assert(m > 0); // 调试检查，可删
    long long g = (long long)binaryGcd(a, m);
```

```
    if (b % g)
    {
        return nullopt;
    }
    long long q = m / g;
    if (q == 1)
    {
        return Cong{0, 1};
    }
    long long r = mulMod(b / g, *invMod(a / g, q), q);
    return Cong{r, q};
}
```

复杂度 $O(\log m)$。

7.1.005 CRT / exCRT

合并两个同余式 $x \equiv a_0 \pmod{a_1}$、$x \equiv b_0 \pmod{b_1}$：令 $g = \gcd(a_1, b_1)$，差 $b_0 - a_0$ 能被 g 整除时有解，否则无解。用 `exgcd` 解出步长 k，新解 $x = a_0 + a_1 k$，新模为 $\operatorname{lcm}(a_1, b_1)$。逐个 `crt` 折叠即得任意多式的 exCRT；模数两两互素时退化为经典 CRT。`crt` 在方程无解或新模超过 `long long` 正数范围时均返回空。依赖 7.1.003 扩展欧几里得 的 `exgcd`。

```
// 依赖 7.1.004 线性同余方程与同余方程组
optional<Cong> crt(Cong a, Cong b)
{
    // 合并 x≡a[0] (mod a[1]) 与 x≡b[0] (mod b[1]); 返回 {最小非负解,lcm 模}, 无解或新模越界时返回空。
    assert(a[1] > 0 && b[1] > 0); // 调试检查, 可删
    auto [g, x, y] = exgcd(a[1], b[1]);
    __int128 d = (__int128)b[0] - a[0];
    if (d % g)
    {
        return nullopt;
    }
    long long q = (long long)((__int128)b[1] / g);
    __int128 k = 0;
    if (q > 1)
    {
        k = d / g * x % q;
        if (k < 0)
        {
            k += q;
        }
    }
    __int128 m = (__int128)a[1] / g * b[1];
    if (m > numeric_limits<long long>::max())
    {
        return nullopt;
    }
    __int128 r = ((__int128)a[1] * k + a[0]) % m;
    if (r < 0)
```

```
    {
        r += m;
    }
    return Cong{(long long)r, (long long)m};
}
```

折叠 n 个方程共 $O(n \log V)$，V 为模数上界。若已知模数两两互素，也可直接乘出总模再逐项配逆元，常数更小但溢出风险更高，一般仍用上面的两两合并。

7.1.006 模逆元：快速幂、exgcd、线性与批量

素数模 p 下由费马小定理 $a^{-1} \equiv a^{p-2} \pmod p$，`invPrime` 用快速幂求单个逆元。批量求 $1..n$ 的逆元用经典线性递推：由 $p = \lfloor p/i \rfloor\, i + (p \bmod i)$ 两边模 p 整理得 $i^{-1} \equiv -\lfloor p/i \rfloor\, (p \bmod i)^{-1} \pmod p$，一趟 $O(n)$ 填满 `invTable`。任意给定数组则用 `batchInv`：先算前缀积，只求一次总积逆元，再逆序撤掉当前元素。三者都要求模数为素数、被求逆元素非零；任意互素合数模的单个逆元见 7.1.003 扩展欧几里得 的 `invMod`。

```
// 依赖 7.1.001 快速幂
int invPrime(int a, int mod)
{
    // 返回非零剩余 a 在素数模 mod 下的逆元
    assert(0 < a && a < mod); // 调试检查，可删
    int ans = 1;
    int b = mod - 2;
    while (b)
    {
        if (b & 1)
        {
            ans = mulMod(ans, a, mod);
        }
        a = mulMod(a, a, mod);
        b >>= 1;
    }
    return ans;
}

vector<int> invTable(int n, int mod)
{
    // 返回 1..n 在素数模 mod 下的逆元表，位置 0 保持 0
    assert(0 <= n && n < mod); // 调试检查，可删
    vector<int> inv(n + 1);
    if (n >= 1)
    {
        inv[1] = 1;
    }
    for (int i = 2; i <= n; i++)
    {
        inv[i] = mod - mulMod(mod / i, inv[mod % i], mod);
    }
```

```
    return inv;
}

vector<int> batchInv(const vector<int> &a, int mod)
{
    // 返回 a 中每个非零剩余在素数模 mod 下的逆元
    int n = a.size();
    vector<int> pre(n + 1, 1);
    for (int i = 0; i < n; i++)
    {
        assert(0 < a[i] && a[i] < mod); // 调试检查，可删
        pre[i + 1] = mulMod(pre[i], a[i], mod);
    }
    int cur = invPrime(pre[n], mod);
    vector<int> ans(n);
    for (int i = n - 1; i >= 0; i--)
    {
        ans[i] = mulMod(cur, pre[i], mod);
        cur = mulMod(cur, a[i], mod);
    }
    return ans;
}
```

单个 `invPrime` 为 $O(\log p)$；`invTable` 为 $O(n)$；`batchInv` 为 $O(n+\log p)$，预处理后读取任一数组元素的逆元才是 $O(1)$。两种批量接口均用 $O(n)$ 空间。固定模 `998244353` 且有大量在线逆元询问时，可用本条 HPP 附带的 `InvTable<998244353>::get(x)`：预处理 $O(2^{22})$、约 20 MiB 常驻表，之后单次 $O(1)$；它是专项接口，不替代通用 `invMod`、`invPrime` 与批量逆元。

7.1.007 静态取模类

静态取模类是跨章节的基础值类型，完整实现、常见模数的加减乘除验证与复杂度统一放在 1.1.008 静态取模类。本编号只保留旧目录和外部引用的兼容入口，不重复维护第二份实现。需要运行期模数时使用 7.1.008 动态取模类，需要接近 64 位上界的运行时模数时使用 7.1.001 快速幂 的 `mulMod`，不能把固定取模类硬套到不同语义上。

7.1.008 动态取模类

模数在运行期才确定、或一道题要分阶段换模数时用 `DynMod`。模数是全类共享的静态状态，`setMod` 一改，所有 `DynMod` 对象的含义随之改变——因此不同模数的对象不能同时存活、混算。核心是 `Barrett` 约减：预存 $\lceil 2^{64}/m \rceil$ 级别的定点倒数，把 $z \bmod m$ 换成一次乘法估商加一步修正，避开慢速硬件除法（推导见 7.1.009 Barrett 约减）。要求 $m < 2^{31}$，从而 $a, b < 2^{31}$、乘积 $< 2^{62}$ 落在 `unsigned long long` 内。

```
struct Barrett
{
    unsigned m;             // 当前约减使用的正模数
    unsigned long long im;  // m>1 时为 ceil(2^64/m)，作估商用的定点倒数

```

```
    Barrett(unsigned m = 1) : m(m), im(0)
    {
        // m 为正模数，构造时预计算定点倒数 im。
        assert(m > 0); // 调试检查，可删
        im = ~0ULL / m + 1;
    }

    unsigned mul(unsigned a, unsigned b) const
    {
        // 返回 a*b mod m，要求 a、b 均在 [0,m) 内。
        if (m == 1)
        {
            return 0;
        }
        unsigned long long z = 1ULL * a * b;
        unsigned long long x = (unsigned __int128)z * im >> 64;
        unsigned v = z - x * m;
        if (m <= v)   // 估商偏大 1 时 v 因无符号回绕呈现为大值，补一个 m 收回 [0,m)
        {
            v += m;
        }
        return v;
    }
};

struct DynMod
{
    inline static Barrett bt;  // 全类共享的约减器，由 setMod 设定当前模数
    unsigned x = 0;            // 当前模数下的最小非负代表

    static void setMod(unsigned m)
    {
        // 把后续所有 DynMod 对象的模数设为 m
        assert(m > 0 && m < (1U << 31)); // 调试检查，可删
        bt = Barrett(m);
    }
    static unsigned mod()
    {
        // 无参数；返回当前设定的模数。
        return bt.m;
    }

    DynMod(long long v = 0)
    {
        // v 规范到 [0,mod) 建立剩余类。
        v %= mod();
        if (v < 0)
        {
            v += mod();
        }
        x = v;
    }
```

```
58
59      DynMod &operator+=(DynMod b)
60      {
61          // b 是同模剩余类；原地完成模加并返回当前对象引用。
62          x += b.x;
63          if (x >= mod())
64          {
65              x -= mod();
66          }
67          return *this;
68      }
69      DynMod &operator-=(DynMod b)
70      {
71          // b 是同模剩余类；原地完成模减并返回当前对象引用。
72          x += mod() - b.x;
73          if (x >= mod())
74          {
75              x -= mod();
76          }
77          return *this;
78      }
79      DynMod &operator*=(DynMod b)
80      {
81          // b 是同模剩余类；原地完成模乘并返回当前对象引用。
82          x = bt.mul(x, b.x);
83          return *this;
84      }
85      friend DynMod operator+(DynMod a, DynMod b)
86      {
87          // a、b 是同模剩余类；返回 a+b。
88          return a += b;
89      }
90
91      friend DynMod operator-(DynMod a, DynMod b)
92      {
93          // a、b 是同模剩余类；返回 a-b。
94          return a -= b;
95      }
96
97      friend DynMod operator*(DynMod a, DynMod b)
98      {
99          // a、b 是同模剩余类；返回 a*b。
100         return a *= b;
101     }
102     bool operator==(const DynMod &b) const = default;
103
104     DynMod pow(unsigned long long n) const
105     {
106         // n 是非负指数；返回当前剩余类的 n 次幂。
107         DynMod a = *this;
108         DynMod r = 1;
109         while (n)
```

```
        {
            if (n & 1)
            {
                r *= a;
            }
            a *= a;
            n >>= 1;
        }
        return r;
    }
};
```

用法：`DynMod::setMod(m)` 后即可像普通整数一样加减乘与 `pow`。不提供除法（合数模一般不可逆）；素数模要逆元用 `pow(mod() - 2)`。加减乘 $O(1)$，`pow(n)` 为 $O(\log(n+1))$，对象 $O(1)$ 空间。

7.1.009 Barrett 约减

Barrett 约减用乘法代替除法算 $z \bmod m$。当 $m > 1$ 时取定点倒数 $\mathrm{im} = \lceil 2^{64}/m \rceil$，它不小于真倒数，于是估商 $\hat{q} = \lfloor z \cdot \mathrm{im}/2^{64} \rfloor$ 满足 $\hat{q} \in \{q,\, q+1\}$（$q = \lfloor z/m \rfloor$）。把 $v = z - \hat{q}\,m$ 截断到 32 位无符号：$\hat{q} = q$ 时 $v \in [0, m)$；$\hat{q} = q+1$ 时真值落在 $[-m, 0)$，按无符号回绕表现为一个 $\geq m$ 的大数。因此 `if (m <= v) v += m;` 用一次回绕加法把两种情形都收回 $[0, m)$；$m = 1$ 直接返回 0。要求 $m < 2^{31}$，乘积 $z = ab < 2^{62}$，中间量 $z \cdot \mathrm{im}$ 用 `unsigned __int128` 承载。

实现就是 7.1.008 动态取模类 里的 `struct Barrett`，此处不再重复打印。与 7.1.010 Montgomery 模乘 相比，Barrett 不要求模数为奇、无需进出域的编解码，代价是每次约减多一步条件修正。

7.1.010 Montgomery 模乘

Montgomery 把元素 a 编码成 $aR \bmod P$（本实现 $R = 2^{32}$），使模乘只用乘、加、移位而无需硬件除法。核心是 `reduce`：给定 $x < PR$，用 P 关于 R 的负逆 `r`（$Pr \equiv -1 \pmod R$）补齐低 32 位使 $x + ((x\,r) \bmod R)P$ 整除 R，右移 32 位即得 $xR^{-1} \bmod P$（落在 $[0, 2P)$，`val` 再规约一次）。于是一次模乘 = 一次普通乘 + 一次 `reduce`；构造时乘以 $r2 = R^2 \bmod P$ 把普通值编码进域。要求 P 为奇数（负逆 `r` 才存在）且 $P < 2^{30}$（保证 $2P$ 不溢出 32 位）。奇素数下大量模乘通常快于 7.1.009 Barrett 约减。

```
template <unsigned P> struct Mont
{
    unsigned x = 0;   // Montgomery 域表示，即 (真值 * 2^32) mod P，范围 [0,2P)

    static_assert(P > 0 && (P & 1));
    static_assert(P < (1U << 30));

    static constexpr unsigned inv()
    {
        // 返回 P 关于 2^32 的负逆 r：满足 P*r ≡ -1 (mod 2^32)，牛顿迭代 5 次收敛。
        unsigned x = P;
        for (int i = 0; i < 5; i++)
```

```
        {
            x *= 2 - P * x;
        }
        return -x;
    }

    static constexpr unsigned r = inv(); // r 是 -P^{-1} mod 2^32，用于 Montgomery 约减。
    static constexpr unsigned r2 = -(unsigned long long)P % P; // r2 是 2^64 mod P，用于转入
    ↪ Montgomery 表示。

    static constexpr unsigned reduce(unsigned long long x)
    {
        // 输入 x < 2^32 * P，返回 x * 2^-32 mod P（结果在 [0,2P) 内）。
        return (x + (unsigned long long)((unsigned)x * r) * P) >> 32;
    }

    Mont(long long v = 0)
    {
        // 把普通整数 v 编码进 Montgomery 域
        v %= P;
        if (v < 0)
        {
            v += P;
        }
        x = reduce((unsigned long long)v * r2);
    }

    unsigned val() const
    {
        // 无参数；返回当前剩余类的最小非负整数代表。
        unsigned v = reduce(x);
        return v >= P ? v - P : v;
    }

    Mont &operator+=(const Mont &o)
    {
        // o 是同模 Montgomery 数；原地完成域内加法并返回当前对象引用。
        x += o.x;
        if (x >= 2 * P)
        {
            x -= 2 * P;
        }
        return *this;
    }

    Mont &operator-=(const Mont &o)
    {
        // o 是同模 Montgomery 数；原地完成域内减法并返回当前对象引用。
        x -= o.x;
        if (x >= 2 * P)
        {
            x += 2 * P;
```

```
        }
        return *this;
    }

    Mont &operator*=(const Mont &o)
    {
        // o 是同模 Montgomery 数；原地完成域内乘法并返回当前对象引用。
        x = reduce((unsigned long long)x * o.x);
        return *this;
    }

    Mont pow(long long n) const
    {
        // n 是非负指数；返回当前元素的 n 次幂。
        assert(n >= 0); // 调试检查，可删
        Mont a = *this;
        Mont r = 1;
        while (n)
        {
            if (n & 1)
            {
                r *= a;
            }
            a *= a;
            n >>= 1;
        }
        return r;
    }

    Mont invMod() const
    {
        // 当前元素必须非零且 P 必须为素数；返回乘法逆元。
        assert(val() != 0); // 调试检查，可删
        return pow(P - 2);
    }

    Mont &operator/=(const Mont &o)
    {
        // o 是非零同模 Montgomery 数；原地除以 o 并返回当前对象引用。
        return *this *= o.invMod();
    }

    friend Mont operator+(Mont a, const Mont &b)
    {
        // a、b 是同模 Montgomery 数；返回 a+b。
        return a += b;
    }
    friend Mont operator-(Mont a, const Mont &b)
    {
        // a、b 是同模 Montgomery 数；返回 a-b。
        return a -= b;
    }
```

```
    friend Mont operator*(Mont a, const Mont &b)
    {
        // a、b 是同模 Montgomery 数；返回 a*b。
        return a *= b;
    }
    friend Mont operator/(Mont a, const Mont &b)
    {
        // a、b 是同模 Montgomery 数且 b 非零；返回 a/b。
        return a /= b;
    }
    friend bool operator==(const Mont &a, const Mont &b)
    {
        // a、b 是同模 Montgomery 数；返回二者是否代表同一剩余类。
        return a.val() == b.val();
    }
};
```

用 `Mont<P>` 要求 `P` 为奇数且 $P < 2^{30}$。加减乘除 $O(1)$，`pow`/`invMod` $O(\log P)$，对象 $O(1)$ 空间；除法与 `invMod` 走费马小定理，额外要求 `P` 为素数。

7.1.011 Long Long 防溢出乘法

计算 $a \cdot b \bmod m$ 而中间不溢出。若两个规范余数的乘积能放进 `long long`，直接乘再取模最快；否则把这一条乘法升到 `__int128`，即 7.1.001 快速幂 的 `mulMod`，$O(1)$ 且无精度问题。若编译器不支持 `__int128`，退路有二：龟速乘（把 b 二进制拆开、逐位倍增累加，$O(\log(b+1))$）；或浮点估商法 $ab - \lfloor ab/m \rfloor m$，用 `long double` 估商后校正余数，$O(1)$ 但需留意精度。竞赛环境几乎总有 `__int128`，完整 64 位模数直接用 `mulMod` 即可。

7.1.012 快速整除判定

判断一个可能超过机器字长、以字符串给出的十进制整数能否被 m 整除。做法是 Horner 逐位取模：从高位到低位维护 $r = (10r + d_i) \bmod m$，扫完若 $r = 0$ 即整除。支持前导正负号，m 取绝对值且须非零，$O(|s|)$ 一趟完成，不需高精度除法。

```
// 依赖 7.1.002 GCD、快速 GCD、LCM
namespace fastDiv
{
bool divisible(const string &s, int m)
{
    // s 是带可选正负号的十进制整数串，m 是非零除数；返回 s 是否能被 m 整除。
    assert(m != 0); // 调试检查，可删
    auto d = magnitude(m);
    unsigned long long r = 0;
    bool small = d <= 3037000499ULL;
    int p = !s.empty() && (s[0] == '+' || s[0] == '-');
    assert(p < (int)s.size()); // 调试检查，可删
    for (; p < (int)s.size(); p++)
    {
```

```
        assert(isdigit((unsigned char)s[p])); // 调试检查，可删
        unsigned digit = (unsigned)(s[p] - '0');
        if (small)
        {
            r = (r * 10 + digit) % d;
        }
        else
        {
            r = ((unsigned __int128)r * 10 + digit) % d;
        }
    }
    return r == 0;
}
}
```

复杂度 $O(|s|)$ 时间、$O(1)$ 空间；小除数的逐位 Horner 只用 64 位运算，除数绝对值超过 `3037000499` 时才对单步 `10r+d_i` 使用 `unsigned __int128`。

7.1.013 模意义真分数还原

已知某既约真分数 p/q 对模 m 的值 $x \equiv p\,q^{-1} \pmod m$，且有界 $|p| \le a$、$0 < q \le b$，要还原出 p, q。做法是对 (m, x) 跑扩展欧几里得（等价于 x/m 的连分数展开）并同步维护 t 系数：当余数 r_1 首次不超过分子界 a 时停下，(r_1, t_1) 即候选分子、分母。再校验 $t_1 \neq 0$、$|t_1| \le b$、$\gcd(r_1, t_1) = 1$ 且 $x\,t_1 \equiv r_1 \pmod m$，全通过才唯一还原，否则返回空；当 $2ab < m$ 时解必唯一。典型用途：模意义下做完一串有理运算后把结果读回真分数。

```
optional<pair<long long, long long>> ratRebuild(long long x, long long m, long long a, long
↪ long b)
{
    // x 是模 m 的剩余，a、b 是分子绝对值与分母上界；在 2ab<m 时返回唯一既约真分数。
    assert(m > 0 && a >= 0 && b > 0); // 调试检查，可删
    assert((__int128)2 * a * b < m); // 调试检查，可删
    x %= m;
    if (x < 0)
    {
        x += m;
    }
    __int128 r0 = m;
    __int128 r1 = x;
    __int128 t0 = 0;
    __int128 t1 = 1;
    auto mag = [](__int128 v)
    {
        return v < 0 ? -v : v;
    };
    while (mag(r1) > a)
    {
        if (!r1)
        {
```

```
            return nullopt;
        }
        __int128 q = r0 / r1;
        __int128 nr = r0 - q * r1;
        __int128 nt = t0 - q * t1;
        r0 = r1;
        r1 = nr;
        t0 = t1;
        t1 = nt;
    }
    if (!t1 || mag(t1) > b)
    {
        return nullopt;
    }
    if (t1 < 0)
    {
        r1 = -r1;
        t1 = -t1;
    }
    if (mag(r1) >= t1 || gcd((long long)mag(r1), (long long)t1) != 1)
    {
        return nullopt;
    }
    if (((__int128)x * t1 - r1) % m)
    {
        return nullopt;
    }
    return pair{(long long)r1, (long long)t1};
}
```

`ratRebuild(x, m, a, b)` 要求 $2ab < m$，该界保证满足条件的既约真分数至多一个；返回 `{分子, 分母}`（分母恒正），不存在时返回空。复杂度 $O(\log m)$ 时间、$O(1)$ 空间。

7.1.014 整数 k 次方根

求最大整数 x 使 $x^k \le n$，即 $\lfloor n^{1/k} \rfloor$。这里在 $[1,\ 2^{64/k+1}]$ 上二分，判定 `ok(x)` 用 `unsigned __int128` 边乘边比、一超过 n 立即返回假，从而避免 x^k 溢出。$k = 1$ 或 $n \le 1$ 直接返回 n，$k \ge 64$ 时答案必为 1。相比先浮点估根再修正，二分更短且无精度坑。

```
unsigned long long kthRoot(unsigned long long n, int k)
{
    // n 是被开方的非负整数，k 是正整数次数；返回 floor(n^(1/k))。
    assert(k > 0); // 调试检查，可删
    if (k == 1 || n <= 1)
    {
        return n;
    }
    if (k >= 64)
    {
        return 1;
```

```
    }
    auto ok = [&](unsigned long long x)
    {
        unsigned __int128 y = 1;
        for (int i = 0; i < k; i++)
        {
            y *= x;
            if (y > n)
            {
                return false;
            }
        }
        return true;
    };
    unsigned long long l = 1;
    unsigned long long r = min(n, 1ULL << (64 / k + 1));
    while (l < r)
    {
        unsigned long long m = l + (r - l + 1) / 2;
        if (ok(m))
        {
            l = m;
        }
        else
        {
            r = m - 1;
        }
    }
    return l;
}
```

`kthRoot(n, k)` 接收无符号 n、正整数 k，返回 $\lfloor n^{1/k} \rfloor$。复杂度 $O(k \log(n+1))$ 时间、$O(1)$ 空间。

7.1.015 线性同余不等式

求最小非负整数 x 使 $l \le dx \bmod m \le r$，常用于「带模线性函数首次落入某区间」的问题。核心是 Euclid 风格递归：$2d > m$ 时借 $dx \bmod m$ 与 $(m-d)x \bmod m$ 的对称把系数翻小；若当前步 $\lfloor (l-1)/d \rfloor < \lfloor r/d \rfloor$ 说明区间内已含 d 的倍数，直接取 $\lfloor (l-1)/d \rfloor + 1$；否则把问题递归到模 d。每层至少对较大参数取模一次，深度 $O(\log m)$。

```
int minModInt(int m, int d, int l, int r)
{
    // 参数满足 m>0 且 0<=d,l<=r<m；返回使 l<=d*x mod m<=r 的最小非负 x，无解返回 -1。
    assert(m > 0 && 0 <= d && d < m && 0 <= l && l <= r && r < m); // 调试检查，可删
    if (l == 0)
    {
        return 0;
    }
    if (d == 0)
    {
```

```
        return -1;
    }
    if (d > m - d)
    {
        return minModInt(m, m - d, m - r, m - l);
    }
    if ((l - 1) / d < r / d)
    {
        return (l - 1) / d + 1;
    }
    auto k = minModInt(d, (-m % d + d) % d, l % d, r % d);
    if (k == -1)
    {
        return -1;
    }
    return (int)(((__int128)k * m + l - 1) / d + 1);
}
```

`minModInt(m, d, l, r)` 要求 $m > 0$、$0 \le d < m$、$0 \le l \le r < m$，返回满足条件的最小非负 x，无解返回 `-1`。比较 $2d > m$ 改写为无溢出的 `d>m-d`，最终乘法在 `__int128` 中完成；复杂度 $O(\log m)$ 时间与递归栈。

7.1.016 基于值域优化的 GCD

当所有查询数都落在 $[1, n]$ 且需要海量 gcd 时，可 $O(n)$ 预处理后单次 $O(1)$ 作答。关键：把每个 $x \le n$ 分解成三个因子 $x = f_0 f_1 f_2$，每个因子要么 $\le \sqrt{n}$、要么本身为质数（线性筛时顺带维护）；再打出 $\le \sqrt{n}$ 两数的 gcd 小表 `tab`。查询 $\gcd(x, y)$ 时对 x 的三个因子依次与 y 求 gcd 并从 y 中除去：小因子查 `tab`，质因子只需判 y 能否被其整除，三步相乘即答案。依据是任意数都能写成三个「各自 $\le \sqrt{n}$ 或为质数」的因子之积。

```
struct ValueGcd
{
    int n;                      // 允许查询的最大值
    int b;                      // 小因子表边界 floor(sqrt(n))+1
    vector<char> prime;         // prime[x] 表示 x 是否为质数
    vector<array<int, 3>> fac; // x 的三个受限因子
    vector<vector<int>> tab;    // 小值两两 gcd 表

    ValueGcd(int n) : n(n), b(sqrtl(n) + 1), prime(n + 1, true), fac(n + 1), tab(b + 1,
    ↪ vector<int>(b + 1))
    {
        // 参数 n 是闭值域上界；预处理分解和小值 GCD 表，无返回值。
        prime[0] = false;
        if (n)
        {
            prime[1] = false;
        }
        for (int i = 0; i <= b; i++)
        {
```

```
            tab[i][0] = i;
            tab[0][i] = i;
        }
        for (int i = 1; i <= b; i++)
        {
            for (int j = i; j <= b; j++)
            {
                tab[i][j] = tab[i][j % i];
                tab[j][i] = tab[i][j];
            }
        }
        vector<int> ps;
        if (n >= 1)
        {
            fac[1] = {1, 1, 1};
        }
        for (int i = 2; i <= n; i++)
        {
            if (prime[i])
            {
                ps.push_back(i);
                fac[i] = {1, 1, i};
            }
            for (auto p : ps)
            {
                if (i > n / p)
                {
                    break;
                }
                prime[i * p] = false;
                fac[i * p] = fac[i];
                auto at = min_element(fac[i * p].begin(), fac[i * p].end());
                *at *= p;
                if (i % p == 0)
                {
                    break;
                }
            }
        }
    }

    int small(int x, int y) const
    {
        // 参数 x 是不超过 b 或为质数的因子，y 在 [0,n]；返回 gcd(x,y)。
        if (x == 1)
        {
            return 1;
        }
        auto t = y % x;
        if (prime[x])
        {
            return t ? 1 : x;
```

```
        }
        return tab[t][x];
    }

    int get(int x, int y) const
    {
        // 参数 x、y 均在 [1,n]；返回二者最大公约数，查询阶段为常数次查表。
        assert(1 <= x && x <= n && 1 <= y && y <= n); // 调试检查，可删
        auto g0 = small(fac[x][0], y);
        y /= g0;
        auto g1 = small(fac[x][1], y);
        y /= g1;
        auto g2 = small(fac[x][2], y);
        return g0 * g1 * g2;
    }
};
```

构造 `ValueGcd(n)` 预处理值域上界 n，再 `get(x, y)` 查询，要求 $1 \le x, y \le n$（`get` 内含断言，不接受 0）。预处理 $O(n + (\sqrt{n})^2) = O(n)$ 时间与空间，单次查询 $O(1)$。查询数超出值域时退化用标准 `gcd` 即可。海量小值 gcd 才值得上此结构，否则直接 `gcd` 更省事。

7.1.017 高斯整数与最大公约数

高斯整数环 $\mathbb{Z}[i]$ 的元素写作 $x + yi$，范数 $N(x + yi) = x^2 + y^2$ 满足乘法性。对非零 b，把复商 $(a\bar{b})/N(b)$ 的实、虚部分别舍入到最近整数得到商 q，则余数 $a - qb$ 的两个坐标误差都不超过 $1/2$，因而其范数严格小于 $N(b)$。这使普通欧几里得算法在 $\mathbb{Z}[i]$ 中同样终止；最大公约数只确定到乘以 $\pm 1, \pm i$ 的单位元，本接口不强行规定代表元。

```
struct GaussianInt
{
    int x, y; // x、y 分别是实部与虚部。

    GaussianInt(int x = 0, int y = 0) : x(x), y(y)
    {
        // x、y 是高斯整数 x+yi 的两个系数。
    }

    bool zero() const
    {
        // 无参数；返回当前高斯整数是否为 0。
        return x == 0 && y == 0;
    }

    GaussianInt conjugate() const
    {
        // 无参数；返回共轭 x-yi。
        return {x, -y};
    }

```

```
    GaussianInt &operator+=(GaussianInt b)
    {
        // b 是加数；原地加 b 并返回自身。
        x += b.x;
        y += b.y;
        return *this;
    }

    GaussianInt &operator-=(GaussianInt b)
    {
        // b 是减数；原地减 b 并返回自身。
        x -= b.x;
        y -= b.y;
        return *this;
    }

    GaussianInt &operator*=(GaussianInt b)
    {
        // b 是乘数；原地乘 b 并返回自身，题目范围保证结果可由 64 位表示。
        __int128 nx = (__int128)x * b.x - (__int128)y * b.y;
        __int128 ny = (__int128)x * b.y + (__int128)y * b.x;
        x = (int)nx;
        y = (int)ny;
        return *this;
    }

    friend GaussianInt operator+(GaussianInt a, GaussianInt b)
    {
        // a、b 是高斯整数；返回 a+b。
        return a += b;
    }

    friend GaussianInt operator-(GaussianInt a, GaussianInt b)
    {
        // a、b 是高斯整数；返回 a-b。
        return a -= b;
    }

    friend GaussianInt operator*(GaussianInt a, GaussianInt b)
    {
        // a、b 是高斯整数；返回 a*b。
        return a *= b;
    }

    static int nearestDiv(__int128 a, __int128 b)
    {
        // a 是被除数，b 是正除数；返回距 a/b 最近的整数，半整数向正无穷取整。
        assert(b > 0);
        return (int)(a >= 0 ? (a + b / 2) / b : -((-a + b / 2) / b));
    }

    GaussianInt div(GaussianInt b) const
```

```
    {
        // b 非零；返回使余数范数严格小于 b 范数的一组商。
        assert(!b.zero());
        __int128 d = (__int128)b.x * b.x + (__int128)b.y * b.y;
        return {nearestDiv((__int128)x * b.x + (__int128)y * b.y, d),
                nearestDiv((__int128)y * b.x - (__int128)x * b.y, d)};
    }

    GaussianInt mod(GaussianInt b) const
    {
        // b 非零；返回当前值除以 b 的欧几里得余数。
        return *this - div(b) * b;
    }

    static GaussianInt gcd(GaussianInt a, GaussianInt b)
    {
        // a、b 是高斯整数；返回任意一个高斯整数最大公约数。
        while (!b.zero())
        {
            GaussianInt r = a.mod(b);
            a = b;
            b = r;
        }
        return a;
    }
};
```

`GaussianInt::gcd(a,b)` 返回任意单位元等价的高斯整数 GCD；`div(b)` 的 `b` 必须非零，`mod(b)` 返回欧几里得余数。LC 的坐标绝对值不超过 10^9，中间乘积以 `__int128` 计算，最终输入输出在 `long long` 范围内；单次复杂度为欧几里得迭代次数，通常为 $O(\log \max(N(a), N(b)))$，额外空间 $O(1)$。普通整数只需 7.1.001 GCD 与扩展 GCD，不要为整数 GCD 引入高斯除法。

7.2 素数与因数分解

7.2.001 埃氏筛、线性筛

线性筛（欧拉筛）让每个合数只被它的最小质因子筛掉一次，$O(n)$ 求出 $[2, n]$ 的全部质数，并顺带填出最小质因子表 `minp`。有了 `minp`，任意 $x \le n$ 都能在 $O(\log x)$ 内反复除最小质因子完成分解。相比埃氏筛的 $O(n \log \log n)$，线性筛不重复标记、还免费带出 `minp`，本书作默认；只要素数列表、值域也不大时埃氏筛更短。

```
struct Sieve
{
    vector<int> minp;   // minp[x] 是 x 的最小质因子
    vector<int> primes; // 升序质数表

    Sieve(int n) : minp(n + 1)
    {
        // n 是非负筛表上界；线性筛 [2,n] 并填充 minp 与 primes。
```

```
        for (int i = 2; i <= n; i++)
        {
            if (!minp[i])
            {
                minp[i] = i;
                primes.push_back(i);
            }
            for (int p : primes)
            {
                if (p > minp[i] || i > n / p)
                {
                    break;
                }
                minp[i * p] = p;
            }
        }
    }

    vector<pair<int, int>> factor(int x) const
    {
        // x 在 [1,minp.size()) 内；返回按质因子升序的 {质因子, 指数}。
        assert(1 <= x && x < (int)minp.size()); // 调试检查，可删
        vector<pair<int, int>> ans;
        while (x > 1)
        {
            int p = minp[x];
            int c = 0;
            while (x % p == 0)
            {
                x /= p;
                c++;
            }
            ans.push_back({p, c});
        }
        return ans;
    }
};

vector<int> enumPrimes(int n)
{
    // n 是非负上界；仅标记奇数并返回 [2,n] 的全部质数。
    if (n < 2)
    {
        return {};
    }
    vector<bool> composite(n / 2 + 1);
    for (int p = 3; p <= n / p; p += 2)
    {
        if (composite[p / 2])
        {
            continue;
        }
```

```
        for (int x = p * p; x <= n; x += p * 2)
        {
            composite[x / 2] = true;
        }
    }
    vector<int> primes{2};
    for (int x = 3; x <= n; x += 2)
    {
        if (!composite[x / 2])
        {
            primes.push_back(x);
        }
    }
    return primes;
}
```

构造 `Sieve(n)` 后 `primes` 即质数表、`factor(x)` 分解 $x \le n$。筛 $O(n)$ 时间与空间，`factor` 为 $O(\log x)$。只需枚举大上界内质数时用 `enumPrimes(n)`：它只存奇数合数位，筛标记占约 $n/2$ 位，时间 $O(n \log\log n)$；返回表仍占 $O(\pi(n))$ 个整数。单个大整数分解改用 7.2.004 Pollard-Rho。

7.2.002 区间筛

当区间右端 r 很大（如 10^{12}）而区间长度 $r-l$ 仍可存下时，用区间筛求 $[l,r)$ 内的质数。先普通筛出 $\le \sqrt{r}$ 的质数，再对每个这样的质数 p，从 $\ge p^2$ 的首个 p 倍数起在长度 $r-l$ 的布尔数组上标记合数；$0,1$ 需显式排除。平方根依赖 7.1.014 整数 k 次方根 的 `kthRoot`。

```
vector<long long> segPrimes(long long l, long long r)
{
    // 返回半开区间 [l,r) 内的全部素数
    l = max(l, 2LL);
    if (l >= r)
    {
        return {};
    }
    int n = kthRoot((unsigned long long)(r - 1), 2);
    vector<bool> a(n + 1, true);
    vector<bool> b((size_t)(r - l), true);
    vector<int> p;
    for (int i = 2; i <= n; i++)
    {
        if (!a[i])
        {
            continue;
        }
        p.push_back(i);
        if (1LL * i * i <= n)
        {
            for (int j = i * i; j <= n; j += i)
            {
                a[j] = false;
```

```
            }
        }
    }
    for (long long x : p)
    {
        long long q = l / x + (l % x != 0);
        long long first = q > (r - 1) / x ? r : q * x;
        long long square = x > (r - 1) / x ? r : x * x;
        long long s = max(square, first);
        for (long long y = s; y < r; )
        {
            b[(size_t)(y - l)] = false;
            if (y > r - x)
            {
                break;
            }
            y += x;
        }
    }
    vector<long long> ans;
    for (long long x = l; x < r; x++)
    {
        if (b[x - l])
        {
            ans.push_back(x);
        }
    }
    return ans;
}
```

`segPrimes(l, r)` 返回 $[l, r)$ 内的全部质数（升序）；要求 $r > l$ 且 `r-l` 能转为无符号 64 位，区间长度 `r-l` 能作为容器大小。首个倍数和平方均用 64 位除法边界计算，不在循环中使用 128 位下标。设 $m = r - l$，时间 $O(\sqrt{r}\log\log r + m\log\log r)$，空间 $O(\sqrt{r} + m)$。整体值域不大时直接用 7.2.001 埃氏筛、线性筛。

7.2.003 Miller-Rabin

确定性判素：把 $n - 1 = d \cdot 2^s$，对每个底数 a 检查 $a^d \bmod n$ 是否为 1，或连续平方过程中是否出现 $n - 1$；两者都不满足则 n 是合数。对整个 64 位范围使用固定底数

$$2, 325, 9375, 28178, 450775, 9780504, 1795265022.$$

这组底数已被验证能排除全部强伪素数，因此结论确定、无需随机重复。乘法用 `unsigned __int128` 防溢出；`mul64`/`pow64` 也供 7.2.004 Pollard-Rho 复用。

```
unsigned long long mul64(unsigned long long a, unsigned long long b, unsigned long long mod)
{
    // a、b 是乘数，mod 是正模数；返回 a*b 对 mod 的最小非负余数。
```

```
    return (unsigned __int128)a * b % mod;
}

unsigned long long pow64(unsigned long long a, unsigned long long b, unsigned long long mod)
{
    // 返回 a^b 对正模数 mod 的最小非负余数
    assert(mod > 0); // 调试检查，可删
    unsigned long long ans = 1 % mod;
    a %= mod;
    while (b)
    {
        if (b & 1)
        {
            ans = mul64(ans, a, mod);
        }
        a = mul64(a, a, mod);
        b >>= 1;
    }
    return ans;
}

bool isPrime(unsigned long long n)
{
    // n 是 64 位无符号整数；返回 n 是否为质数，本底数集合在完整 64 位范围内确定。
    if (n < 2)
    {
        return false;
    }
    for (auto p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL, 23ULL, 29ULL, 31ULL,
    ↪  37ULL})
    {
        if (n % p == 0)
        {
            return n == p;
        }
    }
    int s = countr_zero(n - 1);
    auto d = (n - 1) >> s;
    for (auto a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL})
    {
        if (a % n == 0)
        {
            continue;
        }
        auto x = pow64(a % n, d, n);
        if (x == 1 || x == n - 1)
        {
            continue;
        }
        bool ok = false;
        for (int r = 1; r < s; r++)
        {
```

```
            x = mul64(x, x, n);
            if (x == n - 1)
            {
                ok = true;
                break;
            }
        }
        if (!ok)
        {
            return false;
        }
    }
    return true;
}
```

`isPrime(n)` 接收 `unsigned long long`；7 个底数各 $O(\log n)$ 次 u128 模乘，$O(1)$ 空间。合数要拆因子用 7.2.004 Pollard-Rho。快速检查分两行写：

- $2, 17, 2305843009213693951$ 判为质数；
- $1, 341, 3825123056546413051$ 判为合数。

7.2.004 Pollard-Rho

对 64 位合数随机找一个非平凡因子。在伪随机序列 $x \leftarrow x^2 + c \pmod n$ 上用 Brent 倍增区间判环；每 128 步把差值乘积取一次 `gcd`，将逐步 `gcd` 的开销摊薄。若批量乘积直接得到 n，再从该块起点逐步回退，保证不会漏掉块内较早出现的非平凡因子。拿到因子后递归分解两侧，配合 7.2.003 Miller-Rabin 的 `isPrime` 在质数处停下、`mul64` 做模乘，最终得到保留重数的升序质因子表。

```
unsigned long long pollard(unsigned long long n)
{
    // n 是合数且 n>1；用 Brent 分块乘积返回非平凡因子，依赖 7.2.003 的 mul64。
    if (n % 2 == 0)
    {
        return 2;
    }
    static mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    while (true)
    {
        auto c = rng() % (n - 1) + 1;
        auto y = rng() % (n - 1) + 1;
        auto f = [&](unsigned long long z)
        {
            return (unsigned long long)(((unsigned __int128)mul64(z, z, n) + c) % n);
        };
        unsigned long long x = 0, ys = 0, g = 1, q = 1, r = 1;
        constexpr unsigned long long m = 128;
        while (g == 1)
        {
            x = y;
            q = 1;
```

```
            for (unsigned long long i = 0; i < r; i++)
            {
                y = f(y);
            }
            for (unsigned long long k = 0; k < r && g == 1; k += m)
            {
                ys = y;
                auto lim = min(m, r - k);
                for (unsigned long long i = 0; i < lim; i++)
                {
                    y = f(y);
                    auto d = x > y ? x - y : y - x;
                    q = mul64(q, d, n);
                }
                g = gcd(q, n);
            }
            r <<= 1;
        }
        if (g == n)
        {
            do
            {
                ys = f(ys);
                auto d = x > ys ? x - ys : ys - x;
                g = gcd(d, n);
            } while (g == 1);
        }
        if (g != n)
        {
            return g;
        }
    }
}

vector<unsigned long long> factor64(unsigned long long n)
{
    // n 是正整数；返回 n 的全部质因子并保留重数，结果升序，依赖 7.2.003 的 isPrime。
    assert(n > 0); // 调试检查，可删
    vector<unsigned long long> ans;
    auto dfs = [&](auto &&self, unsigned long long x) -> void
    {
        if (x == 1)
        {
            return;
        }
        if (isPrime(x))
        {
            ans.push_back(x);
            return;
        }
        auto d = pollard(x);
        self(self, d);
```

```
        self(self, x / d);
    };
    dfs(dfs, n);
    sort(ans.begin(), ans.end());
    return ans;
}
```

依赖 7.2.003 Miller-Rabin 的 `mul64`、`isPrime`。`factor64(n)` 返回升序质因子（含重数），$n = 1$ 返回空。Brent 版仍为期望约 $O(n^{1/4})$ 次模运算，但每块只做一次 `gcd`；递归栈 $O(\log n)$，无有限最坏保证。验证：`factor64(600851475143)` 得 $71, 839, 1471, 6857$。

7.2.005 质因数分解

对单个中小整数试除到 $\sqrt{n}$：合数必有 $\le \sqrt{n}$ 的质因子，逐个除尽后剩下大于 1 的部分只能是质数。只试 2 与奇数即可。批量分解筛表内整数用 7.2.001 埃氏筛、线性筛 的 `minp`，接近 64 位的大数用 7.2.004 Pollard-Rho。

```
vector<pair<int, int>> trialFactor(int n)
{
    // n 是正整数；返回按质因子递增排列的 {质因子, 指数}，适合单次分解中小整数。
    assert(n > 0); // 调试检查，可删
    vector<pair<int, int>> ans;
    for (int p = 2; p <= n / p; p += p == 2 ? 1 : 2)
    {
        if (n % p)
        {
            continue;
        }
        int e = 0;
        while (n % p == 0)
        {
            n /= p;
            e++;
        }
        ans.push_back({p, e});
    }
    if (n > 1)
    {
        ans.push_back({n, 1});
    }
    return ans;
}
```

`trialFactor(n)` 返回升序 `{质因子, 指数}`。$O(\sqrt{n})$ 时间、输出外 $O(1)$ 空间，适合单次分解不大的整数。验证：`trialFactor(360)` 得 $(2,3),(3,2),(5,1)$，`trialFactor(1000000007)` 得 $(1000000007, 1)$。

7.2.006 约数枚举、约数个数和

枚举单个整数的全部正约数：成对枚举到 $\sqrt{n}$，每找到 d 就同时得到 n/d，最后拼成升序表。若已有质因数分解 $n=\prod p_i^{e_i}$，则约数个数为 $\prod(e_i+1)$、约数和为 $\prod \frac{p_i^{e_i+1}-1}{p_i-1}$（各质数幂的等比和之积），`divCntSum` 直接由指数算出、不再分解。批量场景先用 7.2.001 埃氏筛、线性筛 筛 `minp` 再分解。

```
vector<int> divisors(int n)
{
    // n 是正整数；返回 n 的全部正约数，已升序且不重复。
    assert(n > 0); // 调试检查，可删
    vector<int> a;
    vector<int> b;
    for (int d = 1; d <= n / d; d++)
    {
        if (n % d == 0)
        {
            continue;
        }
        a.push_back(d);
        if (d != n / d)
        {
            b.push_back(n / d);
        }
    }
    reverse(b.begin(), b.end());
    a.insert(a.end(), b.begin(), b.end());
    return a;
}

pair<int, int> divCntSum(const vector<pair<int, int>> &fac)
{
    // fac 的每项是不同质因子 p 及其指数 e；返回约数个数与约数和，结果须能放入有符号 64 位。
    int cnt = 1;
    int sum = 1;
    for (auto [p, e] : fac)
    {
        int pw = 1;
        int cur = 1;
        for (int i = 1; i <= e; i++)
        {
            pw *= p;
            cur += pw;
        }
        cnt *= e + 1;
        sum *= cur;
    }
    return {cnt, sum};
}
```

`divisors(n)` 返回升序约数表，$O(\sqrt{n})$ 时间、$O(d(n))$ 空间；`divCntSum(fac)` 输入 `{p,e}` 列表、返回 `{约数个数，约数和}`，$O(\sum e_i)$，本身不负责分解。

7.2.007 因数最多的数

求不超过 lim 且约数个数最多的最小正整数（反素数 / 高合成数）。关键观察：最优数的质因子必是从 2 起若干连续最小质数，且指数非增。于是按质数递增、指数非增地 DFS，当前约数数为各指数加一之积，乘积超界即剪枝；前 16 个质数足以覆盖 64 位范围。

```
namespace highlyComp
{
array<int, 2> mostDivisors(int lim)
{
    // lim 是正上界；返回的两维依次是最小整数及其因数个数。
    assert(lim >= 1); // 调试检查，可删
    constexpr array<int, 16> ps = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53};
    int best = 1;
    int bestCnt = 1;
    auto dfs = [&](auto &&self, int p, int last, int x, int cnt) -> void
    {
        // self 是递归自身，p 是质数下标，last 是指数上界，x 是当前乘积，cnt 是因数个数。
        if (cnt > bestCnt || (cnt == bestCnt && x < best))
        {
            best = x;
            bestCnt = cnt;
        }
        if (p == (int)ps.size())
        {
            return;
        }
        for (int e = 1; e <= last && x <= lim / ps[p]; e++)
        {
            x *= ps[p];
            self(self, p + 1, e, x, cnt * (e + 1));
        }
    };
    dfs(dfs, 0, 63, 1, 1);
    return {best, bestCnt};
}
}
```

`mostDivisors(lim)` 返回 {最小整数，其约数个数}。搜索节点数即满足非增指数约束的候选数，远小于逐个枚举，递归深度 $O(\log \mathrm{lim})$。

7.2.008 阶乘最后非零位

求 $n!$ 去掉末尾全部零后的个位数字（十进制）。末尾零由成对的 $2,5$ 产生，而 2 远多于 5，故零的个数等于 $n!$ 中 5 的指数。递归地把 $\lfloor n/5 \rfloor$ 段的贡献、本段 $n \bmod 5$ 的尾积、以及多出来的 2 的 4 周期相乘，$O(\log_5 n)$ 得结果；表 $d = \{1,1,2,6,4\}$ 是 $0!$ 到 $4!$ 的末位。

```
namespace lastNzDigit
{
```

```
int lastDigit(int n)
{
    // n 是非负整数；返回 n! 去掉末尾全部零后的个位数字。
    assert(n >= 0); // 调试检查，可删
    constexpr array<int, 5> d = {1, 1, 2, 6, 4};
    if (n < 5)
    {
        return d[n];
    }
    int p = n / 5;
    int two = array<int, 4>{6, 2, 4, 8}[p % 4];
    return lastDigit(p) * d[n % 5] * two % 10;
}
}
```

`lastDigit(n)` 返回 $n!$ 去尾零后的个位，$O(\log_5 n)$ 时间与递归栈。

7.2.009 阶乘模合数

计算 $n! \bmod p^e$（p 质数），是扩展 Lucas 的底层步骤。把 $n!$ 拆成质因子 p 的幂与不含 p 的单位部分两块：单位部分按完整 p^e 周期递归，`unitPrefix[i]` 预存一个周期内 $1..i$ 中删去 p 倍数后的前缀积，$\text{unitFact}(n) = \text{unitPrefix}[p^e]^{\lfloor n/p^e \rfloor} \cdot \text{unitPrefix}[n \bmod p^e] \cdot \text{unitFact}(\lfloor n/p \rfloor)$；$p$ 的幂指数由 Legendre 公式 $v_p(n!) = \sum_{j\ge 1} \lfloor n/p^j \rfloor$ 得到，指数达到 e 时整体为 0。本版直接预处理整个周期，只用于 $p^e \le 5 \times 10^6$；更大模数不能只删掉容量断言继续使用。依赖 7.1.001 快速幂 的 `powerMod`。

```
class PPFactorial
{
    // p 是质数，e 是指数，mod=p^e。
    int p, e, mod = 1;
    // unitPrefix[i] 是 1..i 中删去 p 的倍数后的乘积模 mod。
    vector<int> unitPrefix;

    int unitFact(int n) const
    {
        // n 是非负整数；返回 n! 删除全部质因子 p 后的单位部分模 mod。
        if (n == 0)
        {
            return 1;
        }
        int z = mulMod(powerMod(unitPrefix[mod], n / mod, mod), unitPrefix[n % mod], mod);
        return mulMod(z, unitFact(n / p), mod);
    }

  public:
    PPFactorial(int p, int e) : p(p), e(e)
    {
        // p 是质数，e 是正指数；预处理模 p^e 的单位前缀乘积，无返回值。
        assert(p >= 2 && e >= 1); // 调试检查，可删
        for (int i = 0; i < e; i++)
```

```
        {
            assert(mod <= 5000000 / p); // 调试检查，可删
            mod *= p;
        }
        unitPrefix.assign(mod + 1, 1);
        for (int i = 1; i <= mod; i++)
        {
            unitPrefix[i] = mulMod(unitPrefix[i - 1], i % p ? i : 1, mod);
        }
    }

    int modulus() const
    {
        // 无参数；返回构造时确定的质数幂模数 p^e。
        return mod;
    }

    int exponent(int n) const
    {
        // n 是非负整数；返回 n! 中质因子 p 的指数。
        assert(n >= 0); // 调试检查，可删
        int ans = 0;
        while (n)
        {
            n /= p;
            ans += n;
        }
        return ans;
    }

    int fact(int n) const
    {
        // n 是非负整数；返回 n! mod p^e。
        assert(n >= 0); // 调试检查，可删
        int q = exponent(n);
        if (q >= e)
        {
            return 0;
        }
        return mulMod(unitFact(n), powerMod(p, q, mod), mod);
    }

    array<int, 2> split(int n) const
    {
        // n 是非负整数；返回 {n! 的 p-adic 指数，删除 p 因子后的单位部分}。
        assert(n >= 0); // 调试检查，可删
        return {exponent(n), unitFact(n)};
    }
};
```

构造 `PPFactorial q(p, e)`，要求 p 质数、当前打印实现限制 $p^e \le 5 \times 10^6$（预处理开一个 p^e 长的前缀积表）。`modulus()` 返回构造时的 p^e；`fact(n)` 返回 $n! \bmod p^e$，`split(n)` 返回 {p-adic 指数，单位部

分}。预处理 $O(p^e)$ 时空，单次查询 $O(\log_p n)$。验证：模 $2^4 = 16$ 时 `fact(3)=6`、`fact(4)=8`、`fact(6)=0`，`split(6)` 的指数为 4。

7.2.010 哥德巴赫猜想及竞赛用结论

对不小于 4 的偶数 n，筛出 $[0, n]$ 的素数表后从小到大枚举 p，命中第一个使 p 与 $n-p$ 同为素数的拆分即返回。竞赛中猜想在题目给定范围内已由打表验证，这里只做确定性构造、不作为正确性证明。

```
vector<char> primeTable(int n)
{
    // n 是非负闭上界；返回 [0,n] 的素数标记表。
    assert(n >= 0); // 调试检查，可删
    vector<char> is(n + 1, true);
    is[0] = false;
    if (n >= 1)
    {
        is[1] = false;
    }
    for (int i = 2; i <= n / i; i++)
    {
        if (!is[i])
        {
            continue;
        }
        for (int j = i * i; j <= n; j += i)
        {
            is[j] = false;
        }
    }
    return is;
}

pair<int, int> goldbach(int n)
{
    // 参数 n 是不小于 4 的偶数；返回一组和为 n 的素数，未找到时返回 {-1,-1}。
    if (n < 4 || n % 2)
    {
        return {-1, -1};
    }
    auto is = primeTable(n);
    for (int p = 2; p <= n; p++)
    {
        if (is[p] && is[n - p])
        {
            return {p, n - p};
        }
    }
    return {-1, -1};
}
```

`goldbach(n)` 要求 n 为不小于 4 的偶数，返回一组和为 n 的素数 `{p, n-p}`，非法或未找到时返回 `{-1,-1}`。

筛表 $O(n \log \log n)$、搜索 $O(n)$、空间 $O(n)$。

7.2.011 麦乐鸡定理 / 两数 Frobenius 问题

对互质正整数 a, b，不能写成 $ax + by$（$x, y \geq 0$）的最大整数由两数 Frobenius 定理直接给出，为 $ab - a - b$；若 $\gcd(a, b) \neq 1$，则不可表示的数有无穷多个、无最大值。

```
__int128 frobenius2(int a, int b)
{
    // 互质正整数 a、b 的最大不可表示数
    assert(a > 0 && b > 0); // 调试检查，可删
    assert(gcd(a, b) == 1); // 调试检查，可删
    return (__int128)a * b - a - b;
}
```

`frobenius2(a, b)` 要求 a, b 为互质正整数；返回类型为 `__int128`，避免 $ab - a - b$ 在乘法时溢出。非互质输入没有最大不可表示数，不属于接口定义域。公式本身为 $O(1)$ 时间、$O(1)$ 空间；启用调试断言时，互质检查另需 $O(\log \min(a, b))$ 时间。

7.3 算术函数与筛法

7.3.001 欧拉函数

线性筛顺带求 $\varphi(1..n)$：质数 p 处 $\varphi(p) = p-1$；用最小质因子 p 扩展 $i \cdot p$ 时，若 $p \mid i$ 则 $\varphi(ip) = \varphi(i) \cdot p$，否则 $\varphi(ip) = \varphi(i) \cdot (p - 1)$，即 φ 在质数幂上的乘性递推。

```
vector<int> phiTable(int n)
{
    // n 是非负筛表上界；返回 0..n 的欧拉函数值，phi[1]=1。
    vector<int> phi(n + 1), minp(n + 1), primes;
    if (n >= 1)
    {
        phi[1] = 1;
    }
    for (int i = 2; i <= n; i++)
    {
        if (minp[i] == 0)
        {
            minp[i] = i;
            primes.push_back(i);
            phi[i] = i - 1;
        }
        for (auto p : primes)
        {
            if (p > minp[i] || i > n / p)
            {
                break;
            }
```

```
            minp[i * p] = p;
            if (i % p == 0)
            {
                phi[i * p] = phi[i] * p;
                break;
            }
            phi[i * p] = phi[i] * (p - 1);
        }
    }
    return phi;
}
```

`phiTable(n)` 返回 `phi[0..n]`、`phi[1]=1`，$O(n)$ 时空。一般积性函数用 7.3.003 积性函数，整除偏序上的累计与反演见 7.3.005 莫比乌斯反演，大范围前缀和见 7.3.006 杜教筛。

7.3.002 莫比乌斯函数

线性筛求 $\mu(1..n)$：扩展 $i \cdot p$ 时若 $p \mid i$ 则出现平方因子、$\mu(ip)=0$，否则 $\mu(ip)=-\mu(i)$，符号由不同质因子个数的奇偶决定。

```
vector<int> mobiusTable(int n)
{
    // n 是非负筛表上界；返回 0..n 的莫比乌斯函数值，mu[1]=1。
    vector<int> mu(n + 1), minp(n + 1), primes;
    if (n >= 1)
    {
        mu[1] = 1;
    }
    for (int i = 2; i <= n; i++)
    {
        if (minp[i] == 0)
        {
            minp[i] = i;
            primes.push_back(i);
            mu[i] = -1;
        }
        for (auto p : primes)
        {
            if (p > minp[i] || i > n / p)
            {
                break;
            }
            minp[i * p] = p;
            if (i % p == 0)
            {
                mu[i * p] = 0;
                break;
            }
            mu[i * p] = -mu[i];
        }
```

```
    }
    return mu;
}
```

`mobiusTable(n)` 返回 `mu[0..n]`、`mu[1]=1`，位置 0 无数论含义，$O(n)$ 时空。反演见 7.3.005 莫比乌斯反演。

7.3.003 积性函数

给定 f 在质数幂上的值 $f(p^e)$（只需积性、不必完全积性），线性筛批量求 $f(1..n)$。维护每个数的最小质因子 p、其当前指数 e 与幂 p^e：若 $p \mid x$ 就在同一质数上升幂，$f(y) = f(x/p^e) \cdot f(p^{e+1})$；否则新质因子，$f(y) = f(x) \cdot f(p)$。

```
template <class T = long long, class F> vector<T> multTable(int n, F primePower)
{
    // n 是非负上界；primePower(p,e) 返回 f(p^e)，且 f 必须是积性函数；返回 f[0..n]，位置 0 不使用。
    assert(n >= 0); // 调试检查，可删
    vector<T> f(n + 1);
    vector<int> minp(n + 1);
    vector<int> exp(n + 1);
    vector<int> pp(n + 1);
    vector<int> primes;
    if (n >= 1)
    {
        f[1] = 1;
        pp[1] = 1;
    }
    for (int x = 2; x <= n; x++)
    {
        if (minp[x] == 0)
        {
            minp[x] = x;
            exp[x] = 1;
            pp[x] = x;
            primes.push_back(x);
            f[x] = primePower(x, 1);
        }
        for (auto p : primes)
        {
            if (p > minp[x] || x > n / p)
            {
                break;
            }
            int y = x * p;
            minp[y] = p;
            if (p == minp[x])
            {
                exp[y] = exp[x] + 1;
                pp[y] = pp[x] * p;
                f[y] = f[x / pp[x]] * primePower(p, exp[y]);
```

```
                break;
            }
            exp[y] = 1;
            pp[y] = p;
            f[y] = f[x] * primePower(p, 1);
        }
    }
    return f;
}
```

`multTable<T>(n, primePower)` 的回调返回类型 `T` 的 $f(p^e)$；所有质数幂函数值和中间乘积都必须能由 `T` 表示，模意义函数直接令 `T` 为模整数。下标、最小质因子和 $p^e \le n$ 仍使用普通 `int`，不随结果类型扩大。时间、空间均为 $O(n)$；φ、μ 的专用版见 7.3.001 欧拉函数、7.3.002 莫比乌斯函数。

例如令 $f(p^e) = e + 1$，即可线性筛出约数个数：

```
signed main()
{
    auto primePower = [](int p, int e)
    {
        (void)p;
        return e + 1;
    };
    auto d = multTable<int>(6, primePower);
    assert((d == vector<int>{0, 1, 2, 2, 3, 2, 4}));
}
```

7.3.004 Dirichlet 卷积

计算两个算术函数的 Dirichlet 卷积 $(f*g)(n) = \sum_{d\mid n} f(d)\,g(n/d)$。枚举因子 d 再遍历其倍数 $dk \le n$ 累加 $f(d)g(k)$，即调和级数式的 $O(n \log n)$ 求和；单位元 ε 只在 1 处为 1。

```
template <class T> vector<T> dirichlet(const vector<T> &a, const vector<T> &b)
{
    // a、b 在下标 1..n 保存两个算术函数；返回同长度的 Dirichlet 卷积，位置 0 不使用。
    assert(!a.empty() && !b.empty()); // 调试检查，可删
    int n = min(a.size(), b.size()) - 1;
    vector<T> c(n + 1);
    for (int d = 1; d <= n; d++)
    {
        for (int k = 1; k <= n / d; k++)
        {
            c[d * k] += a[d] * b[k];
        }
    }
    return c;
}
```

下标 $1..n$ 保存函数值、位置 0 留空；`dirichlet(a, b)` 两数组长度不同时只算公共上界，$O(n \log n)$ 时间、$O(n)$ 空间。若只需整除前缀和 / 反演，用 7.3.005 莫比乌斯反演 的原地变换更省空间。

7.3.005 莫比乌斯反演

整除偏序上的一对互逆变换。Zeta 为 $g(x)=\sum_{d\mid x} f(d)$。不能按所有 d 直接把已经更新过的 `a[d]` 再加到倍数，否则同一原始约数会沿 $1\mid 2\mid 4$ 等链重复累计；正确原地做法是逐个质数处理一个指数维度。对每个质数 p，按 x 递增加 `a[x]` 到 `a[xp]`，便恰好补上指数少一的状态；逆变换反序减回。

```
template <class T> void divisorZeta(vector<T> &a)
{
    // a 在下标 1..n 保存原函数；原地改成每个位置的约数和，位置 0 不使用。
    assert(!a.empty()); // 调试检查，可删
    int n = a.size() - 1;
    vector<char> comp(n + 1);
    vector<int> p;
    for (int x = 2; x <= n; x++)
    {
        if (!comp[x])
        {
            p.push_back(x);
        }
        for (auto q : p)
        {
            if (q > n / x)
            {
                break;
            }
            comp[x * q] = true;
            if (x % q == 0)
            {
                break;
            }
        }
    }
    for (auto q : p)
    {
        for (int x = 1; x <= n / q; x++)
        {
            a[x * q] += a[x];
        }
    }
}

template <class T> void divisorMu(vector<T> &a)
{
    // a 是 divisorZeta 的结果；原地恢复下标 1..n 的原函数，位置 0 不使用。
    assert(!a.empty()); // 调试检查，可删
    int n = a.size() - 1;
    vector<char> comp(n + 1);
    vector<int> p;
    for (int x = 2; x <= n; x++)
    {
        if (!comp[x])
```

```
        {
            p.push_back(x);
        }
        for (auto q : p)
        {
            if (q > n / x)
            {
                break;
            }
            comp[x * q] = true;
            if (x % q == 0)
            {
                break;
            }
        }
    }
    for (auto it = p.rbegin(); it != p.rend(); it++)
    {
        int q = *it;
        for (int x = n / q; x >= 1; x--)
        {
            a[x * q] -= a[x];
        }
    }
}
```

下标 $1..n$，`divisorZeta(a)` 求约数和、`divisorMu(a)` 由其恢复原函数，二者严格互逆。筛素数加各质数维更新为 $O(n \log\log n)$ 时间，素数表与标记为 $O(n)$ 额外空间。固定例 `a={0,1,0,0,0}` 变换后应为 `{0,1,1,1,1}`，可直接识别旧实现对位置 4 的重复累计。

7.3.006 杜教筛

求积性函数 f 的大范围前缀和 $S(n)=\sum_{i=1}^{n} f(i)$ 而不必线性筛到 n。选一个便于求和的 g，由 $\sum_{i\le n}(f*g)(i)=\sum_{d\le n} g(d)\,S(\lfloor n/d\rfloor)$ 得 $g(1)S(n)=\sum_{i\le n}(f*g)(i)-\sum_{d\ge 2} g(d)\,S(\lfloor n/d\rfloor)$，右端对 d 整除分块递归。此处 μ,φ 都取 $g=1$：$\mu*1=\varepsilon$ 故 $\sum(f*g)=1$，$\varphi*1=\mathrm{id}$ 故 $\sum(f*g)=\frac{n(n+1)}{2}$。预筛到约 $n^{2/3}$，并对 $\lfloor n/i\rfloor$ 的 $O(\sqrt{n})$ 种取值记忆化。

```
struct DuSieve
{
    int n;                          // 预筛上界
    vector<int> p;                  // 预筛范围内的素数
    vector<int> mu;                 // 莫比乌斯函数值
    vector<int> phi;                // 欧拉函数值
    vector<bool> vis;               // 线性筛合数标记
    vector<long long> smu;          // 莫比乌斯函数前缀和
    vector<__int128> sphi;          // 欧拉函数前缀和
    unordered_map<long long, long long> cmu; // 大参数 Mertens 值
    unordered_map<long long, __int128> cphi; // 大参数欧拉函数前缀和

```

```
    DuSieve(long long m)
    {
        // m 是最大查询上界；预筛约 m^(2/3) 内的 mu、phi 及其前缀和。
        assert(m >= 0); // 调试检查，可删
        n = max(1LL, (long long)(powl(m, 2.0L / 3) + 2));
        mu.resize(n + 1);
        phi.resize(n + 1);
        vis.resize(n + 1);
        smu.resize(n + 1);
        sphi.resize(n + 1);
        mu[1] = phi[1] = 1;
        for (int i = 2; i <= n; i++)
        {
            if (!vis[i])
            {
                p.push_back(i);
                mu[i] = -1;
                phi[i] = i - 1;
            }
            for (int q : p)
            {
                if (1LL * i * q > n)
                {
                    break;
                }
                vis[i * q] = true;
                if (i % q == 0)
                {
                    phi[i * q] = phi[i] * q;
                    break;
                }
                mu[i * q] = -mu[i];
                phi[i * q] = phi[i] * (q - 1);
            }
        }
        for (int i = 1; i <= n; i++)
        {
            smu[i] = smu[i - 1] + mu[i];
            sphi[i] = sphi[i - 1] + phi[i];
        }
    }

    long long sumMu(long long x)
    {
        // x 是查询点；返回 sum_{i=1}^x mu(i)，超出预筛范围时整除分块递归并记忆化。
        assert(x >= 0); // 调试检查，可删
        if (x <= n)
        {
            return smu[x];
        }
        if (cmu.contains(x))
        {
```

```
            return cmu[x];
        }
        __int128 ans = 1;
        long long l = 2;
        while (l <= x)
        {
            long long r = x / (x / l);
            ans -= (__int128)(r - l + 1) * sumMu(x / l);
            if (r == x)
            {
                break;
            }
            l = r + 1;
        }
        return cmu[x] = (long long)ans;
    }

    __int128 sumPhi(long long x)
    {
        // x 是查询点；返回 sum_{i=1}^x phi(i)，超出预筛范围时整除分块递归并记忆化。
        assert(x >= 0); // 调试检查，可删
        if (x <= n)
        {
            return sphi[x];
        }
        if (cphi.contains(x))
        {
            return cphi[x];
        }
        __int128 ans = (__int128)x * ((__int128)x + 1) / 2;
        long long l = 2;
        while (l <= x)
        {
            long long r = x / (x / l);
            ans -= (__int128)(r - l + 1) * sumPhi(x / l);
            if (r == x)
            {
                break;
            }
            l = r + 1;
        }
        return cphi[x] = ans;
    }
};
```

构造 `DuSieve S(n)`（n 是最大查询上界）后调用 `S.sumMu(x)` / `S.sumPhi(x)`，要求 $0 \le x \le n$。`sumPhi` 返回 `__int128`，三角和、块长乘法和缓存都保持宽类型；`sumMu` 的中间差也先提升再转回其必在 $[-x, x]$ 内的结果。预筛 $O(n^{2/3})$ 时空；缓存后同参查询 $O(1)$。更一般的积性函数前缀和用 7.3.007 Min_25 筛。

7.3.007 Min_25 筛

求积性函数 f 的前缀和 $\sum_{i=1}^{n} f(i)$，要求 f 在质数处等于一个低次多项式、在质数幂处可快速求值。分两步：先对每个不同的 $\lfloor n/i \rfloor$ 求「不超过它的素数的幂和」——`primeCount` 求素数个数、`primeSum` 求素数一次和，逐个质数扣掉最小质因子恰为该质数的合数贡献；再 `dfs` 枚举每个质数幂，把质数幂上的函数值与其后缀素数贡献拼起来。本模板取质数处 $f(p) = A + Bp$，由 `linearPPow(A,B)` 求和。

```
struct Min25
{
    long long n;   // 前缀和上界
    long long sq;  // sq 是 floor(sqrt(n))。
    long long hi;  // 大商与小商的分界
    long long s;   // 不同整除商的数组长度
    vector<int> p; // 不超过 sq 的素数
    vector<Z> buf; // 各离散商处的素数 f 值前缀和
    Z A;           // f(p^e)=A*e+B*p 的指数系数
    Z B;           // f(p^e)=A*e+B*p 的质数系数
    Z ans;         // 当前查询的答案

    Min25(long long n) : n(n), sq(0)
    {
        // n 是前缀和上界（n>0）；筛出不超过 sqrt(n) 的素数并预置离散商的分界。
        assert(n > 0); // 调试检查，可删
        sq = sqrtl(n);
        while ((sq + 1) <= n / (sq + 1))
        {
            sq++;
        }
        while (sq > n / sq)
        {
            sq--;
        }
        hi = n ? n / sq : 0;
        if (hi != 1 && n / (hi - 1) == sq)
        {
            hi--;
        }
        s = hi + sq;
        vector<bool> isp(sq + 1, true);
        for (int i = 2; i <= sq; i++)
        {
            if (isp[i])
            {
                p.push_back(i);
                if (1LL * i * i <= sq)
                {
                    for (int j = i * i; j <= sq; j += i)
                    {
                        isp[j] = false;
                    }
                }
```

```
            }
        }
    }

    int id(long long x) const
    {
        // x 是某个整除商 n/l；返回它在离散数组中的下标。
        return x <= sq ? s - x : n / x;
    }

    vector<Z> primeCount() const
    {
        // 无参数；返回各离散整除商对应的不超过该值的素数个数。
        vector<Z> h(s);
        for (long long x = 1; x < s; x++)
        {
            long long v = x < hi ? n / x : s - x;
            h[x] = v - 1;
        }
        int cnt = 0;
        for (long long q : p)
        {
            for (long long x = 1; x < s; x++)
            {
                long long v = x < hi ? n / x : s - x;
                if (v < q * q)
                {
                    break;
                }
                h[x] -= h[id(v / q)] - cnt;
            }
            cnt++;
        }
        return h;
    }

    vector<Z> primeSum() const
    {
        // 无参数；返回各离散整除商对应的不超过该值的素数和。
        vector<Z> h(s);
        for (long long x = 1; x < s; x++)
        {
            long long v = x < hi ? n / x : s - x;
            if (v & 1)
            {
                h[x] = Z(v) * Z(v / 2 + 1) - 1;
            }
            else
            {
                h[x] = Z(v / 2) * Z(v + 1) - 1;
            }
        }
```

```
        for (long long q : p)
        {
            Z before = h[id(q - 1)];
            for (long long x = 1; x < s; x++)
            {
                long long v = x < hi ? n / x : s - x;
                if (v < q * q)
                {
                    break;
                }
                h[x] -= Z(q) * (h[id(v / q)] - before);
            }
        }
        return h;
    }

    Z f(long long q, int e) const
    {
        // q 是质数 p，e 是指数；返回质数幂上的函数值 f(p^e)=A·e+B·p。
        return A * e + B * q;
    }

    void dfs(int i, int e, long long prod, Z cur)
    {
        // i、e 指当前质数及指数，prod 是已选乘积，cur 是此前质因子贡献；递归累加后续贡献。
        ans += cur * f(p[i], e + 1);
        long long lim = n / prod;
        if (lim >= 1LL * p[i] * p[i])
        {
            dfs(i, e + 1, prod * p[i], cur);
        }
        cur *= f(p[i], e);
        ans += cur * (buf[id(lim)] - buf[id(p[i])]);
        for (int j = i + 1; j < (int)p.size() && 1LL * p[j] * p[j] <= lim; j++)
        {
            dfs(j, 1, prod * p[j], cur);
        }
    }

    Z linearPPow(Z a, Z b)
    {
        // 对 f(p^e)=a*e+b*p，返回 sum_{i=1}^n f(i)
        A = a;
        B = b;
        auto c = primeCount();
        auto d = primeSum();
        buf.resize(s);
        for (int i = 0; i < s; i++)
        {
            buf[i] = c[i] * A + d[i] * B;
        }
        ans = buf[id(n)] + 1;
```

```
        for (int i = 0; i < (int)p.size(); i++)
        {
            dfs(i, 1, p[i], 1);
        }
        return ans;
    }
};
```

依赖 1.1.008 静态取模类 的取模类 `Z`（本节不重复打印）。构造 `Min25 q(n)` 后，`q.linearPPow(A,B)` 只适用于完整定义为 $f(p^e)=Ae+Bp$ 的积性函数；只知道质数处 $f(p)=A+Bp$ 不足以确定质数幂，不能据此套用。离散商按对应数值递减扫描，低于 p^2 后立即结束该质数的更新，表长 $O(\sqrt{n})$，本通用递归版约 $O\left(\frac{n^{3/4}}{\log n}\right)$ 时间、$O(\sqrt{n})$ 空间。只求 μ,φ 前缀和时 7.3.006 杜教筛 更简单。

7.3.008 Meissel-Lehmer 质数计数

单点求 $\pi(n)$（不超过 n 的素数个数）而不枚举全部素数，适合少量大整数查询。核心是 Legendre 函数 $\phi(x,a)$——$[1,x]$ 中不被前 a 个素数整除的数的个数，满足 $\phi(x,a)=\phi(x,a-1)-\phi(\lfloor x/p_a\rfloor,a-1)$。Lehmer 公式把 $\pi(x)$ 写成 $\phi(x,a)$ 加上跨 $\sqrt[4]{x},\sqrt[3]{x},\sqrt{x}$ 若干区间的修正项，从而递归到更小规模；不超过 N 的部分预筛 `pi` 直接查表。

```
// 依赖 7.1.014 整数 k 次方根
struct LehmerPi
{
    // N 是预筛上界，不超过 N 的查询直接用 pi 表回答。
    static constexpr int N = 5000000;
    // p 是不超过 N 的素数表；pi[x] 是不超过 x 的素数个数（x<=N）。
    vector<int> p, pi;
    // memPhi 缓存 Legendre 函数 phi(x,s) 的值。
    unordered_map<unsigned long long, long long> memPhi;
    // memPi 缓存大参数处的 pi(x)。
    unordered_map<long long, long long> memPi;

    LehmerPi() : pi(N + 1)
    {
        // 无参数；线性筛出不超过 N 的素数及其计数前缀 pi。
        vector<bool> c(N + 1);
        for (int i = 2; i <= N; i++)
        {
            if (!c[i])
            {
                p.push_back(i);
            }
            for (int q : p)
            {
                if (1LL * i * q > N)
                {
                    break;
                }
                c[i * q] = true;
```

```
                if (i % q == 0)
                {
                    break;
                }
            }
            pi[i] = p.size();
        }
    }

    long long phi(long long x, int s)
    {
        // Legendre phi: 返回 [1,x] 中不被前 s 个素数整除的整数个数。
        assert(x >= 0 && 0 <= s && s <= (int)p.size()); // 调试检查，可删
        if (!x)
        {
            return 0;
        }
        if (!s)
        {
            return x;
        }
        if (s == 1)
        {
            return x - x / 2;
        }
        if (x <= N && 1LL * p[s - 1] * p[s - 1] > x)
        {
            return max(1LL, pi[x] - s + 1);
        }
        if (s < 100 && x < 100000)
        {
            unsigned long long k = (unsigned long long)x << 7 | s;
            if (auto it = memPhi.find(k); it != memPhi.end())
            {
                return it->second;
            }
            return memPhi[k] = phi(x, s - 1) - phi(x / p[s - 1], s - 1);
        }
        return phi(x, s - 1) - phi(x / p[s - 1], s - 1);
    }

    long long operator()(long long x)
    {
        // x 是查询点；返回 pi(x)= 不超过 x 的素数个数。
        assert(0 <= x && x <= (long long)N * N); // 调试检查，可删
        if (x <= N)
        {
            return pi[x];
        }
        if (auto it = memPi.find(x); it != memPi.end())
        {
            return it->second;
```

```
        }
        long long a = (*this)(kthRoot((unsigned long long)x, 4));
        long long b = (*this)(kthRoot((unsigned long long)x, 2));
        long long c = (*this)(kthRoot((unsigned long long)x, 3));
        long long ans = phi(x, a) + (b + a - 2) * (b - a + 1) / 2;
        for (long long i = a; i < b; i++)
        {
            long long w = x / p[i];
            ans -= (*this)(w);
            if (i < c)
            {
                long long z = (*this)(kthRoot((unsigned long long)w, 2));
                for (long long j = i; j < z; j++)
                {
                    ans -= (*this)(w / p[j]) - j;
                }
            }
        }
        return memPi[x] = ans;
    }
};
```

构造 `LehmerPi pi;` 一次性预筛 $N = 5 \times 10^6$，随后 `pi(x)` 返回 $\pi(x)$。固定表版本要求 $0 \le x \le N^2 = 2.5 \times 10^{13}$，保证外层访问的全部 $\le \sqrt{x}$ 素数已经筛出；更大的查询必须把 N 随最大输入提高，不能沿用本表。各整数根调用 7.1.014 整数 k 次方根 的 `kthRoot` 校正。预筛 $O(N)$ 时空，单次查询约 $O\left(\frac{x^{2/3}}{\log x}\right)$。

7.3.009 整除分块

把 $\lfloor n/i \rfloor$ 相等的连续下标并成一块：固定 n 时商在 $[l,\ \lfloor n/\lfloor n/l \rfloor \rfloor]$ 内不变，每次跳到该块右端加一，只需枚举约 $2\sqrt{n}$ 个块。`divBlocks` 返回每块 $[l, r]$ 及其公共商，`quotients` 只取不同商值（升序）。

```
struct DivBlock
{
    // l、r 是该块闭区间的左右端点；q 是块内公共商 floor(n/l)。
    long long l, r, q;
    bool operator==(const DivBlock &) const = default;
};

vector<DivBlock> divBlocks(long long n)
{
    // n 是正整数；返回所有极大闭区间 [l,r]，段内 floor(n/i) 同为 q。
    assert(n > 0); // 调试检查，可删
    vector<DivBlock> a;
    long long l = 1;
    while (l <= n)
    {
        long long q = n / l;
        long long r = n / q;
        a.push_back({l, r, q});
        if (r == n)
```

```
        {
            break;
        }
        l = r + 1;
    }
    return a;
}

vector<long long> quotients(long long n)
{
    // n 是正整数；返回 floor(n/i) 的全部不同取值（升序）。
    vector<long long> a;
    for (const auto &block : divBlocks(n))
    {
        a.push_back(block.q);
    }
    reverse(a.begin(), a.end());
    return a;
}
```

`divBlocks(n)` 返回 `{l, r, q}` 列表，每段对整除型求和贡献 $(r-l+1)F(q)$；`quotients(n)` 返回 $\lfloor n/i \rfloor$ 的不同取值。块数不超过 $2\lfloor\sqrt{n}\rfloor$，$O(\sqrt{n})$ 时间与空间。它是 7.3.006 杜教筛 递归的基础。

7.3.010 调和级数分块

这里的“调和级数分块”与 7.3.009 整除分块 是同一分解——把 $\lfloor n/l \rfloor$ 相同的下标合成极大闭区间；名字只是强调按每段贡献 $\sum_q F(q)\cdot(\text{段长})$ 求和时，段数与调和级数求和的阶数相当。直接复用该条的 `divBlocks(n)`（每段 `{l, r, q}`，$q=\lfloor n/l \rfloor$），整段贡献 $(r-l+1)F(q)$。例如 $n=10$ 依次得到 $[1,1,10],[2,2,5],[3,3,3],[4,5,2],[6,10,1]$。

> 备注：本条不再单独实现。原资料这里有一份返回 `array<int,3>`、参数为 `int` 的 `divBlocks`，但在本章 `#define int long long` 下 `int` 与 `long long` 是同一类型，它与 7.3.009 整除分块 的 `divBlocks(long long)` 构成同签名重定义、无法同时编译，故合并为引用。

7.3.011 素数、$\omega(n)$ 与 $d(n)$

线性筛同时递推三个量：不同质因子数 ω、含重数质因子数 Ω、约数个数 d。用最小质因子 p 扩展 $y=xp$ 时，若 $p \mid x$（升幂，旧指数 e）则 ω 不变、$\Omega{+}1$，$d(y)=d(x)/(e{+}1)\cdot(e{+}2)$；否则遇新质因子，$\omega,\Omega$ 各 $+1$、d 翻倍。

```
vector<array<int, 3>> factorStats(int n)
{
    // n 是筛表上界；返回 s[x]={omega(x),Omega(x),d(x)}，依次为不同质因子数、含重数质因子数、约数数。
    vector<array<int, 3>> s(n + 1);
    vector<int> minp(n + 1), exp(n + 1), primes;
    if (n >= 1)
    {
        s[1] = {0, 0, 1};
```

```
    }
    for (int x = 2; x <= n; x++)
    {
        if (minp[x] == 0)
        {
            minp[x] = x;
            exp[x] = 1;
            primes.push_back(x);
            s[x] = {1, 1, 2};
        }
        for (auto p : primes)
        {
            if (p > minp[x] || x > n / p)
            {
                break;
            }
            int y = x * p;
            minp[y] = p;
            if (p == minp[x])
            {
                exp[y] = exp[x] + 1;
                s[y] = {s[x][0], s[x][1] + 1, s[x][2] / (exp[x] + 1) * (exp[y] + 1)};
                break;
            }
            exp[y] = 1;
            s[y] = {s[x][0] + 1, s[x][1] + 1, s[x][2] * 2};
        }
    }
    return s;
}
```

`factorStats(n)` 返回 `s[0..n]`，`s[x]={omega(x), Omega(x), d(x)}`，`s[1]={0,0,1}`，$O(n)$ 时空。单个大整数可先用 7.2.004 Pollard-Rho 分解，再由质因子表直接算出这些统计量。

7.3.012 商集上的 Dirichlet 卷积与除法

固定上界 N 后，$\lfloor N/i \rfloor$ 只有 $O(\sqrt{N})$ 个不同值。本条的数组在这些商值上保存算术函数前缀和：第 `i` 项表示 $F(v_i)=\sum_{k\le v_i} f(k)$，而不是 `f(i)`。利用小参数与大商值分治，可直接得到 $f*g$ 的同类前缀和表；若 $f(1)$ 可逆，还能由 $H=F*G$ 反解 G。这适合杜教筛、Min_25 一类只访问整除商集的批量运算。

```
// 依赖 1.1.008 静态取模类
template <int Mod> using StaticModInt = MInt<(int)Mod>;

struct QuotDirich
{
    unsigned long long lim, sq, t; // lim 是 N, sq 是 floor(sqrt N), t 是小值段 1..t 的上界。
    int n; // n 是 lim/i 的不同取值个数（表长）。

```

```
    static unsigned long long isqrt(unsigned long long x)
    {
        // x 是非负整数；返回 floor(sqrt(x))。
        unsigned long long r = sqrtl((long double)x);
        while (r + 1 <= x / (r + 1))
        {
            r++;
        }
        while (r && r > x / r)
        {
            r--;
        }
        return r;
    }

    static unsigned long long icbrt(unsigned long long x)
    {
        // x 是非负整数；返回 floor(cuberoot(x))。
        unsigned long long r = cbrtl((long double)x);
        auto ok = [&](unsigned long long y)
        {
            return y == 0 || y <= x / y / y;
        };
        while (ok(r + 1))
        {
            r++;
        }
        while (!ok(r))
        {
            r--;
        }
        return r;
    }

    QuotDirich(unsigned long long N) : lim(N)
    {
        // N 是固定查询上界；建立其全部整除商值的双端下标。
        assert(1 <= N && N <= (1ULL << 50)); // 调试检查，可删。
        sq = isqrt(N);
        t = sq * sq + sq <= N ? sq : sq - 1;
        n = t + sq + 1;
    }

    int id(unsigned long long x) const
    {
        // x 必须是 lim/i 的某个取值；返回它在表中的下标。
        return x <= t ? x : n - lim / x;
    }

    unsigned long long value(int i) const
    {
        // i 是表下标；返回对应商值。
```

```
        return i <= (int)t ? i : lim / (n - i);
    }

    template <class T, class F> vector<T> sumTable(F prefix) const
    {
        // prefix(x) 返回原函数在 1..x 的前缀和；生成商集表。
        vector<T> a(n);
        for (int i = 1; i < n; i++)
        {
            a[i] = prefix(value(i));
        }
        return a;
    }

    template <class T> vector<T> conv(vector<T> F, vector<T> G) const
    {
        // F、G 是两函数的商集前缀和表；返回 Dirichlet 卷积的商集前缀和表。
        assert((int)F.size() == n && (int)G.size() == n); // 调试检查，可删。
        if (lim == 1)
        {
            return {T(0), F[1] * G[1]};
        }
        vector<T> f(n), g(n), H(n);
        for (int i = 1; i < n; i++)
        {
            f[i] = F[i] - F[i - 1];
            g[i] = G[i] - G[i - 1];
        }
        unsigned long long k = icbrt(lim);
        unsigned long long s = k * k;
        for (unsigned long long a = 1; a <= k; a++)
        {
            H[id(a * a)] += f[a] * g[a];
            if (a * (a + 1) <= t)
            {
                for (unsigned long long b = a + 1; b <= t / a; b++)
                {
                    H[a * b] += f[a] * g[b] + f[b] * g[a];
                }
            }
            unsigned long long q = min(s / a, t);
            for (unsigned long long b = max(a, t / a) + 1; b <= q; b++)
            {
                H[id(a * b)] += f[a] * g[b] + f[b] * g[a];
            }
            if (lim / sq <= s / a)
            {
                unsigned long long p = lim / (s / a + 1) + 1;
                for (unsigned long long b = p; b <= sq; b++)
                {
                    H[id(a * (lim / b))] += f[a] * g[n - b] + g[a] * f[n - b];
                }
```

```
            }
        }
        for (int i = 1; i < n; i++)
        {
            H[i] += H[i - 1];
        }
        for (unsigned long long z = 1; lim / z > s; z++)
        {
            unsigned long long m = lim / z;
            unsigned long long u = isqrt(m);
            H[n - z] = T(0);
            for (unsigned long long a = 1; a <= u; a++)
            {
                int j = id(m / a);
                H[n - z] += f[a] * G[j] + g[a] * F[j];
            }
            H[n - z] -= F[u] * G[u];
        }
        return H;
    }

    template <class T> vector<T> divide(vector<T> H, vector<T> F) const
    {
        // H、F 是商集前缀和表且 f(1) 可逆；返回满足 H=F*G 的 G 表。
        assert((int)H.size() == n && (int)F.size() == n && F[1] != T(0)); // 调试检查，可删。
        if (lim == 1)
        {
            return {T(0), H[1] / F[1]};
        }
        T inv = T(1) / F[1];
        for (auto &x : F)
        {
            x *= inv;
        }
        vector<T> f(n), g(n), h(n);
        for (int i = 1; i < n; i++)
        {
            f[i] = F[i] - F[i - 1];
            h[i] = H[i] - H[i - 1];
        }
        unsigned long long k = icbrt(lim);
        unsigned long long s = max(sq, k * k);
        g[1] = H[1];
        for (int i = 2; i < n; i++)
        {
            unsigned long long a = value(i);
            if (a > s)
            {
                break;
            }
            g[i] = h[i] - g[1] * f[i];
            if (a * a <= s)
```

```
            {
                h[id(a * a)] -= f[i] * g[i];
            }
            unsigned long long u = min<unsigned long long>(i - 1, s / a);
            for (unsigned long long b = 2; b <= u; b++)
            {
                h[id(a * b)] -= f[i] * g[b] + f[b] * g[i];
            }
        }
        vector<T> G(n);
        for (int i = 1; i < n; i++)
        {
            G[i] = G[i - 1] + g[i];
        }
        for (unsigned long long z = lim / (s + 1); z > 0; z--)
        {
            unsigned long long m = lim / z;
            unsigned long long u = isqrt(m);
            G[n - z] = H[n - z] - g[1] * F[n - z] + F[u] * G[u];
            for (unsigned long long a = 2; a <= u; a++)
            {
                int j = id(m / a);
                G[n - z] -= f[a] * G[j] + g[a] * F[j];
            }
        }
        for (auto &x : G)
        {
            x *= inv;
        }
        return G;
    }
};
```

先构造 `QuotDirich q(N)`，再用 `q.sumTable<T>(prefix)` 生成表；`q.conv(F,G)` 求卷积，`q.divide(H,F)` 反解且要求 $f(1)$ 在 `T` 中可逆。固定素数模直接使用 1.1.008 静态取模类 的 `MInt<P>`；`StaticModInt<Mod>` 只是兼容旧调用名的单行别名，不维护第二份取模实现。这里公开限制 $1 \le N \le 2^{50}$，时间约 $O(N^{2/3})$、空间 $O(\sqrt{N})$；实际常数和最优分界会随平台变化。小 N 直接使用 7.3.004 Dirichlet 卷积 更简单。

下面取常数函数 $f(n)=g(n)=1$；其卷积为约数个数 $d(n)$，并由卷积表反解除原函数：

```
signed main()
{
    QuotDirich q(10);
    auto F = q.sumTable<Z>([](unsigned long long x)
    {
        return Z(x);
    });
    auto H = q.conv(F, F);
    assert(q.divide(H, F) == F);
    assert(H[q.id(1)].val() == 1);
    assert(H[q.id(2)].val() == 3);
```

```
    assert(H[q.id(3)].val() == 5);
    assert(H[q.id(5)].val() == 10);
    assert(H[q.id(10)].val() == 27);
}
```

7.3.013 无平方因子数计数

无平方因子数的指标为 $\sum_{k^2|x}\mu(k)$，故不超过 n 的个数

$$Q(n)=\sum_{k\le\sqrt{n}}\mu(k)\left\lfloor\frac{n}{k^2}\right\rfloor.$$

取 $I\approx n^{1/5}$、$D=\lfloor\sqrt{n/I}\rfloor$，$k\le D$ 直接求和；余下部分按 $i=\lfloor n/k^2\rfloor<I$ 分组并望远镜化为 $\sum_{i=1}^{I-1}M(\lfloor\sqrt{n/i}\rfloor)-(I-1)M(D)$。这些大 Mertens 值再用 $M(x)=1-\sum_{j=2}^{x}M(\lfloor x/j\rfloor)$ 递推；所需的大商恰好已经按 i 从大到小算出。为控制 $n\le10^{18}$ 时的内存，μ 用 `int`，$M(0..D)$ 用 `int`。

```
long long sqrtFloor(long long n)
{
    // n 是非负整数；返回 floor(sqrt(n))，先用浮点估计再整数校正。
    long long x = sqrtl((long double)n);
    while ((x + 1) <= n / (x + 1))
    {
        x++;
    }
    while (x > n / x)
    {
        x--;
    }
    return x;
}

vector<int> squarefreeMu(int n)
{
    // n 是上界；返回 Mobius 函数值，mu[1..n] 有效（mu[0] 无意义）。
    vector<int> mu(n + 1, 1);
    vector<bool> composite(n + 1);
    for (int p = 2; p <= n; p++)
    {
        if (composite[p])
        {
            continue;
        }
        mu[p] = -1;
        for (int j = p * 2; j <= n; j += p)
        {
            composite[j] = true;
            mu[j] = -mu[j];
        }
        if (p <= n / p)
        {
```

```
            for (int j = p * p; j <= n; j += p * p)
            {
                mu[j] = 0;
            }
        }
    }
    return mu;
}

long long countSqFree(long long n)
{
    // n 是上界；返回 [1,n] 中无平方因子数的个数，n<=0 时为 0。
    if (n <= 0)
    {
        return 0;
    }
    int I = max<long long>(1, powl((long double)n, 0.2L));
    int D = sqrtFloor(n / I);
    auto mu = squarefreeMu(D);
    vector<int> mertens(D + 1);
    long long direct = 0;
    for (int k = 1; k <= D; k++)
    {
        mertens[k] = mertens[k - 1] + mu[k];
        direct += (long long)mu[k] * (n / k / k);
    }
    mu.clear();
    mu.shrink_to_fit();

    vector<long long> large;
    large.reserve(I - 1);
    long long sum = 0;
    for (int i = I - 1; i >= 1; i--)
    {
        long long x = sqrtFloor(n / i);
        long long s = sqrtFloor(x);
        long long cur = 1;
        for (int q = 1; q <= x / (s + 1); q++)
        {
            cur -= (x / q - x / (q + 1)) * mertens[q];
        }
        for (int j = 2; j <= s; j++)
        {
            long long y = x / j;
            if (y <= D)
            {
                cur -= mertens[y];
            }
            else
            {
                cur -= large[I - j * j * i - 1];
            }
```

```
        }
        large.push_back(cur);
        sum += cur;
    }
    return direct + sum - (I - 1) * (long long)mertens[D];
}
```

`countSqFree(n)` 返回 $[1,n]$ 中无平方因子的正整数个数，$n \le 0$ 返回 0。时间约 $O(n^{2/5}\log\log n)$、空间 $O(n^{2/5})$；$n = 10^{18}$ 时结果仍不超过 n，64 位足够。小上界已经有 7.3.002 莫比乌斯函数 的线性筛，本条用于无法筛到 $\sqrt{n}$ 的范围。

下面检查小范围边界，并保留 10^{18} 的已核对结果：

```
signed main()
{
    assert(countSqFree(0) == 0);
    assert(countSqFree(1) == 1);
    assert(countSqFree(10) == 7);
    assert(countSqFree(100) == 61);
    assert(countSqFree(1000000000000000000LL) == 607927101854022750LL);
}
```

7.4 离散对数与剩余

7.4.001 BSGS / exBSGS

求最小非负 x 使 $a^x \equiv b \pmod m$。BSGS 要求 $\gcd(a,m)=1$：取 $n=\lceil\sqrt{m}\rceil$，哈希表存小步 $a^j\ (0\le j<n)$，再按大步枚举 i 匹配 $b\cdot a^{-ni}$，命中即 $x=ni+j$。exBSGS 去掉互质要求——反复约去 $\gcd(a,m)$ 并累计指数偏移，直到底数与模互质再转 BSGS。依赖 7.1.001 快速幂 的 `mulMod`、`powerMod`，7.1.002 GCD、快速 GCD、LCM 的 `binaryGcd`，7.1.003 扩展欧几里得 的 `invMod` 及 7.1.014 整数 k 次方根 的 `kthRoot`。

```
// 依赖 7.1.002 GCD、快速 GCD、LCM
// 依赖 7.1.003 扩展欧几里得
// 依赖 7.1.014 整数 k 次方根
optional<long long> bsgs(long long a, long long b, long long m,
                         long long ord = -1)
{
    // 在 [0,ord) 内求最小解 x；ord=-1 时使用通用上界 m，无解返回空。
    assert(m > 0); // 调试检查，可删
    if (m == 1)
    {
        return 0;
    }
    a %= m;
    b %= m;
    if (a < 0)
    {
        a += m;
```

```
    }
    if (b < 0)
    {
        b += m;
    }
    assert(binaryGcd(a, m) == 1); // 调试检查，可删
    if (ord == -1)
    {
        ord = m;
    }
    assert(ord > 0); // 调试检查，可删
    long long n = kthRoot((unsigned long long)ord, 2);
    if (n <= (ord - 1) / n && n * n < ord)
    {
        n++;
    }
    unordered_map<long long, long long> baby;
    long long x = 1;
    for (long long j = 0; j < n; j++)
    {
        if (!baby.contains(x))
        {
            baby[x] = j;
        }
        x = mulMod(x, a, m);
    }
    long long step = powerMod(*invMod(a, m), n, m);
    x = b;
    long long rounds = ord / n + (ord % n != 0);
    for (long long i = 0; i < rounds; i++)
    {
        if (auto it = baby.find(x); it != baby.end())
        {
            long long j = it->second;
            if (j < ord && i <= (ord - 1 - j) / n)
            {
                return i * n + j;
            }
        }
        x = mulMod(x, step, m);
    }
    return nullopt;
}

optional<long long> exbsgs(long long a, long long b, long long m)
{
    // a 是底数，b 是右端值，m 是模数（不要求与 a 互质）；返回满足 a^x=b (mod m) 的最小非负 x，不存在时返回空。
    assert(m > 0); // 调试检查，可删
    a %= m;
    b %= m;
    if (a < 0)
```

```
    {
        a += m;
    }
    if (b < 0)
    {
        b += m;
    }
    if (m == 1 || b == 1 % m)
    {
        return 0;
    }
    long long k = 1;
    long long z = 0;
    while (true)
    {
        long long g = (long long)binaryGcd(a, m);
        if (g == 1)
        {
            break;
        }
        if (b == k)
        {
            return z;
        }
        if (b % g)
        {
            return nullopt;
        }
        b /= g;
        m /= g;
        z++;
        k = mulMod(k, a / g, m);
    }
    if (m == 1)
    {
        return z;
    }
    auto x = bsgs(a, mulMod(b, *invMod(k, m), m), m);
    if (!x)
    {
        return nullopt;
    }
    return *x + z;
}
```

`bsgs(a,b,m)` 要求 $\gcd(a,m)=1$，默认在 $[0,m)$ 搜索；已知底数阶不超过 `ord` 时传第四参，把时空降为 $O(\sqrt{ord})$。模 1 的最小解约定为 0。`exbsgs` 无互素要求，返回最小非负解或空。哈希操作为期望 $O(1)$；固定群阶可分解时用 7.4.002 Pohlig-Hellman。

7.4.002 Pohlig-Hellman

当模素数 p 的乘法群阶 $p-1=\prod q_i^{e_i}$ 只含较小质因子时，把离散对数分解到各质数幂阶子群求解：对每个 $q_i^{e_i}$，用 $g^{(p-1)/q_i^{e_i}}$、$h^{(p-1)/q_i^{e_i}}$ 在 $q_i^{e_i}$ 阶子群里跑 BSGS 得 $x \bmod q_i^{e_i}$，再逐步 CRT 合并成 $x \bmod (p-1)$。依赖 7.1.001 快速幂 的 `powerMod`、7.1.003 扩展欧几里得 的 `invMod` 与 7.4.001 BSGS / exBSGS 的 `bsgs`。

```cpp
// 依赖 7.4.001 BSGS / exBSGS
optional<int> PH(int g, int h, int p)
{
    // g 是模素数 p 的原根，h 是非零剩余；返回最小非负 x 使 g^x=h，无解返回空。
    assert(p >= 2); // 调试检查，可删
    int left = p - 1;
    int x = 0;
    int curMod = 1;
    vector<int> fac;
    for (int q = 2; q <= left / q; q++)
    {
        if (left % q)
        {
            continue;
        }
        int pe = 1;
        while (left % q == 0)
        {
            left /= q;
            pe *= q;
        }
        fac.push_back(pe);
    }
    if (left > 1)
    {
        fac.push_back(left);
    }
    for (auto pe : fac)
    {
        int a = powerMod(g, (p - 1) / pe, p);
        int b = powerMod(h, (p - 1) / pe, p);
        auto y = bsgs(a, b, p, pe);
        if (!y)
        {
            return nullopt;
        }
        int t = (*y - x) % pe;
        if (t < 0)
        {
            t += pe;
        }
        auto inv = invMod(curMod % pe, pe);
        if (!inv)
        {
            return nullopt;
```

```
        }
        t = mulMod(t, *inv, pe);
        x += curMod * t;
        curMod *= pe;
        x %= curMod;
    }
    h %= p;
    if (h < 0)
    {
        h += p;
    }
    return powerMod(g, x, p) == h ? optional<int>(x) : nullopt;
}
```

`PH(g, h, p)`：p 素数、g 为模 p 原根、h 非零，返回最小非负 x 使 $g^x \equiv h$，无解返回 `nullopt`。时间约 $O(\sum_i \sqrt{q_i^{e_i}})$、空间 $O(\max_i \sqrt{q_i^{e_i}})$；$p-1$ 含大质因子时该质数幂上的 BSGS 仍可能慢。原根由 7.4.003 原根与全部原根筛 求。

7.4.003 原根与全部原根筛

模素数 p 的原根 g 是满足「对 $p-1$ 的每个不同质因子 q 都有 $g^{(p-1)/q} \neq 1$」的最小正整数；枚举 $g = 2, 3, \ldots$ 逐个验证即可（最小原根通常很小）。取到一个原根后，全部原根为 g^k（$\gcd(k, p-1) = 1$），共 $\varphi(p-1)$ 个。依赖 7.1.001 快速幂 的 `powerMod` 和 7.2.004 Pollard-Rho 的 `factor64`；64 位输入不能用试除法分解 $p-1$。

```
// 依赖 7.2.004 Pollard-Rho
long long primRoot(long long p)
{
    // p 是素数；返回模 p 的最小正原根，p=2 时返回 1。
    assert(p >= 2); // 调试检查，可删
    if (p == 2)
    {
        return 1;
    }
    long long n = p - 1;
    auto raw = factor64(n);
    raw.erase(unique(raw.begin(), raw.end()), raw.end());
    for (long long g = 2;; g++)
    {
        bool ok = true;
        for (auto d : raw)
        {
            if (powerMod(g, n / d, p) == 1)
            {
                ok = false;
            }
        }
        if (ok)
        {
```

```
            return g;
        }
    }
}

vector<long long> primRoots(long long p)
{
    // p 是素数；返回模 p 的全部原根并按数值升序排列。
    long long g = primRoot(p);
    long long n = p - 1;
    vector<long long> a;
    for (long long k = 1; k <= n; k++)
    {
        if (gcd(k, n) == 1)
        {
            a.push_back(powerMod(g, k, p));
        }
    }
    sort(a.begin(), a.end());
    return a;
}
```

`primRoot(p)` 返回模素数 p 的最小原根（$p=2$ 时为 1）；`primRoots(p)` 返回全部 $\varphi(p-1)$ 个原根（升序）。Pollard-Rho 分解 $p-1$ 的期望时间通常约为最小质因子四次根量级；若最小原根为 g，逐个验证为 $O(g\,\omega(p-1)\log p)$，实践中 g 通常很小；枚举并排序全部原根为 $O(p\log p)$。仅需离散对数时用 7.4.001 BSGS / exBSGS 或 7.4.002 Pohlig-Hellman。

7.4.004 二次剩余

素数模 p 下求 $x^2\equiv n$ 的一个根。$p=2$ 时根就是 $n \bmod 2$；奇素数先用 Euler 判据 $n^{(p-1)/2}$ 判定是否为二次剩余，$p\equiv 3 \pmod 4$ 时直接取 $x=n^{(p+1)/4}$，其余情况用 Tonelli-Shanks。写 $p-1=q\cdot 2^s$（q 奇），取一个非剩余 z，以 $c=z^q$ 逐步消去 $t=n^q$ 的 2-幂阶误差。返回 $\min(x,p-x)$，另一根为 $p-x$。依赖 7.1.001 快速幂 的 `mulMod`、`powerMod`。

```
optional<long long> sqrtMod(long long n, long long p)
{
    // n 是被开方数，p 是素数模；返回较小的平方根，非二次剩余返回空。
    assert(p >= 2); // 调试检查，可删
    n %= p;
    if (n < 0)
    {
        n += p;
    }
    if (n == 0)
    {
        return 0;
    }
    if (p == 2)
    {
```

```
        return n;
    }
    if (powerMod(n, (p - 1) / 2, p) != 1)
    {
        return nullopt;
    }
    if (p % 4 == 3)
    {
        long long x = powerMod(n, (p + 1) / 4, p);
        return min(x, p - x);
    }
    long long q = p - 1;
    long long s = 0;
    while (!(q & 1))
    {
        q >>= 1;
        s++;
    }
    long long z = 2;
    while (powerMod(z, (p - 1) / 2, p) != p - 1)
    {
        z++;
    }
    long long c = powerMod(z, q, p);
    long long x = powerMod(n, (q + 1) / 2, p);
    long long t = powerMod(n, q, p);
    long long m = s;
    while (t != 1)
    {
        long long i = 1;
        long long y = mulMod(t, t, p);
        while (y != 1)
        {
            y = mulMod(y, y, p);
            i++;
        }
        long long b = powerMod(c, 1ULL << (m - i - 1), p);
        x = mulMod(x, b, p);
        c = mulMod(b, b, p);
        t = mulMod(t, c, p);
        m = i;
    }
    return min(x, p - x);
}
```

`sqrtMod(n, p)`（p 为素数）返回较小的那个根或 `nullopt`；奇素数下另一根为 $p-x$，$n\equiv 0$ 时返回 0。给定二次非剩余后，Tonelli-Shanks 主体用 $O(\log^2 p)$ 次模乘；当前代码从 2 起试找非剩余，这一步最坏检查 $O(p)$ 个候选、每次需 $O(\log p)$ 次模乘，实践中通常很快。额外空间 $O(1)$。一般 k 次剩余用 7.4.005 K 次剩余。

7.4.005 K 次剩余

求素数模 p 下 $x^k \equiv a$ 的一组解。`kthRootBSGS` 取原根并离散对数，适合少量查询；默认的 `kthResidue` 先解掉与 $p-1$ 互素的指数部分，再对 $\gcd(k,p-1)$ 的每个质数幂逐层提根，适合批量查询。依赖 7.1.001 快速幂 的 `mulMod`、`powerMod` 与 7.1.003 扩展欧几里得 的 `invMod`；简单版本还依赖 7.4.001 BSGS / exBSGS 和 7.4.003 原根与全部原根筛。

```
// 依赖 7.4.003 原根与全部原根筛
optional<int> kthRootBSGS(int a, int k, int p)
{
    // a 是目标剩余，k 是非负指数，p 是素数；返回一组 x 使 x^k=a mod p，无解返回空。
    assert(k >= 0 && p >= 2); // 调试检查，可删
    a %= p;
    if (a < 0)
    {
        a += p;
    }
    if (k == 0)
    {
        return a == 1 ? optional<int>(0) : nullopt;
    }
    if (a == 0)
    {
        return 0;
    }
    int g = primRoot(p);
    auto b = bsgs(g, a, p);
    if (!b)
    {
        return nullopt;
    }
    int d = gcd(k, p - 1);
    if (*b % d)
    {
        return nullopt;
    }
    int mod = (p - 1) / d;
    if (mod == 1)
    {
        return 1;
    }
    auto inv = invMod(k / d, mod);
    if (!inv)
    {
        return nullopt;
    }
    int y = mulMod(*b / d, *inv, mod);
    return powerMod(g, y, p);
}
```

头文件随后实现 `ppRoot` 与默认 `kthResidue`：后者把指数约化到 $p-1$，检查 $a^{(p-1)/\gcd(k,p-1)}=1$，再

逐质数幂提根。`kthResidue(a,k,p)`（p 素数、$k \geq 0$）返回任意一根或 `nullopt`；按 $0^0 = 1$ 约定，$k = 0$ 时仅 $a \equiv 1 \pmod{p}$ 有解并返回 0。简单的 `kthRootBSGS` 为 $O(\sqrt{p})$ 期望时间空间；默认接口避免整群离散对数，单个质数幂步骤约需 $O(\sqrt{eq}\log p)$ 次模运算，其中 $q^e \mid \gcd(k, p-1)$。测试对小素数全部暴力比对，并覆盖 LC 的零指数语义。

7.4.006 扩展欧拉定理与欧拉降幂

扩展欧拉定理允许非互素底数在指数足够大时把指数化为 $(b \bmod \varphi(m)) + \varphi(m)$。幂塔由下向上沿 φ 链递归，模数在 $O(\log m)$ 层后降到 1；关键是不能只保留指数余数，还要知道真实指数是否已经达到当前模数。`Enc` 分开保存余数与 `large` 标志，`towerCapped` 用带上限的乘幂判断真实塔值是否达到阈值；零底数则按 $0^0 = 1$ 单独处理奇偶层。模幂依赖 7.1.001 快速幂 的 `powerMod`。

```
class PowerTower
{
    struct Enc
    {
        long long rem = 0; // 对当前模数的余数
        bool large = false; // 真实值是否不小于当前模数
    };

    static long long phi(long long n)
    {
        // 返回正整数 n 的 Euler phi 值
        long long ans = n;
        for (long long p = 2; p <= n / p; p++)
        {
            if (n % p)
            {
                continue;
            }
            while (n % p == 0)
            {
                n /= p;
            }
            ans = ans / p * (p - 1);
        }
        if (n > 1)
        {
            ans = ans / n * (n - 1);
        }
        return ans;
    }

    static long long mulCap(long long a, long long b, long long limit)
    {
        // 返回 min(a*b,limit)，参数均非负
        if (!a || !b)
        {
            return 0;
        }
```

```
        if (a > (limit - 1) / b)
        {
            return limit;
        }
        return min(limit, a * b);
    }

    static long long powerCapped(long long base, long long exponent, long long limit)
    {
        // 返回 min(base^exponent,limit)
        long long result = 1;
        while (exponent && result < limit)
        {
            if (exponent & 1)
            {
                result = mulCap(result, base, limit);
            }
            exponent >>= 1;
            if (exponent)
            {
                base = mulCap(base, base, limit);
            }
        }
        return result;
    }

    static long long towerCapped(long long base, long long height,
                                 long long limit)
    {
        // 返回 height 层幂塔与 limit 的较小值，约定 0^0=1
        if (height == 0)
        {
            return min(1LL, limit);
        }
        if (base == 0)
        {
            long long v = height & 1 ? 0 : 1;
            return min(v, limit);
        }
        if (base == 1)
        {
            return min(1LL, limit);
        }
        if (limit <= 1)
        {
            return limit;
        }

        long long need = 0;
        long long v = 1;
        while (v < limit)
        {
```

```
            v = mulCap(v, base, limit);
            need++;
        }
        long long e = towerCapped(base, height - 1, need);
        if (e >= need)
        {
            return limit;
        }
        return powerCapped(base, e, limit);
    }

    static Enc evaluate(long long base, long long height, long long mod)
    {
        // 返回幂塔对 mod 的余数，并独立记录真实值是否越过 mod
        if (height == 0)
        {
            return {1 % mod, 1 >= mod};
        }
        if (base == 0)
        {
            long long v = height & 1 ? 0 : 1;
            return {v % mod, v >= mod};
        }
        if (mod == 1)
        {
            return {0, towerCapped(base, height, 1) == 1};
        }
        long long ph = phi(mod);
        auto e = evaluate(base, height - 1, ph);
        unsigned long long exponent = (unsigned long long)e.rem;
        if (e.large)
        {
            exponent += (unsigned long long)ph;
        }
        long long rem = powerMod(base, exponent, mod);
        bool large = towerCapped(base, height, mod) == mod;
        return {rem, large};
    }

  public:
    // 计算 height 层 base 幂塔对 mod 的余数。
    static long long eval(long long base, long long height, long long mod)
    {
        // 返回 height 层 base 幂塔对 mod 的余数
        assert(base >= 0 && height >= 0 && mod > 0); // 调试检查，可删
        return evaluate(base, height, mod).rem;
    }
};
```

`PowerTower::eval(base,height,mod)` 要求三者分别非负、非负、正，$height = 0$ 视作 1，并约定 $0^0 = 1$；因此零底数时奇数层为 0、正偶数层为 1。余数与“真实值是否达到模数”分成 `Enc` 两个字段，不再用 `residue+mod` 塞进可能溢出的 `long long`。递归层数不超过 $\min(height, O(\log mod))$，每层试

除求 φ 最坏 $O(\sqrt{mod})$，栈 $O(\log mod)$。

7.4.007 Carmichael 函数

Carmichael 函数 $\lambda(n)$ 是模 n 既约剩余类群中所有元素阶的最小公倍数，也是对任意 $\gcd(a,n)=1$ 都满足 $a^m \equiv 1 \pmod n$ 的最小正指数 m（它整除 $\varphi(n)$，往往更小）。分解 $n=\prod p^k$ 后逐个素数幂取值再合并：奇素数幂与 $2,4$ 处 $\lambda=\varphi$，而 $\lambda(2^k)=2^{k-2}\ (k\ge 3)$；最终 $\lambda(n)=\mathrm{lcm}$ 各素数幂之值。

```cpp
long long carmichael(long long n)
{
    // n>=1；试除分解后返回 Carmichael 函数 lambda(n)。
    assert(n >= 1); // 调试检查，可删
    long long ans = 1;
    for (long long p = 2; p <= n / p; p++)
    {
        if (n % p)
        {
            continue;
        }
        long long q = 1;
        int k = 0;
        while (n % p == 0)
        {
            n /= p;
            q *= p;
            k++;
        }
        long long z = q / p * (p - 1);
        if (p == 2 && k >= 3)
        {
            z /= 2;
        }
        ans = lcm(ans, z);
    }
    if (n > 1)
    {
        ans = lcm(ans, n - 1);
    }
    return ans;
}
```

`carmichael(n)`（$n\ge 1$）返回 $\lambda(n)$，试除分解 $O(\sqrt{n})$ 时间、$O(1)$ 额外空间；各质数幂及逐步最小公倍数均不超过原 n，故正 `long long` 输入下不会溢出。验证：$n=8,15,16,561$ 依次得 $2,4,4,80$（$\lambda(561)=\mathrm{lcm}(\lambda(3),\lambda(11),\lambda(17))=\mathrm{lcm}(2,10,16)=80$）。

7.4.008 固定模批量离散对数

素数 p 与原根 g 固定、查询 a_i 很多时，不应为每个询问各做一次 BSGS。取 $B\approx\sqrt{(p-1)/Q}$，预存 g^{Bx}；随后对 $0\le y<B$ 共享扫描全部查询，若 $a_i g^y=g^{Bx}$，则答案为 $Bx-y \pmod{p-1}$。表大

小与扫描次数分别约为 p/B、QB，平衡后总期望时间和空间量级均为 $O(\sqrt{pQ})$。下面的开放寻址表把非零域元素和指数压在同一个 64 位槽中，避免 unordered_map 在大询问下的节点开销；它与 1.4.004 防 hack 哈希 用途不同。

```cpp
// 依赖 7.1.001 快速幂
struct DLogHash
{
    // tab 是开放寻址表，非空槽高 32 位存域元素、低 32 位存指数，0 表示空槽（域元素非零）。
    vector<unsigned long long> tab;

    DLogHash(int n) : tab(max(3LL, n * 10 / 7 + 3))
    {
        // n 是预计插入的键数；表长取约 1.43n，装载率不超过 0.7。
    }

    unsigned long long hash(unsigned long long x) const
    {
        // x 是键；返回 splitmix64 混合后的散列值。
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }

    void insert(int key, int val)
    {
        // key 是 [1,p) 内的域元素，val 是正指数；同一键只保留最小的指数。
        int p = hash(key) % tab.size();
        while (tab[p] && (int)(tab[p] >> 32) != key)
        {
            if (++p == (int)tab.size())
            {
                p = 0;
            }
        }
        if (!tab[p] || val < (int)(tab[p] & 0xffffffffULL))
        {
            tab[p] = ((unsigned long long)key << 32) | val;
        }
    }

    int find(int key) const
    {
        // 返回 key 对应的最小指数，不存在时返回 -1。
        int p = hash(key) % tab.size();
        while (tab[p])
        {
            if ((int)(tab[p] >> 32) == key)
            {
                return (int)(tab[p] & 0xffffffffULL);
            }
            if (++p == (int)tab.size())
```

```
            {
                p = 0;
            }
        }
        return -1;
    }
};

vector<int> batchDLog(int p, int g, const vector<int> &a)
{
    // p 是不超过 1e9 的素数，g 是模 p 的原根，a[i] 在 [1,p) 内；
    // 返回 ans[i] 为满足 g^x = a[i] (mod p) 的最小 x，范围 [0,p-2]。
    assert(p >= 2 && 1 <= g && g < p); // 调试检查，可删。
    int qn = a.size();
    vector<int> ans(qn, -1);
    if (qn == 0)
    {
        return ans;
    }
    if (p == 2)
    {
        fill(ans.begin(), ans.end(), 0);
        return ans;
    }
    // 巨步表存 g^{Bx}（1<=x<=lim），小步对所有查询共享 g^y（0<=y<B）。
    int B = sqrtl((long double)(p - 1) / qn) + 1;
    int lim = (p - 2 + B - 1) / B;
    DLogHash tab(lim);
    int step = powerMod(g, B, p), cur = step;
    for (int x = 1; x <= lim; x++)
    {
        tab.insert(cur, x);
        cur = cur * step % p;
    }
    int gy = 1; // gy 是 g^y。
    for (int y = 0; y < B; y++)
    {
        for (int i = 0; i < qn; i++)
        {
            assert(1 <= a[i] && a[i] < p); // 调试检查，可删。
            if (a[i] == 1)
            {
                ans[i] = 0;
                continue;
            }
            // a[i]*g^y = g^{Bx} 说明 a[i] = g^{Bx-y}。
            int x = tab.find(a[i] * gy % p);
            if (x != -1)
            {
                int e = ((x * B - y) % (p - 1) + p - 1) % (p - 1);
                if (ans[i] == -1 || e < ans[i])
                {
```

```
                    ans[i] = e;
                }
            }
        }
        gy = gy * g % p;
    }
    return ans;
}
```

`batchDLog(p,g,a)` 要求 $p \le 10^9$ 为素数、g 为原根且每个查询在 $[1,p)$，返回唯一的 $x_i \in [0,p-2]$ 使 $g^{x_i} \equiv a_i \pmod p$；空查询直接返回空。$Q$ 极大时表可能占数百 MiB，应按题目内存限制评估；单次或少量查询直接用 7.4.001 BSGS / exBSGS，群阶光滑时用 7.4.002 Pohlig-Hellman。

下面在模 17、原根 3 下检查单位元及一批可直接快速幂复核的指数：

```
signed main()
{
    assert(batchDLog(17, 3, {1, 3, 9, 10, 16}) ==
           vector<int>({0, 1, 2, 3, 8}));
}
```

7.4.009 Hensel 简单根提升

Hensel 引理把多项式 $f \in \mathbb{Z}[x]$ 模素数 p 的单根提升为模 p^k 的根。设 $f(x_0) \equiv 0 \pmod q$（$q = p^i$）且 $f'(x_0) \not\equiv 0 \pmod p$，令 $x_1 = x_0 + tq$，由 Taylor 展开

$$f(x_0 + tq) \equiv f(x_0) + tq\, f'(x_0) \pmod{q^2},$$

在模 qp 下要求 $\frac{f(x_0)}{q} + t\, f'(x_0) \equiv 0 \pmod p$，于是

$$t \equiv -\frac{f(x_0)}{q} \cdot f'(x_0)^{-1} \pmod p.$$

$f'(x_1) \equiv f'(x_0) \pmod p$，所以逆元只需在模 p 下求一次；每轮把模数乘 p，提升后的根在模 p^k 下唯一。

```
// 依赖 7.2.003 Miller-Rabin
optional<int> hensel(const vector<int> &a, int x, int p, int k)
{
    // a 是整系数多项式 f，低次在前；p 是素数，x 是 f 模 p 的根；p^k 不超过 LLONG_MAX。
    // 当 x 是单根（f'(x) 模 p 非零）时返回 f 模 p^k 的唯一提升根（在 [0,p^k) 内），否则返回空。
    assert(isPrime(p) && k >= 1); // 调试检查，可删。
    auto eval = [&](int y, int mod, bool der)
    {
        // 返回 f(y) 或 f'(y)（der 为真）模 mod 的非负值，秦九韶求值。
        __int128 s = 0;
        for (int i = (int)a.size() - 1; i >= der; i--)
        {
            s = (s * y + (__int128)a[i] * (der ? i : 1)) % mod;
        }
        return (int)((s + mod) % mod);
```

```
    };
    x = (x % p + p) % p;
    int d = eval(x, p, true);
    if (eval(x, p, false) || !d)
    {
        return nullopt;
    }
    // 牛顿迭代：若 f(x)≡0 (mod q)，则 x - f(x)/f'(x) 是模 qp 的根，且 f'(x) 的逆只需模 p。
    int inv = pow64(d, p - 2, p), q = p;
    for (int i = 1; i < k; i++)
    {
        assert((__int128)q * p <= LLONG_MAX); // 调试检查，可删。
        int nq = q * p, f = eval(x, nq, false);
        int t = (__int128)(p - f / q) * inv % p;
        x += t * q;
        q = nq;
    }
    return x;
}
```

`hensel(a,x,p,k)` 中 `a` 是系数（低次在前），`p` 为素数，`x` 是模 p 的根，p^k 须不超过 `LLONG_MAX`；返回 $[0,p^k)$ 内的提升根。若 x 不是模 p 的根，或 $f'(x)\equiv 0 \pmod p$（重根，可能无解也可能有多个提升），返回空。求值用 `__int128` 防止乘积溢出。时间 $O(k\deg f)$，额外空间 $O(1)$。素性判定与模幂复用 7.2.003 Miller-Rabin 的 `isPrime` 与 `pow64`。

模 p 的平方根本身用 7.4.004 二次剩余 求。下面把 $x^2\equiv 2 \pmod 7$ 的根 3 提升到模 $7^4=2401$：

```
signed main()
{
    auto r = hensel({-2, 0, 1}, 3, 7, 4);
    assert(r && *r == 2166 && (*r * *r - 2) % 2401 == 0);
    assert(!hensel({0, 0, 1}, 0, 7, 3)); // f'(0)=0，不是单根。
}
```

7.5 类欧几里得

7.5.001 `floor_sum`

计算 $\sum_{i=0}^{n-1}\left\lfloor\dfrac{ai+b}{m}\right\rfloor$，把 $O(n)$ 枚举降到对数级。做法同 AtCoder Library：先扣除 a,b 中 $\ge m$ 的整商贡献，再令 $y=an+b$，利用格点三角形关于直线的对称把问题交换 a 与 m 递归，参数如辗转相除般缩小，直到 $an+b<m$ 终止；开头两段先把 $a<0$、$b<0$ 规约为非负。

```
__int128 floorSum(long long n, long long m, long long a, long long b)
{
    // n 是项数，m 是正除数，a、b 是线性系数；返回取整和，要求精确中间量与答案可由 __int128 表示。
    assert(n >= 0 && m > 0); // 调试检查，可删
    __int128 N = n;
    __int128 M = m;
```

```
    __int128 A = a;
    __int128 B = b;
    __int128 ans = 0;
    auto divFloor = [](__int128 x, __int128 y)
    {
        __int128 q = x / y;
        __int128 r = x % y;
        return q - (r < 0);
    };
    __int128 q = divFloor(A, M);
    ans += N * (N - 1) / 2 * q;
    A -= q * M;
    q = divFloor(B, M);
    ans += N * q;
    B -= q * M;
    while (true)
    {
        if (A >= M)
        {
            ans += N * (N - 1) / 2 * (A / M);
            A %= M;
        }
        if (B >= M)
        {
            ans += N * (B / M);
            B %= M;
        }
        __int128 y = A * N + B;
        if (y < M)
        {
            break;
        }
        N = y / M;
        B = y % M;
        swap(A, M);
    }
    return ans;
}
```

`floorSum(n,m,a,b)` 要求 $m > 0$、$n \geq 0$，a, b 可取完整 `long long`；负数规约、三角和与 $AN + B$ 均用 `__int128` 计算，但接口仍要求所有精确中间量及最终取整和落在 `__int128` 范围内，极端的 $n^2|a|/m$ 可能超界。复杂度 $O(\log \max(|a|, m))$ 时间、$O(1)$ 空间。需要聚合更多格点量时用 7.5.003 万能欧几里得；求 $(ai + b) \bmod m$ 的最小值见 7.5.004 Min of Mod of Linear。

7.5.002 类欧几里得格点统计

基础线性取整和 $\sum \left\lfloor \frac{ai+b}{m} \right\rfloor$ 由 7.5.001 floor_sum 的 `floorSum` 提供，本条不重复实现。若要同时聚合横步数、纵步数及其交叉项等格点统计量（一次矩、覆盖面积等），改用 7.5.003 万能欧几里得 的 `floorPath`：把每种待累加量定义成一个幺半群元素，一次遍历即可得到全部统计。

备注：原资料本条重复粘贴了一份与 7.5.001 floor_sum 完全相同的 `floorSum`（且只算和、并未真的算矩），属重复代码；同名同签名与 7.5.001 冲突，故删去，改按上面的分工引用。

7.5.003 万能欧几里得

万能欧几里得把直线 $y=\frac{pi+r}{q}$ 下方的取整阶梯看成一串操作：i 从 1 走到 $length$，每当 $\lfloor\frac{pi+r}{q}\rfloor$ 增加 1 就写一个纵步 U，每到一个整数 i 就写一个横步 R。要求的统计量只要能写成某个幺半群上的乘积 $\prod(\text{步})$，就能像类欧几里得一样递归：$p\ge q$ 时每个 R 前必然先有 $\lfloor p/q\rfloor$ 个 U，把 R 换成 $U^{\lfloor p/q\rfloor}R$ 后令 $p\leftarrow p \bmod q$；$p<q$ 时交换 U、R 的角色（相当于把图形沿 $y=x$ 翻转），问题规模像辗转相除一样下降。这里的 `Euclid` 以 (x,y,xy) 记录横步数、纵步数以及“每个横步之前已有的纵步数之和”，合并时交叉项补 $y\cdot x'$，于是 $xy=\sum_{i=1}^{length}\lfloor\frac{pi+r}{q}\rfloor$。

`floorMoment` 求更一般的矩 $T_{p,q}=\sum_{i=0}^{n} i^p f(i)^q$，$f(i)=\lfloor\frac{ai+b}{c}\rfloor$，一次给出所有 $p+q\le d$ 的表：

- $a=0$ 时 f 是常数，$T_{p,q}=S_p(n)\,(b/c)^q$，其中 $S_p(x)=\sum_{i=0}^{x} i^p$ 是 $p+1$ 次多项式，由 $\sum_{k\le p}\binom{p+1}{k}S_k(x)=(x+1)^{p+1}$ 逐个求出。
- $a\ge c$ 或 $b\ge c$ 时 $f(i)=\lfloor a/c\rfloor i+\lfloor b/c\rfloor+g(i)$，$g$ 用 $(a \bmod c, b \bmod c)$；把 f^q 按三项式展开，归约到 g 的表。
- $a,b<c$ 时令 $h=f(n)$，利用 $f(i)^q=\sum_{j<f(i)}\big((j+1)^q-j^q\big)$ 以及 $f(i)>j \iff i>g(j)$，$g(j)=\lfloor\frac{cj+c-b-1}{a}\rfloor$，交换求和次序得

$$T_{p,q}=h^q S_p(n)-\sum_{u<q}\binom{q}{u}\sum_{r}[x^r]S_p\cdot\sum_{j=0}^{h-1} j^u g(j)^r,$$

右边是参数为 $(h-1,c,c-b-1,a)$ 的同类表，参数像辗转相除一样变小。

```cpp
// 依赖 1.1.008 静态取模类
struct Euclid
{
    __int128 x = 0;  // 已拼接路径的横步总数
    __int128 y = 0;  // 已拼接路径的纵步总数
    __int128 xy = 0; // 每个横步之前的纵步数之和

    Euclid operator+(const Euclid &other) const
    {
        // 把 other 路径接在当前路径后，返回合并结果。
        return {x + other.x, y + other.y,
                xy + other.xy + y * other.x};
    }
};

inline Euclid monoidPower(Euclid value, long long exponent)
{
    // 返回 value 在幺半群 + 下的 exponent 次幂。
    assert(exponent >= 0); // 调试检查，可删
    Euclid res;
    while (exponent)
    {
        if (exponent & 1)
```

```
        {
            res = res + value;
        }
        value = value + value;
        exponent >>= 1;
    }
    return res;
}

inline Euclid floorPath(long long p, long long q, long long r,
                        long long length, Euclid up, Euclid right)
{
    // p、q、r 定义取整直线，length 是横坐标上界；up、right 是纵步与横步元素，返回整段聚合。
    assert(p >= 0 && q > 0 && 0 <= r && r < q && length >= 0); // 调试检查，可删
    if (!length)
    {
        return {};
    }
    if (p >= q)
    {
        return floorPath(p % q, q, r, length, up,
                         monoidPower(up, p / q) + right);
    }
    long long height = (long long)(((__int128)length * p + r) / q);
    if (!height)
    {
        return monoidPower(right, length);
    }
    long long first = (long long)(((__int128)q - r - 1) / p);
    long long suffix = (long long)((__int128)length -
                                   ((__int128)q * height - r - 1) / p);
    return monoidPower(right, first) + up +
           floorPath(q, p, (q - r - 1) % p, height - 1,
                     right, up) +
           monoidPower(right, suffix);
}

using Z7 = MInt<1000000007>; // 矩表固定模 1e9+7，换模数只改这里。
constexpr int floorDeg = 10;  // floorDeg 是 p+q 的上界。
// FloorTable[p][q] 存 sum i^p floor(...)^q，只有 p+q<=d 的项有效。
using FloorTable = array<array<Z7, floorDeg + 1>, floorDeg + 1>;

struct FloorPre
{
    // C[n][k] 是组合数；S[p] 是 S_p(x)=sum_{i=0}^{x} i^p 的系数（低次在前，0^0 记为 1）。
    array<array<Z7, floorDeg + 2>, floorDeg + 2> C;
    array<vector<Z7>, floorDeg + 1> S;

    FloorPre()
    {
        // 无参数；由 sum_k C(p+1,k) S_k(x) = (x+1)^{p+1} 逐个求出幂和多项式。
        for (int n = 0; n <= floorDeg + 1; n++)
```

```
        {
            C[n][0] = 1;
            for (int k = 1; k <= n; k++)
            {
                C[n][k] = C[n - 1][k - 1] + C[n - 1][k];
            }
        }
        for (int p = 0; p <= floorDeg; p++)
        {
            S[p].assign(p + 2, 0);
            for (int j = 0; j <= p + 1; j++)
            {
                S[p][j] = C[p + 1][j];
            }
            for (int k = 0; k < p; k++)
            {
                for (int j = 0; j < (int)S[k].size(); j++)
                {
                    S[p][j] -= C[p + 1][k] * S[k][j];
                }
            }
            Z7 iv = Z7(p + 1).inv();
            for (auto &x : S[p])
            {
                x *= iv;
            }
        }
    }

    Z7 eval(int p, int x) const
    {
        // 返回 S_p(x)，x>=0。
        Z7 ans = 0;
        for (int i = (int)S[p].size() - 1; i >= 0; i--)
        {
            ans = ans * x + S[p][i];
        }
        return ans;
    }
};

FloorTable floorMoment(int n, int a, int b, int c, int d)
{
    // n>=0，a,b>=0，c>0，0<=d<=floorDeg；记 f(i)=floor((a*i+b)/c)，
    // 返回表 T，T[p][q] = sum_{i=0}^{n} i^p f(i)^q（p+q<=d），均模 1e9+7。
    assert(n >= 0 && a >= 0 && b >= 0 && c > 0 && 0 <= d && d <= floorDeg); // 调试检查，可删。
    static const FloorPre pre;
    FloorTable ans{};
    if (!a)
    {
        // f 恒为 b/c。
        Z7 v = b / c;
```

```
        for (int p = 0; p <= d; p++)
        {
            for (int q = 0; p + q <= d; q++)
            {
                ans[p][q] = pre.eval(p, n) * v.pow(q);
            }
        }
        return ans;
    }
    int da = a / c, db = b / c;
    if (da || db)
    {
        // f(i) = da*i + db + g(i)，g 用 (a%c, b%c)；按三项式展开 f^q。
        auto sub = floorMoment(n, a % c, b % c, c, d);
        vector<Z7> pa(d + 1, 1), pb(d + 1, 1);
        for (int i = 1; i <= d; i++)
        {
            pa[i] = pa[i - 1] * da;
            pb[i] = pb[i - 1] * db;
        }
        for (int p = 0; p <= d; p++)
        {
            for (int q = 0; p + q <= d; q++)
            {
                for (int j = 0; j <= q; j++)
                {
                    for (int t = 0; t + j <= q; t++)
                    {
                        // g^j (da*i)^t db^{q-j-t} 的系数是 C(q,j) C(q-j,t)。
                        ans[p][q] += pre.C[q][j] * pre.C[q - j][t] * pa[t] * pb[q - j - t] *
↪   sub[p + t][j];
                    }
                }
            }
        }
        return ans;
    }
    // a<c 且 b<c：交换坐标轴。h=f(n)，f(i)>j 当且仅当 i>g(j)，g(j)=floor((c*j+c-b-1)/a)。
    int h = (int)(((__int128)a * n + b) / c);
    for (int p = 0; p <= d; p++)
    {
        ans[p][0] = pre.eval(p, n);
    }
    if (!h)
    {
        return ans;
    }
    // f(i)^q = sum_{j<f(i)} ((j+1)^q - j^q)，交换求和后 i 取 (g(j), n]。
    auto sub = floorMoment(h - 1, c, c - b - 1, a, d);
    for (int p = 0; p <= d; p++)
    {
        for (int q = 1; p + q <= d; q++)
```

```
        {
            Z7 w = ans[p][0] * Z7(h).pow(q);
            for (int u = 0; u < q; u++)
            {
                for (int r = 0; r <= p + 1; r++)
                {
                    w -= pre.C[q][u] * pre.S[p][r] * sub[u][r];
                }
            }
            ans[p][q] = w;
        }
    }
    return ans;
}

vector<int> floorMoments(const vector<array<int, 6>> &qs)
{
    // qs[i] 的六维依次是 n、a、b、c、p、q（p+q<=floorDeg）；
    // 返回各询问的 sum_{i=0}^{n} i^p floor((a*i+b)/c)^q 模 1e9+7。
    vector<int> ans;
    for (auto [n, a, b, c, p, q] : qs)
    {
        assert(p >= 0 && q >= 0 && p + q <= floorDeg); // 调试检查，可删。
        ans.push_back(floorMoment(n, a, b, c, p + q)[p][q].val());
    }
    return ans;
}
```

`floorPath(p,q,r,length,up,right)` 要求 $p \ge 0$、$q > 0$、$0 \le r < q$，返回 $1 \le i \le length$ 的阶梯路径聚合值；传 `up={0,1,0}`、`right={1,0,0}` 时 `xy` 就是 $\sum_{i=1}^{length} \lfloor \frac{pi+r}{q} \rfloor$，三个字段须落在 `__int128` 范围内。递归 $O(\log \max(p,q))$ 层，每层用 $O(\log length)$ 次合并做快速幂。换别的统计量只需改 `Euclid` 的字段与 `operator+`，保持结合律即可。

`floorMoment(n,a,b,c,d)` 要求 $n \ge 0$、$a, b \ge 0$、$c > 0$、$d \le 10$，返回 `FloorTable`，只有 $p+q \le d$ 的项有效；`floorMoments(qs)` 逐个回答 `{n,a,b,c,p,q}` 询问。模数固定为 10^9+7（改 `Z7` 即可换素数模），依赖 1.1.008 静态取模类。递归 $O(\log \max(a,c))$ 层，每层 $O(d^4)$。只求 $\sum \lfloor \frac{ai+b}{c} \rfloor$ 时用更短的 7.5.001 floor_sum。下面先用小例子核对，再给一个大参数的实测值：

```
signed main()
{
    auto e = floorPath(4, 11, 3, 5, {0, 1, 0}, {1, 0, 0}); // 依次为 floor((4i+3)/11), i=1..5:
    ↪ 0,1,1,1,2。
    assert(e.x == 5 && e.y == 2 && e.xy == 5);
    auto t = floorMoment(10, 3, 2, 5, 3);
    assert(t[1][2] == 1069 && t[0][0] == 11);
    auto v = floorMoments({{100, 7, 3, 4, 2, 3}, {1000000000, 123456789, 987654321, 1000000007,
    ↪ 3, 4}});
    assert(v[0] == 279549271);
    cout << v[1] << '\n';
    // 期望输出：738884790
}
```

7.5.004 Min of Mod of Linear

在 $0 \le i < n$ 中求 $(ai+b) \bmod m$ 的最小值。余数 $r_i = (ai+b) - m\lfloor \frac{ai+b}{m} \rfloor$，而 “$r_i \le x$ 的下标个数” 等于两次 `floorSum` 之差且关于 x 单调，于是二分最小可行的 x 即为答案。依赖 7.5.001 floor_sum 的 `floorSum`。

```
int minModLinear(int n, int m, int a, int b)
{
    // n、m 为正，a、b 是整数；返回 0<=i<n 时 (a*i+b) mod m 的最小值。
    assert(n > 0 && m > 0); // 调试检查，可删
    a %= m;
    b %= m;
    if (a < 0)
    {
        a += m;
    }
    if (b < 0)
    {
        b += m;
    }
    auto count = [&](int x)
    {
        // x 是余数上界；返回余数落在 [0,x] 的下标数量。
        return floorSum(n, m, a, b) - floorSum(n, m, a, b - x - 1);
    };
    int l = -1;
    int r = m - 1;
    while (r - l > 1)
    {
        int x = midpoint(l, r);
        if (count(x))
        {
            r = x;
        }
        else
        {
            l = x;
        }
    }
    return r;
}
```

`minModLinear(n,m,a,b)` 要求 $n, m > 0$，a, b 可负，返回值在 $[0, m)$。计数调用 7.5.001 floor_sum 的宽返回值，二分中点使用 `midpoint`；a, b 先规约到 $[0, m)$，故时间复杂度为 $O(\log^2 m)$，额外空间 $O(1)$。固定例 $n = 5, m = 11, a = 4, b = 3$ 的余数为 $3, 7, 0, 4, 8$，答案为 0。

7.5.005 Stern-Brocot Tree / Farey 序列

在正有理数的 Stern-Brocot 表示与既约分数间互转，并枚举 Farey 序列。Stern-Brocot 树以左右邻分数的 mediant 导航正有理数，同向的连续步用连分数商压成一段（`L`/`R` + 次数）；`farey` 用相邻分数行列式恒

为 1 的递推 $\frac{p}{q}, \frac{r}{s} \to k = \left\lfloor \frac{n+q}{s} \right\rfloor$ 逐个生成 n 阶序列。

```cpp
vector<pair<char, long long>> sternPath(long long p, long long q)
{
    // p、q 是既约正分数的分子与分母；返回该分数在 Stern-Brocot 树中的路径编码（每段为 L/R 及重复次数）。
    assert(p > 0 && q > 0 && gcd(p, q) == 1); // 调试检查，可删
    vector<pair<char, long long>> a;
    while (p != q)
    {
        char c;
        long long k;
        if (p < q)
        {
            c = 'L';
            k = (q - 1) / p;
            q -= k * p;
        }
        else
        {
            c = 'R';
            k = (p - 1) / q;
            p -= k * q;
        }
        a.push_back({c, k});
    }
    return a;
}

pair<long long, long long> sternValue(const vector<pair<char, long long>> &a)
{
    // a 是合法路径编码，且结果可由 long long 表示；返回对应的既约正分数 {p,q}。
    long long p = 1;
    long long q = 1;
    for (auto it = a.rbegin(); it != a.rend(); it++)
    {
        auto [c, k] = *it;
        assert((c == 'L' || c == 'R') && k > 0); // 调试检查，可删
        if (c == 'L')
        {
            assert(k <= (numeric_limits<long long>::max() - q) / p); // 调试检查，可删
            q += k * p;
        }
        else
        {
            assert(k <= (numeric_limits<long long>::max() - p) / q); // 调试检查，可删
            p += k * q;
        }
    }
    return {p, q};
}

vector<pair<int, int>> farey(int n)
```

```
{
    // n 是阶数；返回 n 阶 Farey 序列 F_n，分数按值递增排列。
    assert(n >= 1); // 调试检查，可删
    vector<pair<int, int>> a{{0, 1}};
    int p = 0;
    int q = 1;
    int r = 1;
    int s = n;
    while (r <= n)
    {
        a.push_back({r, s});
        __int128 k = ((__int128)n + q) / s;
        int x = (int)(k * r - p);
        int y = (int)(k * s - q);
        p = r;
        q = s;
        r = x;
        s = y;
    }
    return a;
}
```

`sternPath` 要求 $p, q > 0$ 且 $\gcd(p, q) = 1$；`sternValue` 逆变换回既约分数；`farey(n)` 返回 F_n。路径编解码 $O(\log(p + q))$ 段与空间，Farey 序列 $O(|F_n|)$ 时间空间。

`ratApprox(n,x,y)` 返回分子、分母均不超过 `n` 的最紧下界和上界；不存在的方向分别用 `{0,1}`、`{1,0}` 表示。要求 `n>0,x>=0,y>0`，中间运算使用 `__int128`。

7.5.006 Pell 方程

求非平方正整数 d 的 Pell 方程 $x^2 - dy^2 = 1$ 的最小正整数解（基本解）。展开 $\sqrt{d}$ 的周期连分数并同步递推收敛分数 $\frac{p}{q}$，第一个满足 $p^2 - dq^2 = 1$ 的收敛分数即基本解；分子分母用 `__int128` 递推，以便在越过 `long long` 前及时判溢出。平方根依赖 7.1.014 整数 k 次方根 的 `kthRoot`。

```
optional<pair<long long, long long>> pell(long long d)
{
    // d 是正整数；返回 x^2-d*y^2=1 的基本解，d 为平方数或解越界时返回空。
    assert(d > 0); // 调试检查，可删
    long long a0 = kthRoot((unsigned long long)d, 2);
    if ((__int128)a0 * a0 == d)
    {
        return nullopt;
    }
    __int128 D = d;
    __int128 m = 0;
    __int128 q = 1;
    __int128 a = a0;
    __int128 p0 = 1;
    __int128 p1 = a;
    __int128 q0 = 0;
```

```
    __int128 q1 = 1;
    constexpr unsigned __int128 IMAX = ((unsigned __int128)1 << 127) - 1;
    while (true)
    {
        auto uq1 = (unsigned __int128)q1;
        if (q1 && uq1 > IMAX / (unsigned __int128)D / uq1)
        {
            return nullopt;
        }
        if (p1 * p1 - D * q1 * q1 == 1)
        {
            break;
        }
        m = q * a - m;
        q = (D - m * m) / q;
        a = (a0 + m) / q;
        __int128 p2 = a * p1 + p0;
        __int128 q2 = a * q1 + q0;
        if (p2 > numeric_limits<long long>::max() || q2 > numeric_limits<long long>::max())
        {
            return nullopt;
        }
        p0 = p1;
        p1 = p2;
        q0 = q1;
        q1 = q2;
    }
    return pair{(long long)p1, (long long)q1};
}
```

`pell(d)`（d 正整数）返回基本解，d 为完全平方数或解越过 `long long` 时返回空。若 $\sqrt{d}$ 的连分数周期长度为 L，找到基本解至多检查 $2L$ 个周期项，额外保存 $O(1)$ 个宽整数。验证：$d=2,3,5,6,7$ 依次得 $(3,2),(2,1),(9,4),(5,2),(8,3)$，$d=4$ 返回空。

7.5.007 圆上整点

`twoSqRep(n)` 依据两平方和定理、Cornacchia 和高斯整数乘法返回全部非负表示；$p\equiv 3\pmod 4$ 的奇指数返回空，`n=0` 返回 `{0,0}`。枚举 $x^2+y^2=n$ 的全部整数解，即原点为心、半径平方为 n 的圆周整点。枚举非负 x、判 $n-x^2$ 是否完全平方，再由坐标交换与正负号生成全部解，用 `set` 消去坐标为零或 $|x|=|y|$ 时的重复。

```
vector<pair<long long, long long>> circlePoints(long long n)
{
    // n 是非负半径平方；返回满足 x*x+y*y=n 的全部整数点。
    assert(n >= 0); // 调试检查，可删
    set<pair<long long, long long>> s;
    for (long long x = 0; x == 0 || x <= n / x; x++)
    {
        long long y2 = n - x * x;
```

```
        long long y = kthRoot((unsigned long long)y2, 2);
        if (y * y != y2)
        {
            continue;
        }
        for (int sx : {-1, 1})
        {
            for (int sy : {-1, 1})
            {
                s.insert({sx * x, sy * y});
                s.insert({sx * y, sy * x});
            }
        }
    }
    return {s.begin(), s.end()};
}
```

`circlePoints(n)`（$n \geq 0$）依赖 7.1.014 整数 k 次方根 的 `kthRoot`，返回去重并按字典序排列的全部整点。时间为 $O(\sqrt{n}(\log n + \log(k+1)))$、空间为 $O(k)$（k 为整点数）；非原点圆心整体平移结果即可。验证：$n = 25$ 得 12 个点（$(0, \pm 5), (\pm 5, 0), (\pm 3, \pm 4), (\pm 4, \pm 3)$），$n = 3$ 为空，$n = 0$ 只有 $(0, 0)$。

8. 组合数学

8.1 组合数

8.1.001 阶乘、逆阶乘与组合数

预处理阶乘表 `fac` 与逆阶乘表 `ifac` 后，$\binom{n}{k}=\frac{n!}{k!(n-k)!}$、$A(n,k)=\frac{n!}{(n-k)!}$ 都退化为常数次乘法。逆阶乘不必逐个求逆：先用一次除法得到最大的 $(n!)^{-1}$，再借 $((i-1)!)^{-1}=(i!)^{-1}\cdot i$ 反向递推，全表只用一次求逆。`init` 支持增量扩表，`fac.size()` 已够时直接返回，扩容时只补算新增前缀，所以系数类型 `T` 必须支持除法（模素数或有理数），且所有用到的 $n!$ 在该类型下可逆——否则某个 $i!\equiv 0$ 会让 `T(1)/fac[n]` 失去意义。

```
// 依赖 1.1.008 静态取模类
template <class T> class Comb
{
    // fac[i]、ifac[i] 分别保存 i! 与其乘法逆元。
    vector<T> fac{1}, ifac{1};

  public:
    void init(int n)
    {
        // n 是要支持的最大上标；把阶乘表扩展到 n，无返回值。
        assert(n >= 0); // 调试检查，可删。
        if ((int)fac.size() > n)
        {
            return;
        }
        int old = fac.size();
        fac.resize(n + 1);
        ifac.resize(n + 1);
        for (int i = old; i <= n; i++)
        {
            fac[i] = fac[i - 1] * T(i);
        }
        ifac[n] = T(1) / fac[n];
        for (int i = n; i >= old; i--)
        {
            ifac[i - 1] = ifac[i] * T(i);
        }
    }

    T C(int n, int k)
    {
        // n 是对象数，k 是选择数；返回组合数，k 不在 [0,n] 时返回 0。
        if (k < 0 || k > n)
        {
            return T(0);
        }
        init(n);
```

```
        return fac[n] * ifac[k] * ifac[n - k];
    }

    T A(int n, int k)
    {
        // n 是对象数，k 是有序选取数；返回排列数，k 不在 [0,n] 时返回 0。
        if (k < 0 || k > n)
        {
            return T(0);
        }
        init(n);
        return fac[n] * ifac[n - k];
    }
};
```

以 `Comb<Z> c` 实例化，`Z` 需支持整数构造与四则运算，`c.C(n,k)`、`c.A(n,k)` 在 $k \notin [0,n]$ 时返回 0；`Z` 为模整数时模数必须大于所有用到的 n。扩表到 n 为 $O(n)$ 时间与空间，之后每次查询 $O(1)$。这是素数模且 n 小于模数时的首选；一旦 n 超过素模 p，某个 $p! \equiv 0$ 破坏可逆性，须改用 8.1.003 Lucas 定理 把上下标按 p 进制拆开，合数模则只能走 8.1.004 扩展 Lucas。分母不可逆或只需要整行组合数时换 8.1.002 杨辉三角 的纯加法递推。

准备 1.1.008 静态取模类 后，可用同一个较大的组合数同时检查扩表、逆阶乘和三次模乘；下面两个结果分别是 $\binom{132}{66}$ 在最常用的两个模数下的已知值：

```
signed main()
{
    // 无参数；核对两个常见模数下的组合数与边界，返回 0。
    using Z998 = MInt<998244353>;
    using Z1e9 = MInt<1000000007>;
    Comb<Z998> a;
    Comb<Z1e9> b;
    assert(a.C(132, 66).val() == 241200029);
    assert(b.C(132, 66).val() == 598375978);
    assert(a.C(0, 0).val() == 1);
    assert(a.C(10, 11).val() == 0);
    return 0;
}
```

8.1.002 杨辉三角

对是否选中某个指定元素分类，就得到 Pascal 恒等式 $\binom{i}{j} = \binom{i-1}{j-1} + \binom{i-1}{j}$，每行两端恒为 1。整张表只用加法递推，不涉及除法，因此模数不保证分母可逆、或需要一整行组合数时它比 8.1.001 阶乘、逆阶乘与组合数 更合适；代价是行数不能太大。

```
template <class T> vector<vector<T>> pascalTri(int n)
{
    // n 是最大行号；返回第 0..n 行杨辉三角，其中 a[i][j]=C(i,j)。
    assert(n >= 0); // 调试检查，可删。
    vector<vector<T>> a(n + 1);
```

```
    for (int i = 0; i <= n; i++)
    {
        a[i].assign(i + 1, T(1));
        for (int j = 1; j < i; j++)
        {
            a[i][j] = a[i - 1][j - 1] + a[i - 1][j];
        }
    }
    return a;
}
```

`pascalTri<T>(n)` 返回第 $0..n$ 行，`T` 只需支持加法和从 1 构造，`a[i][j]` 即 $\binom{i}{j}$。$n=5$ 时第 5 行为 `1 5 10 10 5 1`。整表 $O(n^2)$ 时间与空间，只求单个组合数时不该建整表：那种场景用 8.1.001 阶乘、逆阶乘与组合数 的 $O(1)$ 查询，或结果不取模、能放进 `__int128` 时用 8.1.010 球盒模型与鸽巢原理 里的逐项约分 `comb`。

8.1.003 Lucas 定理

Lucas 定理来自把 $(1+x)^n$ 在模素数 p 下按 p 进制分块：$(1+x)^p \equiv 1+x^p \pmod p$，于是 $n=\sum n_i p^i$ 时 $(1+x)^n \equiv \prod_i (1+x^{p^i})^{n_i}$，比较 x^k 系数即得 $\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod p$，任一位 $k_i > n_i$ 则整体为 0。它只处理素数模，把远大于 p 的上标拆成 $O(\log_p n)$ 位、每位组合数在长度 p 的阶乘表上 $O(1)$ 取得。

```
int powPrime(int a, int b, int p)
{
    // a 是底数，b 是非负指数，p 是素数；返回 a^b mod p。
    assert(b >= 0 && p >= 2); // 调试检查，可删。
    a %= p;
    if (a < 0)
    {
        a += p;
    }
    int ans = 1 % p;
    while (b)
    {
        if (b & 1)
        {
            ans = combMulMod(ans, a, p);
        }
        a = combMulMod(a, a, p);
        b >>= 1;
    }
    return ans;
}

int lucas(int n, int k, int p)
{
    // n、k 是组合数参数，p 是素数；返回 C(n,k) mod p。
    assert(n >= 0 && p >= 2); // 调试检查，可删。
```

```
    if (k < 0 || k > n)
    {
        return 0;
    }
    vector<int> fac(p, 1), ifac(p, 1);
    for (int i = 1; i < p; i++)
    {
        fac[i] = combMulMod(fac[i - 1], i, p);
    }
    ifac[p - 1] = powPrime(fac[p - 1], p - 2, p);
    for (int i = p - 1; i; i--)
    {
        ifac[i - 1] = combMulMod(ifac[i], i, p);
    }
    int ans = 1;
    while (n || k)
    {
        int a = n % p, b = k % p;
        if (b > a)
        {
            return 0;
        }
        ans = combMulMod(combMulMod(ans, fac[a], p), ifac[b], p);
        ans = combMulMod(ans, ifac[a - b], p);
        n /= p;
        k /= p;
    }
    return ans;
}
```

`lucas(n,k,p)` 要求 `n>=0`，且 `p>=2` 是能完整建 p 长阶乘表的素数；实测 `lucas(100,20,7)=0`、`lucas(100,30,7)=0`、`lucas(100,50,7)=4`。每次 $O(p+\log_p n)$ 时间、$O(p)$ 空间，同一 `p` 反复查询时应把 `fac/ifac` 提到外层复用。若 $n<p$，可直接用 8.1.001 阶乘、逆阶乘与组合数；若 $n\geq p$ 且 p 大到不能建表，普通阶乘表同样会在 $p!$ 处失去逆元，必须改为逐位按较小一侧乘除、按需缓存阶乘或采用大素模阶乘算法。模数为合数时改用 8.1.004 扩展 Lucas 按质数幂分别处理再 CRT 合并。

8.1.004 扩展 Lucas

合数模 $m=\prod p_i^{e_i}$ 下 $\binom{n}{k}$ 的困难在于阶乘含 p 因子、逆元不存在，扩展 Lucas 把它拆到每个 p^e 上单独算再 CRT 合并。关键是把 $n!$ 分成 p 的幂部分与「单位部分」：$n!=p^{v_p(n!)}\cdot u(n)$，其中 $v_p(n!)=\sum_{i\geq 1}\lfloor n/p^i\rfloor$（`exponent`），$u(n)$ 是删去所有 p 因子后的乘积模 p^e（`split` 的第二项，靠 $[1,p^e]$ 内非 p 倍数之积的周期性快速幂加余项递归求得）。于是 $\binom{n}{k}$ 在 p^e 下的 p 指数为 $v_p(n!)-v_p(k!)-v_p((n-k)!)$，$\geq e$ 则该分量为 0，否则单位部分可逆，乘上 $p^{\text{指数}}$ 即得；各 $p_i^{e_i}$ 两两互质，`crt` 用 $\sum \text{get}\cdot\frac{m}{m_i}\cdot(\frac{m}{m_i})^{-1}$ 合并。本条依赖 7.1.001 快速幂 的 `mulMod`、`powerMod`、`invMod`，并复用 7.2.009 阶乘模合数 的 `PPFactorial::split`、`fact` 与 `modulus`；只新增组合数与 CRT 层，不能另写同义的质数幂单位阶乘。

```
// 依赖 7.1.003 扩展欧几里得
// 依赖 7.2.009 阶乘模合数
```

```
struct BinomMod
{
    struct PP
    {
        // p 是质因子，e 是 p 在总模数中的指数。
        int p, e;
        // fac 维护模 p^e 的单位阶乘与 p-adic 指数。
        PPFactorial fac;

        PP(int p, int e) : p(p), e(e), fac(p, e)
        {
            // p 是质因子，e 是正指数；建立对应的质数幂阶乘表。
        }

        int modulus() const
        {
            // 无参数；返回本分量的质数幂模数 p^e。
            return fac.modulus();
        }

        int comb(int n, int k) const
        {
            // n、k 是组合数参数；返回 C(n,k) mod p^e，k 不在 [0,n] 时返回 0。
            assert(n >= 0); // 调试检查，可删。
            if (k < 0 || k > n)
            {
                return 0;
            }
            auto [en, un] = fac.split(n);
            auto [ek, uk] = fac.split(k);
            auto [er, ur] = fac.split(n - k);
            int q = en - ek - er;
            if (q >= e)
            {
                return 0;
            }
            int mod = modulus();
            int den = mulMod(uk, ur, mod);
            int z = mulMod(un, *invMod(den, mod), mod);
            return mulMod(z, powerMod(p, q, mod), mod);
        }

        int fact(int n) const
        {
            // n 是非负整数；返回 n! mod p^e。
            return fac.fact(n);
        }
    };

    // mod 是查询所用的正模数。
    int mod;
    // a 保存 mod 的全部质数幂分量。
```

```
    vector<PP> a;

    explicit BinomMod(int mod) : mod(mod)
    {
        // mod 是正模数且每个质数幂分量不超过 5e6；分解 mod 并建立单位阶乘表。
        assert(mod > 0); // 调试检查，可删。
        int n = mod;
        for (int p = 2; p <= n / p; p++)
        {
            if (n % p == 0)
            {
                int e = 0;
                while (n % p == 0)
                {
                    n /= p;
                    e++;
                }
                a.emplace_back(p, e);
            }
        }
        if (n > 1)
        {
            a.emplace_back(n, 1);
        }
    }

    template <class F> long long crt(F get) const
    {
        // get(q) 返回模质数幂 q.mod 的余数；返回 CRT 合并后的模 mod 最小非负余数。
        if (mod == 1)
        {
            return 0;
        }
        long long ans = 0;
        for (const auto &q : a)
        {
            int qmod = q.modulus();
            int m = mod / qmod;
            int x = mulMod(get(q), m, mod);
            x = mulMod(x, *invMod(m, qmod), mod);
            ans = ans >= mod - x ? ans - (mod - x) : ans + x;
        }
        return ans;
    }

    int C(int n, int k) const
    {
        // n、k 是组合数参数；返回 C(n,k) mod mod，k 不在 [0,n] 时返回 0。
        assert(n >= 0); // 调试检查，可删。
        return crt(
            [&](const auto &q)
            {
```

```
                return q.comb(n, k);
            });
    }

    int fact(int n) const
    {
        // n 是非负整数；返回 n! mod mod。
        assert(n >= 0); // 调试检查，可删。
        return crt(
            [&](const auto &q)
            {
                return q.fact(n);
            });
    }
};
```

构造 `BinomMod q(mod)` 后调用 `q.C(n,k)` 或 `q.fact(n)`。构造时分解 `mod`、为每个质数幂 p^e 建长度 p^e 的 `f` 表；当前实现用调试断言要求每个 $p^e \le 5 \times 10^6$，避免误建无法承受的表。构造为 $O(\sqrt{m} + \sum p^e)$ 时间与 $O(\sum p^e)$ 空间，单次查询 $O(\sum \log_p n)$。素数模且 n 小于模数时直接用 8.1.001 阶乘、逆阶乘与组合数，素数模且 n 超模数时用 8.1.003 Lucas 定理；合数模才使用本条。需要按 `n,k,mod` 一次性调用而不持有对象时见 8.1.005 任意模数组合数 的薄封装。

公开的 `crt(get)` 可合并调用者按各质数幂计算的余数，例如重建 $23 \bmod 12$：

```
signed main()
{
    BinomMod q(12);
    int x = q.crt([](const auto &part)
    {
        return 23 % part.modulus();
    });
    assert(x == 11);
}
```

下面固定覆盖模 1、质数幂、混合质数幂和越界排名：

```
signed main()
{
    assert(BinomMod(1).C(20, 7) == 0);
    assert(BinomMod(8).C(10, 3) == 0);
    assert(BinomMod(72).C(20, 6) == 24);
    BinomMod q(60);
    assert(q.C(40, 41) == 0);
}
```

8.1.005 任意模数组合数

任意正模数下的组合数与扩展 Lucas 是同一数学算法，这里不重复底层类，只依赖 8.1.004 扩展 Lucas 的 `BinomMod`，包一层按 `n,k,mod` 一次性调用的入口；区别只在于是否预先持有模数对象。

```cpp
// 依赖 7.1.003 扩展欧几里得
// 依赖 7.2.009 阶乘模合数
int binomAnyMod(int n, int k, int mod)
{
    // n、k 是组合数参数，mod 是正模数且每个质数幂分量不超过 5e6；返回 C(n,k) mod mod。
    return BinomMod(mod).C(n, k);
}
```

`binomAnyMod(n,k,mod)` 每次都新建 `BinomMod`，因此同样要求 `mod` 的每个质数幂分量不超过 5×10^6；实测 `binomAnyMod(10,3,12)=0`、`binomAnyMod(10,4,1000)=210`。单次构造加查询的开销与 8.1.004 扩展 Lucas 相同，只适合零散查询；同一 `mod` 反复调用时应持有一个 `BinomMod` 复用预处理。

8.1.006 Vandermonde 卷积

从 m 个第一类元素与 n 个第二类元素中共选 r 个，按第一类恰选了 k 个分类，各类互斥且并起来是全部方案，于是有 Vandermonde 卷积 $\sum_k \binom{m}{k}\binom{n}{r-k}=\binom{m+n}{r}$。下面这段既是它的直接实现，也可当作核对组合数接口的验证：把左侧逐项求和与右侧 $\binom{m+n}{r}$ 比对。求和下标已收紧到 $k\in[\max(0,r-n),\min(m,r)]$，落在此外的组合数必为 0。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 8.1.001 阶乘、逆阶乘与组合数
template <class C> auto vandermonde(int m, int n, int r, C comb)
{
    // m、n 是两组大小，r 是总选择数，comb(x,y) 返回 C(x,y)；返回 Vandermonde 左侧求和。
    assert(m >= 0 && n >= 0); // 调试检查，可删。
    using T = decltype(comb(0, 0));
    T ans{};
    for (int k = max<int>(0, r - n); k <= min(m, r); k++)
    {
        ans += comb(m, k) * comb(n, r - k);
    }
    return ans;
}
```

把任一组合数接口作为回调传给 `vandermonde(m,n,r,comb)`，`comb(x,y)` 对合法参数返回同一系数类型，返回值类型由 `decltype(comb(0,0))` 推断；右侧直接写 `comb(m+n,r)` 即可，固定查验值为 `vandermonde(2,3,2,comb)==10`。共 $O(\min(m,r)-\max(0,r-n)+1)$ 次组合数调用、$O(1)$ 额外空间。`comb` 通常取 8.1.001 阶乘、逆阶乘与组合数 的 `Comb<Z>::C`（素数模）或 8.1.004 扩展 Lucas 的 `BinomMod::C`（合数模）；本条本身只做求和，不预处理阶乘。

固定素数模下可把 `Comb<Z>::C` 包成 Lambda：

```cpp
signed main()
{
    Comb<Z> c;
    auto comb = [&](int n, int k)
    {
        return c.C(n, k);
    };
```

```
    assert(vandermonde(2, 3, 2, comb) == Z(10));
}
```

8.1.007 容斥原理

容斥把并集计数写成所有非空交集按集合个数奇加偶减：$|\bigcup A_i| = \sum_{\emptyset \neq S} (-1)^{|S|+1} |\bigcap_{i \in S} A_i|$，前提是各交集计数针对同一全集。这里的示例把它落到「$[1, n]$ 中至少被某个 d_i 整除的整数个数」：子集 S 对应除数的最小公倍数 lcm，其倍数个数为 $\lfloor n / \operatorname{lcm} \rfloor$，按 `popcount(s)` 的奇偶加减。累乘 `lcm` 时用 `l > n / (d[i]/g)` 提前判溢出并跳过该子集（此时 $\operatorname{lcm} > n$，贡献本就为 0），避免乘积炸掉 `int`。

```
namespace incl_excl
{
int countMult(int n, vector<int> d)
{
    // n 是闭区间上界，d 是正除数表；返回 [1,n] 中至少被一个 d[i] 整除的整数个数。
    assert(n >= 0 && d.size() < 63); // 调试检查，可删。
    for (int x : d)
    {
        assert(x > 0); // 调试检查，可删。
    }
    int ans = 0;
    for (unsigned long long s = 1; s < (1ULL << d.size()); s++)
    {
        int l = 1;
        bool ok = true;
        for (int i = 0; i < (int)d.size(); i++)
        {
            if (!(s >> i & 1))
            {
                continue;
            }
            int g = gcd(l, d[i]);
            if (l > n / (d[i] / g))
            {
                ok = false;
                break;
            }
            l *= d[i] / g;
        }
        if (ok)
        {
            ans += (popcount(s) & 1 ? 1 : -1) * (n / l);
        }
    }
    assert(0 <= ans && ans <= n); // 调试检查，可删。
    return ans;
}
}
```

`countMult(n, d)` 要求 `n>=0`、每个除数为正且个数 $k < 63$（否则子集掩码溢出），返回 $[1, n]$ 内至少被

一个 d_i 整除的整数个数；还须保证容斥过程的带符号部分和能由 `int` 表示。枚举共 $O(2^k k)$ 时间、$O(1)$ 额外空间。除数很多时，先删掉被更小除数整除的冗余项；值域不大可筛出所有倍数，具有额外数论结构时再用 Möbius 或倍数 DP，不能用 8.1.009 单位根反演 直接替代任意除数集合的并集。第 k 大与子集最小值和之间的转换见 8.1.008 扩展 min-max 容斥与 KMAX-MIN 反演。

8.1.008 扩展 min-max 容斥与 KMAX-MIN 反演

普通 min-max 容斥是 $\max(S)=\sum_{\emptyset\neq T\subseteq S}(-1)^{|T|+1}\min(T)$；扩展版把「第 k 大」写成各子集最小值的带二项式系数组合：$\operatorname{kthmax}(S)=\sum_T(-1)^{|T|-k}\binom{|T|-1}{k-1}\min(T)$。按大小聚合后只依赖每个尺寸 j 的子集最小值之和 mn_j，得 第k 大 $=\sum_{j\ge k}(-1)^{j-k}\binom{j-1}{k-1}mn_j$。为避免巨大交错部分和，代码分别在三个互素大素数下递推二项式系数，再用 CRT 恢复唯一的 64 位有符号答案；三模乘积远大于整个 64 位值域。

```
namespace minmax_inv
{
int kthMaxMinSum(const vector<int> &mn, int k)
{
    // mn[j] 是所有大小为 j 的子集最小值之和且 mn[0] 不用，k 从 1 开始；返回第 k 大元素。
    int n = (int)mn.size() - 1;
    assert(1 <= k && k <= n); // 调试检查，可删。
    constexpr array<long long, 3> ps{998244353, 1000000007, 1000000009};
    assert(n < ps[0]); // 调试检查，可删。
    array<long long, 3> rem{};
    for (int z = 0; z < 3; z++)
    {
        long long mod = ps[z];
        vector<long long> inv(n - k + 1);
        if (n > k)
        {
            inv[1] = 1;
        }
        for (int i = 2; i <= n - k; i++)
        {
            inv[i] = mod - (mod / i) * inv[mod % i] % mod;
        }
        long long c = 1;
        long long ans = 0;
        for (int j = k; j <= n; j++)
        {
            if (j > k)
            {
                c = c * (j - 1) % mod * inv[j - k] % mod;
            }
            long long x = mn[j] % mod;
            if (x < 0)
            {
                x += mod;
            }
            long long v = c * x % mod;
```

```
            if ((j - k) & 1)
            {
                ans -= v;
                if (ans < 0)
                {
                    ans += mod;
                }
            }
            else
            {
                ans += v;
                if (ans >= mod)
                {
                    ans -= mod;
                }
            }
        }
        rem[z] = ans;
    }
    __int128 ans = rem[0];
    __int128 m = ps[0];
    for (int z = 1; z < 3; z++)
    {
        long long mod = ps[z];
        long long a = rem[z] - (long long)(ans % mod);
        if (a < 0)
        {
            a += mod;
        }
        long long b = m % mod;
        long long e = mod - 2;
        long long iv = 1;
        while (e)
        {
            if (e & 1)
            {
                iv = iv * b % mod;
            }
            b = b * b % mod;
            e >>= 1;
        }
        ans += m * (a * iv % mod);
        m *= mod;
    }
    if (ans > m / 2)
    {
        ans -= m;
    }
    assert(numeric_limits<int>::min() <= ans && ans <= numeric_limits<int>::max()); // 调试检查,
    ↪ 可删。
    return (int)ans;
}
```

```
}
```

`kthMaxMinSum(mn, k)` 中 `mn[j]` 是所有大小为 j 的子集最小值之和（`mn[0]` 不用），每个和与最终答案都必须能放进 `int`；`k` 从 1 计，数组长度小于三个模数。三次线性递推与常数次 CRT 共 $O(n)$ 时间、$O(n)$ 额外空间，不会因最终抵消前的部分和而溢出。这套反演用于「各尺寸子集最小值和已知、要还原第 k 名」；只需并集大小时用 8.1.007 容斥原理。

8.1.009 单位根反演

单位根的正交性给出指示函数 $[m \mid t] = \frac{1}{m}\sum_{j=0}^{m-1}\omega^{jt}$（$\omega$ 是 m 次本原单位根）。把它套到生成函数 $A(x)=\sum_k a_k x^k$ 上，取 $t = k - r$ 就能滤出指数模 m 余 r 的那些项：$\sum_{k\equiv r\,(m)} a_k = \frac{1}{m}\sum_{j=0}^{m-1}\omega^{-jr}A(\omega^j)$，非目标指数在求和中相位抵消。这里在模 998244353 的有限域里做，$\omega = 3^{(p-1)/m}$ 要求 $m \mid p-1$；`rootFilter` 对每个 ω^j 用 Horner（`a.rbegin()` 起）求 $A(\omega^j)$，乘上 ω^{-jr} 累加，最后乘 m^{-1}。

```
namespace roots_unity
{
constexpr int mod = 998244353;

int qpow(int a, int n)
{
    // a 是底数，n 是非负指数；返回模 mod 幂。
    assert(n >= 0); // 调试检查，可删。
    a %= mod;
    if (a < 0)
    {
        a += mod;
    }
    int r = 1;
    while (n)
    {
        if (n & 1)
        {
            r = r * a % mod;
        }
        a = a * a % mod;
        n >>= 1;
    }
    return r;
}

int rootFilter(const vector<int> &a, int m, int rem)
{
    // a 是多项式系数，m 是整除 mod-1 的正整数，rem 是余数；返回次数同余 rem 的系数和。
    assert(m > 0 && (mod - 1) % m == 0); // 调试检查，可删。
    rem %= m;
    if (rem < 0)
    {
```

```
        rem += m;
    }
    int w = qpow(3, (mod - 1) / m);
    int ans = 0;
    for (int j = 0; j < m; j++)
    {
        int x = qpow(w, j);
        int y = 0;
        for (auto it = a.rbegin(); it != a.rend(); ++it)
        {
            int c = *it % mod;
            if (c < 0)
            {
                c += mod;
            }
            y = (y * x + c) % mod;
        }
        int e = ((m - rem) % m) * j % (mod - 1);
        ans = (ans + y * qpow(w, e)) % mod;
    }
    return ans * qpow(m, mod - 2) % mod;
}
}
```

`rootFilter(a, m, rem)` 中 `a` 是多项式系数（`a[i]` 为 x^i 系数），`m` 必须整除 $p-1$，`rem` 是目标余数（内部先规约到 $[0,m)$）；返回次数同余 `rem` 的系数和。直接求值 $O(mn)$ 时间、$O(1)$ 额外空间（n 为多项式次数）。当 m 很小而需要频繁分组，或题目本身要卷积时，用第 10 章的 NTT 把多点求值降到 $O(n\log n)$ 更划算；这里的朴素版胜在实现短、无需搬多项式全家桶。它与 8.1.007 容斥原理 都在做「按同余类筛选」，但单位根反演处理的是幂级数系数而非集合并。

8.1.010 球盒模型与鸽巢原理

球盒计数先分清球、盒是否有标号，能否为空，有无容量限制。本条实现最常用的隔板法：无标号球放进有标号盒，允许空盒时为 $\binom{b+m-1}{m-1}$，要求非空时为 $\binom{b-1}{m-1}$。`comb` 每轮先约分再做一次受检乘法，默认用 64 位；需要更宽结果时由调用者显式选择类型。鸽巢下界用商与余数计算，不形成可能溢出的 `objects+boxes-1`。

```
namespace balls_boxes
{
template <class T = long long>
T comb(T n, int k)
{
    // n 是非负上标，k 是下标；返回二项式系数。
    static_assert(numeric_limits<T>::is_integer && numeric_limits<T>::is_signed &&
                  sizeof(T) >= sizeof(long long));
    assert(n >= 0); // 调试检查，可删。
    if (k < 0 || (T)k > n)
    {
        return T(0);
    }
```

```
    if ((T)k > n - k)
    {
        k = (int)(n - k);
    }
    T ans = 1;
    for (int i = 1; i <= k; i++)
    {
        T x = n - k + i;
        int y = i;
        int g = gcd((int)(x % y), y);
        x /= g;
        y /= g;
        assert(ans % y == 0); // 调试检查，可删。
        ans /= y;
        ans *= x;
    }
    return ans;
}

template <class T = long long>
T unlabBoxDist(int balls, int boxes, bool nonempty)
{
    // balls 是球数，boxes 是盒数，nonempty 表示每盒非空；返回方案数。
    static_assert(numeric_limits<T>::is_integer && numeric_limits<T>::is_signed &&
                  sizeof(T) >= sizeof(long long));
    assert(balls >= 0 && boxes >= 0); // 调试检查，可删。
    if (boxes == 0)
    {
        return T(balls == 0);
    }
    if (nonempty)
    {
        return balls < boxes ? T(0) : comb<T>((T)balls - 1, boxes - 1);
    }
    T b = balls, m = boxes;
    return comb<T>(b + m - 1, boxes - 1);
}

int pigeonBound(int objects, int boxes)
{
    // objects 是对象数，boxes 是正盒数；返回必有一盒至少包含的对象数。
    assert(objects >= 0 && boxes > 0); // 调试检查，可删。
    return objects / boxes + (objects % boxes != 0);
}
}
```

两个计数函数默认返回 `long long`；调用者须保证上标和答案能由 `T` 表示，需要 128 位精确值时调用 `comb<__int128>` 或 `unlabBoxDist<__int128>`。`comb` 为 $O(\min(k, n-k))$ 时间、$O(1)$ 空间。超过 128 位用 14.1.001 高精度整数（BigInt），只求模意义答案则用 8.1.001 阶乘、逆阶乘与组合数 或 8.1.004 扩展 Lucas。

8.1.011 q-二项式与 q-Lucas

Gaussian 二项式由 $\begin{bmatrix} n \\ k \end{bmatrix}_q = \prod_{i=1}^{k} \frac{[n-k+i]_q}{[i]_q}$ 定义，其中 $[i]_q = 1 + q + \cdots + q^{i-1}$。模素数 p 时，直接除法会在某个 $[i]_q = 0$ 后失效；令 d 为首个正的零 q-整数，若 $q = 0$ 或遍历到 $p-1$ 仍无零则取 $d = p$。对 $0 \le n, k < p$ 有 q-Lucas 分解 $\begin{bmatrix} n \\ k \end{bmatrix}_q = \binom{\lfloor n/d \rfloor}{\lfloor k/d \rfloor} \begin{bmatrix} n \bmod d \\ k \bmod d \end{bmatrix}_q \pmod p$。低位的 q-阶乘均非零，高位是普通组合数；$q = 0$ 时合法组合数恒为 1，$q = 1$ 时退化为普通组合数，二者都无需另写特判。

```
class QBinomPrime
{
    // p 是素数模，q 是规范到 [0,p) 的参数，d 是首个零 q-整数的位置；无零时 d=p。
    int p, q, d;
    // fac/ifac 是 0..d-1 的 q-阶乘及逆元，cfac/cifac 是高位普通阶乘及逆元。
    vector<int> fac, ifac, cfac, cifac;

    int mul(int a, int b) const
    {
        // a、b 是模 p 乘数；返回 ab mod p。
        return combMulMod(a, b, p);
    }

    int power(int a, int n) const
    {
        // a 是模 p 底数，n 是非负指数；返回 a^n mod p。
        int r = 1;
        while (n)
        {
            if (n & 1)
            {
                r = mul(r, a);
            }
            a = mul(a, a);
            n >>= 1;
        }
        return r;
    }

    int small(int n, int k) const
    {
        // 0<=n<d，k 是下标；返回对应低位 q-二项式，k 不在 [0,n] 时返回 0。
        if (k < 0 || k > n)
        {
            return 0;
        }
        return mul(mul(fac[n], ifac[k]), ifac[n - k]);
    }

    int ordinary(int n, int k) const
    {
        // n 是高位上标，k 是高位下标；返回普通 C(n,k) mod p。
        if (k < 0 || k > n)
        {
```

```
            return 0;
        }
        return mul(mul(cfac[n], cifac[k]), cifac[n - k]);
    }

public:
    QBinomPrime(int p, int q) : p(p), q(q)
    {
        // p 是可承担 O(p) 预处理的素数，q 是 q-二项式参数；建立 q-Lucas 查询表。
        assert(p >= 2); // 调试检查，可删。
        this->q %= p;
        if (this->q < 0)
        {
            this->q += p;
        }
        fac = {1};
        vector<int> num;
        int x = 1;
        for (int i = 1; i < p; i++)
        {
            if (x == 0)
            {
                break;
            }
            num.push_back(x);
            fac.push_back(mul(fac.back(), x));
            x = (mul(this->q, x) + 1) % p;
        }
        d = fac.size();
        ifac.resize(d);
        ifac[d - 1] = power(fac[d - 1], p - 2);
        for (int i = d - 1; i >= 1; i--)
        {
            ifac[i - 1] = mul(ifac[i], num[i - 1]);
        }

        int m = (p - 1) / d;
        cfac.assign(m + 1, 1);
        cifac.assign(m + 1, 1);
        for (int i = 1; i <= m; i++)
        {
            cfac[i] = mul(cfac[i - 1], i);
        }
        cifac[m] = power(cfac[m], p - 2);
        for (int i = m; i >= 1; i--)
        {
            cifac[i - 1] = mul(cifac[i], i);
        }
    }

    int C(int n, int k) const
    {
```

```
        // n、k 是 q-二项式参数且 0<=n<p；返回 [n choose k]_q mod p，k 越界时返回 0。
        assert(0 <= n && n < p); // 调试检查，可删。
        if (k < 0 || k > n)
        {
            return 0;
        }
        return mul(ordinary(n / d, k / d), small(n % d, k % d));
    }
};
```

构造 `QBinomPrime cb(p,q)` 后用 `cb.C(n,k)`，要求 `p` 为素数、$0 \le n < p$，并且 p 小到能承受线性预处理。固定查验值为 `QBinomPrime(7,2).C(6,3)==2`。设 q-整数周期为 d，构造 $O(d + p/d)$ 时间与空间，单次查询 $O(1)$；同一 `p,q` 的多组询问必须复用对象。若 $n \ge p$，还要继续按 p 进制处理普通组合数高位，不能直接套本接口；普通二项式见 8.1.003 Lucas 定理，多项式形式的快速运算见第 10 章。

8.1.012 二项式恒等式与反演

约定 $n \ge 0$ 且 $\binom{n}{k} = 0$ 对所有 $k \notin [0, n]$ 成立。二项式定理及其系数形式为

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^{n-k} y^k,$$
$$[x^k](1+x)^n = \binom{n}{k}.$$

分别代入 $x = 1$ 与 $x = -1$，得到

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n, \qquad \sum_{k=0}^{n} (-1)^k \binom{n}{k} = [n = 0].$$

对 $(1+x)^n$ 求导后乘 x，再令 $x = 1$，可依次得到加权和。下式要求 $n \ge 2$；$n = 0, 1$ 直接代入求和式。

$$\sum_{k} k \binom{n}{k} = n 2^{n-1},$$
$$\sum_{k} k(k-1) \binom{n}{k} = n(n-1) 2^{n-2},$$
$$\sum_{k} k^2 \binom{n}{k} = n(n+1) 2^{n-2}.$$

Vandermonde 卷积见 8.1.006 Vandermonde 卷积。令其中一侧退化，可得 hockey-stick 恒等式；其条件为 $n \ge r \ge 0$：

$$\sum_{k=r}^{n} \binom{k}{r} = \binom{n+1}{r+1}.$$

二项式反演必须成对使用同一符号约定。对任意序列，若

$$b_n = \sum_{k=0}^{n} \binom{n}{k} a_k,$$

则恰有

$$a_n = \sum_{k=0}^{n} (-1)^{n-k} \binom{n}{k} b_k.$$

这些等式是整数恒等式，取模时可直接逐项取模，不涉及除法；按 $(-1)^k$ 书写的等价反演式需要同时调整两边符号，不能只替换一个求和式。

8.2 经典数列与计数

8.2.001 Fibonacci / Lucas 数

矩阵快速幂 $\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$ 展开后可消去冗余乘法，得到倍增恒等式 $F_{2n} = F_n(2F_{n+1} - F_n)$、$F_{2n+1} = F_n^2 + F_{n+1}^2$。`fibPair` 每次递归把下标折半、同时维护相邻对 $\{F_n, F_{n+1}\}$，比通用 2×2 矩阵幂常数更小；Lucas 数则由 $L_n = 2F_{n+1} - F_n$ 直接换算。模乘调用 `combMulMod`，常见小模数直接使用 64 位乘法，接近 `long long` 上界时才局部使用 `__int128`；$n = 0$ 时返回 $\{0, 1 \bmod mod\}$，兼容 `mod=1`。

```
array<int, 2> fibPair(unsigned long long n, int mod)
{
    // n 是下标，mod 是正模数；返回 {F_n,F_{n+1}} mod mod。
    assert(mod > 0); // 调试检查，可删。
    if (n == 0)
    {
        return {0, 1 % mod};
    }
    auto [a, b] = fibPair(n >> 1, mod);
    int t = b >= mod - b ? b - (mod - b) : b + b;
    t -= a;
    if (t < 0)
    {
        t += mod;
    }
    int c = combMulMod(a, t, mod);
    int x = combMulMod(a, a, mod);
    int y = combMulMod(b, b, mod);
    int d = x >= mod - y ? x - (mod - y) : x + y;
    if (n & 1)
    {
        int next = c >= mod - d ? c - (mod - d) : c + d;
        return {d, next};
    }
```

```
    return {c, d};
}

int lucasNumber(unsigned long long n, int mod)
{
    // n 是下标，mod 是正模数；返回 Lucas 数 L_n mod mod。
    assert(mod > 0); // 调试检查，可删。
    if (n == 0)
    {
        return 2 % mod;
    }
    auto [fn, fn1] = fibPair(n, mod);
    int ans = fn1 >= mod - fn1 ? fn1 - (mod - fn1) : fn1 + fn1;
    ans -= fn;
    return ans < 0 ? ans + mod : ans;
}
```

`fibPair(n,mod)` 返回 $\{F_n, F_{n+1}\} \bmod mod$，`lucasNumber(n,mod)` 返回 $L_n \bmod mod$；模 10^9+7 下 `fibPair(10)={55,89}`、`lucasNumber(10)=123`。$O(\log n)$ 时间、$O(\log n)$ 递归栈。需要精确大整数时沿用倍增式，改以 14.1.001 高精度整数（BigInt） 保存两项并去掉取模。这是唯一按下标 $O(\log n)$ 求值的数列；下面 Catalan、Motzkin、Bell 等都是 $O(n^2)$ 或 $O(n\sqrt{n})$ 的整段递推，求单个高位项时反而应回到对应的闭式或矩阵幂。

8.2.002 Catalan 数

合法括号序列、凸多边形三角剖分、二叉树形态等都由 Catalan 数计数。按第一个匹配位置（或根）把对象拆成互相独立的左右两块，得卷积递推 $C_i = \sum_{j=0}^{i-1} C_j C_{i-1-j}$，$C_0 = 1$；闭式 $C_n = \frac{1}{n+1}\binom{2n}{n}$ 只在分母可逆或整数约分时才直接用。下面返回 $C_0, \dots, C_n$，`T` 只需支持加法、乘法和从 1 构造。

```
template <class T> vector<T> catalan(int n)
{
    // n 是要计算的最大下标；返回 C_0 到 C_n 的 Catalan 数。
    assert(n >= 0); // 调试检查，可删。
    vector<T> a(n + 1);
    a[0] = 1;
    for (int i = 1; i <= n; i++)
    {
        for (int j = 0; j < i; j++)
        {
            a[i] += a[j] * a[i - 1 - j];
        }
    }
    return a;
}
```

`catalan<T>(n)` 返回 $C_0, \dots, C_n$，卷积递推为 $O(n^2)$ 时间、$O(n)$ 输出空间。模素数且 $2n$ 小于模数时，用 8.1.001 阶乘、逆阶乘与组合数 按闭式预处理整段只需 $O(n)$；用形式幂级数时则由 $C(x) = 1 + xC(x)^2$ 做 Newton 迭代。图的生成树计数不属于这类一维递推，见 8.2.011 Matrix-Tree / Kirchhoff 生成树计数。

前若干项 $C_0, C_1, \ldots$：$1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900$（64 位有符号内到 $C_{35} = 3116285494907301262$ 不溢出，C_{36} 溢出）。

8.2.003 Motzkin 数

Motzkin 数统计不越过横轴、每步为上/下/平的格路（也对应圆上非交弦等结构）。按首步分类：若首步是水平步，剩下是长度 $i-1$ 的 Motzkin 路；若首步上升，则到首次回零之间夹一段 Motzkin 路、之后再接一段，于是 $M_i = M_{i-1} + \sum_{j=0}^{i-2} M_j M_{i-2-j}$，$M_0 = 1$，即一个线性项加 Catalan 型卷积。

```cpp
template <class T> vector<T> motzkin(int n)
{
    // n 是要计算的最大下标；返回 M_0 到 M_n 的 Motzkin 数。
    assert(n >= 0); // 调试检查，可删。
    vector<T> a(n + 1);
    a[0] = 1;
    for (int i = 1; i <= n; i++)
    {
        a[i] = a[i - 1];
        for (int j = 0; j + 2 <= i; j++)
        {
            a[i] += a[j] * a[i - 2 - j];
        }
    }
    return a;
}
```

`motzkin<T>(n)` 返回 $M_0, \ldots, M_n$，卷积递推 $O(n^2)$ 时间、$O(n)$ 空间。与 8.2.002 Catalan 数 同族但多了水平步：只有上、下步且必须回到起点时是 Catalan，允许平步则是 Motzkin。

前若干项 $M_0, M_1, \ldots$：$1, 1, 2, 4, 9, 21, 51, 127, 323, 835, 2188, 5798, 15511$。

8.2.004 Bell 数

Bell 数 B_n 数把 n 个带标号元素划分成若干非空无序集合的方案。递推 $B_{n+1} = \sum_{k=0}^{n} \binom{n}{k} B_k$ 来自按「含最后一个元素的块」大小分类。代码用等价的 Bell 三角实现：每行首项抄上一行末项，同行向右做前缀累加 $a[i][j] = a[i][j-1] + a[i-1][j-1]$，每行首项 `a[i][0]` 即 B_i，避免显式算组合数。

```cpp
template <class T> vector<T> bell(int n)
{
    // n 是要计算的最大下标；返回 B_0 到 B_n 的 Bell 数。
    assert(n >= 0); // 调试检查，可删。
    vector<vector<T>> a(n + 1, vector<T>(n + 1));
    vector<T> b(n + 1);
    a[0][0] = 1;
    b[0] = 1;
    for (int i = 1; i <= n; i++)
    {
        a[i][0] = a[i - 1][i - 1];
```

```
        b[i] = a[i][0];
        for (int j = 1; j <= i; j++)
        {
            a[i][j] = a[i][j - 1] + a[i - 1][j - 1];
        }
    }
    return b;
}
```

`bell<T>(n)` 返回 $B_0,\dots,B_n$，Bell 三角为 $O(n^2)$ 时间与空间。Bell 数是第二类 Stirling 数按块数求和 $B_n=\sum_k S(n,k)$，需要按块数分开计数时用 8.2.005 Stirling 数 的第二类；只关心总划分数则本条足够。模素数下需要整段大规模结果时，可由 EGF e^{e^x-1} 调用第 10 章形式幂级数指数。

前若干项 $B_0,B_1,\dots$：$1,1,2,5,15,52,203,877,4140,21147,115975,678570$。

8.2.005 Stirling 数

第一类 Stirling 数按下标计排列的环数，第二类计集合划分成指定块数的方案。加入第 i 个新元素时，无符号第一类满足 $\begin{bmatrix}i\\j\end{bmatrix}=\begin{bmatrix}i-1\\j-1\end{bmatrix}+(i-1)\begin{bmatrix}i-1\\j\end{bmatrix}$；代码采用对应下降幂 $x^{\underline{n}}$ 系数的有符号版本 $s(i,j)=s(i-1,j-1)-(i-1)s(i-1,j)$，需要环数时取绝对值。第二类中，新元素要么独占新块，要么并入已有 j 块之一，故 $\begin{Bmatrix}i\\j\end{Bmatrix}=\begin{Bmatrix}i-1\\j-1\end{Bmatrix}+j\begin{Bmatrix}i-1\\j\end{Bmatrix}$。

```
template <class T> vector<vector<T>> stirling1(int n)
{
    // n 是最大行号；返回有符号第一类 Stirling 数 s(i,j) 的第 0..n 行。
    assert(n >= 0); // 调试检查，可删。
    vector<vector<T>> a(n + 1, vector<T>(n + 1));
    a[0][0] = 1;
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= i; j++)
        {
            a[i][j] = a[i - 1][j - 1] - T(i - 1) * a[i - 1][j];
        }
    }
    return a;
}

template <class T> vector<vector<T>> stirling2(int n)
{
    // n 是最大行号；返回第二类 Stirling 数 S(i,j) 的第 0..n 行。
    assert(n >= 0); // 调试检查，可删。
    vector<vector<T>> a(n + 1, vector<T>(n + 1));
    a[0][0] = 1;
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= i; j++)
        {
            a[i][j] = a[i - 1][j - 1] + T(j) * a[i - 1][j];
        }
```

```
    }
    return a;
}
```

`stirling1<T>(n)`、`stirling2<T>(n)` 各返回一张 $(n+1)\times(n+1)$ 的三角表，$a[i][j]$ 为对应 Stirling 数；均 $O(n^2)$ 时间与空间。只查一个第二类 $S(n,k)$ 且模素数时，可用 $S(n,k)=\frac{1}{k!}\sum_j(-1)^j\binom{k}{j}(k-j)^n$ 配 8.1.001 阶乘、逆阶乘与组合数，以 $O(k\log n)$ 时间求值；需要小规模整表时用本条。集合划分不限块数就求和成 8.2.004 Bell 数。

在 $m,n\ge 0$ 时，交错幂和与第二类 Stirling 数满足下式；当 $m=n=0$ 时约定 $0^0=1$：

$$\sum_{k=0}^{n}(-1)^k\binom{n}{k}k^m=(-1)^n n!\left\{{m\atop n}\right\}.$$

因此 $m<n$ 时为 0，$m=n$ 时为 $(-1)^n n!$。当模素数 p 且 $n<p$ 时，因 $n!$ 可逆，可以反解为

$$\left\{{m\atop n}\right\}=\frac{1}{n!}\sum_{k=0}^{n}(-1)^{n-k}\binom{n}{k}k^m\pmod p.$$

若模数为合数，或 $p\mid n!$，不能直接使用这个分式形式。

前几行速查。代码返回有符号第一类 $s(n,k)$，下表只展示无符号绝对值 $|s(n,k)|$：

$n\backslash k$	0	1	2	3	4	5	6	7
0	1	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0
2	0	1	1	0	0	0	0	0
3	0	2	3	1	0	0	0	0
4	0	6	11	6	1	0	0	0
5	0	24	50	35	10	1	0	0
6	0	120	274	225	85	15	1	0
7	0	720	1764	1624	735	175	21	1

第二类 $\left\{{n\atop k}\right\}$（划分成 k 个非空块）：

$n\backslash k$	0	1	2	3	4	5	6	7
0	1	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0
2	0	1	1	0	0	0	0	0
3	0	1	3	1	0	0	0	0
4	0	1	7	6	1	0	0	0
5	0	1	15	25	10	1	0	0
6	0	1	31	90	65	15	1	0
7	0	1	63	301	350	140	21	1

8.2.006 Eulerian 数

Eulerian 数 $\left\langle {n \atop k} \right\rangle$ 计 n 元排列中恰有 k 个下降位置（$a_i > a_{i+1}$）的排列数，下标 k 从 0 起。把最大元素插入长度 $i-1$ 排列的 i 个空隙：插进已有的某个下降处或末尾（保持下降数，共 $j+1$ 种），或插进上升处新增一个下降（$i-j$ 种），得递推 $\left\langle {i \atop j} \right\rangle = (j+1)\left\langle {i-1 \atop j} \right\rangle + (i-j)\left\langle {i-1 \atop j-1} \right\rangle$。有些资料把递增段数 $k+1$ 作为列下标，本实现始终用下降位置数 k。

```cpp
template <class T> vector<vector<T>> eulerian(int n)
{
    // n 是最大排列长度；返回第 0..n 行 Eulerian 数，列下标是下降位置数。
    assert(n >= 0); // 调试检查，可删。
    vector<vector<T>> a(n + 1, vector<T>(n + 1));
    a[0][0] = 1;
    for (int i = 1; i <= n; i++)
    {
        for (int j = 0; j < i; j++)
        {
            if (j)
            {
                a[i][j] += T(i - j) * a[i - 1][j - 1];
            }
            a[i][j] += T(j + 1) * a[i - 1][j];
        }
    }
    return a;
}
```

`eulerian<T>(n)` 返回三角表，下标为元素数与下降数，$O(n^2)$ 时间与空间。Eulerian 数与 Worpitzky 恒等式、$\sum k^n$ 幂和相关联；若只需幂和本身，8.2.007 Bernoulli 数与幂和 的 Faulhaber 公式更直接。它与第二类 Stirling 数可互相线性表出，按题目要「下降数分布」还是「块数分布」二选一。

前几行速查 $\left\langle {n \atop k} \right\rangle$（$k$ 为下降位置数，从 0 起；每行对称、行和为 $n!$）：

$n\backslash k$	0	1	2	3	4	5	6
1	1						
2	1	1					
3	1	4	1				
4	1	11	11	1			
5	1	26	66	26	1		
6	1	57	302	302	57	1	
7	1	120	1191	2416	1191	120	1

8.2.007 Bernoulli 数与幂和

本条采用 $B_1 = +\frac{1}{2}$ 的 Bernoulli 约定，由差分表以 $a[j] = \frac{1}{j+1}$ 起、反复做 $a[j-1] \leftarrow j(a[j-1]-a[j])$ 得到。对应的 Faulhaber 公式是 $\sum_{i=1}^{n} i^m = \frac{1}{m+1}\sum_{k=0}^{m} \binom{m+1}{k} B_k n^{m+1-k}$；例如 $m=1$ 给出 $\frac{n^2+n}{2}$。因递推含除法，系数类型 `T` 必须支持除法，模素数时各分母须可逆。

```cpp
template <class T> vector<T> bernoulli(int n)
{
    // n 是要计算的最大下标；返回 B_0 到 B_n，采用 B_1=+1/2 的约定。
    assert(n >= 0); // 调试检查，可删。
    vector<T> a(n + 1), b(n + 1);
    for (int m = 0; m <= n; m++)
    {
        a[m] = T(1) / T(m + 1);
        for (int j = m; j; j--)
        {
            a[j - 1] = T(j) * (a[j - 1] - a[j]);
        }
        b[m] = a[0];
    }
    return b;
}
```

`bernoulli<T>(n)` 返回 $B_0,\dots,B_n$，$O(n^2)$ 时间、$O(n)$ 空间。本条符号约定下前四项为 $1,\frac{1}{2},\frac{1}{6},0$，可优先用 $B_1=+\frac{1}{2}$ 排查约定混用。要算 $\sum_{i=1}^{n} i^m$ 时，先取 $B_0,\dots,B_m$ 再代入 Faulhaber 公式；系数预处理后可把它作为关于 n 的 $m+1$ 次多项式求值。需要更快地求整段 Bernoulli 数时，用 EGF $\frac{x}{e^x-1}$ 配 10.2.003 乘法逆元。

8.2.008 错排数

错排数 D_n 是没有任何元素留在原位的排列数（空排列计为 1）。考虑元素 1 被放到位置 j：若 j 上的元素回放到位置 1（对换），剩下是 $n-2$ 个元素的错排；否则把「位置 1 禁放 1」的约束转嫁给 j，化为 $n-1$ 个元素的错排。j 有 $n-1$ 种取法，故 $D_n=(n-1)(D_{n-1}+D_{n-2})$，$D_0=1,D_1=0$。

```cpp
template <class T> vector<T> derangements(int n)
{
    // n 是最大规模；返回 d[0..n]，d[i] 是 i 个元素无固定点排列数。
    assert(n >= 0); // 调试检查，可删。
    vector<T> d(n + 1);
    d[0] = 1;
    if (n >= 1)
    {
        d[1] = 0;
    }
    for (int i = 2; i <= n; i++)
    {
        d[i] = T(i - 1) * (d[i - 1] + d[i - 2]);
    }
    return d;
}
```

`derangements<T>(n)` 返回 `d[0..n]`，`T` 支持加法、乘法与整数构造即可（大整数或模整数）；$O(n)$ 时间与输出空间。前七项为 $1,0,1,2,9,44,265$。等价的容斥闭式是 $D_n=n!\sum_{k=0}^{n}\frac{(-1)^k}{k!}$，模素数下想 $O(1)$ 单点查询就配 8.1.001 阶乘、逆阶乘与组合数 预处理；只需整段小 n 时本条线性递推更短。

8.2.009 分拆数与五边形数

分拆数 $p(n)$ 数把 n 写成无序正整数之和的方案数，生成函数为 $\prod_{k\ge 1}(1-x^k)^{-1}$。对其倒数 $\prod(1-x^k)$ 用 Euler 五边形定理 $\prod_{k\ge 1}(1-x^k)=\sum_j(-1)^j x^{j(3j-1)/2}$（$j$ 取全体整数），只在广义五边形数 $g=\frac{k(3k\pm 1)}{2}$ 处非零，于是得正负交替递推 $p(n)=\sum_{k\ge 1}(-1)^{k-1}(p(n-g_k^-)+p(n-g_k^+))$。每个 n 只枚举到 $g\le n$ 的 $O(\sqrt{n})$ 个五边形数，把朴素 $O(n^2)$ 降到 $O(n\sqrt{n})$。

```cpp
template <class T> vector<T> partitions(int n)
{
    // n 是要计算的最大整数；返回 p(0) 到 p(n)。
    assert(n >= 0); // 调试检查，可删。
    vector<T> a(n + 1);
    a[0] = 1;
    for (int i = 1; i <= n; i++)
    {
        for (int k = 1;; k++)
        {
            int x = k / 2;
            int y = 3 * k - 1;
            if (k & 1)
            {
                x = k;
                y /= 2;
            }
            if (x > i / y)
            {
                break;
            }
            int g = x * y;
            if (k & 1)
            {
                a[i] += a[i - g];
            }
            else
            {
                a[i] -= a[i - g];
            }
            if (g <= i - k)
            {
                g += k;
                if (k & 1)
                {
                    a[i] += a[i - g];
                }
                else
                {
                    a[i] -= a[i - g];
                }
            }
        }
    }
```

```
    return a;
}
```

`partitions<T>(n)` 返回 $p(0), \dots, p(n)$，固定查验值为 $p(0)=1$、$p(5)=7$、$p(10)=42$；$O(n\sqrt{n})$ 时间、$O(n)$ 空间。这是无限制的整数分拆；若限制份数或每份大小（如「至多 k 份」「各份互异」），改用完全背包式的二维 DP 或对应生成函数。模素数下需要更长前缀时，可按五边形定理写出分母的稀疏系数，再调用 10.2.003 乘法逆元 在 $O(n \log n)$ 时间求逆。

8.2.010 有根树、无根树计数

树计数要先分清是否有根、节点是否带标号、儿子是否有序。本条实现最干净的一类——带标号无根树，即 Cayley 公式：n 个带标号点的无根树共 n^{n-2} 棵；若根标号预先固定，每棵无根树只有一种置根方式，数量仍为 n^{n-2}；若把“根是哪一个标号点”也视为结构的一部分，数量才是 $n \cdot n^{n-2} = n^{n-1}$。`cayley` 返回无根树数和后一种“根被区分”的有根树数，$n=1$ 单独处理。无标号树不在本条闭式范围内。

```
namespace tree_count
{
int qpow(int a, int n, int mod)
{
    // a、n、mod 是底数、非负指数和正模数；返回模幂。
    assert(n >= 0 && mod > 0); // 调试检查，可删。
    a %= mod;
    if (a < 0)
    {
        a += mod;
    }
    int r = 1 % mod;
    while (n)
    {
        if (n & 1)
        {
            r = combMulMod(r, a, mod);
        }
        a = combMulMod(a, a, mod);
        n >>= 1;
    }
    return r;
}

array<int, 2> cayley(int n, int mod)
{
    // n 是有标号点数，mod 是正模数；返回 {无根树数，根标号被区分的有根树数}。
    assert(n >= 1 && mod > 0); // 调试检查，可删。
    int unrooted = n == 1 ? 1 % mod : qpow(n % mod, n - 2, mod);
    int rooted = qpow(n % mod, n - 1, mod);
    return {unrooted, rooted};
}
}
```

`cayley(n, mod)` 返回的两维依次是无根树数、根标号被区分的有根树数，固定查验值为 `cay-`

`ley(4,1000000007)` 返回 `{16,64}`。两次模幂共 $O(\log n)$ 时间、$O(1)$ 空间；若根固定为某个已知标号，答案取第一项。Cayley 公式只适用于完全图的标号树；给定图上数生成树要用 8.2.011 Matrix-Tree / Kirchhoff 生成树计数 的 Laplacian 行列式，带度数限制的标号树则用 Prüfer 序列计数。

8.2.011 Matrix-Tree / Kirchhoff 生成树计数

Matrix-Tree（Kirchhoff）定理把生成树计数化成行列式。无向图的 Laplacian $L = D - A$（D 度数对角阵、A 邻接矩阵）任意删掉同编号的一行一列，剩下 $(n-1)$ 阶子式的行列式就是生成树数量；证明来自 Cauchy–Binet 展开到关联矩阵。有向图则按朝向选入度或出度 Laplacian，`arboCount` 用入度版统计从 `root` 向外、每个非根点恰有一条入边的树形图数。平行边按重数累加；自环在 Laplacian 的度数项与邻接项中抵消，不属于生成树。行列式在有限域 `Z` 上用高斯消元 `detField` 求：逐列找非零主元（找不到说明奇异，返回 0），换行时翻转符号，消元后累乘对角元。

```
// 依赖 1.1.008 静态取模类
Z detField(vector<vector<Z>> a)
{
    // a 是有限域上的方阵；返回其行列式。
    int n = a.size();
    for (const auto &row : a)
    {
        assert((int)row.size() == n); // 调试检查，可删。
    }
    Z ans = 1;
    for (int c = 0; c < n; c++)
    {
        int p = c;
        while (p < n && a[p][c] == Z(0))
        {
            p++;
        }
        if (p == n)
        {
            return 0;
        }
        if (p != c)
        {
            swap(a[p], a[c]);
            ans = -ans;
        }
        ans *= a[c][c];
        Z iv = a[c][c].inv();
        for (int i = c + 1; i < n; i++)
        {
            Z q = a[i][c] * iv;
            for (int j = c; j < n; j++)
            {
                a[i][j] -= q * a[c][j];
            }
        }
```

```
    }
    return ans;
}

Z spanTreeCnt(int n, const vector<pair<int, int>> &e)
{
    // n 是点数，e 是无向边端点列表且允许重边；返回生成树数量。
    assert(n >= 0); // 调试检查，可删。
    for (auto [u, v] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
    }
    if (n <= 1)
    {
        return 1;
    }
    vector a(n - 1, vector<Z>(n - 1));
    for (auto [u, v] : e)
    {
        if (u < n - 1)
        {
            a[u][u] += 1;
        }
        if (v < n - 1)
        {
            a[v][v] += 1;
        }
        if (u < n - 1 && v < n - 1)
        {
            a[u][v] -= 1;
            a[v][u] -= 1;
        }
    }
    return detField(move(a));
}

Z arboCount(int n, int root, const vector<pair<int, int>> &e)
{
    // n 是点数，root 是根，e 是有向边端点列表且允许重边；返回从 root 向外的生成树数量。
    assert(n >= 1 && 0 <= root && root < n); // 调试检查，可删。
    for (auto [u, v] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
    }
    if (n == 1)
    {
        return 1;
    }
    vector<int> id(n, -1);
    for (int u = 0, k = 0; u < n; u++)
    {
        if (u ≠ root)
```

```
        {
            id[u] = k++;
        }
    }
    vector a(n - 1, vector<Z>(n - 1));
    for (auto [u, v] : e)
    {
        if (v == root)
        {
            continue;
        }
        a[id[v]][id[v]] += 1;
        if (u != root)
        {
            a[id[v]][id[u]] -= 1;
        }
    }
    return detField(move(a));
}
```

先准备 1.1.008 静态取模类 的素数域 `Z`。`spanTreeCnt(n, e)` 接受点编号为 `0..n-1` 的无向边，`arboCount(n, root, e)` 接受有向边 $u \to v$ 并统计从 `root` 向外的树形图；空无向图 `n=0` 约定返回 1，有向接口要求 `n>=1`。自环可保留在边表中并自动抵消，平行边分别计数，非单点图存在孤立点时行列式为 0。整体 $O(n^3 + m)$ 时间、$O(n^2)$ 额外空间。完全图上有闭式时优先 8.2.010 有根树、无根树计数 的 Cayley 公式；只有一般图才动用行列式。5.9.003 BEST 定理与有向欧拉回路计数 需要朝根的树形图，因此应反转原图全部边后调用 `arboCount`。

下面用三角形、孤立点和有向图各核对一个已知答案：

```
signed main()
{
    assert(spanTreeCnt(3, {{0, 1}, {1, 2}, {2, 0}}).val() == 3);
    assert(spanTreeCnt(3, {{0, 1}}).val() == 0);
    assert(arboCount(3, 0, {{0, 1}, {0, 2}, {1, 2}}).val() == 2);
}
```

8.2.012 Pólya / Burnside 置换计数

Burnside 引理说群 G 作用下的轨道数（本质不同方案数）等于各元素不动点数的平均：$|X/G| = \frac{1}{|G|}\sum_{g\in G}|\mathrm{Fix}(g)|$。用 c 种颜色给位置染色时，一个置换 g 的染色不动点要求同一循环上位置同色，故 $|\mathrm{Fix}(g)| = c^{\mathrm{cyc}(g)}$，这就是 Pólya 计数把循环长度代入库存多项式的常数版本。`burnside` 对群里每个位置置换数出循环数 `cyc`，累加 c^{cyc}，最后除以 $|G|$。传入的 `group` 必须是真正的群（含单位元、对合成封闭），否则平均无意义。

```
namespace polya
{
template <class T = long long>
T burnside(const vector<vector<int>> &group, int colors)
```

```
{
    // group 中每项是位置置换，colors 是颜色数；返回本质不同染色数。
    static_assert(numeric_limits<T>::is_integer && numeric_limits<T>::is_signed &&
                  sizeof(T) >= sizeof(long long));
    assert(!group.empty() && colors >= 0); // 调试检查，可删。
    int n = group[0].size();
    T sum = 0;
    for (const auto &p : group)
    {
        assert((int)p.size() == n); // 调试检查，可删。
        vector<int> cnt(n);
        for (int x : p)
        {
            assert(0 <= x && x < n); // 调试检查，可删。
            cnt[x]++;
        }
        for (int x : cnt)
        {
            assert(x == 1); // 调试检查，可删。
        }
        vector<int> vis(n);
        int cyc = 0;
        for (int i = 0; i < n; i++)
        {
            if (vis[i])
            {
                continue;
            }
            cyc++;
            int x = i;
            while (!vis[x])
            {
                vis[x] = 1;
                x = p[x];
            }
        }
        T ways = 1;
        for (int i = 0; i < cyc; i++)
        {
            ways *= colors;
        }
        sum += ways;
    }
    int sz = group.size();
    assert(sum % sz == 0); // 调试检查，可删。
    return sum / sz;
}
}
```

`burnside(group, colors)` 要求 `group` 是完整有限群，并返回本质不同染色数；调用者须保证幂和总和能由 `T` 表示，需要 128 位精确计数时调用 `burnside<__int128>`。算法访问 $O(|G|n)$ 个置换位置。结果超过 128 位时换 14.1.001 高精度整数（BigInt）；群规模本身指数级时应改用循环指标解析求和。

四个位置只考虑旋转、使用两种颜色时共有 6 个轨道，可用这组固定群检查循环分解与平均数：

```cpp
signed main()
{
    // 无参数；核对四点旋转群的二染色轨道数，返回 0。
    vector<vector<int>> rot{
        {0, 1, 2, 3}, {1, 2, 3, 0},
        {2, 3, 0, 1}, {3, 0, 1, 2}};
    assert(polya::burnside(rot, 2) == 6);
    return 0;
}
```

8.2.013 杨氏矩阵与钩子公式

形状为分拆 λ（行长非增）的 Young 图，其标准 Young 表（每行每列严格递增地填 $1..n$）数量由钩长公式给出：$f^\lambda = \dfrac{n!}{\prod_{\text{格}c} h(c)}$，其中格 c 的钩长 $h(c) = 1 +$ 右侧同行格数 $+$ 下方同列格数。直接形成 $n!$ 可能在除掉钩长前溢出。代码用线性筛得到最小质因子，先按 Legendre 公式统计 $n!$ 的质因数指数，再逐格分解钩长并扣除指数，最后只乘答案真正保留的质因子；列高 `col[j]` 让钩长可在 $O(1)$ 时间写成 `row[i]-j+col[j]-i-1`。

```cpp
namespace young_tab
{
template <class T = long long>
T youngTab(const vector<int> &row)
{
    // row 是非增的行长分拆；返回标准 Young 表数量。
    static_assert(numeric_limits<T>::is_integer && numeric_limits<T>::is_signed &&
                  sizeof(T) >= sizeof(long long));
    size_t cells = 0;
    for (int i = 0; i < (int)row.size(); i++)
    {
        assert(row[i] >= 0); // 调试检查，可删。
        if (i)
        {
            assert(row[i - 1] >= row[i]); // 调试检查，可删。
        }
        cells += row[i];
    }
    int n = (int)cells;
    vector<int> spf(n + 1), ps;
    for (int i = 2; i <= n; i++)
    {
        if (!spf[i])
        {
            spf[i] = i;
            ps.push_back(i);
        }
        for (int p : ps)
        {
```

```
            if (p > spf[i] || i > n / p)
            {
                break;
            }
            spf[i * p] = p;
        }
    }
    vector<int> e(n + 1);
    for (int p : ps)
    {
        for (int x = n / p; x; x /= p)
        {
            e[p] += x;
        }
    }
    vector<int> col(row.empty() ? 0 : row[0]);
    for (int x : row)
    {
        for (int j = 0; j < x; j++)
        {
            col[j]++;
        }
    }
    for (int i = 0; i < (int)row.size(); i++)
    {
        for (int j = 0; j < row[i]; j++)
        {
            int x = row[i] - j + col[j] - i - 1;
            while (x > 1)
            {
                int p = spf[x];
                e[p]--;
                x /= p;
            }
        }
    }
    T ans = 1;
    for (int p : ps)
    {
        assert(e[p] >= 0); // 调试检查，可删。
        for (int i = 0; i < e[p]; i++)
        {
            ans *= p;
        }
    }
    return ans;
}

}
```

`youngTab(row)` 要求行长非负且非增，空形状返回 1；调用者须保证结果能由 `T` 表示，需要 128 位时调用 `youngTab<__int128>`。线性筛、列高与指数表使用 $O(n+\lambda_1)$ 空间，全部分解最坏为 $O(n\log n)$ 时间。

超过 128 位的精确答案换 14.1.001 高精度整数（BigInt）。

下面同时检查非矩形小形状与 2×10 矩形；后者的标准 Young 表数恰为 Catalan 数 $C_{10} = 16796$：

```
signed main()
{
    // 无参数；核对两个 Young 图形状的标准表数量，返回 0。
    assert(young_tab::youngTab({2, 1}) == 2);
    assert(young_tab::youngTab({10, 10}) == 16796);
    return 0;
}
```

8.2.014 固定 k 的第一类 Stirling 数

有符号第一类 Stirling 数的指数生成函数为 $\frac{\log(1+x)^k}{k!} = \sum_{n \geq k} s(n,k)\frac{x^n}{n!}$。先写出 $\log(1+x) = \sum_{i \geq 1}(-1)^{i-1}\frac{x^i}{i}$，调用多项式快速幂，最后把第 n 项乘回 $n!/k!$，即可一次得到固定 k 的整列；符号与 8.2.005 Stirling 数 的 `stirling1` 完全一致。

```
// 依赖 1.1.008 静态取模类
// 依赖 7.4.004 二次剩余
// 依赖 10.2.002 乘法、除法、取模
// 依赖 10.2.004 导数、积分
// 依赖 10.2.005 ln、exp、开根、快速幂
vector<Z> stirling1K(int n, int k)
{
    // n 是最大行号，k 是固定列号；返回 s(k,k) 到 s(n,k)，模数由 Z 决定。
    assert(0 <= k && k <= n && n < mod); // 调试检查，可删。
    Poly f(n + 1);
    for (int i = 1; i <= n; i++)
    {
        f[i] = (i & 1 ? Z(1) : Z(-1)) / Z(i);
    }
    Poly g = fpsPow(f, k, n + 1);
    Z fk = 1;
    for (int i = 1; i <= k; i++)
    {
        fk *= i;
    }
    Z ifk = fk.inv();
    Z fac = 1;
    vector<Z> ans(n - k + 1);
    for (int i = 0; i <= n; i++)
    {
        if (i >= k)
        {
            ans[i - k] = g[i] * fac * ifk;
        }
        fac *= i + 1;
    }
    return ans;
```

```
}
```

本函数依赖 10.2.001 多项式封装 的 `Poly/Z` 与 10.2.005 ln、exp、开根、快速幂 的 `fpsPow`。`stirling1K(n,k)[i-k]` 等于 $s(i,k)$，要求 $n < \mathrm{mod}$ 以保证积分分母和阶乘可逆；时间 $O(M(n))$、输出与工作空间 $O(n)$。若只查一个小 `n,k`，直接用 8.2.005 Stirling 数 的二维递推更简单。

8.2.015 固定 k 的第二类 Stirling 数

固定块数的第二类 Stirling 数满足普通生成函数 $\sum_{n\ge k} S(n,k)x^{n-k} = \prod_{i=1}^{k}(1-ix)^{-1}$。代码用平衡乘积树构造分母，避免从左到右相乘退化成二次复杂度，再调用 FPS 逆元截断到 `n-k+1` 项；第 t 项就是 $S(k+t,k)$。

```
// 依赖 8.2.014 固定 k 的第一类 Stirling 数
// 依赖 10.2.003 乘法逆元
vector<Z> stirling2K(int n, int k)
{
    // n 是最大行号，k 是固定列号；返回 S(k,k) 到 S(n,k)，模数由 Z 决定。
    assert(0 <= k && k <= n); // 调试检查，可删。
    auto build = [&](auto &&self, int l, int r) -> Poly
    {
        // self 是递归自身，[l,r) 是因子下标；返回这一段 (1-ix) 的乘积。
        if (r - l == 1)
        {
            return Poly{1, -Z(l)};
        }
        int m = (l + r) / 2;
        return self(self, l, m) * self(self, m, r);
    };
    Poly den = k == 0 ? Poly{1} : build(build, 1, k + 1);
    Poly ans = fpsInv(den, n - k + 1);
    return vector<Z>(ans.begin(), ans.end());
}
```

本函数依赖 10.2.001 多项式封装 的乘法和 10.2.003 乘法逆元 的 `fpsInv`。构造乘积树为 $O(M(k)\log k)$，求逆为 $O(M(n-k))$，峰值空间 $O(n+k)$；`k=0` 返回 $S(0,0)=1$ 及其后的全零项。若只需固定 `n` 的整行，仍用 8.2.005 Stirling 数 或第二类的有限和公式。

下面直接检查四条可手算的固定列，不在正文再放二维递推作参考实现：

```
signed main()
{
    assert(stirling1K(5, 1) == vector<Z>({1, -1, 2, -6, 24}));
    assert(stirling1K(6, 2) == vector<Z>({1, -3, 11, -50, 274}));
    assert(stirling2K(5, 1) == vector<Z>({1, 1, 1, 1, 1}));
    assert(stirling2K(6, 2) == vector<Z>({1, 3, 7, 15, 31}));
    return 0;
}
```

8.2.016 小素数大下标第一类 Stirling 数

当素数 p 很小而 n, k 可到 64 位时，二维递推不再可行。先用 Pascal 表保存 $0 \le a, b < p$ 的组合数，`comb` 按 Lucas 定理逐位查询。对 $n = ip + j$，再把 $k - i = a(p-1) + b$ 调整为公式要求的余数范围，有 $s(n,k) = (-1)^{i+a}\binom{i}{a}s(j,b) \pmod p$；越界条件直接贡献 0。低位 $s(j,b)$ 只需预处理到 $p-1$。

```
class SmallPComb
{
  protected:
    // p 是小素数，c[i][j] 是 C(i,j) mod p，其中 0<=j<=i<p。
    int p;
    vector<vector<int>> c;

    int comb(int n, int k) const
    {
        // n、k 是非负 64 位组合数参数；返回 C(n,k) mod p，k 越界时返回 0。
        if (k < 0 || k > n)
        {
            return 0;
        }
        int ans = 1;
        while (n)
        {
            int a = n % p;
            int b = k % p;
            if (b > a)
            {
                return 0;
            }
            ans = ans * c[a][b] % p;
            n /= p;
            k /= p;
        }
        return ans;
    }

  public:
    explicit SmallPComb(int p) : p(p)
    {
        // p 是可承担 O(p^2) 表的素数；建立一位 Lucas 所需的 Pascal 三角。
        assert(2 <= p && p <= numeric_limits<int>::max()); // 调试检查，可删。
        c.resize(p);
        c[0] = {1};
        for (int i = 1; i < p; i++)
        {
            c[i].assign(i + 1, 0);
            for (int j = 0; j <= i; j++)
            {
                int x = 0;
                if (j < i)
                {
```

```
                    x += c[i - 1][j];
                }
                if (j)
                {
                    x += c[i - 1][j - 1];
                }
                c[i][j] = x % p;
            }
        }
    }
};

class Stirling1P : public SmallPComb
{
    // s[i][j] 是有符号第一类 Stirling 数 s(i,j) mod p，其中 0<=j<=i<p。
    vector<vector<int>> s;

  public:
    explicit Stirling1P(int p) : SmallPComb(p), s(p)
    {
        // p 是小素数；建立第一类低位表与 Lucas 表。
        s[0] = {1};
        for (int i = 1; i < p; i++)
        {
            s[i].assign(i + 1, 0);
            for (int j = 0; j <= i; j++)
            {
                int x = 0;
                if (j)
                {
                    x += s[i - 1][j - 1];
                }
                if (j < i)
                {
                    x += s[i - 1][j] * (p - i + 1) % p;
                }
                s[i][j] = x % p;
            }
        }
    }

    int get(int n, int k) const
    {
        // n、k 是非负 64 位下标；返回有符号 s(n,k) mod p，k>n 时返回 0。
        assert(n >= 0); // 调试检查，可删。
        if (k < 0 || k > n)
        {
            return 0;
        }
        int i = n / p;
        int j = n % p;
        if (i > k)
```

```
        {
            return 0;
        }
        int a = (k - i) / (p - 1);
        int b = (k - i) % (p - 1);
        if (b == 0 && j > 0)
        {
            b += p - 1;
            a--;
        }
        if (a < 0 || i < a || b > j)
        {
            return 0;
        }
        int ans = comb(i, a) * s[j][b] % p;
        if (((i + a) & 1) && ans)
        {
            ans = p - ans;
        }
        return ans;
    }
};
```

构造 `Stirling1P q(p)` 后 `q.get(n,k)` 在 $O(\log_p n)$ 时间返回有符号 $s(n,k) \bmod p$；预处理为 $O(p^2)$ 时间与空间，表项显式使用 `int`，避免 `#define int long long` 把 $p = 5000$ 时的内存翻倍。`p` 必须是不超过 `int` 上界的小素数，且内存要能容纳两张三角表；中等规模整表仍优先 8.2.005 Stirling 数。

8.2.017 小素数大下标第二类 Stirling 数

第二类需要不同的周期式，不能复用第一类的符号公式。对 $n < p$ 直接查低位表；否则写 $k = ip + j$，再把 $n - i = a(p-1) + b$ 调整到 $1 \le b \le p-1$。当 $b < p-1$ 时答案为 $\binom{a}{i} S(b, j)$；当 $b = p-1$ 时，`j=0` 的答案是 $\binom{a}{i-1}$，否则为 $\binom{a}{i} S(p-1, j)$。

```
// 依赖 8.2.016 小素数大下标第一类 Stirling 数
class Stirling2P : public SmallPComb
{
    // s[i][j] 是第二类 Stirling 数 S(i,j) mod p，其中 0<=j<=i<p。
    vector<vector<int>> s;

  public:
    explicit Stirling2P(int p) : SmallPComb(p), s(p)
    {
        // p 是小素数；建立第二类低位表与 Lucas 表。
        s[0] = {1};
        for (int i = 1; i < p; i++)
        {
            s[i].assign(i + 1, 0);
            for (int j = 0; j <= i; j++)
            {
                int x = 0;
```

```
                if (j)
                {
                    x += s[i - 1][j - 1];
                }
                if (j < i)
                {
                    x += s[i - 1][j] * j % p;
                }
                s[i][j] = x % p;
            }
        }
    }

    int get(int n, int k) const
    {
        // n、k 是非负 64 位下标；返回 S(n,k) mod p，k>n 时返回 0。
        assert(n >= 0); // 调试检查，可删。
        if (k < 0 || k > n)
        {
            return 0;
        }
        if (n < p)
        {
            return s[n][k];
        }
        int i = k / p;
        int j = k % p;
        int a = (n - i) / (p - 1);
        int b = (n - i) % (p - 1);
        if (b == 0)
        {
            b = p - 1;
            a--;
        }
        if (a < 0 || j > b)
        {
            return 0;
        }
        if (b < p - 1)
        {
            return comb(a, i) * s[b][j] % p;
        }
        if (j == 0)
        {
            return comb(a, i - 1);
        }
        return comb(a, i) * s[p - 1][j] % p;
    }
};
```

`Stirling2P q(p)` 同样以 $O(p^2)$ 时间与空间预处理，`q.get(n,k)` 为 $O(\log_p n)$；它与 8.2.016 小素数大下标第一类 Stirling 数 只共享 `SmallPComb` 的 Lucas 表，低位递推和高位公式彼此独立。固定 k 的整列

输出用 8.2.015 固定 k 的第二类 Stirling 数，不要为单次大下标查询构造长多项式。

模 5 时先用小值核对定义递推，再用 $s(n,n-1)=-\binom{n}{2}$ 与 $S(n,2)=2^{n-1}-1$ 检查大下标分支：

```
signed main()
{
    int n = 100000000000000003LL;
    Stirling1P a(5);
    Stirling2P b(5);
    assert(a.get(6, 2) == 4);
    assert(a.get(n, n - 1) == 2);
    assert(b.get(6, 3) == 0);
    assert(b.get(n, 2) == 3);
    cout << "OK\n";
    return 0;
}
```

8.3 生成与构造

8.3.001 排列组合生成

这四种枚举都短到不该封装，直接展开在使用处即可。排序后反复 `next_permutation` 无重地枚举多重集合的不同排列；`0..2^n-1` 的二进制掩码天然枚举全部子集；Gosper Hack（`x&-x` 取最低位、`s+lowbit` 进位后补齐低位）在相同 popcount 的掩码间按数值递增跳转，等价于组合的 colex 顺序，并非元素下标序列的字典序；Gray 编码 $g=x\oplus(x\gg 1)$ 使相邻编号只差一位，解码则对不断右移的前缀连续异或还原。下面的 `main` 把四段写在一起，末尾 `assert(y == x)` 验证 Gray 编解码互逆。

```
signed main()
{
    // 无参数；直接展示排列、子集、固定大小子集与 Gray Code 的展开写法，返回 0。
    vector<int> a{1, 1, 2};
    sort(a.begin(), a.end());
    do
    {
        // 此处直接处理当前字典序排列 a。
    } while (next_permutation(a.begin(), a.end()));

    int n = 5;
    for (int s = 0; s < (1LL << n); s++)
    {
        // s 的第 i 位表示是否选择编号 i 的元素。
    }

    int k = 3;
    if (k == 0)
    {
        int s = 0;
        // 空集是唯一的 0 元子集。
    }
```

```
    else
    {
        int s = (1LL << k) - 1;
        while (s < (1LL << n))
        {
            // 此处直接处理当前恰含 k 个元素的子集 s。
            int x = s & -s;
            int y = s + x;
            s = y | ((s ^ y) >> 2) / x;
        }
    }

    int x = 23;
    int g = x ^ (x >> 1);
    int y = g;
    for (int t = y >> 1; t; t >>= 1)
    {
        y ^= t;
    }
    assert(y == x);
}
```

直接把对应循环抄到 `solve`/`main` 里，不要包装成回调。示例要求 $0 \le n < 63$、$0 \le k \le n$，Gray 编解码按非负整数处理。设不同排列数为 P，排列枚举 $O(Pn)$；全部子集 $O(2^n)$；固定大小子集 $O(\binom{n}{k})$；Gray 编码 $O(1)$、解码 $O(\log x)$，都只用 $O(1)$ 额外空间。只需要「第几个排列」而非全部枚举时用 8.3.002 康托展开与逆展开 直接跳到目标；子集枚举里要按权和分组则配合状压 DP。

8.3.002 康托展开与逆展开

康托展开把排列映到它的字典序排名（从 0 起）：第 i 位的权是 $(n-1-i)!$，系数是「p_i 右边比它小的元素个数」，累加即排名。逆展开反过来，从高位到低位用 $k/(i-1)!$ 选出当前剩余元素里第几小的、再取模递降。`cantor` 用树状数组维护右侧更小计数把逐位统计降到 $O(n \log n)$；`invCantor` 每步从候选 `vector` 删除选中元素，故为 $O(n^2)$。两向都要求元素互异，且排名从 0 还是 1 起必须全程统一。

```
unsigned long long cantor(const vector<int> &p)
{
    // p 是由 0 到 n-1 组成的排列；返回其从零开始的字典序排名。
    int n = p.size();
    assert(n <= 20); // 调试检查，可删。
    vector<int> seen(n);
    for (int x : p)
    {
        assert(0 <= x && x < n && !seen[x]); // 调试检查，可删。
        seen[x] = 1;
    }
    vector<unsigned long long> f(n + 1, 1);
    for (int i = 1; i <= n; i++)
    {
        f[i] = f[i - 1] * i;
```

```
    }
    vector<int> bit(n + 1);
    auto add = [&](int x)
    {
        // x 是要加入的排列值；把它在树状数组中的计数加一。
        for (++x; x <= n; x += x & -x)
        {
            bit[x]++;
        }
    };
    auto sum = [&](int x)
    {
        // x 是值域右端；返回已加入值中严格小于 x 的个数。
        int ans = 0;
        for (; x; x -= x & -x)
        {
            ans += bit[x];
        }
        return ans;
    };
    unsigned long long ans = 0;
    for (int i = n - 1; i >= 0; i--)
    {
        ans += f[n - 1 - i] * sum(p[i]);
        add(p[i]);
    }
    return ans;
}

vector<int> invCantor(int n, unsigned long long k)
{
    // n 是排列长度，k 是从零开始的排名；返回由 0 到 n-1 组成的对应排列。
    assert(0 <= n && n <= 20); // 调试检查，可删。
    vector<unsigned long long> f(n + 1, 1);
    for (int i = 1; i <= n; i++)
    {
        f[i] = f[i - 1] * i;
    }
    assert(k < f[n]); // 调试检查，可删。
    vector<int> a(n), ans;
    iota(a.begin(), a.end(), 0);
    for (int i = n; i; i--)
    {
        unsigned long long q = k / f[i - 1];
        k %= f[i - 1];
        assert(q < a.size()); // 调试检查，可删。
        ans.push_back(a[q]);
        a.erase(a.begin() + q);
    }
    return ans;
}
```

`cantor(p)` 接受由 $0,1,\dots,n-1$ 组成的排列并返回其康托排名，`invCantor(n, k)` 由排名还原排列。`cantor` 为 $O(n\log n)$、`invCantor` 为 $O(n^2)$，都 $O(n)$ 空间；排名用 `unsigned long long` 存，故 $n\le 20$（$21!$ 已溢出 64 位）。更大的精确排名改用 14.1.001 高精度整数（BigInt） 保存阶乘与答案；若只求排名模某个数，则把 `f[i]` 和累加过程取模，但模排名不能再唯一逆展开。只需按字典序枚举而非跳转到指定名次时用 8.3.001 排列组合生成 的 `next_permutation`。

下面用三组已知排名检查重复换位、首尾边界与逆展开：

```
signed main()
{
    assert(cantor({0, 1, 2, 3}) == 0);
    assert(cantor({1, 0, 3, 2}) == 7);
    assert(cantor({3, 2, 1, 0}) == 23);
    assert((invCantor(4, 7) == vector<int>{1, 0, 3, 2}));
    cout << "OK\n";
    return 0;
}
```

8.3.003 勾股数枚举

Euclid 参数化给出全部本原勾股数组：取 $m>n\ge 1$、$\gcd(m,n)=1$ 且 m,n 一奇一偶，则 $(m^2-n^2,\ 2mn,\ m^2+n^2)$ 是本原三元组，且每个本原解恰对应一组这样的 (m,n)；再乘正整数就得到全部非本原解。`pythagorean` 按此枚举斜边 $c=m^2+n^2\le c_{\max}$ 的本原组，`(m-n)%2==0`（同奇偶）或 $\gcd\neq 1$ 时跳过，并把两条直角边排成 $a\le b$。

```
// PTriple 的三个分量依次是两条直角边 a、b 与斜边 c。
using PTriple = array<int, 3>;

vector<PTriple> pythagorean(long long cmax)
{
    // cmax 是非负斜边上界；返回全部满足 a<=b、c<=cmax 的本原三元组 {a,b,c}。
    assert(cmax >= 0); // 调试检查，可删。
    vector<PTriple> ans;
    for (long long m = 2; m <= (cmax - 1) / m; m++)
    {
        long long mm = m * m;
        for (long long n = 1; n < m; n++)
        {
            if ((m - n) % 2 == 0 || gcd(m, n) != 1)
            {
                continue;
            }
            long long nn = n * n;
            if (mm > cmax - nn)
            {
                break;
            }
            long long a = mm - nn;
            long long b = 2 * m * n;
```

```
            long long c = mm + nn;
            if (a > b)
            {
                swap(a, b);
            }
            ans.push_back({a, b, c});
        }
    }
    return ans;
}
```

`pythagorean(cmax)` 返回斜边不超过 `cmax` 的全部本原三元组，`PTriple[0..2]` 为两直角边与斜边。m 到约 $\sqrt{c_{\max}}$、内层 $n<m$，实际有效 (m,n) 对约 $O(c_{\max})$，空间为答案数。需要全部（含非本原）解时把每个本原组乘 $1,2,\dots$ 直到超限；只统计数量而不列举时用数论方法按 c 的因子结构计数，不必逐个枚举。

8.3.004 阿达马矩阵

Sylvester 阿达马矩阵元素全为 ±1、任意两行正交，用于正交编码和构造题。从 $H_1=[1]$ 出发按分块递推 $H_{2m}=\begin{bmatrix}H_m & H_m\\ H_m & -H_m\end{bmatrix}$ 倍增：若 $H_mH_m^T=mI$，则展开可验证 $H_{2m}H_{2m}^T=2mI$，故正交性对全部 2 的幂阶成立。代码每轮把 $m\times m$ 的块复制到左上、右上、左下，右下取相反数，直到边长达到 n。

```
vector<vector<int>> hadamard(int n)
{
    // n 是正的 2 的幂；返回元素只取 1 和 -1、任意两条不同行正交的 Sylvester 阿达马矩阵。
    assert(n > 0 && has_single_bit((unsigned long long)n)); // 调试检查，可删。
    assert((unsigned long long)n <= numeric_limits<size_t>::max()); // 调试检查，可删。
    size_t side = (size_t)n;
    assert(side <= numeric_limits<size_t>::max() / sizeof(int) / side); // 调试检查，可删。
    vector<vector<int>> a{{1}};
    while ((int)a.size() < n)
    {
        int m = a.size();
        vector b(2 * m, vector<int>(2 * m));
        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < m; j++)
            {
                b[i][j] = a[i][j];
                b[i][j + m] = a[i][j];
                b[i + m][j] = a[i][j];
                b[i + m][j + m] = -a[i][j];
            }
        }
        a.swap(b);
    }
    return a;
}
```

`hadamard(n)` 要求 `n` 是正的 2 的幂（`has_single_bit` 断言），返回 $n\times n$ 的 ±1 矩阵；$O(n^2)$ 时间与输

出空间。$n=4$ 时每个元素为 ± 1，每行与自身内积为 4、与其他行内积为 0。题目要 $0/1$ 矩阵时把 -1 映射成 0，但映射后普通整数内积不再为 0（需改用异或/汉明距离度量正交）。非 2 的幂阶的阿达马矩阵（如 12、20 阶）需要 Paley 等其他构造，本条不覆盖；填数构造题里的幻方见 8.3.005 幻方。

8.3.005 幻方

幻方要把 $1,2,\dots,n^2$ 各恰用一次填进 $n\times n$ 方阵，使每行、每列、两条主对角线的和都等于幻和 $\frac{n(n^2+1)}{2}$。除 $n=2$ 无解外，按阶数奇偶分三套经典构造：奇数阶用 Siamese（罗伯法），从中上格填 1，每步向右上移动，越界则绕环、遇占用则改为下移一格；双偶阶（$4\mid n$）先按行填 $1,2,\dots,n^2$，再对满足 $i \bmod 4 = j \bmod 4$ 或 $(i \bmod 4)+(j \bmod 4)=3$ 的格取补数 n^2+1-v；单偶阶（$n\equiv 2 \pmod 4$）用 Strachey/LUX 法，把方阵分成四个 $m=\frac{n}{2}$ 阶子块，各子块填同一个奇数阶幻方 b 加偏移 $0,2m^2,3m^2,m^2$（左上、右上、左下、右下），再交换左上/左下块的前 $k=\frac{n-2}{4}$ 列（跳过中心行首列并单独换中心行的第 k 列）与后 $k-1$ 列，使四块的行列和重新配平。

```
vector<vector<int>> magicSquare(int n)
{
    // n 是阶数；返回恰好填入 1 到 n^2 的幻方，n=2 时无解并返回空矩阵。
    assert(n > 0); // 调试检查，可删。
    assert(n <= numeric_limits<int>::max() / n); // 调试检查，可删。
    assert((unsigned long long)n <= numeric_limits<size_t>::max()); // 调试检查，可删。
    size_t side = (size_t)n;
    assert(side <= numeric_limits<size_t>::max() / sizeof(int) / side); // 调试检查，可删。
    int total = n * n;
    if (n == 2)
    {
        return {};
    }
    if (n & 1)
    {
        vector a(n, vector<int>(n));
        int x = 0;
        int y = n / 2;
        for (int v = 1; v <= total; v++)
        {
            a[x][y] = v;
            int nx = (x + n - 1) % n;
            int ny = (y + 1) % n;
            if (a[nx][ny])
            {
                x = (x + 1) % n;
            }
            else
            {
                x = nx;
                y = ny;
            }
        }
        return a;
    }
```

```
    if (n % 4 == 0)
    {
        vector a(n, vector<int>(n));
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                int v = i * n + j + 1;
                bool rev = i % 4 == j % 4 || i % 4 + j % 4 == 3;
                a[i][j] = rev ? total + 1 - v : v;
            }
        }
        return a;
    }
    int m = n / 2;
    int k = (n - 2) / 4;
    int z = m * m;
    auto b = magicSquare(m);
    vector a(n, vector<int>(n));
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < m; j++)
        {
            a[i][j] = b[i][j];
            a[i][j + m] = b[i][j] + 2 * z;
            a[i + m][j] = b[i][j] + 3 * z;
            a[i + m][j + m] = b[i][j] + z;
        }
    }
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < k; j++)
        {
            if (i != m / 2 || j)
            {
                swap(a[i][j], a[i + m][j]);
            }
        }
        for (int j = n - k + 1; j < n; j++)
        {
            swap(a[i][j], a[i + m][j]);
        }
    }
    swap(a[m / 2][k], a[m / 2 + m][k]);
    return a;
}
```

`magicSquare(n)` 要求 `n>0` 且 n^2、所有块偏移和填充值均能由当前 64 位 `int` 表示；代码用除法断言检查乘法前边界，并另行检查矩阵元素总数可由 `size_t` 分配。$n=1$ 得 `{{1}}`，$n=2$ 返回空矩阵表示无解，其余阶数均有构造。时间与输出空间均为 $O(n^2)$；同为矩阵构造的 ±1 正交阵见 8.3.004 阿达马矩阵。

```
signed main()
{
    assert(magicSquare(2).empty());
    auto a = magicSquare(3);
    assert(a.size() == 3 && a[0].size() == 3 && a[1][1] == 5);
}
```

9. 线性代数与数值算法

9.1 矩阵

9.1.001 普通矩阵与取模矩阵

矩阵 A 记作行数 n、列数 m 的二维数组，`Mat<T>` 把元素类型 T 参数化，只要 T 支持所在环的加法与乘法就能复用同一套加乘与快速幂。乘积 $C = AB$ 定义为 $C_{ij} = \sum_k A_{ik}B_{kj}$，要求左矩阵列数等于右矩阵行数，一般不交换；实现里三重循环特意按 i, k, j 顺序累加，使最内层对 `x[i]`、`y[k]` 都连续访问，比教科书的 i, j, k 更缓存友好。`eye(n)` 造的 n 阶单位矩阵既是快速幂的初值，也是空乘积的单位元。元素类型若用原生整数，连乘会溢出；需要取模时把 `T` 换成模整数类型（每次乘加后自动约减），本条不内置取模、由 `T` 承担。

```
template <class T> struct Mat
{
    // n、m 是行数与列数；a 始终应有 n 行，每行恰有 m 个元素。
    int n, m;
    vector<vector<T>> a;

    Mat(int n = 0, int m = 0) : n(n), m(m), a(n, vector<T>(m))
    {
        // 建立 n 行 m 列的零矩阵；不要直接改变 a 的行数或行宽。
    }

    static Mat eye(int n)
    {
        // n 是矩阵阶数；返回 n 阶单位矩阵。
        Mat a(n, n);
        for (int i = 0; i < n; i++)
        {
            a[i][i] = 1;
        }
        return a;
    }

    vector<T> &operator[](int x)
    {
        // x 是行下标；返回该行的可修改引用。
        return a[x];
    }
    const vector<T> &operator[](int x) const
    {
        // x 是行下标；返回该行的只读引用。
        return a[x];
    }

    friend Mat operator*(const Mat &x, const Mat &y)
    {
        // x、y 是乘法左右两侧的矩阵；返回维数相容时的普通乘积。
```

```
        assert((int)x.a.size() == x.n && (int)y.a.size() == y.n && x.m == y.n); // 调试检查，可删。
        // 调试检查，可删。
        for (const auto &r : x.a)
        {
            assert((int)r.size() == x.m);
        }
        // 调试检查，可删。
        for (const auto &r : y.a)
        {
            assert((int)r.size() == y.m);
        }
        Mat z(x.n, y.m);
        for (int i = 0; i < x.n; i++)
        {
            for (int k = 0; k < x.m; k++)
            {
                for (int j = 0; j < y.m; j++)
                {
                    z[i][j] += x[i][k] * y[k][j];
                }
            }
        }
        return z;
    }

    Mat pow(unsigned long long b) const
    {
        // b 是非负指数；返回当前方阵的 b 次幂。
        assert(n == m); // 调试检查，可删。
        Mat x = *this;
        Mat ans = eye(n);
        while (b)
        {
            if (b & 1)
            {
                ans = ans * x;
            }
            b >>= 1;
            if (b)
            {
                x = x * x;
            }
        }
        return ans;
    }
};
```

以 `Mat<T>(n,m)` 建零矩阵、`Mat<T>::eye(n)` 建单位阵，`operator*` 要求维数相容，`pow(b)` 要求方阵，$b=0$ 时返回单位阵。由于 `a` 是公开成员，若直接改坏其行数或行宽，后续运算不再合法；代码中的断言只用于调试。$n\times m$ 乘 $m\times k$ 是 $O(nmk)$ 时间、$O(nk)$ 结果空间；n 阶方阵的 b 次幂是 $O(n^3\log(b+1))$ 时间、$O(n^2)$ 空间。转移是普通加乘时本条即够；只需裸函数时用 9.1.002 矩阵快速幂；转移是 min-加、

max-加等半环运算时改用 9.1.004 +min / +max 半环矩阵；非零元远少于 n^2 时用 9.1.008 稀疏矩阵；要行列式或逆矩阵则转 9.1.006 行列式（质数模数 / 任意模数） 与 9.1.005 矩阵求逆（质数模数 / 任意模数）。

9.1.002 矩阵快速幂

常系数线性递推 $f_n = \sum_i c_i f_{n-i}$ 与固定状态转移都能写成向量左乘同一个转移方阵 M，第 b 步的状态即 M^b 乘初始向量，于是把递推的 $O(b)$ 步压成对 M 求幂。快速幂把指数按二进制拆开：$M^b = \prod_{b\text{ 的置位}i} M^{2^i}$，边平方边在对应位乘入答案。只有方阵能做普通幂，答案初值必须是同阶单位矩阵。本条给出不依赖 9.1.001 普通矩阵与取模矩阵 类封装的裸函数版，`T` 只需支持加法、乘法与从 0、1 构造。

```cpp
template <class T> vector<vector<T>> matPow(vector<vector<T>> a, unsigned long long b)
{
    // a 是方阵，b 是非负指数；返回 a 的 b 次幂，T 必须支持加法、乘法和从 0、1 构造。
    int n = a.size();
    // 调试检查，可删。
    for (const auto &r : a)
    {
        assert((int)r.size() == n);
    }
    vector<vector<T>> ans(n, vector<T>(n));
    for (int i = 0; i < n; i++)
    {
        ans[i][i] = 1;
    }
    auto mul = [&](const vector<vector<T>> &x, const vector<vector<T>> &y)
    {
        // x、y 是同阶方阵；返回普通矩阵乘积。
        vector<vector<T>> z(n, vector<T>(n));
        for (int i = 0; i < n; i++)
        {
            for (int k = 0; k < n; k++)
            {
                for (int j = 0; j < n; j++)
                {
                    z[i][j] += x[i][k] * y[k][j];
                }
            }
        }
        return z;
    };
    while (b)
    {
        if (b & 1)
        {
            ans = mul(ans, a);
        }
        b >>= 1;
        if (b)
        {
            a = mul(a, a);
```

```
        }
    }
    return ans;
}
```

调用 `matPow(a,b)`，`a` 必须是方阵，$b = 0$ 时返回单位矩阵；斐波那契转移 $\begin{pmatrix}1 & 1\\ 1 & 0\end{pmatrix}$ 的 10 次幂应为 $\begin{pmatrix}89 & 55\\ 55 & 34\end{pmatrix}$。$O(n^3 \log(b+1))$ 时间、$O(n^2)$ 额外空间。转移是 min-加或 max-加运算时改用 9.1.004 `+min` / `+max` 半环矩阵；需要携带行列信息或下标运算符时用 9.1.001 普通矩阵与取模矩阵 的 `Mat<T>::pow`。

下面只调用本条的裸函数，检查手算三次幂、零次幂、单位阵与 Fibonacci 转移：

```
signed main()
{
    vector<vector<long long>> a{{1, 2}, {3, 4}};
    vector<vector<long long>> cube{{37, 54}, {81, 118}};
    assert(matPow(a, 3) == cube);
    assert(matPow(a, 0) == vector<vector<long long>>({{1, 0}, {0, 1}}));
    assert(matPow(vector<vector<long long>>{{1, 0}, {0, 1}}, 7) == vector<vector<long
    ↪  long>>({{1, 0}, {0, 1}}));
    auto f = matPow(vector<vector<long long>>{{1, 1}, {1, 0}}, 10);
    cout << f[0][0] << ' ' << f[0][1] << ' ' << f[1][0] << ' ' << f[1][1] << '\n'; // 期望输出:
 ↪  89 55 55 34
    return 0;
}
```

9.1.003 Frobenius 标准形矩阵快速幂（高级版）

基础二进制矩阵幂每个指数位都做一次 n^3 乘法。若指数很大且矩阵阶数也高，可先随机选 Krylov 向量，把空间拆成循环子空间，使 $P^{-1}AP$ 成为若干伴随块；每个 d 阶块上的幂只需算 x^k 对块多项式取模，最后再乘 P 与 P^{-1} 还原。构造基底时会验证新块多项式对旧块的整除关系；验证失败就丢弃本轮随机向量并重建，因此这是 Las Vegas 算法：运行时间随机，返回结果不带错误概率。

```
template <class T> struct NaivePolyMul
{
    vector<T> operator()(const vector<T> &a, const vector<T> &b) const
    {
        // a、b 是系数从低次到高次排列的多项式；返回普通卷积。
        if (a.empty() || b.empty())
        {
            return {};
        }
        vector<T> c(a.size() + b.size() - 1);
        for (int i = 0; i < (int)a.size(); i++)
        {
            for (int j = 0; j < (int)b.size(); j++)
            {
                c[i + j] += a[i] * b[j];
```

```
            }
        }
        return c;
    }
};

template <class T> struct FieldBasis
{
    // n 是向量维数，rk 是当前秩。
    int n, rk = 0;
    // basis 保存插入的原向量；row 是按首个非零位置存放的阶梯基，way 给出 row 关于 basis 的系数。
    vector<vector<T>> basis, row, way;

    FieldBasis(int n) : n(n), row(n, vector<T>(n)), way(n, vector<T>(n))
    {
        // 建立 n 维空线性基。
    }

    pair<bool, vector<T>> solveOrAdd(vector<T> x)
    {
        // x 是待查询向量；可表示时返回 true 及其关于 basis 的系数，否则插入 x 并返回 false。
        vector<T> original = x;
        vector<T> coef(n);
        for (int i = 0; i < n; i++)
        {
            if (row[i][i] == T(1))
            {
                coef[i] = x[i];
                for (int j = i; j < n; j++)
                {
                    x[j] -= coef[i] * row[i][j];
                }
            }
        }
        int p = 0;
        while (p < n && x[p] == T(0))
        {
            p++;
        }
        if (p == n)
        {
            vector<T> ans(rk);
            for (int i = 0; i < n; i++)
            {
                for (int j = 0; j < rk; j++)
                {
                    ans[j] += coef[i] * way[i][j];
                }
            }
            return {true, ans};
        }
        T z = T(1) / x[p];
```

```
        for (int j = p; j < n; j++)
        {
            x[j] *= z;
        }
        for (auto &v : coef)
        {
            v *= z;
        }
        basis.push_back(move(original));
        row[p] = move(x);
        way[p][rk] = z;
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < rk; j++)
            {
                way[p][j] -= coef[i] * way[i][j];
            }
        }
        rk++;
        return {false, {}};
    }

    void rollback(int r)
    {
        // r 是此前的秩；撤销最近插入的向量，直到秩恢复为 r。
        while (rk > r)
        {
            rk--;
            basis.pop_back();
            for (int i = 0; i < n; i++)
            {
                if (way[i][rk] != T(0))
                {
                    fill(row[i].begin(), row[i].end(), T(0));
                    fill(way[i].begin(), way[i].end(), T(0));
                }
            }
        }
    }
};

template <class T, class Conv = NaivePolyMul<T>> struct FrobPower
{
    using Poly = vector<T>;
    using Matrix = vector<vector<T>>;

    // n 是矩阵阶数；a 是原矩阵，p 的列是 Krylov 基，ip 是 p 的逆矩阵。
    int n;
    Matrix a, p, ip;
    // start 保存各循环子空间的起始向量；relation 保存对应伴随块的递推系数。
    vector<Poly> start, relation;
    // conv 计算多项式卷积，rng 产生试探向量，trials 记录完整重建次数。
```

```
    Conv conv;
    mt19937_64 rng;
    int trials = 0;

    FrobPower(Matrix a, Conv conv = Conv(), unsigned long long seed = 0)
        : n(a.size()), a(move(a)), conv(move(conv)), rng(seed)
    {
        // a 是域上的方阵，conv 是卷积器，seed 是固定随机种子；构造时完成 Frobenius 分解。
        // 调试检查，可删。
        for (const auto &r : this->a)
        {
            assert((int)r.size() == n);
        }
        do
        {
            trials++;
        } while (!trial());
    }

    Matrix pow(unsigned long long b)
    {
        // b 是非负指数；返回原矩阵的 b 次幂。
        Matrix x(n, vector<T>(n));
        int s = 0;
        for (const auto &g : relation)
        {
            int d = g.size();
            Poly f = polyPowMod(g, b);
            for (int j = 0; j < d; j++)
            {
                for (int i = 0; i < (int)f.size(); i++)
                {
                    x[s + i][s + j] = f[i];
                }
                if (j + 1 < d)
                {
                    f.insert(f.begin(), T(0));
                    divmod(f, g);
                }
            }
            s += d;
        }
        return matrixMul(matrixMul(p, x), ip);
    }

private:
    static void trim(Poly &f)
    {
        // f 的系数按低次到高次排列；原地删除高次零项。
        while (!f.empty() && f.back() == T(0))
        {
            f.pop_back();
```

```
        }
    }

    Poly divmod(Poly &f, const Poly &g)
    {
        // f 是被除式，g 表示 x^d-sum(g[i]x^i)；原地留下余数并返回商。
        trim(f);
        int d = g.size();
        Poly q;
        for (int i = (int)f.size() - 1; i >= d; i--)
        {
            q.push_back(f[i]);
            for (int j = 0; j < d; j++)
            {
                f[i - d + j] += f[i] * g[j];
            }
            f[i] = T(0);
        }
        trim(f);
        reverse(q.begin(), q.end());
        return q;
    }

    Poly polyMulMod(const Poly &x, const Poly &y, const Poly &g)
    {
        // x、y 是被乘多项式，g 表示模多项式；返回乘积的余式。
        Poly z = conv(x, y);
        divmod(z, g);
        return z;
    }

    Poly polyPowMod(const Poly &g, unsigned long long b)
    {
        // g 表示模多项式，b 是非负指数；返回 x^b 的余式。
        Poly ans{T(1)};
        Poly x{T(0), T(1)};
        divmod(x, g);
        while (b)
        {
            if (b & 1)
            {
                ans = polyMulMod(ans, x, g);
            }
            b >>= 1;
            if (b)
            {
                x = polyMulMod(x, x, g);
            }
        }
        return ans;
    }

```

```
    Matrix matrixMul(const Matrix &x, const Matrix &y)
    {
        // x、y 是 n 阶方阵；返回普通矩阵乘积。
        Matrix z(n, vector<T>(n));
        for (int i = 0; i < n; i++)
        {
            for (int k = 0; k < n; k++)
            {
                for (int j = 0; j < n; j++)
                {
                    z[i][j] += x[i][k] * y[k][j];
                }
            }
        }
        return z;
    }

    optional<Matrix> matrixInv(Matrix x)
    {
        // x 是 n 阶方阵；可逆时返回逆矩阵，否则返回空。
        Matrix y(n, vector<T>(n));
        for (int i = 0; i < n; i++)
        {
            y[i][i] = T(1);
        }
        for (int col = 0; col < n; col++)
        {
            int row = col;
            while (row < n && x[row][col] == T(0))
            {
                row++;
            }
            if (row == n)
            {
                return nullopt;
            }
            swap(x[row], x[col]);
            swap(y[row], y[col]);
            T z = T(1) / x[col][col];
            for (int j = 0; j < n; j++)
            {
                x[col][j] *= z;
                y[col][j] *= z;
            }
            for (int i = 0; i < n; i++)
            {
                if (i == col)
                {
                    continue;
                }
                T q = x[i][col];
                for (int j = 0; j < n; j++)
```

```
                {
                    x[i][j] -= q * x[col][j];
                    y[i][j] -= q * y[col][j];
                }
            }
        }
        return y;
    }

    vector<T> apply(const vector<T> &v)
    {
        // v 是 n 维列向量；返回 a*v。
        vector<T> w(n);
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                w[i] += a[i][j] * v[j];
            }
        }
        return w;
    }

    vector<T> randomVector()
    {
        // 返回由 rng 生成的 n 维试探向量。
        vector<T> v(n);
        for (auto &x : v)
        {
            x = T((long long)(rng() & 0x7FFFFFFFFFFFFFFFULL));
        }
        return v;
    }

    bool trial()
    {
        // 从空基底尝试构造一次 Frobenius 分解；成功时写入 p、ip 与 relation。
        start.clear();
        relation.clear();
        FieldBasis<T> s(n);
        while (s.rk < n)
        {
            int oldRank = s.rk;
            vector<T> v = randomVector();
            start.push_back(v);
            while (true)
            {
                auto [solved, coef] = s.solveOrAdd(v);
                if (!solved)
                {
                    v = apply(v);
                    continue;
```

```
                }
                Poly f(coef.begin() + oldRank, coef.end());
                if (f.empty())
                {
                    return false;
                }
                relation.push_back(f);
                if (start.size() == 1)
                {
                    break;
                }
                v = start.back();
                int pos = 0;
                for (int k = 0; k + 1 < (int)start.size(); k++)
                {
                    int d = relation[k].size();
                    Poly rem(coef.begin() + pos, coef.begin() + pos + d);
                    Poly q = divmod(rem, f);
                    if (!rem.empty())
                    {
                        return false;
                    }
                    for (int i = 0; i < (int)q.size(); i++)
                    {
                        for (int j = 0; j < n; j++)
                        {
                            v[j] -= q[i] * s.basis[pos + i][j];
                        }
                    }
                    pos += d;
                }
                s.rollback(oldRank);
                start.back() = v;
                for (int i = 0; i < (int)f.size(); i++)
                {
                    if (s.solveOrAdd(v).first)
                    {
                        return false;
                    }
                    if (i + 1 < (int)f.size())
                    {
                        v = apply(v);
                    }
                }
                break;
            }
        }
        p.assign(n, vector<T>(n));
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
```

```
                p[i][j] = s.basis[j][i];
            }
        }
        auto inv = matrixInv(p);
        if (!inv)
        {
            return false;
        }
        ip = move(*inv);
        return true;
    }
};
```

`FrobPower<T> fp(a,{},seed)` 构造后用 `fp.pow(k)`。`T` 必须是域；每轮分解与求逆为 $O(n^3)$，失败后会从空基底重试，`trials` 可查看实际轮数。一次 `pow` 的块内求幂为 $O(\sum_i d_i^2 \log(k+1))$，乘 P、P^{-1} 还原为 $O(n^3)$，峰值空间为 $O(n^2)$。第二模板参数只替换卷积器；当前 `divmod` 仍是二次取模，因此仅接入快速卷积只能减小卷积部分的常数。若 $M(d)$ 表示长度 d 的卷积复杂度，要达到 $O(\sum_i M(d_i)\log(k+1))$ 还必须同时换成快速多项式取模。有限域大小为 p 时单轮失败概率为 $O(n/p)$。它依赖域除法、卷积和随机 Krylov 分解，只适合大矩阵或超大指数；一般情况优先使用 9.1.002 矩阵快速幂。

固定查验时可在模 998244353 的域上取 `seed=0` 与 $A=\begin{pmatrix}1&1&0\\0&1&1\\0&0&1\end{pmatrix}$；`fp.pow(5)` 应为 $\begin{pmatrix}1&5&10\\0&1&5\\0&0&1\end{pmatrix}$。再分别对零矩阵、单位阵和两个 Krylov 块组成的分块矩阵核对指数 $0,1,2,17$ 的预先算定结果，可覆盖空乘积、不可逆矩阵和多块分解；完整随机检验放在本地测试文件。

9.1.004 `+min` / `+max` 半环矩阵

普通矩阵乘法 $C_{ij}=\sum_k A_{ik}B_{kj}$ 只要求标量加法与乘法构成半环——满足结合律、乘法对加法分配、有各自单位元——就能整套照搬。把 $(+,\times)$ 换成 $(\min,+)$ 得 min-plus（热带）半环，此时 M^b 的 (i,j) 项恰是 $i\to j$ 恰好 b 条边的最短路，正是 Floyd 分治与定长最短路的代数形式；换成 $(\max,+)$ 则给定长最长路。两者的加法单位元分别是 $+\infty$、$-\infty$ 哨兵，乘法单位元都是 0。`semiringMul` 与 `semiringPow` 把 `add`、`mul` 与单位元 `zero`、`one` 抽成回调，因此同一份幂运算能服务任意半环，不必为每种运算重写。

```
template <class T, class Add, class Mul>
vector<vector<T>> semiringMul(const vector<vector<T>> &a, const vector<vector<T>> &b, T zero,
 ↪  Add add, Mul mul)
{
    // a、b 是待乘矩阵，zero 是加法单位元；add、mul 定义半环加法与乘法。
    int n = a.size();
    int m = b.size();
    int k = m ? b[0].size() : 0;
    // 调试检查，可删。
    for (const auto &r : a)
    {
        assert((int)r.size() == m);
    }
    // 调试检查，可删。
```

```
    for (const auto &r : b)
    {
        assert((int)r.size() == k);
    }
    vector c(n, vector<T>(k, zero));
    for (int i = 0; i < n; i++)
    {
        for (int z = 0; z < m; z++)
        {
            for (int j = 0; j < k; j++)
            {
                c[i][j] = add(c[i][j], mul(a[i][z], b[z][j]));
            }
        }
    }
    return c;
}

template <class T, class Add, class Mul>
vector<vector<T>> semiringPow(vector<vector<T>> a, unsigned long long b, T zero, T one, Add
 ↪ add, Mul mul)
{
    // a 是方阵，b 是非负指数；zero、one 是加法与乘法单位元。
    int n = a.size();
    // 调试检查，可删。
    for (const auto &r : a)
    {
        assert((int)r.size() == n);
    }
    vector ans(n, vector<T>(n, zero));
    for (int i = 0; i < n; i++)
    {
        ans[i][i] = one;
    }
    while (b)
    {
        if (b & 1)
        {
            ans = semiringMul(ans, a, zero, add, mul);
        }
        b >>= 1;
        if (b)
        {
            a = semiringMul(a, a, zero, add, mul);
        }
    }
    return ans;
}
```

`semiringMul(a,b,zero,add,mul)` 做一次半环乘法，`semiringPow(a,b,zero,one,add,mul)` 对方阵求 b 次幂。min-plus 不能把普通 `+` 直接作为 `mul`：不可达哨兵与负边相加后会变成有限值。安全回调可写成：

```cpp
signed main()
{
    // 0->1 权 2, 1->2 权 3, 0->2 权 10; 求恰好走 2 条边的最短路。
    vector<vector<int>> g{{INF, 2, 10}, {INF, INF, 3}, {INF, INF, INF}};
    auto minAdd = [](int x, int y)
    {
        return min(x, y);
    };
    auto minMul = [](int x, int y)
    {
        return x == INF || y == INF ? INF : x + y;
    };
    auto d = semiringPow(g, 2, INF, 0LL, minAdd, minMul);
    assert(d[0][2] == 5 && d[0][1] == INF && d[1][1] == INF);
}
```

这里还要求所有有限距离及其加法都在 `long long` 范围内且不等于 `INF`。max-plus 同理使用 `-INF`，只要任一因子为 `-INF` 就返回 `-INF`。$n\times m$ 与 $m\times k$ 的半环乘法是 $O(nmk)$，n 阶方阵快速幂是 $O(n^3\log(b+1))$，空间 $O(n^2)$。转移是普通加乘时回到 9.1.001 普通矩阵与取模矩阵、9.1.002 矩阵快速幂；若求任意步数最短路而非恰好 k 条边，用图论章的 Floyd 或 Bellman-Ford 更直接。

9.1.005 矩阵求逆（质数模数 / 任意模数）

求方阵 A 的模逆矩阵，即找 B 使 $AB\equiv I$。素数模 p 下每个非零元都可逆，直接 Gauss-Jordan 消元即可；难点在合数模：此时非零元未必可逆，不能随便拿主元求逆。关键事实是 A 模 p^k 可逆当且仅当 $\det(A)$ 不被 p 整除，等价于消元时每一列都能找到模 p 非零的主元（模 p^k 的单位）。于是把模数分解成互质的质数幂 $\prod p_i^{k_i}$，`matInvPP` 在每个 $p_i^{k_i}$ 上单独消元（主元用扩欧求 `invInt` 得到模 p^k 逆元），再对每个位置用 CRT 把各分量唯一合并回模 mod。该接口的模数在运行期确定且允许合数，不能使用固定素数域 `MInt<P>`，宽乘法显式复用 7.1.001 快速幂 的 `mulMod`。任一分量不可逆则整体不可逆，返回空。

```cpp
// 依赖 7.1.001 快速幂
int invInt(int a, int mod)
{
    // a 与 mod 互质且 mod>1; 返回 a 模 mod 的最小非负逆元。
    assert(mod > 1); // 调试检查, 可删。
    a %= mod;
    if (a < 0)
    {
        a += mod;
    }
    int b = mod;
    __int128 x = 1;
    __int128 y = 0;
    while (b)
    {
        int q = a / b;
        a -= q * b;
        swap(a, b);
        x -= q * y;
```

```
        swap(x, y);
    }
    assert(a == 1); // 调试检查，可删。
    x %= mod;
    if (x < 0)
    {
        x += mod;
    }
    return (int)x;
}

optional<vector<vector<int>>> matInvPP(vector<vector<int>> a, int p, int pk)
{
    // a 是方阵，p 是质数，pk 是 p 的正次幂；返回模 pk 的逆矩阵，不可逆时返回空。
    int n = a.size();
    assert(p >= 2 && pk >= p); // 调试检查，可删。
    for (auto &row : a)
    {
        assert((int)row.size() == n); // 调试检查，可删。
        for (auto &x : row)
        {
            x %= pk;
            if (x < 0)
            {
                x += pk;
            }
        }
    }
    vector b(n, vector<int>(n));
    for (int i = 0; i < n; i++)
    {
        b[i][i] = 1;
    }
    for (int c = 0; c < n; c++)
    {
        int r = c;
        while (r < n && a[r][c] % p == 0)
        {
            r++;
        }
        if (r == n)
        {
            return nullopt;
        }
        swap(a[r], a[c]);
        swap(b[r], b[c]);
        int z = invInt(a[c][c], pk);
        for (int j = 0; j < n; j++)
        {
            a[c][j] = mulMod(a[c][j], z, pk);
            b[c][j] = mulMod(b[c][j], z, pk);
        }
```

```
        for (int i = 0; i < n; i++)
        {
            if (i == c)
            {
                continue;
            }
            int q = a[i][c];
            for (int j = 0; j < n; j++)
            {
                a[i][j] = (a[i][j] - mulMod(q, a[c][j], pk)) % pk;
                b[i][j] = (b[i][j] - mulMod(q, b[c][j], pk)) % pk;
                if (a[i][j] < 0)
                {
                    a[i][j] += pk;
                }
                if (b[i][j] < 0)
                {
                    b[i][j] += pk;
                }
            }
        }
    }
    return b;
}

optional<vector<vector<int>>> matInvAny(const vector<vector<int>> &a, int mod)
{
    // a 是方阵，mod>1 可为素数或合数；返回模 mod 的逆矩阵，不存在时返回空。
    assert(mod > 1); // 调试检查，可删。
    vector<pair<int, int>> fac;
    int x = mod;
    for (int p = 2; p <= x / p; p++)
    {
        if (x % p)
        {
            continue;
        }
        int pk = 1;
        while (x % p == 0)
        {
            x /= p;
            pk *= p;
        }
        fac.push_back({p, pk});
    }
    if (x > 1)
    {
        fac.push_back({x, x});
    }
    int n = a.size();
    int cur = 1;
    vector ans(n, vector<int>(n));
```

```
    for (auto [p, pk] : fac)
    {
        auto b = matInvPP(a, p, pk);
        if (!b)
        {
            return nullopt;
        }
        int z = invInt(cur % pk, pk);
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                int t = ((*b)[i][j] - ans[i][j]) % pk;
                if (t < 0)
                {
                    t += pk;
                }
                t = mulMod(t, z, pk);
                // 已处理模数 cur 与当前 pk 的乘积不超过总模数，故 cur*t 不溢出。
                ans[i][j] += cur * t;
            }
        }
        cur *= pk;
    }
    return ans;
}
```

调用 `matInvAny(a,mod)`，`a` 是方阵、`mod>1`，返回空表示不可逆；素数模也走同一接口，此时只有一个质数幂分量。矩阵 $\begin{pmatrix}2&3\\3&2\end{pmatrix}$ 模 6 可逆且逆恰为自身，$\begin{pmatrix}2&0\\0&1\end{pmatrix}$ 模 6 不可逆。若记模数为 M，试除分解最坏为 $O(\sqrt{M})$，随后是 $O(\omega(M)n^3)$ 时间、$O(n^2)$ 空间，其中 $\omega(M)$ 是 M 的不同质因子数。只要行列式而不需要整个逆矩阵时用 9.1.006 行列式（质数模数 / 任意模数）；解线性方程组时用 9.2.001 高斯消元：唯一解、无解、多解，避免多算 n 列。

下面直接检查两组已知逆矩阵和一组不可逆矩阵；不可逆样例的行列式为 2，与模数 6 不互质：

```
signed main()
{
    auto a = matInvAny({{1, 2}, {3, 4}}, 5);
    assert(a && *a == vector<vector<int>>({{3, 1}, {4, 2}}));
    auto b = matInvAny({{2, 3}, {3, 2}}, 6);
    assert(b && *b == vector<vector<int>>({{2, 3}, {3, 2}}));
    assert(!matInvAny({{2, 0}, {0, 1}}, 6));
    cout << (*b)[0][0] << ' ' << (*b)[0][1] << ' ' << (*b)[1][0] << ' ' << (*b)[1][1] << '\n';
    // 期望输出：2 3 3 2
    return 0;
}
```

9.1.006 行列式（质数模数 / 任意模数）

行列式在行初等变换下的行为完全确定：交换两行变号，某行乘常数 c 则整体乘 c，某行加另一行的倍数不变。把矩阵消成上三角后行列式就是对角线乘积（乘上累计的交换符号）。素数模下 `detPrime` 直接取主元逆元（费马小定理求 a^{mod-2}）消去下方各行。合数模不能求逆，`detAny` 改用辗转相除式消元：对同列的两行反复取商作整倍数相减再交换，就像对这两个元素跑欧几里得算法，把下方元素辗转消成 0，全程只用整数乘减、绝不请求模逆——代价是每列多花 $O(\log mod)$ 轮相减。任一列主元最终为 0 则行列式为 0。

```
// 依赖 7.1.001 快速幂
int detPrime(vector<vector<int>> a, int mod)
{
    // a 是方阵，mod 是素数；返回行列式对 mod 的最小非负余数。
    assert(mod > 1); // 调试检查，可删。
    int n = a.size();
    int ans = 1;
    for (auto &row : a)
    {
        assert((int)row.size() == n); // 调试检查，可删。
        for (auto &x : row)
        {
            x %= mod;
            if (x < 0)
            {
                x += mod;
            }
        }
    }
    for (int c = 0; c < n; c++)
    {
        int r = c;
        while (r < n && a[r][c] == 0)
        {
            r++;
        }
        if (r == n)
        {
            return 0;
        }
        if (r != c)
        {
            swap(a[r], a[c]);
            ans = (mod - ans) % mod;
        }
        ans = mulMod(ans, a[c][c], mod);
        int z = 1;
        int b = mod - 2;
        int x = a[c][c];
        while (b)
        {
            if (b & 1)
```

```
            {
                z = mulMod(z, x, mod);
            }
            b >>= 1;
            if (b)
            {
                x = mulMod(x, x, mod);
            }
        }
        for (int i = c + 1; i < n; i++)
        {
            int q = mulMod(a[i][c], z, mod);
            for (int j = c; j < n; j++)
            {
                a[i][j] = (a[i][j] - mulMod(q, a[c][j], mod)) % mod;
                if (a[i][j] < 0)
                {
                    a[i][j] += mod;
                }
            }
        }
    }
    return ans;
}

int detAny(vector<vector<int>> a, int mod)
{
    // a 是方阵，mod>1 可为合数；返回行列式对 mod 的最小非负余数，全程不做模逆。
    assert(mod > 1); // 调试检查，可删。
    int n = a.size();
    int ans = 1;
    for (auto &row : a)
    {
        assert((int)row.size() == n); // 调试检查，可删。
        for (auto &x : row)
        {
            x %= mod;
            if (x < 0)
            {
                x += mod;
            }
        }
    }
    for (int c = 0; c < n; c++)
    {
        for (int i = c + 1; i < n; i++)
        {
            while (a[i][c])
            {
                int q = a[c][c] / a[i][c];
                for (int j = c; j < n; j++)
                {
```

```
                    a[c][j] = (a[c][j] - mulMod(q, a[i][j], mod)) % mod;
                    if (a[c][j] < 0)
                    {
                        a[c][j] += mod;
                    }
                }
                swap(a[c], a[i]);
                ans = (mod - ans) % mod;
            }
        }
        if (a[c][c] == 0)
        {
            return 0;
        }
        ans = mulMod(ans, a[c][c], mod);
    }
    return ans;
}
```

模数是编译期已知且小于 2^{30} 的素数时，热点可改用 `detPrimeFast<P>`：矩阵改成连续 64 位数组，每行累计 8 次消元后才统一取模，避免内层每个元素一次 `%`。核心消元如下，完整初始化、换行与收尾见对应头文件：

```
template <int P> int detPrimeFast(const vector<vector<long long>> &src)
{
    // P 是小于 2^30 的素数，src 是方阵；返回 det(src) mod P，每行累计 8 次消元后统一取模。
    static_assert(1 < P && P < (1U << 30));
    int n = src.size();
    vector<unsigned long long> a((size_t)n * n);
    vector<int> cnt(n);
    for (int i = 0; i < n; i++)
    {
        assert(src[i].size() == n); // 调试检查，可删
        for (int j = 0; j < n; j++)
        {
            long long x = src[i][j] % (long long)P;
            if (x < 0)
            {
                x += P;
            }
            a[(size_t)i * n + j] = x;
        }
    }
    auto power = [](unsigned long long x, int b)
    {
        unsigned long long ans = 1;
        for (; b; b >>= 1, x = x * x % P)
        {
            if (b & 1)
            {
                ans = ans * x % P;
```

```
            }
        }
        return (int)ans;
    };
    bool neg = false;
    for (int i = 0; i < n; i++)
    {
        for (int j = i; j < n; j++)
        {
            a[(size_t)j * n + i] %= P;
        }
        int p = i;
        while (p < n && !a[(size_t)p * n + i])
        {
            p++;
        }
        if (p == n)
        {
            return 0;
        }
        if (p != i)
        {
            for (int j = i; j < n; j++)
            {
                swap(a[(size_t)i * n + j], a[(size_t)p * n + j]);
            }
            swap(cnt[i], cnt[p]);
            neg = !neg;
        }
        auto row = a.data() + (size_t)i * n;
        for (int j = i; j < n; j++)
        {
            row[j] %= P;
        }
        int iv = power(row[i], P - 2);
        for (int k = i + 1; k < n; k++)
        {
            auto cur = a.data() + (size_t)k * n;
            int q = cur[i] ? P - (unsigned long long)iv * cur[i] % P : 0;
            int j = i;
            for (; j + 4 <= n; j += 4)
            {
                cur[j] += (unsigned long long)q * row[j];
                cur[j + 1] += (unsigned long long)q * row[j + 1];
                cur[j + 2] += (unsigned long long)q * row[j + 2];
                cur[j + 3] += (unsigned long long)q * row[j + 3];
            }
            for (; j < n; j++)
            {
                cur[j] += (unsigned long long)q * row[j];
            }
            if (++cnt[k] == 8)
```

```
            {
                cnt[k] = 0;
                for (int j = i; j < n; j++)
                {
                    cur[j] %= P;
                }
            }
        }
    }
    unsigned long long ans = neg ? P - 1 : 1;
    for (int i = 0; i < n; i++)
    {
        ans = ans * (a[(size_t)i * n + i] % P) % P;
    }
    return ans;
}
```

已知 `mod` 为素数时调用 `detPrime(a,mod)`，可能为合数时调用 `detAny(a,mod)`；两者都接受负元素并返回最小非负余数。固定小素数且矩阵较大时调用 `detPrimeFast<P>(a)`；它仍是便携 C++，未绑定 AVX2，但要求 $P<2^{30}$ 以证明 8 轮延迟取模不溢出 64 位。矩阵 $\begin{pmatrix}2&3\\3&2\end{pmatrix}$ 的整数行列式为 -5，故 `detPrime(a,7)` 返回 2、`detAny(a,6)` 返回 1。三版渐进复杂度不变；本机 400×400 随机矩阵的 5 轮中位数约为 `8.2ms`，逐元素取模版约为 `25.8ms`。运行时模数或大模数不能用固定版。若记模数为 M，素数模版本为 $O(n^3+n\log M)$，任意模版本为 $O(n^3\log M)$ 次基本算术，空间均为 $O(n^2)$。需要在模意义下解方程或求逆而非只要行列式时用 9.2.001 高斯消元：唯一解、无解、多解、9.1.005 矩阵求逆（质数模数 / 任意模数）；要 $\det(xI-A)$ 的整条特征多项式时用 9.1.007 矩阵特征多项式。

下面用两个手算行列式的固定矩阵检查素数模、合数模、负元素和奇异矩阵；只保留一个 3×3 例，不做大规模随机对拍：

```
signed main()
{
    vector<vector<int>> a{{2, 3}, {3, 2}};
    assert(detPrime(a, 7) == 2);
    assert(detAny(a, 6) == 1);
    vector<vector<int>> b{{1, 2, 3}, {0, 1, 4}, {5, 6, 0}};
    assert(detPrime(b, 7) == 1 && detAny(b, 6) == 1);
    assert(detAny({{1, 2}, {2, 4}}, 6) == 0);
    cout << detAny(a, 6) << ' ' << detPrime(b, 7) << '\n'; // 期望输出：1 1
    return 0;
}
```

9.1.007 矩阵特征多项式

特征多项式 $p(x)=\det(xI-A)$ 的系数装着行列式、迹等谱信息，也是 Cayley-Hamilton（$p(A)=0$）与常系数矩阵递推降幂的基础。`charPoly` 先用相似变换把 A 化成上 Hessenberg 形 H（$i>j+1$ 时 $h_{i,j}=0$）：对第 j 列在第 $j+1$ 行以下选非零主元，交换行与对应列，再用行变换 $r_i\leftarrow r_i-qr_{j+1}$ 消去下方元素，并同步做列变换 $c_{j+1}\leftarrow c_{j+1}+qc_i$，保持相似从而特征多项式不变。之后记 p_i 为 H 左上

i 阶主子阵的特征多项式，$p_0 = 1$，沿最后一列展开（下标从 1 开始）得

$$p_i(x) = (x - h_{i,i})\,p_{i-1}(x) - \sum_{m=1}^{i-1} h_{i-m,i}\Big(\prod_{t=0}^{m-1} h_{i-t,i-t-1}\Big)p_{i-m-1}(x),$$

p_n 即所求。消元要做除法，`T` 必须是域（模素数的取模类或浮点）。返回顺序是从常数项到最高次项：`c[n]` 恒为 1，`c[0]` 是 $(-1)^n \det A$。`detAPlusBz(a,b)` 在同一框架下求 $\det(A + Bz)$。

```
template <class T> vector<T> charPoly(const vector<vector<T>> &a)
{
    // a 是域上的方阵；返回 det(xI-a) 从常数项到最高次项的系数。
    int n = a.size();
    // 调试检查，可删。
    for (const auto &r : a)
    {
        assert((int)r.size() == n);
    }
    vector h = a;
    for (int col = 0; col + 2 < n; col++)
    {
        int p = col + 1;
        while (p < n && h[p][col] == T(0))
        {
            p++;
        }
        if (p == n) continue;
        if (p != col + 1)
        {
            swap(h[p], h[col + 1]);
            for (int i = 0; i < n; i++)
            {
                swap(h[i][p], h[i][col + 1]);
            }
        }
        T inv = T(1) / h[col + 1][col];
        for (int row = col + 2; row < n; row++)
        {
            if (h[row][col] == T(0)) continue;
            T q = h[row][col] * inv;
            for (int j = col; j < n; j++)
            {
                h[row][j] -= q * h[col + 1][j];
            }
            for (int i = 0; i < n; i++)
            {
                h[i][col + 1] += q * h[i][row];
            }
        }
    }
    vector<vector<T>> p(n + 1);
    p[0] = {T(1)};
    for (int i = 1; i <= n; i++)
```

```
    {
        p[i].assign(i + 1, T(0));
        for (int j = 0; j < i; j++)
        {
            p[i][j] -= p[i - 1][j] * h[i - 1][i - 1];
            p[i][j + 1] += p[i - 1][j];
        }
        T prod = 1;
        for (int len = 1; len < i; len++)
        {
            prod *= h[i - len][i - len - 1];
            T q = prod * h[i - len - 1][i - 1];
            for (int j = 0; j <= i - len - 1; j++)
            {
                p[i][j] -= q * p[i - len - 1][j];
            }
        }
    }
    return p[n];
}

template <class T> vector<T> detAPlusBz(vector<vector<T>> a,
                                        vector<vector<T>> b)
{
    // a、b 是域上的同阶方阵；返回 det(a+bz) 从常数项到最高次项的系数，允许 b 奇异。
    int n = a.size(), shift = 0;
    assert((int)b.size() == n); // 调试检查，可删
    for (int i = 0; i < n; i++)
    {
        assert((int)a[i].size() == n && (int)b[i].size() == n); // 调试检查，可删
    }
    T scale = T(1);
    for (int col = 0; col < n; col++)
    {
        while (true)
        {
            for (int p = 0; p < col; p++)
            {
                T w = b[p][col];
                for (int i = 0; i < n; i++)
                {
                    a[i][col] -= w * a[i][p];
                }
                b[p][col] = T(0);
            }
            int p = col;
            while (p < n && b[p][col] == T(0))
            {
                p++;
            }
            if (p < n) break;
            if (++shift > n) return vector<T>(n + 1);
```

```
            for (int i = 0; i < n; i++)
            {
                b[i][col] = a[i][col], a[i][col] = T(0);
            }
        }
        int p = col;
        while (b[p][col] == T(0))
        {
            p++;
        }
        if (p != col) scale = -scale, swap(a[p], a[col]), swap(b[p], b[col]);
        scale *= b[col][col];
        T inv = T(1) / b[col][col];
        for (int j = 0; j < n; j++)
        {
            a[col][j] *= inv, b[col][j] *= inv;
        }
        for (int i = col + 1; i < n; i++)
        {
            T w = b[i][col];
            for (int j = 0; j < n; j++)
            {
                a[i][j] -= w * a[col][j], b[i][j] -= w * b[col][j];
            }
        }
    }
    for (auto &row : a)
    {
        for (T &x : row)
        {
            x = -x;
        }
    }
    for (int col = 0; col + 1 < n; col++)
    {
        int p = col + 1;
        while (p < n && a[p][col] == T(0))
        {
            p++;
        }
        if (p == n) continue;
        swap(a[p], a[col + 1]);
        for (int i = 0; i < n; i++)
        {
            swap(a[i][p], a[i][col + 1]);
        }
        T inv = T(1) / a[col + 1][col];
        for (int i = col + 2; i < n; i++)
        {
            T w = a[i][col] * inv;
            for (int j = 0; j < n; j++)
            {
```

```
                a[i][j] -= w * a[col + 1][j], a[j][col + 1] += w * a[j][i];
            }
        }
    }
    vector<vector<T>> f(n + 1);
    f[0] = {T(1)};
    for (int m = 1; m <= n; m++)
    {
        f[m].assign(m + 1, T(0));
        for (int i = 0; i < m; i++)
        {
            T w = a[i][m - 1];
            for (int k = i + 1; k < m; k++)
            {
                w *= a[k][k - 1];
            }
            for (int d = 0; d <= i; d++)
            {
                f[m][d] -= w * f[i][d];
            }
        }
        for (int d = 0; d < m; d++)
        {
            f[m][d + 1] += f[m - 1][d];
        }
    }
    vector<T> ans(n + 1);
    for (int d = 0; d + shift <= n; d++)
    {
        ans[d] = f[n][d + shift] * scale;
    }
    return ans;
}
```

调用 `charPoly(a)` 传入域上的方阵，返回长度 $n+1$ 的 $\det(xI-A)$ 系数向量；Hessenberg 相似变换与递推共 $O(n^3)$ 时间、$O(n^2)$ 空间。`detAPlusBz(a,b)` 返回 $\det(A+Bz)$，允许 B 奇异并自动处理最低非零次数的平移，复杂度同为 $O(n^3)$。只要行列式时仍使用 9.1.006 行列式（质数模数 / 任意模数）。

下面直接核对空矩阵、对角矩阵与一个非对角矩阵的已知特征多项式：

```
signed main()
{
    auto same = [](const vector<long double> &a, const vector<long double> &b)
    {
        if (a.size() != b.size())
        {
            return false;
        }
        for (int i = 0; i < (int)a.size(); i++)
        {
            if (fabsl(a[i] - b[i]) > 1E-10L)
            {
```

```
                return false;
            }
        }
        return true;
    };
    assert(same(charPoly<long double>({}), {1}));
    assert(same(charPoly<long double>({{1, 0}, {0, 2}}), {2, -3, 1})); // 即 x^2-3x+2。
    auto c = charPoly<long double>({{1, 2}, {3, 4}});
    assert(same(c, {-2, -5, 1}));                                      // 即 x^2-5x-2。
    return 0;
}
```

9.1.008 稀疏矩阵

当矩阵非零元个数 nnz 远少于 $n \times m$ 时，稠密存储既费内存又让每次乘法白算大量零。`SparseMat` 按行存（列下标，值），只保留非零项，矩阵乘向量 $y = Ax$ 就退化为遍历每行的非零表累加 $y_i \mathrel{+}= v \cdot x_j$。它专为幂法、共轭梯度等只需反复算 Ax 的线性迭代设计；若做稀疏乘稀疏且结果趋于稠密，或反复自乘导致填充增长，稀疏表的优势会消失，此时应转回稠密矩阵。`add(i,j,x)` 中 `i`、`j` 分别是行、列下标，同一位置可加入多项并在乘法时累加。

```
template <class T> struct SparseMat
{
    // n、m 是行数与列数，a[i] 存第 i 行的 (列下标, 值)。
    int n, m;
    vector<vector<pair<int, T>>> a;

    SparseMat(int n, int m) : n(n), m(m), a(n)
    {
        // 建立 n 行 m 列的零矩阵。
    }

    void add(int i, int j, T x)
    {
        // 在 (i,j) 加一项 x；重复位置会在乘法时累加，零项应由调用方过滤。
        assert(0 <= i && i < n && 0 <= j && j < m); // 调试检查，可删。
        a[i].push_back({j, x});
    }

    vector<T> operator*(const vector<T> &x) const
    {
        // x 是 m 维列向量；返回 A*x。
        assert((int)x.size() == m); // 调试检查，可删。
        vector<T> y(n);
        for (int i = 0; i < n; i++)
        {
            for (auto [j, v] : a[i])
            {
                y[i] += v * x[j];
            }
```

```
        }
        return y;
    }
};
```

以 `SparseMat<T> s(n,m)` 建 $n \times m$ 零矩阵，`s.add(i,j,x)` 只加入非零项，`s*x` 返回长度 n 的乘积向量。同一位置可加入多次，乘法时各项自然相加；若把零值也放入表中，算法仍正确但会浪费时间和空间。单次矩阵乘向量为 $O(\mathrm{nnz})$，存储为 $O(n + \mathrm{nnz})$。矩阵稠密时回到 9.1.001 普通矩阵与取模矩阵；固定步数的半环最短路用 9.1.004 `+min` / `+max` 半环矩阵，那是稠密表示。

9.1.009 Strassen 矩阵乘法

把两个 $n \times n$ 矩阵各切成四块 $m \times m (m = n/2)$，朴素分块乘法要算八次子块乘积。Strassen 用七次线性组合乘积 $p_1, \dots, p_7$ 拼出全部四块结果，把递归式从 $T(n) = 8T(n/2)$ 降到 $T(n) = 7T(n/2) + O(n^2)$，复杂度为 $O(n^{\log_2 7})$。本实现在规模 ≤ 32 时退回朴素乘法；入口先把行数、公共维数和列数补齐到同一个二次幂，算完再裁回原形。实现放在 `strassen_mul` 命名空间内，避免其中的矩阵别名与 9.1.001 普通矩阵与取模矩阵 的 `Mat` 结构体重名。

```
namespace strassen_mul
{
template <class T> using Mat = vector<vector<T>>;

template <class T> Mat<T> matAdd(const Mat<T> &a, const Mat<T> &b, int sgn = 1)
{
    // a、b 是同阶方阵；sgn=1 时相加，sgn=-1 时相减。
    int n = a.size();
    Mat<T> c(n, vector<T>(n));
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            c[i][j] = a[i][j] + (sgn == 1 ? b[i][j] : -b[i][j]);
        }
    }
    return c;
}

template <class T> Mat<T> strassenRec(const Mat<T> &a, const Mat<T> &b)
{
    // a、b 是同阶二次幂方阵；返回其乘积。
    int n = a.size();
    if (n <= 32)
    {
        Mat<T> c(n, vector<T>(n));
        for (int i = 0; i < n; i++)
        {
            for (int k = 0; k < n; k++)
            {
                for (int j = 0; j < n; j++)
```

```
                {
                    c[i][j] += a[i][k] * b[k][j];
                }
            }
        }
        return c;
    }
    int m = n / 2;
    Mat<T> x[2][2], y[2][2];
    for (int p = 0; p < 2; p++)
    {
        for (int q = 0; q < 2; q++)
        {
            x[p][q].assign(m, vector<T>(m));
            y[p][q].assign(m, vector<T>(m));
            for (int i = 0; i < m; i++)
            {
                copy_n(a[i + p * m].begin() + q * m, m, x[p][q][i].begin());
                copy_n(b[i + p * m].begin() + q * m, m, y[p][q][i].begin());
            }
        }
    }
    auto p1 = strassenRec(matAdd(x[0][0], x[1][1]), matAdd(y[0][0], y[1][1]));
    auto p2 = strassenRec(matAdd(x[1][0], x[1][1]), y[0][0]);
    auto p3 = strassenRec(x[0][0], matAdd(y[0][1], y[1][1], -1));
    auto p4 = strassenRec(x[1][1], matAdd(y[1][0], y[0][0], -1));
    auto p5 = strassenRec(matAdd(x[0][0], x[0][1]), y[1][1]);
    auto p6 = strassenRec(matAdd(x[1][0], x[0][0], -1), matAdd(y[0][0], y[0][1]));
    auto p7 = strassenRec(matAdd(x[0][1], x[1][1], -1), matAdd(y[1][0], y[1][1]));
    Mat<T> c(n, vector<T>(n));
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < m; j++)
        {
            c[i][j] = p1[i][j] + p4[i][j] - p5[i][j] + p7[i][j];
            c[i][j + m] = p3[i][j] + p5[i][j];
            c[i + m][j] = p2[i][j] + p4[i][j];
            c[i + m][j + m] = p1[i][j] - p2[i][j] + p3[i][j] + p6[i][j];
        }
    }
    return c;
}

template <class T> Mat<T> strassen(const Mat<T> &a, const Mat<T> &b)
{
    // a 是 n*m 矩阵，b 是 m*k 矩阵；返回 n*k 乘积。
    int n = a.size();
    int m = b.size();
    int k = m ? b[0].size() : 0;
    // 调试检查，可删。
    for (const auto &r : a)
    {
```

```
        assert((int)r.size() == m);
    }
    // 调试检查，可删。
    for (const auto &r : b)
    {
        assert((int)r.size() == k);
    }
    if (!n || !m || !k)
    {
        return Mat<T>(n, vector<T>(k));
    }
    int z = (int)bit_ceil((unsigned long long)max({n, m, k}));
    Mat<T> x(z, vector<T>(z));
    Mat<T> y(z, vector<T>(z));
    for (int i = 0; i < n; i++)
    {
        copy(a[i].begin(), a[i].end(), x[i].begin());
    }
    for (int i = 0; i < m; i++)
    {
        copy(b[i].begin(), b[i].end(), y[i].begin());
    }
    x = strassenRec(x, y);
    x.resize(n);
    for (auto &v : x)
    {
        v.resize(k);
    }
    return x;
}
}
```

调用 `strassen_mul::strassen(a,b)`，`a` 是 $n\times m$、`b` 是 $m\times k$，返回 $n\times k$ 乘积；零行或零列矩阵返回对应零形。补零尺寸用 64 位无符号整数求 $N=\text{bit_ceil}(\max(n,m,k))$，要求该尺寸和对应矩阵均可分配。元素类型 `T` 必须同时容纳最终乘积与递归中的块加减，最终答案可表示并不能单独保证 Strassen 的临时值可表示。时间 $O(N^{\log_2 7})$、峰值空间 $O(N^2)$；常规规模或模矩阵仍优先用 9.1.001 普通矩阵与取模矩阵。

9.1.010 Woodbury 矩阵恒等式

已知 A^{-1}，若 A 又叠加一个低秩修正变成 $A+UCV$（U 是 $n\times k$、C 是 $k\times k$、V 是 $k\times n$，$k\ll n$），从头重算逆是 $O(n^3)$，太浪费。Woodbury 恒等式 $(A+UCV)^{-1}=A^{-1}-A^{-1}U(C^{-1}+VA^{-1}U)^{-1}VA^{-1}$ 把它转成只对一个 $k\times k$ 矩阵 $C^{-1}+VA^{-1}U$ 求逆，其余都是矩阵乘法。要求 A、C 以及那个 $k\times k$ 中间矩阵都可逆。$k=1$ 的特例即 Sherman–Morrison。下面 `inv` 用带部分选主元的高斯消元求逆（`long double`），`woodbury(ai,u,ci,v)` 直接吃 A^{-1}、C^{-1} 与低秩因子。

```
namespace woodbury
{
using Mat = vector<vector<long double>>;
```

```cpp

Mat mul(const Mat &a, const Mat &b)
{
    // a、b 是维数相容矩阵；返回矩阵乘积。
    if (a.empty())
    {
        return {};
    }
    if (b.empty())
    {
        for (const auto &r : a)
        {
            assert(r.empty()); // 调试检查，可删。
        }
        return Mat(a.size());
    }
    int n = a.size();
    int k = b.size();
    int m = b[0].size();
    // 调试检查，可删。
    for (const auto &r : b)
    {
        assert((int)r.size() == m);
    }
    Mat c(n, vector<long double>(m));
    for (int i = 0; i < n; i++)
    {
        assert((int)a[i].size() == k); // 调试检查，可删。
        for (int p = 0; p < k; p++)
        {
            for (int j = 0; j < m; j++)
            {
                c[i][j] += a[i][p] * b[p][j];
            }
        }
    }
    return c;
}

Mat inv(Mat a)
{
    // a 是可逆方阵；返回高斯消元求得的逆矩阵。
    int n = a.size();
    Mat b(n, vector<long double>(2 * n));
    for (int i = 0; i < n; i++)
    {
        assert((int)a[i].size() == n); // 调试检查，可删。
        copy(a[i].begin(), a[i].end(), b[i].begin());
        b[i][n + i] = 1;
    }
    for (int j = 0; j < n; j++)
    {
```

```
        int p = j;
        for (int i = j + 1; i < n; i++)
        {
            if (abs(b[i][j]) > abs(b[p][j]))
            {
                p = i;
            }
        }
        assert(abs(b[p][j]) > 1E-18L); // 调试检查，可删。
        swap(b[p], b[j]);
        long double x = b[j][j];
        for (auto &v : b[j])
        {
            v /= x;
        }
        for (int i = 0; i < n; i++)
        {
            if (i == j)
            {
                continue;
            }
            x = b[i][j];
            for (int k = 0; k < 2 * n; k++)
            {
                b[i][k] -= x * b[j][k];
            }
        }
    }
    Mat ans(n, vector<long double>(n));
    for (int i = 0; i < n; i++)
    {
        copy(b[i].begin() + n, b[i].end(), ans[i].begin());
    }
    return ans;
}

Mat woodbury(const Mat &ai, const Mat &u, const Mat &ci, const Mat &v)
{
    // ai、ci 分别是 A 与 C 的逆，u、v 是低秩因子；返回 (A+UCV) 的逆。
    int n = ai.size();
    int k = ci.size();
    assert((int)u.size() == n && (int)v.size() == k); // 调试检查，可删。
    // 调试检查，可删。
    for (const auto &r : ai)
    {
        assert((int)r.size() == n);
    }
    // 调试检查，可删。
    for (const auto &r : u)
    {
        assert((int)r.size() == k);
    }
```

```
    // 调试检查，可删。
    for (const auto &r : ci)
    {
        assert((int)r.size() == k);
    }
    // 调试检查，可删。
    for (const auto &r : v)
    {
        assert((int)r.size() == n);
    }
    if (n == 0)
    {
        return {};
    }
    if (k == 0)
    {
        return ai;
    }
    Mat left = mul(ai, u);
    Mat right = mul(v, ai);
    Mat mid = ci;
    Mat vaiu = mul(right, u);
    for (int i = 0; i < (int)mid.size(); i++)
    {
        for (int j = 0; j < (int)mid.size(); j++)
        {
            mid[i][j] += vaiu[i][j];
        }
    }
    Mat sub = mul(mul(left, inv(mid)), right);
    Mat ans = ai;
    for (int i = 0; i < (int)ans.size(); i++)
    {
        for (int j = 0; j < (int)ans.size(); j++)
        {
            ans[i][j] -= sub[i][j];
        }
    }
    return ans;
}
}
```

调用 `woodbury::woodbury(ai,u,ci,v)`，其中 `ai=A^{-1}`、`ci=C^{-1}` 需调用方预先备好（或用同命名空间的 `inv` 现算）。`k=0` 是合法的零秩修正，此时 `ci`、`v` 为空、`u` 每行为空，函数直接返回 `ai`。低秩为 k、原维数为 n 时 $O(n^2k+nk^2+k^3)$ 时间、$O(n^2+k^2)$ 空间，当 $k \ll n$ 时远优于重新求逆。若修正不是低秩、或没有现成 A^{-1}，直接对 $A+UCV$ 用 9.1.005 矩阵求逆（质数模数 / 任意模数）（模意义）或本条 `inv`（浮点）从头求逆。

9.1.011 伴随矩阵

伴随矩阵满足 $A\,\mathrm{adj}(A)=\det(A)I$。满秩时直接用 $\mathrm{adj}(A)=\det(A)A^{-1}$；秩为 $n-1$ 时，右零空间与左零空间都为一维，伴随矩阵是两个零向量外积乘一个标量，标量由一个 $(n-1)\times(n-1)$ 余子式确定；秩至多 $n-2$ 时所有余子式都为零，伴随矩阵就是零矩阵。下面实现只要求素数模，按行最简形确定秩和零向量，整个过程是确定性的，不依赖随机增广。

```
// 依赖 7.1.001 快速幂
// 依赖 9.1.005 矩阵求逆（质数模数 / 任意模数）
// 依赖 9.1.006 行列式（质数模数 / 任意模数）
vector<vector<int>> adjugateP(vector<vector<int>> a, int mod)
{
    // a 是方阵，mod 是素数；返回 A 的伴随矩阵。
    assert(mod > 1); // 调试检查，可删。
    int n = a.size();
    for (auto &r : a)
    {
        assert((int)r.size() == n); // 调试检查，可删。
        for (auto &x : r)
        {
            x %= mod;
            if (x < 0)
            {
                x += mod;
            }
        }
    }
    if (n == 0)
    {
        return {};
    }
    auto power = [&](int x, int b)
    {
        // x 是非零剩余，b 是非负指数；返回 x 的 b 次幂对 mod 的余数。
        int ans = 1;
        while (b)
        {
            if (b & 1)
            {
                ans = mulMod(ans, x, mod);
            }
            x = mulMod(x, x, mod);
            b >>= 1;
        }
        return ans;
    };
    auto reduce = [&](vector<vector<int>> m)
    {
        // m 是 n 阶方阵；返回秩、行最简形和每列对应的主元行，非主元列记为 -1。
        vector<int> pivot(n, -1);
        int rk = 0;
```

```
        for (int col = 0; col < n && rk < n; col++)
        {
            int row = rk;
            while (row < n && m[row][col] == 0)
            {
                row++;
            }
            if (row == n)
            {
                continue;
            }
            swap(m[row], m[rk]);
            int z = power(m[rk][col], mod - 2);
            for (int j = 0; j < n; j++)
            {
                m[rk][j] = mulMod(m[rk][j], z, mod);
            }
            for (int i = 0; i < n; i++)
            {
                if (i == rk)
                {
                    continue;
                }
                int q = m[i][col];
                for (int j = 0; j < n; j++)
                {
                    m[i][j] = (m[i][j] - mulMod(q, m[rk][j], mod)) % mod;
                    if (m[i][j] < 0)
                    {
                        m[i][j] += mod;
                    }
                }
            }
            pivot[col] = rk++;
        }
        return tuple<int, vector<vector<int>>, vector<int>>(rk, move(m), move(pivot));
    };
    auto [rk, r, pivot] = reduce(a);
    vector ans(n, vector<int>(n));
    if (rk == n)
    {
        auto inv = matInvPP(a, mod, mod);
        int det = detPrime(a, mod);
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                ans[i][j] = mulMod(det, (*inv)[i][j], mod);
            }
        }
        return ans;
    }
```

```
    if (rk <= n - 2)
    {
        return ans;
    }
    int fx = find(pivot.begin(), pivot.end(), -1) - pivot.begin();
    vector<int> x(n);
    x[fx] = 1;
    for (int col = 0; col < n; col++)
    {
        if (pivot[col] != -1)
        {
            x[col] = (mod - r[pivot[col]][fx]) % mod;
        }
    }
    vector<vector<int>> at(n, vector<int>(n));
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            at[i][j] = a[j][i];
        }
    }
    auto [rk2, rt, pivot2] = reduce(at);
    (void)rk2;
    int fy = find(pivot2.begin(), pivot2.end(), -1) - pivot2.begin();
    vector<int> y(n);
    y[fy] = 1;
    for (int col = 0; col < n; col++)
    {
        if (pivot2[col] != -1)
        {
            y[col] = (mod - rt[pivot2[col]][fy]) % mod;
        }
    }
    int i0 = 0, j0 = 0;
    while (x[i0] == 0)
    {
        i0++;
    }
    while (y[j0] == 0)
    {
        j0++;
    }
    vector<vector<int>> minor;
    for (int i = 0; i < n; i++)
    {
        if (i == j0)
        {
            continue;
        }
        vector<int> row;
        for (int j = 0; j < n; j++)
```

```
        {
            if (j != i0)
            {
                row.push_back(a[i][j]);
            }
        }
        minor.push_back(move(row));
    }
    int cof = detPrime(minor, mod);
    if ((i0 + j0) & 1 && cof)
    {
        cof = mod - cof;
    }
    int scale = mulMod(cof, power(x[i0], mod - 2), mod);
    scale = mulMod(scale, power(y[j0], mod - 2), mod);
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            ans[i][j] = mulMod(mulMod(scale, x[i], mod), y[j], mod);
        }
    }
    return ans;
}
```

`adjugateP(a,mod)` 覆盖满秩、秩 $n-1$ 和秩至多 $n-2$ 三种情况，返回值总在 $[0,mod)$。时间为 $O(n^3)$，空间为 $O(n^2)$；满秩分支复用本章的 9.1.005 矩阵求逆（质数模数 / 任意模数） 和 9.1.006 行列式（质数模数 / 任意模数）。只要逆矩阵而不要伴随矩阵时不要额外乘行列式。

下面分别检查三种秩：

```
signed main()
{
    assert(adjugateP({{1, 2}, {3, 4}}, 5) == vector<vector<int>>({{4, 3}, {2, 1}}));
    assert(adjugateP({{1, 2}, {2, 4}}, 5) == vector<vector<int>>({{4, 3}, {3, 1}}));
    assert(adjugateP({{0, 0}, {0, 0}}, 5) == vector<vector<int>>({{0, 0}, {0, 0}}));
    assert(adjugateP({{0}}, 5) == vector<vector<int>>({{1}}));
    cout << "OK\n";
    return 0;
}
```

9.1.012 Hafnian 与一般图完美匹配计数

对 $2n\times 2n$ 对称矩阵 A，Hafnian 是所有完美匹配边权乘积之和：$\mathrm{haf}(A)=\sum_{M}\prod_{(i,j)\in M}A_{ij}$。邻接矩阵取 $0/1$ 时，它就是一般图完美匹配数。直接枚举匹配约有 $(2n-1)!!$ 项；Bjorklund 型算法把顶点预先两两分组，把交错环贡献写成集合函数 `cyc`，再调用 10.1.013 集合形式幂级数 的 `setExp`。子集卷积与集合指数的表示、工作区和边界完全适用，因此本条只保留 Hafnian 自己的状态转移。

```cpp
// 依赖 10.1.013 集合形式幂级数
template <class T> T hafnian(const vector<vector<T>> &a)
{
    // a 是 2n 阶对称矩阵；返回其 Hafnian。
    int N = a.size();
    assert(N % 2 == 0); // 调试检查，可删。
    int n = N / 2;
    assert(n <= 20); // 调试检查，可删。
    // 调试检查，可删。
    for (int i = 0; i < N; i++)
    {
        assert((int)a[i].size() == N);
        for (int j = 0; j < N; j++)
        {
            assert(a[i][j] == a[j][i]);
        }
    }
    vector<T> cyc(1LL << n);
    for (int i = 0; i < n; i++)
    {
        int u = 2 * i;
        int v = u + 1;
        int w = 2 * i;
        cyc[1LL << i] += a[u][v];
        vector<T> dp(w * (1LL << i));
        for (int j = 0; j < i; j++)
        {
            int x = 2 * j;
            int y = x + 1;
            dp[w * (1LL << j) + x] += a[u][y];
            dp[w * (1LL << j) + y] += a[u][x];
        }
        for (int mask = 0; mask < (1LL << i); mask++)
        {
            for (int j = 0; j < i; j++)
            {
                int x = 2 * j;
                int y = x + 1;
                cyc[mask | (1LL << i)] += dp[w * mask + x] * a[v][x];
                cyc[mask | (1LL << i)] += dp[w * mask + y] * a[v][y];
                int rest = ((1LL << i) - 1) ^ mask;
                while (rest)
                {
                    int k = countr_zero((unsigned)rest);
                    rest &= rest - 1;
                    int p = 2 * k;
                    int q = p + 1;
                    int to = mask | (1LL << k);
                    dp[w * to + p] += dp[w * mask + x] * a[x][q];
                    dp[w * to + p] += dp[w * mask + y] * a[y][q];
                    dp[w * to + q] += dp[w * mask + x] * a[x][p];
                    dp[w * to + q] += dp[w * mask + y] * a[y][p];
```

```
                }
            }
        }
    }
    return setExp(cyc).back();
}
```

`hafnian(a)` 要求矩阵阶数为 $2n$ 且对称，主对角线不会参与答案；`T` 必须构成交换环，空矩阵的 Hafnian 按空乘积定义为 1。本条显式依赖第 10 章的 `setExp`，拼接或单独使用头文件时都要带上该公共头。按子集大小分层计数，时间为 $O(n^2 2^n)$，峰值空间为 $O(n2^n)$，因此代码把 `n` 限为 20。它计算的是一般图完美匹配的带权和；二分图完美匹配计数对应行列式或 permanent 的其他专用算法，不能混用。

下面用 4×4 矩阵直接核对三种配对之和 $1\cdot 6+2\cdot 5+3\cdot 4=28$：

```
signed main()
{
    vector<vector<long long>> a{{0, 1, 2, 3}, {1, 0, 4, 5}, {2, 4, 0, 6}, {3, 5, 6, 0}};
    assert(hafnian(vector<vector<long long>>{}) == 1);
    assert(hafnian(vector<vector<long long>>{{0, 7}, {7, 0}}) == 7);
    cout << hafnian(a) << '\n'; // 期望输出：28
    return 0;
}
```

9.1.013 Permanent / Ryser

n 阶方阵的积和式 $\operatorname{perm}(A)=\sum_{\sigma}\prod_{i}a_{i,\sigma(i)}$ 与行列式只差符号，却没有已知的多项式算法（#P 完全）。Ryser 公式用容斥写成

$$\operatorname{perm}(A)=\sum_{S\subseteq\{0,\dots,n-1\}}(-1)^{n-|S|}\prod_{i=0}^{n-1}\sum_{j\in S}a_{i,j},$$

右边展开后每一项是每行各选一列的乘积，只有所选列恰好覆盖全部列（即构成排列）的项系数为 1，其余项被容斥抵消。按 Gray 码枚举 S，相邻两个集合只差一列，每行的列和可以 $O(1)$ 更新，所以每个集合只需 $O(n)$。

```
int permanent(const vector<vector<int>> &a, int mod)
{
    // a 是 n<=24 的方阵，mod>0；返回积和式 perm(a) 模 mod 的非负值。
    // Ryser 公式：perm = sum_{S} (-1)^{n-|S|} prod_i sum_{j in S} a[i][j]，按格雷码每次改一列。
    int n = a.size();
    assert(mod > 0 && n <= 24); // 调试检查，可删。
    vector<int> row(n); // row[i] 是当前列集合 S 在第 i 行上的元素和（模 mod）。
    int ans = 0;
    for (int t = 1; t < (1LL << n); t++)
    {
        int s = t ^ (t >> 1), b = __builtin_ctzll(t), sg = s >> b & 1 ? 1 : -1;
        int prod = 1 % mod;
        for (int i = 0; i < n; i++)
        {
            row[i] = (((__int128)row[i] + (__int128)sg * a[i][b]) % mod + mod) % mod;
```

```
            prod = (__int128)prod * row[i] % mod;
        }
        if ((n - __builtin_popcountll(s)) & 1)
        {
            prod = (mod - prod) % mod;
        }
        ans = ((__int128)ans + prod) % mod;
    }
    return n ? ans : 1 % mod;
}
```

`permanent(a,mod)` 要求方阵、$n \le 24$、`mod>0`，元素可为任意整数，返回 $\operatorname{perm}(A) \bmod mod$ 的非负值；空矩阵的积和式为 1。时间 $O(2^n n)$，额外空间 $O(n)$；中间量用 `__int128`，`mod` 可以取到 `LLONG_MAX`。

0/1 矩阵的积和式等于二分图完美匹配数；一般图完美匹配计数用 9.1.012 Hafnian 与一般图完美匹配计数，行列式见 9.1.006 行列式（质数模数 / 任意模数）。

```
signed main()
{
    vector<vector<int>> a{{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    assert(permanent(a, 1000000007) == 450);
    assert(permanent(vector<vector<int>>(10, vector<int>(10, 1)), 1000000007) == 3628800); //
    ↪ 全 1 矩阵的积和式是 10!。
    assert(permanent({}, 7) == 1);
}
```

9.1.014 GF(2) 位集高斯消元

GF(2) 上的线性方程组 $Ax = b$ 中加法是异或；每行压成 64 位字的位集后，消去一行只需 $\lfloor \frac{n+64}{64} \rfloor$ 次字异或。按列找主元做 Gauss-Jordan 消元，得到简化阶梯形后：

- 某个全零系数行的右端为 1 时方程组无解；
- 自由元全取 0 时，主元变量等于所在行的右端，得到一组特解；
- 令某个自由元 f 为 1、其余自由元为 0，主元变量等于所在行第 f 位，得到齐次方程组的一个基向量。

解集是特解加上基向量张成的空间，共 $2^{n-\mathrm{rank}}$ 个解。

```
namespace gf2
{
using Row = vector<unsigned long long>; // Row 是压位的 0/1 行，第 i 位存在第 i/64 个字的第 i%64
↪ 位。

bool bit(const Row &r, int i)
{
    // 返回 r 的第 i 位。
    return r[i / 64] >> (i % 64) & 1;
}

void flip(Row &r, int i)
{
```

```
    // 翻转 r 的第 i 位。
    r[i / 64] ^= 1ULL << (i % 64);
}

struct Result
{
    bool consistent; // consistent 表示方程组有解。
    int rank, free;  // rank 是系数矩阵的秩，free = n - rank 是自由元个数。
    Row solution;    // solution 是一组特解（自由元取 0），无解时无意义。
    vector<Row> basis; // basis 是齐次方程组解空间的一组基，共 free 个。
};

Result solve(vector<Row> a, int n)
{
    // a 是 GF(2) 上的增广矩阵，每行第 0..n-1 位是系数，第 n 位是右端，行长须为 (n+64)/64 个字；
    // 返回可解性、秩、特解与齐次解基。复杂度 O(m n^2 / 64)。
    int rank = 0, m = a.size();
    vector<int> where(n, -1); // where[c] 是以第 c 列为主元的行号，自由元为 -1。
    for (int c = 0; c < n; c++)
    {
        int p = rank;
        while (p < m && !bit(a[p], c))
        {
            p++;
        }
        if (p == m)
        {
            continue;
        }
        swap(a[p], a[rank]);
        where[c] = rank;
        for (int i = 0; i < m; i++)
        {
            if (i != rank && bit(a[i], c))
            {
                for (int j = 0; j < (int)a[i].size(); j++)
                {
                    a[i][j] ^= a[rank][j];
                }
            }
        }
        rank++;
    }
    Result res{true, rank, n - rank, Row((n + 63) / 64), {}};
    for (int i = rank; i < m; i++)
    {
        if (bit(a[i], n))
        {
            res.consistent = false;
            return res;
        }
    }
```

```
    // 消成简化阶梯形后，主元变量 = 右端 xor (该行上自由元的取值)。
    for (int c = 0; c < n; c++)
    {
        if (where[c] >= 0 && bit(a[where[c]], n))
        {
            flip(res.solution, c);
        }
    }
    for (int f = 0; f < n; f++)
    {
        if (where[f] < 0)
        {
            // 令自由元 f 为 1、其余自由元为 0，得到一个齐次解。
            Row v((n + 63) / 64);
            flip(v, f);
            for (int c = 0; c < n; c++)
            {
                if (where[c] >= 0 && bit(a[where[c]], f))
                {
                    flip(v, c);
                }
            }
            res.basis.push_back(move(v));
        }
    }
    return res;
}
}
```

`gf2::solve(a,n)` 中 `a` 是 m 行增广矩阵，每行的第 $0\dots n-1$ 位是系数、第 n 位是右端，行长为 $\lfloor\frac{n+64}{64}\rfloor$ 个字；`bit`、`flip` 读写单个位。返回的 `Result` 含可解性、秩、自由元个数、特解与齐次解基，特解和基向量的行长为 $\lceil\frac{n}{64}\rceil$ 个字。时间 $O(\frac{mn^2}{64})$，空间 $O(\frac{mn}{64})$。

一般模数下的消元见 9.1.005 矩阵求逆（质数模数 / 任意模数） 与 9.1.006 行列式（质数模数 / 任意模数）。下面解 $x_0\oplus x_1=1$、$x_1\oplus x_2=0$：

```
signed main()
{
    int n = 3;
    vector<gf2::Row> a(2, gf2::Row((n + 64) / 64));
    a[0][0] = 0b1011; // x0^x1=1，第 3 位是右端。
    a[1][0] = 0b0110; // 方程 x1^x2=0。
    auto r = gf2::solve(a, n);
    assert(r.consistent && r.rank == 2 && r.free == 1);
    assert(r.solution[0] == 0b001 && r.basis[0][0] == 0b111);
}
```

9.2 线性方程与规划

9.2.001 高斯消元：唯一解、无解、多解

在素数模域 $\mathbb{F}_p$ 上解 $Ax \equiv b$，并完整区分无解、唯一解与多解。Gauss-Jordan 消元逐列找非零主元、用主元逆元把该列化成单位列（主元行系数变 1、同列其他行清零），处理完得到行最简形。判定规则：若某行左侧系数全零而右端非零，方程 $0 \equiv c \neq 0$ 矛盾即无解；否则每个主元列的变量由该行右端唯一定出（特解），每个非主元列是自由变量，对应一条齐次解基——把该自由变量置 1、其余自由变量置 0，主元变量取相应负系数，就得到 $Ax \equiv 0$ 的一个基向量。通解即“特解 + 齐次基的任意线性组合”。`basis` 为空即唯一解。

```cpp
// 依赖 7.1.001 快速幂
struct ModLinearSol
{
    // part 是一组特解，basis 是齐次解空间基；全部值在 [0,mod) 内。
    vector<int> part;
    vector<vector<int>> basis;
};

optional<ModLinearSol> gaussMod(vector<vector<int>> a, int n, int mod)
{
    // a 有 n+1 列且最后一列是右端，mod 是素数；返回特解与齐次基，无解时返回空。
    assert(n >= 0 && mod > 1); // 调试检查，可删。
    for (auto &r : a)
    {
        assert((int)r.size() == n + 1); // 调试检查，可删。
        for (auto &x : r)
        {
            x %= mod;
            if (x < 0)
            {
                x += mod;
            }
        }
    }
    int m = a.size();
    int rk = 0;
    vector<int> pivot(n, -1);
    auto power = [&](int x, int b)
    {
        // x 是非零剩余，b 是非负指数；返回 x^b mod mod。
        int ans = 1;
        while (b)
        {
            if (b & 1)
            {
                ans = mulMod(ans, x, mod);
            }
            x = mulMod(x, x, mod);
            b >>= 1;
```

```
        }
        return ans;
    };
    for (int col = 0; col < n && rk < m; col++)
    {
        int row = rk;
        while (row < m && a[row][col] == 0)
        {
            row++;
        }
        if (row == m)
        {
            continue;
        }
        swap(a[row], a[rk]);
        int inv = power(a[rk][col], mod - 2);
        for (int j = col; j <= n; j++)
        {
            a[rk][j] = mulMod(a[rk][j], inv, mod);
        }
        for (int i = 0; i < m; i++)
        {
            if (i == rk)
            {
                continue;
            }
            int q = a[i][col];
            for (int j = col; j <= n; j++)
            {
                a[i][j] -= mulMod(q, a[rk][j], mod);
                if (a[i][j] < 0)
                {
                    a[i][j] += mod;
                }
            }
        }
        pivot[col] = rk++;
    }
    for (int i = rk; i < m; i++)
    {
        if (a[i][n])
        {
            return nullopt;
        }
    }
    ModLinearSol sol{vector<int>(n), {}};
    for (int col = 0; col < n; col++)
    {
        if (pivot[col] != -1)
        {
            sol.part[col] = a[pivot[col]][n];
        }
```

```
    }
    for (int free = 0; free < n; free++)
    {
        if (pivot[free] != -1)
        {
            continue;
        }
        vector<int> v(n);
        v[free] = 1;
        for (int col = 0; col < n; col++)
        {
            if (pivot[col] != -1)
            {
                v[col] = (mod - a[pivot[col]][free]) % mod;
            }
        }
        sol.basis.push_back(move(v));
    }
    return sol;
}
```

调用 `gaussMod(a,n,mod)`，`a` 是 m 行 $n+1$ 列增广矩阵（最后一列是右端），`mod` 必须是素数；显式传 `n` 是为了让 `m=0` 时仍能表示变量数。返回空即无解，`sol.basis.empty()` 即唯一解，齐次基数量恒为 `n-rk`。零条方程会返回全零特解和 n 个单位基向量；零个变量时，每个合法方程只剩右端，全部为零才相容。连同输入归一化和齐次基输出，时间为 $O(m(n+1)+mn\min(m,n)+n^2)$，空间为 $O(m(n+1)+n^2)$。实数方程用 9.2.002 高斯消元：列主元 或 9.2.003 高斯消元最小范数解；异或方程参见 9.2.004 线性基（参见数据结构）；三对角结构用 9.2.007 追赶法解三对角/周期方程。

下面用五个固定系统覆盖唯一解、多解、无解、零方程和零变量，并直接核对齐次基维数：

```
signed main()
{
    auto one = gaussMod({{1, 1, 3}, {1, -1, 1}}, 2, 5);
    assert(one && one->part == vector<int>({2, 1}) && one->basis.empty());
    auto many = gaussMod({{1, 1, 1}}, 2, 5);
    assert(many && many->basis.size() == 1);
    assert(many->basis[0] == vector<int>({4, 1}));
    assert(!gaussMod({{1, 1}, {1, 2}}, 1, 5));
    auto none = gaussMod({}, 3, 5);
    assert(none && none->part == vector<int>(3, 0) && none->basis.size() == 3);
    assert(gaussMod({{0}}, 0, 5) && !gaussMod({{1}}, 0, 5));
    cout << one->part[0] << ' ' << one->part[1] << ' ' << none->basis.size() << '\n'; // 期望输
    ↪ 出: 2 1 3
    return 0;
}
```

异或方程组（$\mathbb{F}_2$）直接用 9.1.014 GF(2) 位集高斯消元 的 `gf2::solve`，它同时给出特解与齐次解基。三元域专用接口 `solveMod3(mat,vars)` 用两个机器位集记录系数 $1,2$，返回任意一组解，无解返回空。

三元域位并行实现如下：

```cpp
struct Mod3Row
{
    // n 是列数；one、two 分别记录系数为 1、2 的列，未记录列系数为 0。
    int n = 0;
    vector<unsigned long long> one, two;

    Mod3Row(int n = 0) : n(n), one((n + 63) / 64), two((n + 63) / 64) {}

    int get(int i) const
    {
        // i 是列下标；返回该列在 F3 中的系数。
        return one[i >> 6] >> (i & 63) & 1ULL ? 1 :
               (two[i >> 6] >> (i & 63) & 1ULL ? 2 : 0);
    }

    void set(int i, int x)
    {
        // i 是列下标，x 是 0、1 或 2；设置该列系数。
        unsigned long long b = 1ULL << (i & 63);
        one[i >> 6] &= ~b, two[i >> 6] &= ~b;
        if (x == 1) one[i >> 6] |= b;
        if (x == 2) two[i >> 6] |= b;
    }

    void add(const Mod3Row &b)
    {
        // b 是同宽行；把 b 加到当前行，系数按模 3 运算。
        assert(n == b.n);
        for (int i = 0; i < (int)one.size(); i++)
        {
            auto az = ~(one[i] | two[i]), bz = ~(b.one[i] | b.two[i]);
            auto o = (one[i] & bz) | (two[i] & b.two[i]) | (az & b.one[i]);
            auto t = (one[i] & b.one[i]) | (two[i] & bz) | (az & b.two[i]);
            one[i] = o, two[i] = t;
        }
    }

    void sub(const Mod3Row &b)
    {
        // b 是同宽行；从当前行减去 b，系数按模 3 运算。
        assert(n == b.n);
        for (int i = 0; i < (int)one.size(); i++)
        {
            auto az = ~(one[i] | two[i]), bz = ~(b.one[i] | b.two[i]);
            auto o = (one[i] & bz) | (two[i] & b.one[i]) | (az & b.two[i]);
            auto t = (one[i] & b.two[i]) | (two[i] & bz) | (az & b.one[i]);
            one[i] = o, two[i] = t;
        }
    }
};

inline optional<vector<int>> solveMod3(const vector<vector<int>> &mat, int vars)
```

```
{
    // mat 每行含 vars 个系数和一个右端，vars 是变量数；返回任意一组 F3 解，无解时返回空。
    assert(vars >= 0); // 调试检查，可删
    int rows = mat.size(), rk = 0;
    vector<Mod3Row> a(rows, Mod3Row(vars + 1));
    for (int i = 0; i < rows; i++)
    {
        assert((int)mat[i].size() == vars + 1); // 调试检查，可删
        for (int j = 0; j <= vars; j++)
        {
            int x = mat[i][j] % 3;
            if (x < 0) x += 3;
            a[i].set(j, x);
        }
    }
    vector<int> pivot(vars, -1);
    for (int col = 0; col < vars && rk < rows; col++)
    {
        int p = rk;
        while (p < rows && !a[p].get(col))
        {
            p++;
        }
        if (p == rows) continue;
        swap(a[p], a[rk]);
        int lead = a[rk].get(col);
        for (int i = 0; i < rows; i++)
        {
            if (i == rk || !a[i].get(col)) continue;
            if (a[i].get(col) == lead) a[i].sub(a[rk]);
            else a[i].add(a[rk]);
        }
        pivot[col] = rk++;
    }
    for (int i = rk; i < rows; i++)
    {
        if (a[i].get(vars)) return nullopt;
    }
    vector<int> ans(vars);
    for (int col = 0; col < vars; col++)
    {
        if (pivot[col] != -1)
ans[col] = a[pivot[col]].get(col) * a[pivot[col]].get(vars) % 3;
    }
    return ans;
}
```

`solveMod3(mat,vars)` 中每行前 `vars` 个数是系数、最后一个是右端，任意整数先归一到 $\{0,1,2\}$；自由变量取 0，返回任意一组 $\mathbb{F}_3$ 解，无解返回空。每个系数用两个位集按位存储，消去一行只需常数次字运算，时间为 $O(\frac{nm\cdot\min(n,m)}{w})$（$n$ 为方程数、m 为变量数、w 为机器字位数），空间 $O(\frac{nm}{w})$。

9.2.002 高斯消元：列主元

浮点消元里若拿一个绝对值极小的数当主元（除数），舍入误差会被放大很多倍，最终解可能面目全非。列主元（部分选主元）法在当前列的候选行中挑绝对值最大者交换到主元位，让除数尽量大、误差最小。这里用它求浮点线性方程的唯一解：消元后统计秩，秩等于变量数才有唯一解并回代；否则（出现矛盾行或秩不足）返回空，把多解、最小范数解交给专门接口处理。判零阈值取公共前导里的 `eps`。

```
optional<vector<long double>> gaussPivot(vector<vector<long double>> a, int n)
{
    // a 有 n+1 列且最后一列是右端；唯一解时返回解，否则返回空。
    assert(n >= 0); // 调试检查，可删。
    // 调试检查，可删。
    for (const auto &r : a)
    {
        assert((int)r.size() == n + 1);
    }
    int m = a.size();
    int rk = 0;
    vector<int> pivot(n, -1);
    for (int col = 0; col < n && rk < m; col++)
    {
        int row = rk;
        for (int i = rk + 1; i < m; i++)
        {
            if (fabsl(a[i][col]) > fabsl(a[row][col]))
            {
                row = i;
            }
        }
        if (fabsl(a[row][col]) <= eps)
        {
            continue;
        }
        swap(a[row], a[rk]);
        for (int j = n; j >= col; j--)
        {
            a[rk][j] /= a[rk][col];
        }
        for (int i = 0; i < m; i++)
        {
            if (i == rk)
            {
                continue;
            }
            long double q = a[i][col];
            for (int j = col; j <= n; j++)
            {
                a[i][j] -= q * a[rk][j];
            }
        }
        pivot[col] = rk++;
```

```
    }
    for (int i = rk; i < m; i++)
    {
        if (fabsl(a[i][n]) > eps)
        {
            return nullopt;
        }
    }
    if (rk < n)
    {
        return nullopt;
    }
    vector<long double> x(n);
    for (int col = 0; col < n; col++)
    {
        x[col] = a[pivot[col]][n];
    }
    return x;
}
```

调用 `gaussPivot(a,n)`，`a` 是 m 行 $n+1$ 列的增广矩阵；显式变量数使空方程组也有确定含义：$m=n=0$ 有唯一空解，$m=0<n$ 时是多解。函数只用于求浮点唯一解，返回空可能是无解也可能是多解。连同输入检查，时间为 $O(m(n+1)+mn\min(m,n))$，空间为 $O(m(n+1))$。素数模方程用 9.2.001 高斯消元：唯一解、无解、多解；相容多解的最小范数解用 9.2.003 高斯消元最小范数解；三对角结构用 9.2.007 追赶法解三对角/周期方程。

下面固定检查唯一解、需要换行的主元、多解、不相容与空系统：

```
signed main()
{
    auto x = gaussPivot({{0, 1, 2}, {1, 1, 3}}, 2);
    assert(x && fabsl((*x)[0] - 1) < 1E-10L && fabsl((*x)[1] - 2) < 1E-10L);
    assert(!gaussPivot({{1, 1, 2}}, 2));
    assert(!gaussPivot({{1, 1}, {1, 2}}, 1));
    assert(gaussPivot({}, 0) && !gaussPivot({}, 2));
    cout << llround((*x)[0]) << ' ' << llround((*x)[1]) << '\n'; // 期望输出：1 2
    return 0;
}
```

9.2.003 高斯消元最小范数解

当实数约束 $Ax=b$ 相容但欠定（解不唯一）时，把自由变量随手置零得到的解依赖消元顺序、没有几何意义。这里求的是欧氏范数 $\|x\|_2$ 最小的那个唯一解。它是带约束极小化 $\min\|x\|^2$ s.t. $Ax=b$，拉格朗日 KKT 条件给出 $x+A^T\lambda=0$、$Ax=b$，把 x 与乘子 λ 拼成未知量就是一个对称增广线性系统 $\begin{pmatrix} I & A^T \\ A & 0 \end{pmatrix}\begin{pmatrix} x \\ \lambda \end{pmatrix}=\begin{pmatrix} 0 \\ b \end{pmatrix}$。用列主元消元解这个 $(n+m)$ 维系统，约束相关只让 λ 不唯一，x 部分仍唯一确定，取出前 n 维即最小范数解。

```cpp
optional<vector<long double>> minNormSolve(const vector<vector<long double>> &a, const vector<long double> &b, int n)
{
    // a 是 m*n 系数矩阵，b 是 m 维右端；返回 Ax=b 的最小二范数解，不相容时返回空。
    int m = a.size();
    int s = n + m;
    assert(n >= 0); // 调试检查，可删。
    assert((int)b.size() == m); // 调试检查，可删。
    // 调试检查，可删。
    for (const auto &r : a)
    {
        assert((int)r.size() == n);
    }
    vector mat(s, vector<long double>(s + 1));
    for (int j = 0; j < n; j++)
    {
        mat[j][j] = 1;
        for (int i = 0; i < m; i++)
        {
            mat[j][n + i] = a[i][j];
        }
    }
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            mat[n + i][j] = a[i][j];
        }
        mat[n + i][s] = b[i];
    }
    int rk = 0;
    vector<int> pivot(s, -1);
    for (int col = 0; col < s && rk < s; col++)
    {
        int row = rk;
        for (int i = rk + 1; i < s; i++)
        {
            if (fabsl(mat[i][col]) > fabsl(mat[row][col]))
            {
                row = i;
            }
        }
        if (fabsl(mat[row][col]) <= eps)
        {
            continue;
        }
        swap(mat[row], mat[rk]);
        for (int j = s; j >= col; j--)
        {
            mat[rk][j] /= mat[rk][col];
        }
        for (int i = 0; i < s; i++)
```

```
        {
            if (i == rk)
            {
                continue;
            }
            long double q = mat[i][col];
            for (int j = col; j <= s; j++)
            {
                mat[i][j] -= q * mat[rk][j];
            }
        }
        pivot[col] = rk++;
    }
    for (int i = rk; i < s; i++)
    {
        if (fabsl(mat[i][s]) > 1E-10L)
        {
            return nullopt;
        }
    }
    vector<long double> z(s);
    for (int col = 0; col < s; col++)
    {
        if (pivot[col] != -1)
        {
            z[col] = mat[pivot[col]][s];
        }
    }
    z.resize(n);
    return z;
}
```

调用 `minNormSolve(a,b,n)`，`a` 各行长度为 `n`、`b` 长度等于行数，返回空表示约束不相容。显式传 `n` 后，零条约束会正确返回 n 维零向量。单个方程 $x+y=1$ 的最小范数解是 $(0.5,0.5)$，重复约束不改变结果。复杂度为 $O((m+n)^3)$，空间为 $O((m+n)^2)$。这里解的是 KKT 增广系统，不是正规方程；它不会把条件数直接平方，但矩阵不定且在冗余约束下奇异，病态问题仍应换 QR 或 SVD。唯一解用 9.2.002 高斯消元：列主元 更省；素数模多解结构用 9.2.001 高斯消元：唯一解、无解、多解 的齐次基表示。

下面检查单约束、重复约束、不相容约束和零约束：

```
signed main()
{
    auto x = minNormSolve({{1, 1}}, {1}, 2);
    assert(x && fabsl((*x)[0] - 0.5L) < 1E-10L && fabsl((*x)[1] - 0.5L) < 1E-10L);
    auto y = minNormSolve({{1, 1}, {1, 1}}, {1, 1}, 2);
    assert(y && fabsl((*y)[0] - 0.5L) < 1E-10L && fabsl((*y)[1] - 0.5L) < 1E-10L);
    assert(!minNormSolve({{1, 1}, {1, 1}}, {1, 2}, 2));
    auto z = minNormSolve({}, {}, 2);
    assert(z && *z == vector<long double>({0, 0}));
    cout << fixed << setprecision(1) << (*x)[0] << ' ' << (*x)[1] << '\n'; // 期望输出：0.5 0.5
    return 0;
```

```
}
```

9.2.004 线性基（参见数据结构）

整数按二进制看成 $\mathbb{F}_2$（二元域）上的向量，异或就是向量加法，一组数的所有异或子集和正是这些向量张成的线性子空间。线性基按最高位存一组极大无关向量：插入 x 时从高位向低位消元，若最终在空档留下非零主元则秩加一，否则它可由现有基表示。`has` 判可表示性，`maxXor` 贪心得最大异或值，`rk` 是秩，`kth` 在化简后的基上取第 k 小可表示值。本条只给出线性代数视角的索引，代表实现与增强接口统一放在数据结构章，避免同一代码维护两份。

本条不重复打印代码；请直接使用 3.8.001 线性基 的代表实现。模板参数 `W` 是位宽，`add(x)` 返回是否使秩增加，`has(x)`、`maxXor(x)`、`kth(k)`、`merge(o)` 的语义和边界检查均以该条为准。`add`、`has`、`maxXor` 均 $O(W)$，`kth` 需一次 $O(W^2)$ 化简，空间 $O(W)$。解异或方程组也使用同一按位消元思路；系数不在 $\mathbb{F}_2$ 而在素数模域时改用 9.2.001 高斯消元：唯一解、无解、多解。

9.2.005 Simplex 单纯形

线性规划求 $\max c^T x$ 使 $Ax \le b$、$x \ge 0$。可行域是若干半空间的交；若有限最优值存在，就能在某个顶点取到。单纯形维护一个基可行解，每轮选一个可改进目标的非基变量入基，再按最小比值法则选基变量出基。退化换基时目标值可以不变，直到没有可改进方向才得到最优解。初始基未必可行，故先引入一个人工变量，第一阶段把它压到 0 以找到可行基，再在第二阶段优化原目标。三种结局必须分清——不可行（第一阶段无法把人工变量压到 0）、无界（存在改进方向却没有正系数约束它）、最优。本实现按 Bland 规则选编号最小的可改进入基变量，最小比值相等时选编号最小的离基变量；精确算术下这能避免退化循环，浮点版仍由成员容差 `e` 决定何时视为相等。

```
struct Simplex
{
    // m 是约束数，n 是变量数。
    int m, n;
    // b、z 分别记录各行的基变量和各列的非基变量编号。
    vector<int> b, z;
    // a 是单纯形表。
    vector<vector<long double>> a;
    // e 是浮点比较容差，linf 标记无界或不可行。
    static constexpr long double e = 1E-12L;
    static constexpr long double linf = numeric_limits<long double>::infinity();

    Simplex(const vector<vector<long double>> &A, const vector<long double> &B, const
    ↪ vector<long double> &C)
        : m(B.size()), n(C.size()), b(m), z(n + 1), a(m + 2, vector<long double>(n + 2))
    {
        // A、B 给出 Ax<=B，C 是最大化目标系数；隐含 x>=0。
        assert((int)A.size() == m); // 调试检查，可删。
        for (int i = 0; i < m; i++)
        {
            assert((int)A[i].size() == n); // 调试检查，可删。
            for (int j = 0; j < n; j++)
```

```
            {
                a[i][j] = A[i][j];
            }
        }
        for (int i = 0; i < m; i++)
        {
            b[i] = n + i;
            a[i][n] = -1;
            a[i][n + 1] = B[i];
        }
        for (int j = 0; j < n; j++)
        {
            z[j] = j;
            a[m][j] = -C[j];
        }
        z[n] = -1;
        a[m + 1][n] = 1;
    }

private:
    void pivot(int r, int s)
    {
        // r 是离基变量所在行，s 是入基变量所在列；用 a[r][s] 换基并更新整张表。
        long double v = 1 / a[r][s];
        for (int i = 0; i < m + 2; i++)
        {
            if (i != r)
            {
                for (int j = 0; j < n + 2; j++)
                {
                    if (j != s)
                    {
                        a[i][j] -= a[r][j] * a[i][s] * v;
                    }
                }
            }
        }
        for (int j = 0; j < n + 2; j++)
        {
            if (j != s)
            {
                a[r][j] *= v;
            }
        }
        for (int i = 0; i < m + 2; i++)
        {
            if (i != r)
            {
                a[i][s] *= -v;
            }
        }
        a[r][s] = v;
```

```
        swap(b[r], z[s]);
    }

    bool phase(int p)
    {
        // p=1 时找可行基，p=2 时优化原目标；返回 false 表示存在无界改进方向。
        int x = p == 1 ? m + 1 : m;
        while (true)
        {
            int s = -1;
            for (int j = 0; j <= n; j++)
            {
                if (z[j] == -1 || a[x][j] >= -e)
                {
                    continue;
                }
                if (s < 0 || z[j] < z[s])
                {
                    s = j;
                }
            }
            if (s < 0)
            {
                return true;
            }
            int r = -1;
            for (int i = 0; i < m; i++)
            {
                if (a[i][s] <= e)
                {
                    continue;
                }
                if (r < 0 || a[i][n + 1] / a[i][s] < a[r][n + 1] / a[r][s] - e ||
                    (fabsl(a[i][n + 1] / a[i][s] - a[r][n + 1] / a[r][s]) <= e && b[i] < b[r]))
                {
                    r = i;
                }
            }
            if (r < 0)
            {
                return false;
            }
            pivot(r, s);
        }
    }

public:
    pair<long double, vector<long double>> solve()
    {
        // 返回最优值与变量；不可行返回 -linf，无界返回 linf，二者的解向量均为空。
        if (m == 0)
        {
```

```
            for (int j = 0; j < n; j++)
            {
                if (a[m][j] < -e)
                {
                    return {linf, {}};
                }
            }
            return {0, vector<long double>(n)};
        }
        int r = min_element(a.begin(),
                            a.begin() + m,
                            [&](auto &x, auto &y)
                            {
                                return x[n + 1] < y[n + 1];
                            }) -
                a.begin();
        if (a[r][n + 1] < -e)
        {
            pivot(r, n);
            if (!phase(1) || a[m + 1][n + 1] < -e)
            {
                return {-linf, {}};
            }
            if (fabsl(a[m + 1][n + 1]) > e)
            {
                return {-linf, {}};
            }
            auto it = find(b.begin(), b.end(), -1);
            if (it != b.end())
            {
                r = it - b.begin();
                int s = 0;
                for (int j = 1; j <= n; j++)
                {
                    if (fabsl(a[r][j]) > fabsl(a[r][s]))
                    {
                        s = j;
                    }
                }
                if (fabsl(a[r][s]) > e)
                {
                    pivot(r, s);
                }
            }
        }
        if (!phase(2))
        {
            return {linf, {}};
        }
        vector<long double> x(n);
        for (int i = 0; i < m; i++)
        {
```

```
            if (b[i] < n)
            {
                x[b[i]] = a[i][n + 1];
            }
        }
        return {a[m][n + 1], x};
    }
};
```

以 `Simplex(A,B,C)` 构造（A 是 $m \times n$ 约束系数、B 是 m 维右端、C 是 n 维目标系数，隐含 $x \ge 0$），`solve()` 返回最优值与解向量：不可行时值为 $-\infty$、无界时为 $+\infty$，均配空向量。每次换基为 $O(mn)$，最坏换基次数是指数级，空间为 $O(mn)$。等式约束要拆成两个不等式；纯粹解方程用 9.2.001 高斯消元：唯一解、无解、多解 或 9.2.002 高斯消元：列主元；分式目标的恰选问题用 9.2.006 0-1 分数规划。

下面固定检查普通最优、第一阶段、不可行、无界，再用经典退化循环例检查 Bland 选元：

```
signed main()
{
    auto [val, x] = Simplex({{1, 1}, {1, 0}, {0, 1}}, {4, 2, 3}, {3, 2}).solve();
    assert(fabsl(val - 10) < 1E-10L && x.size() == 2);
    auto [phaseOne, y] = Simplex({{-1}}, {-1}, {-1}).solve();
    assert(fabsl(phaseOne + 1) < 1E-10L && y.size() == 1);
    auto [infeasible, p] = Simplex({{1}, {-1}}, {0, -1}, {1}).solve();
    assert(isinf(infeasible) && infeasible < 0 && p.empty());
    auto [unbounded, q] = Simplex({}, {}, {1}).solve();
    assert(isinf(unbounded) && unbounded > 0 && q.empty());
    auto [degenerate, z] = Simplex({{0.5L, -5.5L, -2.5L, 9},
                                    {0.5L, -1.5L, -0.5L, 1},
                                    {1, 0, 0, 0}},
                                   {0, 0, 1}, {10, -57, -9, -24})
                               .solve();
    assert(fabsl(degenerate - 1) < 1E-10L && z.size() == 4);
    cout << llround(val) << ' ' << llround(phaseOne) << '\n'; // 期望输出：10 -1
    return 0;
}
```

9.2.006 0-1 分数规划

要在若干物品里恰选 k 个，最大化比值 $\dfrac{\sum a_i}{\sum b_i}$（$b_i > 0$）。分式不能直接贪心，但可以参数化：比值 $\ge r$ 当且仅当 $\sum a_i - r\sum b_i = \sum(a_i - rb_i) \ge 0$。固定 r 后 $a_i - rb_i$ 就是每个物品的独立权重，取最大的 k 个之和判它是否非负即可——这个判定关于 r 单调，于是对 r 二分。所有分母 b_i 必须为正，否则单调性不成立。（另一路线是 Dinkelbach 迭代，收敛更快但实现更绕，这里用固定轮数二分足够。）

```
namespace frac_prog
{
long double maxRatio(const vector<long double> &a, const vector<long double> &b, int k)
{
    // a、b 是分子分母贡献且 b[i] 为正，k 是恰选数量；返回最大比值。
    int n = a.size();
```

```
    assert((int)b.size() == n && 1 <= k && k <= n); // 调试检查，可删。
    long double l = numeric_limits<long double>::infinity();
    long double r = -l;
    for (int i = 0; i < n; i++)
    {
        assert(b[i] > 0); // 调试检查，可删。
        l = min(l, a[i] / b[i]);
        r = max(r, a[i] / b[i]);
    }
    vector<long double> c(n);
    for (int it = 0; it < 100; it++)
    {
        long double mid = (l + r) / 2;
        for (int i = 0; i < n; i++)
        {
            c[i] = a[i] - mid * b[i];
        }
        nth_element(c.begin(), c.end() - k, c.end());
        long double s = accumulate(c.end() - k, c.end(), 0.0L);
        if (s >= 0)
        {
            l = mid;
        }
        else
        {
            r = mid;
        }
    }
    return (l + r) / 2;
}
}
```

调用 `frac_prog::maxRatio(a,b,k)`，`a`、`b` 是各物品分子分母贡献（`b[i]>0`）、`k` 是恰选数量，返回最大比值。上下界取所有单项比值的最小值与最大值，因此最优值为负时也正确。固定 100 轮二分，每轮用 `nth_element` 在线性期望时间取前 k 大，总时间为 $O(n)$ 期望、空间为 $O(n)$。复杂可行性约束可把内层判定换成对应最优化；连续线性规划用 9.2.005 Simplex 单纯形。

下面用负最优值和普通正值各检查一次：

```
signed main()
{
    long double x = frac_prog::maxRatio({-5, -2, -9}, {1, 1, 1}, 2);
    long double y = frac_prog::maxRatio({1, 8}, {1, 2}, 1);
    assert(fabsl(x + 3.5L) < 1E-10L && fabsl(y - 4) < 1E-10L);
    cout << fixed << setprecision(1) << x << ' ' << y << '\n'; // 期望输出：-3.5 4.0
    return 0;
}
```

9.2.007 追赶法解三对角/周期方程

三对角矩阵每行只有主对角 b_i 与相邻的下对角 a_i、上对角 c_i 三个非零元。对它做一般高斯消元是 $O(n^3)$ 的浪费，因为消去下对角后不会产生新的填充。追赶法（Thomas 算法）就是这个专门化：正向“追”一遍，用 a_i/b_{i-1} 把每行下对角消掉、同步更新主对角与右端；再反向“赶”一遍回代求解，总共 $O(n)$。周期三对角在左下角 $A_{n-1,0}$ 与右上角 $A_{0,n-1}$ 多两个耦合元素，破坏了带状结构；这里用 Sherman-Morrison 把周期矩阵写成“三对角 + 秩一修正”，解两次普通三对角方程（一次对原右端、一次对修正向量）再线性组合得到答案。所有主元 b_{i-1} 消元时必须非零可除（对角占优时天然成立）。

```cpp
template <class T> optional<vector<T>> tridiagonal(vector<T> a, vector<T> b, vector<T> c,
    vector<T> d)
{
    // a、b、c 是下、主、上对角线，d 是右端；零主元时返回空。
    int n = b.size();
    assert((int)a.size() == n && (int)c.size() == n && (int)d.size() == n); // 调试检查，可删。
    if (n == 0)
    {
        return vector<T>{};
    }
    for (int i = 1; i < n; i++)
    {
        if (b[i - 1] == T(0))
        {
            return nullopt;
        }
        T q = a[i] / b[i - 1];
        b[i] -= q * c[i - 1];
        d[i] -= q * d[i - 1];
    }
    if (b[n - 1] == T(0))
    {
        return nullopt;
    }
    vector<T> x(n);
    x[n - 1] = d[n - 1] / b[n - 1];
    for (int i = n - 2; i >= 0; i--)
    {
        if (b[i] == T(0))
        {
            return nullopt;
        }
        x[i] = (d[i] - c[i] * x[i + 1]) / b[i];
    }
    return x;
}

template <class T>
optional<vector<T>> cyclicTri(vector<T> a, vector<T> b, vector<T> c, T tr, T bl, const
    vector<T> &d)
{
    // tr、bl 是右上角与左下角系数；追赶法或秩一修正失败时返回空。
```

```
    int n = b.size();
    assert(n >= 3); // 调试检查，可删。
    assert((int)a.size() == n && (int)c.size() == n && (int)d.size() == n); // 调试检查，可删。
    T g = b[0] == T(0) ? T(1) : -b[0];
    b[0] -= g;
    b[n - 1] -= tr * bl / g;
    vector<T> u(n);
    u[0] = g;
    u[n - 1] = bl;
    auto x = tridiagonal(a, b, c, d);
    auto z = tridiagonal(a, b, c, u);
    if (!x || !z)
    {
        return nullopt;
    }
    T den = T(1) + (*z)[0] + tr * (*z)[n - 1] / g;
    if (den == T(0))
    {
        return nullopt;
    }
    T q = ((*x)[0] + tr * (*x)[n - 1] / g) / den;
    for (int i = 0; i < n; i++)
    {
        (*x)[i] -= q * (*z)[i];
    }
    return x;
}
```

普通三对角方程调用 `tridiagonal(a,b,c,d)`，约定 `a[0]`、`c[n-1]` 不使用；空系统返回一个存在但长度为零的解。周期方程调用 `cyclicTri(a,b,c,tr,bl,d)`，要求 $n \geq 3$。两者返回 `nullopt` 表示无主元交换的追赶过程失败，并不等价于原矩阵一定奇异。`T` 必须提供域上的除法；若 `T` 是整数，则每一步除法都必须整除，否则应改用有理数或模整数。两者均为 $O(n)$ 时间、$O(n)$ 空间。一般矩阵用 9.2.002 高斯消元：列主元；模意义下用 9.2.001 高斯消元：唯一解、无解、多解。

下面检查空系统、普通三对角、周期三对角与非零行列式但首主元为零的失败分支：

```
signed main()
{
    auto empty = tridiagonal<long double>({}, {}, {}, {});
    assert(empty && empty->empty());
    auto x = tridiagonal<long double>({0, -1, -1}, {2, 2, 2}, {-1, -1, 0}, {1, 0, 1});
    assert(x);
    for (auto v : *x)
    {
        assert(fabsl(v - 1) < 1E-10L);
    }
    auto y = cyclicTri<long double>({0, -1, -1, -1}, {4, 4, 4, 4},
                                    {-1, -1, -1, 0}, -1, -1, {2, 2, 2, 2});
    assert(y);
    for (auto v : *y)
    {
```

```
        assert(fabsl(v - 1) < 1E-10L);
    }
    assert(!tridiagonal<long double>({0, 1}, {0, 1}, {1, 0}, {1, 2}));
}
```

9.2.008 Pfaffian

偶数阶斜对称矩阵 $A^T = -A$ 的 Pfaffian 满足 $\mathrm{pf}(A)^2 = \det(A)$。对前两个下标选非零配对主元后，其余部分做一次斜对称 Schur 补，Pfaffian 就乘上该主元；若交换一个下标到主元位置，必须同步交换对应行列并让答案变号。重复成对消元即可在 $O(n^3)$ 内求值，不需要先算行列式再开平方。

```
template <class T> T pfaffian(vector<vector<T>> a)
{
    // a 是偶数阶斜对称矩阵，T 是域；返回 Pfaffian。
    int n = a.size();
    assert(n % 2 == 0); // 调试检查，可删。
    // 调试检查，可删。
    for (int i = 0; i < n; i++)
    {
        assert((int)a[i].size() == n && a[i][i] == T(0));
        for (int j = 0; j < n; j++)
        {
            assert(a[i][j] == -a[j][i]);
        }
    }
    T ans = 1;
    for (int k = 0; k < n; k += 2)
    {
        int p = k + 1;
        while (p < n && a[k][p] == T(0))
        {
            p++;
        }
        if (p == n)
        {
            return T(0);
        }
        if (p != k + 1)
        {
            swap(a[p], a[k + 1]);
            for (int i = 0; i < n; i++)
            {
                swap(a[i][p], a[i][k + 1]);
            }
            ans = -ans;
        }
        T v = a[k][k + 1];
        ans *= v;
        for (int i = k + 2; i < n; i++)
        {
```

```
            for (int j = i + 1; j < n; j++)
            {
                a[i][j] -= (a[k][i] * a[k + 1][j] - a[k][j] * a[k + 1][i]) / v;
                a[j][i] = -a[i][j];
            }
        }
    }
    return ans;
}
```

`pfaffian(a)` 要求偶数阶斜对称矩阵，`T` 必须是可除的域；模意义下通常使用奇素数模的模整数。空矩阵返回空乘积 1。时间为 $O(n^3)$，按值传入的矩阵副本占 $O(n^2)$ 空间，除此之外只需 $O(1)$ 工作空间。Pfaffian 是带符号的配对和；不带符号的一般图完美匹配计数应使用 9.1.012 Hafnian 与一般图完美匹配计数，二者不能互换。

下面用 4×4 闭式 $a_{01}a_{23}-a_{02}a_{13}+a_{03}a_{12}$ 检查普通主元与交换主元：

```
signed main()
{
    vector<vector<long double>> a{{0, 1, 2, 3}, {-1, 0, 4, 5}, {-2, -4, 0, 6}, {-3, -5, -6, 0}};
    vector<vector<long double>> b{{0, 0, 2, 3}, {0, 0, 4, 5}, {-2, -4, 0, 6}, {-3, -5, -6, 0}};
    assert(fabsl(pfaffian(vector<vector<long double>>{}) - 1) < 1E-10L);
    assert(fabsl(pfaffian(a) - 8) < 1E-10L);
    cout << llround(pfaffian(b)) << '\n'; // 期望输出：2
    return 0;
}
```

9.2.009 对称矩阵 LDL^T 分解

对称矩阵在不交换主元且各步枢轴非零时可分解为 $A=LDL^T$，其中 L 是单位下三角矩阵、$D=\operatorname{diag}(d_0,\dots,d_{n-1})$。比较等式两侧第 (j,j) 与 (i,j) 项可依次得到

$$d_j = a_{jj} - \sum_{k<j} l_{jk}^2 d_k,$$
$$l_{ij} = \frac{a_{ij} - \sum_{k<j} l_{ik} d_k l_{jk}}{d_j} \qquad (i>j),$$

因此按列递推即可原地确定全部因子。解 $Ax=b$ 时依次做单位下三角前代 $Ly=b$、对角求解 $Dz=y$ 和单位上三角回代 $L^Tx=z$。与 Cholesky 不同，这个分解允许 D 中出现负数，因而能处理一部分不定矩阵；但当前版本不做对称主元交换，某个 d_j 为零时即使 A 可逆也会失败。

```
struct LDLT
{
    // l 是单位下三角因子，满足 A=l*diag(d)*l^T；只使用其下三角部分。
    vector<vector<long double>> l;
    // d 是对角因子；每个元素都必须远离零枢轴。
    vector<long double> d;
    // eps 是判断对称误差和零枢轴的绝对容差。
```

```
    long double eps;
    // valid 表示构造时是否成功完成无主元交换的分解。
    bool valid;

    explicit LDLT(const vector<vector<long double>> &a, long double tol = 1E-12L)
        : l(a.size(), vector<long double>(a.size())), d(a.size()), eps(tol), valid(true)
    {
        // a 是方阵且应对称，tol 是非负的对称性与枢轴判定容差；保存 A 的 LDL^T 分解。
        assert(tol >= 0); // 调试检查，可删。
        int n = (int)a.size();
        for (int i = 0; i < n; i++)
        {
            assert((int)a[i].size() == n); // 调试检查，可删。
        }
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                assert(fabsl(a[i][j] - a[j][i]) <= eps); // 调试检查，可删。
            }
            l[i][i] = 1;
        }
        for (int j = 0; j < n && valid; j++)
        {
            long double diag = a[j][j];
            for (int k = 0; k < j; k++)
            {
                diag -= l[j][k] * l[j][k] * d[k];
            }
            if (fabsl(diag) <= eps)
            {
                valid = false;
                break;
            }
            d[j] = diag;
            for (int i = j + 1; i < n; i++)
            {
                long double value = a[i][j];
                for (int k = 0; k < j; k++)
                {
                    value -= l[i][k] * d[k] * l[j][k];
                }
                l[i][j] = value / d[j];
            }
        }
    }

    optional<vector<long double>> solve(const vector<long double> &b) const
    {
        // b 是长度等于矩阵阶数的右端向量；返回 Ax=b 的解，无效分解时返回空。
        assert((int)b.size() == (int)d.size()); // 调试检查，可删。
        if (!valid)
```

```
        {
            return nullopt;
        }
        int n = (int)d.size();
        vector<long double> y = b;
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < i; j++)
            {
                y[i] -= l[i][j] * y[j];
            }
        }
        vector<long double> z(n);
        for (int i = 0; i < n; i++)
        {
            z[i] = y[i] / d[i];
        }
        vector<long double> x = z;
        for (int i = n - 1; i >= 0; i--)
        {
            for (int j = i + 1; j < n; j++)
            {
                x[i] -= l[j][i] * x[j];
            }
        }
        return x;
    }
};
```

构造 `LDLT fact(a,e)` 后先检查 `fact.valid`，再可对多个右端重复调用 `fact.solve(b)`；输入必须是 $n\times n$ 对称矩阵，`e` 是同时用于对称性断言与零枢轴判断的绝对容差，调用者应按数据尺度选取。空矩阵有唯一空分解和空解；失败只说明当前顺序下无主元分解不可用，不等价于矩阵奇异，此时改用 9.2.002 高斯消元：列主元，病态对称系统则应使用带对称主元交换的 LDL^T 或 QR。分解耗时 $O(n^3)$、空间 $O(n^2)$，每个右端的前代与回代耗时 $O(n^2)$、额外空间 $O(n)$。

下面用一个 D 含负数的不定矩阵核对解 $(1,-2,3)$，并检查空矩阵和“矩阵可逆但首枢轴为零”的失败分支：

```
signed main()
{
    LDLT fact({{4, 8, -4}, {8, 21, 7}, {-4, 7, 47}});
    auto x = fact.solve({-24, -13, 123});
    assert(fact.valid && x);
    assert(fabsl((*x)[0] - 1) < 1E-10L);
    assert(fabsl((*x)[1] + 2) < 1E-10L);
    assert(fabsl((*x)[2] - 3) < 1E-10L);
    auto empty = LDLT({}).solve({});
    assert(empty && empty->empty());
    LDLT zeroPivot({{0, 1}, {1, 0}});
    assert(!zeroPivot.valid && !zeroPivot.solve({1, 1}));
```

```
    cout << llround((*x)[0]) << ' ' << llround((*x)[1]) << ' ' << llround((*x)[2]) << '\n'; // 期望输出：1 -2 3
    return 0;
}
```

9.3 数值计算

9.3.001 自适应 Simpson

Simpson 公式用一条抛物线拟合三点近似定积分：$\int_l^r f \approx \frac{r-l}{6}\left(f(l)+4f(\frac{l+r}{2})+f(r)\right)$，对三次以内多项式精确。自适应细分是它的实用化：把区间二分后左右两段的 Simpson 和 S_{lr} 与整段 S 相比；对足够光滑的函数，两半的总误差约为整段误差的 $1/16$，故差值 $S_{lr}-S$ 能估计误差。当 $|S_{lr}-S| \le 15\varepsilon$ 时，用 $S_{lr}+\frac{S_{lr}-S}{15}$ 做 Richardson 外推；否则把误差预算 ε 均分给两半递归。变化平缓处自然少分、剧烈处自动加密，代码中的 `err` 就是 ε。

```
template <class F>
long double simpson(F f, long double l, long double r, long double err = 1E-12L, int dep = 24)
{
    // f 是被积函数，[l,r] 是区间，err 是绝对误差目标，dep 是递归上限；返回定积分近似值。
    assert(err > 0 && dep >= 0); // 调试检查，可删。
    auto simpson = [](long double ql, long double qr, long double vl, long double vm, long double vr)
    {
        // ql、qr 是端点，vl、vm、vr 是左端、中点、右端函数值；返回一次 Simpson 近似。
        return (vl + 4 * vm + vr) * (qr - ql) / 6;
    };
    long double m = (l + r) / 2;
    long double fl = f(l);
    long double fm = f(m);
    long double fr = f(r);
    auto dfs = [&](auto &&self,
                   long double ql,
                   long double qr,
                   long double vl,
                   long double vm,
                   long double vr,
                   long double whole,
                   long double tol,
                   int depth) -> long double
    {
        // self 是递归入口，whole 是当前段近似，tol 是误差预算，depth 是剩余层数；返回当前段积分值。
        long double mid = (ql + qr) / 2;
        long double x = (ql + mid) / 2;
        long double y = (mid + qr) / 2;
        long double fx = f(x);
        long double fy = f(y);
        long double a = simpson(ql, mid, vl, fx, vm);
        long double b = simpson(mid, qr, vm, fy, vr);
        if (!depth || fabsl(a + b - whole) <= 15 * tol)
```

```
        {
            return a + b + (a + b - whole) / 15;
        }
        return self(self, ql, mid, vl, fx, vm, a, tol / 2, depth - 1) +
               self(self, mid, qr, vm, fy, vr, b, tol / 2, depth - 1);
    };
    return dfs(dfs, l, r, fl, fm, fr, simpson(l, r, fl, fm, fr), err, dep);
}
```

调用 `simpson(f,l,r,err,dep)`，要求 `err>0`、`dep>=0`，且 `f` 在采样点处处有定义；对 $f(x)=x^4$ 在 $[0,1]$ 积分结果应在 10^{-10} 内逼近 0.2。间断、尖峰或强振荡函数要在奇点处主动分段，`dep` 只是递归上限、并非误差保证。若实际细分出 K 个区间，则有 $O(K)$ 次函数求值，递归栈不超过 `dep` 层。被积函数很光滑、想要更高阶收敛时用 9.3.002 Romberg 的外推；本条的优势在于对局部剧烈变化的函数按需加密、实现简短。

```
signed main()
{
    auto f = [](long double x) { return x * x * x * x; };
    assert(fabsl(simpson(f, 0, 1) - .2L) < 1E-10L);
}
```

9.3.002 Romberg 积分

复合梯形公式的误差按步长有 $h^2,h^4,h^6,\dots$ 的偶次幂展开（Euler–Maclaurin），这正好适合 Richardson 外推逐阶消去。Romberg 先用梯形法在步长 $h,h/2,h/4,\dots$ 上算出第一列 $R_{i,0}$——第 i 层复用上一层的值、只补新加入的中点，不重算旧点；再按 $R_{i,j}=\dfrac{4^jR_{i,j-1}-R_{i-1,j-1}}{4^j-1}$ 组合相邻两层，每做一次 j 就消掉一阶误差，对光滑函数收敛极快。本实现只滚动保存一维数组，原地从高层往低层外推。

```
template <class F> long double romberg(F f, long double l, long double r, int levels = 15)
{
    // f 是被积函数，[l,r] 是区间，levels 是外推层数；返回 Romberg 积分近似值。
    assert(1 <= levels && levels <= 25); // 调试检查，可删。
    vector<long double> a(levels);
    a[0] = (f(l) + f(r)) * (r - l) / 2;
    for (int i = 1; i < levels; i++)
    {
        unsigned long long cnt = 1ULL << i;
        long double h = (r - l) / cnt;
        long double sum = 0;
        for (unsigned long long j = 1; j < cnt; j += 2)
        {
            sum += f(l + j * h);
        }
        a[i] = a[i - 1] / 2 + h * sum;
        long double p = 4;
        for (int j = i - 1; j >= 0; j--)
        {
            a[j] = (p * a[j + 1] - a[j]) / (p - 1);
```

```
            p *= 4;
        }
    }
    return a[0];
}
```

调用 `romberg(f,l,r,levels)`，`levels` 每加 1 采样点约翻倍，限制为 `1<=levels<=25`；最高层约有 2^{24} 个小区间，继续放宽通常已没有竞赛实用性。外推因子使用 `long double`，不会发生整数连乘溢出。对 $f(x)=x^4$ 在 $[0,1]$ 应收敛到 0.2。若外推层数为 L，函数求值次数为 $O(2^L)$，空间为 $O(L)$。端点有可积奇异时须先变量代换；函数有间断或尖峰时用 9.3.001 自适应 Simpson 更合适。

```
signed main()
{
    auto f = [](long double x) { return x * x * x * x; };
    assert(fabsl(romberg(f, 0, 1, 8) - .2L) < 1E-12L);
}
```

9.3.003 Newton 多项式求根

Newton 迭代用当前点的切线零点逼近函数零点：把 f 在 x 处一阶泰勒展开 $f(x)+f'(x)\Delta\approx 0$ 解出步长，得 $x\leftarrow x-\dfrac{f(x)}{f'(x)}$。在单根附近它二次收敛（有效位数每步约翻倍），但对任意初值并不保证收敛：初值离根远、或迭代路过导数为零的驻点时步长会爆炸甚至发散。本实现把零导数与非有限数判为失败，用相对位移 $|y-x|\le\varepsilon\max(1,|y|)$ 判收敛；耗尽迭代上限仍未满足该条件就返回空。这里不对导数或函数值使用固定绝对阈值，因而把 f 整体乘一个很小的非零常数不会改变判定。

```
template <class F, class G> optional<long double> newton(F f, G df, long double x, int it = 80)
{
    // f、df 是函数及导数，x 是初值，it 是迭代上限；收敛时返回一个根，导数为零或发散返回空。
    assert(it >= 0); // 调试检查，可删。
    for (int i = 0; i < it; i++)
    {
        long double d = df(x);
        if (d == 0 || !isfinite(d))
        {
            return nullopt;
        }
        long double y = x - f(x) / d;
        if (!isfinite(y))
        {
            return nullopt;
        }
        if (fabsl(y - x) <= eps * max(1.L, fabsl(y)))
        {
            return y;
        }
        x = y;
    }
    return nullopt;
```

```
}
```

调用 `newton(f,df,x0)` 传函数、导数与初值，要求 `it>=0`；返回空表示当前初值未能可靠收敛。若实际迭代 I 轮，就有 $O(I)$ 次函数与导数求值和 $O(1)$ 空间；二次收敛阶不等于最坏迭代次数保证。没有好初值但已知一个异号区间 $[l,r]$（$f(l)f(r)<0$）时，用有界二分更稳、必定收敛；专门求实系数三次方程全部实根时用 9.3.004 一元三次方程，它先用导数临界点定位单调段再二分，不依赖初值。

```
signed main()
{
    auto f = [](long double x) { return x * x - 2; };
    auto df = [](long double x) { return 2 * x; };
    auto root = newton(f, df, 1.L);
    assert(root && fabsl(*root * *root - 2) < 1E-12L);
}
```

9.3.004 一元三次方程

求实系数三次方程 $ax^3+bx^2+cx+d=0$ 的全部互异实根，思路是“分段单调 + 二分”。次数按归一化前的精确零判断；随后把系数除以最大绝对值，要求原来非零的最高有效系数归一化后仍非零，这一比例范围取决于平台的 `long double`。三次函数的导数至多两个实根，它们把实轴切成至多三段单调区间，再由 Cauchy 根界包住全部实根并逐段二分。

```
optional<vector<long double>> cubicRoots(long double a, long double b, long double c, long
↪ double d)
{
    // a、b、c、d 是有限实系数；返回升序互异实根，恒等式返回空 optional，无实根返回存在的空向量。
    assert(isfinite(a) && isfinite(b) && isfinite(c) && isfinite(d)); // 调试检查，可删。
    bool hasA = a != 0;
    bool hasB = b != 0;
    bool hasC = c != 0;
    long double sc = max({fabsl(a), fabsl(b), fabsl(c), fabsl(d)});
    if (sc == 0)
    {
        return nullopt;
    }
    a /= sc;
    b /= sc;
    c /= sc;
    d /= sc;
    assert((!hasA || a != 0) && (!hasB || b != 0) && (!hasC || c != 0)); // 调试检查，可删。
    if (!hasA)
    {
        if (!hasB)
        {
            if (!hasC)
            {
                return vector<long double>{};
            }
            return vector<long double>{-d / c};
```

```
        }
        long double det = c * c - 4 * b * d;
        if (det < -eps)
        {
            return vector<long double>{};
        }
        det = max(0.L, det);
        if (det == 0)
        {
            return vector<long double>{-c / (2 * b)};
        }
        long double s = sqrtl(det);
        long double q = -0.5L * (c + copysignl(s, c));
        vector<long double> ans{q / b, d / q};
        sort(ans.begin(), ans.end());
        if (ans[0] == ans[1])
        {
            ans.pop_back();
        }
        return ans;
    }
    auto f = [&](long double x)
    {
        return ((a * x + b) * x + c) * x + d;
    };
    long double lim = 1 + max({fabsl(b), fabsl(c), fabsl(d)}) / fabsl(a);
    vector<long double> key{-lim, lim};
    long double det = b * b - 3 * a * c;
    if (det >= -eps)
    {
        det = sqrtl(max(0.L, det));
        long double x = (-b - det) / (3 * a);
        long double y = (-b + det) / (3 * a);
        if (-lim < x && x < lim)
        {
            key.push_back(x);
        }
        if (-lim < y && y < lim)
        {
            key.push_back(y);
        }
    }
    sort(key.begin(), key.end());
    vector<long double> ans;
    for (auto x : key)
    {
        if (fabsl(f(x)) <= eps)
        {
            ans.push_back(x);
        }
    }
    for (int i = 0; i + 1 < (int)key.size(); i++)
```

```
    {
        long double l = key[i];
        long double r = key[i + 1];
        long double fl = f(l);
        long double fr = f(r);
        if (fl == 0 || fr == 0 || (fl < 0) == (fr < 0))
        {
            continue;
        }
        for (int t = 0; t < 100; t++)
        {
            long double m = (l + r) / 2;
            long double fm = f(m);
            if ((fl < 0) == (fm < 0))
            {
                l = m;
                fl = fm;
            }
            else
            {
                r = m;
                fr = fm;
            }
        }
        ans.push_back((l + r) / 2);
    }
    sort(ans.begin(), ans.end());
    ans.erase(unique(ans.begin(),
                     ans.end(),
                     [&](auto x, auto y)
                     {
                         return fabsl(x - y) <= 100 * eps * max({1.L, fabsl(x), fabsl(y)});
                     }),
              ans.end());
    return ans;
}
```

调用 `cubicRoots(a,b,c,d)`：四个系数必须有限，且原来非零的最高有效系数除以最大绝对值后仍可表示为非零；MSVC 的 `long double` 通常等同 `double`，范围小于 GCC x86 的扩展精度。返回 `nullopt` 表示恒等式，存在但为空的向量表示没有实根，否则返回升序互异实根。原始系数精确零只用于次数分流，归一化判别式用相对其各项量级的容差，根去重使用相对根大小的 `100*eps`，三者不是同一零判断。内部二分固定 100 轮，时间、空间均为 $O(1)$。

下面检查三个实根、退化二次、无实根、重根与恒等式：

```
signed main()
{
    auto a = cubicRoots(1, -6, 11, -6);
    assert(a && a->size() == 3);
    auto b = cubicRoots(0, -1, 3, -2);
    assert(b && b->size() == 2 && fabsl((*b)[0] - 1) < 1E-10L && fabsl((*b)[1] - 2) < 1E-10L);
```

```
    auto wide = cubicRoots(0, 1, 1E16L, 1);
    assert(wide && wide->size() == 2 && fabsl((*wide)[1] + 1E-16L) < 1E-20L);
    auto none = cubicRoots(0, 1, 0, 1);
    assert(none && none->empty());
    auto repeated = cubicRoots(1, 0, 0, 0);
    assert(repeated && repeated->size() == 1 && fabsl((*repeated)[0]) < 1E-10L);
    assert(!cubicRoots(0, 0, 0, 0));
    cout << llround((*a)[0]) << ' ' << llround((*a)[1]) << ' ' << llround((*a)[2]) << '\n'; // 期望输出：1 2 3
    return 0;
}
```

9.3.005 线性插值与幂和

很多计数题的答案关于某个变量 x 是次数已知的多项式（典型如幂和 $\sum_{i=1}^{x} i^k$ 是 x 的 $k+1$ 次多项式），却只能算出前若干个点值。给定 $f(0), f(1), \dots, f(n-1)$ 这 n 个等距样本，f 的次数小于 n，就能用拉格朗日插值唯一恢复 $f(x)$。等距点让公式大幅简化：$f(x) = \sum_i y_i \prod_{j \neq i} \frac{x-j}{i-j}$ 中，分母 $\prod_{j \neq i}(i-j) = (-1)^{n-1-i}\, i!\,(n-1-i)!$ 可用阶乘逆元 $O(1)$ 得到，分子的连乘用前缀积 `pre` 与后缀积 `suf` 拆开，于是整体 $O(n)$。模数须为素数以保证阶乘可逆。`interpSeq` 里 $x < n$ 直接返回样本值。

```
// 依赖 7.1.001 快速幂
inline long long interpSeq(const vector<long long> &y, long long x, long long mod)
{
    // y[i]=f(i)，x 是查询点，mod 是素数；返回 f(x) 对 mod 的最小非负余数。
    int n = y.size();
    assert(n > 0 && n < mod); // 调试检查，可删。
    x %= mod;
    if (x < 0)
    {
        x += mod;
    }
    if (x < n)
    {
        long long ans = y[x] % mod;
        return ans < 0 ? ans + mod : ans;
    }
    vector<long long> fac(n, 1), ifac(n, 1), pre(n + 1, 1), suf(n + 1, 1);
    for (int i = 1; i < n; ++i)
    {
        fac[i] = mulMod(fac[i - 1], i, mod);
    }
    ifac[n - 1] = powerMod(fac[n - 1], mod - 2, mod);
    for (int i = n - 1; i; --i)
    {
        ifac[i - 1] = mulMod(ifac[i], i, mod);
    }
    for (int i = 0; i < n; ++i)
    {
        long long d = x >= i ? x - i : mod - (i - x);
```

```
        pre[i + 1] = mulMod(pre[i], d, mod);
    }
    for (int i = n - 1; i >= 0; --i)
    {
        long long d = x >= i ? x - i : mod - (i - x);
        suf[i] = mulMod(suf[i + 1], d, mod);
    }
    long long ans = 0;
    for (int i = 0; i < n; ++i)
    {
        long long v = y[i] % mod;
        if (v < 0)
        {
            v += mod;
        }
        long long term = mulMod(v, pre[i], mod);
        term = mulMod(term, suf[i + 1], mod);
        term = mulMod(term, ifac[i], mod);
        term = mulMod(term, ifac[n - 1 - i], mod);
        if ((n - 1 - i) & 1)
        {
            term = term ? mod - term : 0;
        }
        ans = ans >= mod - term ? ans - (mod - term) : ans + term;
    }
    return ans;
}
```

调用 `interpSeq(values,x,mod)`，要求样本非空、f 次数小于样本数、`mod` 为素数且 `values.size()<mod`；空样本不能唯一确定多项式，属于不满足前提。`x` 和样本值可以为负，函数统一返回最小非负余数。该接口保留运行时素数模并支持接近 `long long` 上界的查询，因此不能实例化固定 `MInt<P>`；模乘和模幂统一复用 7.1.001 快速幂，末尾模加用减法比较避免溢出。若模数为 M，预处理与单次求值为 $O(n+\log M)$ 时间、$O(n)$ 空间。求幂和 $\sum_{i=0}^{x} i^k$ 时先算 $k+2$ 个前缀样本再插值；固定常用素数且多次调用时，优先用 1.1.008 静态取模类 改写元素类型。非等距样本改用一般拉格朗日插值或第 10 章的多点求值。

下面用 $f(x)=x^2$ 检查区间外正查询和负查询，再检查负样本的直接返回：

```
signed main()
{
    assert(interpSeq({0, 1, 4, 9}, 5, 998244353) == 25);
    constexpr long long mod = 9'223'372'036'854'775'783LL;
    assert(interpSeq({0, 1, 4}, mod - 1, mod) == 1);
    assert(interpSeq({0, 1, 4}, -1, 7) == 1);
    assert(interpSeq({0, -1}, 1, 5) == 4);
    cout << interpSeq({0, 1, 4, 9}, 5, 998244353) << '\n'; // 期望输出：25
    return 0;
}
```

9.3.006 指数乘多项式有限和

给定次数不超过 d 的多项式 f 在 $0,1,\dots,d$ 的值，求 $\sum_{i=0}^{n-1} r^i f(i)$。当 $r=1$ 时，前缀和本身是 $d+1$ 次多项式，算出前 $d+2$ 个值后等距插值即可；当 $r\neq 0,1$ 时，先求形式无穷和常数 $C=\sum_{i\geq 0} r^i f(i)$，则 $r^{-n}(S_n-C)$ 是关于 n 的 d 次多项式，同样由前 $d+2$ 个值插值得到。`r=0` 必须单独处理，因为一般分支会请求它的逆元。

```cpp
template <class T> T fieldPow(T a, unsigned long long b)
{
    // a 是域元素，b 是非负指数；返回 a 的 b 次幂。
    T ans = 1;
    while (b)
    {
        if (b & 1)
        {
            ans *= a;
        }
        b >>= 1;
        if (b)
        {
            a *= a;
        }
    }
    return ans;
}

template <class T> T interpIota(const vector<T> &y, long long x)
{
    // y[i] 是多项式在 i 处的值，x 是非负查询点；返回该多项式在 x 处的值。
    int n = y.size();
    assert(n > 0 && x >= 0); // 调试检查，可删。
    if (x < n)
    {
        return y[x];
    }
    vector<T> fac(n, T(1)), ifac(n, T(1));
    vector<T> pre(n + 1, T(1)), suf(n + 1, T(1));
    for (int i = 1; i < n; i++)
    {
        fac[i] = fac[i - 1] * T(i);
    }
    ifac[n - 1] = T(1) / fac[n - 1];
    for (int i = n - 1; i; i--)
    {
        ifac[i - 1] = ifac[i] * T(i);
    }
    for (int i = 0; i < n; i++)
    {
        pre[i + 1] = pre[i] * T(x - i);
    }
    for (int i = n - 1; i >= 0; i--)
```

```
    {
        suf[i] = suf[i + 1] * T(x - i);
    }
    T ans = 0;
    for (int i = 0; i < n; i++)
    {
        T v = y[i] * pre[i] * suf[i + 1] * ifac[i] * ifac[n - 1 - i];
        ans += (n - 1 - i) & 1 ? -v : v;
    }
    return ans;
}

template <class T> T sumExpPLim(T r, vector<T> f)
{
    // f[i] 是多项式在 i 处的值且 r≠1；返回生成函数给出的形式无穷和。
    assert(!f.empty() && r != T(1)); // 调试检查，可删。
    T q = r / (T(1) - r);
    T term = T(1) / (T(1) - r);
    T ans = 0;
    for (int k = 0; k < (int)f.size(); k++)
    {
        ans += f[0] * term;
        for (int i = 0; i + 1 < (int)f.size() - k; i++)
        {
            f[i] = f[i + 1] - f[i];
        }
        term *= q;
    }
    return ans;
}

template <class T> T sumExpPoly(T r, const vector<T> &f, long long n)
{
    // f[i] 是多项式在 i 处的值，n 是项数；返回前 n 项的指数加权和。
    assert(!f.empty() && n >= 0); // 调试检查，可删。
    if (n == 0)
    {
        return T(0);
    }
    if (r == T(0))
    {
        return f[0];
    }
    int d = f.size() - 1;
    vector<T> s(d + 2);
    T pw = 1;
    for (int i = 0; i <= d; i++)
    {
        s[i + 1] = s[i] + pw * f[i];
        pw *= r;
    }
    if (r == T(1))
```

```
    {
        return interpIota(s, n);
    }
    T limit = sumExpPLim(r, f);
    T inv = T(1) / r;
    T ipw = 1;
    for (int i = 0; i <= d + 1; i++)
    {
        s[i] = (s[i] - limit) * ipw;
        ipw *= inv;
    }
    return limit + fieldPow(r, (unsigned long long)n) * interpIota(s, n);
}
```

`sumExpPoly(r,f,n)` 要求 `f[i]` 是次数小于 `f.size()` 的多项式在连续整数点的值，`n>=0`。`T` 必须是域，且 $1, 2, \dots, d+1$ 可逆；素数模下通常要求 `d+1<mod`。有限差分占 $O(d^2)$ 时间，其他步骤为 $O(d + \log n)$，空间为 $O(d)$。若已有快速卷积，可用二项式变换加速各阶差分的计算；普通竞赛次数下本版更短。

下面覆盖 `r=0`、`r=1` 与一般分支：

```
signed main()
{
    assert(fabsl(sumExpPoly(2.L, vector<long double>{0, 1}, 4) - 34) < 1E-10L);
    assert(fabsl(sumExpPoly(1.L, vector<long double>{0, 1}, 5) - 10) < 1E-10L);
    assert(fabsl(sumExpPoly(0.L, vector<long double>{7, 8}, 100) - 7) < 1E-10L);
    cout << llround(sumExpPoly(2.L, vector<long double>{0, 1}, 4)) << '\n'; // 期望输出：34
    return 0;
}
```

9.3.007 指数乘多项式无穷和

把 $f(i)$ 写成 Newton 展开 $f(i) = \sum_k \Delta^k f(0) \binom{i}{k}$，再用 $\sum_{i \ge 0} r^i \binom{i}{k} = \dfrac{r^k}{(1-r)^{k+1}}$，即可得到上条 `sumExpPLim(r,f)`。调用时 `f[i]` 必须是次数不超过 d 的多项式在 $i = 0, 1, \dots, d$ 处的值，`r=1` 非法，`r=0` 合法且结果为 `f(0)`。这里的“无穷和”是在域上按生成函数得到的有理式；当 `T` 是实数时，只有 $|r| < 1$ 才同时等于通常意义下的收敛级数。时间为 $O(d^2)$，空间为 $O(d)$；实现与有限和共享，故不重复打印。

```
// 依赖 9.3.006 指数乘多项式有限和
signed main()
{
    long double a = sumExpPLim(0.5L, vector<long double>{1});
    long double b = sumExpPLim(0.5L, vector<long double>{0, 1});
    assert(fabsl(a - 2) < 1E-10L && fabsl(b - 2) < 1E-10L);
    assert(fabsl(sumExpPLim(0.L, vector<long double>{7, 8}) - 7) < 1E-10L);
    cout << fixed << setprecision(1) << a << ' ' << b << '\n'; // 期望输出：2.0 2.0
    return 0;
}
```

9.3.008 LLL 浮点实用版

LLL 算法把整数格的一组基 $b_0,\dots,b_{n-1}$ 变换成同一格中较短、较接近正交的基。设 b_i^* 是 Gram-Schmidt 正交化向量，$\mu_{i,j}=\frac{\langle b_i,b_j^*\rangle}{\langle b_j^*,b_j^*\rangle}$。算法维护下标 k（初值 1），反复执行：

1. 尺寸约简：对 $j=k-1,\dots,0$，令 $q=\operatorname{round}(\mu_{k,j})$，$b_k\leftarrow b_k-qb_j$，使 $|\mu_{k,j}|\le\frac12$；此时正交化向量不变，只需同步更新 $\mu_{k,\cdot}$。
2. Lovász 条件：若 $\|b_k^*\|^2\ge(\delta-\mu_{k,k-1}^2)\|b_{k-1}^*\|^2$，则 $k\leftarrow k+1$；否则交换 b_k,b_{k-1}，重新正交化并令 $k\leftarrow\max(1,k-1)$。

结束时第一个向量满足 $\|b_0\|\le\left(\frac{4}{4\delta-1}\right)^{(n-1)/2}\lambda_1$，$\lambda_1$ 是格中最短非零向量长度。

```
optional<vector<vector<int>>> lll(vector<vector<int>> b, long double delta = 0.75L)
{
    // b 是 n 个长度为 m 的整数向量，须线性无关；delta 取 (1/4,1)，越接近 1 约简越强。
    // 返回同一格的 LLL 约简基（第一个向量长度不超过最短向量的 (4/(4delta-1))^{(n-1)/2} 倍）；
    // 线性相关时返回空。Gram-Schmidt 用 long double 计算，系数很大时可能有精度误差。
    int n = b.size();
    if (!n)
    {
        return b;
    }
    int m = b[0].size();
    assert(0.25L < delta && delta < 1); // 调试检查，可删。
    // st[i] 是 b[i] 的 Gram-Schmidt 正交化向量，nm[i]=|st[i]|^2，mu[i][j]=<b[i],st[j]>/nm[j]。
    vector<vector<long double>> st(n, vector<long double>(m)), mu(n, vector<long double>(n));
    vector<long double> nm(n);
    auto gs = [&]() -> bool
    {
        // 重新做 Gram-Schmidt；出现零向量（线性相关）时返回 false。
        for (int i = 0; i < n; i++)
        {
            for (int t = 0; t < m; t++)
            {
                st[i][t] = b[i][t];
            }
            for (int j = 0; j < i; j++)
            {
                long double dot = 0;
                for (int t = 0; t < m; t++)
                {
                    dot += b[i][t] * st[j][t];
                }
                mu[i][j] = dot / nm[j];
                for (int t = 0; t < m; t++)
                {
                    st[i][t] -= mu[i][j] * st[j][t];
                }
            }
            nm[i] = 0;
            for (auto x : st[i])
```

```
            {
                nm[i] += x * x;
            }
            if (nm[i] < 1e-9L)
            {
                return false;
            }
        }
        return true;
    };
    if (!gs())
    {
        return nullopt;
    }
    int k = 1;
    while (k < n)
    {
        // 尺寸约简：让 |mu[k][j]| <= 1/2。
        for (int j = k - 1; j >= 0; j--)
        {
            int q = llroundl(mu[k][j]);
            if (q)
            {
                for (int t = 0; t < m; t++)
                {
                    b[k][t] -= q * b[j][t];
                }
                // 正交向量不变，只需同步 mu[k][*]。
                for (int i = 0; i < j; i++)
                {
                    mu[k][i] -= q * mu[j][i];
                }
                mu[k][j] -= q;
            }
        }
        // Lovász 条件成立则前进，否则交换相邻两向量并后退。
        if (nm[k] >= (delta - mu[k][k - 1] * mu[k][k - 1]) * nm[k - 1])
        {
            k++;
        }
        else
        {
            swap(b[k], b[k - 1]);
            gs();
            k = max(1LL, k - 1);
        }
    }
    return b;
}
```

`lll(b,delta)` 中 `b` 是 n 个长为 m 的整数向量，须线性无关，$\delta \in (\frac{1}{4}, 1)$，默认 0.75；返回约简后的基，线性相关时返回空。Gram-Schmidt 用 `long double`，坐标很大时可能因精度问题得不到严格的约简基，

也可能在 `long long` 坐标上溢出，需要严格结果时应改用有理数或整数版本。交换次数关于输入规模是多项式级，每次交换重新正交化 $O(n^2 m)$。

典型用途是背包型密码、求小整数关系与丢番图逼近；二维格的最短向量用 Gauss 约简更简单。下面约简一个三维格，变换前后行列式的绝对值都是 3：

```
signed main()
{
    auto b = lll({{1, 1, 1}, {-1, 0, 2}, {3, 5, 6}});
    assert(b && *b == vector<vector<int>>({{0, 1, 0}, {1, 0, 1}, {-2, 0, 1}}));
    assert(!lll({{1, 1}, {2, 2}}));
}
```

10. 多项式、变换与线性递推

10.1 卷积与变换

10.1.001 DFT / IDFT

DFT 把长度 n 的序列映到 n 个单位根上的点值 $A_k = \sum_{j=0}^{n-1} a_j \omega_n^{jk}$，其中 $\omega_n = e^{2\pi i/n}$；逆变换换成共轭单位根 ω_n^{-jk} 再除以 n，单位根正交性 $\sum_k \omega_n^{k(i-j)} = n\,[i=j]$ 保证两次变换互逆。这份朴素双重循环只用于小规模验算和理解定义，不是快速卷积主力。

```cpp
using Complex = complex<long double>;

vector<Complex> dftNaive(const vector<Complex> &a, bool inv = false)
{
    // a 是时域或频域序列，inv=false/true 表示 DFT/IDFT；返回变换结果，逆变换已除以长度。
    int n = a.size();
    vector<Complex> b(n);
    long double pi = acosl(-1);
    for (int k = 0; k < n; k++)
    {
        for (int j = 0; j < n; j++)
        {
            long double ang = 2 * pi * j * k / n * (inv ? -1 : 1);
            b[k] += a[j] * Complex(cosl(ang), sinl(ang));
        }
        if (inv)
        {
            b[k] /= n;
        }
    }
    return b;
}
```

`dftNaive(a,false)` 得频域点值，`dftNaive(A,true)` 还原时域，浮点结果按误差比较；序列 `{1,2,3,4}` 正逆变换后实部在 10^{-12} 内还原、虚部量级约 10^{-19}。$O(n^2)$ 时间、$O(n)$ 输出空间。规模一大必须换 10.1.002 FFT 的 $O(n \log n)$ 蝶形；模意义精确卷积用 10.1.003 NTT，目标模数不支持则用 10.1.004 MTT 任意模数卷积。

10.1.002 FFT

FFT 用分治蝴蝶在 $O(n \log n)$ 内算 DFT/IDFT：按下标奇偶把 $A_k = E_k + \omega_n^k O_k$、$A_{k+n/2} = E_k - \omega_n^k O_k$ 展开，迭代版先做位逆序置换再自底向上合并各层蝶形，逆变换用共轭角度并整体除以长度。整数卷积把系数放进复数实部，变换后逐点相乘再逆变换、结果四舍五入取整；`long double` 精度下系数与长度不过大时误差可控（本机随机 ± 500、长度百级时最大绝对误差约 3.5×10^{-12}）。

```cpp
using C = complex<long double>;

```

```cpp
void fft(vector<C> &a, bool inv = false)
{
    // a 是待变换序列，inv 表示是否做逆变换；原地完成 FFT，逆变换已除以长度。
    int n = a.size();
    assert(n > 0 && n <= (1 << 23) && has_single_bit((unsigned long long)n)); // 调试检查，可删。
    int j = 0;
    for (int i = 1; i < n; i++)
    {
        int k = n >> 1;
        while (j & k)
        {
            j ^= k;
            k >>= 1;
        }
        j ^= k;
        if (i < j)
        {
            swap(a[i], a[j]);
        }
    }
    const long double pi = acosl(-1);
    for (int len = 2; len <= n; len <<= 1)
    {
        long double ang = 2 * pi / len * (inv ? -1 : 1);
        C wlen(cosl(ang), sinl(ang));
        for (int l = 0; l < n; l += len)
        {
            C w{1.0L, 0.0L};
            for (int j = 0; j < len / 2; j++)
            {
                C x = a[l + j];
                C y = a[l + j + len / 2] * w;
                a[l + j] = x + y;
                a[l + j + len / 2] = x - y;
                w *= wlen;
            }
        }
    }
    if (inv)
    {
        for (C &x : a)
        {
            x /= n;
        }
    }
}

vector<long double> convolution(const vector<long double> &a, const vector<long double> &b)
{
    // a、b 是实系数序列；返回二者的线性卷积，任一输入为空时返回空。
    if (a.empty() || b.empty())
    {
```

```
        return {};
    }
    int sz = a.size() + b.size() - 1;
    int n = bit_ceil((unsigned long long)sz);
    vector<C> x(n);
    vector<C> y(n);
    copy(a.begin(), a.end(), x.begin());
    copy(b.begin(), b.end(), y.begin());
    fft(x);
    fft(y);
    for (int i = 0; i < n; i++)
    {
        x[i] *= y[i];
    }
    fft(x, true);
    vector<long double> c(sz);
    for (int i = 0; i < sz; i++)
    {
        c[i] = x[i].real();
    }
    return c;
}
```

`convolution(a,b)` 返回长度 $n+m-1$ 的实系数卷积，内部补齐到 `bit_ceil(n+m-1)`；`fft(a,inv)` 原地变换要求长度是 2 的幂。长度 N 的变换 $O(N\log N)$、卷积 $O((n+m)\log(n+m))$，空间 $O(N)$。系数或长度过大导致浮点误差失控时改用 10.1.003 NTT 的精确整数变换；目标模数不含 NTT 单位根时在此 FFT 上拆系数即 10.1.004 MTT 任意模数卷积。

下面用空输入、单项、负系数和结果长度为 9 的固定例核对补长与舍入；输出 `OK` 即通过：

```
signed main()
{
    assert(convolution({}, {1}).empty());
    auto a = convolution(vector<long double>{3}, vector<long double>{-7});
    assert(a.size() == 1 && llroundl(a[0]) == -21);
    auto b = convolution(vector<long double>{1, 2, 3}, vector<long double>{4, -5});
    assert(b.size() == 4 && llroundl(b[0]) == 4 && llroundl(b[1]) == 3 &&
           llroundl(b[2]) == 2 && llroundl(b[3]) == -15);
    auto c = convolution(vector<long double>{1, -2, 3, -4, 5},
                         vector<long double>{6, 7, -8, 9, -10});
    vector<long long> want{6, -5, -4, 22, -50, 114, -106, 85, -50};
    assert(c.size() == want.size());
    for (int i = 0; i < (int)c.size(); i++)
    {
        assert(llroundl(c[i]) == want[i]);
    }
    cout << "OK\n";
    return 0;
}
```

10.1.003 NTT

NTT 在有限域上做整数精确的 DFT。设素模 p、原根 g、变换长度 $N \mid p-1$，取 $\omega_N = g^{(p-1)/N}$，则 $\omega_N^N = 1$ 且 $0 < k < N$ 时 $\omega_N^k \neq 1$。正变换 $A_k = \sum_j a_j \omega_N^{jk}$；由单位根正交性 $\sum_k \omega_N^{k(i-j)} = N[i=j]$ 得逆变换 $a_j = N^{-1}\sum_k A_k \omega_N^{-jk}$。按奇偶拆下标并用 $\omega_N^{N/2} = -1$ 得一层蝶形 $A_k = E_k + \omega_N^k O_k$、$A_{k+N/2} = E_k - \omega_N^k O_k$。

`FastNTT` 把相邻两层合成 radix-4 蝶形，用 `rate2`、`rate3` 按块更新旋转因子，不做显式位逆序。长度 N 的结果中相邻位置 `2*i,2*i+1` 对应 $z, -z$，前者与长度 $N/2$ 变换的第 i 个频点对应同一个 z^2；10.2.012 多项式复合 正是复用这个排列契约。`NTT32` 是另一种受限工作接口：它直接处理 `int*`，缓存位逆序表和一块 `unsigned long long` 缓冲区；乘单位根后仍立即取模，只延迟蝶形加减结果的归一化。长度上限固定为 2^{17}，此时尚未归一化的中间值与单位根相乘仍严格落在 64 位无符号范围内；它服务 10.2.005 `ln`、`exp`、开根、快速幂 的 16 叉在线卷积，不能替代最大长度 2^{23} 的通用 `FastNTT`。两者共用本章的 `Z`、模数和单位根计算，没有再复制模整数、卷积或 FPS 基础层。

```
// 依赖 1.1.008 静态取模类
constexpr int NTT_LIM = 1 << 23;

bool nttFit(int n, int m)
{
    // n、m 是两输入序列长度；返回一次不超过 NTT_LIM 的变换能否覆盖其线性卷积。
    assert(n >= 0 && m >= 0); // 调试检查，可删。
    return n == 0 || m == 0 || (__int128)n + m - 1 <= NTT_LIM;
}

bool convFit(int n, int m)
{
    // n、m 是两输入序列长度；返回公共 conv 能否计算其线性卷积。
    // 结果只超上限一至两项时，conv 会拆掉两端最高项再做一次较短卷积。
    assert(n >= 0 && m >= 0); // 调试检查，可删。
    return n == 0 || m == 0 || min(n, m) <= 64 ||
           (__int128)n + m - 1 <= NTT_LIM + 2;
}

class FastNTT
{
    // root[i]、iroot[i] 分别保存 2 的 i 次单位根及其逆元。
    static inline array<int, 24> root;
    static inline array<int, 24> iroot;
    // rate2、irate2 保存二进制蝶形跨块时的旋转因子更新量。
    static inline array<int, 22> rate2;
    static inline array<int, 22> irate2;
    // rate3、irate3 保存四进制蝶形跨块时的旋转因子更新量。
    static inline array<int, 21> rate3;
    static inline array<int, 21> irate3;
    // ready 标记单位根表是否已经初始化。
    static inline bool ready = false;

    static void init()
    {
```

```
        // 无参数；首次调用时预处理各阶单位根和跨块旋转因子，后续调用直接复用。
        if (ready)
        {
            return;
        }
        ready = true;
        root[23] = 31;
        iroot[23] = Z(root[23]).inv().val();
        for (int i = 22; i >= 0; i--)
        {
            root[i] = (unsigned long long)root[i + 1] * root[i + 1] % mod;
            iroot[i] = (unsigned long long)iroot[i + 1] * iroot[i + 1] % mod;
        }
        int prod = 1;
        int iprod = 1;
        for (int i = 0; i <= 21; i++)
        {
            rate2[i] = (unsigned long long)root[i + 2] * prod % mod;
            irate2[i] = (unsigned long long)iroot[i + 2] * iprod % mod;
            prod = (unsigned long long)prod * iroot[i + 2] % mod;
            iprod = (unsigned long long)iprod * root[i + 2] % mod;
        }
        prod = 1;
        iprod = 1;
        for (int i = 0; i <= 20; i++)
        {
            rate3[i] = (unsigned long long)root[i + 3] * prod % mod;
            irate3[i] = (unsigned long long)iroot[i + 3] * iprod % mod;
            prod = (unsigned long long)prod * iroot[i + 3] % mod;
            iprod = (unsigned long long)iprod * root[i + 3] % mod;
        }
    }

public:
    static void dft(Z *a, int n)
    {
        // a 指向长度 n 的连续系数，n 是至多 2^23 的正 2 的幂；原地完成正变换。
        // 长度 n 的相邻位置 2i、2i+1 对应 z、-z；前者与长度 n/2 变换的第 i 个频点对应同一个 z^2。
        assert(n > 0 && has_single_bit((unsigned long long)n) && n <= NTT_LIM); // 调试检查，可
        ↪ 删。
        init();
        int z = n;
        int h = countr_zero((unsigned long long)z);
        int len = 0;
        while (len < h)
        {
            if (h - len == 1)
            {
                int p = 1 << (h - len - 1);
                int rot = 1;
                for (int s = 0; s < (1 << len); s++)
                {
```

```
87                      int off = s << (h - len);
88                      for (int i = 0; i < p; i++)
89                      {
90                          int l = a[off + i].x;
91                          int r = (unsigned long long)a[off + i + p].x * rot % mod;
92                          a[off + i].x = l + r < mod ? l + r : l + r - mod;
93                          a[off + i + p].x = l >= r ? l - r : l + mod - r;
94                      }
95                      if (s + 1 != (1 << len))
96                      {
97                          rot = (unsigned long long)rot * rate2[countr_zero((unsigned long
    ↪ long)~s)] % mod;
98                      }
99                  }
100                 len++;
101             }
102             else
103             {
104                 int p = 1 << (h - len - 2);
105                 int rot = 1;
106                 int imag = root[2];
107                 for (int s = 0; s < (1 << len); s++)
108                 {
109                     int rot2 = (unsigned long long)rot * rot % mod;
110                     int rot3 = (unsigned long long)rot2 * rot % mod;
111                     int off = s << (h - len);
112                     for (int i = 0; i < p; i++)
113                     {
114                         unsigned long long m2 = (unsigned long long)mod * mod;
115                         unsigned long long a0 = a[off + i].x;
116                         unsigned long long a1 = (unsigned long long)a[off + i + p].x * rot;
117                        unsigned long long a2 = (unsigned long long)a[off + i + 2 * p].x * rot2;
118                        unsigned long long a3 = (unsigned long long)a[off + i + 3 * p].x * rot3;
119                        unsigned long long x = (unsigned long long)Z(a1 + m2 - a3).val() * imag;
120                         unsigned long long na2 = m2 - a2;
121                         a[off + i].x = (a0 + a2 + a1 + a3) % mod;
122                         a[off + i + p].x = (a0 + a2 + 2 * m2 - a1 - a3) % mod;
123                         a[off + i + 2 * p].x = (a0 + na2 + x) % mod;
124                         a[off + i + 3 * p].x = (a0 + na2 + m2 - x) % mod;
125                     }
126                     if (s + 1 != (1 << len))
127                     {
128                         rot = (unsigned long long)rot * rate3[countr_zero((unsigned long
    ↪ long)~s)] % mod;
129                     }
130                 }
131                 len += 2;
132             }
133         }
134     }
135
136     static void dft(vector<Z> &a)
```

```
    {
        // a 的长度是至多 2^23 的正 2 的幂；转发到连续内存正变换，无返回值。
        dft(a.data(), a.size());
    }

    static void idft(Z *a, int n)
    {
        // a 指向本类 dft 产生的 n 个频域值；原地逆变换、除以 n 并恢复自然顺序系数。
        assert(n > 0 && has_single_bit((unsigned long long)n) && n <= NTT_LIM); // 调试检查，可
        ↪ 删。
        init();
        int z = n;
        int h = countr_zero((unsigned long long)z);
        int len = h;
        while (len)
        {
            if (len == 1)
            {
                int p = 1 << (h - len);
                int rot = 1;
                for (int s = 0; s < (1 << (len - 1)); s++)
                {
                    int off = s << (h - len + 1);
                    for (int i = 0; i < p; i++)
                    {
                        int l = a[off + i].x;
                        int r = a[off + i + p].x;
                        a[off + i].x = l + r < mod ? l + r : l + r - mod;
                        a[off + i + p].x = (unsigned long long)(l + mod - r) * rot % mod;
                    }
                    if (s + 1 != (1 << (len - 1)))
                    {
                        rot = (unsigned long long)rot * irate2[countr_zero((unsigned long
    ↪ long)~s)] % mod;
                    }
                }
                len--;
            }
            else
            {
                int p = 1 << (h - len);
                int rot = 1;
                int imag = iroot[2];
                for (int s = 0; s < (1 << (len - 2)); s++)
                {
                    int rot2 = (unsigned long long)rot * rot % mod;
                    int rot3 = (unsigned long long)rot2 * rot % mod;
                    int off = s << (h - len + 2);
                    for (int i = 0; i < p; i++)
                    {
                        unsigned long long a0 = a[off + i].x;
                        unsigned long long a1 = a[off + i + p].x;
```

```
                        unsigned long long a2 = a[off + i + 2 * p].x;
                        unsigned long long a3 = a[off + i + 3 * p].x;
                        unsigned long long x = Z((mod + a2 - a3) * imag).val();
                        a[off + i].x = (a0 + a1 + a2 + a3) % mod;
                        a[off + i + p].x = (a0 + mod - a1 + x) % mod * rot % mod;
                        a[off + i + 2 * p].x = (a0 + a1 + 2 * mod - a2 - a3) % mod * rot2 % mod;
                        a[off + i + 3 * p].x = (a0 + 2 * mod - a1 - x) % mod * rot3 % mod;
                    }
                    if (s + 1 != (1 << (len - 2)))
                    {
                        rot = (unsigned long long)rot * irate3[countr_zero((unsigned long long)~s)] % mod;
                    }
                }
                len -= 2;
            }
        }
        Z iv = Z(n).inv();
        for (int i = 0; i < z; i++)
        {
            a[i] *= iv;
        }
    }

    static void idft(vector<Z> &a)
    {
        // a 必须来自本类 dft；转发到连续内存逆变换，无返回值。
        idft(a.data(), a.size());
    }

    static vector<Z> conv(vector<Z> a, vector<Z> b)
    {
        // a、b 按低次到高次保存系数；返回普通线性卷积，任一输入为空时返回空。
        if (a.empty() || b.empty())
        {
            return {};
        }
        int n = a.size();
        int m = b.size();
        int s = n + m - 1;
        if (min(n, m) <= 64)
        {
            vector<Z> c(s);
            for (int i = 0; i < n; i++)
            {
                for (int j = 0; j < m; j++)
                {
                    c[i + j] += a[i] * b[j];
                }
            }
            return c;
        }
```

```
        assert(convFit(n, m)); // 调试检查，可删。
        int z = bit_ceil((unsigned long long)s);
        if (s - 2 <= z / 2)
        {
            Z al = a.back();
            Z bl = b.back();
            a.pop_back();
            b.pop_back();
            auto x = a;
            auto y = b;
            auto c = conv(move(a), move(b));
            c.resize(s);
            c[s - 1] = al * bl;
            for (int i = 0; i < n - 1; i++)
            {
                c[i + m - 1] += x[i] * bl;
            }
            for (int i = 0; i < m - 1; i++)
            {
                c[i + n - 1] += y[i] * al;
            }
            return c;
        }
        bool same = a == b;
        a.resize(z);
        b.resize(z);
        dft(a);
        if (same)
        {
            b = a;
        }
        else
        {
            dft(b);
        }
        for (int i = 0; i < z; i++)
        {
            a[i] *= b[i];
        }
        idft(a);
        a.resize(s);
        return a;
    }
};

class NTT32
{
    // 本类服务原始 int 切片，复用位逆序表与 unsigned long long 工作区，并在蝶形间延迟取模。
    // 它与 FastNTT 的 Z 容器接口用途不同；2^17 上限保证最终归一化乘法不会溢出 unsigned long long。
    int mx = 0; // 允许的最大变换长度，所有实际长度必须整除 mx。
    vector<int> w; // w[l..2l) 保存长度 2l 蝶形使用的正单位根。
    vector<unsigned long long> buf; // 蝶形使用的延迟取模工作区。
```

```
290     array<vector<int>, 24> rev; // rev[k] 保存长度 2^k 的二进制翻转下标。
291
292     const vector<int> &getRev(int n)
293     {
294         // n 是不超过 mx 的二次幂；返回对应二进制翻转下标表的引用。
295         int k = countr_zero((unsigned long long)n);
296         auto &r = rev[k];
297         if (r.empty())
298         {
299             r.resize(n);
300             for (int i = 1; i < n; i++)
301             {
302                 r[i] = (r[i >> 1] >> 1) | ((int)(i & 1) << (k - 1));
303             }
304         }
305         return r;
306     }
307
308     void trans(int *a, int n, bool inv)
309     {
310         // a 指向长度 n 且各项在 [0,mod) 内的预分配切片，inv 表示是否逆变换；使用 unsigned long long
            ↪ 工作区原地完成变换。
311         assert(n > 0 && has_single_bit((unsigned long long)n) && n <= mx && mx % n == 0); // 调
            ↪ 试检查，可删。
312         const auto &r = getRev(n);
313         for (int i = 0; i < n; i++)
314         {
315             buf[i] = a[r[i]];
316         }
317         for (int h = 1; h < n; h <<= 1)
318         {
319             const int *o = w.data() + h;
320             for (int l = 0; l < n; l += 2 * h)
321             {
322                 for (int j = 0; j < h; j++)
323                 {
324                     unsigned long long x = buf[l + j];
325                     unsigned long long y = buf[l + j + h] * o[j] % mod;
326                     buf[l + j] = x + y;
327                     buf[l + j + h] = x + mod - y;
328                 }
329             }
330         }
331         if (!inv)
332         {
333             for (int i = 0; i < n; i++)
334             {
335                 a[i] = buf[i] % mod;
336             }
337             return;
338         }
339         int k = countr_zero((unsigned long long)n);
```

```
        int iv = mod - ((mod - 1) >> k);
        a[0] = buf[0] * iv % mod;
        for (int i = 1; i < n; i++)
        {
            a[i] = buf[n - i] * iv % mod;
        }
    }

  public:
    NTT32(int n = 0) : mx(n), w(n), buf(n)
    {
        // n 是最大变换长度，必须为 0 或不超过 2^17 的二次幂；预处理各级连续单位根。
        assert(n == 0 || (has_single_bit((unsigned long long)n) && n <= (1 << 17))); // 调试检查，可删。
        if (n == 0)
        {
            return;
        }
        if (n == 1)
        {
            w[0] = 1;
            return;
        }
        int r = Z(3).pow((mod - 1) / n).val();
        w[n / 2] = 1;
        for (int i = n / 2 + 1; i < n; i++)
        {
            w[i] = (unsigned long long)w[i - 1] * r % mod;
        }
        for (int i = n / 2 - 1; i; i--)
        {
            w[i] = w[i << 1];
        }
    }

    void dft(int *a, int n)
    {
        // a 指向长度 n 且各项在 [0,mod) 内的预分配切片；原地完成自然频点顺序的正变换，无返回值。
        trans(a, n, false);
    }

    void idft(int *a, int n)
    {
        // a 指向本对象 dft 产生的 n 个频域值；原地完成逆变换并除以 n，无返回值。
        trans(a, n, true);
    }
};

void dft(vector<Z> &a)
{
    // a 的长度是至多 2^23 的正 2 的幂；调用卡常正变换，结果满足 FastNTT 的递归配对排列。
    FastNTT::dft(a);
```

```
}

void dft(Z *a, int n)
{
    // a 指向长度 n 的连续系数；调用卡常正变换，结果满足 FastNTT 的递归配对排列。
    FastNTT::dft(a, n);
}

void idft(vector<Z> &a)
{
    // a 必须来自全局 dft；调用卡常逆变换并恢复自然顺序系数。
    FastNTT::idft(a);
}

void idft(Z *a, int n)
{
    // a 指向长度 n 且来自全局 dft 的频域值；调用卡常逆变换并恢复自然顺序系数。
    FastNTT::idft(a, n);
}

vector<Z> conv(vector<Z> a, vector<Z> b)
{
    // a、b 按低次到高次保存系数；返回卡常 NTT 计算的普通线性卷积。
    return FastNTT::conv(move(a), move(b));
}
```

当总长度超过单次 2^{23} NTT 上限时，把每个输入按 `NTT_LIM/2` 分块。每个输入块只做一次正变换；按块下标的和在频域累加乘积，再逐个逆变换输出块。这样设块数为 A, B 时变换次数为 $A + B + AB$（而不是对每个块对重复做两次正变换），峰值空间为 $O((A + B + 1)\,NTT_LIM)$，总时间为 $O((A + B + AB)\,NTT_LIM \log NTT_LIM)$。

```
vector<Z> convLarge(const vector<Z> &a, const vector<Z> &b)
{
    // a、b 的总长可以超过单次 NTT 上限；返回分块线性卷积。
    if (a.empty() || b.empty()) return {};
    if (convFit(a.size(), b.size())) return conv(a, b);
    constexpr int block = NTT_LIM / 2;
    vector<Z> ans(a.size() + b.size() - 1);
    vector<vector<Z>> x, y;
    for (int i = 0; i < (int)a.size(); i += block)
    {
        vector<Z> v(a.begin() + i, a.begin() + min<int>(a.size(), i + block));
        v.resize(NTT_LIM);
        dft(v);
        x.push_back(move(v));
    }
    for (int i = 0; i < (int)b.size(); i += block)
    {
        vector<Z> v(b.begin() + i, b.begin() + min<int>(b.size(), i + block));
        v.resize(NTT_LIM);
        dft(v);
```

```
        y.push_back(move(v));
    }
    for (int s = 0; s + 1 < (int)x.size() + (int)y.size(); s++)
    {
        vector<Z> z(NTT_LIM);
        for (int i = max<int>(0, s - (int)y.size() + 1);
             i <= s && i < (int)x.size(); i++)
        {
            int j = s - i;
            for (int k = 0; k < NTT_LIM; k++)
            {
                z[k] += x[i][k] * y[j][k];
            }
        }
        idft(z);
        int offset = s * block;
        int len = min<int>(NTT_LIM, ans.size() - offset);
        for (int k = 0; k < len; k++)
        {
            ans[offset + k] += z[k];
        }
    }
    return ans;
}
```

下面的普通二进制蝶形版先做位逆序置换、再逐层 radix-2 蝶形，代码直观且频域值是自然顺序，适合调试或需要观察点值的场合。它是固定模数 `998244353` 的独立板子，只使用 `vector<int>`，不依赖 `Z`；系数可以是任意整数，卷积入口会先规约到模数范围。它与卡常版的中间排列不同，两者的正逆变换不能交叉配对。

```
namespace plain_ntt
{
constexpr int mod = 998244353, G = 3;

int qpow(int a, int b)
{
    // a 是底数，b 是非负指数；返回 a^b mod 998244353。
    int r = 1;
    for (; b; b >>= 1, a = (long long)a * a % mod)
    {
        if (b & 1) r = (long long)r * a % mod;
    }
    return r;
}

void dft(vector<int> &a, bool inv = false)
{
    // a 的长度是正的 2 的幂；原地完成自然顺序的正/逆 NTT。
    int n = a.size();
    assert(n > 0 && has_single_bit((unsigned long long)n)); // 调试检查，可删。
    for (int i = 1, j = 0; i < n; i++)
```

```
    {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
        {
            j ^= bit;
        }
        if ((j ^= bit) > i) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1)
    {
        int wlen = qpow(G, (mod - 1) / len);
        if (inv) wlen = qpow(wlen, mod - 2);
        for (int i = 0; i < n; i += len)
        {
            int w = 1;
            for (int j = 0; j < len / 2; j++)
            {
                int u = a[i + j], v = (long long)a[i + j + len / 2] * w % mod;
                a[i + j] = u + v < mod ? u + v : u + v - mod;
                a[i + j + len / 2] = u - v < 0 ? u - v + mod : u - v;
                w = (long long)w * wlen % mod;
            }
        }
    }
    if (inv)
    {
        int iv = qpow(n, mod - 2);
        for (int &x : a)
        {
            x = (long long)x * iv % mod;
        }
    }
}

void idft(vector<int> &a)
{
    // a 是自然顺序的频域值；原地完成逆 NTT 并除以长度。
    dft(a, true);
}

vector<int> conv(vector<int> a, vector<int> b)
{
    // a、b 按低次到高次保存系数；返回模 mod 的普通线性卷积。
    if (a.empty() || b.empty()) return {};
    int s = a.size() + b.size() - 1, n = 1;
    while (n < s)
    {
        n <<= 1;
    }
    for (int &x : a)
    {
        x = (x % mod + mod) % mod;
```

```
    }
    for (int &x : b)
    {
        x = (x % mod + mod) % mod;
    }
    a.resize(n), b.resize(n);
    dft(a), dft(b);
    for (int i = 0; i < n; i++)
    {
        a[i] = (long long)a[i] * b[i] % mod;
    }
    idft(a), a.resize(s);
    return a;
}
}
```

在 AMD Ryzen 9 7945HX、g++ 14.2.0、`-O2` 下，两边等长且系数独立随机，取四轮单次卷积的中位耗时（输入长 n 时实际变换长 $2n$）：

每边系数数 n	radix-4 卡常版	radix-2 普通版	加速比
32768	3.91 ms	5.47 ms	1.40x
131072	17.39 ms	25.26 ms	1.45x
262144	35.62 ms	52.69 ms	1.48x

同机、GCC 14.2.0、`-std=c++20 -O2 -DNDEBUG` 下，固定种子 `20260816`，每轮反复执行同样数量的正逆变换并取 7 次中位数；构造根表不计入计时：

变换长度	正逆对数	`FastNTT`	`NTT32`	加速比
64	32768	27.218 ms	15.383 ms	1.77x
256	8192	30.397 ms	18.495 ms	1.64x
1024	2048	35.486 ms	22.294 ms	1.59x
4096	512	41.219 ms	25.659 ms	1.61x
16384	128	46.546 ms	31.552 ms	1.48x
65536	32	52.093 ms	39.809 ms	1.31x
131072	16	54.827 ms	44.594 ms	1.23x

保留普通版是为了调试和自然顺序频域值，不把本机常数差异当成渐进复杂度差异。默认用全局 `dft`、`idft`、`conv`，调试时用 `plain_ntt::dft/idft/conv`；`NTT32` 只在调用方已经采用原始 32 位切片、固定小变换和复用工作区时使用。`FastNTT` 与普通版的变换长度须为正的 2 的幂且不超过 2^{23}，`NTT32` 的实际长度须整除构造时最大长度且不超过 2^{17}。长度 N 的变换为 $O(N\log N)$，卷积为 $O((n+m)\log(n+m))$，工作空间为 $O(N)$。模数不含所需单位根时用 10.1.004 MTT 任意模数卷积；按位卷积用 10.1.006 FWT / FST。

下面核对长度 100000 的确定性系数与滚动校验和：

```
signed main()
{
    int n = 100000;
    vector<Z> a(n), b(n);
    for (int i = 0; i < n; i++)
    {
        a[i] = (i * i + 3 * i + 7) % mod;
        b[i] = (5 * i * i + 11) % mod;
    }
    auto c = conv(a, b);
    assert(c[0].val() == 77 && c[99999].val() == 971178005 && c[199998].val() == 532417205);
    unsigned long long h = 0;
    for (int i = 0; i < (int)c.size(); i++)
    {
        h = h * 1000003 + c[i].val() + i;
    }
    assert(h == 6385078395933506484ULL);
}
```

10.1.004 MTT 任意模数卷积

目标模数不含 NTT 需要的 2 的幂阶单位根时，`convMod` 先规范系数。短边不超过 64 时直接枚举；模数不超过 10^9+7 且变换长度不超过 2^{20} 时，分别在三个 NTT 素数下卷积，再用 Garner 重建后对目标模数规约，三个素数的乘积严格大于该范围内单个卷积系数上界。其余范围仅在误差界允许时用复数 FFT：把每个系数按 $B=2^{15}$ 拆成高低两段，分别卷积后重组；无法证明浮点安全时保守退回朴素实现。

```
namespace arbModConv
{
struct NTT
{
    int mod, root; // mod 是素数模数，root 是其原根。

    int pow(int a, int n) const
    {
        // a 是模 mod 剩余，n 是非负指数；返回 a^n mod mod。
        unsigned long long ans = 1;
        while (n)
        {
            if (n & 1)
            {
                ans = ans * a % mod;
            }
            a = (unsigned long long)a * a % mod;
            n >>= 1;
        }
        return ans;
    }

    void dft(vector<int> &a, bool inv) const
    {
```

```
        // a 长度是支持范围内的 2 的幂；inv 为 false/true 时原地正/逆 NTT。
        int n = a.size();
        for (int i = 1, j = 0; i < n; i++)
        {
            int b = n >> 1;
            for (; j & b; b >>= 1)
            {
                j ^= b;
            }
            j ^= b;
            if (i < j)
            {
                swap(a[i], a[j]);
            }
        }
        for (int len = 2; len <= n; len <<= 1)
        {
            int w = pow(root, (mod - 1) / len);
            if (inv)
            {
                w = pow(w, mod - 2);
            }
            for (int l = 0; l < n; l += len)
            {
                unsigned long long cur = 1;
                for (int i = 0; i < len / 2; i++)
                {
                    int x = a[l + i];
                    int y = cur * a[l + i + len / 2] % mod;
                    a[l + i] = x + y < mod ? x + y : x + y - mod;
                    a[l + i + len / 2] = x >= y ? x - y : x + mod - y;
                    cur = cur * w % mod;
                }
            }
        }
        if (inv)
        {
            int iv = pow(n, mod - 2);
            for (auto &x : a)
            {
                x = (unsigned long long)x * iv % mod;
            }
        }
    }

    vector<int> conv(const vector<unsigned long long> &x,
                     const vector<unsigned long long> &y) const
    {
        // x、y 是普通整数系数；返回它们在 mod 下的线性卷积。
        int need = x.size() + y.size() - 1;
        int n = bit_ceil((unsigned long long)need);
        vector<int> a(n), b(n);
```

```
        for (int i = 0; i < (int)x.size(); i++)
        {
            a[i] = x[i] % mod;
        }
        for (int i = 0; i < (int)y.size(); i++)
        {
            b[i] = y[i] % mod;
        }
        dft(a, false);
        dft(b, false);
        for (int i = 0; i < n; i++)
        {
            a[i] = (unsigned long long)a[i] * b[i] % mod;
        }
        dft(a, true);
        a.resize(need);
        return a;
    }
};

inline int inv(int a, int mod)
{
    // a 与素数 mod 互素；返回 a 在模 mod 下的逆元。
    unsigned long long ans = 1, b = a;
    int e = mod - 2;
    while (e)
    {
        if (e & 1)
        {
            ans = ans * b % mod;
        }
        b = b * b % mod;
        e >>= 1;
    }
    return ans;
}
}

long long normConv(long long x, long long mod)
{
    // mod 是正模数；返回 x 在 [0,mod) 内的代表元。
    x %= mod;
    if (x < 0)
    {
        x += mod;
    }
    return x;
}

vector<long long> convNaiveMod(const vector<long long> &a, const vector<long long> &b,
                               long long mod)
{
```

```cpp
    // a、b 已规范到 [0,mod)，mod 是正模数；用 128 位乘加返回精确模卷积，任一输入为空时返回空。
    assert(mod > 0); // 调试检查，可删。
    if (a.empty() || b.empty())
    {
        return {};
    }
    vector<long long> c(a.size() + b.size() - 1);
    for (int i = 0; i < (int)a.size(); i++)
    {
        for (int j = 0; j < (int)b.size(); j++)
        {
            c[i + j] = ((__int128)c[i + j] + (__int128)a[i] * b[j]) % mod;
        }
    }
    return c;
}

vector<long long> convNTTMod(const vector<long long> &a, const vector<long long> &b,
                             long long mod)
{
    // a、b 已规范到 [0,mod)，mod 不超过 1E9+7；用三个 NTT 素数返回精确模卷积。
    assert(0 < mod && mod <= 1000000007); // 调试检查，可删。
    if (a.empty() || b.empty())
    {
        return {};
    }
    int need = a.size() + b.size() - 1;
    int n = bit_ceil((unsigned long long)need);
    assert(n <= (1 << 20)); // 调试检查，可删；所选三个素数都支持该长度。
    constexpr array<int, 3> prime{998244353, 985661441, 943718401};
    constexpr array<int, 3> root{3, 3, 7};
    vector<unsigned long long> x(a.begin(), a.end()), y(b.begin(), b.end());
    array<vector<int>, 3> rem;
    for (int k = 0; k < 3; k++)
    {
        rem[k] = arbModConv::NTT{prime[k], root[k]}.conv(x, y);
    }
    unsigned long long inv01 = arbModConv::inv(prime[0] % prime[1], prime[1]);
    unsigned long long p01mod2 = (unsigned long long)prime[0] * prime[1] % prime[2];
    unsigned long long inv012 = arbModConv::inv(p01mod2, prime[2]);
    unsigned long long p0mod = prime[0] % mod;
    unsigned long long p01mod = p0mod * (prime[1] % mod) % mod;
    vector<long long> ans(need);
    for (int i = 0; i < need; i++)
    {
        unsigned long long c0 = rem[0][i];
        unsigned long long c1 = (rem[1][i] + prime[1] - c0 % prime[1]) % prime[1] * inv01 %
            prime[1];
        unsigned long long seen2 = (c0 + (unsigned long long)prime[0] * c1) % prime[2];
        unsigned long long c2 = (rem[2][i] + prime[2] - seen2) % prime[2] * inv012 % prime[2];
        ans[i] = (c0 % mod + p0mod * (c1 % mod) + p01mod * (c2 % mod)) % mod;
    }
```

```
    return ans;
}

vector<long long> convMod(vector<long long> a, vector<long long> b, long long mod)
{
    // a、b 是整数系数，mod 是正模数；返回最小非负代表元组成的线性卷积。
    assert(mod > 0); // 调试检查，可删。
    if (a.empty() || b.empty())
    {
        return {};
    }
    for (auto &x : a)
    {
        x = normConv(x, mod);
    }
    for (auto &x : b)
    {
        x = normConv(x, mod);
    }
    if (min(a.size(), b.size()) <= 64)
    {
        return convNaiveMod(a, b, mod);
    }
    int s = a.size() + b.size() - 1;
    int n = bit_ceil((unsigned long long)s);
    if (mod <= 1000000007 && n <= (1 << 20))
    {
        return convNTTMod(a, b, mod);
    }
    const long long B = 1 << 15;
    unsigned __int128 t = min(a.size(), b.size());
    unsigned __int128 hi = (mod - 1) / B;
    unsigned __int128 lim = (unsigned __int128)1 << 48;
    auto bounded = [&](unsigned __int128 x, unsigned __int128 y, unsigned __int128 z)
    {
        if (x && y > lim / x)
        {
            return false;
        }
        x *= y;
        return !x || z <= lim / x;
    };
    bool safe = numeric_limits<long double>::digits >= 64 && n <= (1 << 20) &&
                bounded(t, B - 1, B - 1) && bounded(t, 2 * (B - 1), hi) &&
                bounded(t, hi, hi);
    if (!safe)
    {
        return convNaiveMod(a, b, mod);
    }
    vector<long double> a0(a.size());
    vector<long double> a1(a.size());
    vector<long double> b0(b.size());
```

```
    vector<long double> b1(b.size());
    for (int i = 0; i < (int)a.size(); i++)
    {
        a0[i] = a[i] % B;
        a1[i] = a[i] / B;
    }
    for (int i = 0; i < (int)b.size(); i++)
    {
        b0[i] = b[i] % B;
        b1[i] = b[i] / B;
    }
    auto c0 = convolution(a0, b0);
    auto c1 = convolution(a0, b1);
    auto q = convolution(a1, b0);
    auto c2 = convolution(a1, b1);
    vector<long long> c(c0.size());
    for (int i = 0; i < (int)c.size(); i++)
    {
        long long x = llroundl(c0[i]) % mod;
        long long y = (llroundl(c1[i]) + llroundl(q[i])) % mod;
        long long z = llroundl(c2[i]) % mod;
        __int128 v = x;
        v += (__int128)y * B % mod;
        v += (__int128)z * B % mod * B % mod;
        c[i] = v % mod;
    }
    return c;
}
```

依赖 10.1.002 FFT 的 `convolution`，调用 `convMod(a,b,p)`；输入允许负数、`p` 须为正，任一输入为空时返回空，否则返回长度 $n+m-1$。三素数路径适用于 $p \le 10^9+7$、补齐长度 $N \le 2^{20}$，时间 $O(N \log N)$、峰值空间 $O(N)$，且因 $N(p-1)^2$ 小于三个素数乘积而能精确重建。其余快速路径要求 `long double` 至少有 64 位尾数，并按 $t=\min(n,m)$、$H=\lfloor (p-1)/2^{15} \rfloor$ 检查三段卷积的绝对系数上界均不超过 2^{48}；无法证明安全时才退回 `__int128` 朴素模卷积的 $O(nm)$。Windows GCC 的 `long double` 可能只有 53 位尾数，但常用十亿级模数仍走精确三素数路径，不会退化为平方复杂度。

下面检查负系数、模数 1、普通卷积和接近 `LLONG_MAX` 时的精确回退：

```
signed main()
{
    assert((convMod({-1, 2}, {3, -4}, 5) == vector<long long>{2, 0, 2}));
    assert((convMod({7, -8}, {9}, 1) == vector<long long>{0, 0}));
    assert((convMod({1, 2, 3}, {4, 5}, 1000000007) == vector<long long>{4, 13, 22, 15}));
    long long big = numeric_limits<long long>::max();
    assert(convMod({big - 1}, {big - 1}, big)[0] == 1);
}
```

10.1.005 二元 NTT / 二元多项式卷积

二维卷积用 Kronecker 打包降成一次一维卷积。令结果列数 $C = c_a + c_b - 1$，把二维下标 (i, j) 映到 $iC + j$；任意乘积项的列指数满足 $j_a + j_b < C$，不会向下一行产生进位，因此一维卷积下标恰好还原为二维次数。`poly2::Mat` 用连续行主序保存二维系数；普通 `poly2::conv(a,b)` 调一次公共 `conv`，三参数重载同时求 `a*c` 与 `b*c`，共享右因子 `c` 的正变换，只需三次正变换和两次逆变换。10.2.011 Power Projection 使用这个真实的第二调用场景，因此该连续矩阵层不是一次性包装。

```
// 依赖 1.1.008 静态取模类
namespace poly2
{
struct Mat
{
    int n = 0; // 二维系数表的行数。
    int m = 0; // 二维系数表的列数。
    vector<Z> a; // 按行连续保存的 n*m 个系数。

    Mat(int r = 0, int c = 0) : n(r), m(c), a(r * c)
    {
        // r、c 是二维系数表的行数与列数；构造全零系数表。
    }

    Z &operator()(int i, int j)
    {
        // i、j 是行列下标；返回对应系数的可修改引用。
        return a[i * m + j];
    }

    const Z &operator()(int i, int j) const
    {
        // i、j 是行列下标；返回对应系数的只读引用。
        return a[i * m + j];
    }
};

vector<Z> pack(const Mat &a, int w)
{
    // a 是行主序二维系数表，w 不小于其列数；返回以 w 为行跨度的 Kronecker 打包结果。
    if (a.n == 0 || a.m == 0)
    {
        return {};
    }
    assert(w >= a.m); // 调试检查，可删。
    vector<Z> x((a.n - 1) * w + a.m);
    for (int i = 0; i < a.n; i++)
    {
        for (int j = 0; j < a.m; j++)
        {
            x[i * w + j] = a(i, j);
        }
    }
```

```cpp
    return x;
}

Mat unpack(const vector<Z> &a, int n, int m)
{
    // a 是行跨度为 m 的 Kronecker 系数，n、m 是目标形状；返回对应行主序二维系数表。
    if (n == 0 || m == 0)
    {
        return {};
    }
    assert((int)a.size() >= n * m); // 调试检查，可删。
    Mat c(n, m);
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            c(i, j) = a[i * m + j];
        }
    }
    return c;
}

Mat conv(const Mat &a, const Mat &b)
{
    // a、b 是行主序二维系数表；用 Kronecker 打包返回二维线性卷积。
    if (a.n == 0 || a.m == 0 || b.n == 0 || b.m == 0)
    {
        return {};
    }
    int n = a.n + b.n - 1;
    int m = a.m + b.m - 1;
    auto c = ::conv(pack(a, m), pack(b, m));
    c.resize(n * m);
    return unpack(c, n, m);
}

array<Mat, 2> conv(const Mat &a, const Mat &b, const Mat &c)
{
    // a、b 是同形左因子，c 是公共右因子；返回 {a*c,b*c}，两次卷积共享 c 的正变换。
    assert(a.n == b.n && a.m == b.m); // 调试检查，可删。
    if (a.n == 0 || a.m == 0 || c.n == 0 || c.m == 0)
    {
        return {};
    }
    int n = a.n + c.n - 1;
    int m = a.m + c.m - 1;
    vector<Z> x = pack(a, m);
    vector<Z> y = pack(b, m);
    vector<Z> z = pack(c, m);
    if (min({x.size(), y.size(), z.size()}) <= 64)
    {
        return {unpack(::conv(x, z), n, m), unpack(::conv(y, z), n, m)};
```

```
    }
    int s = max(x.size() + z.size() - 1, y.size() + z.size() - 1);
    assert(nttFit(x.size(), z.size()) && nttFit(y.size(), z.size())); // 调试检查，可删。
    int len = bit_ceil((unsigned long long)s);
    x.resize(len);
    y.resize(len);
    z.resize(len);
    dft(x);
    dft(y);
    dft(z);
    for (int i = 0; i < len; i++)
    {
        x[i] *= z[i];
        y[i] *= z[i];
    }
    idft(x);
    idft(y);
    x.resize(n * m);
    y.resize(n * m);
    return {unpack(x, n, m), unpack(y, n, m)};
}
}

vector<vector<Z>> conv2(const vector<vector<Z>> &a, const vector<vector<Z>> &b)
{
    // a、b 是矩形二维系数表；返回二维线性卷积，任一维为空时返回空。
    if (a.empty() || b.empty())
    {
        return {};
    }
    int m1 = a[0].size();
    int m2 = b[0].size();
    for (const auto &r : a)
    {
        assert((int)r.size() == m1); // 调试检查，可删。
    }
    for (const auto &r : b)
    {
        assert((int)r.size() == m2); // 调试检查，可删。
    }
    if (m1 == 0 || m2 == 0)
    {
        return {};
    }
    poly2::Mat x(a.size(), m1), y(b.size(), m2);
    for (int i = 0; i < x.n; i++)
    {
        for (int j = 0; j < x.m; j++)
        {
            x(i, j) = a[i][j];
        }
    }
```

```
    for (int i = 0; i < y.n; i++)
    {
        for (int j = 0; j < y.m; j++)
        {
            y(i, j) = b[i][j];
        }
    }
    auto z = poly2::conv(x, y);
    vector<vector<Z>> ans(z.n, vector<Z>(z.m));
    for (int i = 0; i < z.n; i++)
    {
        for (int j = 0; j < z.m; j++)
        {
            ans[i][j] = z(i, j);
        }
    }
    return ans;
}
```

依赖 10.1.003 NTT 的 `conv`、`dft` 和 `idft`。外部接口 `conv2(a,b)` 接受矩形 `vector<vector<Z>>`；内部需要连续布局或共享右因子时使用 `poly2::Mat`。任一输入没有行或列时返回空，否则结果两维各为输入之和减一。设输出矩形为 $R \times C$，打包长度为 $\Theta(RC)$，时间 $O(RC\log(RC))$、额外空间 $O(RC)$；三参数重载与两次独立二维卷积同阶，但少做一次公共右因子的正变换。实际规模仍受一维 NTT 的 2^{23} 长度上限约束。

10.1.006 FWT / FST

FWT 把按位卷积对角化。异或用 Hadamard 蝶形 $(x, y) \mapsto (x + y, x - y)$，逆变换同形再除以 n；或卷积是子集和（高维前缀和）$a_{i+h} \mathrel{+}= a_i$，逆用减法；与卷积是超集和，方向相反。三种变换各配各的逆，绝不能混用。公开接口按方向拆成 `orFwt`/`orConv`、`andFwt`/`andConv`、`xorFwt`/`xorConv`；`BitOp` 和 `bitConv` 仅保留兼容旧代码。`orFWT` 还提供按掩码优先、每个掩码连续保存 d 个通道的批量入口；正逆方向作为模板参数放到内层循环之外，四参数重载只承担兼容旧调用的单次分派。子集卷积还要求两部分不交，不属于这三个逐位卷积，见 10.1.011 子集卷积。

```
// 依赖 1.1.008 静态取模类
enum class BitOp
{
    Or,
    And,
    Xor
};

template <bool Inv = false, class T> void orFWT(T *a, int n, int d)
{
    // a 指向 n 行、每行 d 项的连续集合函数，n 是 2 的幂；沿行掩码做批量 OR 变换。
    assert(a && n > 0 && d > 0 && n <= numeric_limits<int>::max() && d <=
    ↪  numeric_limits<int>::max() &&
           has_single_bit((unsigned long long)n)); // 调试检查，可删。
    int m = n;
```

```
    int w = d;
    for (int h = 1; h < m; h <<= 1)
    {
        for (int l = 0; l < m; l += h << 1)
        {
            for (int s = 0; s < h; s++)
            {
                T *x = a + (size_t)(l + s) * w;
                T *y = x + (size_t)h * w;
                if constexpr (!Inv)
                {
                    for (int k = 0; k < w; k++)
                    {
                        y[k] += x[k];
                    }
                }
                else
                {
                    for (int k = 0; k < w; k++)
                    {
                        y[k] -= x[k];
                    }
                }
            }
        }
    }
}

template <class T> void orFWT(T *a, int n, int d, bool inv)
{
    // a、n、d 的含义同三参数版本；inv 指定正变换或逆变换。
    if (inv)
    {
        orFWT<true>(a, n, d);
    }
    else
    {
        orFWT<false>(a, n, d);
    }
}

template <bool Inv = false, class T> void orFWT(vector<T> &a)
{
    // a 是按掩码编号的一维集合函数；原地完成 OR 正变换或逆变换。
    assert(!a.empty() && a.size() <= numeric_limits<int>::max() && has_single_bit(a.size()));
    ↪ // 调试检查，可删。
    int n = a.size();
    for (int h = 1; h < n; h <<= 1)
    {
        for (int i = 0; i < n; i += h << 1)
        {
            for (int j = 0; j < h; j++)
```

```
            {
                if constexpr (!Inv)
                {
                    a[i + j + h] += a[i + j];
                }
                else
                {
                    a[i + j + h] -= a[i + j];
                }
            }
        }
    }
}

template <class T> void fwt(vector<T> &a, BitOp op, bool inv = false)
{
    // a 是按掩码编号的系数，op 指定或、与或异或，inv 指定正变换或逆变换；原地完成变换。
    int n = a.size();
    assert(n > 0 && has_single_bit((unsigned long long)n)); // 调试检查，可删。
    if (op == BitOp::Or)
    {
        if (inv)
        {
            orFWT<true>(a);
        }
        else
        {
            orFWT<false>(a);
        }
        return;
    }
    for (int h = 1; h < n; h <<= 1)
    {
        for (int i = 0; i < n; i += h << 1)
        {
            for (int j = 0; j < h; j++)
            {
                T x = a[i + j];
                T y = a[i + j + h];
                if (op == BitOp::Xor)
                {
                    a[i + j] = x + y;
                    a[i + j + h] = x - y;
                }
                else
                {
                    if (!inv)
                    {
                        a[i + j] += y;
                    }
                    else
                    {
```

```
                        a[i + j] -= y;
                    }
                }
            }
        }
    }
    if (op == BitOp::Xor && inv)
    {
        for (T &x : a)
        {
            x /= T(n);
        }
    }
}

template <class T> vector<T> bitConv(vector<T> a, vector<T> b, BitOp op)
{
    // a、b 是等长的掩码系数，op 指定卷积下标使用或、与或异或；返回对应的按位卷积。
    assert(!a.empty() && a.size() == b.size()); // 调试检查，可删。
    fwt(a, op);
    fwt(b, op);
    for (int i = 0; i < (int)a.size(); i++)
    {
        a[i] *= b[i];
    }
    fwt(a, op, true);
    return a;
}
```

模数是编译期已知的奇素数时，可把通用模整数换成固定 32 位剩余并让变换长度参与模板实例化，消除蝶形里的运行时运算分派：

```
template <int P> int fwtAdd(int x, int y)
{
    // x、y 在 [0,P) 内；返回 (x+y) mod P。
    x += y;
    if (x >= P)
    {
        x -= P;
    }
    return x;
}

template <int P> int fwtSub(int x, int y)
{
    // x、y 在 [0,P) 内；返回 (x-y) mod P。
    x += P - y;
    if (x >= P)
    {
        x -= P;
    }
```

```
    return x;
}

template <int P, BitOp Op, bool Inv, int N> void fwtRec(int *a)
{
    // a 指向长 N 的数组；编译期长度递归完成 Op 型 FWT（Inv 为真时做逆变换），原地修改。
    constexpr int h = N / 2;
    if constexpr (h)
    {
        fwtRec<P, Op, Inv, h>(a);
        fwtRec<P, Op, Inv, h>(a + h);
        for (int i = 0; i < h; i++)
        {
            int x = a[i], y = a[i + h];
            if constexpr (Op == BitOp::Or)
            {
                a[i + h] = Inv ? fwtSub<P>(y, x) : fwtAdd<P>(x, y);
            }
            else if constexpr (Op == BitOp::And)
            {
                a[i] = Inv ? fwtSub<P>(x, y) : fwtAdd<P>(x, y);
            }
            else
            {
                int s = fwtAdd<P>(x, y);
                int d = fwtSub<P>(x, y);
                if constexpr (Inv)
                {
                    s = (s + (s & 1) * P) >> 1;
                    d = (d + (d & 1) * P) >> 1;
                }
                a[i] = s;
                a[i + h] = d;
            }
        }
    }
}

template <int P, BitOp Op, bool Inv = false, int N = 1>
void fwtMod(int *a, int n)
{
    // a 指向长 n 的数组，n 是 2 的幂；把运行期 n 分派到相同编译期长度的 fwtRec。
    static_assert(1 < P && P < (1U << 31));
    static_assert(Op != BitOp::Xor || (P & 1));
    assert(a && n && has_single_bit((unsigned long long)n)); // 调试检查，可删
    if constexpr (N <= (1U << 30))
    {
        if (n == N)
        {
            fwtRec<P, Op, Inv, N>(a);
        }
        else
```

```
        {
            fwtMod<P, Op, Inv, N << 1>(a, n);
        }
    }
}

template <int P, BitOp Op, bool Inv = false> void fwtMod(vector<int> &a)
{
    fwtMod<P, Op, Inv>(a.data(), a.size());
}

template <int P, BitOp Op>
vector<int> bitConvMod(vector<int> a, vector<int> b)
{
    // a、b 是等长（2 的幂）且值在 [0,P) 的数组；返回位运算卷积 c[i op j] += a[i]b[j] 模 P。
    assert(!a.empty() && a.size() == b.size()); // 调试检查，可删
    fwtMod<P, Op>(a);
    fwtMod<P, Op>(b);
    for (int i = 0; i < a.size(); i++)
    {
        a[i] = (unsigned long long)a[i] * b[i] % P;
    }
    fwtMod<P, Op, true>(a);
    return a;
}
```

`fwt(a,op,inv)` 的 `op` 取 `BitOp::Or`、`BitOp::And` 或 `BitOp::Xor`；正变换后逐点相乘，再用同一枚举值逆变换。异或逆变换要求长度在系数域中可逆。`orFWT<Inv>(a,n,d)` 要求 n 为二次幂，连续区间恰有 nd 项；时间 $O(nd\log n)$、额外空间 $O(1)$。`bitConv` 封装通用流程。固定模热点调用 `fwtMod<P,Op,Inv>(a)` 或 `bitConvMod<P,Op>(a,b)`，输入必须已经落在 $[0,P)$；$P<2^{31}$，异或还要求 P 为奇数。本机长度 2^{20} 的 AND 正逆变换中，固定模版 7 轮中位数约为 `4ms`，通用 `MInt` 版约为 `62ms`；代价是代码体积更大且不支持运行时模数。两版渐进复杂度均为 $O(N\log N)$。稀疏因子异或卷积可用附带的 `sparseXor(n,factors)`，其模数固定为 `1000000000000125953`，以局部 Walsh、平方根分解和全局逆变换做到 $O(2^n n+\sum 2^d d)$；它不替代普通 FWT。2 在模下不可逆时改用 10.1.007 偶模 FWT 与光速乘，每位取值超过 2 种用 10.1.008 K 进制 FWT，要求不交拆分时用 10.1.011 子集卷积。

下面用同一组四项系数直接核对三种卷积；这些结果可手算复查：

```
signed main()
{
    vector<Z> a{1, 2, 3, 4};
    vector<Z> b{5, 6, 7, 8};
    assert((bitConv(a, b, BitOp::Xor) == vector<Z>{70, 68, 62, 60}));
    assert((bitConv(a, b, BitOp::Or) == vector<Z>{5, 28, 43, 184}));
    assert((bitConv(a, b, BitOp::And) == vector<Z>{103, 52, 73, 32}));
    cout << "OK\n";
    return 0;
}
```

10.1.007 偶模 FWT 与光速乘

偶模数下 2 不可逆，普通异或逆 FWT 末尾的除以 n 会失败。把模数临时提升到 $M = n \cdot mod$，在 $\mathbb{Z}/M\mathbb{Z}$ 上做正 FWT、逐点乘、再做一次正 FWT；异或 FWT 的平方是 nI，所得每项在 $[0, M)$ 内都恰能被 n 整除，先作整数除法再模 mod 即可。下面用 `unsigned __int128` 保存 M，模加减避免先形成可能溢出的和，模乘在 $M > 2^{64}$ 时改用倍增加法。

```
unsigned __int128 addMod128(unsigned __int128 x, unsigned __int128 y,
                            unsigned __int128 mod)
{
    // x、y 均小于正模数 mod；返回 (x+y) mod mod，计算中不发生无符号回绕。
    return x >= mod - y ? x - (mod - y) : x + y;
}

unsigned __int128 subMod128(unsigned __int128 x, unsigned __int128 y,
                            unsigned __int128 mod)
{
    // x、y 均小于正模数 mod；返回 (x-y) mod mod。
    return x >= y ? x - y : mod - (y - x);
}

unsigned __int128 mulMod128(unsigned __int128 x, unsigned __int128 y,
                            unsigned __int128 mod)
{
    // x、y 均小于正模数 mod；返回 (x*y) mod mod。
    if (mod <= ((unsigned __int128)1 << 64))
    {
        return x * y % mod;
    }
    unsigned __int128 r = 0;
    while (y)
    {
        if (y & 1)
        {
            r = addMod128(r, x, mod);
        }
        y >>= 1;
        if (y)
        {
            x = addMod128(x, x, mod);
        }
    }
    return r;
}

void xorFwt(vector<unsigned __int128> &a, unsigned __int128 mod)
{
    // a 的各项均小于正模数 mod；原地完成异或 FWT。
    int n = a.size();
    for (int len = 1; len < n; len <<= 1)
    {
```

```
        for (int l = 0; l < n; l += len << 1)
        {
            for (int i = 0; i < len; i++)
            {
                unsigned __int128 x = a[l + i];
                unsigned __int128 y = a[l + len + i];
                a[l + i] = addMod128(x, y, mod);
                a[l + len + i] = subMod128(x, y, mod);
            }
        }
    }
}

vector<long long> xorConv(const vector<long long> &a, const vector<long long> &b,
                          long long mod)
{
    // a、b 是等长整数序列，mod 是正模数；返回最小非负代表元组成的异或卷积。
    assert(mod > 0 && !a.empty() && a.size() == b.size() &&
           has_single_bit(a.size())); // 调试检查，可删。
    int n = a.size();
    unsigned __int128 lifted = (unsigned __int128)(unsigned long long)mod * n;
    vector<unsigned __int128> x(n);
    vector<unsigned __int128> y(n);
    for (int i = 0; i < n; i++)
    {
        long long u = a[i] % mod;
        long long v = b[i] % mod;
        if (u < 0)
        {
            u += mod;
        }
        if (v < 0)
        {
            v += mod;
        }
        x[i] = (unsigned long long)u;
        y[i] = (unsigned long long)v;
    }
    xorFwt(x, lifted);
    xorFwt(y, lifted);
    for (int i = 0; i < n; i++)
    {
        x[i] = mulMod128(x[i], y[i], lifted);
    }
    xorFwt(x, lifted);
    vector<long long> c(n);
    for (int i = 0; i < n; i++)
    {
        assert(x[i] % n == 0); // 调试检查，可删。
        c[i] = (long long)(x[i] / n % (unsigned long long)mod);
    }
    return c;
```

```
}
```

`xorConv(a,b,mod)` 接受负系数，要求两输入等长、非空且长度为 2 的幂，`mod` 为正的有符号 64 位整数；实现使用提升模数 $M = n \cdot mod$，要求 $M < 2^{128}$。通常 $M \le 2^{64}$，逐点模乘可直接用 `unsigned __int128`，总时间 $O(n \log n)$；更大的 M 用二进制模乘，时间为 $O(n(\log n + \log M))$。额外空间均为 $O(n)$。模数为奇数且长度在模下可逆时直接用 10.1.006 FWT / FST 更短。

下面分别覆盖普通偶模、模数 1 和提升模数超过 2^{64} 的分支：

```
signed main()
{
    assert((xorConv({1, 2, 3, 0}, {3, -1, 2, 1}, 4) ==
            vector<long long>{3, 0, 1, 2}));
    assert((xorConv({7, -8}, {9, 10}, 1) == vector<long long>{0, 0}));
    long long m = numeric_limits<long long>::max() - 1;
    assert((xorConv({m - 1, m - 2}, {m - 3, m - 4}, m) ==
            vector<long long>{11, 10}));
    cout << "OK\n";
    return 0;
}
```

10.1.008 K 进制 FWT

把每位两种取值推广到 k 种：在每一位上做 k 点 DFT，使逐位模 k 加法（无进位）的循环卷积对角化。系数域必须含 k 次单位根 $\omega_k = g^{(P-1)/k}$，即要求 $k \mid P-1$；逆变换用 ω_k^{-1} 并除以总长 n。

```
// 依赖 1.1.008 静态取模类
void kFWT(vector<Z> &a, int k, bool inv = false)
{
    // a 按 k 进制向量编号，k 是每一位的取值数；inv 指定正变换或逆变换，结果写回 a。
    assert(!a.empty() && k > 1 && (Z::mod() - 1) % k == 0); // 调试检查，可删。
    int n = a.size();
    int q = n;
    while (q % k == 0)
    {
        q /= k;
    }
    assert(q == 1); // 调试检查，可删。
    Z w = Z(3).pow((Z::mod() - 1) / k);
    if (inv)
    {
        w = w.inv();
    }
    vector<Z> pw(k, 1);
    vector<Z> x(k);
    vector<Z> y(k);
    for (int i = 1; i < k; i++)
    {
        pw[i] = pw[i - 1] * w;
    }
```

```
    for (int len = 1; len < n; len *= k)
    {
        for (int l = 0; l < n; l += len * k)
        {
            for (int p = 0; p < len; p++)
            {
                for (int i = 0; i < k; i++)
                {
                    x[i] = a[l + i * len + p];
                }
                fill(y.begin(), y.end(), Z(0));
                for (int i = 0; i < k; i++)
                {
                    for (int j = 0; j < k; j++)
                    {
                        y[i] += x[j] * pw[i * j % k];
                    }
                }
                for (int i = 0; i < k; i++)
                {
                    a[l + i * len + p] = y[i];
                }
            }
        }
    }
    if (inv)
    {
        Z q = Z(n).inv();
        for (Z &x : a)
        {
            x *= q;
        }
    }
}
```

本章公共前导固定 $P = 998244353$、原根 3。调用 `kFWT(a,k,inv)` 时长度须为 k 的非负整数次幂且 $k \mid P-1$；正变换后逐点相乘，再以 `inv=true` 逆变换。预存一轮单位根幂后，长度 N 的变换为 $O(kN\log_k N + k\log P)$ 时间、$O(k)$ 临时空间。$k=2$ 即退化为 10.1.006 FWT / FST 的异或卷积。

10.1.009 全在线卷积：基准版与 NTT 版

两序列系数同步逐项到达时，第 n 次加入后要立即得到 $c_n = \sum_{i=0}^{n} a_i b_{n-i}$。`OnlineConv` 直接枚举本项，适合小规模；`OnlineNTT` 按 p+2 的最低位把新贡献分成二进制块，复用 10.1.003 NTT 批量更新当前及未来答案，仍在每次 add 结束时立即返回 c_p。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.1.003 NTT
template <class T> struct OnlineConv
{
    vector<T> a; // 第一序列已经加入的系数。
```

```
    vector<T> b; // 第二序列已经加入的系数。
    vector<T> c; // 从第 0 项到当前项的卷积系数。

    T add(T x, T y)
    {
        // x、y 分别是两序列新到达的系数；追加二者并返回刚确定的卷积项。
        int n = a.size();
        a.push_back(x);
        b.push_back(y);
        c.push_back({});
        for (int i = 0; i <= n; i++)
        {
            c[n] += a[i] * b[n - i];
        }
        return c[n];
    }
};

struct OnlineNTT
{
    // a、b 是已加入系数，h 是已经批量累加的卷积项。
    vector<Z> a, b, h, x, y;
    // fa[z]、fb[z] 保存长度 2^(z+1) 的前缀变换。
    vector<vector<Z>> fa, fb;
    int p = 0;

    Z add(Z u, Z v)
    {
        // u、v 是下标 p 的新系数；追加后立即返回第 p 项卷积。
        a.push_back(u);
        b.push_back(v);
        int z = countr_zero((unsigned long long)(p + 2));
        int w = 1LL << z;
        int s;
        assert(2 * w <= (1 << 23)); // 调试检查，可删：受当前 NTT 最大长度限制。
        if (p + 2 == w)
        {
            x = a;
            x.resize(2 * w);
            dft(x);
            fa.emplace_back(x.begin(), x.begin() + w);
            y = b;
            y.resize(2 * w);
            dft(y);
            fb.emplace_back(y.begin(), y.begin() + w);
            for (int i = 0; i < 2 * w; i++)
            {
                x[i] *= y[i];
            }
            s = w - 2;
            h.resize(2 * s + 2);
        }
```

```
        else
        {
            x.assign(a.end() - w, a.end());
            x.resize(2 * w);
            dft(x);
            y.assign(b.end() - w, b.end());
            y.resize(2 * w);
            dft(y);
            for (int i = 0; i < 2 * w; i++)
            {
                x[i] = x[i] * fb[z][i] + y[i] * fa[z][i];
            }
            s = w - 1;
        }
        idft(x);
        for (int i = 0; i <= s; i++)
        {
            h[p + i] += x[s + i];
        }
        return h[p++];
    }
};
```

两个版本的 `add(x,y)` 都立即返回新确定的卷积项。基准版第 n 项 $O(n)$、前 n 项累计 $O(n^2)$；NTT 版累计 $O(n\log^2 n)$ 时间，各层缓存长度构成几何级数，故总空间为 $O(n)$。后者仅支持模 998244353 的 `z`，且受当前 2^{23} 阶单位根限制。项数不大时基准版更短、常数也更小。

10.1.010 整除关系卷积

在下标的整除偏序上做卷积。约数方向 Zeta 变换令 $b_n=\sum_{d\mid n}a_d$，Möbius 逆用减法回收；倍数方向令 $b_n=\sum_{n\mid m}a_m$，逆同理。它们是位偏序 FWT 的数论类比，配合逐点乘可算 GCD/LCM 卷积。四个变换都按“枚举因子—枚举其倍数”推进，Zeta 与 Möbius 的循环方向相反才能保证就地正确。

```
template <class T> void divisorZeta(vector<T> &a)
{
    // a[n] 是下标 n 的系数；原地改成所有约数系数之和，a[0] 不参与。
    int n = a.size();
    for (int d = n - 1; d >= 1; d--)
    {
        for (int m = d * 2; m < n; m += d)
        {
            a[m] += a[d];
        }
    }
}

template <class T> void divMob(vector<T> &a)
{
    // a[n] 是约数 Zeta 变换后的系数；原地恢复原序列，a[0] 不参与。
```

```
    int n = a.size();
    for (int d = 1; d < n; d++)
    {
        for (int m = d * 2; m < n; m += d)
        {
            a[m] -= a[d];
        }
    }
}

template <class T> void multipleZeta(vector<T> &a)
{
    // a[n] 是下标 n 的系数；原地改成所有倍数系数之和，a[0] 不参与。
    int n = a.size();
    for (int d = 1; d < n; d++)
    {
        for (int m = d * 2; m < n; m += d)
        {
            a[d] += a[m];
        }
    }
}

template <class T> void mulMob(vector<T> &a)
{
    // a[n] 是倍数 Zeta 变换后的系数；原地恢复原序列，a[0] 不参与。
    int n = a.size();
    for (int d = n - 1; d >= 1; d--)
    {
        for (int m = d * 2; m < n; m += d)
        {
            a[d] -= a[m];
        }
    }
}
```

`divisorZeta`/`divMob` 是约数方向的正逆变换，`multipleZeta`/`mulMob` 是倍数方向。四者均 $O(n \log n)$ 时间（调和级数求和）、$O(1)$ 额外空间。位集合上的对应变换见 10.1.006 FWT / FST。

10.1.011 子集卷积

子集卷积 $c_S = \sum_{T \subseteq S} a_T b_{S \setminus T}$ 要求两部分不交且并为 S，不能用或卷积逐点乘替代（后者会重复计入有交的拆分）。技巧是引入占位维 popcount：把 a_S 放到第 $|S|$ 层，对每层做或方向子集和（zeta），在每个掩码上按层做加法卷积 $h_k = \sum_i f_i g_{k-i}$，再 Möbius 逆回收；因为不交时 $|T| + |S \setminus T| = |S|$ 恰落在第 $|S|$ 层，取该层即排除所有有交拆分。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.1.006 FWT / FST
template <class T> struct Ranked
{
```

```
    int n = 0; // 集合变量数，掩码范围是 [0,2^n)。
    vector<T> a; // 按掩码优先连续保存，每个掩码依次存放 0..n 秩。

    explicit Ranked(int m = 0) : n(m)
    {
        // m 是集合变量数；构造全零秩表。
        assert(0 <= m && m <= 30); // 调试检查，可删。
        a.resize((1LL << m) * (m + 1));
    }

    T &operator()(int s, int k)
    {
        // s 是集合掩码，k 是秩；返回对应元素的可修改引用。
        return a[s * (n + 1) + k];
    }

    const T &operator()(int s, int k) const
    {
        // s 是集合掩码，k 是秩；返回对应元素的只读引用。
        return a[s * (n + 1) + k];
    }
};

template <class T> Ranked<T> rankedZeta(const vector<T> &a)
{
    // a 是长度为 2 的幂的集合函数；返回按子集大小分层后的子集 Zeta 变换。
    assert(!a.empty() && has_single_bit(a.size())); // 调试检查，可删。
    int n = countr_zero((unsigned long long)a.size());
    int m = a.size();
    int d = n + 1;
    Ranked<T> r(n);
    for (int s = 0; s < m; s++)
    {
        r.a[(size_t)s * d + popcount((unsigned long long)s)] = a[s];
    }
    orFWT<false>(r.a.data(), m, d);
    return r;
}

template <class T> vector<T> rankedMobius(Ranked<T> r)
{
    // r 是按子集大小分层的 Zeta 变换；返回每个集合对应秩层的 Möbius 逆变换系数。
    int n = r.n;
    assert(0 <= n && n <= 30); // 调试检查，可删。
    int m = 1LL << n;
    assert((int)r.a.size() == m * (n + 1)); // 调试检查，可删。
    orFWT<true>(r.a.data(), m, n + 1);
    vector<T> a(m);
    for (int s = 0; s < (int)m; s++)
    {
        a[s] = r.a[(size_t)s * (n + 1) + popcount((unsigned long long)s)];
    }
```

```
    return a;
}

template <class T> vector<T> subsetConv(const vector<T> &a, const vector<T> &b)
{
    // a、b 是等长且长度为 2 的幂的集合函数；返回 c[S]=sum(T subset S) a[T]*b[S\T]。
    assert(a.size() == b.size() && !a.empty() && has_single_bit(a.size())); // 调试检查，可删。
    int n = countr_zero((unsigned long long)a.size());
    int m = a.size();
    int d = n + 1;
    auto f = rankedZeta(a);
    auto g = rankedZeta(b);
    for (int s = 0; s < m; s++)
    {
        T *x = f.a.data() + (size_t)s * d;
        const T *y = g.a.data() + (size_t)s * d;
        for (int k = n; k >= 0; k--)
        {
            T v{};
            for (int i = 0; i <= k; i++)
            {
                v += x[i] * y[k - i];
            }
            x[k] = v;
        }
    }
    return rankedMobius(move(f));
}
```

`Ranked` 只保存一块按掩码优先的 $(n+1)2^n$ 连续缓冲区，`rankedZeta` 与 `rankedMobius` 统一调用 10.1.006 FWT / FST 的批量公共入口；这里没有第二份私有变换。`subsetConv(a,b)` 的长度须为 2^n 且当前实现要求 $0 \le n \le 30$，`T` 支持加减乘，下标二进制位表示集合；`a={1,2,3,4}`、`b={5,6,7,8}` 得 `{5,16,22,60}`。时间 $O(n^2 2^n)$、额外空间 $O(n2^n)$。或、与、异或卷积不要求两部分不交，分别使用 10.1.006。

下面直接核对空集合和两个二进制位都参与的固定结果：

```
signed main()
{
    assert((subsetConv(vector<Z>{2}, vector<Z>{3}) == vector<Z>{6}));
    assert((subsetConv(vector<Z>{1, 2, 3, 4}, vector<Z>{5, 6, 7, 8}) ==
            vector<Z>{5, 16, 22, 60}));
    cout << "OK\n";
    return 0;
}
```

10.1.012 Min-Plus 卷积

Min-plus 卷积 $c_k = \min_i(a_i + b_{k-i})$ 一般没有 FFT 型加速，只在附加结构下才快。两序列差分都单调不降（离散凸）时，答案差分是两条输入差分的有序归并；一凸一任意时，Monge 性保证最优决策点随 k 单调，可用分治搜索每行最优。无结构时只能枚举所有合法下标对。默认输入、差分和返回值使用 `long`

long；可能超过 64 位时显式选择 Acc=__int128，不拿有限常量冒充无穷大。

```cpp
template <class F> vector<int> monoMin(int rows, int cols, F better)
{
    // better(r,x,y) 判断第 r 行的候选 y 是否不劣于 x；返回每行的最优列下标。
    assert(rows > 0 && cols > 0); // 调试检查，可删。
    vector<int> p(rows);
    auto dfs = [&](auto &&self, int u, int d, int l, int r) -> void
    {
        if (u == d)
        {
            return;
        }
        int x = (u + d) / 2;
        int y = l;
        for (int j = l + 1; j < r; j++)
        {
            if (better(x, y, j))
            {
                y = j;
            }
        }
        p[x] = y;
        self(self, u, x, l, y + 1);
        self(self, x + 1, d, y, r);
    };
    dfs(dfs, 0, rows, 0, cols);
    return p;
}

template <class Acc = long long>
vector<Acc> minPlus(const vector<Acc> &a, const vector<Acc> &b)
{
    // a、b 是任意整数序列；返回精确的 min-plus 卷积，任一输入为空时返回空。
    if (a.empty() || b.empty())
    {
        return {};
    }
    int n = a.size();
    int m = b.size();
    vector<Acc> c(n + m - 1);
    for (int s = 0; s < n + m - 1; s++)
    {
        int l = max<int>(0, s - m + 1);
        int r = min<int>(n - 1, s);
        Acc best = a[l] + b[s - l];
        for (int i = l + 1; i <= r; i++)
        {
            best = min(best, a[i] + b[s - i]);
        }
        c[s] = best;
    }
```

```cpp
    return c;
}

template <class Acc = long long>
vector<Acc> minPlusCC(const vector<Acc> &a, const vector<Acc> &b)
{
    // a、b 的相邻差分均单调不降；返回精确的 min-plus 卷积，任一输入为空时返回空。
    if (a.empty() || b.empty())
    {
        return {};
    }
    int n = a.size();
    int m = b.size();
    vector<Acc> c{a[0] + b[0]};
    int i = 1;
    int j = 1;
    while (i < n || j < m)
    {
        if (j == m || (i < n && a[i] - a[i - 1] <= b[j] - b[j - 1]))
        {
            c.push_back(c.back() + a[i] - a[i - 1]);
            i++;
        }
        else
        {
            c.push_back(c.back() + b[j] - b[j - 1]);
            j++;
        }
    }
    return c;
}

template <class Acc = long long>
vector<Acc> minPlusCA(const vector<Acc> &a, const vector<Acc> &b)
{
    // a 的相邻差分单调不降，b 任意；返回精确的 min-plus 卷积，任一输入为空时返回空。
    if (a.empty() || b.empty())
    {
        return {};
    }
    int n = b.size();
    int m = a.size();
    auto better = [&](int s, int x, int y)
    {
        auto valid = [&](int j)
        {
            return j <= s && s - j < m;
        };
        if (!valid(y))
        {
            return false;
        }
```

```
        if (!valid(x))
        {
            return true;
        }
        return b[x] + a[s - x] >= b[y] + a[s - y];
    };
    vector<int> p = monoMin(n + m - 1, n, better);
    vector<Acc> c(n + m - 1);
    for (int s = 0; s < (int)c.size(); s++)
    {
        c[s] = b[p[s]] + a[s - p[s]];
    }
    return c;
}
```

`minPlus` 是一般 $O(nm)$ 版；`minPlusCC` 要求两序列离散凸，时间 $O(n+m)$；`minPlusCA` 要求 `a` 离散凸、`b` 任意，借 `monoMin` 分治在 $O((n+m)\log(n+m))$ 时间内完成。三者默认返回 `vector<long long>`，必须保证每次加法、相邻差分和结果项都能装入 `Acc`；需要宽结果时显式调用 `minPlus<__int128>`、`minPlusCC<__int128>` 或 `minPlusCA<__int128>` 并传入宽类型序列。三者额外空间均为 $O(n+m)$，每个结果项都有至少一个合法候选，不使用不可达哨兵。它与加法卷积 10.1.003 NTT 无关，是 $(\min,+)$ 半环上的运算，常见于背包合并与凸优化 DP。

直接调用 `monoMin` 时，`better(r,x,y)` 要判断新列 `y` 是否优于旧列 `x`：

```
signed main()
{
    auto better = [](int r, int x, int y)
    {
        return abs(r - y) < abs(r - x);
    };
    assert((monoMin(3, 3, better) == vector<int>{0, 1, 2}));
}
```

下面用两个短例分别核对完整 64 位边界和两种凸优化：

```
signed main()
{
    vector<long long> a{0, 1, 4}, b{5, 0, 6};
    assert((minPlus(vector<long long>{1, 2}, vector<long long>{-1, 3}) ==
            vector<long long>{0, 1, 5}));
    assert((minPlusCC(a, vector<long long>{0, 2, 7}) ==
            vector<long long>{0, 1, 3, 6, 11}));
    assert((minPlusCA(a, b) == vector<long long>{5, 0, 1, 4, 10}));
    vector<__int128> wide{(__int128)6000000000000000000LL, (__int128)7000000000000000000LL};
    auto wc = minPlus<__int128>(wide, vector<__int128>{1, 2});
    assert(wc[0] == (__int128)6000000000000000001LL);
    cout << "OK\n";
    return 0;
}
```

10.1.013 集合形式幂级数

把长度 2^n 的集合函数看成以子集为下标的形式幂级数，乘法取 10.1.011 子集卷积。若 $f_\emptyset = 0$，则 $\exp_*(f) = \sum_{k\ge 0} f^{*k}/k!$；若 $g_\emptyset = 1$，则 $\log_*(g)$ 是它的逆。按最高二进制位拆块后，同一块内任意两项都含该最高位，乘积为零，因此指数新块是输入新块与已知低块指数的子集卷积；对数反向求解 $f_1 * g_0 = g_1$，在 ranked zeta 域按秩作常数项为 1 的三角消元。

`compEgf(c,f)` 计算 $\sum_{k=0}^{n} c_k f^{*k}/k!$。它逐个加入集合变量，在 OR zeta 域维护“已使用的变量数”和秩；`setCompose` 先在 $x = f_\emptyset$ 处求 $p^{(k)}(x)$，再令空集系数归零并调用该公共 EGF 复合，所以允许 $f_\emptyset \neq 0$。`projEgf` 是这条线性映射的转置:对子集卷积算子先把掩码取补集顺序,再调用同一个 `subsetConv`；`setPowerProj` 最后把非零常数项按二项式展开卷回去。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 10.1.006 FWT / FST
// 依赖 10.1.011 子集卷积
namespace setfps
{
template <class T> vector<T> compEgf(const vector<T> &c, const vector<T> &f)
{
    // c[k] 是外层 EGF 的第 k 个分子系数，f 是常数项为 0 的集合幂级数；返回 sum c[k]*f^k/k!。
    assert(!c.empty() && !f.empty() && has_single_bit(f.size()) && f[0] == T(0)); // 调试检查，可删。
    int n = c.size() - 1;
    assert(n <= 30 && (int)f.size() == (1LL << n)); // 调试检查，可删。
    vector<vector<vector<T>>> a(n + 1);
    for (int i = 0; i <= n; i++)
    {
        a[i].assign(i + 1, vector<T>(1LL << i));
        a[i][0][0] = c[n - i];
    }
    for (int v = 0; v < n; v++)
    {
        int z = 1LL << v;
        vector h(v + 1, vector<T>(z));
        for (int s = 0; s < z; s++)
        {
            h[popcount((unsigned long long)s)][s] = f[z + s];
        }
        for (auto &x : h)
        {
            fwt(x, BitOp::Or);
        }
        for (int i = v + 1; i <= n; i++)
        {
            for (int j = 0; j <= i; j++)
            {
                copy_n(a[i][j].begin(), z, a[i][j].begin() + z);
            }
        }
        for (int i = v; i < n; i++)
        {
```

```
            for (int j = 0; j <= v; j++)
            {
                for (int k = 0; j + k <= i; k++)
                {
                    for (int s = 0; s < z; s++)
                    {
                        a[i + 1][j + k + 1][z + s] += h[j][s] * a[i][k][s];
                    }
                }
            }
        }
    }
    for (auto &x : a[n])
    {
        fwt(x, BitOp::Or, true);
    }
    vector<T> ans(f.size());
    for (int s = 0; s < (int)f.size(); s++)
    {
        ans[s] = a[n][popcount((unsigned long long)s)][s];
    }
    return ans;
}

template <class T> vector<T> transConv(const vector<T> &f, vector<T> x)
{
    // f 是固定右因子，x 是输出线性泛函；返回子集卷积算子关于 f 的转置作用结果。
    assert(!f.empty() && f.size() == x.size() && has_single_bit(f.size())); // 调试检查，可删。
    reverse(x.begin(), x.end());
    x = subsetConv(x, f);
    reverse(x.begin(), x.end());
    return x;
}

template <class T> vector<T> projEgf(const vector<T> &f, const vector<T> &w)
{
    // f 是常数项为 0 的集合幂级数，w 是线性泛函权重；返回 <w,f^k/k!>（0<=k<=n）。
    assert(!f.empty() && f.size() == w.size() && has_single_bit(f.size()) && f[0] == T(0)); //
    ↪ 调试检查，可删。
    int n = countr_zero((unsigned long long)f.size());
    vector<vector<T>> block(n);
    for (int j = 0; j < n; j++)
    {
        block[j] = vector<T>(f.begin() + (1LL << j), f.begin() + (2LL << j));
    }
    vector<T> dp = w;
    vector<T> ans(n + 1);
    ans[0] = w[0];
    for (int i = 0; i < n; i++)
    {
        vector<T> ndp(1LL << (n - i - 1));
        for (int j = 0; j < n - i; j++)
```

```
        {
            vector<T> x(dp.begin() + (1LL << j), dp.begin() + (2LL << j));
            x = transConv(block[j], move(x));
            for (int k = 0; k < (int)x.size(); k++)
            {
                ndp[k] += x[k];
            }
        }
        dp = move(ndp);
        ans[i + 1] = dp[0];
    }
    return ans;
}
}

template <class T> vector<T> setExp(const vector<T> &f)
{
    // f[0] 必须为 0；返回子集卷积意义下的 exp(f)。
    assert(!f.empty() && has_single_bit(f.size()) && f[0] == T(0)); // 调试检查，可删。
    int n = countr_zero((unsigned long long)f.size());
    vector<T> g(f.size());
    g[0] = T(1);
    for (int i = 0; i < n; i++)
    {
        int m = 1LL << i;
        vector<T> a(f.begin() + m, f.begin() + 2 * m);
        vector<T> b(g.begin(), g.begin() + m);
        a = subsetConv(a, b);
        copy(a.begin(), a.end(), g.begin() + m);
    }
    return g;
}

template <class T> vector<T> setLog(const vector<T> &g)
{
    // g[0] 必须为 1；返回子集卷积意义下的 log(g)。
    assert(!g.empty() && has_single_bit(g.size()) && g[0] == T(1)); // 调试检查，可删。
    int n = countr_zero((unsigned long long)g.size());
    vector<T> f(g.size());
    for (int i = n - 1; i >= 0; i--)
    {
        int m = 1LL << i;
        vector<T> a(g.begin() + m, g.begin() + 2 * m);
        vector<T> b(g.begin(), g.begin() + m);
        auto x = rankedZeta(a);
        auto y = rankedZeta(b);
        for (int s = 0; s < m; s++)
        {
            for (int d = 0; d <= i; d++)
            {
                for (int k = 0; k < d; k++)
                {
```

```
                    x(s, d) -= x(s, k) * y(s, d - k);
                }
            }
        }
        a = rankedMobius(move(x));
        copy(a.begin(), a.end(), f.begin() + m);
    }
    return f;
}

template <class T> vector<T> setCompose(const vector<T> &p, const vector<T> &f)
{
    // p[k] 是普通多项式系数，f 是集合幂级数；返回普通多项式 p(f)。
    assert(!f.empty() && has_single_bit(f.size())); // 调试检查，可删。
    int n = countr_zero((unsigned long long)f.size());
    if (p.empty())
    {
        return vector<T>(f.size());
    }
    vector<T> q = p;
    vector<T> c(n + 1);
    T x = f[0];
    for (int k = 0; k <= n && !q.empty(); k++)
    {
        T v{};
        for (auto it = q.rbegin(); it != q.rend(); ++it)
        {
            v = v * x + *it;
        }
        c[k] = v;
        for (int i = 1; i < (int)q.size(); i++)
        {
            q[i - 1] = q[i] * T(i);
        }
        q.pop_back();
    }
    vector<T> g = f;
    g[0] = T(0);
    return setfps::compEgf(c, g);
}

template <class T> vector<T> setPowerProj(const vector<T> &f, const vector<T> &w, int m)
{
    // f 是集合幂级数，w 是同长线性泛函权重，m 是输出项数；返回 <w,f^k>（0<=k<m）。
    // T 必须是交换域，且 1..m-1 在其中可逆。
    assert(m >= 0 && !f.empty() && f.size() == w.size() && has_single_bit(f.size())); // 调试检
    ↪ 查，可删。
    if (m == 0)
    {
        return {};
    }
    T c = f[0];
```

```
    vector<T> g = f;
    g[0] = T(0);
    auto q = setfps::projEgf(g, w);
    vector<T> e(m, T(1));
    for (int i = 1; i < m; i++)
    {
        e[i] = e[i - 1] * c / T(i);
    }
    vector<T> ans(m);
    for (int i = 0; i < min(m, (int)q.size()); i++)
    {
        for (int j = 0; i + j < m; j++)
        {
            ans[i + j] += q[i] * e[j];
        }
    }
    T fac = T(1);
    for (int i = 0; i < m; i++)
    {
        if (i)
        {
            fac *= T(i);
        }
        ans[i] *= fac;
    }
    return ans;
}
```

设集合函数长度为 2^n，普通多项式 `p` 的长度为 r，投影输出长度为 m。`setExp`、`setLog`、`compEgf`、`projEgf` 的时间均为 $O(n^2 2^n)$、额外空间为 $O(n2^n)$；`setCompose` 另需 $O(nr)$ 求前 $n+1$ 阶导数值，`setPowerProj` 另需 $O(nm)$ 合并常数项。除 `setPowerProj` 要求 `T` 中 $1, \dots, m-1$ 可逆外，其余接口只使用交换环的加减乘和整数倍。

Library Checker 的 `exp/log` 参考实现都把外层函数转成 `compEgf`，算法同阶；当前主入口保留最高位分块递推，因为在 $n = 20$、GCC 14.2.0、`-O2 -pipe` 的同机复测中，模板 core 分别为 `669.447/601.345 ms`，参考为 `719.323/712.555 ms`，峰值工作集约 `97.80/96.27 MiB`，参考两项均约 `209.00 MiB`。这些数值只说明当前实现与环境的取舍，不是跨平台结论。普通下标多项式的 FPS 运算见 10.2 多项式基础运算。

下面在两个元素的全集上分别检查指数、对数与二次复合的固定系数：

```
signed main()
{
    vector<Z> f{0, 2, 3, 5};
    assert(setExp(f) == vector<Z>({1, 2, 3, 11}));
    assert(setLog(vector<Z>{1, 2, 3, 11}) == f);
    assert(setCompose(vector<Z>{7, 11, 13}, f) == vector<Z>({7, 22, 33, 211}));
    assert(setPowerProj(f, vector<Z>{7, 11, 13, 17}, 4) ==
           vector<Z>({7, 146, 204, 0}));
}
```

10.1.014 多元卷积

给定各维长度 $n_0, n_1, \dots, n_{k-1}$，用最低位在前的混合进制把坐标压成下标。目标卷积只保留每一维坐标和都小于对应 n_i 的项，也就是混合进制无进位加法；它不同于把矩阵展平后直接卷积。令 $\chi(x) = \sum_i \lfloor x/(n_0 n_1 \cdots n_i) \rfloor \bmod k$，按 χ 分成 k 组做普通卷积，再只取输出下标所属的组，恰好滤掉发生进位的项。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 7.1.008 动态取模类
// 依赖 7.4.003 原根与全部原根筛
// 依赖 10.1.003 NTT
// 依赖 10.1.004 MTT 任意模数卷积
vector<Z> multiConv(const vector<int> &base, const vector<Z> &f, const vector<Z> &g)
{
    // base 依次给出各维长度，f、g 按最低维在前展平；返回各维无进位卷积。
    int k = base.size();
    int n = 1;
    for (int x : base)
    {
        assert(x > 0 && n <= (1 << 22) / x); // 调试检查，可删；同时保证 NTT 长度可用。
        n *= x;
    }
    assert((int)f.size() == n && (int)g.size() == n); // 调试检查，可删。
    if (n == 1)
    {
        return {f[0] * g[0]};
    }
    auto cls = [&](int x)
    {
        int c = 0;
        for (int b : base)
        {
            x /= b;
            c += x;
        }
        return c % k;
    };
    int len = bit_ceil((unsigned long long)(2 * n - 1));
    vector a(k, vector<Z>(len));
    vector b(k, vector<Z>(len));
    for (int i = 0; i < n; i++)
    {
        a[cls(i)][i] = f[i];
        b[cls(i)][i] = g[i];
    }
    for (int i = 0; i < k; i++)
    {
        FastNTT::dft(a[i]);
        FastNTT::dft(b[i]);
    }
    vector c(k, vector<Z>(len));
```

```
    for (int i = 0; i < k; i++)
    {
        for (int j = 0; j < k; j++)
        {
            for (int p = 0; p < len; p++)
            {
                c[(i + j) % k][p] += a[i][p] * b[j][p];
            }
        }
    }
    for (auto &v : c)
    {
        FastNTT::idft(v);
    }
    vector<Z> ans(n);
    for (int i = 0; i < n; i++)
    {
        ans[i] = c[cls(i)][i];
    }
    return ans;
}
```

`multiConv(base,f,g)` 要求所有维长为正、两数组长度都等于 $N=\prod_i n_i$，并受当前 NTT 的 2^{23} 长度上限约束。若维数为 k，时间 $O(kN\log N+k^2N)$、空间 $O(kN)$；维数很少时常数可接受，所有维长均为 2 时应优先用专门的 10.1.011 子集卷积。

下面用 2×2 系数表检查两维无进位卷积：

```
signed main()
{
    vector<int> base{2, 2};
    vector<Z> a{1, 2, 3, 4};
    vector<Z> b{5, 6, 7, 8};
    assert(multiConv(base, a, b) == vector<Z>({5, 16, 22, 60}));
}
```

题目若允许任意素数模数且要求每一维按 $x_i^{n_i}=1$ 循环折叠，不能调用上面的无进位版本。`multiCycConv` 先在每个维度的几何序列上做 DFT，频域逐点相乘，再按逆根变换并除以 N；长度超过 100 时 `geomDft` 用 10.1.004 任意模卷积 的三素数 NTT + Garner，避免对大模数退化为平方枚举。

```
vector<DynMod> geomDft(vector<DynMod> f, DynMod r)
{
    // f 是低次在前的系数，r 是非零单位根；返回 f(1),f(r),...,f(r^(n-1))。
    int n = f.size();
    if (n <= 100)
    {
        vector<DynMod> ans(n);
        DynMod x = 1;
        for (int i = 0; i < n; i++)
        {
            DynMod pw = 1;
```

```
            for (int j = 0; j < n; j++)
            {
                ans[i] += f[j] * pw;
                pw *= x;
            }
            x *= r;
        }
        return ans;
    }
    vector<DynMod> a(2 * n - 1), b(n);
    DynMod pw = 1;
    a[0] = 1;
    for (int i = 1; i < 2 * n - 1; i++)
    {
        a[i] = a[i - 1] * pw;
        pw *= r;
    }
    DynMod ir = r.pow(DynMod::mod() - 2);
    pw = 1;
    b[0] = 1;
    for (int i = 1; i < n; i++)
    {
        b[i] = b[i - 1] * pw;
        pw *= ir;
    }
    for (int i = 0; i < n; i++)
    {
        f[i] *= b[i];
    }
    reverse(f.begin(), f.end());
    vector<long long> x(n), y(2 * n - 1);
    for (int i = 0; i < n; i++)
    {
        x[i] = f[i].x;
    }
    for (int i = 0; i < 2 * n - 1; i++)
    {
        y[i] = a[i].x;
    }
    auto c = convMod(move(x), move(y), DynMod::mod());
    vector<DynMod> ans(n);
    for (int i = 0; i < n; i++)
    {
        ans[i] = DynMod(c[n - 1 + i]) * b[i];
    }
    return ans;
}

vector<long long> multiCycConv(long long p, const vector<int> &base,
                               const vector<long long> &f, const vector<long long> &g)
{
    // p 是不超过 1E9 的素数，各维长度整除 p-1；f、g 最低维在前，返回逐维循环卷积。
```

```
    assert(2 <= p && p <= 1000000000); // 调试检查，可删。
    DynMod::setMod(p);
    int n = 1;
    for (int x : base)
    {
        assert(x >= 2 && (p - 1) % x == 0 && n <= 262144 / x); // 调试检查，可删。
        n *= x;
    }
    assert((int)f.size() == n && (int)g.size() == n); // 调试检查，可删。
    vector<DynMod> a(n), b(n);
    for (int i = 0; i < n; i++)
    {
        a[i] = f[i];
        b[i] = g[i];
    }
    DynMod gen = primRoot(p);
    auto transform = [&](vector<DynMod> &v, bool inv)
    {
        int step = 1;
        for (int len : base)
        {
            DynMod root = gen.pow((p - 1) / len);
            if (inv)
            {
                root = root.pow(p - 2);
            }
            for (int block = 0; block < n; block += step * len)
            {
                for (int off = 0; off < step; off++)
                {
                    vector<DynMod> row(len);
                    for (int j = 0; j < len; j++)
                    {
                        row[j] = v[block + off + step * j];
                    }
                    row = geomDft(move(row), root);
                    for (int j = 0; j < len; j++)
                    {
                        v[block + off + step * j] = row[j];
                    }
                }
            }
            step *= len;
        }
    };
    transform(a, false);
    transform(b, false);
    for (int i = 0; i < n; i++)
    {
        a[i] *= b[i];
    }
    transform(a, true);
```

```
    DynMod scale = DynMod(n).pow(p - 2);
    vector<long long> ans(n);
    for (int i = 0; i < n; i++)
    {
        ans[i] = (a[i] * scale).x;
    }
    return ans;
}
```

该循环版要求 p 为素数、$n_i \mid p-1$、$n_i \geq 2$ 且 $N \leq 262144$；输入数组按最低维在前展平。每个维度总共处理 N/n_i 个长度 n_i 的 DFT，几何点求值的总复杂度为 $O(N \sum_i \log n_i)$，三素数卷积的系数重建为常数倍，额外空间为 $O(N + \max n_i)$。当 $N = 1$（$K = 0$）时函数退化为一个标量乘法。

10.1.015 模 2^n 乘法卷积

这里的运算是 $c_k = \sum_{ij \equiv k \pmod{2^n}} a_i b_j$，下标相乘而不是相加。把非零剩余类按 $v_2(x)$ 分层；固定层后，奇数部分属于 $(\mathbb{Z}/2^m\mathbb{Z})^\times$，当 $m \geq 2$ 时可唯一写成符号 ± 1 与 5 的幂。于是每层在一个长度至多 2 的符号轴和一个二进制循环轴上做变换，不同 2-进赋值的层再按加法合并，超过 n 的赋值统一落到下标 0。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.1.003 NTT
vector<Z> mulMod2n(const vector<Z> &a, const vector<Z> &b)
{
    // a、b 的长度必须同为 2 的幂；返回下标乘法模数组长度的卷积。
    assert(a.size() == b.size() && !a.empty() && has_single_bit(a.size())); // 调试检查，可删。
    int n = countr_zero((unsigned long long)a.size());
    assert(n <= 22); // 调试检查，可删；该上限控制 O(n2^n) 工作量与三组分层缓冲区。
    int mask = (1LL << n) - 1;
    vector<vector<vector<Z>>> x(n + 1), y(n + 1), z(n + 1);
    auto shape = [&](int v)
    {
        int h = n - v >= 2 ? 2 : 1;
        int w = 1LL << max<int>(n - v - 2, 0);
        return pair{h, w};
    };
    for (int v = 0; v <= n; v++)
    {
        auto [h, w] = shape(v);
        x[v].assign(h, vector<Z>(w));
        y[v].assign(h, vector<Z>(w));
        z[v].assign(h, vector<Z>(w));
        int p = (1LL << v) & mask;
        for (int j = 0; j < w; j++)
        {
            x[v][0][j] = a[p];
            y[v][0][j] = b[p];
            if (h == 2)
            {
                x[v][1][j] = a[(1LL << n) - p];
```

```
                y[v][1][j] = b[(1LL << n) - p];
            }
            p = 5 * p & mask;
        }
    }
    for (int v = 0; v <= n; v++)
    {
        auto [h, w] = shape(v);
        for (int i = 0; i < h; i++)
        {
            FastNTT::dft(x[v][i]);
            FastNTT::dft(y[v][i]);
        }
        if (h == 2)
        {
            for (int j = 0; j < w; j++)
            {
                Z a0 = x[v][0][j], a1 = x[v][1][j];
                Z b0 = y[v][0][j], b1 = y[v][1][j];
                x[v][0][j] = a0 + a1;
                x[v][1][j] = a0 - a1;
                y[v][0][j] = b0 + b1;
                y[v][1][j] = b0 - b1;
            }
        }
    }
    for (int p = 0; p <= n; p++)
    {
        for (int q = 0; q <= n; q++)
        {
            int v = min(n, p + q);
            auto [h, w] = shape(v);
            for (int i = 0; i < h; i++)
            {
                for (int j = 0; j < w; j++)
                {
                    z[v][i][j] += x[p][i][j] * y[q][i][j];
                }
            }
        }
    }
    Z inv2 = Z(2).inv();
    for (int v = 0; v <= n; v++)
    {
        auto [h, w] = shape(v);
        for (int i = 0; i < h; i++)
        {
            FastNTT::idft(z[v][i]);
        }
        if (h == 2)
        {
            for (int j = 0; j < w; j++)
```

```
            {
                Z a0 = z[v][0][j], a1 = z[v][1][j];
                z[v][0][j] = (a0 + a1) * inv2;
                z[v][1][j] = (a0 - a1) * inv2;
            }
        }
    }
    vector<Z> c(1LL << n);
    for (int v = 0; v <= n; v++)
    {
        auto [h, w] = shape(v);
        int p = (1LL << v) & mask;
        for (int j = 0; j < w; j++)
        {
            c[p] = z[v][0][j];
            if (h == 2)
            {
                c[(1LL << n) - p] = z[v][1][j];
            }
            p = 5 * p & mask;
        }
    }
    return c;
}
```

`mulMod2n(a,b)` 支持长度 2^n，当前实现要求 $n \le 22$；这个界限制的是 $O(n2^n)$ 工作量和 `x,y,z` 三组各含约 2^n 个 `Z` 的分层缓冲区，不是公共 NTT 的阶数上限。最大单层变换来自 $v=0$，长度仅为 2^{n-2}。总时间 $O(n2^n)$、额外空间 $O(2^n)$。系数仍在模 `998244353` 的 `Z` 中计算，只有下标乘法取模 2^n，不要把它与系数模数混淆。普通加法下标卷积用 10.1.003 NTT。

下面检查长度 1、2、4 的固定下标乘法卷积：

```
signed main()
{
    assert(mulMod2n(vector<Z>{2}, vector<Z>{3}) == vector<Z>({6}));
    assert(mulMod2n(vector<Z>{1, 2}, vector<Z>{3, 4}) == vector<Z>({13, 8}));
    assert(mulMod2n(vector<Z>{1, 2, 3, 4}, vector<Z>{5, 6, 7, 8}) ==
           vector<Z>({92, 44, 84, 40}));
}
```

10.1.016 $GF(2^{64})$ 卷积

把每个 `unsigned long long` 视为 $GF(2)[x]$ 中次数小于 64 的多项式，并模不可约多项式 $x^{64}+x^4+x^3+x+1$，即可得到 $GF(2^{64})$。加减都是异或；`carrylessMul` 做无进位乘法，`GF64::reduce` 用多项式 Montgomery 约简。乘法群阶 $2^{64}-1$ 没有足够大的二次幂因子，不能套普通 NTT；下面用特征 2 的 additive FFT，在一组 $GF(2)$ 线性无关的基上递归求值。x86-64 的 GCC/Clang 路径使用 PCLMUL 完成常数时间无进位乘法；其他目标保留逐置位的可移植回退。

```cpp
#if defined(__x86_64__) && defined(__GNUC__)
#pragma GCC push_options
#pragma GCC target("pclmul")
#endif

unsigned __int128 carrylessMul(unsigned long long a, unsigned long long b)
{
    // a、b 是按位表示的 GF(2)[x] 多项式；返回 128 位无进位乘积，x86 下用 PCLMULQDQ 指令。
#if defined(__x86_64__) && defined(__GNUC__)
    typedef long long Vec128 __attribute__((vector_size(16)));
    Vec128 z = __builtin_ia32_pclmulqdq128(Vec128{(long long)a, 0},
                                           Vec128{(long long)b, 0}, 0);
    unsigned __int128 r;
    memcpy(&r, &z, sizeof(r));
    return r;
#else
    unsigned __int128 r = 0;
    while (a)
    {
        int k = countr_zero(a);
        r ^= (unsigned __int128)b << k;
        a &= a - 1;
    }
    return r;
#endif
}

constexpr unsigned __int128 clMulConst(unsigned long long a, unsigned long long b)
{
    // a、b 同 carrylessMul；逐位异或的 constexpr 版本，供编译期常量计算。
    unsigned __int128 r = 0;
    for (int i = 0; i < 64; i++)
    {
        if (a >> i & 1)
        {
            r ^= (unsigned __int128)b << i;
        }
    }
    return r;
}

struct GF64
{
    static constexpr unsigned __int128 poly = ((unsigned __int128)1 << 64) | 0b11011; // poly
    ↪ 是 GF(2^64) 的模多项式 x^64+x^4+x^3+x+1。
    static constexpr unsigned long long inv = []
    {
        unsigned long long a = 1;
        for (int i = 0; i < 6; i++)
        {
            a = clMulConst(a, (unsigned long long)clMulConst(a, (unsigned long long)poly));
        }
```

```
        return a;
    }();
    static constexpr unsigned long long r2 = []
    {
        unsigned __int128 r = 1;
        for (int i = 0; i < 128; i++)
        {
            r <<= 1;
            if (r >> 64 & 1)
            {
                r ^= poly;
            }
        }
        return (unsigned long long)r;
    }();

    unsigned long long x = 0; // x 是 Montgomery 表示（v*x^64 mod poly）。

    GF64() = default;
    explicit GF64(unsigned long long v) : x(reduce(carrylessMul(v, r2)))
    {
        // v 是普通表示的域元素；转入 Montgomery 表示。
    }
    static GF64 bit(int x)
    {
        // x 的最低位为 0 或 1；返回域元素 0 或 1。
        return GF64(x & 1);
    }

    static unsigned long long reduce(unsigned __int128 x)
    {
        // x 是 128 位无进位乘积；Montgomery 约减，返回 x*x^{-64} mod poly。
        unsigned long long f = carrylessMul((unsigned long long)x, inv);
        return (unsigned long long)(x >> 64) ^
               (unsigned long long)(carrylessMul(f, (unsigned long long)poly) >> 64) ^ f;
    }

    GF64 &operator+=(GF64 o)
    {
        x ^= o.x;
        return *this;
    }
    GF64 &operator-=(GF64 o)
    {
        return *this += o;
    }
    GF64 &operator*=(GF64 o)
    {
        x = reduce(carrylessMul(x, o.x));
        return *this;
    }
    friend GF64 operator+(GF64 a, GF64 b)
```

```
    {
        // a、b 是域元素；返回 a+b（按位异或）。
        return a += b;
    }
    friend GF64 operator-(GF64 a, GF64 b)
    {
        // a、b 是域元素；返回 a-b，特征 2 下与 a+b 相同。
        return a -= b;
    }
    friend GF64 operator*(GF64 a, GF64 b)
    {
        // a、b 是域元素；返回 a*b。
        return a *= b;
    }

    GF64 pow(unsigned long long n) const
    {
        // n 是指数；返回当前元素的 n 次幂。
        GF64 a = *this, r = bit(1);
        while (n)
        {
            if (n & 1)
            {
                r *= a;
            }
            a *= a;
            n >>= 1;
        }
        return r;
    }
    GF64 invField() const
    {
        // 当前元素非零；返回乘法逆元，即 2^64-2 次幂。
        assert(x != 0); // 调试检查，可删。
        return pow(~(unsigned long long)(0) - 1);
    }
    unsigned long long val() const
    {
        // 无参数；返回普通表示的 64 位值。
        return reduce(x);
    }
    friend bool operator==(GF64 a, GF64 b)
    {
        return a.x == b.x;
    }
};

vector<GF64> gf64SubSum(const vector<GF64> &b)
{
    // b 是 k 个域元素；返回长 2^k 的表，第 s 项是 s 的二进制位选中的 b 之和。
    vector<GF64> a(1LL << b.size());
    for (int k = 0; k < (int)b.size(); k++)
```

```
    {
        for (int s = 0; s < (1LL << k); s++)
        {
            a[(1LL << k) + s] = a[s] + b[k];
        }
    }
    return a;
}

struct GF64FFT
{
    struct Data
    {
        vector<GF64> basis, next, sums, aux; // basis 是本层基，next 是下一层基，sums 是归一化系数的子集和表，aux 是工作区。

        void init()
        {
            // 无参数；由本层 basis 求归一化系数的子集和 sums、下一层基 next，并分配工作区 aux。
            int lg = basis.size();
            aux.resize(1LL << lg);
            GF64 b = basis.back(), ib = b.invField();
            next.resize(lg - 1);
            vector<GF64> g(lg - 1);
            for (int i = 0; i + 1 < lg; i++)
            {
                g[i] = basis[i] * ib;
                next[i] = g[i] * g[i] + g[i];
            }
            sums = gf64SubSum(g);
        }
    };

    vector<Data> d; // d[k] 是长度 2^k 的变换层数据。

    void prepare(int lg)
    {
        // lg 是变换阶数；尚未准备时用随机元素生成线性无关基并逐层预处理 d[0..lg]。
        if ((int)d.size() > lg)
        {
            return;
        }
        mt19937_64 rng;
        vector<GF64> basis;
        while ((int)basis.size() < lg)
        {
            basis.clear();
            for (GF64 x(rng()); !(x == GF64()); x = x * x + x)
            {
                basis.push_back(x);
            }
        }
```

```
        basis.erase(basis.begin(), basis.end() - lg);
        d.resize(lg + 1);
        d[lg].basis = basis;
        for (int k = lg; k > 0; k--)
        {
            d[k].init();
            d[k - 1].basis = d[k].next;
        }
    }

    template <bool inverse = false> void taylor(span<GF64> f)
    {
        for (int n = inverse ? 1 : f.size() / 4;
             inverse ? n * 4 <= (int)f.size() : n >= 1;
             inverse ? n *= 2 : n /= 2)
        {
            for (int s = 0; s < (int)f.size(); s += 4 * n)
            {
                for (int i = 0; i < n; i++)
                {
                    GF64 b = f[s + n + i], c = f[s + 2 * n + i];
                    GF64 z = f[s + 3 * n + i];
                    f[s + n + i] = inverse ? b + c : b + c + z;
                    f[s + 2 * n + i] = c + z;
                }
            }
        }
    }

    template <bool inverse = false> void transform(span<GF64> f)
    {
        int n = f.size();
        if (n == 1)
        {
            return;
        }
        int lg = bit_width((unsigned long long)n) - 1;
        auto &cur = d[lg];
        if (n == 2)
        {
            f[1] = f[0] + f[1];
            return;
        }
        span<GF64> u(cur.aux.data(), n / 2);
        span<GF64> v(cur.aux.data() + n / 2, n / 2);
        if constexpr (!inverse)
        {
            taylor(f);
            for (int i = 0; i < n / 2; i++)
            {
                u[i] = f[2 * i];
                v[i] = f[2 * i + 1];
```

```
            }
            transform(u);
            transform(v);
            for (int i = 0; i < n / 2; i++)
            {
                GF64 a = u[i] + cur.sums[i] * v[i];
                f[i] = a;
                f[i + n / 2] = a + v[i];
            }
        }
        else
        {
            for (int i = 0; i < n / 2; i++)
            {
                v[i] = f[i] + f[n / 2 + i];
                u[i] = f[i] + cur.sums[i] * v[i];
            }
            transform<true>(u);
            transform<true>(v);
            for (int i = 0; i < n / 2; i++)
            {
                f[2 * i] = u[i];
                f[2 * i + 1] = v[i];
            }
            taylor<true>(f);
        }
    }
};

inline GF64FFT gf64FFT;

vector<unsigned long long> convGF64(const vector<unsigned long long> &x, const vector<unsigned
↪  long long> &y)
{
    // x、y 是 GF(2^64) 上多项式的系数（普通表示）；返回乘积系数，规模小时直接相乘，否则用加法 FFT。
    if (x.empty() || y.empty())
    {
        return {};
    }
    int need = x.size() + y.size() - 1;
    int lg = bit_width((unsigned long long)(need - 1));
    int n = 1LL << lg;
    if ((long long)x.size() * (long long)y.size() <=
        (long long)n * (lg + 1) * (lg + 1))
    {
        vector<GF64> c(need);
        for (int i = 0; i < (int)x.size(); i++)
        {
            for (int j = 0; j < (int)y.size(); j++)
            {
                c[i + j] += GF64(x[i]) * GF64(y[j]);
            }
```

```
        }
        vector<unsigned long long> ans(need);
        for (int i = 0; i < need; i++)
        {
            ans[i] = c[i].val();
        }
        return ans;
    }
    if (lg > 3 && need == (1LL << (lg - 1)) + 1)
    {
        vector<unsigned long long> prefix(x.begin(), x.end() - 1);
        vector<unsigned long long> ans = convGF64(prefix, y);
        ans.resize(need);
        GF64 last(x.back());
        for (int j = 0; j < (int)y.size(); j++)
        {
            ans[x.size() - 1 + j] ^= (last * GF64(y[j])).val();
        }
        return ans;
    }
    vector<GF64> a(n), b(n);
    for (int i = 0; i < (int)x.size(); i++)
    {
        a[i] = GF64(x[i]);
    }
    for (int i = 0; i < (int)y.size(); i++)
    {
        b[i] = GF64(y[i]);
    }
    gf64FFT.prepare(lg);
    gf64FFT.transform(span<GF64>(a));
    gf64FFT.transform(span<GF64>(b));
    for (int i = 0; i < n; i++)
    {
        a[i] *= b[i];
    }
    gf64FFT.transform<true>(span<GF64>(a));
    vector<unsigned long long> ans(need);
    for (int i = 0; i < need; i++)
    {
        ans[i] = a[i].val();
    }
    return ans;
}

#if defined(__x86_64__) && defined(__GNUC__)
#pragma GCC pop_options
#endif
```

`convGF64(a,b)` 返回上述固定域中的线性卷积，空输入返回空；输入和输出都是普通多项式基下的 `unsigned long long`。设补齐长度为 N，每层 additive FFT 的 Taylor 变换还要递归处理半长块，当前实现时间为 $O(N\log^2 N)$ 次域运算、空间为 $O(N\log N)$；当结果长度恰为 2^k+1 时会拆下较长输入的

末项，避免补齐长度翻倍。PCLMUL 是本题最大规模下的重要常数优化，非 x86-64 平台的标量回退仍保持相同复杂度但可能无法满足严格时限。模 998244353 的整数卷积仍用 10.1.003 NTT。

下面检查域乘的固定值，并用固定种子强制一组 600×620 卷积进入 additive FFT 后核对滚动校验和：

```
signed main()
{
    mt19937_64 rng(0);
    assert((GF64(2) * GF64(3)).val() == 6);
    assert((GF64(1ULL << 63) * GF64(2)).val() == 0x1BULL);
    vector<unsigned long long> a(600), b(620);
    for (auto &x : a)
    {
        x = rng();
    }
    for (auto &x : b)
    {
        x = rng();
    }
    auto c = convGF64(a, b);
    unsigned long long h = 0;
    for (auto x : c)
    {
        h = h * 1000000007ULL + x;
    }
    assert(h == 3014567480266723489ULL);
}
```

10.1.017 模素数乘法幺半群卷积

下标属于乘法幺半群 $(\mathbb{Z}/P\mathbb{Z}, \times)$ 时，非零元群是循环群。取原根 g，以 g^i 重编号后，非零下标的乘法变为指数模 $P-1$ 的加法，因此先做一次普通线性 NTT 卷积再折回长度 $P-1$。零元不属于该群：积为零恰好是一侧下标为零，需由两数组总和单独累加。

下面显式依赖 10.1.003 NTT 提供的 `Z` 与 `conv`，及 7.4.003 原根 的 `primRoot`。

```
// 依赖 1.1.008 静态取模类
// 依赖 7.4.003 原根与全部原根筛
vector<Z> mulPrimeConv(int p, const vector<Z> &a, const vector<Z> &b)
{
    // p 是不超过 NTT 长度限制的素数，a、b 按 0..p-1 编号；返回 c[k]=sum_{ij=k(mod p)}a[i]b[j]。
    assert(p >= 2 && (int)a.size() == p && (int)b.size() == p);
    if (p == 2)
    {
        return {a[0] * b[0] + a[0] * b[1] + a[1] * b[0], a[1] * b[1]};
    }
    int g = primRoot(p);
    vector<int> pw(p - 1), lg(p);
    for (int i = 0, x = 1; i < p - 1; i++)
    {
```

```
        pw[i] = x;
        lg[x] = i;
        x = x * g % p;
    }
    vector<Z> x(p - 1), y(p - 1);
    for (int i = 1; i < p; i++)
    {
        x[lg[i]] = a[i];
        y[lg[i]] = b[i];
    }
    auto z = conv(x, y);
    for (int i = p - 1; i < (int)z.size(); i++)
    {
        z[i - (p - 1)] += z[i];
    }
    z.resize(p - 1);
    vector<Z> ans(p);
    Z sa = 0, sb = 0;
    for (int i = 0; i < p; i++)
    {
        sa += a[i];
        sb += b[i];
    }
    ans[0] = sa * b[0] + (sb - b[0]) * a[0];
    for (int i = 0; i < p - 1; i++)
    {
        ans[pw[i]] = z[i];
    }
    return ans;
}
```

`mulPrimeConv(p,a,b)` 要求 `p` 为素数、`a,b` 都按余数 $0..p-1$ 存放，并返回同样布局的结果。时间 $O(P\log P)$、空间 $O(P)$，在 LC 中 $P\le 524288$；`p=2` 已单独处理。普通加法下标卷积应直接用 10.1.003 NTT，不要把离散对数变换用于非素数模数或带零元的普通环。

10.1.018 模 2^{64} 精确整数卷积

这里的系数环是整数环 $\mathbb{Z}/2^{64}\mathbb{Z}$，乘法包含进位，和 10.1.016 $GF(2^{64})$ 卷积 的无进位域乘法完全不同。依赖 10.1.004 MTT 任意模数卷积 提供的 `arbModConv::NTT`，选五个互素 NTT 素数分别卷积，每个位置的整数真值小于 $\min(n,m)2^{128}$；LC 的长度上界使五模乘积超过此界。Garner 逐项恢复混合进制系数时，`unsigned long long` 的自然溢出恰实现最终模 2^{64}。

```
vector<unsigned long long> convU64(const vector<unsigned long long> &a,
                                   const vector<unsigned long long> &b)
{
    // a、b 是普通整数系数；返回整数卷积对 2^64 取模的无符号 64 位结果。
    if (a.empty() || b.empty())
    {
        return {};
```

```
    }
    int need = a.size() + b.size() - 1;
    int n = bit_ceil((unsigned long long)need);
    assert(n <= (1 << 20)); // 当前五个 NTT 素数均支持这一长度。
    constexpr array<int, 5> mod{998244353, 985661441, 943718401, 935329793, 918552577};
    constexpr array<int, 5> root{3, 3, 7, 3, 5};
    array<vector<int>, 5> rem;
    for (int k = 0; k < 5; k++)
    {
        rem[k] = arbModConv::NTT{mod[k], root[k]}.conv(a, b);
    }
    vector<unsigned long long> ans(need);
    for (int i = 0; i < need; i++)
    {
        array<int, 5> coef{}; // coef[k] 是第 k 个 Garner 系数。
        unsigned long long cur = 0, mul64 = 1;
        for (int k = 0; k < 5; k++)
        {
            unsigned long long seen = 0, mul = 1;
            for (int j = 0; j < k; j++)
            {
                seen = (seen + mul * coef[j]) % mod[k];
                mul = mul * mod[j] % mod[k];
            }
            int delta = (rem[k][i] + mod[k] - seen % mod[k]) % mod[k];
            coef[k] = (unsigned long long)delta * arbModConv::inv((int)mul, mod[k]) % mod[k];
            cur += mul64 * coef[k];
            mul64 *= mod[k];
        }
        ans[i] = cur;
    }
    return ans;
}
```

`convU64(a,b)` 接受无符号 64 位普通整数系数，返回线性卷积后模 2^{64} 的系数；空输入返回空。五个模数都支持长度至多 2^{20} 的 NTT，故在 $|a|,|b| \le 524288$ 下时间 $O((n+m)\log(n+m))$、额外空间 $O(n+m)$。模正奇数时优先用 10.1.004 任意模数卷积，做无进位域运算才使用 10.1.016 $GF(2^{64})$ 卷积。

10.2 多项式基础运算

10.2.001 多项式封装

`Poly` 是后续形式幂级数的轻量载体，直接继承 `vector<Z>`，系数按 $[x^0, x^1, \dots]$ 低次到高次排列，只提供存储、截断 `cut` 和加减乘等基础运算；乘法转交前置的 NTT `conv`，逆元、插值等独立算法留到各自条目。`norm` 删去尾部零系数以规范次数。`midProd(a,b)` 计算 $c_i = \sum_j b_j a_{i+j}$，即只取反转卷积中真正需要的中段，供等比节点插值避免构造无用系数。

```cpp
// 依赖 1.1.008 静态取模类
struct Poly : vector<Z>
{
    using vector<Z>::vector;
    Poly(const vector<Z> &a = {}) : vector<Z>(a)
    {
        // a 按低次到高次给出系数；构造同系数多项式，无返回值。
    }

    Poly(vector<Z> &&a) : vector<Z>(move(a))
    {
        // a 按低次到高次给出系数；移动构造同系数多项式。
    }

    void norm()
    {
        // 无参数；删除末尾多余零系数，无返回值。
        while (!empty() && back() == Z(0))
        {
            pop_back();
        }
    }

    Poly cut(int n) const
    {
        // n 是非负截断长度；返回恰含前 n 项的形式幂级数。
        assert(n >= 0); // 调试检查，可删。
        Poly a(begin(), begin() + min(n, (int)size()));
        a.resize(n);
        return a;
    }

    Poly &operator+=(const Poly &b)
    {
        // b 是要相加的多项式；返回更新后的当前多项式。
        resize(max(size(), b.size()));
        for (int i = 0; i < (int)b.size(); i++)
        {
            (*this)[i] += b[i];
        }
        return *this;
    }

    Poly &operator-=(const Poly &b)
    {
        // b 是要减去的多项式；返回更新后的当前多项式。
        resize(max(size(), b.size()));
        for (int i = 0; i < (int)b.size(); i++)
        {
            (*this)[i] -= b[i];
        }
        return *this;
```

```
    }

    Poly &operator*=(const Poly &b)
    {
        // b 是乘数多项式；返回卷积后的当前多项式。
        if (this == &b)
        {
            Poly a = b;
            return *this = conv(move(a), move(*this));
        }
        return *this = conv(move(*this), b);
    }

    Poly &operator*=(Z k)
    {
        // k 是标量系数；返回所有系数乘 k 后的当前多项式。
        for (auto &x : *this)
        {
            x *= k;
        }
        return *this;
    }

    friend Poly operator+(Poly a, const Poly &b)
    {
        // a、b 是两个多项式；返回系数和。
        return a += b;
    }

    friend Poly operator-(Poly a, const Poly &b)
    {
        // a、b 是两个多项式；返回系数差。
        return a -= b;
    }

    friend Poly operator*(Poly a, const Poly &b)
    {
        // a、b 是两个多项式；返回多项式乘积。
        return a *= b;
    }

    friend Poly operator*(Poly a, Z k)
    {
        // a 是多项式，k 是标量；返回缩放后的多项式。
        return a *= k;
    }

    friend Poly operator*(Z k, Poly a)
    {
        // k 是标量，a 是多项式；返回缩放后的多项式。
        return a *= k;
    }
```

```cpp
};

Poly midProd(const Poly &a, const Poly &b)
{
    // a、b 满足 a.size()>=b.size()>0；返回 c[i]=sum_j b[j]*a[i+j] 的中段乘积。
    assert(!b.empty() && a.size() >= b.size()); // 调试检查，可删。
    int m = a.size() - b.size() + 1;
    if (min((int)b.size(), m) <= 64)
    {
        Poly c(m);
        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < (int)b.size(); j++)
            {
                c[i] += b[j] * a[i + j];
            }
        }
        return c;
    }
    assert(nttFit(b.size(), m)); // 调试检查，可删。
    int n = bit_ceil((unsigned long long)a.size());
    Poly x = a;
    Poly y(b.rbegin(), b.rend());
    x.resize(n);
    y.resize(n);
    dft(x);
    dft(y);
    for (int i = 0; i < n; i++)
    {
        x[i] *= y[i];
    }
    idft(x);
    return Poly(x.begin() + b.size() - 1, x.begin() + a.size());
}
```

依赖本章公共 `Z` 与 10.1.003 NTT 的 `conv`、`dft`、`idft`，直接用 `a+b`、`a-b`、`a*b`、`a.cut(n)`。加减截断为 $O(\max(n,m))$；乘法与 `midProd` 都是一次补齐长度上的 NTT，时间 $O(N\log N)$、额外空间 $O(N)$。`midProd` 要求 `a.size()>=b.size()>0`，返回长度 `a.size()-b.size()+1`。此后各 FPS 运算都建立在此类型上。

10.2.002 乘法、除法、取模

域上多项式带余除法 $a=bq+r$（$\deg r<\deg b$）商余唯一。快速做法把两式系数反转：$\operatorname{rev}(a)=\operatorname{rev}(b)\operatorname{rev}(q)\pmod{x^{n-m+1}}$，于是 $\operatorname{rev}(q)=\operatorname{rev}(a)\cdot\operatorname{rev}(b)^{-1}$ 截断即可，反转回来得 q，再由 $r=a-bq$ 求余。

```cpp
// 依赖 10.2.003 乘法逆元
Poly fpsInvFast(const Poly &a, int n);

```

```cpp
pair<Poly, Poly> polyDivmod(Poly a, Poly b)
{
    // a 是被除式，b 是非零除式；返回 {商，余式}，余式次数严格小于 b。
    a.norm();
    b.norm();
    assert(!b.empty()); // 调试检查，可删。
    if (a.size() < b.size())
    {
        return {{}, a};
    }
    int n = a.size() - b.size() + 1;
    if (a.size() <= 256 || b.size() <= 256)
    {
        Poly q(n);
        Z iv = b.back().inv();
        for (int i = n - 1; i >= 0; i--)
        {
            q[i] = a[i + b.size() - 1] * iv;
            for (int j = 0; j < (int)b.size(); j++)
            {
                a[i + j] -= q[i] * b[j];
            }
        }
        a.resize(b.size() - 1);
        q.norm();
        a.norm();
        return {move(q), move(a)};
    }
    assert(convFit(n, n) && convFit(b.size(), n)); // 调试检查，可删。
    Poly x = a;
    Poly y = b;
    reverse(x.begin(), x.end());
    reverse(y.begin(), y.end());
    Poly q = (x.cut(n) * fpsInvFast(y, n)).cut(n);
    reverse(q.begin(), q.end());
    Poly r = a - b * q;
    r.resize(b.size() - 1);
    q.norm();
    r.norm();
    return {q, r};
}
```

`polyDivmod(a,b)` 返回 `{q,r}`，会删去输入尾零、`b` 不能是零多项式；`{2,3,1}`（x^2+3x+2）除 `{1,1}`（$x+1$）得商 `{2,1}`、余空。被除式或除式长度不超过 256 时使用 $O((\deg q+1)|b|)$ 的长除，否则用 10.2.003 乘法逆元 的 `fpsInvFast`，时间 $O(M(n))$、空间 $O(n)$（$M(n)$ 为 n 项卷积代价）。同机、GCC 14.2.0、`-O2 -DNDEBUG`、固定随机种子与 7 轮中位数基准中，除式长度不超过 256 的规定网格只有被除式长 1000、除式长 256 时 NTT 快约 4%，其余长除更快；该阈值是本机经验值，不是渐进复杂度结论。除式稀疏时用 10.2.006 稀疏多项式除法 的系数递推更省。

下面用整除、带余除法和被除式次数较小三种短例核对 $a=bq+r$：

```cpp
signed main()
{
    auto [q1, r1] = polyDivmod({2, 3, 1}, {1, 1});
    assert((q1 == Poly{2, 1} && r1.empty()));
    auto [q2, r2] = polyDivmod({1, 0, 1}, {1, 1});
    assert((q2 == Poly{-1, 1} && r2 == Poly{2}));
    auto [q3, r3] = polyDivmod({1}, {1, 0, 1});
    assert(q3.empty() && r3 == Poly{1});
    cout << "OK\n";
    return 0;
}
```

10.2.003 乘法逆元

在系数环 R 上，设 $G(Y)$ 是关于未知级数 Y 的一元多项式，或是各次复合均有定义的形式幂级数，精确根 F 满足 $G(F)=0$。目标截断为 x^n 且 $1\le k<n$ 时，若已有 $F^*\equiv F\pmod{x^k}$，令 $m=\min(2k,n)$；G' 表示对 Y 求形式导数而不是对 x 求导。若 $G'(F^*)$ 在 $R[[x]]/(x^m)$ 中是单位，则

$$F_{\text{new}}(x)=F^*(x)-\frac{G(F^*(x))}{G'(F^*(x))}\pmod{x^m}.$$

系数环 R 为域时，上述单位条件等价于 $G'(F^*)$ 的常数项非零；一般环上则要求该常数项本身是 R 中的单位。令 $E=F^*-F\in(x^k)$，代数恒等式 $G(F+E)=G(F)+G'(F)E+E^2H$ 表明校正后误差属于 (x^{2k})，故 $F_{\text{new}}\equiv F\pmod{x^m}$，每轮正确项数翻倍；$G(F^*)$、导数逆和乘积都只需计算到模 x^m。本条取 $G(g)=ag-1$，于是得到常数项非零的 FPS 逆 g 满足 $ag\equiv1\pmod{x^n}$（不是逐系数求逆），以及迭代 $g\leftarrow g(2-ag)$；$\log n$ 轮到位，总代价由最后一轮卷积主导。基础版直接按该式调用两次通用卷积，便于记忆和修改。

快速版只求每轮新增长出的高半段。若 g 已满足 $ag\equiv1\pmod{x^m}$，定义 c 使 $ag\equiv1+x^mc\pmod{x^{2m}}$，则

$$g_{\text{new}}=g-x^m(gc\bmod x^m),\qquad ag_{\text{new}}\equiv1+x^mc-x^mc\equiv1\pmod{x^{2m}}.$$

用长度 $2m$ 的循环卷积计算时，$a\bmod x^{2m}$ 与次数小于 m 的 g 相乘，次数至少 $2m$ 的回卷项只会落入 $[0,m)$，所以中间乘积的 $[m,2m)$ 正是 x^mc；清空低半段后再乘一次 g，所需高半段同样不受回卷影响。最后一轮若 $k=\min(2m,n)<2m$，代码只保留 $a\bmod x^k$ 并将误差截到 $[m,k)$；两次乘积最高次数至多 $k+m-2$，回卷后仍只落在低于 m 的位置，因此 $[m,k)$ 不变。这样可复用第一次变换得到的 g 的频域值，每轮只做五次长度 $2m$ 的变换。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
Poly fpsInv(const Poly &a, int n)
{
    // a 是输入 FPS，n 是 [0,2^22] 内的截断长度；n>0 时 a[0] 必须非零，返回满足 a*g=1 mod x^n 的 g。
    assert(n >= 0 && n <= (1 << 22)); // 调试检查，可删。
    if (n == 0)
    {
```

```
        return {};
    }
    assert(!a.empty() && a[0] != Z(0)); // 调试检查，可删。
    Poly g{a[0].inv()};
    for (int k = 2; k < 2 * n; k <<= 1)
    {
        g = (g * (Poly{2} - a.cut(k) * g)).cut(k);
    }
    return g.cut(n);
}

Poly fpsInvFast(const Poly &a, int n)
{
    // a 是输入 FPS，n 是 [0,2^23] 内的截断长度；n>0 时 a[0] 必须非零，返回满足 a*g=1 mod x^n 的快速逆元。
    assert(n >= 0 && n <= (1 << 23)); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    assert(!a.empty() && a[0] != Z(0)); // 调试检查，可删。

    int m = min<int>(n, 64); // m 是当前已经求出的逆元项数。
    Poly g(m);
    g[0] = a[0].inv();
    for (int i = 1; i < m; i++)
    {
        Z sum = 0;
        for (int j = 1; j <= i && j < (int)a.size(); j++)
        {
            sum += a[j] * g[i - j];
        }
        g[i] = -sum * g[0];
    }

    vector<Z> fg; // fg 保存当前 g 的频域值，供本轮两次乘法复用。
    vector<Z> err; // err 依次保存 a、ag 的高半误差和最终校正乘积。
    int cap = bit_ceil((unsigned long long)n);
    fg.reserve(cap);
    err.reserve(cap);
    while (m < n)
    {
        int k = min(2 * m, n); // k 是本轮要求的逆元项数。
        int z = 2 * m; // z 是本轮循环卷积的 NTT 长度。
        fg.assign(z, Z(0));
        err.assign(z, Z(0));
        copy(g.begin(), g.end(), fg.begin());
        copy(a.begin(), a.begin() + min(k, (int)a.size()), err.begin());

        dft(fg);
        dft(err);
        for (int i = 0; i < z; i++)
```

```
        {
            err[i] *= fg[i];
        }
        idft(err);
        fill(err.begin(), err.begin() + m, Z(0));
        fill(err.begin() + k, err.end(), Z(0));

        dft(err);
        for (int i = 0; i < z; i++)
        {
            err[i] *= fg[i];
        }
        idft(err);

        g.resize(k);
        for (int i = m; i < k; i++)
        {
            g[i] = -err[i];
        }
        m = k;
    }
    return g;
}
```

`fpsInv(a,n)` 与 `fpsInvFast(a,n)` 都把未给出的高次系数视为零；`n=0` 时直接返回空且不限制 `a`，`n>0` 时要求 `a` 非空且 `a[0]≠0`，返回恰 n 项。基础版较短，适合小规模或调试，但两次通用卷积的变换长度更大，在 10.1.003 NTT 的上限下要求 $n \le 2^{22}$；快速版先用 $O(64^2)$ 递推得到前 64 项，再进入高半段迭代，要求 $n \le 2^{23}$。两版均为 $O(M(n))$ 时间、$O(n)$ 额外空间。多项式除法、`ln` 和开根等公共主流程使用 `fpsInvFast`；`fpsInv` 作为更短的 Newton 基准版独立保留。除式稀疏时改用 10.2.014 稀疏 FPS 运算 的递推。

性能参考：在 Ryzen 9 7945HX、Windows 11、GCC 14.2.0、`-std=c++20 -O2 -DNDEBUG` 且不启用 `-march=native` 的环境中，对种子 `0xC0DEC0FFEE` 生成、常数项非零的稠密随机 FPS，三次预热后取 11 次中位数并重复三个进程，$n = 4096, 65536, 131072$ 时 `fpsInvFast/fpsInv` 分别为 $0.537/1.593$ ms、$11.715/32.639$ ms、$24.563/67.647$ ms，即快 $2.97, 2.79, 2.75$ 倍；$n = 131072$ 时，固定 32 位数组并延迟取模的参考实现为 23.035 ms，只再快 6.6%，但其安全累加界与固定变换上限绑定，故这里不采用。

下面用 $1 + x$ 的交错逆元核对系数，并覆盖零截断：

```
signed main()
{
    assert(fpsInv({}, 0).empty());
    assert(fpsInvFast({}, 0).empty());
    Poly want(65);
    for (int i = 0; i < 65; i++)
    {
        want[i] = (i & 1) ? -1 : 1;
    }
    assert(fpsInv({1, 1}, 65) == want);
```

```
    assert(fpsInvFast({1, 1}, 65) == want);
    Poly constant = fpsInvFast({2}, 65);
    assert(constant[0] == Z(2).inv());
    for (int i = 1; i < 65; i++)
    {
        assert(constant[i] == Z(0));
    }
    cout << "OK\n";
    return 0;
}
```

10.2.004 导数、积分

形式导数把 $a_i x^i$ 变为 $i a_i x^{i-1}$，积分做逆操作 $a_i x^i \mapsto \frac{a_i}{i+1} x^{i+1}$ 并把常数项固定为零，要求 $1..n$ 在模下可逆。积分直接复用 1.1.008 静态取模类 的共享 `invTable<mod>`，表只在首次需要更大下标时线性扩展，不逐项调用费马逆元。`batchInv` 用一组前缀积、一次总逆元和一次反向扫描求数组逐项逆元；零项不进入乘积并在答案中保持零。三者是 FPS、插值和点值平移共用的基础零件。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
Poly deriv(const Poly &a)
{
    // a 是输入多项式；返回形式导数。
    if (a.empty())
    {
        return {};
    }
    Poly b(a.size() - 1);
    for (int i = 1; i < (int)a.size(); i++)
    {
        b[i - 1] = a[i] * i;
    }
    return b;
}

Poly integr(const Poly &a)
{
    // a 是输入多项式；返回常数项为 0 的形式积分，次数必须小于 mod-1。
    assert(a.size() < mod); // 调试检查，可删。
    Poly b(a.size() + 1);
    const auto &iv = invTable<mod>(a.size());
    for (int i = 0; i < (int)a.size(); i++)
    {
        b[i + 1] = a[i] * iv[i + 1];
    }
    return b;
}

vector<Z> batchInv(const vector<Z> &a)
{
```

```
    // a 是允许含零的数组；返回逐项逆元，零位置仍为零。
    vector<Z> pre(a.size() + 1, 1);
    vector<Z> ans(a.size());
    for (int i = 0; i < (int)a.size(); i++)
    {
        pre[i + 1] = pre[i] * (a[i] == Z(0) ? Z(1) : a[i]);
    }
    Z suf = pre.back().inv();
    for (int i = (int)a.size() - 1; i >= 0; i--)
    {
        if (a[i] != Z(0))
        {
            ans[i] = suf * pre[i];
            suf *= a[i];
        }
    }
    return ans;
}
```

`deriv(a)`、`integr(a)` 均为 $O(n)$ 时间与输出空间；积分长度须小于模数，常数项由调用方按需另补。共享逆元表若需增长，本次调用另做与新增表长成正比的预处理，后续较短积分直接复用。`batchInv(a)` 允许零项，做一次模逆和 $O(n)$ 次乘法。它们直接服务 10.2.005 `ln`、`exp`、开根、快速幂、10.2.008 拉格朗日单点求值、多点插值 和 10.2.009 连续点值插值与点值平移。

10.2.005 `ln`、`exp`、开根、快速幂

$\ln f$ 由 $(\ln f)' = f' f^{-1}$ 积分得到，要求 $f(0) = 1$。令 $g = \exp f$，由 $g' = f' g$ 可得在线递推 $i g_i = \sum_{j=1}^{i} j a_j g_{i-j}$；默认 `fpsExp` 用 16 叉分块 NTT 加速这条递推，另保留 `fpsExpNewton` 的 $g \leftarrow g(1 - \ln g + f)$ 倍增实现。开根用 $g \leftarrow \frac{1}{2}(g + f g^{-1})$：先剥离最低次项，次数必须为偶；`fpsSqrt(a,n,s)` 使用调用方给出的首项平方根，`fpsSqrt(a,n)` 则复用 7.4.004 Tonelli-Shanks 模平方根 的 `sqrtMod` 自动选择一组。幂 f^k 先提出最低次 x^p 与首系数 c，对归一化部分做 $\exp(k \ln)$ 再乘回 $c^k x^{pk}$。

```
// 依赖 1.1.008 静态取模类
// 依赖 7.4.004 二次剩余
// 依赖 10.2.001 多项式封装
// 依赖 10.2.003 乘法逆元
// 依赖 10.2.004 导数、积分
Poly fpsLog(const Poly &a, int n)
{
    // a 的常数项必须为 1，n 是不超过 2^22 的截断长度；返回 ln(a) mod x^n。
    assert(n >= 0 && n <= (1 << 22)); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    assert(!a.empty() && a[0] == Z(1)); // 调试检查，可删。
    Poly f = a.cut(n);
    return integr((deriv(f) * fpsInvFast(f, n)).cut(n - 1)).cut(n);
}
```

```cpp

Poly fpsExpNewton(const Poly &a, int n)
{
    // a 的常数项必须为 0，n 是 [0,2^22] 内的截断长度；用 Newton 迭代返回 exp(a) mod x^n。
    assert(n >= 0 && n <= (1 << 22)); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    assert(a.empty() || a[0] == Z(0)); // 调试检查，可删。
    Poly g{1};
    for (int k = 2; k < 2 * n; k <<= 1)
    {
        Poly q = a.cut(k) - fpsLog(g, k);
        q[0] += 1;
        g = (g * q).cut(k);
    }
    return g.cut(n);
}

namespace rexp
{
constexpr int B = 16;
constexpr int S = 64;
constexpr int LIM = 1 << 20;

struct RExp
{
    int n; // 实际截断长度。
    int sz; // 补齐后的二次幂长度。
    NTT32 nt; // 预分配切片使用的公共二进制 NTT。
    vector<int> dg; // dg[i]=i*g[i]，是输入指数多项式的导数系数。
    vector<int> f; // 已经求出的 exp(g) 系数。
    vector<int> iv; // iv[i] 是 i 的模逆元，iv[0]=iv[1]=1。
    vector<int> t; // 当前层逆变换的临时数组。
    vector<int> wf; // 各层答案块的预分配频域工作区。
    vector<int> wg; // 各层导数块的预分配频域工作区。
    vector<unsigned long long> sum; // 当前频点的延迟取模乘积和。

    static size_t wsize(int n)
    {
        // n 是当前 16 叉分治区间长度；返回各递归层频域块所需的 int 元素总数。
        size_t ans = 0;
        while (n > S)
        {
            ans += 2 * n;
            n /= B;
        }
        return ans;
    }

    RExp(const Poly &g, int m)
```

```
        : n(m), sz(bit_ceil((unsigned long long)max<int>(1, m))), nt(sz > S ? sz / 8 : 0),
          dg(sz), f(sz), iv(m, 1),
          t(sz > S ? sz / 8 : 0), wf(wsize(sz)), wg(wsize(sz)), sum(t.size())
    {
        // g 是常数项为 0 的输入 FPS，m 是实际截断长度；初始化在线卷积所需全部状态。
        // dg[i]=i*g[i]，使 exp(g)'=g'*exp(g) 成为在线卷积递推。
        for (int i = 1; i < min(m, (int)g.size()); i++)
        {
            dg[i] = (unsigned long long)g[i].x * i % mod;
        }
        f[0] = 1;
        const auto &ziv = invTable<mod>(m - 1);
        for (int i = 1; i < m; i++)
        {
            iv[i] = ziv[i].val();
        }
    }

    void dfs(int l, int w, int *pf, int *pg)
    {
        // l、w 确定当前区间，pf、pg 是本层答案块与导数块的频域工作区；补全该区间答案，无返回值。
        if (w <= S)
        {
            int r = min(n, l + w);
            for (int i = l; i < r; i++)
            {
                unsigned long long v = 0;
                for (int j = l; j < i; j++)
                {
                    v += (unsigned long long)f[j] * dg[i - j];
                    if (((j - l) & 15) == 15)
                    {
                        v %= mod;
                    }
                }
                int x = f[i] + v % mod;
                if (x >= mod)
                {
                    x -= mod;
                }
                f[i] = (unsigned long long)x * iv[i] % mod;
            }
            return;
        }

        int b = w / B;
        int len = 2 * b;
        array<int *, B - 1> tf, tg;
        for (int i = 0; i < B - 1; i++)
        {
            tf[i] = pf + i * len;
            tg[i] = pg + i * len;
```

```
        }

        // 卷积核只与下标差有关，同一层的所有结点复用这些频域块。
        if (l == 0)
        {
            for (int i = 0; i < B - 1 && (i + 1) * b < this->n; i++)
            {
                copy_n(dg.data() + i * b, len, tg[i]);
                nt.dft(tg[i], len);
            }
        }

        for (int i = 0; i < B && l + i * b < this->n; i++)
        {
            if (i)
            {
                fill(sum.begin(), sum.begin() + len, (unsigned long long)(0));
                for (int j = 0; j < i; j++)
                {
                    for (int k = 0; k < len; k++)
                    {
                        sum[k] += (unsigned long long)tf[j][k] * tg[i - j - 1][k];
                    }
                }
                for (int k = 0; k < len; k++)
                {
                    t[k] = sum[k] % mod;
                }
                nt.idft(t.data(), len);
                for (int k = 0; k < b && l + i * b + k < this->n; k++)
                {
                    int &x = f[l + i * b + k];
                    x += t[k + b];
                    if (x >= mod)
                    {
                        x -= mod;
                    }
                }
            }

            dfs(l + i * b, b, pf + 2 * w, pg + 2 * w);
            if (i + 1 < B && l + (i + 1) * b < this->n)
            {
                fill(tf[i], tf[i] + len, (int)0);
                copy_n(f.data() + l + i * b, b, tf[i]);
                nt.dft(tf[i], len);
            }
        }
        fill(pf, pf + (B - 1) * len, (int)0);
    }

    Poly run()
```

```
    {
        // 无参数；执行在线分治并返回输入 FPS 指数的前 n 项。
        dfs(0, sz, wf.data(), wg.data());
        Poly ans(n);
        for (int i = 0; i < n; i++)
        {
            ans[i].x = f[i];
        }
        return ans;
    }
};
}

Poly fpsExpRelax(const Poly &a, int n)
{
    // a 的常数项必须为 0，n 不超过 2^20；用 16 叉在线卷积返回 exp(a) mod x^n。
    assert(n >= 0 && n <= rexp::LIM); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    assert(a.empty() || a[0] == Z(0)); // 调试检查，可删。
    return rexp::RExp(a, n).run();
}

Poly fpsExp(const Poly &a, int n)
{
    // a 的常数项必须为 0，n 是截断长度；默认使用在线分块算法，超长时回退 Newton 迭代。
    assert(n >= 0 && n <= (1 << 22)); // 调试检查，可删。
    if (n <= rexp::LIM)
    {
        return fpsExpRelax(a, n);
    }
    return fpsExpNewton(a, n);
}

optional<Poly> fpsSqrtAt(const Poly &a, int n, int p, Z s)
{
    // a[p] 是前 n 项中的首个非零系数，s 是它的一组平方根；返回对应 FPS 平方根，无解时返回空。
    assert(0 <= p && p < min(n, (int)a.size()) && a[p] != Z(0)); // 调试检查，可删。
    if ((p & 1) || s * s != a[p])
    {
        return nullopt;
    }
    int m = n - p;
    Poly b(a.begin() + p, a.begin() + min((int)a.size(), p + m));
    b.resize(m);
    Poly g{s};
    Z iv2 = Z(2).inv();
    for (int k = 2; k < 2 * m; k <<= 1)
    {
        g = ((g + (b.cut(k) * fpsInvFast(g, k)).cut(k)) * iv2).cut(k);
```

```
    }
    Poly ans(p / 2);
    g = g.cut(n - p / 2);
    ans.insert(ans.end(), g.begin(), g.end());
    return ans.cut(n);
}

optional<Poly> fpsSqrt(const Poly &a, int n, Z s)
{
    // a 是输入 FPS，n 是不超过 2^22 的截断长度，s 是首个非零系数的一组平方根；无平方根时返回空。
    assert(n >= 0 && n <= (1 << 22)); // 调试检查，可删。
    if (n == 0)
    {
        return Poly{};
    }
    int p = 0;
    while (p < min(n, (int)a.size()) && a[p] == Z(0))
    {
        p++;
    }
    if (p == min(n, (int)a.size()))
    {
        return Poly(n);
    }
    return fpsSqrtAt(a, n, p, s);
}

optional<Poly> fpsSqrt(const Poly &a, int n)
{
    // a 是输入 FPS，n 是不超过 2^22 的截断长度；自动求首个非零系数平方根，返回一组平方根或空。
    assert(n >= 0 && n <= (1 << 22)); // 调试检查，可删。
    if (n == 0)
    {
        return Poly{};
    }
    int p = 0;
    while (p < min(n, (int)a.size()) && a[p] == Z(0))
    {
        p++;
    }
    if (p == min(n, (int)a.size()))
    {
        return Poly(n);
    }
    if (p & 1)
    {
        return nullopt;
    }
    auto s = sqrtMod(a[p].val(), mod);
    if (!s)
    {
        return nullopt;
```

```
    }
    return fpsSqrtAt(a, n, p, Z(*s));
}

Poly fpsPow(const Poly &a, unsigned long long k, int n)
{
    // a 是输入 FPS，k 是非负指数，n 是不超过 2^22 的截断长度；返回 a^k mod x^n。
    assert(n >= 0 && n <= (1 << 22)); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    if (k == 0)
    {
        Poly ans(n);
        ans[0] = 1;
        return ans;
    }
    int lim = min(n, (int)a.size());
    int p = 0;
    while (p < lim && a[p] == Z(0))
    {
        p++;
    }
    if (p == lim || (p && k > (unsigned long long)(n - 1) / p))
    {
        return Poly(n);
    }
    Z c = a[p];
    int sh = p * k;
    int m = n - sh;
    Poly b(a.begin() + p, a.begin() + min((int)a.size(), p + m));
    b.resize(m);
    b *= c.inv();
    b = fpsExp(fpsLog(b, m) * Z(k % mod), m) * c.pow(k);
    Poly ans(sh);
    ans.insert(ans.end(), b.begin(), b.end());
    return ans.cut(n);
}
```

`fpsLog(a,n)` 要求 $a_0 = 1$；`fpsExp(a,n)` 与两个显式指数接口要求 $a_0 = 0$；`fpsSqrt(a,n,s)` 要求 s^2 等于首个非零系数，无解返回空。`fpsLog`、`fpsExpNewton`、`fpsSqrt` 均为 $O(M(n))$，`fpsExpRelax` 的 16 叉在线卷积为 $O(n \log^2 n)$；后者渐进复杂度更差，但使用紧凑的 32 位频域数组，在 $n \le 2^{20}$ 时作为默认实现，超过后 `fpsExp` 回退 Newton。当前 NTT 上限下 Newton 路径和依赖基础逆元的接口最多处理 2^{22} 项，各实现峰值空间均为 $O(n)$，`fpsPow` 的复杂度随其调用的指数实现。

性能参考：在 Ryzen 9 7945HX、Windows 11、GCC 14.2.0、`-std=c++20 -O2 -DNDEBUG` 且不启用 `-march=native` 的环境中，对种子 `20260816` 生成的稠密随机 FPS 交替运行两种指数各 7 次，$n = 100000$ 时 `fpsExpRelax`/`fpsExpNewton` 中位数为 $12.123/79.994$ ms，快 6.60 倍；$n = 131072$ 时为 $14.929/78.355$ ms，快 5.25 倍。因此主接口保留 relaxed 路径，Newton 作为更优渐进复杂度和超长输入的独立实现。Newton 路径建立在 10.2.003 乘法逆元 与 10.2.004 导数、积分 上；输入稀疏时用

10.2.014 稀疏 FPS 运算 的 $O(nk)$ 递推。

下面用 $\log(1+x)$、$\exp(x)$ 和一组固定平方的已知系数，分别检查四个接口：

```
signed main()
{
    int n = 6;
    Poly h{1, 2, 3, 4, 5, 6};
    assert(fpsLog(Poly{1, 1}, n) ==
           Poly({0, 1, -Z(2).inv(), Z(3).inv(), -Z(4).inv(), Z(5).inv()}));
    assert(fpsExp(Poly{0, 1}, n) ==
           Poly({1, 1, Z(2).inv(), Z(6).inv(), Z(24).inv(), Z(120).inv()}));
    assert(fpsPow(h, 3, n) == Poly({1, 6, 21, 56, 126, 252}));
    auto r = fpsSqrt(Poly{1, 4, 10, 20, 35, 56}, n, 1);
    assert(r && *r == h);
    assert(!fpsSqrt(Poly{0, 1}, 2, 1));
    cout << "OK\n";
    return 0;
}
```

10.2.006 稀疏多项式除法

除式只有少量非零项时，把 $a = bq$ 展开成 $a_i = \sum_j b_j q_{i-j}$，从低次到高次解出 $q_i = b_0^{-1}(a_i - \sum_{j\ge 1} b_j q_{i-j})$，内层只枚举 b 的非零项，常数项 b_0 必须可逆。公共 `spNorm` 先把任意次序的 {次数，系数} 排序，合并重复次数并删除相消后的零项；稀疏除法与后续全部稀疏 FPS 接口共用这一入口。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
vector<pair<int, Z>> spNorm(vector<pair<int, Z>> f)
{
    // f 是任意次序的稀疏项；返回按次数递增、合并同次且删除零系数后的稀疏项。
    auto cmp = [](const auto &a, const auto &b)
    {
        return a.first < b.first;
    };
    if (!is_sorted(f.begin(), f.end(), cmp))
    {
        sort(f.begin(), f.end(), cmp);
    }
    vector<pair<int, Z>> g;
    for (auto [i, x] : f)
    {
        assert(i >= 0); // 调试检查，可删。
        if (!g.empty() && g.back().first == i)
        {
            g.back().second += x;
        }
        else
        {
            g.push_back({i, x});
```

```
        }
    }
    g.erase(remove_if(g.begin(), g.end(), [](const auto &x)
    {
        return x.second == Z(0);
    }), g.end());
    return g;
}

Poly sparseDiv(const Poly &a, vector<pair<int, Z>> b, int n)
{
    // a 是稠密分子，b 保存非零的 {次数，系数}，n 是商的截断长度；返回 a/b mod x^n。
    assert(n >= 0); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    b = spNorm(move(b));
    b.erase(lower_bound(b.begin(), b.end(), n, [](const auto &x, int y)
    {
        return x.first < y;
    }), b.end());
    assert(!b.empty() && b[0].first == 0 && b[0].second != Z(0)); // 调试检查，可删。
    Z iv = b[0].second.inv();
    Poly q(n);
    for (int i = 0; i < n; i++)
    {
        q[i] = i < (int)a.size() ? a[i] : Z(0);
        for (int j = 1; j < (int)b.size() && b[j].first <= i; j++)
        {
            q[i] -= b[j].second * q[i - b[j].first];
        }
        q[i] *= iv;
    }
    return q;
}
```

`sparseDiv(a,b,n)` 允许 `b` 乱序、重复次数和显式零系数；`spNorm` 后若 `n>0`，必须留下唯一的非零常数项。次数不小于 `n` 的除式项不会影响截断商并会被删除。分子 `{1}` 除 `{{0,1},{2,-1}}` 取 8 项得 `{1,0,1,0,1,0,1,0}`。设规范化后有效项数为 k，已按次数排列时为 $O(nk)$，乱序时另有 $O(k\log k)$ 排序，空间 $O(n+k)$。除式稠密时用 10.2.002 乘法、除法、取模；稀疏求逆等运算见 10.2.014 稀疏 FPS 运算。

10.2.007 多点求值

一次求多项式在许多横坐标上的全部点值。`multiEval` 把乘积树保存在 NTT 域：叶保存 $X-x_i$ 在二次单位根处的值，内部逐点相乘并用一次倍长变换扩展频点；根结点恢复 $P(X)=\prod_i(X-x_i)$ 的低次系数，再通过反转逆元得到向下传递的初始余式信息。向下时，当前频域值乘另一侧子树的乘积，逆变换后取高半，即得到对应子树的信息，直到线性叶输出点值。

`FastNTT` 的长度 $2k$ 频域排列中，前后两个连续 k 块分别补出原长度 k 频点的两组平方根；`twice` 正是复用 10.1.003 NTT 已写明的排列契约，不另建位逆序 NTT。补齐叶取 $x_i = 0$，只增加末尾无用点值，不改变原点顺序。很小的输入直接逐点 Horner，避免建树常数。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
vector<Z> multiEval(const Poly &f, const vector<Z> &x)
{
    // f 是待求值多项式，x 是求值点；用频域乘积树返回与 x 同序的点值。
    int s = x.size();
    if (s == 0)
    {
        return {};
    }
    if (f.empty())
    {
        return vector<Z>(s);
    }
    if (s == 1 || f.size() <= 64 || (__int128)f.size() * s <= 4096)
    {
        vector<Z> ans(s);
        for (int i = 0; i < s; i++)
        {
            for (auto it = f.rbegin(); it != f.rend(); ++it)
            {
                ans[i] = ans[i] * x[i] + *it;
            }
        }
        return ans;
    }

    int n = bit_ceil((unsigned long long)s);
    assert(n <= NTT_LIM && convFit(f.size(), f.size())); // 调试检查，可删。
    vector<Poly> tr(2 * n);
    for (int i = 0; i < n; i++)
    {
        Z y = i < s ? -x[i] : Z(0);
        tr[n + i] = {y + 1, y - 1};
    }

    auto twice = [&](Poly &a)
    {
        // a 是长度 k 的频域值；原地扩展成同一多项式的长度 2k 频域值。
        int k = a.size();
        Z w = 31;
        for (int i = 23; i > bit_width((unsigned long long)k); i--)
        {
            w *= w;
        }
        Z v = 1;
        Poly b = a;
        idft(b);
```

```
        for (int i = 0; i < k; i++)
        {
            b[i] *= v;
            v *= w;
        }
        dft(b);
        a.insert(a.end(), b.begin(), b.end());
    };

    for (int p = n - 1; p; p--)
    {
        auto &a = tr[p << 1];
        auto &b = tr[p << 1 | 1];
        int k = a.size();
        tr[p].resize(k);
        for (int i = 0; i < k; i++)
        {
            tr[p][i] = a[i] * b[i] - 1;
        }
        if (p == 1)
        {
            continue;
        }
        twice(tr[p]);
        for (int i = 0; i < 2 * k; i++)
        {
            tr[p][i] += i < k ? Z(1) : Z(-1);
        }
    }

    Poly a = tr[1];
    idft(a);
    a.push_back(1);
    reverse(a.begin(), a.end());
    a.resize(f.size());
    a = fpsInvFast(a, f.size());
    reverse(a.begin(), a.end());
    a *= f;
    a.erase(a.begin(), a.begin() + f.size() - 1);
    a.resize(n);

    vector<Z> ans(n);
    auto dfs = [&](auto &&self, int p, Poly cur) -> void
    {
        if (p >= n)
        {
            ans[p - n] = cur[0];
            return;
        }
        int k = cur.size();
        dft(cur);
        for (int q = p << 1; q <= (p << 1 | 1); q++)
```

```
        {
            Poly b = tr[q ^ 1];
            for (int i = 0; i < k; i++)
            {
                b[i] *= cur[i];
            }
            idft(b);
            self(self, q, Poly(b.begin() + k / 2, b.end()));
        }
    };
    dfs(dfs, 1, move(a));
    ans.resize(s);
    return ans;
}
```

`PolyTree` 保留普通系数乘积树。它适合同一批点复用多次，且 10.2.008 拉格朗日单点求值、多点插值 需要直接访问根多项式。建树后先算 $a \bmod t_{\text{root}}$，再自顶向下取模；子树不超过 128 个实际点时用 Horner 直接求当前余式，省去大量小规模逆元与卷积。

```
struct PolyTree
{
    static constexpr int S = 128; // 子树点数不超过 S 时直接逐点 Horner 求值。
    // n 保存求值点个数，z 保存补齐到的 2 的幂叶子数。
    int n;
    int z;
    // x 保存各求值点，t 保存乘积树多项式。
    vector<Z> x;
    vector<Poly> t;

    PolyTree(const vector<Z> &points)
        : n(points.size()), z(bit_ceil((unsigned long long)max<int>(1, n))), x(points)
    {
        // points 是全部求值点；建立乘积树。
        assert(z <= NTT_LIM); // 调试检查，可删。
        t.assign(2 * z, Poly{1});
        for (int i = 0; i < n; i++)
        {
            t[z + i] = {-x[i], 1};
        }
        for (int i = z - 1; i; i--)
        {
            t[i] = t[i << 1] * t[i << 1 | 1];
        }
    }

    vector<Z> eval(const Poly &a) const
    {
        // a 是待求值多项式；返回 a(x[i])，顺序与构造参数 x 相同。
        if (n == 0)
        {
            return {};
```

```
        }
        int m = a.size();
        while (m && a[m - 1] == Z(0))
        {
            m--;
        }
        int b = t[1].size();
        if (m >= b && m > 256 && b > 256)
        {
            int q = m - b + 1;
            assert(convFit(q, q) && convFit(b, q)); // 调试检查，可删。
        }
        vector<Z> ans(n);
        auto dfs = [&](auto &&self, int p, int l, int w, Poly r) -> void
        {
            int cnt = max<int>(0, min(n, l + w) - l);
            if (cnt <= S)
            {
                for (int i = l; i < l + cnt; i++)
                {
                    Z v = 0;
                    for (auto it = r.rbegin(); it != r.rend(); ++it)
                    {
                        v = v * x[i] + *it;
                    }
                    ans[i] = v;
                }
                return;
            }
            if (l + w / 2 >= n)
            {
                self(self, p << 1, l, w / 2,
                     polyDivmod(move(r), t[p << 1]).second);
                return;
            }
            Poly left = polyDivmod(r, t[p << 1]).second;
            Poly right = polyDivmod(move(r), t[p << 1 | 1]).second;
            self(self, p << 1, l, w / 2, move(left));
            self(self, p << 1 | 1, l + w / 2, w / 2, move(right));
        };
        dfs(dfs, 1, 0, z, polyDivmod(a, t[1]).second);
        return ans;
    }
};
```

`multiEval(f,x)` 返回与 `x` 同序的点值，允许重复点、空点集与零多项式。设 $F = |f|$、$N = |x|$，要求补齐点数不超过 `NTT_LIM` 且 `convFit(F,F)`；时间 $O(F \log F + N \log^2 N)$，树占 $O(N \log N)$、根逆元与临时量占 $O(F + N)$ 空间。`PolyTree` 依赖 10.2.002 乘法、除法、取模，建树和单次求值均为 $O(M(N) \log N)$ 量级，树占 $O(N \log N)$；构造 `PolyTree tr(x)` 后可多次调用 `tr.eval(a)`。

在 Ryzen 9 7945HX、Windows 11、GCC 14.2.0、`-std=c++20 -O2 -DNDEBUG -pipe` 且不启用 `-march=native` 的环境中，对种子 `20260824` 生成、$F = N$ 的稠密随机输入交错运行 5 轮并取中位数；

$N=4096,16384,65536,131072$ 时，`PolyTree/multiEval` 分别为 $7.028/4.845$、$37.503/21.813$、$186.172/100.043$、$414.858/209.294$ ms。单次大规模求值优先 `multiEval`；需要显式根多项式或复用树时使用 `PolyTree`。原 `PolyTree` 的 128 点阈值来自同机 $N=131072$ 的既有基准，保留不变。其逆问题见 10.2.008 拉格朗日单点求值、多点插值。

10.2.008 拉格朗日单点求值、多点插值

两件事。等距样本 $y_i=f(i)$ 求单点 $f(x)$：拉格朗日基 $\prod_{j\neq i}\frac{x-j}{i-j}$ 的分子用前后缀积 $O(n)$ 拼出、分母是两个阶乘之积带符号，免去逐项 $O(n^2)$。一般互异点插值先求 $P'(x_i)$（$P=\prod(x-x_i)$），用一次前缀积与一次总逆元批量得到全部倒数，再递归按 $q=q_Lt_R+q_Rt_L$ 合并；左右中间多项式在返回后立即释放，不再额外保存整棵插值树。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
// 依赖 10.2.004 导数、积分
Z lagrangeSeq(const vector<Z> &y, long long x)
{
    // y[i]=f(i) 且 deg(f)<y.size()，x 是求值点；返回 f(x)。
    int n = y.size();
    assert(n > 0 && n <= mod); // 调试检查，可删。
    if (0 <= x && x < n)
    {
        return y[x];
    }
    vector<Z> pre(n + 1, 1);
    vector<Z> suf(n + 1, 1);
    vector<Z> fac(n + 1, 1);
    Z p = x;
    for (int i = 0; i < n; i++)
    {
        pre[i + 1] = pre[i] * (p - Z(i));
        fac[i + 1] = fac[i] * Z(i + 1);
    }
    for (int i = n - 1; i >= 0; i--)
    {
        suf[i] = suf[i + 1] * (p - Z(i));
    }
    Z ans = 0;
    for (int i = 0; i < n; i++)
    {
        Z c = pre[i] * suf[i + 1] / fac[i] / fac[n - 1 - i];
        if ((n - 1 - i) & 1)
        {
            c = -c;
        }
        ans += y[i] * c;
    }
    return ans;
}
```

```
Poly interpPoints(const vector<Z> &x, const vector<Z> &y)
{
    // x 是互不相同的横坐标，y 是对应点值；返回次数小于 x.size() 的唯一插值多项式。
    assert(x.size() == y.size()); // 调试检查，可删。
    if (x.empty())
    {
        return {};
    }
    PolyTree tr(x);
    auto d = tr.eval(deriv(tr.t[1]));
    auto iv = batchInv(d);
    vector<Z> w(tr.n);
    for (int i = 0; i < tr.n; i++)
    {
        assert(d[i] != Z(0)); // 调试检查，可删。
        w[i] = y[i] * iv[i];
    }
    auto dfs = [&](auto &&self, int p) -> Poly
    {
        if (p >= tr.z)
        {
            int i = p - tr.z;
            return i < tr.n ? Poly{w[i]} : Poly{};
        }
        Poly left = self(self, p << 1);
        Poly right = self(self, p << 1 | 1);
        return left * tr.t[p << 1 | 1] + right * tr.t[p << 1];
    };
    return dfs(dfs, 1).cut(tr.n);
}
```

等距用 `lagrangeSeq(y,x)`（要求 `y[i]=f(i)`），一般互异点用 `interpPoints(x,y)`；点 $(0,1),(1,4),(2,9)$ 插得 $1+2x+x^2$，`lagrangeSeq({1,4,9},10)=121`。前者 $O(n)$，后者 $O(M(n)\log n)$ 时间、$O(n\log n)$ 空间。依赖 10.2.007 多点求值 的 `PolyTree`；批量平移等距点值另见 10.2.009 连续点值插值与点值平移。

10.2.009 连续点值插值与点值平移

两种平移。系数平移 $f(x+c)$ 用 $[x^k]f(x+c)=\sum_{i\ge k}a_i\binom{i}{k}c^{i-k}$，把它写成两个数组（$a_i i!$ 与 $\frac{c^j}{j!}$）的一次卷积再乘 $\frac{1}{k!}$。连续点值平移由 $f(0),\dots,f(n-1)$ 求 $f(c),\dots,f(c+m-1)$：拉格朗日分母只依赖下标差、写成卷积，`batchInv` 一次求出所有分母逆元。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
// 依赖 10.2.004 导数、积分
Poly taylorShift(const Poly &a, Z c)
{
    // a 保存 f(x) 的系数，c 是平移量；返回 f(x+c) 的系数。
    int n = a.size();
```

```
    if (n == 0)
    {
        return {};
    }
    assert(n <= mod && convFit(n, n)); // 调试检查，可删。
    vector<Z> fac(n, 1);
    vector<Z> ifac(n, 1);
    for (int i = 1; i < n; i++)
    {
        fac[i] = fac[i - 1] * i;
    }
    ifac[n - 1] = fac[n - 1].inv();
    for (int i = n - 1; i; i--)
    {
        ifac[i - 1] = ifac[i] * i;
    }
    Poly x(n);
    Poly y(n);
    Z pw = 1;
    for (int i = 0; i < n; i++)
    {
        x[n - 1 - i] = a[i] * fac[i];
        y[i] = pw * ifac[i];
        pw *= c;
    }
    Poly z = x * y;
    Poly ans(n);
    for (int i = 0; i < n; i++)
    {
        ans[i] = z[n - 1 - i] * ifac[i];
    }
    return ans;
}

vector<Z> shiftSamples(const vector<Z> &y, int c, int m)
{
    // y[i]=f(i)，c 是首个整数点，m 是输出长度；返回模 mod 下 f(c)..f(c+m-1)。
    int n = y.size();
    assert(n > 0 && n < mod && 0 <= m && m <= mod); // 调试检查，可删。
    if (m == 0)
    {
        return {};
    }
    assert(convFit(n, n + m - 1)); // 调试检查，可删。
    vector<Z> fac(n + 1, 1);
    vector<Z> ifac(n + 1, 1);
    for (int i = 1; i <= n; i++)
    {
        fac[i] = fac[i - 1] * i;
    }
    ifac[n] = fac[n].inv();
    for (int i = n; i; i--)
```

```
    {
        ifac[i - 1] = ifac[i] * i;
    }
    vector<Z> a(n);
    vector<Z> den(n + m - 1);
    vector<int> xs(m);
    for (int i = 0; i < n; i++)
    {
        a[i] = y[i] * ifac[i] * ifac[n - 1 - i];
        if ((n - 1 - i) & 1)
        {
            a[i] = -a[i];
        }
    }
    Z cur = Z(c) - Z(n - 1);
    for (auto &x : den)
    {
        x = cur;
        cur += 1;
    }
    cur = c;
    for (auto &x : xs)
    {
        x = cur.val();
        cur += 1;
    }
    auto iv = batchInv(den);
    auto sum = conv(a, iv);
    vector<Z> step(m);
    for (int i = 0; i < m; i++)
    {
        int x = xs[i];
        if (x >= n)
        {
            step[i] = x - n;
        }
    }
    auto istep = batchInv(step);
    vector<Z> ans(m);
    Z fall = 0;
    int pre = -2;
    for (int i = 0; i < m; i++)
    {
        int x = xs[i];
        if (x < n)
        {
            ans[i] = y[x];
            pre = x;
            continue;
        }
        if (pre >= n && x == pre + 1)
        {
```

```
            fall *= Z(x) * istep[i];
        }
        else
        {
            fall = 1;
            for (int j = 0; j < n; j++)
            {
                fall *= x - j;
            }
        }
        ans[i] = fall * sum[i + n - 1];
        pre = x;
    }
    return ans;
}
```

系数平移 `taylorShift(a,c)` 中 `{1,2,3}` 平移 $c=4$ 得 `{57,26,3}`；点值平移 `shiftSamples(y,c,m)` 把参数按模 `998244353` 解释，允许输出区间跨过模数回绕。样本 `{1,4,9}` 从 $c=3$ 取 4 项得 `{16,25,36,49}`，从 $c=998244351$ 取 4 项得 `{1,0,1,4}`。前者要求 `convFit(n,n)`，后者要求 `convFit(n,n+m-1)`；这些条件分别覆盖实际构造的两段卷积。Taylor 平移为 $O(M(n))$，点值平移为 $O(M(n+m)+n+m)$，空间线性。两者复用 10.2.004 导数、积分 的 `batchInv` 和 10.1.003 NTT 的 `conv`，本条不再复制批量求逆。

10.2.010 快速阶乘

在多项式友好模域算远小于 P 的 $n!$。取 $B\approx\sqrt{n}$，构造 $F(x)=\prod_{j=1}^{B}(x+j)$，则 $n!$ 的整块部分是 $F(0)F(B)F(2B)\cdots$ 的乘积——用乘积树建 F、多点求值批量取这些块值，再补乘尾部零头。n 接近模数时先用 Wilson 定理 $(P-1)!\equiv-1$ 把 $n!$ 换成较短的 $(P-1-n)!$。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
Z fastFact(int n)
{
    // n 满足 0<=n<Z::mod()；返回 n!，使用分块多项式乘积与多点求值降低单次复杂度。
    assert(0 <= n && n < Z::mod()); // 调试检查，可删。
    if (n <= 1)
    {
        return 1;
    }
    if (n > (Z::mod() - 1) / 2)
    {
        int m = Z::mod() - 1 - n;
        Z s = m & 1 ? Z(1) : Z(-1);
        return s / fastFact(m);
    }
    int b = max<int>(1, sqrtl(n));
    while ((b + 1) * (b + 1) <= n)
    {
        b++;
    }
```

```
    while (b * b > n)
    {
        b--;
    }
    int q = n / b;
    vector<Z> roots(b);
    vector<Z> xs(q);
    for (int i = 0; i < b; i++)
    {
        roots[i] = -Z(i + 1);
    }
    for (int i = 0; i < q; i++)
    {
        xs[i] = Z(i * b);
    }
    PolyTree prod(roots);
    PolyTree eval(xs);
    auto val = eval.eval(prod.t[1]);
    Z ans = 1;
    for (auto x : val)
    {
        ans *= x;
    }
    for (int x = q * b + 1; x <= n; x++)
    {
        ans *= x;
    }
    return ans;
}
```

批量查询固定取 $B = 2^{15}$。先预处理所有 $(kB)!$；块内部分把因子叶 $x+1,\dots,x+B-1$ 与对应余数的查询叶交错排列，树上向右传播左侧因子前缀，并对查询点多项式取模。这样每个查询一次得到 $\prod_{j=1}^{r}(kB+j)$，不会对每个 n 从 1 线性乘到 n。

```
vector<Z> factBatch(const vector<int> &q)
{
    // q 是所有待查询的非负阶乘下标；返回与 q 同序的 n!。
    // 先用多点求值预处理整块阶乘，再用一棵乘积树批量求全部块内前缀积。
    if (q.empty()) return {};
    constexpr int LOG_BLOCK = 15;
    constexpr int BLOCK = 1 << LOG_BLOCK;
    static const vector<Z> blockFac = []
    {
        int cnt = (mod - 1) / BLOCK;
        vector<Z> roots(BLOCK), xs(cnt);
        for (int i = 0; i < BLOCK; i++)
        {
            roots[i] = -Z(i + 1);
        }
        for (int i = 0; i < cnt; i++)
        {
```

```
            xs[i] = Z(i * BLOCK);
        }
        Poly f;
        {
            PolyTree prod(roots);
            f = prod.t[1];
        }
        vector<Z> val;
        {
            PolyTree eval(xs);
            val = eval.eval(f);
        }
        vector<Z> fac(cnt + 1, 1);
        for (int i = 0; i < cnt; i++)
        {
            fac[i + 1] = fac[i] * val[i];
        }
        return fac;
    }();

    vector<Z> ans(q.size());
    vector<vector<pair<int, int>>> queries(BLOCK);
    int leaves = BLOCK - 1;
    for (int i = 0; i < (int)q.size(); i++)
    {
        int n = q[i];
        assert(0 <= n && n < mod); // 调试检查，可删。
        int base = n / BLOCK * BLOCK;
        int r = n % BLOCK;
        ans[i] = blockFac[n / BLOCK];
        if (r)
        {
            queries[r].push_back({base, i});
            leaves++;
        }
    }

    int z = bit_ceil((unsigned long long)max<int>(1, leaves));
    vector<int> queryLeaf(q.size(), -1);
    vector<Poly> factors(2 * z, Poly{1}), points(2 * z, Poly{1});
    int at = 0;
    for (int r = 1; r < BLOCK; r++)
    {
        factors[z + at++] = {Z(r), Z(1)};
        for (auto [x, id] : queries[r])
        {
            queryLeaf[id] = z + at;
            points[z + at++] = {-Z(x), Z(1)};
        }
    }
    for (int p = z - 1; p > 1; p--)
    {
```

```
        factors[p] = factors[p << 1] * factors[p << 1 | 1];
        points[p] = points[p << 1] * points[p << 1 | 1];
    }
    points[1] = {Z(1)};
    for (int p = 1; p < z; p++)
    {
        Poly prefix = points[p];
        points[p << 1] = polyDivmod(prefix, points[p << 1]).second;
        points[p << 1 | 1] = polyDivmod(prefix * factors[p << 1],
                                        points[p << 1 | 1]).second;
    }
    for (int i = 0; i < (int)q.size(); i++)
    {
        if (queryLeaf[i] != -1)
        {
            const Poly &r = points[queryLeaf[i]];
            ans[i] *= r.empty() ? Z(0) : r[0];
        }
    }
    return ans;
}
```

依赖 10.2.007 多点求值 的 `PolyTree` 和 10.2.002 多项式除法与取模 的 `polyDivmod`。`fastFact(n)` 要求 $0 \le n < P$，时间 $O(M(\sqrt{n})\log n)$、空间 $O(\sqrt{n}\log n)$；`factBatch(q)` 要求所有下标位于 $[0,P)$，令 $T=|q|$，固定 $B=2^{15}$ 时预处理约 $O(M(B)\log B)$，块内批处理约 $O(M(B+T)\log(B+T))$，空间 $O((B+T)\log(B+T))$。T 很小或 n 较小时直接线性阶乘表更省，见 8.1.001 阶乘、逆阶乘与组合数。在 Ryzen 9 7945HX、Windows 11、GCC 14.2.0、`gnu++20 -O2 -pipe` 下，对 $T=100000$、混合小值/近模数/均匀随机的固定输入预热后交替五轮，模板与受控参考端到端中位数为 $638.6/522.8$ ms，即模板约为参考的 1.221 倍；五轮完整输出哈希一致。该数据只说明当前上限输入通过 1.25 倍工程阈值，不代表模板常数普遍更优。

10.2.011 Power Projection

给定 f 和线性泛函 $L(\sum a_i x^i)=\sum_{i=0}^{n-1} a_i w_i$，要批量求 $L(f^k)$。令 $R(x)=\sum_{i=0}^{n-1} w_{n-1-i}x^i$，则 $L(h)=[x^{n-1}]R(x)h(x)$，从而

$$\sum_{k\ge 0} L(f^k)y^k=[x^{n-1}]\frac{R(x)}{1-yf(x)}.$$

实现对这个二元有理式做 Bostan–Mori 消元：每轮把分子分母同乘 $Q(-x,y)$，因为 $Q(x,y)Q(-x,y)$ 只含 x 的偶次幂，只需按目标次数奇偶保留一半行并令 $x^2\mapsto x$。这样 x 方向每轮减半、所需 y 系数窗口逐轮加倍；二维卷积用 Kronecker 间隔打包成一次一维 NTT。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
// 依赖 10.2.003 乘法逆元
vector<Z> powerProj(const Poly &f, const vector<Z> &w, int m)
{
```

```
    // f 是输入多项式，w[i] 是线性泛函在 x^i 上的值，m 是输出项数；返回 L(f^k) (0<=k<m)。
    assert(m >= 0 && m <= (1 << 22) && w.size() <= (1 << 20)); // 调试检查，可删。
    if (m == 0)
    {
        return {};
    }
    int n = w.size();
    vector<Z> ans(m);
    if (n == 0)
    {
        return ans;
    }
    if (m == 1)
    {
        return {w[0]};
    }
    if (n <= 32 && m <= 32)
    {
        Poly p{1};
        Poly ff = f.cut(n);
        for (int k = 0; k < m; k++)
        {
            for (int i = 0; i < min(n, (int)p.size()); i++)
            {
                ans[k] += w[i] * p[i];
            }
            p = (p * ff).cut(n);
        }
        return ans;
    }

    // 将 L(h)=sum w[i]h[i] 改写为 [x^(n-1)] reverse(w)*h,
    // 再用 Bostan-Mori 递归同时求出多个 [x^(n-1)] f^k。
    int k = n - 1;
    Poly ff = f.cut(n);
    poly2::Mat p(n, 2), q(n, 2);
    q(0, 0) = 1;
    for (int i = 0; i < n; i++)
    {
        p(i, 0) = w[n - 1 - i];
        q(i, 1) = -ff[i];
    }
    while (k > 0)
    {
        auto r = q;
        for (int i = 1; i < r.n; i += 2)
        {
            for (int j = 0; j < r.m; j++)
            {
                r(i, j) = -r(i, j);
            }
        }
```

```
        auto z = poly2::conv(p, q, r);
        int rn = k / 2 + 1;
        int cm = min(m, z[0].m);
        poly2::Mat np(rn, cm), nq(rn, cm);
        int o = k & 1;
        for (int i = 0; i < rn; i++)
        {
            int pi = 2 * i + o;
            int qi = 2 * i;
            for (int j = 0; j < cm; j++)
            {
                np(i, j) = z[0](pi, j);
                nq(i, j) = z[1](qi, j);
            }
        }
        p = move(np);
        q = move(nq);
        k /= 2;
    }
    Poly a(m), b(m);
    for (int i = 0; i < min(m, p.m); i++)
    {
        a[i] = p(0, i);
        b[i] = q(0, i);
    }
    return (a * fpsInvFast(b, m)).cut(m);
}
```

依赖 10.1.005 二元 NTT / 二元多项式卷积 的连续 `poly2::Mat` 与共享右因子变换，以及 10.2.003 乘法逆元 的 `fpsInvFast`。`powerProj(f,w,m)` 中 `w[i]=L(x^i)`，返回长度 m 的

$$L(f^0),\dots,L(f^{m-1}),$$

其中每次幂只保留前 `w.size()` 项；`m=0` 返回空，`w` 为空时返回 m 个零。当前实现要求 `w.size()<=2^20`、`m<=2^22`，小于等于 32×32 时自动走朴素递推。设 $n=|w|$，一般时间为 $O(M(n)\log n+M(m))$、峰值空间 $O(n+m)$；当 $m=\Theta(n)$ 时即 $O(n\log^2 n)$。取 `f=1+x`、`w={2,3,5,7}` 时得到 `{2,5,13,33}`。

性能参考：在 Ryzen 9 7945HX、Windows 11、GCC 14.2.0、`-std=c++20 -O2 -DNDEBUG` 且不启用 `-march=native` 的环境中，种子 `20260816` 的 $n=m=131072$ 稠密随机输入单次运行约 1090.03 ms；这是包含分配与二维打包常数的本机数据，不代表其他平台的固定耗时。

10.2.012 多项式复合

求 $f(g(x)) \bmod x^n$。当 $g(0)=0$ 时使用 Kinoshita–Li 的 Graeffe 型递归：分母与其负变量版本相乘后，$Q(z)Q(-z)$ 只保留偶次项；公共 `dft` 已保证相邻频点对应 $z,-z$，并与半长变换中的 z^2 保持同一编号，因此逐对相乘即可把当前系数宽度减半、块数加倍，并保持 `b*w` 不变。递归返回时再用同一对频点恢复分子所需切片，全程不显式生成 $1,g,g^2,\dots$，也不另建私有 NTT。若 $g(0)=c\neq0$，先令 $h=g-c$

并用 Taylor 平移把外层改写成 $f(x+c)$，再计算平移后外层与 h 的零常数项复合；$n \le 32$ 时保留朴素分支减少常数。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
// 依赖 10.2.004 导数、积分
namespace comp
{
void kl(vector<Z> &p, vector<Z> q, int b, int w)
{
    // p 是待更新分子，q 是当前分母，b、w 是二维块数与块宽；原地完成 Kinoshita-Li 递归消元。
    // b*w 保持不变；公共 dft 的相邻配对与半长递归排列保证 q[2i]q[2i+1] 可直接作为半长频域值。
    assert((int)p.size() == 2 * b * w); // 调试检查，可删。
    assert((int)q.size() == 2 * b * w); // 调试检查，可删。
    if (w == 1)
    {
        return;
    }

    q.resize(4 * b * w);
    q[2 * b * w] = 1;
    dft(q);
    if (w > 2)
    {
        vector<Z> nq(2 * b * w);
        for (int i = 0; i < 4 * b * w; i += 2)
        {
            nq[i / 2] = q[i] * q[i + 1];
        }
        idft(nq);
        assert(nq[0] == Z(1)); // 调试检查，可删。
        nq[0] = 0;
        for (int i = 0; i < 2 * b; i++)
        {
            fill(nq.begin() + i * w + w / 2, nq.begin() + (i + 1) * w, Z(0));
        }
        kl(p, move(nq), 2 * b, w / 2);
    }

    dft(p);
    for (int i = 0; i < 4 * b * w; i += 2)
    {
        Z x = q[i];
        q[i] = p[i / 2] * q[i + 1];
        q[i + 1] = p[i / 2] * x;
    }
    idft(q);
    for (int i = 0; i < b; i++)
    {
        auto dst = p.begin() + 2 * i * w;
        auto src = q.begin() + 2 * (i + b) * w;
        copy_n(src, w, dst);
```

```
        fill(dst + w, dst + 2 * w, Z(0));
    }
}

Poly zero(const Poly &f, const Poly &g, int n)
{
    // g[0]=0；用 Kinoshita-Li 算法返回 f(g(x)) mod x^n。
    assert(n > 0 && (g.empty() || g[0] == Z(0))); // 调试检查，可删。
    int s = bit_ceil((unsigned long long)n);
    assert(s <= (1 << 21)); // 调试检查，可删；递归中的最大 NTT 长度为 4*s。
    vector<Z> p(2 * s), q(2 * s);
    for (int i = 0; i < min({n, s, (int)f.size()}); i++)
    {
        p[2 * i] = f[i];
    }
    for (int i = 0; i < min({n, s, (int)g.size()}); i++)
    {
        q[i] = -g[i];
    }
    kl(p, move(q), 1, s);
    Poly ans(n);
    for (int i = 0; i < n; i++)
    {
        ans[i] = p[i];
    }
    return ans;
}
}

Poly composePoly(const Poly &f, const Poly &g, int n)
{
    // f 是外层多项式，g 是内层多项式，n 是不超过 2^21 的截断项数；返回 f(g(x)) mod x^n。
    assert(n >= 0 && n <= (1 << 21)); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    if (f.empty())
    {
        return {};
    }
    Z c = g.empty() ? Z(0) : g[0];
    Poly b = g.cut(n);
    b[0] -= c;
    if (all_of(b.begin(), b.end(), [](Z x) { return x == Z(0); }))
    {
        Z v = 0;
        for (auto it = f.rbegin(); it != f.rend(); ++it)
        {
            v = v * c + *it;
        }
        Poly ans{v};
```

```
        ans.norm();
        return ans;
    }
    if (c != Z(0))
    {
        assert((int)f.size() <= mod && convFit(f.size(), f.size())); // 调试检查，可删。
    }
    Poly a = c == Z(0) ? f : taylorShift(f, c);
    a.resize(min(n, (int)a.size()));
    if (n <= 32)
    {
        Poly ans(n), p{1};
        for (int k = 0; k < (int)a.size(); k++)
        {
            for (int i = 0; i < (int)p.size(); i++)
            {
                ans[i] += a[k] * p[i];
            }
            p = (p * b).cut(n);
        }
        ans.norm();
        return ans;
    }
    Poly ans = comp::zero(a, b, n);
    ans.norm();
    return ans;
}
```

依赖 10.2.009 连续点值插值与点值平移，并固定在模 `998244353` 的 2^{23} 阶 NTT 上。`composePoly(f,g,n)` 在 `n=0` 或 `f` 为空时返回空；快速分支令 $s=2^{\lceil\log_2 n\rceil}$，要求 $s\le 2^{21}$，因为递归最大变换长度为 $4s\le 2^{23}$。当 $g(0)\neq 0$ 时还会对完整 `f` 做 Taylor 平移，故外层长度也必须落在底层卷积上限内。主递归每层做常数次长度 $O(s)$ 的 NTT，共 $O(n\log^2 n)$ 时间；各层保留分母缓冲区，峰值空间为 $O(n\log n)$，非零常数项另加一次 $O(M(|f|))$ 平移。`f=1+2x+3x^2`、`g=4+5x`、$n=4$ 得 `57+130x+75x^2`；更短但更慢的替代是 Brent–Kung 的 $O(\sqrt{n}\,M(n))$ 实现，复合逆见 10.3.005 拉格朗日反演。

同一环境下，对 $n=131072$ 的稠密随机输入，$g(0)=0$ 与非零常数项两种单次运行分别约 400.43 ms、410.63 ms；连同幂投影、复合逆和一次正向校验的整个基准进程峰值工作集约 45.88 MB。

10.2.013 下降幂多项式乘法

下降幂基 $(x)_i=x(x-1)\cdots(x-i+1)$ 下的乘法。把系数除以阶乘转到 EGF 式表示后，下降幂乘法变成点值意义的乘法，处理完再转回：具体是三次卷积——`a`、`b` 各与 $\frac{1}{i!}$ 卷、逐点乘 $i!$、再与带符号的 $\frac{(-1)^i}{i!}$ 卷。

```
// 依赖 1.1.008 静态取模类
vector<Z> fallMul(const vector<Z> &a, const vector<Z> &b)
{
    // a、b 是下降幂基系数；返回二者乘积的同基系数。
    if (a.empty() || b.empty())
```

```
    {
        return {};
    }
    int n = a.size() + b.size() - 1;
    assert(n <= mod); // 调试检查，可删。
    vector<Z> fac(n);
    vector<Z> ifac(n);
    vector<Z> e(n);
    fac[0] = 1;
    for (int i = 1; i < n; i++)
    {
        fac[i] = fac[i - 1] * i;
    }
    ifac[n - 1] = fac[n - 1].inv();
    for (int i = n - 1; i; i--)
    {
        ifac[i - 1] = ifac[i] * i;
    }

    vector<Z> x = conv(a, ifac);
    vector<Z> y = conv(b, ifac);
    x.resize(n);
    y.resize(n);
    for (int i = 0; i < n; i++)
    {
        x[i] *= y[i] * fac[i];
    }
    for (int i = 0; i < n; i++)
    {
        e[i] = i & 1 ? -ifac[i] : ifac[i];
    }
    x = conv(x, e);
    x.resize(n);
    return x;
}
```

依赖 10.1.003 NTT 的 `Z` 与 `conv`。`fallMul(a,b)` 中 `a[i]`、`b[i]` 是下降幂基 $(x)_i$ 的系数，返回同基排列。设 $s=n+m-1$，三次卷积 $O(N\log N)$ 时间、$O(N)$ 空间（$N=2^{\lceil\log_2 s\rceil}$）。普通幂基与下降幂基之间的点值互转另见 10.2.008 拉格朗日单点求值、多点插值 与 10.2.009 连续点值插值与点值平移。

10.2.014 稀疏 FPS 运算

FPS 只有少量非零项时，逆、对数、指数、幂和平方根都能按非零项递推，复杂度依赖有效非零项数 k 而非 n^2。前置 10.2.006 稀疏多项式除法 的 `spNorm` 负责排序、合并同次项和删除零项；各递推共同复用 1.1.008 静态取模类 的 `invTable<mod>`，不再维护另一份稀疏专用逆元表。exp 由 $g'=gf'$，log 由 $(\log f)'=f'/f$，pow 由 $g'f=kgf'$；把最低项写成 $f=x^p cu$ 且 $u(0)=1$ 后，幂先平移 pk 次，平方根要求 p 为偶数并对单位部分使用指数 $\frac{1}{2}$ 的同类递推。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 7.4.004 二次剩余
// 依赖 10.2.001 多项式封装
// 依赖 10.2.006 稀疏多项式除法
vector<Z> sparseInv(int n, vector<pair<int, Z>> f)
{
    // n 是截断长度，f 是常数项非零的稀疏 FPS；返回 1/f 的前 n 项。
    assert(0 <= n && n <= Z::mod()); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    f = spNorm(move(f));
    assert(!f.empty() && f[0].first == 0 && f[0].second != Z(0)); // 调试检查，可删。
    vector<Z> g(n);
    g[0] = f[0].second.inv();
    for (int i = 1; i < n; i++)
    {
        Z sum = 0;
        for (int j = 1; j < (int)f.size() && f[j].first <= i; j++)
        {
            sum += f[j].second * g[i - f[j].first];
        }
        g[i] = -sum * g[0];
    }
    return g;
}

vector<Z> sparseExp(int n, vector<pair<int, Z>> f)
{
    // n 是截断长度，f 保存常数项为 0 的稀疏 FPS；规范化后返回 exp(f) 的前 n 项。
    assert(0 <= n && n <= Z::mod()); // 调试检查，可删。
    f = spNorm(move(f));
    assert(f.empty() || f[0].first > 0 || f[0].second == Z(0)); // 调试检查，可删。
    vector<Z> g(n);
    if (!n)
    {
        return g;
    }
    const auto &iv = invTable<mod>(n - 1);
    g[0] = 1;
    for (int i = 1; i < n; i++)
    {
        for (auto [j, x] : f)
        {
            if (!j)
            {
                continue;
            }
            if (j > i)
            {
                break;
```

```
            }
            g[i] += Z(j) * x * g[i - j];
        }
        g[i] *= iv[i];
    }
    return g;
}

vector<Z> sparseLog(int n, vector<pair<int, Z>> f)
{
    // n 是截断长度，f 保存常数项为 1 的稀疏 FPS；规范化后返回 ln(f) 的前 n 项。
    assert(0 <= n && n <= Z::mod()); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    f = spNorm(move(f));
    assert(!f.empty() && f[0].first == 0 && f[0].second == Z(1)); // 调试检查，可删。
    vector<Z> q(max<int>(0, n - 1));
    vector<Z> g(n);
    const auto &iv = invTable<mod>(n - 1);
    for (int i = 0; i + 1 < n; i++)
    {
        for (auto [j, x] : f)
        {
            if (j == i + 1)
            {
                q[i] += Z(j) * x;
            }
            if (!j)
            {
                continue;
            }
            if (j > i)
            {
                break;
            }
            q[i] -= x * q[i - j];
        }
        g[i + 1] = q[i] * iv[i + 1];
    }
    return g;
}

vector<Z> sparsePow(int n, vector<pair<int, Z>> f, long long k)
{
    // n 是截断长度，f 是可乱序且允许重复次数的稀疏 FPS，k 是非负指数；规范化后返回 f^k 的前 n 项。
    assert(0 <= n && n <= Z::mod() && k >= 0); // 调试检查，可删。
    vector<Z> g(n);
    if (!n)
    {
        return g;
```

```
    }
    if (k == 0)
    {
        g[0] = 1;
        return g;
    }
    auto h = spNorm(move(f));
    if (h.empty())
    {
        return g;
    }
    int p = h[0].first;
    if ((__int128)p * k >= n)
    {
        return g;
    }
    int sh = p * k;
    int m = n - sh;
    Z c = h[0].second;
    vector<pair<int, Z>> q;
    for (auto [i, x] : h)
    {
        if (i - p >= m)
        {
            break;
        }
        q.push_back({i - p, x / c});
    }
    const auto &iv = invTable<mod>(m - 1);
    vector<Z> r(m);
    r[0] = 1;
    long long km = k % mod;
    for (int i = 1; i < m; i++)
    {
        for (int j = 1; j < (int)q.size() && q[j].first <= i; j++)
        {
            int d = q[j].first;
            r[i] += Z((km + 1) * d - i) * q[j].second * r[i - d];
        }
        r[i] *= iv[i];
    }
    Z scale = c.pow(k);
    for (int i = 0; i < m; i++)
    {
        g[sh + i] = scale * r[i];
    }
    return g;
}

optional<vector<Z>> spSqrt(int n, vector<pair<int, Z>> f, Z root)
{
    // n 是截断长度，f 已按次数规范化，root 是首项的一组平方根；返回对应平方根，无解时返回空。
```

```
    assert(0 <= n && n <= Z::mod()); // 调试检查，可删。
    vector<Z> ans(n);
    if (n == 0)
    {
        return ans;
    }
    if (f.empty())
    {
        return ans;
    }
    int p = f[0].first;
    if (p >= n)
    {
        return ans;
    }
    if (p & 1)
    {
        return nullopt;
    }
    if (root * root != f[0].second)
    {
        return nullopt;
    }
    int sh = p / 2;
    int m = n - sh;
    Z c = f[0].second;
    vector<pair<int, Z>> q;
    for (auto [i, x] : f)
    {
        if (i - p >= m)
        {
            break;
        }
        q.push_back({i - p, x / c});
    }
    const auto &iv = invTable<mod>(m - 1);
    Z half = Z(2).inv();
    vector<Z> r(m);
    r[0] = 1;
    for (int i = 1; i < m; i++)
    {
        for (int j = 1; j < (int)q.size() && q[j].first <= i; j++)
        {
            int d = q[j].first;
            r[i] += (Z(3 * d) * half - Z(i)) * q[j].second * r[i - d];
        }
        r[i] *= iv[i];
    }
    for (int i = 0; i < m; i++)
    {
        ans[sh + i] = root * r[i];
    }
```

```
    return ans;
}

optional<vector<Z>> sparseSqrt(int n, vector<pair<int, Z>> f, Z root)
{
    // n 是截断长度，f 是可乱序且允许重复次数的稀疏 FPS，root 是首项平方根；返回对应平方根，无解时返回空。
    return spSqrt(n, spNorm(move(f)), root);
}

optional<vector<Z>> sparseSqrt(int n, vector<pair<int, Z>> f)
{
    // n 是截断长度，f 是稀疏 FPS；自动选择首个非零系数的一组平方根，返回对应平方根的前 n 项，无平方根时返回空。
    assert(0 <= n && n <= Z::mod()); // 调试检查，可删。
    auto g = spNorm(move(f));
    if (n == 0 || g.empty() || g[0].first >= n)
    {
        return vector<Z>(n);
    }
    if (g[0].first & 1)
    {
        return nullopt;
    }
    auto root = sqrtMod(g[0].second.val(), mod);
    if (!root)
    {
        return nullopt;
    }
    return spSqrt(n, move(g), Z(*root));
}
```

所有接口要求 $0 \le n \le P$，并都会先调用 `spNorm`，所以输入可以乱序、含重复次数和显式零项。规范化后 `sparseExp` 要求常数项为 0，`sparseLog` 在 `n>0` 时要求常数项为 1，`sparseInv` 要求常数项非零；零截断直接返回空。`sparsePow` 支持任意最低项和首系数，指数 `k` 非负；`sparseSqrt(n,f,root)` 固定首个非零系数的一组平方根，自动重载复用 `sqrtMod` 选择一组，无解返回空。

若最低非零项次数为 $p > 0$，模 x^n 的等式 $g^2 = f$ 只唯一约束 g 的前 $n - p/2$ 项，最后 $p/2$ 项可以任意；实现把缺失的归一化高次系数视为零并返回一个确定代表，显式 `root` 也只固定受约束前缀的首项选择。规范化为 $O(k \log k)$，逆元表为 $O(n)$，其余递推为 $O(nk)$ 时间、$O(n + k)$ 空间。稠密输入用 10.2.005 ln、exp、开根、快速幂。

下面覆盖零截断、任意最低项幂和显式首根平方根：

```
signed main()
{
    assert(sparseInv(0, {}).empty());
    assert(sparseLog(0, {}).empty());
    assert(sparseInv(5, {{0, 1}, {2, -1}}) ==
           vector<Z>({1, 0, 1, 0, 1}));
    assert(sparsePow(8, {{2, 2}}, 3) ==
```

```
            vector<Z>({0, 0, 0, 0, 0, 0, 8, 0}));
    auto r = sparseSqrt(6, {{2, 4}}, 2);
    assert(r && *r == vector<Z>({0, 2, 0, 0, 0, 0}));
}
```

10.2.015 有限域多项式求根与因式分解

固定模 `998244353` 的主版本先用导数与 half-GCD 分离各重数的平方自由部分，再利用 $\gcd(f, x^{p^d}-x)$ 按不可约因子次数分组。固定次数组用 Cantor–Zassenhaus 随机二分：对随机 $a(x)$ 计算 $a^{(p^d-1)/2} \bmod f$，其与 1 的差通常能切出非平凡因子；指数不构造为大整数，而按 a^{p^i} 的 d 个 Frobenius 共轭逐段相乘。只求域内不同根时，先取 $\gcd(f, x^p - x)$ 直接筛出一次因子，再递归拆分，避免完整分解所有高次不可约因子。另有运行时素数模 `factorPrime`，用朴素乘除和 Frobenius 幂处理小次数输入，包含 $p = 2$、导数恒零和重数为特征倍数的情形。

```
// 依赖 7.1.001 快速幂
// 依赖 10.2.001 多项式封装
// 依赖 10.2.002 乘法、除法、取模
// 依赖 10.2.020 多项式 half-GCD 与模逆
vector<pair<Poly, int>> squareFree(Poly f)
{
    // f 是次数小于 mod 的非零多项式；返回全部首一平方自由因子及其重数。
    f = polyMonic(move(f));
    assert(!f.empty() && (int)f.size() <= Z::mod());
    if (f.size() == 1)
    {
        return {};
    }
    Poly d(f.size() - 1);
    for (int i = 1; i < (int)f.size(); i++)
    {
        d[i - 1] = f[i] * i;
    }
    Poly c = polyGcd(f, d);
    Poly w = polyDivmod(f, c).first;
    vector<pair<Poly, int>> ans;
    for (int k = 1; w.size() > 1; k++)
    {
        Poly y = polyGcd(w, c);
        Poly z = polyDivmod(w, y).first;
        if (z.size() > 1)
        {
            ans.push_back({polyMonic(move(z)), k});
        }
        w = move(y);
        c = polyDivmod(c, w).first;
    }
    return ans;
}

```

```
vector<pair<Poly, int>> distinctDeg(Poly f)
{
    // f 是首一平方自由多项式；返回 {同次数不可约因子乘积，不可约因子次数}。
    vector<pair<Poly, int>> ans;
    Poly x{0, 1}, h = polyMod(x, f);
    for (int d = 1; 2 * d <= (int)f.size() - 1; d++)
    {
        h = polyPowMod(h, Z::mod(), f);
        Poly g = polyGcd(f, h - x);
        if (g.size() > 1)
        {
            ans.push_back({g, d});
            f = polyDivmod(f, g).first;
            if (f.size() == 1)
            {
                break;
            }
            h = polyMod(move(h), f);
        }
    }
    if (f.size() > 1)
    {
        ans.push_back({f, (int)f.size() - 1});
    }
    return ans;
}

void splitEqual(Poly f, int d, mt19937_64 &rng, vector<Poly> &out)
{
    // f 的不可约因子次数均为 d，rng 是随机源，out 接收结果；递归加入全部首一不可约因子，无返回值。
    if ((int)f.size() - 1 == d)
    {
        out.push_back(polyMonic(move(f)));
        return;
    }
    while (true)
    {
        Poly a(f.size() - 1);
        for (auto &x : a)
        {
            x = rng() % Z::mod();
        }
        Poly b{1}, y = move(a);
        for (int i = 0; i < d; i++)
        {
            b = polyMod(b * polyPowMod(y, (Z::mod() - 1) / 2, f), f);
            y = polyPowMod(move(y), Z::mod(), f);
        }
        b -= Poly{1};
        Poly g = polyGcd(f, b);
        if (g.size() == 1 || g.size() == f.size())
        {
```

```
            continue;
        }
        Poly q = polyDivmod(move(f), g).first;
        splitEqual(move(g), d, rng, out);
        splitEqual(move(q), d, rng, out);
        return;
    }
}

vector<pair<Poly, int>> factorPoly(Poly f, unsigned long long seed = 0)
{
    // f 是次数小于 mod 的非零多项式，seed 是随机种子；返回 F_mod 上全部首一不可约因子及重数。
    f.norm();
    assert(!f.empty() && (int)f.size() <= Z::mod());
    mt19937_64 rng(seed);
    vector<pair<Poly, int>> ans;
    auto sf = squareFree(polyMonic(move(f)));
    for (auto &[g, k] : sf)
    {
        auto dd = distinctDeg(move(g));
        for (auto &[h, d] : dd)
        {
            vector<Poly> part;
            splitEqual(move(h), d, rng, part);
            for (auto &q : part)
            {
                ans.push_back({move(q), k});
            }
        }
    }
    sort(ans.begin(), ans.end(), [](const auto &a, const auto &b)
    {
        if (a.first.size() != b.first.size())
        {
            return a.first.size() < b.first.size();
        }
        for (int i = a.first.size() - 1; i >= 0; i--)
        {
            if (a.first[i] != b.first[i])
            {
                return a.first[i].val() < b.first[i].val();
            }
        }
        return a.second < b.second;
    });
    return ans;
}

vector<Z> finiteRoots(const Poly &f, unsigned long long seed = 0)
{
    // f 是非零多项式；用完整因式分解返回所有有限根，根按代数重数重复。
    vector<Z> ans;
```

```
    for (auto [g, k] : factorPoly(f, seed))
    {
        if (g.size() == 2)
        {
            Z x = -g[0] / g[1];
            for (int i = 0; i < k; i++)
            {
                ans.push_back(x);
            }
        }
    }
    sort(ans.begin(), ans.end(), [](Z a, Z b) { return a.val() < b.val(); });
    return ans;
}

void splitRoots(Poly f, mt19937_64 &rng, vector<Z> &ans)
{
    // f 是首一且全部因子互异的一次因子积；递归把所有不同根加入 ans。
    int n = f.size() - 1;
    if (n == 1)
    {
        ans.push_back(-f[0]);
        return;
    }
    while (true)
    {
        Poly a(n);
        for (auto &x : a)
        {
            x = rng() % mod;
        }
        Poly b = polyPowMod(move(a), (mod - 1) / 2, f);
        Poly g = polyGcd(f, b - Poly{1});
        if (g.size() == 1 || g.size() == f.size())
        {
            g = polyGcd(f, b);
        }
        if (g.size() == 1 || g.size() == f.size())
        {
            continue;
        }
        Poly q = polyDivmod(move(f), g).first;
        splitRoots(move(g), rng, ans);
        splitRoots(move(q), rng, ans);
        return;
    }
}

vector<Z> distinctRoot(Poly f, unsigned long long seed = 0)
{
    // f 是非零多项式；返回 F_p 中所有不同有限根，按代表元升序排列且不重复。
    f.norm();
```

```
    assert(!f.empty()); // 调试检查，可删。
    if (f.size() == 1)
    {
        return {};
    }
    Poly x{0, 1};
    Poly h = polyPowMod(x, mod, f);
    Poly g = polyGcd(f, h - x);
    if (g.size() == 1)
    {
        return {};
    }
    mt19937_64 rng(seed);
    vector<Z> ans;
    splitRoots(polyMonic(move(g)), rng, ans);
    sort(ans.begin(), ans.end(), [](Z a, Z b)
    {
        return a.val() < b.val();
    });
    ans.erase(unique(ans.begin(), ans.end()), ans.end());
    return ans;
}

namespace dfac
{
using DPoly = vector<int>;

// 公共 Z/NTT 固定模数，不适用于这里的运行时素数；次数至多百级，故保留朴素运算。
// p 是当前模数，state 是确定性随机状态；公开入口会重置二者，因此本实现不可重入。
int p;
unsigned long long state;

int norm(int x)
{
    // x 是任意整数；返回其模 p 的最小非负代表。
    x %= p;
    if (x < 0)
    {
        x += p;
    }
    return x;
}

void trim(DPoly &a)
{
    // a 是低次在前的动态模多项式；原地删除末尾零系数。
    while (!a.empty() && a.back() == 0)
    {
        a.pop_back();
    }
}

```

```
int deg(const DPoly &a)
{
    // a 是动态模多项式；返回忽略末尾零项后的次数，零多项式返回 -1。
    int n = a.size();
    while (n && a[n - 1] == 0)
    {
        n--;
    }
    return n - 1;
}

DPoly monic(DPoly a)
{
    // a 是动态模多项式；返回首一化结果，零多项式仍为空。
    trim(a);
    if (a.empty())
    {
        return a;
    }
    int iv = powerMod(a.back(), p - 2, p);
    for (auto &x : a)
    {
        x = x * iv % p;
    }
    return a;
}

DPoly add(DPoly a, const DPoly &b)
{
    // a、b 是动态模多项式；返回 a+b。
    a.resize(max(a.size(), b.size()));
    for (int i = 0; i < (int)b.size(); i++)
    {
        a[i] += b[i];
        if (a[i] >= p)
        {
            a[i] -= p;
        }
    }
    trim(a);
    return a;
}

DPoly sub(DPoly a, const DPoly &b)
{
    // a、b 是动态模多项式；返回 a-b。
    a.resize(max(a.size(), b.size()));
    for (int i = 0; i < (int)b.size(); i++)
    {
        a[i] -= b[i];
        if (a[i] < 0)
        {
```

```
            a[i] += p;
        }
    }
    trim(a);
    return a;
}

DPoly mul(const DPoly &a, const DPoly &b)
{
    // a、b 是动态模多项式；返回朴素乘积，适用于次数至多 100 的运行时模数题。
    if (a.empty() || b.empty())
    {
        return {};
    }
    DPoly c(a.size() + b.size() - 1);
    for (int i = 0; i < (int)a.size(); i++)
    {
        for (int j = 0; j < (int)b.size(); j++)
        {
            c[i + j] = (c[i + j] + a[i] * b[j]) % p;
        }
    }
    trim(c);
    return c;
}

pair<DPoly, DPoly> divmod(DPoly a, const DPoly &b)
{
    // a 是被除式，b 是非零动态模多项式；返回 {商，余式}。
    int n = deg(a), m = deg(b);
    assert(m >= 0); // 调试检查，可删。
    a.resize(n + 1);
    if (n < m)
    {
        return {{}, a};
    }
    DPoly q(n - m + 1);
    int iv = powerMod(b[m], p - 2, p);
    for (int i = n; i >= m; i--)
    {
        int x = a[i] * iv % p;
        q[i - m] = x;
        for (int j = 0; j <= m; j++)
        {
            a[i - m + j] = norm(a[i - m + j] - x * b[j] % p);
        }
    }
    trim(q);
    trim(a);
    return {q, a};
}

```

```
DPoly gcd(DPoly a, DPoly b)
{
    // a、b 是动态模多项式；返回首一化 gcd。
    trim(a);
    trim(b);
    while (!b.empty())
    {
        DPoly r = divmod(move(a), b).second;
        a = move(b);
        b = move(r);
    }
    return monic(move(a));
}

DPoly mulMod(const DPoly &a, const DPoly &b, const DPoly &m)
{
    // a、b 是动态模多项式，m 是非零模多项式；返回 a*b mod m。
    return divmod(mul(a, b), m).second;
}

DPoly powMod(DPoly a, int e, const DPoly &m)
{
    // a 是动态模多项式，e 是非负指数，m 是非零模多项式；返回 a^e mod m。
    DPoly r = divmod({1}, m).second;
    a = divmod(move(a), m).second;
    while (e)
    {
        if (e & 1)
        {
            r = mulMod(r, a, m);
        }
        e >>= 1;
        if (e)
        {
            a = mulMod(a, a, m);
        }
    }
    return r;
}

unsigned long long rnd()
{
    // 无参数；返回确定性 xorshift 伪随机数并更新 state。
    state ^= state << 7;
    state ^= state >> 9;
    return state;
}

DPoly rndPoly(int n)
{
    // n 是严格次数上界；返回次数小于 n 的随机动态模多项式。
    DPoly a(n);
```

```
    for (auto &x : a)
    {
        x = rnd() % p;
    }
    trim(a);
    return a;
}

DPoly edfOdd(const DPoly &a, int d, const DPoly &m)
{
    // a 是测试多项式，d 是不可约因子次数，m 是待拆多项式；返回奇素数域的 EDF 分裂值。
    DPoly x = powMod(a, (p - 1) / 2, m);
    DPoly r = divmod({1}, m).second;
    for (int i = 0; i < d; i++)
    {
        r = mulMod(r, x, m);
        if (i + 1 < d)
        {
            x = powMod(move(x), p, m);
        }
    }
    return r;
}

DPoly edfTwo(const DPoly &a, int d, const DPoly &m)
{
    // a 是测试多项式，d 是不可约因子次数，m 是待拆多项式；返回 F_2 的 EDF 迹值。
    DPoly x = divmod(a, m).second;
    DPoly r;
    for (int i = 0; i < d; i++)
    {
        r = add(move(r), x);
        if (i + 1 < d)
        {
            x = mulMod(x, x, m);
        }
    }
    return r;
}

void splitEq(DPoly f, int d, vector<DPoly> &ans)
{
    // f 是首一且全部不可约因子次数为 d 的多项式；递归把全部首一不可约因子加入 ans。
    int n = deg(f);
    if (n == d)
    {
        ans.push_back(move(f));
        return;
    }
    DPoly g;
    do
    {
```

```
        DPoly a = rndPoly(n);
        DPoly h = p == 2 ? edfTwo(a, d, f)
                         : sub(edfOdd(a, d, f), {1});
        g = gcd(f, move(h));
    } while (deg(g) <= 0 || deg(g) == n);
    DPoly q = divmod(move(f), g).first;
    splitEq(move(g), d, ans);
    splitEq(move(q), d, ans);
}

struct Fac
{
    // f 是首一不可约因子，e 是其代数重数。
    DPoly f;
    int e;
};

vector<Fac> factor(DPoly f)
{
    // f 是非零动态模多项式；返回全部首一不可约因子及其重数。
    f = monic(move(f));
    vector<Fac> ans;
    if (deg(f) <= 0)
    {
        return ans;
    }
    DPoly x{0, 1};
    DPoly h = x;
    for (int d = 1; deg(f) > 0 && 2 * d <= deg(f); d++)
    {
        h = powMod(move(h), p, f);
        DPoly g = gcd(f, sub(h, x));
        if (deg(g) <= 0)
        {
            continue;
        }
        vector<DPoly> part;
        splitEq(move(g), d, part);
        for (auto &q : part)
        {
            int e = 0;
            while (deg(f) >= deg(q))
            {
                auto [u, r] = divmod(f, q);
                if (!r.empty())
                {
                    break;
                }
                f = move(u);
                e++;
            }
            ans.push_back({move(q), e});
```

```
        }
    }
    if (deg(f) > 0)
    {
        ans.push_back({monic(move(f)), 1});
    }
    return ans;
}
}

struct DynFactor
{
    // f 是首一不可约因子，e 是其在输入多项式中的代数重数。
    vector<int> f;
    int e;
};

vector<DynFactor> factorPrime(vector<int> f, int p,
                              unsigned long long seed = 88172645463325252ULL)
{
    // f 是低次在前系数，p 是素数模数；返回 F_p 上的首一不可约分解及重数。
    assert(p >= 2); // 调试检查，可删。
    using namespace dfac;
    dfac::p = p;
    state = seed ? seed : 88172645463325252ULL;
    for (auto &x : f)
    {
        x = norm(x);
    }
    trim(f);
    assert(!f.empty()); // 调试检查，可删。
    auto part = factor(move(f));
    vector<DynFactor> ans;
    for (auto &x : part)
    {
        ans.push_back({move(x.f), x.e});
    }
    sort(ans.begin(), ans.end(), [](const auto &a, const auto &b)
    {
        if (a.f.size() != b.f.size())
        {
            return a.f.size() < b.f.size();
        }
        return a.f < b.f;
    });
    return ans;
}
```

`factorPoly(f,seed)` 忽略原多项式的非零常数倍，返回全部首一不可约因子及重数；`finiteRoots` 按代数重数展开所有一次因子，`distinctRoot` 只返回互异有限根。固定模接口排除零多项式并要求 $\deg f < 998244353$，因此导数恒零时只能是常数；half-GCD 来自 10.2.020 多项式 half-GCD 与模逆。设 $n = \deg f$，完整分解是随机化算法，期望时间约 $O(nM(n)\log p)$，卷积与递归临时空间至少为线性量级；需

要复现实验时显式传固定 seed。

factorPrime(f,p,seed) 的 p 必须是素数，系数按低次到高次给出并先规范到 $[0,p)$；返回的每个 DynFactor 保存首一不可约因子和代数重数。标量模幂直接复用 7.1.001 基础、位运算与边界安全 的 powerMod；运行时模数的多项式表示、朴素乘除与 gcd 无法由固定模 Z/Poly/NTT 承担，才保留为局部实现。它不用 NTT，适合次数约百以内的运行时模数题；为这一级规模引入动态 NTT 反而增加约束。按朴素乘除、各次数 Frobenius 扫描和随机拆分保守估计，期望时间为 $O(n^4 \log p)$ 次小模运算，空间 $O(n^2)$。该版本通过反复整除记录重数，能覆盖 $p=2$ 和特征整除重数的情形；大次数固定模仍优先使用 factorPoly。

下面检查固定分解的因子、重数与根；实际开发时也应把输出因子乘回原首一式作为证书检查：

```
signed main()
{
    Poly f{72, -84, 14, 17, -8, 1}; // 即 (x-2)^2(x-3)^2(x+2)。
    auto fac = factorPoly(f, 0);
    assert((fac == vector<pair<Poly, int>>({{{2, 1}, 1}, {{-3, 1}, 2}, {{-2, 1}, 2}})));
    assert(finiteRoots(f, 0) == vector<Z>({2, 2, 3, 3, -2}));
    assert(distinctRoot(f, 0) == vector<Z>({2, 3, -2}));
    auto dyn = factorPrime({1, 0, 0, 0, 1}, 2);
    assert(dyn.size() == 1 && dyn[0].f == vector<int>({1, 1}) && dyn[0].e == 4);
}
```

10.2.016 等差、等比节点求值与插值

等差节点 $c, c+1, \dots$ 已由 10.2.009 连续点值插值与点值平移 的 shiftSamples 覆盖；这里补等比节点 $a, ar, ar^2, \dots$。恒等式 $ij=\binom{i+j}{2}-\binom{i}{2}-\binom{j}{2}$ 把 r^{ij} 拆成三个三角数幂，使全部点值化为一次卷积。插值用等比节点的重心权与 q-二项式系数恢复系数；triPow(r,n)[i]=r^{i(i-1)/2} 是两方向共同底座，其中所需卷积中段直接调用 midProd，不再先求完整卷积再切片。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.2.004 导数、积分
vector<Z> triPow(Z r, int n)
{
    // r 是公比，n 是非负长度；返回 a[i]=r^(i(i-1)/2) 的前 n 项。
    assert(n >= 0); // 调试检查，可删。
    vector<Z> a(n);
    if (!n)
    {
        return a;
    }
    a[0] = 1;
    Z pw = 1;
    for (int i = 0; i + 1 < n; i++)
    {
        a[i + 1] = a[i] * pw;
        pw *= r;
    }
    return a;
}
```

```
21
22  vector<Z> geomEval(vector<Z> f, Z a, Z r, int m)
23  {
24      // f 是低次在前的多项式，a、r 给出点 a*r^i，m 是点数；返回依次对应的 m 个点值。
25      assert(m >= 0);
26      if (m == 0)
27      {
28          return {};
29      }
30      int n = f.size();
31      if (n == 0)
32      {
33          return vector<Z>(m);
34      }
35      if (r == Z(0))
36      {
37          vector<Z> ans(m, f[0]);
38          Z x = 1, y = 0;
39          for (auto c : f)
40          {
41              y += c * x;
42              x *= a;
43          }
44          ans[0] = y;
45          return ans;
46      }
47      assert(convFit(n, n + m - 1)); // 调试检查，可删。
48      Z pw = 1;
49      for (auto &x : f)
50      {
51          x *= pw;
52          pw *= a;
53      }
54      auto p = triPow(r, n + m - 1);
55      auto ip = triPow(r.inv(), max(n, m));
56      for (int i = 0; i < n; i++)
57      {
58          f[i] *= ip[i];
59      }
60      reverse(f.begin(), f.end());
61      auto z = conv(f, p);
62      vector<Z> ans(m);
63      for (int i = 0; i < m; i++)
64      {
65          ans[i] = z[i + n - 1] * ip[i];
66      }
67      return ans;
68  }
69
70  vector<Z> geomInterp(vector<Z> y, Z a, Z r)
71  {
72      // y[i]=f(a*r^i)，a、r 非零且 r^i≠1(1<=i<y.size())；返回次数小于 y.size() 的 f。
```

```
    int n = y.size();
    if (n <= 1)
    {
        return y;
    }
    assert(a != Z(0) && r != Z(0));
    assert(nttFit(n, n)); // 调试检查，可删。
    vector<Z> pw(2 * n - 1);
    pw[0] = 1;
    Z ir = r.inv();
    for (int i = 1; i < 2 * n - 1; i++)
    {
        pw[i] = pw[i - 1] * r;
    }
    auto tpw = triPow(r, 2 * n - 1);
    auto itpw = triPow(ir, 2 * n - 1);
    vector<Z> s(n, 1);
    for (int i = 1; i < n; i++)
    {
        assert(pw[i] != Z(1));
        s[i] = s[i - 1] * (Z(1) - pw[i]);
    }
    auto is = batchInv(s);
    Z sn = s[n - 1] * (Z(1) - pw[n]);
    for (int i = 0; i < n; i++)
    {
        y[i] *= tpw[n - 1 - i] * itpw[n - 1] * is[i] * is[n - 1 - i];
        if (i & 1)
        {
            y[i] = -y[i];
        }
        y[i] *= itpw[i];
    }
    auto f = midProd(Poly(tpw), Poly(y));
    for (int i = 0; i < n; i++)
    {
        f[i] *= itpw[i];
    }
    vector<Z> g(n);
    g[0] = 1;
    for (int i = 1; i < n; i++)
    {
        g[i] = tpw[i] * sn * is[i] * is[n - i];
        if (i & 1)
        {
            g[i] = -g[i];
        }
    }
    f = conv(f, g);
    f.resize(n);
    reverse(f.begin(), f.end());
    Z ia = a.inv();
```

```
    Z ap = 1;
    for (auto &x : f)
    {
        x *= ap;
        ap *= ia;
    }
    return f;
}
```

`geomEval(f,a,r,m)` 返回 $f(ar^0),\dots,f(ar^{m-1})$，允许 $r=0$；非退化分支要求 `convFit(n,n+m-1)`。`geomInterp(y,a,r)` 恢复次数小于 n 的唯一多项式，$n>1$ 时要求 $a,r\neq 0$、$1,r,\dots,r^{n-1}$ 互异且 `nttFit(n,n)`。两者依赖 10.2.001 多项式封装 的 `midProd`、10.2.004 导数、积分 的 `batchInv` 和公共 `conv`，时间分别为 $O(M(n+m))$、$O(M(n))$，空间线性。

下面检查等比节点、$r=0$ 退化和插值复原：

```
signed main()
{
    vector<Z> f{1, 2, 3};
    assert(geomEval(f, 2, 3, 4) == vector<Z>({17, 121, 1009, 8857}));
    assert(geomInterp(geomEval(f, 2, 3, 3), 2, 3) == f);
    assert(geomEval({1, 2, 3}, 7, 0, 4) == vector<Z>({162, 1, 1, 1}));
}
```

10.2.017 二元形式幂级数逆

在矩形截断 $0\le i<n,0\le j<m$ 下求 g 使 $fg=1$。一元 Newton 的 $g\leftarrow 2g-fg^2$ 不能把两维同时翻倍，因为矩形外的项不构成所需理想；改按总次数分层：若 g 已在 $i+j<L$ 正确，则 fg^2 的 $L\le i+j<2L$ 部分取负就是新系数，低次保持不动。每轮仍调用 10.1.005 二元 NTT / 二元多项式卷积 的 `conv2`，输入先按当前总次数清零；第二次卷积前把 g^2 裁到当前矩形，避免把必然不会使用的高行高列继续打包进一维 NTT。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.1.003 NTT
// 依赖 10.1.005 二元 NTT / 二元多项式卷积
vector<vector<Z>> fpsInv2(const vector<vector<Z>> &f, int n, int m)
{
    // f 是二维 FPS，n、m 是两维截断长度；返回满足 f*g=1 mod (x^n,y^m) 的 g。
    assert(n >= 0 && m >= 0);
    if (n == 0 || m == 0)
    {
        return {};
    }
    assert(!f.empty() && !f[0].empty() && f[0][0] != Z(0));
    assert(((__int128)2 * n - 1) * ((__int128)2 * m - 1) <= NTT_LIM + 2); // 调试检查，可删。
    vector<vector<Z>> g(1, vector<Z>(1, f[0][0].inv()));
    for (int len = 1; len < n + m - 1; len <<= 1)
    {
        int rn = min(n, len * 2);
```

```
        int cm = min(m, len * 2);
        vector<vector<Z>> a(rn, vector<Z>(cm));
        for (int i = 0; i < rn && i < (int)f.size(); i++)
        {
            for (int j = 0; j < cm && j < (int)f[i].size(); j++)
            {
                if (i + j < len * 2)
                {
                    a[i][j] = f[i][j];
                }
            }
        }
        auto gg = conv2(g, g);
        gg.resize(rn);
        for (auto &row : gg)
        {
            row.resize(cm);
        }
        auto ggf = conv2(move(gg), move(a));
        g.resize(rn);
        for (auto &row : g)
        {
            row.resize(cm);
        }
        for (int i = 0; i < rn; i++)
        {
            for (int j = 0; j < cm; j++)
            {
                if (len <= i + j && i + j < len * 2)
                {
                    g[i][j] = -ggf[i][j];
                }
            }
        }
    }
    g.resize(n);
    for (auto &row : g)
    {
        row.resize(m);
    }
    return g;
}
```

`fpsInv2(f,n,m)` 要求常数项非零，返回 $n \times m$ 的矩形截断逆；输入可以比目标矩形小，缺项按零处理。最外层两次二维卷积打包后的结果长度不超过 $(2n-1)(2m-1)$，因此还要求该值不超过公共 `conv` 可处理的 `NTT_LIM+2`。总时间为 $O(nm(\log n+\log m))$ 次模域运算、峰值空间 $O(nm)$，常数约为每层两次二维卷积；只做二维乘法直接用 `conv2`，一元逆用 10.2.003 乘法逆元。

下面检查 $1+x+y$ 的 3×3 矩形截断逆：

```
signed main()
{
    vector<vector<Z>> f{{1, 1, 0}, {1, 0, 0}, {0, 0, 0}};
    vector<vector<Z>> inv{{1, -1, 1}, {-1, 2, -3}, {1, -3, 6}};
    assert(fpsInv2(f, 3, 3) == inv);
}
```

10.2.018 多项式序列乘积

给定一组长度差异很大的多项式时，固定层数的完全二叉乘积树会让短多项式过早与长多项式相乘。这里按当前系数长度建立小根堆，每次取最短的两个相乘再放回，等价于对卷积规模做 Huffman 式合并；`conv` 接收移动后的缓冲区，已经合并的中间多项式不会继续占用系数存储。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
Poly productPolys(vector<Poly> a)
{
    // a 是待相乘的多项式序列；返回保持卷积长度的总乘积，空序列返回常数 1。
    Z c = 1;
    vector<Poly> f;
    f.reserve(a.size());
    for (auto &x : a)
    {
        if (x.empty())
        {
            return {};
        }
        if (x.size() == 1)
        {
            c *= x[0];
        }
        else
        {
            f.push_back(move(x));
        }
    }
    if (f.empty())
    {
        return {c};
    }
    vector<Poly> pool;
    pool.reserve(2 * f.size());
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> q;
    for (auto &x : f)
    {
        int p = pool.size();
        pool.push_back(move(x));
        q.push({pool[p].size(), p});
    }
    while (q.size() > 1)
```

```
    {
        int x = q.top().second;
        q.pop();
        int y = q.top().second;
        q.pop();
        assert(convFit(pool[x].size(), pool[y].size())); // 调试检查，可删。
        vector<Z> z = conv(move(pool[x]), move(pool[y]));
        int p = pool.size();
        pool.emplace_back(move(z));
        q.push({pool[p].size(), p});
    }
    Poly ans = move(pool[q.top().second]);
    ans *= c;
    return ans;
}
```

`productPolys(a)` 返回全部元素的通常乘积，空序列的乘积定义为常数 1，序列中出现零多项式时结果为空多项式。设 k 是多项式个数、$D=1+\sum_i \deg a_i$ 是非零输入总乘积的系数长度，在准线性卷积模型下时间为 $O(M(D)\log k + k\log k)$，峰值额外空间为 $O(D+k)$；规模接近时普通分层乘积树常数相近，规模悬殊时优先队列更稳。

下面检查空积与三个因子的总乘积：

```
signed main()
{
    assert(productPolys({}) == Poly{1});
    assert((productPolys({Poly{1, 1}, Poly{1, -1}, Poly{2}}) ==
            Poly{2, 0, -2}));
}
```

10.2.019 单项式基转 Newton 基

对节点 $p_0,\dots,p_{n-1}$ 定义 $B_0(x)=1$、$B_i(x)=\prod_{j=0}^{i-1}(x-p_j)$。目标是把低次在前的单项式表示 $f(x)$ 转成唯一的 Newton 表示 $f(x)=\sum_{i=0}^{n-1} b_i B_i(x)$；节点允许重复，因为 B_i 始终是次数为 i 的首一多项式。实现先用乘积树构造各段节点多项式，再通过反转多项式与 FPS 逆完成一次批量商信息提取，最后沿树递归恢复全部 b_i。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
vector<Z> monoToNewton(const Poly &f, const vector<Z> &p)
{
    // f 是低次在前的单项式基系数，p 是 Newton 基的节点；两者等长，返回各级前缀乘积的系数。
    int n = f.size();
    assert((int)p.size() == n); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    int z = bit_ceil((unsigned long long)n);
```

```
    assert(z <= NTT_LIM / 2); // 调试检查，可删。
    vector<Poly> prod(2 * z, Poly{1});
    for (int i = 0; i < z; i++)
    {
        Z x = i < n ? p[i] : Z(0);
        prod[z + i] = {-x, 1};
    }
    for (int i = z - 1; i; i--)
    {
        prod[i] = prod[i << 1] * prod[i << 1 | 1];
    }

    Poly a = f.cut(z);
    reverse(a.begin(), a.end());
    Poly root = prod[1];
    reverse(root.begin(), root.end());
    Poly h = (a * fpsInvFast(root, z)).cut(z);
    reverse(h.begin(), h.end());

    vector<Z> ans(n);
    auto dfs = [&](auto &&self, int u, Poly cur) -> void
    {
        if (u >= z)
        {
            int i = u - z;
            if (i < n)
            {
                ans[i] = cur[0];
            }
            return;
        }
        int m = prod[u << 1 | 1].size() - 1;
        Poly q = cur * prod[u << 1 | 1];
        Poly left(q.begin() + m, q.begin() + 2 * m);
        Poly right(cur.begin() + m, cur.end());
        self(self, u << 1, move(left));
        self(self, u << 1 | 1, move(right));
    };
    dfs(dfs, 1, move(h));
    return ans;
}
```

`monoToNewton(f,p)` 要求 `f.size()==p.size()`，返回的第 i 项就是 B_i 的系数；补齐长度 $z=2^{\lceil\log_2 n\rceil}$ 还须满足 $z\le \texttt{NTT_LIM}/2$，从而根节点乘积和反转卷积均落在公共 NTT 可调用域内。p_{n-1} 只参与统一的树形计算，不改变次数小于 n 的最终表示。依赖 10.2.003 乘法逆元，时间 $O(M(n)\log n)$，乘积树占 $O(n\log n)$ 系数空间。若只需在互异节点间做点值插值，直接使用 10.2.008 拉格朗日单点求值、多点插值；这里解决的是基转换，节点不要求互异。

```
signed main()
{
    assert(monoToNewton({11, -13, 6}, {1, 2, 3}) ==
```

```
            vector<Z>({4, 5, 6}));
}
```

10.2.020 多项式 half-GCD 与模逆

普通多项式欧几里得算法每轮都对完整余式做一次除法。half-GCD 只观察高半系数，递归求出一个 2×2 多项式矩阵，把余式对 $(f,g)^T$ 的第二项次数至少降低一半；必要时补一次欧几里得步，再继续递归。`half` 的输入必须已经严格次数递减，因此 `polyGcd` 遇到等次数余式时先做普通 Euclid 一步，不能把等长切片递归下去。矩阵始终表示若干次变换 $(a,b)\mapsto(b,a-qb)$ 的乘积，因此最终首行同时给出 Bézout 系数，满足 $d=xf+yg$。

矩阵乘法的多个卷积共享同一变换长度：本实现直接调用 10.1.003 快速 NTT 的公共 `dft`、`idft`，每个输入只正变换一次，再在频域完成两项点积；没有复制单位根表或蝶形。欧几里得步直接复用 `polyDivmod` 已算出的余式，并按稀疏矩阵公式更新变换矩阵，避免再乘一次原余式对。长度不超过 128 时普通 Euclid 常数更小，因此作为递归叶子。

```
// 依赖 10.2.001 多项式封装
// 依赖 10.2.002 乘法、除法、取模
Poly polyMonic(Poly a)
{
    // a 是输入多项式；返回删除高次零项后的首一化结果，零多项式返回空。
    a.norm();
    if (!a.empty())
    {
        a *= a.back().inv();
    }
    return a;
}

Poly polyMod(Poly a, const Poly &m)
{
    // a 是被除式，m 是非零模多项式；返回 a 除以 m 的余式。
    return polyDivmod(move(a), m).second;
}

Poly polyPowMod(Poly a, long long e, const Poly &m)
{
    // a 是底数多项式，e 是非负指数，m 是次数至少为 1 的模多项式；返回 a^e mod m。
    assert(e >= 0 && m.size() > 1); // 调试检查，可删。
    Poly r{1};
    a = polyMod(move(a), m);
    while (e)
    {
        if (e & 1)
        {
            r = polyMod(r * a, m);
        }
        e >>= 1;
        if (e)
```

```
        {
            a = polyMod(a * a, m);
        }
    }
    return r;
}

namespace hg
{
using Pair = array<Poly, 2>;
using Mat = array<Poly, 4>;

Poly add(Poly a, const Poly &b)
{
    // a、b 是两个多项式；返回去除高次零项后的 a+b。
    a += b;
    a.norm();
    return a;
}

Poly sub(Poly a, const Poly &b)
{
    // a、b 是两个多项式；返回去除高次零项后的 a-b。
    a -= b;
    a.norm();
    return a;
}

int mulLen(const Poly &a, const Poly &b)
{
    // a、b 是两个多项式；返回乘积系数数，任一多项式为零时返回 0。
    return a.empty() || b.empty() ? 0 : a.size() + b.size() - 1;
}

Mat mul(const Mat &a, const Mat &b)
{
    // a、b 是按行展开的 2×2 多项式矩阵；返回矩阵乘积。
    array<int, 4> s{
        max(mulLen(a[0], b[0]), mulLen(a[1], b[2])),
        max(mulLen(a[0], b[1]), mulLen(a[1], b[3])),
        max(mulLen(a[2], b[0]), mulLen(a[3], b[2])),
        max(mulLen(a[2], b[1]), mulLen(a[3], b[3]))}; // s[i] 是结果第 i 项在消去高次零前的长度。
    int n = bit_ceil((unsigned long long)max({1LL, s[0], s[1], s[2], s[3]}));
    bool zero = false; // zero 表示存在零输入，此时逐项卷积更适合稀疏矩阵。
    for (const auto &x : a)
    {
        zero |= x.empty();
    }
    for (const auto &x : b)
    {
        zero |= x.empty();
    }
```

```
    if (zero || n < 64)
    {
        return {add(a[0] * b[0], a[1] * b[2]),
                add(a[0] * b[1], a[1] * b[3]),
                add(a[2] * b[0], a[3] * b[2]),
                add(a[2] * b[1], a[3] * b[3])};
    }
    assert(n <= (1 << 23)); // 调试检查，可删。
    array<Poly, 8> f{a[0], a[1], a[2], a[3], b[0], b[1], b[2], b[3]};
    for (auto &x : f)
    {
        x.resize(n);
        dft(x);
    }
    Mat c{Poly(n), Poly(n), Poly(n), Poly(n)};
    for (int i = 0; i < n; i++)
    {
        c[0][i] = f[0][i] * f[4][i] + f[1][i] * f[6][i];
        c[1][i] = f[0][i] * f[5][i] + f[1][i] * f[7][i];
        c[2][i] = f[2][i] * f[4][i] + f[3][i] * f[6][i];
        c[3][i] = f[2][i] * f[5][i] + f[3][i] * f[7][i];
    }
    for (int i = 0; i < 4; i++)
    {
        idft(c[i]);
        c[i].resize(s[i]);
        c[i].norm();
    }
    return c;
}

Pair mul(const Mat &a, const Pair &b)
{
    // a 是按行展开的 2×2 多项式矩阵，b 是两维列向量；返回矩阵与列向量的乘积。
    int s0 = max(mulLen(a[0], b[0]), mulLen(a[1], b[1]));
    int s1 = max(mulLen(a[2], b[0]), mulLen(a[3], b[1]));
    int n = bit_ceil((unsigned long long)max({1LL, s0, s1}));
    bool zero = b[0].empty() || b[1].empty();
    for (const auto &x : a)
    {
        zero |= x.empty();
    }
    if (zero || n < 128)
    {
        return {add(a[0] * b[0], a[1] * b[1]),
                add(a[2] * b[0], a[3] * b[1])};
    }
    assert(n <= (1 << 23)); // 调试检查，可删。
    array<Poly, 6> f{a[0], a[1], a[2], a[3], b[0], b[1]};
    for (auto &x : f)
    {
        x.resize(n);
```

```
        dft(x);
    }
    Pair c{Poly(n), Poly(n)};
    for (int i = 0; i < n; i++)
    {
        c[0][i] = f[0][i] * f[4][i] + f[1][i] * f[5][i];
        c[1][i] = f[2][i] * f[4][i] + f[3][i] * f[5][i];
    }
    idft(c[0]);
    idft(c[1]);
    c[0].resize(s0);
    c[1].resize(s1);
    c[0].norm();
    c[1].norm();
    return c;
}

Mat id()
{
    // 无参数；返回 2×2 多项式单位矩阵。
    Mat a;
    a[0] = {1};
    a[3] = {1};
    return a;
}

Mat step(const Poly &q)
{
    // q 是一次欧几里得除法的商；返回把 (a,b) 变为 (b,a-qb) 的矩阵。
    Mat a;
    a[1] = {1};
    a[2] = {1};
    a[3] = sub({}, q);
    return a;
}

Mat stepMul(Mat a, const Poly &q)
{
    // a 是变换矩阵，q 是欧几里得商；返回 step(q)*a，并利用 step(q) 的稀疏结构。
    Poly x = q * a[2];
    Poly y = q * a[3];
    return {move(a[2]), move(a[3]),
            sub(move(a[0]), x), sub(move(a[1]), y)};
}

Mat halfSmall(Pair a)
{
    // a 是次数递减且第一项长度不超过 128 的余式对；用普通 Euclid 返回半降矩阵。
    int m = a[0].size() / 2;
    Mat ans = id();
    while (!a[1].empty() && (int)a[1].size() > m)
    {
```

```
        auto [q, r] = polyDivmod(move(a[0]), a[1]);
        ans = stepMul(move(ans), q);
        a = {move(a[1]), move(r)};
    }
    return ans;
}

Mat half(const Pair &a)
{
    // a[0]、a[1] 是次数递减的余式对；返回把第二项次数降到第一项一半以下的变换矩阵。
    int m = a[0].size() / 2;
    if ((int)a[1].size() <= m)
    {
        return id();
    }
    if ((int)a[0].size() <= 128)
    {
        return halfSmall(a);
    }
    Mat ans = half({Poly(a[0].begin() + m, a[0].end()),
                    Poly(a[1].begin() + m, a[1].end())});
    Pair b = mul(ans, a);
    if ((int)b[1].size() <= m)
    {
        return ans;
    }
    auto [q, r] = polyDivmod(move(b[0]), b[1]);
    ans = stepMul(move(ans), q);
    b = {move(b[1]), move(r)};
    if ((int)b[1].size() <= m)
    {
        return ans;
    }
    int cut = 2 * m + 1 - b[0].size();
    Mat suf = half({Poly(b[0].begin() + cut, b[0].end()),
                    Poly(b[1].begin() + cut, b[1].end())});
    return mul(suf, ans);
}

Mat all(const Pair &a)
{
    // a[0]、a[1] 是次数递减的余式对；返回把第二项降为零的完整变换矩阵。
    Mat ans = half(a);
    Pair b = mul(ans, a);
    if (b[1].empty())
    {
        return ans;
    }
    auto [q, r] = polyDivmod(move(b[0]), b[1]);
    ans = stepMul(move(ans), q);
    b = {move(b[1]), move(r)};
    if (b[1].empty())
```

```
    {
        return ans;
    }
    return mul(all(b), ans);
}
}

Poly polyGcd(Poly f, Poly g)
{
    // f、g 是不全为零的多项式；两者非空时长度均不超过 2^22，返回首一 gcd，不计算最终 Bézout 系数。
    f.norm();
    g.norm();
    assert(!f.empty() || !g.empty()); // 调试检查，可删。
    if (f.empty())
    {
        return polyMonic(move(g));
    }
    if (g.empty())
    {
        return polyMonic(move(f));
    }
    assert(max(f.size(), g.size()) <= (1 << 22)); // 调试检查，可删。
    if (f.size() < g.size())
    {
        swap(f, g);
    }
    hg::Pair a{move(f), move(g)};
    while (!a[1].empty())
    {
        if (a[1].size() < a[0].size() &&
            (int)a[1].size() > (int)a[0].size() / 2)
        {
            hg::Mat t = hg::half(a);
            a = hg::mul(t, a);
            if (a[1].empty())
            {
                break;
            }
        }
        Poly r = polyDivmod(move(a[0]), a[1]).second;
        a = {move(a[1]), move(r)};
    }
    return polyMonic(move(a[0]));
}

tuple<Poly, Poly, Poly> polyExtGcd(Poly f, Poly g)
{
    // f、g 是不全为零的多项式；两者非空时长度均不超过 2^22，返回 {d,x,y}，满足 d=xf+yg=gcd(f,g)，d
    ↪ 未首一化。
    f.norm();
    g.norm();
    assert(!f.empty() || !g.empty()); // 调试检查，可删。
```

```
    if (g.empty())
    {
        return {f, Poly{1}, Poly{}};
    }
    if (f.empty())
    {
        return {g, Poly{}, Poly{1}};
    }
    assert(max(f.size(), g.size()) <= (1 << 22)); // 调试检查，可删。
    if (f.size() < g.size())
    {
        auto [d, y, x] = polyExtGcd(move(g), move(f));
        return {move(d), move(x), move(y)};
    }
    using namespace hg;
    auto [q, r] = polyDivmod(f, g);
    Mat ans = step(q);
    Pair rem{g, move(r)};
    if (!rem[1].empty())
    {
        ans = mul(all(rem), ans);
    }
    Poly d = add(f * ans[0], g * ans[1]);
    return {move(d), move(ans[0]), move(ans[1])};
}

optional<Poly> polyInvMod(Poly f, Poly m)
{
    // f 是待求逆多项式，m 是非零模多项式；两者非空且 m 非常数时长度均不超过 2^22，可逆时返回次数小于 m 的逆元，否则返回空。
    f.norm();
    m.norm();
    assert(!m.empty()); // 调试检查，可删。
    if (m.size() == 1)
    {
        return Poly{};
    }
    if (!f.empty())
    {
        assert(max(f.size(), m.size()) <= (1 << 22)); // 调试检查，可删。
    }
    auto [d, x, y] = polyExtGcd(move(f), m);
    if (d.size() != 1)
    {
        return nullopt;
    }
    x *= d[0].inv();
    return polyDivmod(move(x), move(m)).second;
}
```

`polyExtGcd(f,g)` 要求二者不全为零，两者非空时最大长度不超过 2^{22}，返回未首一化的 `{d,x,y}`；同一长度界也用于 `polyGcd`。`polyInvMod(f,m)` 要求 $\deg m \geq 1$，当 `f` 非零时两输入同样不超过 2^{22}；仅当

$\gcd(f,m)$ 为非零常数时返回次数小于 m 的逆元，否则返回空。这个界保证 half-GCD 内部两项点积补齐后的公共 NTT 长度不超过 2^{23}。设 $n=\max(\deg f,\deg g)$，时间 $O(M(n)\log n)$；父层矩阵与余式、子层高半切片以及一次批量变换的长度都按层构成几何级数，峰值额外空间为 $O(n)$。只需要首一 gcd 时由 10.2.015 有限域多项式求根与因式分解 的 `polyGcd` 调用本条；小次数现场实现仍可用普通欧几里得算法换取更短代码。

```
signed main()
{
    auto inv = polyInvMod({0, 1}, {1, 0, 1});
    assert(inv && *inv == Poly({0, -1}));
    assert(!polyInvMod({1, 1}, {1, 2, 1}));
}
```

10.2.021 拉格朗日非连续点单点求值

给定互异横坐标 $x_0,x_1,\dots,x_{n-1}$ 及对应点值 $y_i=f(x_i)$，在一个查询点 t 求次数小于 n 的插值多项式 $f(t)$。直接枚举每个拉格朗日基函数

$$f(t)=\sum_{i=0}^{n-1} y_i\prod_{j\neq i}\frac{t-x_j}{x_i-x_j}$$

即可；它不要求横坐标连续或等距，代码短但时间为 $O(n^2)$。这里使用已有的模整数类 `Z`，因此所有分母必须在模下可逆；若只需一次求值且 n 较小，优先用本条，若要恢复多项式或处理大规模数据则用 10.2.008 拉格朗日单点求值、多点插值 的 `interpPoints`。

```
// 依赖 1.1.008 静态取模类
Z lagrange(const vector<Z> &x, const vector<Z> &y, Z t)
{
    // x 是互异横坐标，y 是对应点值，t 是查询点；返回插值多项式在 t 处的值。
    assert(x.size() == y.size() && !x.empty()); // 调试检查，可删。
    int n = x.size();
    for (int i = 0; i < n; i++)
    {
        if (x[i] == t)
        {
            return y[i];
        }
    }
    Z ans = 0;
    for (int i = 0; i < n; i++)
    {
        Z c = y[i];
        for (int j = 0; j < n; j++)
        {
            if (i != j)
            {
                c *= (t - x[j]) / (x[i] - x[j]);
            }
        }
```

```
        ans += c;
    }
    return ans;
}
```

例如三点 $(0,1),(1,4),(2,9)$ 插值为 $1+2x+x^2$，`lagrange({0,1,2},{1,4,9},10)` 返回 121。横坐标必须两两不同，且每个 x_i-x_j 在模下非零；时间 $O(n^2)$、额外空间 $O(1)$。它只求一个点，不生成多项式系数；连续整数点应使用 `lagrangeSeq`，需要多个任意点时使用 `interpPoints`。

10.2.022 计数版子集和生成函数

给定非负整数多重集 a，对每个 $0\le s\le T$ 求和为 s 的子集个数（模 998244353），即 $\prod_i(1+x^{a_i})$ 的前 $T+1$ 项。把权值相同的元素合并，设权值 v 出现 c_v 次，取对数

$$\ln\prod_i(1+x^{a_i})=\sum_{v\ge1}c_v\ln(1+x^v)=\sum_{v\ge1}c_v\sum_{k\ge1}\frac{(-1)^{k+1}}{k}x^{kv},$$

对每个出现过的 v 只需枚举 $k\le T/v$，总项数不超过调和级数 $T\ln T$；分母 $k\le T<998244353$，模意义下可逆。对这个对数做 `fpsExp` 即得答案；权值为 0 的元素可选可不选，整体乘 2^{c_0}。

```
// 依赖 10.2.005 ln、exp、开根、快速幂
vector<Z> subsetCount(const vector<int> &a, int T)
{
    // a 是非负整数权，0<=T<2^22；返回 f[0..T]，f[s] 是和为 s 的子集个数模 998244353。
    // prod (1+x^v) = exp(sum_v sum_k (-1)^{k+1} x^{kv}/k)，按权值合并后调和级数求 ln，再 exp。
    assert(0 <= T && T < (1LL << 22)); // 调试检查，可删。
    vector<int> cnt(T + 1); // cnt[v] 是权值为 v 的元素个数（只统计 1..T）。
    int zeros = 0;
    for (int x : a)
    {
        assert(x >= 0); // 调试检查，可删。
        if (!x)
        {
            zeros++;
        }
        else if (x <= T)
        {
            cnt[x]++;
        }
    }
    const auto &iv = invTable<mod>(T);
    Poly lg(T + 1);
    for (int v = 1; v <= T; v++)
    {
        if (cnt[v])
        {
            for (int k = 1; k * v <= T; k++)
            {
                lg[k * v] += Z(cnt[v]) * (k & 1 ? iv[k] : -iv[k]);
            }
```

```
        }
    }
    // 权为 0 的元素可选可不选，整体乘 2^zeros。
    Poly f = fpsExp(lg, T + 1);
    Z sc = Z(2).pow(zeros);
    for (auto &x : f)
    {
        x *= sc;
    }
    return f;
}
```

`subsetCount(a,T)` 要求权值非负、$0 \le T < 2^{22}$，返回长 $T+1$ 的 `vector<Z>`，大于 T 的权值不影响结果。时间 $O(T \log T)$，空间 $O(T)$。依赖 10.2.005 `ln、exp、开根、快速幂` 的 `fpsExp` 与第 1 章的逆元表 `invTable`。

只问可达性时用 12.1.010 布尔母函数 / 有界子集和可达性；nT 不大时直接做 01 背包更简单。下面核对多重集 $\{1,2,3,3,0\}$：

```
signed main()
{
    auto f = subsetCount({1, 2, 3, 3, 0}, 6);
    vector<int> want{2, 2, 2, 6, 4, 4, 6}, got;
    for (auto x : f)
    {
        got.push_back(x.val());
    }
    assert(got == want);
}
```

10.3 生成函数

10.3.001 普通生成函数 OGF

普通生成函数 $A(x)=\sum a_n x^n$ 编码无标号序列，x^n 系数直接是大小 n 的对象数；加法对应互斥选择、乘法（卷积）对应有序拼接、$\frac{1}{1-A}$ 对应序列。这段只给最基础的系数乘法示意，要求所有乘加结果均在有符号 64 位范围内；需要取模或大规模计算时改用多项式全家桶。

```
// 依赖 1.1.008 静态取模类
namespace ogf
{
vector<int> ogfProduct(const vector<int> &a, const vector<int> &b)
{
    // a、b 是 OGF 系数；返回乘积系数，即普通卷积。
    if (a.empty() || b.empty())
    {
        return {};
    }
    vector<int> c(a.size() + b.size() - 1);
```

```
    for (int i = 0; i < (int)a.size(); i++)
    {
        for (int j = 0; j < (int)b.size(); j++)
        {
            c[i + j] += a[i] * b[j];
        }
    }
    return c;
}
}
```

无标号计数用 OGF、有标号用 10.3.002 指数生成函数 EGF，两者系数归一方式不同、不能混算。这里的 `ogfProduct` 是 $O(nm)$ 朴素卷积示意；真正计算依赖 10.2.001 多项式封装 与 10.1.003 NTT。

10.3.002 指数生成函数 EGF

指数生成函数 $A(x)=\sum \frac{a_n}{n!}x^n$ 编码有标号对象，乘法 $\frac{a_i}{i!}\cdot\frac{b_j}{j!}$ 自动带上 $\binom{i+j}{i}$，正好表示把标签集合划给两部分，所以有标号结构的不交合并对应 EGF 乘法。这里演示乘 $n!$ 前系数的卷积，要求帕斯卡表和全部乘加中间值均在有符号 64 位范围内。

```
namespace egf
{
vector<int> egfProduct(const vector<int> &a, const vector<int> &b)
{
    // a[n]、b[n] 是乘 n! 前的 EGF 系数；返回有标号乘积系数。
    if (a.empty() || b.empty())
    {
        return {};
    }
    vector<int> ans(a.size() + b.size() - 1);
    vector<int> c(b.size(), 1);
    for (int i = 0; i < (int)a.size(); i++)
    {
        if (i)
        {
            for (int j = 1; j < (int)b.size(); j++)
            {
                c[j] += c[j - 1];
            }
        }
        for (int j = 0; j < (int)b.size(); j++)
        {
            ans[i + j] += c[j] * a[i] * b[j];
        }
    }
    return ans;
}
}
```

有标号用 EGF、无标号用 10.3.001 普通生成函数 OGF，系数不可直接互换。实现逐行维护 $\binom{i+j}{i}$，对长

度分别为 n, m 的输入用 $O(nm)$ 时间、$O(n+m)$ 空间；实际模意义大规模计算以 10.2.001 多项式封装 承载，指数、对数等运算见 10.2.005 ln、exp、开根、快速幂。

10.3.003 q-int / q-Binomial

q-整数 $[n]_q = 1 + q + \cdots + q^{n-1}$、q-阶乘与高斯二项式 $\binom{n}{k}_q$ 把普通计数按维数或逆序数加权：$\binom{n}{k}_q$ 数 $\mathbb{F}_q$ 上 n 维空间的 k 维子空间，或按逆序对数加权的选择。下面使用递推 $\binom{n}{k}_q = \binom{n-1}{k}_q + q^{n-k}\binom{n-1}{k-1}_q$，$q = 1$ 退化为普通组合数；所有整数系数须在有符号 64 位范围内。

```
namespace gauss_binom
{
vector<int> qBinomial(int n, int k)
{
    // n、k 是高斯二项式参数；返回按 q 次数升序的整数系数。
    if (k < 0 || k > n)
    {
        return {};
    }
    k = min(k, n - k);
    int w = n - k;
    vector<int> a{1};
    for (int i = 1; i <= k; i++)
    {
        vector<int> b(i * w + 1);
        for (int j = 0; j <= i * w; j++)
        {
            if (j < (int)a.size())
            {
                b[j] += a[j];
            }
            if (j >= w + i && j - w - i < (int)a.size())
            {
                b[j] -= a[j - w - i];
            }
            if (j >= i)
            {
                b[j] += b[j - i];
            }
        }
        a = move(b);
    }
    return a;
}
}
```

`qBinomial(n,k)` 返回按 q 次数升序的整数系数多项式。实现先用对称性令 $k \le n-k$，再逐项应用

$$(1-q^i)\binom{n-k+i}{i}_q = (1-q^{n-k+i})\binom{n-k+i-1}{i-1}_q,$$

因此每轮只保留一个系数数组。令 $D=k(n-k)$ 为答案次数，时间 $O(kD)$、空间 $O(D)$；$q=1$ 特化为普通组合数，见 8.1.001 阶乘、逆阶乘与组合数。

10.3.004 二项式型生成函数

二项式变换 $b_n=\sum_k \binom{n}{k}a_k$ 及其反演 $a_n=\sum_k(-1)^{n-k}\binom{n}{k}b_k$，对应 EGF 乘以 e^x / e^{-x}。二项式型序列满足 $p_n(x+y)=\sum_k \binom{n}{k}p_k(x)p_{n-k}(y)$，其 EGF 是某形式幂级数的幂，适合处理卷积型多项式序列。以下整数版要求帕斯卡表和全部乘加中间值均在有符号 64 位范围内。

```cpp
namespace binom_trans
{
vector<int> binomTrans(const vector<int> &a, bool inverse = false)
{
    // a 是原序列，inverse 表示做逆变换；返回二项式变换结果。
    int n = a.size();
    vector<int> b(n), c{1};
    for (int i = 0; i < n; i++)
    {
        if (i)
        {
            c.push_back(0);
        }
        for (int j = i; j >= 1; j--)
        {
            c[j] += c[j - 1];
        }
        for (int j = 0; j <= i; j++)
        {
            int sign = inverse && ((i - j) & 1) ? -1 : 1;
            b[i] += sign * c[j] * a[j];
        }
    }
    return b;
}
}
```

`binomTrans(a,inverse)` 做正/逆二项式变换。实现只滚动保存一行 Pascal 系数，用 $O(n^2)$ 时间、$O(n)$ 空间；写成模意义 EGF 乘 $e^{\pm x}$ 可借 10.2.005 ln、exp、开根、快速幂 降到 $O(n\log n)$。它是容斥的生成函数形式，与 10.3.002 指数生成函数 EGF 同源。

10.3.005 拉格朗日反演

由隐式方程 $F=x\phi(F)$（$\phi(0)\neq 0$）求递归结构计数时，拉格朗日反演给出 $[x^n]F=\frac{1}{n}[x^{n-1}]\phi(x)^n$，更一般地 $[x^n]H(F)=\frac{1}{n}[x^{n-1}]H'(x)\phi(x)^n$。`lagrangeCoef` 直接用一次 FPS 幂计算指定系数。

完整复合逆则把全部系数打包。设 $f(x)=ax+O(x^2)$、$a\neq 0$，先归一化 $F=f/a=x+O(x^2)$，令 $d=n-1$ 并定义

$$A(z)=\sum_{i=1}^{d}\frac{d}{i}[x^d]F(x)^i z^{d-i}.$$

由拉格朗日反演逐项比较可得 $F^{\langle -1\rangle}(z)\equiv zA(z)^{-1/d}\pmod{z^{d+1}}$。因此先用 10.2.011 Power Projection 一次求出全部 $[x^d]F^i$，再做一次单位 FPS 的分数次幂，最后代入 $z=x/a$ 恢复原函数的逆；这避免对每个次数分别计算不同幂。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 7.4.004 二次剩余
// 依赖 10.1.003 NTT
// 依赖 10.2.001 多项式封装
// 依赖 10.2.003 乘法逆元
// 依赖 10.2.004 导数、积分
// 依赖 10.2.005 ln、exp、开根、快速幂
// 依赖 10.2.011 Power Projection
// 依赖 10.2.012 多项式复合
namespace lagrange
{
int lagrangeCoef(vector<int> phi, int n)
{
    // phi 是 phi(t) 的低次系数，n 为正；返回方程 w=x*phi(w) 中 [x^n]w。
    assert(n > 0 && n < mod && !phi.empty() && phi[0] != 0); // 调试检查，可删。
    Poly f(n);
    for (int i = 0; i < min(n, (int)phi.size()); i++)
    {
        f[i] = phi[i];
    }
    return (fpsPow(f, n, n)[n - 1] / Z(n)).val();
}
}

namespace cinv
{
Poly powUnit(const Poly &f, Z k, int n)
{
    // f[0]=1，k 是模意义指数，n 是截断长度；返回 f^k mod x^n。
    assert(n >= 0 && (n == 0 || (!f.empty() && f[0] == Z(1)))); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    return fpsExp(fpsLog(f, n) * k, n);
}
}

Poly compInverse(const Poly &f, int n)
{
    // f[0]=0、f[1]≠0，n 是截断项数；返回满足 f(g(x))=x mod x^n 的 g。
    assert(n >= 0 && n <= (1 << 20)); // 调试检查，可删。
    if (n == 0)
```

```
    {
        return {};
    }
    if (n == 1)
    {
        return Poly(1);
    }
    assert(!f.empty() && f[0] == Z(0)); // 调试检查，可删。
    assert(f.size() >= 2 && f[1] != Z(0)); // 调试检查，可删。

    Z a = f[1], ia = a.inv();
    Poly h = f.cut(n) * ia;
    int d = n - 1;
    vector<Z> w(d + 1);
    w[d] = 1;
    auto c = powerProj(h, w, d + 1);

    Poly q(d);
    for (int i = 1; i <= d; i++)
    {
        q[d - i] = Z(d) * c[i] / Z(i);
    }
    assert(q[0] == Z(1)); // 调试检查，可删。
    q = cinv::powUnit(q, -Z(d).inv(), d);

    Poly ans(n);
    Z pw = ia;
    for (int i = 1; i < n; i++)
    {
        ans[i] = q[i - 1] * pw;
        pw *= ia;
    }
    return ans;
}
```

`lagrangeCoef(phi,n)` 返回 $w = x\phi(w)$ 中 $[x^n]w$，要求 $\phi(0) \neq 0$、$0 < n < 998244353$；受基础 FPS 逆上限约束，当前实现实际要求 $n \leq 2^{22}$。它的复杂度继承 10.2.005 `ln`、`exp`、开根、快速幂 的 `fpsPow`：默认范围 $n \leq 2^{20}$ 为 $O(n \log^2 n)$，更长时走 Newton 路径。

`compInverse(f,n)` 要求 $f_0 = 0$、$n \leq 2^{20}$，当 $n \geq 2$ 时还要求 $f_1 \neq 0$；返回恰 n 项并满足 $f(g(x)) \equiv x \pmod{x^n}$，`n=0` 返回空、`n=1` 返回零常数项。幂投影和分数次幂合计 $O(n \log^2 n)$ 时间、$O(n)$ 峰值额外空间。正向复合见 10.2.012 多项式复合。同一 Ryzen 9 7945HX / GCC 14.2.0 / `-O2 -DNDEBUG` 环境下，种子 `20260816` 的 $n = 131072$ 稠密随机输入单次求逆约 1162.32 ms。

下面用 $w = x(1+w)^2$ 的前四项 $1, 2, 5, 14$ 快速核对下标与除以 n：

```
signed main()
{
    using namespace lagrange;
    assert(lagrangeCoef({1, 2, 1}, 1) == 1);
    assert(lagrangeCoef({1, 2, 1}, 4) == 14);
```

```
    Poly inv = compInverse({0, 1, 1}, 6);
    assert(inv == Poly({0, 1, -1, 2, -5, 14}));
    assert(composePoly({0, 1, 1}, inv, 6) == Poly({0, 1}));
    cout << "OK\n";
    return 0;
}
```

10.3.006 两边界间非降序列计数

计数满足 $0 \le x_0 \le \cdots \le x_{n-1}$ 且 $A_i \le x_i < B_i$ 的整数序列，输入边界本身不要求单调。先把上界做后缀最小、下界做前缀最大，问题变成上下两条单调边界之间的网格路径计数。`block` 沿中线分割一个单调网格块；跨过分界线的转移是二项式系数和，`enumF`、`enumG` 分别把整段所需值化为一次 NTT 卷积。主过程沿边界交替向上、向右推进，每次把边界差分作为下一块起始值。

```
// 依赖 1.1.008 静态取模类
struct IncSeq
{
    int sz; // 规范化后的边界点数，包含末尾哨兵点。
    bool bad = false; // 是否已经判定不存在合法序列。
    vector<int> lo, hi; // 单调下边界与单调含端点上边界。
    vector<Z> fac, ifac; // 阶乘与逆阶乘表。

    vector<Z> enumF(int n, vector<Z> v, int m)
    {
        // n 是 v 的长度，m 是输出长度；返回 ans[i]=sum(v[j]*C(n-1+i-j,i))。
        reverse(v.begin(), v.end());
        for (int i = 0; i < n; i++)
        {
            v[i] *= ifac[i];
        }
        reverse(v.begin(), v.end());
        assert(convFit(n, n + m - 1)); // 调试检查，可删。
        vector<Z> c(n + m - 1);
        for (int i = 0; i < n + m - 1; i++)
        {
            c[i] = fac[i];
        }
        auto z = conv(v, c);
        vector<Z> ans(m);
        for (int i = 0; i < m; i++)
        {
            ans[i] = z[n - 1 + i] * ifac[i];
        }
        return ans;
    }

    vector<Z> enumG(int n, const vector<Z> &v, int l)
    {
        // n 是 v 与输出的长度，l 是固定下标；返回 ans[i]=sum(v[j]*C(l+i-j,l),j=0..i)。
        assert(convFit(n, n)); // 调试检查，可删。
```

```
        vector<Z> c(n);
        for (int i = 0; i < n; i++)
        {
            c[i] = fac[l + i] * ifac[i];
        }
        auto z = conv(v, c);
        vector<Z> ans(n);
        for (int i = 0; i < n; i++)
        {
            ans[i] = z[i] * ifac[l];
        }
        return ans;
    }

    vector<Z> block(int n, const vector<int> &h, const vector<Z> &st)
    {
        // h 是 n 列单调网格块的高度，st 是各列底边入口值；返回最右边界各高度的路径计数。
        int m = h[n - 1];
        if (n == 1)
        {
            vector<Z> ans(m + 1);
            fill(ans.begin(), ans.end(), st[0]);
            return ans;
        }
        if (n == 2)
        {
            vector<Z> ans(m + 1);
            for (int i = 0; i <= m; i++)
            {
                ans[i] = st[0] * min(i + 1, h[0] + 1) + st[1];
            }
            return ans;
        }

        int mid = n / 2, cut = h[mid];
        vector<int> left(mid + 1), right(n - mid);
        vector<Z> ls(mid + 1), rs(n - mid);
        for (int i = 0; i <= mid; i++)
        {
            left[i] = h[i];
            ls[i] = i == mid ? Z(0) : st[i];
        }
        for (int i = mid; i < n; i++)
        {
            right[i - mid] = h[i] - cut;
            rs[i - mid] = st[i];
        }

        auto lv = block(mid + 1, left, ls);
        for (int i = cut; i >= 1; i--)
        {
            lv[i] -= lv[i - 1];
```

```
        }
        auto lf = enumF(cut + 1, lv, n - mid);
        auto lg = enumG(cut + 1, lv, n - mid - 1);
        auto rf = enumF(n - mid, rs, cut + 1);
        auto rg = enumG(n - mid, rs, cut);
        vector<Z> pre(cut);
        for (int i = 0; i < cut; i++)
        {
            pre[i] = lg[i] + rf[i];
        }
        for (int i = 0; i < n - mid; i++)
        {
            rs[i] = lf[i] + rg[i];
        }
        for (int i = n - mid - 1; i >= 1; i--)
        {
            rs[i] -= rs[i - 1];
        }
        auto rv = block(n - mid, right, rs);
        vector<Z> ans(m + 1);
        copy(pre.begin(), pre.end(), ans.begin());
        for (int i = 0; i < (int)rv.size(); i++)
        {
            ans[i + cut] = rv[i];
        }
        return ans;
    }

    IncSeq(vector<int> a, vector<int> b) : sz(a.size()), lo(move(a)), hi(move(b))
    {
        // a、b 是非空等长的半开下、上界；构造后保存可供 get 计算的单调网格边界。
        assert(sz > 0 && (int)hi.size() == sz); // 调试检查，可删。
        for (int i = 0; i < sz; i++)
        {
            assert(lo[i] >= 0 && hi[i] >= 0); // 调试检查，可删。
            hi[i]--;
            if (lo[i] > hi[i] || (i && hi[i] < lo[i - 1]))
            {
                bad = true;
            }
        }
        for (int i = sz - 2; i >= 0; i--)
        {
            hi[i] = min(hi[i], hi[i + 1]);
        }
        for (int i = 1; i < sz; i++)
        {
            lo[i] = max(lo[i], lo[i - 1]);
        }
        int al = lo[sz - 1], bl = hi[sz - 1];
        for (int i = sz - 1; i >= 1; i--)
        {
```

```
            lo[i] = lo[i - 1];
        }
        int dec = lo[0];
        for (auto &x : lo)
        {
            x -= dec;
        }
        for (auto &x : hi)
        {
            x -= dec;
        }
        sz++;
        lo.push_back(al - dec);
        // 哨兵只需比原末端高一格，使边界游走在原终点后经过唯一的收束竖边。
        hi.push_back(bl + 1 - dec);
        for (int i = 0; i < sz; i++)
        {
            if (lo[i] > hi[i] || (i && hi[i] < lo[i - 1]))
            {
                bad = true;
            }
        }

        if (bad)
        {
            return;
        }
        int m = sz + hi.back() - 1;
        assert(0 < m && m <= mod); // 调试检查，可删。
        fac.assign(m, 1);
        ifac.assign(m, 1);
        for (int i = 1; i < m; i++)
        {
            fac[i] = fac[i - 1] * i;
        }
        ifac[m - 1] = fac[m - 1].inv();
        for (int i = m - 1; i; i--)
        {
            ifac[i - 1] = ifac[i] * i;
        }
    }

    Z get()
    {
        // 无参数；返回原半开边界内非降整数序列数，不可行时返回 0。
        if (bad)
        {
            return 0;
        }
        int dist = upper_bound(lo.begin(), lo.end(), lo[0]) - lo.begin();
        int px = 0, py = lo[0], qx = dist - 1, qy = lo[0];
        if (qx == 0)
```

```
        {
            qy = hi[0];
        }
        vector<Z> cur(abs(qx - px) + abs(qy - py) + 1);
        cur[0] = 1;
        while (qx != sz - 1 || qy != hi[sz - 1])
        {
            for (int i = cur.size() - 1; i >= 1; i--)
            {
                if (i == 1 && px == 0 && py == 0)
                {
                    break;
                }
                cur[i] -= cur[i - 1];
            }
            if (py == qy)
            {
                vector<int> h(qx - px + 1);
                for (int i = 0; i <= qx - px; i++)
                {
                    h[i] = hi[px + i] - py;
                }
                cur = block(qx - px + 1, h, cur);
                px = qx;
                py = qy;
                qy = hi[qx];
            }
            else
            {
                int to = upper_bound(lo.begin(), lo.end(), qy) - lo.begin();
                vector<int> h(qy - py + 1);
                for (int i = 0; i <= qy - py; i++)
                {
                    h[i] = upper_bound(lo.begin(), lo.end(), py + i) - lo.begin() - px;
                }
                for (int i = qy - py; i >= 0; i--)
                {
                    h[i] -= h[0];
                }
                cur = block(qy - py + 1, h, cur);
                px = qx;
                py = qy;
                qx = to - 1;
            }
        }
        return cur.back();
    }
};

Z countIncSeq(const vector<int> &a, const vector<int> &b)
{
    // a、b 是非空等长的半开下、上界；返回其中全部非降整数序列数。
```

```
    return IncSeq(a, b).get();
}
```

`countIncSeq(a,b)` 直接接收半开上界，返回满足 $a_i \le x_i < b_i$ 且 $x_0 \le x_1 \le \cdots \le x_{n-1}$ 的非降整数序列数；数组必须非空、等长且所有端点非负，不可行时返回 0，并在建立阶乘表前结束。末尾哨兵只比原末端上界高一格，使边界游走经过唯一收束竖边。`enumF(n,v,m)` 与 `enumG(n,v,l)` 分别要求 `convFit(n,n+m-1)` 与 `convFit(n,n)`，每个递归块都在调用公共卷积前检查实际长度。两者访问的最大阶乘下标为 `cut+n-mid`，全局恰不超过 `sz+hi.back()-2`，所以表长严格取 `sz+hi.back()-1`，不是经验倍增。`block` 在右递归前只保留长度 `cut` 的左前缀，等右侧返回后才分配完整答案，避免父层高度数组沿递归栈叠成 $\Omega(H\log n)$。令 H 为规范化后最大边界高度，在 $n, H \le 131072$ 的典型约束下，时间约 $O((n+H)\log^2(n+H))$、峰值空间 $O(n+H)$，依赖 10.1.003 NTT 的 `Z` 与 `conv`。边界很小时，$O(nH)$ 前缀和 DP 更短。

下面检查单点、非单调边界、不可行边界和一组 Catalan 型结果：

```
signed main()
{
    assert(countIncSeq({3}, {7}) == Z(4));
    assert(countIncSeq({1, 0, 2}, {4, 3, 5}) == Z(9));
    assert(countIncSeq({2, 0}, {3, 2}) == Z(0));
    assert(countIncSeq({0, 0, 0}, {2, 3, 4}) == Z(14));
}
```

10.4 线性递推

10.4.001 Berlekamp-Massey（BM / 最小线性递推）

Berlekamp–Massey 从域上序列前缀恢复最短常系数线性递推。维护当前递推 C 与上一次导致阶数增长的递推 B：算每项的预测差 d，非零时用错位缩放的 B 消去它；仅当前缀长度达到 $2L$ 才把阶数提升到 $n+1-L$ 并记下这次的 B。常用于只会造前若干项、却要求远项的题。

```
// 依赖 1.1.008 静态取模类
template <class T> vector<T> BM(const vector<T> &s)
{
    // s 是已知序列前缀；返回最短 rec，使 s[n]=sum(rec[i-1]*s[n-i])。
    vector<T> c, old;
    int len = 0, gap = 1;
    T last = 1;
    for (int n = 0; n < (int)s.size(); n++)
    {
        T d = s[n];
        for (int i = 1; i <= len; i++)
        {
            d += c[i - 1] * s[n - i];
        }
        if (d == T{})
        {
            gap++;
            continue;
```

```
        }
        auto pre = c;
        T q = -d / last;
        if ((int)c.size() < (int)old.size() + gap)
        {
            c.resize(old.size() + gap);
        }
        c[gap - 1] += q;
        for (int i = 0; i < (int)old.size(); i++)
        {
            c[i + gap] += q * old[i];
        }
        if (2 * len <= n)
        {
            len = n + 1 - len;
            old = move(pre);
            last = d;
            gap = 1;
        }
        else
        {
            gap++;
        }
    }
    for (auto &x : c)
    {
        x = -x;
    }
    return c;
}
```

`BM(s)` 的系数类型须构成域，返回 `rec` 后按 `a[n]=sum(rec[i-1]*a[n-i])` 续推；Fibonacci 前缀 `0,1,1,2,3,5,8,13` 恢复 `{1,1}`，常数列 `7,7,7,7` 恢复 `{1}`。$O(NL)$ 时间（最坏 $O(N^2)$）、$O(L)$ 空间（N 样本数、L 阶数）。已知递推求远项用 10.4.003 Linear Recurrence 或 10.4.005 Bostan-Mori / 分式第 N 项；把递推写成关系多项式见 10.4.002 最短递推式。

下面用 Fibonacci、常数列和全零列核对系数方向与零阶递推：

```
signed main()
{
    assert((BM<Z>({0, 1, 1, 2, 3, 5, 8, 13}) == vector<Z>{1, 1}));
    assert((BM<Z>({7, 7, 7, 7}) == vector<Z>{1}));
    assert(BM<Z>({0, 0, 0, 0}).empty());
    cout << "OK\n";
    return 0;
}
```

10.4.002 最短递推式

把最短递推写成常数项为 1 的关系多项式 $Q(x)=1-r_0x-\cdots-r_{k-1}x^k$。它直接就是序列 OGF 的分母，与多项式取模、Bostan–Mori 都以此为接口。此条只在 BM 结果上包一层符号转换，不重复 BM 本体。

```cpp
template <class T> vector<T> shortRel(const vector<T> &s)
{
    // s 是序列前缀；返回最短关系多项式 q，满足 sum(q[i]*s[n-i])=0 且 q[0]=1。
    auto rec = BM(s);
    vector<T> q(rec.size() + 1);
    q[0] = 1;
    for (int i = 0; i < (int)rec.size(); i++)
    {
        q[i + 1] = -rec[i];
    }
    return q;
}
```

`shortRel(s)` 返回 `q={1,-rec[0], ... }`，依赖 10.4.001 Berlekamp-Massey（BM / 最小线性递推） 的 `BM`，成本即 BM 的 $O(NL)$。得到 Q 后配上分子可交 10.4.005 Bostan-Mori / 分式第 N 项 求任意项，构造分子分母见 10.4.004 递推式转分式。

10.4.003 Linear Recurrence

已知 k 项初值与 k 阶递推求第 n 项。特征多项式 $C(x)=x^k-r_0x^{k-1}-\cdots-r_{k-1}$，则 $a_n=\sum_i([x^i](x^n \bmod C))a_i$：用快速幂在模 C 意义下算 x^n，每次多项式平方或乘法后做 $O(k^2)$ 暴力取模。不负责从样本猜递推。

```cpp
template <class T> vector<T> recMul(const vector<T> &a, const vector<T> &b,
                                    const vector<T> &rec)
{
    // a、b 是次数小于 k 的系数；返回乘积模特征多项式后的 k 项系数。
    int k = rec.size();
    assert(k > 0 && (int)a.size() == k && (int)b.size() == k); // 调试检查，可删
    vector<T> c(2 * k - 1);
    for (int i = 0; i < k; i++)
    {
        for (int j = 0; j < k; j++)
        {
            c[i + j] += a[i] * b[j];
        }
    }
    for (int i = 2 * k - 2; i >= k; i--)
    {
        for (int j = 1; j <= k; j++)
        {
            c[i - j] += c[i] * rec[j - 1];
        }
    }
```

```
    c.resize(k);
    return c;
}

template <class T>
vector<T> recCoef(const vector<T> &rec, unsigned long long n)
{
    // 返回 x^n 模特征多项式的 k 个系数，系数下标对应 x^0..x^(k-1)。
    int k = rec.size();
    assert(k > 0); // 调试检查，可删
    vector<T> ans(k), x(k);
    ans[0] = T(1);
    if (k == 1)
    {
        x[0] = rec[0];
    }
    else
    {
        x[1] = T(1);
    }
    while (n)
    {
        if (n & 1)
        {
            ans = recMul(ans, x, rec);
        }
        n >>= 1;
        if (n)
        {
            x = recMul(x, x, rec);
        }
    }
    return ans;
}

template <class T> T linearRec(const vector<T> &init, const vector<T> &rec, unsigned long long
↪ n)
{
    // init 是至少 k 项初值，rec 定义 a[n]=sum(rec[i-1]*a[n-i])，n 是下标；返回 a[n]。
    int k = rec.size();
    assert(k > 0 && (int)init.size() >= k); // 调试检查，可删。
    if (n < (unsigned long long)k)
    {
        return init[n];
    }
    auto ans = recCoef(rec, n);
    T res{};
    for (int i = 0; i < k; i++)
    {
        res += ans[i] * init[i];
    }
    return res;
```

```
}
```

`linearRec(init,rec,n)` 的 `rec` 方向须与 10.4.001 Berlekamp-Massey（BM / 最小线性递推）一致，`n` 从 0 起，只把 `init` 的前 k 项视为初值；`init={0,1}, rec={1,1}` 时第 $0,1,10,50$ 项为 $0,1,55,12586269025$。$O(k^2 \log n)$ 时间、$O(k)$ 空间。k 大且有快速卷积时把暴力取模换成 NTT，或直接用 10.4.005 Bostan-Mori / 分式第 N 项 降到 $O(k \log k \log n)$。

下面核对 Fibonacci 的近项、远项以及“多给的初值不覆盖递推”这一接口语义：

```cpp
signed main()
{
    assert(linearRec<long long>({0, 1}, {1, 1}, 0) == 0);
    assert(linearRec<long long>({0, 1}, {1, 1}, 10) == 55);
    assert(linearRec<long long>({0, 1}, {1, 1}, 50) == 12586269025LL);
    assert(linearRec<long long>({0, 1, 999}, {1, 1}, 2) == 1);
    cout << "OK\n";
    return 0;
}
```

10.4.004 递推式转分式

把递推及初值转成 OGF $A(x) = \frac{P(x)}{Q(x)}$。分母 $Q = 1 - \sum_i r_i x^{i+1}$，把 $A(x)Q(x)$ 截到前 k 项，高次因递推恒等而抵消，剩下的低次就是分子 P，于是 $A = \frac{P}{Q}$。得到 P, Q 后即可喂给 Bostan–Mori。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
pair<Poly, Poly> recFraction(const vector<Z> &init, const vector<Z> &rec)
{
    // init 是至少 k 项初值，rec 定义 a[n]=sum(rec[i-1]*a[n-i])；返回生成函数 P(x)/Q(x)。
    int k = rec.size();
    assert(k > 0 && (int)init.size() >= k); // 调试检查，可删。
    Poly q(k + 1);
    q[0] = 1;
    for (int i = 0; i < k; i++)
    {
        q[i + 1] = -rec[i];
    }
    Poly a(init.begin(), init.begin() + k);
    Poly p = (a * q).cut(k);
    p.norm();
    return {p, q};
}
```

`recFraction(init,rec)` 返回 `{p,q}`，用 10.2.001 多项式封装 的 `Z`、`Poly`，初值至少含 k 项。$O(M(k))$ 时间、$O(k)$ 空间。直接消费者是 10.4.005 Bostan-Mori / 分式第 N 项；只需关系多项式 Q 见 10.4.002 最短递推式。

10.4.005 Bostan-Mori / 分式第 N 项

直接求有理 FPS $\frac{P(x)}{Q(x)}$ 的第 n 项。令 $Q^-(x)=Q(-x)$，则 $Q(x)Q^-(x)$ 只含偶次项；由 $\frac{P}{Q}=\frac{PQ^-}{QQ^-}$，在分子 PQ^- 中取与 n 同奇偶的系数、分母取偶次系数并做 $x^2\mapsto x$，所求下标减半而答案不变，$\log n$ 轮后 $n=0$ 直接读 $\frac{p_0}{q_0}$。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
// 依赖 10.4.004 递推式转分式
pair<Poly, Poly> bmStep(Poly q)
{
    // q 的首尾系数非零；返回 {q(-x),v(x)}，其中 q(x)q(-x)=v(x^2)。
    assert(!q.empty() && q[0] != Z(0) && q.back() != Z(0)); // 调试检查，可删。
    Poly neg = q;
    for (int i = 1; i < (int)neg.size(); i += 2)
    {
        neg[i] = -neg[i];
    }
    Poly a = conv(move(q), neg);
    Poly nq((a.size() + 1) / 2);
    for (int i = 0; i < (int)nq.size(); i++)
    {
        nq[i] = a[2 * i];
    }
    return {move(neg), move(nq)};
}

Z bostanMori(Poly p, Poly q, unsigned long long n)
{
    // p、q 是分子分母且 q[0] 非零，n 是所求下标；返回 [x^n]p(x)/q(x)。
    p.norm();
    q.norm();
    assert(!q.empty() && q[0] != Z(0)); // 调试检查，可删。
    while (n)
    {
        auto [neg, nq] = bmStep(move(q));
        Poly a = conv(move(p), move(neg));
        Poly np;
        np.reserve((a.size() + 1) / 2);
        for (int i = n & 1; i < (int)a.size(); i += 2)
        {
            np.push_back(a[i]);
        }
        p = move(np);
        q = move(nq);
        n >>= 1;
    }
    return p.empty() ? Z(0) : p[0] / q[0];
}
```

`bostanMori(p,q,n)` 要求 `q[0]≠0`；$p=x$、$q=1-x-x^2$ 的第 10 项为 55（即 Fibonacci）。令

$k=\max(|p|,|q|)$，则为 $O(M(k)\log n)$ 时间、$O(k)$ 空间。输入若来自递推，先用 10.4.004 递推式转分式 构造 P,Q；阶数小时 10.4.003 Linear Recurrence 常数更小。

下面把 Fibonacci 递推转成分式，直接检查分子、分母和几个已知项：

```
signed main()
{
    auto [p, q] = recFraction({0, 1}, {1, 1});
    assert(p == Poly({0, 1}) && q == Poly({1, -1, -1}));
    assert(bostanMori(p, q, 0) == 0);
    assert(bostanMori(p, q, 10) == 55);
    assert(bostanMori(p, q, 50) == Z(12586269025LL));
    return 0;
}
```

10.4.006 有理生成函数远处连续区间

单次 `bostanMori` 适合一个远项；连续取同一有理 FPS 的远处区间时，逐项查询会重复做同样的下标折半。这里先把 $Q(0)$ 归一化为 1，并做快速长除 $P/Q=A+R/Q$。有限多项式 A 直接填入答案；严格真分式部分先由 `invTail(q,k)` 一次求出

$$[x^k]\frac{1}{Q},\dots,[x^{k+d-1}]\frac{1}{Q},\qquad d=\deg Q.$$

`invTail` 每轮使用 `bmStep` 的 $Q(x)Q(-x)=Q_1(x^2)$，把区间起点按奇偶映到约一半的位置；降到 $k\le d$ 后用一次 FPS 逆取得局部块，再沿保存的 $Q(-x)$ 逐层回推。令这段逆元组成 T，截取 $U=(QT)\bmod x^d$，则远处移位在商环中的状态可由 $S\equiv RU\pmod Q$ 表示，最后展开 $S/Q \bmod x^m$ 就得到严格真分式从第 k 项起的 m 项。整个过程复用公共 `bmStep`、`conv`、`fpsInvFast` 和 `polyDivmod`，没有另一套递推乘法或私有 NTT。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
// 依赖 10.2.002 乘法、除法、取模
// 依赖 10.2.003 乘法逆元
// 依赖 10.4.005 Bostan-Mori / 分式第 N 项
Poly invTail(Poly q, unsigned long long k)
{
    // q 的次数 d>0 且首尾系数非零；返回 [x^k..x^(k+d-1)]1/q(x)。
    q.norm();
    assert(q.size() > 1 && q[0] != Z(0) && q.back() != Z(0)); // 调试检查，可删。
    int d = q.size() - 1;
    assert(d <= NTT_LIM); // 调试检查，可删。
    if (k <= (unsigned long long)d)
    {
        assert(k <= (unsigned long long)(NTT_LIM - d)); // 调试检查，可删。
    }
    else
    {
        assert(convFit(d + 1, 2 * d - 1)); // 调试检查，可删。
```

```
    }
    vector<pair<Poly, int>> st;
    st.reserve(64);
    while (k > (unsigned long long)d)
    {
        auto [neg, nq] = bmStep(move(q));
        int t = (k - (unsigned long long)d) & 1ULL;
        st.push_back({move(neg), t});
        q = move(nq);
        k = (k - (unsigned long long)d + t) / 2;
    }

    int s = k;
    Poly a = fpsInvFast(q, s + d);
    a.erase(a.begin(), a.begin() + s);
    while (!st.empty())
    {
        Poly neg = move(st.back().first);
        int t = st.back().second;
        st.pop_back();
        Poly b(2 * d - 1);
        for (int i = 0; i < d; i++)
        {
            b[2 * i] = a[i];
        }
        Poly f = conv(move(neg), move(b));
        int l = d - t;
        a = Poly(f.begin() + l, f.begin() + l + d);
    }
    return a;
}

Poly ratBlock(Poly p, Poly q, unsigned long long k, int m)
{
    // p/q 是有理 FPS，q[0] 非零；返回从第 k 项起的 m 项。
    // 设 L=log(1+k/d)，时间 O(M(|p|)+M(d)L+M(d+m))，空间 O(|p|+dL+d+m)。
    // 所有实际卷积长度须不超过公共 NTT 的 2^23 上限。
    assert(m >= 0); // 调试检查，可删。
    q.norm();
    assert(!q.empty() && q[0] != Z(0)); // 调试检查，可删。
    if (!m)
    {
        return {};
    }
    assert(k <= numeric_limits<unsigned long long>::max() -
                    (unsigned long long)(m - 1)); // 调试检查，可删。
    p.norm();
    Z iv = q[0].inv();
    p *= iv;
    q *= iv;

    if (q.size() == 1)
```

```
    {
        Poly ans(m);
        if (k < (unsigned long long)p.size())
        {
            int n = min<unsigned long long>(m, (unsigned long long)p.size() - k);
            copy_n(p.begin() + (size_t)k, n, ans.begin());
        }
        return ans;
    }

    auto [a, r] = polyDivmod(move(p), q);
    Poly ans(m);
    if (k < (unsigned long long)a.size())
    {
        int n = min<unsigned long long>(m, (unsigned long long)a.size() - k);
        for (int i = 0; i < n; i++)
        {
            ans[i] = a[(size_t)k + i];
        }
    }

    int d = q.size() - 1;
    if (r.empty())
    {
        return ans;
    }
    assert(m <= NTT_LIM && convFit(d + 1, d)); // 调试检查，可删。
    Poly t = invTail(q, k);
    Poly u = conv(q, move(t));
    u.resize(d);
    Poly v = conv(move(u), move(r));
    Poly s = polyDivmod(move(v), q).second;
    assert(convFit(s.size(), m)); // 调试检查，可删。
    Poly iq = fpsInvFast(q, m);
    Poly b = conv(move(s), move(iq));
    b.resize(m);
    ans += b;
    return ans;
}
```

`ratBlock(p,q,k,m)` 返回 $[x^k]\frac{p(x)}{q(x)},\dots,[x^{k+m-1}]\frac{p(x)}{q(x)}$；`q[0]` 必须非零，`m=0` 返回空，且 $k+m-1$ 不能发生无符号回绕。常数分母在归一化后直接从有限多项式截取，时间为 $O(|p|+m)$；非常数分母先长除，使严格真分式和有限多项式部分同时成立，例如 $p=1+x^3, q=1-x$ 的前六项为 $1,1,1,2,2,2$。设 $N=\max(|p|,d+1)$、$L=1+\log(1+k/d)$，一般时间为 $O(M(N)+M(d)L+M(d+m))$，额外空间为 $O(N+dL+d+m)$。所有实际卷积的 NTT 长度必须不超过 2^{23}。只求一个远项时 10.4.005 Bostan-Mori / 分式第 N 项 更短，连续区间才使用本条。

下面用 Fibonacci、重根分母、非严格真分式、常数分母和空区间作固定核对；模数类 `Z` 沿用本章公共前导：

```cpp
signed main()
{
    assert((ratBlock({0, 1}, {1, -1, -1}, 10, 4) ==
            vector<Z>{55, 89, 144, 233}));
    assert((ratBlock({1}, {1, -2, 1}, 100, 3) ==
            vector<Z>{101, 102, 103}));
    assert((ratBlock({1, 0, 0, 1}, {1, -1}, 0, 6) ==
            vector<Z>{1, 1, 1, 2, 2, 2}));
    assert((ratBlock({2, 4, 6}, {2}, 1, 3) ==
            vector<Z>{2, 3, 0}));
    assert(ratBlock({1}, {1, -1}, 0, 0).empty());
    cout << "OK\n";
    return 0;
}
```

11. 计算几何

11.1 二维基础对象

11.1.001 浮点误差与精确整数几何

统一整数几何与浮点几何的判号规则。浮点类型统一记为 `Real`，实际使用 `long double`；整数坐标的叉积应优先保持精确，只有交点、距离、角度和圆等需要除法或开方的计算才转为 `Real`。`sgn` 对整数精确判号，对 `Real` 使用相对误差，因而只依赖符号的算法可以直接写成 `Point<T>` 共用。坐标乘积可能超过 `long long` 时，由调用者在专用算法中显式转为 `__int128`。

```cpp
using Real = long double;
constexpr Real eps = 1E-12L;

template <class T> int sgn(T x)
{
    // x 是整数；返回 -1、0、1 表示负、零、正。
    return (x > 0) - (x < 0);
}

int sgn(Real x)
{
    // x 表示待判断的浮点数；返回 -1、0、1 表示负、近似零、正。
    Real t = eps * max<Real>(1, fabsl(x));
    return (x > t) - (x < -t);
}

template <class T> bool eq(T x, T y)
{
    // x、y 是整数；返回二者是否相等。
    return x == y;
}

bool eq(Real x, Real y)
{
    // x、y 表示两个浮点数；使用相对误差判断它们是否近似相等。
    return fabsl(x - y) <= eps * max<Real>(1, max(fabsl(x), fabsl(y)));
}
```

整数与浮点计算都调用 `sgn(x)` 或 `eq(x,y)`；传入整数时不会产生浮点转换。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.002 Point / Vector

表示二维点和二维向量，提供坐标读写、比较、加减与数乘。Point 同时承担点和向量语义，是本章其余二维模板的共同参数类型。点和向量都存坐标，但语义不同：点减点得到向量，点加向量得到点；比较、哈

希和浮点相等必须采用与坐标类型一致的规则。

```cpp
// 依赖 11.1.001 浮点误差与精确整数几何
template <class T> struct Point
{
    T x; // x 保存横坐标。
    T y; // y 保存纵坐标。

    Point(const T &x = 0, const T &y = 0) : x(x), y(y)
    {
        // x、y 表示初始坐标；构造点或向量。
    }

    template <class U> explicit operator Point<U>() const
    {
        // U 表示目标坐标类型；返回逐坐标 C 风格转换后的点。
        return Point<U>((U)x, (U)y);
    }

    Point &operator+=(const Point &p)
    {
        // p 表示另一个向量；把 p 加到当前向量并返回自身。
        x += p.x;
        y += p.y;
        return *this;
    }

    Point &operator-=(const Point &p)
    {
        // p 表示另一个向量；从当前向量减去 p 并返回自身。
        x -= p.x;
        y -= p.y;
        return *this;
    }

    Point &operator*=(const T &k)
    {
        // k 表示缩放倍数；原地数乘并返回自身。
        x *= k;
        y *= k;
        return *this;
    }

    Point &operator/=(const T &k)
    {
        // k 表示非零除数；原地除法并返回自身。
        assert(k != T(0));
        x /= k;
        y /= k;
        return *this;
    }

```

```
51      Point operator-() const
52      {
53          // 无参数；返回反向向量。
54          return {-x, -y};
55      }
56
57      friend Point operator+(Point a, const Point &b)
58      {
59          // a、b 表示两个向量；返回逐坐标之和。
60          return a += b;
61      }
62
63      friend Point operator-(Point a, const Point &b)
64      {
65          // a、b 表示两个点或向量；返回 a-b。
66          return a -= b;
67      }
68
69      friend Point operator*(Point a, const T &k)
70      {
71          // a 表示向量，k 表示倍数；返回缩放后的向量。
72          return a *= k;
73      }
74
75      friend Point operator*(const T &k, Point a)
76      {
77          // k 表示倍数，a 表示向量；返回缩放后的向量。
78          return a *= k;
79      }
80
81      friend Point operator/(Point a, const T &k)
82      {
83          // a 表示向量，k 表示非零除数；返回缩放后的向量。
84          return a /= k;
85      }
86
87      friend bool operator<(const Point &a, const Point &b)
88      {
89          // a、b 表示两个点；返回按 x、y 的字典序比较结果。
90          return a.x != b.x ? a.x < b.x : a.y < b.y;
91      }
92
93      friend bool operator==(const Point &a, const Point &b)
94      {
95          // a、b 表示两个点；返回坐标是否精确相等。
96          return a.x == b.x && a.y == b.y;
97      }
98  };
99
100 using P = Point<Real>;
101
102 template <class T> bool eq(Point<T> a, Point<T> b)
```

```
{
    // a、b 表示两个点；整数坐标精确比较，浮点坐标按统一误差比较。
    return eq(a.x, b.x) && eq(a.y, b.y);
}
```

使用 `Point<int>` 保存整数点，使用 `P` 即 `Point<Real>` 保存需要除法或开方的浮点点；排序使用的比较运算始终精确比较坐标，误差相等判断显式调用 `eq(a,b)`。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.003 点积、叉积、长度、夹角、旋转

计算点积、叉积、模长、夹角和旋转。点积描述投影与夹角，叉积描述有向面积和左右关系，两者是直线、多边形与圆算法的基础。点积刻画投影和夹角，叉积符号刻画转向、平行四边形有向面积；旋转使用正交矩阵，有向夹角把叉积和点积交给 `atan2`，无需先求模长或限制输入范围。二维 `norm` 返回的是模长的平方（与三维 11.6.001 的 `norm` 同义），外接圆公式、投影参数、旋转卡壳的直径平方都按平方长度取用；原稿此处误写成 `return abs(a);`（返回模长）会破坏这些调用，已修正为 `return square(a);`。

```
// 依赖 11.1.002 Point / Vector
template <class T> T dot(const Point<T> &a, const Point<T> &b)
{
    // a、b 表示两个向量；返回点积。
    return a.x * b.x + a.y * b.y;
}

template <class T> T cross(const Point<T> &a, const Point<T> &b)
{
    // a、b 表示两个向量；返回有向叉积。
    return a.x * b.y - a.y * b.x;
}

template <class T> T cross(const Point<T> &a, const Point<T> &b, const Point<T> &c)
{
    // a、b、c 表示三个点；返回三角形 abc 的有向面积两倍。
    return cross(b - a, c - a);
}

template <class T> T square(const Point<T> &a)
{
    // a 表示向量；返回模长平方。
    return dot(a, a);
}

template <class T> Real abs(const Point<T> &a)
{
    // a 表示向量；返回其欧氏模长。
    return sqrtl((Real)square(a));
}
```

```cpp

template <class T> T norm(const Point<T> &a)
{
    // a 表示向量；返回模长平方。
    return square(a);
}

template <class T> Real angle(const Point<T> &a, const Point<T> &b)
{
    // a、b 是非零向量；返回 a 到 b 的有向夹角，范围为 (-pi,pi]。
    assert(a.x != T{} || a.y != T{}); // 调试检查，可删。
    assert(b.x != T{} || b.y != T{}); // 调试检查，可删。
    return atan2l(cross(a, b), dot(a, b));
}

template <class T> Point<Real> rotate(const Point<T> &a, Real t)
{
    // a 表示向量，t 表示逆时针旋转弧度；返回旋转后的浮点向量。
    return {a.x * cosl(t) - a.y * sinl(t), a.x * sinl(t) + a.y * cosl(t)};
}

template <class T> Point<T> rot(const Point<T> &a)
{
    // a 表示向量；返回逆时针旋转 90 度后的向量。
    return {-a.y, a.x};
}
```

`dot(a,b)` 与 `cross(a,b)` 分别返回点积和二维叉积标量，`cross(a,b,c)` 判断 $a \to b$ 到 $a \to c$ 的转向。`angle(a,b)` 用 `atan2l(cross(a,b),dot(a,b))` 返回从 a 到 b 的有向夹角，范围为 $(-\pi,\pi]$；普通无向夹角取 `fabsl(angle(a,b))`。这些函数直接返回坐标类型 `T`，调用者必须保证乘积与加减结果能由 `T` 表示；本书的 `Point<int>` 因 `#define int long long` 使用 64 位坐标。只有确实可能超过 64 位的专用算法才在调用处显式转为 `__int128`，普通几何调用不承担隐藏的宽类型开销。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用本条目精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.004 极角排序与象限分类

把向量按极角从小到大排序，并在同一射线上按距离排序。适用于扫描射线、按方向合并边、半平面预处理和角度统计。先按上半平面或象限分类，再用叉积比较同半平面向量方向；共线时用长度打破平局。排序比较器必须保持严格弱序，因此这里对整数和 `Real` 都使用坐标与叉积的原值比较，不在比较器中混入 `eps`。

```cpp
// 依赖 11.1.002 Point / Vector
template <class T> int half(Point<T> p)
{
    // p 是向量；返回其所在的极角半平面编号。
    return p.y < 0 || (p.y == 0 && p.x < 0);
```

```cpp
}

// 以原点为极点，按 [-pi, pi) 逆时针排序；同射线按距离升序。
template <class T> bool polarCmp(Point<T> a, Point<T> b)
{
    // a、b 是相对同一原点的向量；返回 a 是否应排在 b 前面。
    int x = half(a);
    int y = half(b);
    if (x != y)
    {
        return x < y;
    }
    auto c = cross(a, b);
    return c ? c > 0 : norm(a) < norm(b);
}

// 绕极点 o 排序。
template <class T> void polarSort(vector<Point<T>> &a, Point<T> o = {0, 0})
{
    // 参数 a 表示 要原地按极角排序的点或向量序列；o 表示 极角排序使用的原点；以 o 为原点原地按极角排序
    ↪ 点集，无返回值。
    sort(a.begin(),
         a.end(),
         [&](Point<T> x, Point<T> y)
         {
             return polarCmp(x - o, y - o);
         });
}
```

调用 `polarSort(a,o)` 按相对原点 `o` 的极角排序；零向量以及同射线点的先后规则见 `polarCmp`。整数坐标最稳；浮点坐标应来自稳定输入，不能把计算误差很大的近似方向直接交给排序器。排序 $O(n \log n)$ 时间、$O(\log n)$ 比较栈空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.005 Line / Ray / Segment

表示直线、射线与线段。Line 保存两个端点；浮点算法内部需要点加方向向量时使用 DirLine，并明确两种表示的转换。直线由一点和非零方向向量表示，射线额外限制参数非负，线段限制参数在 `[0,1]`；同一几何对象的边界条件不能混用。

```cpp
// 依赖 11.1.002 Point / Vector
template <class T> struct Line
{
    Point<T> a; // a 保存直线、射线或线段的起点。
    Point<T> b; // b 保存直线上的第二点；作为射线时方向为 b-a。

    Line(Point<T> a = {}, Point<T> b = {}) : a(a), b(b)
    {
```

```
        // a、b 表示两个端点；构造端点式直线。
    }
};

struct DirLine
{
    P p; // p 保存直线上一点。
    P v; // v 保存非零方向向量。

    P at(Real t) const
    {
        // t 表示方向参数；返回 p+v*t。
        return p + v * t;
    }
};

using L = DirLine;

template <class T> DirLine toDir(const Line<T> &l)
{
    // l 表示端点式直线；返回供浮点算法使用的点方向式直线。
    auto v = l.b - l.a;
    assert(l.a != l.b); // 调试检查，可删。
    return {(P)l.a, (P)v};
}
```

通常构造 Line{a,b} 表示端点式直线或线段；只有调用旧的浮点圆与半平面接口时用 toDir(l) 转为 DirLine。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.006 平行、垂直、共线、同侧/异侧

判断两条方向的平行、垂直、共线以及点在线的哪一侧。整数输入时这些关系可只用点积和叉积精确完成。平行由方向叉积为零判断，垂直由点积为零判断，共线还需点到直线方向的叉积为零；同侧异侧看两个有向叉积的符号乘积。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.1.005 Line / Ray / Segment
template <class T> bool parallel(const Line<T> &a, const Line<T> &b)
{
    // a、b 表示两条端点式直线；返回方向是否平行。
    return sgn(cross(a.b - a.a, b.b - b.a)) == 0;
}

template <class T> bool perp(const Line<T> &a, const Line<T> &b)
{
    // a、b 表示两条端点式直线；返回方向是否垂直。
```

```
    return sgn(dot(a.b - a.a, b.b - b.a)) == 0;
}

template <class T> int side(const Line<T> &l, const Point<T> &p)
{
    // l 表示有向直线，p 表示查询点；返回叉积符号，正数在左侧。
    return sgn(cross(l.b - l.a, p - l.a));
}

bool parallel(L a, L b)
{
    // a、b 表示两条点方向式直线；返回方向是否平行。
    return sgn(cross(a.v, b.v)) == 0;
}
```

parallel(a,b)、perp(a,b) 判断方向关系；side(l,p) 返回正、零、负分别表示左侧、线上、右侧。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.007 直线交点、投影、对称点

求非平行直线交点、点在线上的投影和关于直线的对称点。退化直线或平行直线必须由调用者提前排除。非平行直线交点参数由两个叉积之比得到；投影参数是点差在方向上的点积除以方向长度平方，对称点是投影点的两倍减原点。

```
// 依赖 11.1.005 Line / Ray / Segment
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
template <class T> Point<Real> lineMeet(const Line<T> &a, const Line<T> &b)
{
    // a、b 表示两条不平行直线；返回唯一交点。
    auto u = (P)a.a;
    auto v = (P)(a.b - a.a);
    auto p = (P)b.a;
    auto q = (P)(b.b - b.a);
    assert(sgn(cross(v, q)) != 0); // 调试检查，可删。
    return u + v * (cross(p - u, q) / cross(v, q));
}

template <class T> Point<Real> projection(const Line<T> &l, const Point<T> &p)
{
    // l 表示非退化直线，p 表示查询点；返回 p 在直线上的正交投影。
    auto a = (P)l.a;
    auto v = (P)(l.b - l.a);
    auto q = (P)p;
    assert(sgn(square(v)) != 0); // 调试检查，可删。
    return a + v * (dot(q - a, v) / square(v));
}

```

```cpp
template <class T> Point<Real> reflection(const Line<T> &l, const Point<T> &p)
{
    // l 表示非退化直线，p 表示查询点；返回 p 关于直线的对称点。
    return projection(l, p) * 2 - (P)p;
}

P isect(L a, L b)
{
    // a、b 表示两条不平行的点方向式直线；返回唯一交点。
    assert(sgn(cross(a.v, b.v)) != 0); // 调试检查，可删。
    return a.at(cross(b.p - a.p, b.v) / cross(a.v, b.v));
}

P proj(L l, P p)
{
    // l 表示非退化点方向式直线，p 表示查询点；返回正交投影。
    assert(sgn(square(l.v)) != 0); // 调试检查，可删。
    return l.at(dot(p - l.p, l.v) / square(l.v));
}

P refl(L l, P p)
{
    // l 表示非退化点方向式直线，p 表示查询点；返回关于直线的对称点。
    return proj(l, p) * 2 - p;
}

Real dis(L l, P p)
{
    // l 表示非退化点方向式直线，p 表示查询点；返回点到直线距离。
    assert(sgn(square(l.v)) != 0); // 调试检查，可删。
    return fabsl(cross(l.v, p - l.p)) / abs(l.v);
}
```

`lineMeet(a,b)` 要求方向叉积非零；`projection(l,p)` 与 `reflection(l,p)` 要求 `l.a≠l.b`。这些接口接受整数或浮点端点，但除法后的结果统一返回 `Point<Real>`。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.008 点到直线/线段距离

计算点到直线、射线或线段的最短距离。线段距离需要先判断投影是否落在端点之间，不能直接套点到直线公式。点到直线距离是叉积绝对值除以方向长度；点到线段先看投影参数，落在线段外时取到相应端点距离。

```cpp
// 依赖 11.1.001 浮点误差与精确整数几何
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.005 Line / Ray / Segment
template <class T> Real distancePL(const Point<T> &p, const Line<T> &l)
```

```
{
    // p 表示点，l 表示非退化直线；返回点到无限直线的距离。
    assert(l.a != l.b); // 调试检查，可删。
    return fabsl((Real)cross(l.b - l.a, p - l.a)) / abs(l.b - l.a);
}

template <class T> Real distancePS(const Point<T> &p, const Line<T> &l)
{
    // p 表示点，l 表示线段；返回点到闭线段的最短距离。
    if (l.a == l.b)
    {
        return abs(p - l.a);
    }
    if (dot(p - l.a, l.b - l.a) <= 0)
    {
        return abs(p - l.a);
    }
    if (dot(p - l.b, l.a - l.b) <= 0)
    {
        return abs(p - l.b);
    }
    return distancePL(p, l);
}
```

distancePL(p,l) 把 l 当无限直线，distancePS(p,l) 把 l 当闭线段；退化线段应直接取点距。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.009 线段相交与距离

判断两条闭线段是否相交，并计算两线段最短距离。实现同时覆盖端点接触与共线重叠。线段相交先用包围盒排斥，再比较两端相对另一线段的叉积符号；不相交时两线段距离是四个端点到对方线段距离的最小值。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.005 Line / Ray / Segment
template <class T> bool onSegment(const Point<T> &p, const Line<T> &l)
{
    // p 表示查询点，l 表示闭线段；返回 p 是否在线段上。
    return sgn(cross(p - l.a, l.b - l.a)) == 0 && sgn(dot(p - l.a, p - l.b)) <= 0;
}

template <class T> bool segMeet(const Line<T> &a, const Line<T> &b)
{
    // a、b 表示两条闭线段；返回它们是否至少有一个公共点。
    int c1 = sgn(cross(a.b - a.a, b.a - a.a));
    int c2 = sgn(cross(a.b - a.a, b.b - a.a));
    int c3 = sgn(cross(b.b - b.a, a.a - b.a));
```

```
    int c4 = sgn(cross(b.b - b.a, a.b - b.a));
    if (c1 == 0 && onSegment(b.a, a))
    {
        return true;
    }
    if (c2 == 0 && onSegment(b.b, a))
    {
        return true;
    }
    if (c3 == 0 && onSegment(a.a, b))
    {
        return true;
    }
    if (c4 == 0 && onSegment(a.b, b))
    {
        return true;
    }
    return ((c1 > 0) != (c2 > 0)) && ((c3 > 0) != (c4 > 0));
}

template <class T> Real distanceSS(const Line<T> &a, const Line<T> &b)
{
    // a、b 表示两条闭线段；返回线段间最短距离。
    if (segMeet(a, b))
    {
        return 0;
    }
    return min({distancePS(a.a, b), distancePS(a.b, b), distancePS(b.a, a), distancePS(b.b,
    ↪ a)});
}

template <class T> bool onSeg(Point<T> p, Point<T> a, Point<T> b)
{
    // p 表示查询点，a、b 表示线段端点；返回 p 是否在线段上。
    return onSegment(p, Line<T>{a, b});
}

template <class T> bool segIsect(Point<T> a, Point<T> b, Point<T> c, Point<T> d)
{
    // a、b 与 c、d 分别表示两条闭线段；返回是否相交。
    return segMeet(Line<T>{a, b}, Line<T>{c, d});
}

template <class T> Real segDis(Point<T> a, Point<T> b, Point<T> c, Point<T> d)
{
    // a、b 与 c、d 分别表示两条闭线段；返回最短距离。
    return distanceSS(Line<T>{a, b}, Line<T>{c, d});
}
```

segMeet(a,b) 含边界相交，distanceSS(a,b) 返回闭线段间距离；整数坐标可精确判断相交。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。

线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.010 线段中垂线

由两个不同点构造线段中垂线，供外心、等距轨迹和 Voronoi 边界使用。中点是直线上一点，原线段方向旋转 90 度得到中垂线方向；两端点重合时方向不确定。

```cpp
// 依赖 11.1.001 浮点误差与精确整数几何
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.1.005 Line / Ray / Segment
optional<Line<Real>> midPerp(P a, P b)
{
    // a、b 表示两个点；重合时返回空，否则返回端点式中垂线。
    if (eq(a, b))
    {
        return nullopt;
    }
    auto m = (a + b) / 2;
    return Line<Real>{m, m + rot(b - a)};
}
```

调用 `midPerp(a,b)`；两点重合返回 `nullopt`，否则返回 `Line<Real>`。中点可能出现二分之一，因此本接口固定使用浮点点。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.011 四点组成矩形 / 正方形判定

判断四个无序点是否组成非退化矩形或正方形。通过六条两两距离避免依赖输入环形顺序。枚举六个两两距离平方并排序。矩形应有两组相等边和两条相等对角线且满足勾股关系；正方形进一步要求四条边相等。

```cpp
// 依赖 11.1.002 Point / Vector
template <class T> int rectKind(const array<Point<T>, 4> &p)
{
    // p[0..3] 表示四个无序点；返回 0 非矩形、1 矩形、2 正方形。
    vector<T> d;
    for (int i = 0; i < 4; i++)
    {
        for (int j = i + 1; j < 4; j++)
        {
            d.push_back(square(p[i] - p[j]));
        }
    }
    sort(d.begin(), d.end());
    if (sgn(d[0]) == 0 || !eq(d[0], d[1]) || !eq(d[2], d[3]) || !eq(d[4], d[5]) || !eq(d[0] +
    ↪ d[2], d[4]))
    {
```

```
        return 0;
    }
    return eq(d[0], d[2]) ? 2 : 1;
}
```

调用 `rectKind(p)`，`p[0..3]` 是四个无序点；整数坐标精确判断，`Point<Real>` 按统一误差判断。返回 0、1、2 分别表示否、矩形、正方形。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.012 三角形内心、外心、垂心与旁心

计算非退化三角形的内心、外心、垂心和三个旁心，供圆、距离和三角形构造题直接使用。内心与旁心使用边长加权；外心由两条中垂线求交；垂心可由向量恒等式 $H = A + B + C - 2O$ 得到。

```
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
array<P, 6> triCenters(P a, P b, P c)
{
    // a、b、c 表示逆时针或顺时针非共线三角形；依次返回内心、外心、垂心和三个旁心。
    auto x = abs(b - c);
    auto y = abs(c - a);
    auto z = abs(a - b);
    auto in = (a * x + b * y + c * z) / (x + y + z);
    auto d = 2 * cross(b - a, c - a);
    assert(sgn(d) != 0); // 调试检查，可删。
    auto bb = square(b - a);
    auto cc = square(c - a);
    P out{a.x + ((c.y - a.y) * bb - (b.y - a.y) * cc) / d, a.y + ((b.x - a.x) * cc - (c.x -
    ↪ a.x) * bb) / d};
    auto orth = a + b + c - out * 2;
    auto exa = (-a * x + b * y + c * z) / (-x + y + z);
    auto exb = (a * x - b * y + c * z) / (x - y + z);
    auto exc = (a * x + b * y - c * z) / (x + y - z);
    return {in, out, orth, exa, exb, exc};
}
```

调用 triCenters(a,b,c)，返回数组依次为内心、外心、垂心、a 对旁心、b 对旁心、c 对旁心；三点不得共线。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.2 多边形

11.2.001 多边形面积、周长、重心

计算简单多边形的有向面积、周长和重心。有向面积还能判断顶点方向，重心公式要求面积非零。鞋带公式的叉积和是有向面积的两倍；整数点先用 `area2` 保留精确整数，需要通常意义的面积、周长或重心时再由接口显式返回 `Real`。

```cpp
// 依赖 11.1.002 Point / Vector
template <class T> T area2(const vector<Point<T>> &a)
{
    // a 是环形点列；返回多边形有向面积的两倍。
    T ans = 0;
    int n = a.size();
    for (int i = 0; i < n; i++)
    {
        ans += cross(a[i], a[(i + 1) % n]);
    }
    return ans;
}

template <class T> Real area(const vector<Point<T>> &a)
{
    // a 是环形点列；返回多边形有向面积。
    return (Real)area2(a) / 2;
}

template <class T> Real perimeter(const vector<Point<T>> &a)
{
    // a 是环形点列；返回多边形周长。
    Real ans = 0;
    int n = a.size();
    for (int i = 0; i < n; i++)
    {
        ans += abs(a[(i + 1) % n] - a[i]);
    }
    return ans;
}

// 非退化简单多边形的重心；顺逆时针均可。
template <class T> Point<Real> cen(const vector<Point<T>> &a)
{
    // a 是非退化简单多边形；返回面积形心。
    P ans{};
    Real s = 0;
    int n = a.size();
    for (int i = 0; i < n; i++)
    {
        Real c = (Real)cross(a[i], a[(i + 1) % n]);
        ans += ((P)a[i] + (P)a[(i + 1) % n]) * c;
        s += c;
```

```
    }
    assert(sgn(s) != 0); // 调试检查，可删。
    return ans / (3 * s);
}
```

`area2(poly)` 返回与坐标同类型的有向面积两倍值，整数坐标时没有除法误差；`area(poly)`、`perimeter(poly)` 和 `cen(poly)` 分别返回 `Real` 面积、周长和重心。调用者须保证叉积和能由 `T` 表示。各接口均为 $O(n)$ 时间、$O(1)$ 额外空间。只做点包含使用 11.2.002 点在多边形内与有符号绕数；整条线段包含必须用 11.2.003 线段是否在多边形内。凸多边形交可用 11.2.007 多边形交、并，由半平面描述的凸区域改用 11.3.009 半平面交；一般非凸布尔运算不在当前模板范围。

11.2.002 点在多边形内与有符号绕数

同时提供简单多边形点分类和一般闭合轮廓的有符号绕数。`inPoly` 对简单多边形用向右射线的奇偶交点数返回外部、边界、内部；`windingNum` 对可自交、可重复绕行的闭合点列，按向上穿越加一、向下穿越减一。两者都先判边界并采用纵坐标半开规则，因此零长度边、重复顶点和射线穿过顶点不会重复计数；绕数正负随轮廓方向反转。

```
// 依赖 11.1.002 Point / Vector
template <class T> int inPoly(const vector<Point<T>> &a, Point<T> p)
{
    // a 是简单多边形，p 是查询点；返回 0、1、2 表示外部、边界、内部。
    bool in = false;
    int n = (int)a.size();
    for (int i = 0; i < n; i++)
    {
        Point<T> x = a[i];
        Point<T> y = a[(i + 1) % n];
        if (onSeg(p, x, y))
        {
            return 1;
        }
        if (x.y > y.y)
        {
            swap(x, y);
        }
        if (sgn(x.y - p.y) <= 0 && sgn(p.y - y.y) < 0 && sgn(cross(x, y, p)) > 0)
        {
            in = !in;
        }
    }
    return in ? 2 : 0;
}

template <class T> pair<bool, int> windingNum(const vector<Point<T>> &a, Point<T> p)
{
    // a 是按顺序连接并首尾闭合的轮廓，p 是查询点；返回是否在边界上及有符号绕数，边界上时绕数置零。
    int ans = 0;
    int n = (int)a.size();
```

```
    for (int i = 0; i < n; i++)
    {
        Point<T> x = a[i];
        Point<T> y = a[(i + 1) % n];
        if (onSeg(p, x, y))
        {
            return {true, 0};
        }
        int sx = sgn(x.y - p.y);
        int sy = sgn(y.y - p.y);
        int c = sgn(cross(x, y, p));
        if (sx <= 0 && sy > 0 && c > 0)
        {
            ans++;
        }
        if (sy <= 0 && sx > 0 && c < 0)
        {
            ans--;
        }
    }
    return {false, ans};
}
```

`inPoly(a,p)` 返回 $0,1,2$ 表示外部、边界、内部，只用于按边界顺序给出的简单多边形；`windingNum(a,p)` 返回的 `first` 表示点是否在边界上，`second` 是有符号绕数，边界上固定为 0。点列会自动首尾相连，空点列绕数为 0；一次查询为 $O(n)$ 时间、$O(1)$ 额外空间。整条线段包含见 11.2.003 线段是否在多边形内，多个轮廓的覆盖层面积见 11.2.011 多个简单多边形的覆盖层面积。

11.2.003 线段是否在多边形内

判断整条闭线段是否位于简单多边形内部或边界上，特别处理穿过凹顶点和沿边界重合的情况。端点在多边形内还不充分：凹多边形中线段可能穿出再穿入。实现枚举线段与每条边的交，并在经过顶点时结合相邻边方向判断是否越过外部。

```
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.1.009 线段相交与距离
// 依赖 11.2.002 点在多边形内与有符号绕数
bool segInPoly(const vector<P> &a, P s, P t)
{
    // a 是简单多边形，s、t 是线段端点；返回闭线段是否完全位于内部或边界。
    if (!inPoly(a, s) || !inPoly(a, t))
    {
        return false;
    }
    vector<Real> q{0, 1};
    P v = t - s;
    Real z = norm(v);
    if (!sgn(z))
    {
```

```
        return true;
    }
    int n = a.size();
    for (int i = 0; i < n; i++)
    {
        P x = a[i];
        P y = a[(i + 1) % n];
        P w = y - x;
        Real d = cross(v, w);
        if (sgn(d))
        {
            Real u = cross(x - s, w) / d;
            Real k = cross(x - s, v) / d;
            if (sgn(u) >= 0 && sgn(u - 1) <= 0 && sgn(k) >= 0 && sgn(k - 1) <= 0)
            {
                q.push_back(u);
            }
        }
        else if (!sgn(cross(v, x - s)))
        {
            q.push_back(clamp(dot(x - s, v) / z, Real(0), Real(1)));
            q.push_back(clamp(dot(y - s, v) / z, Real(0), Real(1)));
        }
    }
    sort(q.begin(), q.end());
    q.erase(unique(q.begin(),
                   q.end(),
                   [](Real x, Real y)
                   {
                       return !sgn(x - y);
                   }),
            q.end());
    for (int i = 0; i + 1 < (int)q.size(); i++)
    {
        P p = s + v * ((q[i] + q[i + 1]) / 2);
        if (!inPoly(a, p))
        {
            return false;
        }
    }
    return true;
}
```

调用 `segInPoly(poly,s,t)`。端点在内部并不足够，函数还会按所有交点分段检查中点；退化为点时按点包含处理。$O(n\log n)$ 时间、$O(n)$ 空间。只做点包含使用 11.2.002 点在多边形内与有符号绕数；整条线段包含必须用 11.2.003 线段是否在多边形内。凸多边形交可用 11.2.007 多边形交、并，由半平面描述的凸区域改用 11.3.009 半平面交；一般非凸布尔运算不在当前模板范围。可据此手算自查：L 形多边形 {(0,0),(4,0),(4,1),(1,1),(1,4),(0,4)}：线段 (0,0)-(4,1) 在内部或边界，线段 (0,4)-(4,0) 穿过缺口外部；期望前者为真、后者为假。

11.2.004 多边形切割

用一条有向直线裁剪多边形，只保留左半平面。连续应用即可完成凸多边形与若干半平面的交。逐边判断顶点在切割有向直线哪侧，保留目标侧顶点，并在跨侧边上插入交点；这是 Sutherland-Hodgman 的单半平面裁剪步骤。

```cpp
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.1.005 Line / Ray / Segment
// 依赖 11.1.007 直线交点、投影、对称点
vector<P> cutPoly(const vector<P> &a, L l)
{
    // a 是环形点列，l 是有向边界；返回其左闭半平面内的裁剪结果。
    vector<P> b;
    int n = a.size();
    for (int i = 0; i < n; i++)
    {
        P x = a[i];
        P y = a[(i + 1) % n];
        int sx = sgn(cross(l.v, x - l.p));
        int sy = sgn(cross(l.v, y - l.p));
        if (sx >= 0)
        {
            b.push_back(x);
        }
        if (sx * sy < 0)
        {
            b.push_back(isect({x, y - x}, l));
        }
    }
    return b;
}
```

调用 `cutPoly(poly,l)`，`l` 是点方向式有向直线，返回其左侧与边界上的多边形；输入应按环形顺序给出。$O(n)$ 时间、$O(n)$ 结果空间。只做点包含使用 11.2.002 点在多边形内与有符号绕数；整条线段包含必须用 11.2.003 线段是否在多边形内。凸多边形交可用 11.2.007 多边形交、并，由半平面描述的凸区域改用 11.3.009 半平面交；一般非凸布尔运算不在当前模板范围。

11.2.005 多边形的核

求简单多边形的核，即能看见多边形全部位置的点集。逆时针多边形的核是所有边左半平面的交。简单多边形的核是其每条有向边内侧半平面的交；按顶点方向确定内侧后连续做半平面裁剪，最终为空即不存在可见全多边形的点。

```cpp
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.1.005 Line / Ray / Segment
// 依赖 11.1.007 直线交点、投影、对称点
// 依赖 11.2.001 多边形面积、周长、重心
```

```cpp
// 依赖 11.2.004 多边形切割
vector<P> polyKernel(const vector<P> &a)
{
    // a 是简单多边形；返回核多边形，核为空时返回空序列。
    if (a.size() < 3)
    {
        return {};
    }
    vector<P> b = a;
    Real s = area(a);
    assert(sgn(s) != 0); // 调试检查，可删。
    int n = a.size();
    for (int i = 0; i < n && !b.empty(); i++)
    {
        P x = a[i];
        P y = a[(i + 1) % n];
        if (s < 0)
        {
            swap(x, y);
        }
        b = cutPoly(b, {x, y - x});
    }
    return b;
}
```

`polyKernel(poly)` 接受顺时针或逆时针简单多边形，空返回值表示核为空。基础版逐边调用 11.2.004 多边形切割，$O(n^2)$ 时间、$O(n)$ 空间；边数很大时可改用 11.3.009 半平面交 的 $O(n \log n)$ 实现。只做点包含使用 11.2.002 点在多边形内与有符号绕数；整条线段包含必须用 11.2.003 线段是否在多边形内。凸多边形交可用 11.2.007 多边形交、并，一般非凸布尔运算不在当前模板范围。

11.2.006 Ear Clipping 三角剖分

把简单多边形拆成不相交三角形。Ear Clipping 容易实现并能恢复三角形编号，但最坏复杂度较高。Ear Clipping 维护当前多边形链，凸顶点且其耳三角形内部不含其他顶点时可剪下；每次删除一个耳，直到只剩最后一个三角形。

```cpp
// 依赖 11.1.002 Point / Vector
template <class T> vector<array<int, 3>> triangulate(const vector<Point<T>> &a)
{
    // a 是简单多边形；返回三角形对应的原顶点下标。
    int n = (int)a.size();
    if (n < 3)
    {
        return {};
    }
    T s = area2(a);
    assert(sgn(s) != 0); // 调试检查，可删。
    vector<int> p(n);
    iota(p.begin(), p.end(), 0);
```

```
    if (s < 0)
    {
        reverse(p.begin(), p.end());
    }
    bool changed = true;
    while (changed && p.size() > 3)
    {
        changed = false;
        int m = (int)p.size();
        for (int i = 0; i < m; i++)
        {
            int x = p[(i + m - 1) % m];
            int y = p[i];
            int z = p[(i + 1) % m];
            if (sgn(cross(a[x], a[y], a[z])) == 0)
            {
                p.erase(p.begin() + i);
                changed = true;
                break;
            }
        }
    }
    vector<array<int, 3>> ans;
    auto inside = [&](Point<T> q, Point<T> x, Point<T> y, Point<T> z)
    {
        return sgn(cross(x, y, q)) >= 0 && sgn(cross(y, z, q)) >= 0
            && sgn(cross(z, x, q)) >= 0;
    };
    while (p.size() > 3)
    {
        bool ok = false;
        int m = (int)p.size();
        for (int i = 0; i < m; i++)
        {
            int x = p[(i + m - 1) % m];
            int y = p[i];
            int z = p[(i + 1) % m];
            if (sgn(cross(a[x], a[y], a[z])) <= 0)
            {
                continue;
            }
            bool bad = false;
            for (int w : p)
            {
                if (w != x && w != y && w != z && inside(a[w], a[x], a[y], a[z]))
                {
                    bad = true;
                    break;
                }
            }
            if (bad)
            {
```

```
                continue;
            }
            ans.push_back({x, y, z});
            p.erase(p.begin() + i);
            ok = true;
            break;
        }
        if (!ok)
        {
            return {};
        }
    }
    if (p.size() == 3)
    {
        ans.push_back({p[0], p[1], p[2]});
    }
    return ans;
}
```

调用 `triangulate(poly)`，输入必须是不自交、面积非零的简单多边形；返回的 `array<int,3>` 三维分别是原多边形三个顶点下标。连续共线的边界点会先被删除，因而不一定出现在三角形中；少于三个点返回空序列。最坏 $O(n^3)$ 时间、$O(n)$ 空间。只做点包含使用 11.2.002 点在多边形内与有符号绕数；整条线段包含必须用 11.2.003 线段是否在多边形内。凸多边形交可用 11.2.007 多边形交、并，由半平面描述的凸区域改用 11.3.009 半平面交；一般非凸布尔运算不在当前模板范围。

11.2.007 多边形交、并

计算两个凸多边形的交与并面积，并给出小规模 Delaunay 三角剖分边集。非凸多边形布尔运算不属于当前接口。凸多边形交可连续半平面裁剪；一般多边形布尔运算需求交点、按参数切边并依据内外状态选择边段，不能只拼顶点集合。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.1.005 Line / Ray / Segment
// 依赖 11.1.007 直线交点、投影、对称点
// 依赖 11.1.009 线段相交与距离
// 依赖 11.2.001 多边形面积、周长、重心
vector<P> convexIsect(vector<P> a, const vector<P> &b)
{
    // a、b 是逆时针凸包；返回包含退化情形的交集凸包。
    auto in = [&](P p, const vector<P> &q)
    {
        if (q.empty())
        {
            return false;
        }
        if (q.size() == 1)
        {
            return eq(p, q[0]);
```

```
        }
        if (q.size() == 2)
        {
            return onSeg(p, q[0], q[1]);
        }
        for (int i = 0; i < (int)q.size(); i++)
        {
            if (sgn(cross(q[i], q[(i + 1) % q.size()], p)) < 0)
            {
                return false;
            }
        }
        return true;
    };
    auto add = [&](vector<P> &q, P p)
    {
        for (P x : q)
        {
            if (eq(x, p))
            {
                return;
            }
        }
        q.push_back(p);
    };
    auto segSeg = [&](P x, P y, P u, P v)
    {
        vector<P> q;
        if (onSeg(x, u, v))
        {
            add(q, x);
        }
        if (onSeg(y, u, v))
        {
            add(q, y);
        }
        if (onSeg(u, x, y))
        {
            add(q, u);
        }
        if (onSeg(v, x, y))
        {
            add(q, v);
        }
        if (q.empty() && segIsect(x, y, u, v))
        {
            add(q, isect({x, y - x}, {u, v - u}));
        }
        return q;
    };
    auto segPoly = [&](P x, P y, const vector<P> &q)
    {
```

```
        if (q.empty())
        {
            return vector<P>{};
        }
        if (q.size() == 1)
        {
            return onSeg(q[0], x, y) ? vector<P>{q[0]} : vector<P>{};
        }
        if (q.size() == 2)
        {
            return segSeg(x, y, q[0], q[1]);
        }
        P d = y - x;
        Real lo = 0;
        Real hi = 1;
        for (int i = 0; i < (int)q.size(); i++)
        {
            P e = q[(i + 1) % q.size()] - q[i];
            Real c = cross(e, x - q[i]);
            Real k = cross(e, d);
            if (sgn(k) == 0)
            {
                if (sgn(c) < 0)
                {
                    return vector<P>{};
                }
                continue;
            }
            Real t = -c / k;
            if (k > 0)
            {
                lo = max(lo, t);
            }
            else
            {
                hi = min(hi, t);
            }
        }
        if (sgn(lo - hi) > 0)
        {
            return vector<P>{};
        }
        vector<P> r{x + d * lo};
        if (!eq(lo, hi))
        {
            r.push_back(x + d * hi);
        }
        return r;
    };
    auto clean = [&](vector<P> q)
    {
        vector<P> r;
```

```
        for (P p : q)
        {
            if (r.empty() || !eq(r.back(), p))
            {
                r.push_back(p);
            }
        }
        if (r.size() > 1 && eq(r.front(), r.back()))
        {
            r.pop_back();
        }
        if (r.size() > 2 && sgn(area(r)) == 0)
        {
            P x = *min_element(r.begin(), r.end());
            P y = *max_element(r.begin(), r.end());
            return eq(x, y) ? vector<P>{x} : vector<P>{x, y};
        }
        return r;
    };
    if (a.empty() || b.empty())
    {
        return {};
    }
    if (a.size() == 1)
    {
        return in(a[0], b) ? a : vector<P>{};
    }
    if (b.size() == 1)
    {
        return in(b[0], a) ? b : vector<P>{};
    }
    if (a.size() == 2)
    {
        return segPoly(a[0], a[1], b);
    }
    if (b.size() == 2)
    {
        return segPoly(b[0], b[1], a);
    }
    for (int i = 0; i < (int)b.size() && !a.empty(); i++)
    {
        P p = b[i];
        P q = b[(i + 1) % b.size()];
        vector<P> c;
        for (int j = 0; j < (int)a.size(); j++)
        {
            P x = a[j];
            P y = a[(j + 1) % a.size()];
            int sx = sgn(cross(p, q, x));
            int sy = sgn(cross(p, q, y));
            if (sx >= 0)
            {
```

```
                c.push_back(x);
            }
            if (sx * sy < 0)
            {
                c.push_back(isect({x, y - x}, {p, q - p}));
            }
        }
        a.swap(c);
    }
    return clean(a);
}

Real convexUnion(const vector<P> &a, const vector<P> &b)
{
    // a、b 是逆时针凸包；返回二者并集面积。
    return area(a) + area(b) - area(convexIsect(a, b));
}
```

`convexIsect(a,b)` 返回两个逆时针凸包的交，空包、点和线段也可直接传入；`convexUnion(a,b)` 由容斥返回并面积。两个非退化凸多边形相交最坏 $O(nm)$，退化情形 $O(n+m)$；Delaunay 单独见 11.3.011。只做点包含使用 11.2.002 点在多边形内与有符号绕数；整条线段包含必须用 11.2.003 线段是否在多边形内。由半平面描述的凸区域改用 11.3.009 半平面交；一般非凸布尔运算不在当前模板范围。

11.2.008 多边形与圆面积交

计算圆与简单多边形的相交面积。把每条有向边对圆心的面积贡献分成三角形段和扇形段累加。把每条有向边与圆围成的面积贡献分段计算：圆内段用三角形叉积，圆外段用扇形角；沿多边形边累加有向贡献后取绝对值。

```
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
Real circlePoly(const vector<P> &a, P o, Real r)
{
    // a 是环形点列，o、r 是圆心和非负半径；返回多边形与圆的相交面积。
    assert(sgn(r) >= 0); // 调试检查，可删。
    auto edge = [&](P a, P b)
    {
        P v = b - a;
        array<Real, 4> q{0, 1};
        int m = 2;
        Real aa = norm(v);
        Real bb = 2 * dot(a, v);
        Real cc = norm(a) - r * r;
        Real d = bb * bb - 4 * aa * cc;
        if (sgn(d) >= 0 && sgn(aa) != 0)
        {
            d = sqrtl(max(Real(0), d));
            for (Real t : {(-bb - d) / (2 * aa), (-bb + d) / (2 * aa)})
            {
                if (sgn(t) > 0 && sgn(t - 1) < 0)
```

```
                {
                    q[m++] = t;
                }
            }
        }
        for (int i = 1; i < m; i++)
        {
            for (int j = i; j > 0 && q[j] < q[j - 1]; j--)
            {
                swap(q[j], q[j - 1]);
            }
        }
        Real ans = 0;
        for (int i = 0; i + 1 < m; i++)
        {
            P x = a + v * q[i];
            P y = a + v * q[i + 1];
            P z = (x + y) / 2;
            if (sgn(norm(z) - r * r) <= 0)
            {
                ans += cross(x, y) / 2;
            }
            else
            {
                ans += atan2l(cross(x, y), dot(x, y)) * r * r / 2;
            }
        }
        return ans;
    };
    Real ans = 0;
    for (int i = 0, n = a.size(); i < n; i++)
    {
        ans += edge(a[i] - o, a[(i + 1) % n] - o);
    }
    return fabsl(ans);
}
```

调用 `circlePoly(poly,o,r)`；`poly` 按环形顺序给出，`o`、`r` 是圆心和非负半径，返回无符号相交面积。接口直接接收计算所需的三个量，不为章节先后顺序增加圆类型模板参数。$O(n)$ 时间、$O(1)$ 额外空间。只做点包含使用 11.2.002 点在多边形内与有符号绕数；整条线段包含必须用 11.2.003 线段是否在多边形内。凸多边形交可用 11.2.007 多边形交、并，由半平面描述的凸区域改用 11.3.009 半平面交；一般非凸布尔运算不在当前模板范围。

把下面的固定样例接在本章公共前导、上述实现与 11.4.001 Circle 基本对象 之后即可快速检查：半径为 1 的圆完全位于边长为 4 的正方形内，答案应为 π。

```
signed main()
{
    vector<P> a{{-2, -2}, {2, -2}, {2, 2}, {-2, 2}};
    cout << fixed << setprecision(12) << circlePoly(a, {0, 0}, 1) << '\n';
}
```

```
// 期望输出：3.141592653590
```

11.2.009 Pick 定理与格点多边形

对格点简单多边形使用 Pick 定理计算内部格点数与边界格点数。坐标、面积两倍值和 gcd 均保持整数精确。顶点为格点的简单多边形满足 `A=I+B/2-1`；边界格点数由每条边的 `gcd(|dx|,|dy|)` 累加，面积应用叉积保持两倍整数形式。

```
pair<long long, long long> pick(const vector<pair<long long, long long>> &a)
{
    // a 是非退化格点简单多边形；返回内部、边界格点数。
    using i128 = __int128_t;
    int n = (int)a.size();
    assert(n >= 3); // 调试检查，可删。
    auto ab = [&](i128 x)
    {
        return x < 0 ? -x : x;
    };
    auto gcd128 = [&](i128 x, i128 y)
    {
        while (y != 0)
        {
            i128 r = x % y;
            x = y;
            y = r;
        }
        return x;
    };
    i128 s = 0;
    i128 b = 0;
    for (int i = 0; i < n; i++)
    {
        auto [x, y] = a[i];
        auto [u, v] = a[(i + 1) % n];
        s += (i128)x * v - (i128)y * u;
        i128 dx = ab((i128)x - u);
        i128 dy = ab((i128)y - v);
        b += gcd128(dx, dy);
    }
    s = ab(s);
    assert(s > 0); // 调试检查，可删。
    assert((s - b) % 2 == 0); // 调试检查，可删。
    i128 in = (s - b + 2) / 2;
    assert(in >= 0 && in <= LLONG_MAX && b <= LLONG_MAX); // 调试检查，可删。
    return {(long long)in, (long long)b};
}
```

调用 `pick(poly)`，返回 `{内部格点数，边界格点数}`；顶点必须构成面积非零的格点简单多边形，且两个答案均能用 `long long` 表示。鞋带项、坐标差、绝对值和最大公约数都在 `__int128` 中计算。$O(n)$ 时间、$O(1)$ 额外空间。只做点包含使用 11.2.002 点在多边形内与有符号绕数；整条线段包含必须用 11.2.003 线

段是否在多边形内。凸多边形交可用 11.2.007 多边形交、并，由半平面描述的凸区域改用 11.3.009 半平面交；一般非凸布尔运算不在当前模板范围。

11.2.010 三角形点集批量计数

给定候选顶点集 A 和待统计点集 B，预处理后回答大量“三个 A 中顶点构成的三角形严格包含多少个 B 点”。把三个顶点按 (y,x) 排序后，内部区域可由若干条按低端到高端定向的边左侧半平面相加减；`edgeL[u][v]` 只数纵坐标严格位于两端之间且在边左侧的点，`edgeM` 单独记录边界，横向穿过中间顶点时再用 `pointL`、`pointM` 补齐同高点。坐标绝对值不超过 10^9 时叉积绝对值不超过 8×10^{18}，这里显式使用 `long long`，不额外扩大全部表项。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
struct TriPointCnt
{
    vector<Point<long long>> a; // a 是候选顶点集 A。
    vector<int> pointL, pointM; // pointL[u]、pointM[u] 是与 a[u] 同高且在其左侧、与其重合的 B 点数。
    vector<vector<int>> edgeL, edgeM; // edgeL[u][v]、edgeM[u][v] 是纵坐标严格夹在有向边 u->v 两端之间、位于其左侧、线上的 B 点数。

    TriPointCnt(const vector<Point<long long>> &vertices,
                        const vector<Point<long long>> &points) : a(vertices)
    {
        // vertices 是候选顶点集 A（count 用其下标），points 是待统计点集 B；预处理同高点计数 pointL/pointM。
        // 并对每条低到高有向边统计纵坐标严格夹在两端之间、位于其左侧/线上的 B 点数 edgeL/edgeM。
        int n = a.size();
        pointL.assign(n, 0);
        pointM.assign(n, 0);
        edgeL.assign(n, vector<int>(n));
        edgeM.assign(n, vector<int>(n));
        for (int u = 0; u < n; u++)
        {
            for (auto p : points)
            {
                if (p.y == a[u].y)
                {
                    pointL[u] += p.x < a[u].x;
                    pointM[u] += p.x == a[u].x;
                }
            }
        }
        for (int u = 0; u < n; u++)
        {
            for (int v = 0; v < n; v++)
            {
                if (a[u].y >= a[v].y)
                {
                    continue;
```

```
                }
                for (auto p : points)
                {
                    if (a[u].y < p.y && p.y < a[v].y)
                    {
                        long long det = cross(a[u] - a[v], p - a[v]);
                        edgeL[u][v] += det < 0;
                        edgeM[u][v] += det == 0;
                    }
                }
            }
        }
    }

    bool lessYX(int u, int v) const
    {
        // u、v 是顶点下标；返回 a[u] 是否按 (y,x) 字典序严格小于 a[v]。
        return pair(a[u].y, a[u].x) < pair(a[v].y, a[v].x);
    }

    int count(int x, int y, int z) const
    {
        // x、y、z 是 A 中三个顶点下标；返回严格位于该三角形内部的 B 点个数，三点共线或重合时返回 0。
        int n = a.size();
        assert(0 <= x && x < n && 0 <= y && y < n && 0 <= z && z < n); // 调试检查，可删。
        if (lessYX(y, x))
        {
            swap(x, y);
        }
        if (lessYX(z, y))
        {
            swap(y, z);
        }
        if (lessYX(y, x))
        {
            swap(x, y);
        }
        long long det = cross(a[x] - a[z], a[y] - a[z]);
        if (det == 0)
        {
            return 0;
        }
        int ans = 0;
        if (a[x].y == a[y].y)
        {
            ans = edgeL[y][z] - edgeL[x][z] - edgeM[x][z];
        }
        else if (a[y].y == a[z].y)
        {
            ans = edgeL[x][z] - edgeL[x][y] - edgeM[x][y];
        }
        else if (det < 0)
```

```
        {
            ans = edgeL[x][z] - edgeL[y][z] - edgeM[y][z] -
                  edgeL[x][y] - edgeM[x][y] - pointL[y] - pointM[y];
        }
        else
        {
            ans = edgeL[x][y] + edgeL[y][z] + pointL[y] -
                  edgeL[x][z] - edgeM[x][z];
        }
        return ans;
    }
};
```

构造 `TriPointCnt q(A,B)` 后调用 `q.count(i,j,k)`；返回值不含三条边上的点，三点共线或重合时返回 0。若 $|A|=N,|B|=M$，预处理 $O(N^2M)$ 时间、$O(N^2)$ 空间，每次查询 $O(1)$；单次点包含可直接用 11.2.002 点在多边形内与有符号绕数。

下面用固定点集检查水平边、边界点、重复点与退化三角形：

```
signed main()
{
    using Q = Point<long long>;
    vector<Q> a{{0, 0}, {4, 0}, {0, 4}, {2, 0}, {0, 0}};
    vector<Q> b{{1, 1}, {2, 1}, {1, 2}, {2, 0}, {0, 0}, {1, 1}, {4, 4}};
    TriPointCnt q(a, b);
    assert(q.count(0, 1, 2) == 4);
    assert(q.count(0, 3, 2) == 2);
    assert(q.count(0, 3, 1) == 0);
    assert(q.count(0, 1, 4) == 0);
}
```

11.2.011 多个简单多边形的覆盖层面积

求多个无洞简单多边形“至少被 k 个覆盖”的面积。对每条逆时针源边，把所有其他边与其支撑直线的交点按参数排序；若某个原子段右侧已被其他代表多边形覆盖 c 次，而源多边形有 w 个完全重复副本，则该有向段依次是第 $c+1$ 至第 $c+w$ 层的边界，按 Green 公式向这一层区间加入 $\frac{1}{2}\operatorname{cross}(s,t)$。整数版用有理参数精确排序和合批，浮点版也只用原始值建立严格弱序，误差判断不进入比较器。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.2.001 多边形面积、周长、重心
struct Fraction128
{
    __int128 num; // 分子。
    __int128 den; // 始终为正的分母。

    Fraction128(__int128 num = 0, __int128 den = 1) : num(num), den(den)
    {
        // num、den 是分子和非零分母；构造符号规范化的有理数。
```

```
        assert(den != 0); // 调试检查，可删。
        if (den < 0)
        {
            this->num = -num;
            this->den = -den;
        }
    }

    friend bool operator<(Fraction128 a, Fraction128 b)
    {
        // a、b 是有理数；返回 a 是否严格小于 b，交叉乘积必须能由 __int128 表示。
        return a.num * b.den < b.num * a.den;
    }

    friend bool operator==(Fraction128 a, Fraction128 b)
    {
        // a、b 是有理数；返回二者是否严格相等，交叉乘积必须能由 __int128 表示。
        return a.num * b.den == b.num * a.den;
    }
};

__int128 covCross(Point<long long> a, Point<long long> b, Point<long long> c)
{
    // a、b 确定有向直线，c 是查询点；返回使用宽整数计算的叉积。
    __int128 x1 = (__int128)b.x - a.x;
    __int128 y1 = (__int128)b.y - a.y;
    __int128 x2 = (__int128)c.x - a.x;
    __int128 y2 = (__int128)c.y - a.y;
    return x1 * y2 - y1 * x2;
}

int covSide(Point<long long> a, Point<long long> b, Point<long long> p)
{
    // a、b 确定整数有向直线，p 是查询点；返回 p 所在侧的精确符号。
    return sgn(covCross(a, b, p));
}

int covSide(P a, P b, P p)
{
    // a、b 确定浮点有向直线，p 是查询点；按统一误差返回 p 所在侧的符号。
    return sgn(cross(a, b, p));
}

int covDir(const vector<Point<long long>> &p)
{
    // p 是整数多边形顶点；返回其有向双倍面积的精确符号。
    __int128 sum = 0;
    for (int i = 0; i < (int)p.size(); i++)
    {
        int j = (i + 1) % p.size();
        sum += (__int128)p[i].x * p[j].y - (__int128)p[i].y * p[j].x;
    }
```

```
    return sgn(sum);
}

int covDir(const vector<P> &p)
{
    // p 是浮点多边形顶点；按统一误差返回其有向双倍面积的符号。
    return sgn(area2(p));
}

Fraction128 covParam(Point<long long> a, Point<long long> b, Point<long long> p)
{
    // a、b 确定整数参数直线，p 是线上点；返回满足 p=a+t*(b-a) 的精确参数 t。
    __int128 x = (__int128)b.x - a.x;
    __int128 y = (__int128)b.y - a.y;
    __int128 u = (__int128)p.x - a.x;
    __int128 v = (__int128)p.y - a.y;
    return {u * x + v * y, x * x + y * y};
}

Real covParam(P a, P b, P p)
{
    // a、b 确定浮点参数直线，p 是线上点；返回满足 p=a+t*(b-a) 的参数 t。
    P v = b - a;
    return dot(p - a, v) / norm(v);
}

Fraction128 covMeet(Point<long long> a, Point<long long> b, Point<long long> c, Point<long
↪ long> d)
{
    // a、b 与 c、d 确定两条不平行整数直线；返回前一条直线交点的精确参数 t。
    __int128 ux = (__int128)b.x - a.x;
    __int128 uy = (__int128)b.y - a.y;
    __int128 vx = (__int128)d.x - c.x;
    __int128 vy = (__int128)d.y - c.y;
    __int128 wx = (__int128)c.x - a.x;
    __int128 wy = (__int128)c.y - a.y;
    return {wx * vy - wy * vx, ux * vy - uy * vx};
}

Real covMeet(P a, P b, P c, P d)
{
    // a、b 与 c、d 确定两条不平行浮点直线；返回前一条直线交点的参数 t。
    P u = b - a;
    P v = d - c;
    return cross(c - a, v) / cross(u, v);
}

Real covVal(Fraction128 x)
{
    // x 是精确有理参数；返回其 long double 近似值。
    return (Real)x.num / (Real)x.den;
}
```

```

Real covVal(Real x)
{
    // x 是浮点参数；返回其原值。
    return x;
}

template <class T, class Key> vector<Real> covAreaImpl(vector<vector<Point<T>>> poly)
{
    // poly 是无洞简单逆时针多边形集；返回各覆盖层面积，Key 是事件参数类型。
    int n = (int)poly.size();
    map<vector<Point<T>>, int> weight;
    for (auto p : poly)
    {
        if (p.size() > 1 && p.front() == p.back())
        {
            p.pop_back();
        }
        assert(p.size() >= 3 && covDir(p) > 0); // 调试检查，可删。
        for (int i = 0; i < (int)p.size(); i++)
        {
            assert(!(p[i] == p[(i + 1) % p.size()])); // 调试检查，可删。
        }
        int pos = min_element(p.begin(), p.end()) - p.begin();
        rotate(p.begin(), p.begin() + pos, p.end());
        weight[p]++;
    }
    vector<vector<Point<T>>> q;
    vector<int> w;
    for (auto &[p, cnt] : weight)
    {
        q.push_back(p);
        w.push_back(cnt);
    }
    for (int i = 0; i < (int)q.size(); i++)
    {
        for (int j = i + 1; j < (int)q.size(); j++)
        {
            for (int x = 0; x < (int)q[i].size(); x++)
            {
                Point<T> a = q[i][x];
                Point<T> b = q[i][(x + 1) % q[i].size()];
                for (int y = 0; y < (int)q[j].size(); y++)
                {
                    Point<T> c = q[j][y];
                    Point<T> d = q[j][(y + 1) % q[j].size()];
                    if (covSide(a, b, c) != 0 || covSide(a, b, d) != 0)
                    {
                        continue;
                    }
                    T al, ar, bl, br;
                    if (a.x != b.x)
```

```
                    {
                        al = min(a.x, b.x);
                        ar = max(a.x, b.x);
                        bl = min(c.x, d.x);
                        br = max(c.x, d.x);
                    }
                    else
                    {
                        al = min(a.y, b.y);
                        ar = max(a.y, b.y);
                        bl = min(c.y, d.y);
                        br = max(c.y, d.y);
                    }
                    assert(!(max(al, bl) < min(ar, br))); // 不支持不同代表多边形正长度共边。
                }
            }
        }
    }
    vector<Real> diff(n + 1);
    for (int i = 0; i < (int)q.size(); i++)
    {
        for (int x = 0; x < (int)q[i].size(); x++)
        {
            Point<T> a = q[i][x];
            Point<T> b = q[i][(x + 1) % q[i].size()];
            Key zero = covParam(a, b, a);
            Key one = covParam(a, b, b);
            vector<pair<Key, int>> e{{zero, 0}, {one, 0}};
            for (int j = 0; j < (int)q.size(); j++)
            {
                if (i == j)
                {
                    continue;
                }
                for (int y = 0; y < (int)q[j].size(); y++)
                {
                    Point<T> c = q[j][y];
                    Point<T> d = q[j][(y + 1) % q[j].size()];
                    int sc = covSide(a, b, c);
                    int sd = covSide(a, b, d);
                    if (sc == 0 && sd == 0)
                    {
                        e.push_back({covParam(a, b, c), 0});
                        e.push_back({covParam(a, b, d), 0});
                        continue;
                    }
                    int delta = 0;
                    if (sc < 0 && sd >= 0)
                    {
                        delta = -w[j];
                    }
                    if (sc >= 0 && sd < 0)
```

```
                    {
                        delta = w[j];
                    }
                    if (!delta)
                    {
                        continue;
                    }
                    Key t;
                    if (sc == 0)
                    {
                        t = covParam(a, b, c);
                    }
                    else if (sd == 0)
                    {
                        t = covParam(a, b, d);
                    }
                    else
                    {
                        t = covMeet(a, b, c, d);
                    }
                    e.push_back({t, delta});
                }
            }
            sort(e.begin(),
                 e.end(),
                 [](const auto &p, const auto &r)
                 {
                     if (p.first < r.first)
                     {
                         return true;
                     }
                     if (r.first < p.first)
                     {
                         return false;
                     }
                     return p.second < r.second;
                 });
            int cnt = 0;
            Key last = e.front().first;
            for (int j = 0; j < (int)e.size();)
            {
                Key t = e[j].first;
                Key l = last < zero ? zero : last;
                Key r = one < t ? one : t;
                if (l < r)
                {
                    assert(0 <= cnt && cnt + w[i] <= n); // 调试检查，可删。
                    P pa = (P)a;
                    P u = (P)(b - a);
                    P s = pa + u * covVal(l);
                    P z = pa + u * covVal(r);
                    Real val = cross(s, z) / 2;
```

```
                    diff[cnt] += val;
                    diff[cnt + w[i]] -= val;
                }
                int k = j;
                int delta = 0;
                while (k < (int)e.size() && e[k].first == t)
                {
                    delta += e[k].second;
                    k++;
                }
                cnt += delta;
                last = t;
                j = k;
            }
        }
    }
    vector<Real> ans(n);
    Real sum = 0;
    for (int i = 0; i < n; i++)
    {
        sum += diff[i];
        ans[i] = sgn(sum) == 0 ? 0 : sum;
    }
    return ans;
}

vector<Real> covArea(vector<vector<Point<long long>>> poly)
{
    // poly 是整数简单逆时针多边形集；事件拓扑精确、面积近似，坐标绝对值须不超过 1E9。
    // 完全重复多边形须仅差循环移位；其余不同多边形不能有正长度共边。
    return covAreaImpl<long long, Fraction128>(move(poly));
}

vector<Real> covArea(vector<vector<P>> poly)
{
    // poly 是浮点简单逆时针多边形集；返回近似覆盖面积，重复须仅差循环移位且不同代表不能正长度共边。
    return covAreaImpl<Real, Real>(move(poly));
}
```

两个 `covArea` 重载都返回长度为多边形数的数组，`ans[k-1]` 表示至少覆盖 k 次的面积。每个多边形必须简单、无洞、顶点逆时针且无零长度边；完全重复只允许顶点序列相差循环移位，不同代表多边形不得有正长度共边，普通横截、相切、点接触和内含均可。整数坐标绝对值须不超过 10^9，此时拓扑事件精确而最终面积仍转为 `Real`；浮点版为近似拓扑，不适合近重合退化输入。设总边数为 E，时间 $O(E^2 \log E)$，额外空间 $O(E+n)$；只有两个凸多边形且只求交并面积时，改用 11.2.007 多边形交、并 更短。

11.2.012 直线平面嵌入的面枚举与对偶图

给定顶点整数坐标和已经画好的连通无向直线平面图，可以直接从边的几何顺序恢复各面，而不必手工提供面邻接。把每条无向边拆成两条有向半边，并在每个顶点把出半边按极角逆时针排序；对于半边 $u \to v$，在 v 处取反向半边 $v \to u$ 的顺时针相邻边作为后继，就会始终沿当前半边左侧面的边界前进。这个后继

关系是半边集合上的置换，其每个环对应一个面；沿环累加鞋带公式得到有符号双倍面积，有限面为正，外部面为最小值。原边两条半边所属面不同时就在两面之间加入带权对偶边，桥的两侧是同一面，因而不会产生对偶自环。

```cpp
// 依赖 11.1.002 Point / Vector
template <class T, class W = long long> struct PlanarEmbed
{
    struct Edge
    {
        int u;    // u 是无向边的一个端点。
        int v;    // v 是无向边的另一个端点。
        W weight; // weight 是穿过该边时对应的对偶边权。
    };

    vector<Point<T>> point;            // point 保存各顶点的互异整数坐标。
    vector<Edge> edge;                 // edge 按加入顺序保存无自环、无重复的无向边。
    vector<array<int, 2>> halfEdge;    // halfEdge[h][0]、[1] 分别是半边 h 的起点、终点。
    vector<int> nextHalfEdge;          // nextHalfEdge[h] 是沿 h 左侧面边界继续走的下一条半边。
    vector<int> faceOfHalf;        // faceOfHalf[h] 是半边 h 左侧面的编号。
    vector<__int128> faceArea2;        // faceArea2[f] 是面 f 边界游走的有符号双倍面积。
    int outer = -1;                // outer 是外部面的编号。
    vector<vector<pair<int, W>>> dual; // dual[f] 的元素是相邻面编号和分隔边权，允许平行边。

    explicit PlanarEmbed(vector<Point<T>> point = {}) : point(move(point))
    {
        // point 是平面嵌入各顶点的互异整数坐标；建立尚未加入边的嵌入。
        static_assert(is_integral_v<T>);
    }

    void addEdge(int u, int v, const W &weight = W(1))
    {
        // u、v 是不同端点编号，weight 是穿过该边的代价；加入一条无向边，无返回值。
        assert(0 <= u && u < (int)point.size());   // 调试检查，可删。
        assert(0 <= v && v < (int)point.size());   // 调试检查，可删。
        assert(u != v && !(point[u] == point[v])); // 调试检查，可删。
        edge.push_back({u, v, weight});
    }

    void build()
    {
        // 无参数；枚举各半边左侧面并填写 nextHalfEdge、faceOfHalf、faceArea2、outer 与 dual，无返回值。
        int n = point.size();
        int m = edge.size();
        halfEdge.assign(2 * m, {});
        nextHalfEdge.assign(2 * m, -1);
        faceOfHalf.assign(2 * m, -1);
        faceArea2.clear();
        outer = -1;
        dual.clear();

        set<Point<T>> uniquePoint(point.begin(), point.end());
```

```
        assert((int)uniquePoint.size() == n); // 调试检查，可删。
        vector<vector<int>> around(n);
        set<pair<int, int>> uniqEdge;
        for (int i = 0; i < m; i++)
        {
            auto [u, v, weight] = edge[i];
            assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
            assert(u != v && !(point[u] == point[v]));  // 调试检查，可删。
            pair<int, int> key{min(u, v), max(u, v)};
            assert(uniqEdge.insert(key).second); // 调试检查，可删。
            halfEdge[2 * i] = {u, v};
            halfEdge[2 * i + 1] = {v, u};
            around[u].push_back(2 * i);
            around[v].push_back(2 * i + 1);
        }

        if (m == 0)
        {
            assert(n <= 1); // 多点输入仍要求连通。
            faceArea2.push_back(0);
            outer = 0;
            dual.resize(1);
            return;
        }

        vector<char> seen(n);
        queue<int> q;
        seen[0] = true;
        q.push(0);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int h : around[u])
            {
                int v = halfEdge[h][1];
                if (!seen[v])
                {
                    seen[v] = true;
                    q.push(v);
                }
            }
        }
        assert(count(seen.begin(), seen.end(), true) == n); // 调试检查，可删。

        auto dir = [&](int h)
        {
            // h 是半边编号；返回先提升到 __int128 再相减得到的方向向量。
            int u = halfEdge[h][0];
            int v = halfEdge[h][1];
            __int128 x = (__int128)point[v].x - (__int128)point[u].x;
            __int128 y = (__int128)point[v].y - (__int128)point[u].y;
```

```
            return pair<__int128, __int128>{x, y};
        };
        auto halfPlane = [](__int128 x, __int128 y)
        {
            // x、y 是非零方向向量坐标；返回极角排序使用的半平面编号。
            return y < 0 || (y == 0 && x < 0);
        };

        vector<int> position(2 * m);
        for (int u = 0; u < n; u++)
        {
            sort(around[u].begin(),
                 around[u].end(),
                 [&](int a, int b)
                 {
                     // a、b 是从 u 出发的半边编号；返回 a 是否在逆时针极角序中先于 b。
                     if (a == b)
                     {
                         return false;
                     }
                     auto [ax, ay] = dir(a);
                     auto [bx, by] = dir(b);
                     int ha = halfPlane(ax, ay);
                     int hb = halfPlane(bx, by);
                     if (ha != hb)
                     {
                         return ha < hb;
                     }
                     __int128 z = ax * by - ay * bx;
                     if (z != 0)
                     {
                         return z > 0;
                     }
                     return a < b;
                 });
            for (int i = 0; i < (int)around[u].size(); i++)
            {
                position[around[u][i]] = i;
                if (i == 0)
                {
                    continue;
                }
                auto [ax, ay] = dir(around[u][i - 1]);
                auto [bx, by] = dir(around[u][i]);
                if (halfPlane(ax, ay) == halfPlane(bx, by))
                {
                    assert(ax * by - ay * bx != 0); // 不允许两条边占用同一射线。
                }
            }
        }

        for (int h = 0; h < 2 * m; h++)
```

```
        {
            int v = halfEdge[h][1];
            int reverse = h ^ 1;
            int d = around[v].size();
            int pos = position[reverse];
            nextHalfEdge[h] = around[v][(pos + d - 1) % d];
        }

        for (int start = 0; start < 2 * m; start++)
        {
            if (faceOfHalf[start] != -1)
            {
                continue;
            }
            int face = faceArea2.size();
            int h = start;
            __int128 area = 0;
            do
            {
                assert(faceOfHalf[h] == -1); // 调试检查，可删。
                faceOfHalf[h] = face;
                auto [u, v] = halfEdge[h];
                area += (__int128)point[u].x * (__int128)point[v].y - (__int128)point[u].y *
    (__int128)point[v].x;
                h = nextHalfEdge[h];
            } while (h != start);
            faceArea2.push_back(area);
        }

        outer = min_element(faceArea2.begin(), faceArea2.end()) - faceArea2.begin();
        dual.assign(faceArea2.size(), {});
        for (int i = 0; i < m; i++)
        {
            int a = faceOfHalf[2 * i];
            int b = faceOfHalf[2 * i + 1];
            if (a == b)
            {
                continue;
            }
            dual[a].push_back({b, edge[i].weight});
            dual[b].push_back({a, edge[i].weight});
        }
    }
};
```

构造 `PlanarEmbed<T,W> embedding(points)`，按输入边顺序调用 `addEdge(u,v,weight)` 后执行 `build()`；第 i 条原边对应半边 $2i$ 与 $2i+1$，`nextHalfEdge` 可还原每个面边界，`faceOfHalf`、`faceArea2`、`outer` 与 `dual` 分别给出半边面号、有符号双倍面积、外部面和带权对偶邻接。无点或单点且无边时返回唯一的零面积外部面；其余输入要求顶点坐标互异、图连通、无自环与重复边，线段只能在公共端点相交，同一顶点的边不能占用同向射线，边内部也不能经过第三个顶点。代码断言连通性等低成本条件，不额外做 $O(E^2)$ 的线段合法性检查；所有坐标差都先提升到 `__int128` 再相减，方向叉积和面积总和仍须能由 `__int128` 表

示。设顶点数、边数为 V, E，时间为 $O((V+E)\log(V+E))$，额外空间为 $O(V+E)$；拿到对偶图后需要求最小穿越代价时使用 5.2.007 平面图对偶最短路。

11.3 凸包

11.3.001 Andrew / Graham 静态凸包

用 Andrew 算法求静态点集凸包。默认删除边上的中间共线点并返回逆时针严格凸包。按坐标排序去重后分别维护下凸壳和上凸壳，遇到不符合指定转向的末点就弹栈；是否保留共线边界点由叉积比较用严格还是非严格决定。

```cpp
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
template <class T> vector<Point<T>> hull(vector<Point<T>> a)
{
    // a 是点集；返回删除边上共线中间点后的逆时针凸包。
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    if (a.size() <= 1)
    {
        return a;
    }
    vector<Point<T>> h;
    for (int t = 0; t < 2; t++)
    {
        int z = h.size();
        for (auto p : a)
        {
            while ((int)h.size() >= z + 2 && sgn(cross(h.back() - h[h.size() - 2], p - h.back())) <= 0)
            {
                h.pop_back();
            }
            h.push_back(p);
        }
        h.pop_back();
        reverse(a.begin(), a.end());
    }
    return h;
}

// 严格凸包的直径平方；输入需逆时针且无共线邻边。
```

`hull(p)` 返回逆时针严格凸包并删除边上中间共线点；整数点全程精确，`Point<Real>` 按统一误差判断转向，但排序与去重仍使用坐标精确比较。$O(n\log n)$ 时间、$O(n)$ 空间。静态建包使用 11.3.001 Andrew / Graham 静态凸包，只插点且需要在线维护才考虑 11.3.002 单点插入动态凸包。最远点对使用 11.3.006 旋转卡壳、凸包直径，平移碰撞与凸多边形距离可转为 11.3.008 闵可夫斯基和。

把下面的固定样例接在本章公共前导与上述实现之后即可快速检查：重复点和边上的共线中间点都应被删除，最终只留下正方形四角。

```
signed main()
{
    vector<P> a{{0, 0}, {1, 0}, {2, 0}, {2, 2}, {0, 2}, {0, 0}, {1, 1}};
    auto h = hull(a);
    cout << h.size() << '\n';
    for (P p : h)
    {
        cout << p.x << ' ' << p.y << '\n';
    }
}
// 期望依次输出四个点：(0,0)、(2,0)、(2,2)、(0,2)。
```

11.3.002 单点插入动态凸包

维护只插点的动态凸包。上壳和下壳分别按横坐标维护严格凸链；新点若不越过相邻壳边就丢弃，否则插入后向两侧删除转向不再严格的点。同一横坐标在上壳只保留最高点、下壳只保留最低点，因此包含查询也只需在两个集合中各找一次相邻点。

```
// 依赖 11.1.002 Point / Vector
template <class T> struct HullChain
{
    int dir;          // dir=1 维护上壳，dir=-1 维护下壳。
    set<Point<T>> a;  // a 按 x、y 保存当前严格凸链。

    HullChain(int dir) : dir(dir)
    {
        // dir 取 1 或 -1；分别构造上壳或下壳。
        assert(abs(dir) == 1); // 调试检查，可删
    }

    T turn(Point<T> x, Point<T> y, Point<T> z) const
    {
        // x、y、z 是三个点；返回按当前上下壳方向修正后的转向。
        return dir * cross(x, y, z);
    }

    bool add(Point<T> p)
    {
        // p 是新点；若 p 改变当前凸链则插入并返回 true。
        auto it = a.lower_bound({p.x, numeric_limits<T>::lowest()});
        if (it != a.end() && it->x == p.x)
        {
            if (dir * (p.y - it->y) <= 0)
            {
                return false;
            }
            a.erase(it);
        }
        it = a.lower_bound({p.x, numeric_limits<T>::lowest()});
        if (it != a.begin() && it != a.end() && turn(*prev(it), *it, p) <= 0)
```

```
        {
            return false;
        }
        auto q = a.insert(p).first;
        while (q ≠ a.begin())
        {
            auto y = prev(q);
            if (y == a.begin() || turn(*prev(y), *y, *q) < 0)
            {
                break;
            }
            a.erase(y);
        }
        while (next(q) ≠ a.end())
        {
            auto y = next(q);
            auto z = next(y);
            if (z == a.end() || turn(*q, *y, *z) < 0)
            {
                break;
            }
            a.erase(y);
        }
        return true;
    }

    bool contains(Point<T> p) const
    {
        // p 是查询点；返回 p 是否不越过当前凸链。
        if (a.empty())
        {
            return false;
        }
        auto it = a.lower_bound({p.x, numeric_limits<T>::lowest()});
        if (it ≠ a.end() && it->x == p.x)
        {
            return dir * (p.y - it->y) <= 0;
        }
        if (it == a.begin() || it == a.end())
        {
            return false;
        }
        return turn(*prev(it), *it, p) <= 0;
    }
};

template <class T> struct DynHull
{
    HullChain<T> up{1};    // up 保存从左到右的上凸链。
    HullChain<T> down{-1}; // down 保存从左到右的下凸链。

    void add(Point<T> p)
```

```
    {
        // p 是新点；把 p 插入动态凸包。
        up.add(p);
        down.add(p);
    }

    bool contains(Point<T> p) const
    {
        // p 是查询点；返回 p 是否在当前凸包内部或边界上。
        return up.contains(p) && down.contains(p);
    }

    vector<Point<T>> points() const
    {
        // 无参数；返回逆时针排列的当前严格凸包点。
        vector<Point<T>> h(up.a.begin(), up.a.end());
        for (auto it = down.a.rbegin(); it != down.a.rend(); it++)
        {
            if (h.empty() || (*it != h.front() && *it != h.back()))
            {
                h.push_back(*it);
            }
        }
        return h;
    }
};
```

构造 `DynHull<int>` `h` 后用 `h.add(p)` 插入整数点，`h.contains(p)` 在 $O(\log n)$ 时间判断包含，`h.points()` 用 $O(h)$ 时间输出当前逆时针严格凸包。每个点至多插入、删除常数次，单次操作摊还 $O(\log n)$、总空间 $O(n)$。集合比较要求坐标值稳定，浮点计算得到的近似点应先转为可靠的离散键，不能直接依赖 `eps` 改变 `set` 顺序。

11.3.003 点在凸包内

在严格逆时针凸包中判断点位于外部、边界还是内部。扇形二分把单次查询降到对数时间。凸多边形固定一个顶点后可按叉积二分查询点所在扇形，再判断点是否在对应三角形；边界、顶点和多边形退化为线段时需单独分类。

```
// 依赖 11.1.002 Point / Vector
template <class T> bool inConvex(const vector<Point<T>> &a, Point<T> p)
{
    // a 是逆时针严格凸包，p 是查询点；边界视为内部。
    int n = (int)a.size();
    if (n == 0)
    {
        return false;
    }
    if (n == 1)
    {
```

```
        return eq(a[0], p);
    }
    if (n == 2)
    {
        return onSeg(p, a[0], a[1]);
    }
    if (sgn(cross(a[0], a[1], p)) < 0 || sgn(cross(a[0], a.back(), p)) > 0)
    {
        return false;
    }
    int l = 1;
    int r = n - 1;
    while (r - l > 1)
    {
        int m = (l + r) / 2;
        if (sgn(cross(a[m] - a[0], p - a[0])) >= 0)
        {
            l = m;
        }
        else
        {
            r = m;
        }
    }
    int k = l;
    return sgn(cross(a[k], a[(k + 1) % n], p)) >= 0;
}
```

调用 `inConvex(h,p)`；`h` 必须是逆时针严格凸包，返回布尔值并把边界视为内部。$O(\log n)$ 时间、$O(1)$ 空间。静态建包使用 11.3.001 Andrew / Graham 静态凸包，只插点且需要在线维护才考虑 11.3.002 单点插入动态凸包。最远点对使用 11.3.006 旋转卡壳、凸包直径，平移碰撞与凸多边形距离可转为 11.3.008 闵可夫斯基和。

11.3.004 凸包极点、切点、最近点查询

查询凸包的循环单峰极点、相对外点的两条切点和最近点。严格逆时针凸包相对外点的可见边形成一段连续链，两端就是切点；从任一切点到另一切点的链上，点到边的投影方向只改变一次，因此可再二分最近边。退化为点或线段时直接处理，查询点在凸包内时最近距离为 0。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.1.009 线段相交与距离
// 依赖 11.3.003 点在凸包内
template <class T, class F> int hullExtreme(const vector<Point<T>> &a, F f)
{
    // a 是严格逆时针凸包，f(x,y) 比较相邻方向；返回对应循环单峰的极值下标。
    int l = 0, r = (int)a.size() - 1, d = 1;
    if (f(a[r], a[l]))
    {
```

```
        swap(l, r);
        d = -1;
    }
    while (d * (r - l) > 1)
    {
        int m = (l + r) / 2;
        if (f(a[m], a[l]) && f(a[m], a[m - d]))
        {
            l = m;
        }
        else
        {
            r = m;
        }
    }
    return l;
}

template <class T> pair<int, int> hullTangents(const vector<Point<T>> &a, Point<T> p)
{
    // a 是严格逆时针凸包，p 是查询点；返回 p 的两个切点下标。
    int n = a.size();
    if (!n || inConvex(a, p))
    {
        return {-1, -1};
    }
    if (n <= 2)
    {
        return {0, n - 1};
    }
    int x = hullExtreme(a, [&](Point<T> u, Point<T> v)
    {
        return sgn(cross(p, v, u)) > 0;
    });
    int y = hullExtreme(a, [&](Point<T> u, Point<T> v)
    {
        return sgn(cross(p, u, v)) > 0;
    });
    return {x, y};
}

P nearSegment(P p, P a, P b)
{
    // p 是查询点，a、b 是不同端点；返回线段 ab 上离 p 最近的点。
    P v = b - a;
    Real t = clamp(dot(p - a, v) / norm(v), Real(0), Real(1));
    return a + v * t;
}

optional<pair<Real, P>> closestHull(const vector<P> &a, P p)
{
    // a 是严格逆时针凸包，p 是查询点；返回最短距离和一个最近点。
```

```
    int n = a.size();
    if (!n)
    {
        return nullopt;
    }
    if (inConvex(a, p))
    {
        return pair<Real, P>{0, p};
    }
    if (n == 1)
    {
        return pair<Real, P>{abs(p - a[0]), a[0]};
    }
    if (n == 2)
    {
        P q = nearSegment(p, a[0], a[1]);
        return pair<Real, P>{abs(p - q), q};
    }
    auto at = [&](int i)
    {
        return a[(i % n + n) % n];
    };
    auto chain = [&](int l, int r)
    {
        if (l > r)
        {
            r += n;
        }
        auto get = [&](int i)
        {
            P q = nearSegment(p, at(i), at(i + 1));
            return pair<Real, P>{abs(p - q), q};
        };
        auto ans = get(l);
        int side = sgn(dot(p - at(l), at(l + 1) - at(l)));
        while (l + 1 < r)
        {
            int m = (l + r) / 2;
            int cur = sgn(dot(p - at(m), at(m + 1) - at(m)));
            if (cur == side)
            {
                l = m;
            }
            else
            {
                r = m;
            }
        }
        auto q = get(l);
        if (q.first < ans.first)
        {
            ans = q;
```

```
        }
        return ans;
    };
    auto [x, y] = hullTangents(a, p);
    auto u = chain(x, y);
    auto v = chain(y, x);
    return u.first < v.first ? u : v;
}
```

调用 `closestHull(h,p)`；有答案时返回 `{最短距离, 最近点}`，空凸包返回 `nullopt`。非退化输入要求 `h` 是没有共线中间点的严格逆时针凸包；外点查询 $O(\log h)$，内部判断同样为 $O(\log h)$，额外空间 $O(1)$。若凸包保留了整段共线边，应先删去中间点，否则切点端点约定和单峰性都会改变。

单独使用 `hullExtreme` 时，比较器定义相邻点中第一点是否更优。例如查最大横坐标：

```
signed main()
{
    vector<Point<int>> h{{0, 0}, {2, 0}, {0, 1}};
    int p = hullExtreme(h, [](Point<int> x, Point<int> y)
    {
        return x.x > y.x;
    });
    assert(p == 1 && h[p].x == 2);
}
```

11.3.005 凸包切线

求凸包相对外部点的两条切线。切点二分已经由 11.3.004 凸包极点、切点、最近点查询 的 `hullTangents` 完整给出，本条只把两个切点转成直线，避免重复一套容易产生不同边界约定的二分。严格凸包上的可见边形成循环连续段，两条切线正好经过该段的两个端点。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.005 Line / Ray / Segment
template <class T> optional<array<Line<T>, 2>> hullTan(const vector<Point<T>> &a, Point<T> p)
{
    // a 是严格逆时针凸包，p 是查询点；外点返回两条切线，否则返回空。
    auto [x, y] = hullTangents(a, p);
    if (x < 0)
    {
        return nullopt;
    }
    return array<Line<T>, 2>{Line<T>{p, a[x]}, Line<T>{p, a[y]}};
}
```

`hullTangents(h,p)` 返回两个切点下标，`hullTan(h,p)` 返回对应点式直线；查询点在内部或边界时前者返回 `{-1,-1}`、后者返回空。严格凸包查询为 $O(\log h)$ 时间、$O(1)$ 空间；共线边的切点不唯一，若题目要求整段接触边，应在返回端点附近另行扩展相等方向。

11.3.006 旋转卡壳、凸包直径

在凸包上用旋转卡壳求最远点对和直径。卡壳指针随边方向单调移动，因此扫描为线性。凸包对踵点随枚举边单调前进；当下一点使边与点的叉积绝对值增大就移动指针，每对最远点会在某条支撑线方向被检查。

```cpp
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
template <class T> T diameter2(const vector<Point<T>> &a)
{
    // a 是逆时针凸包；返回最远点对距离平方。
    int n = (int)a.size();
    if (n <= 1)
    {
        return 0;
    }
    if (n == 2)
    {
        return norm(a[0] - a[1]);
    }
    T ans = 0;
    int j = 1;
    for (int i = 0; i < n; i++)
    {
        while (sgn(cross(a[(i + 1) % n] - a[i], a[(j + 1) % n] - a[j])) > 0)
        {
            j = (j + 1) % n;
        }
        ans = max(ans, norm(a[i] - a[j]));
        ans = max(ans, norm(a[(i + 1) % n] - a[j]));
    }
    return ans;
}

template <class T> pair<int, int> farthestPair(const vector<Point<T>> &p)
{
    // p 是至少含两个点的点集；返回一对欧氏距离最大的原下标。
    assert(p.size() >= 2); // 调试检查，可删。
    map<Point<T>, int> id;
    for (int i = 0; i < (int)p.size(); i++)
    {
        id.try_emplace(p[i], i);
    }
    auto h = hull(p);
    if (h.size() == 1) return {0, 1};
    if (h.size() == 2) return {id[h[0]], id[h[1]]};
    int m = h.size(), j = 1;
    T best = -1;
    pair<int, int> ans{0, 1};
    auto update = [&](int x, int y)
    {
        T d = norm(h[x] - h[y]);
        if (best < d)
```

```
        {
            best = d;
            ans = {id[h[x]], id[h[y]]};
        }
    };
    for (int i = 0; i < m; i++)
    {
        while (cross(h[(i + 1) % m] - h[i],
                     h[(j + 1) % m] - h[j]) > 0)
        {
            j = (j + 1) % m;
        }
        update(i, j);
        update((i + 1) % m, j);
    }
    return ans;
}
```

调用 `diameter2(h)` 时 `h` 按逆时针给出且不得重复首点，并要求严格凸，返回最远点对距离平方，$O(h)$ 时间、$O(1)$ 额外空间。输入是按边界顺序给出的弱凸多边形时调用 `weakDiam2(h)`；它删除共线中间点，单独处理整图共线，并在平行支撑线并列时检查两个候选，$O(h)$ 时间、$O(h)$ 额外空间，顺逆时针均可。`farthestPair(p)` 接受至少两个、可重复且无需预建凸包的点，返回一对最远点的原下标；全相同点返回两个不同下标。它先以 $O(n \log n)$ 建凸包，再用 $O(h)$ 旋转卡壳，额外空间 $O(n)$。静态建包使用 11.3.001 Andrew / Graham 静态凸包，只插点且需要在线维护才考虑 11.3.002 单点插入动态凸包。平移碰撞与凸多边形距离可转为 11.3.008 闵可夫斯基和。

把下面的固定样例接在本章公共前导与上述实现之后即可快速检查：3×4 矩形的直径平方应为 25。

```
signed main()
{
    vector<P> h{{0, 0}, {3, 0}, {3, 4}, {0, 4}};
    cout << diameter2(h) << '\n';
}
// 期望输出：25
```

11.3.007 最大面积三角形、四边形

求凸包顶点组成的最大面积三角形或四边形。三角形可双指针，当前四边形基准版保留更高复杂度以便核对。固定凸包一条边时，对边面积最大的第三点沿边推进而单调移动；四边形可用两条对角线两侧最远点面积之和，避免枚举四点。

```
// 依赖 11.1.002 Point / Vector
template <class T> tuple<T, int, int, int> maxTriangle(const vector<Point<T>> &a)
{
    // a 是逆时针凸包；返回最大双倍面积和三个顶点下标。
    int n = (int)a.size();
    tuple<T, int, int, int> ans{0, -1, -1, -1};
    if (n < 3)
    {
```

```
        return ans;
    }
    auto area = [&](int i, int j, int k)
    {
        T s = cross(a[i % n], a[j % n], a[k % n]);
        return s < 0 ? -s : s;
    };
    for (int i = 0; i < n; i++)
    {
        int k = i + 2;
        for (int j = i + 1; j < n; j++)
        {
            if (k <= j)
            {
                k = j + 1;
            }
            while (k + 1 < i + n && sgn(area(i, j, k + 1) - area(i, j, k)) > 0)
            {
                k++;
            }
            T s = area(i, j, k);
            if (s > get<0>(ans))
            {
                ans = {s, i, j % n, k % n};
            }
        }
    }
    return ans;
}

template <class T> T maxQuad(const vector<Point<T>> &a)
{
    // a 是逆时针凸包；返回最大双倍面积。
    int n = (int)a.size();
    T ans = 0;
    auto area = [&](int i, int j, int k)
    {
        T s = cross(a[i], a[j], a[k]);
        return s < 0 ? -s : s;
    };
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            T x = 0;
            T y = 0;
            for (int k = i + 1; k < j; k++)
            {
                x = max(x, area(i, j, k));
            }
            for (int k = j + 1; k < n; k++)
            {
```

```
                y = max(y, area(i, j, k));
            }
            for (int k = 0; k < i; k++)
            {
                y = max(y, area(i, j, k));
            }
            ans = max(ans, x + y);
        }
    }
    return ans;
}
```

`maxTriangle(h)` 返回双倍面积和三个顶点下标，`maxQuad(h)` 返回双倍面积；输入必须是逆时针凸包。最大三角形 $O(h^2)$，当前最大四边形 $O(h^3)$。静态建包使用 11.3.001 Andrew / Graham 静态凸包，只插点且需要在线维护才考虑 11.3.002 单点插入动态凸包。最远点对使用 11.3.006 旋转卡壳、凸包直径，平移碰撞与凸多边形距离可转为 11.3.008 闵可夫斯基和。

11.3.008 闵可夫斯基和

计算两个凸多边形的闵可夫斯基和。规范化顶点起点后按极角归并边向量，可用于平移碰撞和凸多边形距离。两个逆时针凸多边形的闵可夫斯基和，可从各自最低点出发按极角归并边向量并累加；结果再去除共线冗余点得到凸包。

```
// 依赖 11.1.002 Point / Vector
template <class T> vector<Point<T>> minkowski(vector<Point<T>> a, vector<Point<T>> b)
{
    // a、b 是两个点集；返回其凸包的闵可夫斯基和。
    a = hull(a);
    b = hull(b);
    if (a.empty() || b.empty())
    {
        return {};
    }
    if (a.size() <= 2 || b.size() <= 2)
    {
        vector<Point<T>> c;
        for (Point<T> x : a)
        {
            for (Point<T> y : b)
            {
                c.push_back(x + y);
            }
        }
        return hull(c);
    }
    auto edge = [](const vector<Point<T>> &p)
    {
        vector<Point<T>> e(p.size());
        for (int i = 0; i < (int)p.size(); i++)
```

```
        {
            e[i] = p[(i + 1) % p.size()] - p[i];
        }
        return e;
    };
    auto x = edge(a);
    auto y = edge(b);
    vector<Point<T>> c = {a[0] + b[0]};
    int i = 0;
    int j = 0;
    while (i < (int)x.size() || j < (int)y.size())
    {
        if (j == (int)y.size() || (i < (int)x.size() && sgn(cross(x[i], y[j])) > 0))
        {
            c.push_back(c.back() + x[i++]);
        }
        else if (i == (int)x.size() || sgn(cross(x[i], y[j])) < 0)
        {
            c.push_back(c.back() + y[j++]);
        }
        else
        {
            c.push_back(c.back() + x[i++] + y[j++]);
        }
    }
    c.pop_back();
    return hull(c);
}
```

调用 `minkowski(a,b)`；函数内部先对两组点建严格凸包，再按极角归并边向量。$O((n+m)\log(n+m))$ 时间、$O(n+m)$ 空间。静态建包使用 11.3.001 Andrew / Graham 静态凸包，只插点且需要在线维护才考虑 11.3.002 单点插入动态凸包。最远点对使用 11.3.006 旋转卡壳、凸包直径，平移碰撞与凸多边形距离可转为 11.3.008 闵可夫斯基和。

11.3.009 半平面交

求一组浮点有向半平面的交。先按方向排序并去除同向冗余直线，再用双端队列维护有效边界。每条有向直线保留左侧。按极角排序后，同向直线只留约束更紧者；当队尾或队首交点落到新半平面外时弹出，对列最终相邻直线交点构成交多边形。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.1.006 平行、垂直、共线、同侧/异侧
// 依赖 11.1.007 直线交点、投影、对称点
// 依赖 11.2.001 多边形面积、周长、重心
struct HP
{
    P p; // p 保存边界直线上的一点。
    P v; // v 保存边界直线的方向。
```

```
    Real a; // a 保存 v 在 [0,2pi) 内的极角。

    HP(P p, P q) : p(p), v(q - p), a(atan2l(v.y, v.x))
    {
        // p、q 是边界直线上两个不同点；构造有向直线 p→q，保留左侧闭半平面，a 为方向极角并规范到 [0,2π)。
        assert(!eq(p, q)); // 调试检查，可删。
        if (a < 0)
        {
            a += 2 * acosl(-1);
        }
    }

    bool out(P q) const
    {
        // q 是查询点；返回其是否严格位于右侧。
        return sgn(cross(v, q - p)) < 0;
    }
};

optional<vector<P>> halfPlane(vector<HP> a)
{
    // a 中每条有向边保留左侧；空 optional 表示非空但无界。
    auto feasible = [&](vector<HP> q) -> optional<P>
    {
        mt19937 rng(0);
        shuffle(q.begin(), q.end(), rng);
        P x{};
        const Real inf = numeric_limits<Real>::infinity();
        for (int i = 0; i < (int)q.size(); i++)
        {
            if (!q[i].out(x))
            {
                continue;
            }
            Real lo = -inf;
            Real hi = inf;
            for (int j = 0; j < i; j++)
            {
                Real c = cross(q[j].v, q[i].p - q[j].p);
                Real k = cross(q[j].v, q[i].v);
                if (sgn(k) == 0)
                {
                    if (sgn(c) < 0)
                    {
                        return nullopt;
                    }
                    continue;
                }
                Real t = -c / k;
                if (k > 0)
                {
                    lo = max(lo, t);
```

```
                }
                else
                {
                    hi = min(hi, t);
                }
            }
            if (sgn(lo - hi) > 0)
            {
                return nullopt;
            }
            Real t = 0;
            if (isfinite(lo))
            {
                t = lo;
            }
            else if (isfinite(hi))
            {
                t = hi;
            }
            x = q[i].p + q[i].v * t;
        }
        return x;
    };
    if (!feasible(a))
    {
        return vector<P>{};
    }
    if (a.empty())
    {
        return nullopt;
    }
    vector<Real> ang;
    for (const HP &h : a)
    {
        ang.push_back(h.a);
    }
    sort(ang.begin(), ang.end());
    Real pi = acosl(-1);
    for (int i = 0; i < (int)ang.size(); i++)
    {
        Real x = ang[i];
        Real y = i + 1 < (int)ang.size() ? ang[i + 1] : ang[0] + 2 * pi;
        if (sgn(y - x - pi) >= 0)
        {
            return nullopt;
        }
    }
    sort(a.begin(), a.end(), [](const HP &x, const HP &y)
         {
             if (x.a != y.a)
             {
                 return x.a < y.a;
```

```
            }
            return cross(x.v, y.p - x.p) < 0;
        });
    vector<HP> b;
    for (HP h : a)
    {
        if (!b.empty() && parallel({b.back().p, b.back().v}, {h.p, h.v})
            && sgn(dot(b.back().v, h.v)) > 0)
        {
            if (h.out(b.back().p))
            {
                b.back() = h;
            }
            continue;
        }
        b.push_back(h);
    }
    if (b.size() > 1 && parallel({b.front().p, b.front().v}, {b.back().p, b.back().v})
        && sgn(dot(b.front().v, b.back().v)) > 0)
    {
        if (b.front().out(b.back().p))
        {
            b.back() = b.front();
        }
        b.erase(b.begin());
    }
    deque<HP> q;
    for (HP h : b)
    {
        while (q.size() > 1
               && h.out(isect({q[q.size() - 2].p, q[q.size() - 2].v},
                              {q.back().p, q.back().v})))
        {
            q.pop_back();
        }
        while (q.size() > 1
               && h.out(isect({q[0].p, q[0].v}, {q[1].p, q[1].v})))
        {
            q.pop_front();
        }
        q.push_back(h);
    }
    while (q.size() > 2
           && q.front().out(isect({q[q.size() - 2].p, q[q.size() - 2].v},
                                  {q.back().p, q.back().v})))
    {
        q.pop_back();
    }
    while (q.size() > 2
           && q.back().out(isect({q[0].p, q[0].v}, {q[1].p, q[1].v})))
    {
        q.pop_front();
```

```
    }
    if (q.size() < 3)
    {
        return vector<P>{};
    }
    vector<P> p;
    for (int i = 0; i < (int)q.size(); i++)
    {
        const HP &x = q[i];
        const HP &y = q[(i + 1) % q.size()];
        assert(!parallel({x.p, x.v}, {y.p, y.v})); // 调试检查，可删。
        P z = isect({x.p, x.v}, {y.p, y.v});
        if (p.empty() || !eq(p.back(), z))
        {
            p.push_back(z);
        }
    }
    if (p.size() > 1 && eq(p.front(), p.back()))
    {
        p.pop_back();
    }
    if (p.size() > 2 && sgn(area(p)) == 0)
    {
        P x = *min_element(p.begin(), p.end());
        P y = *max_element(p.begin(), p.end());
        return eq(x, y) ? vector<P>{x} : vector<P>{x, y};
    }
    return p;
}
```

把约束写成 `HP(p,q)`，表示有向直线 $p \to q$ 的左闭半平面。`halfPlane(lines)` 返回 `nullopt` 表示非空但无界，返回空向量表示空集，其余返回有界交的逆时针顶点；点和线段形式的有界交也会保留。可行性增量检查期望 $O(n)$、最坏 $O(n^2)$，边界构造 $O(n \log n)$，空间 $O(n)$。静态建包使用 11.3.001 Andrew / Graham 静态凸包，只插点且需要在线维护才考虑 11.3.002 单点插入动态凸包。最远点对使用 11.3.006 旋转卡壳、凸包直径，平移碰撞与凸多边形距离可转为 11.3.008 闵可夫斯基和。

下面四组固定约束依次检查乱序矩形、同向冗余边、空集和无界半条带。

```
signed main()
{
    vector<HP> box{{{0, 1}, {0, 0}}, {{1, 0}, {1, 1}},
                   {{1, 1}, {0, 1}}, {{0, 0}, {1, 0}},
                   {{2, 0}, {2, 1}}};
    auto a = halfPlane(box);
    assert(a && eq(area(*a), 1));
    auto empty = halfPlane(vector<HP>{{{0, 0}, {0, 1}},
                                      {{1, 1}, {1, 0}}}); // x<=0 与 x>=1 无交。
    assert(empty && empty->empty());
    auto unbounded = halfPlane(vector<HP>{{{0, 1}, {0, 0}},
                                          {{1, 0}, {1, 1}},
                                          {{0, 0}, {1, 0}}});
```

```
    assert(!unbounded);
    cout << "OK\n";
}
```

11.3.010 整数半平面交

求若干整数线性不等式 `a*x+b*y+c>=0` 的公共区域。排序、删线和交点内外判定全部使用 `__int128`，适合不能接受浮点误差的凸多边形裁剪与线性约束可行域。返回边界直线和有理交点，不把非整数交点截断。把每条约束写成整数系数半平面。按边界方向极角排序，同向约束只保留更严格的一条；队列两端是否失效由三条直线系数的行列式直接判定，无需求出浮点交点。最终相邻边界的交点以两个分子和一个正分母保存，所以全程保持精确。

```
namespace integer_hpi
{
// ILine[0..2] 依次是整数半平面 a*x+b*y+c>=0 的系数 a、b、c。
using ILine = array<int, 3>;
// IRatPoint[0..2] 依次是有理点横坐标分子、纵坐标分子和公共正分母。
using IRatPoint = array<__int128, 3>;

int half(int x, int y)
{
    // x、y 是非零方向向量的坐标；返回它在极角排序中所属的半平面编号。
    return y < 0 || (y == 0 && x < 0);
}

__int128 det(const ILine &p, const ILine &q)
{
    // p、q 是两条半平面的系数；返回它们法向量的叉积。
    return (__int128)p[0] * q[1] - (__int128)p[1] * q[0];
}

bool sameDir(const ILine &p, const ILine &q)
{
    // p、q 是两条半平面；返回它们的边界方向是否相同，而不是相反。
    return det(p, q) == 0 && (__int128)p[0] * q[0] + (__int128)p[1] * q[1] > 0;
}

bool stricter(const ILine &p, const ILine &q)
{
    // p、q 必须同向；返回 p 是否比 q 的可行半平面更小。
    auto ab = [](__int128 x)
    {
        return x < 0 ? -x : x;
    };
    __int128 xp = p[0] ? ab(p[0]) : ab(p[1]);
    __int128 xq = q[0] ? ab(q[0]) : ab(q[1]);
    return (__int128)p[2] * xq < (__int128)q[2] * xp;
}

```

```cpp
bool compatible(const ILine &p, const ILine &q)
{
    // p、q 的法向量必须反向；返回两个闭半平面是否相容。
    __int128 d = (__int128)p[0] * p[0] + (__int128)p[1] * p[1];
    __int128 x = -(__int128)p[0] * p[2];
    __int128 y = -(__int128)p[1] * p[2];
    return (__int128)q[0] * x + (__int128)q[1] * y + (__int128)q[2] * d >= 0;
}

bool outside(const ILine &a, const ILine &b, const ILine &c)
{
    // a 是待检查半平面，b、c 是两条不平行边界；返回 b、c 交点是否严格落在 a 外。
    __int128 d = det(b, c);
    assert(d != 0); // 调试检查，可删。
    __int128 x = (__int128)b[1] * c[2] - (__int128)b[2] * c[1];
    __int128 y = (__int128)b[2] * c[0] - (__int128)b[0] * c[2];
    __int128 v = (__int128)a[0] * x + (__int128)a[1] * y + (__int128)a[2] * d;
    return v != 0 && ((v < 0) != (d < 0));
}

optional<vector<ILine>> halfPlane(vector<ILine> a)
{
    // a 是全部整数半平面；空 optional 表示非空但无界。
    for (auto &l : a)
    {
        assert(l[0] != 0 || l[1] != 0); // 调试检查，可删。
    }
    sort(a.begin(),
         a.end(),
         [](const ILine &p, const ILine &q)
         {
             int hp = half(p[1], -p[0]);
             int hq = half(q[1], -q[0]);
             if (hp != hq)
             {
                 return hp < hq;
             }
             auto d = det(p, q);
             if (d != 0)
             {
                 return d > 0;
             }
             if (sameDir(p, q))
             {
                 return stricter(p, q);
             }
             return false;
         });

    vector<ILine> b;
    for (auto l : a)
    {
```

```
        if (!b.empty() && sameDir(b.back(), l))
        {
            continue;
        }
        b.push_back(l);
    }

    if (b.empty())
    {
        return nullopt;
    }
    auto ab = [](__int128 x)
    {
        return x < 0 ? -x : x;
    };
    auto gcd128 = [&](__int128 x, __int128 y)
    {
        x = ab(x);
        y = ab(y);
        while (y != 0)
        {
            __int128 r = x % y;
            x = y;
            y = r;
        }
        return x;
    };
    map<pair<__int128, __int128>, ILine> dir;
    for (const ILine &l : b)
    {
        __int128 g = gcd128(l[0], l[1]);
        __int128 x = (__int128)l[0] / g;
        __int128 y = (__int128)l[1] / g;
        auto it = dir.find({-x, -y});
        if (it != dir.end() && !compatible(l, it->second))
        {
            return vector<ILine>{};
        }
        dir[{x, y}] = l;
    }
    for (int i = 0; i < (int)b.size(); i++)
    {
        if (det(b[i], b[(i + 1) % b.size()]) <= 0)
        {
            return nullopt;
        }
    }

    deque<ILine> q;
    for (auto l : b)
    {
        while (q.size() >= 2 && outside(l, q[q.size() - 2], q.back()))
```

```
        {
            q.pop_back();
        }
        while (q.size() >= 2 && outside(l, q[0], q[1]))
        {
            q.pop_front();
        }
        q.push_back(l);
    }
    while (q.size() >= 3 && outside(q.front(), q[q.size() - 2], q.back()))
    {
        q.pop_back();
    }
    while (q.size() >= 3 && outside(q.back(), q[0], q[1]))
    {
        q.pop_front();
    }
    if (q.size() < 3)
    {
        return vector<ILine>{};
    }
    return vector<ILine>{q.begin(), q.end()};
}

vector<IRatPoint> vertices(const vector<ILine> &q)
{
    // q 是 halfPlane 返回的有界边界；返回相邻边界交点的“横分子、纵分子、正分母”。
    vector<IRatPoint> p;
    auto ab = [](__int128 z)
    {
        return z < 0 ? -z : z;
    };
    auto gcd128 = [&](__int128 u, __int128 v)
    {
        u = ab(u);
        v = ab(v);
        while (v != 0)
        {
            __int128 r = u % v;
            u = v;
            v = r;
        }
        return u;
    };
    for (int i = 0; i < (int)q.size(); i++)
    {
        auto a = q[i];
        auto b = q[(i + 1) % q.size()];
        __int128 d = det(a, b);
        __int128 x = (__int128)a[1] * b[2] - (__int128)a[2] * b[1];
        __int128 y = (__int128)a[2] * b[0] - (__int128)a[0] * b[2];
        if (d < 0)
```

```
        {
            x = -x;
            y = -y;
            d = -d;
        }
        __int128 g = gcd128(gcd128(x, y), d);
        x /= g;
        y /= g;
        d /= g;
        IRatPoint z{x, y, d};
        if (p.empty() || p.back() != z)
        {
            p.push_back(z);
        }
    }
    if (p.size() > 1 && p.front() == p.back())
    {
        p.pop_back();
    }
    return p;
}
}
```

在 `integer_hpi` 命名空间中，把约束写成 `ILine{a,b,c}`，表示 `a*x+b*y+c>=0`。`halfPlane(lines)` 返回 `nullopt` 表示非空但无界，返回空向量表示空集，其余返回逆时针边界；再用 `vertices(*q)` 得到约分后的有理交点，点和线段形式的有界交会自动删除相邻重复点。系数三次乘积必须落在 `__int128` 范围内。$O(n \log n)$ 时间、$O(n)$ 空间；其中 n 为半平面数，排序后每条边至多入队和出队一次。静态建包使用 11.3.001 Andrew / Graham 静态凸包，只插点且需要在线维护才考虑 11.3.002 单点插入动态凸包。最远点对使用 11.3.006 旋转卡壳、凸包直径，平移碰撞与凸多边形距离可转为 11.3.008 闵可夫斯基和。可据此手算自查：四条约束 `{1,0,0}`、`{0,1,0}`、`{-1,0,2}`、`{0,-1,3}` 表示矩形 `[0,2]x[0,3]`；期望返回 4 条边，四个有理顶点分母均为 1，面积为 6。再加入 `{1,0,-4}` 后交集为空；只保留前三条时交集无界。

11.3.011 Delaunay 三角剖分 / Voronoi 图

分治合并左右两侧的 Delaunay 图。每个点的出边按旋转顺序组成双向循环链，合并时从共同下切线向上扫描；若候选边的对点落入外接圆，就立刻删除该边，再连接新的跨区边。这个结构既给出线性条数的平面边，也保留了恢复三角面和 Voronoi 对偶所需的嵌入顺序。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
using IP = Point<int>;
using IW = __int128_t;

struct DelaunayG
{
    struct Edge
    {
        int to = 0;           // to 保存有向边终点。
        int ccw = 0, cw = 0; // ccw、cw 是同起点边的逆时针、顺时针相邻边。
```

```
        int rev = 0;          // rev 保存反向边编号。
        bool on = false;      // on 表示边当前是否存在。
    };

    vector<IP> p;       // p 是按 x、y 排序且去重后的点。
    vector<Edge> e;     // e 保存可回收的旋转边结构。
    vector<int> free;   // free 保存已删除边的空闲编号。

    DelaunayG(vector<IP> p) : p(move(p))
    {
        // p 必须按字典序严格递增；构造 Delaunay 平面图。
        if (this->p.size() >= 2)
        {
            build(0, this->p.size());
        }
    }

    static bool inCircle(IP a, IP b, IP c, IP d)
    {
        // a、b、c、d 是整数点；返回 d 是否严格位于有向圆 abc 内。
        a -= d, b -= d, c -= d;
        auto cr = [](IP x, IP y)
        {
            return (IW)x.x * y.y - (IW)x.y * y.x;
        };
        auto sq = [](IP x)
        {
            return (IW)x.x * x.x + (IW)x.y * x.y;
        };
        IW z = cr(b, c) * sq(a) + cr(c, a) * sq(b) + cr(a, b) * sq(c);
        return z > 0;
    }

    int turn(int a, int b, int c) const
    {
        // a、b、c 是点下标；返回三点的精确转向。
        auto z = cross(p[b] - p[a], p[c] - p[a]);
        return (z > 0) - (z < 0);
    }

    int slot()
    {
        // 无参数；返回一个可写边编号。
        if (free.empty())
        {
            e.push_back({});
            return (int)e.size() - 1;
        }
        int x = free.back();
        free.pop_back();
        return x;
    }
```

```
64
65      pair<int, int> makeEdge(int u, int v)
66      {
67          // u、v 是端点；建立一对反向边并返回编号。
68          int a = slot(), b = slot();
69          e[a] = {v, a, a, b, true};
70          e[b] = {u, b, b, a, true};
71          return {a, b};
72      }
73
74      void eraseOne(int x)
75      {
76          // x 是有向边；从其起点的旋转链中删除它。
77          e[e[x].ccw].cw = e[x].cw;
78          e[e[x].cw].ccw = e[x].ccw;
79          e[x].on = false;
80      }
81
82      void eraseEdge(int x)
83      {
84          // x 是有向边；删除它及其反向边并回收编号。
85          int y = e[x].rev;
86          eraseOne(x);
87          eraseOne(y);
88          free.push_back(x);
89          free.push_back(y);
90      }
91
92      void insertCcw(int x, int at)
93      {
94          // x 是新边，at 是同起点边；把 x 插到 at 的逆时针一侧。
95          int y = e[at].ccw;
96          e[x].ccw = y, e[y].cw = x;
97          e[x].cw = at, e[at].ccw = x;
98      }
99
100     void insertCw(int x, int at)
101     {
102         // x 是新边，at 是同起点边；把 x 插到 at 的顺时针一侧。
103         int y = e[at].cw;
104         e[x].cw = y, e[y].ccw = x;
105         e[x].ccw = at, e[at].cw = x;
106     }
107
108     pair<int, int> next(int x) const
109     {
110         // x 是当前出边；返回逆时针相邻点及对应出边。
111         int v = e[x].to;
112         return {v, e[e[x].rev].ccw};
113     }
114
115     pair<int, int> prev(int x) const
```

```
    {
        // x 是当前出边；返回顺时针相邻点及对应出边。
        int y = e[x].cw;
        return {e[y].to, e[y].rev};
    }

    tuple<int, int, int, int> bottom(int a, int ea, int b, int eb) const
    {
        // a、b 在两个凸壳上，ea、eb 是出边；返回共同下切线状态。
        while (true)
        {
            auto [x, ex] = prev(ea);
            if (turn(b, a, x) > 0)
            {
                a = x, ea = ex;
                continue;
            }
            auto [y, ey] = next(eb);
            if (turn(a, b, y) < 0)
            {
                b = y, eb = ey;
                continue;
            }
            return {a, ea, b, eb};
        }
    }

    pair<int, int> extreme(int a, int ea, bool low) const
    {
        // a、ea 是凸壳入口，low 控制取最小或最大编号；返回极值点状态。
        pair<int, int> ans{a, ea};
        int u = a, x = ea;
        do
        {
            tie(u, x) = next(x);
            ans = low ? min(ans, pair{u, x}) : max(ans, pair{u, x});
        } while (x != ea);
        return ans;
    }

    pair<int, int> join(int a, int ea, int b, int eb)
    {
        // a、b 是左右 Delaunay 图入口；合并两侧并返回新外壳入口。
        tie(a, ea) = extreme(a, ea, false);
        tie(b, eb) = extreme(b, eb, true);
        auto [al, eal, bl, ebl] = bottom(a, ea, b, eb);
        auto [bu, ebu, au, eau] = bottom(b, eb, a, ea);
        ebl = e[ebl].cw;
        ebu = e[ebu].cw;
        auto [ab, ba] = makeEdge(al, bl);
        insertCw(ab, eal);
        insertCcw(ba, ebl);
```

```
        if (al == au)
        {
            eau = ab;
        }
        if (bl == bu)
        {
            ebu = ba;
        }
        int ap = al, eap = eal, bp = bl, ebp = ebl;
        while (ap ≠ au || bp ≠ bu)
        {
            int a2 = e[eap].to, b2 = e[ebp].to;
            int na = e[eap].ccw, nb = e[ebp].cw;
            if (eap ≠ eau && na ≠ ab)
            {
                int a1 = e[na].to;
                if (inCircle(p[ap], p[bp], p[a2], p[a1]))
                {
                    eraseEdge(eap);
                    eap = na;
                    continue;
                }
            }
            if (ebp ≠ ebu && nb ≠ ba)
            {
                int b1 = e[nb].to;
                if (inCircle(p[b2], p[ap], p[bp], p[b1]))
                {
                    eraseEdge(ebp);
                    ebp = nb;
                    continue;
                }
            }
            bool takeA = ebp == ebu;
            if (eap ≠ eau && ebp ≠ ebu)
            {
                if (turn(ap, bp, b2) < 0)
                {
                    takeA = true;
                }
                else if (turn(a2, ap, bp) < 0)
                {
                    takeA = false;
                }
                else
                {
                    takeA = inCircle(p[ap], p[bp], p[b2], p[a2]);
                }
            }
            if (takeA)
            {
                na = e[e[eap].rev].ccw;
```

```
                auto [x, y] = makeEdge(a2, bp);
                insertCw(x, na);
                insertCcw(y, ebp);
                eap = na, ap = a2;
            }
            else
            {
                nb = e[e[ebp].rev].cw;
                auto [x, y] = makeEdge(b2, ap);
                insertCcw(x, nb);
                insertCw(y, eap);
                ebp = nb, bp = b2;
            }
        }
        return {al, ab};
    }

    pair<int, int> build(int l, int r)
    {
        // l、r 是有序点的半开区间；递归构造并返回外壳入口。
        if (r - l == 2)
        {
            auto [x, y] = makeEdge(l, l + 1);
            return {l, x};
        }
        if (r - l == 3)
        {
            int u = l, v = l + 1, w = l + 2;
            auto [uv, vu] = makeEdge(u, v);
            auto [vw, wv] = makeEdge(v, w);
            int z = turn(u, v, w);
            if (!z)
            {
                insertCcw(vu, vw);
                return {u, uv};
            }
            auto [uw, wu] = makeEdge(u, w);
            if (z > 0)
            {
                insertCw(uv, uw), insertCw(vw, vu), insertCw(wu, wv);
                return {u, uv};
            }
            insertCcw(uv, uw), insertCcw(vw, vu), insertCcw(wu, wv);
            return {v, vu};
        }
        int m = (l + r) / 2;
        auto [a, ea] = build(l, m);
        auto [b, eb] = build(m, r);
        return join(a, ea, b, eb);
    }

    vector<pair<int, int>> edges() const
```

```
    {
        // 无参数；返回每条当前无向边一次。
        vector<pair<int, int>> ans;
        for (int i = 0; i < (int)e.size(); i++)
        {
            if (e[i].on && i > e[i].rev)
            {
                ans.push_back(minmax(e[i].to, e[e[i].rev].to));
            }
        }
        sort(ans.begin(), ans.end());
        return ans;
    }
};

struct DelaunayRes
{
    struct VorEdge
    {
        P a;     // a 是线段端点或射线、直线上的一点。
        P b;     // type=0 时 b 是另一端点，否则 b 是方向向量。
        int type; // type 为 0、1、2 时分别表示线段、射线、直线。
    };

    vector<pair<int, int>> edge; // edge 保存 Delaunay 无向边原下标。
    vector<array<int, 3>> tri;   // tri 保存逆时针三角面原下标。
    vector<P> center;            // center 保存与 tri 同序的外心。
    vector<VorEdge> vor;         // vor 保存完整 Voronoi 边。
};

DelaunayRes delaunay(const vector<IP> &input)
{
    // input 是整数点集；返回 Delaunay 三角剖分及其 Voronoi 对偶。
    vector<int> ord(input.size());
    iota(ord.begin(), ord.end(), 0);
    sort(ord.begin(), ord.end(), [&](int x, int y)
    {
        return input[x] != input[y] ? input[x] < input[y] : x < y;
    });
    vector<IP> p;
    vector<int> id;
    for (int x : ord)
    {
        if (p.empty() || p.back() != input[x])
        {
            p.push_back(input[x]);
            id.push_back(x);
        }
    }
    DelaunayRes ans;
    if (p.size() < 2)
    {
```

```
        return ans;
    }
    DelaunayG g(p);
    auto es = g.edges();
    int n = p.size();
    vector<vector<int>> adj(n);
    for (auto [u, v] : es)
    {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    auto halfInt = [](IP x)
    {
        return x.y < 0 || (x.y == 0 && x.x < 0);
    };
    map<pair<int, int>, int> pos;
    for (int u = 0; u < n; u++)
    {
        sort(adj[u].begin(), adj[u].end(), [&](int x, int y)
        {
            IP a = p[x] - p[u], b = p[y] - p[u];
            int ha = halfInt(a), hb = halfInt(b);
            auto z = cross(a, b);
            return ha != hb ? ha < hb : (z ? z > 0 : norm(a) < norm(b));
        });
        for (int i = 0; i < (int)adj[u].size(); i++)
        {
            pos[{u, adj[u][i]}] = i;
        }
    }
    set<pair<int, int>> used;
    vector<array<int, 3>> tri;
    for (auto [s, t] : es)
    {
        for (auto [u0, v0] : {pair{s, t}, pair{t, s}})
        {
            if (used.contains({u0, v0}))
            {
                continue;
            }
            vector<int> face;
            int u = u0, v = v0;
            while (!used.contains({u, v}))
            {
                used.insert({u, v});
                face.push_back(u);
                int k = pos[{v, u}];
                int w = adj[v][(k - 1 + adj[v].size()) % adj[v].size()];
                u = v, v = w;
            }
            IW area = 0;
            for (int i = 0; i < (int)face.size(); i++)
```

```
            {
                area += cross(p[face[i]], p[face[(i + 1) % face.size()]]);
            }
            if (area > 0)
            {
                assert(face.size() == 3); // 调试检查，可删
                tri.push_back({face[0], face[1], face[2]});
            }
        }
    }
    map<pair<int, int>, vector<int>> side;
    for (auto e : es)
    {
        side[e];
        ans.edge.push_back(minmax(id[e.first], id[e.second]));
    }
    for (int i = 0; i < (int)tri.size(); i++)
    {
        auto x = tri[i];
        for (int k = 0; k < 3; k++)
        {
            side[minmax(x[k], x[(k + 1) % 3])].push_back(i);
        }
        P a = (P)p[x[0]], u = (P)(p[x[1]] - p[x[0]]), v = (P)(p[x[2]] - p[x[0]]);
        P c = a + (rot(u) * norm(v) - rot(v) * norm(u)) / (2 * cross(u, v));
        ans.center.push_back(c);
        ans.tri.push_back({id[x[0]], id[x[1]], id[x[2]]});
    }
    for (auto [e, f] : side)
    {
        if (f.size() == 2)
        {
            ans.vor.push_back({ans.center[f[0]], ans.center[f[1]], 0});
        }
        else if (f.size() == 1)
        {
            int u = e.first, v = e.second;
            auto x = tri[f[0]];
            bool forward = false;
            for (int k = 0; k < 3; k++)
            {
                forward |= x[k] == u && x[(k + 1) % 3] == v;
            }
            if (!forward)
            {
                swap(u, v);
            }
            P d{(Real)(p[v].y - p[u].y), (Real)(p[u].x - p[v].x)};
            ans.vor.push_back({ans.center[f[0]], d, 1});
        }
        else
        {
```

```
            P a = (P)p[e.first], b = (P)p[e.second];
            ans.vor.push_back({(a + b) / 2, rot(b - a), 2});
        }
    }
    sort(ans.edge.begin(), ans.edge.end());
    return ans;
}
```

`delaunay(p)` 要求整数坐标，返回 Delaunay 边、逆时针三角形、外心和 Voronoi 对偶；`VorEdge` 的 `type=0/1/2` 分别表示线段、射线和直线，后两者的 `b` 是方向向量。重复点只保留首次出现者；全部共线时三角形为空，Delaunay 边是相邻点链，Voronoi 输出相邻点中垂线。分治构造 $O(n \log n)$ 时间、$O(n)$ 边空间，圆内判定要求 $12D^4 < 2^{127}$，其中 D 是任意两坐标之差的最大绝对值。共圆时 Delaunay 对角线不唯一，本实现确定性选择其中一套。

```
signed main()
{
    vector<IP> p{{0, 0}, {2, 0}, {2, 2}, {0, 2}, {1, 1}};
    auto d = delaunay(p);
    assert(d.tri.size() == 4 && d.edge.size() == 8 && d.vor.size() == 8);
}
```

11.3.012 三角形四心（内心、外心、垂心、费马点）

汇总三角形内心、外心、垂心与费马点。前三者复用 11.1.012，费马点按最大角是否达到 120 度分情况。内心按三条对边长度加权；外心解两条等距线性方程；垂心使用 $H = A + B + C - 2O$；若某内角不小于 120°，费马点就是该顶点，否则连接向外构造的两个等边三角形顶点求交。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
P fermatPoint(P a, P b, P c)
{
    // a、b、c 表示非共线三角形顶点；返回到三顶点距离和最小的费马点。
    assert(sgn(cross(a, b, c)) != 0); // 调试检查，可删。
    const Real pi = acosl(-1);
    auto ang = [&](P x, P y, P z)
    {
        // x 表示角顶点，y、z 表示另外两点；返回夹角弧度。
        auto u = y - x;
        auto v = z - x;
        auto q = dot(u, v) / abs(u) / abs(v);
        return acosl(clamp(q, (Real)-1, (Real)1));
    };
    auto A = ang(a, b, c);
    auto B = ang(b, c, a);
    auto C = ang(c, a, b);
    if (sgn(A - 2 * pi / 3) >= 0)
    {
        return a;
    }
```

```
    if (sgn(B - 2 * pi / 3) >= 0)
    {
        return b;
    }
    if (sgn(C - 2 * pi / 3) >= 0)
    {
        return c;
    }
    auto x = abs(b - c) / sinl(A + pi / 3);
    auto y = abs(c - a) / sinl(B + pi / 3);
    auto z = abs(a - b) / sinl(C + pi / 3);
    return (a * x + b * y + c * z) / (x + y + z);
}
```

内心、外心、垂心调用 11.1.012 的 `triCenters`。费马点若某角不小于 120 度就是该顶点；否则先取三线坐标 $\csc(A+60^\circ):\csc(B+60^\circ):\csc(C+60^\circ)$，再分别乘对边长转换成重心坐标。$O(1)$ 时间和 $O(1)$ 额外空间。静态建包使用 11.3.001 Andrew / Graham 静态凸包，只插点且需要在线维护才考虑 11.3.002 单点插入动态凸包。最远点对使用 11.3.006 旋转卡壳、凸包直径，平移碰撞与凸多边形距离可转为 11.3.008 闵可夫斯基和。

11.3.013 凸包分层

凸包分层反复删除当前点集的全部凸包边界点。若每层重新 Andrew，最坏可达 $O(n^2\log n)$。下面先按横坐标把点放进完全二叉树，每个结点保存左右儿子上凸壳的公切桥；删除一个点后只重算其 $O(\log n)$ 个祖先，每次找新桥再花 $O(\log n)$。同时维护原点集的上凸壳与取反点集的上凸壳，就能拼出每一层的逆时针边界。

```
// 依赖 11.1.002 Point / Vector
struct UpperHull
{
    struct Node
    {
        int l = -1, r = -1, bl = -1, br = -1; // l、r 是子树最左存活点与最右存活点加一（r=-1 表示子
        ↪ 树为空），bl、br 是左右儿子上凸壳之间公切桥的端点。
    };

    int n, sz; // n 是点数，sz 是补齐到 2 的幂的叶子数。
    vector<Point<long long>> p; // p 是按坐标排序后的点。
    vector<Node> tr; // tr 是按堆式编号的线段树结点。
    vector<int> orig, pos;

    UpperHull(vector<Point<long long>> a) : n(a.size()), p(move(a))
    {
        // a 是互异整数点；建立包含全部点的动态上凸壳。
        orig.resize(n);
        iota(orig.begin(), orig.end(), 0);
        sort(orig.begin(), orig.end(), [&](int x, int y) { return p[x] < p[y]; });
        vector<Point<long long>> q(n);
        pos.resize(n);
```

```
        for (int i = 0; i < n; i++)
        {
            q[i] = p[orig[i]];
            pos[orig[i]] = i;
        }
        p.swap(q);
        sz = 1;
        while (sz < n)
        {
            sz <<= 1;
        }
        tr.resize(2 * sz);
        for (int i = 0; i < n; i++)
        {
            tr[sz + i] = {i, i + 1, i, i};
        }
        for (int i = sz - 1; i; i--)
        {
            pull(i);
        }
    }

    bool has(int x) const
    {
        // x 是线段树结点编号；返回该结点子树中是否还有未删除的点。
        return tr[x].r != -1;
    }

    void pull(int x)
    {
        // 这里的叉积还要与坐标相乘，局部扩为 __int128。
        auto cross128 = [](Point<long long> a, Point<long long> b)
        {
            return (__int128)a.x * b.y - (__int128)a.y * b.x;
        };
        if (!has(2 * x) && !has(2 * x + 1))
        {
            tr[x] = {};
            return;
        }
        if (!has(2 * x))
        {
            tr[x] = tr[2 * x + 1];
            return;
        }
        if (!has(2 * x + 1))
        {
            tr[x] = tr[2 * x];
            return;
        }
        int u = 2 * x, v = 2 * x + 1;
        long long split = p[tr[v].l].x;
```

```
        while (u < sz || v < sz)
        {
            if (u < sz && !has(2 * u))
            {
                u = 2 * u + 1;
                continue;
            }
            if (u < sz && !has(2 * u + 1))
            {
                u = 2 * u;
                continue;
            }
            if (v < sz && !has(2 * v))
            {
                v = 2 * v + 1;
                continue;
            }
            if (v < sz && !has(2 * v + 1))
            {
                v = 2 * v;
                continue;
            }
            int a = tr[u].bl, b = tr[u].br, c = tr[v].bl, d = tr[v].br;
            if (a != b && cross128(p[b] - p[a], p[c] - p[a]) > 0)
            {
                u = 2 * u;
            }
            else if (c != d && cross128(p[c] - p[b], p[d] - p[b]) > 0)
            {
                v = 2 * v + 1;
            }
            else if (a == b)
            {
                v = 2 * v;
            }
            else if (c == d)
            {
                u = 2 * u + 1;
            }
            else
            {
                __int128 x1 = cross128(p[b] - p[a], p[c] - p[a]);
                __int128 x2 = cross128(p[a] - p[b], p[d] - p[b]);
                if (x1 + x2 == 0 || x1 * p[d].x + x2 * p[c].x < (__int128)split * (x1 + x2))
                {
                    u = 2 * u + 1;
                }
                else
                {
                    v = 2 * v;
                }
            }
```

```
        }
        tr[x].l = tr[2 * x].l;
        tr[x].r = tr[2 * x + 1].r;
        tr[x].bl = tr[u].l;
        tr[x].br = tr[v].l;
    }

    void erase(int id)
    {
        // id 是原输入编号；从当前点集中删除该点。
        int x = sz + pos[id];
        tr[x] = {};
        for (x >>= 1; x; x >>= 1)
        {
            pull(x);
        }
    }

    vector<int> get() const
    {
        // 无参数；返回当前上凸壳从左到右的原输入编号。
        vector<int> ans;
        auto dfs = [&](auto &&self, int x, int l, int r) -> void
        {
            if (!has(x) || l >= r)
            {
                return;
            }
            if (x >= sz)
            {
                ans.push_back(tr[x].l);
                return;
            }
            if (!has(2 * x))
            {
                self(self, 2 * x + 1, l, r);
                return;
            }
            if (!has(2 * x + 1))
            {
                self(self, 2 * x, l, r);
                return;
            }
            if (r <= tr[x].bl)
            {
                self(self, 2 * x, l, r);
                return;
            }
            if (tr[x].br <= l)
            {
                self(self, 2 * x + 1, l, r);
                return;
```

```
            }
            self(self, 2 * x, l, tr[x].bl + 1);
            self(self, 2 * x + 1, tr[x].br, r);
        };
        if (n)
        {
            dfs(dfs, 1, 0, n);
        }
        for (auto &x : ans)
        {
            x = orig[x];
        }
        return ans;
    }
};

vector<vector<int>> convexLayers(const vector<Point<long long>> &p)
{
    // p 是互异且坐标绝对值不超过 1e9 的整数点；返回各层逆时针点编号。
    if (p.empty())
    {
        return {};
    }
    vector<Point<long long>> q;
    q.reserve(p.size());
    for (auto [x, y] : p)
    {
        assert(llabs(x) <= 1000000000 && llabs(y) <= 1000000000); // 调试检查，可删。
        q.push_back({-x, -y});
    }
    UpperHull up(p), down(q);
    vector<vector<int>> ans;
    int done = 0;
    while (done < (int)p.size())
    {
        auto a = down.get();
        auto b = up.get();
        if (a.size() == p.size() - done && b.size() == a.size())
        {
            // 剩余点全共线时两条壳相同，只保留其中一条。
        }
        else if (a.size() >= 2)
        {
            a.pop_back();
            b.pop_back();
            a.insert(a.end(), b.begin(), b.end());
            reverse(a.begin(), a.end());
        }
        ans.push_back(a);
        for (int x : a)
        {
            up.erase(x);
```

```
            down.erase(x);
            done++;
        }
    }
    return ans;
}
```

`convexLayers(p)` 返回每层原输入编号；同一凸包边上的全部共线点属于该层，点必须互异且坐标绝对值不超过 10^9。预处理 $O(n \log n)$，每次删除 $O(\log^2 n)$，总时间 $O(n \log^2 n)$、空间 $O(n)$。只求一层时直接用 11.3.001 Andrew / Graham 静态凸包，其代码更短。

下面用两层正方形和中心点检查层号；每层顶点在结果内的循环起点不作约定：

```
signed main()
{
    vector<Point<long long>> p{{0, 0}, {4, 0}, {4, 4}, {0, 4},
                               {1, 1}, {3, 1}, {3, 3}, {1, 3}, {2, 2}};
    auto lay = convexLayers(p);
    assert(lay.size() == 3);
    vector<int> bel(p.size());
    for (int i = 0; i < (int)lay.size(); i++)
    {
        for (int x : lay[i])
        {
            bel[x] = i + 1;
        }
    }
    assert(bel == vector<int>({1, 1, 1, 1, 2, 2, 2, 2, 3}));
}
```

11.3.014 点集最小 / 最大面积三角形

在同一点集中同时求不同下标三点的最小、最大双倍面积 $\left|\operatorname{cross}(b-a, c-a)\right|$。整数版扫描所有无向点对的方向：点按该方向法向投影排序，同一直线上的连续块在越过这一方向时整体反转；对一条基边，块外相邻点给出最小非零候选，排列两端给出最大候选，同方向事件必须整批处理。重复坐标或三点共线会使最小值为 0。浮点事件难以可靠判同向，因而单独保留直接枚举版本。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
optional<pair<__int128, __int128>> triAreaRng2(const vector<Point<long long>> &a)
{
    // a 是整数点集；返回不同下标三点的最小、最大双倍面积，少于三个点时返回空。
    // 坐标绝对值不超过 1E9，以保证方向排序和面积乘积可由 __int128 表示。
    // 坐标差之间的叉积必须能由 __int128 表示。
    int n = (int)a.size();
    if (n < 3)
    {
        return nullopt;
    }
```

```
    vector<Point<long long>> p = a;
    sort(p.begin(), p.end());
    auto last = unique(p.begin(), p.end());
    bool dup = last != p.end();
    p.erase(last, p.end());
    if (p.size() < 3)
    {
        return pair<__int128, __int128>{0, 0};
    }
    int m = (int)p.size();
    struct Event
    {
        int u;      // 有向差向量的起点下标。
        int v;      // 有向差向量的终点下标。
        __int128 x; // 规范到上半平面后的横坐标差。
        __int128 y; // 规范到上半平面后的纵坐标差。
    };
    vector<Event> e;
    for (int i = 0; i < m; i++)
    {
        for (int j = i + 1; j < m; j++)
        {
            __int128 x = (__int128)p[j].x - p[i].x;
            __int128 y = (__int128)p[j].y - p[i].y;
            if (y < 0 || (y == 0 && x < 0))
            {
                x = -x;
                y = -y;
            }
            e.push_back({i, j, x, y});
        }
    }
    sort(e.begin(),
         e.end(),
         [](const Event &u, const Event &v)
         {
             __int128 c = u.x * v.y - u.y * v.x;
             if (c != 0)
             {
                 return c > 0;
             }
             return pair(u.u, u.v) < pair(v.u, v.v);
         });
    vector<int> ord(m), pos(m);
    iota(ord.begin(), ord.end(), 0);
    sort(ord.begin(),
         ord.end(),
         [&](int x, int y)
         {
             return tuple(p[x].y, p[x].x, x) < tuple(p[y].y, p[y].x, y);
         });
    for (int i = 0; i < m; i++)
```

```
    {
        pos[ord[i]] = i;
    }
    auto triArea = [&](int i, int j, int k)
    {
        __int128 x1 = (__int128)p[j].x - p[i].x;
        __int128 y1 = (__int128)p[j].y - p[i].y;
        __int128 x2 = (__int128)p[k].x - p[i].x;
        __int128 y2 = (__int128)p[k].y - p[i].y;
        __int128 s = x1 * y2 - y1 * x2;
        return s < 0 ? -s : s;
    };
    __int128 mn = 0;
    __int128 mx = 0;
    bool hasMin = dup;
    vector<int> fa(m, -1);
    auto find = [&](auto &&self, int x) -> int
    {
        return fa[x] == x ? x : fa[x] = self(self, fa[x]);
    };
    for (int l = 0; l < (int)e.size();)
    {
        int r = l + 1;
        while (r < (int)e.size() && e[l].x * e[r].y == e[l].y * e[r].x)
        {
            r++;
        }
        vector<int> touched;
        for (int i = l; i < r; i++)
        {
            for (int x : {e[i].u, e[i].v})
            {
                if (fa[x] == -1)
                {
                    fa[x] = x;
                    touched.push_back(x);
                }
            }
            int x = find(find, e[i].u);
            int y = find(find, e[i].v);
            if (x != y)
            {
                fa[x] = y;
            }
        }
        map<int, vector<int>> block;
        for (int x : touched)
        {
            block[find(find, x)].push_back(x);
        }
        for (int i = l; i < r; i++)
        {
```

```
            for (int x : {ord.front(), ord.back()})
            {
                if (x != e[i].u && x != e[i].v)
                {
                    mx = max(mx, triArea(e[i].u, e[i].v, x));
                }
            }
        }
        for (auto &[root, v] : block)
        {
            int x = m;
            int y = -1;
            for (int z : v)
            {
                x = min(x, pos[z]);
                y = max(y, pos[z]);
            }
            assert(y - x + 1 == (int)v.size()); // 调试检查，可删。
            if (v.size() >= 3)
            {
                mn = 0;
                hasMin = true;
            }
            else if (!hasMin || mn != 0)
            {
                int u = v[0];
                int z = v[1];
                for (int k : {x - 1, y + 1})
                {
                    if (0 <= k && k < m)
                    {
                        __int128 cur = triArea(u, z, ord[k]);
                        if (!hasMin || cur < mn)
                        {
                            mn = cur;
                            hasMin = true;
                        }
                    }
                }
            }
            reverse(ord.begin() + x, ord.begin() + y + 1);
            for (int k = x; k <= y; k++)
            {
                pos[ord[k]] = k;
            }
        }
        for (int x : touched)
        {
            fa[x] = -1;
        }
        l = r;
    }
```

```
    assert(hasMin); // 调试检查，可删。
    return pair<__int128, __int128>{mn, mx};
}

optional<pair<Real, Real>> triAreaRng2(const vector<P> &a)
{
    // a 是浮点点集；返回不同下标三点的最小、最大双倍面积，少于三个点时返回空。
    int n = (int)a.size();
    if (n < 3)
    {
        return nullopt;
    }
    Real mn = numeric_limits<Real>::infinity();
    Real mx = 0;
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            for (int k = j + 1; k < n; k++)
            {
                Real s = fabsl(cross(a[i], a[j], a[k]));
                if (sgn(s) == 0)
                {
                    s = 0;
                }
                mn = min(mn, s);
                mx = max(mx, s);
            }
        }
    }
    return pair<Real, Real>{mn, mx};
}
```

整数重载接收 `vector<Point<long long>>` 并返回 `pair<__int128,__int128>`；坐标绝对值须不超过 10^9，少于三个下标返回 `nullopt`，重复下标坐标仍可同时选入三角形。其时间 $O(n^2 \log n)$、空间 $O(n^2)$。`Point<Real>` 重载返回 `pair<Real,Real>`，使用 $O(n^3)$ 时间和 $O(1)$ 额外空间。只求凸包最大三角形且坐标类型范围足够时可用 11.3.007 最大面积三角形、四边形；需要宽整数结果时使用本条。

11.4 圆

11.4.001 点、线、圆的位置关系

判断点与圆、直线与圆、两圆的相离、相切、相交或包含关系，为交点、切线和面积公式选择正确分支。点与圆比较距离平方和半径平方，直线与圆比较圆心到直线距离；两圆关系由圆心距与半径和、半径差比较，重合需单独状态。

```
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
struct C
{
```

```
    P o; // o 保存圆心。
    Real r; // r 保存非负半径。
};

bool in(C c, P p)
{
    // c 是圆，p 是查询点；返回 p 是否在圆内或圆上。
    assert(sgn(c.r) >= 0); // 调试检查，可删。
    return sgn(abs(p - c.o) - c.r) <= 0;
}

// 0 相离，1 外切，2 相交，3 内切，4 内含，5 重合。
int circleRel(C a, C b)
{
    // a、b 是圆；返回二者的位置关系编号。
    assert(sgn(a.r) >= 0 && sgn(b.r) >= 0); // 调试检查，可删。
    Real d = abs(a.o - b.o);
    if (!sgn(d) && !sgn(a.r - b.r))
    {
        return 5;
    }
    int x = sgn(d - a.r - b.r);
    int y = sgn(d - fabsl(a.r - b.r));
    if (x > 0)
    {
        return 0;
    }
    if (!x)
    {
        return 1;
    }
    if (y > 0)
    {
        return 2;
    }
    if (!y)
    {
        return 3;
    }
    return 4;
}
```

`C{o,r}` 保存圆心与半径；`in(c,p)` 判断点在圆内或圆上，`circleRel(a,b)` 返回编码 0 至 5，依次表示相离、外切、相交、内切、内含、重合。$O(1)$ 时间和 $O(1)$ 额外空间。圆的交点、位置关系、切线统一建立在 11.4.002 直线与圆、两圆交点 的 Circle 上。覆盖全部点且半径未知用 11.4.006 最小圆覆盖；半径固定、只最大化覆盖点数用 11.4.007 固定半径圆覆盖最多点，两者不能互换。

11.4.002 直线与圆、两圆交点

求直线与圆、两圆的全部交点。圆的基础类型和位置关系在 11.4.001 定义，本项只打印交点算法。直线圆交先把圆心投影到直线，再沿单位方向偏移弦半长；两圆交先在圆心连线上求公共弦中点，再沿垂线偏移。

```
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.1.007 直线交点、投影、对称点
// 依赖 11.4.001 点、线、圆的位置关系
vector<P> lineCircle(L l, C c)
{
    // l 是直线，c 是圆；返回按直线参数排列的交点。
    assert(sgn(norm(l.v)) != 0 && sgn(c.r) >= 0); // 调试检查，可删。
    P h = proj(l, c.o);
    Real d = norm(h - c.o);
    if (sgn(d - c.r * c.r) > 0)
    {
        return {};
    }
    Real t = sqrtl(max(Real(0), c.r * c.r - d)) / abs(l.v);
    if (!sgn(t))
    {
        return {h};
    }
    return {h - l.v * t, h + l.v * t};
}

vector<P> circleIsect(C a, C b)
{
    // a、b 是圆；返回有限的公共点。
    assert(sgn(a.r) >= 0 && sgn(b.r) >= 0); // 调试检查，可删。
    Real d = abs(b.o - a.o);
    if (sgn(d) == 0)
    {
        if (sgn(a.r) == 0 && sgn(b.r) == 0)
        {
            return {a.o};
        }
        return {};
    }
    if (sgn(d - a.r - b.r) > 0 || sgn(d - fabsl(a.r - b.r)) < 0)
    {
        return {};
    }
    Real x = (a.r * a.r - b.r * b.r + d * d) / (2 * d);
    Real h = sqrtl(max(Real(0), a.r * a.r - x * x));
    P v = (b.o - a.o) / d;
    P p = a.o + v * x;
    if (!sgn(h))
    {
        return {p};
    }
```

```
    return {p + rot(v) * h, p - rot(v) * h};
}
```

`lineCircle(l,c)` 与 `circleIsect(a,b)` 返回 0 至 2 个交点。重合的正半径圆有无穷多交点，以空结果表示并由 11.4.001 预先分类；重合的两个零半径圆只有一个公共点，返回该圆心。$O(1)$ 时间和 $O(1)$ 额外空间。圆的交点、位置关系、切线统一建立在 11.4.002 直线与圆、两圆交点 的 Circle 上。覆盖全部点且半径未知用 11.4.006 最小圆覆盖；半径固定、只最大化覆盖点数用 11.4.007 固定半径圆覆盖最多点，两者不能互换。

11.4.003 两圆相交面积

计算两圆公共部分面积。根据圆心距先处理相离与包含，再用两个扇形减三角形。两圆无交、内含时面积直接为零或小圆面积；相交时分别由余弦定理求圆心角，两段扇形面积减对应三角形面积后相加。

```
// 依赖 11.4.001 点、线、圆的位置关系
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
Real circleArea(C a, C b)
{
    // a、b 是圆；返回二者公共面积。
    assert(sgn(a.r) >= 0 && sgn(b.r) >= 0); // 调试检查，可删。
    Real d = abs(a.o - b.o);
    if (sgn(d - a.r - b.r) >= 0)
    {
        return 0;
    }
    if (sgn(d - fabsl(a.r - b.r)) <= 0)
    {
        Real r = min(a.r, b.r);
        return acosl(-1) * r * r;
    }
    Real x = acosl(clamp((a.r * a.r + d * d - b.r * b.r) / (2 * a.r * d), Real(-1), Real(1)));
    Real y = acosl(clamp((b.r * b.r + d * d - a.r * a.r) / (2 * b.r * d), Real(-1), Real(1)));
    return a.r * a.r * x + b.r * b.r * y - a.r * d * sinl(x);
}
```

调用 `circleArea(a,b)` 返回两圆公共面积；半径必须非负。$O(1)$ 时间和 $O(1)$ 额外空间。圆的交点、位置关系、切线统一建立在 11.4.002 直线与圆、两圆交点 的 Circle 上。覆盖全部点且半径未知用 11.4.006 最小圆覆盖；半径固定、只最大化覆盖点数用 11.4.007 固定半径圆覆盖最多点，两者不能互换。

11.4.004 圆的切线与两圆公切线

求外点到圆的切点以及两圆公切线。重合圆有无穷多公切线，必须单独处理。点到圆切线由直角三角形求切点方向；两圆公切线可统一处理外切和内切的有符号半径，再按圆心距与半径差判断切线条数。

```
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.4.001 点、线、圆的位置关系
vector<P> tangents(C c, P p)
```

```
{
    // c 是圆，p 是查询点；返回不同的切点。
    assert(sgn(c.r) >= 0); // 调试检查，可删。
    P v = p - c.o;
    Real d = norm(v);
    Real z = d - c.r * c.r;
    if (sgn(d) == 0)
    {
        return {};
    }
    if (sgn(c.r) == 0)
    {
        return {c.o};
    }
    if (sgn(z) < 0)
    {
        return {};
    }
    P q = c.o + v * (c.r * c.r / d);
    if (!sgn(z))
    {
        return {q};
    }
    P w = rot(v) * (c.r * sqrtl(z) / d);
    return {q + w, q - w};
}

// 返回两圆公切线在两圆上的切点对；重合圆有无穷多条，返回空。
vector<pair<P, P>> tangents(C a, C b)
{
    // a、b 是圆；返回每条不同公切线上的两个切点。
    assert(sgn(a.r) >= 0 && sgn(b.r) >= 0); // 调试检查，可删。
    if (sgn(a.r) == 0)
    {
        vector<pair<P, P>> ans;
        for (P q : tangents(b, a.o))
        {
            ans.push_back({a.o, q});
        }
        return ans;
    }
    if (sgn(b.r) == 0)
    {
        vector<pair<P, P>> ans;
        for (P q : tangents(a, b.o))
        {
            ans.push_back({q, b.o});
        }
        return ans;
    }
    P d = b.o - a.o;
    Real z = norm(d);
```

```
    if (!sgn(z))
    {
        return {};
    }
    vector<pair<P, P>> ans;
    for (int s : {-1, 1})
    {
        Real r = a.r - s * b.r;
        Real h2 = z - r * r;
        if (sgn(h2) < 0)
        {
            continue;
        }
        for (int t : {-1, 1})
        {
            if (!sgn(h2) && t == 1)
            {
                continue;
            }
            P v = (d * r + rot(d) * (sqrtl(max(Real(0), h2)) * t)) / z;
            ans.push_back({a.o + v * a.r, b.o + v * (s * b.r)});
        }
    }
    return ans;
}
```

调用 `tangents(c,p)` 或 `tangents(a,b)`，分别返回不同切点和不同公切线上的接触点对；重合的正半径圆返回空表示无穷多条。零半径圆按点退化：外部查询点只返回圆心一次，一个点圆与普通圆的公切线转成点到圆的切线，两个不同点圆返回连接二点的一组接触点。$O(1)$ 时间和 $O(1)$ 额外空间。圆的交点、位置关系、切线统一建立在 11.4.002 直线与圆、两圆交点 的 Circle 上。覆盖全部点且半径未知用 11.4.006 最小圆覆盖；半径固定、只最大化覆盖点数用 11.4.007 固定半径圆覆盖最多点，两者不能互换。

11.4.005 三点外接圆、三角形内切圆

构造直径圆、三点外接圆和非退化三角形的内切圆。外接圆圆心由两条垂直平分线确定；三点共线时 `circle(a,b,c)` 延续旧接口，返回覆盖三点的最小直径圆。内心按对边长加权，若双倍面积为 $\left|\operatorname{cross}(b-a, c-a)\right|$、周长为 p，则内切圆半径为该双倍面积除以 p；共线或重复点没有唯一内切圆。

```
// 依赖 11.4.001 点、线、圆的位置关系
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
C circle(P a, P b)
{
    // a、b 是直径端点；返回对应的圆。
    P o = (a + b) / 2;
    return {o, abs(a - o)};
}

C circle(P a, P b, P c)
{
```

```
    // a、b、c 是三个点；共线时返回覆盖三点的最小圆。
    P u = b - a;
    P v = c - a;
    Real d = 2 * cross(u, v);
    if (sgn(d) == 0)
    {
        if (norm(a - b) >= norm(a - c) && norm(a - b) >= norm(b - c))
        {
            return circle(a, b);
        }
        if (norm(a - c) >= norm(b - c))
        {
            return circle(a, c);
        }
        return circle(b, c);
    }
    P o = a + P{norm(u) * v.y - norm(v) * u.y, u.x * norm(v) - v.x * norm(u)} / d;
    return {o, abs(a - o)};
}

optional<C> incircle(P a, P b, P c)
{
    // a、b、c 是三角形顶点；返回内切圆，三点共线或有重复点时返回空。
    Real s = fabsl(cross(b - a, c - a));
    if (sgn(s) == 0)
    {
        return nullopt;
    }
    Real x = abs(b - c);
    Real y = abs(c - a);
    Real z = abs(a - b);
    Real sum = x + y + z;
    P o = (a * x + b * y + c * z) / sum;
    return C{o, s / sum};
}
```

`circle(a,b)` 返回以两点为直径端点的圆，`circle(a,b,c)` 返回三点外接圆或共线退化下的最小覆盖圆；`incircle(a,b,c)` 返回内切圆，退化三角形返回 `nullopt`。各接口均为 $O(1)$ 时间和 $O(1)$ 额外空间。覆盖任意点集使用 11.4.006 最小圆覆盖，不要把共线退化约定当作一般最小圆覆盖算法。

11.4.006 最小圆覆盖

求覆盖全部平面点的最小圆。随机增量算法在固定边界点后退化为两点或三点定圆。若当前最小圆不包含新点 p，则新圆边界必须经过 p；继续扫描旧点，若仍不包含 q，则边界还必须经过 q；第三层同理用三点外接圆。随机顺序使期望总工作量为线性。

```
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.4.001 点、线、圆的位置关系
// 依赖 11.4.005 三点外接圆、三角形内切圆
C minCircle(vector<P> a, int seed = 0)
```

```
{
    // a 是点集，seed 是打乱种子；返回最小覆盖圆。
    mt19937 rng(seed);
    shuffle(a.begin(), a.end(), rng);
    C c{{0, 0}, 0};
    for (int i = 0; i < (int)a.size(); i++)
    {
        if (!in(c, a[i]))
        {
            c = {a[i], 0};
            for (int j = 0; j < i; j++)
            {
                if (!in(c, a[j]))
                {
                    c = circle(a[i], a[j]);
                    for (int k = 0; k < j; k++)
                    {
                        if (!in(c, a[k]))
                        {
                            c = circle(a[i], a[j], a[k]);
                        }
                    }
                }
            }
        }
    }
    return c;
}

string minCircleBd(const vector<Point<long long>> &p,
                                        unsigned long long seed = 0x9E3779B97F4A7C15ULL)
{
    // p 是整数点集，seed 是确定性洗牌种子；返回各点是否位于最小包围圆圆周的 0/1 串。
    struct Support
    {
        int size = 0; // size 是确定当前圆的边界点数（0 到 3）。
        array<Point<long long>, 3> p; // p[0..size) 是这些边界点。
    };
    auto cross128 = [](Point<long long> a, Point<long long> b)
    {
        return (__int128)a.x * b.y - (__int128)a.y * b.x;
    };
    auto norm128 = [](Point<long long> a)
    {
        return (__int128)a.x * a.x + (__int128)a.y * a.y;
    };
    auto diameter = [&](Point<long long> a, Point<long long> b, Point<long long> x)
    {
        Point<long long> u{2 * x.x - a.x - b.x, 2 * x.y - a.y - b.y};
        __int128 lhs = norm128(u), rhs = norm128(a - b);
        return lhs < rhs ? 1 : lhs == rhs ? 0 : -1;
    };
```

```
    auto contain = [&](const Support &s, Point<long long> x)
    {
        if (s.size == 0) return -1;
        if (s.size == 1) return x == s.p[0] ? 0 : -1;
        if (s.size == 2) return diameter(s.p[0], s.p[1], x);
        auto a = s.p[0] - x, b = s.p[1] - x, c = s.p[2] - x;
        __int128 det = norm128(a) * cross128(b, c) +
                       norm128(b) * cross128(c, a) +
                       norm128(c) * cross128(a, b);
        __int128 ori = cross128(s.p[1] - s.p[0], s.p[2] - s.p[0]);
        det = ori > 0 ? det : -det;
        return det > 0 ? 1 : det == 0 ? 0 : -1;
    };
    auto reduce = [&](Support &s)
    {
        if (s.size != 3 || cross128(s.p[1] - s.p[0], s.p[2] - s.p[0]) != 0) return;
        __int128 d01 = norm128(s.p[0] - s.p[1]);
        __int128 d12 = norm128(s.p[1] - s.p[2]);
        __int128 d02 = norm128(s.p[0] - s.p[2]);
        if (d12 >= d01 && d12 >= d02)
        {
            s.p[0] = s.p[1];
            s.p[1] = s.p[2];
        }
        else if (d02 >= d01)
        {
            s.p[1] = s.p[2];
        }
        s.size = 2;
    };
    vector<Point<long long>> q = p;
    mt19937_64 rng(seed);
    shuffle(q.begin(), q.end(), rng);
    Support s;
    for (int i = 0; i < (int)q.size(); i++)
    {
        if (contain(s, q[i]) >= 0) continue;
        s = {1, {q[i], {}, {}}};
        for (int j = 0; j < i; j++)
        {
            if (contain(s, q[j]) >= 0) continue;
            s = {2, {q[i], q[j], {}}};
            for (int k = 0; k < j; k++)
            {
                if (contain(s, q[k]) >= 0) continue;
                s = {3, {q[i], q[j], q[k]}};
                reduce(s);
            }
        }
    }
    string ans(p.size(), '0');
    for (int i = 0; i < (int)p.size(); i++)
```

```
    {
        ans[i] = contain(s, p[i]) == 0 ? '1' : '0';
    }
    return ans;
}
```

调用 `minCircle(a,seed)` 返回覆盖 `a` 中全部点的浮点最小圆；空点集返回圆心 (0,0)、半径 0。`minCircleBd(p,seed)` 用 2/3 个整数支撑点和 `__int128` 行列式精确判断圆内、圆外和圆周，返回与原输入同序的 0/1 串；它避免先求浮点圆心后用误差比较，适用于要求精确输出哪些整数点在最小圆圆周上的题目。两个接口在随机排列下期望 $O(n)$、最坏 $O(n^3)$ 时间，$O(n)$ 空间；整数版要求所有坐标差及其两倍仍可由 `long long` 表示，相关平方乘叉积可由 `__int128` 表示。圆的交点、位置关系、切线统一建立在 11.4.002 直线与圆、两圆交点 的 Circle 上。半径固定、只最大化覆盖点数用 11.4.007 固定半径圆覆盖最多点，两者不能互换。

下面用单点、共线点和正方形检查零半径、直径圆与外接圆分支：

```
signed main()
{
    C a = minCircle({P{3, -2}});
    assert(abs(a.o - P{3, -2}) < eps && fabsl(a.r) < eps);

    C b = minCircle({P{-2, 0}, P{0, 0}, P{4, 0}});
    assert(abs(b.o - P{1, 0}) < eps && fabsl(b.r - 3) < eps);

    C c = minCircle({P{-1, -1}, P{-1, 1}, P{1, 1}, P{1, -1}});
    assert(abs(c.o) < eps && fabsl(c.r - sqrtl(2)) < eps);
}
```

11.4.007 固定半径圆覆盖最多点

给定半径，求一个圆最多覆盖多少输入点。枚举一对点确定的候选圆心方向，再做角区间扫描。固定一个点在覆盖圆边界上，其他可覆盖点给出圆心极角区间；对跨越 `2pi` 的区间拆分后做角度扫描，最大重叠数加固定点即答案。

```
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
int fixedCover(const vector<P> &a, Real r)
{
    // a 是点集，r 是固定半径；返回最多覆盖点数。
    assert(sgn(r) >= 0); // 调试检查，可删。
    int ans = a.empty() ? 0 : 1;
    const Real pi = acosl(-1);
    for (int i = 0; i < (int)a.size(); i++)
    {
        vector<pair<Real, int>> e;
        int same = 1;
        for (int j = 0; j < (int)a.size(); j++)
        {
            if (i == j)
            {
```

```
                continue;
            }
            Real d = abs(a[j] - a[i]);
            if (!sgn(d))
            {
                same++;
                continue;
            }
            if (sgn(d - 2 * r) > 0)
            {
                continue;
            }
            Real x = atan2l(a[j].y - a[i].y, a[j].x - a[i].x);
            Real y = acosl(clamp(d / (2 * r), Real(-1), Real(1)));
            Real l = x - y;
            Real h = x + y;
            while (l < 0)
            {
                l += 2 * pi;
                h += 2 * pi;
            }
            while (l >= 2 * pi)
            {
                l -= 2 * pi;
                h -= 2 * pi;
            }
            if (h <= 2 * pi)
            {
                e.push_back({l, 1});
                e.push_back({h, -1});
            }
            else
            {
                e.push_back({l, 1});
                e.push_back({2 * pi, -1});
                e.push_back({0, 1});
                e.push_back({h - 2 * pi, -1});
            }
        }
        sort(e.begin(),
             e.end(),
             [](auto x, auto y)
             {
                 return x.first != y.first ? x.first < y.first : x.second > y.second;
             });
        ans = max(ans, same);
        int cur = same;
        for (auto [x, v] : e)
        {
            cur += v;
            ans = max(ans, cur);
        }
```

```
    }
    return ans;
}
```

调用 `fixedCover(p,r)`；`r` 必须非负，返回某个半径为 `r` 的圆最多能覆盖的输入点数。$O(n^2 \log n)$ 时间、$O(n)$ 空间。圆的交点、位置关系、切线统一建立在 11.4.002 直线与圆、两圆交点 的 Circle 上。覆盖全部点且半径未知用 11.4.006 最小圆覆盖；半径固定、只最大化覆盖点数用 11.4.007 固定半径圆覆盖最多点，两者不能互换。

11.4.008 圆反演与阿波罗尼斯圆

以圆心 O、半径 $R > 0$ 的圆做反演时，点 $P \neq O$ 的像满足 $\overrightarrow{OP'} = \frac{R^2}{|OP|^2}\overrightarrow{OP}$。过 O 的直线保持为直线，不过 O 的直线变为过 O 的圆；过 O 的正半径圆变为直线，其余圆仍为圆。实现直接使用解析公式，并用 `variant<Line<Real>,C>` 表达对象类型变化；不同定点 a, b 的阿波罗尼斯轨迹约定满足 $|X-a| = k|X-b|$。

```
// 依赖 11.1.005 Line / Ray / Segment
// 依赖 11.4.001 点、线、圆的位置关系
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
using InvertObj = variant<Line<Real>, C>;

optional<P> invert(P p, C c)
{
    // p 是待反演点，c 是正半径反演圆；返回像点，圆心没有有限像。
    assert(sgn(c.r) > 0); // 调试检查，可删。
    P v = p - c.o;
    if (sgn(norm(v)) == 0)
    {
        return nullopt;
    }
    return c.o + v * (c.r * c.r / norm(v));
}

InvertObj invertLine(Line<Real> l, C c)
{
    // l 是待反演直线，c 是正半径反演圆；返回像直线或像圆。
    assert(!eq(l.a, l.b)); // 调试检查，可删。
    assert(sgn(c.r) > 0);  // 调试检查，可删。
    P v = l.b - l.a;
    P h = l.a + v * (dot(c.o - l.a, v) / norm(v));
    P d = h - c.o;
    if (sgn(norm(d)) == 0)
    {
        return Line<Real>{c.o, c.o + v};
    }
    P q = c.o + d * (c.r * c.r / norm(d));
    return C{(c.o + q) / 2, abs(q - c.o) / 2};
}

```

```
optional<InvertObj> invertCircle(C x, C c)
{
    // x 是待反演圆，c 是正半径反演圆；返回像直线或像圆，仅反演中心这个点圆返回空。
    assert(sgn(x.r) >= 0); // 调试检查，可删。
    assert(sgn(c.r) > 0);  // 调试检查，可删。
    P v = x.o - c.o;
    Real d2 = norm(v);
    if (sgn(x.r) == 0 && sgn(d2) == 0)
    {
        return nullopt;
    }
    Real k = c.r * c.r;
    Real z = d2 - x.r * x.r;
    if (sgn(z) == 0)
    {
        P p = c.o + v * (k / (2 * d2));
        return InvertObj{Line<Real>{p, p + rot(v)}};
    }
    return InvertObj{C{c.o + v * (k / z), k * x.r / fabsl(z)}};
}

optional<C> apollonius(P a, P b, Real k)
{
    // a、b 是不同定点，k 是到 a 的距离除以到 b 的距离；返回轨迹圆，k=1 的直线轨迹返回空。
    assert(!eq(a, b) && sgn(k) >= 0); // 调试检查，可删。
    if (sgn(k - 1) == 0)
    {
        return nullopt;
    }
    Real z = 1 - k * k;
    P o = (a - b * (k * k)) / z;
    return C{o, abs(a - b) * fabsl(k / z)};
}
```

`invert(p,c)`、`invertLine(l,c)` 和 `invertCircle(x,c)` 分别反演点、直线和圆，反演圆半径必须为正。反演中心没有有限像；位于反演中心的零半径源圆返回 `nullopt`，其余点圆以零半径像圆表示。`apollonius(a,b,k)` 要求 $a \neq b$、$k \geq 0$，并按 $|X-a| = k|X-b|$ 定义比值；$k = 1$ 时轨迹为垂直平分线，因返回类型只表示圆而返回 `nullopt`。所有接口均为 $O(1)$ 时间和 $O(1)$ 额外空间。

11.4.009 圆并与多圆覆盖层面积

计算非负半径圆集的圆并面积，或同时求“至少被 k 个圆覆盖”的全部层面积。对每个正半径圆，把其他圆在其圆周上覆盖的部分转成角区间并扫描覆盖数；覆盖数固定的一段圆弧通过 Green 公式贡献边界积分。完全相同的圆按下标把各自整圆边界分配到相邻覆盖层，同心内含、内外切和零半径圆都不产生错误角事件。

```
// 依赖 11.4.001 点、线、圆的位置关系
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
Real circleUnion(const vector<C> &a)
```

```
{
    // a 是非负半径圆集；返回至少被一个圆覆盖的面积。
    const Real pi = acosl(-1);
    Real ans = 0;
    for (int i = 0; i < (int)a.size(); i++)
    {
        assert(sgn(a[i].r) >= 0); // 调试检查，可删。
        vector<pair<Real, int>> e;
        bool cover = false;
        for (int j = 0; j < (int)a.size(); j++)
        {
            if (i == j)
            {
                continue;
            }
            Real d = abs(a[j].o - a[i].o);
            if (sgn(d + a[i].r - a[j].r) <= 0 && (sgn(a[i].r - a[j].r) < 0 || i > j))
            {
                cover = true;
                break;
            }
            if (sgn(d - a[i].r - a[j].r) >= 0 || sgn(d - fabsl(a[i].r - a[j].r)) <= 0)
            {
                continue;
            }
            Real x = atan2l(a[j].o.y - a[i].o.y, a[j].o.x - a[i].o.x);
            Real y = acosl(clamp((d * d + a[i].r * a[i].r - a[j].r * a[j].r) / (2 * d *
↪ a[i].r), Real(-1), Real(1)));
            Real l = x - y;
            Real r = x + y;
            while (l < 0)
            {
                l += 2 * pi;
                r += 2 * pi;
            }
            while (l >= 2 * pi)
            {
                l -= 2 * pi;
                r -= 2 * pi;
            }
            e.push_back({l, 1});
            e.push_back({min(r, 2 * pi), -1});
            if (r > 2 * pi)
            {
                e.push_back({0, 1});
                e.push_back({r - 2 * pi, -1});
            }
        }
        if (cover)
        {
            continue;
        }
```

```
        e.push_back({0, 0});
        e.push_back({2 * pi, 0});
        sort(e.begin(),
             e.end(),
             [](auto x, auto y)
             {
                 return x.first != y.first ? x.first < y.first : x.second > y.second;
             });
        int cnt = 0;
        Real last = 0;
        for (auto [x, v] : e)
        {
            if (!cnt)
            {
                P p = a[i].o + P{cosl(last), sinl(last)} * a[i].r;
                P q = a[i].o + P{cosl(x), sinl(x)} * a[i].r;
                Real t = x - last;
                ans += cross(p, q) / 2 + a[i].r * a[i].r * (t - sinl(t)) / 2;
            }
            cnt += v;
            last = x;
        }
    }
    return ans;
}

vector<Real> circleCov(const vector<C> &a)
{
    // a 是非负半径圆集；返回各覆盖层面积，ans[k-1] 表示至少被 k 个圆覆盖的面积。
    int n = (int)a.size();
    vector<Real> ans(n);
    const Real pi = acosl(-1);
    P shift = n ? a[0].o : P{};
    for (int i = 0; i < n; i++)
    {
        assert(sgn(a[i].r) >= 0); // 调试检查，可删。
        if (sgn(a[i].r) == 0)
        {
            continue;
        }
        vector<pair<Real, int>> e;
        int cnt = 0;
        for (int j = 0; j < n; j++)
        {
            assert(sgn(a[j].r) >= 0); // 调试检查，可删。
            if (i == j || sgn(a[j].r) == 0)
            {
                continue;
            }
            Real d = abs(a[j].o - a[i].o);
            if (eq(a[i].o, a[j].o) && eq(a[i].r, a[j].r))
            {
```

```
                cnt += j < i;
                continue;
            }
            if (sgn(d + a[i].r - a[j].r) <= 0)
            {
                cnt++;
                continue;
            }
            if (sgn(d - a[i].r - a[j].r) >= 0 || sgn(d - fabsl(a[i].r - a[j].r)) <= 0)
            {
                continue;
            }
            Real mid = atan2l(a[j].o.y - a[i].o.y, a[j].o.x - a[i].o.x);
            if (mid < 0)
            {
                mid += 2 * pi;
            }
            Real ang = acosl(clamp((d * d + a[i].r * a[i].r - a[j].r * a[j].r) / (2 * d *
↪  a[i].r), Real(-1), Real(1)));
            Real l = mid - ang;
            Real r = mid + ang;
            while (l < 0)
            {
                l += 2 * pi;
            }
            while (r >= 2 * pi)
            {
                r -= 2 * pi;
            }
            if (l < r)
            {
                e.push_back({l, 1});
                e.push_back({r, -1});
            }
            else
            {
                cnt++;
                e.push_back({r, -1});
                e.push_back({l, 1});
            }
        }
        sort(e.begin(),
             e.end(),
             [](auto x, auto y)
             {
                 return x.first != y.first ? x.first < y.first : x.second < y.second;
             });
        auto add = [&](Real l, Real r)
        {
            // l、r 是当前圆弧的起止角；把该弧的格林积分加入当前覆盖层。
            if (l == r)
            {
```

```
                return;
            }
            assert(0 <= cnt && cnt < n); // 调试检查，可删。
            P o = a[i].o - shift;
            Real sum =
                a[i].r * a[i].r * (r - l) + o.x * a[i].r * (sinl(r) - sinl(l)) - o.y * a[i].r *
↪ (cosl(r) - cosl(l));
            ans[cnt] += sum / 2;
        };
        Real last = 0;
        for (int j = 0; j < (int)e.size();)
        {
            int k = j;
            int delta = 0;
            Real x = e[j].first;
            add(last, x);
            while (k < (int)e.size() && e[k].first == x)
            {
                delta += e[k].second;
                k++;
            }
            cnt += delta;
            last = x;
            j = k;
        }
        add(last, 2 * pi);
    }
    for (Real &x : ans)
    {
        if (sgn(x) == 0)
        {
            x = 0;
        }
    }
    return ans;
}
```

`circleUnion(a)` 是只求第一层的短接口；`circleCov(a)` 返回长度为圆数的数组，`ans[k-1]` 表示至少覆盖 k 次的面积。输入圆半径均须非负，零半径圆不贡献面积。两者都使用 `Real` 数值计算，结果允许浮点误差；n 个圆的时间 $O(n^2 \log n)$，逐圆复用事件数组时额外空间 $O(n)$。固定半径只求最多覆盖点数使用 11.4.007 固定半径圆覆盖最多点。

11.4.010 最近圆对

求两圆边界之间的最小间隙，并在圆集合中找最近的一对。当前版本为确定性枚举基准，直接枚举全部圆对；距离定义是圆周间距还是圆心距必须在接口固定。若需更低复杂度，须另行证明按圆心分治时中线带的筛选条件，不能把点对最近的常数邻居结论直接套到半径不同的圆上。

```cpp
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.4.001 点、线、圆的位置关系
Real circleGap(C a, C b)
{
    // a、b 是圆；返回两条圆周之间的最短距离。
    assert(sgn(a.r) >= 0 && sgn(b.r) >= 0); // 调试检查，可删。
    Real d = abs(a.o - b.o);
    return max({Real(0), d - a.r - b.r, fabsl(a.r - b.r) - d});
}

optional<tuple<Real, int, int>> nearCircles(const vector<C> &a)
{
    // a 是圆集；返回最短圆周距离及对应下标。
    if (a.size() < 2)
    {
        return nullopt;
    }
    tuple<Real, int, int> ans{numeric_limits<Real>::infinity(), -1, -1};
    for (int i = 0; i < (int)a.size(); i++)
    {
        for (int j = i + 1; j < (int)a.size(); j++)
        {
            if (circleGap(a[i], a[j]) < get<0>(ans))
            {
                ans = {circleGap(a[i], a[j]), i, j};
            }
        }
    }
    return ans;
}
```

`circleGap(a,b)` 返回两条圆周间的非负最短距离；两圆相交或相切时为 0，一圆严格位于另一圆内时为半径差减圆心距。`nearCircles(a)` 返回最小圆周距离及圆下标，少于两个圆时返回 `nullopt`。$O(n^2)$ 时间、$O(1)$ 额外空间。圆的交点、位置关系、切线统一建立在 11.4.002 直线与圆、两圆交点 的 Circle 上。覆盖全部点且半径未知用 11.4.006 最小圆覆盖；半径固定、只最大化覆盖点数用 11.4.007 固定半径圆覆盖最多点，两者不能互换。

11.4.011 圆弧表示与圆弧重心

表示圆弧并计算均匀弧线的重心。弧长趋近零时应退化到端点，整圆弧需单独约定方向。圆弧由圆心、半径和有向起止角表示；弧长重心位于角平分方向，距圆心为 `r*sin(theta/2)/(theta/2)`，整圆极限回到圆心。

```cpp
// 依赖 11.4.001 点、线、圆的位置关系
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
P arcCentroid(C c, Real l, Real r)
{
    // c 是圆，l、r 是有向起止角；返回均匀弧线的重心。
    assert(sgn(c.r) >= 0); // 调试检查，可删。
```

```
    Real t = r - l;
    if (sgn(t) == 0)
    {
        return c.o + P{cosl(l), sinl(l)} * c.r;
    }
    Real z = 2 * c.r * sinl(t / 2) / t;
    return c.o + P{cosl((l + r) / 2), sinl((l + r) / 2)} * z;
}
```

调用 `arcCentroid(c,a,b)`，a、b 是均匀有向弧线的起止角；跨越 2π 时先按所需方向规范角度。零长度弧返回对应端点，整圆返回圆心。$O(1)$ 时间和 $O(1)$ 额外空间。圆的交点、位置关系、切线统一建立在 11.4.002 直线与圆、两圆交点 的 Circle 上。覆盖全部点且半径未知用 11.4.006 最小圆覆盖；半径固定、只最大化覆盖点数用 11.4.007 固定半径圆覆盖最多点，两者不能互换。

11.4.012 圆上整点

枚举圆心在整点、半径平方为整数的圆周整点。平移到圆心后问题变为求全部整数解 $x^2+y^2=n$；本条与 7.5.007 是同一知识在几何章的完整重复。枚举 $0 \le x \le \lfloor\sqrt{n}\rfloor$，检查 $n-x^2$ 是否为完全平方数 y^2；每组非负解通过交换坐标和改变符号生成全部象限解，最后用集合去重。若原圆心为整点 (a,b)，把结果统一平移为 $(a+x,b+y)$。

```
vector<pair<long long, long long>> circlePoints(long long n)
{
    // n 是非负半径平方；返回全部整数解。
    assert(n >= 0); // 调试检查，可删。
    set<pair<long long, long long>> s;
    for (long long x = 0; x == 0 || x <= n / x; x++)
    {
        long long y2 = n - x * x;
        long long y = sqrtl(y2);
        while (y + 1 <= y2 / (y + 1))
        {
            y++;
        }
        while (y > 0 && y > y2 / y)
        {
            y--;
        }
        if (y * y != y2)
        {
            continue;
        }
        for (int sx : {-1, 1})
        {
            for (int sy : {-1, 1})
            {
                s.insert({sx * x, sy * y});
                s.insert({sx * y, sy * x});
            }
```

```
        }
    }
    return {s.begin(), s.end()};
}
```

调用 `circlePoints(n)` 枚举 $x^2+y^2=n$ 的全部整数解。参数要求 `n>=0`；返回结果已去重并按二元组字典序排列。圆心为整点 `(a,b)` 时，再把每个结果平移 `(a,b)`。枚举 $O(\sqrt{n}\log k)$ 时间、$O(k)$ 空间，其中 k 为圆周整点数。圆的交点、位置关系、切线统一建立在 11.4.002 直线与圆、两圆交点 的 Circle 上。覆盖全部点且半径未知用 11.4.006 最小圆覆盖；半径固定、只最大化覆盖点数用 11.4.007 固定半径圆覆盖最多点，两者不能互换。可据此手算自查：`n=25` 时应返回 12 个点：坐标绝对值对为 `(0,5)`、`(3,4)` 及其交换和全部符号组合；`n=3` 返回空集，`n=0` 只返回 `(0,0)`。

11.5 距离、扫描线与特殊坐标

11.5.001 平面最近点对

用分治求平面最近点对，同时返回原下标。按 x 分治、按 y 合并，并只检查中线附近常数个候选点。按 x 坐标分成两半，递归求两侧答案后按 y 合并。跨中线候选只需保留横向距离小于当前答案的点；平面装箱性质保证每个点只向后检查常数个点。

```
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
template <class T> struct ClosestPair
{
    struct Q
    {
        Point<T> p; // p 保存二维点。
        int id; // id 保存原下标。
    };

    vector<Q> a; // a 保存递归中按 x 或 y 排序的点。
    vector<Q> b; // b 保存归并缓冲区。
    T ans; // ans 保存当前最小距离平方。
    pair<int, int> ids; // ids 保存对应的两个原下标。

    void upd(Q x, Q y)
    {
        // x、y 是两个带原下标的点；尝试更新答案。
        T d = norm(x.p - y.p);
        if (d < ans)
        {
            ans = d;
            ids = {x.id, y.id};
        }
    }

    void solve(int l, int r)
    {
        // 递归求 [l,r) 的最近点对，并把该段按 y 坐标归并。
```

```
        if (r - l <= 3)
        {
            for (int i = l; i < r; i++)
            {
                for (int j = l; j < i; j++)
                {
                    upd(a[i], a[j]);
                }
            }
            sort(a.begin() + l,
                 a.begin() + r,
                 [](Q x, Q y)
                 {
                     return x.p.y < y.p.y;
                 });
            return;
        }
        int m = (l + r) / 2;
        T x = a[m].p.x;
        solve(l, m);
        solve(m, r);
        merge(a.begin() + l,
              a.begin() + m,
              a.begin() + m,
              a.begin() + r,
              b.begin() + l,
              [](Q x, Q y)
              {
                  return x.p.y < y.p.y;
              });
        copy(b.begin() + l, b.begin() + r, a.begin() + l);
        vector<Q> s;
        for (int i = l; i < r; i++)
        {
            if ((a[i].p.x - x) * (a[i].p.x - x) > ans)
            {
                continue;
            }
            for (int j = (int)s.size() - 1; j >= 0; j--)
            {
                if ((a[i].p.y - s[j].p.y) * (a[i].p.y - s[j].p.y) > ans)
                {
                    break;
                }
                upd(a[i], s[j]);
            }
            s.push_back(a[i]);
        }
    }

    optional<pair<T, pair<int, int>>> work(const vector<Point<T>> &p)
    {
```

```
        // p 是输入点集；返回最小距离平方与一对原下标。
        if (p.size() < 2)
        {
            return nullopt;
        }
        a.resize(p.size());
        b.resize(p.size());
        for (int i = 0; i < (int)p.size(); i++)
        {
            a[i] = {p[i], i};
        }
        sort(a.begin(),
             a.end(),
             [](Q x, Q y)
             {
                 return x.p.x != y.p.x ? x.p.x < y.p.x : x.p.y < y.p.y;
             });
        for (int i = 1; i < (int)a.size(); i++)
        {
            if (a[i - 1].p == a[i].p)
            {
                return pair<T, pair<int, int>>{0, {a[i - 1].id, a[i].id}};
            }
        }
        ans = norm(a[0].p - a[1].p);
        ids = {a[0].id, a[1].id};
        solve(0, (int)a.size());
        return pair<T, pair<int, int>>{ans, ids};
    }
};
```

构造 `ClosestPair<T> q` 后调用 `q.work(points)`；少于两个点返回空，否则返回类型为 `T` 的距离平方和一对原下标。整数点全程精确，`Point<Real>` 返回浮点距离平方；调用者须保证坐标差及平方和能由 `T` 表示。排序后先识别重复点并直接返回 0，避免零距离条带退化。时间 $O(n \log n)$，空间 $O(n)$；曼哈顿度量见 11.5.002 最远曼哈顿距离。

下面的固定样例同时核对距离和原下标；重复点应返回原下标 0、2，跨分治最近对的下标对应距离必须为 1。

```
signed main()
{
    ClosestPair<Real> q;
    vector<P> a{{0, 0}, {4, 0}, {0, 0}, {2, 2}};
    auto x = q.work(a);
    assert((x && x->first == 0 && x->second == pair<int, int>{0, 2}));
    vector<P> b{{-3, 0}, {-1, 0}, {0, 0}, {4, 0}};
    auto y = q.work(b);
    assert(y && eq(y->first, norm(b[y->second.first] - b[y->second.second])));
    assert(eq(y->first, 1));
    cout << "OK\n";
}
```

11.5.002 最远曼哈顿距离

求点集中最远曼哈顿距离。二维时维护 x+y 与 x-y 的极值即可线性完成。曼哈顿最远距离可枚举每个坐标符号组合，把点映成线性式；任意点对距离等于某个符号下两映射值之差，二维只需四种符号。

```cpp
// 依赖 11.1.002 Point / Vector
template <class T> T farManhattan(const vector<Point<T>> &a)
{
    // a 是点集；返回最远点对的曼哈顿距离。
    if (a.empty())
    {
        return 0;
    }
    T lo1 = a[0].x + a[0].y, hi1 = lo1;
    T lo2 = a[0].x - a[0].y, hi2 = lo2;
    for (auto p : a)
    {
        T u = p.x + p.y, v = p.x - p.y;
        lo1 = min(lo1, u);
        hi1 = max(hi1, u);
        lo2 = min(lo2, v);
        hi2 = max(hi2, v);
    }
    return max(hi1 - lo1, hi2 - lo2);
}
```

调用 `farManhattan(points)` 返回与坐标同类型的最远曼哈顿距离。整数版本要求每个 $x+y$、$x-y$ 及最终距离都能由 `T` 表示；坐标接近完整 64 位边界时，在题目代码中显式改用更宽坐标。空集和单点集返回 0；$O(n)$ 时间、$O(1)$ 额外空间。欧氏最近点对见 11.5.001 平面最近点对。

11.5.003 曼哈顿距离与切比雪夫距离转换

在曼哈顿距离与切比雪夫距离之间转换坐标，便于把菱形范围变成轴对齐正方形。二维变换 `u=x+y,v=x-y` 把曼哈顿距离变成切比雪夫距离；逆变换要除以二，因此整数格上的奇偶性必须一致。

```cpp
// 依赖 11.1.002 Point / Vector
template <class T> T manhattan(Point<T> a, Point<T> b)
{
    // a、b 是两个点；返回曼哈顿距离。
    T x = a.x - b.x, y = a.y - b.y;
    return (x < 0 ? -x : x) + (y < 0 ? -y : y);
}

template <class T> T chebyshev(Point<T> a, Point<T> b)
{
    // a、b 是两个点；返回切比雪夫距离。
    T x = a.x - b.x, y = a.y - b.y;
    x = x < 0 ? -x : x;
    y = y < 0 ? -y : y;
    return max(x, y);
```

```cpp
}

// (x,y)->(x+y,x-y)：原坐标曼哈顿距离等于新坐标切比雪夫距离。
template <class T> Point<T> toCheb(Point<T> p)
{
    // p 是平面点；返回对应切比雪夫坐标。
    return {p.x + p.y, p.x - p.y};
}

// 逆变换；若使用整数坐标，u,v 必须同奇偶。
P fromCheb(P p)
{
    // p 是浮点切比雪夫坐标；返回原坐标。
    return {(p.x + p.y) / 2, (p.x - p.y) / 2};
}

Point<long long> fromCheb(Point<long long> p)
{
    // p 是同奇偶的整数切比雪夫坐标；返回精确原坐标。
    assert((p.x - p.y) % 2 == 0); // 调试检查，可删。
    return {(p.x + p.y) / 2, (p.x - p.y) / 2};
}
```

`manhattan`、`chebyshev` 与 `toCheb` 由整数和 `Real` 共用并返回坐标类型。逆变换的整数版要求两个新坐标同奇偶，`Real` 版直接除以二；整数调用必须保证坐标加减、绝对值与距离均能由类型 `T` 表示。各函数均为 $O(1)$；点集最远距离见 11.5.002 最远曼哈顿距离。

11.5.004 矩形面积并、周长并

计算轴对齐矩形面积并或周长并。核心是扫描线和线段树覆盖长度，直接复用数据结构章节的实现。按横坐标扫描矩形左右边，线段树维护纵轴覆盖长度、覆盖段数以及左右端是否被覆盖。条带面积为覆盖长度乘横向宽度，横边长度为覆盖段数的两倍乘横向宽度，事件前后覆盖长度之差贡献竖边。

```cpp
template <class T> struct Rectangle
{
    // x1、y1、x2、y2 依次是矩形的左、下、右、上边界。
    T x1, y1, x2, y2;
};

template <class T, class Acc = T> class RectUnion
{
    struct Event
    {
        T x, y1, y2; // x 是扫描线位置，[y1,y2) 是该事件覆盖的纵向区间。
        int d; // d 为加入 1 或删除 -1。
    };

    vector<T> ys; // ys 保存离散后的纵坐标。
    vector<int> cov, seg, lc, rc; // 覆盖标记、段数和端点状态。
    vector<Acc> len; // len 保存覆盖长度。
```

```
18
19      void pull(int p, int l, int r)
20      {
21          // p 是节点编号，[l,r) 是纵坐标下标区间；由覆盖标记和儿子信息更新节点。
22          if (cov[p])
23          {
24              len[p] = (Acc)ys[r] - (Acc)ys[l];
25              seg[p] = lc[p] = rc[p] = 1;
26          }
27          else if (r - l == 1)
28          {
29              len[p] = 0;
30              seg[p] = lc[p] = rc[p] = 0;
31          }
32          else
33          {
34              int ls = p * 2, rs = ls + 1;
35              len[p] = len[ls] + len[rs];
36              seg[p] = seg[ls] + seg[rs] - (rc[ls] && lc[rs]);
37              lc[p] = lc[ls];
38              rc[p] = rc[rs];
39          }
40      }
41
42      void add(int p, int l, int r, int ql, int qr, int d)
43      {
44          // p、[l,r) 是当前节点，[ql,qr) 是修改区间，d 是覆盖次数增量。
45          if (ql <= l && r <= qr)
46          {
47              cov[p] += d;
48              pull(p, l, r);
49              return;
50          }
51          int m = midpoint(l, r);
52          if (ql < m)
53          {
54              add(p * 2, l, m, ql, qr, d);
55          }
56          if (m < qr)
57          {
58              add(p * 2 + 1, m, r, ql, qr, d);
59          }
60          pull(p, l, r);
61      }
62
63      pair<Acc, Acc> calc(const vector<Rectangle<T>> &a)
64      {
65          // a 是输入矩形集合；返回值 first、second 依次为面积并和周长并。
66          vector<Event> ev;
67          ys.clear();
68          for (auto [x1, y1, x2, y2] : a)
69          {
```

```
            if (x1 >= x2 || y1 >= y2)
            {
                continue;
            }
            ev.push_back({x1, y1, y2, 1});
            ev.push_back({x2, y1, y2, -1});
            ys.push_back(y1);
            ys.push_back(y2);
        }
        if (ev.empty())
        {
            return {};
        }
        ranges::sort(ys);
        ys.erase(unique(ys.begin(), ys.end()), ys.end());
        ranges::sort(ev, {}, &Event::x);
        int n = ys.size() * 4;
        cov.assign(n, 0);
        seg.assign(n, 0);
        lc.assign(n, 0);
        rc.assign(n, 0);
        len.assign(n, 0);

        Acc ar = 0;
        Acc peri = 0;
        T px = ev[0].x;
        for (int i = 0; i < (int)ev.size();)
        {
            int j = i;
            while (j < (int)ev.size() && ev[j].x == ev[i].x)
            {
                j++;
            }
            Acc dx = (Acc)ev[i].x - (Acc)px;
            ar += len[1] * dx;
            peri += (Acc)2 * seg[1] * dx;

            // 同一横坐标先加入再删除，逐次长度变化之和恰为新旧覆盖集合的对称差长度。
            for (int d : {1, -1})
            {
                for (int k = i; k < j; k++)
                {
                    if (ev[k].d != d)
                    {
                        continue;
                    }
                    Acc old = len[1];
                    int l = lower_bound(ys.begin(), ys.end(), ev[k].y1) - ys.begin();
                    int r = lower_bound(ys.begin(), ys.end(), ev[k].y2) - ys.begin();
                    add(1, 0, (int)ys.size() - 1, l, r, d);
                    Acc delta = len[1] - old;
                    peri += delta < 0 ? -delta : delta;
```

```
                }
            }
            px = ev[i].x;
            i = j;
        }
        return {ar, peri};
    }

  public:
    Acc area(const vector<Rectangle<T>> &a)
    {
        // a 是输入矩形集合；返回全部轴平行矩形并集的面积。
        return calc(a).first;
    }

    Acc perimeter(const vector<Rectangle<T>> &a)
    {
        // a 是输入矩形集合；返回全部轴平行矩形并集的周长。
        return calc(a).second;
    }

    Acc solve(const vector<Rectangle<T>> &a)
    {
        // a 是输入矩形集合；兼容旧接口并返回矩形并集面积。
        return area(a);
    }
};
```

构造 `RectUnion<int> q` 使用 64 位坐标与累计；若坐标差、覆盖长度或面积可能超过 64 位，改用 `RectUnion<int, __int128>`。矩形字段依次为左、下、右、上边界，退化矩形会被忽略。n 个矩形产生 $2n$ 个事件，时间 $O(n \log n)$、空间 $O(n)$。至少二重覆盖见 3.3.015 扫描线求矩形至少二重覆盖面积。

11.5.005 六边形坐标系

表示六边形网格的轴坐标，计算网格距离并与二维显示坐标互换。六边形轴坐标用两个坐标并隐含第三坐标之和为零；相邻移动改变两个轴，距离是三个立方坐标绝对差之和的一半。

```
// 依赖 11.1.002 Point / Vector
struct Hex
{
    int q; // q 是第一轴坐标。
    int r; // r 是第二轴坐标。

    Hex operator+(Hex b) const
    {
        // b 是另一组轴坐标；返回逐分量相加的轴坐标（平移 b）。
        return {q + b.q, r + b.r};
    }
    Hex operator-(Hex b) const
    {
```

```
        // b 是另一组轴坐标；返回逐分量相减的轴坐标，即从 b 到当前格的位移。
        return {q - b.q, r - b.r};
    }
};

int hexDist(Hex a, Hex b = {})
{
    // a、b 是轴坐标；返回六边形网格距离。
    __int128 q = (__int128)a.q - b.q;
    __int128 r = (__int128)a.r - b.r;
    auto ab = [](__int128 x)
    {
        return x < 0 ? -x : x;
    };
    __int128 d = (ab(q) + ab(r) + ab(q + r)) / 2;
    assert(d <= LLONG_MAX); // 调试检查，可删。
    return (int)d;
}

P hexPoint(Hex a, Real size = 1)
{
    // a 是轴坐标，size 是单元边长；返回平面显示坐标。
    return {size * sqrtl(3) * (a.q + a.r / 2.L), size * 1.5L * a.r};
}
```

构造轴坐标 `Hex{q,r}`，第三轴隐含为 `-q-r`；`hexDist(a,b)` 求格距，`hexPoint(a,size)` 转为平面显示坐标。时间和额外空间均为 $O(1)$。

11.5.006 球面经纬度与球面距离

由经纬度计算球面大圆距离，并给出高维球面公式入口。它是球面几何而不是平面欧氏距离。经纬度先转为单位球三维向量，中心角由点积的反余弦或 haversine 公式得到，再乘球半径；小角度优先 haversine 以减少消减误差。

```
// 依赖 11.1.001 浮点误差与精确整数几何
Real sphereDist(Real lat1, Real lon1, Real lat2, Real lon2, Real r = 1)
{
    // lat1、lon1、lat2、lon2 是弧度，r 是非负球半径。
    assert(sgn(r) >= 0); // 调试检查，可删。
    Real x = sinl((lat2 - lat1) / 2);
    Real y = sinl((lon2 - lon1) / 2);
    Real h = x * x + cosl(lat1) * cosl(lat2) * y * y;
    return 2 * r * asinl(sqrtl(clamp(h, Real(0), Real(1))));
}
```

调用 `sphereDist(lat1,lon1,lat2,lon2,r)`，经纬度均使用弧度，`r` 是球半径。haversine 在两点很近时避免点积接近 1 造成的消减误差；对跖点附近仍受浮点舍入影响，代码把中间值限制在 $[0,1]$。时间、额外空间均为 $O(1)$；平面欧氏距离使用 11.1.002 Point / Vector。

11.5.007 多线段任意相交对

在线段集中找出任意一对有公共点的闭线段。扫描线按横坐标成批处理左端插入、竖直线查询和右端删除；活动集的比较器只由线段端点确定，在尚未发现交点时与当前纵向次序一致，不读取会在容器外变化的扫描坐标。每次相邻关系建立时立即判交，同横坐标端点、竖直线段和零长度线段另行批处理，因此严格交、端点接触和共线重叠都能返回。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.1.005 Line / Ray / Segment
// 依赖 11.1.009 线段相交与距离
__int128 swCross(Point<long long> a, Point<long long> b, Point<long long> c)
{
    // a、b 确定有向直线，c 是查询点；返回使用宽整数计算的叉积。
    __int128 x1 = (__int128)b.x - a.x;
    __int128 y1 = (__int128)b.y - a.y;
    __int128 x2 = (__int128)c.x - a.x;
    __int128 y2 = (__int128)c.y - a.y;
    return x1 * y2 - y1 * x2;
}

Real swCross(P a, P b, P c)
{
    // a、b 确定有向直线，c 是查询点；返回浮点叉积。
    return cross(b - a, c - a);
}

bool swOnSeg(Point<long long> p, Line<long long> l)
{
    // p 是整数点，l 是整数闭线段；返回 p 是否在线段上，方向判定使用宽整数。
    return swCross(l.a, l.b, p) == 0 && min(l.a.x, l.b.x) <= p.x && p.x <= max(l.a.x, l.b.x) &&
           min(l.a.y, l.b.y) <= p.y && p.y <= max(l.a.y, l.b.y);
}

bool swMeet(Line<long long> a, Line<long long> b)
{
    // a、b 是整数闭线段；返回二者是否有公共点，叉积使用宽整数。
    __int128 c1 = swCross(a.a, a.b, b.a);
    __int128 c2 = swCross(a.a, a.b, b.b);
    __int128 c3 = swCross(b.a, b.b, a.a);
    __int128 c4 = swCross(b.a, b.b, a.b);
    if (c1 == 0 && swOnSeg(b.a, a))
    {
        return true;
    }
    if (c2 == 0 && swOnSeg(b.b, a))
    {
        return true;
    }
    if (c3 == 0 && swOnSeg(a.a, b))
    {
```

```
        return true;
    }
    if (c4 == 0 && swOnSeg(a.b, b))
    {
        return true;
    }
    return ((c1 > 0) != (c2 > 0)) && ((c3 > 0) != (c4 > 0));
}

bool swMeet(Line<Real> a, Line<Real> b)
{
    // a、b 是浮点闭线段；按统一浮点误差返回二者是否有公共点。
    return segMeet(a, b);
}

template <class T> optional<pair<int, int>> anySegMeet(vector<Line<T>> seg)
{
    // seg 是 long long 或 Real 闭线段集；返回任意一对相交线段的原下标，不存在时返回空。
    // 整数坐标绝对值须不超过 4E18；浮点活动次序使用原值严格比较。
    int n = (int)seg.size();
    vector<pair<Point<T>, int>> endpoint;
    for (int i = 0; i < n; i++)
    {
        endpoint.push_back({seg[i].a, i});
        endpoint.push_back({seg[i].b, i});
    }
    sort(endpoint.begin(), endpoint.end());
    for (int i = 0; i < (int)endpoint.size();)
    {
        int j = i;
        int who = -1;
        while (j < (int)endpoint.size() && endpoint[j].first == endpoint[i].first)
        {
            if (who != -1 && who != endpoint[j].second)
            {
                return pair<int, int>{min(who, endpoint[j].second), max(who,
                ↪ endpoint[j].second)};
            }
            who = endpoint[j].second;
            j++;
        }
        i = j;
    }
    vector<tuple<T, int, int>> event; // 三维依次为横坐标、0 插入/1 竖线查询/2 删除、线段下标。
    for (int i = 0; i < n; i++)
    {
        if (seg[i].b.x < seg[i].a.x)
        {
            swap(seg[i].a, seg[i].b);
        }
        if (seg[i].a.x == seg[i].b.x)
        {
```

```
            event.push_back({seg[i].a.x, 1, i});
        }
        else
        {
            event.push_back({seg[i].a.x, 0, i});
            event.push_back({seg[i].b.x, 2, i});
        }
    }
    sort(event.begin(), event.end());
    Point<T> probe{};
    auto cmp = [&](int i, int j)
    {
        if (i == j)
        {
            return false;
        }
        if (j == -1)
        {
            auto c = swCross(seg[i].a, seg[i].b, probe);
            return c > 0;
        }
        if (i == -1)
        {
            auto c = swCross(seg[j].a, seg[j].b, probe);
            return c <= 0;
        }
        if (seg[i].a.x >= seg[j].a.x)
        {
            auto c = swCross(seg[j].a, seg[j].b, seg[i].a);
            if (c != 0)
            {
                return c < 0;
            }
        }
        else
        {
            auto c = swCross(seg[i].a, seg[i].b, seg[j].a);
            if (c != 0)
            {
                return c > 0;
            }
        }
        return i < j;
    };
    set<int, decltype(cmp)> active(cmp);
    for (int l = 0; l < (int)event.size();)
    {
        int r = l + 1;
        while (r < (int)event.size() && get<0>(event[r]) == get<0>(event[l]))
        {
            r++;
        }
```

```
148         for (int i = l; i < r; i++)
149         {
150             auto [x, type, id] = event[i];
151             if (type != 0)
152             {
153                 continue;
154             }
155             auto it = active.insert(id).first;
156             if (it != active.begin())
157             {
158                 int j = *prev(it);
159                 if (swMeet(seg[id], seg[j]))
160                 {
161                     return pair<int, int>{min(id, j), max(id, j)};
162                 }
163             }
164             if (next(it) != active.end())
165             {
166                 int j = *next(it);
167                 if (swMeet(seg[id], seg[j]))
168                 {
169                     return pair<int, int>{min(id, j), max(id, j)};
170                 }
171             }
172         }
173         vector<int> vertical;
174         for (int i = l; i < r; i++)
175         {
176             if (get<1>(event[i]) == 1)
177             {
178                 vertical.push_back(get<2>(event[i]));
179             }
180         }
181         sort(vertical.begin(),
182              vertical.end(),
183              [&](int i, int j)
184              {
185                  T il = min(seg[i].a.y, seg[i].b.y);
186                  T ir = max(seg[i].a.y, seg[i].b.y);
187                  T jl = min(seg[j].a.y, seg[j].b.y);
188                  T jr = max(seg[j].a.y, seg[j].b.y);
189                  return tuple(il, ir, i) < tuple(jl, jr, j);
190              });
191         int who = -1;
192         T high{};
193         for (int id : vertical)
194         {
195             T low = min(seg[id].a.y, seg[id].b.y);
196             T curHigh = max(seg[id].a.y, seg[id].b.y);
197             if (who != -1 && swMeet(seg[id], seg[who]))
198             {
199                 return pair<int, int>{min(id, who), max(id, who)};
```

```
            }
            if (who == -1 || curHigh > high)
            {
                who = id;
                high = curHigh;
            }
            probe = {get<0>(event[l]), low};
            auto it = active.lower_bound(-1);
            if (it != active.begin())
            {
                int j = *prev(it);
                if (swMeet(seg[id], seg[j]))
                {
                    return pair<int, int>{min(id, j), max(id, j)};
                }
            }
            if (it != active.end() && swMeet(seg[id], seg[*it]))
            {
                return pair<int, int>{min(id, *it), max(id, *it)};
            }
        }
        for (int i = l; i < r; i++)
        {
            auto [x, type, id] = event[i];
            if (type != 2)
            {
                continue;
            }
            auto it = active.find(id);
            assert(it != active.end()); // 调试检查，可删。
            auto nx = next(it);
            if (it != active.begin() && nx != active.end())
            {
                int u = *prev(it);
                int v = *nx;
                if (swMeet(seg[u], seg[v]))
                {
                    return pair<int, int>{min(u, v), max(u, v)};
                }
            }
            active.erase(it);
        }
        l = r;
    }
    return nullopt;
}
```

`anySegMeet(seg)` 接收 `vector<Line<long long>>` 或 `vector<Line<Real>>`，返回任意相交线段的一对原下标，不存在时返回 `nullopt`。整数版在坐标绝对值不超过 4×10^{18} 时用 `__int128` 宽叉积；浮点版关系判断使用统一误差，但活动集和事件排序仍使用原始值严格比较。时间 $O(n\log n)$、空间 $O(n)$；只判断给定一对线段时直接用 11.1.009 线段相交与距离。

11.6 三维几何

11.6.001 三维点、向量、直线、平面

定义三维点、向量、直线和平面，并提供点积、叉积、模长和单位化，作为本节所有空间几何算法的共同底座；具体关系、相交、距离和体积函数放在各自主讲条目。三维点差得到向量，直线用点加方向参数表示，平面用点和法向量或一般式表示；零方向和零法向量不是合法对象。

```
// 依赖 11.1.001 浮点误差与精确整数几何
struct P3
{
    Real x; // x 保存横坐标。
    Real y; // y 保存纵坐标。
    Real z; // z 保存竖坐标。

    P3 operator+(P3 p) const
    {
        // p 是另一三维点或向量；返回逐分量之和。
        return {x + p.x, y + p.y, z + p.z};
    }
    P3 operator-(P3 p) const
    {
        // p 是另一三维点或向量；返回逐分量之差，即从 p 指向当前点的向量。
        return {x - p.x, y - p.y, z - p.z};
    }
    P3 operator*(Real k) const
    {
        // k 是实数标量；返回各分量乘以 k 的向量。
        return {x * k, y * k, z * k};
    }
    P3 operator/(Real k) const
    {
        // k 是非零实数标量；返回各分量除以 k 的向量。
        assert(sgn(k) != 0); // 调试检查，可删。
        return {x / k, y / k, z / k};
    }
};

Real dot(P3 a, P3 b)
{
    // a、b 是三维向量；返回点积 a·b。
    return a.x * b.x + a.y * b.y + a.z * b.z;
}
P3 cross(P3 a, P3 b)
{
    // a、b 是三维向量；返回右手系叉积 a×b，与 a、b 都垂直。
    return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x};
}
Real norm(P3 a)
{
    // a 是三维向量；返回模长平方 |a|^2。
```

```
    return dot(a, a);
}
Real abs(P3 a)
{
    // a 是三维向量；返回模长 |a|。
    return sqrtl(norm(a));
}
P3 unit(P3 a)
{
    // a 是非零三维向量；返回同方向单位向量 a/|a|。
    return a / abs(a);
}

struct L3
{
    P3 p; // p 是直线上的一点。
    P3 v; // v 是非零方向向量。
};

struct Plane
{
    P3 p; // p 是平面上的一点。
    P3 n; // n 是非零法向量。
};
```

`P3`、`L3`、`Plane` 分别表示三维点向量、点方向式直线、点法式平面；三维浮点接口与二维部分共用 `Real`。本条只提供底座运算。$O(1)$ 时间和 $O(1)$ 额外空间。所有空间算法先使用 11.6.001 三维点、向量、直线、平面 的 P3/L3/Plane。小规模凸包可用 11.6.006 三维凸包 枚举版，规模大或存在大量共面点时应换可靠增量实现；覆盖球使用 11.6.007 最小覆盖球，不能用平面最小圆。

11.6.002 共线、共面、平行、垂直

统一判断三维点、直线和平面的共线、共面、包含、平行和垂直关系。共线由叉积为零判定，共面由混合积为零判定；直线方向与平面法向的叉积或点积分别给出平行、垂直关系。浮点判号先除以相关向量长度，使判断不随方向向量的普通倍数缩放而改变。整数点另用 `IP3`，把坐标差先提升到 `__int128`；`mixedSign(a,b,c,d)` 返回 $(b-a)\cdot((c-a)\times(d-a))$ 的精确符号。若四个整数点坐标绝对值不超过 10^{12}，混合积的中间量不超过 $6(2\times10^{12})^3<2^{127}$，因此 `collinearEx(IP3,IP3,IP3)`、`coplanarEx(IP3,IP3,IP3,IP3)` 和 `mixedSign` 不发生有符号溢出。精确版本使用独立名称，避免 `collinear({},{},{})` 这类既有浮点调用产生重载二义性。

```
// 依赖 11.6.001 三维点、向量、直线、平面
struct IP3
{
    long long x; // x 是整数点的第一维坐标。
    long long y; // y 是整数点的第二维坐标。
    long long z; // z 是整数点的第三维坐标。
};

int mixedSign(IP3 a, IP3 b, IP3 c, IP3 d)
```

```
{
    // a、b、c、d 是整数空间点；返回 (b-a)·((c-a)×(d-a)) 的精确符号。
    __int128 ux = (__int128)b.x - a.x;
    __int128 uy = (__int128)b.y - a.y;
    __int128 uz = (__int128)b.z - a.z;
    __int128 vx = (__int128)c.x - a.x;
    __int128 vy = (__int128)c.y - a.y;
    __int128 vz = (__int128)c.z - a.z;
    __int128 wx = (__int128)d.x - a.x;
    __int128 wy = (__int128)d.y - a.y;
    __int128 wz = (__int128)d.z - a.z;
    __int128 det = ux * (vy * wz - vz * wy) - uy * (vx * wz - vz * wx) + uz * (vx * wy - vy * wx);
    return (det > 0) - (det < 0);
}

bool collinearEx(IP3 a, IP3 b, IP3 c)
{
    // a、b、c 是整数空间点；返回三点是否共线，重合点视为共线。
    __int128 ux = (__int128)b.x - a.x;
    __int128 uy = (__int128)b.y - a.y;
    __int128 uz = (__int128)b.z - a.z;
    __int128 vx = (__int128)c.x - a.x;
    __int128 vy = (__int128)c.y - a.y;
    __int128 vz = (__int128)c.z - a.z;
    return uy * vz - uz * vy == 0 && uz * vx - ux * vz == 0 && ux * vy - uy * vx == 0;
}

bool coplanarEx(IP3 a, IP3 b, IP3 c, IP3 d)
{
    // a、b、c、d 是整数空间点；返回四点是否共面，退化点集视为共面。
    return mixedSign(a, b, c, d) == 0;
}

bool collinear(P3 a, P3 b, P3 c)
{
    // a、b、c 是三个空间点；返回三点是否共线，重合点视为共线。
    P3 u = b - a;
    P3 v = c - a;
    if (sgn(abs(u)) == 0 || sgn(abs(v)) == 0)
    {
        return true;
    }
    return sgn(abs(cross(u, v)) / abs(u) / abs(v)) == 0;
}

bool coplanar(P3 a, P3 b, P3 c, P3 d)
{
    // a、b、c、d 是四个空间点；返回四点是否共面，重合或共线退化视为共面。
    P3 u = b - a;
    P3 v = c - a;
    P3 w = d - a;
    if (sgn(abs(u)) == 0 || sgn(abs(v)) == 0 || sgn(abs(w)) == 0)
```

```
    {
        return true;
    }
    return sgn(dot(u, cross(v, w)) / abs(u) / abs(v) / abs(w)) == 0;
}

bool onLine3(P3 p, L3 l)
{
    // p 是查询点，l 是非退化直线；返回 p 是否在 l 上。
    assert(sgn(abs(l.v)) != 0); // 调试检查，可删。
    P3 u = p - l.p;
    if (sgn(abs(u)) == 0)
    {
        return true;
    }
    return sgn(abs(cross(u, l.v)) / abs(u) / abs(l.v)) == 0;
}

bool onSegment3(P3 p, P3 a, P3 b)
{
    // p 是查询点，a、b 是闭线段端点；返回 p 是否在线段上，a=b 时线段是单点。
    P3 v = b - a;
    if (sgn(abs(v)) == 0)
    {
        return sgn(abs(p - a)) == 0;
    }
    if (!onLine3(p, {a, v}))
    {
        return false;
    }
    Real t = dot(p - a, v) / norm(v);
    return t >= -eps && t <= 1 + eps;
}

bool onPlane3(P3 p, Plane s)
{
    // p 是查询点，s 是非退化平面；返回 p 是否在 s 上。
    assert(sgn(abs(s.n)) != 0); // 调试检查，可删。
    return sgn(dot(p - s.p, s.n) / abs(s.n)) == 0;
}

bool parallel(L3 a, L3 b)
{
    // a、b 是两条非退化直线；返回二者方向是否平行。
    assert(sgn(abs(a.v)) != 0 && sgn(abs(b.v)) != 0); // 调试检查，可删。
    return sgn(abs(cross(a.v, b.v)) / abs(a.v) / abs(b.v)) == 0;
}

bool parallel(Plane a, Plane b)
{
    // a、b 是两个非退化平面；返回二者是否平行。
    assert(sgn(abs(a.n)) != 0 && sgn(abs(b.n)) != 0); // 调试检查，可删。
```

```
    return sgn(abs(cross(a.n, b.n)) / abs(a.n) / abs(b.n)) == 0;
}

bool parallel(L3 a, Plane b)
{
    // a 是非退化直线，b 是非退化平面；返回直线方向是否平行于平面。
    assert(sgn(abs(a.v)) != 0 && sgn(abs(b.n)) != 0); // 调试检查，可删。
    return sgn(dot(a.v, b.n) / abs(a.v) / abs(b.n)) == 0;
}

bool perp(L3 a, L3 b)
{
    // a、b 是两条非退化直线；返回二者方向是否垂直。
    assert(sgn(abs(a.v)) != 0 && sgn(abs(b.v)) != 0); // 调试检查，可删。
    return sgn(dot(a.v, b.v) / abs(a.v) / abs(b.v)) == 0;
}

bool perp(Plane a, Plane b)
{
    // a、b 是两个非退化平面；返回二者法向量是否垂直。
    assert(sgn(abs(a.n)) != 0 && sgn(abs(b.n)) != 0); // 调试检查，可删。
    return sgn(dot(a.n, b.n) / abs(a.n) / abs(b.n)) == 0;
}
```

`collinear` 把含重合点的三点视为共线，`coplanar` 把重合或共线退化视为共面；整数版要求各坐标绝对值不超过 10^{12}，浮点版使用统一误差。`onSegment3` 支持端点相同的单点线段。`onLine3`、`onPlane3` 以及 `parallel`、`perp` 的直线方向或平面法向必须非零，接口中用 `assert` 检查。所有函数均为 $O(1)$ 时间和 $O(1)$ 额外空间；求交见 11.6.003 直线与平面、两平面求交，最近点和关系分类见 11.6.010 投影、最近点与空间距离。

11.6.003 直线与平面、两平面求交

求直线与平面交点、两平面交线。平行或重合时返回空 optional，由调用者区分题目语义。直线平面交参数由法向量点积方程解出；两非平行平面交线方向是两法向叉积，再联立两个平面方程求线上一点。

```
// 依赖 11.6.001 三维点、向量、直线、平面
optional<P3> linePlane(L3 l, Plane s)
{
    // 参数 l 表示 待求交的三维直线；s 表示 待求交的平面；返回直线与平面的唯一交点；平行或重合时返回空。
    assert(sgn(abs(l.v)) != 0 && sgn(abs(s.n)) != 0); // 调试检查，可删。
    Real d = dot(l.v, s.n);
    if (sgn(d / abs(l.v) / abs(s.n)) == 0)
    {
        return nullopt;
    }
    return l.p + l.v * (dot(s.p - l.p, s.n) / d);
}

optional<L3> planePlane(Plane a, Plane b)
```

```
{
    // 参数 a 表示 第一个平面；b 表示 第二个平面；返回两平面的交线；平行或重合时返回空。
    assert(sgn(abs(a.n)) != 0 && sgn(abs(b.n)) != 0); // 调试检查，可删。
    P3 v = cross(a.n, b.n);
    Real d = norm(v);
    if (sgn(d / norm(a.n) / norm(b.n)) == 0)
    {
        return nullopt;
    }
    Real x = dot(a.n, a.p);
    Real y = dot(b.n, b.p);
    P3 p = cross(b.n * x - a.n * y, v) / d;
    return L3{p, v};
}
```

`linePlane` 与 `planePlane` 返回 `optional`；空值表示没有唯一交点或交线，平行与重合需由题意进一步区分。$O(1)$ 时间和 $O(1)$ 额外空间。所有空间算法先使用 11.6.001 三维点、向量、直线、平面 的 P3/L3/Plane。小规模凸包可用 11.6.006 三维凸包 枚举版，规模大或存在大量共面点时应换可靠增量实现；覆盖球使用 11.6.007 最小覆盖球，不能用平面最小圆。

11.6.004 点线面距离与夹角

计算点到直线、点到平面、两异面直线的距离，以及向量夹角。点线距离由点差与方向叉积长度除以方向长度，点面距离由平面方程绝对值除以法向长度；夹角统一由规范化点积或叉积计算。

```
// 依赖 11.1.001 浮点误差与精确整数几何
// 依赖 11.6.001 三维点、向量、直线、平面
Real dis(P3 p, L3 l)
{
    // p 是点，l 是直线；返回点线距离。
    assert(sgn(abs(l.v)) != 0); // 调试检查，可删。
    return abs(cross(p - l.p, l.v)) / abs(l.v);
}
Real dis(P3 p, Plane s)
{
    // p 是点，s 是平面；返回点面距离。
    assert(sgn(abs(s.n)) != 0); // 调试检查，可删。
    return fabsl(dot(p - s.p, s.n)) / abs(s.n);
}
Real dis(L3 a, L3 b)
{
    // a、b 是直线；返回二者最短距离。
    assert(sgn(abs(a.v)) != 0 && sgn(abs(b.v)) != 0); // 调试检查，可删。
    P3 n = cross(a.v, b.v);
    if (sgn(abs(n) / abs(a.v) / abs(b.v)) == 0)
    {
        return dis(a.p, b);
    }
    return fabsl(dot(b.p - a.p, n)) / abs(n);
}
```

```
Real angle(P3 a, P3 b)
{
    // a、b 是非零向量；返回夹角弧度。
    assert(sgn(abs(a)) != 0 && sgn(abs(b)) != 0); // 调试检查，可删。
    return acosl(clamp(dot(a, b) / abs(a) / abs(b), Real(-1), Real(1)));
}
```

调用三个 `dis` 重载和 `angle`；夹角返回弧度，零向量不能用于夹角。$O(1)$ 时间和 $O(1)$ 额外空间。所有空间算法先使用 11.6.001 三维点、向量、直线、平面 的 P3/L3/Plane。小规模凸包可用 11.6.006 三维凸包 枚举版，规模大或存在大量共面点时应换可靠增量实现；覆盖球使用 11.6.007 最小覆盖球，不能用平面最小圆。

11.6.005 空间线段与三角形相交

判断空间线段是否穿过三角形。先与三角形所在平面求交，再用同侧叉积判点在三角形内。线段与三角形交可用 Möller-Trumbore 解射线参数和重心坐标，再限制参数在线段范围；共面时需投影到主平面做二维相交。

```
// 依赖 11.1.009 线段相交与距离
// 依赖 11.6.001 三维点、向量、直线、平面
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
bool segTri(P3 p, P3 q, P3 a, P3 b, P3 c)
{
    // p、q 是线段端点，a、b、c 是非退化三角形顶点。
    P3 v = q - p;
    P3 e1 = b - a;
    P3 e2 = c - a;
    P3 n = cross(e1, e2);
    assert(sgn(abs(n)) != 0); // 调试检查，可删。
    P3 h = cross(v, e2);
    Real d = dot(e1, h);
    Real scale = abs(v) * abs(e1) * abs(e2);
    if (fabsl(d) <= eps * max(Real(1), scale))
    {
        if (sgn(dot(n, p - a) / abs(n)) != 0
            || sgn(dot(n, q - a) / abs(n)) != 0)
        {
            return false;
        }
        int k = 0;
        if (fabsl(n.y) > fabsl(n.x))
        {
            k = 1;
        }
        if (fabsl(n.z) > (k == 0 ? fabsl(n.x) : fabsl(n.y)))
        {
            k = 2;
        }
        auto proj = [&](P3 x)
```

```
        {
            if (k == 0)
            {
                return P{x.y, x.z};
            }
            if (k == 1)
            {
                return P{x.x, x.z};
            }
            return P{x.x, x.y};
        };
        P x = proj(p);
        P y = proj(q);
        P A = proj(a);
        P B = proj(b);
        P C = proj(c);
        auto inside = [&](P z)
        {
            int x1 = sgn(cross(A, B, z));
            int x2 = sgn(cross(B, C, z));
            int x3 = sgn(cross(C, A, z));
            return !(min({x1, x2, x3}) < 0 && max({x1, x2, x3}) > 0);
        };
        return inside(x) || inside(y) || segIsect(x, y, A, B)
            || segIsect(x, y, B, C) || segIsect(x, y, C, A);
    }
    Real z = 1 / d;
    P3 s = p - a;
    Real u = dot(s, h) * z;
    if (u < -eps || u > 1 + eps)
    {
        return false;
    }
    Real w = dot(v, cross(s, e1)) * z;
    if (w < -eps || u + w > 1 + eps)
    {
        return false;
    }
    Real t = dot(e2, cross(s, e1)) * z;
    return t >= -eps && t <= 1 + eps;
}
```

调用 `segTri(p,q,a,b,c)`；三角形必须非退化，返回值包含端点落在三角形边界的情形。$O(1)$ 时间和 $O(1)$ 额外空间。所有空间算法先使用 11.6.001 三维点、向量、直线、平面 的 P3/L3/Plane。小规模凸包可用 11.6.006 三维凸包 枚举版，规模大或存在大量共面点时应换可靠增量实现；覆盖球使用 11.6.007 最小覆盖球，不能用平面最小圆。

11.6.006 三维凸包

构造三维点集凸包的外向三角面。随机增量维护当前面集：新点能从外侧看到的面全部删除，可见面和保留面交界处的有向边恰好组成地平线，只沿这些边连接新点即可。先把四个不共面的点换到最前面；全部点共面时没有三维面，返回空。

```cpp
// 依赖 11.6.001 三维点、向量、直线、平面
vector<array<int, 3>> hull3(vector<P3> p, int seed = 0)
{
    // p 是点集，seed 是固定洗牌种子；返回外侧朝向的三角面原下标。
    int n = p.size();
    vector<int> id(n);
    iota(id.begin(), id.end(), 0);
    mt19937 rng(seed);
    for (int i = n - 1; i > 0; i--)
    {
        int j = rng() % (i + 1);
        swap(p[i], p[j]);
        swap(id[i], id[j]);
    }
    auto vol = [&](array<int, 3> f, P3 q)
    {
        return dot(cross(p[f[1]] - p[f[0]], p[f[2]] - p[f[0]]), q - p[f[0]]);
    };
    int b = -1;
    for (int i = 1; i < n; i++)
    {
        if (sgn(abs(p[i] - p[0])) != 0)
        {
            b = i;
            break;
        }
    }
    if (b < 0)
    {
        return {};
    }
    swap(p[1], p[b]);
    swap(id[1], id[b]);
    b = -1;
    for (int i = 2; i < n; i++)
    {
        if (sgn(abs(cross(p[1] - p[0], p[i] - p[0]))) != 0)
        {
            b = i;
            break;
        }
    }
    if (b < 0)
    {
        return {};
```

```
    }
    swap(p[2], p[b]);
    swap(id[2], id[b]);
    b = -1;
    for (int i = 3; i < n; i++)
    {
        if (sgn(vol({0, 1, 2}, p[i])) != 0)
        {
            b = i;
            break;
        }
    }
    if (b < 0)
    {
        return {};
    }
    swap(p[3], p[b]);
    swap(id[3], id[b]);

    vector<array<int, 3>> f{{0, 1, 2}, {0, 2, 1}};
    vector<vector<int>> seen(n, vector<int>(n, -1));
    for (int i = 3; i < n; i++)
    {
        vector<array<int, 3>> keep, out;
        for (auto x : f)
        {
            if (sgn(vol(x, p[i])) > 0)
            {
                out.push_back(x);
                for (int k = 0; k < 3; k++)
                {
                    seen[x[k]][x[(k + 1) % 3]] = i;
                }
            }
            else
            {
                keep.push_back(x);
            }
        }
        for (auto x : out)
        {
            for (int k = 0; k < 3; k++)
            {
                int u = x[k], v = x[(k + 1) % 3];
                if (seen[v][u] != i)
                {
                    keep.push_back({u, v, i});
                }
            }
        }
        f.swap(keep);
    }
```

```
    for (auto &x : f)
    {
        for (int &u : x)
        {
            u = id[u];
        }
    }
    return f;
}
```

调用 `hull3(p,seed)` 返回外向三角面的原下标，固定种子默认 0；输入允许重复点，若无法找到四个不共面点则返回空。随机顺序下凸包面数为 $O(n)$，增量扫描期望 $O(n^2)$ 时间；有向边时间戳矩阵占 $O(n^2)$ 空间，适合竞赛中常见的几千点规模。大量共面壳点会选择其中一套三角剖分，不保证保留面内所有点；需要输出共面多边形面时应再按平面合并相邻三角面。

11.6.007 最小覆盖球

求覆盖全部三维点的最小球。随机增量时边界集最多四点，可分别由一至四点确定球。随机打乱点后增量维护覆盖球；遇到球外点时把它依次固定为球面点，并在更内层重建由二、三、四个边界点确定的球。

```
// 依赖 11.6.001 三维点、向量、直线、平面
struct Sphere
{
    P3 o; // o 保存球心。
    Real r; // r 保存非负半径。
};

bool in(Sphere s, P3 p)
{
    // s 是球，p 是查询点；返回 p 是否在球内或球面上。
    assert(sgn(s.r) >= 0); // 调试检查，可删。
    return sgn(abs(p - s.o) - s.r) <= 0;
}

optional<Sphere> sphere4(P3 a, P3 b, P3 c, P3 d)
{
    // a、b、c、d 是四点；共面时没有唯一外接球。
    P3 u = b - a;
    P3 v = c - a;
    P3 w = d - a;
    Real det = dot(u, cross(v, w));
    if (fabsl(det) <= eps * max(Real(1), abs(u) * abs(v) * abs(w)))
    {
        return nullopt;
    }
    P3 o = a + (cross(v, w) * norm(u) + cross(w, u) * norm(v) + cross(u, v) * norm(w)) / (2 *
    ↪ det);
    return Sphere{o, abs(o - a)};
}

```

```
optional<Sphere> sphere3(P3 a, P3 b, P3 c)
{
    // a、b、c 是三点；共线时没有唯一外接球。
    P3 u = b - a;
    P3 v = c - a;
    P3 w = cross(u, v);
    Real z = 2 * norm(w);
    if (sgn(abs(w)) == 0)
    {
        return nullopt;
    }
    P3 o = a + (cross(w, u) * norm(v) + cross(v, w) * norm(u)) / z;
    return Sphere{o, abs(o - a)};
}

Sphere boundSphere(const vector<P3> &p)
{
    // p 含至多四个边界点；返回覆盖它们的最小球。
    if (p.empty())
    {
        return {{}, 0};
    }
    Sphere ans{{}, numeric_limits<Real>::infinity()};
    int n = p.size();
    auto add = [&](optional<Sphere> s)
    {
        if (!s || s->r >= ans.r)
        {
            return;
        }
        for (P3 x : p)
        {
            if (!in(*s, x))
            {
                return;
            }
        }
        ans = *s;
    };
    for (int i = 0; i < n; i++)
    {
        add(Sphere{p[i], 0});
    }
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < i; j++)
        {
            add(Sphere{(p[i] + p[j]) / 2, abs(p[i] - p[j]) / 2});
        }
    }
    for (int i = 0; i < n; i++)
    {
```

```
        for (int j = 0; j < i; j++)
        {
            for (int k = 0; k < j; k++)
            {
                add(sphere3(p[i], p[j], p[k]));
            }
        }
    }
    if (n == 4)
    {
        add(sphere4(p[0], p[1], p[2], p[3]));
    }
    return ans;
}

Sphere minSphere(vector<P3> p, unsigned seed = 712367)
{
    // p 是点集，seed 是打乱种子；返回最小覆盖球。
    mt19937 rng(seed);
    shuffle(p.begin(), p.end(), rng);
    Sphere s{{}, -1};
    for (int i = 0; i < (int)p.size(); i++)
    {
        if (s.r >= 0 && in(s, p[i]))
        {
            continue;
        }
        s = {p[i], 0};
        for (int j = 0; j < i; j++)
        {
            if (in(s, p[j]))
            {
                continue;
            }
            s = boundSphere({p[i], p[j]});
            for (int k = 0; k < j; k++)
            {
                if (in(s, p[k]))
                {
                    continue;
                }
                s = boundSphere({p[i], p[j], p[k]});
                for (int l = 0; l < k; l++)
                {
                    if (!in(s, p[l]))
                    {
                        s = boundSphere({p[i], p[j], p[k], p[l]});
                    }
                }
            }
        }
    }
```

```
    if (p.empty())
    {
        s = {{}, 0};
    }
    return s;
}
```

调用 `minSphere(p,seed)`，返回 `Sphere{o,r}`；空点集返回原点和半径 0。随机排列期望 $O(n)$、最坏 $O(n^4)$ 时间，$O(n)$ 空间。所有空间算法先使用 11.6.001 三维点、向量、直线、平面 的 P3/L3/Plane。小规模凸包可用 11.6.006 三维凸包 枚举版，规模大或存在大量共面点时应换可靠增量实现；覆盖球使用 11.6.007 最小覆盖球，不能用平面最小圆。

11.6.008 四面体体积与内切球

计算四面体体积与内切球。体积来自标量三重积，内心是按对面面积加权的顶点平均。四面体有向体积是三条边混合积的六分之一；内心按四个对面三角形面积加权顶点，内切球半径为三倍体积除以表面积。

```
// 依赖 11.1.001 浮点误差与精确整数几何
// 依赖 11.6.001 三维点、向量、直线、平面
Real tetraVolume(P3 a, P3 b, P3 c, P3 d)
{
    // a、b、c、d 是四面体顶点；返回无符号体积。
    return fabsl(dot(b - a, cross(c - a, d - a))) / 6;
}

pair<P3, Real> tetraSphere(P3 a, P3 b, P3 c, P3 d)
{
    // a、b、c、d 是非退化四面体顶点；返回内心和内切球半径。
    Real x = abs(cross(b - a, c - a));
    Real y = abs(cross(b - a, d - a));
    Real z = abs(cross(c - a, d - a));
    Real w = abs(cross(c - b, d - b));
    Real s = x + y + z + w;
    assert(sgn(s) != 0 && sgn(tetraVolume(a, b, c, d)) != 0); // 调试检查，可删。
    P3 o = (d * x + c * y + b * z + a * w) / s;
    return {o, 3 * tetraVolume(a, b, c, d) / (s / 2)};
}
```

调用 `tetraVolume` 与 `tetraSphere`；后者返回内心和内切球半径，四点必须构成非退化四面体。$O(1)$ 时间和 $O(1)$ 额外空间。所有空间算法先使用 11.6.001 三维点、向量、直线、平面 的 P3/L3/Plane。小规模凸包可用 11.6.006 三维凸包 枚举版，规模大或存在大量共面点时应换可靠增量实现；覆盖球使用 11.6.007 最小覆盖球，不能用平面最小圆。

11.6.009 高维球体积、超球坐标与立体角

计算高维球体积、超球坐标相关量和立体角。公式依赖 Gamma 函数，数值范围与精度必须单独检查。d 维单位球体积满足 `V_d=pi^(d/2)/Gamma(d/2+1)`；超球坐标的体积元含逐层正弦幂，立体角是单位球面上对应区域测度。

```cpp
// 依赖 11.1.001 浮点误差与精确整数几何
Real sphereVolume(Real r, int n = 3)
{
    // r 是非负半径，n 是非负维数；返回 n 维球体积。
    assert(sgn(r) >= 0 && n >= 0); // 调试检查，可删。
    if (n == 0)
    {
        return 1;
    }
    if (sgn(r) == 0)
    {
        return 0;
    }
    Real x = Real(n) / 2 * logl(acosl(-1)) + n * logl(r)
        - lgammal(Real(n) / 2 + 1);
    return expl(x);
}
```

调用 `sphereVolume(r,n)`；`r` 与 `n` 必须非负，0 维球体积按约定为 1。函数在对数域使用 `lgammal`，最后一次 `expl` 仍可能在真实结果超出 `Real` 范围时得到无穷。$O(1)$ 时间和 $O(1)$ 额外空间。所有空间算法先使用 11.6.001 三维点、向量、直线、平面 的 P3/L3/Plane。小规模凸包可用 11.6.006 三维凸包 枚举版，规模大或存在大量共面点时应换可靠增量实现；覆盖球使用 11.6.007 最小覆盖球，不能用平面最小圆。

11.6.010 投影、最近点与空间距离

点到直线 l 的投影参数为 $t=\frac{(p-l.p)\cdot l.v}{\|l.v\|^2}$，点到线段时再把 t 截到 $[0,1]$；两条非平行直线由两个正交条件组成的二元方程求最近点对，据距离区分相交与异面。三个平面有唯一交点当且仅当 $D=n_a\cdot(n_b\times n_c)\neq 0$，Cramer 公式直接给出交点。本节依赖 11.6.002 共线、共面、平行、垂直、11.6.003 直线与平面、两平面求交 和 11.6.004 点线面距离与夹角。`lineLine3` 允许直线退化，任一方向为零时返回 `LineRel3::Degenerate`；平行或重合直线没有唯一最近点见证，故 `closest` 为空。

```cpp
// 依赖 11.6.001 三维点、向量、直线、平面
// 依赖 11.6.004 点线面距离与夹角
struct PointSeg3
{
    P3 point;      // point 是线段上的最近点。
    Real t;        // t 满足 point=a+(b-a)*t，范围为 [0,1]。
    Real distance; // distance 是查询点到线段的距离。
};

struct ClosestPair3
{
    P3 onA;        // onA 是第一个对象上的最近点。
    P3 onB;        // onB 是第二个对象上的最近点。
    Real tA;       // tA 是 onA 在第一个对象参数式中的参数。
    Real tB;       // tB 是 onB 在第二个对象参数式中的参数。
    Real distance; // distance 是两个对象之间的距离。
};

```

```cpp
enum class LineRel3
{
    Intersecting,
    Skew,
    Parallel,
    Coincident,
    Degenerate
};

struct LineClosest3
{
    LineRel3 relation;         // relation 是两条直线的空间关系。
    Real distance;                  // distance 是两条直线的最短距离，所有关系下都有效。
    optional<ClosestPair3> closest; // closest 在相交、异面或退化点线关系下保存唯一最近点对。
};

P3 projection(L3 l, P3 p)
{
    // l 是非退化直线，p 是查询点；返回 p 在 l 上的正交投影。
    assert(norm(l.v) != 0); // 调试检查，可删。
    return l.p + l.v * (dot(p - l.p, l.v) / norm(l.v));
}

P3 projection(Plane s, P3 p)
{
    // s 是非退化平面，p 是查询点；返回 p 在 s 上的正交投影。
    assert(norm(s.n) != 0); // 调试检查，可删。
    return p - s.n * (dot(p - s.p, s.n) / norm(s.n));
}

PointSeg3 pointSeg3(P3 p, P3 a, P3 b)
{
    // p 是查询点，a、b 是闭线段端点；返回最近点、线段参数和距离。
    P3 v = b - a;
    Real t = 0;
    if (norm(v) != 0)
    {
        t = clamp(dot(p - a, v) / norm(v), Real(0), Real(1));
    }
    P3 q = a + v * t;
    return {q, t, abs(p - q)};
}

LineClosest3 lineLine3(L3 a, L3 b)
{
    // a、b 是两条可退化直线；返回关系、距离以及存在唯一解时的最近点对。
    Real aa = norm(a.v);
    Real bb = norm(b.v);
    if (aa == 0 && bb == 0)
    {
        Real distance = abs(a.p - b.p);
        return {LineRel3::Degenerate, distance, ClosestPair3{a.p, b.p, 0, 0, distance}};
```

```
    }
    if (aa == 0)
    {
        Real t = dot(a.p - b.p, b.v) / bb;
        P3 q = b.p + b.v * t;
        Real distance = abs(a.p - q);
        return {LineRel3::Degenerate, distance, ClosestPair3{a.p, q, 0, t, distance}};
    }
    if (bb == 0)
    {
        Real s = dot(b.p - a.p, a.v) / aa;
        P3 q = a.p + a.v * s;
        Real distance = abs(q - b.p);
        return {LineRel3::Degenerate, distance, ClosestPair3{q, b.p, s, 0, distance}};
    }
    P3 n = cross(a.v, b.v);
    Real d = norm(n);
    if (d == 0)
    {
        LineRel3 relation = onLine3(a.p, b) ? LineRel3::Coincident : LineRel3::Parallel;
        Real distance = relation == LineRel3::Coincident ? 0 : dis(a.p, b);
        return {relation, distance, nullopt};
    }
    P3 w = b.p - a.p;
    Real s = dot(cross(w, b.v), n) / d;
    Real t = dot(cross(w, a.v), n) / d;
    P3 x = a.p + a.v * s;
    P3 y = b.p + b.v * t;
    Real distance = abs(x - y);
    LineRel3 relation = sgn(distance) == 0 ? LineRel3::Intersecting : LineRel3::Skew;
    return {relation, distance, ClosestPair3{x, y, s, t, distance}};
}

ClosestPair3 segSeg3(P3 a, P3 b, P3 c, P3 d)
{
    // a、b 与 c、d 是两条闭线段端点；返回一组最近点、两段参数和距离。
    P3 u = b - a;
    P3 v = d - c;
    P3 w = a - c;
    Real aa = norm(u);
    Real ab = dot(u, v);
    Real bb = norm(v);
    Real aw = dot(u, w);
    Real bw = dot(v, w);
    Real s = 0;
    Real t = 0;
    if (aa == 0 && bb == 0)
    {
        s = t = 0;
    }
    else if (aa == 0)
    {
```

```
        t = clamp(bw / bb, Real(0), Real(1));
    }
    else if (bb == 0)
    {
        s = clamp(-aw / aa, Real(0), Real(1));
    }
    else
    {
        P3 n = cross(u, v);
        Real det = norm(n);
        if (det != 0)
        {
            s = clamp(dot(cross(c - a, v), n) / det, Real(0), Real(1));
        }
        t = (ab * s + bw) / bb;
        if (t < 0)
        {
            t = 0;
            s = clamp(-aw / aa, Real(0), Real(1));
        }
        else if (t > 1)
        {
            t = 1;
            s = clamp((ab - aw) / aa, Real(0), Real(1));
        }
    }
    P3 x = a + u * s;
    P3 y = c + v * t;
    ClosestPair3 ans{x, y, s, t, abs(x - y)};
    auto take = [&](P3 p, P3 q, Real ta, Real tb)
    {
        Real distance = abs(p - q);
        if (distance < ans.distance)
        {
            ans = {p, q, ta, tb, distance};
        }
    };
    PointSeg3 acd = pointSeg3(a, c, d);
    PointSeg3 bcd = pointSeg3(b, c, d);
    PointSeg3 cab = pointSeg3(c, a, b);
    PointSeg3 dab = pointSeg3(d, a, b);
    take(a, acd.point, 0, acd.t);
    take(b, bcd.point, 1, bcd.t);
    take(cab.point, c, cab.t, 0);
    take(dab.point, d, dab.t, 1);
    return ans;
}

Real angle(L3 a, L3 b)
{
    // a、b 是两条非退化无向直线；返回范围 [0,pi/2] 的较小夹角弧度。
    assert(norm(a.v) != 0 && norm(b.v) != 0); // 调试检查，可删。
```

```
    Real x = fabsl(dot(a.v, b.v) / abs(a.v) / abs(b.v));
    return acosl(clamp(x, Real(0), Real(1)));
}

Real angle(L3 a, Plane b)
{
    // a 是非退化直线，b 是非退化平面；返回范围 [0,pi/2] 的夹角弧度。
    assert(norm(a.v) != 0 && norm(b.n) != 0); // 调试检查，可删。
    Real x = fabsl(dot(a.v, b.n) / abs(a.v) / abs(b.n));
    return asinl(clamp(x, Real(0), Real(1)));
}

Real angle(Plane a, Plane b)
{
    // a、b 是两个非退化无向平面；返回范围 [0,pi/2] 的较小夹角弧度。
    assert(norm(a.n) != 0 && norm(b.n) != 0); // 调试检查，可删。
    Real x = fabsl(dot(a.n, b.n) / abs(a.n) / abs(b.n));
    return acosl(clamp(x, Real(0), Real(1)));
}

optional<P3> planesMeet3(Plane a, Plane b, Plane c)
{
    // a、b、c 是三个非退化平面；返回唯一公共点，不唯一或不存在时返回空。
    assert(norm(a.n) != 0 && norm(b.n) != 0 && norm(c.n) != 0); // 调试检查，可删。
    Real det = dot(a.n, cross(b.n, c.n));
    if (det == 0)
    {
        return nullopt;
    }
    Real x = dot(a.n, a.p);
    Real y = dot(b.n, b.p);
    Real z = dot(c.n, c.p);
    P3 num = cross(b.n, c.n) * x + cross(c.n, a.n) * y + cross(a.n, b.n) * z;
    return num * (1 / det);
}
```

调用 `projection`、`pointSeg3`、`lineLine3` 或 `segSeg3` 取得投影与最近点信息；两个线段接口都允许端点重合，此时退化段按单点处理。`angle` 的三个重载返回无向较小夹角，`planesMeet3` 仅在三平面有唯一公共点时返回值。所有接口均为 $O(1)$ 时间和 $O(1)$ 额外空间；投影与夹角接口要求相应直线、平面非退化，只求标量距离或向量夹角时直接使用 11.6.004 点线面距离与夹角 更短。

11.6.011 射线与 AABB 查询

轴对齐包围盒可在三个坐标轴上分别解参数不等式，再把三个闭区间与查询参数域取交；射线使用 $[0,\infty)$，线段使用 $[0,1]$，因此盒内射线起点的进入参数固定为 0。平面查询同样只接受非负参数。本节的射线接口显式允许零方向，但此时 `rayPlane3` 和 `rayAABB3` 都返回 `RayState3::Degenerate`，不把静止点当作普通射线命中。

```
// 依赖 11.6.001 三维点、向量、直线、平面
// 依赖 11.6.002 共线、共面、平行、垂直
struct Ray3
{
    P3 p; // p 是射线起点。
    P3 v; // v 是射线方向，查询函数允许零方向并返回 Degenerate。
};

struct AABB3
{
    P3 lo; // lo 是三个坐标的闭区间下界。
    P3 hi; // hi 是三个坐标的闭区间上界。
};

struct ParamPoint3
{
    Real t;   // t 是参数式 p+v*t 中的参数。
    P3 point; // point 是参数 t 对应的空间点。
};

struct ParamRange3
{
    Real enter; // enter 是命中闭集的最小参数。
    Real exit;  // exit 是离开闭集的最大参数。
    P3 first;   // first 是 enter 对应的点。
    P3 last;    // last 是 exit 对应的点。
};

enum class RayState3
{
    None,
    Hit,
    Degenerate
};

struct RayPtQuery3
{
    RayState3 state; // state 表示未命中、命中或零方向退化。
    ParamPoint3 hit;      // hit 只在 state=Hit 时有效。
};

struct RayIntQuery3
{
    RayState3 state; // state 表示未命中、命中或零方向退化。
    ParamRange3 hit;   // hit 只在 state=Hit 时有效。
};

bool in(AABB3 b, P3 p)
{
    // b 是合法闭包围盒，p 是查询点；返回 p 是否在盒内或边界上。
    assert(b.lo.x <= b.hi.x && b.lo.y <= b.hi.y && b.lo.z <= b.hi.z); // 调试检查，可删。
    auto inside = [&](Real x, Real l, Real r)
```

```
    {
        return (x > l || eq(x, l)) && (x < r || eq(x, r));
    };
    return inside(p.x, b.lo.x, b.hi.x) && inside(p.y, b.lo.y, b.hi.y) && inside(p.z, b.lo.z,
    ↪ b.hi.z);
}

optional<ParamRange3> aabbRange3(P3 p, P3 v, AABB3 b, Real lower, Real upper)
{
    // p+v*t 是参数直线，b 是合法闭包围盒，[lower,upper] 是参数域；返回命中参数区间，未命中返回空。
    assert(lower <= upper);                                                    // 调试检查，可删。
    assert(b.lo.x <= b.hi.x && b.lo.y <= b.hi.y && b.lo.z <= b.hi.z); // 调试检查，可删。
    array<Real, 3> origin{p.x, p.y, p.z};
    array<Real, 3> dir{v.x, v.y, v.z};
    array<Real, 3> lo{b.lo.x, b.lo.y, b.lo.z};
    array<Real, 3> hi{b.hi.x, b.hi.y, b.hi.z};
    if (v.x == 0 && v.y == 0 && v.z == 0)
    {
        if (!in(b, p))
        {
            return nullopt;
        }
        return ParamRange3{lower, upper, p, p};
    }
    Real enter = lower;
    Real exit = upper;
    for (int i = 0; i < 3; i++)
    {
        if (dir[i] == 0)
        {
            bool outside = (origin[i] < lo[i] && !eq(origin[i], lo[i])) || (origin[i] > hi[i]
            ↪ && !eq(origin[i], hi[i]));
            if (outside)
            {
                return nullopt;
            }
            continue;
        }
        Real x = (lo[i] - origin[i]) / dir[i];
        Real y = (hi[i] - origin[i]) / dir[i];
        if (x > y)
        {
            swap(x, y);
        }
        enter = max(enter, x);
        exit = min(exit, y);
        if (enter > exit && !eq(enter, exit))
        {
            return nullopt;
        }
    }
    if (enter > exit)
```

```
    {
        Real t = (enter + exit) / 2;
        enter = exit = t;
    }
    return ParamRange3{enter, exit, p + v * enter, p + v * exit};
}

RayPtQuery3 rayPlane3(Ray3 r, Plane s)
{
    // r 是可退化射线，s 是非退化平面；返回闭平面的首次非负命中。
    assert(sgn(abs(s.n)) != 0); // 调试检查，可删。
    if (r.v.x == 0 && r.v.y == 0 && r.v.z == 0)
    {
        return {RayState3::Degenerate, {}};
    }
    Real d = dot(r.v, s.n);
    if (d == 0)
    {
        if (onPlane3(r.p, s))
        {
            return {RayState3::Hit, {0, r.p}};
        }
        return {RayState3::None, {}};
    }
    Real t = dot(s.p - r.p, s.n) / d;
    if (t < 0)
    {
        return {RayState3::None, {}};
    }
    return {RayState3::Hit, {t, r.p + r.v * t}};
}

RayIntQuery3 rayAABB3(Ray3 r, AABB3 b)
{
    // r 是可退化射线，b 是合法闭包围盒；返回盒内的非负参数区间。
    assert(b.lo.x <= b.hi.x && b.lo.y <= b.hi.y && b.lo.z <= b.hi.z); // 调试检查，可删。
    if (r.v.x == 0 && r.v.y == 0 && r.v.z == 0)
    {
        return {RayState3::Degenerate, {}};
    }
    auto hit = aabbRange3(r.p, r.v, b, 0, numeric_limits<Real>::infinity());
    if (!hit)
    {
        return {RayState3::None, {}};
    }
    return {RayState3::Hit, *hit};
}

optional<ParamRange3> segmentAABB3(P3 a, P3 b, AABB3 box)
{
    // a、b 是闭线段端点，box 是合法闭包围盒；返回参数域 [0,1] 内的命中区间，未命中返回空。
    P3 v = b - a;
```

```
    if (v.x == 0 && v.y == 0 && v.z == 0)
    {
        if (!in(box, a))
        {
            return nullopt;
        }
        return ParamRange3{0, 1, a, a};
    }
    return aabbRange3(a, v, box, 0, 1);
}

bool sphereAABB3(Sphere s, AABB3 b)
{
    // s 是非负半径闭球，b 是合法闭包围盒；返回两个闭集是否相交。
    assert(sgn(s.r) >= 0);                                          // 调试检查，可删。
    assert(b.lo.x <= b.hi.x && b.lo.y <= b.hi.y && b.lo.z <= b.hi.z); // 调试检查，可删。
    P3 q{clamp(s.o.x, b.lo.x, b.hi.x), clamp(s.o.y, b.lo.y, b.hi.y), clamp(s.o.z, b.lo.z,
    ↪ b.hi.z)};
    return in(b, q) && in(s, q);
}
```

`RayState3::None`、`Hit`、`Degenerate` 分别表示未命中、命中和零方向射线；射线与平面共面时首次命中为 $t = 0$。`aabbRange3` 与 `segmentAABB3` 未命中时返回 `nullopt`，退化线段若位于盒内则返回完整参数域 $[0, 1]$；`sphereAABB3` 通过逐坐标夹取得盒内最近点判断闭球与闭盒是否相交。包围盒必须满足各维下界不大于上界，球半径必须非负。所有查询均为 $O(1)$ 时间和 $O(1)$ 额外空间，并依赖 11.6.007 最小覆盖球 的 `Sphere` 及 11.6.010 投影、最近点与空间距离 的基础接口。

11.6.012 空间三角形与平面裁剪

空间三角形先以法向量确定承载平面，再用同向边叉积判断闭集包含关系；最近点若投影落在内部就用重心坐标表示，否则比较三条边，三角形与平面之交则按顶点符号分类为空、点、线段或整三角形。除仅计算面积的接口外，三角形参数必须非退化；`clipPolygon3` 只支持顶点有序的共面凸多边形，并保留裁剪平面法向正侧的闭半空间。

```
// 依赖 11.6.001 三维点、向量、直线、平面
// 依赖 11.6.002 共线、共面、平行、垂直
// 依赖 11.6.004 点线面距离与夹角
// 依赖 11.6.010 投影、最近点与空间距离
// 依赖 11.6.011 射线与 AABB 查询
enum class TriPlaneType
{
    Empty,
    Point,
    Segment,
    Triangle
};

struct TriPlaneRes3
{
```

```
    TriPlaneType type; // type 是空、点、线段或整三角形四类交集。
    vector<P3> points;       // points 按 type 保存 0、1、2 或 3 点；整三角形保持输入序，线段点按三条边枚举序。
};

struct TriClose3
{
    P3 point;                   // point 是三角形闭集上的最近点。
    array<Real, 3> barycentric; // barycentric 依次是三个顶点的重心坐标。
    Real distance;              // distance 是查询点到三角形的距离。
};

bool inTri3(P3 p, P3 a, P3 b, P3 c)
{
    // p 是查询点，a、b、c 是非退化三角形；返回 p 是否在三角形闭集内。
    P3 n = cross(b - a, c - a);
    assert(sgn(abs(n)) != 0); // 调试检查，可删。
    if (!onPlane3(p, {a, n}))
    {
        return false;
    }
    Real x = dot(cross(b - a, p - a), n) / abs(b - a) / abs(n);
    Real y = dot(cross(c - b, p - b), n) / abs(c - b) / abs(n);
    Real z = dot(cross(a - c, p - c), n) / abs(a - c) / abs(n);
    return sgn(x) >= 0 && sgn(y) >= 0 && sgn(z) >= 0;
}

TriClose3 pointTri3(P3 p, P3 a, P3 b, P3 c)
{
    // p 是查询点，a、b、c 是非退化三角形；返回最近点、重心坐标和距离。
    P3 u = b - a;
    P3 v = c - a;
    P3 n = cross(u, v);
    assert(sgn(abs(n)) != 0); // 调试检查，可删。
    P3 q = projection(Plane{a, n}, p);
    if (inTri3(q, a, b, c))
    {
        P3 w = q - a;
        Real uu = norm(u);
        Real uv = dot(u, v);
        Real vv = norm(v);
        Real uw = dot(u, w);
        Real vw = dot(v, w);
        Real d = uu * vv - uv * uv;
        Real y = (vv * uw - uv * vw) / d;
        Real z = (uu * vw - uv * uw) / d;
        return {q, {1 - y - z, y, z}, abs(p - q)};
    }
    PointSeg3 ab = pointSeg3(p, a, b);
    PointSeg3 bc = pointSeg3(p, b, c);
    PointSeg3 ca = pointSeg3(p, c, a);
    TriClose3 ans{ab.point, {1 - ab.t, ab.t, 0}, ab.distance};
```

```
    if (bc.distance < ans.distance)
    {
        ans = {bc.point, {0, 1 - bc.t, bc.t}, bc.distance};
    }
    if (ca.distance < ans.distance)
    {
        ans = {ca.point, {ca.t, 0, 1 - ca.t}, ca.distance};
    }
    return ans;
}

TriPlaneRes3 triPlane3(P3 a, P3 b, P3 c, Plane s)
{
    // a、b、c 是非退化三角形，s 是非退化平面；返回完整交集分类与顶点。
    assert(sgn(abs(cross(b - a, c - a))) != 0 && sgn(abs(s.n)) != 0); // 调试检查，可删。
    array<P3, 3> p{a, b, c};
    array<Real, 3> d;
    array<int, 3> side;
    for (int i = 0; i < 3; i++)
    {
        d[i] = dot(p[i] - s.p, s.n) / abs(s.n);
        side[i] = sgn(d[i]);
    }
    if (side[0] == 0 && side[1] == 0 && side[2] == 0)
    {
        return {TriPlaneType::Triangle, {a, b, c}};
    }
    vector<P3> out;
    auto add = [&](P3 q)
    {
        for (P3 x : out)
        {
            if (sgn(abs(x - q)) == 0)
            {
                return;
            }
        }
        out.push_back(q);
    };
    for (int i = 0; i < 3; i++)
    {
        int j = (i + 1) % 3;
        if (side[i] == 0)
        {
            add(p[i]);
        }
        if (side[i] * side[j] < 0)
        {
            add(p[i] + (p[j] - p[i]) * (d[i] / (d[i] - d[j])));
        }
    }
    if (out.empty())
```

```
    {
        return {TriPlaneType::Empty, {}};
    }
    if (out.size() == 1)
    {
        return {TriPlaneType::Point, out};
    }
    assert(out.size() == 2); // 调试检查，可删。
    return {TriPlaneType::Segment, out};
}

vector<P3> clipPolygon3(const vector<P3> &poly, Plane s)
{
    // poly 是有序共面凸多边形，s 是非退化裁剪平面；返回法向正侧闭半空间内的顶点序列。
    assert(sgn(abs(s.n)) != 0); // 调试检查，可删。
    if (poly.empty())
    {
        return {};
    }
    vector<P3> out;
    int n = (int)poly.size();
    for (int i = 0; i < n; i++)
    {
        P3 a = poly[i];
        P3 b = poly[(i + 1) % n];
        Real x = dot(a - s.p, s.n) / abs(s.n);
        Real y = dot(b - s.p, s.n) / abs(s.n);
        int sx = sgn(x);
        int sy = sgn(y);
        if (sx >= 0)
        {
            out.push_back(a);
        }
        if (sx * sy < 0)
        {
            out.push_back(a + (b - a) * (x / (x - y)));
        }
    }
    vector<P3> clean;
    for (P3 p : out)
    {
        if (clean.empty() || sgn(abs(clean.back() - p)) != 0)
        {
            clean.push_back(p);
        }
    }
    if (clean.size() > 1 && sgn(abs(clean.front() - clean.back())) == 0)
    {
        clean.pop_back();
    }
    return clean;
}
```

```

Real triArea3(P3 a, P3 b, P3 c)
{
    // a、b、c 是三角形顶点；返回无符号面积，退化时返回 0。
    return abs(cross(b - a, c - a)) / 2;
}

Real polygonArea3(const vector<P3> &poly)
{
    // poly 是顶点有序的共面简单多边形；返回无符号面积，少于三个点时返回 0。
    if (poly.size() < 3)
    {
        return 0;
    }
    P3 sum{};
    for (int i = 1; i + 1 < (int)poly.size(); i++)
    {
        sum = sum + cross(poly[i] - poly[0], poly[i + 1] - poly[0]);
    }
    return abs(sum) / 2;
}

RayPtQuery3 rayTriangle3(Ray3 r, P3 a, P3 b, P3 c)
{
    // r 是可退化射线，a、b、c 是非退化三角形；返回三角形闭集的首次命中。
    P3 n = cross(b - a, c - a);
    assert(sgn(abs(n)) != 0); // 调试检查，可删。
    if (r.v.x == 0 && r.v.y == 0 && r.v.z == 0)
    {
        return {RayState3::Degenerate, {}};
    }
    Real d = dot(r.v, n);
    if (d != 0)
    {
        Real t = dot(a - r.p, n) / d;
        if (t < 0)
        {
            return {RayState3::None, {}};
        }
        P3 p = r.p + r.v * t;
        if (!inTri3(p, a, b, c))
        {
            return {RayState3::None, {}};
        }
        return {RayState3::Hit, {t, p}};
    }
    if (!onPlane3(r.p, {a, n}))
    {
        return {RayState3::None, {}};
    }
    array<P3, 3> p{a, b, c};
    Real enter = 0;
```

```
    Real exit = numeric_limits<Real>::infinity();
    for (int i = 0; i < 3; i++)
    {
        int j = (i + 1) % 3;
        P3 edge = p[j] - p[i];
        Real scale = abs(edge) * abs(n);
        Real x = dot(cross(edge, r.p - p[i]), n) / scale;
        Real y = dot(cross(edge, r.v), n) / scale;
        if (y == 0)
        {
            if (sgn(x) < 0)
            {
                return {RayState3::None, {}};
            }
            continue;
        }
        Real t = -x / y;
        if (y > 0)
        {
            enter = max(enter, t);
        }
        else
        {
            exit = min(exit, t);
        }
        if (enter > exit && !eq(enter, exit))
        {
            return {RayState3::None, {}};
        }
    }
    return {RayState3::Hit, {enter, r.p + r.v * enter}};
}
```

调用 `inTri3`、`pointTri3`、`triPlane3` 和 `rayTriangle3` 完成常数规模查询；零方向射线返回 `RayState3::Degenerate`。`polygonArea3` 只接受顶点有序的共面简单多边形。`triArea3` 为 $O(1)$，含 m 个顶点的 `clipPolygon3` 与 `polygonArea3` 均为 $O(m)$ 时间，前者使用 $O(m)$、后者使用 $O(1)$ 额外空间，其余接口均为 $O(1)$ 时间和空间；射线首次命中沿用 11.6.011 射线与 AABB 查询，只判断闭线段与三角形是否相交可用 11.6.005 空间线段与三角形相交。

11.6.013 球、空间圆与球冠 / 球带

直线与球面之交先投影球心并由弦长恢复至多两个参数，平面与球面之交返回空间圆，两球关系则由球心距和半径和、半径差分类；球冠与球带使用旋转体截面积分公式。所有 `Sphere`、`Circle3` 及面积体积接口都要求半径非负；`raySphere3` 只保留非负参数，所以射线从球内出发时只返回出口交点。

```
// 依赖 11.6.001 三维点、向量、直线、平面
// 依赖 11.6.002 共线、共面、平行、垂直
// 依赖 11.6.004 点线面距离与夹角
// 依赖 11.6.010 投影、最近点与空间距离
```

```cpp
// 依赖 11.6.011 射线与 AABB 查询
struct Circle3
{
    P3 o;   // o 是空间圆圆心。
    P3 n;   // n 是圆所在平面的非零法向量。
    Real r; // r 是非负半径，0 表示单点圆。
};

enum class SphereRel3
{
    Separate,
    ExtTangent,
    Intersecting,
    IntTangent,
    Contained,
    Coincident
};

struct SpherePair3
{
    SphereRel3 relation; // relation 是两个球面的相对关系。
    optional<Circle3> circle; // circle 在相切或相交时保存交圆，相切时半径为 0。
};

struct RayPtsQuery3
{
    RayState3 state;     // state 表示未命中、命中或零方向退化。
    vector<ParamPoint3> hits; // hits 在命中时按非负参数递增保存球面交点。
};

vector<ParamPoint3> lineSphere3(L3 l, Sphere s)
{
    // l 是非退化直线，s 是非负半径球；返回按参数递增的 0 至 2 个球面交点。
    assert((l.v.x != 0 || l.v.y != 0 || l.v.z != 0) && s.r >= 0); // 调试检查，可删。
    Real vv = norm(l.v);
    Real middle = dot(s.o - l.p, l.v) / vv;
    P3 foot = l.p + l.v * middle;
    Real d2 = norm(foot - s.o);
    Real radius2 = s.r * s.r;
    Real tol = eps * max(d2, radius2);
    if (d2 > radius2 + tol)
    {
        return {};
    }
    Real chord2 = radius2 - d2;
    if (fabsl(chord2) <= tol)
    {
        return {{middle, foot}};
    }
    Real half = sqrtl(max(Real(0), chord2) / vv);
    Real x = middle - half;
    Real y = middle + half;
```

```
    return {{x, l.p + l.v * x}, {y, l.p + l.v * y}};
}

RayPtsQuery3 raySphere3(Ray3 r, Sphere s)
{
    // r 是可退化射线，s 是非负半径球；返回所有非负参数的球面交点。
    assert(s.r >= 0); // 调试检查，可删。
    if (r.v.x == 0 && r.v.y == 0 && r.v.z == 0)
    {
        return {RayState3::Degenerate, {}};
    }
    vector<ParamPoint3> out;
    for (ParamPoint3 hit : lineSphere3({r.p, r.v}, s))
    {
        if (hit.t < 0)
        {
            continue;
        }
        out.push_back(hit);
    }
    if (out.empty())
    {
        return {RayState3::None, {}};
    }
    return {RayState3::Hit, out};
}

optional<Circle3> planeSphere3(Plane p, Sphere s)
{
    // p 是非退化平面，s 是非负半径球；返回交圆，相离时返回空，相切时半径为 0。
    assert(sgn(abs(p.n)) != 0 && sgn(s.r) >= 0); // 调试检查，可删。
    Real h = dot(s.o - p.p, p.n) / abs(p.n);
    if (fabsl(h) > s.r && !eq(fabsl(h), s.r))
    {
        return nullopt;
    }
    P3 o = projection(p, s.o);
    Real r2 = max(Real(0), s.r * s.r - h * h);
    return Circle3{o, unit(p.n), sqrtl(r2)};
}

SpherePair3 spherePair3(Sphere a, Sphere b)
{
    // a、b 是两个非负半径球；返回球面关系及相切或相交时的交圆。
    assert(sgn(a.r) >= 0 && sgn(b.r) >= 0); // 调试检查，可删。
    Real d = abs(b.o - a.o);
    Real sum = a.r + b.r;
    Real diff = fabsl(a.r - b.r);
    if (eq(d, Real(0)) && eq(a.r, b.r))
    {
        return {SphereRel3::Coincident, nullopt};
    }
```

```
    if (d < diff && !eq(d, diff))
    {
        return {SphereRel3::Contained, nullopt};
    }
    SphereRel3 relation;
    if (eq(d, diff))
    {
        relation = SphereRel3::IntTangent;
    }
    else if (d > sum && !eq(d, sum))
    {
        return {SphereRel3::Separate, nullopt};
    }
    else if (eq(d, sum))
    {
        relation = SphereRel3::ExtTangent;
    }
    else
    {
        relation = SphereRel3::Intersecting;
    }
    P3 u = (b.o - a.o) / d;
    Real x = (d * d + a.r * a.r - b.r * b.r) / (2 * d);
    Real r2 = max(Real(0), a.r * a.r - x * x);
    return {relation, Circle3{a.o + u * x, u, sqrtl(r2)}};
}

vector<ParamPoint3> lineCircle3(L3 l, Circle3 c)
{
    // l 是非退化直线，c 是法向非零的空间圆；返回按直线参数递增的交点。
    assert((l.v.x != 0 || l.v.y != 0 || l.v.z != 0) && sgn(abs(c.n)) != 0 && c.r >= 0); // 调试
    ↪ 检查，可删。
    Real d = dot(l.v, c.n);
    if (d != 0)
    {
        Real t = dot(c.o - l.p, c.n) / d;
        P3 p = l.p + l.v * t;
        Real d2 = norm(p - c.o);
        Real radius2 = c.r * c.r;
        if (fabsl(d2 - radius2) > eps * max(d2, radius2))
        {
            return {};
        }
        return {{t, p}};
    }
    if (sgn(dot(l.p - c.o, c.n) / abs(c.n)) != 0)
    {
        return {};
    }
    return lineSphere3(l, {c.o, c.r});
}

```

```
Real sphereLens3(Sphere a, Sphere b)
{
    // a、b 是两个非负半径闭球；返回二者公共部分体积。
    assert(sgn(a.r) >= 0 && sgn(b.r) >= 0); // 调试检查，可删。
    Real d = abs(b.o - a.o);
    Real sum = a.r + b.r;
    Real diff = fabsl(a.r - b.r);
    if (eq(d, Real(0)) || d < diff || eq(d, diff))
    {
        return sphereVolume(min(a.r, b.r), 3);
    }
    if (d > sum || eq(d, sum))
    {
        return 0;
    }
    const Real pi = acosl(-1);
    Real h = sum - d;
    Real q = d * d + 2 * d * sum - 3 * (a.r - b.r) * (a.r - b.r);
    return pi * h * h * q / (12 * d);
}

Real sphereArea(Real r)
{
    // r 是非负球半径；返回三维球面面积。
    assert(r >= 0); // 调试检查，可删。
    return 4 * acosl(-1) * r * r;
}

Real capArea(Real r, Real h)
{
    // r 是非负球半径，h 是范围 [0,2r] 的球冠高；返回球冠曲面面积。
    assert(r >= 0 && (h > 0 || eq(h, Real(0))) && (h < 2 * r || eq(h, 2 * r))); // 调试检查，可删。
    h = clamp(h, Real(0), 2 * r);
    return max(Real(0), 2 * acosl(-1) * r * h);
}

Real capVolume(Real r, Real h)
{
    // r 是非负球半径，h 是范围 [0,2r] 的球冠高；返回球冠体积。
    assert(r >= 0 && (h > 0 || eq(h, Real(0))) && (h < 2 * r || eq(h, 2 * r))); // 调试检查，可删。
    h = clamp(h, Real(0), 2 * r);
    return max(Real(0), acosl(-1) * h * h * (r - h / 3));
}

Real zoneArea(Real r, Real low, Real high)
{
    // r 是非负球半径，[low,high] 是相对球心轴坐标区间；返回球带曲面面积。
    assert(r >= 0 && (low > -r || eq(low, -r)) && (low < high || eq(low, high)) &&
           (high < r || eq(high, r))); // 调试检查，可删。
    low = clamp(low, -r, r);
    high = clamp(high, -r, r);
    if (low > high)
```

```
    {
        low = high = (low + high) / 2;
    }
    return max(Real(0), 2 * acosl(-1) * r * (high - low));
}

Real zoneVol3(Real r, Real low, Real high)
{
    // r 是非负球半径，[low,high] 是相对球心轴坐标区间；返回两截面间体积。
    assert(r >= 0 && (low > -r || eq(low, -r)) && (low < high || eq(low, high)) &&
           (high < r || eq(high, r))); // 调试检查，可删。
    low = clamp(low, -r, r);
    high = clamp(high, -r, r);
    if (low > high)
    {
        low = high = (low + high) / 2;
    }
    auto primitive = [&](Real z)
    {
        return r * r * z - z * z * z / 3;
    };
    return max(Real(0), acosl(-1) * (primitive(high) - primitive(low)));
}
```

调用 `lineSphere3`、`raySphere3`、`planeSphere3`、`spherePair3` 或 `lineCircle3` 查询边界交集；球冠高满足 $0 \le h \le 2r$，球带轴坐标满足 $-r \le low \le high \le r$。`sphereLens3` 返回两闭球公共体积，其余公式接口分别返回球面、球冠或球带的面积与体积。所有接口均为 $O(1)$ 时间和 $O(1)$ 额外空间，交点向量至多含两个元素；实现依赖 11.6.009 高维球体积、超球坐标与立体角 的球体积及 11.6.011 射线与 AABB 查询 的射线结果类型。

11.6.014 共面基底、凸包与网格截面

非退化平面可构造右手正交基，把共面点投到二维后复用 11.3.001 二维凸包；闭合三角网格的体积由各有向面与固定原点组成的四面体代数体积求和，截面则把同一网格边上的交点按共享边编号合并并沿面定向连成环。网格必须是闭合、方向一致、共享顶点使用同一编号的二流形；截平面必须处于一般位置，经过网格顶点、边或面时 `meshSection3` 返回 `nullopt`。

```
// 依赖 11.6.001 三维点、向量、直线、平面
// 依赖 11.6.002 共线、共面、平行、垂直
struct PlaneBasis3
{
    P3 o; // o 是局部二维坐标原点。
    P3 u; // u 是局部 x 轴单位向量。
    P3 v; // v 是局部 y 轴单位向量。
    P3 n; // n 是满足 cross(u,v)=n 的单位法向量。
};

struct MeshLoop3
{
```

```
    vector<P3> points; // points 是不重复首点的有向截面环。
    Real area;   // area 是沿截平面法向观察的有向面积。
    Real perimeter;    // perimeter 是截面环周长。
};

PlaneBasis3 planeBasis3(Plane s)
{
    // s 是非退化平面；返回以 s.p 为原点的右手正交基。
    assert(sgn(abs(s.n)) != 0); // 调试检查，可删。
    P3 n = unit(s.n);
    P3 axis;
    if (fabsl(n.x) <= fabsl(n.y) && fabsl(n.x) <= fabsl(n.z))
    {
        axis = {1, 0, 0};
    }
    else if (fabsl(n.y) <= fabsl(n.z))
    {
        axis = {0, 1, 0};
    }
    else
    {
        axis = {0, 0, 1};
    }
    P3 u = unit(cross(axis, n));
    P3 v = cross(n, u);
    return {s.p, u, v, n};
}

P planeCoord3(PlaneBasis3 b, P3 p)
{
    // b 是右手正交平面基，p 是空间点；返回 p 的局部二维投影坐标。
    P3 q = p - b.o;
    return {dot(q, b.u), dot(q, b.v)};
}

P3 planePoint3(PlaneBasis3 b, P p)
{
    // b 是右手正交平面基，p 是局部二维坐标；返回对应的空间平面点。
    return b.o + b.u * p.x + b.v * p.y;
}

vector<P3> coplanarHull(const vector<P3> &points, Plane s)
{
    // points 是 s 上的点集，s 是非退化平面；返回沿 s.n 观察逆时针的凸包顶点。
    PlaneBasis3 basis = planeBasis3(s);
    vector<P> projected;
    map<P, int> original;
    for (int i = 0; i < (int)points.size(); i++)
    {
        assert(onPlane3(points[i], s)); // 调试检查，可删。
        P q = planeCoord3(basis, points[i]);
        projected.push_back(q);
```

```
        if (!original.count(q))
        {
            original[q] = i;
        }
    }
    vector<P3> out;
    for (P q : hull(projected))
    {
        out.push_back(points[original[q]]);
    }
    return out;
}

Real meshVolume3(const vector<P3> &points, const vector<array<int, 3>> &faces)
{
    // points 是网格顶点，faces 每项三维依次是按一致外向顺序的三个顶点下标；返回封闭网格体积。
    if (faces.empty())
    {
        return 0;
    }
    assert(!points.empty() && 0 <= faces[0][0] && faces[0][0] < (int)points.size()); // 调试检
    ↪ 查，可删。
    P3 o = points[faces[0][0]];
    Real sum = 0;
    for (auto f : faces)
    {
        for (int x : f)
        {
            assert(0 <= x && x < (int)points.size()); // 调试检查，可删。
        }
        P3 a = points[f[0]] - o;
        P3 b = points[f[1]] - o;
        P3 c = points[f[2]] - o;
        assert(sgn(abs(cross(b - a, c - a))) != 0); // 调试检查，可删。
        sum += dot(a, cross(b, c));
    }
    return fabsl(sum) / 6;
}

optional<vector<MeshLoop3>> meshSection3(const vector<P3> &points, const vector<array<int, 3>>
↪ &faces, Plane s)
{
    // points 是网格顶点，faces 每项三维依次是按一致外向顺序的三个顶点下标，s 是截平面；返回截面环，退
    ↪ 化时返回空。
    assert(sgn(abs(s.n)) != 0); // 调试检查，可删。
    vector<Real> distance(points.size());
    vector<int> side(points.size());
    for (int i = 0; i < (int)points.size(); i++)
    {
        distance[i] = dot(points[i] - s.p, s.n) / abs(s.n);
        side[i] = sgn(distance[i]);
        if (side[i] == 0)
```

```
        {
            return nullopt;
        }
    }
    map<pair<int, int>, int> id; // id 把无向网格边映射到唯一截面点编号。
    vector<P3> nodePoint;        // nodePoint 保存各截面点的空间坐标。
    vector<int> next;            // next 保存有向截面环中的后继点编号。
    vector<int> indegree;        // indegree 保存各截面点在有向截面图中的入度。
    auto getNode = [&](int a, int b)
    {
        pair<int, int> edge = minmax(a, b);
        auto it = id.find(edge);
        if (it != id.end())
        {
            return it->second;
        }
        Real t = distance[a] / (distance[a] - distance[b]);
        int z = (int)nodePoint.size();
        id[edge] = z;
        nodePoint.push_back(points[a] + (points[b] - points[a]) * t);
        next.push_back(-1);
        indegree.push_back(0);
        return z;
    };
    for (auto f : faces)
    {
        for (int x : f)
        {
            assert(0 <= x && x < (int)points.size()); // 调试检查，可删。
        }
        P3 fn = cross(points[f[1]] - points[f[0]], points[f[2]] - points[f[0]]);
        if (sgn(abs(fn)) == 0)
        {
            return nullopt;
        }
        vector<pair<int, int>> cut;
        for (int i = 0; i < 3; i++)
        {
            int a = f[i];
            int b = f[(i + 1) % 3];
            if (side[a] * side[b] < 0)
            {
                cut.push_back({a, b});
            }
        }
        if (cut.empty())
        {
            continue;
        }
        if (cut.size() != 2)
        {
            return nullopt;
```

```
        }
        int a = getNode(cut[0].first, cut[0].second);
        int b = getNode(cut[1].first, cut[1].second);
        P3 dir = cross(s.n, fn);
        Real turn = dot(nodePoint[b] - nodePoint[a], dir);
        if (sgn(turn / abs(nodePoint[b] - nodePoint[a]) / abs(dir)) == 0)
        {
            return nullopt;
        }
        if (turn < 0)
        {
            swap(a, b);
        }
        if (next[a] != -1 || indegree[b] != 0)
        {
            return nullopt;
        }
        next[a] = b;
        indegree[b]++;
    }
    for (int i = 0; i < (int)nodePoint.size(); i++)
    {
        if (next[i] == -1 || indegree[i] != 1)
        {
            return nullopt;
        }
    }
    PlaneBasis3 basis = planeBasis3(s);
    vector<char> used(nodePoint.size());
    vector<MeshLoop3> out;
    for (int start = 0; start < (int)nodePoint.size(); start++)
    {
        if (used[start])
        {
            continue;
        }
        vector<P3> loop;
        int cur = start;
        while (!used[cur])
        {
            used[cur] = true;
            loop.push_back(nodePoint[cur]);
            cur = next[cur];
        }
        if (cur != start || loop.size() < 3)
        {
            return nullopt;
        }
        Real area2 = 0;
        Real perimeter = 0;
        for (int i = 1; i + 1 < (int)loop.size(); i++)
        {
```

```
            area2 += dot(cross(loop[i] - loop[0], loop[i + 1] - loop[0]), basis.n);
        }
        for (int i = 0; i < (int)loop.size(); i++)
        {
            int j = (i + 1) % loop.size();
            perimeter += abs(loop[j] - loop[i]);
        }
        out.push_back({loop, area2 / 2, perimeter});
    }
    return out;
}
```

调用 `planeCoord3` 与 `planePoint3` 在局部二维坐标和空间点之间转换，`coplanarHull` 返回沿给定法向观察的逆时针凸包；`meshSection3` 的空 `optional` 表示退化输入，含空向量的有效结果表示平面未切中网格。平面基与坐标转换为 $O(1)$ 时间和空间；设点数、面数和参与截面的边数分别为 V、F、E，共面凸包为 $O(V\log V)$ 时间和 $O(V)$ 额外空间，体积为 $O(F)$ 时间和 $O(1)$ 额外空间，截面为 $O(V+F\log E)$ 时间和 $O(V+E)$ 额外空间。本节依赖 11.3.001 Andrew / Graham 静态凸包 和 11.6.012 空间三角形与平面裁剪。

11.6.015 反射、二面角与球面多边形

点 p 关于平面的反射是 $p'=2\,\mathrm{projection}(p)-p$；有向二面角把两个面法向投到公共边的正交补，再按公共边右手方向从第一投影转向第二投影，两个投影都必须非零。对单位球面方向 a,b,c，令 $y=a\cdot(b\times c)$、$x=1+a\cdot b+b\cdot c+c\cdot a$，短大圆弧三角形的立体角为 $\Omega=2\,\mathrm{atan2}(y,x)$；一般简单球面多边形由 Gauss–Bonnet 得到 $A/R^2\equiv 2\pi-\sum_i\tau_i\pmod{4\pi}$，其中 τ_i 是有向顶点转角。顶点必须位于同一正半径球面，边取短大圆弧且相邻点不得对跖；允许非凸和跨越半球，但不允许自交或把边替成长大圆弧。返回模 $4\pi R^2$ 的主值有向面积，范围为 $(-2\pi R^2,2\pi R^2]$。

```
// 依赖 11.6.001 三维点、向量、直线、平面
// 依赖 11.6.004 点线面距离与夹角
// 依赖 11.6.010 投影、最近点与空间距离
P3 reflection(Plane s, P3 p)
{
    // s 是非退化平面，p 是查询点；返回 p 关于 s 的镜像点。
    P3 q = projection(s, p);
    return q * 2 - p;
}

Real signedDih3(P3 edge, P3 na, P3 nb)
{
    // edge 是非零有向边，两法向的正交投影非零；返回绕 edge 按右手方向从 A 到 B 的 [-pi,pi] 角。
    assert(sgn(abs(edge)) != 0 && sgn(abs(na)) != 0 && sgn(abs(nb)) != 0); // 调试检查，可删。
    P3 e = unit(edge);
    P3 a = na - e * dot(na, e);
    P3 b = nb - e * dot(nb, e);
    assert(sgn(abs(a)) != 0 && sgn(abs(b)) != 0); // 调试检查，可删。
    return atan2l(dot(e, cross(a, b)), dot(a, b));
}
```

```
21
22  Real dihedral3(P3 edge, P3 na, P3 nb)
23  {
24      // edge 是非零公共边，na、nb 是两面非零法向；返回范围 [0,pi] 的无向二面角。
25      return fabsl(signedDih3(edge, na, nb));
26  }
27
28  P3 fromLatLon3(Real lat, Real lon, Real r = 1)
29  {
30      // lat、lon 是纬度和经度弧度，r 是非负半径；返回球心在原点的球面点。
31      const Real pi = acosl(-1);
32      assert((lat > -pi / 2 || eq(lat, -pi / 2)) && (lat < pi / 2 || eq(lat, pi / 2)) &&
33             r >= 0); // 调试检查，可删。
34      Real z = cosl(lat);
35      return {r * z * cosl(lon), r * z * sinl(lon), r * sinl(lat)};
36  }
37
38  pair<Real, Real> toLatLon3(P3 p)
39  {
40      // p 是以原点为球心的非零球面点；返回纬度与范围 [-pi,pi] 的经度弧度。
41      assert(p.x != 0 || p.y != 0 || p.z != 0); // 调试检查，可删。
42      Real xy = hypotl(p.x, p.y);
43      Real lon = xy == 0 ? 0 : atan2l(p.y, p.x);
44      return {atan2l(p.z, xy), lon};
45  }
46
47  Real solidAngle3(P3 a, P3 b, P3 c)
48  {
49      // a、b、c 是非零方向且两两不反向；返回三条短大圆弧围成的主值有向立体角。
50      assert((a.x != 0 || a.y != 0 || a.z != 0) && (b.x != 0 || b.y != 0 || b.z != 0) &&
51             (c.x != 0 || c.y != 0 || c.z != 0)); // 调试检查，可删。
52      auto normalize = [&](P3 p)
53      {
54          Real len = abs(p);
55          return P3{p.x / len, p.y / len, p.z / len};
56      };
57      a = normalize(a);
58      b = normalize(b);
59      c = normalize(c);
60      assert(norm(a + b) != 0 && norm(b + c) != 0 && norm(c + a) != 0); // 调试检查，可删。
61      Real y = dot(a, cross(b, c));
62      Real x = 1 + dot(a, b) + dot(b, c) + dot(c, a);
63      return 2 * atan2l(y, x);
64  }
65
66  Real spherePoly3(const vector<P3> &poly, Sphere s)
67  {
68      // poly 是球面简单多边形顶点，s 是正半径球；返回 (-2pi*r^2,2pi*r^2] 内的主值有向面积。
69      assert(s.r > 0); // 调试检查，可删。
70      auto normalize = [&](P3 p)
71      {
72          Real len = abs(p);
```

```
        return P3{p.x / len, p.y / len, p.z / len};
    };
    vector<P3> dir;
    for (P3 p : poly)
    {
        P3 q = p - s.o;
        Real len = abs(q);
        assert(fabsl(len - s.r) <= eps * max(len, s.r)); // 调试检查，可删。
        P3 u = normalize(q);
        if (dir.empty() || norm(dir.back() - u) != 0)
        {
            dir.push_back(u);
        }
    }
    if (dir.size() > 1 && norm(dir.front() - dir.back()) == 0)
    {
        dir.pop_back();
    }
    if (dir.size() < 3)
    {
        return 0;
    }
    for (int i = 0; i < (int)dir.size(); i++)
    {
        int j = (i + 1) % dir.size();
        assert(norm(dir[i] + dir[j]) != 0); // 调试检查，可删。
    }
    Real turn = 0;
    int n = (int)dir.size();
    for (int i = 0; i < n; i++)
    {
        P3 pre = dir[(i + n - 1) % n];
        P3 cur = dir[i];
        P3 next = dir[(i + 1) % n];
        P3 ni = normalize(cross(pre, cur));
        P3 no = normalize(cross(cur, next));
        P3 vin = cross(ni, cur);
        P3 vout = cross(no, cur);
        turn += atan2l(dot(cur, cross(vin, vout)), dot(vin, vout));
    }
    const Real pi = acosl(-1);
    Real ans = turn > 0 ? 2 * pi - turn : -2 * pi - turn;
    if (turn == 0)
    {
        ans = 2 * pi;
    }
    while (ans <= -2 * pi)
    {
        ans += 4 * pi;
    }
    while (ans > 2 * pi)
    {
```

```
        ans -= 4 * pi;
    }
    return ans * s.r * s.r;
}
```

调用 `reflection`、`signedDih3` 与 `dihedral3` 处理平面和二面角，后两者分别返回 $[-\pi,\pi]$ 和 $[0,\pi]$ 内的弧度；`fromLatLon3` 与 `toLatLon3` 在球面点和纬经度弧度之间转换，位于两极且横向分量为零时经度约定返回 0。`solidAngle3` 返回三条短大圆弧围成的主值有向立体角。`spherePoly3` 对 n 个顶点使用 $O(n)$ 时间和 $O(n)$ 额外空间，其余接口均为 $O(1)$ 时间和空间，并依赖 11.6.007 最小覆盖球 的 `Sphere` 和 11.6.010 投影、最近点与空间距离 的投影接口。

12. 动态规划与优化

本章遵循统一类型约定：下标、状态编号、容量和普通计数使用 `int`（公共前导中的 `#define int long long` 会在提交中将其提升为 `long long`）；可能超过 32 位的数值、代价和答案显式使用 `long long`。只有确实依赖 64 位机器字或无符号范围的位集、插头状态、汉诺塔步数、数位 DP 计数和凸包差值比较保留 `unsigned long long`，不得将这些类型扩散到普通变量。

12.1 背包

12.1.001 01 背包

01 背包让每件物品至多用一次，状态 $f[j]$ 是恰好占用容量 j 时的最大价值，转移 $f[j] \leftarrow \max(f[j],\ f[j-w]+v)$。关键在容量必须倒序枚举：倒序时 `f[j-w]` 尚未在本轮被同一件物品更新，仍代表”未放入该物品”的旧状态；一旦改成正序，`f[j-w]` 可能已含这件物品，就退化成可重复取的完全背包。不可达容量用 $-\infty$（这里取 `-INF`）标记并在转移前判掉，避免把无效状态当作价值 0 向后传播。

```
vector<int> zeroOnePack(int m, const vector<pair<int, int>> &a)
{
    // m 是容量，a 的每项是物品体积和价值；返回每个恰好容量的最大价值，不可达为 -INF。
    assert(m >= 0); // 调试检查，可删。
    vector<int> f(m + 1, -INF);
    f[0] = 0;
    for (auto [w, v] : a)
    {
        assert(w > 0); // 调试检查，可删。
        for (int j = m; j >= w; j--)
        {
            if (f[j - w] != -INF)
            {
                chmax(f[j], f[j - w] + v);
            }
        }
    }
    return f;
}
```

调用 `zeroOnePack(m,a)`，容量非负、物品体积为正，且所有可达价值和须能由 `int` 表示；返回的 `f[j]` 是恰好装满容量 j 的最优值，若求容量不超过 m 的答案取 `max(f)`。时间 $O(nm)$、空间 $O(m)$。容量循环方向是这一族的分水岭：可重复取时改用 12.1.002 完全背包 的正序，件数有限用 12.1.003 多重背包，组内互斥用 12.1.006 分组背包，物品间存在选取依赖时不能拍平成普通物品、须走 12.1.007 有依赖背包。

12.1.002 完全背包

完全背包每种物品可取任意次，状态与转移和 01 背包完全相同，只有容量改成正序枚举：正序时 `f[j-w]` 可能已在本轮被同种物品更新过，于是”再加一件”自然叠加，等价于该物品用无限次。体积必须严格为正，否则 `j-w>=j`、正序循环没有推进方向甚至自我引用。同样用 `-INF` 标记不可达并在转移前判掉。

```cpp
vector<int> completePack(int m, const vector<pair<int, int>> &a)
{
    // m 是容量，a 的每项是物品体积和价值且每种可取任意次；返回各恰好容量的最大价值。
    assert(m >= 0); // 调试检查，可删。
    vector<int> f(m + 1, -INF);
    f[0] = 0;
    for (auto [w, v] : a)
    {
        assert(w > 0); // 调试检查，可删。
        for (int j = w; j <= m; j++)
        {
            if (f[j - w] != -INF)
            {
                chmax(f[j], f[j - w] + v);
            }
        }
    }
    return f;
}
```

调用 `completePack(m,a)`，语义与 12.1.001 一致但每种可无限取；容量非负、物品体积为正，且所有可达价值和须能由 `int` 表示。时间 $O(nm)$、空间 $O(m)$。当同一种物品有明确件数上限时它会超额取用，应换 12.1.003 多重背包；混合出现多类物品见 12.1.004 混合背包。

12.1.003 多重背包

多重背包给每种物品一个件数上限 c。直接把它拆成 c 件 01 物品会带来 $O(c)$ 因子，这里用二进制拆分：把 c 分成 $1, 2, 4, \ldots$ 直到剩余，每组当作一件体积价值成倍的 01 物品，用 $O(\log c)$ 组就能凑出 $0 \sim c$ 之间任意取用件数。拆出的每组仍按 01 背包倒序更新，因此天然不会超额。件数极大时可换成按容量余数分组的单调队列优化（见 12.1.009），二进制拆分胜在实现短、常数稳。

```cpp
vector<int> boundedPack(int m, const vector<array<int, 3>> &a)
{
    // m 是容量；a[i][0..2] 依次是体积、价值和最多可选件数；返回各恰好容量的最大价值。
    assert(m >= 0); // 调试检查，可删。
    vector<int> f(m + 1, -INF);
    f[0] = 0;
    for (auto [w, v, c] : a)
    {
        assert(w > 0 && c >= 0); // 调试检查，可删。
        int k = 1;
        while (c > 0)
        {
            int z = min(k, c);
            c -= z;
            if (z <= m / w)
            {
                int ww = w * z;
                int vv = v * z;
```

```
                for (int j = m; j >= ww; j--)
                {
                    if (f[j - ww] != -INF)
                    {
                        chmax(f[j], f[j - ww] + vv);
                    }
                }
            }
            if (c > 0)
            {
                k = k <= c / 2 ? k * 2 : c;
            }
        }
    }
    return f;
}
```

调用 `boundedPack(m,a)`，`a[i][0..2]` 依次是体积、价值、件数，要求体积为正、件数非负。时间 $O(m\sum_i \log(c_i+1))$、空间 $O(m)$。一个小样例：容量 10、物品 `{3,4,2}` 与 `{2,3,3}`，恰好装满容量 10 的最大价值为 14。件数无上限时退回 12.1.002 完全背包，与 01、完全物品混杂时用 12.1.004 混合背包 统一处理。

12.1.004 混合背包

混合背包在同一份输入里同时出现 01、完全、多重三类物品，做法是所有物品共享同一个 f 数组，但逐物品按其类型选定更新方式：完全物品正序、01 物品倒序（相当于件数为 1 的多重）、多重物品做二进制拆分后倒序。这里用件数字段 -1 / 0 / 正数作类型标记；每一类的循环方向必须在处理该物品时就地确定，不能全局统一，否则完全与 01 会互相污染。

```
vector<int> mixedPack(int m, const vector<array<int, 3>> &a)
{
    // m 是容量；a[i] 是体积、价值、件数，件数 -1/0/正数分别表示 01/完全/多重物品。
    assert(m >= 0); // 调试检查，可删。
    vector<int> f(m + 1, -INF);
    f[0] = 0;
    for (auto [w, v, c] : a)
    {
        assert(w > 0 && c >= -1); // 调试检查，可删。
        if (c == 0)
        {
            for (int j = w; j <= m; j++)
            {
                if (f[j - w] != -INF)
                {
                    chmax(f[j], f[j - w] + v);
                }
            }
            continue;
        }
```

```
        if (c == -1)
        {
            c = 1;
        }
        int k = 1;
        while (c > 0)
        {
            int z = min(k, c);
            c -= z;
            if (z <= m / w)
            {
                int ww = w * z;
                int vv = v * z;
                for (int j = m; j >= ww; j--)
                {
                    if (f[j - ww] != -INF)
                    {
                        chmax(f[j], f[j - ww] + vv);
                    }
                }
            }
            if (c > 0)
            {
                k = k <= c / 2 ? k * 2 : c;
            }
        }
    }
    return f;
}
```

调用 `mixedPack(m,a)`，第三维为 `-1`、`0`、正数分别表示 01、完全、多重物品。最坏时间 $O(m\sum_i \log(c_i+1))$、空间 $O(m)$，其中完全物品每种只花 $O(m)$。若全部物品同类，直接用对应的 12.1.001 / 12.1.002 / 12.1.003 更清晰；组内互斥仍须 12.1.006 分组背包，本函数不处理分组语义。

12.1.005 二维费用背包

二维费用背包每件物品同时消耗两种资源、只可选一次，状态升为 $f[x][y]$——恰好用掉第一维费用 x、第二维费用 y 时的最大价值，转移 $f[x][y] \leftarrow \max(f[x][y],\ f[x-a][y-b]+v)$。它就是把 01 背包的倒序思想推到两维：两层容量都必须倒序，才能保证同一件物品在本轮至多被计入一次。函数最后遍历整张表取最大值，得到”两维都不超上限”的答案。

```
int knapsack2(int cap1, int cap2, const vector<array<int, 3>> &a)
{
    // cap1、cap2 是两维容量；a[i][0..2] 依次是第一费用、第二费用、价值；返回不超容量的最大总价值。
    assert(cap1 >= 0 && cap2 >= 0); // 调试检查，可删。
    vector f(cap1 + 1, vector<int>(cap2 + 1, -INF));
    f[0][0] = 0;
    for (auto [x, y, v] : a)
    {
```

```
        assert(x >= 0 && y >= 0 && (x > 0 || y > 0)); // 调试检查，可删。
        for (int i = cap1; i >= x; i--)
        {
            for (int j = cap2; j >= y; j--)
            {
                if (f[i - x][j - y] != -INF)
                {
                    chmax(f[i][j], f[i - x][j - y] + v);
                }
            }
        }
    }
    int ans = 0;
    for (const auto &row : f)
    {
        for (int x : row)
        {
            chmax(ans, x);
        }
    }
    return ans;
}
```

调用 `knapsack2(cap1,cap2,a)`，每个 `array<int,3>` 的三维依次是第一费用、第二费用、价值，返回允许两维都有剩余时的最大价值。时间 $O(nAB)$、空间 $O(AB)$，其中 n 为物品数、A, B 为两维容量上限。一个样例：两维容量 $(4,5)$、物品 `{(2,3,6),(2,2,5),(3,2,7)}`，答案为 11。这是普通 01 背包 12.1.001 的直接推广，费用维数继续增加时按同样的倒序方式再加层即可，但空间随维数指数增长；它与依赖背包 12.1.007、子集和 12.1.010 是不同问题，不要混用。

12.1.006 分组背包

分组背包要求每组至多选一件。诀窍是每组开始前先把当前 f 快照成 `h`，组内每件物品都只从旧数组 `f` 转移、写进新数组 `h`，处理完整组再 `swap`。这样组内任何两件都来自同一份进组前状态，天然互斥；若图省事在同一数组里连续更新，本组前一件的结果会喂给后一件，导致同组多件被同时选中。允许整组不选，因为初始的 `h=f` 已保留”不选本组”的状态。

```
vector<int> groupPack(int m, const vector<vector<pair<int, int>>> &a)
{
    // m 是容量，a[i] 是第 i 组全部“体积、价值”物品；每组至多选一件，返回各恰好容量最大价值。
    assert(m >= 0); // 调试检查，可删。
    vector<int> f(m + 1, -INF);
    f[0] = 0;
    for (auto &g : a)
    {
        auto h = f;
        for (auto [w, v] : g)
        {
            assert(w > 0); // 调试检查，可删。
            for (int j = w; j <= m; j++)
```

```
            {
                if (f[j - w] != -INF)
                {
                    chmax(h[j], f[j - w] + v);
                }
            }
        }
        f.swap(h);
    }
    return f;
}
```

调用 `groupPack(m,a)`，外层每个 `a[i]` 是一组 (体积, 价值) 物品，返回各恰好容量的最大价值，允许整组不选。时间 $O(m\sum_i |G_i|)$、空间 $O(m)$。当每组只有一件时它退化成 01 背包 12.1.001；若组之间还存在”选子必选父”的层级约束，分组模型无法表达，须改用 12.1.007 有依赖背包。

12.1.007 有依赖背包

有依赖背包（金明的预算方案型）中物品构成父子依赖森林：选任何非根物品前必须先选它的父物品。树形 DP 用一个虚根 `rt` 把所有依赖树根挂在一起；真实节点只有在自身已选的状态上才能合并非空儿子状态，从而不会出现“没选父亲却选了后代”。完整实现统一放在 6.5.004 树上依赖背包，本条只复用该接口，不再维护第二份同义代码。

`treeDepPack` 的接口为 `template<class Acc = long long> vector<optional<Acc>> treeDepPack(const vector<int> &w, const vector<Acc> &val, const vector<int> &par, int cap)`：`w[u]` 是正体积，`val[u]` 是价值，`par[u]` 是父节点编号，依赖树根用 `-1` 表示，`par` 必须构成森林，且 `cap >= 0`。返回长度为 `cap + 1` 的数组，其中第 `c` 项是恰好占用容量 `c` 且满足全部父子依赖的最大价值；不可达容量为 `nullopt`。求容量不超过 `cap` 的答案时只在有值的元素中取最大值。默认累计类型是 `long long`；当价值和可能超出 64 位时，显式传入 `treeDepPack<__int128>` 和 `vector<__int128>`。所有可达状态及其加法中间结果必须能由 `Acc` 表示。

```
// 依赖 6.5.004 树上依赖背包
signed main()
{
    vector<__int128> val{3, 5, 8};
    auto f = treeDepPack<__int128>(vector<int>{2, 2, 3}, val,
                                   vector<int>{-1, 0, 0}, 5);
    assert(f[5] && *f[5] == 11);
}
```

合并儿子的 max-plus 卷积保守时间为 $O(nC^2)$、额外空间为 $O(nC)$，其中 n 是物品数、$C = cap$；依赖链的递归深度为 $O(n)$，实际可承受深度取决于评测环境的线程栈。样例中体积 `{2,2,3}`、价值 `{3,5,8}`、父节点 `{-1,0,0}`、容量 5 时最大值为 `11`（选点 `0` 与 `2`）。没有父子依赖时用 12.1.001 01 背包；只有组内至多选一时用 12.1.006 分组背包；树边只表示合并顺序、并不限制是否选父节点时用 12.1.009 树上背包及单调队列优化。

12.1.008 背包方案数与具体方案

这条在求 01 背包最优值的同时统计最优方案数并恢复一组具体方案。方案数是把最优值 DP 里的 `max` 换成”当价值相等时累加方案数”：`f[j]` 被严格刷新则方案数继承 `cnt[j-w]`，与旧值打平则 `cnt[j]+=cnt[j-w]`，对 `mod` 取模。方案恢复靠一条前驱链——每次严格刷新 `f[j]` 时把 (物品下标, 上一状态在 `path` 中的位置) 压进 `path`、并让 `at[j]` 指向它，最后从最优容量沿 `prev` 回溯即得被选物品序列。注意方案计数按物品编号区分、天然是”组合”意义（每件物品处理一次），若要”排列”数需改变枚举顺序。

```
struct PackResult
{
    // val 是不超过容量时的最优总价值。
    long long val;
    // ways 是模 mod 的最优方案数。
    int ways;
    // pick 是恢复出的一组物品下标，按输入顺序排列。
    vector<int> pick;
};

PackResult packPlan(int cap, const vector<pair<int, long long>> &a, int mod = 1000000007)
{
    // cap 是容量上限，a 是各 01 物品的体积与价值，mod 是计数模数；返回最优价值、方案数及一组方案。
    struct Path
    {
        // item 是本步选择的物品下标，pre 是上一步前驱节点编号。
        int item;
        int pre;
    };
    assert(cap >= 0 && mod > 0); // 调试检查，可删。
    int n = a.size();
    vector<long long> f(cap + 1, -INF);
    vector<int> cnt(cap + 1), at(cap + 1, -1);
    vector<Path> path;
    f[0] = 0;
    cnt[0] = 1;
    for (int i = 0; i < n; i++)
    {
        auto [w, v] = a[i];
        assert(w > 0); // 调试检查，可删。
        for (int j = cap; j >= w; j--)
        {
            if (f[j - w] == -INF)
            {
                continue;
            }
            long long x = f[j - w] + v;
            if (x > f[j])
            {
                f[j] = x;
                cnt[j] = cnt[j - w];
                path.push_back({i, at[j - w]});
                at[j] = path.size() - 1;
```

```
            }
            else if (x == f[j] && x > -INF)
            {
                cnt[j] = addMod(cnt[j], cnt[j - w], mod);
            }
        }
    }
    long long val = *max_element(f.begin(), f.end());
    int ways = 0;
    int last = -1;
    for (int j = 0; j <= cap; j++)
    {
        if (f[j] == val)
        {
            ways = addMod(ways, cnt[j], mod);
            last = at[j];
        }
    }
    vector<int> pick;
    while (last >= 0)
    {
        pick.push_back(path[last].item);
        last = path[last].pre;
    }
    reverse(pick.begin(), pick.end());
    return {val, ways, pick};
}
```

调用 `packPlan(cap,a,mod)`，容量非负、体积为正、模数为正，`a` 每项是 (体积, 价值)；返回 `PackResult{val,ways,pick}`，分别是最优价值、模 `mod` 的最优方案数和一组按输入顺序排列的物品下标。时间 $O(nC)$；状态数组占 $O(C)$，持久前驱节点最坏占 $O(nC)$。只要最优值不需要方案信息时直接用 12.1.00101 背包 更省内存；要把恢复空间严格压到 $O(C)$，可分治重算转移，而不能继续永久保存每次刷新。

12.1.009 树上背包及单调队列优化

这条给出树上背包中一个可被单调队列优化的清晰模型：每个树节点挂一类有件数上限的物品，先在本点线性完成这类多重背包，再把子树答案卷积合并上来。多重背包的单调队列优化按体积余数 r 把容量分成若干链 $r, r+w, r+2w, \dots$，在每条链上件数上限就化成滑动窗口长度：维护 `pre[x]-j*v`（旧价值减去到当前格所需件数的线性代价）的窗口最大值，窗口宽度即件数上限 cnt，于是本点转移降到 $O(C)$。子树之间仍是普通 max-plus 卷积。注意本函数的树边只决定合并顺序，不表示父子选取依赖，这与 12.1.007 不同。

```
template <class Acc = long long>
vector<optional<Acc>> treePack(const vector<vector<int>> &g,
                               const vector<array<long long, 3>> &a,
                               int root,
                               int cap)
{
    // a[u] 是 {体积, 价值, 件数}；返回恰用各容量的最大价值，nullopt 表示不可达。
```

```
    int n = g.size();
    assert(n > 0 && (int)a.size() == n && 0 <= root && root < n && cap >= 0); // 调试检查，可删
    auto dfs = [&](auto &&self, int u, int p) -> vector<optional<Acc>>
    {
        int w = a[u][0];
        Acc v = (Acc)a[u][1];
        int cnt = a[u][2];
        assert(w > 0 && cnt >= 0); // 调试检查，可删
        vector<optional<Acc>> pre(cap + 1), f(cap + 1);
        pre[0] = (Acc)0;
        for (int r = 0; r < w && r <= cap; r++)
        {
            deque<int> q;
            for (int j = 0, x = r; x <= cap; j++, x += w)
            {
                while (!q.empty() && q.front() < j - cnt)
                {
                    q.pop_front();
                }
                if (pre[x])
                {
                    Acc cur = *pre[x] - (Acc)j * v;
                    while (!q.empty())
                    {
                        int y = r + q.back() * w;
                        Acc old = *pre[y] - (Acc)q.back() * v;
                        if (old > cur)
                        {
                            break;
                        }
                        q.pop_back();
                    }
                    q.push_back(j);
                }
                if (!q.empty())
                {
                    int y = r + q.front() * w;
                    f[x] = *pre[y] + (Acc)(j - q.front()) * v;
                }
            }
        }
        for (int x : g[u])
        {
            if (x == p)
            {
                continue;
            }
            auto h = self(self, x, u);
            auto nf = f;
            for (int i = 0; i <= cap; i++)
            {
                for (int j = 1; i + j <= cap; j++)
```

```
                {
                    if (f[i] && h[j])
                    {
                        Acc cur = *f[i] + *h[j];
                        if (!nf[i + j] || *nf[i + j] < cur)
                        {
                            nf[i + j] = cur;
                        }
                    }
                }
            }
            f.swap(nf);
        }
        return f;
    };
    return dfs(dfs, root, -1);
}
```

调用 `treePack(g,a,root,cap)`，`g` 必须是覆盖全部点的非空无向树，`a[u][0..2]` 是点 u 的物品体积、价值、件数上限；返回的 `f[c]` 是恰用容量 c 的最大价值，`nullopt` 表示不可达。默认状态、队列评分与卷积和都用 `long long`，必须保证这些中间值可表示；需要宽范围时调用 `treePack<__int128>`。单点树、物品 `{2,3,3}`、容量 7 时，容量 $0,2,4,6$ 的答案为 $0,3,6,9$。保守时间 $O(nC^2)$、额外空间 $O(nC)$；链形树的递归深度为 $O(n)$，实际可承受深度取决于评测环境的线程栈。单节点的有界物品步骤是 $O(C)$。非树上的单个多重背包用 12.1.003 多重背包，单调队列原理见 12.3.001 单调队列优化。

12.1.010 布尔母函数 / 有界子集和可达性

有界子集和判定问题：每种权值 w_i 最多用 cnt_i 次，问某个目标和是否可达。二进制拆分能做，但这里用一个更巧的 $O(kS)$ 布尔母函数：`rem[s]` 记录”和 s 可达时，当前这种权值还剩几次可用”。处理权值 w_i 时正序扫 s——若 s 在上一轮已可达就把额度重置为 cnt_i（本轮一次没用）；否则若 $s-w_i$ 可达且那里还有剩余次数，就从它继承并消耗一次（`rem[s]=rem[s-w_i]-1`），两者都不满足则 s 不可达置 -1。这样无需拆分就限制了每种的使用次数。

```
bool boundSubset(const vector<int> &cnt, const vector<int> &w, int sum)
{
    // cnt、w 分别是每类物品数量和正权值，sum 是目标和；返回目标是否可达。
    assert(cnt.size() == w.size() && sum >= 0); // 调试检查，可删。
    for (int i = 0; i < (int)w.size(); i++)
    {
        assert(cnt[i] >= 0 && w[i] > 0); // 调试检查，可删。
    }
    vector<int> rem(sum + 1, -1);
    rem[0] = 0;
    for (int i = 0; i < (int)w.size(); i++)
    {
        for (int s = 0; s <= sum; s++)
        {
            if (rem[s] >= 0)
            {
```

```
                rem[s] = cnt[i];
            }
            else if (s < w[i] || rem[s - w[i]] <= 0)
            {
                rem[s] = -1;
            }
            else
            {
                rem[s] = rem[s - w[i]] - 1;
            }
        }
    }
    return rem[sum] >= 0;
}
```

调用 `boundSubset(cnt,w,sum)`，`cnt[i]`、`w[i]` 分别是第 i 种物品的非负数量与正权值、下标一一对应，`sum` 是目标和，返回布尔是否可达（原调用说明误写成 `array<int,2>` 输入与”可达表”返回，与函数签名不符，已按实际签名更正）。时间 $O(kS)$、空间 $O(S)$，k 为种类数、S 为目标和。只判可达性而不限件数时，用 12.2.008 子集和 Bitset 优化 的按字并行更快；需要方案数或最优价值则退回普通 12.1.003 多重背包。

12.1.011 基础区间 DP

区间 DP 以区间长度为阶段，$dp_{l,r}$ 表示半开区间 $[l,r)$ 的最优值，由所有分割点 $k \in (l,r)$ 的两段合并而来；记录最优分割点 `cut[l][r]` 后可以递归还原方案。这里给出三个基本模型：

- 石子合并：每次合并相邻两堆，代价为两堆之和，$dp_{l,r}=\min_k(dp_{l,k}+dp_{k,r})+\sum_{i=l}^{r-1}a_i$。环形把数组复制一倍，答案是所有长度为 n 的窗口中的最小值。
- 矩阵链乘：第 i 个矩阵为 $p_i\times p_{i+1}$，$dp_{l,r}=\min_k(dp_{l,k}+dp_{k,r}+p_lp_kp_r)$。
- 括号补全：$dp_{l,r}$ 是使 $[l,r)$ 合法的最少插入数；两端匹配时可取 $dp_{l+1,r-1}$，再与所有分割取最小值。

```
namespace interval_dp
{
struct Result
{
    __int128 cost = 0;        // cost 是最优总代价。
    int start = 0;            // start 是环形时最优的断环起点，链形为 0。
    vector<vector<int>> cut; // cut[l][r] 是区间 [l,r) 最优的分割点，长度为 1 的区间为 -1。

    vector<array<int, 3>> plan(int l, int r) const
    {
        // 按后序返回区间 [l,r) 的所有合并 {x,m,y}，表示把 [x,m) 与 [m,y) 合并。
        vector<array<int, 3>> res;
        auto dfs = [&](auto &&self, int x, int y) -> void
        {
            if (y - x <= 1)
            {
                return;
```

```
            }
            int m = cut[x][y];
            self(self, x, m);
            self(self, m, y);
            res.push_back({x, m, y});
        };
        dfs(dfs, l, r);
        return res;
    }
};

Result stones(vector<int> a, bool ring = false)
{
    // a 是各堆石子数（可为负），ring 表示首尾相邻；每次合并相邻两堆、代价为两堆之和，
    // 返回最小总代价与方案。环形把数组复制一倍后取所有长度为 n 的窗口。O(n^3)。
    int n = a.size();
    if (!n)
    {
        return {};
    }
    if (ring)
    {
        auto b = a;
        a.insert(a.end(), b.begin(), b.end());
    }
    int m = a.size();
    vector<__int128> pre(m + 1);
    for (int i = 0; i < m; i++)
    {
        pre[i + 1] = pre[i] + a[i];
    }
    vector<vector<__int128>> dp(m, vector<__int128>(m + 1)); // dp[l][r] 是合并 [l,r) 的最小代
    价。
    Result res;
    res.cut.assign(m, vector<int>(m + 1, -1));
    for (int len = 2; len <= n; len++)
    {
        for (int l = 0; l + len <= m; l++)
        {
            int r = l + len;
            for (int k = l + 1; k < r; k++)
            {
                __int128 v = dp[l][k] + dp[k][r] + pre[r] - pre[l];
                // 求最大总代价：把这里的 v < dp[l][r] 和下面环形窗口的 dp[l][l + n] < res.cost 都改
                成 >。
                if (k == l + 1 || v < dp[l][r])
                {
                    dp[l][r] = v;
                    res.cut[l][r] = k;
                }
            }
        }
```

```
    }
    res.cost = dp[0][n];
    for (int l = 1; ring && l < n; l++)
    {
        if (dp[l][l + n] < res.cost)
        {
            res.cost = dp[l][l + n];
            res.start = l;
        }
    }
    return res;
}

Result matrixChain(const vector<int> &p)
{
    // p 有 n+1 个正整数，第 i 个矩阵是 p[i] x p[i+1]；返回最少标量乘法次数与加括号方案。O(n^3)。
    assert(!p.empty()); // 调试检查，可删。
    int n = p.size() - 1;
    Result res;
    res.cut.assign(n, vector<int>(n + 1, -1));
    vector<vector<__int128>> dp(n, vector<__int128>(n + 1));
    for (int len = 2; len <= n; len++)
    {
        for (int l = 0; l + len <= n; l++)
        {
            int r = l + len;
            for (int k = l + 1; k < r; k++)
            {
                __int128 v = dp[l][k] + dp[k][r] + (__int128)p[l] * p[k] * p[r];
                if (k == l + 1 || v < dp[l][r])
                {
                    dp[l][r] = v;
                    res.cut[l][r] = k;
                }
            }
        }
    }
    if (n)
    {
        res.cost = dp[0][n];
    }
    return res;
}

int brackets(const string &s)
{
    // s 只含 ()[]{}；返回最少插入多少个括号能使其合法。O(n^3)。
    // 只有一种括号时用贪心即可：未匹配的右括号数 + 剩余左括号数，O(n)。
    int n = s.size();
    vector<vector<int>> dp(n + 1, vector<int>(n + 1)); // dp[l][r] 是区间 [l,r) 的答案。
    for (int i = 0; i < n; i++)
    {
```

```
        dp[i][i + 1] = 1;
    }
    for (int len = 2; len <= n; len++)
    {
        for (int l = 0; l + len <= n; l++)
        {
            int r = l + len;
            dp[l][r] = len;
            int i = string("()[]{}").find(s[l]); // 左括号在偶数位，其右括号紧随其后；找不到为 -1。
            if (i >= 0 && i % 2 == 0 && "()[]{}"[i + 1] == s[r - 1])
            {
                dp[l][r] = dp[l + 1][r - 1];
            }
            for (int k = l + 1; k < r; k++)
            {
                dp[l][r] = min(dp[l][r], dp[l][k] + dp[k][r]);
            }
        }
    }
    return dp[0][n];
}
}
```

`stones(a,ring)` 允许负数堆，返回最小代价、环形时最优窗口的起点 `start` 以及分割表；`plan(l,r)` 按后序给出合并步骤 $\{x, m, y\}$，环形时传入 `plan(start,start+n)`。`matrixChain(p)` 要求 p 非空且元素为正，`brackets(s)` 只接受 `()[]{}`。三者都是 $O(n^3)$ 时间、$O(n^2)$ 空间，代价用 `__int128` 累加。改求最大代价时按代码注释同步修改两处比较。

堆大小非负时石子合并满足四边形不等式，可以用 12.3.002 四边形不等式优化 降到 $O(n^2)$；有负数时该条件不成立，只能用这里的 $O(n^3)$。只有一种括号时贪心即可：未匹配的右括号数加上剩余左括号数。下面的环形样例中首尾两个 1 相邻，环形答案比链形更小：

```
signed main()
{
    auto s = interval_dp::stones({1, 9, 9, 1});
    auto r = interval_dp::stones({1, 9, 9, 1}, true);
    assert(s.cost == 40 && r.cost == 33 && r.start == 1);
    assert((s.plan(0, 4) == vector<array<int, 3>>{{0, 1, 2}, {2, 3, 4}, {0, 2, 4}}));
    assert(interval_dp::matrixChain({10, 30, 5, 60}).cost == 4500);
    assert(interval_dp::brackets("([)]") == 2);
}
```

12.2 常见 DP

12.2.001 数位 DP

数位 DP 从高位到低位扫描整数、统计 $[0, n]$ 中满足数位约束的个数。这份实现把”约束”抽象成一个自动机：调用者提供状态转移 `next(q,d,ns)` 和终态判定 `accept(q,st)`，本体只负责管理两个通用维度——`lim`（是否仍贴着上界 n，贴界时最高位只能枚举到 n 的对应位）和 `st`（是否已经填过非零位，即是否走

出前导零)。每读一位，就对 $(\mathrm{lim}, \mathrm{st}, q)$ 的计数向 $(\mathrm{nl}, \mathrm{ns}, \mathrm{nq})$ 分摊。前导零必须靠 `st` 单独标记而不能当普通的数字 0，否则形如”数位递增” “无前导零段”这类约束会算错；`lim` 维度保证不会数超过 n。

```cpp
template<class Next, class Accept>
unsigned long long digitDP(long long n, int m, int s, Next next, Accept accept)
{
    // n 是区间右端点，m 是状态数，s 是初态；next 转移状态，accept 判定终态；返回被接受的整数个数。
    if (n < 0)
    {
        return 0;
    }
    assert(m > 0 && 0 <= s && s < m); // 调试检查，可删。
    string a = to_string(n);
    vector dp(2, vector(2, vector<unsigned long long>(m)));
    dp[1][0][s] = 1;
    for (char c : a)
    {
        vector ndp(2, vector(2, vector<unsigned long long>(m)));
        for (int lim = 0; lim < 2; lim++)
        {
            for (int st = 0; st < 2; st++)
            {
                for (int q = 0; q < m; q++)
                {
                    for (int d = 0; d <= (lim ? c - '0' : 9); d++)
                    {
                        int nl = lim && d == c - '0';
                        int ns = st || d;
                        int nq = next(q, d, ns);
                        assert(0 <= nq && nq < m); // 调试检查，可删。
                        ndp[nl][ns][nq] += dp[lim][st][q];
                    }
                }
            }
        }
        dp.swap(ndp);
    }
    unsigned long long ans = 0;
    for (int lim = 0; lim < 2; lim++)
    {
        for (int st = 0; st < 2; st++)
        {
            for (int q = 0; q < m; q++)
            {
                if (accept(q, st))
                {
                    ans += dp[lim][st][q];
                }
            }
        }
    }
    return ans;
```

```
}
```

调用 `digitDP(n,m,s,next,accept)`：`n` 是区间右端点（$n < 0$ 返回 0），`m` 是正状态数，`s` 是初态；`next(q,d,ns)` 返回合法新状态，`accept(q,st)` 判断终态，其中 `ns`、`st` 表示是否已经读入非零位。返回类型能表示 $[0, 2^{63}-1]$ 的全部 2^{63} 个整数；求 $[l, r]$ 用两次调用之差。时间 $O(10m \log n)$、空间 $O(m)$。集合状态压缩用 12.2.002 状压 DP，棋盘轮廓局部转移用 12.2.011 插头 DP。

下面固定检查“不含数字 4”和数字 0 的 `st` 语义：

```
signed main()
{
    auto nextf = [](int q, int d, int ns)
    {
        // q 是当前状态，d 是本位数字，ns 表示读入后是否已开始；返回新状态。
        (void)ns;
        return q == 1 || d == 4;
    };
    auto no4 = [](int q, int st)
    {
        // q 是终态，st 表示是否已开始；返回该整数是否不含数字 4。
        (void)st;
        return q == 0;
    };
    assert(digitDP(49, 2, 0, nextf, no4) == 36);
    auto same = [](int q, int d, int ns)
    {
        // 这个单状态自动机不改变状态。
        (void)d;
        (void)ns;
        return q;
    };
    auto started = [](int q, int st)
    {
        // 只接受已经读入非零位的整数，因此排除数字 0。
        (void)q;
        return st;
    };
    assert(digitDP(0, 1, 0, same, started) == 0);
    auto all = [](int q, int st)
    {
        // 接受所有终态，包括始终未开始的数字 0。
        (void)q;
        (void)st;
        return true;
    };
    assert(digitDP(numeric_limits<long long>::max(), 1, 0, same, all)
           == (1ULL << 63));
    cout << "OK\n";
}
```

12.2.002 状压 DP

状压 DP 把”规模较小的集合中哪些元素已用”编码成位掩码，适合旅行商、集合划分、匹配这类未来只依赖已选集合而非顺序的问题。下例是最经典的旅行商：$dp[s][u]$ 表示已访问集合恰为 s、当前停在 u 的最小费用，转移枚举下一个未访问点 v：$dp[s \mid 2^v][v] \leftarrow dp[s][u] + w[u][v]$，最后在全集上加回到起点 0 的边取最小。$n \le 22$ 时状态数 $2^n n$ 尚可承受；复杂度必须按实际枚举的状态数与转移数算，不能笼统写成 2^n。

```cpp
// 知识点 12.2.002：旅行商状态压缩 DP。
namespace bitmask_dp
{
int tsp(const vector<vector<int>> &w)
{
    // w 是完整有向图权值方阵；返回从 0 出发访问所有点并回到 0 的最小费用。
    int n = w.size();
    assert(1 <= n && n <= 22); // 调试检查，可删。
    for (auto &row : w)
    {
        assert((int)row.size() == n); // 调试检查，可删。
    }
    if (n == 1)
    {
        return 0;
    }
    vector<vector<int>> dp(1LL << n, vector<int>(n));
    vector<vector<char>> ok(1LL << n, vector<char>(n));
    dp[1][0] = 0;
    ok[1][0] = true;
    for (int s = 1; s < (1LL << n); s++)
    {
        for (int u = 0; u < n; u++)
        {
            if (!(s >> u & 1) || !ok[s][u])
            {
                continue;
            }
            for (int v = 0; v < n; v++)
            {
                if (s >> v & 1)
                {
                    continue;
                }
                int ns = s | 1LL << v;
                int nd = dp[s][u] + w[u][v];
                if (!ok[ns][v] || nd < dp[ns][v])
                {
                    dp[ns][v] = nd;
                    ok[ns][v] = true;
                }
            }
        }
```

```
    }
    int ans = 0;
    bool found = false;
    for (int u = 0; u < n; u++)
    {
        if (ok.back()[u])
        {
            int cur = dp.back()[u] + w[u][0];
            if (!found || cur < ans)
            {
                ans = cur;
                found = true;
            }
        }
    }
    return ans;
}
}
```

调用 `bitmask_dp::tsp(w)`，`w` 是 $1 \le n \le 22$ 的完整有向图权值方阵，所有路径费用须能由 `int` 表示；返回从 0 出发遍历所有点再回到 0 的最小费用，单点图返回 0，不读取自环权值。时间 $O(2^n n^2)$、空间 $O(2^n n)$。集合划分可在此框架上改成枚举子集；整数各位约束用 12.2.001 数位 DP，只判子集和可达性时用 12.2.008 子集和 Bitset 优化。

12.2.003 棋盘分割

棋盘分割把矩形棋盘沿格线递归切成 k 个子矩形，最小化各块元素和的平方和。状态是当前子矩形四条边界与块数，转移枚举最后一刀及两侧块数分配；$k = 1$ 时代价是整块和的平方。记忆值用 `optional` 区分不可达状态，不拿某个可能被合法答案达到的整数充当无穷大。

```
// 知识点 12.2.003：矩形棋盘切成 k 块的最小平方和。
namespace board_part
{
template <class Acc = long long>
Acc splitBoard(const vector<vector<int>> &a, int parts)
{
    // a 是非空棋盘，parts 是块数；返回最小平方和。
    static_assert(numeric_limits<Acc>::is_integer && numeric_limits<Acc>::is_signed &&
                  sizeof(Acc) >= sizeof(long long));
    assert(!a.empty() && !a[0].empty()); // 调试检查，可删。
    int n = a.size();
    int m = a[0].size();
    assert(1 <= parts && parts <= n * m); // 调试检查，可删。
    vector<vector<Acc>> s(n + 1, vector<Acc>(m + 1));
    for (int i = 0; i < n; i++)
    {
        assert((int)a[i].size() == m); // 调试检查，可删。
        for (int j = 0; j < m; j++)
        {
```

```
            s[i + 1][j + 1] = s[i][j + 1] + s[i + 1][j] - s[i][j] + a[i][j];
        }
    }
    auto sum = [&](int x1, int y1, int x2, int y2)
    {
        // 四个参数定义半开子矩形；返回元素和。
        return s[x2][y2] - s[x1][y2] - s[x2][y1] + s[x1][y1];
    };
    // key[0..4] 依次是上、左、下、右边界与块数。
    using Key = array<int, 5>;
    map<Key, optional<Acc>> memo;
    auto dfs = [&](auto &&self, int x1, int y1, int x2, int y2, int k) -> optional<Acc>
    {
        // self 是递归自身，四坐标定义矩形，k 是块数；返回最小平方和。
        Key key{x1, y1, x2, y2, k};
        if (auto it = memo.find(key); it != memo.end())
        {
            return it->second;
        }
        if (k == 1)
        {
            Acc x = sum(x1, y1, x2, y2);
            return memo[key] = x * x;
        }
        if (k > (x2 - x1) * (y2 - y1))
        {
            return memo[key] = nullopt;
        }
        optional<Acc> ans;
        for (int x = x1 + 1; x < x2; x++)
        {
            for (int p = 1; p < k; p++)
            {
                auto l = self(self, x1, y1, x, y2, p);
                auto r = self(self, x, y1, x2, y2, k - p);
                if (l && r && (!ans || *l + *r < *ans))
                {
                    ans = *l + *r;
                }
            }
        }
        for (int y = y1 + 1; y < y2; y++)
        {
            for (int p = 1; p < k; p++)
            {
                auto l = self(self, x1, y1, x2, y, p);
                auto r = self(self, x1, y, x2, y2, k - p);
                if (l && r && (!ans || *l + *r < *ans))
                {
                    ans = *l + *r;
                }
            }
```

```
        }
        return memo[key] = ans;
    };
    return *dfs(dfs, 0, 0, n, m, parts);
}
}
```

`splitBoard(a,parts)` 返回合法切分的最小平方和；需要宽累计时调用 `splitBoard<__int128>`。调用者须保证二维前缀和、任意区域和的平方及候选代价之和都能由 `Acc` 表示。内部 `optional` 只区分某个子矩形无法切出指定块数的状态，不暴露到公开接口。状态数 $O(n^2m^2k)$，最坏时间 $O(n^3m^2k^2+n^2m^3k^2)$，空间 $O(n^2m^2k+nm)$。

12.2.004 两台机器工作调度

两台相同机器并行做一批独立任务、最小化完工时间（makespan），本质是把总工时尽量对半分：设一台机器的负载为 s，另一台就是 $\text{sum}-s$，完工时间是二者较大者 $\max(s,\text{sum}-s)$，s 越接近 $\text{sum}/2$ 越优。于是问题化成”哪些负载 s 可由任务子集凑出”的 01 子集和可达性 DP，`dp[s]` 用布尔标记，最后在所有可达 s 上取 $\max(s,\text{sum}-s)$ 的最小值。

```
// 知识点 12.2.004：独立任务分配给两台机器的最短完工时间。
namespace twoMachSched
{
int twoMachines(const vector<int> &a)
{
    // a 是各任务的非负耗时；返回两台相同机器并行完成全部任务的最小 makespan。
    __int128 total = 0;
    for (int x : a)
    {
        assert(x >= 0); // 调试检查，可删。
        total += x;
    }
    // 本算法按总耗时分配数组，故总耗时还必须能作为容器下标。
    // 调试检查，可删。
    assert(total <= numeric_limits<int>::max()
           && total < (__int128)numeric_limits<size_t>::max());
    int sum = (int)total;
    vector<char> dp(sum + 1);
    dp[0] = 1;
    for (int x : a)
    {
        for (int s = sum; s >= x; s--)
        {
            dp[s] |= dp[s - x];
        }
    }
    int ans = sum;
    for (int s = 0; s <= sum; s++)
    {
        if (dp[s])
```

```
        {
            ans = min(ans, max(s, sum - s));
        }
    }
    return ans;
}
}
```

调用 `twoMachSched::twoMachines(a)`，`a` 是各任务的非负耗时，返回两台相同机器并行的最小 makespan。时间 $O(nS)$、空间 $O(S)$，S 为总耗时。子集和可达性判定本身可用 12.2.008 子集和 Bitset 优化 把常数压到 $1/64$；机器数超过两台时对半分不再成立，得转多维背包或状压。

12.2.005 找硬币问题

找硬币问题的关键是先分清模型：硬币是否可重复、顺序是否计入、要最少枚数还是方案数或只判可达，三者的初值与合并运算各不相同。下例是最常见的一种——面值无限、顺序无关，同时求最少枚数与无序方案数：外层枚举面值、内层容量正序（完全背包语义，一种面值可反复用），`best[s]=\min(best[s],best[s-x]+1)` 取最少枚数、`ways[s]+=ways[s-x]` 累加方案数。外层枚举面值而非金额，保证每种组合只按面值升序统计一次，得到的是无序方案数（若把两层循环对调则统计有序序列数）。

```
// 知识点 12.2.005：完全背包找硬币的最少枚数与方案数。
namespace coin_problem
{
array<int, 2> coinChange(const vector<int> &coin, int target, int mod)
{
    // coin 是不同正面值，target 是目标金额，mod 是方案数模数；返回最少枚数和无序方案数，无解时第一维为
    ↪ -1。
    assert(target >= 0 && mod > 0); // 调试检查，可删。
    set<int> seen;
    for (int x : coin)
    {
        assert(x > 0 && seen.insert(x).second); // 调试检查，可删。
    }
    const int inf = numeric_limits<int>::max() / 4;
    vector<int> best(target + 1, inf);
    vector<int> ways(target + 1);
    best[0] = 0;
    ways[0] = 1;
    for (int x : coin)
    {
        for (int s = x; s <= target; s++)
        {
            if (best[s - x] != inf)
            {
                best[s] = min(best[s], best[s - x] + 1);
            }
            ways[s] = addMod(ways[s], ways[s - x], mod);
        }
    }
```

```
    return {best[target] == inf ? -1 : best[target], ways[target]};
}
}
```

调用 `coin_problem::coinChange(coin,target,mod)`，`coin` 是不同正面值、`target` 是目标金额、`mod` 是方案数模数，返回 `{最少枚数，无序方案数}`，无解时第一维为 -1。时间 $O(kS)$、空间 $O(S)$，k 为面值数、S 为金额。硬币有件数上限时正序不再成立，改用 12.1.003 多重背包 的二进制拆分或 12.1.010 的剩余次数 DP；只关心某金额能否凑出用 12.2.008 子集和 Bitset 优化。

12.2.006 汉诺塔

经典三柱汉诺塔的解是递归的：“移 n 个盘 = 先把上 $n-1$ 个移到辅助柱，再移最大盘，再把 $n-1$ 个移回”，总步数 2^n-1。本条不做递归模拟，而是 $O(1)$ 回答”第 k 步移哪个盘、从哪到哪”：观察到第 k 步移动的盘号等于 k 二进制末尾零的个数加一（`countr_zero(k)+1`），因为盘 i 每 2^i 步动一次；而某个盘每次移动的方向固定（由 $n-$ disk 的奇偶决定顺时针或逆时针），出现次数 `before` 决定它当前在哪根柱上。多柱（Frame–Stewart）版本步数不再是 2^n-1，该公式不适用。

```
// 知识点 12.2.006：汉诺塔第 k 步移动。
namespace hanoi
{
array<int, 3> kthMove(int n, unsigned long long k)
{
    // n 是盘数，k 从 1 开始且不超过 2^n-1；返回盘号、起始柱和目标柱，柱编号为 0..2。
    assert(1 <= n && n < 64 && 1 <= k && k < (1ULL << n)); // 调试检查，可删。
    int disk = countr_zero(k) + 1;
    unsigned long long before = (k >> disk) + 1;
    int dir = ((n - disk) & 1) ? 1 : -1;
    int from = (dir * (int)(before - 1) % 3 + 3) % 3;
    int to = (from + dir + 3) % 3;
    return {disk, from, to};
}
}
```

调用 `hanoi::kthMove(n,k)`，n 是盘数（$1 \le n < 64$）、k 从 1 开始且不超过 2^n-1，返回 `{盘号，起始柱，目标柱}`，柱编号 $0\ldots2$。时间与空间 $O(1)$。要输出整段移动序列时逐 k 调用即得，总输出量 2^n-1 是指数级、复杂度须按输出规模计而非递归层数；多柱最优步数问题不在此列。

12.2.007 最长上升/公共子序列

最长上升子序列（LIS）与最长公共子序列（LCS）状态和复杂度不同。LIS 维护“长度为 $k+1$ 的上升子序列的最小末值”`tail[k]`，对每个元素用 `lower_bound` 找到应替换的位置；`at[k]` 记录该末值来自哪个下标，`pre[i]` 记录方案前驱，从最后一个末值即可恢复一条严格 LIS。LCS 是两序列前缀 DP，其完整实现统一放在 4.5.001 最长公共子序列，本条不再复制。

```
template<class T>
vector<T> lis(const vector<T> &a)
{
```

```
    // a 是输入序列；返回一条严格最长上升子序列。
    int n = a.size();
    vector<T> tail;
    vector<int> at;
    vector<int> pre(n, -1);
    for (int i = 0; i < n; i++)
    {
        int k = lower_bound(tail.begin(), tail.end(), a[i]) - tail.begin();
        if (k > 0)
        {
            pre[i] = at[k - 1];
        }
        if (k == (int)tail.size())
        {
            tail.push_back(a[i]);
            at.push_back(i);
        }
        else
        {
            tail[k] = a[i];
            at[k] = i;
        }
    }
    if (at.empty())
    {
        return {};
    }
    vector<T> ans;
    for (int x = at.back(); x != -1; x = pre[x])
    {
        ans.push_back(a[x]);
    }
    reverse(ans.begin(), ans.end());
    return ans;
}
```

`lis(a)` 返回一条严格最长上升子序列，空输入返回空序列；把 `lower_bound` 改成 `upper_bound` 可求最长不下降子序列。时间 $O(n\log n)$、额外空间 $O(n)$。普通 LCS、Bitset LCS 和 SCS 使用 4.5.001 最长公共子序列 的 `lcs`、`lcsBitset` 与 `scs`，不要在本章复制实现。

12.2.008 子集和 Bitset 优化

子集和可达性判定用一个位集表示”哪些和可达”，加入体积 w 时执行 `bits |= bits << w`——第 s 位为 1 表示和 s 可达，左移 w 位再或回去，就等价于”每个原可达和再加一件 w “。这样把布尔 DP 的每字 64 个状态并行处理，常数降到 $1/64$。`SubsetSum` 手写了跨 `unsigned long long` 字的移位（`q=w>>6` 是整字偏移、`r=w&63` 是字内偏移，跨字时补上前一字被移出的高位），并在每次 `add` 后把超过上限 n 的高位掩掉。注意这条 `bits |= bits<<w` 是 01 物品语义（每次调用只加一件）；完全背包那种”同一件反复取”不能靠单次移位表达，需在字内做前缀 or。

```cpp
struct SubsetSum
{
    // n 是维护的最大子集和。
    int n;
    // a 的第 i 位表示子集和 i 当前是否可达。
    vector<unsigned long long> a;

    static size_t words(int n)
    {
        // n 是最大子集和；返回位集所需的机器字数。
        assert(n >= 0); // 调试检查，可删。
        return (size_t)(n / 64 + 1);
    }

    SubsetSum(int n) : n(n), a(words(n))
    {
        // n 是维护的最大子集和；初始时只有和 0 可达。
        a[0] = 1;
    }

    void add(int x)
    {
        // x 是新加入的一件 01 物品的非负重量；更新可达和。
        assert(x >= 0); // 调试检查，可删。
        int q = x >> 6, r = x & 63;
        for (int i = (int)a.size() - 1; i >= q; i--)
        {
            unsigned long long z = a[i - q] << r;
            if (r && i > q)
            {
                z |= a[i - q - 1] >> (64 - r);
            }
            a[i] |= z;
        }
        if ((n & 63) != 63)
        {
            a.back() &= (1ULL << ((n & 63) + 1)) - 1;
        }
    }

    bool has(int x) const
    {
        // 参数 x 表示 要查询是否可达的子集和；返回目标子集和 x 当前是否可达。
        return 0 <= x && x <= n && (a[x >> 6] >> (x & 63) & 1);
    }
};
```

用 `SubsetSum ss(n)` 建对象（n 是需维护的最大和），`ss.add(x)` 加入一件体积 x，`ss.has(x)` 查询和 x 是否可达。加入 k 件的总时间 $O\left(\frac{kS}{64}\right)$、空间 $O\left(\frac{S}{64}\right)$，$S$ 为上限。它是 12.2.004 两台机器工作调度 等可达性问题的加速器；需要方案数或最优价值时它不适用，回到普通 01 背包 12.1.001；有件数上限的可达性判定用 12.1.010。

下面检查跨越 64 位块的移位与末字掩码：

```
signed main()
{
    SubsetSum s(65);
    for (int x : {1, 63, 64})
    {
        s.add(x);
    }
    assert(s.has(0) && s.has(64) && s.has(65) && !s.has(62));
}
```

12.2.009 树形 DP 与动态 DP

树上最大权独立集的静态递推是 $f_{u,0}=\sum_v \max(f_{v,0},f_{v,1})$、$f_{u,1}=w_u+\sum_v f_{v,0}$，其中 $0/1$ 表示不选或选择 u。代码用递归 lambda 在回溯时合并子树状态；链形树的递归深度为 $O(n)$，实际可承受深度取决于评测环境的线程栈。支持单点修改的动态版本要把重链转移写成 max-plus 矩阵并维护矩阵积，完整实现见 6.4.005 动态 DP，本章不重复打印其 `GlobalTree` 底层。

```
template <class Acc = long long>
Acc treeMWIS(const vector<vector<int>> &g, const vector<Acc> &w, int root = 0)
{
    // g 是无向树，w 是点权，root 是根；返回允许空集的最大权独立集权值。
    int n = g.size();
    assert((int)w.size() == n); // 调试检查，可删。
    if (n == 0)
    {
        return 0;
    }
    assert(0 <= root && root < n); // 调试检查，可删。
    int deg = 0;
    for (int u = 0; u < n; u++)
    {
        deg += g[u].size();
        for (int v : g[u])
        {
            assert(0 <= v && v < n); // 调试检查，可删。
        }
    }
    assert(deg == 2 * (n - 1)); // 调试检查，可删。
    vector<int> par(n, -2);
    int seen = 0;
    auto dfs = [&](auto &&self, int u, int p) -> array<Acc, 2>
    {
        par[u] = p;
        seen++;
        array<Acc, 2> f{0, w[u]};
        for (int v : g[u])
        {
            if (v == p)
```

```
            {
                continue;
            }
            assert(par[v] == -2); // 调试检查，可删。
            auto h = self(self, v, u);
            f[0] += max(h[0], h[1]);
            f[1] += h[0];
        }
        return f;
    };
    auto f = dfs(dfs, root, -1);
    assert(seen == n); // 调试检查，可删。
    return max(f[0], f[1]);

}
```

调用 `treeMWIS(g,w,root)` 求静态树的最大权独立集，空树返回 0，非空输入必须是覆盖全部点的无向树；默认点权、DP 状态与返回值均为 `long long`，必须保证所有子树状态能装入 `Acc`。需要宽答案时调用 `treeMWIS<__int128>` 并传入 `vector<__int128>` 点权。时间和额外空间均为 $O(n)$。需要修改点权时使用 6.4.005 动态 DP 的 `DynamicMWIS`：预处理 $O(n\log n)$、单点修改 $O(\log n)$、查询 $O(1)$。其他树形状态不能只替换矩阵元素，必须重新证明链上转移满足结合律。

12.2.010 不同子序列计数

统计一个序列的不同子序列数量（含空串则加一）。设 sum 为”当前所有不同子序列数”（初值 1，代表空串），逐元素处理：加入元素 x 时，理论上每个已有子序列后面接一个 x 都产生新串，于是 sum 翻倍；但若 x 之前出现过，上次处理 x 那一刻已经统计过”以 x 结尾”的那批串，它们会被重复计入，须减去 x 上次出现前的 sum。故递推 $\text{sum} \leftarrow 2\,\text{sum} - \text{last}[x]$，其中 `last[x]` 记 x 上次处理前的 sum。全程对 `mod` 取模，减法先加 `mod` 防负。

```
// 返回不同非空子序列个数，值相同的元素视为相同字符。
int distinctSub(const vector<long long> &a, int mod = 998244353)
{
    // a 是输入序列，mod 是正模数；返回不同非空子序列数量模 mod。
    assert(mod > 0); // 调试检查，可删。
    unordered_map<long long, int> last;
    int sum = 1 % mod;
    for (long long x : a)
    {
        int old = last[x];
        int twice = sum >= mod - sum ? sum - (mod - sum) : sum + sum;
        int next = twice - old;
        if (next < 0)
        {
            next += mod;
        }
        last[x] = sum;
        sum = next;
    }
```

```
    return sum == 0 ? mod - 1 : sum - 1;
}
```

调用 `distinctSub(a,mod)`，`a` 是输入序列（值相同即视为同字符），返回不同非空子序列个数模 `mod`（末尾 -1 去掉空串）。期望 $O(n)$ 时间、$O(k)$ 空间，k 为不同值数；用 `unordered_map` 存 `last`，哈希退化时最坏 $O(n^2)$，值域小时可换定长数组稳定到 $O(n)$。它统计的是”去重后的子序列数”，与不去重的子序列计数（直接 2^n）是不同问题，别混用。

12.2.011 插头 DP

沿棋盘扫描线（轮廓线）逐格转移，状态只保存轮廓上每个位置的插头——即回路/路径穿过该格线的连通信息。相邻状压 DP 只记「有无」，插头 DP 还要记连通性，用于铺砖、哈密顿回路、连通块构造这类要求整体连通的问题。连通性用括号表示法编码：轮廓上有 $m+1$ 个插头，每个取值 $\{0$ 无，1 左括号(，2 右括号$)\}$，同一条链的两端记成一对匹配括号，2 bit 一个插头压进一个整数。

下例是最经典的 URAL1519 型问题：用一条闭合回路恰好覆盖棋盘全部空格，求方案数。处理格 (i,j) 时看它的左插头 $L=\text{plug}[j]$ 与上插头 $U=\text{plug}[j+1]$，按二者取值分情形转移（下插头写回 $\text{plug}[j]$、右插头写回 $\text{plug}[j+1]$）：

- $L=U=0$：本格是回路拐角「，新建一对插头，下 $=($、右 $=)$；
- 恰一个非空：回路直行或拐弯，把该插头改道向右或向下（两种转移）；
- $L=U=1$（两左括号）：两段合并，需把 U 匹配的) 改成 (；
- $L=U=2$（两右括号）：把 L 匹配的 (改成)；
- $L=2,U=1$ $()$ $()$：两条不同链首尾相连，直接消去两插头；
- $L=1,U=2$ $(())$：相邻括号互为配对，闭合成回路——只有在最后一个空格、且闭合后轮廓再无其它插头时才计入答案（否则会漏成多条回路）。

障碍格要求左、上插头都为空。每行处理完要求最右插头为空，再整体左移一位对齐下一行。

```
// 插头 DP（括号表示法）：统计用一条闭合回路覆盖棋盘全部空格的方案数（URAL1519 型）。
// 轮廓线上有 m+1 个插头，每个取值 {0 无，1 左括号'('，2 右括号')'}，2 bit 编码。
template <class Acc = long long>
struct PlugDP
{
    int n; // n 是棋盘行数。
    int m; // m 是棋盘列数。
    vector<vector<int>> a; // a[i][j] 为 1 表示障碍，0 表示必须覆盖的空格。

    PlugDP(vector<vector<int>> g) : n(g.size()), m(0), a(move(g))
    {
        // g 是矩形棋盘，1 表示障碍，0 表示必须覆盖的空格；初始化棋盘。
        assert(n > 0 && !a[0].empty()); // 调试检查，可删。
        m = a[0].size();
        assert(2 * (m + 1) <= 64); // 调试检查，可删。
        for (int i = 0; i < n; i++)
        {
            assert((int)a[i].size() == m); // 调试检查，可删。
        }
```

```
    }

    int get(unsigned long long s, int p) const
    {
        // s 是插头状态，p 是插头位置；返回该位置的 0/1/2 编码。
        return (s >> (2 * p)) & 3;
    }

    unsigned long long set(unsigned long long s, int p, int v) const
    {
        // s 是插头状态，p 是插头位置，v 是新的 0/1/2 编码；返回修改后的状态。
        return (s & ~(3ULL << (2 * p))) | ((unsigned long long)v << (2 * p));
    }

    Acc solve()
    {
        // 无参数；返回覆盖全部空格且只有一条闭合回路的方案数。
        int ex = -1;
        int ey = -1;
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < m; j++)
            {
                if (!a[i][j])
                {
                    ex = i;
                    ey = j;
                }
            }
        }
        if (ex < 0)
        {
            return 0;
        }

        map<unsigned long long, Acc> cur;
        cur[0] = 1;
        Acc ans = 0;
        for (int i = 0; i < n; i++)
        {
            map<unsigned long long, Acc> nxt;
            for (auto [s, c] : cur)
            {
                nxt[s << 2] += c;
            }
            cur = nxt;
            for (int j = 0; j < m; j++)
            {
                map<unsigned long long, Acc> nx;
                for (auto [s, c] : cur)
                {
                    int L = get(s, j), U = get(s, j + 1);
```

```
                if (a[i][j])                        // 障碍：左、上插头必须都空
                {
                    if (!L && !U)
                    {
                        nx[s] += c;
                    }
                    continue;
                }
                if (!L && !U)                       // ┌：新建一对插头
                {
                    if (i + 1 < n && j + 1 < m)
                    {
                        nx[set(set(s, j, 1), j + 1, 2)] += c;
                    }
                }
                else if (!L || !U)                  // 直行/拐弯：单插头可向右或向下
                {
                    int t = L ? L : U;
                    if (j + 1 < m)
                    {
                        nx[set(set(s, j, 0), j + 1, t)] += c;
                    }
                    if (i + 1 < n)
                    {
                        nx[set(set(s, j, t), j + 1, 0)] += c;
                    }
                }
                else if (L == 1 && U == 1)          // ( ( ：找 U 的配对 ) 改成 (
                {
                    unsigned long long t = set(set(s, j, 0), j + 1, 0);
                    int cnt = 1;
                    for (int k = j + 2; k <= m; k++)
                    {
                        int v = get(t, k);
                        if (v == 1)
                        {
                            cnt++;
                        }
                        else if (v == 2)
                        {
                            cnt--;
                            if (cnt == 0)
                            {
                                t = set(t, k, 1);
                                break;
                            }
                        }
                    }
                    nx[t] += c;
                }
                else if (L == 2 && U == 2)          // ) ) ：找 L 的配对 ( 改成 )
                {
```

```
                        unsigned long long t = set(set(s, j, 0), j + 1, 0);
                        int cnt = 1;
                        for (int k = j - 1; k >= 0; k--)
                        {
                            int v = get(t, k);
                            if (v == 2)
                            {
                                cnt++;
                            }
                            else if (v == 1)
                            {
                                cnt--;
                                if (cnt == 0)
                                {
                                    t = set(t, k, 2);
                                    break;
                                }
                            }
                        }
                        nx[t] += c;
                    }
                    else if (L == 2 && U == 1)           // ) ( : 两条链合并，两插头消去
                    {
                        nx[set(set(s, j, 0), j + 1, 0)] += c;
                    }
                    else                                 // ( ) : 相邻配对，闭合回路
                    {
                        // 仅当此为末空格、且闭合后轮廓再无其它插头（整条回路收尾）才计入
                        if (i == ex && j == ey && set(set(s, j, 0), j + 1, 0) == 0)
                        {
                            ans += c;
                        }
                    }
                }
                cur = nx;
            }
            map<unsigned long long, Acc> f;
            for (auto [s, c] : cur)
            {
                if (!get(s, m))
                {
                    f[s] += c;
                }
            }
            cur = f;
        }
        return ans;
    }
};
```

用 `PlugDP q(g)` 构造非空矩形棋盘，1 表示障碍、0 表示必须覆盖的空格，宽度须满足 $m \le 31$；默认 `q.solve()` 返回 `long long` 方案数，必须保证方案数累加能装入 `Acc`，可能超过 64 位时使用

`PlugDP<__int128>`。全为障碍时返回 0。状态数随轮廓宽度指数增长，应把较小的一维放在宽度方向。铺砖、指定端点路径和最优权值需要重新定义插头转移，不能只改计数合并运算。

下面用三张全空棋盘的已知答案和一张带障碍图检查主要转移：

```
signed main()
{
    vector<array<int, 3>> chk{{2, 3, 1}, {3, 3, 0}, {4, 4, 6}};
    for (auto [n, m, want] : chk)
    {
        vector<vector<int>> g(n, vector<int>(m));
        assert(PlugDP(g).solve() == want);
    }
    vector<vector<int>> g{{0, 0, 0, 0}, {0, 1, 1, 0}, {0, 0, 0, 0}};
    assert(PlugDP(g).solve() == 1);
    cout << "OK\n";
}
```

设轮廓合法状态数为 S，时间 $O(nmS)$、额外空间 $O(S)$。

12.3 DP 优化

12.3.001 单调队列优化

当转移是 $f[i]=c_i+\min_{i-k\le j<i} f[j]$ 时，只需维护最近 k 个 $f[j]$ 的最小值。双端队列按 f 值单调递增：队首过期就弹出，新状态入队前弹掉所有不优于它的队尾；每个下标至多进出各一次，便把朴素 $O(nk)$ 降到 $O(n)$。通用的窗口最值、单调栈和最大矩形实现见 2.2.008 单调栈 / 单调队列，本条只保留 DP 转移。

```
vector<int> windowDP(const vector<int> &c, int k)
{
    // c[i] 是进入状态 i 的代价，k 是前驱窗口长度；返回 f[0]=c[0] 的最小代价 DP。
    if (c.empty())
    {
        return {};
    }
    assert(k > 0); // 调试检查，可删。
    int n = c.size();
    vector<int> f(n);
    deque<int> q;
    f[0] = c[0];
    q.push_back(0);
    for (int i = 1; i < n; i++)
    {
        while (!q.empty() && q.front() < i - k)
        {
            q.pop_front();
        }
        f[i] = f[q.front()] + c[i];
        while (!q.empty() && f[q.back()] >= f[i])
        {
```

```
            q.pop_back();
        }
        q.push_back(i);
    }
    return f;
}
```

`windowDP(c,k)` 返回上述递推的全部状态，空代价序列返回空；非空时 $k>0$，所有状态值须能由 `int` 表示。时间 $O(n)$、额外空间 $O(n)$。代价满足四边形不等式的区间合并用 12.3.002 四边形不等式优化，最优决策随位置单调用 12.3.003 分治优化，转移写成直线求最值用 12.3.004 斜率优化 / 李超树优化。

12.3.002 四边形不等式优化

半开区间合并递推为 $dp[l][r]=\min_{l<k<r}\left(dp[l][k]+dp[k][r]\right)+w(l,r)$，单元素基例 $dp[i][i+1]=0$。若代价 w 满足四边形不等式（$w(a,c)+w(b,d)\le w(a,d)+w(b,c)$，$a\le b\le c\le d$）且区间单调，则最优决策点满足 $\mathrm{opt}[l][r-1]\le \mathrm{opt}[l][r]\le \mathrm{opt}[l+1][r]$。Knuth 优化把断点限制在这两个旧决策之间，总枚举量降到 $O(n^2)$；使用前必须证明条件。

```
// 知识点 12.3.002：满足 Knuth 条件的区间 DP。
namespace quad_opt
{
template <class Acc = long long, class Cost>
Acc knuth(int n, Cost w)
{
    // n 是序列长度，w(l,r) 返回半开区间代价；返回 dp[0][n]，调用者必须证明 Knuth 单调性条件。
    assert(n >= 0); // 调试检查，可删。
    vector<vector<Acc>> dp(n + 1, vector<Acc>(n + 1));
    vector<vector<int>> opt(n + 1, vector<int>(n + 1));
    for (int i = 0; i < n; i++)
    {
        opt[i][i] = i;
        opt[i][i + 1] = i + 1;
    }
    opt[n][n] = n;
    for (int len = 2; len <= n; len++)
    {
        for (int l = 0; l + len <= n; l++)
        {
            int r = l + len;
            optional<Acc> val;
            int lo = max(l + 1, opt[l][r - 1]);
            int hi = min(r - 1, opt[l + 1][r]);
            for (int k = lo; k <= hi; k++)
            {
                Acc cur = dp[l][k] + dp[k][r] + (Acc)w(l, r);
                if (!val || cur < *val)
                {
                    val = cur;
                    opt[l][r] = k;
```

```
                }
            }
            assert(val); // Knuth 条件下该区间至少有一个候选。
            dp[l][r] = *val;
        }
    }
    return dp[0][n];
}
}
```

`knuth(n,w)` 默认返回 `long long` 的 $dp[0][n]$；$n=0$ 或 1 返回 0。调用者须证明 Knuth 条件，并保证区间代价和候选加法能由 `Acc` 表示；需要宽状态时调用 `knuth<__int128>`。局部 `optional` 只表示本区间尚未枚举候选，因此最优值恰等于类型上界也不会和哨兵混淆。时间、空间均为 $O(n^2)$。

下面检查空序列、单元素与三堆石子的已知最优值：

```
signed main()
{
    auto cost = [](const vector<int> &a)
    {
        int n = a.size();
        vector<int> pre(n + 1);
        for (int i = 0; i < n; i++)
        {
            pre[i + 1] = pre[i] + a[i];
        }
        auto w = [&](int l, int r)
        {
            return pre[r] - pre[l];
        };
        return quad_opt::knuth(n, w);
    };
    assert(cost({}) == 0);
    assert(cost({7}) == 0);
    assert(cost({1, 2, 3}) == 9);
    assert(cost({4, 3, 3, 4}) == 28);
}
```

12.3.003 分治优化

分治优化针对分层转移 $dp[i]=\min_{j<i}\left(\mathrm{pre}[j]+w(j,i)\right)$，当本层最优决策点 $\mathrm{opt}(i)$ 随 i 单调不减时适用。做法是分治处理下标区间 $[l,r]$：先算中点 mid，在给定候选决策区间 $[ql,qr]$ 里暴力找出它的最优决策 $best$；由单调性，$[l,mid-1]$ 的最优决策都 $\le best$、$[mid+1,r]$ 的都 $\ge best$，于是递归时把候选区间分别收窄到 $[ql,best]$ 和 $[best,qr]$。每层递归的决策枚举量之和是 $O(n)$、递归 $O(\log n)$ 层，故代价函数被调用 $O(n\log n)$ 次。它与 Knuth 的区别：这里是”一层依赖上一层”的转移，Knuth 是”区间拆两段”的转移。

```
// 知识点 12.3.003：决策单调的分治优化。
namespace dc_dp
```

```
{
template <class Acc = long long, class Cost>
vector<Acc> divideLayer(const vector<Acc> &pre, int n, Cost w)
{
    // pre[j] 是上一层状态，n 是本层最大下标，w(j,i) 是转移代价；返回本层 dp，调用者必须保证最优决策单调。
    assert(n >= 0 && (int)pre.size() == n + 1); // 调试检查，可删。
    vector<Acc> dp(n + 1);
    auto solve = [&](auto &&self, int l, int r, int ql, int qr) -> void
    {
        // self 是递归自身，[l,r] 是待算区间，[ql,qr] 是候选决策范围；填充 dp，无返回值。
        if (l > r)
        {
            return;
        }
        int mid = l + (r - l) / 2;
        int best = ql;
        optional<Acc> val;
        for (int j = ql; j <= min(qr, mid - 1); j++)
        {
            Acc cur = pre[j] + (Acc)w(j, mid);
            if (!val || cur < *val)
            {
                val = cur;
                best = j;
            }
        }
        assert(val); // 候选区间非空，且 pre 中没有不可达状态。
        dp[mid] = *val;
        self(self, l, mid - 1, ql, best);
        self(self, mid + 1, r, best, qr);
    };
    solve(solve, 1, n, 0, n - 1);
    return dp;
}
}
```

`divideLayer(pre,n,w)` 要求 `pre[0..n-1]` 全部是可达状态，且最优决策单调；默认返回 `long long` 的本层结果。若上一层含不可达状态，应先过滤候选或另用 `optional` 状态，不能拿类型上界传入本函数。所有转移和须能由 `Acc` 表示；需要宽状态时调用 `divideLayer<__int128>`。单层调用代价函数 $O(n \log n)$ 次，额外空间 $O(n + \log n)$。

下面检查空层、单点、并列决策与平方代价的已知结果：

```
signed main()
{
    auto solve = [](const vector<int> &s, const vector<int> &pre)
    {
        int n = (int)s.size() - 1;
        auto w = [&](int j, int i)
        {
```

```
            long long d = s[i] - s[j];
            return d * d;
        };
        return dc_dp::divideLayer(pre, n, w);
    };
    assert(solve({0}, {0}) == vector<long long>({0}));
    assert(solve({0, 5}, {7, 0}) == vector<long long>({0, 32}));
    assert(solve({0, 1, 2, 3}, {0, 0, 0, 0}) == vector<long long>({0, 1, 1, 1}));
}
```

12.3.004 斜率优化 / 李超树优化

把转移 $dp[i] = \min_j(a_j x_i + b_j)$ 中的每个决策看成直线即可做斜率优化。若斜率按不增顺序加入、查询横坐标也不减，维护下凸壳的双端队列就能均摊 $O(1)$：新线使队尾交点次序逆转时弹尾，查询时第二条线不劣于第一条便弹头。求最大值时只有在斜率、截距和函数值都能安全取反时，才把每条线同时取反、复用本最小值凸壳并对非空查询结果取反；这样斜率加入顺序也随取反自然变成原斜率不减，不能只把查询比较改成 `>=`，遇到最小负数要完整反向比较。斜率或查询无序时改用 3.7.001 李超线段树，本章不复制数据结构实现。

```
template <class Acc = long long>
struct MonoHull
{
    // 当前凸壳按最小函数值维护。改求最大值可在 k、b 与函数值都能安全取反时同时取反，
    // 继续调用本结构，最后把非空 query 结果取反；否则必须完整反向交点与比较判定。
    using U128 = unsigned __int128;

    struct Line
    {
        // k、b 是直线 y=kx+b 的斜率和截距。
        int k;
        int b;

        Acc get(int x) const
        {
            // x 是查询横坐标；返回直线在 x 处的函数值。
            return (Acc)k * x + b;
        }
    };
    // q 是按交点顺序排列的下凸壳，last 是上一次查询横坐标。
    deque<Line> q;
    optional<int> last;

    static pair<bool, U128> product(int a, int b, int c, int d)
    {
        // 返回 (a-b)(c-d) 的符号与绝对值，true 表示负数。
        auto diff = [](int x, int y)
        {
            if (x >= y)
            {
```

```
                return pair{false, (unsigned long long)x - (unsigned long long)y};
            }
            return pair{true, (unsigned long long)y - (unsigned long long)x};
        };
        auto x = diff(a, b), y = diff(c, d);
        return {x.first != y.first, (U128)x.second * y.second};
    }

    static bool less(pair<bool, U128> a, pair<bool, U128> b)
    {
        // 比较两个符号-绝对值表示的整数。
        if (a.first != b.first)
        {
            return a.first;
        }
        return a.first ? a.second > b.second : a.second < b.second;
    }

    void add(int k, int b)
    {
        // k、b 是新直线的斜率和截距；要求斜率按不增顺序加入。
        assert(q.empty() || k <= q.back().k); // 调试检查，可删。
        if (!q.empty() && k == q.back().k)
        {
            if (q.back().b <= b)
            {
                return;
            }
            q.pop_back();
        }
        Line c{k, b};
        while (q.size() >= 2)
        {
            Line a = q[q.size() - 2];
            Line d = q.back();
            auto l = product(d.b, a.b, d.k, c.k);
            auto r = product(c.b, d.b, a.k, d.k);
            if (less(l, r))
            {
                break;
            }
            q.pop_back();
        }
        q.push_back(c);
    }

    optional<Acc> query(int x)
    {
        // x 是不小于上次查询的横坐标；无直线时返回空，否则返回最小函数值。
        assert(!last || *last <= x); // 调试检查，可删。
        last = x;
        while (q.size() >= 2 && q[1].get(x) <= q[0].get(x))
```

```
        {
            q.pop_front();
        }
        if (q.empty())
        {
            return nullopt;
        }
        return q.front().get(x);
    }
};
```

用 `MonoHull h` 建空凸壳，斜率不增插入、横坐标不减查询。函数值默认用 `long long`，需要宽答案时使用 `MonoHull<__int128>`；交点比较把完整 64 位差的绝对值乘积放进 `unsigned __int128`，不会溢出有符号 128 位。无直线时返回空；插入和查询均摊 $O(1)$，空间 $O(q)$。顺序不单调时改用 3.7.001 李超线段树。

12.3.005 WQS 二分

WQS 二分（又称带权二分 / Aliens trick）处理「恰好取 K 个」且最优值关于 K 下凸的问题。设 $g(K)$ 为恰取 K 个的最优代价，g 下凸。去掉个数限制、给每次「取」附加一个罚 λ，做无约束最优化

$$h(\lambda) = \min_{c} \left(g(c) + \lambda c \right).$$

λ 越大越抑制取用，最优个数 $c^*(\lambda)$ 单调不增；二分 λ 使 $c^* = K$，再由凸共轭关系

$$g(K) = \max_{\lambda} \left(h(\lambda) - \lambda K \right)$$

反推真实代价。由于斜率 $g(c) - g(c+1)$ 为整数，整数 λ 足以命中每个凸包顶点。两个实现要点：

- check 的 DP 在同代价时按「段数最多」打破平局，使 $c^*(\lambda)$ 单调、且能覆盖凸包边上的每个整点；
- 恢复答案时取「最多段数仍 $\geq K$ 的最大 λ」。若误取「段数 $\leq K$ 的最小 λ」，会选到相邻凸包边、令 $h(\lambda) - \lambda K$ 偏小（实测偏小 1）。

下例把正数数组恰好分成 K 个连续非空段，最小化 $\sum$(段和)^2。罚值上界不再写死：设总和为 S，当 $\lambda = S^2 + 1$ 时，任何至少两段的含罚代价都严格大于单段方案，因此它一定跨过所有 $K > 1$ 的目标边界。

```
template <class Acc = long long>
Acc wqsPartition(const vector<int> &pre, int K)
{
    // pre 是正数数组的前缀和，K 是目标段数；返回恰分 K 段的最小平方和。
    static_assert(numeric_limits<Acc>::is_integer && numeric_limits<Acc>::is_signed &&
                  sizeof(Acc) >= sizeof(long long));
    int n = (int)pre.size() - 1;
    assert(n >= 1 && 1 <= K && K <= n && pre[0] == 0); // 调试检查，可删。
    for (int i = 1; i <= n; i++)
    {
        assert(pre[i] > pre[i - 1]); // 调试检查，可删。
```

```
    }
    Acc total = pre[n];
    if (K == 1)
    {
        return total * total;
    }
    auto check = [&](Acc lambda)
    {
        // lambda 是每段罚值；返回含罚最优代价及同代价下的最多段数。
        vector<Acc> dp(n + 1);
        vector<int> cnt(n + 1);
        for (int i = 1; i <= n; i++)
        {
            bool found = false;
            for (int j = 0; j < i; j++)
            {
                Acc seg = (Acc)pre[i] - pre[j];
                Acc cur = dp[j] + seg * seg + lambda;
                int c = cnt[j] + 1;
                if (!found || cur < dp[i] || (cur == dp[i] && c > cnt[i]))
                {
                    dp[i] = cur;
                    cnt[i] = c;
                    found = true;
                }
            }
        }
        return pair<Acc, int>{dp[n], cnt[n]};
    };
    Acc lo = 0;
    Acc hi = total * total + 1;
    Acc best = 0;
    while (lo <= hi)
    {
        Acc mid = lo + (hi - lo) / 2;
        if (check(mid).second >= K)
        {
            best = mid;
            lo = mid + 1;
        }
        else
        {
            hi = mid - 1;
        }
    }
    return check(best).first - best * K;
}
```

`wqsPartition(pre,K)` 默认用 `long long` 返回恰好 K 段的最小平方和，需要宽代价时调用 `wqsPartition<__int128>`。调用者须保证 S^2+1、每个 `dp[j]+seg^2+lambda`、二分边界及最终的 `best*K` 都能由 `Acc` 表示。当前 `check` 为 $O(n^2)$，总时间 $O(n^2 \log S)$、额外空间 $O(n)$；接入斜率优化后可进一步降低检查复杂度。

下面检查一段、每点单独成段和一组最优切分不在端点的例子：

```
signed main()
{
    assert(wqsPartition(vector<int>{0, 5}, 1) == 25);
    assert(wqsPartition(vector<int>{0, 1, 2, 3, 4}, 4) == 4);
    assert(wqsPartition(vector<int>{0, 3, 4, 8, 9, 14}, 3) == 66);
}
```

12.3.006 SMAWK 全单调矩阵优化

当转移代价形成全单调矩阵（特别是 Monge 矩阵）时，SMAWK 在线性时间 $O(H+W)$ 内求出 H 行、W 列矩阵每行的最优列。回调 `better(r,x,y)` 表示第 `r` 行的 `y` 是否优于 `x`；相等时应返回 `false` 以保留最左最优列。它比本章分治优化的 $O(H \log W)$ 更快，但要求更强：所有行的最优列必须单调，且候选列的比较满足全单调性。

```
template <class F>
vector<int> smawk(int rows, int cols, F better)
{
    // rows、cols 是矩阵行列数；better(row, oldCol, newCol) 表示新列严格更优；返回每行最左最优列。
    assert(rows >= 0 && cols > 0);
    if (rows == 0)
    {
        return {};
    }
    auto solve = [&](auto &&self, const vector<int> &rs, const vector<int> &cs) -> vector<int>
    {
        vector<int> reduced;
        reduced.reserve(cs.size());
        for (int col : cs)
        {
            while (!reduced.empty())
            {
                int row = rs[(int)reduced.size() - 1];
                if (!better(row, reduced.back(), col))
                {
                    break;
                }
                reduced.pop_back();
            }
            if ((int)reduced.size() < (int)rs.size())
            {
                reduced.push_back(col);
            }
        }
        vector<int> odd;
        for (int i = 1; i < (int)rs.size(); i += 2)
        {
            odd.push_back(rs[i]);
        }
```

```
        vector<int> ans(rs.size());
        if (!odd.empty())
        {
            vector<int> oa = self(self, odd, reduced);
            for (int i = 0; i < (int)oa.size(); i++)
            {
                ans[2 * i + 1] = oa[i];
            }
        }
        int p = 0;
        for (int i = 0; i < (int)rs.size(); i += 2)
        {
            int lim = (i + 1 == (int)rs.size() ? reduced.back() : ans[i + 1]);
            int best = reduced[p];
            while (reduced[p] < lim)
            {
                ++p;
                if (better(rs[i], best, reduced[p]))
                {
                    best = reduced[p];
                }
            }
            ans[i] = best;
        }
        return ans;
    };
    vector<int> rs(rows), cs(cols);
    iota(rs.begin(), rs.end(), 0);
    iota(cs.begin(), cs.end(), 0);
    return solve(solve, rs, cs);
}
```

空间复杂度为 $O(H+W)$（递归栈计入 $O(\log H)$）。若仅能证明最优列单调而不能证明全单调，应使用 `monoMin` 或 `divideLayer`；误用 SMAWK 会产生错误答案。

```
signed main()
{
    vector<vector<int>> a{{0, 4, 9}, {1, 1, 5}, {4, 2, 2}, {8, 3, 1}};
    auto p = smawk(4, 3, [&](int r, int x, int y) { return a[r][y] < a[r][x]; });
    assert((p == vector<int>{0, 0, 1, 2}));
}
```

12.3.007 Slope Trick

Slope Trick 维护分段线性凸函数

$$f(x)=mn+\sum_{a\in L}\max(a-x,0)+\sum_{b\in R}\max(x-b,0),$$

其中 L 用大根堆、R 用小根堆保存拐点，并保持 $\max L\le\min R$，于是 f 在 $[\max L,\min R]$ 上取到最小值 mn。加入 $\max(a-x,0)$ 时，若 $a>\min R$，最小值增加 $a-\min R$；之后把 a 放入 R、再把

R 的最小拐点移到 L，恢复不变量。$\max(x-a,0)$ 对称处理。`dl`、`dr` 是 L、R 整体平移量的懒标记。

常用的函数变换都只改堆或标记：$f(x) \leftarrow \min_{x-b\le y\le x-a} f(y)$ 等价于左半段右移 a、右半段右移 b；前缀最小值 $\min_{y\le x} f(y)$ 是删去右侧上升段，后缀最小值对称。

```
struct SlopeTrick
{
    // 维护分段线性凸函数 f(x) = mn + sum_{a in L} max(a-x,0) + sum_{b in R} max(x-b,0),
    // 且 max L <= min R, [max L, min R] 即最小值区间。dl、dr 是 L、R 整体的平移量（懒标记）。
    priority_queue<int> l;                          // l 存 L 中各拐点减去 dl。
    priority_queue<int, vector<int>, greater<>> r;  // r 存 R 中各拐点减去 dr。
    int dl = 0, dr = 0, mn = 0;                     // mn 是 f 的最小值。

    void addLeft(int a)
    {
        // 给 f 加上 max(a-x,0)。
        if (!r.empty())
        {
            mn += max(0LL, a - (r.top() + dr));
        }
        r.push(a - dr);
        int v = r.top() + dr;
        r.pop();
        l.push(v - dl);
    }

    void addRight(int a)
    {
        // 给 f 加上 max(x-a,0)。
        if (!l.empty())
        {
            mn += max(0LL, l.top() + dl - a);
        }
        l.push(a - dl);
        int v = l.top() + dl;
        l.pop();
        r.push(v - dr);
    }

    void addAbs(int a)
    {
        // 给 f 加上 |x-a|。
        addLeft(a);
        addRight(a);
    }

    void shift(int a, int b)
    {
        // a<=b; f(x) <- min_{x-b<=y<=x-a} f(y)，即左半段右移 a、右半段右移 b；a==b 时为平移。
        assert(a <= b); // 调试检查，可删。
        dl += a;
        dr += b;
```

```
    }

    void prefixMin()
    {
        // f(x) <- min_{y<=x} f(y)：去掉右侧上升段。
        r = {};
    }

    void suffixMin()
    {
        // f(x) <- min_{y>=x} f(y)：去掉左侧下降段。
        l = {};
    }

    pair<optional<int>, optional<int>> argmin() const
    {
        // 返回最小值区间的左右端点，无界一侧为空。
        optional<int> x, y;
        if (!l.empty())
        {
            x = l.top() + dl;
        }
        if (!r.empty())
        {
            y = r.top() + dr;
        }
        return {x, y};
    }

    int eval(int x) const
    {
        // 返回 f(x)，O(|L|+|R|) 并复制堆，仅用于调试或少量查询。
        auto a = l;
        auto b = r;
        int ans = mn;
        for (; !a.empty(); a.pop())
        {
            ans += max(0LL, a.top() + dl - x);
        }
        for (; !b.empty(); b.pop())
        {
            ans += max(0LL, x - b.top() - dr);
        }
        return ans;
    }
};
```

公开成员 `mn` 是当前最小值；`addLeft`、`addRight`、`addAbs` 加入一次项，`shift(a,b)` 要求 $a \le b$，`prefixMin`、`suffixMin` 取前后缀最小值，`argmin()` 返回最小值区间的两个端点（无界一侧为空），`eval(x)` 复制两个堆求函数值，只用于调试或少量查询。加入与平移为 $O(\log n)$，`argmin` 为 $O(1)$，`eval` 为 $O(n \log n)$；n 是加入的拐点数，所有值须在 `long long` 范围内。

典型模型是“把数列改成满足单调或差分约束的最小代价”。例如单调不降化：设 $f_i(x)$ 是前 i 项且 $b_i = x$ 的最小代价，则 $f_i(x) = \min_{y \le x} f_{i-1}(y) + |x - a_i|$，每读入 a_i 先取前缀最小值再加入 $|x - a_i|$。值域小时也可以直接做 $O(nV)$ 的 DP；其他凸性结构见 12.3.004 斜率优化 / 李超树优化 与 12.3.005 WQS 二分。

```
signed main()
{
    vector<int> a{3, 1, 4, 1, 5, 9, 2, 6}; // 改成单调不降的最小代价 sum |a_i-b_i|。
    SlopeTrick f;
    for (int x : a)
    {
        f.prefixMin();
        f.addAbs(x);
    }
    auto [l, r] = f.argmin();
    assert(f.mn == 12 && *l == 6 && *r == 6 && f.eval(0) == 31);
}
```

13. 博弈论、概率与杂项数学

13.1 博弈论

13.1.001 Bash 与扩展 Bash

单堆 n 颗石子、每次取 1 到 m 颗、取走最后一颗者胜，这就是 Bash 博弈。把”轮到某人时必输”的局面称为 P 局面，其余为 N 局面：$n=0$ 是 P 局面（无子可取即已输），而 n 是 $m+1$ 的倍数时先手不论取几颗，对手都能补足到下一个 $m+1$ 的倍数，始终把 P 局面还给先手，直至 0，故 $n \bmod (m+1)=0$ 恰为 P 局面。反之余数非零时先手一步取到最近的倍数即可反制。要点是这条结论只在”连续可取 $1..m$、正常取胜”规则下成立，一旦改成不连续可取集合或最后取者负，必须回到必败态递推或 SG 重新求周期，见 13.1.008 取石子游戏。

```cpp
bool bashWin(long long n, long long m)
{
    // n 是当前石子数，m 是单步最多取石数；返回先手是否必胜。
    assert(n >= 0 && m >= 1); // 调试检查，可删。
    if (m == numeric_limits<long long>::max())
    {
        return n != 0;
    }
    return n % (m + 1) != 0;
}
```

`bashWin(n, m)` 要求 $n \geq 0$、$m \geq 1$，返回先手是否必胜，$m = LLONG_MAX$ 时单独按 `n!=0` 处理，其他情况用 64 位取模；$O(1)$ 时间与空间。多堆 Bash 是独立子游戏之和，须先对每堆求 SG 值（单堆 Bash 的 SG 值即 $n \bmod (m+1)$）再异或，见 13.1.004 SG；取数集合不连续时改用 13.1.008 取石子游戏 的通用 SG 表。

13.1.002 Nim、Anti-Nim、Nim-K

普通 Nim 是若干独立石子堆、每次从任一堆取任意正数、取走最后一颗者胜。Bouton 定理指出先手必胜当且仅当所有堆大小的异或和 $\bigoplus a_i \neq 0$。Anti-Nim（Misère，取走最后一颗者负）只在全为小堆时改变结论：若所有堆都不超过 1，忽略零堆后，非空堆数为正偶数时先手必胜；全空局面无步可走，先手败。只要存在大于 1 的堆，仍以异或和非零判胜。Nim-K（每步可动至多 k 堆）则检查每一二进制位上的 1 数量模 $k+1$，此处代码只给普通与 Misère 两种。

```cpp
bool nimWin(const vector<unsigned long long> &a)
{
    // 参数 a 表示 普通 Nim 各堆的非负石子数；返回 Nim 局面的先手是否必胜。
    unsigned long long x = 0;
    for (unsigned long long v : a)
    {
        x ^= v;
    }
    return x != 0;
}
```

```cpp

bool antiNimWin(const vector<unsigned long long> &a)
{
    // 参数 a 表示 反 Nim 各堆的非负石子数；返回反 Nim 局面的先手是否必胜。
    bool allOne = true;
    unsigned long long x = 0;
    int one = 0;
    for (unsigned long long v : a)
    {
        allOne &= v <= 1;
        x ^= v;
        one += v == 1;
    }
    return allOne ? one > 0 && one % 2 == 0 : x != 0;
}
```

`nimWin(a)`、`antiNimWin(a)` 接受各堆非负石子数 `a`，返回先手是否必胜，均 $O(n)$ 时间、$O(1)$ 额外空间。判必胜后要还原到具体一步时，对 Nim 找满足 $a_i \oplus X < a_i$ 的堆并取到 $a_i \oplus X$（X 为总异或）。只能移动石子而非取走的阶梯型规则见 13.1.003 阶梯 Nim，任意状态图上的一般合成见 13.1.004 SG。

13.1.003 阶梯 Nim

阶梯 Nim 把石子摆在编号 $0, 1, 2, \ldots$ 的台阶上，每步可把某台阶上任意正数颗石子移到下一级，移到第 0 级即“落地”不再参与。只有奇数编号台阶的石子异或和决定胜负：从奇数级移到偶数级对应 Nim 取石；对手从偶数级移到奇数级后，先手可把等量石子再下移一级抵消。因此先手必胜当且仅当奇数下标石子数的异或和非零。

```cpp
bool stairNimWin(const vector<unsigned long long> &a)
{
    // 参数 a 表示 从第 0 层开始记录的各阶梯石子数；返回阶梯 Nim 局面的先手是否必胜。
    unsigned long long x = 0;
    for (int i = 1; i < (int)a.size(); i += 2)
    {
        x ^= a[i];
    }
    return x != 0;
}
```

`stairNimWin(a)` 中 `a[i]` 为第 i 级台阶石子数，返回先手是否必胜，$O(n)$ 时间、$O(1)$ 额外空间。它本质是把奇数下标当作 13.1.002 Nim 的堆再异或；若移动规则不是”下一级”而是任意 DAG 上的转移，则退回 13.1.004 SG 逐状态求 mex。

13.1.004 SG、Anti-SG、Multi-SG、Every-SG

有向无环状态图上的每个局面 x，其 Sprague-Grundy 值定义为 $\mathrm{SG}(x) = \mathrm{mex}\{\mathrm{SG}(y) : x \to y\}$，其中 mex 是不在集合中的最小非负整数，无出边的终局 SG 值为 0。$\mathrm{SG}(x) = 0$ 即 P 局面。Sprague-Grundy 定理进一步说明：多个独立子游戏并列时，整体 SG 值等于各子游戏 SG 值的异或，于是普通取石、Nim、阶梯 Nim 都是它的特例。变体只改合成或终局规则而非这条 mex 递推本身——Anti-SG（取到终局者负）用 SJ

定理按”是否存在 SG> 1 的堆”分类，Multi-SG 允许一步把一堆拆成多堆需重定义后继集合，Every-SG 要求每个子游戏都走到底、按各子游戏到终局的步数奇偶合成，都不能只套一个 mex 公式。下面给出最基础的 DAG 上记忆化求全体 SG 值。

```cpp
vector<int> sg(const vector<vector<int>> &to)
{
    // to[x] 是状态 x 的后继编号表；返回每个 DAG 状态的 Sprague-Grundy 值。
    int n = to.size();
    vector<int> deg(n);
    for (int x = 0; x < n; x++)
    {
        for (int y : to[x])
        {
            assert(0 <= y && y < n); // 调试检查，可删。
            deg[y]++;
        }
    }
    queue<int> q;
    for (int x = 0; x < n; x++)
    {
        if (deg[x] == 0)
        {
            q.push(x);
        }
    }
    vector<int> ord;
    while (!q.empty())
    {
        int x = q.front();
        q.pop();
        ord.push_back(x);
        for (int y : to[x])
        {
            deg[y]--;
            if (deg[y] == 0)
            {
                q.push(y);
            }
        }
    }
    assert((int)ord.size() == n); // 调试检查，可删。
    reverse(ord.begin(), ord.end());
    vector<int> f(n);
    for (int x : ord)
    {
        vector<int> a;
        for (int y : to[x])
        {
            a.push_back(f[y]);
        }
        sort(a.begin(), a.end());
        a.erase(unique(a.begin(), a.end()), a.end());
```

```
        int g = 0;
        while (binary_search(a.begin(), a.end(), g))
        {
            g++;
        }
        f[x] = g;
    }
    return f;
}
```

`sg(to)` 要求 `to[x]` 中每个编号合法且整张状态图为 DAG，返回第 x 个状态的 $\mathrm{SG}(x)$；拓扑逆序避免合法长链的递归栈风险。设状态数 V、边数 E、最大出度 Δ，时间 $O((V+E)\log\Delta)$、空间 $O(V+E)$。有环博弈不能沿用 DAG mex；单堆规则用 13.1.008 取石子游戏 的 `subGameSG`。

13.1.005 Wythoff 博弈

Wythoff 博弈有两堆石子 (a,b)，每步可从任一堆取任意正数，或从两堆同时取相同正数，取到最后者胜。其 P 局面为 $(\lfloor k\varphi\rfloor,\lfloor k\varphi\rfloor+k)$ 及交换，$k\ge 0$，$\varphi=\frac{1+\sqrt{5}}{2}$。两列 $\lfloor k\varphi\rfloor$ 与 $\lfloor k\varphi^2\rfloor$ 构成互补 Beatty 序列。令较小堆为 a、差为 $k=b-a$，无需计算可能超出 `int` 的 $\lfloor k\varphi\rfloor$；直接用精确等价条件 $a^2-ak\le k^2<(a+1)^2-(a+1)k$ 判断当前 a 是否就是该冷位置的较小堆。

```
// 知识点 13.1.005: Wythoff 两堆取石博弈。
namespace wythoff_game
{
bool firstWins(int a, int b)
{
    // a、b 是非负石子数；返回先手是否必胜。
    assert(a >= 0 && b >= 0); // 调试检查，可删。
    if (a > b)
    {
        swap(a, b);
    }
    int k = b - a;
    __int128 x = a;
    __int128 d = k;
    __int128 z = d * d;
    bool cold = x * x - x * d <= z
                && z < (x + 1) * (x + 1) - (x + 1) * d;
    return !cold;
}
}
```

`wythoff_game::firstWins(a,b)` 接受完整非负 `int` 范围内的两堆石子数，返回先手是否必胜，$O(1)$ 时间与空间；判定全程在 `__int128` 中完成，不依赖浮点近似，也不要求冷位置坐标能由 `int` 表示。其他移动规则回到 13.1.004 SG。下面检查前几组冷局面、对称性和大数热局面：

```
signed main()
{
    for (auto [a, b] : vector<pair<int, int>>{{0, 0}, {1, 2}, {3, 5}, {4, 7}, {6, 10}, {8, 13}})
```

```
    {
        assert(!wythoff_game::firstWins(a, b));
        assert(!wythoff_game::firstWins(b, a));
    }
    assert(wythoff_game::firstWins(1, 1));
    assert(wythoff_game::firstWins(2, 3));
    assert(wythoff_game::firstWins(numeric_limits<int>::max(),
                                   numeric_limits<int>::max()));
}
```

13.1.006 Fibonacci 博弈

Fibonacci 博弈(Zeckendorf 博弈)为单堆 n 颗石子:先手首次可取 1 到 $n-1$ 颗(不能一次取空),此后每步取的数量至多为对方上一步的两倍,取到最后一颗者胜。结论是先手必败当且仅当 n 恰为 Fibonacci 数。依据是 Zeckendorf 定理——每个正整数都能唯一写成若干不相邻 Fibonacci 数之和;当 n 不是 Fibonacci 数时,其 Zeckendorf 分解中最小的那一项 F 满足"取走 F 后对手无法一步取完剩余、且 $2F$ 不足以够到下一个更大的 Zeckendorf 项",于是取 F 就是必胜首步,反复施加把对手逼入 Fibonacci 数的冷态。下面的 `zeckMin` 正是求这个最小 Fibonacci 项。

```
// 知识点 13.1.006: Fibonacci 取石的最优首取量。
namespace fib_game
{
int zeckMin(int n)
{
    // n 是正石子数;返回 Zeckendorf 分解中的最小 Fibonacci 项。
    assert(n > 0); // 调试检查,可删。
    vector<int> f = {1, 2};
    while (f.back() <= n - f[f.size() - 2])
    {
        f.push_back(f.back() + f[f.size() - 2]);
    }
    int last = 0;
    for (auto it = f.rbegin(); it != f.rend(); ++it)
    {
        if (*it <= n)
        {
            n -= *it;
            last = *it;
        }
    }
    return last;
}
}
```

`fib_game::zeckMin(n)` 要求 $n > 0$,返回其 Zeckendorf 分解的最小 Fibonacci 项:该值等于 n 时说明 n 本身是 Fibonacci 数(先手必败),否则它就是必胜首取量。Fibonacci 数按 φ^k 指数增长,故枚举到 n 只需 $O(\log n)$ 项,时间与空间均 $O(\log n)$。这是单堆特殊规则,与依赖异或合成的 13.1.002 Nim、13.1.004 SG 不同一类,规则再变则回到 13.1.008 取石子游戏 的暴力 SG 递推验证周期。

13.1.007 树上删边、无向图删边游戏

树上删边游戏（Green Hackenbush）中，一棵以地面为根的树，每步删去一条边，失去地面连通性的部分整片脱落。Colon Principle 给出递推：挂着子树的一条边贡献 $\mathrm{SG}(\text{子树})+1$，同一父节点下各分支独立取异或，因此 $\mathrm{SG}(u)=\bigoplus_{v\in\text{子节点}}(\mathrm{SG}(v)+1)$，叶子为 0。一般无向图的环不是独立分支，不能直接套树公式。

```cpp
// 知识点 13.1.007：树上删边 Green Hackenbush 的 SG 值。
namespace edge_del
{
int treeSg(const vector<vector<int>> &g, int root = 0)
{
    // g 是无向树，root 是地面根；返回整个局面的 SG 值。
    int n = g.size();
    assert(n > 0 && 0 <= root && root < n); // 调试检查，可删。
    int deg = 0;
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u])
        {
            assert(0 <= v && v < n); // 调试检查，可删。
            deg++;
        }
    }
    assert(deg == 2 * (n - 1)); // 调试检查，可删。
    vector<int> par(n, -2);
    int seen = 0;
    auto dfs = [&](auto &&self, int u, int p) -> int
    {
        par[u] = p;
        seen++;
        int ans = 0;
        for (int v : g[u])
        {
            if (v == p)
            {
                continue;
            }
            assert(par[v] == -2); // 调试检查，可删。
            ans ^= self(self, v, u) + 1;
        }
        return ans;
    };
    int ans = dfs(dfs, root, -1);
    assert(seen == n); // 调试检查，可删。
    return ans;
}
}
```

`edge_del::treeSg(g,root)` 要求 `g` 是以合法根覆盖全部点的无向树，返回整局 SG 值，非零即先手必胜；时间与额外空间均为 $O(n)$，链形树的递归深度为 $O(n)$，实际可承受深度取决于评测环境的线程栈。多

棵接地树并列时异或各自结果；含环图不能调用本函数。

13.1.008 取石子游戏

减法游戏（Subtraction Game）是最一般的单堆取石模型：给定允许取走的正数集合 S，每步从堆里取走 S 中某个数量，无法行动者负。其 SG 值由 $\mathrm{SG}(x)=\mathrm{mex}\{\mathrm{SG}(x-d):d\in S,\ d\le x\}$ 逐点递推，$\mathrm{SG}(0)=0$，先手必胜当且仅当 $\mathrm{SG}(n)\neq 0$。有限取数集合的 SG 序列最终一定进入周期（状态转移只依赖前 $\max S$ 项），但周期长度须由算出的表验证，不能凭规则臆断。因单点 SG 值不超过后继个数 $|S|$，判 mex 时只需大小 $|S|+1$ 的标记数组。这份 `subGameSG` 是 Bash、限取集合等 13.1.001 Bash 无法用简单模运算覆盖的规则的通用求解器。

```
// 知识点 13.1.008：固定取法集合的取石子 SG 表。
namespace stone_game
{
vector<int> subGameSG(int n, const vector<int> &take)
{
    // n 是最大石子数，take 是每次允许取走的正数；返回 0..n 的 SG 值。
    assert(n >= 0); // 调试检查，可删。
    vector<int> step = take;
    sort(step.begin(), step.end());
    step.erase(unique(step.begin(), step.end()), step.end());
    for (int d : step)
    {
        assert(d > 0); // 调试检查，可删。
    }
    vector<int> sg(n + 1);
    for (int x = 1; x <= n; x++)
    {
        vector<char> seen(step.size() + 1);
        for (int d : step)
        {
            if (d <= x && sg[x - d] < (int)seen.size())
            {
                seen[sg[x - d]] = 1;
            }
        }
        while (sg[x] < (int)seen.size() && seen[sg[x]])
        {
            sg[x]++;
        }
    }
    return sg;
}
}
```

`stone_game::subGameSG(n,take)` 要求 $n\ge 0$ 且每种取数为正；函数先去重，再返回 $0..n$ 的 SG 表。设去重后取数种类为 k，时间 $O(nk+k\log k)$、空间 $O(n+k)$。多堆合成时异或各堆 SG；连续取 $1..m$ 时直接用 13.1.001 Bash。

13.1.009 64 位 nim-product

有限 nimber 在异或和 nim-product 下构成特征为 2 的域。直接按 mex 定义相乘不可用；利用 Conway 域塔，先把 $[0, 2^{16})$ 的非零元素写成一个原始元的幂，再把 32 位和 64 位元素分别视为二次扩域元素做 Karatsuba 式合并。`basis` 给出该原始元乘法在二进制基下的线性映射，指数表中的偏移 3、6 分别完成乘 2^{15} 及其平方；这些常数属于 nimber 域结构，不是普通有限域任取的不可约多项式。

```
struct NimProduct64
{
    inline static array<int, 131075> ex{}; // ex[i] 是 16 位 nimber 原始元的 i 次幂，周期 65535，尾部延长供指数相加后直接查表。
    inline static array<int, 65536> lg{}; // lg[x] 是非零元 x 关于原始元的离散对数。
    inline static bool ready = false; // ready 表示两张表是否已建立。

    static void init()
    {
        // 无参数；首次调用时建 16 位 nimber 原始元的幂表 ex（周期 65535，尾部延长供指数和取用）与离散对数表 lg。
        if (ready)
        {
            return;
        }
        ready = true;
        const array<int, 16> basis = {
            10279, 15417, 35722, 52687, 44124, 62628, 15661, 5686,
            3862, 1323, 334, 647, 61560, 20636, 4267, 8445
        };
        array<int, 65536> nxt{};
        for (int i = 0; i < 16; i++)
        {
            for (int s = 0; s < (1LL << i); s++)
            {
                nxt[s | 1LL << i] = nxt[s] ^ basis[i];
            }
        }
        ex[0] = 1;
        for (int i = 0; i < 65534; i++)
        {
            ex[i + 1] = nxt[ex[i]];
        }
        for (int i = 0; i < 65535; i++)
        {
            lg[ex[i]] = i;
            ex[65535 + i] = ex[i];
        }
        for (int i = 0; i < 5; i++)
        {
            ex[131070 + i] = ex[i];
        }
    }

```

```
    static int p16(int a, int b, int shift = 0)
    {
        // a、b 是 16 位 nimber，须已 init；返回 a*b*g^shift（g 为原始元），a 或 b 为 0 时返回 0。
        // shift 是附加指数：3 即再乘 nimber 2^15，6 即再乘其平方。
        if (!a || !b)
        {
            return 0;
        }
        return ex[(int)lg[a] + lg[b] + shift];
    }

    static int p32high(int a, int b)
    {
        // a、b 是 32 位 nimber，须已 init；返回 nim 积 a*b*2^31，供 64 位合并时的低半部分使用。
        int al = a & 65535, ah = a >> 16;
        int bl = b & 65535, bh = b >> 16;
        int x = p16(al, bl, 3);
        int y = p16(ah, bh, 6);
        int z = p16(al ^ ah, bl ^ bh, 3);
        return (int)(y ^ z) << 16 | p16(z ^ x, 1, 3);
    }

    static int p32(int a, int b)
    {
        // a、b 是 32 位 nimber，须已 init；按 16 位子域上的二次扩域做 Karatsuba 合并，返回 nim 积
        ↪ a*b。
        int al = a & 65535, ah = a >> 16;
        int bl = b & 65535, bh = b >> 16;
        int c = p16(al, bl);
        return (int)(p16(al ^ ah, bl ^ bh) ^ c) << 16 |
               (p16(ah, bh, 3) ^ c);
    }

    static int product32(int a, int b)
    {
        // a、b 是 32 位 nimber；自动初始化后返回它们的积。
        init();
        return p32(a, b);
    }

    static unsigned long long product(unsigned long long a, unsigned long long b)
    {
        // a、b 是 64 位 nimber；自动初始化后按 32 位子域的二次扩域合并，返回 nim 积 a*b。
        init();
        int al = a & 0xffffffffULL, ah = a >> 32;
        int bl = b & 0xffffffffULL, bh = b >> 32;
        int c = p32(al, bl);
        return (unsigned long long)(p32(al ^ ah, bl ^ bh) ^ c) << 32 |
               (p32high(ah, bh) ^ c);
    }
};

```

```cpp
unsigned long long nimProduct(unsigned long long a, unsigned long long b)
{
    // a、b 是 64 位 nimber；返回 nim 积 a*b，不要求调用方预先初始化。
    return NimProduct64::product(a, b);
}

int nimProduct32(int a, int b)
{
    // a、b 是 32 位 nimber；返回乘积，不要求调用方预先初始化。
    return NimProduct64::product32(a, b);
}
```

`nimProduct(a,b)` 返回两个 64 位 nimber 的积；输入只占 32 位时用 `nimProduct32(a,b)`，它会自行初始化并直接调用更短的 32 位扩域路径。`p16`、`p32`、`p32high` 是要求已经初始化的底层接口，不应从题目入口直接调用。nim 和仍用 `a^b`，不能把这里的乘法拿去替代模乘。首次调用用 $O(2^{16})$ 时间和约 256 KiB 空间建表，之后每次乘法为 $O(1)$；需要 SG 理论回看 13.1.004 SG、Anti-SG、Multi-SG、Every-SG。

下面检查零元、单位元与低阶 nimber 乘法；完整随机校验放在本地测试文件：

```cpp
signed main()
{
    assert(nimProduct(0, 123456789) == 0);
    assert(nimProduct(1, 123456789) == 123456789);
    assert(nimProduct(2, 2) == 3);
    assert(nimProduct(2, 3) == 1);
    assert(nimProduct(3, 3) == 2);
}
```

13.2 概率与分布

13.2.001 常见概率公式

这几条公式都建立在同一个事件空间上。条件概率 $P(A \mid B)=\dfrac{P(A\cap B)}{P(B)}$ 要求 $P(B)>0$；把样本空间按互斥且完备的 $\{B_i\}$ 划分，得全概率公式 $P(A)=\sum_i P(A \mid B_i)P(B_i)$；两者结合即 Bayes 公式 $P(B_i \mid A)=\dfrac{P(A \mid B_i)P(B_i)}{\sum_j P(A \mid B_j)P(B_j)}$，把先验 $P(B_i)$ 与似然 $P(A \mid B_i)$ 更新为后验。实现里 `bayes` 传入的 `like` 就是各假设下的似然（不必是概率、无需归一），分母 z 为证据的全概率，`assert` 挡住 $z=0$ 的退化。要点是概率为零不等于事件不可能（连续分布上单点概率恒为零），且独立性 $P(A\cap B)=P(A)P(B)$ 是额外假设、不能默认成立。

```cpp
// 知识点 13.2.001：条件概率、Bayes 与全概率公式。
namespace prob_formula
{
long double condProb(long double pab, long double pb)
{
    // pab 是 P(AnB)，pb 是 P(B)>0；返回 P(A|B)。
    assert(0 <= pab && pab <= pb && pb > 0 && pb <= 1); // 调试检查，可删。
```

```
    return pab / pb;
}

vector<long double> bayes(const vector<long double> &prior, const vector<long double> &like)
{
    // prior 是先验概率，like 是各假设下证据似然；返回观察证据后的后验分布。
    assert(prior.size() == like.size()); // 调试检查，可删。
    assert(!prior.empty()); // 调试检查，可删。
    long double z = 0;
    long double sum = 0;
    for (int i = 0; i < (int)prior.size(); i++)
    {
        assert(prior[i] >= 0 && like[i] >= 0); // 调试检查，可删。
        sum += prior[i];
        z += prior[i] * like[i];
    }
    assert(fabsl(sum - 1) <= 1E-12L); // 调试检查，可删。
    assert(z > 0); // 调试检查，可删。
    vector<long double> ans(prior.size());
    for (int i = 0; i < (int)prior.size(); i++)
    {
        ans[i] = prior[i] * like[i] / z;
    }
    return ans;
}
}
```

`prob_formula::condProb(pab,pb)` 要求 $0 \le P(A \cap B) \le P(B) \le 1$ 且 $P(B) > 0$；`bayes(prior,like)` 要求非空先验为非负且和为 1，似然非负，证据总权重为正。返回后验分布，时间与输出空间 $O(n)$。随机变量矩见 13.2.003 随机变量期望与方差。

13.2.002 常见概率分布

每种分布由参数、支持集和归一化概率确定。Bernoulli 与二项分布要求 $0 \le p \le 1$、$n \in \mathbb{Z}_{\ge 0}$；几何分布要求 $0 < p \le 1$；负二项要求 $r \in \mathbb{Z}_{>0}$、$0 < p \le 1$；Poisson 要求 $\lambda \ge 0$，$\lambda = 0$ 时退化为 $X = 0$。超几何要求 $N \in \mathbb{Z}_{>0}$、$0 \le K, n \le N$，且 $\max(0, n - N + K) \le k \le \min(n, K)$；表中方差式用于 $N > 1$，$N = 1$ 时方差为 0。连续均匀分布要求 $a < b$，指数分布要求 $\lambda > 0$，正态分布要求 $\sigma > 0$。

离散分布：

分布	参数	$P(X = k)$
Bernoulli	p	$p^k(1-p)^{1-k},\ k \in \{0,1\}$
二项	n, p	$\binom{n}{k} p^k (1-p)^{n-k}$
几何（首次成功）	p	$(1-p)^{k-1} p,\ k \ge 1$
负二项（第 r 次成功）	r, p	$\binom{k-1}{r-1} p^r (1-p)^{k-r}$
Poisson	λ	$\dfrac{e^{-\lambda}\lambda^k}{k!}$

分布	参数	$P(X=k)$
超几何	N,K,n	$\frac{\binom{K}{k}\binom{N-K}{n-k}}{\binom{N}{n}}$

分布	$E[X]$	$\mathrm{Var}(X)$
Bernoulli	p	$p(1-p)$
二项	np	$np(1-p)$
几何	$\frac{1}{p}$	$\frac{1-p}{p^2}$
负二项	$\frac{r}{p}$	$\frac{r(1-p)}{p^2}$
Poisson	λ	λ
超几何	$\frac{nK}{N}$	$\frac{nK(N-K)(N-n)}{N^2(N-1)}$

连续分布（f 为密度）：

分布	参数	$f(x)$
均匀	a,b	$\frac{1}{b-a},\ x\in[a,b]$
指数	λ	$\lambda e^{-\lambda x},\ x\ge 0$
正态	μ,σ^2	$\frac{1}{\sigma\sqrt{2\pi}}e^{-(x-\mu)^2/(2\sigma^2)}$

分布	$E[X]$	$\mathrm{Var}(X)$
均匀	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$
指数	$\frac{1}{\lambda}$	$\frac{1}{\lambda^2}$
正态	μ	σ^2

二项概率使用对数 Gamma 计算，避免组合数上溢与幂下溢先形成 $\infty\cdot 0$：

```
long double binomialPmf(int n, int k, long double p)
{
    // n、k、p 是试验数、成功数和单次成功率；返回二项分布 P(X=k)。
    assert(n >= 0 && 0 <= p && p <= 1); // 调试检查，可删。
    if (k < 0 || k > n)
    {
        return 0;
    }
    if (p == 0)
    {
        return k == 0;
    }
    if (p == 1)
    {
        return k == n;
```

```
    }
    long double z = lgammal((long double)n + 1) - lgammal((long double)k + 1)
                  - lgammal((long double)(n - k) + 1);
    z += k * logl(p) + (n - k) * log1pl(-p);
    return expl(z);
}

array<long double, 2> geomMoments(long double p)
{
    // p 是每次成功概率；返回几何分布（首次成功试验数）的 {期望, 方差}。
    assert(0 < p && p <= 1); // 调试检查，可删。
    return {1 / p, (1 - p) / (p * p)};
}
```

`binomialPmf(n,k,p)` 对 $k \notin [0,n]$ 返回 0，$p \in \{0,1\}$ 时直接返回退化分布的概率，其余情形用 `lgammal` 求对数后再取指数，时间 $O(1)$；`geomMoments(p)` 要求 $0 < p \le 1$，返回首次成功所需试验次数的期望 $1/p$ 与方差 $(1-p)/p^2$。

13.2.003 随机变量期望与方差

离散随机变量满足 $E[X]=\sum_i p_i x_i$、$\operatorname{Var}(X)=\sum_i p_i (x_i-E[X])^2$。直接计算 $E[X^2]-E[X]^2$ 会让两个平方量级的大数相减，在大均值、低方差数据上丢失全部有效位；代码改用带权 Welford，逐步更新总权重、均值和中心二阶矩。期望恒线性而方差含协方差项，只有协方差为零时方差才能直接相加。

```
// 知识点 13.2.003：随机变量期望、方差与协方差。
namespace exp_var
{
array<long double, 2> moments(const vector<long double> &x, const vector<long double> &p)
{
    // x 是取值，p 是对应概率且总和为 1；返回期望与方差。
    assert(x.size() == p.size()); // 调试检查，可删。
    assert(!x.empty()); // 调试检查，可删。
    long double sum = 0;
    long double mean = 0;
    long double m2 = 0;
    for (int i = 0; i < (int)x.size(); i++)
    {
        assert(p[i] >= 0); // 调试检查，可删。
        if (p[i] == 0)
        {
            continue;
        }
        long double next = sum + p[i];
        long double d = x[i] - mean;
        mean += d * p[i] / next;
        m2 += p[i] * d * (x[i] - mean);
        sum = next;
    }
    assert(abs(sum - 1) <= 1E-12L); // 调试检查，可删。
    return {mean, max((long double)0, m2 / sum)};
```

```
}
}
```

`exp_var::moments(x, p)` 要求 `x` 与 `p` 等长、`p` 之和为 1（代码用 `assert` 容差校验），返回 `{期望, 方差}`，$O(n)$ 时间、$O(1)$ 额外空间。各标准分布的期望方差闭式见 13.2.002 常见概率分布；状态相互依赖、无法直接求和时应建立条件期望方程（全期望公式 $E[X] = E[E[X \mid Y]]$）逐步消元，这类递推常配合 13.2.001 常见概率公式 的全概率展开。

13.2.004 随机算法与 Xorshift

Xorshift 用若干移位异或更新非零状态，速度快、状态小。本实现采用经典 $13, 7, 17$ 参数；该状态转移在非零状态上为满周期，但这不等于密码学随机。`operator()` 用拒绝采样消除直接取模造成的机器字区间偏差；“均匀”仍建立在伪随机输出足够接近均匀的模型上，不能由可逆性单独推出。对随机质量敏感时使用 `mt19937_64` 或更强发生器。

```
struct RNG
{
    // s 是 xorshift 伪随机数生成器的非零状态。
    unsigned long long s;

    RNG(unsigned long long seed = chrono::steady_clock::now().time_since_epoch().count()) :
    ↪ s(seed ? seed : 1)
    {
        // seed 是随机种子；0 会替换为 1。
    }

    unsigned long long next()
    {
        // 无参数；推进随机数状态并返回下一个随机数。
        s ^= s << 13;
        s ^= s >> 7;
        s ^= s << 17;
        return s;
    }

    unsigned long long operator()(unsigned long long n)
    {
        // n 是正上界；返回 [0,n) 内的伪随机整数。
        assert(n > 0); // 调试检查，可删。
        unsigned long long lim = -n % n;
        unsigned long long x;
        do
        {
            x = next();
        } while (x < lim);
        return x % n;
    }
};
```

`RNG rng(seed)` 用给定种子构造（默认取时钟计数），`rng.next()` 返回下一个 64 位随机数、`rng(n)` 返

回 $[0,n)$ 内均匀整数（要求 $n>0$），二者均 $O(1)$。要固定复现（如本章各自测的对拍）时传入确定种子。抗 hack 的哈希盐、大素数基数取值见 13.3.006 常用常数与 NTT / 哈希素数表；标准库等价物是 `mt19937_64`，随机质量更高但状态更大、稍慢。

13.3 常用公式表

13.3.001 三角恒等式、导数与不定积分表

手算推导索引。角度默认弧度制，积分默认略去 $+C$，换元后需重新核对定义域。

三角恒等式：

$$\sin^2 x+\cos^2 x=1,\quad 1+\tan^2 x=\sec^2 x,\quad 1+\cot^2 x=\csc^2 x$$

$$\begin{aligned}\sin(x\pm y)&=\sin x\cos y\pm\cos x\sin y,\\ \cos(x\pm y)&=\cos x\cos y\mp\sin x\sin y,\\ \tan(x\pm y)&=\frac{\tan x\pm\tan y}{1\mp\tan x\tan y}.\end{aligned}$$

$$\begin{aligned}\sin 2x&=2\sin x\cos x,\\ \cos 2x&=\cos^2 x-\sin^2 x=2\cos^2 x-1=1-2\sin^2 x,\\ \tan 2x&=\frac{2\tan x}{1-\tan^2 x}.\end{aligned}$$

$$\sin^2\tfrac{x}{2}=\frac{1-\cos x}{2},\quad \cos^2\tfrac{x}{2}=\frac{1+\cos x}{2},\quad \tan\tfrac{x}{2}=\frac{1-\cos x}{\sin x}=\frac{\sin x}{1+\cos x}$$

积化和差：$\sin x\cos y=\frac12[\sin(x+y)+\sin(x-y)]$，$\cos x\cos y=\frac12[\cos(x+y)+\cos(x-y)]$，$\sin x\sin y=\frac12[\cos(x-y)-\cos(x+y)]$。

和差化积：$\sin x+\sin y=2\sin\frac{x+y}{2}\cos\frac{x-y}{2}$，$\cos x+\cos y=2\cos\frac{x+y}{2}\cos\frac{x-y}{2}$，$\cos x-\cos y=-2\sin\frac{x+y}{2}\sin\frac{x-y}{2}$。

万能代换 $t=\tan\frac{x}{2}$：$\sin x=\dfrac{2t}{1+t^2}$，$\cos x=\dfrac{1-t^2}{1+t^2}$，$\tan x=\dfrac{2t}{1-t^2}$。

导数表（法则：$(uv)'=u'v+uv'$，$\left(\frac{u}{v}\right)'=\dfrac{u'v-uv'}{v^2}$，$[f(g(x))]'=f'(g)\,g'$）。基本初等函数与三角：

f	f'	f	f'	f	f'
x^a	ax^{a-1}	a^x	$a^x\ln a$	$\log_a x$	$\dfrac{1}{x\ln a}$
e^x	e^x	$\ln x$	$\dfrac{1}{x}$		
$\sin x$	$\cos x$	$\cos x$	$-\sin x$	$\tan x$	$\sec^2 x$

f	f'	f	f'	f	f'
$\cot x$	$-\csc^2 x$	$\sec x$	$\sec x \tan x$	$\csc x$	$-\csc x \cot x$

反三角、双曲与反双曲（源资料中 $(\cot x)'$、$(\operatorname{arccsc} x)'$ 等有正负号 / 定义域笔误，此处取数学上正确的写法）：

f	f'	f	f'
$\arcsin x$	$\dfrac{1}{\sqrt{1-x^2}}$	$\arccos x$	$-\dfrac{1}{\sqrt{1-x^2}}$
$\arctan x$	$\dfrac{1}{1+x^2}$	$\operatorname{arccot} x$	$-\dfrac{1}{1+x^2}$
$\operatorname{arcsec} x$	$\dfrac{1}{\lvert x\rvert\sqrt{x^2-1}}$	$\operatorname{arccsc} x$	$-\dfrac{1}{\lvert x\rvert\sqrt{x^2-1}}$
$\sinh x$	$\cosh x$	$\cosh x$	$\sinh x$
$\tanh x$	$\operatorname{sech}^2 x$	$\coth x$	$-\operatorname{csch}^2 x$
$\operatorname{sech} x$	$-\operatorname{sech} x \tanh x$	$\operatorname{csch} x$	$-\operatorname{csch} x \coth x$
$\operatorname{arcsinh} x$	$\dfrac{1}{\sqrt{x^2+1}}$	$\operatorname{arccosh} x$	$\dfrac{1}{\sqrt{x^2-1}}$
$\operatorname{arctanh} x$	$\dfrac{1}{1-x^2}$	$\operatorname{arccoth} x$	$\dfrac{1}{1-x^2}$
$\operatorname{arcsech} x$	$-\dfrac{1}{x\sqrt{1-x^2}}$	$\operatorname{arccsch} x$	$-\dfrac{1}{\lvert x\rvert\sqrt{1+x^2}}$

不定积分表（分部积分 $\int u\,dv = uv - \int v\,du$，以下一律省略 $+C$）。基本表：

被积函数	原函数	被积函数	原函数
$x^a\ (a \neq -1)$	$\dfrac{x^{a+1}}{a+1}$	$\dfrac{1}{x}$	$\ln\lvert x\rvert$
e^x	e^x	a^x	$\dfrac{a^x}{\ln a}$
$\sin x$	$-\cos x$	$\cos x$	$\sin x$
$\sec^2 x$	$\tan x$	$\csc^2 x$	$-\cot x$
$\dfrac{1}{a^2+x^2}$	$\dfrac{1}{a}\arctan\dfrac{x}{a}$	$\dfrac{1}{\sqrt{1-x^2}}$	$\arcsin x$
$\dfrac{1}{\sqrt{x^2 \pm a^2}}$	$\ln\left\lvert x+\sqrt{x^2 \pm a^2}\right\rvert$	$\ln x$	$x\ln x - x$

二次三项式 $ax^2+bx+c\ (a>0)$：

$$\int \frac{\mathrm{d}x}{ax^2+bx+c} = \begin{cases} \dfrac{2}{\sqrt{4ac-b^2}}\arctan\dfrac{2ax+b}{\sqrt{4ac-b^2}}, & b^2<4ac, \\ -\dfrac{2}{2ax+b}, & b^2=4ac, \\ \dfrac{1}{\sqrt{b^2-4ac}}\ln\left\lvert\dfrac{2ax+b-\sqrt{b^2-4ac}}{2ax+b+\sqrt{b^2-4ac}}\right\rvert, & b^2>4ac. \end{cases}$$

$$\int \frac{x\,\mathrm{d}x}{ax^2+bx+c} = \frac{1}{2a}\ln|ax^2+bx+c| - \frac{b}{2a}\int \frac{\mathrm{d}x}{ax^2+bx+c}$$

含根式 $\sqrt{ax^2+bx+c}\,(a>0)$ 与 $\sqrt{c+bx-ax^2}$：

$$\int \frac{\mathrm{d}x}{\sqrt{ax^2+bx+c}} = \frac{1}{\sqrt{a}}\ln\left|2ax+b+2\sqrt{a}\sqrt{ax^2+bx+c}\right|,$$
$$\int \frac{\mathrm{d}x}{\sqrt{c+bx-ax^2}} = \frac{1}{\sqrt{a}}\arcsin\frac{2ax-b}{\sqrt{b^2+4ac}}.$$

$$\int \frac{\mathrm{d}x}{\sqrt{(x-a)(b-x)}} = 2\arcsin\sqrt{\frac{x-a}{b-a}},$$
$$\int \sqrt{(x-a)(b-x)}\,\mathrm{d}x = \frac{2x-a-b}{4}\sqrt{(x-a)(b-x)} + \frac{(b-a)^2}{4}\arcsin\sqrt{\frac{x-a}{b-a}}.$$

上式要求 $a<b$ 且 $a<x<b$；端点处按单侧连续延拓。

三角函数：

$$\int \tan x\,\mathrm{d}x = -\ln|\cos x|,$$
$$\int \cot x\,\mathrm{d}x = \ln|\sin x|,$$
$$\int \sec x\,\mathrm{d}x = \ln|\sec x+\tan x|,$$
$$\int \csc x\,\mathrm{d}x = \ln\left|\tan\frac{x}{2}\right|.$$

$$\int \sec x\tan x\,\mathrm{d}x = \sec x,$$
$$\int \csc x\cot x\,\mathrm{d}x = -\csc x,$$
$$\int \sin^2 x\,\mathrm{d}x = \frac{x}{2}-\frac{1}{4}\sin 2x,$$
$$\int \cos^2 x\,\mathrm{d}x = \frac{x}{2}+\frac{1}{4}\sin 2x.$$

降次递推（前两式要求 $n\neq 0$；$n=0$ 时被积函数为 1，原函数为 x）：

$$\int \sin^n x\,\mathrm{d}x = -\frac{1}{n}\sin^{n-1}x\cos x + \frac{n-1}{n}\int \sin^{n-2}x\,\mathrm{d}x,$$
$$\int \cos^n x\,\mathrm{d}x = \frac{1}{n}\cos^{n-1}x\sin x + \frac{n-1}{n}\int \cos^{n-2}x\,\mathrm{d}x.$$

$$\int \cos^m x \sin^n x \,\mathrm{d}x = \frac{1}{m+n}\cos^{m-1} x \sin^{n+1} x + \frac{m-1}{m+n}\int \cos^{m-2} x \sin^n x \,\mathrm{d}x, \qquad m+n \neq 0.$$

有理三角式与 x 的乘积:

$$\int \frac{\mathrm{d}x}{a+b\sin x} = \begin{cases} \dfrac{2}{\sqrt{a^2-b^2}} \arctan \dfrac{a\tan\frac{x}{2}+b}{\sqrt{a^2-b^2}}, & a^2 > b^2, \\ -\dfrac{2}{a\tan\frac{x}{2}+b}, & a^2 = b^2,\ (a,b) \neq (0,0), \\ \dfrac{1}{\sqrt{b^2-a^2}} \ln \left| \dfrac{a\tan\frac{x}{2}+b-\sqrt{b^2-a^2}}{a\tan\frac{x}{2}+b+\sqrt{b^2-a^2}} \right|, & a^2 < b^2. \end{cases}$$

当 $ab \neq 0$ 时,

$$\int \frac{\mathrm{d}x}{a^2\cos^2 x + b^2 \sin^2 x} = \frac{1}{ab}\arctan\left(\frac{b}{a}\tan x\right).$$

当 $a \neq 0$ 时,

$$\int x \sin ax \,\mathrm{d}x = \frac{1}{a^2}\sin ax - \frac{1}{a}x\cos ax,$$
$$\int x \cos ax \,\mathrm{d}x = \frac{1}{a^2}\cos ax + \frac{1}{a}x\sin ax.$$

当 $a = 0$ 时，两式的原函数分别为 0 与 $\frac{x^2}{2}$。

反三角函数（$a > 0$）:

$$\int \arcsin \frac{x}{a}\,\mathrm{d}x = x\arcsin\frac{x}{a} + \sqrt{a^2-x^2},$$
$$\int \arccos \frac{x}{a}\,\mathrm{d}x = x\arccos\frac{x}{a} - \sqrt{a^2-x^2},$$
$$\int \arctan \frac{x}{a}\,\mathrm{d}x = x\arctan\frac{x}{a} - \frac{a}{2}\ln(a^2+x^2).$$

指数函数:

$$\int xe^{ax}\,\mathrm{d}x = \frac{1}{a^2}(ax-1)e^{ax},$$
$$\int x^n e^{ax}\,\mathrm{d}x = \frac{1}{a}x^n e^{ax} - \frac{n}{a}\int x^{n-1}e^{ax}\,\mathrm{d}x, \qquad a \neq 0.$$

当 $a = 0$ 时，第一式原函数为 $\frac{x^2}{2}$，第二式退化为普通幂函数积分。

$$\int e^{ax}\sin bx\,\mathrm{d}x=\frac{e^{ax}(a\sin bx-b\cos bx)}{a^2+b^2},$$
$$\int e^{ax}\cos bx\,\mathrm{d}x=\frac{e^{ax}(b\sin bx+a\cos bx)}{a^2+b^2},\qquad a^2+b^2>0.$$

当 $a=b=0$ 时，两式的原函数分别为 0 与 x。

对数函数：

$$\int \ln x\,\mathrm{d}x=x\ln x-x,$$
$$\int \frac{\mathrm{d}x}{x\ln x}=\ln|\ln x|,$$
$$\int x^n\ln x\,\mathrm{d}x=\frac{x^{n+1}}{n+1}\left(\ln x-\frac{1}{n+1}\right),\qquad n\neq -1.$$

当 $n=-1$ 时，$\int x^{-1}\ln x\,\mathrm{d}x=\frac{1}{2}(\ln x)^2$；另有 $\int(\ln x)^n\,\mathrm{d}x=x(\ln x)^n-n\int(\ln x)^{n-1}\,\mathrm{d}x$。

C++ 标准库 `<cmath>` 直接提供的相关特殊函数（`std::` 前缀）：$\int_0^1 t^{x-1}(1-t)^{y-1}\mathrm{d}t=B(x,y)=$ `beta(x,y)`；$\Gamma(x)=$ `tgamma(x)`；指数积分 $\mathrm{Ei}(x)=$ `expint(x)`，于是 $\int_x^\infty \frac{e^{-t}}{t}\mathrm{d}t=-\mathrm{Ei}(-x)=-$ `expint(-x)`；$\zeta(x)=$ `riemann_zeta(x)`；$\frac{2}{\sqrt{\pi}}\int_0^x e^{-t^2}\mathrm{d}t=$ `erf(x)`；第一 / 第二类椭圆积分 `ellint_1(k,phi)`、`ellint_2(k,phi)`（椭圆周长 $=4a\,$`ellint_2(e,pi/2)`，离心率 $e=\sqrt{1-b^2/a^2}$）。

多项式的逐项求导/积分可直接对系数操作：

```cpp
vector<long double> derivative(const vector<long double> &a)
{
    // a[i] 是 x^i 系数；返回导函数系数，常数项导数为 {0}。
    if (a.size() <= 1)
    {
        return {0};
    }
    vector<long double> b(a.size() - 1);
    for (int i = 1; i < (int)a.size(); i++)
    {
        b[i - 1] = i * a[i];
    }
    return b;
}

vector<long double> integral(const vector<long double> &a, long double c = 0)
{
    // a[i] 是 x^i 系数，c 是积分常数；返回一个原函数的系数。
    vector<long double> b(a.size() + 1);
    b[0] = c;
    for (int i = 0; i < (int)a.size(); i++)
    {
        b[i + 1] = a[i] / (i + 1);
    }
```

```
    return b;
}
```

`derivative(a)` 与 `integral(a,c)` 都按低次在前的系数表操作，分别返回导函数与常数项为 c 的原函数，时间与空间均为 $O(n)$；常数多项式的导数约定为 $\{0\}$。

13.3.002 调和数与幂和

用于估算复杂度、化简整式求和。$\gamma \approx 0.5772156649$ 为 Euler-Mascheroni 常数。

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2},$$
$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6},$$
$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2.$$

$$\sum_{i=1}^{n} i^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}, \quad \sum_{i=1}^{n} i^5 = \frac{n^2(n+1)^2(2n^2+2n-1)}{12}$$

一般幂和是关于 n 的 $k+1$ 次多项式，可用 Bernoulli 数（8.2.007）闭式或对 $k+2$ 个点拉格朗日求值：

$$\sum_{i=0}^{n-1} i^k = \frac{1}{k+1} \sum_{j=0}^{k} \binom{k+1}{j} B_j \, n^{k+1-j}$$

调和数与整除分块的渐近式（右侧仅用于估计，不作精确值）：

$$H_n = \sum_{i=1}^{n} \frac{1}{i} = \ln n + \gamma + \frac{1}{2n} - \frac{1}{12n^2} + O(n^{-4}),$$
$$\sum_{i=1}^{n} \left\lfloor \frac{n}{i} \right\rfloor = n \ln n + (2\gamma - 1)n + O(\sqrt{n}).$$

两条精确调和恒等式为

$$\sum_{k=1}^{n} H_k = (n+1)H_n - n, \qquad \sum_{k=1}^{n} kH_k = \frac{n(n+1)}{2} H_n - \frac{n(n-1)}{4}.$$

```
array<__int128, 3> powerSums(int n)
{
    // n 是非负整数；返回 1..n 的一次、二次、三次幂和。
    assert(0 <= n && n <= 5107605666LL); // 调试检查，可删。
    __int128 x = n;
    __int128 s1 = x * (x + 1) / 2;
```

```
    __int128 a = x;
    __int128 b = x + 1;
    __int128 c = 2 * x + 1;
    if (a % 2 == 0)
    {
        a /= 2;
    }
    else
    {
        b /= 2;
    }
    if (a % 3 == 0)
    {
        a /= 3;
    }
    else if (b % 3 == 0)
    {
        b /= 3;
    }
    else
    {
        c /= 3;
    }
    return {s1, a * b * c, s1 * s1};
}

long double harmonic(int n)
{
    // n 是非负整数；返回直接累加得到的第 n 个调和数。
    assert(n >= 0); // 调试检查，可删。
    long double ans = 0;
    for (int i = 1; i <= n;)
    {
        ans += 1.0L / i;
        if (i == n)
        {
            break;
        }
        i++;
    }
    return ans;
}
```

`powerSums(n)` 用 `__int128` 返回前三个精确幂和，要求 $0 \le n \le 5107605666$；该上界恰保证最大的三次幂和不超过 $2^{127}-1$，二次幂和先约分再乘，避免中间式先溢出。时间、空间均为 $O(1)$。`harmonic(n)` 直接累加，时间 $O(n)$、额外空间 $O(1)$；大 n 只需近似时用上面的渐近式。

13.3.003 三角形、四边形与圆公式

设三角形三边 a,b,c 分别对应角 A,B,C，半周长为 $s=(a+b+c)/2$，面积为 S，内外接圆半径为 r,R。以下公式按 HIT 几何模板 PDF 物理第 5 页逐式核对：

$$\frac{a}{\sin A}=\frac{b}{\sin B}=\frac{c}{\sin C}=2R,\qquad c^2=a^2+b^2-2ab\cos C$$

$$S=\frac{ab\sin C}{2}=\sqrt{s(s-a)(s-b)(s-c)}=rs=\frac{abc}{4R}$$

量	公式
a 边中线 m_a	$\frac{1}{2}\sqrt{2b^2+2c^2-a^2}=\frac{1}{2}\sqrt{b^2+c^2+2bc\cos A}$
A 角平分线 t_a	$\frac{\sqrt{bc((b+c)^2-a^2)}}{b+c}=\frac{2bc\cos(A/2)}{b+c}$
a 边高 h_a	$b\sin C=c\sin B=\sqrt{b^2-\left(\frac{a^2+b^2-c^2}{2a}\right)^2}$
内切圆半径 r	$\frac{S}{s}=4R\sin\frac{A}{2}\sin\frac{B}{2}\sin\frac{C}{2}=\sqrt{\frac{(s-a)(s-b)(s-c)}{s}}=s\tan\frac{A}{2}\tan\frac{B}{2}\tan\frac{C}{2}$
外接圆半径 R	$\frac{abc}{4S}=\frac{a}{2\sin A}=\frac{b}{2\sin B}=\frac{c}{2\sin C}$

四边形边长依次为 a,b,c,d，对角线为 D_1,D_2，对角线夹角为 α，两条对角线中点距离为 M，则

$$a^2+b^2+c^2+d^2=D_1^2+D_2^2+4M^2,\qquad S=\frac{1}{2}D_1D_2\sin\alpha.$$

若四边形内接于圆，另有 Ptolemy 定理与 Brahmagupta 公式

$$ac+bd=D_1D_2,\qquad S=\sqrt{(s-a)(s-b)(s-c)(s-d)},$$

其中 $s=\frac{1}{2}(a+b+c+d)$。

正 n 边形的中心角 $\alpha=\frac{2\pi}{n}$、内角 $\beta=\frac{(n-2)\pi}{n}$。边长与面积满足

$$a=2\sqrt{R^2-r^2}=2R\sin\frac{\alpha}{2}=2r\tan\frac{\alpha}{2},$$

$$S=\frac{nar}{2}=nr^2\tan\frac{\alpha}{2}=\frac{nR^2\sin\alpha}{2}=\frac{na^2}{4\tan(\alpha/2)}.$$

圆半径为 r，圆心角为 θ，弧长为 l，弦长为 a，弓高为 h，则

量	公式
弧长	$l=r\theta$

量	公式
弦长	$a=2\sqrt{2rh-h^2}=2r\sin\frac{\theta}{2}$
弓高	$h=r-\sqrt{r^2-\frac{a^2}{4}}=r\left(1-\cos\frac{\theta}{2}\right)=\frac{a}{2}\tan\frac{\theta}{4}$
扇形面积	$S_1=\frac{rl}{2}=\frac{r^2\theta}{2}$
弓形面积	$S_2=\frac{rl-a(r-h)}{2}=\frac{r^2(\theta-\sin\theta)}{2}$

退化（三点共线）时 $S=0$，代入前先验三角不等式，`acos` 参数需 `clamp` 到 $[-1,1]$ 防浮点越界：

```cpp
array<long double, 3> triangle(long double a, long double b, long double c)
{
    // a、b、c 是合法三角形边长；返回 {面积，c 对应角（弧度），外接圆半径}。
    assert(a > 0 && b > 0 && c > 0 && a + b > c && a + c > b && b + c > a); // 调试检查，可删。
    long double s = (a + b + c) / 2;
    long double area = sqrt(s * (s - a) * (s - b) * (s - c));
    long double angle = acos(clamp((a * a + b * b - c * c) / (2 * a * b), -1.0L, 1.0L));
    return {area, angle, a * b * c / (4 * area)};
}
```

`triangle(a,b,c)` 返回面积、角 C 与外接圆半径；表中所有角均用弧度。代码为 $O(1)$，接近退化时应使用与输入尺度匹配的浮点误差策略。

13.3.004 常用面积、体积公式

平面面积还常用：平行四边形 ah，梯形 $\frac{1}{2}(a+b)h$，圆 πr^2，椭圆 πab。三角形、正多边形、扇形与弓形的完整公式见 13.3.003 三角形、四边形与圆公式。下表按 HIT 几何模板 PDF 物理第 6 至 7 页核对；A, A_1, A_2 是底面积，p, p_1, p_2 是底面周长，h 是垂高，l 是斜高或母线。

立体	侧面积 L	全表面积 T	体积 V
直棱柱	ph	$ph+2A$	Ah
正棱锥	$\frac{1}{2}pl$	$\frac{1}{2}pl+A$	$\frac{1}{3}Ah$
正棱台	$\frac{1}{2}(p_1+p_2)l$	$\frac{1}{2}(p_1+p_2)l+A_1+A_2$	$\frac{h}{3}(A_1+A_2+\sqrt{A_1A_2})$
圆柱（半径 r）	$2\pi rh$	$2\pi r(h+r)$	πr^2h
圆锥（底半径 r）	πrl	$\pi r(l+r)$	$\frac{1}{3}\pi r^2h$
圆台（底半径 r_1, r_2）	$\pi(r_1+r_2)l$	$\pi(r_1+r_2)l+\pi(r_1^2+r_2^2)$	$\frac{\pi h}{3}(r_1^2+r_1r_2+r_2^2)$
球（半径 R）	—	$4\pi R^2$	$\frac{4}{3}\pi R^3$
球台（球半径 R，底圆半径 r_1, r_2）	$2\pi Rh$	$\pi(2Rh+r_1^2+r_2^2)$	$\frac{\pi h}{6}(3r_1^2+3r_2^2+h^2)$
球扇形（球半径 R，球冠高 h、底圆半径 r_0）	—	$\pi R(2h+r_0)$	$\frac{2}{3}\pi R^2h$

圆锥母线 $l=\sqrt{h^2+r^2}$，圆台母线 $l=\sqrt{h^2+(r_1-r_2)^2}$。斜高、垂高、侧面积与全表面积不可混用；所有长度必须先统一单位。常用圆体的关键量：

```cpp
array<long double, 4> roundBodies(long double r, long double h, long double r2)
{
    // r 圆/球半径，h 锥台高，r2 锥台另一半径；返回 {圆面积，球面积，球体积，圆台体积}。
    assert(r >= 0 && h >= 0 && r2 >= 0);
    long double pi = acos(-1.0L);
    return {pi * r * r, 4 * pi * r * r, 4 * pi * r * r * r / 3, pi * h * (r * r + r * r2 + r2 *
    ↪  r2) / 3};
}
```

`roundBodies(r,h,r2)` 只返回圆面积、球面积、球体积和圆台体积，用于快速计算；其余量直接按表代入。函数为 $O(1)$，输入须为非负有限实数。

13.3.005 日期公式：Zeller / 基姆拉尔森

由公历年月日直接算星期时，可把 1、2 月视作上一年的 13、14 月；更通用的做法是先把日期映射为相对 1970-01-01 的纪元日数，再由日数模 7 求星期。完整、唯一的 `Date` 实现放在 14.1.005 日期与时间：它提供闰年、月天数、日期与纪元日互转及星期接口。本条只保留公式入口，避免两份日期边界和注释漂移。

13.3.006 常用常数与 NTT / 哈希素数表

常数：$\pi = \arccos(-1)$，$e = \exp(1)$，黄金比 $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.6180339887$，Euler-Mascheroni $\gamma \approx 0.5772156649$，$\ln 2 \approx 0.6931471806$。

NTT 友好素数 $p = c \cdot 2^k + 1$，原根 g，可做长度至多 2^k 的变换：

p	分解	g	最大 2^k
998244353	$119 \cdot 2^{23} + 1$	3	2^{23}
1004535809	$479 \cdot 2^{21} + 1$	3	2^{21}
469762049	$7 \cdot 2^{26} + 1$	3	2^{26}
167772161	$5 \cdot 2^{25} + 1$	3	2^{25}
1224736769	$73 \cdot 2^{24} + 1$	3	2^{24}
2013265921	$15 \cdot 2^{27} + 1$	31	2^{27}
2281701377	$17 \cdot 2^{27} + 1$	3	2^{27}

需要更大变换长度（如大位宽 NTT、超长卷积或挑三模数）时的完整表，$p = r \cdot 2^k + 1$，g 为最小原根：

p	r	k	g
17	1	4	3
97	3	5	5
193	3	6	5
257	1	8	3
7681	15	9	17
12289	3	12	11
40961	5	13	3

p	r	k	g
65537	1	16	3
786433	3	18	10
5767169	11	19	3
7340033	7	20	3
23068673	11	21	3
104857601	25	22	3
167772161	5	25	3
469762049	7	26	3
998244353	119	23	3
1004535809	479	21	3
2013265921	15	27	31
2281701377	17	27	3

较大的模数单独列出，避免打印时与前四列挤在一起：

p	r	k	g
3221225473	3	30	5
75161927681	35	31	3
77309411329	9	33	7
206158430209	3	36	22
2061584302081	15	37	7
2748779069441	5	39	3
6597069766657	3	41	5
39582418599937	9	42	5
79164837199873	9	43	5
263882790666241	15	44	7
1231453023109121	35	45	3
1337006139375617	19	46	3
3799912185593857	27	47	5
4222124650659841	15	48	19
7881299347898369	7	50	6
31525197391593473	7	52	3
180143985094819841	5	55	6
1945555039024054273	27	56	5
4179340454199820289	29	57	3

任意模数卷积常取 $\{998244353, 1004535809, 469762049\}$ 三模数 CRT 合并。设三模乘积为 P；无符号恢复要求真实卷积系数落在 $[0, P)$，有符号对称恢复要求绝对值严格小于 $P/2$。若输入系数绝对值分别不超过 A, B，还须核对 $\min(n, m)AB < P/2$，不能只凭“十亿级系数”判断安全。

字符串/表哈希大素数：单哈希用 10^9+7、10^9+9、998244353；需抗 hack 的双哈希取 Mersenne 素数 $2^{31}-1 = 2147483647$ 与 $2^{61}-1 = 2305843009213693951$（后者乘法用 `__int128` 或快速取

模)。基数取大于字符集的随机奇数(如 131、13331 或运行时随机)。

```cpp
constexpr long double pi = 3.14159265358979323846264338327950288 4L;
constexpr long double e = 2.71828182845904523536028747135266249 8L;
constexpr long double phi = 1.61803398874989484820458683436563811 8L;
// ntt[i] 三维依次是质数、原根、可用的最大二次幂指数。
constexpr array<array<int, 3>, 7> ntt = {{{998244353, 3, 23},
                                          {1004535809, 3, 21},
                                          {469762049, 3, 26},
                                          {167772161, 3, 25},
                                          {1224736769, 3, 24},
                                          {2013265921, 31, 27},
                                          {2281701377, 3, 27}}};
```

常用数量级只保留明确标注的上下界附近素数;需要随机素数时仍应运行素性测试,不能把速查表当成随机源。

k	不小于 10^k 的常用素数	小于 10^{k+1} 的靠后素数
2	101	997
3	1009	9973
4	10007	99991
5	100003	999983
6	1000003	9999991
7	10000019	99999989
8	100000007	999999937
9	1000000007	9999999967

13.3.007 数论数量级与极值表

赛场估复杂度、判溢出用。除注明外均指 64 位有符号范围内。计算某具体上界的最大约数个数或反素数本身用 7.2.007 因数最多的数。

Stirling 估计 $\ln(n!) = n\ln n - n + \frac{1}{2}\ln(2\pi n) + O(1/n)$;阶乘增长极快,64 位内最大阶乘是 $20! = 2432902008176640000$($21!$ 溢出),$13!$ 已超 32 位。

上界 N	$\pi(N)$ 素数个数	$\max_{n\le N} d(n)$
10^3	168	32
10^6	78498	240
10^9	50847534	1344
10^{12}	37607912018	6720
10^{18}	$\approx 2.47\times 10^{16}$	103680

约数个数最大值的部分达到者为

$$840,\quad 720720,\quad 735134400,$$

分别对应前三行；10^{18} 对应的达到者是

$$897612484786617600.$$

上界 N	$\max_{n\le N}\omega(n)$	最大素数间隙
10^3	4	20
10^4	5	36
10^5	6	72
10^6	7	114
10^7	8	154
10^8	8	220
10^9	9	282
10^{10}	10	354
10^{11}	10	464
10^{12}	11	540
10^{13}	12	674
10^{14}	12	804
10^{15}	13	906
10^{16}	13	1132
10^{17}	14	1220
10^{18}	15	1442

ω 上界即前若干最小素数之积不超过 N：$2\cdot 3\cdots 23 = 223092870 < 10^9$（9 个），前 15 个素数之积 $\approx 6.15\times 10^{17} < 10^{18}$。表中素数间隙指相邻素数 $p<q\le N$ 的最大 $q-p$，因此只有两个端点都不超过 N 的间隙才计入。约数和 $\sigma(n)$、欧拉函数极小值 $\varphi(n)$ 也在这些高合成数附近取极端。

逐数量级细表：10^k 前 / 后第一个素数（以相对 10^k 的偏移给出，如 10^9 前第一个素数是 10^9-63），以及 $\le 10^k$ 的 $\max\omega(n)$、$\max d(n)$。挑「贴着 10^k 的素数模数」或估极值时直接查：

k	10^k 前第一质数	10^k 后第一质数	$\max\omega$	$\max d$	k	10^k 前第一质数	10^k 后第一质数	$\max\omega$	$\max d$
1	-3	$+1$	2	4	11	-23	$+3$	10	4032
2	-3	$+1$	3	12	12	-11	$+39$	11	6720
3	-3	$+13$	4	32	13	-29	$+37$	12	10752
4	-27	$+7$	5	64	14	-27	$+31$	12	17280
5	-9	$+3$	6	128	15	-11	$+37$	13	26880
6	-17	$+3$	7	240	16	-63	$+61$	13	41472
7	-9	$+19$	8	448	17	-3	$+3$	14	64512
8	-11	$+7$	8	768	18	-11	$+3$	15	103680
9	-63	$+7$	9	1344	19	-39	$+51$	16	161280
10	-33	$+19$	10	2304					

13.3.008 裂项相消、有限差分与几何级数

裂项相消先把通项写成相邻两项之差。对整数 $a \le b$，最基本的望远镜求和为

$$\sum_{k=a}^{b} \bigl(f(k+1)-f(k)\bigr) = f(b+1)-f(a).$$

下列分式要求求和范围内的分母非零；给出的有限和均取整数 $n \ge 1$：

$$\begin{aligned}
\frac{1}{k(k+1)} &= \frac{1}{k}-\frac{1}{k+1},\\
\sum_{k=1}^{n} \frac{1}{k(k+1)} &= 1-\frac{1}{n+1},\\
\frac{1}{k(k+1)(k+2)} &= \frac{1}{2}\left(\frac{1}{k(k+1)}-\frac{1}{(k+1)(k+2)}\right),\\
\sum_{k=1}^{n} \frac{1}{k(k+1)(k+2)} &= \frac{1}{4}-\frac{1}{2(n+1)(n+2)}.
\end{aligned}$$

阶乘分母也常直接裂成相邻两项：

$$\frac{k}{(k+1)!} = \frac{1}{k!}-\frac{1}{(k+1)!}, \qquad \sum_{k=1}^{n}\frac{k}{(k+1)!} = 1-\frac{1}{(n+1)!}.$$

有限几何级数及其加权式如下。对整数 $n \ge 0$，令 $m = n+1$。当 $q \ne 1$ 时，定义

$$\begin{aligned}
P_2(m,q) &= m^2(1-q)^2 + 2mq(1-q) + q(1+q),\\
P_3(m,q) &= m^3(1-q)^3 + 3m^2q(1-q)^2\\
&\quad + 3mq(1+q)(1-q) + q(1+4q+q^2),\\
P_4(m,q) &= m^4(1-q)^4 + 4m^3q(1-q)^3\\
&\quad + 6m^2q(1+q)(1-q)^2\\
&\quad + 4mq(1+4q+q^2)(1-q)\\
&\quad + q(1+11q+11q^2+q^3).
\end{aligned}$$

则二、三、四阶加权和为

$$\begin{aligned}
\sum_{x=0}^{n} x^2 q^x &= \frac{q(1+q)-q^m P_2(m,q)}{(1-q)^3},\\
\sum_{x=0}^{n} x^3 q^x &= \frac{q(1+4q+q^2)-q^m P_3(m,q)}{(1-q)^4},\\
\sum_{x=0}^{n} x^4 q^x &= \frac{q(1+11q+11q^2+q^3)-q^m P_4(m,q)}{(1-q)^5}.
\end{aligned}$$

零阶与一阶式为

$$\sum_{x=0}^{n} q^x = \frac{1-q^m}{1-q}, \qquad \sum_{x=0}^{n} xq^x = \frac{q - mq^m + (m-1)q^{m+1}}{(1-q)^2}.$$

在模意义下，上述闭式还要求 $1-q$ 可逆。若 $q=1$，改用

$$\sum_{x=0}^{n} x^2 = \frac{n(n+1)(2n+1)}{6},$$
$$\sum_{x=0}^{n} x^3 = \left(\frac{n(n+1)}{2}\right)^2,$$
$$\sum_{x=0}^{n} x^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}.$$

这些式子可从有限几何级数反复求导并乘以 q 得到。有限差分定义为 $\Delta f(k) = f(k+1) - f(k)$；对整数 $m \geq 0$，有

$$\Delta^m f(0) = \sum_{j=0}^{m} (-1)^{m-j} \binom{m}{j} f(j).$$

若 f 是次数小于 m 的多项式，右侧为 0；这可以直接化简交错二项式和，也可用于差分表和 Newton 展开。组合数相关的交错幂和见 8.2.005 Stirling 数，含 $\lfloor n/i \rfloor$ 的和优先按 7.3.009 整除分块 枚举 $O(\sqrt{n})$ 个商值块。

13.3.009 泛型复数与三次单位根代数

`Complex<T>` 表示 $a+bi$，其中 $i^2=-1$，共轭与范数分别为 $\overline{z}=a-bi$、$N(z)=z\overline{z}=a^2+b^2$；因此 $N(z)$ 可逆时有 $z^{-1}=\overline{z}/N(z)$。`Complex3<T>` 表示 $a+b\omega$，其中 $\omega^2+\omega+1=0$，把乘积中的 ω^2 替换为 $-1-\omega$，可得 $(a+b\omega)(c+d\omega)=(ac-bd)+(ad+bc-bd)\omega$；这里的共轭把 ω 换成 ω^2，故 $\overline{a+b\omega}=(a-b)-b\omega$、$N(a+b\omega)=a^2-ab+b^2$。两种类型都只要求底层 `T` 提供实际调用所需的环或域运算，适合复用模整数、有理数和浮点类型。

```
template <class T> struct Complex
{
    T a; // 实部。
    T b; // 虚部的系数。

    Complex(const T &a_ = T(), const T &b_ = T()) : a(a_), b(b_)
    {
        // a_ 是实部，b_ 是虚部的系数；构造 a_+b_*i。
    }

    Complex operator+(const Complex &rhs) const
    {
        // rhs 是加数；返回当前复数与 rhs 的和。
```

```
        return {a + rhs.a, b + rhs.b};
    }

    Complex operator-(const Complex &rhs) const
    {
        // rhs 是减数；返回当前复数减 rhs。
        return {a - rhs.a, b - rhs.b};
    }

    Complex operator*(const Complex &rhs) const
    {
        // rhs 是乘数；返回当前复数与 rhs 的积。
        return {a * rhs.a - b * rhs.b, a * rhs.b + b * rhs.a};
    }

    Complex operator/(const Complex &rhs) const
    {
        // rhs 是除数；返回当前复数除以 rhs，要求 rhs 的范数在 T 中可逆。
        return *this * rhs.inv();
    }

    Complex &operator+=(const Complex &rhs)
    {
        // rhs 是加数；把当前复数增加 rhs 并返回自身引用。
        return *this = *this + rhs;
    }

    Complex &operator-=(const Complex &rhs)
    {
        // rhs 是减数；把当前复数减去 rhs 并返回自身引用。
        return *this = *this - rhs;
    }

    Complex &operator*=(const Complex &rhs)
    {
        // rhs 是乘数；把当前复数乘以 rhs 并返回自身引用。
        return *this = *this * rhs;
    }

    Complex &operator/=(const Complex &rhs)
    {
        // rhs 是除数；把当前复数除以 rhs 并返回自身引用，要求 rhs 的范数在 T 中可逆。
        return *this = *this / rhs;
    }

    bool operator==(const Complex &rhs) const
    {
        // rhs 是待比较复数；返回两个系数是否分别相等。
        return a == rhs.a && b == rhs.b;
    }

    bool operator≠(const Complex &rhs) const
```

```
    {
        // rhs 是待比较复数；返回两个复数是否不等。
        return !(*this == rhs);
    }

    bool isZero() const
    {
        // 返回当前复数是否为加法零元。
        return a == T() && b == T();
    }

    Complex conj() const
    {
        // 返回当前复数的共轭。
        return {a, -b};
    }

    T norm() const
    {
        // 返回当前复数与其共轭之积 a^2+b^2。
        return a * a + b * b;
    }

    Complex inv() const
    {
        // 返回当前复数的乘法逆元，要求范数在 T 中非零且可逆。
        T n = norm();
        assert(n != T());
        return {a / n, -b / n};
    }
};

template <class T> Complex<T> conj(const Complex<T> &z)
{
    // z 是待共轭复数；返回 z 的共轭。
    return z.conj();
}

template <class T> struct Complex3
{
    T a; // 常数项的系数。
    T b; // 三次单位根 omega 的系数。

    Complex3(const T &a_ = T(), const T &b_ = T()) : a(a_), b(b_)
    {
        // a_ 是常数项系数，b_ 是 omega 的系数；构造 a_+b_*omega。
    }

    Complex3 operator+(const Complex3 &rhs) const
    {
        // rhs 是加数；返回当前元素与 rhs 的和。
        return {a + rhs.a, b + rhs.b};
```

```
    }

    Complex3 operator-(const Complex3 &rhs) const
    {
        // rhs 是减数；返回当前元素减 rhs。
        return {a - rhs.a, b - rhs.b};
    }

    Complex3 operator*(const Complex3 &rhs) const
    {
        // rhs 是乘数；返回按 omega^2=-1-omega 化简后的积。
        T bd = b * rhs.b;
        return {a * rhs.a - bd, a * rhs.b + b * rhs.a - bd};
    }

    Complex3 operator/(const Complex3 &rhs) const
    {
        // rhs 是除数；返回当前元素除以 rhs，要求 rhs 的范数在 T 中可逆。
        return *this * rhs.inv();
    }

    Complex3 &operator+=(const Complex3 &rhs)
    {
        // rhs 是加数；把当前元素增加 rhs 并返回自身引用。
        return *this = *this + rhs;
    }

    Complex3 &operator-=(const Complex3 &rhs)
    {
        // rhs 是减数；把当前元素减去 rhs 并返回自身引用。
        return *this = *this - rhs;
    }

    Complex3 &operator*=(const Complex3 &rhs)
    {
        // rhs 是乘数；把当前元素乘以 rhs 并返回自身引用。
        return *this = *this * rhs;
    }

    Complex3 &operator/=(const Complex3 &rhs)
    {
        // rhs 是除数；把当前元素除以 rhs 并返回自身引用，要求 rhs 的范数在 T 中可逆。
        return *this = *this / rhs;
    }

    bool operator==(const Complex3 &rhs) const
    {
        // rhs 是待比较元素；返回两个系数是否分别相等。
        return a == rhs.a && b == rhs.b;
    }

    bool operator≠(const Complex3 &rhs) const
```

```
    {
        // rhs 是待比较元素；返回两个元素是否不等。
        return !(*this == rhs);
    }

    bool isZero() const
    {
        // 返回当前元素是否为加法零元。
        return a == T() && b == T();
    }

    Complex3 conj() const
    {
        // 返回把 omega 替换为 omega^2 后的共轭元素。
        return {a - b, -b};
    }

    T norm() const
    {
        // 返回当前元素与其共轭之积 a^2-a*b+b^2。
        return a * a - a * b + b * b;
    }

    Complex3 inv() const
    {
        // 返回当前元素的乘法逆元，要求范数在 T 中非零且可逆。
        T n = norm();
        assert(n != T());
        return {(a - b) / n, -b / n};
    }
};

template <class T> Complex3<T> conj(const Complex3<T> &z)
{
    // z 是待共轭元素；返回 z 的共轭。
    return z.conj();
}
```

直接以两个系数构造对象；每次运算都是 $O(1)$ 时间和空间，数值范围由 `T` 负责。`inv()`、`operator/` 和 `operator/=` 不仅要求范数非零，还要求该范数在 `T` 中存在可用的乘法逆元；模整数应工作在相应可逆元上，有理数和浮点数可直接使用，而原生整数除法会截断，所以整数实例通常只用于加、减、乘、共轭与范数。复数没有与代数结构相容的自然全序，本实现不定义 `<` 或 `>`；若题意确实按模长排序，应显式比较 `norm()`，只需要普通浮点复数时也可直接使用标准库 `complex<T>`。

14. 其他实用模板

14.1 其他实用模板

14.1.001 高精度整数（BigInt）

带符号任意精度整数，内部按 10^9 一块、低位块在前存放绝对值，符号单独记在 `s` 里（取 $-1,0,1$）。加减法在同号时走 `addAbs`、异号时先比较绝对值再走 `subAbs`；乘法是 $O(nm)$ 的分块卷积，单项乘积加旧块与进位严格小于 $10^{18}+2\times10^9$，用 `unsigned long long` 即可。除法 `divmod` 从最高块到最低块逐位定商——每步把当前余数左移一块，再二分出最大的 $q_i\in[0,B)$ 使 `mulAbs(d,q_i)` 不超过余数，商向零取整、余数与被除数同号。每次运算后都要 `norm` 删掉最高端的零块并把空值的符号归零，这是比较与 `str` 输出正确的前提；忘记归一化会让 0 带上残留符号、进而破坏 `operator<`。

```
struct BigInt
{
    // B 是每个低位块使用的进制 10^9。
    static constexpr int B = 1000000000;
    // a 按低位在前保存绝对值的 10^9 进制块。
    vector<int> a;
    // s 保存符号：-1、0、1 分别表示负数、零、正数。
    int s = 0;

    BigInt(long long x = 0)
    {
        // x 是待转换的 64 位整数；构造同值高精度整数。
        *this = x;
    }
    BigInt(const string &x)
    {
        // x 是带可选单个正负号的十进制整数串；构造同值高精度整数。
        assign(x);
    }

    BigInt &operator=(long long x)
    {
        // x 是待转换的 64 位整数；返回赋值后的当前对象。
        a.clear();
        s = (x > 0) - (x < 0);
        unsigned long long y = x < 0 ? (unsigned long long)(-(x + 1)) + 1 : (unsigned long
        ↪  long)x;
        while (y)
        {
            a.push_back(y % B);
            y /= B;
        }
        return *this;
    }

    BigInt &assign(const string &x)
```

```
    {
        // x 是带可选单个正负号且至少含一位数字的十进制整数串；返回赋值后的当前对象。
        a.clear();
        int p = 0;
        s = 1;
        if (!x.empty() && (x[0] == '-' || x[0] == '+'))
        {
            s = x[0] == '-' ? -1 : 1;
            p = 1;
        }
        assert(p < (int)x.size()); // 调试检查，可删。
        for (int i = p; i < (int)x.size(); i++)
        {
            assert('0' <= x[i] && x[i] <= '9'); // 调试检查，可删。
        }
        for (int r = (int)x.size(); r > p; r -= 9)
        {
            int l = max(p, r - 9);
            int v = 0;
            for (int i = l; i < r; i++)
            {
                v = v * 10 + x[i] - '0';
            }
            a.push_back(v);
        }
        norm();
        return *this;
    }

    string str() const
    {
        // 无参数；返回当前高精度整数的十进制字符串。
        if (!s)
        {
            return "0";
        }
        string x = s < 0 ? "-" : "";
        x += to_string(a.back());
        for (int i = (int)a.size() - 2; i >= 0; i--)
        {
            string t = to_string(a[i]);
            x += string(9 - t.size(), '0') + t;
        }
        return x;
    }

    void norm()
    {
        // 删除最高端零块并同步零值符号，无返回值。
        while (!a.empty() && !a.back())
        {
            a.pop_back();
```

```
        }
        if (a.empty())
        {
            s = 0;
        }
    }

    static int cmpAbs(const BigInt &x, const BigInt &y)
    {
        // x、y 是待比较整数；按 |x| 与 |y| 的大小返回 -1、0 或 1。
        if (x.a.size() != y.a.size())
        {
            return x.a.size() < y.a.size() ? -1 : 1;
        }
        for (int i = (int)(x.a.size()) - 1; i >= 0; i--)
        {
            if (x.a[i] != y.a[i])
            {
                return x.a[i] < y.a[i] ? -1 : 1;
            }
        }
        return 0;
    }

    friend bool operator==(const BigInt &x, const BigInt &y)
    {
        // x、y 是待比较整数；返回二者是否相等。
        return x.s == y.s && x.a == y.a;
    }
    friend bool operator<(const BigInt &x, const BigInt &y)
    {
        // x、y 是待比较整数；返回 x 是否小于 y。
        if (x.s != y.s)
        {
            return x.s < y.s;
        }
        if (!x.s)
        {
            return false;
        }
        int c = cmpAbs(x, y);
        return x.s > 0 ? c < 0 : c > 0;
    }
    friend bool operator!=(const BigInt &x, const BigInt &y)
    {
        // x、y 是待比较整数；返回二者是否不相等。
        return !(x == y);
    }
    friend bool operator>(const BigInt &x, const BigInt &y)
    {
        // x、y 是待比较整数；返回 x 是否大于 y。
        return y < x;
```

```
    }
    friend bool operator<=(const BigInt &x, const BigInt &y)
    {
        // x、y 是待比较整数；返回 x 是否不大于 y。
        return !(y < x);
    }
    friend bool operator>=(const BigInt &x, const BigInt &y)
    {
        // x、y 是待比较整数；返回 x 是否不小于 y。
        return !(x < y);
    }

    BigInt operator-() const
    {
        // 无参数；返回当前整数的相反数。
        BigInt x = *this;
        x.s = -x.s;
        return x;
    }

    static BigInt addAbs(const BigInt &x, const BigInt &y)
    {
        // x、y 是两个整数；返回 |x|+|y|。
        BigInt z;
        z.s = 1;
        int n = max(x.a.size(), y.a.size());
        z.a.resize(n);
        long long c = 0;
        for (int i = 0; i < n; i++)
        {
            c += (i < (int)(x.a.size()) ? x.a[i] : 0);
            c += (i < (int)(y.a.size()) ? y.a[i] : 0);
            z.a[i] = c % B;
            c /= B;
        }
        if (c)
        {
            z.a.push_back(c);
        }
        z.norm();
        return z;
    }

    // 要求 |x| >= |y|。
    static BigInt subAbs(const BigInt &x, const BigInt &y)
    {
        // x、y 满足 |x⊳⊰y|；返回 |x|-|y|。
        BigInt z;
        z.s = 1;
        z.a.resize(x.a.size());
        long long c = 0;
        for (int i = 0; i < (int)(x.a.size()); i++)
```

```
        {
            c = (long long)(x.a[i]) - c - (i < (int)(y.a.size()) ? y.a[i] : 0);
            if (c < 0)
            {
                c += B;
                z.a[i] = c;
                c = 1;
            }
            else
            {
                z.a[i] = c;
                c = 0;
            }
        }
        z.norm();
        return z;
    }

    BigInt &operator+=(const BigInt &y)
    {
        // y 是加数；把 y 加到当前整数并返回当前对象。
        if (!y.s)
        {
            return *this;
        }
        if (!s)
        {
            return *this = y;
        }
        if (s == y.s)
        {
            int t = s;
            *this = addAbs(*this, y);
            s = t;
        }
        else if (cmpAbs(*this, y) >= 0)
        {
            int t = s;
            *this = subAbs(*this, y);
            if (a.size())
            {
                s = t;
            }
        }
        else
        {
            int t = y.s;
            *this = subAbs(y, *this);
            if (a.size())
            {
                s = t;
            }
```

```
        }
        return *this;
    }
    BigInt &operator-=(const BigInt &y)
    {
        // y 是减数；从当前整数减去 y 并返回当前对象。
        return *this += -y;
    }

    BigInt &operator*=(const BigInt &y)
    {
        // y 是乘数；把当前整数乘以 y 并返回当前对象。
        if (!s || !y.s)
        {
            return *this = 0;
        }
        vector<int> c(a.size() + y.a.size());
        for (int i = 0; i < (int)a.size(); i++)
        {
            unsigned long long v = 0;
            for (int j = 0; j < (int)(y.a.size()) || v; j++)
            {
                unsigned long long w = c[i + j] + v;
                if (j < (int)(y.a.size()))
                {
                    w += (unsigned long long)a[i] * y.a[j];
                }
                c[i + j] = w % B;
                v = w / B;
            }
        }
        a = move(c);
        s *= y.s;
        norm();
        return *this;
    }

    static BigInt mulAbs(const BigInt &x, int y)
    {
        // x 是整数，y 满足 0<=y<B；返回 |x|*y。
        BigInt z;
        if (!y || !x.s)
        {
            return z;
        }
        z.s = 1;
        z.a.resize(x.a.size());
        unsigned long long c = 0;
        for (int i = 0; i < (int)x.a.size(); i++)
        {
            unsigned long long w = c + (unsigned long long)x.a[i] * y;
            z.a[i] = w % B;
```

```
            c = w / B;
        }
        if (c)
        {
            z.a.push_back(c);
        }
        return z;
    }

    // 商向 0 取整，余数与被除数同号；除数不能为 0。
    static pair<BigInt, BigInt> divmod(const BigInt &x, const BigInt &y)
    {
        // x 是被除数，y 是非零除数；返回向零取整的商和与 x 同号的余数。
        assert(y.s); // 调试检查，可删。
        BigInt q, r, d = y;
        d.s = 1;
        q.a.resize(x.a.size());
        q.s = x.s * y.s;
        for (int i = (int)(x.a.size()) - 1; i >= 0; i--)
        {
            r.a.insert(r.a.begin(), x.a[i]);
            r.s = 1;
            r.norm();
            int l = 0, h = B - 1;
            while (l < h)
            {
                int m = l + (h - l + 1) / 2;
                if (cmpAbs(mulAbs(d, m), r) <= 0)
                {
                    l = m;
                }
                else
                {
                    h = m - 1;
                }
            }
            q.a[i] = l;
            r = subAbs(r, mulAbs(d, l));
        }
        q.norm();
        if (r.a.size())
        {
            r.s = x.s;
        }
        return {q, r};
    }

    BigInt &operator/=(const BigInt &y)
    {
        // y 是非零除数；把当前整数改成向零取整的商并返回当前对象。
        return *this = divmod(*this, y).first;
    }
```

```
    BigInt &operator%=(const BigInt &y)
    {
        // y 是非零除数；把当前整数改成与原值同号的余数并返回当前对象。
        return *this = divmod(*this, y).second;
    }

    friend BigInt operator+(BigInt x, const BigInt &y)
    {
        // x、y 是加数；返回 x+y。
        return x += y;
    }
    friend BigInt operator-(BigInt x, const BigInt &y)
    {
        // x 是被减数，y 是减数；返回 x-y。
        return x -= y;
    }
    friend BigInt operator*(BigInt x, const BigInt &y)
    {
        // x、y 是乘数；返回 x*y。
        return x *= y;
    }
    friend BigInt operator/(BigInt x, const BigInt &y)
    {
        // x 是被除数，y 是非零除数；返回向零取整的商。
        return x /= y;
    }
    friend BigInt operator%(BigInt x, const BigInt &y)
    {
        // x 是被除数，y 是非零除数；返回与 x 同号的余数。
        return x %= y;
    }
};
```

用 `BigInt x(十进制字符串)` 或 `BigInt(long long)` 构造；字符串必须是可选单个正负号后接至少一位十进制数字，否则抛出 `invalid_argument`。四则与取模直接用重载的 `+ - * / %`，除数必须非零；输出调用 `str()`，本类不重载流运算符。该类适合常规十进制中等位数；百万位十进制题使用下方 `HexBigInt::fromDecimal` / `toDecimal`，把十进制分组转换到 Karatsuba 与 Burnikel–Ziegler 路径。设两数块数为 n, m：比较与加减为 $O(\max(n,m))$，乘法为 $O(nm)$，当前逐块二分除法为 $O(n(n+m)\log B)$，额外空间为 $O(n+m)$。需要幂或进制转换见 14.1.006 高精度进制转换与快速幂，只求向下取整平方根见 14.1.016 高精度整数平方根。

除法逐位定商和跨块进位最易写错。下面检查 10^9 分块边界和一组固定四则答案；长位数压力测试放在本地测试文件：

```
signed main()
{
    assert(BigInt::addAbs(0, 0).str() == "0");
    assert((BigInt("999999999") + 1).str() == "1000000000");
    assert((BigInt("1000000000") - 1).str() == "999999999");

    BigInt x("1234567890123"), y("987654321");
```

```
    assert((x + y).str() == "12355555444444");
    assert((x - y).str() == "1233580235802");
    assert((x * y).str() == "121932631124783417148 3");
    assert((x / y).str() == "1249");
    assert((x % y).str() == "987643194");
    assert((-x / y).str() == "-1249");
}
```

14.1.002 分数类与有理数还原

有理数类用 64 位分子和正分母保存唯一既约表示。构造时统一分母符号并约分；乘除先交叉约分，既能减小中间量，也省去额外的大整数包装。比较时的交叉乘法有较高溢出风险，只在这一处局部使用 `__int128`。标题所说的“有理数还原”若指由近似值反推最简分数，还需另配 Stern–Brocot 树或连分数逼近，本条只负责精确运算。

```
struct Frac
{
    long long x, y; // x、y 是既约分子和正分母。

    Frac(long long x = 0, long long y = 1) : x(x), y(y)
    {
        // x、y 是分子和非零分母；构造既约分数 x/y。
        assert(y); // 调试检查，可删。
        if (y < 0)
        {
            x = -x;
            y = -y;
        }
        long long g = gcd(llabs(x), y);
        this->x = x / g;
        this->y = y / g;
    }

    friend Frac operator+(const Frac &a, const Frac &b)
    {
        // a、b 是两个分数；返回 a+b。
        long long g = gcd(a.y, b.y);
        return {a.x * (b.y / g) + b.x * (a.y / g), a.y / g * b.y};
    }
    friend Frac operator-(const Frac &a, const Frac &b)
    {
        // a 是被减数，b 是减数；返回 a-b。
        long long g = gcd(a.y, b.y);
        return {a.x * (b.y / g) - b.x * (a.y / g), a.y / g * b.y};
    }
    friend Frac operator*(const Frac &a, const Frac &b)
    {
        // a、b 是两个分数；返回 a*b。
        long long g = gcd(llabs(a.x), b.y);
        long long h = gcd(llabs(b.x), a.y);
```

```
        return {a.x / g * (b.x / h), a.y / h * (b.y / g)};
    }
    friend Frac operator/(const Frac &a, const Frac &b)
    {
        // a 是被除数，b 是非零除数；返回 a/b。
        assert(b.x); // 调试检查，可删。
        long long g = gcd(llabs(a.x), llabs(b.x));
        long long h = gcd(a.y, b.y);
        return {a.x / g * (b.y / h), a.y / h * (b.x / g)};
    }
    friend bool operator==(const Frac &a, const Frac &b)
    {
        // a、b 是两个分数；返回二者是否相等。
        return a.x == b.x && a.y == b.y;
    }
    friend bool operator<(const Frac &a, const Frac &b)
    {
        // a、b 是两个分数；返回 a 是否小于 b。
        return (__int128)a.x * b.y < (__int128)b.x * a.y;
    }
};
```

用 `Frac(x,y)` 构造，其中 $y \neq 0$；随后直接使用 `+ - * / < ==`。调用者须保证取绝对值、分母变号和四则中间量能由 `long long` 表示；接近完整 64 位边界时改用 14.1.001 高精度整数（BigInt）。构造与四则各做常数次欧几里得算法，最坏 $O(\log M)$ 时间、$O(1)$ 空间，M 是操作数绝对值的最大值；比较为 $O(1)$。Python 等价物见 14.1.014 Python：`datetime`、`Fraction`、递归深度与文件 IO。

下面用可手算的固定值覆盖负分母、四则和比较：

```
signed main()
{
    assert(Frac(2, -4) == Frac(-1, 2));
    assert(Frac(1, 6) + Frac(1, 3) == Frac(1, 2));
    assert(Frac(5, 6) - Frac(1, 4) == Frac(7, 12));
    assert(Frac(-7, 3) < Frac(-2, 1));
    assert(Frac(2, 3) * Frac(9, 4) == Frac(3, 2));
    assert(Frac(2, 3) / Frac(4, 9) == Frac(3, 2));
    cout << "OK\n";
    return 0;
}
```

14.1.003 表达式解析：中缀转后缀、表达式树

递归下降求值：`expr` 处理最低优先级的 $+$, $-$，`term` 处理 $*$, $/$，`atom` 迭代读完连续一元正负号，再处理括号或整数。三层调用自然实现优先级，二元运算的循环实现左结合。各层用 `bool` 区分成功与失败，`eval` 还要求完整消费输入，因此空串、残缺表达式、多余字符、除零和 64 位中间结果溢出都返回 `nullopt`，不会越界或产生未定义行为。

```
struct Expr
{
    // s 保存待求值表达式。
    string s;
    // p 保存下一个待读字符的位置。
    int p = 0;

    void skip()
    {
        // 无参数；把读位置 p 向后移过 s 中连续的空白字符。
        while (p < (int)s.size() && isspace((unsigned char)s[p]))
        {
            p++;
        }
    }

    static bool take(__int128 v, long long &x)
    {
        // v 是中间结果，x 接收 64 位值；可表示时写入 x 并返回 true。
        if (v < LLONG_MIN || v > LLONG_MAX)
        {
            return false;
        }
        x = (long long)v;
        return true;
    }

    bool expr(long long &x)
    {
        // x 接收当前位置起的加减表达式值；语法或数值失败时返回 false。
        if (!term(x))
        {
            return false;
        }
        skip();
        while (p < (int)s.size() && (s[p] == '+' || s[p] == '-'))
        {
            char op = s[p++];
            long long y;
            if (!term(y))
            {
                return false;
            }
            __int128 v = op == '+' ? (__int128)x + y : (__int128)x - y;
            if (!take(v, x))
            {
                return false;
            }
            skip();
        }
        return true;
    }
```

```
53
54      bool term(long long &x)
55      {
56          // x 接收当前位置起的乘除项值；语法或数值失败时返回 false。
57          if (!atom(x))
58          {
59              return false;
60          }
61          skip();
62          while (p < (int)s.size() && (s[p] == '*' || s[p] == '/'))
63          {
64              char op = s[p++];
65              long long y;
66              if (!atom(y) || (op == '/' && !y))
67              {
68                  return false;
69              }
70              skip();
71              __int128 v = op == '*' ? (__int128)x * y : (__int128)x / y;
72              if (!take(v, x))
73              {
74                  return false;
75              }
76          }
77          return true;
78      }
79
80      bool atom(long long &x)
81      {
82          // x 接收当前位置起的一元表达式值；语法或数值失败时返回 false。
83          skip();
84          bool neg = false;
85          while (p < (int)s.size() && (s[p] == '+' || s[p] == '-'))
86          {
87              neg ^= s[p] == '-';
88              p++;
89              skip();
90          }
91          if (p >= (int)s.size())
92          {
93              return false;
94          }
95          if (s[p] == '(')
96          {
97              p++;
98              if (!expr(x) || p >= (int)s.size() || s[p] != ')')
99              {
100                 return false;
101             }
102             p++;
103             return !neg || take(-(__int128)x, x);
104         }
```

```
        if (s[p] < '0' || s[p] > '9')
        {
            return false;
        }
        __int128 v = 0;
        while (p < (int)s.size() && s[p] >= '0' && s[p] <= '9')
        {
            v = v * 10 + s[p++] - '0';
            if (v > (__int128)LLONG_MAX + 1)
            {
                return false;
            }
        }
        if (neg)
        {
            v = -v;
        }
        return take(v, x);
    }

    optional<long long> eval(const string &t)
    {
        // t 是待求值表达式；成功时返回整数值，语法或数值失败时返回空。
        s = t;
        p = 0;
        long long x;
        if (!expr(x))
        {
            return nullopt;
        }
        skip();
        if (p != (int)s.size())
        {
            return nullopt;
        }
        return x;
    }
};
```

`Expr::eval(t)` 支持 `+ - * /`、括号、连续一元正负号和任意空白，成功时返回整数值，失败时返回空。除法向零截断。长度为 n 的表达式用 $O(n)$ 时间，递归栈为 $O(d)$，其中 d 是括号嵌套深度；一元符号已迭代处理，不增加递归层数。精确有理数中间值可改用 14.1.002 分数类与有理数还原，任意精度整数可改用 14.1.001 高精度整数（BigInt）。

下面用固定表达式分别覆盖优先级、左结合除法、运算符后一元负号、连续一元符号、嵌套括号和失败分支：

```
signed main()
{
    Expr e;
    assert(e.eval("2+3*4") == 14);
```

```
    assert(e.eval("8/3*3") == 6);
    assert(e.eval("2*-3") == -6);
    assert(e.eval("--5") == 5);
    assert(e.eval("-(3+4)*2") == -14);
    assert(e.eval("((2+3)*(4-1))") == 15);
    assert(e.eval("-9223372036854775808") == LLONG_MIN);
    assert(!e.eval(""));
    assert(!e.eval("1+"));
    assert(!e.eval("(1+2"));
    assert(!e.eval("1/0"));
    assert(!e.eval("1a"));
    cout << "OK\n";
    return 0;
}
```

14.1.004 德州扑克牌型判断（五张牌与多手牌最佳五张）

`pokerScore` 把五张牌压成一个可字典序比较的评分数组 `z`：`z[0]` 是牌型等级（8 同花顺、7 四条、6 葫芦、5 同花、4 顺子、3 三条、2 两对、1 一对、0 高牌），其后各维依次填决胜的关键点数，于是两手牌直接按 `array` 字典序比大小即可。点数用 $2\text{—}14$（A 记 14），先降序排序，并特判把 A-5-4-3-2 的“轮子顺”改写成 $5,4,3,2,1$ 使最小顺子也能被检出；`flush` 看五张花色是否全同，`straight` 看相邻差是否恒为 1，再用按出现次数降序的 `(cnt,rank)` 对区分四条 / 葫芦 / 三条 / 两对 / 一对。判定顺序必须“同花顺 → 四条 → 葫芦 →……”从强到弱，否则同花顺会被同花或顺子截胡。七张或更多牌取最佳五张时，在外层枚举五张子集分别调用本函数取字典序最大评分即可，本条只实现五张的核心比较。

```
array<int, 6> pokerScore(vector<int> r, vector<int> suit)
{
    // r、suit 依次给出五张牌的点数与花色；返回可直接字典序比较的六元评分。
    assert(r.size() == 5 && suit.size() == 5); // 调试检查，可删。
    set<pair<int, int>> card;
    for (int i = 0; i < 5; i++)
    {
        assert(2 <= r[i] && r[i] <= 14); // 调试检查，可删。
        assert(0 <= suit[i] && suit[i] < 4); // 调试检查，可删。
        card.insert({r[i], suit[i]});
    }
    assert(card.size() == 5); // 调试检查，可删。
    sort(r.begin(), r.end(), greater<>());
    if (r == vector<int>{14, 5, 4, 3, 2})
    {
        r = {5, 4, 3, 2, 1};
    }
    bool flush = count(suit.begin(), suit.end(), suit[0]) == 5;
    bool straight = true;
    for (int i = 1; i < 5; i++)
    {
        straight &= r[i - 1] == r[i] + 1;
    }
    map<int, int, greater<>> f;
```

```
    for (int x : r)
    {
        f[x]++;
    }
    vector<pair<int, int>> a;
    for (auto [x, c] : f)
    {
        a.push_back({c, x});
    }
    sort(a.begin(), a.end(), greater<>());
    array<int, 6> z{};
    if (straight && flush)
    {
        z[0] = 8;
        z[1] = r[0];
    }
    else if (a[0].first == 4)
    {
        z[0] = 7;
        z[1] = a[0].second;
        z[2] = a[1].second;
    }
    else if (a[0].first == 3 && a[1].first == 2)
    {
        z[0] = 6;
        z[1] = a[0].second;
        z[2] = a[1].second;
    }
    else if (flush)
    {
        z[0] = 5;
        copy(r.begin(), r.end(), z.begin() + 1);
    }
    else if (straight)
    {
        z[0] = 4;
        z[1] = r[0];
    }
    else if (a[0].first == 3)
    {
        z[0] = 3;
        z[1] = a[0].second;
        vector<int> q;
        for (auto [c, x] : a)
        {
            if (c == 1)
            {
                q.push_back(x);
            }
        }
        sort(q.begin(), q.end(), greater<>());
        copy(q.begin(), q.end(), z.begin() + 2);
```

```
    }
    else if (a[0].first == 2 && a[1].first == 2)
    {
        z[0] = 2;
        z[1] = max(a[0].second, a[1].second);
        z[2] = min(a[0].second, a[1].second);
        z[3] = a[2].second;
    }
    else if (a[0].first == 2)
    {
        z[0] = 1;
        z[1] = a[0].second;
        vector<int> q;
        for (auto [c, x] : a)
        {
            if (c == 1)
            {
                q.push_back(x);
            }
        }
        sort(q.begin(), q.end(), greater<>());
        copy(q.begin(), q.end(), z.begin() + 2);
    }
    else
    {
        copy(r.begin(), r.end(), z.begin() + 1);
    }
    return z;
}
```

调用 `pokerScore(r,suit)`，两向量必须按相同次序描述五张互不重复的实体牌，点数在 $[2,14]$，花色在 $[0,4)$；返回数组可直接用 `<`、`>` 比较。单手判定为 $O(1)$ 时间和空间；七张牌枚举 $\binom{7}{5}=21$ 个子集仍为 $O(1)$。胜率估计需在外层采样，不属于牌型判定。

下面固定检查顺子同花、A-5 轮子顺、葫芦、两对踢脚和普通高牌；完整牌型覆盖放在本地测试文件：

```
signed main()
{
    assert((pokerScore({14, 13, 12, 11, 10}, {0, 0, 0, 0, 0})
            == array<int, 6>{8, 14, 0, 0, 0, 0}));
    assert((pokerScore({4, 4, 4, 2, 2}, {0, 1, 2, 0, 1})
            == array<int, 6>{6, 4, 2, 0, 0, 0}));
    assert((pokerScore({14, 5, 4, 3, 2}, {0, 1, 2, 3, 0})
            == array<int, 6>{4, 5, 0, 0, 0, 0}));
    assert((pokerScore({14, 14, 2, 2, 9}, {0, 1, 0, 1, 2})
            == array<int, 6>{2, 14, 2, 9, 0, 0}));
    assert((pokerScore({14, 11, 9, 5, 2}, {0, 1, 2, 3, 0})
            == array<int, 6>{0, 14, 11, 9, 5, 2}));
    cout << "OK\n";
    return 0;
}
```

14.1.005 日期与时间换算

公历日期与“距 1970-01-01 的天数”互转，核心是 Howard Hinnant 的纪元–年–日算法：`days()` 先把 ≤ 2 月的月份并入上一年（`yy=y-(m<=2)`），以 400 年为一个纪元（恰 146097 天）算出纪元编号 `era`、纪元内年 `yo` 与内日 `dy`，合成后减 719468 把原点对齐到 1970-01-01；`fromDays` 是它逐步的逆变换。闰年按公历“四年一闰、百年不闰、四百年再闰”判定，月天数查表并对二月补闰日——所以任何日期加减都必须先转成连续日序，绝不能按固定 365 天硬算。星期由 `(days()%7+11)%7` 给出（0 为周日），$+11$ 是把可能为负的取模结果掰回 $[0,7)$ 的偏移。

```
struct Date
{
    // y 保存公历年份。
    int y;
    // m 保存公历月份。
    int m;
    // d 保存公历月内日期。
    int d;

    static bool leap(int y)
    {
        // y 是公历年份；返回该年是否为闰年。
        return y % 4 == 0 && (y % 100 != 0 || y % 400 == 0);
    }

    static int monthDays(int y, int m)
    {
        // y、m 是公历年份和月份；返回该月的天数。
        static constexpr int a[] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
        assert(1 <= m && m <= 12); // 调试检查，可删。
        return a[m] + (m == 2 && leap(y));
    }

    bool valid() const
    {
        // 无参数；返回当前年月日是否为合法公历日期。
        return 1 <= m && m <= 12 && 1 <= d && d <= monthDays(y, m);
    }

    // 与 1970-01-01 相差的天数，可为负。
    long long days() const
    {
        // 无参数；返回当前日期与 1970-01-01 相差的天数。
        assert(valid()); // 调试检查，可删。
        __int128 yy = (__int128)y - (m <= 2);
        __int128 era = (yy >= 0 ? yy : yy - 399) / 400;
        __int128 yo = yy - era * 400;
        __int128 mp = m + (m > 2 ? -3 : 9);
        __int128 dy = (153 * mp + 2) / 5 + d - 1;
        __int128 de = yo * 365 + yo / 4 - yo / 100 + dy;
        __int128 z = era * 146097 + de - 719468;
        assert(LLONG_MIN <= z && z <= LLONG_MAX); // 调试检查，可删。
```

```
        return (long long)z;
    }

    // days() 的逆变换。
    static Date fromDays(long long z)
    {
        // z 是相对 1970-01-01 的天数；返回对应公历日期。
        __int128 zz = (__int128)z + 719468;
        __int128 era = (zz >= 0 ? zz : zz - 146096) / 146097;
        __int128 de = zz - era * 146097;
        __int128 yo = (de - de / 1460 + de / 36524 - de / 146096) / 365;
        __int128 yy = yo + era * 400;
        __int128 dy = de - (365 * yo + yo / 4 - yo / 100);
        __int128 mp = (5 * dy + 2) / 153;
        __int128 dd = dy - (153 * mp + 2) / 5 + 1;
        __int128 mm = mp + (mp < 10 ? 3 : -9);
        yy += mm <= 2;
        return {(long long)yy, (long long)mm, (long long)dd};
    }

    // 周日为 0，周一为 1，…，周六为 6。
    int weekday() const
    {
        // 无参数；返回当前日期的星期编号，周日为 0。
        return (int)((days() % 7 + 11) % 7);
    }
};
```

构造 `Date{y,m,d}` 后可用 `valid()` 判断日期是否合法；`days()` 要求日期合法，并要求相对 1970-01-01 的日序可由 `long long` 表示。`Date::fromDays(z)` 接受任意 64 位日序，`weekday()` 返回 $[0,7)$，周日为 0。所有接口均为 $O(1)$ 时间和空间。Python 的 `date`、`timedelta` 见 14.1.014 Python：datetime、Fraction、递归深度与文件 IO。

下面固定检查纪元、世纪闰年和日期往返：

```
signed main()
{
    assert((Date{1970, 1, 1}.days() == 0));
    assert((Date{1970, 1, 1}.weekday() == 4));
    assert(!Date::leap(1900));
    assert(Date::leap(2000));
    long long z = Date{2000, 2, 29}.days();
    Date d = Date::fromDays(z);
    assert(d.y == 2000 && d.m == 2 && d.d == 29);
    assert((Date{2024, 2, 29}.valid() && !Date{2023, 2, 29}.valid()));
}
```

14.1.006 高精度进制转换与快速幂

两个只依赖字符串或大整数的小工具。`convertBase` 在 $2-62$ 进制间转换任意长带符号整数，数码严格按 `0-9A-Za-z` 映射到 $0-61$，因此大小写在 37 进制以上含义不同；它反复对目标进制做整型短除，每次余数逆序拼成答案，零统一输出为 `0`。`bigPow` 是 `BigInt` 上的二进制快速幂——按指数 `e` 的二进制位决定是否把当前底数乘入答案、同时底数自乘，乘法次数从 e 降到 $O(\log e)$，底层乘法复用 14.1.001 高精度整数（BigInt）；`convertBase` 本身不用 `BigInt`。

```
// 依赖 14.1.001 高精度整数（BigInt）
string convertBase(string s, int from, int to)
{
    // s 是字符序 0-9A-Za-z 下带可选正负号的 from 进制整数；返回同字符序、无前导零的 to 进制字符串。
    assert(!s.empty()); // 调试检查，可删。
    assert(2 <= from && from <= 62 && 2 <= to && to <= 62); // 调试检查，可删。
    const string digit = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz";
    bool neg = s[0] == '-';
    int p = neg || s[0] == '+';
    assert(p < (int)s.size()); // 调试检查，可删。
    vector<int> a;
    for (; p < (int)s.size(); p++)
    {
        size_t pos = digit.find(s[p]);
        assert(pos != string::npos && pos < (size_t)from); // 调试检查，可删。
        int x = pos;
        a.push_back(x);
    }
    while (a.size() > 1 && a[0] == 0)
    {
        a.erase(a.begin());
    }
    string ans;
    while (!(a.size() == 1 && a[0] == 0))
    {
        vector<int> b;
        int rem = 0;
        for (int x : a)
        {
            int cur = rem * from + x;
            if (!b.empty() || cur / to)
            {
                b.push_back(cur / to);
            }
            rem = cur % to;
        }
        ans.push_back(digit[rem]);
        a = b.empty() ? vector<int>{0} : move(b);
    }
    if (ans.empty())
    {
        return "0";
    }
```

```
    reverse(ans.begin(), ans.end());
    return neg ? "-" + ans : ans;
}

BigInt bigPow(BigInt a, int e)
{
    // a 是任意精度整数，e 是非负整数；返回 a^e，依赖 14.1.001 的 BigInt 乘法。
    assert(e >= 0); // 调试检查，可删。
    BigInt ans = 1;
    while (e)
    {
        if (e & 1)
        {
            ans *= a;
        }
        e >>= 1;
        if (e)
        {
            a *= a;
        }
    }
    return ans;
}
```

`convertBase(s,from,to)` 要求 `s` 除可选首字符 `+`、`-` 外至少有一位合法数码；`from`、`to` 均在 $[2,62]$，输出沿用同一字符序并去掉前导零。先引入 14.1.001 高精度整数（BigInt） 后可调用 `bigPow(a,e)`，其中 $e \geq 0$。设转换途中数值最多 D 位，反复短除用 $O(D^2)$ 时间、$O(D)$ 空间；快速幂做 $O(\log e)$ 次高精度乘法。`convertBase("-z",62,10)` 返回 `-61`，`convertBase("61",10,62)` 返回 `z`，`bigPow(BigInt("12"),10).str()` 返回 `61917364224`。

14.1.007 9×9 数独字典序最小解

标准 9×9 数独求解，用三个位掩码 `row`/`col`/`box` 各记一行、一列、一宫已用的数字（第 v 位表示数字 v，常量 `1022` 即二进制 `1111111110`、圈定候选 $1\text{—}9$）。要得到标题承诺的字典序最小解，`dfs` 必须严格按行优先顺序逐格填、每格从最小候选试起、找到首解即停：先固定的靠前格因此拿到尽可能小的值，整盘按行读出才最小。原实现此处改用了最少候选数（MRV）启发式挑格，能快速求出某个合法解却不保证字典序最小——空盘下它给出的第 4 行是 `231674895`，而真正字典序最小解的第 4 行为 `214365897`；下面已改回行优先扫描，参数 `z` 也回归“下一个待检查格编号”的本义。`solve` 先把初盘已填数字灌入三个掩码并顺带检测冲突，再调 `dfs`。

```
struct Sudoku
{
    // a 保存 9x9 棋盘，0 表示空格，1..9 表示已填数字。
    array<array<int, 9>, 9> a{};
    // row[i] 的第 v 位表示第 i 行是否已使用数字 v。
    array<int, 9> row{};
    // col[j] 的第 v 位表示第 j 列是否已使用数字 v。
    array<int, 9> col{};
```

```
    // box[b] 的第 v 位表示第 b 个 3x3 宫是否已使用数字 v。
    array<int, 9> box{};

    bool dfs(int z = 0)
    {
        // 参数 z 表示 按行优先扫描时下一个待检查的格编号；从该格起继续深度优先搜索，成功返回 true。
        while (z < 81 && a[z / 9][z % 9])
        {
            z++;
        }
        if (z == 81)
        {
            return true;
        }
        int x = z / 9;
        int y = z % 9;
        int b = x / 3 * 3 + y / 3;
        int m = ~(row[x] | col[y] | box[b]) & 1022;
        while (m)
        {
            int v = countr_zero((unsigned)m);
            m &= m - 1;
            a[x][y] = v;
            row[x] |= 1 << v;
            col[y] |= 1 << v;
            box[b] |= 1 << v;
            if (dfs(z + 1))
            {
                return true;
            }
            row[x] ^= 1 << v;
            col[y] ^= 1 << v;
            box[b] ^= 1 << v;
            a[x][y] = 0;
        }
        return false;
    }

    bool solve()
    {
        // 无参数；验证初盘后求字典序最小完整数独，成功返回 true，矛盾时返回 false。
        row.fill(0);
        col.fill(0);
        box.fill(0);
        for (int i = 0; i < 9; i++)
        {
            for (int j = 0; j < 9; j++)
            {
                assert(0 <= a[i][j] && a[i][j] <= 9); // 调试检查，可删。
                if (a[i][j])
                {
                    int b = i / 3 * 3 + j / 3;
```

```
                    int q = 1 << a[i][j];
                    if ((row[i] | col[j] | box[b]) & q)
                    {
                        return false;
                    }
                    row[i] |= q;
                    col[j] |= q;
                    box[b] |= q;
                }
            }
        }
        return dfs();
    }
};
```

填好标准 9×9 初盘后调用 `solve()`，其中 0 表示空格，非零格必须在 $[1,9]$。成功时 `a` 就地改成按行展开后字典序最小的完整解；取值合法但已有行、列或宫冲突时返回 `false`。设空格数为 k，最坏 $O(9^k)$ 时间、$O(k)$ 递归栈。需要更快地求任意一组解时可改用 Dancing Links；同类构造见 14.1.018 N 皇后构造解。

14.1.008 装箱问题

装箱是 NP-hard，这里给三档实现。`firstFitDec` 是首次适应下降近似：物品按体积降序，逐个放进第一个装得下的箱（`rem` 按开箱顺序存各箱剩余容量），放不下就开新箱，简单但当前线性找箱为 $O(n^2)$。`firstFit` 保留原输入顺序，复用 3.3.001 单点修改线段树与摘要二分 的 `Seg<MaxInfo>`：每个叶子是一个箱子的剩余容量，区间摘要取最大值，`findFirst` 找最早还能容纳当前物品的箱子；尚未使用的叶子初值也是箱容量，所以第一次走到它就等价于开新箱。`exactBins` 是 $O(n2^n)$ 的子集 DP 求精确最少箱数：`dp[s]` 记“已装物品集合为 s”时按字典序最优的二元组 $(\text{已用箱数},\text{末箱已用容量})$，转移把某个未装物品 i 塞进当前末箱（装得下）或新开一箱、取字典序最小。以 $(\text{箱数},\text{末箱负载})$ 为最小化目标是这套 DP 正确的关键：箱数相同再压低末箱负载，才能给后续物品留最大空隙。

```
// 依赖 3.3.001 单点修改线段树与摘要二分
// 知识点 14.1.008：装箱问题的近似与小规模精确解。
namespace bin_packing
{
int firstFitDec(vector<int> a, int cap)
{
    // a 是物品体积，cap 是箱容量；返回降序首次适应使用的箱数。
    assert(cap >= 0); // 调试检查，可删。
    for (int x : a)
    {
        assert(0 <= x && x <= cap); // 调试检查，可删。
    }
    sort(a.begin(), a.end(), greater<>());
    vector<int> rem;
    for (int x : a)
    {
        int p = 0;
        while (p < (int)rem.size() && rem[p] < x)
        {
```

```
            p++;
        }
        if (p == (int)rem.size())
        {
            rem.push_back(cap);
        }
        rem[p] -= x;
    }
    return rem.size();
}

pair<int, vector<int>> firstFit(const vector<int> &a, int cap)
{
    // 按输入顺序放置，返回使用箱数与每件物品所在的 0 下标箱号。
    assert(cap >= 0); // 调试检查，可删
    int n = (int)a.size();
    vector<int> rem(n, cap), box;
    vector<MaxInfo> init(n, MaxInfo(cap));
    Seg<MaxInfo> st(init);
    box.reserve(n);
    int used = 0;
    for (int x : a)
    {
        assert(0 <= x && x <= cap); // 调试检查，可删
        int p = st.findFirst(0, n, [&](MaxInfo z) { return z.val >= x; });
        assert(p != -1); // 前 i 件物品至多占用 i 个箱子
        rem[p] -= x;
        st.set(p, MaxInfo(rem[p]));
        used = max(used, p + 1);
        box.push_back(p);
    }
    return {used, box};
}

int exactBins(const vector<int> &a, int cap)
{
    // a 是至多 24 个物品体积，cap 是箱容量；返回最少箱数。
    int n = (int)a.size();
    assert(0 <= n && n <= 24); // 调试检查，可删。
    assert(cap >= 0); // 调试检查，可删。
    for (int x : a)
    {
        assert(0 <= x && x <= cap); // 调试检查，可删。
    }
    // State[0..1] 依次是已用箱数与最后一个箱的已用容量。
    using State = array<int, 2>;
    State inf{n + 1, 0};
    vector<State> dp(1LL << n, inf);
    dp[0] = {1, 0};
    for (int s = 0; s < (1LL << n); s++)
    {
        for (int i = 0; i < n; i++)
```

```
        {
            if (s >> i & 1)
            {
                continue;
            }
            State cur = dp[s];
            if (a[i] <= cap - cur[1])
            {
                cur[1] += a[i];
            }
            else
            {
                cur[0]++;
                cur[1] = a[i];
            }
            dp[s | 1LL << i] = min(dp[s | 1LL << i], cur);
        }
    }
    return n == 0 ? 0 : dp.back()[0];
}
}
```

`firstFitDec(a,cap)` 返回 FFD 的近似箱数；`firstFit(a,cap)` 返回按输入顺序 First-Fit 使用的箱数及每件物品的 0 下标箱号；二者都要求物品体积落在 $[0,cap]$，但排序会改变放置顺序，结果不能混称。当前 FFD 为 $O(n^2)$，线段树 First-Fit 为 $O(n\log n)$ 时间、$O(n)$ 空间。`exactBins(a,cap)` 要求物品数 ≤ 24，以 $O(n2^n)$ 时间、$O(2^n)$ 空间返回精确最少箱数。物品多且必须保持到达顺序时用 `firstFit`，允许重排且只需简单近似时用 FFD，$n\le 24$ 要精确最优用 `exactBins`；若还要在容量约束下优化物品价值则退化为背包，属第 12 章动态规划范畴。

14.1.009 选数与网格路径计数

只向下或向右走的网格路径计数。无障碍时从 $(0,0)$ 到 $(h-1,w-1)$ 有 $\binom{h+w-2}{h-1}$ 条路径。有障碍时把障碍与终点按坐标排序，令 `dp[i]` 为到第 i 个点且不经过任何更早障碍的路径数；先计全部路径，再减去每个能到达它的障碍 j 所截获的 $dp_j\binom{(x_i-x_j)+(y_i-y_j)}{x_i-x_j}$。组合数复用第 8 章代表实现，本条只保留容斥转移。

```
// 知识点 14.1.009：障碍点网格最短路径计数。
namespace grid_count
{
template<class T, class C>
T gridPaths(int h, int w, vector<array<int, 2>> ban, C &comb)
{
    // h、w 是网格尺寸，ban[i] 是障碍坐标，comb 提供 C(n,k)；返回模 T::mod() 的路径数。
    assert(h > 0 && w > 0); // 调试检查，可删。
    assert(h - 1 < T::mod() && w - 1 < T::mod() - (h - 1)); // 调试检查，可删。
    for (auto p : ban)
    {
        assert(0 <= p[0] && p[0] < h && 0 <= p[1] && p[1] < w); // 调试检查，可删。
    }
```

```
    sort(ban.begin(), ban.end());
    ban.erase(unique(ban.begin(), ban.end()), ban.end());
    array<int, 2> goal{h - 1, w - 1};
    if (binary_search(ban.begin(), ban.end(), goal))
    {
        return T(0);
    }
    ban.push_back(goal);
    sort(ban.begin(), ban.end());
    int goalPos = lower_bound(ban.begin(), ban.end(), goal) - ban.begin();
    vector<T> dp(ban.size());
    for (int i = 0; i < (int)ban.size(); i++)
    {
        auto [x, y] = ban[i];
        dp[i] = comb.C(x + y, x);
        for (int j = 0; j < i; j++)
        {
            auto [u, v] = ban[j];
            if (u <= x && v <= y)
            {
                dp[i] -= dp[j] * comb.C(x - u + y - v, x - u);
            }
        }
    }
    return dp[goalPos];
}
}
```

先引入 1.1.008 静态取模类 的 `MInt<P>` 与 8.1.001 阶乘、逆阶乘与组合数 的 `Comb<T>`，再以 `Comb<Z> c; grid_count::gridPaths<Z>(h,w,ban,c)` 调用。要求 $h, w > 0$、障碍均在网格内，且 $h + w - 2 < P$，从而所有阶乘在素模 P 下可逆；重复障碍会自动去重，终点是障碍时返回 0。设去重后有 k 个障碍，预处理与 DP 共用 $O(h + w + k^2)$ 时间、$O(h + w + k)$ 空间。网格较小而障碍很多时改用 $O(hw)$ 的逐格递推。

14.1.010 坐标类

二维点、向量、直线和线段基本操作统一使用 11.1.002 Point / Vector 的 `Point<T>`、`Line<T>`、点积、叉积、投影、交点与距离接口，本章不再复制同一实现。该代表条目要求直线方向向量非零；求两直线唯一交点前还须确认它们不平行。

浮点坐标有两套不能混用的关系：排序、`set` 和 `map` 的比较必须采用原始坐标的精确字典序，以保持严格弱序；判断几何重合、共线或方向时才用 `eps` 判号。整点的精确共线与格点计数应直接使用整数坐标和宽化叉积，含角度、投影或距离时再使用浮点版本。所有基础点线操作均为 $O(1)$ 时间和空间。

14.1.011 Python：基础语法、容器、字符串、数学库

Python 3 竞赛常用写法速查。`tokens()` 一次性读入并按空白切分（`sys.stdin.buffer.read().split()` 是大数据量下最快的读法），其余是列表推导、`bisect` 二分、`Counter`/`defaultdict`/`deque`、`heapq` 小根堆

（模拟大根堆时压相反数）、字符串 `split`/`join`/切片，以及 `math` 的 `gcd`/`lcm`/`isqrt`。要点在于 Python 整数自动扩展精度、不会溢出，列表与字符串切片会复制（$O(n)$），集合与字典平均 $O(1)$。

```python
import heapq
import math
import sys
from bisect import bisect_left, bisect_right
from collections import Counter, defaultdict, deque


def tokens():
    # 无参数；读取标准输入剩余内容，返回按空白切分的字节串列表。
    return sys.stdin.buffer.read().split()


# a 是整数表；切片会复制，推导式生成新表。
a = [1, 3, 3, 8]
b = [x * x for x in a if x & 1]
lo = bisect_left(a, 3)
hi = bisect_right(a, 3)

# cnt 是频率表，pos 按值保存出现位置，q 是双端队列。
cnt = Counter(a)
pos = defaultdict(list)
for i, x in enumerate(a):
    pos[x].append(i)
q = deque(a)
q.appendleft(0)

# hp 是小根堆；要模拟大根堆时压入相反数。
hp = a[:]
heapq.heapify(hp)
mn = heapq.heappop(hp)

# 字符串不可修改；split、join 和切片都会产生新字符串。
s = "acm-template"
parts = s.split("-")
joined = "/".join(parts)
rev = s[::-1]

# Python 整数不限位数；数学库函数的返回类型按标准库定义。
g = math.gcd(84, 30)
lcm = math.lcm(84, 30)
root = math.isqrt(10**100)
ans = sum(b)
sys.stdout.write(str(ans) + "\n")
```

整段按 Python 3 运行，示例变量只为展示接口，提交时保留需要的部分即可；`tokens()` 读取剩余全部标准输入。列表末尾追加均摊 $O(1)$、排序 $O(n \log n)$、二分 $O(\log n)$、字典与集合平均 $O(1)$，切片 / 拼接 / 整体读入 $O(n)$。数据结构与流、多项式的 Python 版见 14.1.012 Python：并查集、线段树、最大流、费用流、FFT、NTT，日期与精确分数见 14.1.014 Python：`datetime`、`Fraction`、递归深度与文件 IO。

14.1.012 Python：并查集、线段树、最大流、费用流、FFT、NTT

Python 版的并查集、点修改区间和线段树、Dinic 最大流、SPFA 增广的最小费用最大流、FFT 与 NTT，供允许 Python 且规模适中的题直接取用。`SegTree` 把内部叶数补到 $N = \text{bit_ceil}(n)$，使用长度 $2N$ 的数组；Dinic 的阻塞流搜索使用显式路径栈，不受 Python 递归深度限制；流量、距离与费用直接使用任意精度整数，不设置有限“无穷”哨兵。

```python
import cmath
from collections import deque


class DSU:
    def __init__(self, n):
        # n 是点数；建立 n 个单点集合，无返回值。
        self.fa = list(range(n))
        self.siz = [1] * n

    def find(self, x):
        # x 是点编号；返回 x 所在集合的根并做路径压缩。
        while x != self.fa[x]:
            self.fa[x] = self.fa[self.fa[x]]
            x = self.fa[x]
        return x

    def merge(self, x, y):
        # x、y 是点编号；合并成功返回 True，原本连通返回 False。
        x, y = self.find(x), self.find(y)
        if x == y:
            return False
        if self.siz[x] < self.siz[y]:
            x, y = y, x
        self.fa[y] = x
        self.siz[x] += self.siz[y]
        return True


class SegTree:
    def __init__(self, a):
        # a 是初始数列；建立点修改、区间和线段树，无返回值。
        self.n = 1
        while self.n < len(a):
            self.n <<= 1
        self.tr = [0] * (2 * self.n)
        self.tr[self.n:self.n + len(a)] = a
        for p in range(self.n - 1, 0, -1):
            self.tr[p] = self.tr[p << 1] + self.tr[p << 1 | 1]

    def set(self, p, x):
        # p 是 0 下标位置，x 是新值；完成单点赋值，无返回值。
        p += self.n
        self.tr[p] = x
```

```
        while p > 1:
            p >>= 1
            self.tr[p] = self.tr[p << 1] + self.tr[p << 1 | 1]

    def prod(self, l, r):
        # l、r 定义半开区间；返回该区间元素和。
        l += self.n
        r += self.n
        x = y = 0
        while l < r:
            if l & 1:
                x += self.tr[l]
                l += 1
            if r & 1:
                r -= 1
                y += self.tr[r]
            l >>= 1
            r >>= 1
        return x + y


class Dinic:
    def __init__(self, n):
        # n 是点数；g 保存边的终点、反向边编号和剩余容量。
        self.n = n
        self.g = [[] for _ in range(n)]

    def add(self, u, v, c):
        # u、v 是端点，c 是非负容量；加入一条有向边，无返回值。
        assert 0 <= u < self.n and 0 <= v < self.n
        assert c >= 0
        a = [v, len(self.g[v]), c]
        b = [u, len(self.g[u]), 0]
        self.g[u].append(a)
        self.g[v].append(b)

    def flow(self, s, t):
        # s、t 是源汇；返回最大流并修改残量网络。
        assert 0 <= s < self.n and 0 <= t < self.n and s != t
        ans = 0
        while True:
            dep = [-1] * self.n
            dep[s] = 0
            q = deque([s])
            while q:
                u = q.popleft()
                for v, _, c in self.g[u]:
                    if c and dep[v] < 0:
                        dep[v] = dep[u] + 1
                        q.append(v)
            if dep[t] < 0:
                return ans
```

```
            cur = [0] * self.n

            def send():
                # 无参数；在当前层次图中用显式栈找到一条增广路并返回流量。
                path_v = [s]
                path_e = []
                while path_v:
                    u = path_v[-1]
                    if u == t:
                        f = min(self.g[x][i][2] for x, i in path_e)
                        for x, i in path_e:
                            e = self.g[x][i]
                            e[2] -= f
                            self.g[e[0]][e[1]][2] += f
                        return f
                    while cur[u] < len(self.g[u]):
                        e = self.g[u][cur[u]]
                        if e[2] and dep[e[0]] == dep[u] + 1:
                            break
                        cur[u] += 1
                    if cur[u] == len(self.g[u]):
                        path_v.pop()
                        if path_e:
                            x, _ = path_e.pop()
                            cur[x] += 1
                        continue
                    path_e.append((u, cur[u]))
                    path_v.append(self.g[u][cur[u]][0])
                return 0

            while True:
                f = send()
                if not f:
                    break
                ans += f


class MinCostFlow:
    def __init__(self, n):
        # n 是点数；g 保存终点、反向边编号、容量和单位费用。
        self.n = n
        self.g = [[] for _ in range(n)]

    def add(self, u, v, c, w):
        # u、v 是端点，c 是容量，w 是单位费用；加入有向边，无返回值。
        assert 0 <= u < self.n and 0 <= v < self.n
        assert c >= 0
        a = [v, len(self.g[v]), c, w]
        b = [u, len(self.g[u]), 0, -w]
        self.g[u].append(a)
        self.g[v].append(b)

```

```
    def flow(self, s, t, lim=None):
        # s、t 是源汇，lim 是可选流量上限；返回实际流量与最小费用。
        n = self.n
        assert 0 <= s < n and 0 <= t < n and s != t
        assert lim is None or lim >= 0
        total = 0
        cost = 0
        while lim is None or total < lim:
            dis = [None] * n
            pre = [None] * n
            inq = [False] * n
            depth = [0] * n
            dis[s] = 0
            q = deque([s])
            inq[s] = True
            while q:
                u = q.popleft()
                inq[u] = False
                for i, e in enumerate(self.g[u]):
                    v, _, c, w = e
                    nd = dis[u] + w
                    if c and (dis[v] is None or nd < dis[v]):
                        dis[v] = nd
                        pre[v] = (u, i)
                        depth[v] = depth[u] + 1
                        if depth[v] >= n:
                            raise ValueError("source-reachable negative cycle")
                        if not inq[v]:
                            inq[v] = True
                            q.append(v)
            if pre[t] is None:
                break
            f = None if lim is None else lim - total
            v = t
            while v != s:
                u, i = pre[v]
                c = self.g[u][i][2]
                f = c if f is None else min(f, c)
                v = u
            v = t
            while v != s:
                u, i = pre[v]
                e = self.g[u][i]
                e[2] -= f
                self.g[v][e[1]][2] += f
                cost += f * e[3]
                v = u
            total += f
        return total, cost


def fft(a, inv=False):
```

```python
    # a 是长度为二次幂的复数表，inv 表示逆变换；原地修改 a，无返回值。
    n = len(a)
    if n == 0 or n & (n - 1):
        raise ValueError("FFT length must be a nonzero power of two")
    j = 0
    for i in range(1, n):
        bit = n >> 1
        while j & bit:
            j ^= bit
            bit >>= 1
        j ^= bit
        if i < j:
            a[i], a[j] = a[j], a[i]
    length = 2
    while length <= n:
        wn = cmath.exp((2j if inv else -2j) * cmath.pi / length)
        for l in range(0, n, length):
            w = 1
            half = length >> 1
            for i in range(l, l + half):
                x = a[i]
                y = a[i + half] * w
                a[i] = x + y
                a[i + half] = x - y
                w *= wn
        length <<= 1
    if inv:
        for i in range(n):
            a[i] /= n


def conv_fft(a, b):
    # a、b 是实数系数表；返回普通卷积的浮点近似值。
    if not a or not b:
        return []
    n = 1
    while n < len(a) + len(b) - 1:
        n <<= 1
    x = list(map(complex, a)) + [0j] * (n - len(a))
    y = list(map(complex, b)) + [0j] * (n - len(b))
    fft(x)
    fft(y)
    for i in range(n):
        x[i] *= y[i]
    fft(x, True)
    return [x[i].real for i in range(len(a) + len(b) - 1)]


def ntt(a, inv=False, mod=998244353, root=3):
    # a 是模 mod 系数表，inv 表示逆变换，root 是原根；原地修改 a。
    n = len(a)
    supported = {(998244353, 3), (1004535809, 3), (469762049, 3)}
```

```
    if (mod, root) not in supported:
        raise ValueError("unsupported NTT modulus or primitive root")
    if n == 0 or n & (n - 1) or (mod - 1) % n:
        raise ValueError("invalid NTT length")
    j = 0
    for i in range(1, n):
        bit = n >> 1
        while j & bit:
            j ^= bit
            bit >>= 1
        j ^= bit
        if i < j:
            a[i], a[j] = a[j], a[i]
    length = 2
    while length <= n:
        wn = pow(root, (mod - 1) // length, mod)
        if inv:
            wn = pow(wn, mod - 2, mod)
        for l in range(0, n, length):
            w = 1
            half = length >> 1
            for i in range(l, l + half):
                x = a[i]
                y = a[i + half] * w % mod
                a[i] = (x + y) % mod
                a[i + half] = (x - y) % mod
                w = w * wn % mod
        length <<= 1
    if inv:
        ni = pow(n, mod - 2, mod)
        for i in range(n):
            a[i] = a[i] * ni % mod


def conv_ntt(a, b, mod=998244353, root=3):
    # a、b 是模 mod 系数表；返回精确模卷积，模数与原根必须在受支持表中。
    if not a or not b:
        return []
    n = 1
    while n < len(a) + len(b) - 1:
        n <<= 1
    x = [v % mod for v in a] + [0] * (n - len(a))
    y = [v % mod for v in b] + [0] * (n - len(b))
    ntt(x, mod=mod, root=root)
    ntt(y, mod=mod, root=root)
    for i in range(n):
        x[i] = x[i] * y[i] % mod
    ntt(x, True, mod, root)
    return x[:len(a) + len(b) - 1]
```

所有下标从 0 起。`Dinic.flow(s,t)` 要求端点合法、$s \neq t$、容量非负；显式栈实现不受递归深度限制。`MinCostFlow.flow(s,t,lim)` 中 `lim=None` 表示尽量增广，要求每轮残量图不存在源可达负费用环，检测

到时抛出 `ValueError`。`conv_fft` 返回浮点近似，只有调用者另行证明每个系数误差小于 $\frac{1}{2}$ 时才能舍入成精确整数；无条件精确结果应使用 `conv_ntt` 或多模 NTT 加 CRT。当前 NTT 支持 $(998244353, 3)$、$(1004535809, 3)$、$(469762049, 3)$，并要求补零长度为非零二次幂且整除 $mod - 1$。并查集单次均摊 $O(\alpha(n))$，线段树单次 $O(\log n)$、空间 $O(N)$，其中 $N = \text{bit_ceil}(n)$；Dinic 为 $O(V^2E)$，SPFA 费用流最坏 $O(FVE)$，FFT/NTT 为 $O(n \log n)$。

下面用固定答案检查主要接口；长链和大整数压力测试放在本地测试文件：

```
d = DSU(4)
assert d.merge(0, 1) and not d.merge(1, 0)
st = SegTree([1, 2, 3, 4])
assert st.prod(1, 4) == 9
st.set(2, 10)
assert st.prod(0, 4) == 17

flow = Dinic(4)
for e in [(0, 1, 3), (0, 2, 2), (1, 2, 1), (1, 3, 2), (2, 3, 4)]:
    flow.add(*e)
assert flow.flow(0, 3) == 5

empty = Dinic(4)
empty.add(0, 1, 7)
empty.add(2, 3, 9)
assert empty.flow(0, 3) == 0

mcf = MinCostFlow(3)
mcf.add(0, 1, 2, 10**6)
mcf.add(1, 2, 2, -3)
assert mcf.flow(0, 2) == (2, 2 * (10**6 - 3))

got = conv_fft([1, 2, 3], [4, -1])
assert all(abs(x - y) < 1e-8 for x, y in zip(got, [4, 7, 10, -3]))
assert conv_ntt([1, 2, 3], [4, 5]) == [4, 13, 22, 15]
```

14.1.013 Python：Splay

维护可重复整数的伸展树（Python 版），支持插入、删一个、按排名取值 `kth` 与查询严格小于给定值的元素数 `rank`。用平行数组 `ch/fa/val/cnt/siz` 存节点、第 0 项当空哨兵；每次查找命中或触底后都把相应节点 `_splay` 旋到根，摊还掉最坏深度。`_rotate` 单旋后必须先 `_pull` 儿子再 `_pull` 自己以自底向上重算子树大小，`_splay` 按 zig-zig / zig-zag 分类决定先旋父还是先旋子——次序错会破坏平衡性的摊还保证。

```
class Splay:
    def __init__(self):
        # 无参数；建立空伸展树，各列表第 0 项表示空节点。
        self.ch = [[0, 0]]
        self.fa = [0]
        self.val = [0]
        self.cnt = [0]
        self.siz = [0]
```

```
        self.rt = 0
        self.free = []

    def _new(self, x, f):
        # x 是键值，f 是父节点；创建节点并返回编号。
        if self.free:
            u = self.free.pop()
            self.ch[u] = [0, 0]
            self.fa[u] = f
            self.val[u] = x
            self.cnt[u] = 1
            self.siz[u] = 1
            return u
        self.ch.append([0, 0])
        self.fa.append(f)
        self.val.append(x)
        self.cnt.append(1)
        self.siz.append(1)
        return len(self.val) - 1

    def _recycle(self, u):
        # u 是已从树中断开的节点；清空并回收其编号，无返回值。
        self.ch[u] = [0, 0]
        self.fa[u] = 0
        self.cnt[u] = 0
        self.siz[u] = 0
        self.free.append(u)

    def _pull(self, u):
        # u 是节点编号；根据儿子和重复计数重算子树大小，无返回值。
        self.siz[u] = self.siz[self.ch[u][0]] + self.siz[self.ch[u][1]] + self.cnt[u]

    def _rotate(self, x):
        # x 是待上旋节点；完成一次旋转并维护父子关系，无返回值。
        y = self.fa[x]
        z = self.fa[y]
        d = int(self.ch[y][1] == x)
        b = self.ch[x][d ^ 1]
        if z:
            self.ch[z][int(self.ch[z][1] == y)] = x
        else:
            self.rt = x
        self.fa[x] = z
        self.ch[x][d ^ 1] = y
        self.fa[y] = x
        self.ch[y][d] = b
        if b:
            self.fa[b] = y
        self._pull(y)
        self._pull(x)

    def _splay(self, x, goal=0):
```

```
        # x 是目标节点，goal 是目标父亲；把 x 旋到 goal 下，无返回值。
        while self.fa[x] != goal:
            y = self.fa[x]
            z = self.fa[y]
            if z != goal:
                if (self.ch[y][1] == x) == (self.ch[z][1] == y):
                    self._rotate(y)
                else:
                    self._rotate(x)
            self._rotate(x)

    def _find(self, x):
        # x 是键值；若存在则伸展到根并返回节点，否则伸展最后访问点并返回 0。
        u = self.rt
        last = 0
        while u and self.val[u] != x:
            last = u
            u = self.ch[u][x > self.val[u]]
        if u:
            self._splay(u)
        elif last:
            self._splay(last)
        return u

    def insert(self, x):
        # x 是键值；插入一个 x，无返回值。
        if not self.rt:
            self.rt = self._new(x, 0)
            return
        u = self.rt
        while True:
            if self.val[u] == x:
                self.cnt[u] += 1
                self._pull(u)
                self._splay(u)
                return
            d = int(x > self.val[u])
            if not self.ch[u][d]:
                v = self._new(x, u)
                self.ch[u][d] = v
                self._pull(u)
                self._splay(v)
                return
            u = self.ch[u][d]

    def erase(self, x):
        # x 是键值；删除一个 x，成功返回 True，不存在返回 False。
        u = self._find(x)
        if not u:
            return False
        if self.cnt[u] > 1:
            self.cnt[u] -= 1
```

```
            self._pull(u)
            return True
        l, r = self.ch[u]
        if not l:
            self.rt = r
            if r:
                self.fa[r] = 0
            self._recycle(u)
            return True
        self.rt = l
        self.fa[l] = 0
        v = l
        while self.ch[v][1]:
            v = self.ch[v][1]
        self._splay(v)
        self.ch[v][1] = r
        if r:
            self.fa[r] = v
        self._pull(v)
        self._recycle(u)
        return True

    def kth(self, k):
        # k 是从 0 开始的排名；返回对应键值，越界返回 None。
        if k < 0 or k >= self.siz[self.rt]:
            return None
        u = self.rt
        while True:
            left = self.siz[self.ch[u][0]]
            if k < left:
                u = self.ch[u][0]
            elif k < left + self.cnt[u]:
                self._splay(u)
                return self.val[u]
            else:
                k -= left + self.cnt[u]
                u = self.ch[u][1]

    def rank(self, x):
        # x 是查询值；返回严格小于 x 的元素个数，并伸展最后访问节点。
        u = self.rt
        last = 0
        ans = 0
        while u:
            last = u
            if x <= self.val[u]:
                u = self.ch[u][0]
            else:
                ans += self.siz[self.ch[u][0]] + self.cnt[u]
                u = self.ch[u][1]
        if last:
            self._splay(last)
```

```
        return ans
```

先构造 `Splay()`：`insert(x)` 插一个，`erase(x)` 删一个，`kth(k)` 取第 k 小，`rank(x)` 返回严格小于 x 的元素数。各操作均摊 $O(\log n)$、单次最坏 $O(n)$；空闲编号会复用，数组空间为 $O(M)$，其中 M 是历史上同时存在的不同键数峰值，不随累计插入次数增长。`sortedcontainers.SortedList` 是第三方库，只有评测环境已安装时才能替代本实现；C++ 平衡树见第 3 章数据结构。

下面用一组短操作检查重复计数、排名、删除和编号复用：

```
t = Splay()
for x in [3, 1, 3, 2]:
    t.insert(x)
assert [t.kth(i) for i in range(4)] == [1, 2, 3, 3]
assert t.rank(3) == 2
assert t.erase(3) and [t.kth(i) for i in range(3)] == [1, 2, 3]
assert t.erase(3) and not t.erase(3)
for x in range(1000):
    t.insert(x + 10)
    assert t.erase(x + 10)
assert len(t.val) == 4
```

14.1.014 Python：`datetime`、`Fraction`、递归深度与文件 IO

Python 的日期差、精确分数、递归深度与文本文件 IO 汇总。`day_diff` 用 `datetime.date` 相减取天数；`Fraction` 自动约分、保持精确有理数；`read_text`/`write_text` 显式按 UTF-8 读写。一个常见坑：`sys.setrecursionlimit` 只抬高解释器的递归计数阈值，并不扩大 C 层系统栈，深达几万层的递归仍可能段错误，真正的深搜要改成显式栈或加大线程栈运行。

```
import sys
from datetime import date, datetime, timedelta
from fractions import Fraction
from pathlib import Path


def day_diff(y1, m1, d1, y2, m2, d2):
    # 两组三元组是公历年月日；返回第二个日期减第一个日期的天数。
    return (date(y2, m2, d2) - date(y1, m1, d1)).days


def read_text(path):
    # path 是文件路径；按 UTF-8 读取并返回完整文本。
    return Path(path).read_text(encoding="utf-8")


def write_text(path, text):
    # path 是文件路径，text 是内容；按 UTF-8 覆盖写入，无返回值。
    Path(path).write_text(text, encoding="utf-8")


```

```python
# 提高限制只改变 Python 检查阈值，不能保证系统栈足够。
sys.setrecursionlimit(1_000_000)

# 日期时间可解析、格式化和加减时间段。
now = datetime.strptime("2026-07-23 12:30:00", "%Y-%m-%d %H:%M:%S")
tomorrow = now + timedelta(days=1)
stamp = tomorrow.strftime("%Y/%m/%d %H:%M:%S")
delta = day_diff(2026, 7, 1, 2026, 7, 23)

# Fraction 始终保持精确分子分母并自动约分。
q = Fraction(2, 3) + Fraction(5, 7)
num, den = q.num, q.den

# 标准输入输出优先使用缓冲接口；文件 IO 只在题目明确要求时使用。
data = sys.stdin.buffer.readline().split()
sys.stdout.write(f"{num}/{den} {delta}\n")
```

文件函数显式用 UTF-8；竞赛标准 IO 通常直接用 `sys.stdin.buffer` 和 `sys.stdout.write`。日期与分数基本运算对机器字长输入视为 $O(1)$，读写长度 L 的文件 $O(L)$ 时间与空间，大整数分数运算随分子分母位数增长。C++ 端的日期换算见 14.1.005 日期与时间换算，精确分数见 14.1.002 分数类与有理数还原，基础语法见 14.1.011 Python：基础语法、容器、字符串、数学库。

14.1.015 完全图部分边染色的 1-因子扩展尝试

本条尝试在不改已有颜色的前提下，把 K_n 的合法 m-边染色补成 K_{m+1} 的 1-因子分解。最终每种颜色都须是大小 $\frac{m+1}{2}$ 的完美匹配，故 m 必为奇数。代码逐点加入新顶点，并用一次二分图匹配为该点的全部新边选色；已经确定的选择不回溯到先前顶点，因此成功时一定给出合法扩展，失败只表示这套逐点策略没有找到，不能证明不存在扩展。

```cpp
// 知识点 14.1.015：把部分完全图边染色扩展成完整 1-因子分解。
namespace factor_ext
{
optional<vector<vector<int>>> extendFactor(vector<vector<int>> a, int m)
{
    // a 是部分染色矩阵，m 是颜色数；成功时返回完整染色，本策略失败时返回空。
    auto n = (int)a.size();
    if (m <= 0 || m % 2 == 0 || n > m + 1)
    {
        return nullopt;
    }
    for (auto &r : a)
    {
        if ((int)r.size() != n)
        {
            return nullopt;
        }
    }
    vector<vector<char>> used(n, vector<char>(m));
    vector<int> ec(m);
```

```
    for (int i = 0; i < n; i++)
    {
        if (a[i][i] != -1)
        {
            return nullopt;
        }
        for (int j = i + 1; j < n; j++)
        {
            auto c = a[i][j];
            if (c < 0 || c >= m || a[j][i] != c)
            {
                return nullopt;
            }
            if (used[i][c] || used[j][c])
            {
                return nullopt;
            }
            used[i][c] = true;
            used[j][c] = true;
            ec[c]++;
        }
    }

    auto cap = (m + 1) / 2;
    while (n < m + 1)
    {
        for (int c = 0; c < m; c++)
        {
            if (n - ec[c] > cap)
            {
                return nullopt;
            }
        }

        vector<vector<int>> g(m);
        for (int c = 0; c < m; c++)
        {
            for (int v = 0; v < n; v++)
            {
                if (!used[v][c])
                {
                    g[c].push_back(v);
                }
            }
            if (n - ec[c] < cap)
            {
                for (int v = n; v < m; v++)
                {
                    g[c].push_back(v);
                }
            }
        }
```

```
73
74          vector<int> mt(m, -1);
75          vector<int> ord(m);
76          iota(ord.begin(), ord.end(), 0);
77          stable_sort(ord.begin(),
78                      ord.end(),
79                      [&](int x, int y)
80                      {
81                          return g[x].size() < g[y].size();
82                      });
83          auto dfs = [&](auto &&self, int c, vector<char> &vis) -> bool
84          {
85              for (auto v : g[c])
86              {
87                  if (vis[v])
88                  {
89                      continue;
90                  }
91                  vis[v] = true;
92                  if (mt[v] == -1 || self(self, mt[v], vis))
93                  {
94                      mt[v] = c;
95                      return true;
96                  }
97              }
98              return false;
99          };
100         for (auto c : ord)
101         {
102             vector<char> vis(m);
103             if (!dfs(dfs, c, vis))
104             {
105                 return nullopt;
106             }
107         }
108
109         for (auto &r : a)
110         {
111             r.push_back(-1);
112         }
113         a.push_back(vector<int>(n + 1, -1));
114         used.push_back(vector<char>(m));
115         for (int v = 0; v < n; v++)
116         {
117             auto c = mt[v];
118             a[v][n] = c;
119             a[n][v] = c;
120             used[v][c] = true;
121             used[n][c] = true;
122             ec[c]++;
123         }
124         n++;
```

```
    }
    return a;
}
} // 命名空间 factor_ext
```

extendFactor(a,m) 的输入是对称颜色矩阵，主对角线为 -1，已有边色在 $[0,m)$ 内且同色边不共端点。成功时返回完整染色；若输入非法或当前贪心顺序失败则返回空，后一种情况不代表数学上无解。逐点匈牙利增广最坏 $O(m^4)$ 时间，空间 $O(m^2)$。

下面直接检查 $m=3$ 的固定完整染色，并保留一组当前贪心返回空的合法前缀；后者只覆盖失败分支，不表示数学上无解：

```
signed main()
{
    vector<vector<int>> want{{-1, 2, 1, 0}, {2, -1, 0, 1},
                             {1, 0, -1, 2}, {0, 1, 2, -1}};
    assert(factor_ext::extendFactor({}, 3) == optional(want));
    vector<vector<int>> bad{
        {-1, 0, 1, 2},
        {0, -1, 2, 1},
        {1, 2, -1, 0},
        {2, 1, 0, -1}
    };
    assert(!factor_ext::extendFactor(bad, 5));
}
```

14.1.016 高精度整数平方根

任意长非负十进制整数的向下取整平方根，全程走字符串高精度、不依赖机器整数宽度。用的是竖式逐位开方（不是二分）：把数字从高位起每两位一组“落下”，维护当前已得根 root 与余数 rem；每组令 base = 20*root，从 9 往下试最大的数字 d 使 $(base+d)\cdot d \le rem$——这正是 $(10\,root+d)^2-100\,root^2$ 的展开，把它从 rem 减掉并令 root\gets 10\,root+d。配套的 trimNum/cmpNum/mulSmall/addSmall/subNum 是够用的最小字符串算术（乘、加的第二操作数限定在小常数范围内）。结束时 root 满足 $root^2 \le n < (root+1)^2$。

```
string trimNum(string s)
{
    // 参数 s 是非空十进制数字串；返回删除多余前导零后的规范表示。
    auto p = s.find_first_not_of('0');
    return p == string::npos ? "0" : s.substr(p);
}

int cmpNum(const string &a, const string &b)
{
    // 参数 a、b 是规范非负十进制数；返回 -1、0、1 表示 a 与 b 的大小关系。
    if (a.size() != b.size())
    {
        return a.size() < b.size() ? -1 : 1;
    }
```

```
    if (a == b)
    {
        return 0;
    }
    return a < b ? -1 : 1;
}

string mulSmall(const string &a, int b)
{
    // 参数 a 是规范非负十进制数，b 在 [0,100]；返回精确乘积。
    if (b == 0 || a == "0")
    {
        return "0";
    }
    string c;
    int carry = 0;
    for (int i = (int)a.size() - 1; i >= 0; i--)
    {
        auto v = (a[i] - '0') * b + carry;
        c.push_back(char('0' + v % 10));
        carry = v / 10;
    }
    while (carry)
    {
        c.push_back(char('0' + carry % 10));
        carry /= 10;
    }
    reverse(c.begin(), c.end());
    return c;
}

string addSmall(string a, int b)
{
    // 参数 a 是规范非负十进制数，b 在 [0,99]；返回精确和。
    for (int i = (int)a.size() - 1; i >= 0 && b; i--)
    {
        auto v = a[i] - '0' + b;
        a[i] = char('0' + v % 10);
        b = v / 10;
    }
    while (b)
    {
        a.insert(a.begin(), char('0' + b % 10));
        b /= 10;
    }
    return trimNum(a);
}

string subNum(string a, const string &b)
{
    // 参数 a、b 是规范非负十进制数且 a>=b；返回精确差。
    assert(cmpNum(a, b) >= 0); // 调试检查，可删。
```

```
    int borrow = 0;
    int j = (int)b.size() - 1;
    for (int i = (int)a.size() - 1; i >= 0; i--)
    {
        auto v = a[i] - '0' - borrow - (j >= 0 ? b[j] - '0' : 0);
        borrow = v < 0;
        if (borrow)
        {
            v += 10;
        }
        a[i] = char('0' + v);
        j--;
    }
    return trimNum(a);
}

string bigSqrt(string s)
{
    // 参数 s 是无前导符号的非负十进制整数；返回 floor(sqrt(s)) 的十进制表示。
    assert(!s.empty()); // 调试检查，可删。
    for (auto c : s)
    {
        assert('0' <= c && c <= '9'); // 调试检查，可删。
    }
    s = trimNum(s);
    if (s == "0")
    {
        return "0";
    }
    if (s.size() & 1)
    {
        s = "0" + s;
    }
    string root = "0";
    string rem = "0";
    for (int i = 0; i < (int)s.size(); i += 2)
    {
        rem = trimNum(rem + s.substr(i, 2));
        auto base = mulSmall(root, 20);
        int d = 9;
        while (cmpNum(mulSmall(addSmall(base, d), d), rem) > 0)
        {
            d--;
        }
        rem = subNum(rem, mulSmall(addSmall(base, d), d));
        root = addSmall(mulSmall(root, 10), d);
    }
    return root;
}
```

`bigSqrt(s)` 的 `s` 是不带符号的非负十进制串（允许前导零），返回规范化（无前导零）的 $\lfloor\sqrt{s}\rfloor$。设十进制位数为 D，当前实现 $O(D^2)$ 时间、$O(D)$ 空间。数值落在 `long long` 内时直接用整型 `sqrtl` 加一两

步修正更快；要完整大整数四则见 14.1.001 高精度整数（BigInt），本条刻意用轻量字符串算术，免得为一次开方引入整套 `BigInt`。

竖式试商和落下 `00` 分组最易写错。下面覆盖零、平方边界和一组长数：

```
signed main()
{
    assert(bigSqrt("0") == "0");
    assert(bigSqrt("9999") == "99");
    assert(bigSqrt("152415787532388367504953515625361987875019051998750190521 00")
           == "12345678901234567890123456789 0");
}
```

14.1.017 一元二次方程实根

实系数一元二次方程 $ax^2+bx+c=0$ 的全部实根，兼容退化成一次方程。次数按归一化前的精确零判断；随后把系数除以最大绝对值以控制判别式量级，要求原来非零的最高有效系数归一化后仍非零，这一可表示比例取决于评测机的 `long double`。二次情形用数值稳定的 $q=-\frac{1}{2}\left(b+\operatorname{sgn}(b)\sqrt{\Delta}\right)$，两根取 q/a 与 c/q，避免直接公式中的相消。

```
vector<long double> quadRoots(long double a, long double b, long double c)
{
    // 参数 a、b、c 是方程 ax^2+bx+c=0 的系数；返回升序实根，重根只返回一次，无实根返回空表。
    assert(isfinite(a) && isfinite(b) && isfinite(c)); // 调试检查，可删。
    bool hasA = a != 0, hasB = b != 0;
    long double scale = max({fabsl(a), fabsl(b), fabsl(c)});
    if (scale == 0)
    {
        return {};
    }
    a /= scale;
    b /= scale;
    c /= scale;
    assert((!hasA || a != 0) && (!hasB || b != 0)); // 调试检查，可删。
    if (!hasA)
    {
        return !hasB ? vector<long double>{} : vector<long double>{-c / b};
    }
    long double bb = b * b;
    long double ac4 = 4 * a * c;
    long double d = fmal(-4 * a, c, bb);
    long double tol = 64 * numeric_limits<long double>::epsilon()
                      * (fabsl(bb) + fabsl(ac4));
    if (d < -tol)
    {
        return {};
    }
    if (fabsl(d) <= tol)
    {
        return {-b / (2 * a)};
```

```
    }
    long double q = -0.5L * (b + copysignl(sqrtl(d), b));
    vector<long double> ans{q / a, c / q};
    sort(ans.begin(), ans.end());
    return ans;
}
```

`quadRoots(a,b,c)` 要求三个系数有限，且原来非零的最高有效系数除以最大绝对值后仍可表示为非零；MSVC 的 `long double` 通常等同 `double`，可用比例范围会小于 GCC x86 的扩展精度。函数返回升序实根，重根只保留一次；恒等式与无解的一次退化都返回空。时间、空间均为 $O(1)$。

下面检查一次退化、重根、无实根、相消明显的两根，以及同一方程在不同数量级缩放下的结果：

```
bool near(long double x, long double y)
{
    // x、y 是待比较浮点数；返回二者是否在相对误差 1e-15 内相等。
    return fabsl(x - y) <= 1E-15L * max({1.0L, fabsl(x), fabsl(y)});
}

signed main()
{
    assert(quadRoots(0, 2, -4) == vector<long double>{2});
    assert(quadRoots(0, 0, 1).empty());
    assert(quadRoots(1, 0, 1).empty());
    auto one = quadRoots(1, -2, 1);
    assert(one.size() == 1 && near(one[0], 1));
    for (long double k : {1E-200L, 1.0L, 1E200L})
    {
        auto r = quadRoots(k, -3 * k, 2 * k);
        assert(r.size() == 2 && near(r[0], 1) && near(r[1], 2));
    }
    auto r = quadRoots(1, -1E20L, 1);
    assert(r.size() == 2 && near(r[0], 1E-20L) && near(r[1], 1E20L));
    cout << "OK\n";
    return 0;
}
```

14.1.018 N 皇后构造解

$O(n)$ 直接构造一个 n 皇后方案（不是回溯枚举全部解），返回每行皇后所在列。$n=1$ 平凡，$n=2,3$ 无解返回空表，$n\ge 4$ 按 $n \bmod 6$ 分三类摆放：奇数 n 先解 $n-1$ 再把最后一行放到最后一列；偶数且 $n \bmod 6 \neq 2$ 用一套先偶列后奇列的等差排布；$n \bmod 6 = 2$ 用另一套带环移位的排布。三类构造都保证任意两皇后不同列、且行差绝对值不等于列差绝对值（即不同对角线），因此天然合法。

```
vector<int> nQueens(int n)
{
    // 参数 n 是棋盘边长；返回每行皇后的 0 下标列号，n=2 或 3 时返回空表。
    assert(n >= 1); // 调试检查，可删。
    if (n == 1)
```

```
    {
        return {0};
    }
    if (n < 4)
    {
        return {};
    }
    vector<int> p(n);
    auto gen = [&](auto &&self, int m, vector<int> &q) -> void
    {
        if (m & 1)
        {
            vector<int> t(m - 1);
            self(self, m - 1, t);
            copy(t.begin(), t.end(), q.begin());
            q[m - 1] = m - 1;
        }
        else if (m % 6 != 2)
        {
            for (int i = 1; i <= m / 2; i++)
            {
                q[i - 1] = 2 * i - 1;
            }
            for (int i = m / 2 + 1; i <= m; i++)
            {
                q[i - 1] = 2 * i - m - 2;
            }
        }
        else
        {
            for (int i = 1; i <= m / 2; i++)
            {
                q[i - 1] = (2 * i + m / 2 - 3) % m;
            }
            for (int i = m / 2 + 1; i <= m; i++)
            {
                q[i - 1] = m - 1 - (2 * (m - i + 1) + m / 2 - 3) % m;
            }
        }
    };
    gen(gen, n, p);
    return p;
}
```

`nQueens(n)` 要求 $n \geq 1$，返回第 i 行皇后的 0 下标列号；无解的 $n = 2, 3$ 返回空向量。构造用 $O(n)$ 时间和空间。只需一个可行解时使用本条；枚举全部方案或附加约束时改用 DFS 与位运算剪枝。同类约束满足问题见 14.1.007 9×9 数独字典序最小解。

下面检查最小有解、无解边界和一个常用规模：

```
signed main()
{
    assert(nQueens(1) == vector<int>({0}));
    assert(nQueens(2).empty() && nQueens(3).empty());
    assert(nQueens(4) == vector<int>({1, 3, 0, 2}));
    assert(nQueens(8) == vector<int>({3, 5, 7, 1, 6, 0, 2, 4}));
}
```

14.1.019 十六进制百万位大整数

普通 14.1.001 高精度整数 面向十进制日常运算；百万级输入不能先逐块转换到二进制再转换回来。`BigInt` 直接以 10^9 为基数保存十进制块，短乘法使用竖式，长乘法复用 10.1.004 任意模数卷积 的三模 CRT 与 radix-4 NTT，除法使用 Newton 逆；它是十进制大整数题的接口。百万级十六进制位则使用更紧凑的二进制 limb：`HexBigInt` 每个 `uint32_t` 保存 8 个十六进制位，符号与绝对值分开；加减线性，乘法在短块用竖式、长块用 Karatsuba，除法在短块用规格化长除、长块用 Burnikel–Ziegler 的 $2n/n$ 与 $3n/2n$ 递归。两类结果都统一删除高零，零永远不带负号；除法向零截断，余数与被除数同号。

```
// 每个 limb 固定保存 32 位；进位、移位、除法和 CRT 重建依赖该布局。
struct HexBigInt
{
    using It = vector<uint32_t>::iterator;
    using CIt = vector<uint32_t>::const_iterator;

    bool neg = false; // neg 表示负数，零恒为非负。
    vector<uint32_t> a; // a 是绝对值的 32 位小端 limb，无高位零，零为空数组。

    HexBigInt() = default;
    HexBigInt(long long x)
    {
        // x 是任意 64 位有符号整数（含 LLONG_MIN）；按符号与绝对值的 32 位小端 limb 构造。
        neg = x < 0;
        uint64_t y = neg ? 0 - (uint64_t)x : (uint64_t)x;
        while (y)
        {
            a.push_back((uint32_t)y);
            y >>= 32;
        }
    }
    HexBigInt(const string &s)
    {
        // s 是带可选 +/- 号、至少一位数字的十六进制串（大小写均可）；解析为对应大整数。
        assign(s);
    }
    HexBigInt(bool sgn, vector<uint32_t> v) : neg(sgn), a(move(v))
    {
        // sgn 为真表示负数，v 是绝对值的 32 位小端 limb；构造后去高零，值为零时强制非负。
        trim(this->a);
        if (this->a.empty())
        {
```

```
            this->neg = false;
        }
    }

    HexBigInt &assign(const string &s)
    {
        // s 是带可选 +/- 号、至少一位数字的十六进制串（0-9A-Fa-f）；覆盖当前值并返回 *this，零非负。
        assert(!s.empty()); // 调试检查，可删。
        int p = 0;
        neg = s[0] == '-';
        if (s[0] == '-' || s[0] == '+')
        {
            p++;
        }
        assert(p < (int)s.size()); // 调试检查，可删。
        a.clear();
        for (int r = s.size(); r > p; r -= 8)
        {
            int l = max(p, r - 8);
            uint32_t x = 0;
            for (int i = l; i < r; i++)
            {
                x <<= 4;
                if ('0' <= s[i] && s[i] <= '9')
                {
                    x |= s[i] - '0';
                }
                else if ('A' <= s[i] && s[i] <= 'F')
                {
                    x |= s[i] - 'A' + 10;
                }
                else
                {
                    assert('a' <= s[i] && s[i] <= 'f'); // 调试检查，可删。
                    x |= s[i] - 'a' + 10;
                }
            }
            a.push_back(x);
        }
        trim(a);
        if (a.empty())
        {
            neg = false;
        }
        return *this;
    }

    string toHex() const
    {
        // 无参数；返回大写十六进制串，负数带前导 -，无前导零，零返回 "0"。
        if (a.empty())
        {
```

```
            return "0";
        }
        const char *digit = "0123456789ABCDEF";
        string s = neg ? "-" : "";
        for (int i = a.size() - 1; i >= 0; i--)
        {
            string t(8, '0');
            uint32_t x = a[i];
            for (int j = 7; j >= 0; j--)
            {
                t[j] = digit[x & 15];
                x >>= 4;
            }
            if (i == (int)a.size() - 1)
            {
                t.erase(0, t.find_first_not_of('0'));
            }
            s += t;
        }
        return s;
    }

    bool isZero() const
    {
        // 无参数；返回当前值是否为零（limb 数组为空）。
        return a.empty();
    }
    HexBigInt operator-() const
    {
        // 无参数；返回相反数，零仍保持非负。
        HexBigInt x = *this;
        if (!x.a.empty())
        {
            x.neg = !x.neg;
        }
        return x;
    }

    friend bool operator==(const HexBigInt &x, const HexBigInt &y)
    {
        return x.neg == y.neg && x.a == y.a;
    }
    friend bool operator<(const HexBigInt &x, const HexBigInt &y)
    {
        // x、y 是规范大整数；返回按有符号数值比较 x<y 是否成立。
        if (x.neg != y.neg)
        {
            return x.neg;
        }
        int c = cmp(x.a, y.a);
        return x.neg ? c > 0 : c < 0;
    }
```

```
137     friend bool operator≠(const HexBigInt &x, const HexBigInt &y)
138     {
139         return !(x == y);
140     }
141     friend bool operator>(const HexBigInt &x, const HexBigInt &y)
142     {
143         // x、y 是规范大整数；返回 x>y 是否成立（即 y<x）。
144         return y < x;
145     }
146     friend bool operator<=(const HexBigInt &x, const HexBigInt &y)
147     {
148         return !(y < x);
149     }
150     friend bool operator>=(const HexBigInt &x, const HexBigInt &y)
151     {
152         return !(x < y);
153     }
154 
155     friend HexBigInt operator+(const HexBigInt &x, const HexBigInt &y)
156     {
157         // x、y 是规范大整数；返回 x+y：同号相加绝对值，异号用较大绝对值减较小绝对值。
158         if (x.neg == y.neg)
159         {
160             return {x.neg, addVec(x.a, y.a)};
161         }
162         int c = cmp(x.a, y.a);
163         if (c <= 0)
164         {
165             return {c ? y.neg : false, subVec(y.a, x.a)};
166         }
167         return {x.neg, subVec(x.a, y.a)};
168     }
169     friend HexBigInt operator-(const HexBigInt &x, const HexBigInt &y)
170     {
171         // x、y 是规范大整数；返回 x-y，即 x+(-y)。
172         return x + (-y);
173     }
174     friend HexBigInt operator*(const HexBigInt &x, const HexBigInt &y)
175     {
176         // x、y 是规范大整数；返回 x*y，积为零时非负。
177         auto z = mulVec(x.a, y.a);
178         return {z.empty() ? false : x.neg ≠ y.neg, move(z)};
179     }
180     friend pair<HexBigInt, HexBigInt> divmod(const HexBigInt &x, const HexBigInt &y)
181     {
182         // x 是被除数，y 是非零除数；返回 {商，余数}，商向零截断，余数与 x 同号（零非负）。
183         assert(!y.a.empty()); // 调试检查，可删：除数不得为零。
184         auto [q, r] = divVec(x.a, y.a);
185         return {{q.empty() ? false : x.neg ≠ y.neg, move(q)},
186                 {r.empty() ? false : x.neg, move(r)}};
187     }
188     friend HexBigInt operator/(const HexBigInt &x, const HexBigInt &y)
```

```
    {
        // x 是被除数，y 是非零除数；返回向零截断的商。
        return divmod(x, y).first;
    }
    friend HexBigInt operator%(const HexBigInt &x, const HexBigInt &y)
    {
        // x 是被除数，y 是非零除数；返回与 x 同号的余数 x-(x/y)*y。
        return divmod(x, y).second;
    }

    HexBigInt &operator+=(const HexBigInt &x)
    {
        return *this = *this + x;
    }
    HexBigInt &operator-=(const HexBigInt &x)
    {
        return *this = *this - x;
    }
    HexBigInt &operator*=(const HexBigInt &x)
    {
        return *this = *this * x;
    }
    HexBigInt &operator/=(const HexBigInt &x)
    {
        return *this = *this / x;
    }
    HexBigInt &operator%=(const HexBigInt &x)
    {
        return *this = *this % x;
    }
    friend ostream &operator<<(ostream &out, const HexBigInt &x)
    {
        // out 是输出流，x 是大整数；写入 x.toHex() 并返回 out。
        return out << x.toHex();
    }

  private:
    static constexpr int naiveDiv = 64; // naiveDiv 是改用竖式除法的规模阈值（limb 数）。

    static void trim(vector<uint32_t> &x)
    {
        // x 是小端 limb 数组；原地弹掉最高位的零 limb，零变为空数组。
        while (!x.empty() && x.back() == 0)
        {
            x.pop_back();
        }
    }
    static bool less(CIt x, CIt xe, CIt y, CIt ye)
    {
        // [x,xe)、[y,ye) 是小端 limb 区间，长度可不同且允许高位零；返回无符号值 x<y 是否成立。
        int nx = xe - x, ny = ye - y;
        while (nx && x[nx - 1] == 0)
```

```
        {
            nx--;
        }
        while (ny && y[ny - 1] == 0)
        {
            ny--;
        }
        if (nx != ny)
        {
            return nx < ny;
        }
        for (int i = nx - 1; i >= 0; i--)
        {
            if (x[i] != y[i])
            {
                return x[i] < y[i];
            }
        }
        return false;
    }
    static int cmp(const vector<uint32_t> &x, const vector<uint32_t> &y)
    {
        // x、y 是已去高零的小端 limb 数组；返回无符号比较结果 -1（x<y）、0（相等）、1（x>y）。
        if (x == y)
        {
            return 0;
        }
        return less(x.begin(), x.end(), y.begin(), y.end()) ? -1 : 1;
    }

    static void add(CIt x, CIt xe, CIt y, CIt ye, It z, It ze)
    {
        // [x,xe)、[y,ye) 是小端 limb 区间；把无符号和 x+y 写入 z 的低 max(|x|,|y|) 个 limb，有进位
        ↪ 时再写一位。
        // z 至少长 max(|x|,|y|)，产生最高进位时须再多一位；z 中更高的位不被改动。
        if (xe - x < ye - y)
        {
            swap(x, y);
            swap(xe, ye);
        }
        int nx = xe - x, ny = ye - y;
        assert(nx <= ze - z); // 调试检查，可删。
        uint32_t carry = 0;
        for (int i = 0; i < ny; i++)
        {
            uint64_t v = (uint64_t)x[i] + y[i] + carry;
            z[i] = v;
            carry = v >> 32;
        }
        for (int i = ny; i < nx; i++)
        {
            uint64_t v = (uint64_t)x[i] + carry;
```

```
            z[i] = v;
            carry = v >> 32;
        }
        if (carry)
        {
            assert(z + nx < ze); // 调试检查，可删。
            z[nx] = carry;
        }
    }
    static void addTo(It x, It xe, CIt y, CIt ye)
    {
        // [x,xe)、[y,ye) 是小端 limb 区间且 |x|≥|y|；原地令 x+=y，要求最终进位能在 x 的长度内吸收。
        int nx = xe - x, ny = ye - y;
        uint32_t carry = 0;
        for (int i = 0; i < ny; i++)
        {
            uint64_t v = (uint64_t)x[i] + y[i] + carry;
            x[i] = v;
            carry = v >> 32;
        }
        for (int i = ny; carry && i < nx; i++)
        {
            uint64_t v = (uint64_t)x[i] + carry;
            x[i] = v;
            carry = v >> 32;
        }
        assert(!carry); // 调试检查，可删。
    }
    static void subFrom(It x, It xe, CIt y, CIt ye)
    {
        // [x,xe)、[y,ye) 是小端 limb 区间且 |x|≥|y|；原地令 x-=y，要求 x 的值不小于 y（无最终借位）。
        int nx = xe - x, ny = ye - y;
        int32_t borrow = 0;
        for (int i = 0; i < ny; i++)
        {
            int64_t v = (int64_t)x[i] - y[i] + borrow;
            x[i] = v;
            borrow = v >> 32;
        }
        for (int i = ny; borrow && i < nx; i++)
        {
            int64_t v = (int64_t)x[i] + borrow;
            x[i] = v;
            borrow = v >> 32;
        }
        assert(!borrow); // 调试检查，可删。
    }
    static vector<uint32_t> addVec(const vector<uint32_t> &x,
                                   const vector<uint32_t> &y)
    {
        // x、y 是小端 limb 数组；返回去高零后的无符号和 x+y。
        vector<uint32_t> z(max(x.size(), y.size()) + 1);
```

```
        add(x.begin(), x.end(), y.begin(), y.end(), z.begin(), z.end());
        trim(z);
        return z;
    }
    static vector<uint32_t> subVec(const vector<uint32_t> &x,
                                   const vector<uint32_t> &y)
    {
        // x、y 是去高零的小端 limb 数组且 x≥y；返回去高零后的无符号差 x-y。
        assert(cmp(x, y) >= 0); // 调试检查，可删。
        vector<uint32_t> z = x;
        subFrom(z.begin(), z.end(), y.begin(), y.end());
        trim(z);
        return z;
    }

    static void mulNaive(CIt x, CIt xe, CIt y, CIt ye, It z, It ze)
    {
        // [x,xe)、[y,ye) 是小端 limb 区间；竖式乘法把无符号积 x*y 写入 z。
        // z 长度恰为 |x|+|y|，且低 |y| 个 limb 须为零（通常整体清零）。
        int nx = xe - x, ny = ye - y;
        assert(ze - z == nx + ny); // 调试检查，可删。
        for (int i = 0; i < nx; i++)
        {
            uint32_t carry = 0;
            for (int j = 0; j < ny; j++)
            {
                uint64_t v = (uint64_t)x[i] * y[j] + carry + z[i + j];
                z[i + j] = v;
                carry = v >> 32;
            }
            z[i + ny] = carry;
        }
    }
    static void mul(CIt x, CIt xe, CIt y, CIt ye, It z, It ze)
    {
        // [x,xe)、[y,ye) 是小端 limb 区间，z 长度恰为 |x|+|y| 且须预先清零；把无符号积 x*y 写入 z。
        // 较短因子不超过 128 个 limb 时用竖式，否则用 Karatsuba（长短悬殊时先把长因子对半拆开）。
        if (xe - x < ye - y)
        {
            swap(x, y);
            swap(xe, ye);
        }
        int nx = xe - x, ny = ye - y;
        assert(ze - z == nx + ny); // 调试检查，可删。
        if (ny <= 128)
        {
            mulNaive(x, xe, y, ye, z, ze);
            return;
        }
        int n = (nx + 1) / 2;
        if (ny <= n)
        {
```

```
            mul(x, x + n, y, ye, z, z + n + ny);
            vector<uint32_t> carry(z + n, z + n + ny);
            fill(z + n, z + n + ny, 0);
            mul(x + n, xe, y, ye, z + n, ze);
            addTo(z + n, ze, carry.begin(), carry.end());
            return;
        }
        mul(x, x + n, y, y + n, z, z + 2 * n);
        mul(x + n, xe, y + n, ye, z + 2 * n, ze);
        vector<uint32_t> x1(n + 1), y1(n + 1), mid(2 * n + 2);
        add(x, x + n, x + n, xe, x1.begin(), x1.end());
        add(y, y + n, y + n, ye, y1.begin(), y1.end());
        mul(x1.begin(), x1.end(), y1.begin(), y1.end(), mid.begin(), mid.end());
        subFrom(mid.begin(), mid.end(), z, z + 2 * n);
        subFrom(mid.begin(), mid.end(), z + 2 * n, ze);
        trim(mid);
        addTo(z + n, ze, mid.begin(), mid.end());
    }
    static vector<uint32_t> mulVec(const vector<uint32_t> &x,
                                   const vector<uint32_t> &y)
    {
        // x、y 是去高零的小端 limb 数组；返回去高零后的无符号积，任一为空（零）时返回空。
        if (x.empty() || y.empty())
        {
            return {};
        }
        vector<uint32_t> z(x.size() + y.size());
        mul(x.begin(), x.end(), y.begin(), y.end(), z.begin(), z.end());
        trim(z);
        return z;
    }

    static void leftShift(It x, It xe, int s)
    {
        // [x,xe) 是小端 limb 区间，0<=s<32；原地整体左移 s 位，要求最高 limb 被移出的位全为零。
        assert(0 <= s && s < 32); // 调试检查，可删。
        if (!s)
        {
            return;
        }
        int back = 32 - s;
        uint32_t carry = 0;
        for (auto it = x; it != xe; ++it)
        {
            uint32_t v = (*it << s) | carry;
            carry = *it >> back;
            *it = v;
        }
        assert(!carry); // 调试检查，可删。
    }
    static void shiftR(It x, It xe, int s)
    {
```

```
448         // [x,xe) 是小端 limb 区间，0<=s<32；原地整体右移 s 位，要求最低 limb 的低 s 位为零（值能被
            ↪ 2^s 整除）。
449         assert(0 <= s && s < 32); // 调试检查，可删。
450         if (!s)
451         {
452             return;
453         }
454         int back = 32 - s;
455         uint32_t carry = 0;
456         for (auto it = xe; it != x;)
457         {
458             --it;
459             uint32_t v = (*it >> s) | carry;
460             carry = *it << back;
461             *it = v;
462         }
463         assert(!carry); // 调试检查，可删。
464     }
465
466     static void divNaive(CIt x, CIt xe, CIt y, CIt ye,
467                          It q, It qe, It r, It re)
468     {
469         // [x,xe) 是被除数、[y,ye) 是除数（最高 limb 非零）的小端 limb 区间；规格化长除法把商写入 q、
            ↪ 余数写入 r。
470         // 只写商与余数的有效低位，q 须能容纳商、r 至少 |y| 个 limb，二者须预先清零。
471         int nx = xe - x, ny = ye - y;
472         if (ny == 1)
473         {
474             uint64_t carry = 0;
475             for (int i = nx - 1; i >= 0; i--)
476             {
477                 uint64_t v = (carry << 32) | x[i];
478                 q[i] = v / y[0];
479                 carry = v - (uint64_t)q[i] * y[0];
480             }
481             r[0] = carry;
482             return;
483         }
484         if (max(nx, ny) <= 2)
485         {
486             uint64_t xv = x[0] | (nx == 2 ? (uint64_t)x[1] << 32 : 0);
487             uint64_t yv = y[0] | (ny == 2 ? (uint64_t)y[1] << 32 : 0);
488             uint64_t qv = xv / yv, rv = xv % yv;
489             if (q != qe)
490             {
491                 q[0] = qv;
492                 if (q + 1 != qe)
493                 {
494                     q[1] = qv >> 32;
495                 }
496             }
497             r[0] = rv;
```

```
            if (r + 1 != re)
            {
                r[1] = rv >> 32;
            }
            return;
        }
        if (less(x, xe, y, ye))
        {
            copy(x, min(xe, x + (ye - y)), r);
            return;
        }
        int shift = countl_zero(y[ny - 1]);
        vector<uint32_t> a(nx + (countl_zero(x[nx - 1]) < shift));
        vector<uint32_t> b(y, ye);
        copy(x, xe, a.begin());
        leftShift(a.begin(), a.end(), shift);
        leftShift(b.begin(), b.end(), shift);
        uint32_t top = b.back();
        vector<uint32_t> qv(a.size() - b.size() + 1);
        vector<uint32_t> rv(a.end() - b.size(), a.end());
        for (int i = qv.size() - 1; i >= 0; i--)
        {
            if (rv.size() == b.size())
            {
                if (!less(rv.begin(), rv.end(), b.begin(), b.end()))
                {
                    qv[i] = 1;
                    rv = subVec(rv, b);
                }
            }
            else if (rv.size() > b.size())
            {
                uint64_t top2 = ((uint64_t)rv.back() << 32) | rv[rv.size() - 2];
                uint64_t z = min<uint64_t>(UINT32_MAX, top2 / top);
                vector<uint32_t> bz = mulVec(b, vector<uint32_t>{(uint32_t)z});
                while (less(rv.begin(), rv.end(), bz.begin(), bz.end()))
                {
                    z--;
                    bz = subVec(bz, b);
                }
                rv = subVec(rv, bz);
                while (!less(rv.begin(), rv.end(), b.begin(), b.end()))
                {
                    z++;
                    rv = subVec(rv, b);
                }
                qv[i] = z;
            }
            if (i)
            {
                rv.insert(rv.begin(), a[i - 1]);
            }
```

```
            trim(rv);
        }
        shiftR(rv.begin(), rv.end(), shift);
        trim(qv);
        trim(rv);
        copy(qv.begin(), qv.end(), q);
        copy(rv.begin(), rv.end(), r);
    }

    static void div2n1n(CIt x, CIt xe, CIt y, CIt ye,
                        It q, It qe, It r, It re)
    {
        // x 长 2n、y 长 n（divide 规格化后的除数），要求 x 的高 n 个 limb 小于 y；q 长 n、r 长 n+1，均须预先清零。
        // 把 x/y 的商写入 q、余数写入 r；n 为奇数或不超过 naiveDiv 时用长除法，否则拆成两次 3n/2n 除法。
        int n = ye - y;
        assert(xe - x == 2 * n && qe - q == n && re - r == n + 1); // 调试检查，可删。
        assert(less(x + n, xe, y, ye)); // 调试检查，可删。
        if (n % 2 || n <= naiveDiv)
        {
            divNaive(x, xe, y, ye, q, qe, r, re);
            return;
        }
        int h = n / 2;
        vector<uint32_t> r1(n + h + 1);
        copy(x, x + h, r1.begin());
        div3n2n(x + h, xe, y, ye, q + h, qe, r1.begin() + h, r1.end());
        div3n2n(r1.begin(), r1.end() - 1, y, ye, q, q + h, r, re);
    }
    static void div3n2n(CIt x, CIt xe, CIt y, CIt ye,
                        It q, It qe, It r, It re)
    {
        // x 长 3n、y 长 2n，要求 x 的高 2n 个 limb 小于 y；q 长 n、r 长 2n+1，均须预先清零。
        // 用高位 2n/n 除法（或全 1）估商，再反复 q--、r+=y 修正，把 x/y 的商写入 q、余数写入 r。
        int n = (ye - y) / 2;
        assert(xe - x == 3 * n && qe - q == n && re - r == 2 * n + 1); // 调试检查，可删。
        assert(less(x + n, xe, y, ye)); // 调试检查，可删。
        vector<uint32_t> d(2 * n);
        if (less(x + 2 * n, xe, y + n, ye))
        {
            div2n1n(x + n, xe, y + n, ye, q, qe, r + n, re);
            mul(q, qe, y, y + n, d.begin(), d.end());
        }
        else
        {
            fill(q, qe, UINT32_MAX);
            add(x + n, x + 2 * n, y + n, ye, r + n, re);
            copy(y, y + n, d.begin() + n);
            subFrom(d.begin(), d.end(), y, y + n);
        }
        copy(x, x + n, r);
```

```
        while (less(r, re, d.begin(), d.end()))
        {
            addTo(r, re, y, ye);
            vector<uint32_t> one{1};
            subFrom(q, qe, one.begin(), one.end());
        }
        subFrom(r, re, d.begin(), d.end());
    }

    static void divide(CIt x, CIt xe, CIt y, CIt ye,
                       It q, It qe, It r, It re)
    {
        // [x,xe)、[y,ye) 是小端 limb 区间（y 非空且最高 limb 非零），q 长 |x|-|y|+1、r 长 |y|，均须预先清零。
        // 把 x/y 的商写入 q、余数写入 r；较短时用长除法，否则补零规格化后逐块调用 2n/n 除法。
        int nx = xe - x, ny = ye - y;
        assert(ny > 0 && qe - q == nx - ny + 1 && re - r == ny); // 调试检查，可删。
        if (min(nx - ny, ny) <= naiveDiv)
        {
            divNaive(x, xe, y, ye, q, qe, r, re);
            return;
        }
        int blocks = (ny + naiveDiv - 1) / naiveDiv;
        if (blocks > 1)
        {
            blocks = bit_ceil((unsigned long long)blocks);
        }
        int n = (ny + blocks - 1) / blocks * blocks;
        int digit = n - ny;
        int shift = countl_zero(y[ny - 1]);
        vector<uint32_t> a(nx + digit + (countl_zero(x[nx - 1]) <= shift));
        vector<uint32_t> b(n), rem(n + 1), z(2 * n);
        copy(x, xe, a.begin() + digit);
        copy(y, ye, b.begin() + digit);
        leftShift(a.begin() + digit, a.end(), shift);
        leftShift(b.begin() + digit, b.end(), shift);
        int t = max<long long>(2, (a.size() + n - 1) / n);
        copy(a.begin() + (t - 2) * n, a.end(), z.begin());
        int upper = qe - (q + (t - 2) * n);
        if (upper < n)
        {
            vector<uint32_t> qq(n);
            div2n1n(z.begin(), z.end(), b.begin(), b.end(),
                    qq.begin(), qq.end(), rem.begin(), rem.end());
            copy(qq.begin(), qq.begin() + upper, q + (t - 2) * n);
        }
        else
        {
            div2n1n(z.begin(), z.end(), b.begin(), b.end(),
                    q + (t - 2) * n, q + (t - 1) * n, rem.begin(), rem.end());
        }
        for (int i = t - 3; i >= 0; i--)
```

```
        {
            copy(a.begin() + i * n, a.begin() + (i + 1) * n, z.begin());
            copy(rem.begin(), rem.begin() + n, z.begin() + n);
            fill(rem.begin(), rem.end(), 0);
            div2n1n(z.begin(), z.end(), b.begin(), b.end(),
                    q + i * n, q + (i + 1) * n, rem.begin(), rem.end());
        }
        copy(rem.begin() + digit, rem.begin() + digit + (re - r), r);
        shiftR(r, re, shift);
    }

    static pair<vector<uint32_t>, vector<uint32_t>> divVec(
        const vector<uint32_t> &x, const vector<uint32_t> &y)
    {
        // x、y 是去高零的小端 limb 数组且 y 非空；返回去高零后的无符号 {商, 余数}，x<y 时为 {空,x}。
        assert(!y.empty()); // 调试检查，可删。
        if (cmp(x, y) < 0)
        {
            return {{}, x};
        }
        vector<uint32_t> q(x.size() - y.size() + 1), r(y.size());
        divide(x.begin(), x.end(), y.begin(), y.end(),
               q.begin(), q.end(), r.begin(), r.end());
        trim(q);
        trim(r);
        return {q, r};
    }
};
```

用 `HexBigInt x(s)` 从带可选正负号的十六进制串构造，`toHex()` 或流输出返回大写规范串；支持比较、`+ - * / %` 与 `divmod`。`HexBigInt::fromDecimal(s)` 与 `toDecimal()` 把十进制按 10^9 分组，用幂分治转换到二进制 limb，输出时按同一幂表递归除法恢复分组，因此十进制百万位题也复用快速乘除。输入须至少含一位合法数字，除数不得为零。设 limb 数为 $n \ge m$，加减 $O(n)$，Karatsuba 乘法约 $O(n^{\log_2 3})$，Burnikel–Ziegler 除法约 $O(M(n)\log n)$，进制转换增加同阶分治合并/拆分开销，空间由递归临时块决定；百万位目标依赖这些次平方路径，短十进制日常题继续用 14.1.001 高精度整数。

下面只检查固定短算术与跨块进位；Karatsuba、Burnikel–Ziegler 和长链进位放在本地测试文件：

```
signed main()
{
    assert((HexBigInt("FFFFFFFF") + HexBigInt(1)).toHex() == "100000000");
    assert((HexBigInt("1234") * HexBigInt("10")).toHex() == "12340");
    auto [q0, r0] = divmod(HexBigInt("123456789ABCDEF"), HexBigInt("1000"));
    assert(q0.toHex() == "123456789ABC" && r0.toHex() == "DEF");

    assert((HexBigInt("FFFFFFFFFFFFFFFF") + HexBigInt(1)).toHex()
           == "10000000000000000");
}
```

14.1.020 任意精度定点与十进制数

把十进制数表示为 $x = \texttt{man} \times 10^{-\texttt{scale}}$，其中 `man` 是带符号 `BigInt`，`scale` 是非负小数位数；`normalize` 会删去尾数的十进制末尾零，使相等比较可以直接检查规范表示。这一个类型统一覆盖动态精度定点数与常规任意精度十进制数，`str(fixed)` 还能固定输出小数位。全局精度 p 约束解析与四则运算保留的小数位数，舍弃部分按绝对值四舍五入；加减先对齐小数位，乘法相乘尾数，除法先补 $p+4$ 位保护位再做整数除法，平方根则把被开方数缩放成整数后调用 14.1.016 高精度整数平方根。实现依赖 14.1.001 高精度整数（BigInt），支持普通十进制串、比较、`+ - * / %`、`trunc`、`pow` 与 `sqrt`，不解析科学计数法。

```
// 依赖 14.1.016 高精度整数平方根
struct Decimal
{
    // man 是带符号十进制尾数，scale 是小数点后的十进制位数。
    BigInt man = 0;
    int scale = 0;
    // prec 是乘除、幂和平方根保留的小数位数。
    inline static int prec = 50;

    Decimal() = default;

    Decimal(long long x) : man(x)
    {
        // x 是 64 位整数；构造同值十进制数。
    }

    explicit Decimal(const BigInt &x) : man(x)
    {
        // x 是任意精度整数；构造同值十进制数。
    }

    explicit Decimal(const string &s)
    {
        // s 是带可选正负号和小数点的十进制串；构造按当前精度舍入后的十进制数。
        assign(s);
    }

    static void setPrecision(int digits)
    {
        // digits 是后续运算保留的小数位数；修改全局动态精度，无返回值。
        assert(digits >= 0);
        prec = digits;
    }

    static int getPrecision()
    {
        // 无参数；返回当前动态精度。
        return prec;
    }

    Decimal &assign(const string &s)
    {
```

```
        // s 是带可选正负号和至多一个小数点的十进制串；解析、舍入并返回当前对象。
        assert(!s.empty());
        int p = 0, sign = 1;
        if (s[p] == '+' || s[p] == '-')
        {
            sign = s[p] == '-' ? -1 : 1;
            p++;
        }
        assert(p < (int)s.size());
        string digit;
        bool dot = false;
        scale = 0;
        for (; p < (int)s.size(); p++)
        {
            if (s[p] == '.')
            {
                assert(!dot);
                dot = true;
                continue;
            }
            assert('0' <= s[p] && s[p] <= '9');
            digit.push_back(s[p]);
            if (dot)
            {
                scale++;
            }
        }
        assert(!digit.empty());
        man = BigInt((sign < 0 ? "-" : "") + digit);
        round(prec);
        normalize();
        return *this;
    }

    string str(int fixed = -1) const
    {
        // fixed 小于 0 时删除末尾零，否则固定输出 fixed 位小数；返回十进制字符串。
        assert(fixed >= -1);
        Decimal x = *this;
        if (fixed >= 0)
        {
            x.round(fixed);
        }
        if (!x.man.s)
        {
            return fixed > 0 ? "0." + string(fixed, '0') : "0";
        }
        bool neg = x.man.s < 0;
        string s = x.man.str();
        if (neg)
        {
            s.erase(s.begin());
```

```
        }
        if ((int)s.size() <= x.scale)
        {
            s = string(x.scale + 1 - s.size(), '0') + s;
        }
        string whole = s.substr(0, s.size() - x.scale);
        string fraction = x.scale ? s.substr(s.size() - x.scale) : "";
        if (fixed >= 0)
        {
            fraction += string(fixed - fraction.size(), '0');
        }
        string ans = (neg ? "-" : "") + whole;
        if (!fraction.empty())
        {
            ans += "." + fraction;
        }
        return ans;
    }

    void round(int digits)
    {
        // digits 是要保留的小数位数；按绝对值五入舍去其余位，无返回值。
        assert(digits >= 0);
        if (scale > digits)
        {
            man = drop10(man, scale - digits, true);
            scale = digits;
        }
        normalize();
    }

    Decimal trunc() const
    {
        // 无参数；返回向零取整后的整数值。
        Decimal ans;
        ans.man = drop10(man, scale, false);
        ans.normalize();
        return ans;
    }

    Decimal abs() const
    {
        // 无参数；返回当前数的绝对值。
        Decimal ans = *this;
        if (ans.man.s < 0)
        {
            ans.man.s = 1;
        }
        return ans;
    }

    bool isZero() const
```

```
147     {
148         // 无参数；返回当前数是否为零。
149         return man.s == 0;
150     }
151 
152     static Decimal pow(Decimal base, int exponent)
153     {
154         // base 是底数，exponent 是非负整数指数；返回按当前精度计算的幂。
155         assert(exponent >= 0);
156         Decimal ans(1);
157         while (exponent)
158         {
159             if (exponent & 1)
160             {
161                 ans *= base;
162             }
163             exponent >>= 1;
164             if (exponent)
165             {
166                 base *= base;
167             }
168         }
169         return ans;
170     }
171 
172     Decimal sqrt() const
173     {
174         // 无参数；当前数须非负，返回按当前精度四舍五入的平方根。
175         assert(man.s >= 0);
176         if (!man.s)
177         {
178             return Decimal(0);
179         }
180         int work = max(prec + 4, (scale + 1) / 2 + 2);
181         int exponent = 2 * work - scale;
182         assert(exponent >= 0);
183         Decimal ans;
184         ans.man = BigInt(bigSqrt(shift10(man, exponent).str()));
185         ans.scale = work;
186         ans.round(prec);
187         return ans;
188     }
189 
190     Decimal operator-() const
191     {
192         // 无参数；返回当前数的相反数。
193         Decimal ans = *this;
194         ans.man = -ans.man;
195         return ans;
196     }
197 
198     Decimal &operator+=(const Decimal &other)
```

```
    {
        // other 是加数；把它加到当前数并返回当前对象。
        return *this = *this + other;
    }

    Decimal &operator-=(const Decimal &other)
    {
        // other 是减数；从当前数减去它并返回当前对象。
        return *this = *this - other;
    }

    Decimal &operator*=(const Decimal &other)
    {
        // other 是乘数；把当前数乘以它并返回当前对象。
        return *this = *this * other;
    }

    Decimal &operator/=(const Decimal &other)
    {
        // other 是非零除数；把当前数除以它并返回当前对象。
        return *this = *this / other;
    }

    Decimal &operator%=(const Decimal &other)
    {
        // other 是非零除数；把当前数改为向零取商对应的余数并返回当前对象。
        return *this = *this % other;
    }

    friend Decimal operator+(const Decimal &a, const Decimal &b)
    {
        // a、b 是加数；返回按当前精度舍入的和。
        int scale = max(a.scale, b.scale);
        Decimal ans;
        ans.man = shift10(a.man, scale - a.scale) + shift10(b.man, scale - b.scale);
        ans.scale = scale;
        ans.round(prec);
        return ans;
    }

    friend Decimal operator-(const Decimal &a, const Decimal &b)
    {
        // a 是被减数，b 是减数；返回按当前精度舍入的差。
        return a + (-b);
    }

    friend Decimal operator*(const Decimal &a, const Decimal &b)
    {
        // a、b 是乘数；返回按当前精度舍入的积。
        Decimal ans;
        ans.man = a.man * b.man;
        ans.scale = a.scale + b.scale;
```

```
        ans.round(prec);
        return ans;
    }

    friend Decimal operator/(const Decimal &a, const Decimal &b)
    {
        // a 是被除数，b 是非零除数；返回按当前精度舍入的商。
        assert(b.man.s);
        if (!a.man.s)
        {
            return Decimal(0);
        }
        int sign = a.man.s * b.man.s;
        BigInt num = absInt(a.man), den = absInt(b.man);
        int work = prec + 4;
        int exponent = work + b.scale - a.scale;
        if (exponent >= 0)
        {
            num = shift10(num, exponent);
        }
        else
        {
            den = shift10(den, -exponent);
        }
        Decimal ans;
        ans.man = num / den;
        ans.man.s *= sign;
        ans.scale = work;
        ans.round(prec);
        return ans;
    }

    friend Decimal operator%(const Decimal &a, const Decimal &b)
    {
        // a 是被除数，b 是非零除数；返回 a-trunc(a/b)*b。
        assert(b.man.s);
        BigInt num = a.man, den = b.man;
        if (b.scale >= a.scale)
        {
            num = shift10(num, b.scale - a.scale);
        }
        else
        {
            den = shift10(den, a.scale - b.scale);
        }
        Decimal quotient(num / den);
        return a - quotient * b;
    }

    friend bool operator==(const Decimal &a, const Decimal &b)
    {
        // a、b 是待比较十进制数；返回二者是否相等。
```

```
303         return a.scale == b.scale && a.man == b.man;
304     }
305
306     friend bool operator≠(const Decimal &a, const Decimal &b)
307     {
308         // a、b 是待比较十进制数；返回二者是否不等。
309         return !(a == b);
310     }
311
312     friend bool operator<(const Decimal &a, const Decimal &b)
313     {
314         // a、b 是待比较十进制数；返回 a 是否小于 b。
315         int scale = max(a.scale, b.scale);
316         return shift10(a.man, scale - a.scale) < shift10(b.man, scale - b.scale);
317     }
318
319     friend bool operator>(const Decimal &a, const Decimal &b)
320     {
321         // a、b 是待比较十进制数；返回 a 是否大于 b。
322         return b < a;
323     }
324
325     friend bool operator<=(const Decimal &a, const Decimal &b)
326     {
327         // a、b 是待比较十进制数；返回 a 是否不大于 b。
328         return !(b < a);
329     }
330
331     friend bool operator>=(const Decimal &a, const Decimal &b)
332     {
333         // a、b 是待比较十进制数；返回 a 是否不小于 b。
334         return !(a < b);
335     }
336
337     friend ostream &operator<<(ostream &out, const Decimal &x)
338     {
339         // out 是输出流，x 是待输出十进制数；写入去掉末尾零的表示并返回输出流。
340         return out << x.str();
341     }
342
343     friend istream &operator>>(istream &in, Decimal &x)
344     {
345         // in 是输入流，x 接收十进制数；读取一个字符串并返回输入流。
346         string s;
347         in >> s;
348         if (in)
349         {
350             x.assign(s);
351         }
352         return in;
353     }
354
```

```
355   private:
356     static BigInt absInt(BigInt x)
357     {
358         // x 是任意精度整数；返回其绝对值。
359         if (x.s < 0)
360         {
361             x.s = 1;
362         }
363         return x;
364     }
365 
366     static BigInt shift10(BigInt x, int digits)
367     {
368         // x 是任意精度整数，digits 是非负十进制位数；返回 x*10^digits。
369         assert(digits >= 0);
370         if (!x.s || !digits)
371         {
372             return x;
373         }
374         int sign = x.s;
375         x.s = 1;
376         int blocks = digits / 9, rest = digits % 9;
377         if (rest)
378         {
379             int factor = 1;
380             for (int i = 0; i < rest; i++)
381             {
382                 factor *= 10;
383             }
384             x = BigInt::mulAbs(x, factor);
385         }
386         x.a.insert(x.a.begin(), blocks, 0);
387         x.s = sign;
388         return x;
389     }
390 
391     static BigInt drop10(BigInt x, int digits, bool nearest)
392     {
393        // x 是任意精度整数，digits 是舍去位数；nearest 表示是否绝对值五入，返回向零截断或舍入后的整数。
394         assert(digits >= 0);
395         if (!x.s || !digits)
396         {
397             return x;
398         }
399         int sign = x.s;
400         x.s = 1;
401         int blocks = digits / 9, rest = digits % 9;
402         int roundDig = 0;
403         if (rest && blocks < (int)x.a.size())
404         {
405             int factor = 1;
406             for (int i = 1; i < rest; i++)
```

```
            {
                factor *= 10;
            }
            roundDig = x.a[blocks] / factor % 10;
        }
        else if (!rest && blocks && blocks - 1 < (int)x.a.size())
        {
            roundDig = x.a[blocks - 1] / 100000000;
        }
        if (blocks >= (int)x.a.size())
        {
            x = 0;
        }
        else
        {
            x.a.erase(x.a.begin(), x.a.begin() + blocks);
            if (rest)
            {
                int divisor = 1;
                for (int i = 0; i < rest; i++)
                {
                    divisor *= 10;
                }
                unsigned long long carry = 0;
                for (int i = (int)x.a.size() - 1; i >= 0; i--)
                {
                    unsigned long long cur = carry * BigInt::B + x.a[i];
                    x.a[i] = cur / divisor;
                    carry = cur % divisor;
                }
            }
            x.norm();
            if (x.s)
            {
                x.s = sign;
            }
        }
        if (nearest && roundDig >= 5)
        {
            x += BigInt(sign);
        }
        return x;
    }

    void normalize()
    {
        // 无参数；删除尾数与小数位中的十进制末尾零，无返回值。
        man.norm();
        if (!man.s)
        {
            scale = 0;
            return;
```

```
        }
        while (scale >= 9 && !man.a.empty() && man.a[0] == 0)
        {
            man.a.erase(man.a.begin());
            scale -= 9;
        }
        man.norm();
        while (scale && man.a[0] % 10 == 0)
        {
            man = drop10(man, 1, false);
            scale--;
        }
    }
};
```

先用 `Decimal::setPrecision(p)` 设置 $p \ge 0$,再用 `Decimal(s)` 构造;`str()` 删除小数末尾零,`str(fixed)` 固定输出 `fixed` 位并按同一规则舍入。`pow(base,e)` 要求 $e \ge 0$，`sqrt()` 要求当前值非负，除法和取模要求除数非零；`%` 使用向零取商，满足 $a = \operatorname{trunc}(a/b)b + (a \bmod b)$。精度是全局状态，临时修改时应保存并恢复旧值。设参与运算的整数位与保留小数位合计至多 D，当前 `BigInt` 下加减、比较为 $O(D)$，乘除、取模与平方根最坏为 $O(D^2)$，`pow` 为 $O(D^2 \log e)$，额外空间为 $O(D)$；每一步都会舍入，连续运算不保证实数域中的结合律。

下面用固定答案检查符号、舍入、取模、幂和平方根:

```
signed main()
{
    Decimal::setPrecision(6);
    assert((Decimal("1.25") + Decimal("-0.5")).str() == "0.75");
    assert((Decimal("1.2") * Decimal("3.4")).str() == "4.08");
    assert((Decimal(1) / Decimal(8)).str(6) == "0.125000");
    assert((Decimal(2) / Decimal(3)).str(6) == "0.666667");
    assert((Decimal("-5.5") % Decimal(2)).str() == "-1.5");
    assert(Decimal::pow(Decimal("1.5"), 3).str() == "3.375");

    Decimal::setPrecision(30);
    assert(Decimal(2).sqrt().str(30) == "1.41421356237309504880168872421O");
}
```

14.1.021 高精度圆周率：Machin 与 Chudnovsky

Machin 公式把圆周率化为 $\pi = 16\arctan\frac{1}{5} - 4\arctan\frac{1}{239}$，其中 $\arctan x = \sum_{k\ge0}(-1)^k\frac{x^{2k+1}}{2k+1}$ 在本条的 $|x| < 1$ 下收敛；`atanRecip(q)` 逐项递推，直到新项在当前精度下舍入为零。Chudnovsky 版本使用 $\pi = \frac{426880\sqrt{10005}}{S}$，其中 $S = \sum_{k\ge0}\frac{(6k)!(13591409+545140134k)}{(3k)!(k!)^3(-262537412640768000)^k}$，相邻项的阶乘比可约成代码中的整数递推，每项约增加十四位正确数字。两个接口都临时把 14.1.020 任意精度定点与十进制数 的精度提高八位，求和结束后再舍入到目标位数。

```
// 依赖 14.1.020 任意精度定点与十进制数
Decimal atanRecip(int q)
{
```

```
    // q 是大于 1 的正整数；返回按当前 Decimal 精度计算的 arctan(1/q)。
    assert(q > 1);
    Decimal x = Decimal(1) / Decimal(q);
    Decimal square = x * x;
    Decimal term = x;
    Decimal ans(0);
    for (int k = 0;; k++)
    {
        Decimal add = term / Decimal(2 * k + 1);
        if (add.isZero())
        {
            break;
        }
        ans += add;
        term = -term * square;
    }
    return ans;
}

Decimal piMachin(int digits)
{
    // digits 是结果保留的小数位数；返回 Machin 公式计算并四舍五入的圆周率。
    assert(0 <= digits && digits <= numeric_limits<int>::max() - 8);
    int old = Decimal::getPrecision();
    Decimal::setPrecision(digits + 8);
    Decimal ans = Decimal(16) * atanRecip(5) - Decimal(4) * atanRecip(239);
    ans.round(digits);
    Decimal::setPrecision(old);
    return ans;
}

Decimal piChudnovsky(int digits)
{
    // digits 是结果保留的小数位数；返回 Chudnovsky 级数计算并四舍五入的圆周率。
    assert(0 <= digits && digits <= numeric_limits<int>::max() - 8);
    int old = Decimal::getPrecision();
    Decimal::setPrecision(digits + 8);
    BigInt mul = 1;
    BigInt power = 1;
    int linear = 13591409;
    Decimal sum(linear);
    for (int k = 1;; k++)
    {
        int x = 6 + 12 * (k - 1);
        mul *= BigInt(x * x * x - 16 * x);
        mul /= BigInt(k * k * k);
        linear += 545140134;
        power *= BigInt("-262537412640768000");
        Decimal term(mul * BigInt(linear));
        term /= Decimal(power);
        if (term.isZero())
        {
```

```
            break;
        }
        sum += term;
    }
    Decimal ans = Decimal(426880) * Decimal(10005).sqrt() / sum;
    ans.round(digits);
    Decimal::setPrecision(old);
    return ans;
}
```

`piMachin(digits)` 与 `piChudnovsky(digits)` 要求 $0 \le digits \le$ `numeric_limits` $<$ `int` $>::$ `max`$()-8$，返回保留 `digits` 位小数的圆周率并恢复原精度。八位保护位适合竞赛常见位数，但不构成任意规模下的严格误差证明；需要正确舍入时应增加保护位并验证误差界。令 $P = digits + 8$，当前二次复杂度大整数下二者最坏均为 $O(P^3)$ 时间、$O(P)$ 空间；Chudnovsky 约用 $P/14$ 项，通常更快，极高精度改用二分拆分。

下面用已知的四十位小数分别核对两种公式：

```
signed main()
{
    const string expected = "3.1415926535897932384626433832795028841972";
    assert(piMachin(40).str(40) == expected);
    assert(piChudnovsky(40).str(40) == expected);
}
```

15. ACM 常用英文单词

15.1 按字母索引

15.1.001 A

English	中文释义
abbreviation	[数学] 约分;
activity on edge	AOE 网
activity on vertex	AOV 网
add, subtract, multiply and divide	加减乘除
adjacency list	邻接表
adjacency multilist	邻接多重表
adjacency matrix	邻接矩阵
adjacent sequence elements	相邻的元素串
adjacent vertex	相邻顶点
algebraic term	代数项
alphabetical order	字典序
alternately rise and fall	交替上升和下降
Ambiguous	模糊不清
ancestor	祖先
anticlockwise	逆时针
Approximate String Matching	模糊匹配
Arbitrary Precision Arithmetic	高精度计算
arc	弧
arithmetic mean	算数平均值
arithmetic progression	等差数列
geometric progression	等比数列
array	数组
articulation point	连接点
ascending lexicographical order	词典顺序升序排列
ascending order	升序
descending order	降序
aspect ratio	固定长宽比
assemble	组合
assess	评定，评估
assigned	指定的，赋值的
augmenting path graph	增广路径图
augmenting path	增广路径
average search length	平均查找长度
average temperature	顺时针
axis axes	轴

15.1.002 B

English	中文释义
balance merging sort	平衡归并排序
balance two-way merging	二路平衡归并排序
Bandwidth Reduction	带宽压缩
banlanced binary tree	平衡二叉树
base	底边；幂的底数
biconnected graph	重连通图
bidirectional	双向的
binary search tree	二叉查找树
binary search	二分查找
binary sort tree	二叉排序树
binary	二进制
bipartite graph	二部图
Bishop	主教（象）只斜走。格数不限不能越子。每方有两象，黑白格各占 1 个
blank string	空白（空格）串
block search	分块查找
boundary	界限

15.1.003 C

English	中文释义
calculate	计算
Calendrical Calculations	日期
carpet	地毯
chariot	战车（中国象棋）
checkmate	(国际象棋）将死; 输棋，将死; 败局; 败北，挫败;
circular linked list	循环链表
cirular queue	循环队列
Clique	最大团
clockwise order	顺时针方向顺序
Coefficient	系数，率，程度
Collinear	共线的
column major order	以列为主的顺序分配
columns	列
Combinatorial Problems	组合问题
comma	逗号
common superstring	公共父串
compile	编译，汇编
complete binary tree	完全二叉树

English	中文释义
complete graph	完全图
composite numbers	合数
Computational Geometry	计算几何
concave	凹的
connected component	连通分量
Connected Components	连通分支
consecutive	连续的
constant	n 常数，常量 adj 不变的，始终如一的，持续不断的
Constrained and Unconstrained Optimization	最值问题
Convex Hull	凸包
coordinates	坐标
corrupt	腐烂，破坏
counterclockwise（anticlockwise ）	逆时针
critical path	关键路径
Cryptography	密码
Cube root	立方根

15.1.004 D

English	中文释义
D is rounded to 2 decimal places	D 是精确到小数点后 2 位
Data Structures	基本数据结构
data type	数据类型
decimal	n 小数，十进制；adj 小数的，十进制的
decision tree	判定树
Deck	甲板
define	定义，明确，使规定
deformed	变形的
Denominator	分母
denote	代表; 指代; 预示; 意思是; 标志；象征
dense graph	稠密图
Deployed	部署
depth	深度
deque(double-ended queue)	双端列表
descentdant	子孙
destination	终点
Determinants and Permanents	行列式
Diagonal	对角
diagonally	斜对角线的
dial	钟面，拨打
dialing	拨号音打电话，拨电话号码

English	中文释义
Dictionaries	字典
difference	差
digital analysis method	数字分析法
digital search tree	数字查找树
digit	位数; 数字
digraph(directed graph)	有向图
Dimensiona	尺寸
diminishing increment sort	随小增量排序
direct access file	直接存取文件
directed acyclic graph	有向无环图
directory structure	目录结构
directory	（计算机文件或程序的）目录; 指导的咨询的; 管理的
discrete Fourier transform	离散傅里叶变换
disjoint	不相交的
Distinct values	独一无二的值
distinct	不同的；独一无二的
division method	除法
divisor	因子; 分母
doubly linked list	双向链表
doubly linked tree	双链树
Drawing Graphs Nicely	图的描绘
Drawing Trees	树的描绘
duplicated	复制；打印的
duplicates	复制品

15.1.005 E

English	中文释义
Edge and Vertex Connectivity	割边/割点
Edge Coloring	边染色
embed	插入
enable	启用
Entry	进口
equation	方程式; 等式
equivalent equation	同解方程; 等价方程
equivalent	相等的，等效的
estimate	预测
Eulerian Cycle / Chinese Postman Euler	回路/中国邮路
evaluate	评价，估价
evaluated	求…的值
even	偶数的

English	中文释义
excluding	排除，拒绝，驱逐; 除…外，不包括
execute	执行，完成
executed	执行的；生效的
exponent	指数；幂
external sort	外部排序

15.1.006 F

English	中文释义
Facility	设备，设施
factorial	阶乘; 因子的，阶乘的
Factoring and Primality Testing	因子分解/质数判定
Feedback Edge/Vertex Set	最大无环子图
Finite State Machine Minimization	有穷自动机简化
fixed-aggregate data type	固定聚合数据类型
foggiest idea	概念
folding method	折叠法
follow by	跟随，其后
forest	森林
formula	公式
fraction	分数; 小部分
front	队头
full binary tree	满二叉树

15.1.007 G

English	中文释义
gcd （greatest common divisor）	最大公约数
generalized list	广义表
Generating Graphs	图的生成
Generating Partitions	划分生成
Generating Permutations	排列生成
Generating Subsets	子集生成
geometric progression	等比数列
grabh	图
Graph Data Structures	图
Graph Isomorphism	同构
Graph Partition	图的划分
Graph Problems —hard	图论-NP 问题
Graph Problems —polynomial	图论-多项式算法

English	中文释义
greatest integer	最大整数
grid	网格；方格；（地图上的）坐标方格

15.1.008 H

English	中文释义
Hamiltonian Cycle Hamilton	回路
hash search	散列查找
hash table	散列表
head node	头结点
head pointer	头指针
heap sort	堆排序
horizontal or vertical direction	水平和垂直方向
horizontally	adv 水平地
horizontal	adj 水平的
Huffman tree	哈夫曼树

15.1.009 I

English	中文释义
Identifier	标识符，识别码
immediate predecessor	直接前趋
immediate successor	直接后继
immediately allocating method	直接定址法
improper fraction	假分数
in the range of	在…范围内
in the shape of a cross	十字形
incident edge	关联边
indegree	入度
indent	缩进
indentical	相同的
Independent Set	独立集
indexed file	索引文件
indexed non-sequential file	索引非顺序文件
indexed sequential file	顺序
indicating	adj 指示的，标志的
inequality	不等式
infinite	无限的
initial	adj 最初的，词首的，开始的 n 首字母
initial node	初始结点

English	中文释义
initialization	n 初始化，赋初值
initialize	v 初始化
inorder traversal	中序遍历
insertion sort	插入排序
insertion	插入
integer	整数
Interior	内部，本质
internal sort	内部排序
Interpret	解释，执行
intersect	v 相交，交叉
intersection	横切; 相交; 交叉点，交叉线; [数] 交集;
Intersection Detection	碰撞测试
intersect	相交
intervals	间隔时间; 间隔；区间
Invade	侵略
invalid	无效的
inverted file	倒排文件
irreparably	不能恢复地

15.1.010 J

English	中文释义
Job Scheduling	工程安排
justified	合理的，合法化的

15.1.011 K

English	中文释义
Kd-Trees	线段树
Knapsack Problem	背包问题
Knight	骑士（马）每步棋先横走或直走一格，然后再往外斜走一格；或者先斜走一格，最后再往外横走或竖走一格（即走“日”字）。可以越子，没有象棋中的“蹩马腿”限制。

15.1.012 L

English	中文释义
lcm (Least Common Multiple)	最小公倍数
left or right-justified	左对齐 or 右对齐
lexicographically	字典序
like terms ,similar terms	同类项
linear algebra	线性代数
linear equation	线性方程
linear linked list	线性链表
Linear Programming	线性规划
linear structure	线性结构
link field	链域
linked list	链表
literal coefficient	字母系数
logarithm	对数
logical structure	逻辑结构
Longest Common Substring	最长公共子串
loop	环

15.1.013 M

English	中文释义
Maintaining Line Arrangements	平面分割
master file	主文件
Matching	匹配
Matrix Multiplication	矩阵乘法
maximum matching	最大匹配
meadow	草坪
mean	平均值
Medial-Axis Transformation	中轴变换
Median and Selection	中位数
memorable	值得纪念的; 显著的，难忘的; 重大的，著名的
merge sort	归并排序
mid-square method	平方取中法
minimal	adj 最小限度的
minimal volume	最小体积
minimum(cost)spanning tree	最小（代价）生成树
mixed number	带分数
mod	v 求余
modulus	n 系数，模数
Motion Planning	运动规划
motion	多边形
multi-dimentional array	多维数组

English	中文释义
multilinked list	多重链表
multilist file	多重链表文件
multiple	adj 多重的多样的，许多的 n 倍数
multiplication	乘法
municipal	市政的

15.1.014 N

English	中文释义
Nearest Neighbor Search	最近点对查询
negative , positive	负，正
Network Flow	网络流
no special punctuation symbols or spacingrules	无特殊标点符号或间距的规则
non-intersecting	非相交的; 不相交的;
nonlinear structure	非线性结构
notation	标记
numerator	分子
numerical coefficient	数字系数
Numerical Problems	数值问题

15.1.015 O

English	中文释义
Obesity	肥胖
octal (binhex)	adj 八进制的 (十六进制)
odd and even	奇和偶
optimal	最佳的
optimally	最佳
Orbit	轨道
ordered pair	有序对
ordered tree	有序树
Ordinal	有次序的
original equation	原方程
origin	原点
orthogonal list	十字链表
Out degree	出度
Over brim	溢出
overflow	上溢
Overlapping	覆盖

English	中文释义
ox	牛

15.1.016 P

English	中文释义
palindrome	回文
palindromic	回文的
parallel	平行的
parity property	奇偶性
partical order	偏序
Pawn	禁卫军（兵）只能向前直走，每次只能走一格。但走第一步时，可以走一格或两格。兵的吃子方法与行棋方向不一样，它是直走斜吃，即如果兵的斜进一格内有对方棋子，就可以吃掉它而占据该格
phyical structure	物理结构
Pipe	管道
Planarity Detection and Embedding	平面性检测和嵌入
ploygon-shaped faces/ polygon	多边形
ployphase merging sort	多步归并排序
Point Location	位置查询
pointer field	指针域
Polygon Partitioning	多边形分割
positive and negative integers	正整数和负整数
postorder traversal	后序遍历
precision	n 精密，精确度精确
predecessor	前趋
prefix	前缀
preorder traversal	先序遍历
prime	质数
Priority Queues	优先队列
proceed	运行
process	v 加工，处理 n 程序，进程
process a sequence of n distinct integers	处理一串 n 个不同的整数
profile	轮廓
proper fraction	真分数
proportional	成比例的
Protrusions	凸起物
Pyramid	金字塔，渐增

15.1.017 Q

English	中文释义
quadrant	象限，四分之一圆
Queen	皇后横、直、斜都可以走，步数不受限制，但不能越子
quotient	商

15.1.018 R

English	中文释义
radix sort	基数排序
Random Number Generation	随机数生成
random number method	随机数法
Range Search	范围查询
rat, ox, tiger, rabbit, dragon, snake,horse, sheep, monkey, rooster, dog pig	十二生肖
rate of convergence	收敛速度
rear	队尾
rectangular	矩形的，成直角的
Relates	叙述，讲述
replacement selection sort	置换选择排序
respectively	各自的，分别的，独自的
robustness	鲁棒性
Rook	战车横竖均可以走，步数不受限，不能斜走。除王车易位外不能越子。
rooster	鸡
root sign	根号
round()	四舍五入（当取舍位为 5 时，若取舍位数前的小数为奇数则直接舍弃，若为偶数则向上取舍）
rounded to n decimal places	精确到小数点后 n 位
row major order	以行为主的顺序分配
Rows and columns	行与列

15.1.019 S

English	中文释义
Satisfiability	可满足性
scenario	方案；(可能发生的) 情况；
search (sequential search)	线性查找（顺序查找）
searching	查找，线索
segment	段，分段; 分割；环节; 部分; 划分；
selection sort	选择排序
semicolon	分号

English	中文释义
sequence	顺序，序列，连续
serial	连续的; 连载的; 顺序排列的;
series	连续的同类事物，系列
Set and String Problems	集合与串的问题
Set Cover	集合覆盖
Set Data Structures	集合
Set Packing	集合配置
Shape Similarity	相似多边形
Shell	贝壳，脱壳
shelter	遮蔽物
Shortest Common Superstring	最短公共父串
Shortest Path	最短路径
simple cycle	简单回路
simple path	简单路径
Simplifying Polygons	多边形化简
simultaneously	同时的
single linked list	单链表
sink	汇点
solution	解决方案
Solving Linear Equations	线性方程组
source	源点
spanning forest	生成森林
spanning tree	生成树
spares graph	稀疏图
sparse matrix	稀疏矩阵
specify	指定
square root	平方根
square	平方，正方形，广场，方格
Squared	平方
Stack Overflow	堆栈溢出通常是您的程序陷入了无穷递归，或递归嵌套层数过多。
statistical	统计的
Steiner Tree Steiner	树
stem	词根
String Matching	模式匹配
strongly connected graph	强连通图
subgraph	子图
subsequent	随后的，后来的
substring	子串
subtree	子树
successor	后继
sufficient	充足的；足够的；
suffix	后缀

English	中文释义
Supervisor	监督人
symmetric matrix	对称矩阵

15.1.020 T

English	中文释义
tail pointer	尾指针
terminal node	终端结点
Text Compression	压缩
threaded binary tree	线索二叉树
times	乘
Topological Sorting	拓扑排序
toss	扔（硬币）
Transitive Closure and Reduction	传递闭包
transposed matrix	转置矩阵
traversal of tree	树的遍历
traversing binary tree	遍历二叉树
traversing graph	遍历图
tree index	树型索引
triangle	三角形
triangle inequality	三角不等式
Triangulation	三角剖分
triple	三倍的，三方的，三部分的; 增至三倍; 三倍的数 [量]; 三个一组;
Tromino	三格骨牌
Troop	军队，组群
truangular matrix	三角矩阵
two adjacent sequence elements	两个相邻的元素串
two-dimensional array	二维数组
two-dimensional	维数

15.1.021 U

English	中文释义
ultimate	基本的，终极的
unconnected graph	非连通图
underflow	下溢
undigraph(undirected graph)	无向图
union	并集
unique identifier	唯一的标识符
unordered pair	无序对

English	中文释义
unordered tree	无序树
uppercase	大写字母盘; 以大写字母印刷; 大写字母的
uppercase（Capital）	大写字母
（Lowercase letters）	（小写字母）

15.1.022 V

English	中文释义
variable-aggregate data type	可变聚合数据类型
variable	变量
Vertex Coloring	点染色
Vertex Cover	点覆盖
vertex	n 顶点，最高点
vertical	n 垂直线，垂直面 adj 垂直的，顶点的
volume	数量，容量
Voronoi Diagrams Voronoi	图
vulnerable	容易受到攻击的

15.1.023 W

English	中文释义
weakly connected graph	弱连通图
weight	权
weighted average	加权平均值
weighted graph	加权图
wooden planks	木板